SYSTEM OWNER'S HANDBOOK

Complete Handbook

Multi-tenant clinic management platform

Status as of 2026-10-28 · code is the source of truth · written for a software-engineer owner

How to read this document

This is part of the **System Owner's Handbook** for a multi-tenant clinic management product, called "the Platform" throughout. It is written for the product owner, a software engineer, and every chapter can be read on its own.

Status labels. Every feature carries exactly one label, based on the code as of 2026-10-28:

- **[Implemented]**: built end to end.
- **[Partial]**: some of it exists; the text says exactly what is missing.
- **[Not implemented]**: planned, seeded, disabled or only mentioned. It is never described as working.
- **[Unverified]**: the code was read but the behaviour could not be confirmed; the text says what is unconfirmed.

Source of truth. The code is the source of truth. Where older documentation disagreed with the code, the handbook follows the code. Nothing was executed while writing: behaviour is described from reading the code.

Figures. Every figure has a number, a caption and a paragraph that describes it in words. Figures captioned "screenshot pending" are placeholders; the paragraph after them describes the screen from the code. The few real screenshots were taken on 2026-10-23 and show a former working name of the product.

Names. "Organization" is one clinic business (the tenant); "location" is one of its branches. File paths and code names appear in a short "Technical references" line at the end of each feature. Example people and places are fictional.

Table of contents

Part 1 — Product overview

Chapter 1 — What the Platform is 6
Chapter 2 — Roles: who uses the Platform 10
Chapter 3 — A day in the clinic 15

Part 2 — How the system is built

Chapter 4 — Repository and tech stack 21
Chapter 5 — Request lifecycle: from a click to the database and back 27
Chapter 6 — Processes, deployment and environments 36
Chapter 7 — Running the Platform locally 42

Part 3 — Organizations, locations, roles and permissions

Chapter 8 — The multi-tenant model 47
Chapter 9 — Organizations and locations 54
Chapter 10 — Features: entitlement, enablement and location overrides 64
Chapter 11 — Roles and permissions 72

Part 4 — Data model

Chapter 12 — Data model overview 85
Chapter 13 — Identity and sessions 92
Chapter 14 — Organizations, locations, subscriptions and settings 97
Chapter 15 — Access and features 103
Chapter 16 — Staff and payroll 107
Chapter 17 — Patients 111
Chapter 18 — Appointments, queue and encounters 118
Chapter 19 — Prescriptions and plans 126
Chapter 20 — Billing core 134
Chapter 21 — Billing ledgers and closing 141
Chapter 22 — Compensation 146
Chapter 23 — Inventory 151
Chapter 24 — Notifications 156
Chapter 25 — Labs 160
Chapter 26 — Workflow engine and common tables 173
Chapter 27 — Overlaps and ambiguities 179

Part 5 — Module chapters

Chapter 28 — Patients 187
Chapter 29 — Appointments 219
Chapter 30 — Queue 239
Chapter 31 — Consultation 277
Chapter 32 — Prescriptions 298
Chapter 33 — Treatment Plans 322
Chapter 34 — Billing 339

Chapter 35 — Commissions 380
Chapter 36 — Dashboard and Reports 401
Chapter 37 — Employees 421
Chapter 38 — Payroll 439
Chapter 39 — Inventory 454
Chapter 40 — Dental Lab 468
Chapter 41 — Lab Portal 510
Chapter 42 — Lab Finance 537
Chapter 43 — Workflow Engine (Config Studio) 563
Chapter 44 — Notifications 580
Chapter 45 — Subscriptions and Onboarding 597

Part 6 — End-to-end workflows

Chapter 46 — Workflow: Onboarding a New Clinic 614
Chapter 47 — Workflow: Onboarding a Staff Member 619
Chapter 48 — Workflow: The Patient Journey 623
Chapter 49 — Workflow: A Dental Lab Case 627
Chapter 50 — Workflow: Closing the Clinic Day 631
Chapter 51 — Workflow: Month-End Payroll 634
Chapter 52 — Workflow: Correcting an Invoice and Refunding Money 637
Chapter 53 — Workflow: Subscription Expiry and Recovery 641
Chapter 54 — Workflow: How an Access Change Reaches Every Screen 645
Chapter 55 — Workflow: From Purchase Order to Consumption 648

Part 7 — Cross-cutting concerns

Chapter 56 — Authentication and sessions 652
Chapter 57 — Access control in depth 666
Chapter 58 — Clinic customization 675
Chapter 59 — Notification mechanics 684
Chapter 60 — Audit logging 689
Chapter 61 — Error handling 694
Chapter 62 — Time, money and document numbers 701
Chapter 63 — Realtime updates and background jobs 706

Part 8 — Current status

Chapter 64 — Feature Status Matrix 711
Chapter 65 — Known Gaps 737
Chapter 66 — Risks 745
Chapter 67 — Open Questions 749
Chapter 68 — Documentation Conflicts 752

Appendices

Appendix A — Glossary 757
Appendix B — Entity Index 767
Appendix C — API Endpoint Summary 776
Appendix D — Permission Catalog 781
Appendix E — Scheduled Jobs and Background Loops 786

Part 1 — Product overview

Chapter 1 — What the Platform is

This chapter explains what the Platform is, who it serves and which parts are built today. It is the first chapter of the System Owner's Handbook and is written for the product owner, who is a software engineer and needs a clear picture before reading the detailed chapters. It ends with a map of the 18 module chapters, so a reader can find any topic.

1.1 One paragraph summary

The Platform is a **multi-tenant** web application that runs the daily work of clinics. "Multi-tenant" means one running system serves many separate clinic businesses, and each business sees only its own data. One clinic business is an **organization**. An organization has one or more **locations** (branches). Staff log in, and what they see depends on their **role** and on the **features** the organization has switched on. The backend is a NestJS API with a PostgreSQL database. The front end is a React web app. Chapter 4 describes the stack, and Chapter 8 describes how tenants are kept apart.

1.2 What a clinic gets

A clinic uses the Platform to do these jobs:

- keep a **patient** record, including clinical history, vitals, documents and signed consents (Chapter 28);
- book **appointments** and manage the calendar (Chapter 29);
- run a doctor-wise waiting line with **tokens** after the patient arrives (Chapter 30);
- let the doctor work from one screen, the **consultation workspace** (Chapter 31);
- write and print **prescriptions** (Chapter 32);
- plan treatment in steps with a **treatment plan** (Chapter 33);
- bill patients, take **payments**, issue **invoices** and close the cash counter (Chapter 34);
- calculate doctor **commission** (Chapter 35), run reports (Chapter 36), manage **employees** (Chapter 37), **payroll** (Chapter 38) and **inventory** (Chapter 39);
- send work to a dental **lab** and track it, including a separate **lab portal** for the lab (Chapters 40 to 42);
- manage subscription and sign-up (Chapter 45).

1.3 Specialties: general, dental and others

An organization carries one or more **specialty tags**. The code lists six: general, dental, skin, ortho, physio and eye. The sign-up form offers them as choices, for example "Skin / dermatology", "Orthopaedics", "Physiotherapy" and "Eye care". The tags describe which **features** are relevant to a clinic.

What is general. Patients, appointments, the queue, consultation, prescriptions, treatment plans, billing, commissions, reports, employees, payroll and inventory work for any clinic type. Their feature keys carry the tag "general".

What is dental-specific. Two parts are built for dental clinics:

- the **teeth chart** (FDI tooth numbering), whose feature `patients.teeth_chart` is tagged dental and ortho, and the "dental info" record of a patient;

- the **dental lab** ordering and production flow, whose feature is `dental.lab_orders`, tagged dental and ortho. The Platform ships dental lab workflows (for example "Try-in stage" and "Complete denture" stage lists) as seed data for the workflow engine.

Honest summary. Dental is the most developed specialty. A clinic of another type can use every general module today. The tags for skin, ortho, physio and eye exist as labels, but this chapter found no screens or rules written only for skin, physio or eye care **[Partial]**. The workflow engine (Chapter 43) is designed so a new specialty can be added as data, not code, but only dental data is seeded.

1.4 Market defaults visible in the code

The first customers are clinics in Pakistan. Several defaults show this. They are **defaults, not limits**.

Default	What the code does	Can it change?
Currency	An organization's currency column defaults to PKR. Billing screens show "Rs".	It is a column per organization (and one per location), so it can be set.
Timezone	Every location has its own timezone. Clinic dates and times (queue day, "today", no-show times) follow the location's timezone.	Yes, per location.
Urdu on prescriptions	A prescription PDF prints Urdu text in Nastaliq script, right to left.	It applies to typed text; printed labels stay English.
Tax	Sales tax is **exclusive**: it is added on top of the price.	Tax is configured in billing settings.
Roman-Urdu text	The "No access" page shows the headline "Aapke paas is page ki permission nahi", with English text under it.	It is fixed text in the web app.

The table lists the visible regional choices. They show the product is built first for Pakistani clinics, but nothing in them blocks another market. One point is **[Unverified]**: the location record has its own currency column whose database default is USD, while the organization default is PKR, and this chapter did not trace which one each billing screen reads.

1.5 What is and is not built

Every feature in this handbook carries one of four labels: **[Implemented]** (built end to end), **[Partial]** (some of it exists), **[Not implemented]** (planned or only mentioned) and **[Unverified]** (not confirmed from code). The table is the one-page summary of the largest items.

Area	Status	Note
Patients, appointments, queue, consultation, prescriptions	[Implemented]	Some sub-features are backend only; each module chapter lists them.
Billing, payments, invoices, cash close, refunds	[Implemented]	The invoice is issued when payment is collected.
Commissions, payroll, employees	[Implemented]	Payroll records are generated by a monthly job.
Dental lab, lab portal, lab finance	[Implemented]	A lab order is not linked to a consultation.
Reports and dashboards	[Implemented]	Reports can be exported.
Inventory	[Partial]	The screen exists but has no sidebar entry; the daily alert job only writes a server log.

Area	Status	Note
Patient portal	[Not implemented]	Patients cannot log in.
SMS, email and WhatsApp messages to patients	[Not implemented]	SMS and email feature keys are retired. A WhatsApp key exists in the feature catalog, but no patient messaging is built.
Online booking	[Not implemented]	Retired feature key.
Recurring appointments	[Not implemented]	Retired feature key.
Appointment reminders	[Not implemented]	Retired feature key.
Sign-in with Google or GitHub (OAuth)	[Not implemented]	The sign-in is disabled.
Notifications	[Partial]	In-app only.
Chairs	[Not implemented]	A database entity only; no screen or API.
Insurance claims and online payments	[Not implemented]	Retired feature keys.

The table shows that the core clinic workflow from booking to payment is built, while everything that reaches out to the patient (portal, messages, online booking) is not. Self-sign-up with a 14-day trial of the Professional plan exists in the backend and a register page exists in the web app; subscription payments are recorded by a super admin by hand (Chapter 45).

1.6 How a patient flows through the Platform

Figure 1.1 shows the main modules and the usual path of a patient.

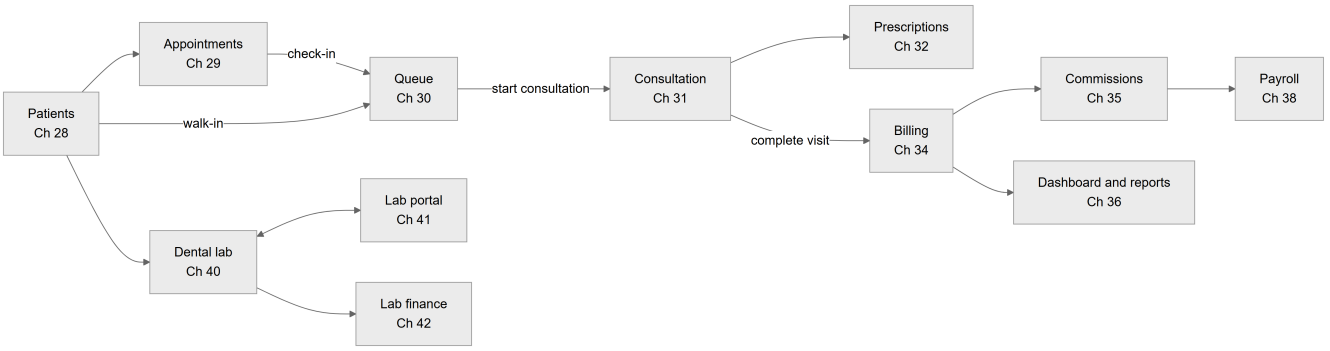

Figure 1.1 — Main modules and the usual flow of a patient, with chapter numbers.

The flowchart reads from left to right. The box "Patients (Ch 28)" has three arrows out of it. One goes to "Appointments (Ch 29)", and an arrow labelled "check-in" leads from Appointments to "Queue (Ch 30)". A second arrow, labelled "walk-in", goes straight from Patients to Queue, because a patient without a booking joins the queue directly. The third goes down to "Dental lab (Ch 40)". From Queue, an arrow labelled "start consultation" leads to "Consultation (Ch 31)". Consultation has two arrows out: one to "Prescriptions (Ch 32)", and one labelled "complete visit" to "Billing (Ch 34)". Billing has two arrows out: one to "Commissions (Ch 35)", which continues to "Payroll (Ch 38)", and one to "Dashboard and reports (Ch 36)". The Dental lab box has a two-way arrow with "Lab portal (Ch 41)", since the clinic and the lab both act on the same order, and a one-way arrow to "Lab finance (Ch 42)".

1.7 Map of the 18 module chapters

Chapter	Module	One line
28	Patients	The patient record and everything attached to it.
29	Appointments	Booking, rescheduling, check-in.
30	Queue	Doctor-wise live waiting line with tokens and a TV display.
31	Consultation	The doctor's one-screen workspace for a visit.
32	Prescriptions	Writing, signing and printing prescriptions, with Urdu.
33	Treatment plans	Planned work per patient and reusable plan templates.
34	Billing	Charges, payments, invoices, refunds, cash close.
35	Commissions	Doctor compensation plans and earnings.
36	Dashboard and reports	The home dashboard, finance overview and reports.
37	Employees	Staff records, assignments, schedules, leave.
38	Payroll	Monthly payroll records and payments.
39	Inventory	Stock, suppliers and purchase orders.
40	Dental lab	The clinic side of lab orders and production.
41	Lab portal	The lab's own login and screens.
42	Lab finance	Lab pricing, bills, statements and payables.
43	Workflow engine	Configurable forms and stages behind lab orders.
44	Notifications	In-app notifications and their rules.
45	Subscriptions and onboarding	Sign-up, trial, plans and subscription status.

The table gives the whole module list in chapter order. The roles that use these modules are described in Chapter 2, and a sample clinic day that touches most of them is told in Chapter 3. Chapters 4 to 27 explain the architecture, tenancy and data model that every module relies on.

1.8 Limitations and known gaps

- The product name is not final. The Platform is the name used in this handbook, and each clinic sets its own display name; the default is "Clinic Portal".
- Non-dental specialties have tags but no specialty-specific screens.
- Everything that reaches the patient directly is missing: no portal, no messages, no online booking.

Chapter 2 — Roles: who uses the Platform

This chapter lists every kind of person who touches the Platform, from the super admin who runs the whole system to the waiting-room TV that needs no login. It is written for the product owner and answers three questions for each audience: what they do day to day, which sidebar entries they use, and what they cannot do. The detailed permission model is in Chapter 11, and the access mechanics are in Chapter 57.

2.1 The audiences at a glance

The Platform checks **permissions**, never role names (Chapter 11). A **role** is only a named bundle of permissions. The audiences in this chapter are therefore described by what their default bundle allows.

Audience	Has a role?	Home page	Sidebar entries (labels)	Cannot do
Super admin	No, platform-wide	Dashboard	Organizations, Subscriptions, Lab Directory, Config Studio (Form schemas, Workflows, Stage templates), plus every clinic entry	Nothing is blocked inside a clinic.
Owner	No, owns one organization	Dashboard	Every clinic entry, including Locations, User Admin and the four "Clinic setup" pages	Platform-only pages: Organizations, Subscriptions, Lab Directory, Config Studio. Cannot enable a feature the super admin has not entitled.
Admin (system role)	Yes	Dashboard	Most clinic entries	Doctor acts: conduct consultations, write prescriptions, own queue and own earnings.
Doctor (system role)	Yes	My Queue	My Queue, Patients (own), Appointments (own), Prescriptions, Lab orders, My earnings, Clinic overview (the page refuses the Doctor role's own scope; see Chapter 36)	Billing, payments, employees, payroll, administration.
Receptionist (system role)	Yes	Appointments	Patients, Appointments, Billing desk, Invoices, Payments, Lab orders	Clinical records, consultation, refunds, voiding invoices, reports.
Assistant (system role)	Yes	Appointments	Patients, Appointments, Lab orders	Billing, prescriptions, consultation.
Accountant (system role)	Yes	Billing desk	Billing desk, Invoices, Payments, Expenses, Reports, Finance overview, Doctor earnings, Billing activity, Lab billing, Lab payables	Patient records beyond prices, queue, booking.
Custom role	Yes, built by the owner	Set per role	Whatever the permissions allow	Anything the owner did not tick; an owner cannot grant what they do not hold.
Lab admin	Lab membership	Lab portal	Lab portal	Anything inside a clinic.
Lab technician	Lab membership	Lab portal	Lab portal	Team management, accepting or rejecting orders.
Magic-link contact	No login	One order page	None	Anything except the one order the link opens.

Audience	Has a role?	Home page	Sidebar entries (labels)	Cannot do
TV display	No login	Display screen	None	Shows no patient names or phone numbers.
Patient	None	None	None	Cannot log in **[Not implemented]**.

The table puts every audience on one page. Doctor, Receptionist, Assistant and Accountant sidebars are deduced from each template's permissions and the sidebar registry, so a clinic that renames or hides entries (Chapter 58) sees a different sidebar. The sections that follow give the day-to-day view of each audience.

Sidebar labels. The sidebar items come from a registry in the web app. Each item appears only when the user holds its permission. The default layout has the groups Clinic, Finance (with Billing and Reports sub-groups), Team, Admin and Clinic setup. Settings is pinned at the bottom for everyone.

2.2 Super admin

The super admin is the person who operates the Platform itself, not a clinic. They are exempt from the tenant checks, the feature check and the permission check.

Day to day. They create and review **organizations**, set which **features** each organization is **entitled** to (optionally with a trial end date), record subscription payments by hand in **Subscriptions**, manage the shared **Lab Directory**, and edit forms, workflows and stage templates in **Config Studio**.

Sidebar. Organizations, Subscriptions, Lab Directory, and the Config Studio group with Form schemas, Workflows and Stage templates. These five entries are marked super admin only in the registry. They also see the clinic entries when they are working inside an organization.

Cannot do. Nothing is blocked by permission. In practice a super admin has no clinic role, so clinic-facing personal screens such as a doctor's own earnings have no staff record behind them.

2.3 Owner

The **owner** is the person who owns one organization. The owner has no role record: they get every permission at its widest scope, and permissions are still dropped when their feature is not active.

Day to day. Setting up the clinic and watching it: **Locations**, **User Admin** (staff logins), and the four setup pages **Roles & permissions**, **Features** (switch features on or off within what the super admin granted), **Sidebar** and **Theme**. They can also use every clinic module.

Cannot do. The owner cannot reach the five platform-only pages, cannot enable a feature that is not entitled, and cannot create permissions that do not exist in the catalog.

2.4 The five system roles

Every organization is seeded with five **system roles**: Admin, Doctor, Receptionist, Assistant and Accountant. They are read-only templates; to change one, the owner clones it into a custom role.

Admin. Runs the clinic day to day. It holds the Full bundle of every module except four clinical acts (conducting consultations, creating prescriptions, own queue, own earnings), but it can run and force-complete any doctor's queue. Home page: Dashboard. Uses: Dashboard, Clinic overview, Patients, Appointments, Billing, Reports, Employees, Payroll, Locations, User Admin and setup pages. Cannot: write prescriptions or conduct a consultation unless the owner clones Doctor for them.

Doctor. Consults patients. Home page: My Queue. A doctor sees only their own patients (scope `own`: people they have an appointment or queue entry with) and their own appointments. Day to day: start the queue, call the next patient, work in the consultation workspace, write and print prescriptions, create lab orders, check My earnings. Uses: My Queue, Patients, Appointments, Prescriptions, Lab orders, Order templates, My earnings, Clinic overview. Cannot: collect payments, see invoices, manage employees, change settings.

Receptionist. The front desk. Home page: Appointments. Day to day: register patients, book and reschedule, check patients in, add walk-ins on the reception queue board, run the TV display, collect payments, apply preset discounts. Uses: Patients, Appointments, Billing desk, Invoices, Payments, Lab orders. Cannot: read clinical notes, issue refunds, void invoices, give custom discounts, run reports.

Assistant. Chairside help. Home page: Appointments. Day to day: view patients at this location including clinical records, edit the teeth chart, book appointments, check patients in, view stock. Uses: Patients, Appointments, Lab orders. Cannot: take payments, write prescriptions, edit patient details.

Accountant. Money. Home page: Billing desk. Day to day: view and edit invoices, void them, issue refunds, close the cash counter, record expenses, see every doctor's earnings, read financial reports, manage lab billing and payables. Uses: Billing desk, Invoices, Payments, Expenses, Finance overview, Reports, Doctor earnings, Billing activity, Lab billing, Lab payables. Cannot: open patient clinical records, run the queue, book appointments.

The **home page** is where a user lands after login. If the user may not open it, the web app falls back to the Dashboard.

2.5 Custom roles

An owner (or a staff member with the roles permission) builds a **custom role** on the Roles & permissions screen. For each module the role gets None, Basic or Full, or an Advanced list of single permissions with a scope. A role also has a home page and members. Custom roles are the way to model a hygienist, a branch manager or a billing clerk. A holder of the roles permission cannot grant anything they do not hold themselves.

Figure 2.1 shows what a restricted custom role looks like on the dashboard.

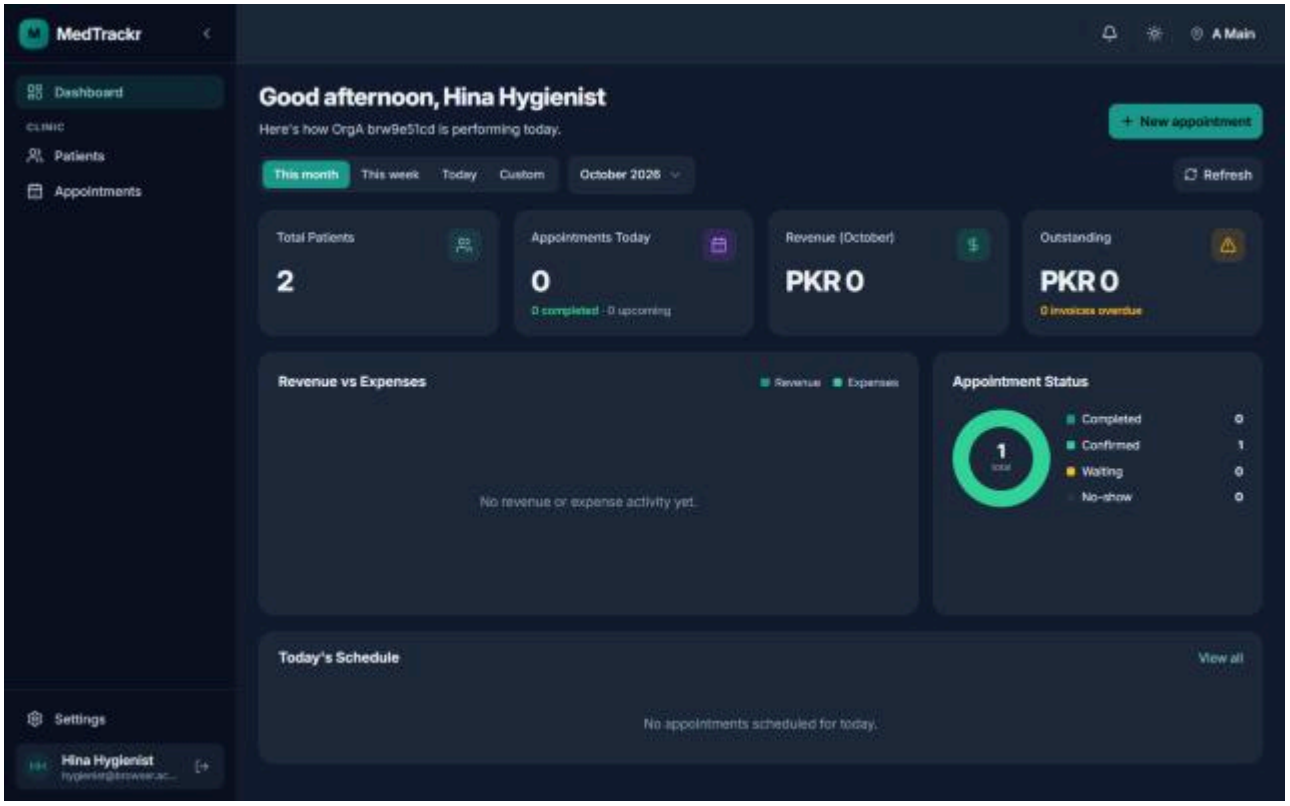

Figure 2.1 — The dashboard for a custom role with only Patients and Appointments.

The screenshot was captured on 2026-10-23 in dark theme and shows the former working name "MedTrackr" in the corner. The dashboard layout predates the redesign of that same date, so it is used only to show the sidebar. The sidebar lists Dashboard, then under the group Clinic only Patients and Appointments, with Settings and the user at the bottom. Entries for billing, reports, employees and administration are absent because the role holds none of their permissions. If this user types the address of a hidden page, the web app shows a "No access" page instead (Chapter 11).

2.6 Lab-side users

Labs are separate from clinics. A **lab** has its own members, who sign in with a normal login but hold a **lab membership** instead of a clinic assignment. They see the **Lab portal** entry in the sidebar (marked for users with at least one lab membership).

- **Lab admin.** Accepts or rejects a submitted order, asks the clinic a question (clarification), moves production steps, manages the lab team (add people, change roles, deactivate), and sees clinics, billing and profile.
- **Lab technician.** Works the production steps (for example finishing an order into quality check and passing it). Cannot accept or reject orders in the seeded dental workflow, and cannot manage the team.
- **Magic-link contact.** A person at the lab with no login. The clinic issues a **magic link** for one order. The link opens one page where the person can read the order, accept it, reject it with a reason or ask a clarification, within the link's expiry and use limit. Chapter 41 covers the lab portal in full.

These users cannot see anything inside a clinic apart from what the order and the clinic-lab link show them. The clinic decides how much patient identity the lab sees: full name or a code only.

2.7 Public and unauthenticated audiences

- **The TV display.** A screen in the waiting room opens the display page for a location. It shows, per doctor, the token being served and the next few tokens. It needs no login and by design shows no patient names or phones. Managing the TV display from a screen is **[Not implemented]**: there is no admin page for it, and the receptionist role only holds the permission to manage it.
- **Visitors before login.** Login, register, forgot password, reset password and verify email pages are public.
- **Patients.** There is **no patient portal [Not implemented]**. Patients appear in the Platform only as records that staff create and as tokens on a ticket.

2.8 Limitations and known gaps

- Sidebar labels per role in the table were deduced from permissions and the registry, not observed in a running app.
- A staff member with no assignment at the active location gets no role permissions there, even if they hold a role elsewhere.
- Super admins have no staff record, so screens built on the employee record do not apply to them.

Chapter 3 — A day in the clinic

This chapter tells the story of one working day at a clinic that uses the Platform, from opening the queue in the morning to the jobs that run overnight. It is written for the product owner, so that the modules described one by one in later chapters can be seen working together. Every person in the story is fictional, and every step names the module, its status and the chapter that explains it.

The cast. Receptionist R1 works the front desk. Dr. Example One is the doctor. Sample Patient A is a patient. The Accountant closes the cash. Example Dental Lab is an outside lab. The clinic is Example Clinic, with a Main location.

Figure 3.1 summarises the whole day before the narrative starts.

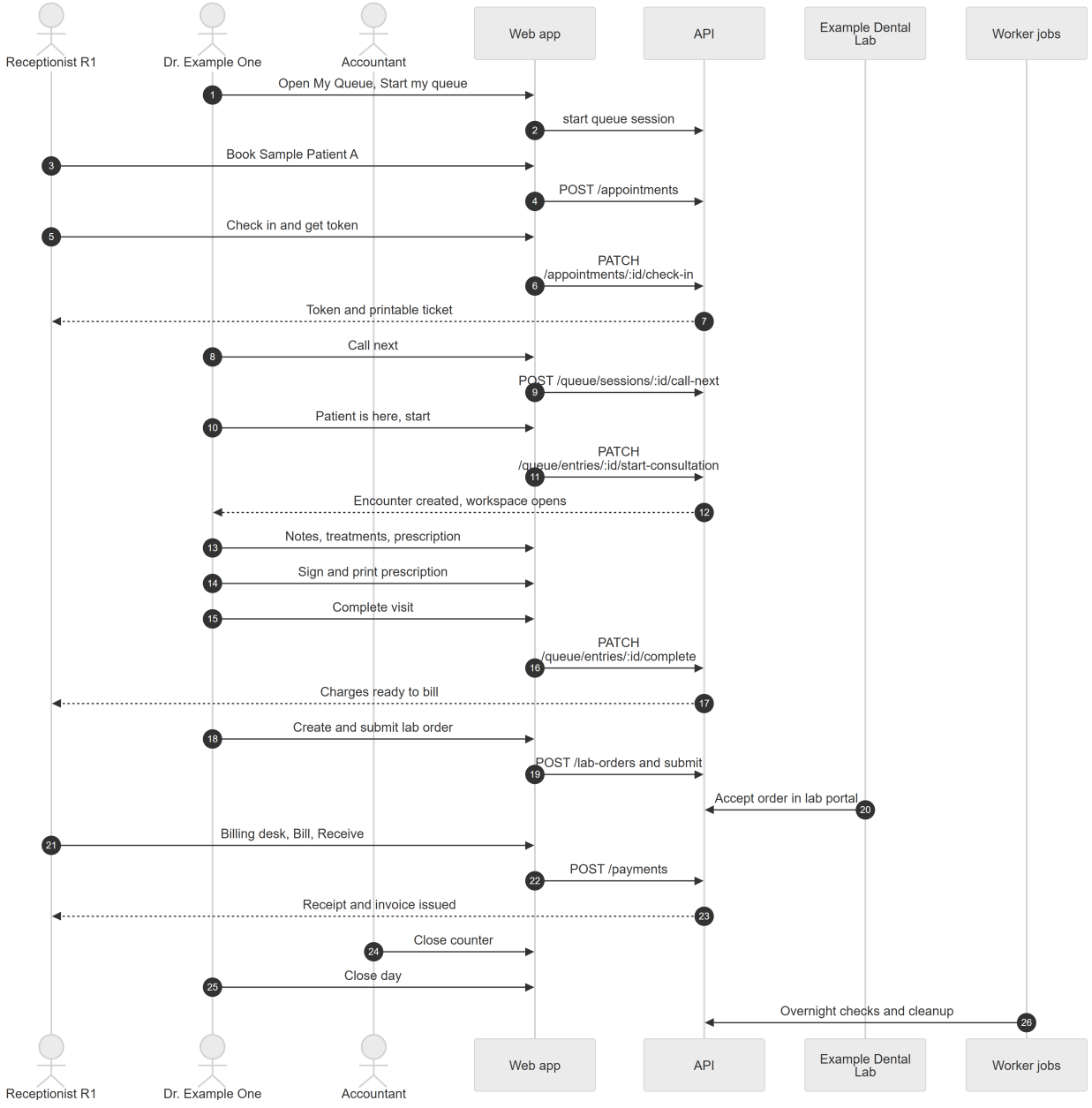

Figure 3.1 — One clinic day from opening the queue to the overnight jobs.

The sequence diagram has seven participants from left to right: Receptionist R1, Dr. Example One, the Accountant, the Web app, the API, Example Dental Lab and Worker jobs. It has 26 numbered messages. In messages 1 and 2 the doctor opens My Queue and starts the queue, and the web app calls the API. Messages 3

and 4 show R1 booking Sample Patient A with a call to create an appointment. Messages 5 to 7 show the check-in: R1 clicks Check in and get token, the web app sends the check-in call, and the API returns a token and a printable ticket to R1. Messages 8 and 9 are the doctor's Call next. Messages 10 to 12 are the doctor starting the consultation, after which the API creates the encounter and the workspace opens. Messages 13 and 14 are notes, treatments and the signed, printed prescription. Messages 15 to 17 are Complete visit, after which charges become ready to bill for R1. Messages 18 to 20 are the lab order: the doctor creates and submits it, and the lab accepts it in the lab portal. Messages 21 to 23 are R1 using the Billing desk to receive the payment, after which the API returns a receipt and issues the invoice. Message 24 is the Accountant closing the counter, message 25 is the doctor closing the day, and message 26 is the worker jobs running overnight checks and cleanup.

3.1 Opening: sign in and start the queue

Receptionist R1 signs in. If R1 may work at more than one location, the app first asks which location to use (Chapter 8, the multi-tenant model). R1's role is Receptionist, so the home page is **Appointments** (Chapter 11, roles and permissions).

Dr. Example One signs in and lands on **My Queue**, the Doctor home page. Opening that page creates the doctor's **queue session** for today if it does not exist yet. A queue session is one doctor's waiting line for one clinic day at one location, and the clinic day follows the Main location's timezone, not the server's. The doctor clicks **Start my queue**. A new session begins in the state "Doctor not started": reception can already add patients, but nobody can be called until the doctor starts. The Queue chapter (Chapter 30) describes the states Doctor not started, Open and Paused.

Status: **[Implemented]**. Module: Queue.

3.2 A booking is made

Earlier, R1 opened **Appointments** and booked Sample Patient A with Dr. Example One. The booking form checks the chosen time slot live and suggests another time when the slot is taken. The booking is saved as an **appointment**. Booking is only about reserving a time; it does not create a queue entry. The Appointments chapter (Chapter 29) covers booking, rescheduling and the status changes of a booking. Reminders to the patient and online booking are **[Not implemented]**.

Status: **[Implemented]**. Module: Appointments.

3.3 Arrival: check-in and token

Sample Patient A arrives. R1 finds the booking and clicks **Check in**, then **Check in & get token**. The Queue chapter (Chapter 30) explains that check-in is the moment the patient joins the doctor's line. The patient gets a **token** that belongs to Dr. Example One's queue for the day. A token is the short number printed on the ticket, such as "E-001": a letter prefix that comes from the doctor's name (or a prefix set on the doctor's assignment), then a three-digit counter. R1 clicks **Print ticket** in the success message, and the browser prints a small slip with the token, the doctor and roughly how many people are ahead.

If the patient has unpaid dues, a red "Rs … due" badge appears next to the patient in the queue, so R1 can ask about it (Billing, Chapter 34, feature "Show dues at check-in"). A person who arrives without a booking is a **walk-in**: R1 clicks **Walk-in** on the reception queue board, picks the doctor, and the patient gets a token the same way.

In the waiting room, the TV display shows the token being served and the next few tokens for each doctor. It needs no login and shows no patient names. The Platform pushes every change to the reception board, the doctor's screen and the TV live, through a realtime connection (Chapter 63).

Status: **[Implemented]**. Modules: Appointments (check-in) and Queue (token, board, TV).

3.4 The doctor's queue

Dr. Example One is on **My Queue**. The doctor clicks **Call next**. The button shows who will be called and why, for example "(E-001 · Appointment due)". The Queue chapter (Chapter 30) explains that a rule engine picks the next patient, either by smart rules or by first come, first served, depending on the clinic's setting (urgent patients go first in both modes). The called token flashes on the TV. When the patient walks in, the doctor clicks **Patient is here — start**.

If a called patient does not come, the receptionist or doctor can skip them, and they can rejoin later. A patient can also be sent for a test and return, or be transferred to another doctor. These actions are described in Chapter 30.

Status: **[Implemented]**. Module: Queue.

3.5 The consultation workspace

Starting the consultation creates the **encounter**, the clinical record of this one visit, and opens the **consultation workspace** at the consultation page. The Consultation chapter (Chapter 31) explains that the clinical record begins only when the doctor really starts, never at booking or check-in. In the workspace the doctor sees the patient at a glance, with allergy and medical alerts built from free-text fields (a structured allergy list is **[Partial]**), writes visit notes that auto-save, and records the **treatments** done today. The doctor can open the patient record without leaving the page.

The doctor does not see treatment prices unless the doctor's role holds the price permission; the default Doctor role does not.

Status: **[Implemented]**. Module: Consultation.

3.6 Prescription and printing

In the same workspace the **Prescription** panel lets the doctor search a medicine, set dose, frequency and duration, or choose a preset. The Prescriptions chapter (Chapter 32) explains the full module. The doctor clicks **Sign & print**. The Platform signs the prescription and shows a PDF preview on the clinic's print layout, with a **Print** button. Urdu text typed by the doctor is printed in Nastaliq script, right to left; the labels stay in English.

Status: **[Implemented]**. Module: Prescriptions.

3.7 Completing the visit

The doctor clicks **Complete**. The toast says "Visit completed" and the doctor returns to My Queue. At the same moment, in the same database transaction, the booked appointment becomes completed and the Platform creates **charges** for the visit: a consultation charge, plus one charge per treatment recorded in the encounter, priced from the clinic's price list. This happens only when the organization has the feature `billing.invoices` enabled. The Consultation chapter (Chapter 31) and the Billing chapter (Chapter 34) describe the automatic charges. No invoice exists yet: the charges wait on the Billing desk.

Status: **[Implemented]**. Modules: Queue (complete), Consultation, Billing (automatic charges).

3.8 A lab order

Sample Patient A needs a crown, so Dr. Example One orders it from Example Dental Lab. The doctor opens **Lab orders**, creates a **lab order** with an item (for example a zirconia crown), and submits it, so its status becomes Submitted. The Dental lab chapter (Chapter 40) covers the clinic side. In this version a lab order is **not** created from inside the consultation workspace and is not linked to an encounter **[Not implemented]**, so the doctor orders it from the separate Lab orders page.

The lab sees the order in its own **lab portal** (Chapter 41). A lab admin there can accept it with an expected date, reject it with a reason or ask a question. Instead of an account, the lab can also receive a **magic link** for the single order: a link that opens one page where the person can accept, reject or ask a clarification without logging in. The clinic can copy that link. The code can also queue the link for delivery through the notification outbox when the lab has a contact number or email (Chapter 44); whether a WhatsApp message actually leaves the Platform is **[Unverified]**, since notifications are in-app only. When the lab accepts, the order moves on, and its later stages are tracked through the workflow engine (Chapter 43).

Status: **[Implemented]**, except that a lab order has no link to the consultation. Modules: Dental lab, Lab portal.

3.9 Collecting payment and the invoice

Sample Patient A goes to the front desk. R1 opens the **Billing desk**. The first section, **Ready to bill**, lists Sample Patient A with the count of charges and the unbilled amount. R1 clicks **Bill**, and the **Collect payment** sheet slides in. R1 can give a discount (the Receptionist role holds the permission for preset discount types only), confirm the amount, choose the **Received in** payment account (for example cash) and click **Receive Rs ...**.

The Billing chapter (Chapter 34) explains that the **invoice is issued at this moment**: when payment is collected for charges that have no invoice yet, the Platform numbers and issues the invoice in the same transaction as the receipt. The result shows the receipt number and the invoice number, with **Print receipt** and **Print invoice** buttons. Exclusive tax, if configured, is added on top of the price. If the patient pays more than the charges, the extra money is held as **advance**. The same transaction records the doctor's **commission entries** (Chapter 35), which the doctor later sees under My earnings.

Status: **[Implemented]**. Module: Billing.

3.10 Cash close

At the end of the shift the cash is closed by someone who holds the cash-close permission. The default Receptionist role does not hold it; the Accountant and Admin roles do. In this story the Accountant opens **Payments** and clicks **Close counter**. The Platform shows the expected amount (payments minus refunds since the previous close). For a cash account the Accountant types the counted amount, adds a note if there is a difference, and clicks the close button. The close is an immutable record with a printable slip, and a difference above the alert amount shows a "Flagged" badge. Each close starts where the previous one ended, so no payment is counted twice. Closing never blocks taking payments (Billing, Chapter 34).

Status: **[Implemented]**. Module: Billing.

3.11 Closing the day

The doctor clicks **Close day** on My Queue. The button is disabled while a patient is still with the doctor. A dialog lists every patient still waiting, skipped or at a test, and for each one the doctor chooses to cancel them or transfer them to another doctor. When the list is settled the session closes and no more tokens are issued

(Chapter 30). A session that is left open with patients still waiting is not cancelled automatically; the system logs an alert for an admin.

Status: **[Implemented]**. Module: Queue.

3.12 Overnight: jobs that run by themselves

When the clinic is closed, scheduled jobs run in the one **worker** process (Appendix E lists them and Chapter 6 explains the process roles). Their schedules are in server time, not in the clinic's timezone. The ones that matter to this day:

Job	When	What it does for the clinic
Queue session day-end	Every 5 minutes	Closes the previous day's queue sessions that are empty. Sessions with waiting patients stay open and log an alert.
Patient credit expiry	Every day at 01:00	Marks prepaid patient credits past their expiry date as expired.
Billing invariant check	Every day at 02:30	Recomputes billing totals, checks the money rules and logs any break. It only logs.
Queue cleanup	Every day at 03:15	Deletes old idempotency keys and published realtime events.
Inventory alerts	Every day at 08:00	Counts low-stock and soon-expiring items and writes to the server log. Nobody is notified **[Partial]**.
Month-end payroll	05:00 on the 1st of each month	Generates payroll records for the month that ended (Chapter 38).
Queue no-show marker	Every 60 seconds, all day	Marks a booked patient as a no-show once they are too late to check in.

The table shows that the Platform does its housekeeping without any user clicking anything. Realtime delivery of queue changes and in-app notifications also runs continuously in the worker, as loops rather than scheduled jobs (Chapter 63).

Status: **[Implemented]**, except inventory alerts, which are **[Partial]**.

3.13 What the day did not include

Some things a reader may expect are not built. The patient received no reminder message, and cannot book online or see anything in a portal. Notifications reach only logged-in staff, in the app. Insurance claims and online payments are not built. These gaps are listed in Chapter 1.

3.14 Limitations and known gaps

- This chapter follows the default flow. A clinic with fewer features enabled, for example without `billing.invoices`, would not get the automatic charges at the end of the visit.
- The ticket prefix example "E-001" is deduced from the token rule (first letter of the doctor's name unless a prefix is set); the actual prefix depends on clinic data.
- The lab order stages after acceptance are described in Chapters 40 to 43, not here.

Part 2 — How the system is built

Chapter 4 — Repository and tech stack

This chapter explains how the source code of the Platform is organised and which technologies it is built on. It is written for the product owner, who is a software engineer and needs a map of the code before reading any module chapter. Everything here was read from the repository itself: the folder tree, the two `package.json` files and the module registrations in the backend.

4.1 One repository, two applications

Status: [Implemented]

The Platform lives in a single Git repository (a "monorepo"). It holds two applications that are built and run separately:

- **`backend/`** — the API server. It is a NestJS application written in TypeScript. It talks to a PostgreSQL database through TypeORM, and to Redis for cross-process messages.
- **`web-app/`** — the web application that clinic staff, owners and super admins use in the browser. It is a React 19 application built with TanStack Start and Vite.

Next to them, `docs/` holds feature documents, test plans and this handbook. The backend also has its own `backend/docs/` folder with the generated entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) and one generated module chapter.

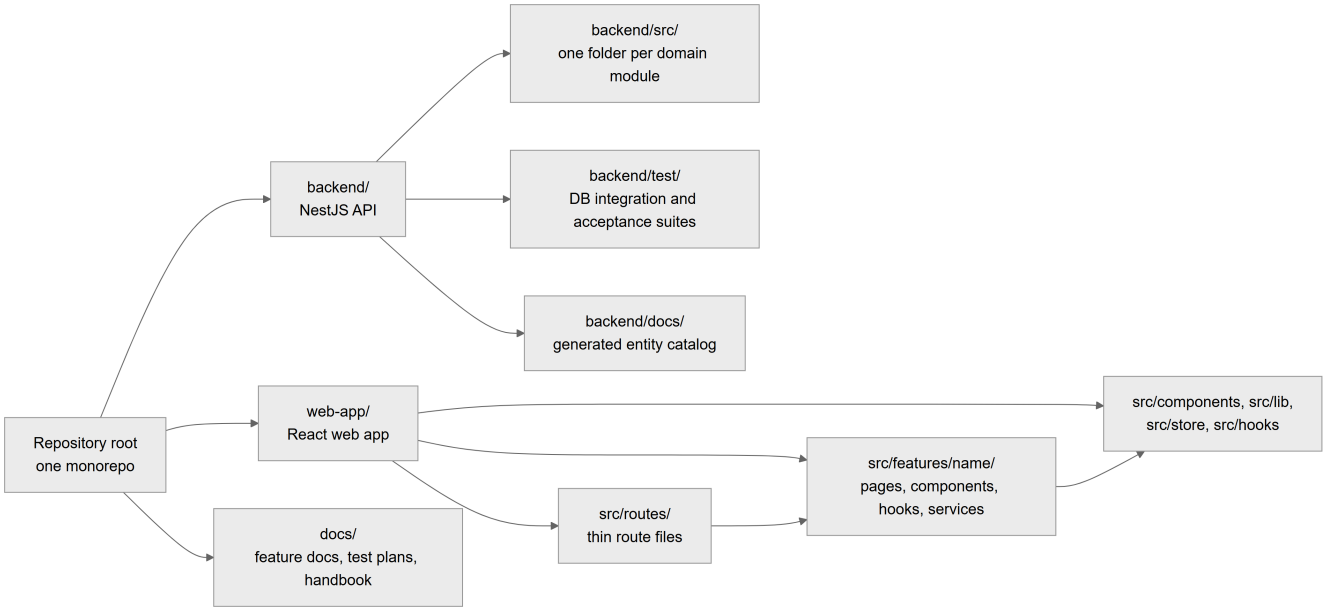

Figure 4.1 — The repository layout at the top level, and the main folders inside each application.

The figure reads from left to right. A single box, "Repository root, one monorepo", has three arrows to its three main folders: `backend/` (the NestJS API), `web-app/` (the React web app) and `docs/` (feature docs, test plans and the handbook). The backend box branches into three folders: `backend/src/` with one folder per domain module, `backend/test/` with the database integration and acceptance suites, and `backend/docs/` with the generated entity catalog. The web-app box branches into `src/routes/` (thin route files), `src/features/<name>/` (pages, components, hooks and services) and the shared folders `src/components` , `src/lib` , `src/store` and `src/hooks` . Two further arrows show the direction of dependencies in the web app: route files point to feature folders, and feature folders point to the shared folders.

There is no workspace tool joining the two applications (no root `package.json`). Each one has its own `package.json`, its own `node_modules` and its own commands.

Technical references: `backend/package.json` · `web-app/package.json` · `README.md` (root)

4.2 The web application's real folder layout

Status: [Implemented]

The web app keeps business logic out of route files. A route file only declares the URL, an optional permission requirement and which feature page to render. The real layout under `web-app/src/` is:

Folder	What it holds
`routes/`	92 file-based route files for TanStack Router. Each one wraps a feature page in `DashboardLayout` and may declare `staticData: { can: … }`, the permission needed to open it (54 routes do). `__root.tsx` is the root route: it restores the session, redirects, and mounts the access providers.
`features/<name>/`	One folder per feature (about 30, for example `patients`, `appointments`, `queue`, `billing`, `roles`, `access`). Inside: `pages/`, `components/`, `hooks/` (TanStack Query and form hooks), `services/` (axios calls), and often `schemas/` (Zod), `types`, `lib/` and `utils/`. A feature's `index.ts` is its public barrel.
`components/`	Shared UI: `ui/` (shadcn/ui primitives), `layout/` (`DashboardLayout`, `Sidebar`, `Topbar`, `LocationSwitcher`), `shared/` (`DataTable`, `ConfirmDialog`, `TooltipButton`, `BackButton`, `DatePicker` and more), `feedback/` (`EmptyState`, `ErrorState`, `LoadingSkeleton`, `NoResultsState`), `forms/`, `data-display/`, `feature/`.
`lib/`	`api.ts` (the configured axios instance), `query-client.ts` (TanStack Query defaults), `currency.ts`, `formatters.ts`, `utils.ts`.
`store/`	Zustand stores: `useAuthStore.ts` (token, active organization and location, kept in memory only), plus small UI stores for patients, appointments, employees, navigation history and general UI.
`hooks/`	Cross-feature hooks such as `useBackNavigation`, `useDebounce`, `useTheme`.
`context/`	`FeatureFlagsContext.tsx` (the active features and the organization's specialty tags).
`test/`	Vitest setup.

The table lists each top-level folder of `web-app/src/` and its role. The pattern to remember is: a route file is thin, a feature folder owns its screens and its API calls, and anything used by many features lives in `components/`, `lib/`, `store/` or `hooks/`.

Where the CLAUDE.md conventions file differs. The root `CLAUDE.md` describes a different, planned layout: route files under `src/app/`, shared code under `src/shared/`, and an `api/` folder in each feature. None of those folders exist. The code uses `src/routes/`, `src/components/`, `src/lib/`, `src/store/` and feature `services/` folders, as the table shows. Some feature folders also deviate from the planned shape (for example `organizations/` and `locations/` keep their views at the folder root, and a few components call the axios instance directly). This handbook always describes the real layout.

Technical references: `web-app/src/routes/__root.tsx` · `web-app/src/features/` · `web-app/src/lib/api.ts` · `web-app/src/store/useAuthStore.ts`

4.3 Backend module map

Status: [Implemented]

The backend is a "modular monolith": one NestJS application whose code is split into modules, one folder per business area under `backend/src/`. All modules are loaded by `AppModule` (`backend/src/app.module.ts`). Every folder is listed here with one line.

Folder	One-line purpose
`access`	Permission catalog (70 permissions), bundles, the 5 system role templates, `AccessResolver`, `AccessGuard`, `GET /me/access`, sidebar layouts (`/admin/navigation`).
`appointments`	Booking: scheduling, validation, booking rules, doctor double-booking protection, appointment status changes.
`auth`	Sign-in, tokens, refresh, sessions endpoints, password reset, email verification, rate limiting, the JWT and tenant guards, `AuthorizationService`. OAuth code exists but is not registered.
`billing`	Charges, invoices, payments, allocations, refunds, discounts, expenses, cash close, ledgers, credit packages, doctor commissions.
`bootstrap`	`http.ts`: starts an HTTP process (body limits, validation pipe, exception filter, CORS, cookies).
`chairs`	One entity file only. No module registers it, so it does nothing at runtime.
`common`	Shared building blocks: audit, clock (`ClockService`), database helpers and migrations runner, decorators, `ResponseWrapper`, exception filter, idempotency, Redis connection helper, number sequences, file storage, request context.
`employees`	The employee person record, assignments, employment roles (including the `/admin/roles` screen API), schedules, leave, doctor availability overrides.
`encounters`	Patient encounters: the clinical record of one consultation.
`engine`	Generic workflow engine: specialties, versioned form schemas, versioned workflows, stage templates (Config Studio).
`feature-flags`	Feature catalog, entitlement, owner switches, location overrides, `FeatureFlagGuard`.
`inventory`	Items, suppliers, purchase orders, stock movements, consumption, low-stock job.
`lab-access`	A lab's answers to orders through the lab portal, magic links or a clinic proxy; work-order QR.
`lab-billing`	Billing the patient for lab work.
`lab-catalog`	Lab services, price windows, clinic-to-lab service mappings, order templates.
`lab-finance`	Glue between lab billing, lab payables, commissions and the journals (no endpoints of its own).
`lab-orders`	Lab orders: drafts, submit, revisions, messages, attachments, work-order print.
`lab-payables`	Vendor bills from labs, lab payments, statements, payables reports.
`lab-production`	Production stages, technician assignments, shipments, remakes, warranties.
`labs`	Platform lab directory, lab staff, each clinic's lab list, lab portal foundation.
`locations`	Locations (branches) and location settings.
`migrations`	Three migration files; the first is a legacy baseline that never runs.
`notifications`	In-app notification rules, templates, outbox, quiet hours, the sending loop.

Folder	One-line purpose
`organizations`	Organizations (clinics), ownership, legacy organization settings.
`patients`	The patient record and its sub-records (vitals, dental info, teeth chart, documents, consents, recalls, referrals, insurance, feedback, plans), import and export, patient audit trail.
`payroll`	Monthly payroll records and the month-end payroll job.
`plan-templates`	Treatment plan templates.
`prescriptions`	Prescriptions, medicine directory, templates, presets, doctor signatures, print layouts.
`queue`	Doctor-wise queue: sessions, entries, tokens, `QueueOrchestrator`, selection engine, policies, Socket.IO gateway, outbox relay, queue jobs.
`reports`	Finance dashboard, clinic overview and 37 report definitions with Excel and PDF export (owns no tables).
`session`	User session records (the refresh-token sessions).
`settings-registry`	`config_values` cascade (platform, organization, location) used by the theme and newer settings.
`subscriptions`	Plans, trial, grace, read-only mode, manual subscription payments, self-signup and the setup wizard, `SubscriptionGuard`.
`treatments`	Treatments: clinical work performed.
`user`	User accounts (logins), user admin, staff logins, owner creation.

The table names every folder in `backend/src/` and what it is responsible for. Besides the folders, `backend/src/` holds the entry points `main.ts`, `main.api.ts` and `main.worker.ts` (one per process role, explained in Chapter 6), `app.module.ts`, a placeholder `app.controller.ts` / `app.service.ts` (a hello endpoint) and `data-sourse.ts`, the TypeORM command-line data source (the misspelling is the real file name).

Note on CLAUDE.md. `CLAUDE.md` describes inventory, invoices, reports and notifications as stubs. In the code all four are built. Only `chairs` is an entity with no module.

Technical references: `backend/src/app.module.ts` · `AppModule` · `backend/src/*/*.module.ts`

4.4 Backend technology

Status: [Implemented] (with the exceptions noted in the table)

Package (major version)	Purpose in the Platform
NestJS 11 (`@nestjs/core`, `common`, `platform-express`)	The application framework: modules, controllers, dependency injection, guards, pipes, interceptors, running on Express.
`@nestjs/typeorm` 11 + TypeORM 0.3 + `pg` 8	Database access to PostgreSQL: entities, repositories, transactions, schema synchronize, migrations. `pg` is also used directly for `LISTEN` connections.
`@nestjs/config` 4	Reads environment variables.
`@nestjs/jwt` 11, `@nestjs/passport` 11, `passport-jwt`, `passport-local`	Access tokens and the JWT strategy behind `JwtAuthGuard`.
`passport-google-oauth20`, `passport-github2`	OAuth strategies. Present but not registered, so OAuth sign-in is off [Not implemented].

Package (major version)	Purpose in the Platform
`bcrypt` 6	Password hashing.
`cookie-parser`	Reads the httpOnly refresh-token cookie.
`class-validator` , `class-transformer`	DTO validation and conversion in the global `ValidationPipe` .
`@nestjs/websockets` 11, `@nestjs/platform-socket.io` , `socket.io` 4	The realtime gateway at `/realtime` .
`ioredis` 5	Redis pub/sub for realtime events and cache invalidation across processes.
`@nestjs/bull` 11 + `bull` 4	A Bull queue named `submissions` is registered, but no code processes or adds jobs to it [Partial: registered, unused].
`@nestjs/schedule` 6	Scheduled jobs (`@Cron` , `@Interval`), loaded only in processes that run jobs.
`@nestjs/cache-manager` , `cache-manager`	Listed as dependencies; no source file imports them [Not implemented].
`@nestjs/mongoose` , `mongoose`	Listed as dependencies; no source file imports them [Not implemented].
`@nestjs/swagger`	Listed; no source file imports it, so there is no generated API documentation [Not implemented].
`cloudinary` 2, `multer` 2	File uploads (logos, documents, attachments).
`nodemailer` 7	Outgoing email (verification and password reset).
`exceljs` 4	Excel export of reports.
`pdf-lib` , `pdfmake` , `fontkit` , `@fontsource/noto-nastaliq-urdu` , `@fontsource/noto-naskh-arabic`	PDF generation, including Urdu text.
`axios` , `slugify` , `useragent` , `rxjs` , `reflect-metadata`	HTTP client, slug generation, user-agent parsing for sessions, and NestJS runtime needs.

The table lists the backend's runtime dependencies by name and what each is for. Development dependencies are the usual NestJS set: the Nest CLI, TypeScript 5, Jest 29 with `ts-jest` , Supertest, `socket.io-client` (for tests), ESLint 9 and Prettier 3.

Technical references: `backend/package.json` · `backend/src/app.module.ts`

4.5 Web application technology

Status: [Implemented]

Package (major version)	Purpose in the Platform
React 19, `react-dom` 19	The UI library.
TanStack Start 1 (`@tanstack/react-start`) with Vite 7 and the Nitro plugin	The application framework and build. Vite builds the app; the TanStack Start and Nitro plugins produce the server bundle that serves it.
TanStack Router 1 (`@tanstack/react-router` , `router-plugin`)	File-based routing from `src/routes/` ; the generated route tree is `routeTree.gen.ts` .
TanStack Query 5	Server-state cache: queries, mutations, invalidation.
TanStack Form 1	Forms.
TanStack Table 8	Tables in list views.

Package (major version)	Purpose in the Platform
Zustand 5	Client state: the auth store and small UI stores.
Zod 4	Form schemas.
axios 1	HTTP client with interceptors (`src/lib/api.ts`).
`socket.io-client` 4	Realtime connection to `/realtime` .
Tailwind CSS 4, shadcn/ui (Radix UI primitives), `class-variance-authority` , `clsx` , `tailwind-merge` , `lucide-react`	Styling, components and icons.
FullCalendar 6	Appointment calendar views.
Recharts 3	Charts.
`@dnd-kit`	Drag and drop (for example the sidebar builder).
`date-fns` 4, `react-day-picker`	Dates and the date picker.
`sonner`	Toast messages.

The table lists the web app's runtime dependencies and their roles. For tests the web app uses Vitest 3 with Testing Library and jsdom; formatting and linting use Prettier and ESLint.

Technical references: `web-app/package.json` · `web-app/vite.config.ts` · `web-app/vitest.config.ts`

4.6 What the repository does not contain

Status: [Not implemented]

There is no Dockerfile, no container compose file, no CI pipeline configuration (no `.github/` folder) and no hosting configuration in the repository. How the Platform is deployed is therefore not described by code. Chapter 6 explains what the code expects from a deployment.

Technical references: repository root listing

Chapter 5 — Request lifecycle: from a click to the database and back

This chapter follows one real request through the whole Platform: from a button in the web app, through the API's security checks, into a database transaction, and back to the screen. It is written for the product owner who wants to know exactly which layers every request passes and where an error can stop it. A second part follows a realtime event from a database commit to every open screen.

5.1 The example: registering a patient

Status: [Implemented]

The traced request is the one a receptionist makes when registering a new patient. On the Patients page the receptionist clicks **Create**, fills the multi-step **New Patient** form and clicks **Add Patient** on the last step. The web app sends `POST /patients`, the API stores the patient and an audit record, and the web app shows the toast "Patient created" and offers to book a first appointment.

This request is a good example because it touches every common layer: the token and tenant headers, all five global guards, DTO validation, a permission check (`patients.create`), a database transaction with an audit row, the response envelope and the cache refresh in the web app. It does not use realtime events; section 5.8 covers those with a queue action instead.

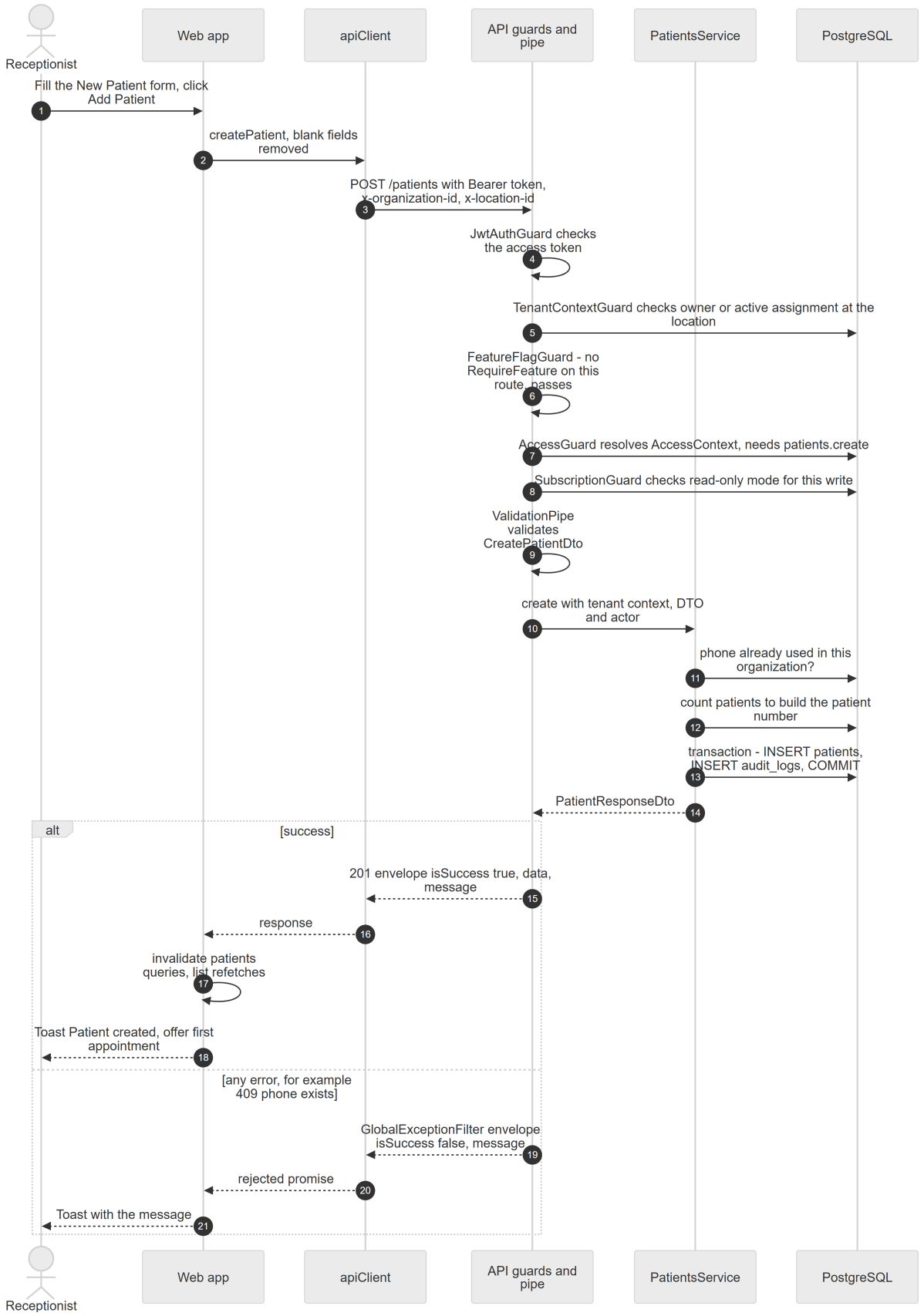

Figure 5.1 — Sequence of one write request, `POST /patients`, from the receptionist's click to the toast.

The diagram has six participants from left to right: the Receptionist, the Web app, the `apiClient` axios instance, the API's guards and validation pipe, `PatientsService`, and PostgreSQL. (1) The receptionist fills the New Patient form and clicks Add Patient. (2) The web app calls `createPatient`, which first removes blank optional fields. (3) The axios instance sends `POST /patients` with the Bearer token and the `x-organization-id` and `x-location-id`

headers. (4) `JwtAuthGuard` checks the access token. (5) `TenantContextGuard` checks in the database that the user owns the organization or has an active assignment at that location. (6) `FeatureFlagGuard` finds no `@RequireFeature` on this route and lets it pass. (7) `AccessGuard` resolves the user's access context and requires `patients.create`. (8) `SubscriptionGuard` checks that the clinic is not in read-only mode, because this is a write. (9) The `ValidationPipe` validates the body against `CreatePatientDto`. (10) The controller calls `PatientsService.create` with the tenant context, the DTO and the actor. (11) The service checks that the phone number is not already used in the organization, (12) counts existing patients to build the next patient number, and (13) inserts the patient and an `audit_logs` row in one transaction and commits. (14) It returns a `PatientResponseDto`. An "alt" box then shows two outcomes. On success the API returns a 201 response whose body is the envelope with `isSuccess: true`, `data` and `message`; the web app invalidates its cached patient lists, which refetch, and shows the toast "Patient created" with the option to book a first appointment. On any error, for example 409 because the phone exists, the `GlobalExceptionFilter` returns the envelope with `isSuccess: false` and a message; the axios promise rejects and the web app shows the message in a toast.

Technical references: `web-app/src/features/patients/components/CreatePatientModal.tsx` · `useCreatePatient` · `backend/src/patients/patients.controller.ts` · `PatientsService.create`

5.2 In the browser: hook, service and the axios instance

Status: [Implemented]

In simple terms. The screen never talks to the server directly. It asks a small "mutation hook" to do the work; the hook calls a service function; the service function uses one shared HTTP client that adds the user's credentials and the clinic and branch the user is working in.

Technically.

- The form component calls `useCreatePatient().mutateAsync(data)`. This is a TanStack Query mutation whose `mutationFn` is `createPatient` from `features/patients/services/patients.api.ts`.
- `createPatient` drops fields that are empty strings (the API would reject an empty `dob` or `email`) and calls `api.post('/patients', body)`. It returns `data.data`, the payload inside the response envelope.
- `api` is the axios instance `apiClient` in `web-app/src/lib/api.ts`:
  - Base URL: the `VITE_API_BASE_URL` build variable, default `http://localhost:5000`.
  - Timeout: 15 seconds. `withCredentials: true`, so the browser sends the refresh-token cookie.
- **Request interceptor.** Before every request it reads the Zustand auth store (`useAuthStore`), which lives in memory only and is never written to browser storage:
  - adds `Authorization: Bearer <access token>` when a token exists;
  - for lab portal URLs (`/lab-portal/...`) adds only `x-lab-id` and no clinic headers;
  - otherwise adds `x-organization-id` (the active organization) and `x-location-id` (the active location).
- **Response interceptor.** Described with the error path in section 5.7.

TanStack Query is configured once in `web-app/src/lib/query-client.ts`: queries are fresh for 5 minutes, are kept for 10 minutes, retry twice and do not refetch when the window regains focus; mutations retry once. Individual hooks may override these values (the access query, for example, uses 30 seconds and refetches on focus).

Technical references: `web-app/src/lib/api.ts` · `apiClient` · `web-app/src/lib/query-client.ts` · `web-app/src/store/useAuthStore.ts`

5.3 Arriving at the API: process start-up and Express middleware

Status: [Implemented]

An HTTP process is started by `bootstrapHttp(role)` in `backend/src/bootstrap/http.ts` (Chapter 6 explains the roles). It configures, in this order:

1. The NestJS application with Nest's own body parser switched off.
2. JSON and URL-encoded body parsers with a **5 MB** limit (consent signatures are sent as inline images).
3. The global `ValidationPipe` (section 5.5).
4. The global exception filter `GlobalExceptionFilter`.
5. CORS for one origin, `CORS_ORIGIN` (default `http://localhost:3000`), with credentials allowed.
6. `cookie-parser`, so the refresh cookie can be read.
7. A small middleware that opens a per-request context (`RequestContextService`, based on Node's AsyncLocalStorage).
8. Listening on `PORT` (default 5000).

Two middleware classes exist in `common/middleware/` (`ContextMiddleware` and `TenantContextMiddleware`) but **no module registers them** [Not implemented]. Tenant checks are done by a guard instead.

For every request NestJS then runs its fixed pipeline: Express middleware → global guards → interceptors (on the way in) → pipes → controller method → interceptors (on the way out) → exception filter if anything threw. The next sections walk this pipeline.

Technical references: `backend/src/bootstrap/http.ts` · `bootstrapHttp` · `backend/src/common/services/request-context.service.ts`

5.4 The five global guards

Status: [Implemented]

Five guards are registered as global guards (`APP_GUARD`) in `backend/src/app.module.ts`. NestJS runs them in registration order, and the first one that refuses stops the request.

#	Guard	Question it answers	Skips when	Refusal
1	`JwtAuthGuard`	Is there a valid, unexpired access token?	The method and path match an entry in `PUBLIC_ROUTES` (login, register, refresh, password reset, email verification, logout, the public theme, the TV display state, lab magic links).	401 with a message such as "Token has expired. Please refresh your token."
2	`TenantContextGuard`	Does this user have access to the organization and location in the headers?	`@Public()`; `@ContextFree()`; super admins. `@RequireSuperAdmin()` routes are checked here instead (non-super-admins get 403 "Super admin access required").	400 "Missing tenant context. Please select an organization and location." or 403 "You do not have access to this organization/location".
3	`FeatureFlagGuard`	Is every feature named by `@RequireFeature` active for this organization and location?	No `@RequireFeature` on the route; `@Public()`; super admins.	403 'The "" feature is not enabled for this location'.

#	Guard	Question it answers	Skips when	Refusal
4	`AccessGuard`	Does the user hold every permission in `@Can(...)`, or is the user a member for `@AnyMember()`?	`@Public()`; routes with neither marker pass (but still get the access context attached when an organization is known).	403 with `{ code: 'NO_PERMISSION', missing: [...] }`.
5	`SubscriptionGuard`	If this is a write, is the clinic out of read-only mode?	Reads (`GET`, `HEAD`, `OPTIONS`); `@Public()`; super admins; paths starting `/auth/`, `/account`, `/me/`, `/admin/subscriptions`; requests with no organization.	402 with `code: 'SUBSCRIPTION_EXPIRED'`.

The table lists the guards in the order they run, what each checks, when it steps aside, and what the caller receives when it refuses. In the patient example all five apply: the route is not public, it carries `@Can('patients.create')`, and it is a write.

Details worth knowing:

- **Access check for the tenant.** `TenantContextGuard` calls `AuthorizationService.validateAccess`. An owner passes for any active location of the organization. Anyone else needs an active assignment at that exact location with portal access switched on. When it passes, the guard stores `request.tenantContext = { organizationId, locationId }`.
- **`@Public()` and the JWT guard.** `JwtAuthGuard` only reads the `PUBLIC_ROUTES` list (`auth/constants/public-routes.ts`). The `@Public()` decorator alone does not let an unauthenticated request through; a route without a token must also be listed there.
- **No path-prefix skips.** `TenantContextGuard` deliberately has no list of path prefixes that bypass it; it relies only on the decorators.
- **Caching.** The access context is cached per user, organization, location and access version (Chapter 11); the enabled-feature set for 30 seconds (Chapter 10); read-only status for 30 seconds per process.

Technical references: `backend/src/app.module.ts` · `JwtAuthGuard` · `TenantContextGuard` · `FeatureFlagGuard` · `AccessGuard` · `SubscriptionGuard` · `backend/src/auth/constants/public-routes.ts`

5.5 Validation pipe and the controller

Status: [Implemented]

Validation. One global `ValidationPipe` validates every request body, query and parameter that is typed as a DTO class with `class-validator` rules. Its settings:

Setting	Effect
`transform: true`	The plain JSON body becomes an instance of the DTO class.
`whitelist: true`	Properties without a validation decorator are removed.
`forbidNonWhitelisted: true`	Instead of silently removing unknown properties, the request is refused with 400 (for example "property id should not exist").
`enableImplicitConversion: true`	Query strings such as `page=2` become numbers when the DTO says so.

The table lists the four options and their effect. The strict `forbidNonWhitelisted` rule matters for owners: a screen that sends a whole database row back (with `id` or timestamps) is refused, which is why some older edit screens fail (the Organizations and Locations chapter, Chapter 9, lists them).

Controller. In the example, `PatientsController.create` receives:

- `@Body() dto: CreatePatientDto` — the validated body;
- `@ActiveTenant() ctx` — `{ organizationId, locationId }` from `request.tenantContext` (or the headers);
- `@RequestActor(ActorSide.CLINIC) actor` — who is acting, for the audit trail.

Other handlers also take `@Access() access`, the resolved access context, when they need scope rules (Chapter 11). Controllers contain no business rules; they call a service and wrap the result.

Technical references: `backend/src/bootstrap/http.ts` · `backend/src/common/decorators/active-tenant.decorator.ts` · `backend/src/common/audit/request-actor.decorator.ts`

5.6 Service, transaction and database

Status: [Implemented]

`PatientsService.create` shows the usual service pattern:

1. **Checks that read.** The phone must be unique inside the organization (409 "A patient with this phone number already exists in this organization"). The next patient number is built from a count of the organization's patients, including deleted ones (`P-00001` , `P-00002` , …).
2. **One transaction for the write.** `dataSource.transaction(...)` saves the `patients` row and writes the audit record through `recordPatientChange` with the same entity manager. Both rows commit together or not at all.
3. **A response DTO.** The saved entity is mapped to `PatientResponseDto` .

Other modules follow the same shape with stronger guarantees where money or queues are involved: row locks (`pessimistic_write`), optimistic version checks, idempotency keys for queue actions, and outbox rows written in the same transaction (section 5.8). All tenant data is filtered by `organization_id` in the service; data that has a scoped permission is filtered further with `applyScope` (Chapter 11).

The database is PostgreSQL. The API connects with the `DATABASE_URL` connection string and always requests an SSL connection.

Technical references: `backend/src/patients/patients.service.ts` · `PatientsService.create` · `recordPatientChange`

5.7 The response envelope and the error path

Status: [Implemented]

Success. Every controller returns `ResponseWrapper.success(data, message)` . The JSON body is always the same envelope:

Field	Meaning
`data`	The payload (here the new patient).
`message`	A human-readable message, for example "Patient created successfully".
`isSuccess`	`true` .
`httpStatusCode`	Always `200` inside a success envelope, even when the HTTP status is 201.
`error` , `devMessage`	Empty on success.

The table describes the five fields of the envelope. The web app reads `res.data.data` for the payload. On the way out, the global `VisibleIfInterceptor` removes fields marked `@VisibleIf` that the caller may not see (for example treatment prices without `patients.price.view`); patient records have no such fields.

Back in the browser, the mutation's `onSuccess` invalidates every query whose key starts with `patients`, so open patient lists refetch, and shows the toast "Patient created". The modal then switches to an optional "Book First Appointment" step.

Errors. Any exception, anywhere after the middleware, reaches `GlobalExceptionFilter`. It logs the error and answers with the HTTP status of the exception (500 for unexpected errors) and an envelope with `isSuccess: false`, `message` (the first message when there are several, such as validation errors; "Something went wrong" for unexpected errors), `error` (the exception body, for example `{ code: 'NO_PERMISSION', missing: [...] }`) and `devMessage`.

Status	Typical cause
400	Validation failed, or tenant headers missing.
401	No token, invalid token or expired token.
402	`SUBSCRIPTION_EXPIRED`: the clinic is read-only.
403	No access to the organization or location, feature not active, `NO_PERMISSION`, or super admin required.
404	Record not found in this organization.
409	Business conflict, such as a duplicate phone number.
500	Unexpected error.

The table maps the common status codes to their causes. Screens show `err.response.data.message` in a toast, or a fallback text such as "Failed to create patient".

Automatic token refresh. The access token lives 15 minutes. When any request gets **401**, the axios response interceptor:

1. ignores 401s from `/auth/login` and `/auth/refresh` themselves;
2. if a refresh is already running, parks the request in a queue;
3. otherwise calls `POST /auth/refresh` (the httpOnly cookie carries the session id and refresh token), stores the new token and context in the auth store, replays the queued requests and retries the original request once;
4. if the refresh fails, or the retried request gets 401 again, clears the auth store and sends the browser to `/login`.

Technical references: `backend/src/common/dtos/ResponseWrapper.ts` · `backend/src/common/exceptions/HttpExceptionHandler.ts` · `GlobalExceptionFilter` · `VisibleIfInterceptor` · `web-app/src/lib/api.ts`

5.8 The realtime path: from a commit to every open screen

Status: [Implemented]

In simple terms. When something changes in a doctor's queue, every screen showing that queue (the doctor screen, the reception board, the waiting-room TV) must update within moments, even when the Platform runs several API servers. The Platform does this with a "transactional outbox": the change and a "please announce this" note are saved together; a separate worker reads the notes and broadcasts them; every API server pushes them to the browsers that are listening.

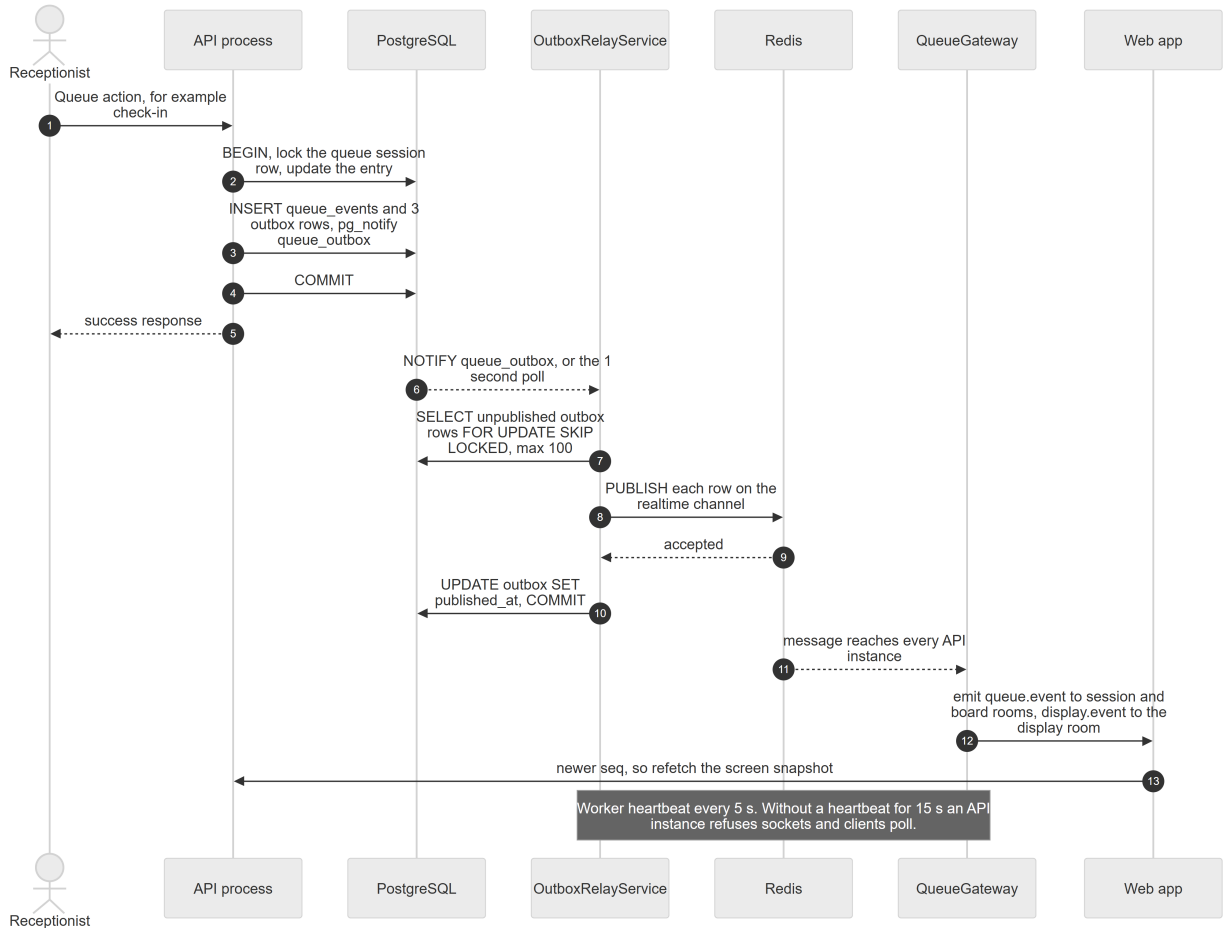

Figure 5.2 — Sequence of a queue change travelling from the database transaction, through the outbox relay and Redis, to the web app.

The diagram has seven participants: the Receptionist, the API process, PostgreSQL, the `OutboxRelayService` (in the worker), Redis, the `QueueGateway` (the Socket.IO gateway in every API process) and the Web app. (1) The receptionist performs a queue action, for example a check-in. (2) The API begins a transaction, locks the queue session row and updates the queue entry. (3) In the same transaction it inserts a `queue_events` audit row and three `outbox` rows, and calls `pg_notify('queue_outbox')`. (4) It commits. (5) The receptionist gets the success response. (6) PostgreSQL delivers the `queue_outbox` notification to the relay (if the notification is missed, the relay also polls every second). (7) The relay selects up to 100 unpublished outbox rows with `FOR UPDATE SKIP LOCKED`. (8) It publishes each row to Redis on the realtime channel, and (9) Redis confirms. (10) The relay stamps `published_at` on those rows and commits. (11) Redis delivers the message to every API instance. (12) Each gateway emits `queue.event` to the session room and the location board room, and `display.event` to the location's display room. (13) The web app sees a newer sequence number and refetches the screen's snapshot over HTTP. A note across the relay and the gateway says that the worker sends a heartbeat every 5 seconds, and that an API instance that has heard no heartbeat for 15 seconds refuses socket connections, so clients fall back to polling.

Facts behind the figure:

- **Three outbox rows per change**, one per room: `session:<sessionId>` (one doctor's queue), `location:<locationId>:board` (the reception board) and `location:<locationId>:display` (the TV). Payloads carry ids, statuses, tokens and a sequence number, never patient names.
- **At-least-once delivery.** If Redis does not accept the batch, the relay's transaction rolls back and the rows are retried. Clients drop duplicates by sequence number.
- **Rooms and access.** Clients connect to the `/realtime` Socket.IO namespace with their access token and `subscribe` to rooms. The gateway checks access per room: display rooms are open without login; session

and board rooms need access to that location; `org:<organizationId>` needs access to the organization.

- **Access changes use the same sockets.** When an organization's access version changes (a role edit, a feature switch), every process hears it over a separate Redis channel and the gateway emits `access.changed` to the `org:<id>` room. The web app then refetches `GET /me/access` (Chapter 11).
- **Single-process mode.** In the `all` role (development), the relay hands rows straight to the local gateway and Redis is not needed for realtime.
- **Notifications** use the same pattern with their own outbox table (`notification_outbox`) and relay loop (Chapter 6).

Technical references: `backend/src/queue/services/queue-orchestrator.service.ts` · `QueueOrchestrator.emit` · `backend/src/queue/realtime/outbox-relay.service.ts` · `RedisRealtimePublisher` · `RedisRealtimeSubscriber` · `QueueGateway` · `web-app/src/features/access/hooks/useAccessSocket.ts`

5.9 Requests that take a different path

Status: [Implemented]

- **Public requests** (login, register, refresh, password reset, verify email, logout, the login-page theme, the TV display state, lab magic links) skip the token check because they are listed in `PUBLIC_ROUTES` , and skip the other guards because they carry `@Public()` .
- **Context-free requests** (`@ContextFree()`) skip the tenant and access checks and check access themselves in the handler. Examples: `/auth/select-location` , `/auth/switch-location` , `GET /me/access` , `GET /feature-flags/my-features` , every `/locations` endpoint.
- **Super-admin requests** (`@RequireSuperAdmin()`) need no tenant headers; the tenant guard checks the super-admin role instead.
- **Lab portal requests** (`/lab-portal/...`) carry `x-lab-id` instead of clinic headers and are protected by their own lab portal guard.

Technical references: `backend/src/common/decorators/` · `backend/src/auth/auth.controller.ts` · `backend/src/labs/`

Chapter 6 — Processes, deployment and environments

This chapter explains how the backend of the Platform runs as operating-system processes, what each process does, what Redis is used for, who owns the database schema, and which environment variables the code reads. It is written for the product owner who has to run or deploy the Platform. Everything comes from the backend code and the current migrations document; no deployment configuration exists in the repository, and that gap is stated where it matters.

6.1 One codebase, three process roles

Status: [Implemented]

The backend is one NestJS application, but it can start in one of three **process roles**. The role decides which parts are switched on. The roles are defined in `backend/src/common/runtime/process-role.ts`.

Role	Entry file	Start command	What it runs	How many
`all`	`main.ts`	`npm run start:dev` (watch mode)	Everything: HTTP API, Socket.IO gateway, outbox relay, notification relay, scheduled jobs. The relay hands realtime events straight to the local gateway, so Redis is not needed for realtime.	One. The development default.
`api`	`main.api.ts`	`npm run start:api` (after a build)	HTTP API and the Socket.IO gateway. Receives realtime events from the worker through Redis. Runs no background jobs.	Two or more, each with its own port.
`worker`	`main.worker.ts`	`npm run start:worker` (after a build)	Outbox relay, notification relay and every scheduled job. No HTTP server at all.	**Exactly one.**

The table lists each role with its entry file, its npm script, what it runs and how many copies should run. The split exists so that the API can be scaled out without running the background jobs more than once: scheduled jobs are only loaded where jobs run (`runsJobs(role)` is true for `all` and `worker`), and HTTP is only started where it is needed (`runsHttp(role)` is true for `all` and `api`).

How the role reaches the code: each entry file calls `AppModule.forRole(role)`. That method adds, per role, the TypeORM connection (with schema ownership only for HTTP roles), the `ScheduleModule` (only where jobs run), `QueueRuntimeModule.forRole(role)` (gateway, Redis subscriber, relay and queue jobs as needed) and `NotificationsRuntimeModule.forRole(role)` (the notification sending loop only where jobs run).

Two oddities in the npm scripts:

- `npm run start:prod` runs `node dist/main`, but the build writes `dist/src/main.js`. Use `start:api` and `start:worker` for built code [Partial].
- The `start:api` and `start:worker` scripts run the compiled `dist/src/...` files, so `npm run build` must run first.

Technical references: `backend/src/common/runtime/process-role.ts` · `ProcessRole` · `runsHttp` · `runsJobs` · `backend/src/main.ts` · `backend/src/main.api.ts` · `backend/src/main.worker.ts` · `AppModule.forRole`

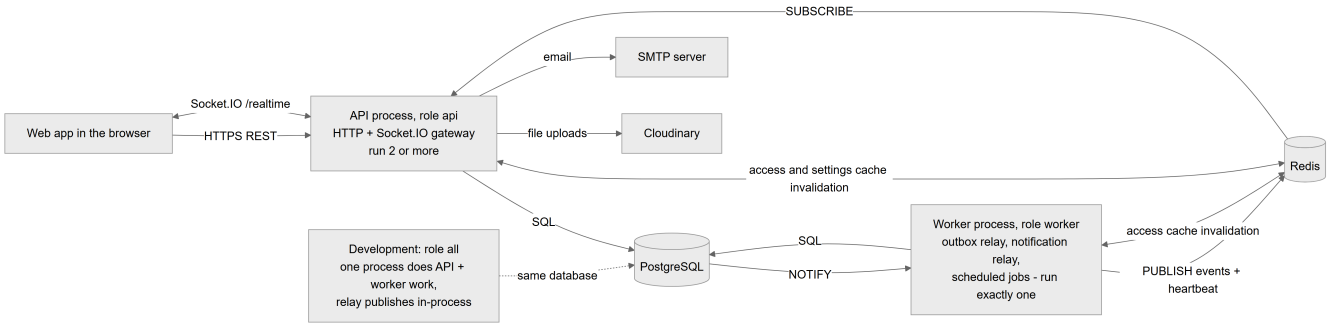

Figure 6.1 — The production process layout: API processes, one worker, PostgreSQL, Redis and the outside services, plus the single development process.

The figure shows the web app in the browser on the left. It sends HTTPS REST requests to the "API process, role api", which runs HTTP and the Socket.IO gateway and is meant to run two or more times; a two-way arrow labelled "Socket.IO /realtime" connects the same two boxes. The API process sends SQL to PostgreSQL, uploads files to Cloudinary and sends email through an SMTP server. The "Worker process, role worker" runs the outbox relay, the notification relay and the scheduled jobs, and must run exactly once; it also sends SQL to PostgreSQL, and PostgreSQL sends it NOTIFY messages. The worker publishes realtime events and a heartbeat to Redis, and Redis delivers them to the API process through a subscription. Two more two-way arrows connect Redis with the API process (access and settings cache invalidation) and with the worker (access cache invalidation). A separate box at the bottom, "Development: role all", shows that in development one process does the work of both API and worker and publishes realtime events in-process; a dotted line shows that it uses the same database.

6.2 What runs where

Status: [Implemented]

Component	`all`	`api`	`worker`
HTTP API (Express, port `PORT`)	yes	yes	no
Socket.IO gateway `/realtime` (`QueueGateway`)	yes	yes	no
Redis realtime subscriber (`RedisRealtimeSubscriber`)	no	yes	no
Queue outbox relay (`OutboxRelayService`)	yes, publishes in-process	no	yes, publishes to Redis
Notification relay (`NotificationRelayService`)	yes	no	yes
Scheduled jobs (`@Cron`, `@Interval`)	yes	no	yes
Postgres extensions, schema synchronize (dev), startup migrations	yes	yes	no
Feature catalog seeding and role template seeding at boot	yes	yes	yes
Access and settings cache invalidation bus (Redis)	yes	yes	yes

The table shows, for each running component, in which role it is active. Seeding runs in every process because every process loads every domain module; both seeders insert only missing rows and ignore conflicts, so concurrent processes are safe.

The relays. Both relays are woken by PostgreSQL `LISTEN` / `NOTIFY` on a dedicated database connection and also poll as a fallback: the queue relay every second, the notification relay every two seconds (which also picks up retries and messages held back by quiet hours). If the `LISTEN` connection fails, they poll only.

Scheduled jobs. There are eight. None of them sets a timezone, so every cron time in the next table is in the **server's** timezone, not the location's.

Job	Schedule	What it does
`queue-no-show-marker`	every 60 seconds	Marks booked patients who never checked in, past start time plus the allowed lateness, as no-show.
`queue-call-timeout`	every 15 seconds	Skips a called patient who never came, when that queue setting is on.
`queue-session-day-end`	every 5 minutes	Closes empty queue sessions from a past clinic day; sessions with waiting patients are left for an admin and logged.
`queue-cleanup`	every day at 03:15 server time	Deletes idempotency keys older than 24 hours and published outbox rows older than 7 days.
Patient credits expiry	every day at 01:00 server time	Marks active patient credits whose expiry date is before today (server-local midnight) as expired.
`billing-invariants`	every day at 02:30 server time	Checks the money invariants for every patient with billing rows and logs violations. Read-only.
Inventory alerts	every day at 08:00 server time	Finds low-stock and expiring items where inventory and inventory alerts are on, and only writes a log line; no notification is created [Partial].
`month-end-payroll`	on the 1st of each month at 05:00 server time	Generates payroll for the previous month for every active location.

The table lists each job, how often it runs and its effect. The queue jobs re-check state under the session lock through `QueueOrchestrator`, so running one twice is harmless. The queue jobs decide "which clinic day" with the location's timezone through `ClockService`; the cron trigger times themselves follow the server clock. The payroll job's code comment says 00:05, but the cron expression means 05:00.

Technical references: `backend/src/queue/queue-runtime.module.ts` · `backend/src/notifications/notifications-runtime.module.ts` · `backend/src/queue/jobs/queue-jobs.service.ts` · `BillingInvariantJob` · `InventoryAlertService` · `PayrollService.runMonthEndPayroll` · `PatientCreditsService.checkExpiredCredits`

6.3 What Redis is used for

Status: [Implemented]

Redis is used only for messages between processes. No business data is stored in it.

Use	Channel or name	Publishers → listeners	Without Redis
Realtime queue events	the queue realtime channel	Worker relay → every API instance; worker heartbeat every 5 s	In `all` mode not needed. In split mode the relay keeps rows unpublished and retries; API instances refuse sockets after 15 s without a heartbeat and clients poll.
Access changes	the access-changed channel	Any process that bumps an organization's access version → every process (access cache, feature cache, socket `access.changed`)	Only the process that made the change hears it; others catch up through their cache lifetimes (15 s for the access version, 30 s for features).
Settings cache invalidation	`settings-registry:invalidate`	Any process → every process	Other processes catch up through the settings cache lifetime.

Use	Channel or name	Publishers → listeners	Without Redis
Bull queue `submissions`	Bull connection	Registered, but no producer or consumer exists	Nothing depends on it.

The table lists the four uses of Redis, the channel names, who talks to whom and what happens when Redis is unavailable. Redis connections are created to fail fast (no offline queue) and reconnect with back-off. The connection string is `REDIS_URL`; the default is a Redis on the local machine. The exact channel names are defined in `realtime-bus.ts` and `access-change-bus.ts`.

Login rate limiting does not use Redis: attempts are counted in each process's memory, so with several API instances each instance counts on its own.

Technical references: `backend/src/common/redis/redis-connection.ts` · `backend/src/queue/realtime/realtime-bus.ts` · `AccessChangeBus` · `backend/src/settings-registry/settings-cache-bus.ts` · `backend/src/auth/guards/rate-limiter.guard.ts`

6.4 Who owns the database schema

Status: [Implemented]

In simple terms. The table structure is defined by the entity classes in the code. In development the API adjusts the database to the entities automatically on start. In production nothing changes the tables automatically: an operator first reviews the planned changes and then applies them. Hand-written migrations add what entities cannot express, such as triggers.

Technically.

- **Entities own table shapes.** Only processes that serve HTTP own the schema (`api` and `all`); the worker never touches it.
- **Start-up order in a schema-owning process:**
  1. `ensurePostgresExtensions` installs the `btree_gist` extension (needed by the doctor double-booking constraint) and a `uuid_generate_v7()` function (default primary key of the lab tables).
  2. The TypeORM connection starts. `synchronize` is **on** unless `NODE_ENV` is `production`.
  3. `runPendingMigrations` records the legacy baseline as applied, then runs pending migrations, each in its own transaction. If one fails, start-up fails.
- **Migrations.** `backend/src/migrations/` holds three files. `1776356529788-create-table.ts` is a legacy baseline that is recorded as applied and never executed. The other two (`doctor-compensation`, `lab-management-foundation`) are hand-written and idempotent: they skip tables that already exist and add triggers and backfills.
- **Production.** `synchronize` is off. The documented procedure is: build; review the SQL with `npm run schema:log:prod`; apply it with `npm run schema:sync:prod` after a backup; start **one** API instance so it runs the migrations; only then start further API instances and the worker (concurrent start-ups take no migration lock). Alternatively run `npm run migration:run:prod` as a release step.
- **CLI data source.** The TypeORM command-line tools use `backend/src/data-sourse.ts`, which loads entities by file pattern. It therefore also sees entity files that no module registers (`chairs` and the old single `inventory` entity), which can show up in `schema:log` output.

Technical references: `backend/src/app.module.ts` · `backend/src/common/database/ensure-extensions.ts` · `backend/src/common/database/migrations.ts` · `runPendingMigrations` · `backend/src/data-sourse.ts` · `docs/database-migrations.md`

6.5 Environments

Status: [Partial]

The code knows only one environment switch: `NODE_ENV`. When it equals `production`:

- TypeORM `synchronize` is off;
- the refresh-token cookie is marked `secure` (sent over HTTPS only).

Everything else is the same in every environment. There is no separate staging configuration, no environment-specific config file and no feature that behaves differently per environment. The README and the migrations document describe a production database hosted on Neon (managed PostgreSQL); the code's SSL settings are written to work with Neon.

Not in the repository [Not implemented]:

- no Dockerfile or container compose file;
- no CI pipeline (no `.github/` folder or other CI configuration);
- no hosting, process-manager or load-balancer configuration.

Whoever deploys the Platform must therefore provide, outside the repository: a PostgreSQL database, a Redis server (required as soon as `api` and `worker` run as separate processes), a way to run two or more `api` processes behind a load balancer and exactly one `worker`, HTTPS, and the web app build. The Socket.IO gateway allows both WebSocket and long-polling transports; the web app connects with WebSocket only, and a code comment notes that long-polling behind several API instances would need sticky sessions.

Technical references: `backend/src/app.module.ts` · `backend/src/auth/auth.service.ts` (`REFRESH_TOKEN_COOKIE_OPTIONS`) · `backend/src/queue/realtime/queue.gateway.ts`

6.6 Environment variables

Status: [Implemented]

Values are never shown in this handbook. The backend reads its variables through `@nestjs/config` (which loads `backend/.env`) or `process.env`. The template file `backend/.env.example` lists some of them.

Variable	Purpose	In `.env.example`
`DATABASE_URL`	PostgreSQL connection string for the API, the worker, the relays' `LISTEN` connections and the CLI.	yes
`PORT`	HTTP port of an API process (default 5000).	yes
`NODE_ENV`	`production` turns `synchronize` off and makes the refresh cookie secure.	yes
`JWT_ACCESS_SECRET`	Signs and checks access tokens (also used by the Socket.IO gateway).	yes
`JWT_REFRESH_SECRET`	Listed in the template, but no code reads it: refresh tokens are random values stored with the session, not JWTs.	yes

Variable	Purpose	In `.env.example`
`EMAIL_TOKEN_SECRET`, `RESET_TOKEN_SECRET`	Listed in the template, but no code reads them: verification and reset tokens are random values.	yes
`REDIS_URL`	Redis for the realtime bus, the access and settings buses and Bull (default: local Redis).	no
`CORS_ORIGIN`	The one web origin allowed by CORS and by the Socket.IO gateway (default `http://localhost:3000`).	no
`FRONTEND_URL`	Base URL of the web app used in email links (default `http://localhost:3000`).	no
`SMTP_USER`, `SMTP_PASS`	Credentials of the outgoing mail account.	yes
`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`	File uploads to Cloudinary.	yes
`MAGIC_LINK_SECRET`	Signs lab magic links; falls back to `JWT_ACCESS_SECRET`, then `JWT_SECRET`.	no
`JWT_SECRET`	Last fallback for the magic-link secret only.	no
`LAB_MAGIC_LINK_TTL_HOURS`	Lifetime of a lab magic link (default 24 hours, at most 7 days).	no
`LAB_LINK_RATE_LIMIT`	Request limit for magic-link pages (default 120).	no
`RATE_LIMIT_LOGIN_MAX`, `RATE_LIMIT_LOGIN_WINDOW_SECONDS`	Login rate limit (default 5 attempts per 900 seconds).	no
`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `GITHUB_CLIENT_ID`, `GITHUB_CLIENT_SECRET`	OAuth sign-in. OAuth is not registered, so these have no effect [Not implemented].	yes
`VITE_API_BASE_URL` (web app)	Base URL of the API for the browser (default `http://localhost:5000`).	no web template exists

The table lists every environment variable the code reads, with its purpose and whether the template file mentions it. Variables only used by the integration test suites (for example `BILLING_IT_DATABASE_URL` and the `*_KEEP_SCHEMA` switches) are described in Chapter 7.

Technical references: `backend/.env.example` (names only) · `backend/src/bootstrap/http.ts` · `backend/src/lab-access/magic-links/magic-link-token.ts` · `backend/src/common/services/mail.service.ts` · `web-app/src/lib/api.ts`

Chapter 7 — Running the Platform locally

This chapter lists what a developer needs to run the Platform on one machine: prerequisites, installation, configuration, start commands, the first super-admin step and the test suites. It is written for the product owner who wants to see the system running or hand it to a new engineer. The steps come from the two `package.json` files, the backend's configuration code and the root README; nothing was executed while writing this chapter.

7.1 Prerequisites

Status: [Implemented]

Need	Why	Notes
Node.js and npm	Both applications are Node projects.	Neither `package.json` pins a Node version. The README recommends Node 20 or newer.
PostgreSQL	The only data store.	The account must be allowed to create the `btree_gist` extension and a function on first start. The API always opens its connection **with SSL** (`ssl: { rejectUnauthorized: false }`), so the server must accept SSL connections; Neon does. Whether a plain local PostgreSQL without SSL works was not verified [Unverified].
Redis	Messages between processes.	Optional when running the single development process (`all` role): realtime then works in-process. The access, settings and Bull connections still try to reach Redis and log warnings while it is missing. Required when `api` and `worker` run as separate processes.
A Cloudinary account	File uploads (logos, patient documents, attachments).	Uploads fail without it; the rest of the Platform runs.
An SMTP mail account	Verification and password-reset emails.	Optional for local work.

The table lists each external dependency, why the Platform needs it and what to know when setting it up locally.

Technical references: `backend/src/app.module.ts` · `backend/src/common/database/ensure-extensions.ts` · `README.md` (root)

7.2 Install

Status: [Implemented]

The two applications are installed separately; there is no root-level install.

1. In `backend/` , run `npm install` .
2. In `web-app/` , run `npm install` .

Technical references: `backend/package.json` · `web-app/package.json`

7.3 Configure

Status: [Implemented]

1. Copy `backend/.env.example` to `backend/.env`. The template names: `DATABASE_URL`, `PORT`, `NODE_ENV`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, the Google and GitHub OAuth pairs, `EMAIL_TOKEN_SECRET`, `RESET_TOKEN_SECRET`, `SMTP_USER`, `SMTP_PASS` and the three `CLOUDINARY_*` variables.
2. Fill in at least `DATABASE_URL` and `JWT_ACCESS_SECRET`. Leave `NODE_ENV` unset or not `production`, so the schema is created from the entities.
3. Add the variables the template does not list when the defaults do not fit: `REDIS_URL` (default: Redis on the local machine), `CORS_ORIGIN` and `FRONTEND_URL` (default `http://localhost:3000`), and optionally the magic-link and rate-limit variables. Chapter 6 lists every variable with its purpose.
4. The web app reads one variable, `VITE_API_BASE_URL` (default `http://localhost:5000`). Put it in `web-app/.env` only if the API runs elsewhere.

Several template entries have no effect: `JWT_REFRESH_SECRET`, `EMAIL_TOKEN_SECRET` and `RESET_TOKEN_SECRET` are not read by any code, and the OAuth variables do nothing because OAuth is not registered.

Technical references: `backend/.env.example` (names only) · `web-app/src/lib/api.ts`

7.4 Run

Status: [Implemented]

Terminal	Command	Result
1	in `backend/`: `npm run start:dev`	One `all`-role process in watch mode on port 5000: HTTP API, Socket.IO, relays and scheduled jobs.
2	in `web-app/`: `npm run dev`	The web app's Vite dev server on port 3000.

The table shows the two commands needed for a working local system. Then open `http://localhost:3000`.

What happens on the first backend start:

1. The `btree_gist` extension and the `uuid_generate_v7()` function are created.
2. TypeORM `synchronize` creates every table from the entities.
3. The legacy migration baseline is recorded and the two hand-written migrations run (triggers and backfills).
4. The feature catalog is seeded with the 24 active features, and the 5 system role templates are seeded into every existing organization.

To try the production split locally: `npm run build`, then `npm run start:api` and `npm run start:worker` in two terminals, with Redis running. Do not start a second worker.

Technical references: `backend/src/main.ts` · `backend/src/bootstrap/http.ts` · `AppModule.forRole` · `FeatureFlagsService.seedCatalog` · `AccessTemplatesService`

7.5 The first super admin

Status: [Partial] — there is no screen, command or seed that creates a super admin; it is a manual database step.

The super admin is the platform operator who creates organizations, entitles features, records subscription payments and runs Config Studio. A user is a super admin when the `users.global_role` column holds `super_admin`.

1. Open `/register` and sign up. Self-signup creates **both** a user and a new clinic: an organization on a 14-day trial with Professional limits, its `MAIN` location and the ownership link. That trial clinic is harmless for local work.

2. Promote the user with one SQL statement against the database:

```
UPDATE users SET global_role = 'super_admin' WHERE email = '<your sign-up email>';
```

3. Sign out and sign in again, so the login response reports the new role.
4. As super admin you choose an organization at sign-in, and you can open the setup pages (Organizations, Locations, Lab Directory, Config Studio, Subscriptions) even before choosing a location. Create an organization and its locations, entitle features, then add employees and give them logins and roles (Chapter 11).

Email verification does not block sign-in.

Technical references: `backend/src/common/enums/global-role.enum.ts` · `backend/src/subscriptions/clinic-setup.service.ts` · `web-app/src/routes/__root.tsx` (`SUPER_ADMIN_SETUP_PATHS`)

7.6 Test suites

Status: [Implemented]

Suite	Command	What it runs	Needs a database
Backend unit	in `backend/` : `npm test`	Jest over the 67 `*.spec.ts` files in `backend/src` , including `access-coverage.spec.ts` , which fails if any endpoint lacks an access decorator.	no
Backend e2e	`npm run test:e2e`	One starter test that boots the full application and calls `GET /` .	yes (it starts the real module)
All DB integration specs	`npm run test:int`	Every `*.int-spec.ts` file under `backend/test/` with the billing suite's setup.	yes
Labs	`npm run test:int:labs`	Lab module integration tests.	yes
Tenancy	`npm run test:int:tenancy`	Tenant isolation tests.	yes
Access core	`npm run test:int:access`	Access resolution, patient scope, sidebar layouts.	yes
Access acceptance	`npm run test:int:acceptance`	The 166-case access-control acceptance plan in 11 spec files.	yes
Web	in `web-app/` : `npm test`	Vitest with Testing Library and jsdom over 34 test files.	no

The table lists each test command, what it covers and whether it needs PostgreSQL. There are 35 integration spec files in total.

How the database suites protect data: each suite creates its **own PostgreSQL schema** (for example `billingit` , `tenancy_it` , `acceptit`), drops and recreates it at the start and drops it at the end, and never touches the `public` schema. They read `BILLING_IT_DATABASE_URL` when set, otherwise `DATABASE_URL` ; for Neon they strip the connection-pooler part of the host name, because schema isolation needs a direct connection. Some suites accept a `*_KEEP_SCHEMA` variable to keep the schema for inspection. The suites share one database, so run them one at a time.

Other useful scripts: `npm run lint` and `npm run format` (backend), `npm run check` (web: Prettier and ESLint fixes), and `npm run access:export` (backend), which regenerates the web app's copy of the permission catalog after `permissions.ts` changes.

Technical references: `backend/package.json` · `backend/test/*.json` · `backend/test/billing/db.config.ts` · `web-app/vitest.config.ts` · `backend/scripts/export-access-catalog.ts`

Part 3 — Organizations, locations, roles and permissions

Chapter 8 — The multi-tenant model

The Platform serves many clinic businesses from one installation and one database, and each business can have several branches. This chapter explains how the data of one clinic is kept apart from another, how a person can work for clinics, how a signed-in user ends up "inside" one clinic and one branch, and how the Platform decides what "today" means for a clinic. It is written for the product owner, who needs this model to reason about every other chapter.

8.1 Organizations, locations and the tenant boundary

Status: [Implemented]

In simple terms. An **organization** is one clinic business; it is the tenant. A **location** is one branch of that business. A dental group with three branches is one organization with three locations. Almost every record in the Platform (patients, appointments, invoices, staff assignments, queue sessions) carries the organization it belongs to, and most also carry a location.

Technically.

- `organizations` is the tenant table. `locations` belongs to exactly one organization (`organization_id` , deleted with it by `ON DELETE CASCADE`). Each location has one `location_settings` row.
- Tenant tables carry an `organization_id` column, and services always filter by the active organization. Data that differs per branch also carries `location_id` .
- All tenants share one database and one schema. Isolation is done in application code (guards plus `organization_id` filters in every query), not by separate databases or database row-level security.
- The **super admin** stands outside all tenants (`users.global_role = 'super_admin'`) and can enter any active organization.

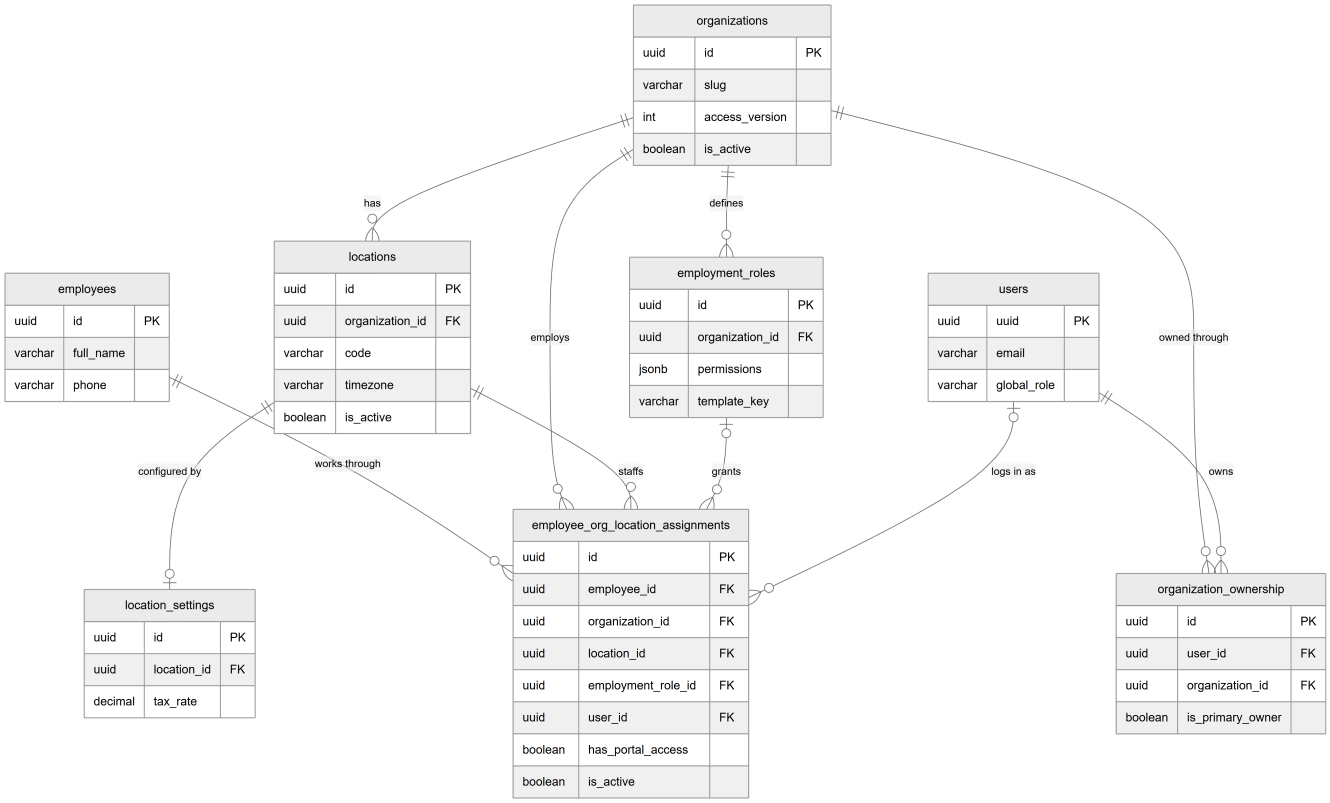

Figure 8.1 — The tables that define who belongs to which organization and location.

The diagram shows eight tables and their relationships. `organizations` (with `id`, `slug`, `access_version`, `is_active`) "has" zero or more `locations` (with `organization_id`, `code`, `timezone`, `is_active`). Each location is "configured by" at most one `location_settings` row (with `location_id` and `tax_rate`). `users` (with `uuid`, `email`, `global_role`) "owns" organizations through `organization_ownership` rows (with `user_id`, `organization_id`, `is_primary_owner`); an organization can have several owners and a user can own several organizations. An organization "defines" zero or more `employment_roles` (with `organization_id`, the `permissions` map and `template_key`). The `employees` table (with `id`, `full_name`, `phone`) has no organization column; a person "works through" zero or more `employee_org_location_assignments`. Each assignment links one employee to one organization ("employs") and one location ("staffs"), optionally to one employment role ("grants"), and optionally to one user account ("logs in as"); it also holds `has_portal_access` and `is_active`.

Technical references: `backend/src/organizations/entities/organization.entity.ts` · `backend/src/locations/entities/location.entity.ts` · `backend/src/auth/services/authorization.service.ts`

8.2 Ownership

Status: [Implemented]

An **owner** is a user linked to an organization through an `organization_ownership` row. Owners are not employees and have no role: the access resolver gives an owner **every permission at its widest scope at every location** of the organization (Chapter 11).

- **How owners are created.** Self-signup creates the clinic and makes the new user its primary owner (`is_primary_owner = true`). Further owners are created from user administration with the "owner" account type and are stored with `is_primary_owner = false`.
- **Several owners, several clinics.** A clinic can have several owners and one user can own several clinics; the pair (`user_id`, `organization_id`) is unique.
- **What an owner can enter.** Every active location of every active organization the owner owns.

Technical references: `backend/src/organizations/entities/organization-ownership.entity.ts` · `backend/src/user/user.service.ts` · `AuthorizationService.isOwner`

8.3 The employee person record and assignments

Status: [Partial] — the data model supports one person at several locations and clinics, but the API only ever creates one assignment per employee.

In simple terms. The Platform separates the **person** from the **job**. The **employee** record holds who the person is (name, phone, national ID number, gender, date of birth, photo). Each **assignment** says where and how that person works: in which organization, at which location, in which role, with which salary, and whether the person may sign in there.

Technically.

- `employees` has **no** `organization_id`. Phone is unique across the whole Platform, and so is the national ID number when given.
- `employee_org_location_assignments` holds, per job: `organization_id`, `location_id`, `employment_role_id` (the **role**, may be empty), `designation`, `employee_code` (unique per organization), an optional queue `token_prefix`, salary fields, `has_portal_access`, `user_id` (the **staff login**, may be empty), `is_primary` and `is_active`. One person can have at most one assignment per location (unique on `employee_id`, `location_id`).
- Permissions are tied to the assignment, not to the person: the same person can be a Doctor at one location and have no role at another.
- **What the API does today.** `POST /employees` creates the person and exactly one assignment, at the location the creator is working in. It refuses a phone number that already exists anywhere on the Platform ("An employee with this phone number already exists"). No endpoint adds a second location or a second clinic to an existing employee; the multi-location model can only be filled outside the API [Partial].
- **Staff logins.** User administration can create a login for an existing employee; that login is linked to all of the person's active assignments in the organization. A login only grants entry where the assignment is active and `has_portal_access` is on.

Technical references: `backend/src/employees/entities/employee.entity.ts` · `backend/src/employees/entities/employee-org-location-assignment.entity.ts` · `EmployeesService.create` · `backend/src/user/user.service.ts`

8.4 Who can enter which clinic and branch

Status: [Implemented]

At sign-in, and whenever a location is selected, the Platform works out the list of places a user may enter (`AuthorizationService.resolveUserAccess`):

Kind of user	Organizations offered	Locations offered
Super admin	Every active organization	Every active location of each
Owner	Every active organization the user owns	Every active location of each
Employee with a staff login	Organizations where the user has an active assignment with portal access, if the organization is active	Exactly those assigned locations that are active

The table shows the three ways to gain entry. When one user is both owner and employee in the same organization, the owner entry wins. Deactivated organizations and locations disappear from the list. The same rules are checked again on every request by `validateAccess` (section 8.5): an owner passes for any active location of the organization, an employee needs an active, portal-enabled assignment at exactly that location.

Technical references: `AuthorizationService.resolveUserAccess` · `AuthorizationService.validateAccess`

8.5 The tenant context on every request

Status: [Implemented]

In simple terms. The web app always tells the API which clinic and which branch the user is working in. The API checks that the user is allowed there before doing anything else.

Technically.

- The access token (a JWT, valid 15 minutes) carries only the user id, email, global role and session id. **It does not carry the organization or location.**
- The web app's axios instance adds two headers to every request from the in-memory auth store: `x-organization-id` and `x-location-id` (lab portal requests carry `x-lab-id` instead).
- `TenantContextGuard`, the second of the five global guards, reads the headers and calls `validateAccess`. Missing headers give 400 "Missing tenant context. Please select an organization and location."; no access gives 403. On success it stores `request.tenantContext`.
- Controllers read the context with `@ActiveTenant()` and pass the organization id (and, where relevant, the location id) to the service, which filters every query by it.
- The guard steps aside for `@Public()` and `@ContextFree()` routes and for super admins, and checks the super-admin role for `@RequireSuperAdmin()` routes. It has **no** path-prefix exceptions.
- The organization and location in the headers also select the access context (Chapter 11) and the active features (Chapter 10).

Technical references: `backend/src/auth/guards/tenant-context.guard.ts` · `TenantContextGuard` · `backend/src/common/decorators/active-tenant.decorator.ts` · `web-app/src/lib/api.ts`

8.6 Sign-in, choosing a location and switching

Status: [Implemented]

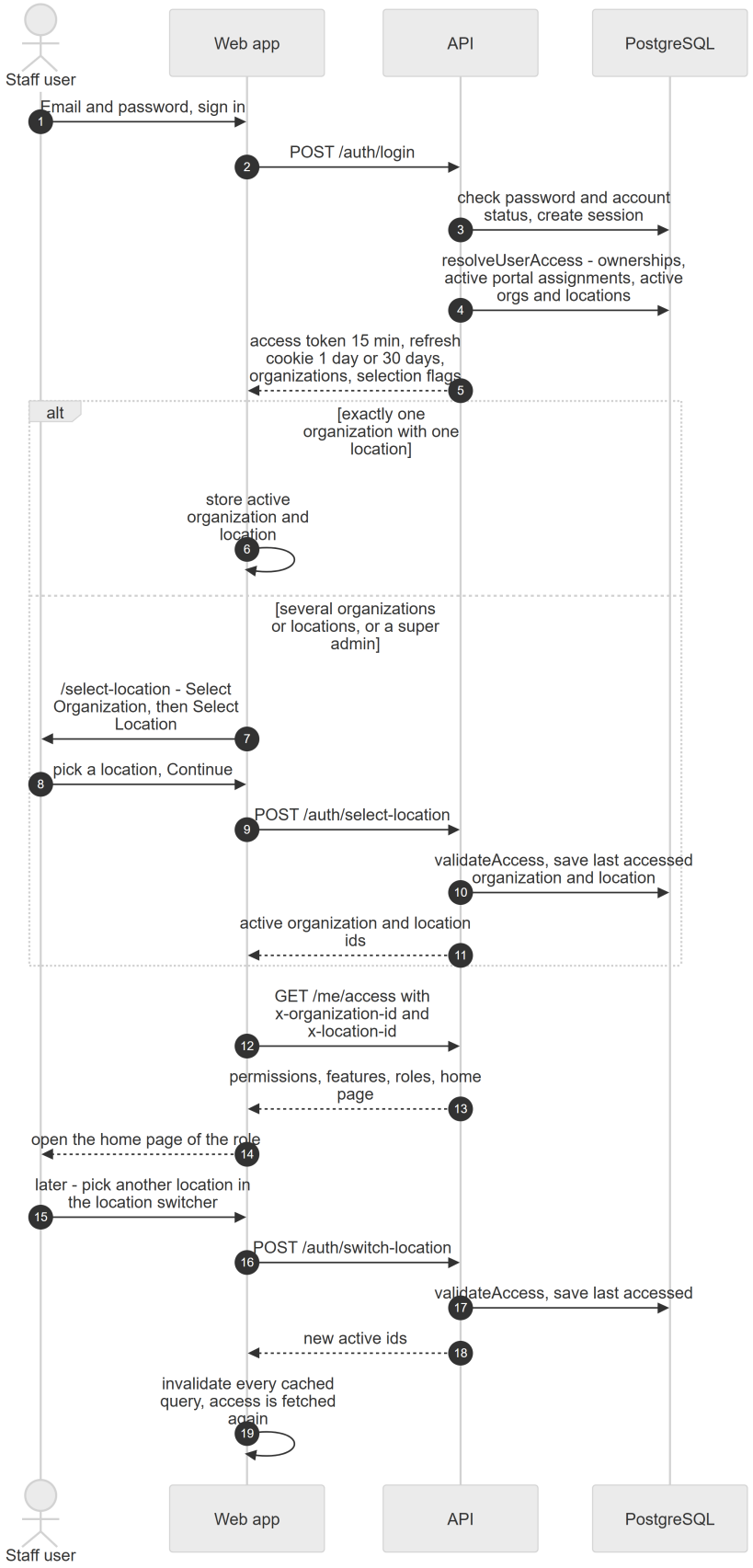

Figure 8.2 — Sequence from sign-in to the first page, and a later location switch.

The diagram has four participants: the Staff user, the Web app, the API and PostgreSQL. (1) The user enters email and password and signs in. (2) The web app calls `POST /auth/login`. (3) The API checks the password and account status and creates a session. (4) It resolves the user's access: ownerships, active portal assignments, active organizations and locations. (5) It returns an access token valid for 15 minutes, sets a refresh cookie valid for 1 day (30 days when the user ticks "keep me signed in"), and returns the list of organizations with two flags

saying whether an organization or a location must still be chosen. An "alt" box shows two paths. If the user has exactly one organization with one location, the web app stores them as active. Otherwise, or for any super admin, the web app shows `/select-location` with "Select Organization" and then "Select Location"; the user picks a location and clicks Continue; the web app calls `POST /auth/select-location`; the API runs `validateAccess` and saves the last accessed organization and location; it returns the active ids. After the box, the web app calls `GET /me/access` with the two tenant headers and receives the permissions, features, roles and home page, then opens the role's home page. The last part shows a later switch: the user picks another location in the location switcher, the web app calls `POST /auth/switch-location`, the API checks access and saves the choice, returns the new ids, and the web app invalidates every cached query so all data and the access context are fetched again.

Rules behind the figure:

- **Automatic choice.** One organization with one location is selected automatically. A super admin always chooses an organization first. When a choice is needed, the API pre-fills the last organization and location the user worked in (`users.last_accessed_organization_id` / `last_accessed_location_id`), if they are still allowed.
- **Where the browser goes.** The root route (`__root.tsx`) redirects: not signed in → `/login`; signed in but no location chosen → `/select-location`. Super admins may open the setup pages (Organizations, Locations, Lab Directory, Config Studio, Subscriptions) without a location. Users who only belong to a lab go to the lab portal.
- **Session restore.** The auth store is in memory only. On page load the web app calls `POST /auth/refresh`; the refresh cookie returns a new token and the same selection data.
- **Switching.** The location switcher in the top bar lists every organization and location the user may enter, grouped by organization. With only one location it shows the name without a menu.

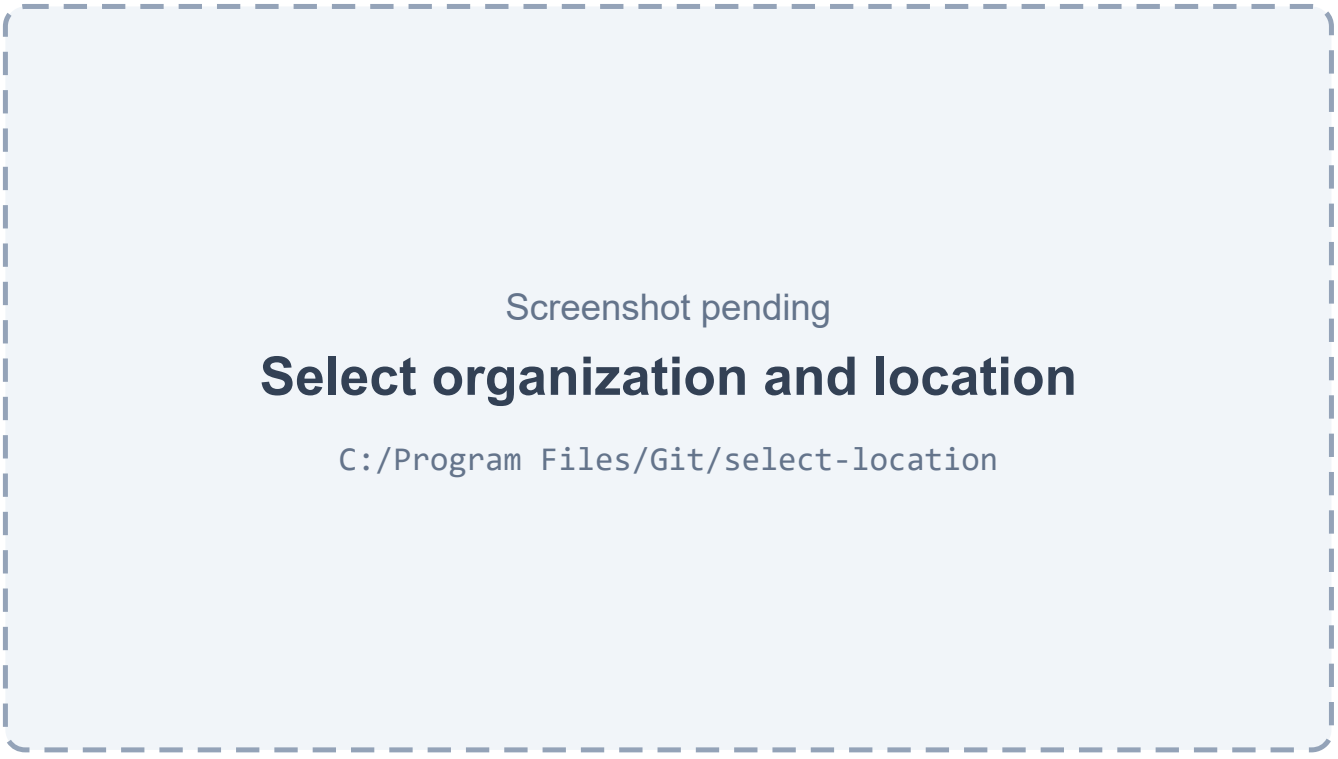

Figure 8.3 — Select organization and location (screenshot pending).

The page appears after sign-in when a choice is needed. Its header reads "Select Organization" or "Select Location", with a welcome line using the user's name ("Choose an organization to continue" or "Choose where you'd like to work today"). The organization step shows one card per organization; picking an organization with a single location selects it at once. The location step shows one card per location of the chosen organization and a **Continue** button. If the organization has no locations yet, the page says "No locations yet" and offers **Create**

First Location, which opens the Locations page. On success a toast says "Location selected" and the user lands on the home page. (The only existing screenshot of this page is excluded from the handbook because it shows compliance badges that the current code no longer displays.)

Technical references: `backend/src/auth/auth.service.ts` (`login` , `selectLocation` , `switchLocation`) · `web-app/src/routes/__root.tsx` · `web-app/src/features/auth/components/LocationSelector.tsx` · `web-app/src/components/layout/LocationSwitcher.tsx`

8.7 Clinic time: the location's timezone

Status: [Implemented]

In simple terms. A clinic's day starts at midnight where the branch is, not where the server is. Each location stores its own timezone, and every "today", "now" and opening-hours check uses it.

Technically.

- `locations.timezone` holds an IANA timezone name. New locations created through `POST /locations` default to `UTC` ; the setup wizard copies the organization's timezone (default `Asia/Karachi`) to every location. The value is not validated; an unknown name falls back to UTC.
- `ClockService` (`common/clock/`) is the single source of clinic time. `timezoneFor(locationId)` reads the timezone (cached for 60 seconds); `today(tz)` gives the local date; `dayRange(tz, date)` gives the UTC start and end of a local day; `at(tz, date, time)` converts a local wall-clock time; `wallClock(tz, instant)` gives the local weekday and time.
- It is used by appointments (working hours, availability, same-day checks), the queue (the queue day and token day), billing, payroll, reports, notifications, employees and the lab modules.
- Scheduled jobs are triggered by the **server** clock (Chapter 6); jobs that need a clinic day compute it through `ClockService` .

Technical references: `backend/src/common/clock/clock.service.ts` · `ClockService` · `backend/src/common/clock/tz.util.ts`

8.8 Limitations and known gaps

- One person cannot be given a second location or clinic through the API (section 8.3) [Partial].
- Deactivating an organization or a location hides it at the next sign-in or selection, but does not end sessions that are already working there; requests keep passing the tenant guard for an owner of a deactivated organization, because `validateAccess` checks the location but not the organization's status for owners [Partial].
- Locations created from the Locations screen get timezone `UTC` and currency `USD` unless changed, because that form has no timezone or currency field (Chapter 9).
- Isolation relies on every service filtering by organization; there is no database-level row security.

Chapter 9 — Organizations and locations

This chapter describes the two records that frame every clinic on the Platform: the **organization** (the clinic business, the tenant) and its **locations** (the branches). It covers their fields, who can create and change them, their settings, and how they are switched off. It is written for the product owner; subscriptions, trials and plan limits get their own chapter (Chapter 45), and only read-only mode is summarised here.

9.1 What these records are for

Status: [Implemented]

- An **organization** answers "who is the clinic?": name, contact details, default timezone and currency, specialties, plan and subscription dates.
- A **location** answers "where does it operate?": address, a short branch code, its own timezone (which defines "today" for queues and reports), its own currency, and per-branch settings such as the tax rate.

The **super admin** manages organizations. Locations are managed by anyone holding the permission `admin.locations.manage` in that organization: owners and super admins always hold it, the system **Admin** role holds it, and a custom role can be given it. The owner's one-time setup wizard (`/setup` , Chapter 45) also edits the organization and its main location.

Sidebar: "Organizations" (super admin only, `/organizations`) and "Locations" (`/locations` , needs `admin.locations.manage`).

Technical references: `backend/src/organizations/` · `backend/src/locations/` · `web-app/src/features/organizations/` · `web-app/src/features/locations/` · `backend/docs/modules/01-organizations-locations.md`

9.2 Organization fields

Status: [Implemented]

Field	Meaning
`name` , `slug`	Display name; `slug` is unique, generated from the name on creation and never regenerated.
`email` , `phone` , `website`	Contact details.
`subscription_plan`	`free` , `starter` , `professional` or `enterprise` . Default `free` . The plan limits are defined in code (`subscriptions/plans.ts`).
`clinic_type`	Free text. No API writes it.
`specialty_tags`	List of `general` , `dental` , `skin` , `ortho` , `physio` , `eye` . Default `general` . Written by the setup wizard; the web app uses it to show specialty-specific screens.
`timezone` , `currency`	Organization defaults: `Asia/Karachi` and `PKR` . The setup wizard copies them to every location.
`max_locations` , `max_users`	Per-organization overrides of the plan limits; empty means "use the plan's limit".
`trial_ends_at` , `paid_until`	Subscription period. Both empty means an **unmanaged** clinic: no expiry and no limits.

Field	Meaning
`onboarding_pending`	True for self-signup clinics until the owner finishes `/setup`.
`access_version`	Counter raised on every access change (roles, assignments, features, ownership); used to invalidate access caches (Chapter 11). Never written by normal saves.
`is_active`	Soft switch-off.

The table explains each column of `organizations` in business terms. Three related tables hang off an organization: `organization_ownership` (its owners), `organization_settings` (legacy settings, section 9.4) and `subscription_payments` (Chapter 45).

Technical references: `backend/src/organizations/entities/organization.entity.ts` · `Organization`

9.3 Managing organizations (super admin)

All endpoints are in `OrganizationsController` and carry `@RequireSuperAdmin()`, so only the super admin can call them, for any organization.

9.3.1 Create an organization

Status: [Partial] — the endpoint works; the screen's form has fields the API refuses.

What it does. Creates a clinic with a name and optional contact details and plan.

How the user does it. On the Organizations page, click **Create**, fill the form and save.

Behind the scenes. `POST /organizations` saves the row, then seeds the organization's feature flags from the catalog defaults (Chapter 10) and the 5 system roles (Chapter 11). The two steps are not one transaction. No owner, no location and no subscription dates are created: the clinic is "unmanaged" until the super admin sets dates or records a payment.

Rules. Name is required (at most 255 characters); email must be valid; phone at most 20, website at most 500 characters; plan must be one of the four enum values. Duplicate slug gives 409.

Known problems (by reading the code, not executed). The form also offers Clinic Type, Timezone and Currency, which the create DTO does not accept, so filling them makes the request fail with 400. The plan selector offers "FREE", "BASIC", "PREMIUM" and "ENTERPRISE", which do not match the API's lower-case values, so choosing a plan also fails with 400. No audit row is written.

Technical references: `OrganizationsController.create` · `OrganizationsService.create` · `CreateOrganizationDto` · `web-app/src/features/organizations/OrganizationManagementView.tsx`

9.3.2 List and view organizations

Status: [Implemented]

`GET /organizations` lists active organizations by name; `GET /organizations/:id` returns one. Deactivated organizations do not appear in the list.

Technical references: `OrganizationsController.findAll` · `OrganizationsController.findOne`

9.3.3 Edit an organization

Status: [Partial] — the endpoint works; the screen sends fields the API refuses.

`PATCH /organizations/:id` accepts name, email, phone, website, plan and `isActive`. The edit dialog sends the whole organization row (including `id`, `slug` and timestamps), which the strict validation pipe refuses with 400 (by reading the code, not executed). A plan change made here is not audited and does not touch the subscription dates.

Technical references: `OrganizationsController.update` · `UpdateOrganizationDto`

9.3.4 Deactivate an organization

Status: [Implemented]

The trash action asks "Deactivate organization?" and calls `DELETE /organizations/:id`, which sets `is_active = false`. Data is kept. The organization disappears from sign-in and selection lists. Sessions that are already working in it are not ended. There is no screen to reactivate it; only `PATCH /organizations/:id` with `isActive: true` does that.

Technical references: `OrganizationsController.deactivate` · `OrganizationsService.deactivate`

Figure 9.1 — Organizations (screenshot pending).

The page is titled "Organizations" with the subtitle "Manage clinic organizations and their settings" and a **Create** button at the top right. A table lists active organizations with the columns Name, Slug, Plan and Status, and an actions column with four icon buttons, each with a tooltip: "Manage features" (opens the feature entitlement page, Chapter 10), "Organization settings", "Edit organization" and "Deactivate organization". The create and edit dialog contains Name, a read-only Slug, Clinic Type, Timezone, Currency and Subscription Plan. Non-super-admins who open the URL are sent to the dashboard.

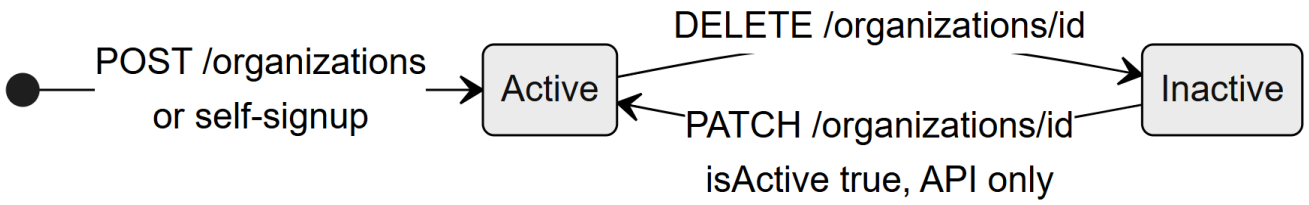

Figure 9.2 — The two states of an organization's `is_active` flag.

The diagram reads left to right. From the start point an organization becomes **Active**, either by `POST /organizations` or by self-signup. `DELETE /organizations/id` moves it to **Inactive**. `PATCH /organizations/id` with `isActive true`, which only the API offers, moves it back to Active. There is no final state: the API never hard-deletes an organization.

9.4 Organization settings (legacy)

Status: [Not implemented] — the settings are stored and editable, but no backend code reads them.

`organization_settings` has one row per organization, created on first read. It holds old feature switches (`feature_appointments`, `feature_inventory`, `feature_finance`, `feature_reminders`, `feature_whatsapp`, `feature_patient_portal`), `allow_location_override`, and booking defaults (`default_slot_duration`, `advance_booking_days`, `cancellation_notice_hours`).

- `GET /organizations/:id/settings` is allowed to the super admin and to owners of that organization.
- `PATCH /organizations/:id/settings` is super-admin only. Limits: slot duration 5–480 minutes, advance booking 1–365 days, notice 0–168 hours.
- **No backend code reads any of these values.** Real feature gating is the feature system (Chapter 10). The web app uses `allow_location_override` only to grey out three inputs on the location settings page.
- The settings screen sends the whole row, which the API should refuse with 400 (by reading the code).

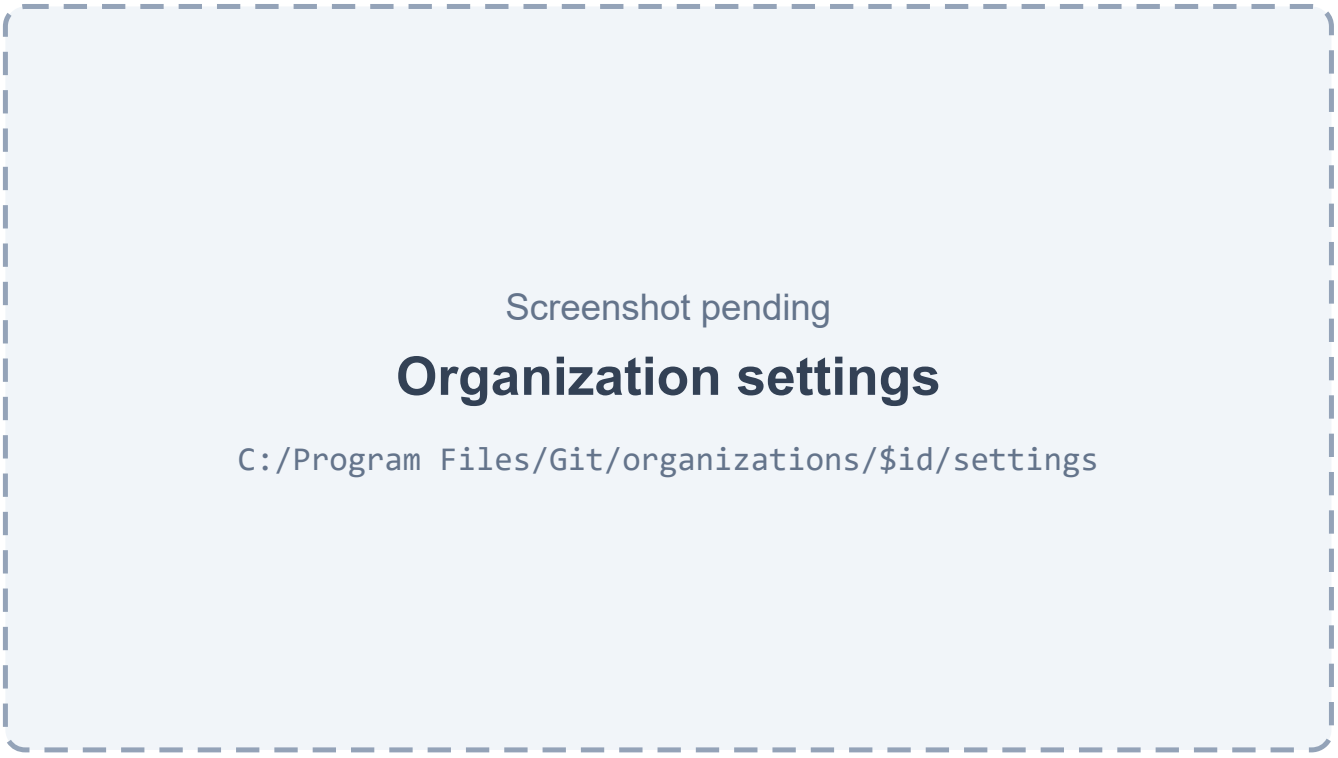

Figure 9.3 — Organization settings (screenshot pending).

The page has a back button to Organizations and two cards. "Global Overrides" holds one switch, **Allow Location Override**. "Booking Defaults" holds three number inputs: Default Slot Duration (mins), Advance Booking Days and Cancellation Notice (hours). A Save button stores the card values. None of these values changes how bookings behave.

Technical references: `backend/src/organizations/entities/organization-settings.entity.ts` · `OrganizationsController.getSettings` · `OrganizationsController.updateSettings` · `web-app/src/features/organizations/OrganizationSettingsFeature.tsx`

9.5 Location fields

Status: [Implemented]

Field	Meaning
`name`	Branch name, for example "Main location".
`code`	Short branch code, unique within the organization. Cannot be changed after creation.
`token_prefix`	Prefix of the daily queue tokens; falls back to `code`. Read by the queue's token engine; no API writes it.
`address`, `city`, `state`, `postal_code`, `country`, `phone`, `email`	Contact details.
`timezone`, `currency`	Defaults `UTC` and `USD` (different from the organization defaults). `ClockService` uses `timezone` for every clinic "today".
`is_main_branch`	A flag; several locations can carry it. The setup wizard treats the first active main branch as "the" main location.
`total_chairs`, `capacity_per_day`, `operating_hours`, `manager_id`, `business_model`	Stored, but no backend code reads them.
`is_active`	Soft switch-off.

The table explains each column of `locations`. Note the timezone default: a location created from the Locations screen gets `UTC`, because that form has no timezone or currency field; only the setup wizard sets the clinic's timezone and currency on locations.

Technical references: `backend/src/locations/entities/location.entity.ts` · `Location`

9.6 Managing locations

All location endpoints are in `LocationsController`. The controller is `@ContextFree()`, because an owner manages locations before choosing one; each handler therefore checks `admin.locations.manage` itself, in the organization of the location being acted on (or, for create and list, the organization in the body or the `x-organization-id` header). An invalid organization gives 400 "Select an organization first"; a missing permission gives 403 `NO_PERMISSION`.

9.6.1 Add a location

Status: [Implemented]

How the user does it. On the Locations page, click **Create Location** and fill Name, Code, Address, City, Country, Phone and Email. A super admin also picks the organization ("Organization (Required)").

Behind the scenes. `POST /locations` first checks the plan's location limit (`SubscriptionsService.assertCanAddLocation`, skipped for super admins), then saves the location and a default settings row as two separate saves (not one transaction). No audit row is written.

Rules and errors. Name and code are required; code at most 50 characters and unique in the organization (409 on duplicates). Plan limit reached gives 403 `PLAN_LIMIT`; an expired subscription gives 402 `SUBSCRIPTION_EXPIRED`.

Technical references: `LocationsController.create` · `LocationsService.create` · `CreateLocationDto`

9.6.2 List, view and edit locations

Status: [Partial] — list and view work; the edit screen sends fields the API refuses.

`GET /locations` returns, for a super admin, every active location of every organization; otherwise the active locations of the active organization (or of every organization where the user holds the permission organization-wide). `GET /locations/:id` returns one. `PATCH /locations/:id` accepts the create fields except `code` and the organization, plus `isActive`. The edit dialog sends the whole row, which the API should refuse with 400 (by reading the code). Setting `isActive: true` reactivates a location **without** the plan-limit check.

Technical references: `LocationsController.findAll` · `LocationsController.findOne` · `LocationsController.update` · `UpdateLocationDto`

9.6.3 Deactivate a location

Status: [Implemented]

The trash action asks "Deactivate location?" and calls `DELETE /locations/:id` (`is_active = false`). Nothing stops deactivating the last location or the main one. Staff assignments, appointments and queue sessions at that location are left as they are; the location stops being offered at sign-in and in the switcher. The list hides inactive locations, so the screen cannot reactivate one.

Technical references: `LocationsController.deactivate` · `LocationsService.deactivate`

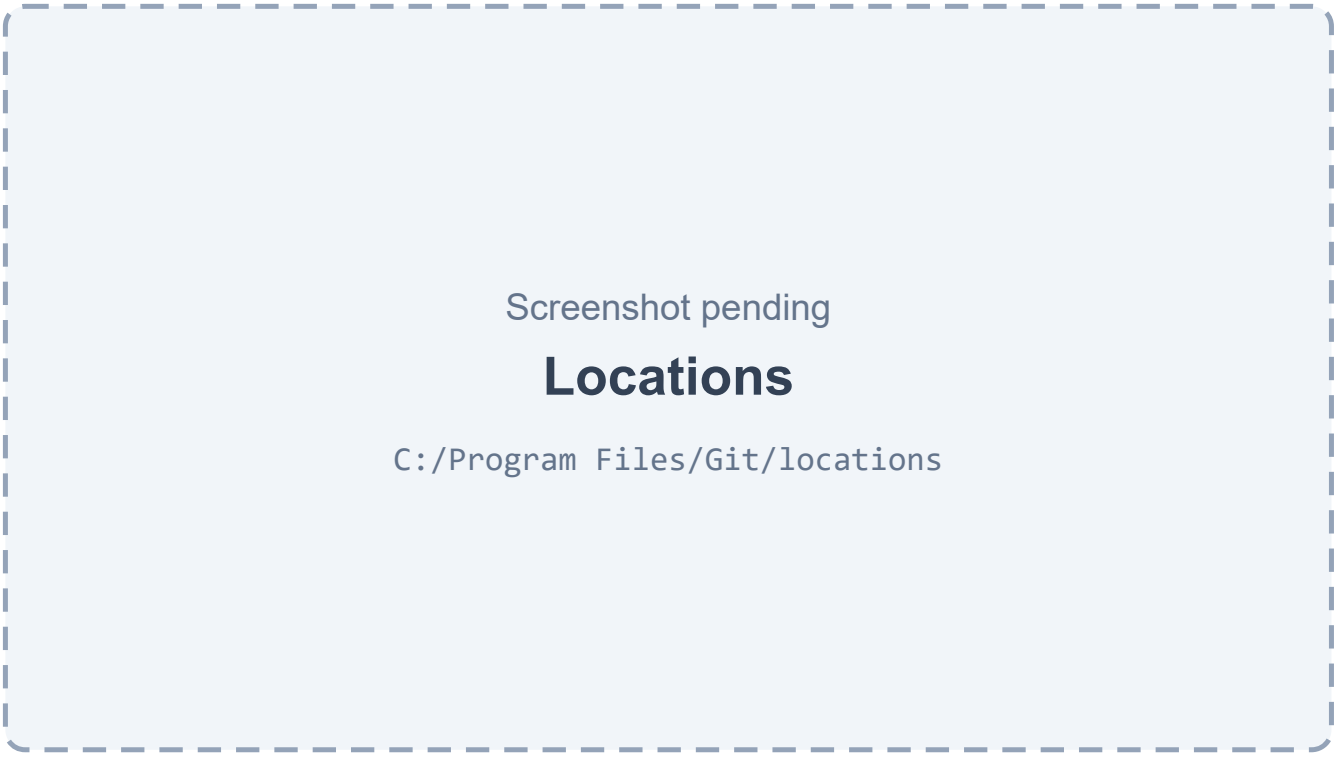

Figure 9.4 — Locations (screenshot pending).

The page is titled "Locations" with the subtitle "Manage clinic locations and their settings" and a **Create Location** button. A table lists active locations with the columns Name, Organization, City, Phone and Status, and three icon actions with tooltips: "Location settings", "Edit location" and "Deactivate location". The create and edit dialog holds the fields listed in section 9.6.1; its code field shows the example "e.g. NYC-01". Toasts confirm "Location created successfully", "Location updated successfully" and "Location deactivated".

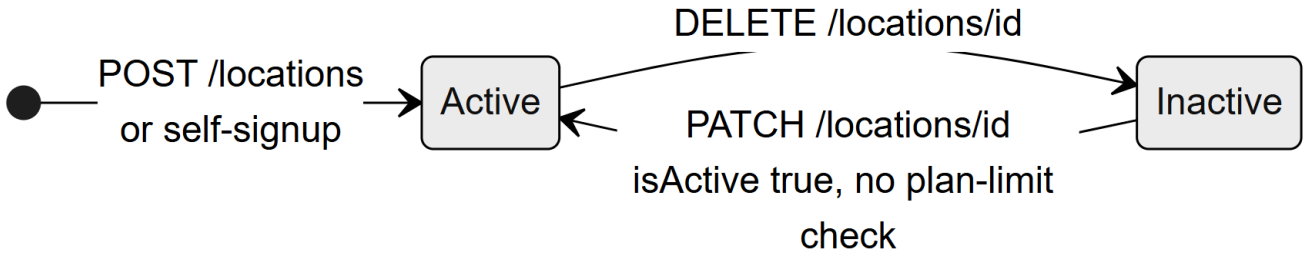

Figure 9.5 — The two states of a location's `is_active` flag.

The diagram reads left to right. A location becomes **Active** through `POST /locations` or self-signup. `DELETE /locations/id` moves it to **Inactive**. `PATCH /locations/id` with `isActive true` moves it back to Active, and the label notes that this path does not check the plan's location limit. There is no final state.

9.7 Location settings

Status: [Partial] — the tax rate and a few other values are used; the edit screen's save is expected to fail, and several stored values do nothing.

Each location has one `location_settings` row, created with the location or on first read.

Setting	Used by
`tax_rate` (percent, two decimals, default 0)	**Used.** Sales tax applied when an invoice is issued (tax is added on top of prices), and by lab billing and purchase orders.
`max_appointments_per_day` (default 50; 0 means no cap)	**Used.** Daily booking cap for the location.
`logo_url`	**Used.** Fallback logo for printed invoices when the clinic theme has none.
`appointment_slot_duration` (default 30)	Read only by the web app, as the default length in the booking form.
`advance_booking_days`, `cancellation_notice_hours`, `reminder_enabled`, `reminder_days_before`, `reminder_hours_before`, `reminder_channels`, `payment_gateway`, `late_fee_percentage`, `primary_color`, `theme`	Stored; no code reads them.

The table lists each setting and whether anything uses it.

- `GET /locations/:id/settings` and `PATCH /locations/:id/settings` both need `admin.locations.manage`. Staff without it (receptionists, doctors) cannot read the slot duration, so their booking form falls back to 30 minutes.
- **Field-name mismatch.** The update DTO names several fields differently from the stored columns (for example `defaultSlotDuration` against `appointmentSlotDuration`, `maxDailyAppointments` against `maxAppointmentsPerDay`). The screen sends the stored names plus `id` and timestamps, so its save should fail with 400; a client sending the DTO names would get success but nothing stored for those fields. In practice, the daily cap and slot duration cannot be changed through the API (by reading the code, not executed). The tax rate has the same name on both sides.
- The tax rate can also be set in the setup wizard's location step (0–100).

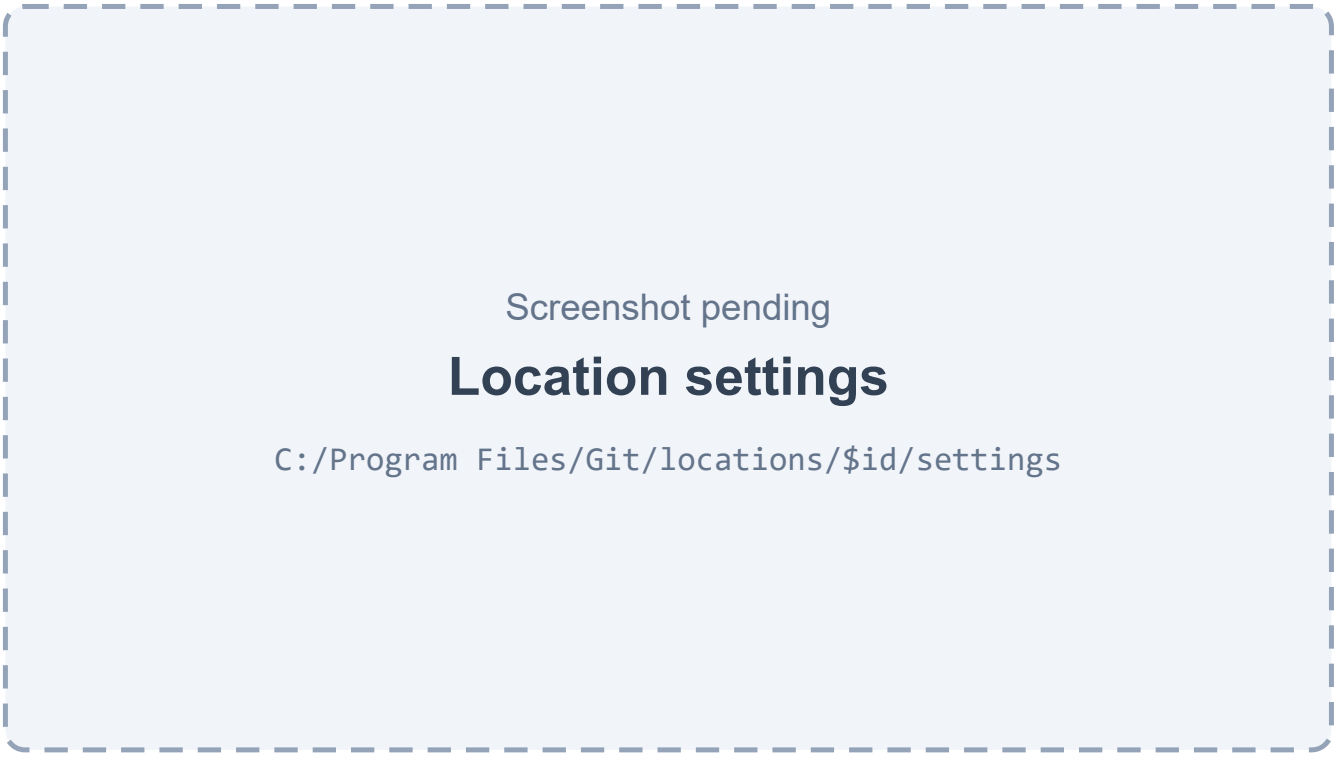

Figure 9.6 — Location settings (screenshot pending).

The page has a back button to Locations. An "Appointment Settings" card holds Slot Duration (mins), Max Appointments / Day, Advance Booking Days and Cancellation Notice (hours); the organization's legacy "Allow Location Override" switch can grey out three of these. A "Payment & Billing" card holds Tax Rate (%) and Late Fee

(%). The page also embeds the queue settings section for this location (the Queue chapter, Chapter 30, covers it). A "Reminder Settings" card exists in the code but is hidden until message delivery exists. Users without the permission see "No access — You don't have permission to manage this location's settings." On success a toast says "Location settings updated successfully".

Technical references: `backend/src/locations/entities/location-settings.entity.ts` · `UpdateLocationSettingsDto` · `LocationsController.getSettings` · `LocationsController.updateSettings` · `backend/src/billing/tax/location-tax-rate.ts` · `web-app/src/features/locations/LocationSettingsFeature.tsx`

9.8 Subscriptions and read-only mode (summary)

Status: [Implemented]

An organization with subscription dates is "managed": it is on a trial (self-signup gives 14 days with Professional limits) or paid until a date, and after that date it gets 3 grace days. When the grace days have passed, the clinic is in **read-only mode**: the fifth global guard, `SubscriptionGuard`, refuses every write with 402 `SUBSCRIPTION_EXPIRED`, while reads, sign-in, the account and setup pages and the super admin keep working. Recording a subscription payment (a manual super-admin action) lifts read-only mode at once; the status is computed on each request and cached for 30 seconds per process. Chapter 45 covers plans, limits, payments and the setup wizard in full.

Technical references: `backend/src/subscriptions/subscription.guard.ts` · `backend/src/subscriptions/plans.ts` · `SubscriptionsService.isReadOnly`

9.9 Limitations and known gaps

- Several edit screens (organization edit, organization settings, location edit, location settings) send whole rows and are expected to fail with 400 under the strict validation pipe; this was found by reading the code and not executed.
- Organizations and locations cannot be reactivated from any screen.
- New locations default to `UTC` and `USD`; timezone and currency are not validated.
- `is_main_branch` is not exclusive, and the last or main location can be deactivated.
- Organization and location create, edit and deactivate write no audit rows; only subscription changes are audited.
- `organization_settings` and most location settings are stored but unused.

9.10 Owner's FAQ

Can an owner create a new organization? No. Only the super admin can, or a new owner through self-signup, which creates a separate clinic.

Can an owner add a branch? Yes, from Locations, as long as the plan's location limit allows it and the subscription has not expired.

Can a branch code be changed? No. The code is fixed at creation; the token prefix falls back to it.

Where is the sales tax rate set? In the location's settings (Tax Rate %) or in the setup wizard's location step. Each branch has its own rate.

What happens to patients and staff when a branch is deactivated? Nothing is deleted or moved. The branch is no longer offered at sign-in and in the location switcher.

What does "unmanaged" mean? An organization with neither trial end nor paid-until date. It never expires and has no plan limits. Organizations created by the super admin start this way.

Chapter 10 — Features: entitlement, enablement and location overrides

A **feature** is a named part of the Platform, such as "Invoices" or "Lab Management", that can be on for one clinic and off for another. This chapter explains the three switches that decide whether a feature is active (the super admin's entitlement, the owner's switch and a per-location override), the 24 features that exist, how the backend enforces them and which screens control them. It is written for the product owner, who sells and configures features per clinic.

10.1 The three switches

Status: [Implemented]

In simple terms. The platform operator decides what a clinic is allowed to have; the clinic owner decides what the clinic actually uses; and the owner can switch a feature off at one branch. A feature works at a branch only when all three agree.

Switch	Who sets it	Stored in	Meaning
Entitled	Super admin	`org_feature_flags.is_entitled`, optional `entitled_until`	The feature is part of this clinic's plan. With `entitled_until`, it is a trial that ends at that moment.
Enabled	Owner (or anyone with `admin.features.manage`); the super admin can also set it	`org_feature_flags.is_enabled`	The clinic has switched the feature on. Only possible while entitled.
Location override	Owner (same permission)	`location_feature_overrides.is_disabled`	Switches the feature off at one location. It can never switch on something the organization has off.

The table lists the three switches, who controls each, where it is stored and what it means. A feature is **active** for an organization when its catalog entry is active, it is entitled (and the trial has not ended), and it is enabled. It is active at a location when, in addition, that location has not switched it off.

Defaults and missing rows.

- When an organization is created (by the super admin or by self-signup), the Platform writes one `org_feature_flags` row per active catalog feature: features that are "on by default" in the catalog are both entitled and enabled; all others are neither.
- If an organization has **no** row for a feature, it counts as entitled, and enabled according to the catalog default. This keeps organizations that predate entitlement working.
- A trial end date must be in the future when set. When it passes, the feature becomes "not entitled" with the reason `trial_expired`; the owner sees it locked.

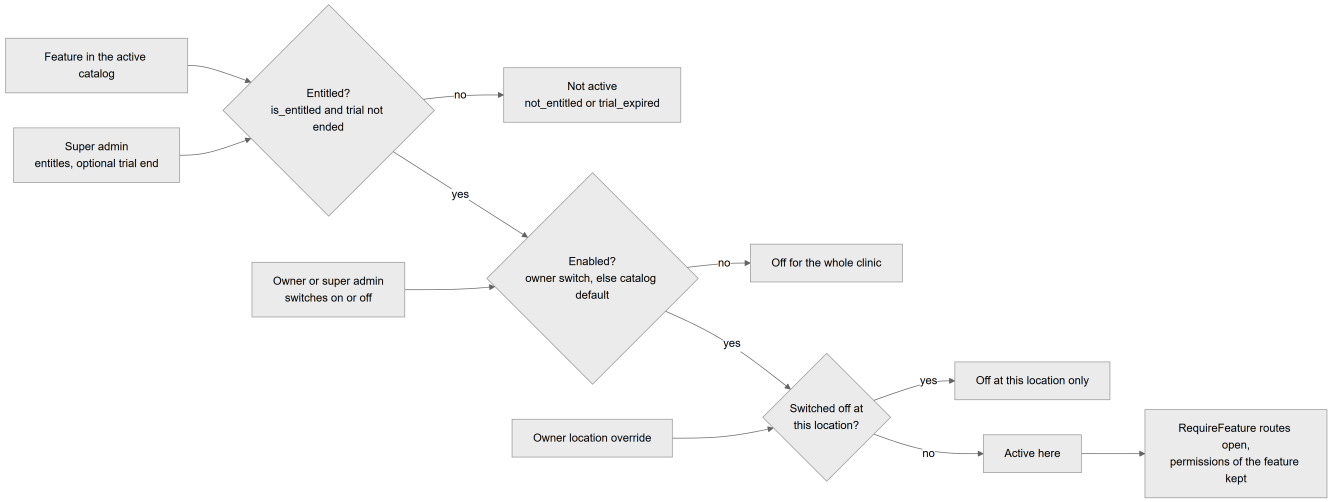

Figure 10.1 — The decision path from a catalog feature to "active at this location".

The flowchart reads left to right. It starts from "Feature in the active catalog" and reaches the first decision, "Entitled? is_entitled and trial not ended", which the super admin controls by entitling with an optional trial end. If the answer is no, the feature is "Not active" with the reason `not_entitled` or `trial_expired`. If yes, the second decision asks "Enabled? owner switch, else catalog default"; the owner (or the super admin) controls this switch. If no, the feature is "Off for the whole clinic". If yes, the third decision asks "Switched off at this location?", controlled by the owner's location override. If yes, the feature is "Off at this location only". If no, the feature is "Active here", and the final box says what that means: routes marked `@RequireFeature` open, and the permissions that belong to the feature are kept.

Technical references: `backend/src/feature-flags/feature-state.ts` · `featureState` · `resolveEnabledFeatures` · `narrowByLocation` · `FeatureFlagsService.seedFlagsForOrg`

10.2 The feature catalog

Status: [Implemented]

The catalog (`feature_catalog`) lists every feature key with a name, a module, a description, specialty tags, a default and a sort order. It is seeded from code on every start: keys that are missing are inserted, existing rows are never changed (so super-admin edits stay), and keys that are not built are deactivated.

There are **24 active features**:

Key	Name	On by default	Specialty tags	What it controls
`appointments.core`	Appointments	yes	general	12 permissions: booking and the whole queue.
`patients.core`	Patient Records	yes	general	9 permissions, including viewing, registering, clinical records and conducting consultations.
`patients.teeth_chart`	Teeth Chart	no	dental, ortho	`patients.chart.edit` ; one endpoint marked `@RequireFeature` .
`patients.treatment_plans`	Treatment Plans	yes	general	`plans.manage` , `plans.templates.manage` ; three endpoints marked `@RequireFeature` .
`patients.prescriptions`	Prescriptions	yes	general	`prescriptions.create` , `prescriptions.library.manage` ; six endpoints marked `@RequireFeature` .

Key	Name	On by default	Specialty tags	What it controls
`patients.documents`	Documents	yes	general	`patients.documents.manage`.
`patients.insurance`	Insurance	no	general	No permission; the three patient insurance endpoints are marked `@RequireFeature`.
`billing.invoices`	Invoices	no	general	9 billing permissions (desk, invoices, payments, refunds, cash close).
`billing.discounts`	Discounts & Coupons	no	general	`billing.discount.apply`, `billing.discount.custom`. Coupons no longer exist; discounts work through discount types.
`billing.expenses`	Expense Tracking	no	general	`expenses.view`, `expenses.manage`.
`billing.financial_summary`	Financial Dashboard	no	general	The financial summary endpoints (`@RequireFeature`) and the dashboard's finance widgets.
`billing.credit_packages`	Credit Packages	no	general	The credit package endpoints (`@RequireFeature`); no page uses them.
`billing.commissions`	Doctor Commissions	no	general	`earnings.own.view`, `earnings.all.view`, `compensation.manage`.
`comms.whatsapp`	WhatsApp	no	general	Nothing. No WhatsApp sending exists.
`comms.in_app`	In-App Notifications	yes	general	Nothing reads this key; in-app notifications work whether it is on or off.
`inventory.core`	Inventory	no	general	`inventory.view`, `inventory.stock.adjust`, `inventory.po.manage`.
`inventory.alerts`	Inventory Alerts	no	general	Read by the daily inventory alert job, which only writes a log line.
`hr.payroll`	Payroll	yes	general	`payroll.manage`.
`hr.leave`	Leave Management	no	general	`leave.approve`.
`hr.commissions`	Commissions	no	general	Nothing. Doctor commissions use `billing.commissions`.
`reports.financial`	Financial Reports	no	general	`reports.financial.view`, `finance.overview.view`.
`reports.clinical`	Clinical Reports	no	general	`reports.clinical.view`.
`reports.exports`	Data Exports	no	general	`reports.export`.
`dental.lab_orders`	Lab Management	no	dental, ortho	7 lab permissions (orders, billing, catalog, payables, links, templates).

The table lists each active key, its display name, whether new clinics get it entitled and enabled, its specialty tags and what it actually controls in the code. Most features work through permissions: when a feature is not active, every permission tied to it is dropped from every role at resolution time (Chapter 11). Eleven permissions belong to no feature ("core") and are always available: staff management and the nine Admin permissions.

Status of individual keys:

- [Implemented]: every key in the table whose "What it controls" names permissions or `@RequireFeature` endpoints.
- [Partial]: `billing.credit_packages` (backend only, no page), `inventory.alerts` (the job only logs), `billing.discounts` (the name still says "Coupons").
- [Not implemented]: `comms.whatsapp` and `hr.commissions` (switching them on changes nothing); `comms.in_app` (the switch has no effect).

Twelve keys are retired [Not implemented]. Their seed entries are commented out, and on every start the Platform deactivates any catalog row with these keys so nobody can switch on a feature that does nothing: `appointments.reminders`, `appointments.recurring`, `appointments.online_booking`, `appointments.waiting_room`, `patients.portal`, `patients.crm`, `billing.pdf_receipts`, `billing.insurance_claims`, `billing.online_payments`, `comms.sms`, `comms.email`, `dental.before_after`. (Some of the underlying ideas exist in other forms, such as the doctor-wise queue and the patient CRM tab, but these keys gate nothing.)

Catalog editing. `GET`, `POST /feature-flags/catalog` and `PATCH /feature-flags/catalog/:id` let the super admin read, add and edit catalog rows [Partial: backend only, no screen calls them].

Technical references: `backend/src/feature-flags/feature-catalog.seed.ts` · `FEATURE_CATALOG_SEED` · `RETIRED_FEATURE_KEYS` · `FeatureFlagsService.seedCatalog` · `backend/src/access/catalog/permissions.ts`

10.3 How the backend enforces features

Status: [Implemented]

There are two mechanisms.

1. **Through permissions (the common case).** Every permission names one feature. `AccessResolver` keeps a permission only if its feature is active at the request's organization and location (`filterByFeatures`). Turning a feature off therefore takes the permission away from every role **without changing the roles**; turning it back on restores access exactly as it was.
2. **Through `@RequireFeature(key)`.** Used where no permission expresses the feature, mainly on `@AnyMember()` routes. `FeatureFlagGuard`, the third global guard, collects every `@RequireFeature` key on the handler and its controller (both apply) and refuses with 403 'The "" feature is not enabled for this location' if any is not active. Super admins and `@Public()` routes skip the check. The location header narrows the set; the guard trusts it without a membership check, which is safe because a location can only remove features.

Caching. `FeatureFlagsService` keeps the active set per organization, and per organization and location, in process memory for 30 seconds. Every change clears the cache in the process that made it. The change also raises the organization's **access version** (automatically for `org_feature_flags` writes, explicitly for location overrides), which is broadcast over Redis so every other process drops its cache at once, and the web app gets `access.changed` and refetches. If Redis is down, other processes catch up within the 30-second lifetime.

Every change is audited in `audit_logs` (entity types `org_feature_flag` and `location_feature_override`), in the same transaction as the change, with the before and after values.

Technical references: `backend/src/feature-flags/guards/feature-flag.guard.ts` · `FeatureFlagGuard` · `requiredFeaturesOf` · `FeatureFlagsService.getEnabledFeatures` · `FEATURE_CACHE_TTL_MS` · `AccessVersionSubscriber` · `FeatureAdminService`

10.4 Super admin: entitle features for a clinic

Status: [Implemented]

What it does. Decides which features a clinic may use, optionally as a trial, and can also flip the clinic's own switch.

How the user does it. On the Organizations page, click the "Manage features" action of a clinic. The page lists features by module. Per feature, the **Entitled** switch grants or withdraws it, a date picker ("No trial end") sets a trial end, and the **Enabled** switch flips the owner's switch. Per module, **Entitle module** and **Withdraw module** act on all features of that module at once.

Behind the scenes — technically.

Action	Endpoint
Load the page	`GET /feature-flags/org/:organizationId`
Entitle or withdraw one feature, with optional trial end	`PATCH /feature-flags/org/:organizationId/:featureKey/entitlement`
Entitle or withdraw a module or a list of keys	`POST /feature-flags/org/:organizationId/entitlement/bulk`
Flip the enabled switch (no entitlement check)	`PATCH /feature-flags/org/:organizationId/:featureKey`
Flip many enabled switches	`POST /feature-flags/org/:organizationId/bulk`
Create missing rows from catalog defaults	`POST /feature-flags/org/:organizationId/seed`

The table maps each action on the screen to its endpoint. All carry `@RequireSuperAdmin()`. Rows are locked (`pessimistic_write`) while being changed, and each changed row writes an audit entry recorded as a platform action.

Rules. The trial end must be a valid date in the future; it is ignored when withdrawing. Re-entitling keeps a running trial; an ended trial is dropped. Unknown keys give 404.

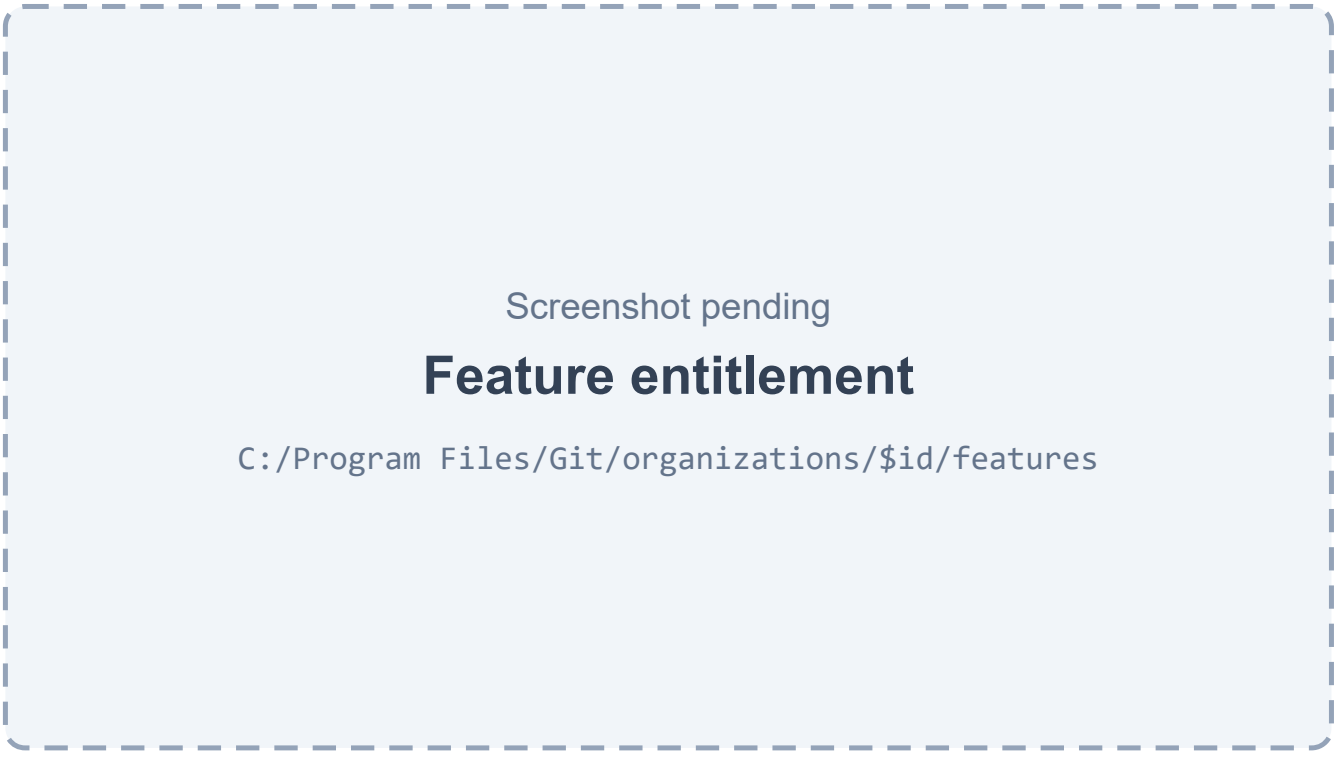

Figure 10.2 — Feature entitlement (screenshot pending).

The page is titled "Feature entitlement" with a back button to Organizations and the subtitle "Entitled = in this clinic's plan (optionally a trial). Enabled = the clinic owner's switch." A search box ("Search features") filters the list. Each module is a card with **Entitle module** and **Withdraw module** buttons. Each feature row shows its name, specialty tags as small badges, a state badge ("Not entitled", "Trial ended" or "Off by clinic" when it

applies), the description, a trial-end date picker, and two labelled switches, **Entitled** and **Enabled**, each with a tooltip. Empty and error states read "No features in the catalog" and "Could not load this organization's features."

Technical references: `backend/src/feature-flags/feature-flags.controller.ts` · `FeatureAdminService.setEntitlement` · `FeatureAdminService.bulkSetEntitlement` · `web-app/src/features/feature-flags/components/OrgFeatureEntitlements.tsx` · `EntitlementRow`

10.5 Owner: switch features on or off

Status: [Implemented]

What it does. Lets the clinic turn the features in its plan on or off for the whole clinic, or off at single locations.

How the user does it. Open **Features** in the clinic setup part of the sidebar (`/admin/features`). Each feature has a switch. Locked features (not entitled, or trial ended) show a lock, a "Locked" badge and a Roman-Urdu note asking the user to contact the platform admin, and their switch is disabled. Turning a feature off opens a confirmation, "Turn off ?", that lists the permissions that stop working and the roles that lose them ("No role uses this feature." when none). A location icon opens "Per-location settings for ", where each location can be switched off or back on.

Behind the scenes — technically.

Action	Endpoint	Rule
Load the page	`GET /admin/features`	Returns modules, features with entitled, enabled, active, trial end, locked reason, permission count, and the locations where each is off; plus the active locations.
Show the impact before turning off	`GET /admin/features/:key/impact`	Lists the feature's permissions and the active roles that grant any of them.
Turn on or off for the clinic	`PATCH /admin/features/:key`	Refused with 403 `NOT_ENTITLED` (with the reason `not_entitled` or `trial_expired`) when the clinic is not entitled.
Switch off or back on at one location	`PUT /admin/features/:key/locations/:locationId`	The location must belong to the clinic. "Back on" only removes the override; it never turns on a feature the clinic has off.

The table maps each owner action to its endpoint and rule. The whole controller requires `admin.features.manage` , a core permission that owners, super admins and the system Admin role hold. The organization always comes from the active tenant, never from the URL. A super admin acting here is recorded in the audit log as a platform action.

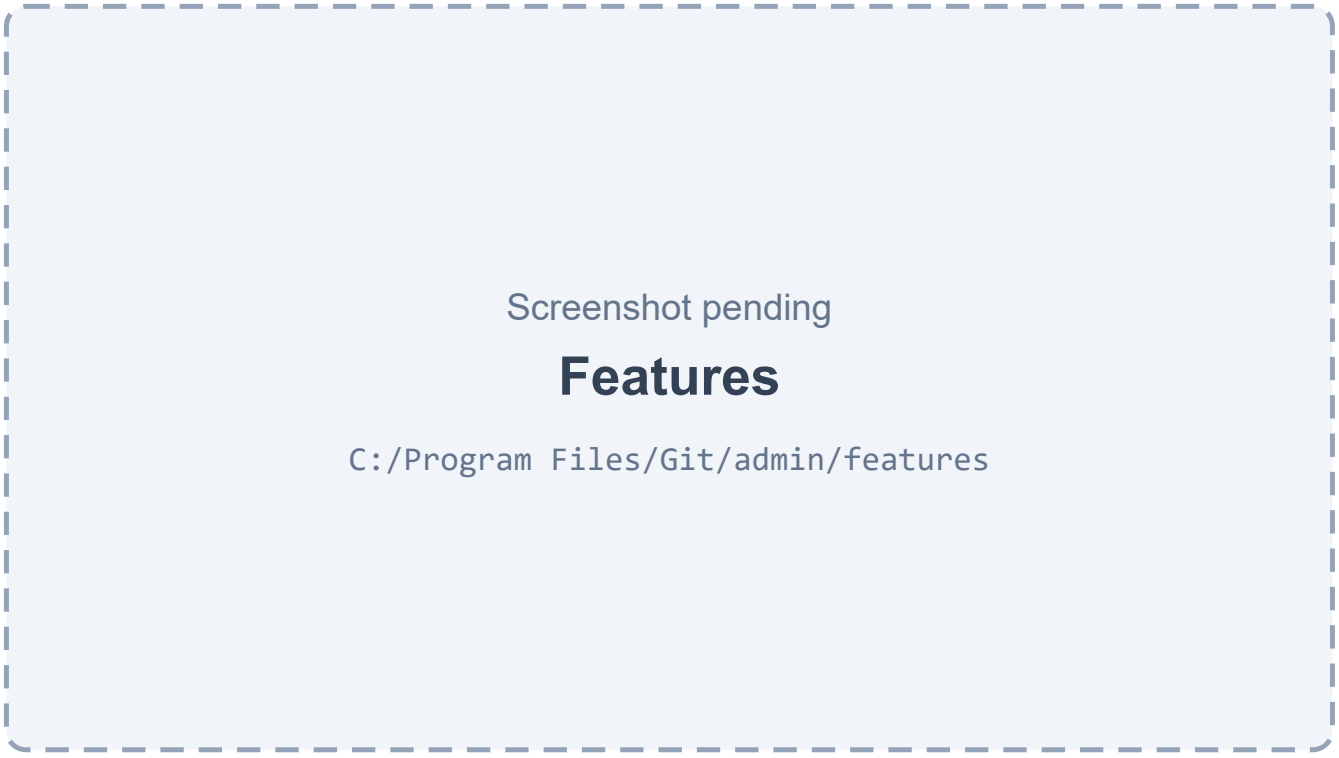

Figure 10.3 — Features (screenshot pending).

The page is titled "Features" with the subtitle "Turn the features in your plan on or off for the clinic, or for one location." A filter row has a search box ("Search features") and two selectors, "All modules" and "All states". Features are grouped by module. Each row shows the name, an "On", "Off" or "Locked" badge, a "Trial until " badge during a trial, an "Off at N locations" badge when overrides exist, the description, a per-location icon button and the main switch with a tooltip such as "Turn off for the clinic". Locked rows show "Plan mein nahi — super admin se rabta karein" or "Trial khatam ho gaya — super admin se rabta karein". The per-location sheet lists each location as "Follows the clinic (on)" or "Off at this location" with a switch. Empty states read "No features yet — The platform has not set up any features for this clinic." and "No locations".

Technical references: `backend/src/feature-flags/admin-features.controller.ts` · `AdminFeaturesController` · `FeatureAdminService.setEnabled` · `FeatureAdminService.setLocationOverride` · `FeatureAdminService.getImpact` · `web-app/src/features/feature-settings/`

10.6 How the web app uses features

Status: [Implemented]

- `GET /me/access` returns the active feature keys for the active organization and location together with the permissions (Chapter 11). Because permissions are already filtered by feature, most screens only ask `can(...)`.
- An older endpoint, `GET /feature-flags/my-features`, returns the same active set; `FeatureFlagsContext` loads it and offers `useFeature(key)` and a `RequireFeaturePage` wrapper, used by six pages and some widgets (for example the dashboard's finance widgets check `billing.financial_summary`).
- When features change, the `access.changed` socket event makes the web app refetch both.

Technical references: `web-app/src/context/FeatureFlagsContext.tsx` · `web-app/src/components/feature/RequireFeaturePage.tsx` · `FeatureFlagsController.getMyFeatures`

10.7 Limitations and known gaps

- Three active keys (`comms.whatsapp` , `comms.in_app` , `hr.commissions`) have no effect; an owner can switch them without any change in behaviour.
- `inventory.alerts` only produces a server log line, no notification.
- Catalog editing has no screen.
- The legacy `organization_settings.feature_*` switches (Chapter 9) are unrelated to this system and do nothing.

10.8 Owner's FAQ

What happens to roles when I turn a feature off? Nothing is deleted. The feature's permissions stop working for everyone; turning the feature back on restores them exactly.

Can I turn a feature on at one branch only? Not directly. Turn it on for the clinic and switch it off at the other branches.

Why can't I switch a feature on? It is not in your plan, or its trial has ended. Only the super admin can entitle it.

What happens when a trial ends? The feature becomes inactive at once (within the 30-second cache), and the owner's switch shows it locked with "trial ended".

Does turning a feature off log people out? No. Open screens receive an access update and lose the related menu items and buttons.

Chapter 11 — Roles and permissions

This chapter explains who may do what inside a clinic on the Platform. It covers the fixed list of 70 permissions, how they are grouped into roles, the five system roles every clinic starts with, how a clinic builds its own roles safely, and how the backend and the web app check permissions on every request and every screen. It is written for the product owner; the deeper customization screens (sidebar builder, theme, the owner's Features page) are covered in the Customization chapter, Chapter 58.

11.1 The model in one paragraph

Status: [Implemented]

The Platform never checks role names. It checks **permissions**, such as `billing.invoice.void`. A **role** is a clinic-owned bundle of permissions, stored as a map `{ permission: scope or true }` in `employment_roles.permissions`. An employee gets a role through an **assignment** at one location (Chapter 8), so the same person can have different roles at different branches. **Owners and super admins** have no role: they always get every permission at its widest scope. A permission whose **feature** is not active (Chapter 10) is dropped at resolution time. Anything not granted is denied (**fail-closed**).

Technical references: `backend/src/access/` · `backend/src/access/catalog/permissions.ts` · `docs/access-control/README.md`

11.2 The 70 permissions

Status: [Implemented]

The catalog is fixed in code; clinics combine its keys into roles but can never invent new ones. Each permission has a label, a module group, a feature, optionally a list of permissions it **implies**, optionally a set of **scopes**, and optionally a **high-risk** mark.

Module (group)	Permissions	Feature
Patients (12)	`patients.view` *, `patients.create`, `patients.edit`, `patients.delete` !, `patients.export` !, `patients.price.view`, `patients.clinical.view`, `patients.clinical.edit`, `patients.documents.manage`, `patients.chart.edit`, `plans.manage`, `plans.templates.manage`	`patients.core`; documents → `patients.documents`; chart → `patients.teeth_chart`; plans → `patients.treatment_plans`
Appointments (5)	`appointments.view` *, `appointments.book`, `appointments.reschedule`, `appointments.cancel`, `appointments.override_rules` !	`appointments.core`
Queue (7)	`queue.board.view`, `queue.checkin`, `queue.entry.manage`, `queue.entry.reprioritize`, `queue.doctor.own`, `queue.doctor.any`, `queue.display.manage`	`appointments.core`
Consultation (3)	`consultation.conduct` *, `prescriptions.create`, `prescriptions.library.manage`	`patients.core`; prescriptions → `patients.prescriptions`

Module (group)	Permissions	Feature
Billing (9)	`billing.desk.use`, `billing.invoice.view`*, `billing.invoice.edit`, `billing.invoice.void`!, `billing.payment.collect`, `billing.refund.issue`!, `billing.price.override`!, `billing.cash_close`, `billing.activity.view`	`billing.invoices`
Discounts (2)	`billing.discount.apply`, `billing.discount.custom`!	`billing.discounts`
Expenses (2)	`expenses.view`, `expenses.manage`	`billing.expenses`
Commissions (3)	`earnings.own.view`, `earnings.all.view`, `compensation.manage`!	`billing.commissions`
Lab (7)	`lab.orders.view`*, `lab.orders.create`, `lab.billing.manage`, `lab.catalog.manage`, `lab.payables.manage`, `lab.links.manage`, `lab.templates.share`	`dental.lab_orders`
Inventory (3)	`inventory.view`, `inventory.stock.adjust`, `inventory.po.manage`	`inventory.core`
Team (4)	`employees.view`, `employees.manage`, `payroll.manage`!, `leave.approve`	core; payroll → `hr.payroll`; leave → `hr.leave`
Reports (4)	`reports.clinical.view`, *`reports.financial.view`*, `reports.export`!, `finance.overview.view`	`reports.clinical`, `reports.financial`, `reports.exports`
Admin (9)	`admin.features.manage`!, `admin.roles.manage`!, `admin.users.manage`!, `admin.nav.manage`, `admin.settings.manage`, `admin.locations.manage`, `admin.audit.view`, `admin.theme.manage`, `notifications.settings.manage`	core (always on)

The table lists every permission by module, with its feature. An asterisk (*) marks the 7 **scoped** permissions; an exclamation mark (!) marks the 13 **high-risk** permissions. "Core" permissions belong to no feature and can never be switched off.

Scopes. A scoped permission is granted with how far it reaches:

- `own` — only records tied to the user as an employee (for example patients the doctor has seen through an appointment or the queue);
- `location` — records of the active location;
- `org` — every record of the organization.

A wider scope includes the narrower ones. Unscoped permissions are stored as `true` and reach the whole organization.

Implies. Granting a permission also grants what it implies, transitively. Examples: `patients.create` implies `patients.view`; `consultation.conduct` implies `patients.clinical.edit`, which implies `patients.clinical.view`; `queue.doctor.any` implies `queue.doctor.own`; `compensation.manage` implies `earnings.all.view`, which implies `earnings.own.view`; `appointments.override_rules` implies `appointments.book`. An implied scoped permission takes the granting permission's scope when it supports it, otherwise its narrowest scope.

Technical references: `backend/src/access/catalog/permissions.ts` · `PERMISSIONS` · `expandImplies` · `normalizeGrants` · `filterByFeatures`

11.3 Bundles: None, Basic and Full, and Advanced mode

Status: [Implemented]

So that owners do not tick 70 boxes, the roles screen offers three ready levels per module:

- **None** — nothing from the module.
- **Basic** — a sensible everyday set. For example: Patients Basic = view at location, register, edit, manage documents; Appointments Basic = view at location, book, reschedule, cancel; Queue Basic = board, check-in, manage entries; Consultation Basic = conduct at `own`, write prescriptions; Billing Basic = desk, view invoices at location, collect payments; Reports Basic = clinical reports at location; Admin Basic = clinic settings and audit log.
- **Full** — every permission of the module at its widest scope.

Advanced opens the module's full list of permissions, each with a checkbox and, for scoped ones, a scope selector. When a role's grants in a module match no level exactly, the module shows **Custom**.

Technical references: `backend/src/access/catalog/bundles.ts` · `BUNDLES` · `matchBundle` · `web-app/src/features/roles/components/ModuleBundleRow.tsx`

11.4 The five system roles

Status: [Implemented]

Every organization gets five **system roles** (templates). They are inserted when the organization is created and checked again on every start, insert-missing only: an existing system role is never changed, and one that is missing is recreated. System roles are **read-only**; to customise one, clone it.

Role	Home page	What it can do by default
Admin	`/dashboard`	Runs the clinic: the Full bundle of every module, minus the clinical acts `consultation.conduct`, `prescriptions.create` and `queue.doctor.own` and minus `earnings.own.view` (it still gets `queue.doctor.own` and `earnings.own.view` back through `queue.doctor.any` and `earnings.all.view`). It can run and force-complete any doctor's queue, manage roles, users, features, locations, the sidebar and the theme.
Doctor	`/queue/doctor`	Own patients (`patients.view` own), clinical records view and edit, dental chart, documents, treatment plans; own appointments; own queue; conduct consultations (own); write prescriptions; own earnings; lab orders (view own, create); clinical reports (own).
Receptionist	`/appointments`	Patients at the location (view, register, edit, see prices, documents); appointments at the location (view, book, reschedule, cancel); queue board, check-in, manage entries, TV display; billing desk, invoices at the location, collect payments, apply preset discounts; view lab orders at the location.
Assistant	`/appointments`	Patients at the location with clinical view and dental chart editing; appointments view and booking; queue board and check-in; inventory view; lab orders view at the location.
Accountant	`/billing`	Treatment prices; billing desk; invoices at the location (view, edit, void); payments, refunds, cash close, billing activity; expenses view and manage; every doctor's earnings; financial reports at the location, finance overview, report export; lab billing and lab payables.

The table lists the five templates with their home page and default powers. The **home page** is where a member of that role lands after sign-in; the web app only uses it if the user may open that page, otherwise it falls back to `/dashboard`. Owners and super admins are not shown here because they have no role.

Technical references: `backend/src/access/catalog/templates.ts` · `ROLE_TEMPLATES` · `AccessTemplatesService`

11.5 Custom roles and the roles screen

Status: [Implemented]

What it does. Lets the clinic create its own roles (for example "Hygienist"), copy a system role and change it, set its home page and see who has it.

How the user does it. Open **Roles & permissions** (`/admin/roles`). The list shows system roles first, then custom ones, each with a "System" or "Custom" badge and its member count; a search box ("Search roles…") and a type filter narrow it. **New role** asks for a name ("e.g. Hygienist") and a description and starts empty. Opening a role shows three tabs:

- **Permissions** — one row per module with None / Basic / Full and **Advanced**;
- **Home page** — the page members land on ("Default (dashboard)" when empty);
- **Members** — everyone with this role, per location ("Nobody has this role yet" when empty; roles are assigned on each employee's page).

The header offers **Delete**, **Clone**, **Discard** and **Save changes**. Leaving with unsaved changes asks "Leave without saving?". Granting a high-risk permission opens a "Grant high-risk" dialog that requires a **Reason**.

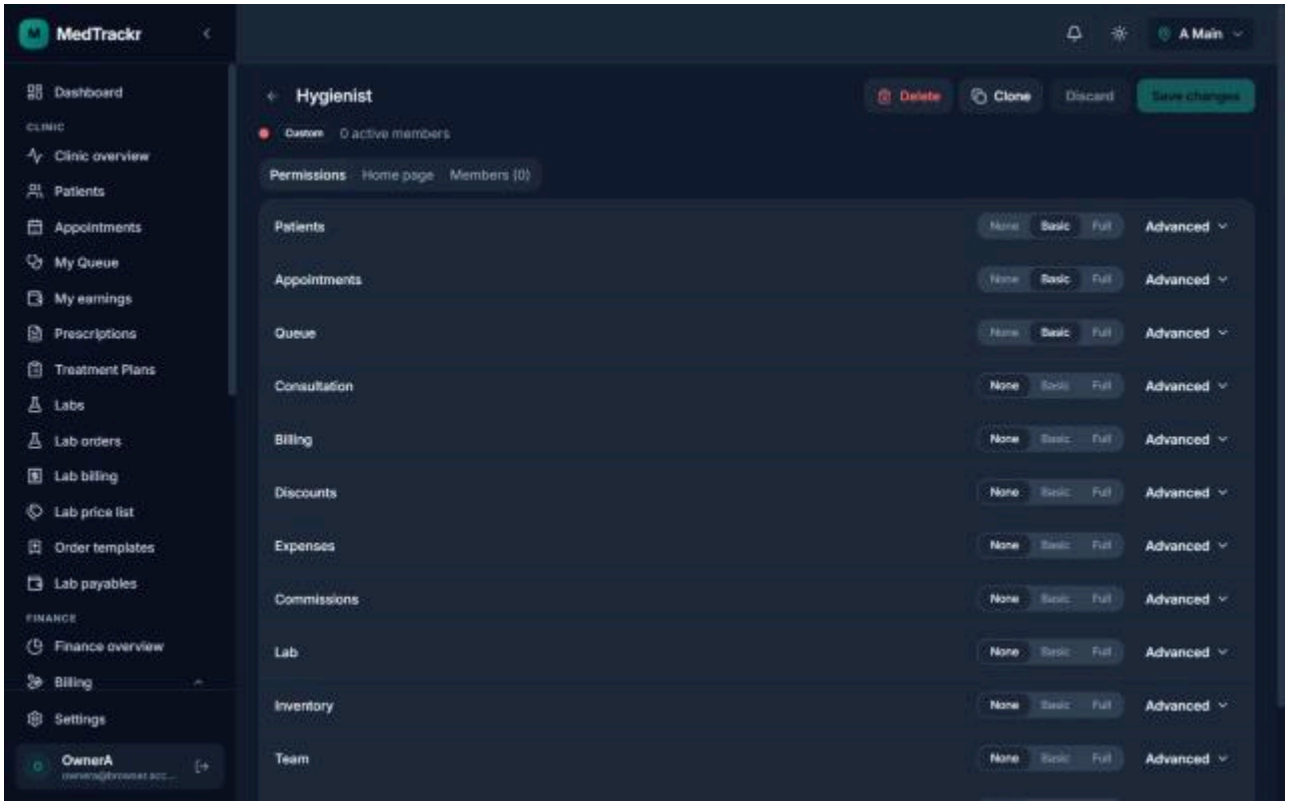

Figure 11.1 — The role editor for a custom role, showing None / Basic / Full per module.

The screenshot was taken on 2026-10-23 in dark theme and shows the former working name "MedTrackr" in the corner; the product is now called the Platform and each clinic sets its own name. The left sidebar shows the full owner menu (Dashboard, the Clinic section with Clinic overview, Patients, Appointments, My Queue, My earnings, Prescriptions, Treatment Plans, Labs, Lab orders, Lab billing, Lab price list, Order templates, Lab payables, and the Finance section with Finance overview and Billing), with Settings and the signed-in test owner at the bottom. The top bar shows the notification bell, the theme toggle and the active location. The main area shows a custom role named "Hygienist" with a back arrow, a "Custom" badge and "0 active members", and the buttons Delete, Clone, Discard and Save changes. Under the header are the tabs Permissions (selected), Home page and Members (0).

Each module (Patients, Appointments, Queue, Consultation, Billing, Discounts, Expenses, Commissions, Lab, Inventory, Team) has a None / Basic / Full selector and an "Advanced" expander; in this role Patients, Appointments and Queue are set to Basic and the other modules to None.

Figure 11.2 — Roles (screenshot pending).

The page is titled "Roles" with the subtitle "What each role can see and do. System roles are templates — clone one to customise it." A **New role** button sits at the top right, and a filter row holds the search box and the "All roles / System / Custom" type filter. Roles appear as cards or table rows with name, badge, description and member count, and row actions for opening, cloning and deleting. Empty state: "No roles yet — Create a role and pick what it can do, module by module."

Behind the scenes — technically.

Action	Endpoint
List roles with bundle summary and member counts	`GET /admin/roles`
One role with its members	`GET /admin/roles/:id`
Create	`POST /admin/roles`
Clone (any role, including system roles)	`POST /admin/roles/:id/clone`
Edit name, description, permissions, home page, active flag	`PATCH /admin/roles/:id`
Delete	`DELETE /admin/roles/:id`
Catalog, bundles and templates for the editor	`GET /access/catalog`

The table maps each action to its endpoint. All need `admin.roles.manage`. Each write runs in one transaction with an `audit_logs` row (entity type `employment_role`) holding before and after values and the reason. A legacy `/employment-roles` controller serves the employee form's role picker (`employees.view`) and applies the same rules to writes.

Rules and validations.

- **Escalation protection.** A user who is not an owner or super admin cannot grant a permission they do not hold, or a wider scope than they hold. Only newly added or widened grants are checked, implies included. Refusal: 403 `ESCALATION` listing the permissions.
- **Reason for high-risk grants.** Newly granted high-risk permissions need a reason, or 400 `REASON_REQUIRED`.
- **Valid maps only.** Unknown keys, a missing scope on a scoped permission, or a scope on an unscoped one give 400 `INVALID_PERMISSIONS`.
- **System roles are read-only** (403 `SYSTEM_ROLE_READ_ONLY`) and cannot be deleted.
- **Unique names** per organization (409 `ROLE_NAME_TAKEN`). A clone without a name becomes " (copy)", "(copy 2)" and so on.
- **No delete while in use.** A role with active members cannot be deleted (409 `ROLE_HAS_MEMBERS`). Deletion is permanent.
- **Home page** must be an in-app path starting with a single `/`.

Technical references: `backend/src/employees/employment-roles.admin.controller.ts` · `EmploymentRolesService` · `backend/src/employees/employment-roles.rules.ts` · `findEscalations` · `highRiskAdded` · `AccessCatalogController` · `web-app/src/features/roles/`

11.6 Resolving access: AccessResolver, cache and access version

Status: [Implemented]

In simple terms. For each user, clinic and branch, the Platform works out once what the user may do, remembers the answer, and throws the answer away as soon as anything relevant changes.

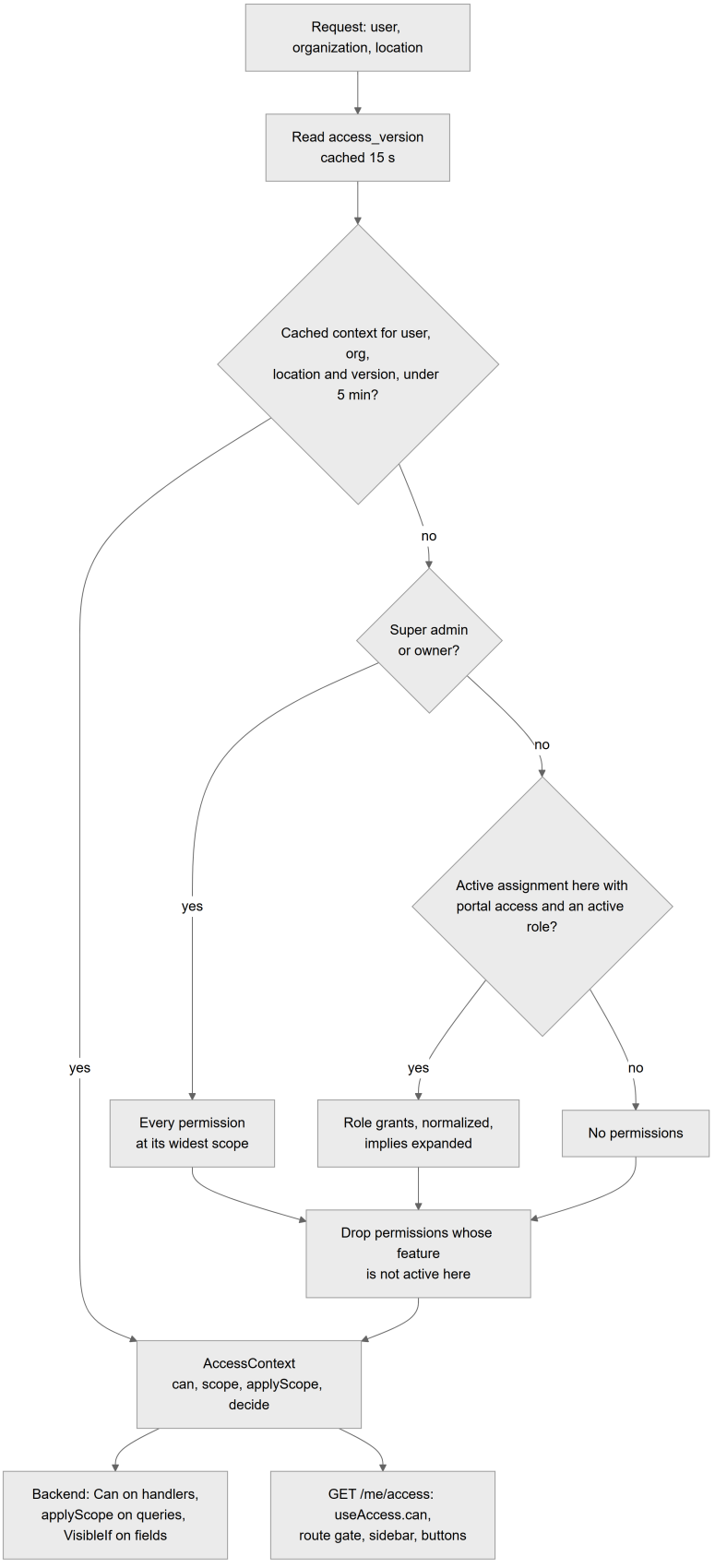

Figure 11.3 — The access resolution path from a request to the permission checks in the backend and the web app.

The flowchart reads top to bottom. A request brings a user, an organization and a location. The resolver first reads the organization's `access_version` (cached for 15 seconds). It then asks whether a cached context exists for this user, organization, location and version that is less than 5 minutes old; if yes, it goes straight to the AccessContext. If not, it asks "Super admin or owner?". If yes, the user gets every permission at its widest scope. If no, it asks whether the user has an active assignment at this location with portal access and an active role. If no,

the user gets no permissions; if yes, the role's grants are normalized and expanded with everything they imply. All three branches meet at "Drop permissions whose feature is not active here", which produces the AccessContext, offering `can`, `scope`, `applyScope` and `decide`. The context feeds two final boxes: the backend (`@Can` on handlers, `applyScope` on queries, `@VisibleIf` on fields) and the web app (`GET /me/access`, `useAccess().can`, the route gate, the sidebar and the buttons).

Technically.

- `AccessResolver.resolve(user, organization, location)` builds an `AccessContext` holding: the user, organization, location, version, `isSuperAdmin`, `isOwner`, `isMember` (super admin, owner or portal-enabled assignment), the employee id, active features, granted permissions, the role and its home page.
- The cache lives in process memory: at most 10,000 entries, 5 minutes each, keyed on user, organization, location and **access version**.
- `organizations.access_version` is raised by one after any committed change to employment roles, assignments (role, location, organization, active flag, portal access, login), organization feature flags or ownership. An entity subscriber does this automatically for TypeORM writes, once per transaction after commit; location overrides and raw SQL bump it explicitly.
- A bump is broadcast over Redis to every process, which drop their cached contexts and feature sets, and the Socket.IO gateway emits `access.changed` to the room `org:<id>`; open browser tabs refetch their access. If Redis is down, other processes catch up within 15 seconds (the version cache).
- **Fail-closed.** A missing key is no access. Roles with only old-style boolean flags give nothing. A role that is inactive, or belongs to another organization, gives nothing. A location header that is not a valid id means no location, so staff get no role permissions.

Technical references: `backend/src/access/access.resolver.ts` · `AccessResolver` · `backend/src/access/access-context.ts` · `AccessContext` · `backend/src/access/version/` · `AccessVersionService` · `AccessVersionSubscriber` · `AccessChangeBus`

11.7 Checking access in the backend

Status: [Implemented]

Every HTTP handler must say who may call it, with exactly one of these markers (on the method or its controller):

Marker	Meaning	Checked by
`@Can('perm', ...)`	All listed permissions are required; class and method lists merge.	`AccessGuard` (403 `NO_PERMISSION` with the missing keys)
`@AnyMember()`	Any active member of the location (owners and super admins too).	`AccessGuard`
`@Public()`	No sign-in (the route must also be in `PUBLIC_ROUTES`).	`JwtAuthGuard` list, all guards skip
`@RequireSuperAdmin()`	Super admin only.	`TenantContextGuard`
`@ContextFree()`	The handler checks access itself (for example `/locations`, `GET /me/access`).	the handler

The table lists the five markers, what they mean and which component enforces them. A controller protected by its own `@UseGuards(...)` (lab portal, magic links) also counts.

- **The coverage test.** `access-coverage.spec.ts` loads every controller and fails if any handler outside the auth and session controllers has none of the markers, or still uses the removed `@Roles()`. At runtime, a handler without a marker would pass the access guard, so this test is the safety net.

- **Row scope.** Handlers that list data take `@Access() access` and call `access.applyScope(queryBuilder, 'patients.view', columns)`. For `own` it adds the caller's employee condition, for `location` the active location condition, for `org` nothing extra; with no grant it adds a condition that matches no rows. Patients, for example, are "own" when the doctor has an appointment or queue entry with them, and "location" when their primary location or an appointment or queue entry is at the active location.
- **Field hiding.** A response property marked `@VisibleIf('perm')` is removed by a global interceptor when the caller lacks the permission; with no access context every guarded field is removed. It is used today for treatment and treatment-plan prices (`patients.price.view`).
- **Range decisions.** `access.decide(perm, { value, min, max })` returns allow, deny or "needs approval". It is used for discounts: changing a preset discount outside its range needs `billing.discount.custom`.
- **Rule for developers.** Code never checks role names; it asks `can(...)`.

Technical references: `backend/src/access/access.guard.ts` · `AccessGuard` · `backend/src/access/decorators/access.decorators.ts` · `backend/src/access/access-coverage.spec.ts` · `backend/src/access/visible-if/visible-if.ts` · `backend/src/patients/access/patient-scope.ts`

11.8 Checking access in the web app

Status: [Implemented]

- `GET /me/access` returns, for the active organization and location: the version, owner and super-admin flags, active features, granted permissions with scopes, the role, the employee id, the home page and the clinic's sidebar layout. `AccessProvider` loads it (fresh for 30 seconds, refetched on window focus and on every `access.changed` event) and serves it through `useAccess()`.
- **Buttons and widgets** ask `useAccess().can('perm')` or wrap content in `<Can perm="...">`.
- **Routes** declare `staticData: { can: ... }` (one permission, or an array meaning any of them). One gate in the root route, `RouteAccessGate`, checks every matched route and shows the **No access** page instead of the screen when the check fails.
- **The sidebar** is built from a developer-owned registry of 39 items, each with its own requirement (a permission, super admin only, owner or super admin, or lab member). Items the user may not open are hidden. The clinic's saved sidebar layout only arranges items; it never grants access.
- **The permission list in the web app** is a generated copy of the backend catalog (`features/access/generated/catalog.ts`), refreshed with `npm run access:export`.

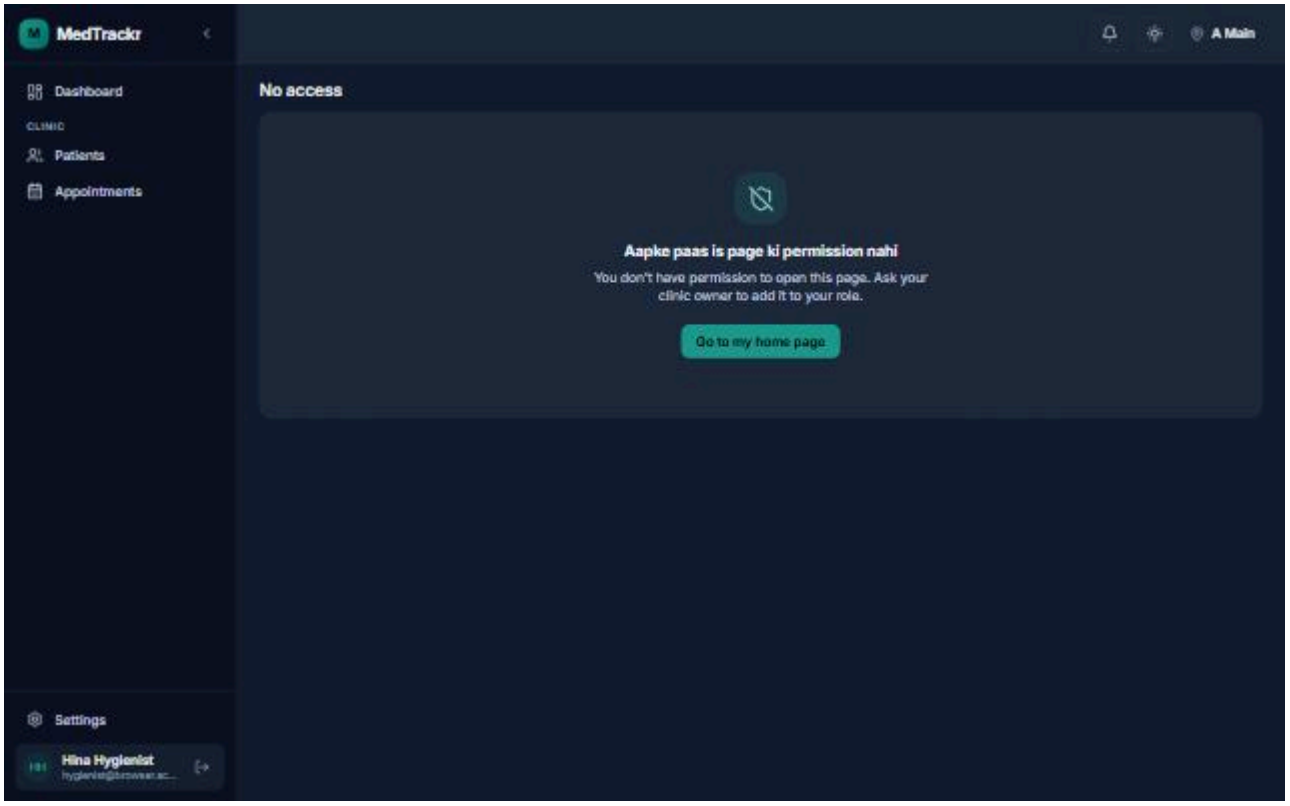

Figure 11.4 — The page shown when a role lacks a page's permission.

The screenshot was taken on 2026-10-23 in dark theme and shows the former working name of the product in the corner. A test user in a custom role opened a page their role does not allow. The sidebar is short: Dashboard, and under "Clinic" only Patients and Appointments, with Settings and the user at the bottom. The main area is titled "No access" and shows a shield icon, the Roman-Urdu headline "Aapke paas is page ki permission nahi" ("You do not have permission for this page"), the English text "You don't have permission to open this page. Ask your clinic owner to add it to your role." and a button "Go to my home page", which opens the role's home page, or the dashboard when none is set.

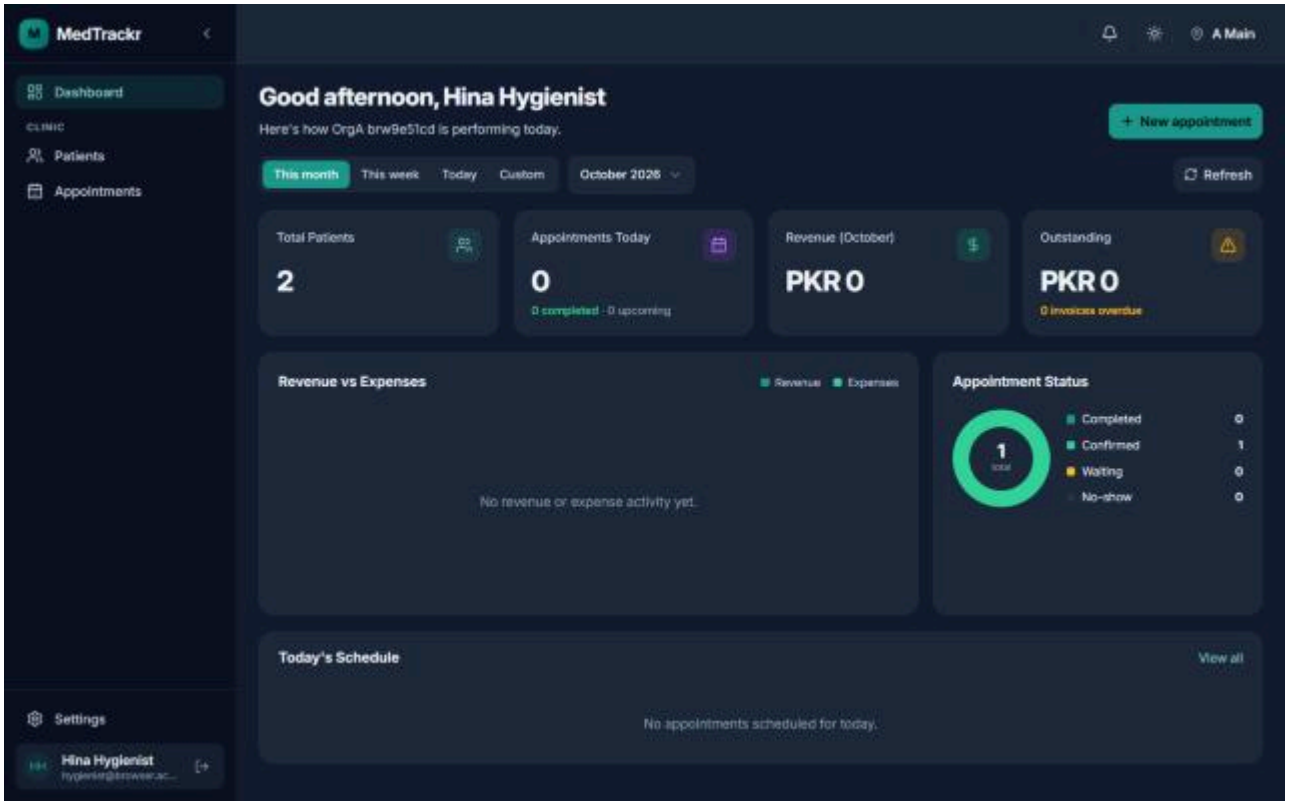

Figure 11.5 — The sidebar of a user in a restricted custom role.

This screenshot from 2026-10-23 (dark theme, former working name in the corner) is used only to show the sidebar; the dashboard content in it predates the dashboard redesign of 2026-10-23. The user holds a custom "Hygienist" role with the Basic level for Patients, Appointments and Queue. The sidebar therefore shows only Dashboard and, under "Clinic", Patients and Appointments, plus Settings at the bottom. Every other menu item is hidden because the role lacks its permission or the item needs an owner or super admin.

Technical references: `backend/src/access/access.controller.ts` · `MeAccessController` · `web-app/src/features/access/components/AccessProvider.tsx` · `RouteAccessGate` · `web-app/src/features/access/components/NoAccess.tsx` · `web-app/src/features/access/nav/nav.registry.ts` · `web-app/src/features/access/nav/resolveNav.ts` · `useAccess`

11.9 Limitations and known gaps

- A system role deleted directly in the database is recreated on the next start; a clinic cannot permanently remove a system role (the API refuses to delete one anyway).
- Missing markers are caught by a unit test, not at runtime.
- `POST /auth/switch-location` still returns eight old permission flags derived from the new permissions; the web app ignores them.
- Permissions follow the assignment's location: a staff member with no assignment at the active location gets no role permissions there.

11.10 Owner's FAQ

Can a receptionist see treatment prices? Yes with the default Receptionist role, which holds `patients.price.view`. Without it, prices are removed from the API responses, not only hidden on screen.

Can I give my manager the right to edit roles without making them all-powerful? Yes: grant `admin.roles.manage`. They still cannot grant anything they do not hold themselves.

Why can't I edit the Doctor role? System roles are read-only templates. Clone it and edit the copy, then assign the copy.

How quickly does a role change take effect? On the next request: the change raises the access version, every server drops its cached answer, and open screens refetch their access.

What does a doctor see with `patients.view` at scope own? Only patients they have an appointment or queue entry with.

Where is the history of role changes? In the audit log, as `employment_role` entries with before and after values and the reason given for high-risk grants.

Part 4 — Data model

Chapter 12 — Data model overview

This chapter explains how the Platform's database is organised as a whole: who owns the schema, which naming and money conventions the tables follow, and how tenant data is kept apart. It is written for the product owner who needs to reason about the data before changing a feature or answering a support question. Chapters 13 to 27 then walk through each business domain table by table; this chapter is the map they share.

12.1 The database in numbers

The Platform stores all of its data in one PostgreSQL database. The backend defines **153 entity classes, one per table**, spread across **31 owning modules** (a module "owns" a table when the table's entity class lives in that module's folder). The schema has 278 enforced foreign keys. It has no views, no table inheritance and no many-to-many junction tables generated by the ORM; every link between two tables is an ordinary column.

The field-level reference for every table is the generated entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`. It lists every column, type, default, index, constraint and foreign key, and it was produced from the same metadata the ORM uses to build the schema. This handbook names only the key columns of each table; when you need the full column list, open the catalog. The catalog has two known defects: the "Notes" column of the ten inventory tables is shifted one row down, and one note claims a trigger on `form_schema_versions` that does not exist.

12.2 Who owns the schema

12.2.1 Entities are the source of truth

Status: [Implemented]

The shape of every table — its columns, primary key, foreign keys, unique keys, indexes, CHECK constraints and EXCLUDE constraints — is declared in TypeORM entity classes in the backend code. Nobody writes `CREATE TABLE` statements by hand for normal tables.

- **In development**, the API process builds and updates the schema itself on start-up. The setting in `app.module.ts` is `synchronize: true` whenever the process owns the schema and `NODE_ENV` is not `production`. The ORM compares the entities with the database and applies the difference.
- **In production**, `synchronize` is off. An engineer runs `npm run schema:log:prod` to print the SQL the entity changes would need, reviews every statement (watching especially for `DROP COLUMN`, `DROP TABLE` and type changes), takes a backup, and then applies it with `npm run schema:sync:prod`.

Only the API process owns the schema. The background worker process never synchronizes and never runs migrations.

Two entity classes are not registered by any running module: `chairs` and `inventory`. The running app only knows tables that some module registers, so in development these two tables are not created. The command-line data source used by `schema:sync:prod` loads entity files by file pattern, so it does include them. A development database and a production database can therefore differ by exactly these two tables.

12.2.2 Migrations: three files, two that run

Status: [Implemented]

The folder `backend/src/migrations/` holds three files. Migrations do the work that `synchronize` cannot do: triggers, SQL functions and data backfills.

File	What it does	Runs?
`1776356529788-create-table`	A partial schema diff taken from an early development database. Describes 17 tables in a shape that no longer exists.	Never. Start-up records it as already applied without executing it.
`1792403456699-doctor-compensation`	Creates the commission tables if they are missing, installs the append-only trigger on `commission_entries`, and backfills data.	Yes, once. Hand-written and idempotent.
`1792490000000-lab-management-foundation`	Creates `audit_logs`, `status_history`, `number_sequences`, `labs`, `lab_members` and `lab_clinic_links` if missing, installs `uuid_generate_v7()` and the append-only triggers on `audit_logs` and `status_history`.	Yes, once. Hand-written and idempotent.

The table lists the three migration files, what each contains and whether it executes. The first file is the "legacy baseline": it is kept in the folder, recorded as applied by a helper called `recordLegacyBaseline`, and must never be edited or deleted. The other two are written so that they give the same result whether or not `synchronize` already created the tables; they skip table creation when the table exists and always (re)install their triggers.

On API start-up the order is fixed: first the required database extensions and functions are created, then `synchronize` runs (development only), then pending migrations run, each in its own transaction. Because 136 of the 153 tables appear in no migration at all, the migration folder cannot be used to rebuild the schema; the entities are the only complete description.

12.2.3 Database objects that live outside the entities

A few database objects cannot be expressed in an entity class. They are created by start-up code or by the two running migrations:

- `btree_gist` **extension.** Needed by the GiST EXCLUDE constraints that compare uuids, such as the rule that stops one doctor from having two overlapping appointments (`appointments.no_doctor_overlap`). Created at start-up before `synchronize`.
- `uuid_generate_v7()` **SQL function.** Produces time-ordered uuids; it is the primary-key default of the newer tables. Created at start-up and also by the lab-management migration.
- `commission_entries` **append-only trigger.** Blocks every DELETE and blocks UPDATE except setting or releasing the settlement link while the settlement is still pending.
- `audit_logs` **and** `status_history` **append-only triggers.** Block every UPDATE and DELETE.
- `migrations` **table.** TypeORM's bookkeeping of which migrations ran.

There are no views, no row-level security policies and no other triggers anywhere in the backend. Several other tables are described as "append-only" or "immutable" in code comments (for example `queue_events`, `patient_consents`, `prescription_history`, `ledger_entries`), but for those the rule is kept by the application only; the database would accept an update.

12.3 Conventions

12.3.1 Primary keys

Kind of primary key	Tables	Where it is used
uuid v4, generated by TypeORM (`uuid_generate_v4()`)	84	The older modules: identity, organizations, locations, staff, patients, appointments, queue, prescriptions, billing, inventory and others.
uuid v7, default `uuid_generate_v7()`	55	The newer modules: every lab table, the workflow engine, notifications, the general-ledger journals, `audit_logs` , `status_history` and `config_values` .
bigint identity	2	`outbox` and `queue_events` , where strict insertion order matters.
Supplied by the application (natural or composite key)	12	For example `idempotency_keys` (organization + key), `location_billing_settings` (the location id), `notification_events` (an event code), `patient_comm_prefs` (the patient id).

The table counts the four kinds of primary key across all 153 tables and names where each appears. A uuid v7 starts with a timestamp, so new rows sort by creation time and indexes stay compact; a uuid v4 is fully random. Both look the same to the application. One naming quirk: the four identity tables (`users` , `oauth_providers` , `security_logs` , `user_sessions`) call their primary-key column `uuid` ; every other table calls it `id` .

12.3.2 Column naming

Columns use `snake_case` (`organization_id` , `created_at`) in every module except one. Ten of the eleven tables of the inventory module use `camelCase` column names such as `organizationId` , `locationId` and `inventoryItemId` . Queries written by hand against these tables must quote the names. The eleventh table, the unregistered `inventory` table, uses snake_case. The Inventory chapter (Chapter 23) covers those tables.

12.3.3 Money

The Platform stores money in two ways, and a reader must check which one a column uses:

- **Integer paisa** (`bigint` or `integer` columns whose names end in `_paisa`). One rupee is 100 paisa, so 1,250.50 PKR is stored as `125050` . The newer money-handling code uses this form: payroll breakdowns, commissions and compensation plans, the general-ledger journals, and every lab billing and payables table.
- **Numeric with two decimals** (`numeric(12,2)` or `numeric(10,2)`). The amount is stored in rupees with two decimal places. Billing v2 (charges, invoices, payments, refunds), subscriptions, inventory, insurance, treatment plans and the legacy `treatments.cost` use this form.

Some tables carry both. `payroll_records` keeps `base_salary_paisa` , `commission_paisa` and `total_paisa` in paisa and also `salary_amount` in rupees, which mirrors the total for older screens. "PKR" is the default currency of a new organization, not a limit: `organizations.currency` and `locations.currency` hold the currency code.

12.3.4 Timestamps and dates

Most tables have `created_at` and many have `updated_at` , filled automatically by the ORM. Older tables store these as `timestamp without time zone` (about 50 tables); newer tables use `timestamp with time zone` (about 70 tables). Business moments that matter to the clinic — appointment start and end, check-in, payment time — are `timestamp with time zone` throughout. Calendar days that belong to a clinic day, such as `queue_sessions.service_date` , are stored as `date` and are computed in the **location's** timezone (`locations.timezone`), not the server's.

12.3.5 Soft delete

The Platform hides records in three ways:

- `is_active` **flag.** Organizations, locations, employees, assignments, roles, catalogs and templates are switched off with `is_active = false`; the row stays. Deactivating an organization or a location is always a soft delete.
- `deleted_at` **column.** Three tables use TypeORM's soft-delete column: `patients`, `patient_documents` and `patient_treatment_plans`. A row with `deleted_at` set is excluded from normal ORM reads.
- **Hard delete.** Some actions really delete rows and rely on `ON DELETE CASCADE` to remove children. For example, permanently deleting a patient deletes the patient row and every child row whose foreign key cascades. Child rows whose foreign key is `RESTRICT` block the delete instead.

`patients` has both a `deleted_at` column and an `is_active` flag. The user-facing activate/deactivate switch uses `is_active`.

12.3.6 Tenant columns

Every row that belongs to one clinic carries `organization_id`, and every row that belongs to one branch also carries `location_id`. The column names are the same in every snake_case module. Whether the column has a foreign key varies by module:

- The older modules (organizations, staff, patients, appointments, encounters, treatments, prescriptions) declare real foreign keys to `organizations` and `locations`, almost always with `ON DELETE CASCADE`.
- The queue tables, nearly all billing v2 tables, the lab tables, `plan_templates`, `audit_logs` and `status_history` keep `organization_id` (and often `location_id`) **without** a foreign key. The database accepts any uuid there; the application is responsible for writing the right one.

12.4 Tenant scope rules

The catalog classifies every table by which tenant columns it carries. The classes are:

Scope class	Meaning	Examples
Global — platform	Shared by all tenants; no tenant column.	`users`, `user_sessions`, `feature_catalog`, `specialties`, `notification_events`, `workflow_effects`
Global — person or lab	Shared across organizations by design.	`employees` (one person can work for several clinics), `labs`, `lab_members`, `lab_services`
Organization	`organization_id` only.	`patients`, `employment_roles`, `org_feature_flags`, `medicine_directory`, `services`
Organization + location	Both columns.	`appointments`, `queue_sessions`, `charges`, `invoices`, `payroll_records`, `prescriptions`
Location only	`location_id` only; the organization is reached through `locations`.	`chairs` (unregistered)
Cascading scope	A `scope_type` + `scope_id` pair that points to the platform, an organization or a location.	`config_values`
Platform or organization	Nullable `organization_id`: null means a platform default shared by all, a value means a clinic's own copy.	`form_schemas`, `workflow_templates`, `stage_templates`, `notification_templates`

Scope class	Meaning	Examples
No tenant column	A child table scoped only through its parent.	the ten tables listed after this table

The table shows each scope class, what it means and typical tables. Ten tenant-data tables have no organization column and are isolated only through their parent: `treatment_plan_items` (through `patient_treatment_plans`), `prescription_history` and `prescription_medicines` (through `prescriptions`), `treatment_plan_template_medicines` (through `treatment_plan_templates`), `cash_close_lines` (through `cash_closes`), `commission_rules` (through `compensation_plans`), and the inventory children `consumption_template_items`, `patient_consumption_items` and `purchase_order_items`. Any query on these tables must join the parent to stay inside one tenant.

The rule that keeps tenants apart at runtime is not in the database. The guard chain resolves the active organization and location from the request headers, and every service filters by `organization_id` (and `location_id` where relevant). The multi-tenant model is described in Chapter 8; this chapter only describes the columns that make it possible.

12.5 The domains and how they connect

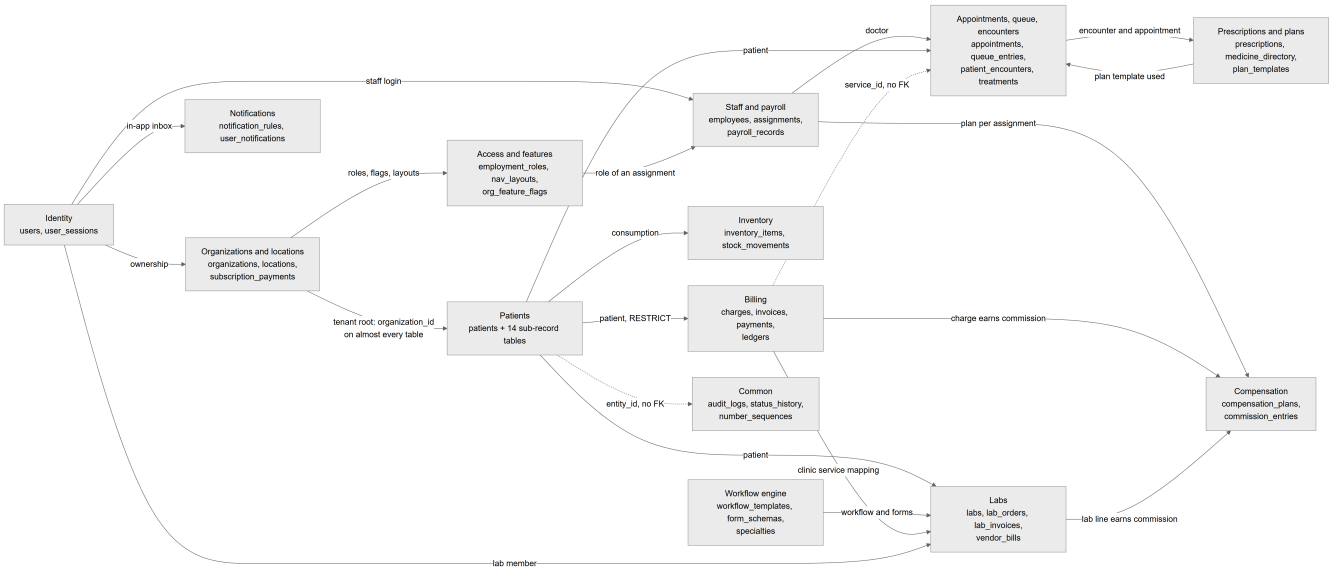

Figure 12.1 — Domains of the data model and the key links between them.

The figure draws the 153 tables as fourteen domain boxes, each labelled with the domain name and two to four of its most important tables. Arrows point from the side that is referenced (the parent) to the side that holds the reference (the child). Solid arrows are links backed by a foreign key; the two dotted arrows are logical links that have no foreign key. The left side holds the foundation: Identity, Organizations and locations, and Access and features. The middle holds the clinic's daily work: Staff and payroll, Patients, Appointments-queue-encounters, Prescriptions and plans, Billing, Inventory, Common and the Workflow engine. The right side holds Compensation and Labs, which consume data from several middle domains. Notifications sits near Identity because its main link is the user's in-app inbox. Every link drawn is explained in the next section.

12.5.1 Every link in the figure

- **Identity → Organizations and locations ("ownership").** `organization_ownership.user_id` links a user account to the organization it owns. Deleting the user or the organization deletes the ownership row (CASCADE).

- **Identity → Staff and payroll ("staff login").** `employee_org_location_assignments.user_id` links an employee's assignment to the user account they log in with. The link is optional; deleting the user sets it to null.
- **Organizations and locations → Patients ("tenant root").** `patients.organization_id` is a foreign key with CASCADE. The same kind of link, `organization_id` and often `location_id`, exists on almost every tenant table in every domain; the figure draws it once to keep the picture readable.
- **Organizations and locations → Access and features ("roles, flags, layouts").** `employment_roles`, `org_feature_flags` and `nav_layouts` each belong to one organization (CASCADE); `location_feature_overrides` belongs to one location (CASCADE).
- **Access and features → Staff and payroll ("role of an assignment").** `employee_org_location_assignments.employment_role_id` names the role a person has at a location. Deleting the role sets it to null.
- **Staff and payroll → Appointments, queue, encounters ("doctor").** `appointments.doctor_id` and `queue_sessions.doctor_id` point to `employees`, both with RESTRICT, so a doctor who has appointments or queue sessions cannot be hard-deleted.
- **Patients → Appointments, queue, encounters ("patient").** `appointments`, `queue_entries`, `patient_encounters` and `treatments` all carry `patient_id` with CASCADE.
- **Appointments, queue, encounters → Prescriptions and plans ("encounter and appointment").** `prescriptions.encounter_id` and `prescriptions.appointment_id` are optional links; deleting the encounter or appointment sets them to null.
- **Prescriptions and plans → Appointments, queue, encounters ("plan template used").** `treatments.plan_template_id` points to `plan_templates`; deleting the template sets it to null.
- **Patients → Billing ("patient, RESTRICT").** `charges`, `invoices`, `payments` and `refunds` carry `patient_id` with RESTRICT. A patient who has any billing row cannot be hard-deleted.
- **Staff and payroll → Compensation ("plan per assignment").** `compensation_plans.assignment_id` ties a salary-and-commission plan to one assignment (CASCADE). In the other direction, `commission_settlements.payroll_record_id` links a settlement to the payroll record that paid it (RESTRICT).
- **Billing → Compensation ("charge earns commission").** `commission_entries.charge_id` points to the charge that earned a commission (RESTRICT), and `charges.commission_plan_id` records which plan applied.
- **Labs → Compensation ("lab line earns commission").** `commission_entries.lab_invoice_line_id` points to a lab invoice line (RESTRICT).
- **Patients → Labs ("patient").** `lab_orders`, `lab_invoices`, `lab_payments` and `lab_refunds` carry `patient_id` with RESTRICT.
- **Workflow engine → Labs ("workflow and forms").** `lab_services` and `lab_orders` point to workflow templates, workflow template versions, stage templates and form schemas (all RESTRICT).
- **Billing → Labs ("clinic service mapping").** `service_mappings.clinic_treatment_id` maps a clinic billing service in `services` to a lab service (RESTRICT).
- **Patients → Inventory ("consumption").** `patient_consumptions.patientId` records materials used for a patient (CASCADE).
- **Identity → Notifications ("in-app inbox").** `user_notifications.user_id` delivers an in-app notification to a user (CASCADE).
- **Identity → Labs ("lab member").** `lab_members.user_id` makes a user a member of a lab (RESTRICT).
- **Patients ⇢ Common ("entity_id, no FK", dotted).** `audit_logs` and `status_history` record history for many kinds of rows through an `entity_type` + `entity_id` pair (and a `subject_id` that groups a patient's sub-records under the patient). The pair is polymorphic, so no foreign key is possible.

- **Billing ⇢ Appointments, queue, encounters ("service_id, no FK", dotted).** `treatments.service_id` names the catalog service billing uses to price the treatment, without a foreign key. `charges.encounter_id` and `charges.appointment_id` are also logical links without foreign keys.

12.6 Where to read next

Each domain has its own chapter with an entity-relationship diagram and every relationship explained: identity (Chapter 13), organizations, locations and subscriptions (Chapter 14), access and features (Chapter 15), staff and payroll (Chapter 16), patients (Chapter 17), appointments, queue and encounters (Chapter 18), prescriptions and plans (Chapter 19), billing (Chapters 20 and 21), compensation (Chapter 22), inventory (Chapter 23), notifications (Chapter 24), labs (Chapter 25), and the workflow engine and common tables (Chapter 26). Overlapping concepts — two prescription tables, four "visit" tables, two outboxes, two sequence mechanisms — are compared in Chapter 27.

Chapter 13 — Identity and sessions

This chapter describes the four tables that hold the people who can sign in to the Platform and the record of what happened when they did. It is for the product owner who needs to know where a login account lives, how a session is kept, and how sign-in events are logged. These tables sit underneath every other domain: a staff login, an owner, a super admin and a lab member are all rows in `users`.

13.1 Context

The identity domain is global. Its tables carry no `organization_id` or `location_id`, because one person has one account and may work for several organizations. What a user may do inside a clinic is not stored here; it comes from ownership (Chapter 14) and from the user's assignments and roles (Chapters 15 and 16). The tables are owned by three small modules: `user` owns `users`, `auth` owns `oauth_providers` and `security_logs`, and `session` owns `user_sessions`. Every column of every table is listed in the entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`.

13.2 The tables

13.2.1 `users` — user account

Status: [Implemented]

One row per login account on the whole Platform. The key columns are:

- `uuid` — the primary key. This table, and the other three in this chapter, name the key column `uuid` instead of `id`.
- `email` — unique across the whole Platform. Two accounts can never share an email address.
- `password` — the stored password hash; the column is nullable.
- `global_role` — null for normal users; the value `super_admin` marks a super admin. No other value exists.
- `status` — `active`, `suspended` or `deleted`, with `suspended_at`, `suspension_reason` and `deleted_at` recording when and why. The account is never removed by this flag; it is a status.
- `email_verification_token`, `email_verification_expires`, `email_verified_at` — used only for self-signup accounts, so accounts created by an administrator are never asked to verify.
- `last_accessed_organization_id`, `last_accessed_location_id` — the organization and location the user last worked in, so the next login can return there.
- `provider` — `local`, `google` or `github`; defaults to `local`.
- `refresh_token` — a deprecated single refresh token kept for backward compatibility. Sessions now keep their own refresh tokens in `user_sessions`.

There is an index on `status` for listing accounts by state.

13.2.2 `oauth_providers` — linked external sign-in

Status: [Not implemented] (table exists, nothing reads or writes it)

Designed to record external sign-in identities (Google, GitHub) linked to an account: `user_id`, `provider`, `provider_id` (the provider's own id for the person), and the email, display name, avatar and raw profile the provider returned. A user can have at most one row per provider (unique `user_id` + `provider`), and lookups by

`provider` + `provider_id` are indexed.

The auth module registers the table, so it is created in the database, but no service injects or queries it. External sign-in itself is disabled: the Google and GitHub strategies exist as files but are not registered, and the module comment says Google/GitHub sign-in is disabled. Expect this table to be empty.

13.2.3 `security_logs` — security event log

Status: [Implemented]

One row per security-relevant event: a successful or failed login, logout, logout of all devices, password change or reset, email verification, session created, revoked or expired, token refreshed, account suspended, deleted or restored, a login from a new device, and suspicious activity. Key columns:

- `user_id` — the account concerned; nullable, because a failed login for an unknown email has no account.
- `event_type` — one of twenty event codes such as `LOGIN_SUCCESS`, `LOGIN_FAILED`, `SESSION_REVOKED`.
- `success` and `error_message` — whether the event succeeded and why not.
- `session_id` — the session involved, when the event concerns a session.
- `metadata` — free-form JSON with event-specific detail.
- The table also records the network address and browser string of the request. Those values are personal data and are not reproduced in this handbook.

Indexes on `created_at` and on `user_id` + `event_type` support "recent events" and "events of this kind for this user" queries. The table has only `created_at`; rows are never updated by design, but the database does not enforce that.

13.2.4 `user_sessions` — login session

Status: [Implemented]

One row per signed-in device or browser. Each session has its own refresh token, so a user can be signed in on several devices and sign one out without affecting the others. Key columns:

- `user_id` — the account.
- `refresh_token_hash` — a hash of this session's refresh token; the token itself is never stored.
- `expires_at` — after this moment the session is expired.
- `is_revoked` — a soft revocation flag: when true the session is no longer valid, and the row stays as history.
- `device`, `browser`, `os` and a free-text `location` describe the device; an `is_current` flag is also stored.
- `last_active_at` — updated on every save of the row.

An index on `user_id` + `is_revoked` makes "this user's live sessions" fast.

13.3 Entity-relationship diagram

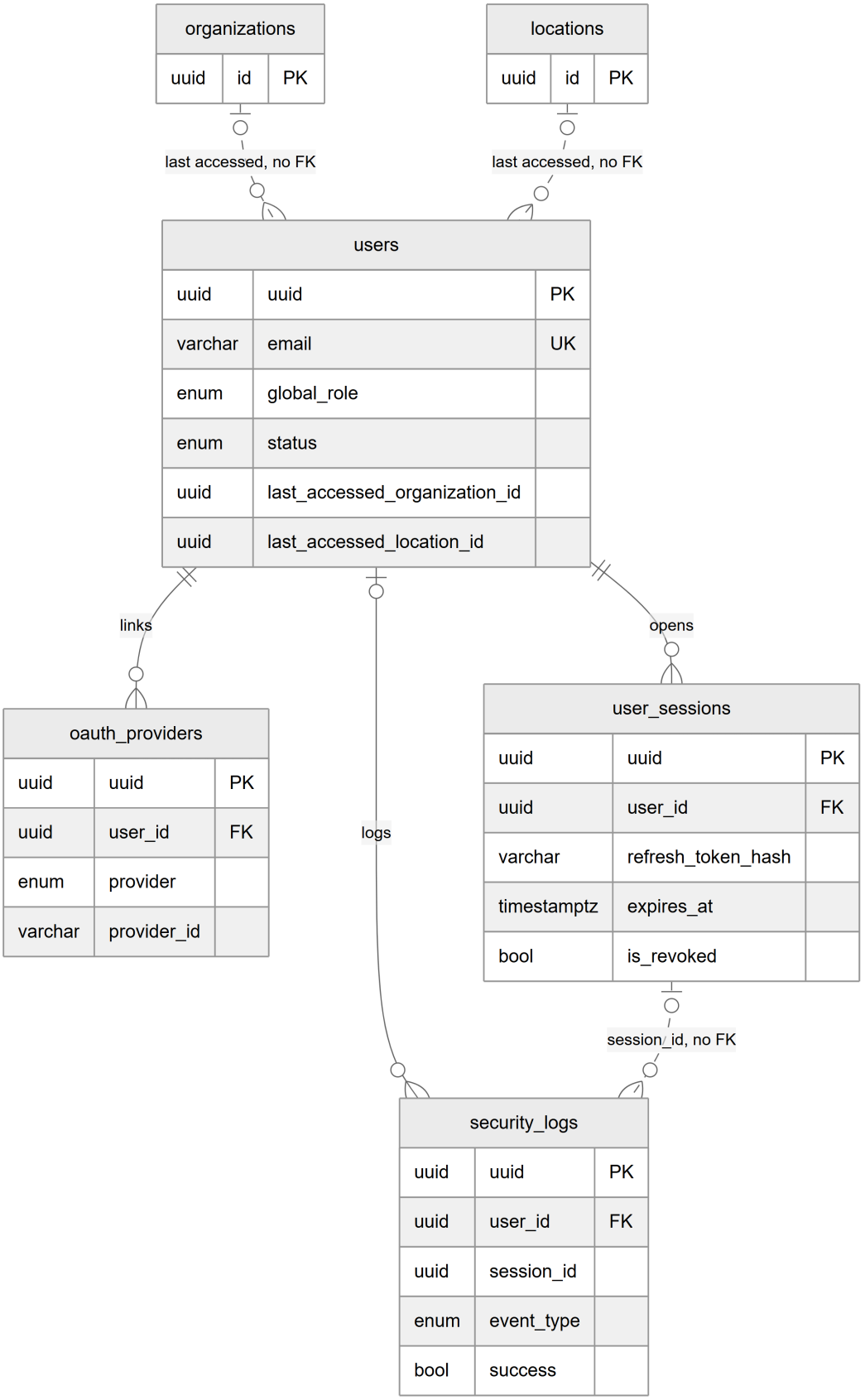

Figure 13.1 — Identity and session tables.

The diagram has `users` in the centre, with the key columns `uuid` (primary key), `email` (unique), `global_role`, `status`, `last_accessed_organization_id` and `last_accessed_location_id`. Under it are three child tables. On the left, `oauth_providers` (`uuid`, `user_id`, `provider`, `provider_id`) hangs from `users` with a solid line

labelled "links": one user, zero or more provider rows, and every provider row must have a user. In the middle, `security_logs` (`uuid` , `user_id` , `session_id` , `event_type` , `success`) hangs from `users` with a solid line labelled "logs" whose user end is optional. On the right, `user_sessions` (`uuid` , `user_id` , `refresh_token_hash` , `expires_at` , `is_revoked`) hangs from `users` with a solid line labelled "opens". A dashed line labelled "session_id, no FK" runs from `user_sessions` to `security_logs` . At the top, `organizations` and `locations` are drawn as context boxes with only their `id` ; each has a dashed line to `users` labelled "last accessed, no FK". Solid lines are foreign keys; dashed lines are logical references the database does not enforce.

13.4 Relationships explained

Foreign-key relationships:

- **`oauth_providers.user_id` → `users` .** Each provider row belongs to exactly one user; a user has zero or more (at most one per provider). Deleting the user deletes their provider rows (ON DELETE CASCADE).
- **`security_logs.user_id` → `users` .** Each log row belongs to zero or one user; a user has zero or more log rows. Deleting the user keeps the log rows and sets `user_id` to null (ON DELETE SET NULL), so the security history survives the account.
- **`user_sessions.user_id` → `users` .** Each session belongs to exactly one user; a user has zero or more sessions. Deleting the user deletes their sessions (ON DELETE CASCADE).

Logical relationships (no foreign key; the database accepts any value):

- **`security_logs.session_id` → `user_sessions` .** The session an event concerns. Zero or one session per log row. If the session row disappears, the log keeps a dangling id.
- **`users.last_accessed_organization_id` → `organizations`** and **`users.last_accessed_location_id` → `locations` .** A convenience pointer for the next login. If the organization or location is removed, the pointer is not cleared by the database.

Links into `users` from other domains (described in their own chapters):

- `organization_ownership.user_id` (CASCADE) makes a user an owner of an organization (Chapter 14).
- `employee_org_location_assignments.user_id` (SET NULL) gives an employee's assignment a staff login (Chapter 16).
- `lab_members.user_id` (RESTRICT) makes a user a member of a lab (Chapter 25). Because of RESTRICT, a user who is a lab member cannot be hard-deleted.
- `user_notifications.user_id` (CASCADE) is the user's in-app inbox (Chapter 24).
- Many tables keep "actor" columns such as `created_by` , `updated_by` , `approved_by` or `recorded_by` . They hold a `users.uuid` but have no foreign key (see Chapter 12).

13.5 Tenant scope

All four tables are global: no `organization_id` , no `location_id` . A user's link to clinics is indirect, through `organization_ownership` and through assignments. This is why `users.email` is unique across the whole Platform rather than per organization. Only the super admin has a platform-wide role, stored in `users.global_role` .

13.6 Ownership and ambiguities

- **`oauth_providers` is registered but unused.** The table is created, but no code path reads or writes it, and external sign-in is disabled. It is a leftover of a planned feature.

- **Two places for a refresh token.** `users.refresh_token` is marked deprecated; the live mechanism is one hashed token per row in `user_sessions`.
- `security_logs.session_id` **has no foreign key**, so a log row can point to a session that no longer exists. This is acceptable for a log but means joins must be outer joins.
- **Append-only by convention.** Nothing in the database prevents an update or delete of `security_logs` rows.

Chapter 14 — Organizations, locations, subscriptions and settings

This chapter describes the tables that define a tenant: the organization (one clinic business), its locations (branches), who owns it, how it pays for the Platform, and where its settings are stored. It is for the product owner who needs to know what a "clinic" is in the database and which table to look at for a given setting. Almost every other table in the Platform points back to `organizations`, and most point to `locations` as well.

14.1 Context

Seven tables belong to this domain. `organizations`, `organization_ownership` and `organization_settings` are owned by the `organizations` module; `locations` and `location_settings` by the `locations` module; `subscription_payments` by the `subscriptions` module; and `config_values` by the `settings-registry` module. The business behaviour of organizations and locations is covered in Chapter 9 and subscriptions in Chapter 45; this chapter covers only the data. Every column is listed in the entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`.

14.2 The tables

14.2.1 `organizations` — organization (the tenant)

Status: [Implemented]

One row per clinic business. This is the root of tenancy: no other table sits over it. Key columns:

- `name` and `slug` — the display name and a short machine name. `slug` is unique across the Platform.
- `clinic_type` and `specialty_tags` — what kind of clinic this is. `specialty_tags` is a JSON list such as `["general"]` or `["dental"]` and is used to decide which features are relevant.
- `timezone` (default `Asia/Karachi`) and `currency` (default `PKR`) — the organization-wide defaults.
- `subscription_plan` — `free`, `starter`, `professional` or `enterprise`.
- `trial_ends_at` and `paid_until` — the subscription window. When both are null the clinic is "unmanaged" (created by a super admin before subscriptions existed) and has no expiry.
- `max_locations` and `max_users` — optional limits.
- `onboarding_pending` — true for a self-signup clinic that has not yet finished the setup wizard.
- `access_version` — a counter increased by one on every access change in the organization (roles, assignments, feature flags, ownership). Access caches use it as their key; it is written only by the access-version service.
- `is_active` — the soft-delete flag. Deactivating an organization sets it to false; the application does not hard-delete organizations.

Indexes: unique `slug`, and an index on `is_active`.

14.2.2 `organization_ownership` — owner link

Status: [Implemented]

Links a user account to an organization it owns: `user_id`, `organization_id` and `is_primary_owner`. The pair (`user_id`, `organization_id`) is unique, so a user is listed at most once per organization. An owner holds every permission in the organization (Chapter 11). An organization can have more than one owner row; `is_primary_owner` marks the main one.

14.2.3 `organization_settings` — organization-wide settings

Status: [Partial] — stored and editable; the feature switches only hide sections on the location settings screen

One row per organization at most (`organization_id` is unique). It holds:

- Booking defaults: `default_slot_duration` (30 minutes), `advance_booking_days` (30), `cancellation_notice_hours` (24).
- `allow_location_override` — whether locations may set their own booking values. The location settings screen shows the organization's values instead when this is false.
- Six older feature switches: `feature_appointments`, `feature_inventory`, `feature_finance`, `feature_reminders` (all default true), `feature_whatsapp` and `feature_patient_portal` (default false).

The feature switches predate the feature-flag system. Today the location settings screen reads them only to hide its appointments, reminders and finance sections. The guards that actually allow or block a feature read `org_feature_flags` (Chapter 15), not these columns. The row is created with defaults the first time the settings are read.

14.2.4 `locations` — location (branch)

Status: [Implemented]

One row per branch of an organization. Key columns:

- `organization_id` — the owning organization.
- `name` and `code` — the code is unique within the organization (`organization_id` + `code`).
- `token_prefix` — a short prefix for daily tokens (for example "MAIN" giving "MAIN-007"); falls back to `code` when empty.
- `timezone` (default `UTC`) and `currency` (default `USD`). The location's timezone is the one the Platform uses to decide what "today" is for appointments, queues and reports at that branch.
- Address fields, contact fields, `operating_hours` (JSON), `capacity_per_day`, `total_chairs`, `business_model`.
- `is_main_branch` — marks the main branch.
- `manager_id` — a reference to the branch manager with no foreign key; the code does not say whether it holds an employee id or a user id.
- `is_active` — the soft-delete flag; deactivating a location sets it to false.

Indexes: unique (`organization_id`, `code`) and (`organization_id`, `is_active`).

Note that the column defaults for a new location (`UTC`, `USD`) differ from the defaults of a new organization (`Asia/Karachi`, `PKR`). Whatever the creating screen sends overrides the default.

14.2.5 `location_settings` — per-location settings

Status: [Implemented]

One row per location at most (`location_id` is unique). It holds booking values (`appointment_slot_duration`, `max_appointments_per_day`, `advance_booking_days`, `cancellation_notice_hours`), reminder values (`reminder_enabled`, `reminder_days_before`, `reminder_hours_before`, `reminder_channels`), money values

(`tax_rate` and `late_fee_percentage` as `numeric(5,2)` percentages, `payment_gateway`) and branding (`logo_url` , `primary_color` , `theme`). Billing reads `tax_rate` as the sales-tax rate of the active location, and the appointments service reads the booking values. The row is created with defaults the first time the settings are read. `organization_id` is nullable here, unlike most tenant tables, and the auto-create path in the locations service sets only `location_id` , so rows created that way have no organization.

14.2.6 `subscription_payments` — subscription payment

Status: [Implemented]

A clinic pays for the Platform outside the app; a super admin then records the payment here. One row per recorded payment: `organization_id` , `plan` , `months` , `amount` (numeric, in rupees), `method` , an optional `reference` , `paid_at` , the covered period `period_start` and `period_end` , `notes` , and `recorded_by` (the super admin's user id, no foreign key). In the same transaction the service sets `organizations.paid_until` to the period end and `organizations.subscription_plan` to the paid plan. An index on (`organization_id` , `paid_at`) supports the payment history list.

14.2.7 `config_values` — settings registry value

Status: [Implemented] for the keys the code registers

A generic store for settings that cascade from the Platform to an organization to a location. Each row is one setting key at one scope:

- `scope_type` — `platform` , `org` or `location` .
- `scope_id` — the organization id or location id; null for the platform.
- `key` — the setting name, for example `theme.brand_color` .
- `value` — JSON; null means "nothing set here, inherit from the wider scope". A row can therefore exist only to carry a lock.
- `is_locked` — freezes the effective value at this scope so lower scopes cannot override it.
- `version` — increased on every write.

Two unique indexes keep one row per scope and key: (`scope_type` , `scope_id` , `key`), and a partial unique index on `key` where `scope_type = 'platform'` . The partial index is needed because PostgreSQL treats nulls as distinct, so two platform rows with a null `scope_id` would otherwise both be allowed. The keys registered in code today are the theme keys (`theme.app_name` , `theme.brand_color` , `theme.sidebar_style` , `theme.default_mode` , `theme.logo_url` , `theme.logo_dark_url` , `theme.favicon_url`) and `subscription.payment_instructions` . The default of `theme.app_name` is "Clinic Portal". The primary key uses the time-ordered `uuid_generate_v7()` default.

14.3 Entity-relationship diagram

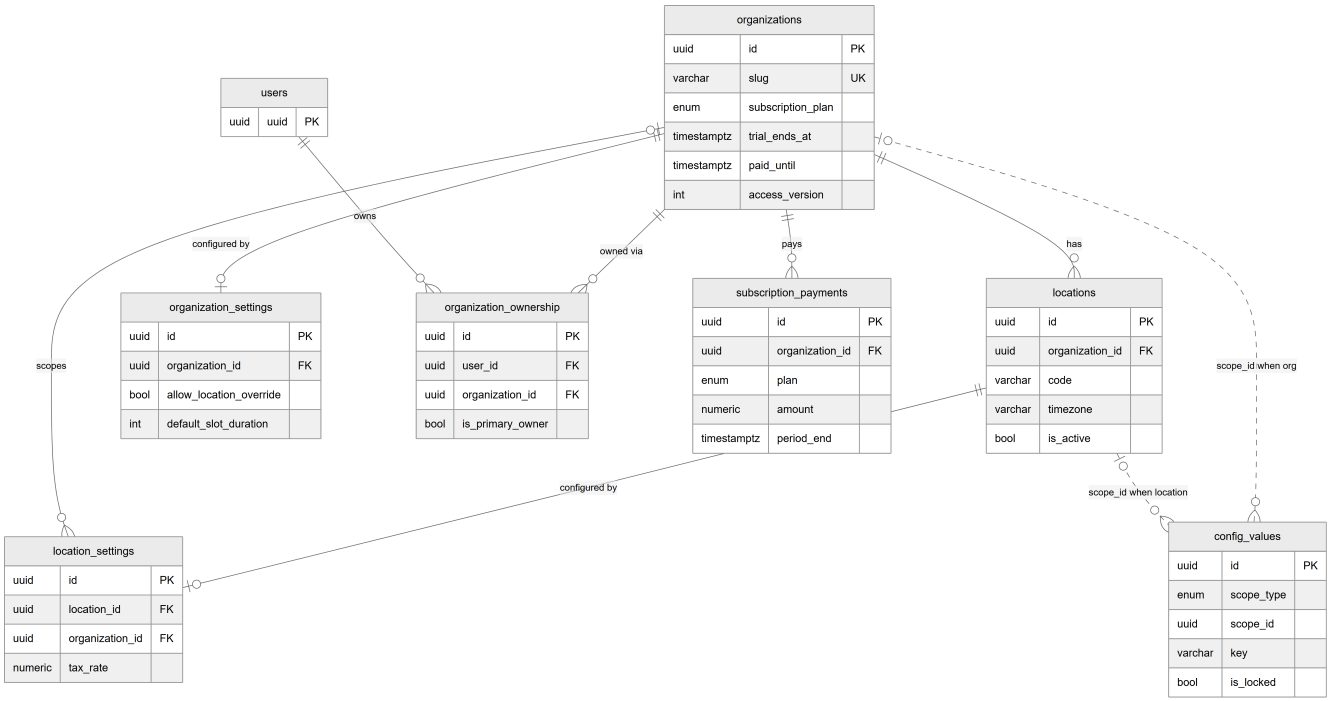

Figure 14.1 — Organizations, locations, ownership, subscriptions and settings.

The diagram places `organizations` at the top, showing `id`, the unique `slug`, `subscription_plan`, `trial_ends_at`, `paid_until` and `access_version`. A small `users` box with only its `uuid` key sits to its left. Solid lines run from `organizations` to five children: `organization_settings` ("configured by", at most one per organization), `organization_ownership` ("owned via"), `subscription_payments` ("pays"), `locations` ("has") and `location_settings` ("scopes", optional on the settings side). `users` also has a solid line to `organization_ownership` labelled "owns". `locations` has a solid line to `location_settings` labelled "configured by", at most one per location. At the bottom right, `config_values` (`id`, `scope_type`, `scope_id`, `key`, `is_locked`) is connected by two dashed lines, one from `organizations` labelled "scope_id when org" and one from `locations` labelled "scope_id when location". Dashed lines mean there is no foreign key.

14.4 Relationships explained

Foreign-key relationships:

- **`locations.organization_id` → `organizations`.** Every location belongs to exactly one organization; an organization has zero or more locations. Deleting the organization deletes its locations (CASCADE).
- **`organization_ownership.organization_id` → `organizations`** and **`organization_ownership.user_id` → `users`.** Each ownership row joins exactly one user to exactly one organization; a user can own several organizations and an organization can have several owners. Deleting either side deletes the ownership row (CASCADE).
- **`organization_settings.organization_id` → `organizations`.** One-to-one: each organization has zero or one settings row, and each settings row belongs to exactly one organization. Deleting the organization deletes the row (CASCADE).
- **`location_settings.location_id` → `locations`.** One-to-one: each location has zero or one settings row. Deleting the location deletes it (CASCADE).
- **`location_settings.organization_id` → `organizations`.** Optional: a settings row may carry its organization. Deleting the organization deletes the row (CASCADE).

- **`subscription_payments.organization_id` → `organizations`.** Each payment belongs to exactly one organization; an organization has zero or more payments. Deleting the organization deletes its payment history (CASCADE).

Logical relationships (no foreign key):

- **`config_values.scope_id` → `organizations` or `locations`.** Polymorphic: `scope_type` says which table the id belongs to, so no single foreign key is possible. Deleting an organization or location leaves its `config_values` rows behind.
- **`locations.manager_id` → `employees` or `users`** (the target is not stated in the code).
- **Actor columns** — `locations.created_by`, `locations.updated_by`, `subscription_payments.recorded_by` and `config_values.updated_by` — hold a user id without a foreign key.
- **`users.last_accessed_organization_id` / `last_accessed_location_id`** point into this domain without a foreign key (Chapter 13).

Links into `organizations` and `locations` from other domains:

- Almost every tenant table in the older modules has `organization_id` → `organizations` with CASCADE, and often `location_id` → `locations` with CASCADE.
- Some newer links use RESTRICT instead: `lab_clinic_links`, `lab_orders` (organization and location), `magic_links`, `order_templates`, `service_mappings`, `lab_service_prices`, `notification_outbox`, and the clinic-specific rows of `form_schemas`, `workflow_templates` and `stage_templates`. An internal lab also points to its owner organization with RESTRICT.
- The queue, billing v2 and lab tables mostly keep `organization_id` with no foreign key at all (Chapter 12).

Because of the RESTRICT links, a hard delete of an organization that has lab, notification or engine rows would be refused by the database; a hard delete of one without them would cascade into most of its data. The application never hard-deletes an organization or a location: it only sets `is_active` to false.

14.5 Tenant scope

- `organizations` is the tenant root and has no tenant column itself.
- `organization_ownership`, `organization_settings`, `locations` and `subscription_payments` are organization-scoped.
- `location_settings` is organization + location scoped.
- `config_values` uses its own cascading scope (`scope_type` + `scope_id`) and is classified as global in the catalog, because platform rows belong to no tenant.

14.6 Ownership and ambiguities

- **Two kinds of feature switch.** `organization_settings.feature_*` columns and `org_feature_flags` rows both describe "is this feature on". Only `org_feature_flags` is enforced (Chapter 15).
- **Two kinds of settings storage.** Typed settings tables (`organization_settings`, `location_settings`, `queue_policies`, `location_billing_settings`) coexist with the generic `config_values` registry. The project's rule is that typed tables stay as they are and new long-tail settings go into the registry.
- **Default mismatch.** A new location defaults to `UTC` and `USD`, while a new organization defaults to `Asia/Karachi` and `PKR`.
- **Specialties stored twice.** `organizations.specialty_tags` and `organizations.clinic_type` describe the clinic's specialties, and the engine's `organization_specialties` table does too (Chapter 26).

- **Legacy baseline.** The never-executed baseline migration describes `organization_admins` and `user_locations` tables that no longer exist; they were replaced by `organization_ownership` and by assignments.

Chapter 15 — Access and features

This chapter describes the tables behind two questions the Platform asks on every request: "is this feature switched on for this clinic and this location?" and "what may this person do here?". It is for the product owner who configures roles and features for clinics, or who needs to explain why a screen is hidden. The behaviour built on these tables — the guards, the permission catalog, the role editor — is explained in Chapters 10 and 11; this chapter covers the stored data.

15.1 Context

Five tables belong to this domain. `feature_catalog`, `org_feature_flags` and `location_feature_overrides` are owned by the `feature-flags` module. `nav_layouts` is owned by the `access` module. `employment_roles` is defined in the `employees` module folder, although the permission model that reads it lives in the `access` module; three modules register it (`employees`, `access` and `feature-flags`). Every column is listed in the entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`.

15.2 The tables

15.2.1 `feature_catalog` — feature catalog entry

Status: [Implemented]

The master list of every feature the Platform offers, one row per feature. It is global: the same list for all clinics. Key columns:

- `key` — the feature key, unique, for example `billing.invoices`.
- `name` and `description` — the human label.
- `module` — a grouping such as `billing`, `appointments` or `patients`.
- `specialty_tags` — the clinic types the feature is relevant to (for example `["general"]`, `["dental"]`).
- `enabled_by_default` — whether a clinic gets it switched on when its flags are created.
- `sort_order` and `is_active` — display order, and whether the feature is still offered at all.

The catalog is seeded from code (`backend/src/feature-flags/feature-catalog.seed.ts`); seeding inserts missing keys and does not overwrite existing rows.

15.2.2 `org_feature_flags` — organization feature state

Status: [Implemented]

One row per organization per feature, unique on (`organization_id`, `feature_key`). It holds two independent switches:

- **Entitled** — `is_entitled`, set by the super admin: the organization may use the feature. It defaults to true so that organizations created before entitlements existed kept every feature they had. `entitled_until` turns an entitlement into a trial: from that moment the feature is no longer entitled. `entitled_by` and `entitled_at` record who and when.
- **Enabled** — `is_enabled`, set by the owner within what is entitled. `enabled_by` records who last switched it.

A feature is **active** for an organization only when its catalog entry is active, it is entitled (and the trial has not expired) and it is enabled. `feature_key` is a copy of `feature_catalog.key`, not a foreign key. An index on (`organization_id`, `is_enabled`) supports "all enabled features of this clinic".

15.2.3 `location_feature_overrides` — location override

Status: [Implemented]

The owner's way to switch an organization-active feature off at one location. One row per location per feature, unique on (`location_id`, `feature_key`). `is_disabled = true` turns the feature off at that location. There is no way to turn a feature on at a location if it is off for the organization; the table can only narrow. `organization_id` is copied from the location for tenant-scoped reads and has no foreign key; `feature_key` is a copy of the catalog key; `updated_by` is the user who last changed it.

15.2.4 `employment_roles` — role

Status: [Implemented]

One row per role in an organization: the five system roles seeded from templates (Admin, Doctor, Receptionist, Assistant, Accountant) and any custom roles the clinic creates. Key columns:

- `name` and `slug` — the slug is unique within the organization (`organization_id` + `slug`).
- `permissions` — a JSON object of granted permissions only, `{ permission: 'own' | 'location' | 'org' | true }`. A missing key means no access. The value is the scope at which the permission applies.
- `template_key` — the system template the role was seeded from, such as `doctor` or `receptionist`; empty for custom roles.
- `is_system_role` — true for seeded roles (the column default is true).
- `home_route` — where members of the role land after login, for example `/queue/doctor`.
- `description` and `is_active`.

An index on (`organization_id`, `is_active`) lists a clinic's live roles. The role a person has is not stored on the person: it is the `employment_role_id` of each of their assignments (Chapter 16), so one person can hold different roles at different locations.

15.2.5 `nav_layouts` — sidebar layout

Status: [Implemented] for the clinic default layout; [Not implemented] for per-role layouts

A clinic's arrangement of the sidebar: which navigation keys go into which section, renamed labels and hidden keys, stored as JSON in `layout`. It only arranges keys that the web app's navigation registry defines; it never grants access. `version` increases on every save and `updated_by` records the user.

`role_id` null means the clinic default layout, which is the only kind the current version writes. A row with a `role_id` would be a layout for one role. Two unique indexes keep at most one default per organization (a partial unique index on `organization_id` where `role_id` is null) and at most one layout per role (`organization_id` + `role_id`).

15.3 Entity-relationship diagram

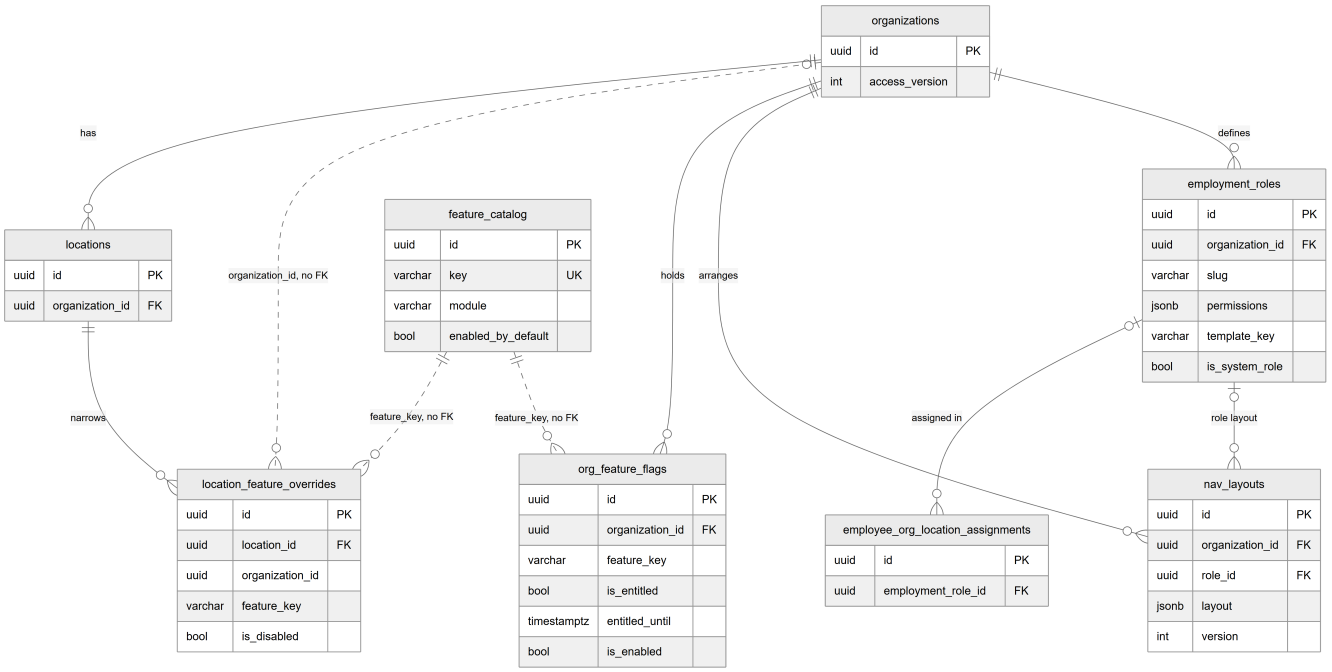

Figure 15.1 — Access and feature tables.

The diagram puts `organizations` (with `id` and `access_version`) at the top. Solid lines run from it to `locations` ("has"), `org_feature_flags` ("holds"), `nav_layouts` ("arranges") and `employment_roles` ("defines"). `employment_roles` (`id`, `organization_id`, `slug`, `permissions`, `template_key`, `is_system_role`) has two solid lines going down: to `nav_layouts` labelled "role layout" and to a context box `employee_org_location_assignments` (only `id` and `employment_role_id`) labelled "assigned in"; both are optional on the role side. `locations` has a solid line to `location_feature_overrides` labelled "narrows". In the middle, `feature_catalog` (`id`, unique `key`, `module`, `enabled_by_default`) has two dashed lines labelled "feature_key, no FK", one to `org_feature_flags` and one to `location_feature_overrides`. A dashed line labelled "organization_id, no FK" also runs from `organizations` to `location_feature_overrides`. Dashed lines are logical links without a foreign key.

15.4 Relationships explained

Foreign-key relationships:

- **`employment_roles.organization_id` → `organizations`.** Each role belongs to exactly one organization; an organization has zero or more roles. Deleting the organization deletes its roles (CASCADE).
- **`nav_layouts.organization_id` → `organizations`.** Each layout belongs to exactly one organization. Deleting the organization deletes its layouts (CASCADE).
- **`nav_layouts.role_id` → `employment_roles`.** Optional: a layout belongs to zero or one role (zero means the clinic default). Deleting a role deletes its own layout (CASCADE); the default layout is unaffected.
- **`employee_org_location_assignments.employment_role_id` → `employment_roles`.** Optional: an assignment has zero or one role; a role is held by zero or more assignments. Deleting a role keeps the assignments and sets their role to null (SET NULL), leaving those people with no role permissions at that location.
- **`org_feature_flags.organization_id` → `organizations`.** Each flag belongs to exactly one organization. Deleting the organization deletes its flags (CASCADE).
- **`location_feature_overrides.location_id` → `locations`.** Each override belongs to exactly one location. Deleting the location deletes its overrides (CASCADE).

Logical relationships (no foreign key):

- **`org_feature_flags.feature_key`** → **`feature_catalog.key`** and **`location_feature_overrides.feature_key`** → **`feature_catalog.key`.** The flag and the override refer to a catalog entry by its key text. The database does not stop a flag for a key that is not in the catalog, and removing a catalog row would not remove its flags.
- **`location_feature_overrides.organization_id`** → **`organizations`.** A copy of the location's organization, for filtering.
- **Actor columns** — `org_feature_flags.enabled_by` and `entitled_by`, `location_feature_overrides.updated_by`, `nav_layouts.updated_by` — hold a user id without a foreign key.

15.5 Tenant scope

- `feature_catalog` is global (one list for the whole Platform).
- `org_feature_flags`, `employment_roles` and `nav_layouts` are organization-scoped.
- `location_feature_overrides` is organization + location scoped.

Every change to roles, assignments, feature flags or ownership increases `organizations.access_version` (Chapter 14). The resolver that turns these tables into a user's permissions caches its result per user, organization, location and access version, so a change takes effect as soon as the version moves.

15.6 Ownership and ambiguities

- **Where `employment_roles` lives.** The entity file is in `employees/`, the permission logic that reads `permissions` is in `access/`, and the project's own overview once described "employee role definitions" as part of `organizations/`. Treat `access` as the functional owner and `employees` as the place the file sits.
- **Feature keys are text, not foreign keys.** A typo in a key would not be caught by the database.
- **Older feature switches.** `organization_settings` has six `feature_*` columns that the guards do not read (Chapter 14).
- **Per-role sidebar layouts** are supported by the schema (`role_id`, its foreign key and its unique index) but the current version only writes the clinic default.

Chapter 16 — Staff and payroll

This chapter describes the tables that record the clinic's staff: the person, where and in which role they work, their weekly hours, leave and day-by-day availability, and the monthly payroll record. It is for the product owner who needs to know how one person can work at several clinics, where a doctor's working hours come from, and what a payroll row contains. The employee and payroll modules are described in Chapters 37 and 38; salary and commission plans are in Chapter 22.

16.1 Context

The domain separates the **person** from the **job**. `employees` holds the person once for the whole Platform. Each place the person works — one organization and one location, with one role — is an **assignment** in `employee_org_location_assignments`. Schedules, leave, availability overrides and payroll all hang from the person and name the location they apply to. Six of the tables are owned by the `employees` module (it also owns `employment_roles`, which Chapter 15 describes) and `payroll_records` is owned by the `payroll` module. Every column is listed in the entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`.

16.2 The tables

16.2.1 `employees` — employee (the person)

Status: [Implemented]

One row per person who works at any clinic on the Platform. It holds only personal data: `full_name`, `phone`, an optional national identity number (`cnic`), `gender`, `dob`, `profile_picture_url` and `is_active`. There is no `organization_id`: the same person record is shared by every organization they work for. For that reason `phone` and `cnic` are **unique across the whole Platform**, not per clinic.

16.2.2 `employee_org_location_assignments` — assignment

Status: [Implemented]

One row per person per location they work at. This is where the job lives. Key columns:

- `employee_id`, `organization_id`, `location_id` — who and where. (`employee_id`, `location_id`) is unique: a person has at most one assignment per location.
- `employment_role_id` — the role at this location (Chapter 15); nullable.
- `user_id` — the staff login used for this assignment; nullable, because not every employee logs in.
- `has_portal_access` — whether this person may log in at this location.
- `designation` and `employment_type` (`full_time`, `part_time`, `visiting`, `contract`).
- `employee_code` — unique within the organization (`organization_id` + `employee_code`).
- `joining_date`, `is_primary` and `is_active` (the soft-delete flag for this job).
- `token_prefix` — the doctor's queue-token prefix at this location (for example "A" giving "A-007"); empty means the first letter of the doctor's name.
- `gross_salary` — deprecated. Salary now lives in versioned compensation plans (Chapter 22); the column is kept for old rows and no longer written once a plan exists.

Indexes support lookups by user, by location and active flag, and by employee and organization.

16.2.3 `employee_schedules` — weekly working hours

Status: [Implemented]

The regular week of a person at one location: one row per weekday (`day_of_week`), with `is_working` , `start_time` , `end_time` and `slot_duration` (default 30 minutes). (`employee_id` , `location_id` , `day_of_week`) is unique, so there is one row per weekday per location. The appointments service reads these rows when it works out a doctor's availability.

16.2.4 `employee_leaves` — leave request

Status: [Implemented]

One row per leave request: `leave_type` (`annual` , `sick` , `unpaid` , `maternity` , `paternity` , `other`), `start_date` , `end_date` , `total_days` , `status` (`pending` , `approved` , `rejected` , `cancelled`), the reason, a review note, `approved_by` and `reviewed_at` . Indexes support "leave in this date range", "pending leave at this location" and "this person's leave by status".

16.2.5 `doctor_availability_overrides` — availability exception

Status: [Partial] — recorded and edited on the employee's availability tab; the appointments and queue services do not read it

An exception to the weekly schedule for one date at one location: `date` , `is_available` (default false, meaning "not available that day"), a reason and optional `override_hours` (JSON) for different hours that day. (`employee_id` , `location_id` , `date`) is unique: one exception per person, location and day. Only the employees module reads this table; the appointments service builds availability from `employee_schedules` without consulting these exceptions.

16.2.6 `payroll_records` — payroll record

Status: [Implemented]

One row per person per location per month. (`employee_id` , `location_id` , `month` , `year`) is unique, so the same person can have a separate record at each location for the same month. Key columns:

- `month` , `year` and `period_label` .
- `base_salary_paisa` , `commission_paisa` , `total_paisa` — the breakdown in integer paisa, where total = base salary + commission.
- `salary_amount` — the total in rupees (`numeric(10,2)`), kept for older screens.
- `status` (`pending` , `paid` , `on_hold`), `paid_at` , `payment_method` (`cash` , `bank_transfer` , `cheque`), `payment_reference` , `paid_by` , `notes` .

The unique key changed over time. The doctor-compensation migration drops the older key (`employee_id` , `month` , `year`) and adds the location-aware one; the entity matches the new key.

16.3 Entity-relationship diagram

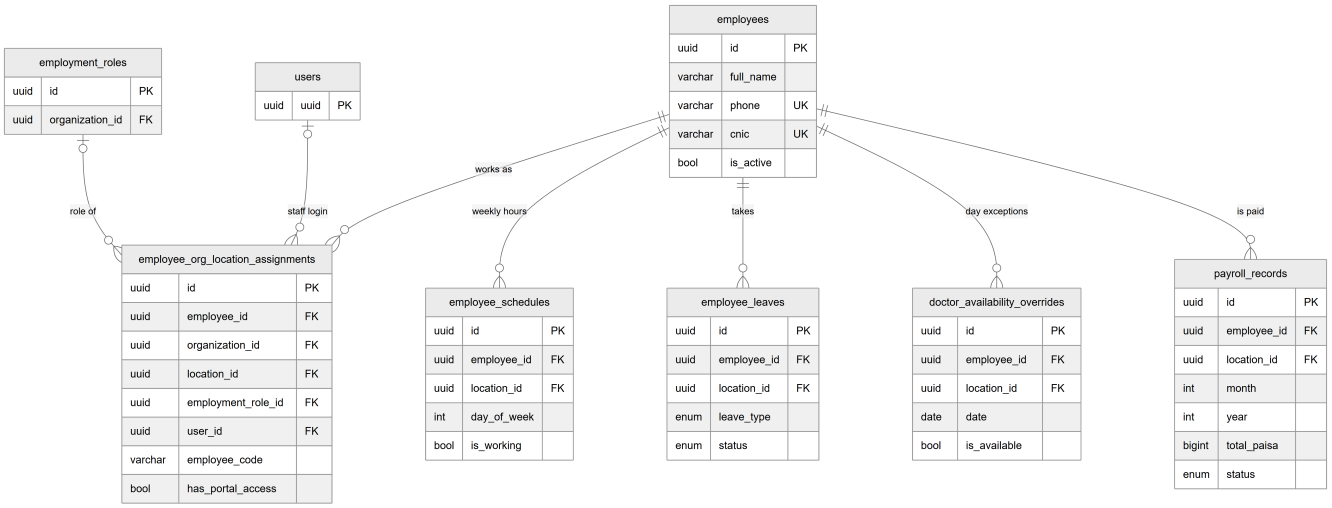

Figure 16.1 — Staff and payroll tables.

The diagram has `employees` at the top centre, showing `id`, `full_name`, `phone` and `cnic` (both marked unique) and `is_active`. Five solid lines fan out from it to its children, each "one employee to zero or more rows": `employee_org_location_assignments` ("works as"), `employee_schedules` ("weekly hours"), `employee_leaves` ("takes"), `doctor_availability_overrides` ("day exceptions") and `payroll_records` ("is paid"). The assignment box, on the left, lists `id`, `employee_id`, `organization_id`, `location_id`, `employment_role_id`, `user_id`, `employee_code` and `has_portal_access`; over it two context boxes, `employment_roles` and `users`, connect to it with optional lines labelled "role of" and "staff login". Each child box shows its `employee_id` and `location_id` foreign keys plus its key business columns (`day_of_week` and `is_working`; `leave_type` and `status`; `date` and `is_available`; `month`, `year`, `total_paisa` and `status`). The links from these tables to `organizations` and `locations` are not drawn, to keep the picture readable; they are listed in the next section.

16.4 Relationships explained

Foreign-key relationships inside the domain:

- **`employee_org_location_assignments.employee_id` → `employees`.** Each assignment belongs to exactly one person; a person has zero or more assignments. Deleting the person deletes the assignments (CASCADE).
- **`employee_org_location_assignments.employment_role_id` → `employment_roles`.** Optional, zero or one role. Deleting the role sets it to null (SET NULL).
- **`employee_org_location_assignments.user_id` → `users`.** Optional, zero or one staff login; a user can be linked to several assignments. Deleting the user sets it to null (SET NULL).
- **`employee_schedules.employee_id`, `employee_leaves.employee_id`, `doctor_availability_overrides.employee_id`, `payroll_records.employee_id` → `employees`.** Each row belongs to exactly one person; a person has zero or more of each. Deleting the person deletes them all (CASCADE).

Tenant foreign keys (not drawn in the figure):

- **`organization_id` → `organizations`** on assignments, schedules, leaves, availability overrides and payroll records: mandatory, CASCADE.
- **`location_id` → `locations`** on the same five tables: mandatory, CASCADE. Deactivating a location never triggers this, because the application only soft-deletes locations.

Logical relationships (no foreign key): the actor columns `created_by` (on assignments, schedules, leaves, overrides and payroll), `approved_by` on leaves and `paid_by` on payroll hold a user id.

Links into this domain from other domains:

- **Into `employees`:** `appointments.doctor_id` and `queue_sessions.doctor_id` (RESTRICT), `charges.doctor_id`, `service_prices.doctor_id` and `lab_orders.doctor_id` (RESTRICT), `prescriptions.doctor_id` (SET NULL), `expenses.recorded_by` and `patient_credits.purchased_by` (SET NULL), and the inventory columns `patient_consumptions.performedBy`, `purchase_orders.createdBy` and `stock_movements.performedBy` (NO ACTION, which behaves like RESTRICT).
- **Into `employee_org_location_assignments`:** `compensation_plans.assignment_id` (CASCADE), and `charges.performing_assignment_id`, `commission_entries.assignment_id` and `commission_settlements.assignment_id` (RESTRICT).
- **Into `payroll_records`:** `commission_settlements.payroll_record_id`, one-to-one and RESTRICT: a payroll record that settled commissions cannot be deleted while the settlement exists.

Because of these RESTRICT links, a person who is the doctor on any appointment, queue session, charge, special price or lab order, or who appears in inventory records, cannot be hard-deleted; the database refuses. The employee service catches that refusal and replies that the employee "has linked inventory records" and should be deactivated instead — the message names inventory even when the blocking row is an appointment or a charge.

16.5 Tenant scope

- `employees` is global: one person record for the whole Platform. The tenant link lives on the assignment.
- `employee_org_location_assignments`, `employee_schedules`, `employee_leaves`, `doctor_availability_overrides` and `payroll_records` are organization + location scoped, with real foreign keys on both columns.

16.6 Ownership and ambiguities

- **Platform-wide uniqueness of personal data.** Because `employees.phone` and `employees.cnic` are unique across the Platform, the database refuses a second person record with the same phone or national identity number, even in a different clinic. The design intends one person record shared by many clinics; how the employee screens handle a person already known elsewhere is covered in Chapter 37.
- **Code comment versus schema.** The comment on the employee delete method says that appointments and invoices keep their rows with the doctor set to null. The schema says otherwise: `appointments.doctor_id`, `queue_sessions.doctor_id` and `charges.doctor_id` are RESTRICT, so such a delete is refused.
- **Deprecated salary column.** `employee_org_location_assignments.gross_salary` is superseded by `compensation_plans`.
- **Two money forms on one row.** `payroll_records` keeps the total in paisa and in rupees; the paisa columns are authoritative for new code.

Chapter 17 — Patients

This chapter describes the patient record and the fourteen sub-record tables that hang from it: clinical history, charts, vitals, plans, documents, consents, contacts, recalls, referrals, feedback, insurance and the per-location visit summary. It is for the product owner who needs to know where a given piece of patient information is stored and what happens to it when a patient is deactivated or deleted. The patient module itself — screens, import, export and permissions — is described in Chapter 28.

17.1 Context

All fifteen tables are owned by the `patients` module. The patient is an organization-level record: one clinic business has one record per patient, shared by all of its locations. Most sub-records also remember the location where they were created. One of the fourteen sub-record tables, `patient_prescriptions`, is the legacy prescription table; it is summarised here and described in full in Chapter 19 next to the current prescription tables. Every column is listed in the entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`.

17.2 The patient record

17.2.1 `patients` — patient

Status: [Implemented]

One row per patient per organization. Key columns:

- `organization_id` — the owning organization.
- `patient_number` — the clinic's own patient number, unique within the organization (`organization_id` + `patient_number`).
- `full_name`, `dob`, `gender`, `blood_group`, `marital_status`, `occupation`.
- Contact fields: `phone` (required, **not** unique — two patients can share a family phone), a WhatsApp number, email, address and city. A national identity number column also exists. These hold personal data and are never reproduced in this handbook.
- `preferred_language` (`urdu` or `english`, default `urdu`) and `communication_pref` (`whatsapp`, `sms`, `call`, `email`, default `whatsapp`).
- `primary_location_id` — the patient's home branch; nullable.
- `source` (`walk_in`, `referral`, `online`, `social_media`, `google`, `other`) and a free-text `referred_by_text`.
- Minor and guardian fields, `medical_history` and `allergies` as free text, emergency-contact fields, `internal_notes` and `tags` (a JSON list).
- `is_active` — the activate/deactivate switch used by the patient screens.
- `is_blacklisted` and `blacklist_reason`.
- `deleted_at` — a TypeORM soft-delete column. The permanent-delete action does not use it; it removes the row.
- `user_id` — a reference to a user account for the patient, with no foreign key. The code does not say what fills it.

Indexes support search by name, phone and national identity number, the active list (`organization_id` + `is_active`), the blacklist filter (`organization_id` + `is_blacklisted`) and lookups by primary location.

17.3 Clinical sub-records

17.3.1 `patient_dental_info` — dental profile

Status: [Implemented]

At most one row per patient (`patient_id` is unique): last check-up date, gum condition, sensitivity, brushing and flossing habits, smoking and tobacco chewing, orthodontic history, prosthetics, previous extractions and surgeries, X-ray notes and dentist notes. It carries `organization_id` but no location: the profile belongs to the patient, not to a branch.

17.3.2 `patient_teeth_chart` — teeth chart snapshot

Status: [Implemented]

One row per charting. `chart_data` is JSON keyed by tooth number in FDI notation (for example `"11"`), each holding condition, treatment, surface and notes; `periodontal_chart` is optional JSON. `is_initial_chart` marks the first chart, `recorded_at` the moment. `dentist_id` records who charted (target table not stated in the code) and `appointment_id` the visit, both without a foreign key.

17.3.3 `patient_vitals` — vitals reading

Status: [Implemented]

One row per measurement: blood pressure (`bp_systolic` , `bp_diastolic`), pulse, temperature, oxygen saturation, weight, height, BMI, risk flags (diabetic, hypertensive, pregnant, on blood thinners, on antibiotics), anaesthesia type and dose, notes and `recorded_at` . `recorded_by` and `anesthesia_administered_by` are user ids without a foreign key; `appointment_id` links the visit without a foreign key.

17.3.4 `patient_treatment_plans` — treatment plan

Status: [Partial] — backend endpoints exist and billing reads the table; no web screen calls the create or update endpoints

A planned course of work for a patient: `plan_number` (unique within the organization), `plan_name` , description, `estimated_cost` , `discount_amount` and `final_estimated_cost` (numeric, rupees), `status` (`draft` , `active` , `completed` , `cancelled` , `on_hold`), start, expected end and actual end dates, `patient_accepted` and `patient_accepted_at` , notes, and `dentist_id` (no foreign key). It has a `deleted_at` soft-delete column. Planned work is different from performed work, which is stored in `treatments` (Chapter 18).

17.3.5 `treatment_plan_items` — treatment plan item

Status: [Partial] — as for its parent plan

One procedure inside a plan: `procedure_name` , `procedure_code` , `tooth_number` , `tooth_surface` , `estimated_sessions` and `completed_sessions` , `estimated_cost` , `status` (`pending` , `in_progress` , `completed` , `skipped` , `postponed`), `sequence_order` , `assigned_to` (a user id, no foreign key) and notes. This table has **no organization column**; it is isolated only through its plan. An index on (`plan_id` , `sequence_order`) returns items in order.

17.3.6 `patient_documents` — patient document

Status: [Implemented]

One row per uploaded file: `document_type` (X-ray, CBCT scan, report, prescription, consent form, insurance card, identity-card copy, before photo, after photo, other), file name, file URL, thumbnail URL, size, MIME type, `tooth_number`, `taken_date`, `is_visible_to_patient`, notes and `uploaded_by` (a user id, no foreign key). It has a `deleted_at` soft-delete column.

17.3.7 `patient_consents` — signed consent

Status: [Implemented]

One row per consent signed: `consent_type` (general treatment, extraction, implant, root canal, orthodontic, sedation, photo release), `consent_version`, the full `consent_text`, `is_agreed`, `agreed_at`, `signed_by_name`, `signed_by_type` (default `patient`), an optional signature image URL and `witnessed_by`. The entity comment declares the table immutable: a revoked consent is a new row with `is_agreed = false`, never an update. The database does not enforce this; it is an application rule.

17.3.8 `patient_prescriptions` — legacy quick prescription

Status: [Implemented] (legacy; still readable and writable from the patient's Clinical tab)

The older, simpler prescription table, with medicines stored as a JSON list. The patient's Clinical tab shows it in a collapsed "Legacy quick-Rx history" section next to the current prescription list. Its columns and its relation to the current `prescriptions` table are described in Chapter 19.

17.4 Relationship and administrative sub-records

17.4.1 `patient_visits` — per-location visit summary

Status: [Partial] — the table exists and a read endpoint returns it; no code writes it and no screen calls the endpoint

Designed as one summary row per patient per location (`patient_id` + `location_id` unique): `first_visit_date`, `last_visit_date`, `visit_count`, `total_spent` (numeric, rupees) and `is_preferred_location`. The encounter entity's comment calls it a patient-to-location aggregate that is "left untouched". No service inserts or updates rows, so on a new database it stays empty. The clinical record of each consultation is `patient_encounters` (Chapter 18), not this table.

17.4.2 `patient_communications` — communication log

Status: [Implemented]

One row per contact with the patient: `type` (`sms`, `whatsapp`, `call`, `email`, `in_person`), `direction` (inbound or outbound), `purpose` (reminder, confirmation, cancellation, recall reminder, follow-up, billing, general, marketing, emergency), subject, message body, `status` (`pending`, `sent`, `delivered`, `read`, `failed`, `received`), sent, delivered and read times, `sent_by`, `is_automated`, `automation_trigger` and an external message id. Rows are written only through the patient module's create endpoint; nothing on the Platform sends these messages. Notifications on the Platform are in-app only and use their own tables (Chapter 24).

17.4.3 `patient_recalls` — recall

Status: [Partial] — backend endpoints exist; no web screen calls them

A reminder that the patient is due back: `recall_type` (six-month check-up, annual cleaning, post-treatment follow-up, implant review, orthodontic review, custom), `custom_name`, `due_date`, up to three reminder-sent timestamps, `status` (`scheduled`, `reminder_sent`, `contacted`, `booked`, `completed`, `ignored`, `overdue`), notes, `dentist_id` and `appointment_id` (no foreign keys).

17.4.4 `patient_referrals` — patient referral

Status: [Partial] — the patient screen lists referrals; no screen creates them

Records that one patient referred another: `referring_patient_id` (required), `referred_patient_id` (set once the referred person is registered), the referred person's name and phone before registration, `referral_date`, reward fields (`reward_given`, `reward_amount` in numeric rupees, `reward_type`, `reward_given_at`) and `status` (`pending`, `registered`, `rewarded`, `cancelled`). It is organization-scoped with no location.

17.4.5 `patient_feedback` — patient feedback

Status: [Partial] — the patient screen lists feedback; no screen creates it

One row per rating: `rating` (integer), `category` (doctor, service, facility, billing, waiting time, general), text, `is_anonymous`, `is_published`, a staff response with `responded_by` and `responded_at`, and `dentist_id` and `appointment_id` without foreign keys.

17.4.6 `patient_insurance` — insurance policy

Status: [Partial] — backend endpoints exist; no web screen calls them

One row per policy: provider name, plan name, policy number, policy holder, group number, coverage details and percentage, `max_annual_benefit` and `used_benefit` (numeric, rupees), valid-from and valid-until dates, card images and `is_active`. It is organization-scoped with no location.

17.5 Entity-relationship diagrams

The fifteen tables are drawn in two diagrams so that each stays readable.

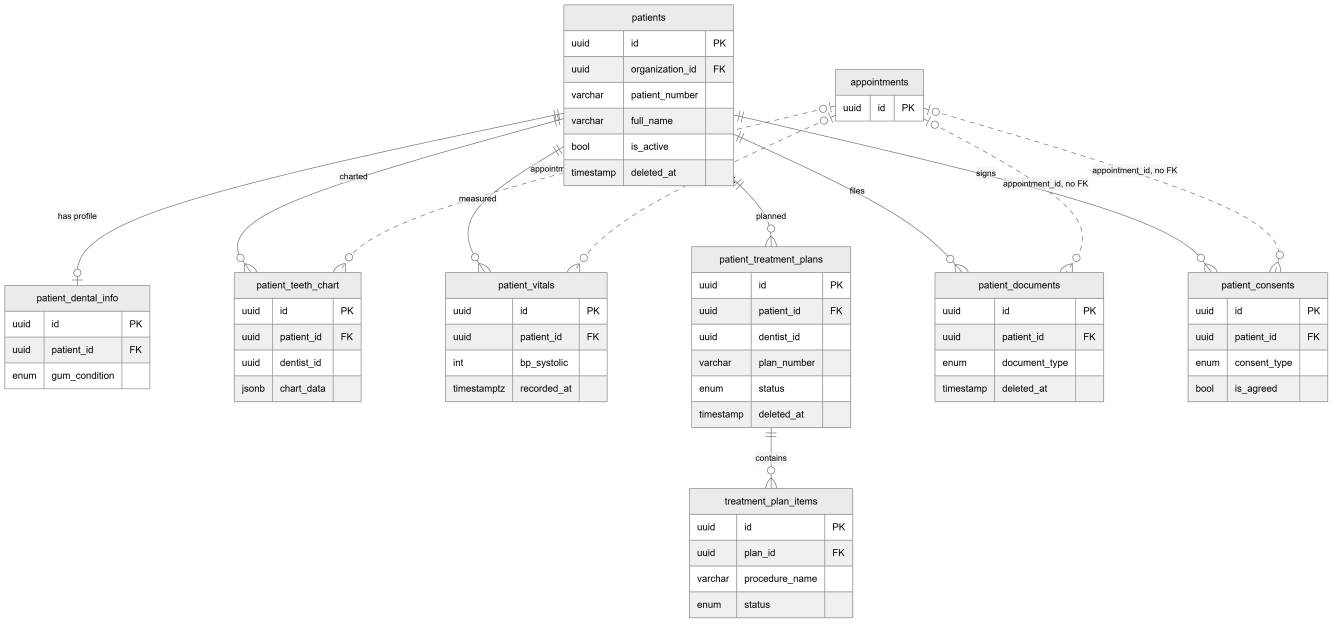

Figure 17.1 — Patient clinical sub-records.

The first diagram has `patients` at the top centre with `id`, `organization_id`, `patient_number`, `full_name`, `is_active` and `deleted_at`. Six solid lines go down from it to `patient_dental_info` ("has profile", at most one per patient), `patient_teeth_chart` ("charted"), `patient_vitals` ("measured"), `patient_treatment_plans` ("planned"), `patient_documents` ("files") and `patient_consents` ("signs"). Under `patient_treatment_plans`, a solid line labelled "contains" leads to `treatment_plan_items` (`id`, `plan_id`, `procedure_name`, `status`). A small `appointments` context box with only `id` sits to the right of `patients`; four dashed lines labelled "appointment_id, no FK" run from it to the teeth chart, vitals, documents and consents boxes. Each child box shows its `patient_id` foreign key and one to three business columns such as `chart_data`, `bp_systolic`, `plan_number`, `document_type` or `consent_type`. The legacy `patient_prescriptions` table is drawn in Figure 19.3, and the tenant links to `organizations` and `locations` are not drawn.

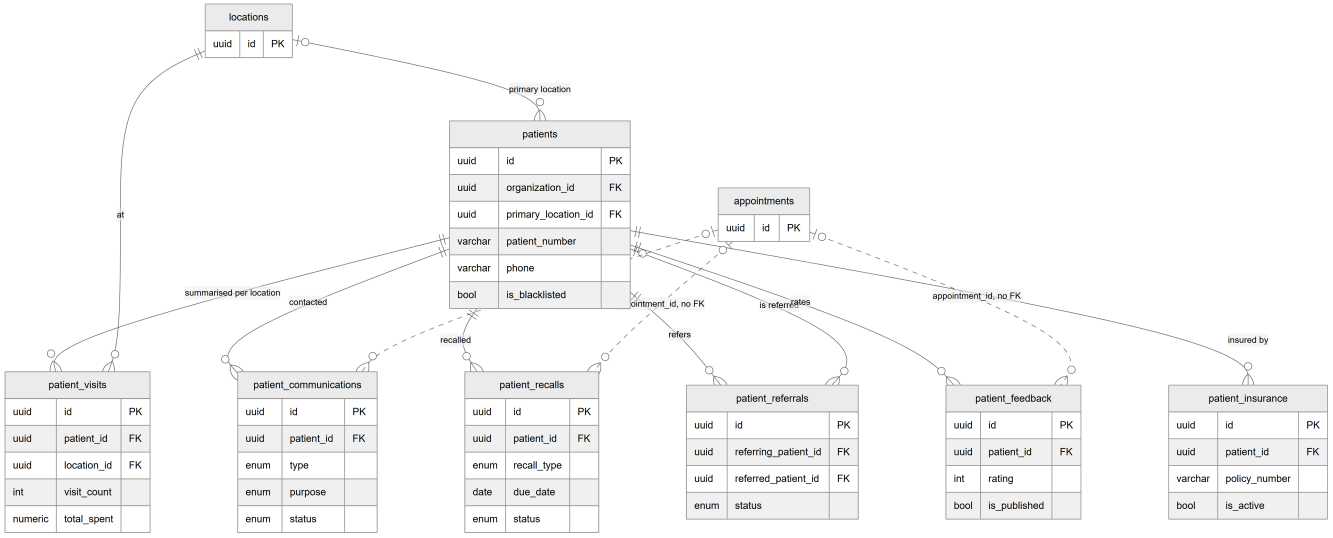

Figure 17.2 — Patient relationship and administrative sub-records.

The second diagram again centres on `patients`, this time showing `primary_location_id`, `phone` and `is_blacklisted`. A `locations` context box at the top left has an optional solid line to `patients` labelled "primary location" and a solid line to `patient_visits` labelled "at". From `patients`, solid lines go down to `patient_visits` ("summarised per location", with `visit_count` and `total_spent`), `patient_communications` ("contacted"), `patient_recalls` ("recalled"), `patient_referrals` (two lines: "refers" for the referring patient and an optional "is referred" for the referred patient), `patient_feedback` ("rates") and `patient_insurance` ("insured by"). An `appointments` context box connects with dashed "appointment_id, no FK" lines to communications, recalls and feedback.

17.6 Relationships explained

Foreign-key relationships inside the domain:

- **`patient_dental_info.patient_id` → `patients`.** One-to-one: a patient has zero or one dental profile. Deleting the patient deletes it (CASCADE).
- **`patient_teeth_chart`, `patient_vitals`, `patient_treatment_plans`, `patient_documents`, `patient_consents`, `patient_prescriptions`, `patient_visits`, `patient_communications`, `patient_recalls`, `patient_feedback`, `patient_insurance` → `patients` (`patient_id`).** Each row belongs to exactly one patient; a patient has zero or more rows in each. Deleting the patient deletes them all (CASCADE).
- **`treatment_plan_items.plan_id` → `patient_treatment_plans`.** Each item belongs to exactly one plan; a plan has zero or more items. Deleting the plan row deletes its items (CASCADE). A soft delete of the plan (`deleted_at`) does not touch the items.

- `patient_referrals.referring_patient_id` → `patients`. Mandatory. Deleting the referring patient deletes the referral (CASCADE).
- `patient_referrals.referred_patient_id` → `patients`. Optional. Deleting the referred patient keeps the referral and sets the link to null (SET NULL).
- `patients.primary_location_id` → `locations`. Optional. Deleting the location sets it to null (SET NULL).
- `patient_visits.location_id` → `locations`. Mandatory; deleting the location deletes its summary rows (CASCADE). The pair (`patient_id`, `location_id`) is unique.

Tenant foreign keys (not drawn):

- `organization_id` → `organizations` on every table except `treatment_plan_items`: mandatory, CASCADE.
- `location_id` → `locations` on teeth charts, vitals, treatment plans, documents, consents, legacy prescriptions, visits, communications, recalls and feedback: mandatory, CASCADE. `patients`, `patient_dental_info`, `patient_referrals` and `patient_insurance` have no location column.

Logical relationships (no foreign key):

- `appointment_id` → `appointments` on communications, consents, documents, feedback, legacy prescriptions, recalls, teeth charts and vitals. The database accepts an appointment id that does not exist.
- `dentist_id` on teeth charts, treatment plans, recalls and feedback. The code does not say whether it holds an employee id or a user id.
- `patients.user_id` → `users`.
- **Actor columns** — `created_by`, `sent_by`, `witnessed_by`, `uploaded_by`, `responded_by`, `prescribed_by`, `recorded_by`, `anesthesia_administered_by`, `assigned_to` — hold user ids.

Links into `patients` from other domains:

- **CASCADE** (deleted with the patient): `appointments`, `queue_entries`, `patient_encounters`, `treatments`, `prescriptions`, `patient_credits`, `patient_consumptions` (inventory) and `patient_comm_prefs` (notifications, one-to-one).
- **RESTRICT** (block the delete): `charges`, `invoices`, `payments`, `refunds`, `lab_orders`, `lab_invoices`, `lab_payments` and `lab_refunds`.
- **No foreign key:** `ledger_entries.patient_id`, `payment_allocations.patient_id`, `ledger_journal_lines.patient_id`, `lab_credit_notes.patient_id`, and the polymorphic history columns of `audit_logs` and `status_history`.

What a permanent delete does. Deleting a patient removes the patient row and every cascading child listed in this section, inside one transaction that first writes an audit row with the patient's last state and a count of linked records. If the patient has any billing or lab-billing row, the RESTRICT links make the database refuse the delete. Deactivating (`is_active = false`) removes nothing.

17.7 Tenant scope

- `patients`, `patient_dental_info`, `patient_referrals` and `patient_insurance` are organization-scoped.
- The other ten sub-record tables are organization + location scoped.
- `treatment_plan_items` has no tenant column and is scoped only through `patient_treatment_plans`; queries must join the plan.

17.8 Ownership and ambiguities

- **Four "visit" concepts.** `patient_visits` (an unused per-location summary), `appointments` (bookings), `queue_entries` (a place in the queue) and `patient_encounters` (the clinical record of a consultation) all describe visits. Chapter 27 compares them.
- **Three planning tables.** `patient_treatment_plans` holds planned work, `treatments` holds performed work, and `plan_templates` holds reusable plans (Chapter 19).
- **Code comment versus schema.** The comment on the patient delete method says invoices and payments are removed by cascade. In the schema, `invoices`, `charges`, `payments` and `refunds` reference the patient with RESTRICT, so a patient with billing history cannot be deleted. No code catches this refusal; what the user sees is [Unverified].
- **Uncertain targets.** The `dentist_id` columns may hold employee ids or user ids.
- **Immutability by convention.** `patient_consents` is declared immutable only in a comment.
- **Two soft-delete mechanisms on one table.** `patients` has both `is_active` and `deleted_at`; the screens use `is_active`.

Chapter 18 — Appointments, queue and encounters

This chapter describes the tables that follow a patient through one visit: the booking, the place in a doctor's queue, the realtime and safety tables the queue needs, the clinical record of the consultation, and the treatments performed. It is for the product owner who needs to know how a visit is stored at each step and which database rules stop double bookings or two patients being with one doctor at once. The behaviour is described in Chapters 29 (appointments), 30 (queue) and 31 (consultation); this chapter covers the data.

18.1 Context

Nine tables belong to this domain, owned by four modules: `appointments` owns `appointments`; `queue` owns `queue_sessions`, `queue_entries`, `queue_events`, `queue_policies`, `outbox` and `idempotency_keys`; `encounters` owns `patient_encounters`; and `treatments` owns `treatments`. A visit moves through them in order: an appointment is booked, at check-in (or walk-in registration) a queue entry is created in the doctor's queue session for that day, when the doctor starts the consultation an encounter is created, and treatments performed are recorded against the encounter. All queue changes go through one service, the `QueueOrchestrator`, which writes the queue tables, an event and an outbox message in one transaction. Every column is listed in the entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`.

18.2 The tables

18.2.1 `appointments` — appointment

Status: [Implemented]

One row per booking. Key columns:

- `organization_id`, `location_id`, `patient_id`, `doctor_id` (an employee).
- `type` (`consultation`, `follow_up`, `treatment`, `checkup`, `emergency`, `cleaning`) and `is_emergency`.
- `status` — `scheduled`, `confirmed`, `checked_in`, `completed`, `cancelled` or `no_show`.
- `start_time`, `end_time` (with time zone) and `duration_minutes`.
- `reason_for_visit` (shown in the queue row) and `notes`.
- `cancel_reason_code` (`no_show`, `patient_left`, `other`) and a free-text `cancel_reason`.
- Moment columns: `checked_in_at`, `completed_at`, `cancelled_at`, `no_show_at`.

The key database rule is the EXCLUDE constraint `no_doctor_overlap`: for the same `doctor_id`, no two appointments may have overlapping time ranges `[start_time, end_time)`, unless the appointment is `cancelled` or `no_show`. It needs the `btree_gist` extension. This is why a doctor can never be double-booked, even if two receptionists book at the same moment. Indexes support the doctor's calendar (`doctor_id`, `start_time`, `end_time`), the location's day (`location_id`, `start_time`), and filters by status, patient and organization.

18.2.2 `queue_sessions` — queue session

Status: [Implemented]

One doctor's queue for one service day at one location. Key columns:

- `organization_id`, `location_id`, `doctor_id`.
- `service_date` — the location-local date of the service day.
- `status` — `waiting_for_doctor`, `open`, `paused`, `closed`, with `pause_reason`, `started_at` and `closed_at`.
- `last_token_number` — the token counter (CHECK: never negative). Because every queue change first locks this row, tokens are gap-free and never duplicated.
- `token_prefix` — the doctor's token prefix, copied when the session is created.
- `smart_queue_enabled` — copied from the queue policy when the session is created; afterwards it changes only through the mode endpoint.
- `last_call_reason`, `event_seq` (the counter that numbers `queue_events`) and `version` (for optimistic checks).

(`organization_id`, `location_id`, `doctor_id`, `service_date`) is unique: one session per doctor per location per day.

18.2.3 `queue_entries` — queue entry

Status: [Implemented]

A patient's place in one queue session. It is created only at check-in of a booked appointment or at walk-in registration — never at booking. Key columns:

- `queue_session_id`, `patient_id`, and `appointment_id` (set for booked visits, null for walk-ins).
- `token_number` and `queue_type` (`appointment` or `walk_in`).
- `status` — `waiting`, `called`, `in_consultation`, `on_hold`, `skipped`, `completed`, `cancelled`, `transferred`, `aborted`.
- `priority_class` (`normal` or `authorized_urgent`) with `urgent_since` and `urgent_reason`.
- `checked_in_at` and `sort_anchor_at` (the first-in-first-out position: the check-in time, or the rejoin time after a rejoin at the end).
- Moment columns: `called_at`, `consultation_started_at`, `hold_started_at`, `returned_from_hold_at`, `skipped_at`, `finished_at`.
- `converted_from_appt_id` (the appointment a walk-in was converted from) and `transferred_from_id` (the entry this one replaced on a transfer to another doctor).
- `version` for optimistic checks.

The database rules on this table are the strongest in the Platform:

- **Unique token per session** — (`queue_session_id`, `token_number`).
- **One active entry per appointment** — a partial unique index on `appointment_id` where the status is waiting, called, in consultation, on hold or skipped.
- **One active entry per patient per session** — a partial unique index on (`queue_session_id`, `patient_id`) for the same statuses.
- **At most one patient with the doctor** — a partial unique index on `queue_session_id` where the status is `called` or `in_consultation`.
- **CHECK: token number greater than zero.**
- **CHECK: type matches appointment** — `queue_type = 'appointment'` exactly when `appointment_id` is set.
- **CHECK: urgent needs a reason** — `priority_class` is `normal`, or both `urgent_since` and `urgent_reason` are set.

A partial index on (`queue_session_id`, `sort_anchor_at`) for waiting entries makes "who is next" fast.

18.2.4 `queue_events` — queue event

Status: [Implemented]

An audit log of every queue change, written in the same transaction as the change. Key columns: `queue_session_id`, `queue_entry_id`, `seq` (from the session's `event_seq`, unique per session, so screens can apply events in order and notice gaps), `event_type` (for example `CHECKED_IN`, `CALLED`, `SKIPPED`, `MODE_CHANGED`), `from_status`, `to_status`, reason code and text, `suggested_entry_id` (when staff selected a patient manually, the entry the selection engine would have called), `actor_id`, `actor_role`, `metadata` and `occurred_at`. The primary key is a bigint identity. The entity comment calls the table append-only (corrections are new events); the database does not enforce it.

18.2.5 `queue_policies` — queue policy

Status: [Implemented]

Queue settings. A row with `location_id` null is the organization default; a row with a location overrides it for that location. Two partial unique indexes keep one default per organization and one override per location. Columns and their CHECK rules:

- `smart_queue_enabled` — smart or manual selection for new sessions.
- `early_window_min` (CHECK 0 to 240), `late_grace_min` (CHECK at least 0), `max_late_min` (CHECK at least `late_grace_min`) — how early and how late a booked patient may check in.
- `walkin_starvation_min` (CHECK greater than 0) — the walk-in waiting threshold, in minutes, used by the selection engine (Chapter 30 explains the rule).
- `skipped_rejoin_policy` — `end`, `original_position` or `manual`.
- `call_timeout_sec` — null turns automatic skip-after-call off; otherwise at least 30 seconds.
- `manual_select_reason_required`.
- `service_day_cutoff` — the local time a service day starts, for example "04:00" for a clinic that works past midnight.

The timezone is not stored here; it is always the location's own `locations.timezone`.

18.2.6 `outbox` — realtime outbox

Status: [Implemented]

The transactional outbox for realtime screens. A queue change writes its realtime message here in the same transaction; a relay later publishes it to the socket room named in `topic` (for example a session room or a location's display room) and sets `published_at`. Screens therefore never see a change that rolled back and never miss one that committed. A partial index on `id` where `published_at` is null lets the relay find unpublished rows quickly. A cleanup job deletes published rows older than 7 days, every day at 03:15 server time. This table is separate from `notification_outbox` (Chapter 24).

18.2.7 `idempotency_keys` — idempotency record

Status: [Implemented]

Stores the result of a mutation keyed by the client's `Idempotency-Key` header, so a retried request returns the saved answer instead of doing the work twice (for example, a check-in whose reply was lost returns the same token). The primary key is (`organization_id`, `key`); other columns are `request_hash`, `response_status`,

`response_body` and `created_at`. The queue writes it, and the shared idempotency service used by the lab modules and the workflow engine writes to the same table with keys namespaced as `scope:key`. The same daily 03:15 cleanup job deletes every row older than 24 hours.

18.2.8 `patient_encounters` — encounter

Status: [Implemented]

The clinical record of one consultation, created when the doctor starts the consultation on a queue entry. `queue_entry_id` is unique: exactly one encounter per queue entry. `appointment_id` is set for booked visits and null for walk-ins. Other key columns: `patient_id`, `organization_id`, `location_id`, `encounter_date`, `reason` (why the patient actually came, which can differ from the booking), `chief_complaint`, `examination`, `diagnosis`, `advice` and `follow_up_date`.

18.2.9 `treatments` — treatment (work performed)

Status: [Implemented]

A treatment actually carried out. It always belongs to a patient and may be linked to the encounter it was performed in. Key columns: `patient_id`, `encounter_id`, `plan_template_id` (the plan template it was based on), `treatment_name`, `notes`, `date`, `service_id` (the catalog service billing uses to price the charge) and `cost` (a legacy fee in numeric rupees, used by billing only when no service is set). Planned work lives in `patient_treatment_plans` (Chapter 17), not here.

18.3 Entity-relationship diagrams

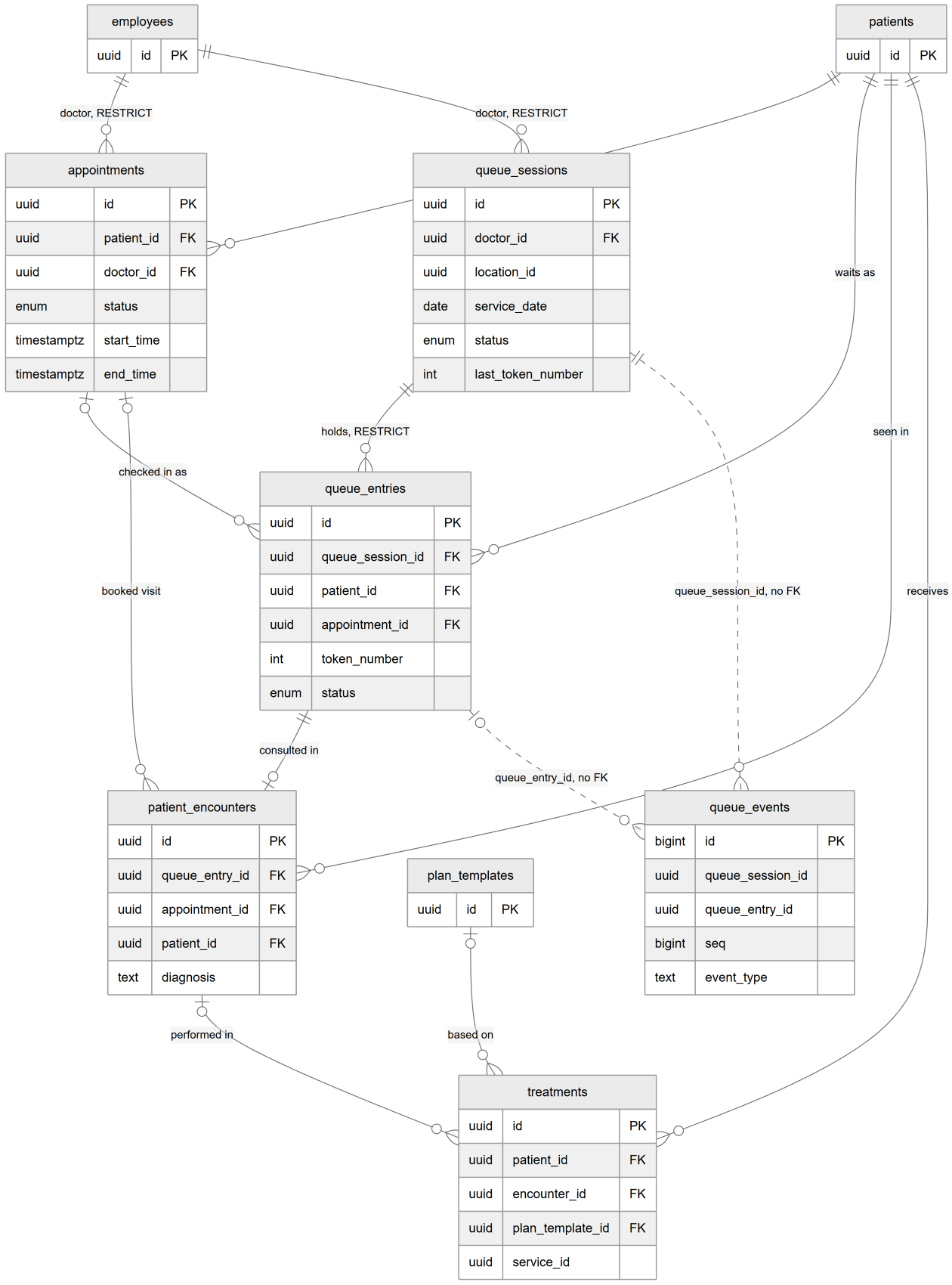

Figure 18.1 — From booking to treatment.

The diagram has two context boxes at the top, `employees` on the left and `patients` on the right, each showing only `id`. From `employees`, two solid lines labelled "doctor, RESTRICT" go to `appointments` and `queue_sessions`. From `patients`, solid lines go to `appointments` ("books"; this label is hidden behind the `queue_sessions` box in

the rendered image), `queue_entries` ("waits as"), `patient_encounters` ("seen in") and `treatments` ("receives"). `appointments` (`id` , `patient_id` , `doctor_id` , `status` , `start_time` , `end_time`) has optional solid lines to `queue_entries` ("checked in as") and to `patient_encounters` ("booked visit"). `queue_sessions` (`id` , `doctor_id` , `location_id` , `service_date` , `status` , `last_token_number`) has a solid line labelled "holds, RESTRICT" to `queue_entries` (`id` , `queue_session_id` , `patient_id` , `appointment_id` , `token_number` , `status`). `queue_entries` has a solid line labelled "consulted in" to `patient_encounters` (at most one encounter per entry). `queue_events` (`id` , `queue_session_id` , `queue_entry_id` , `seq` , `event_type`) is joined by two dashed lines, "queue_session_id, no FK" from `queue_sessions` and "queue_entry_id, no FK" from `queue_entries` . At the bottom, `treatments` (`id` , `patient_id` , `encounter_id` , `plan_template_id` , `service_id`) has optional solid lines from `patient_encounters` ("performed in") and from a `plan_templates` context box ("based on"). Tenant links are not drawn.

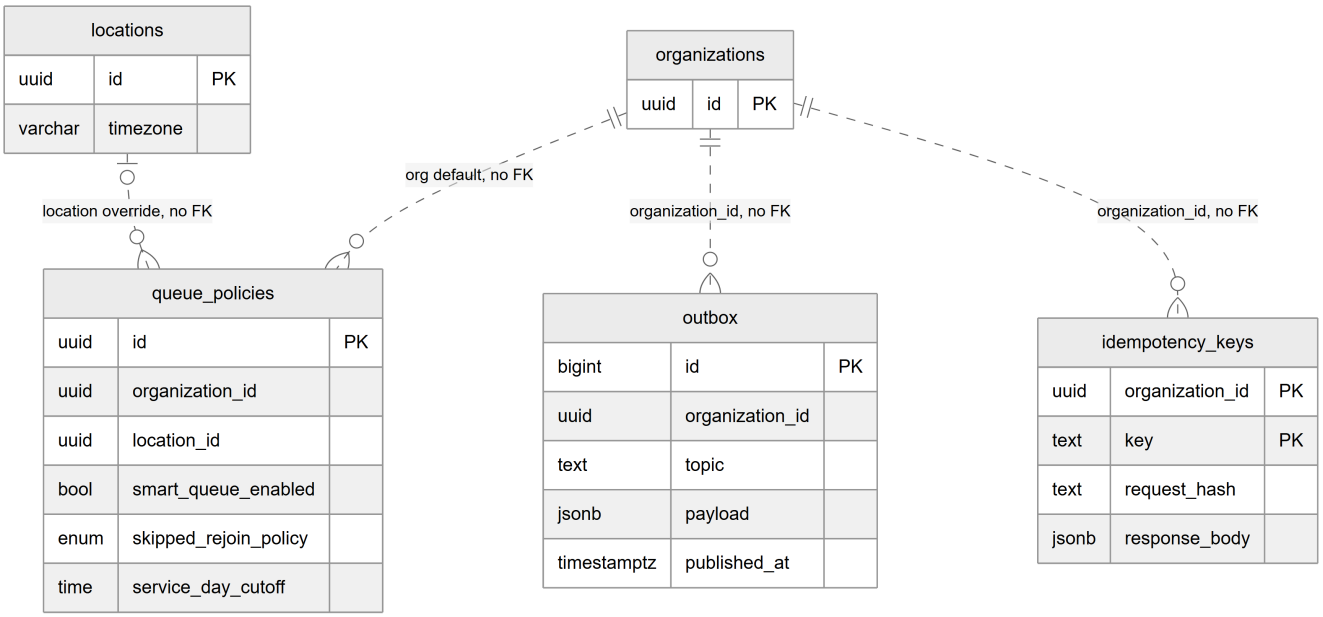

Figure 18.2 — Queue support tables.

The second diagram shows the three queue tables that have no foreign key at all. `organizations` (with `id`) is joined by dashed lines to `queue_policies` ("org default, no FK"), `outbox` ("organization_id, no FK") and `idempotency_keys` ("organization_id, no FK"). `locations` (with `id` and `timezone`) is joined by a dashed line to `queue_policies` labelled "location override, no FK". The boxes show `queue_policies` with `organization_id` , `location_id` , `smart_queue_enabled` , `skipped_rejoin_policy` and `service_day_cutoff` ; `outbox` with a bigint `id` , `organization_id` , `topic` , `payload` and `published_at` ; and `idempotency_keys` with its two-part key `organization_id` + `key` , `request_hash` and `response_body` .

18.4 Relationships explained

Foreign-key relationships:

- **`appointments.patient_id` → `patients` .** Each appointment has exactly one patient; a patient has zero or more. Deleting the patient deletes the appointments (CASCADE).
- **`appointments.doctor_id` → `employees` .** Each appointment has exactly one doctor. A doctor with appointments cannot be deleted (RESTRICT).
- **`appointments.organization_id` → `organizations`** and **`appointments.location_id` → `locations` .** Mandatory, CASCADE.
- **`queue_sessions.doctor_id` → `employees` .** Each session has exactly one doctor. RESTRICT.

- `queue_entries.queue_session_id` → `queue_sessions`. Each entry belongs to exactly one session; a session has zero or more entries. A session with entries cannot be deleted (RESTRICT).
- `queue_entries.patient_id` → `patients`. Mandatory. CASCADE.
- `queue_entries.appointment_id` → `appointments`. Optional; an appointment has zero or more entries over time (only one active, by the partial unique index). Deleting the appointment deletes its entries (CASCADE).
- `patient_encounters.queue_entry_id` → `queue_entries`. Each encounter has exactly one queue entry; an entry has zero or one encounter (unique). Deleting the entry deletes the encounter (CASCADE).
- `patient_encounters.appointment_id` → `appointments`. Optional. Deleting the appointment keeps the encounter and sets the link to null (SET NULL).
- `patient_encounters.patient_id` → `patients`, `organization_id` → `organizations`, `location_id` → `locations`. Mandatory, CASCADE.
- `treatments.patient_id` → `patients`. Mandatory, CASCADE.
- `treatments.encounter_id` → `patient_encounters`. Optional; an encounter has zero or more treatments. Deleting the encounter keeps the treatments with the link set to null (SET NULL).
- `treatments.plan_template_id` → `plan_templates`. Optional. SET NULL.
- `treatments.organization_id` → `organizations` and `location_id` → `locations`. Mandatory, CASCADE.

Logical relationships (no foreign key):

- `queue_events.queue_session_id` → `queue_sessions`, `queue_entry_id` → `queue_entries`, `suggested_entry_id` → `queue_entries`, `actor_id` → `users`.
- `queue_entries.converted_from_appt_id` → `appointments` and `transferred_from_id` → `queue_entries` (a link to another entry of the same table).
- `treatments.service_id` → `services` (billing's catalog, Chapter 20).
- **Tenant columns without a foreign key:** `organization_id` on `queue_sessions`, `queue_entries`, `queue_events`, `queue_policies` and `outbox`; `location_id` on `queue_sessions` and `queue_policies`. `idempotency_keys.organization_id` is part of its primary key, also without a foreign key.
- **Actor columns** — `created_by` and `updated_by` on appointments and encounters, `created_by` on treatments.

Links into this domain from other domains:

- `prescriptions.appointment_id` and `prescriptions.encounter_id` (SET NULL) — Chapter 19.
- `patient_consumptions.appointmentId` (SET NULL) — Chapter 23.
- Logical only: `charges.encounter_id` and `charges.appointment_id`, `lab_orders.appointment_id` and `lab_orders.treatment_id`, `order_item_stages.appointment_id`, and the `appointment_id` columns of the patient sub-records.

18.5 Tenant scope

- `appointments`, `queue_sessions`, `queue_policies`, `patient_encounters` and `treatments` are organization + location scoped.
- `queue_entries`, `queue_events`, `outbox` and `idempotency_keys` carry only `organization_id`; an entry's location is the location of its session.
- The queue tables keep their tenant columns without foreign keys; the orchestrator writes them.

18.6 Ownership and ambiguities

- **Four "visit" tables.** An appointment is the booking, a queue entry is a place in line, an encounter is the clinical record, and `patient_visits` is an unused summary (Chapter 17). Chapter 27 compares them.
- **Two outboxes.** `outbox` carries realtime screen updates; `notification_outbox` carries notifications.
- **Shared idempotency table.** `idempotency_keys` lives in the queue module, but the shared idempotency service used by labs and the workflow engine writes to it too, and the queue's daily cleanup removes those rows after 24 hours as well.
- **Append-only by convention.** `queue_events` is declared append-only only in a comment.
- **RESTRICT on the doctor.** Because sessions and appointments block deletion of their doctor, a doctor who has worked cannot be hard-deleted; deactivation is the only option (Chapter 16).

Chapter 19 — Prescriptions and plans

This chapter describes the tables behind prescriptions and the clinic's reusable libraries: the prescription itself with its medicine lines and version history, the medicine directory, prescription presets, print layouts, doctors' saved signatures, plan templates, and the older prescription table that still exists. It is for the product owner who needs to know where a printed prescription's data comes from and why several tables have "plan" or "template" in their name. The prescription module is described in Chapter 32 and treatment plans in Chapter 33.

19.1 Context

Ten tables are covered here. Eight are owned by the `prescriptions` module: `prescriptions`, `prescription_medicines`, `prescription_history`, `prescription_templates`, `doctor_signatures`, `medicine_directory`, `treatment_plan_templates` and `treatment_plan_template_medicines`. `plan_templates` is owned by the `plan-templates` module. The legacy `patient_prescriptions` table is owned by the `patients` module. The patient's own planned work, `patient_treatment_plans` and `treatment_plan_items`, also belongs to the patients domain and is described in Chapter 17.

The names are easy to confuse, so here is the mapping from screen to table. In the web app's prescription library, **Medicines** is `medicine_directory`, **Presets** (ready-made medicine sets) is `treatment_plan_templates` despite its name, and **Print layouts** is `prescription_templates`. The sidebar entry **Treatment Plans** opens the plan template library, which is `plan_templates`. Every column is listed in the entity catalog, `backend/docs/data-model/ENTITY_CATALOG.md`.

19.2 The prescription record

19.2.1 `prescriptions` — prescription

Status: [Implemented]

The prescription a doctor writes, signs and shares. Key columns:

- `prescription_number` — formatted `RX-<year>-<5 digits>`, for example `RX-2026-00001`.
- `organization_id`, `location_id`, `patient_id` (all required).
- `appointment_id`, `encounter_id`, `doctor_id` — the visit and the doctor; all optional.
- `prescription_template_id` (the print layout) and `treatment_plan_template_id` (the preset it was pre-filled from); both optional.
- Clinical text: `diagnosis`, `chief_complaint`, `further_plan`, and instructions in English and in Urdu.
- `vitals` — a JSON copy of the readings (blood pressure, pulse, temperature, oxygen, weight, height, BMI, respiratory rate).
- `signature_image_url` and `signed_at`.
- `status` — `DRAFT`, `FINALIZED` or `AMENDED`; `version` starts at 1.
- `pdf_url` — a cached generated PDF, cleared when the record is amended.

`prescription_number` is unique across the whole Platform, while the number generator counts per organization (it reads the organization's latest number and adds one). Indexes support lookups by patient, appointment, encounter, doctor and by (`organization_id`, `location_id`).

19.2.2 `prescription_medicines` — prescription medicine line

Status: [Implemented]

One line per medicine on a prescription: `medicine_name` and its Urdu name, `type`, `dosage`, `frequency`, `frequency_timing` (a JSON list such as `["morning","evening"]`), `meal_relation` (before, after or with a meal, empty stomach, none), `duration`, special instructions in English and Urdu, and `sort_order`. `medicine_directory_id` keeps a link to the directory entry it came from, but the name and every other value are copied onto the line, so a finalized prescription stays a faithful record even if the directory entry is renamed or switched off later. This table has no tenant column; it is isolated through its prescription.

19.2.3 `prescription_history` — prescription change history

Status: [Implemented]

One row per change event: `version`, `change_type` (`CREATED`, `AMENDED`, `FINALIZED`, `PRINTED`, `SHARED`), `changed_by` (a user id), `change_notes`, and `previous_data` — a full JSON snapshot of the prescription and its medicines before the change (null for `CREATED`). Any past version can be shown from this table alone. The entity comment calls it immutable; the database does not enforce that. It has no tenant column and is isolated through its prescription.

19.3 The libraries

19.3.1 `medicine_directory` — medicine directory

Status: [Implemented] (screen label "Medicines")

The clinic's own medicine catalogue, which powers the search while writing a prescription. Each row: `name`, `generic_name`, `name_urdu`, `type` (tablet, capsule, syrup, injection, cream, drops and others), `strength`, `manufacturer`, default dosage, frequency, duration and meal relation, Urdu instructions, and `is_active`. Entries are switched off with `is_active`, never hard-deleted, so old prescriptions still resolve. Indexes cover (`organization_id`, `name`) and (`organization_id`, `is_active`).

19.3.2 `treatment_plan_templates` — prescription preset

Status: [Implemented] (screen label "Presets")

A reusable prescription protocol, for example "Root canal aftercare": `name`, `description`, `category` (forty categories such as pain relief, post-extraction, implant, pregnancy-safe), `specialty_tags`, `further_plan`, instructions in English and Urdu, and `is_active`. Choosing a preset in the prescription builder pre-fills the medicines and texts; the doctor can still edit before finalizing. Despite the name, it is not a treatment plan.

19.3.3 `treatment_plan_template_medicines` — preset medicine line

Status: [Implemented]

The medicine lines of a preset, with the same columns as `prescription_medicines` and the same denormalized copy of the directory values. It has no tenant column and is isolated through its preset.

19.3.4 `prescription_templates` — print layout

Status: [Implemented] (screen label "Print layouts")

The look of the printed prescription: `name`, `is_default`, `page_size` (`A4`, `A5`, `LETTER`, `CUSTOM`), `orientation`, custom width and height in millimetres, and JSON blocks for `header` (logo, clinic and doctor names in English and Urdu, qualifications, registration number), `footer` (address, timings, disclaimer) and `styling` (colours, fonts, toggles, margins). `location_id` null means an organization-wide layout usable at any location; a location can also have its own default, and several layouts per location are allowed. Nothing in the database limits a location to one default; an index on (`location_id`, `is_default`) only speeds the lookup.

19.3.5 `doctor_signatures` — saved doctor signature

Status: [Implemented]

A doctor's saved signature image (`signature_image_url`, optional `signature_data`), attached to prescriptions they finalize unless they draw a new one. `employee_id` is unique: one signature per owner, and saving again updates the row in place. Despite its name, the column holds the signed-in **user's** id, not an employee id — the controller passes the user id from the login token. Because the uniqueness is on that id alone, a user who works for two organizations has one signature row, stored with the organization where it was first saved, and the read path does not filter by organization.

19.3.6 `plan_templates` — plan template

Status: [Implemented] (sidebar label "Treatment Plans")

A predefined treatment plan a doctor can pick, for example a root canal with its usual steps: `name`, `category` (seventeen dental categories such as endodontics, prosthodontics, orthodontics, emergency), `description`, `items` (a JSON list of procedures) and `is_active` (inactive templates are hidden from the picker but kept). When a treatment is recorded from the patient screen, the chosen template is saved in `treatments.plan_template_id`. `organization_id` has no foreign key.

19.3.7 `patient_prescriptions` — legacy quick prescription

Status: [Implemented] (legacy; shown and still creatable in the "Legacy quick-Rx history" section of the patient's Clinical tab)

The older prescription table, which coexists with `prescriptions` while consumers move to the newer schema. Each row has `prescription_number` (unique within the organization), `prescription_date`, chief complaint, diagnosis, `medications` as one JSON list, special instructions, `follow_up_date`, print tracking (`is_printed`, `printed_at`, `print_count`), `prescribed_by` (a user id) and `appointment_id`, both without foreign keys. It is organization + location scoped with foreign keys to `patients`, `organizations` and `locations`, all CASCADE.

19.4 Entity-relationship diagrams

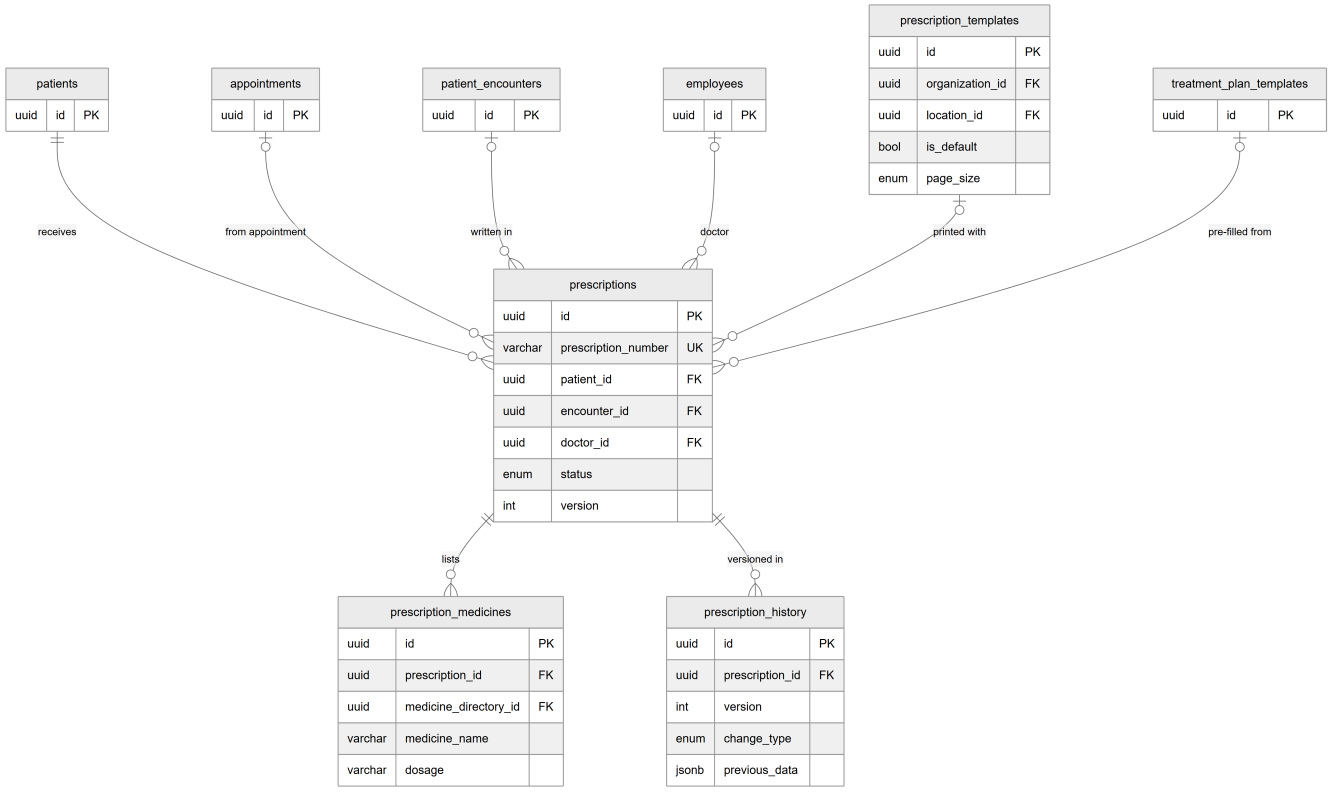

Figure 19.1 — The prescription record.

The diagram centres on `prescriptions`, showing `id`, the unique `prescription_number`, `patient_id`, `encounter_id`, `doctor_id`, `status` and `version`. Six context boxes sit in a row over it, each with an optional or mandatory solid line into `prescriptions`: `patients` ("receives", mandatory), `appointments` ("from appointment"), `patient_encounters` ("written in"), `employees` ("doctor"), `prescription_templates` ("printed with", shown with `organization_id`, `location_id`, `is_default` and `page_size`) and `treatment_plan_templates` ("pre-filled from"). Under `prescriptions`, two solid lines lead to its children: `prescription_medicines` ("lists", with `prescription_id`, `medicine_directory_id`, `medicine_name` and `dosage`) and `prescription_history` ("versioned in", with `prescription_id`, `version`, `change_type` and `previous_data`).

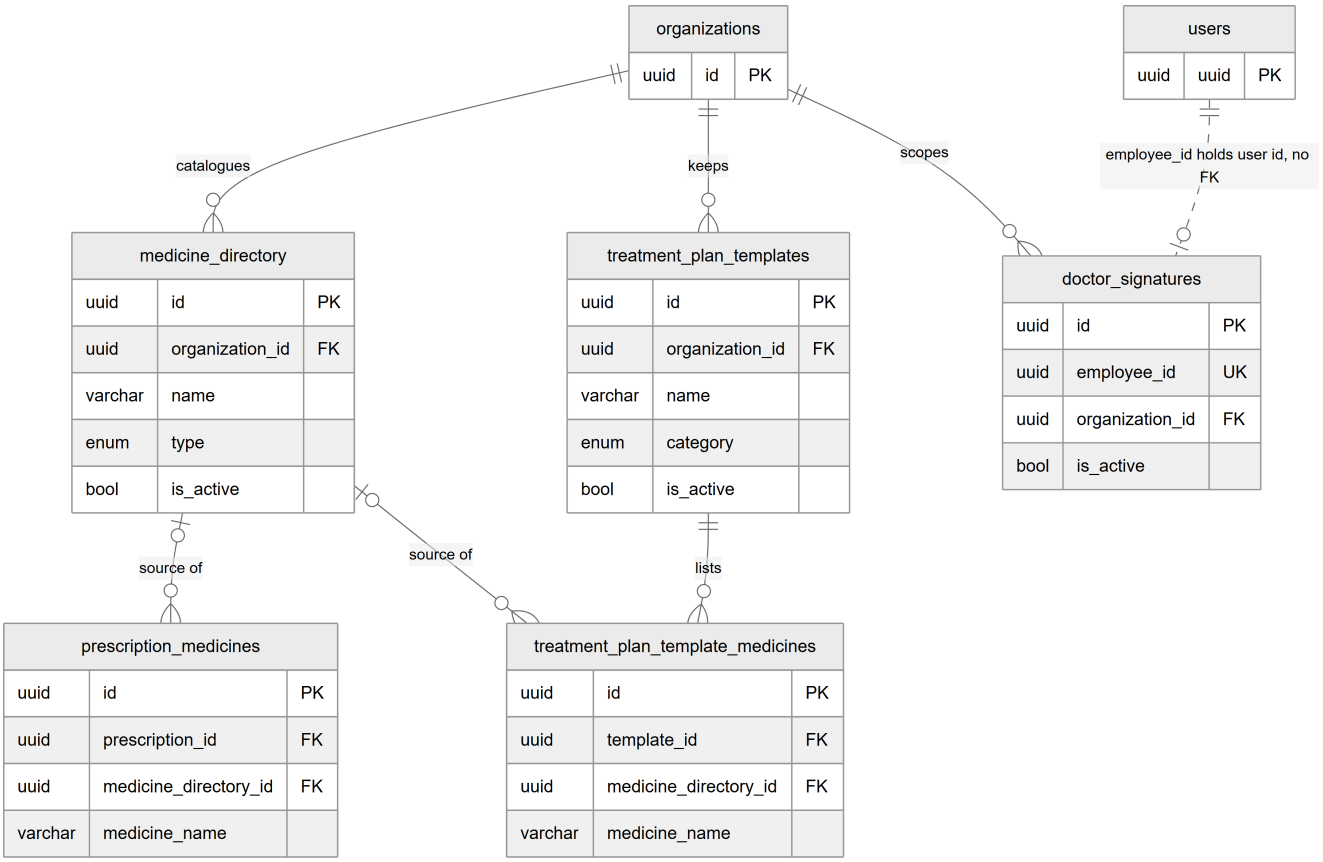

Figure 19.2 — Prescription libraries.

The second diagram shows the libraries. `organizations` has solid lines to `medicine_directory` ("catalogues"), `treatment_plan_templates` ("keeps") and `doctor_signatures` ("scopes"). `medicine_directory` (`id` , `organization_id` , `name` , `type` , `is_active`) has two optional solid lines labelled "source of", one to `prescription_medicines` and one to `treatment_plan_template_medicines` . `treatment_plan_templates` has a solid line labelled "lists" to `treatment_plan_template_medicines` . A `users` box is joined to `doctor_signatures` (`id` , the unique `employee_id` , `organization_id` , `is_active`) by a dashed line labelled "employee_id holds user id, no FK".

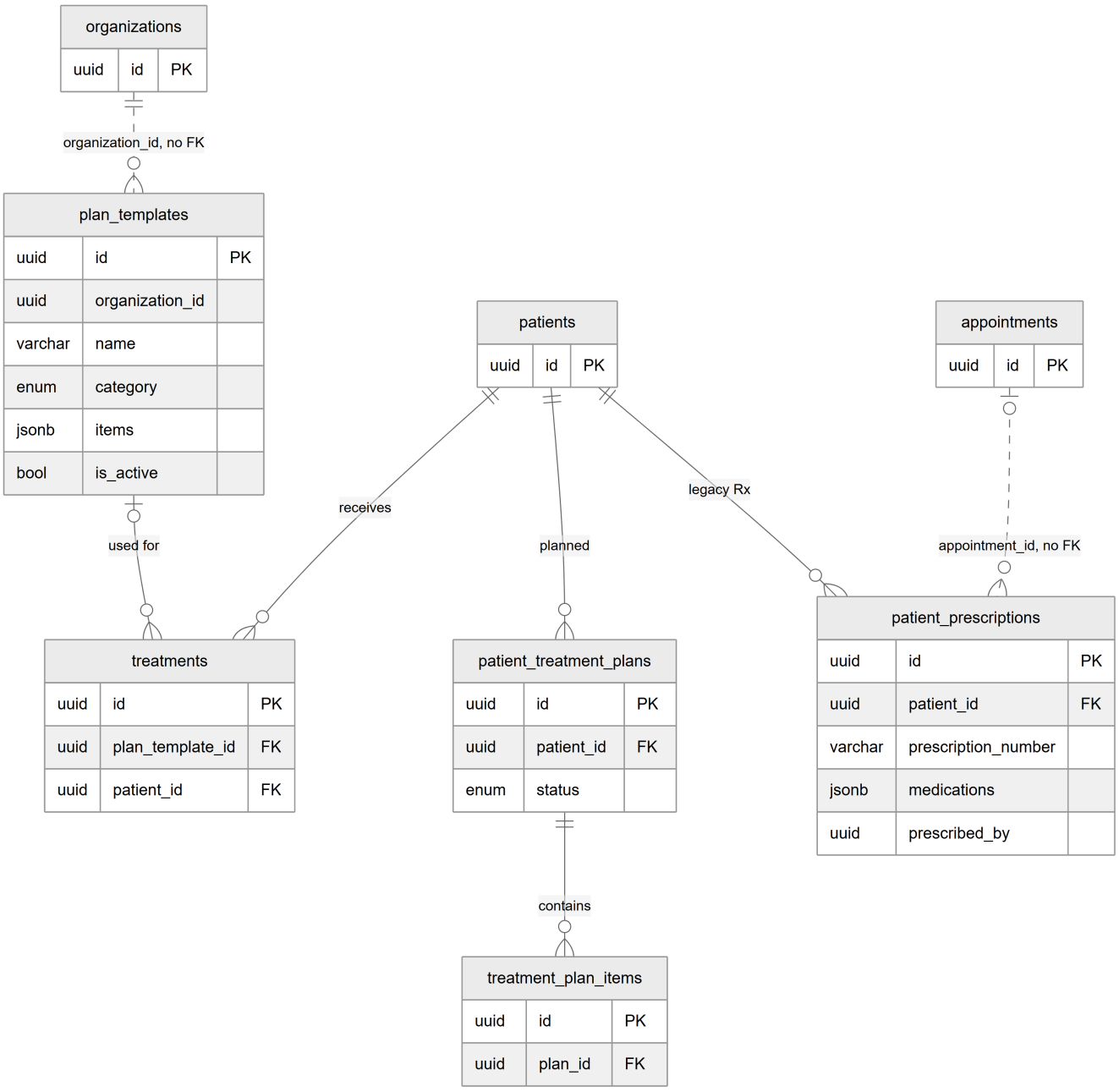

Figure 19.3 — Plan templates and the legacy prescription table.

The third diagram groups the remaining tables. `organizations` is joined to `plan_templates` (`id` , `organization_id` , `name` , `category` , `items` , `is_active`) by a dashed line labelled "organization_id, no FK". `plan_templates` has an optional solid line labelled "used for" to `treatments` . `patients` has solid lines to `treatments` ("receives"), to `patient_treatment_plans` ("planned") and to `patient_prescriptions` ("legacy Rx", showing `prescription_number` , `medications` and `prescribed_by`). `patient_treatment_plans` has a solid line labelled "contains" to `treatment_plan_items` . `appointments` is joined to `patient_prescriptions` by a dashed line labelled "appointment_id, no FK".

19.5 Relationships explained

Foreign-key relationships:

- **`prescriptions.patient_id`** → **`patients`.** Each prescription has exactly one patient; deleting the patient deletes the prescriptions (CASCADE).
- **`prescriptions.organization_id`** → **`organizations`** and **`location_id`** → **`locations`.** Mandatory, CASCADE.

- `prescriptions.appointment_id` → `appointments`, `encounter_id` → `patient_encounters`, `doctor_id` → `employees`, `prescription_template_id` → `prescription_templates`, `treatment_plan_template_id` → `treatment_plan_templates`. All optional (zero or one); deleting the referenced row keeps the prescription and sets the link to null (SET NULL).
- `prescription_medicines.prescription_id` → `prescriptions`. Each line belongs to exactly one prescription; deleting the prescription deletes its lines (CASCADE).
- `prescription_medicines.medicine_directory_id` → `medicine_directory`. Optional; deleting the directory entry keeps the line with the link set to null (SET NULL). The copied name keeps the line readable.
- `prescription_history.prescription_id` → `prescriptions`. Mandatory; deleting the prescription deletes its history (CASCADE).
- `medicine_directory.organization_id`, `treatment_plan_templates.organization_id`, `doctor_signatures.organization_id` → `organizations`. Mandatory, CASCADE.
- `treatment_plan_template_medicines.template_id` → `treatment_plan_templates`. Mandatory, CASCADE.
- `treatment_plan_template_medicines.medicine_directory_id` → `medicine_directory`. Optional, SET NULL.
- `prescription_templates.organization_id` → `organizations` (mandatory, CASCADE) and `location_id` → `locations` (optional; deleting the location deletes its layouts, CASCADE).
- `treatments.plan_template_id` → `plan_templates`. Optional, SET NULL.
- `patient_prescriptions.patient_id` → `patients`, `organization_id` → `organizations`, `location_id` → `locations`. Mandatory, CASCADE.

Logical relationships (no foreign key):

- `doctor_signatures.employee_id` → `users` (holds the user id despite the column name).
- `plan_templates.organization_id` → `organizations`.
- `patient_prescriptions.prescribed_by` → `users` and `appointment_id` → `appointments`.
- **Actor columns:** `prescription_history.changed_by`, `created_by` on `prescription_templates`, `treatment_plan_templates` and `plan_templates`.

19.6 Tenant scope

- `prescriptions`, `prescription_templates` and `patient_prescriptions` are organization + location scoped.
- `medicine_directory`, `treatment_plan_templates`, `doctor_signatures` and `plan_templates` are organization-scoped.
- `prescription_medicines`, `prescription_history` and `treatment_plan_template_medicines` have no tenant column and are isolated through their parent; queries must join it.

19.7 Ownership and ambiguities

- **Two prescription tables.** `prescriptions` is current; `patient_prescriptions` is legacy but still writable from the patient screen. Their numbers come from two different generators, and a prescription can exist in either.
- **Number uniqueness mismatch.** `prescriptions.prescription_number` is unique Platform-wide while numbering restarts in each organization, so the second organization to issue a given number would hit the unique key. The legacy table's number is unique per organization and does not have this problem. The runtime effect has not been tested and is [Unverified].
- **Names that mislead.** `treatment_plan_templates` are prescription presets, not treatment plans; `plan_templates` are treatment plan templates; `prescription_templates` are print layouts.

- **Plan template use differs from its comment.** The `plan_templates` entity comment says using a template copies its procedures into the patient's own plan. In the code, the link that is written is `treatments.plan_template_id`; no screen creates `patient_treatment_plans` rows (Chapter 17).
- **Signature owner.** `doctor_signatures.employee_id` holds a user id, is unique across organizations, and is read without an organization filter.
- **Immutability by convention.** `prescription_history` is declared immutable only in a comment.

Chapter 20 — Billing core

This chapter describes the tables that hold a clinic's money for treatment work: the service catalog, prices, charges, invoices, payments, allocations, refunds, discount types, payment accounts and per-location billing settings. It is written for the product owner who needs to know where every rupee is recorded and which database rules protect it. The design is called "billing v2" in the code; it replaced an older single-table invoice model. The entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) remains the field-level reference; this chapter explains meaning, keys and relationships only.

20.1 The idea in one paragraph

A **charge** is one billable line for a patient, for example one consultation or one filling. It is the atomic unit of billing. An **invoice** is an issued document that groups some of the patient's charges; there are no draft invoices. A **payment** is money received from the patient. It belongs to the patient, not to an invoice. An **allocation** says how much of a payment went to which charge. Money that has been received but not allocated is the patient's **advance**. A **refund** always gives back advance money, and refund sources record which payments funded it. Prices, discounts and the service type are copied onto each charge when it is created, so later catalog edits never change old bills.

Money convention. Every amount in this chapter is a `numeric(12,2)` column in rupees (major units), not paisa. The commission snapshot columns on `charges` are the exception: they use basis points and paisa, because they feed the commission ledger (the Compensation chapter, Chapter 22, explains it).

Key generation. All tables in this chapter use random UUID (version 4) primary keys generated by the database, except `location_billing_settings` , whose key is the location id supplied by the application.

Status: [Implemented]. All tables in this chapter are written by the billing module (`backend/src/billing/`) and shown on billing screens (billing desk, invoices, payments, billing settings).

20.2 Catalog and pricing tables

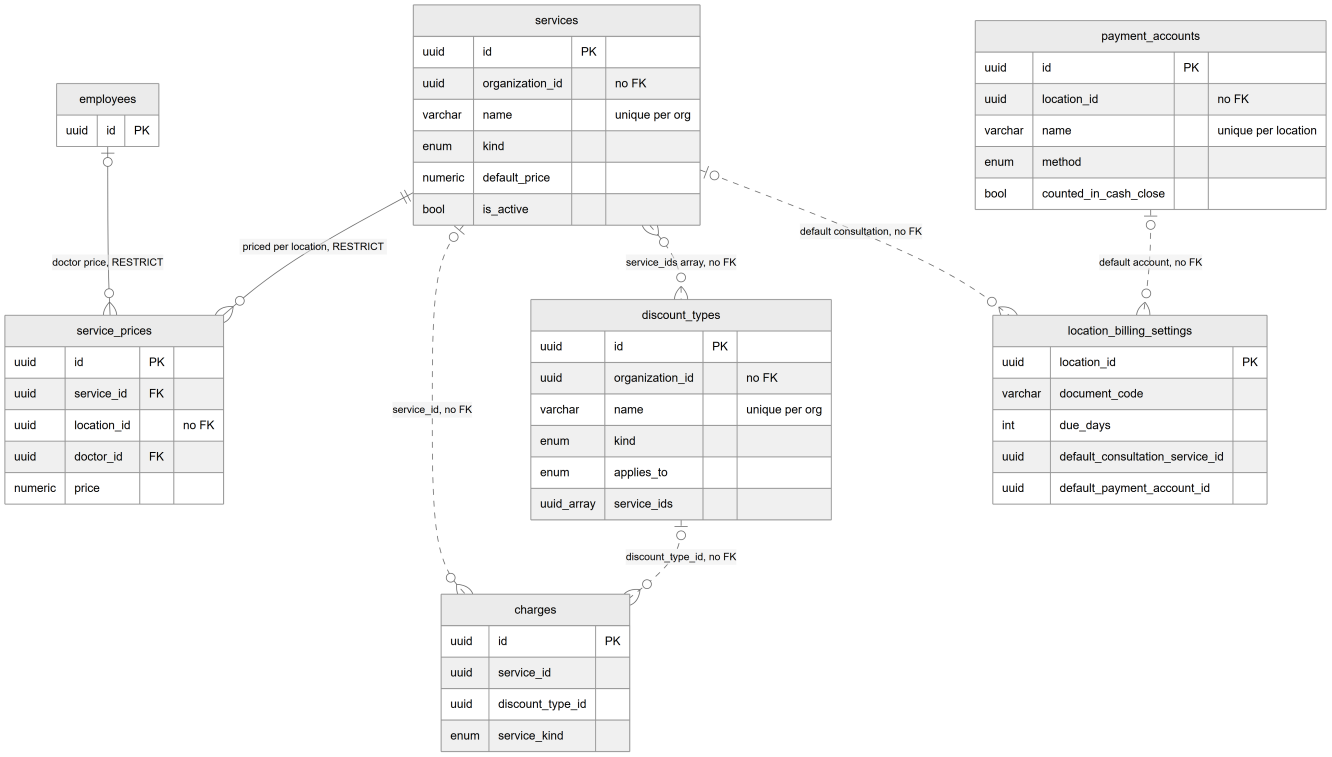

Figure 20.1 — The service catalog, location prices, discount types, payment accounts and location billing settings.

The figure shows seven boxes. `services` is at the centre. A solid line runs from `services` to `service_prices` labelled "priced per location, RESTRICT", and a second solid line runs from `employees` to `service_prices` labelled "doctor price, RESTRICT". Dotted lines, which mean the column has no foreign key, run from `services` to `charges` (service_id), from `discount_types` to `charges` (discount_type_id), from `services` to `discount_types` (the `service_ids` array, a many-to-many link), and from `services` and `payment_accounts` to `location_billing_settings` (the default consultation service and default payment account). Each box lists its primary key and a few key columns; the notes "unique per org" and "unique per location" mark business-unique names, and "no FK" marks tenant columns without a database constraint.

`services` — the service catalog

Code class name: `CatalogService` (the table name and class name differ). One row is something the clinic sells: a consultation, a procedure or "other". It belongs to an organization, not a location. Key columns: `name`, optional `code`, `kind` (CONSULTATION, PROCEDURE, OTHER), `default_price`, `is_active` (soft delete) and `sort_order`.

Constraints that matter:

- The name is unique per organization (`uq_services_org_name`).
- The code is unique per organization when present (`uq_services_org_code`, a partial unique index that ignores rows with no code).

`service_prices` — location and doctor price overrides

One row overrides a service's default price at one location, and optionally for one doctor at that location. Doctors are identified by their employee record, the same way queue sessions identify them.

Constraints that matter:

- At most one location-wide price per service and location (`uq_service_prices_loc` , partial: only rows with no doctor).
- At most one doctor-specific price per service, location and doctor (`uq_service_prices_loc_doc` , partial: only rows with a doctor).

`discount_types` — discount presets

An owner-defined discount, such as "Staff family 20%" or "Senior citizen PKR 500 off". It replaced the older coupon idea. Key columns: `kind` (PERCENTAGE or FIXED), `value` , an optional cap `max_amount` , and `applies_to` (ALL, SERVICE_KINDS or SERVICES) with the matching `service_kinds` or `service_ids` arrays. `reception_editable` , `min_value` and `max_value` let the front desk adjust the value within a range. `requires_note` forces a written reason. `valid_from` and `valid_to` limit the period. `is_active` is the soft-delete flag.

Constraints that matter: the name is unique per organization, and the code is unique per organization when present.

`payment_accounts` — ways a location receives money

One row is one way a location can take money: cash drawer, a bank account, a mobile wallet. Key columns: `name` , `method` (CASH, CARD, BANK_TRANSFER, CHEQUE, ONLINE_WALLET), optional bank details, `requires_reference` (the cashier must type a transaction reference), `counted_in_cash_close` (the account is counted at the end-of-day cash close), `is_active` and `sort_order` .

Constraint that matters: the name is unique per organization and location (`uq_payment_accounts_loc_name`).

`location_billing_settings` — one settings row per location

The primary key is the location id itself, so each location has at most one row. The row is created with defaults the first time it is needed. Key columns:

- `document_code` : the 2–6 character location code printed inside invoice, receipt and refund numbers. It lives here, not on `locations.code` , because the location code is free-form and up to 50 characters.
- `due_days` (default 7): days from issue to due date.
- `invoice_print_format` and `receipt_print_format` (A4 or THERMAL_80).
- Display switches: `show_logo` , `show_previous_balance` , `show_outstanding_at_check_in` , and a `footer_note` .
- `default_consultation_service_id` and `default_payment_account_id` .
- `cash_variance_alert_amount` : the cash-close difference that triggers an alert.

20.3 Charges, invoices, payments and refunds

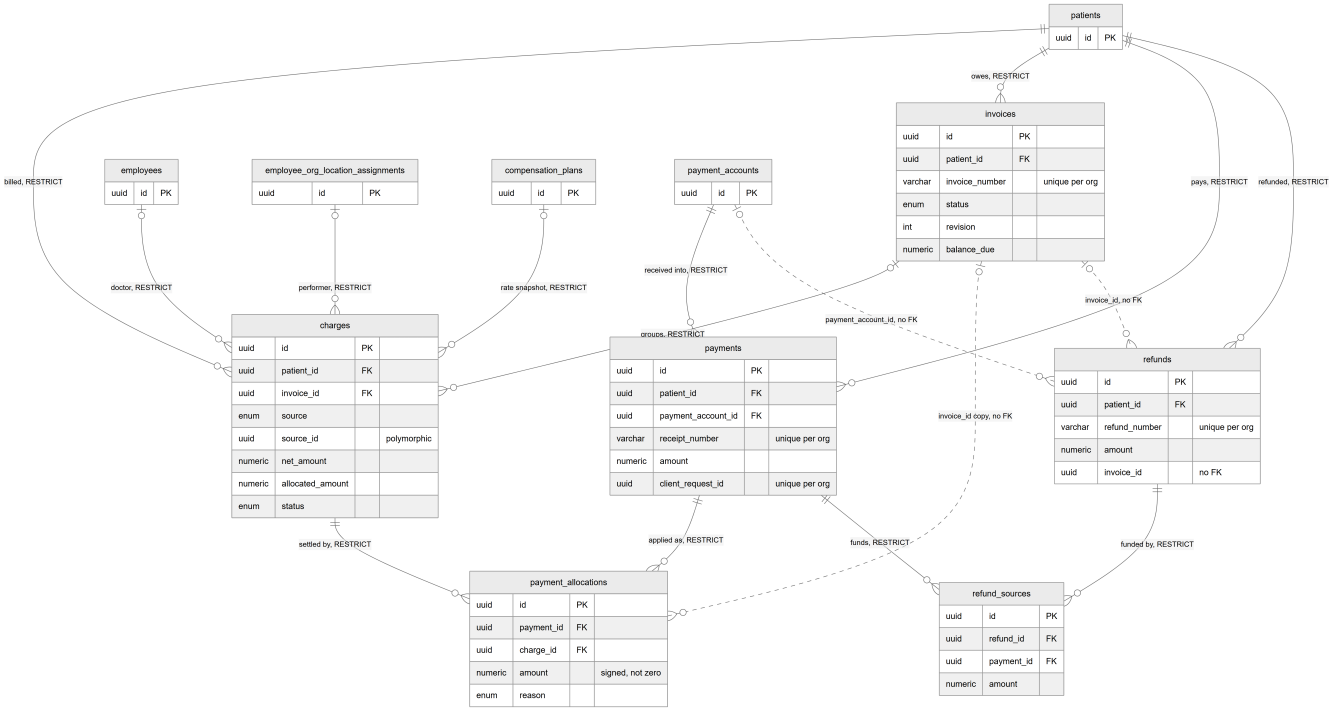

Figure 20.2 — How charges, invoices, payments, allocations, refunds and refund sources connect.

The figure shows `patients` at the top with four solid lines down to `charges` ("billed"), `invoices` ("owes"), `payments` ("pays") and `refunds` ("refunded"), all marked RESTRICT. `invoices` connects to `charges` with "groups, RESTRICT". Three small boxes on the left, `employees`, `employee_org_location_assignments` and `compensation_plans`, each connect to `charges` with "doctor", "performer" and "rate snapshot" (all RESTRICT). `payment_accounts` connects to `payments` with "received into, RESTRICT". At the bottom, `payment_allocations` hangs from both `payments` ("applied as") and `charges` ("settled by"), and `refund_sources` hangs from both `refunds` ("funded by") and `payments` ("funds"). Dotted lines show the logical links without foreign keys: `invoices` to `payment_allocations` (a copied invoice id), `invoices` to `refunds` (invoice_id) and `payment_accounts` to `refunds` (payment_account_id).

`charges` — billable lines

One row is one billable line for one patient at one location. Charges come from four sources (`source`): CONSULTATION, TREATMENT, TREATMENT_PLAN and MANUAL; `source_id` points at the originating record. Key columns:

- Pricing snapshot: `service_id`, `service_kind`, `description`, `quantity`, `unit_price`, `gross_amount`.
- Discount snapshot: `discount_type_id`, `discount_name`, `discount_kind`, `discount_value`, `discount_amount`, `discount_note`, `is_custom_discount`.
- Results: `net_amount`, `tax_amount` (this charge's share of its invoice's sales tax; zero while unbilled) and `allocated_amount` (a cached sum of allocations).
- Billing state: `invoice_id` (null means the charge is not yet invoiced), `status` (ACTIVE or VOID), void details, `needs_pricing` and `source_removed_at` (set when the doctor removed the source treatment after billing).
- Commission snapshot: `doctor_id`, `performing_assignment_id`, `commission_plan_id`, `commission_rate_bps`, `commission_fixed_paisa`.
- Links to the visit: `encounter_id` and `appointment_id`.

The balance of a charge is never stored. The code computes it as net + tax − allocated.

Constraints that matter:

- `uq_charges_source` (partial unique index): an active charge can exist only once per organization, source and source id. This stops the same consultation or treatment being charged twice. Voided charges are excluded, so a corrected charge can be created after a void.
- `chk_charges_commission_rate`: the rate is null or between 0 and 10,000 basis points (0–100%).

`invoices` — issued documents

One row is one issued invoice for one patient at one location. There is no draft state: an invoice is created and issued in one step. Key columns: `invoice_number`, `status` (ISSUED or CANCELLED), `revision` (an edit after issue increases it), `issue_date`, `due_date`, and the cached money columns `subtotal`, `discount_total`, `tax_rate` (copied from the location's settings at issue), `tax_amount`, `total_amount`, `paid_amount`, `balance_due`, `released_amount` and `refunded_amount`. Only the billing totals service writes the cached columns.

Constraint that matters: the invoice number is unique per organization (`uq_invoices_org_number`). Numbers are produced by the document numbering table described in the Billing ledgers chapter (Chapter 21).

`payments` — money received

One row is money received from one patient. It belongs to the patient, not to an invoice. Apart from the cached `allocated_amount` and `refunded_amount`, a payment row never changes. The patient's advance from a payment is amount − allocated − refunded. Key columns: `receipt_number`, `amount`, `payment_account_id`, snapshots of the account's `method` and name, `reference_number`, `payment_date`, `received_at`, `received_by` and `client_request_id`.

Constraints that matter:

- The receipt number is unique per organization.
- `client_request_id` is unique per organization (`uq_payments_org_client_request`). The web app sends a fresh id with each "collect payment" action, so a double click or a network retry cannot record the same payment twice.

`payment_allocations` — which payment paid which charge

One row says "this much of payment P was applied to charge C" (positive amount) or "this much was released back to advance" (negative amount). The table is designed as append-only: rows are never updated or deleted, and corrections are new rows with the opposite sign. The `reason` column records why: PAYMENT, ADVANCE_APPLIED, INVOICE_EDIT_RELEASE or UNAPPLY. `invoice_id` is a copy of the charge's invoice at the time of writing, kept for fast invoice queries.

Constraint that matters: `chk_payment_allocations_amount_nonzero` forbids zero-amount rows. The append-only rule itself is enforced by the application, not by a database trigger.

`refunds` — money returned

One row is money returned to a patient. A refund always comes out of advance money. If `invoice_id` is set, it is an invoice refund; if it is null, it is a plain advance refund. Key columns: `refund_number`, `amount`, `payment_account_id` with method and name snapshots, `reason` (required), `refund_date`, `refunded_at`, `refunded_by` and `client_request_id`.

Constraints that matter: the refund number and the client request id are each unique per organization.

`refund_sources` — which payments funded a refund

One row says how much of one payment's advance was used for one refund. The rows of one refund add up to the refund amount; the application enforces this sum.

20.4 Relationships explained

Foreign-key relationships (the database enforces these):

- **Service → service prices.** Each service price belongs to exactly one service; a service has zero or more prices. A service with prices cannot be deleted (ON DELETE RESTRICT).
- **Doctor → service prices.** A service price may name one doctor (an employee); a doctor can have many prices. Deleting an employee who has prices is blocked (RESTRICT).
- **Patient → charges, invoices, payments, refunds.** Each of these rows belongs to exactly one patient, and a patient can have many of each. A patient with any billing history cannot be deleted (RESTRICT on all four).
- **Invoice → charges.** A charge belongs to zero or one invoice; an invoice groups one or more charges. An invoice that still has charges cannot be deleted (RESTRICT).
- **Doctor → charges.** A charge may name the doctor (employee) who did the work; deleting that employee is blocked (RESTRICT).
- **Assignment → charges.** A charge may name the performing assignment, which is the doctor's link to the organization and location; deleting that assignment is blocked (RESTRICT).
- **Compensation plan → charges.** A charge may record the compensation plan whose rate was copied onto it; that plan cannot be deleted (RESTRICT).
- **Payment account → payments.** Each payment was received into exactly one payment account; an account with payments cannot be deleted (RESTRICT). This is why accounts are switched off with `is_active` instead.
- **Payment → allocations** and **charge → allocations.** Each allocation row belongs to exactly one payment and exactly one charge. A payment or charge with allocations cannot be deleted (RESTRICT).
- **Refund → refund sources** and **payment → refund sources.** Each refund source belongs to exactly one refund and exactly one payment; neither parent can be deleted while sources exist (RESTRICT).

Logical relationships (an id column with no foreign key; the database accepts ids that do not exist):

- `charges.service_id` → `services`; `charges.discount_type_id` → `discount_types`; `charges.encounter_id` → `patient_encounters`; `charges.appointment_id` → `appointments`. Each charge refers to at most one of each.
- `charges.source_id` is polymorphic: its meaning depends on `source` (for example a treatment or an encounter).
- `discount_types.service_ids` is an array of service ids, a many-to-many link with no constraint.
- `location_billing_settings.default_consultation_service_id` → `services` and `default_payment_account_id` → `payment_accounts`.
- `payment_allocations.invoice_id` and `payment_allocations.patient_id` are copies for fast reads.
- `refunds.invoice_id` → `invoices` and `refunds.payment_account_id` → `payment_accounts`. Note that, unlike payments, refunds have no foreign key to the payment account.
- Actor columns (`created_by`, `voided_by`, `discount_applied_by`, `issued_by`, `last_edited_by`, `cancelled_by`, `received_by`, `refunded_by`) hold a user id with no foreign key.

20.5 Tenant scope

- **Organization and location:** `charges`, `invoices`, `payments`, `payment_allocations`, `refunds`, `service_prices`, `payment_accounts`, `location_billing_settings`.
- **Organization only:** `services`, `discount_types` (both shared by every location), and `refund_sources`.
- **No foreign key on the tenant columns.** In every table of this chapter, `organization_id` and `location_id` are plain columns without a foreign key to `organizations` or `locations` (the one exception is that `location_billing_settings.location_id` is the primary key). Tenant isolation therefore depends entirely on the application always filtering by the active organization and location. The Overlaps chapter (Chapter 27) discusses this choice.

20.6 What an owner should remember

- Deleting a patient who has any billing history is impossible by design. Patients are deactivated instead.
- A charge's price and discount never change when the catalog changes, because they are copied at creation.
- The patient's balance and advance are always derived from charges, allocations and refunds; the cached totals are recomputed by one service.
- Duplicate protection exists at three levels: one active charge per source, unique document numbers per organization, and one payment or refund per client request id.

Chapter 21 — Billing ledgers and closing

This chapter describes the record-keeping tables that sit around the core billing tables: the patient statement, the double-entry general ledger, the end-of-day cash close, document numbering, the billing audit log, clinic expenses and prepaid credit packages. It is for the product owner who needs to know which tables are the books of the clinic, which are history, and which features are only partly built. The core billing tables (charges, invoices, payments, allocations, refunds) are covered in the Billing core chapter (Chapter 20). The entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) remains the field-level reference.

21.1 Two different "ledgers"

The Platform has two tables with "ledger" in their names, and they are not the same thing:

- **`ledger_entries` is the patient statement.** It is one-sided, in rupees (`numeric(12,2)`), and has one line per invoice, adjustment, cancellation, payment or refund for a patient. It answers "what does this patient owe us?".
- **`ledger_journals` with `ledger_journal_lines` is the general ledger.** It is true double-entry bookkeeping in paisa (`bigint`), with debits and credits on named accounts. It answers "what are the clinic's balances by account?".

Both are described in this chapter. The Overlaps chapter (Chapter 27) explains why both exist.

21.2 Patient statement, cash close, numbering and audit

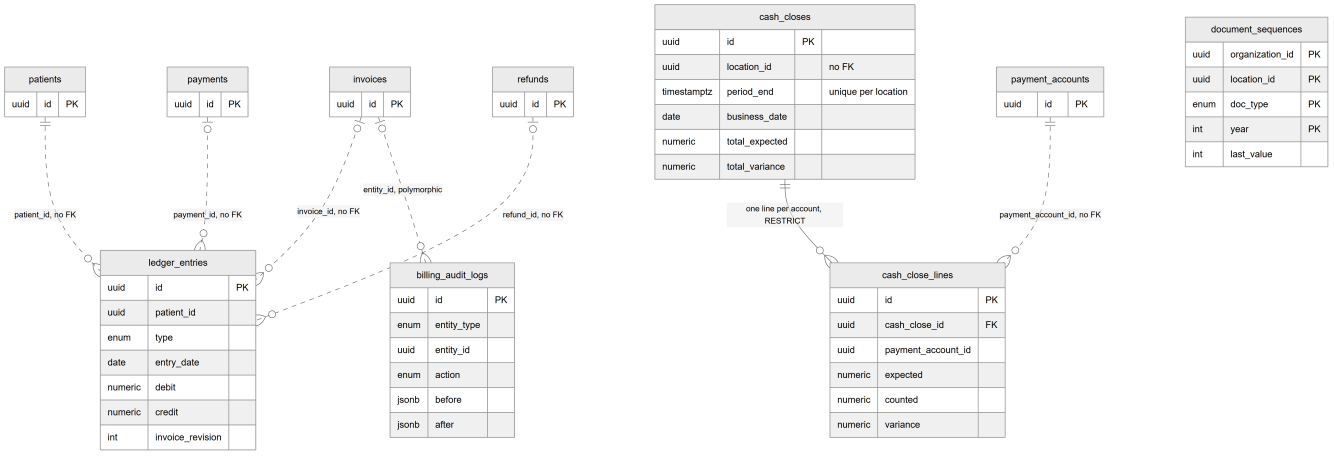

Figure 21.1 — The patient statement, the cash close with its lines, the document counter and the billing audit log.

The figure shows `ledger_entries` connected by dotted lines (no foreign key) to `patients` (patient_id), `invoices` (invoice_id), `payments` (payment_id) and `refunds` (refund_id). `cash_closes` connects with a solid line to `cash_close_lines` , labelled "one line per account, RESTRICT", and `payment_accounts` connects to `cash_close_lines` with a dotted line (payment_account_id, no FK). `billing_audit_logs` receives a dotted line from `invoices` labelled "entity_id, polymorphic", which stands for all the record types the log can point at. `document_sequences` stands alone: its four-part primary key (organization, location, document type, year) and a `last_value` counter are shown, and it has no links.

`ledger_entries` — the patient statement

Status: [Implemented]. Written by the billing ledger service; read by the patient statement endpoints and the printed statement.

One row is one line on a patient's statement at one location. `type` is INVOICE_ISSUED, INVOICE_ADJUSTED, INVOICE_CANCELLED, PAYMENT or REFUND. Exactly one of `debit` and `credit` is above zero. A positive running balance means the patient owes money; a negative one means the clinic holds the patient's advance. `invoice_revision` records which invoice revision an adjustment line belongs to. The table is designed as append-only, but no database trigger enforces it; the application never updates or deletes rows.

`cash_closes` and `cash_close_lines` — end-of-day count

Status: [Implemented]. Written from the cash close sheet on the payments page.

A cash close is an immutable count of a location's counter since the previous close. `cash_closes` holds the period (`period_start` , `period_end`), the clinic `business_date` , the totals `total_expected` , `total_counted` and `total_variance` , an optional `note` and who closed it. `cash_close_lines` holds one line per payment account in that close, with that account's payment and refund totals and counts, `expected` , `counted` and `variance` . The account's name is copied onto the line, so renaming an account later does not change old closes.

Constraint that matters: `uq_cash_closes_loc_end` makes `period_end` unique per organization and location, so two closes cannot end at the same moment.

`document_sequences` — invoice, receipt and refund numbers

Status: [Implemented].

One row is a counter for one organization, location, document type (INVOICE, RECEIPT or REFUND) and year. The primary key is those four columns, and there is no generated id: the application supplies all four values. The numbering service increments `last_value` with a single atomic statement inside the caller's transaction, so two cashiers can never get the same number. The resulting number has the form `<location code>-<INV|REC|RFD>-<year>-<five-digit counter>` , where the location code is `document_code` from `location_billing_settings` and the year is the clinic's year in the location's timezone. Because the counter is per location and per year, numbering restarts at 1 each year at each location. If a location has no document code yet, numbering refuses with the error `BILLING_LOCATION_CODE_MISSING` ("Set a document code for this location in billing settings first").

`billing_audit_logs` — billing's own audit trail

Status: [Implemented]. Read by the billing activity screen.

One row records one billing state change. `entity_type` names what changed (CHARGE, INVOICE, PAYMENT, REFUND, DISCOUNT_TYPE, SERVICE, SERVICE_PRICE, SETTINGS, PAYMENT_ACCOUNT or CASH_CLOSE), `entity_id` points at that record, and `action` names the change (for example CREATED, VOIDED, DISCOUNT_APPLIED, ISSUED, EDITED, CANCELLED, ALLOCATED, REFUNDED, CLOSED). `before` and `after` hold JSON snapshots; for an edited invoice they hold the full snapshot used for invoice revisions. `amount` records the money moved, for activity totals, and `actor_role` records the role of the person who did it. `location_id` is null only for organization-level records such as services and discount types.

The table is described as append-only, but no database trigger protects it. The Platform also has a generic `audit_logs` table (the Workflow engine and common tables chapter, Chapter 26), which billing does not use for these events.

21.3 The general ledger

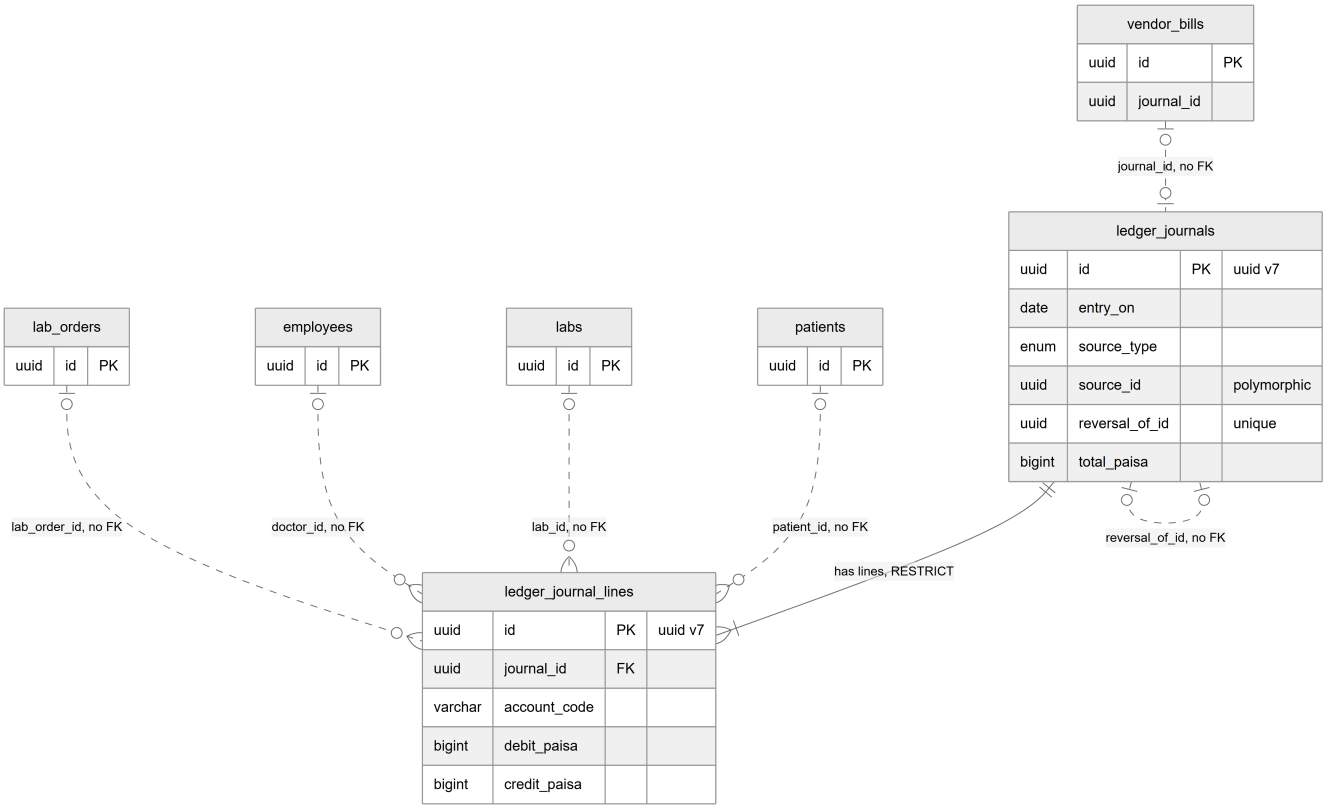

Figure 21.2 — Journals, journal lines and their logical links.

The figure shows `ledger_journals` connected to `ledger_journal_lines` by a solid line labelled "has lines, RESTRICT"; the crow's foot shows that a journal has one or more lines. A dotted loop from `ledger_journals` back to itself is labelled "reversal_of_id, no FK", meaning a journal can be the reversal of one earlier journal. `vendor_bills` points to `ledger_journals` with a dotted line (journal_id, no FK). Four dotted lines run into `ledger_journal_lines` from `patients`, `labs`, `employees` and `lab_orders`; these are optional tags on a line. Both ledger boxes note that their keys are UUID version 7.

`ledger_journals` and `ledger_journal_lines`

Status: [Partial]. The tables and the posting service exist, but in the code only lab money posts journals: lab patient invoices, lab payments, lab credit notes, lab refunds, commission on lab invoice lines, vendor bills and vendor payments. Clinic (non-lab) invoices and payments from Chapter 20 do not post journals, the `expense` source type is never posted, and no screen or report reads the general ledger.

A journal is one balanced double-entry posting. `entry_on` is the accounting date, `source_type` and `source_id` name the business record that caused it (vendor_bill, vendor_payment, expense, invoice, payment, credit_note, refund or commission), `memo` describes it, and `total_paisa` equals the sum of debits, which equals the sum of credits. A mistake is never edited: the application writes a mirror journal whose `reversal_of_id` points at the original.

A journal line is one side of a journal. `account_code` is a code from a fixed chart of accounts kept in the code, not in a table: CASH, BANK, LAB_COST, LAB_PAYABLE, LAB_ADVANCES, PATIENT_RECEIVABLE, PATIENT_ADVANCES, LAB_REVENUE, TAX_PAYABLE, COMMISSION_EXPENSE and COMMISSION_PAYABLE. Each line also carries optional tags: `patient_id`, `lab_id`, `doctor_id` and `lab_order_id`.

Constraints that matter:

- `chk_ledger_journal_lines_side`: each line is either a debit or a credit, never both and never zero.
- `uq_ledger_journals_reversal` (partial unique index): a journal can be reversed at most once.
- The rule "debits equal credits" is checked by the posting service, not by the database.
- No trigger stops updates or deletes; append-only is an application rule.

21.4 Expenses and credit packages

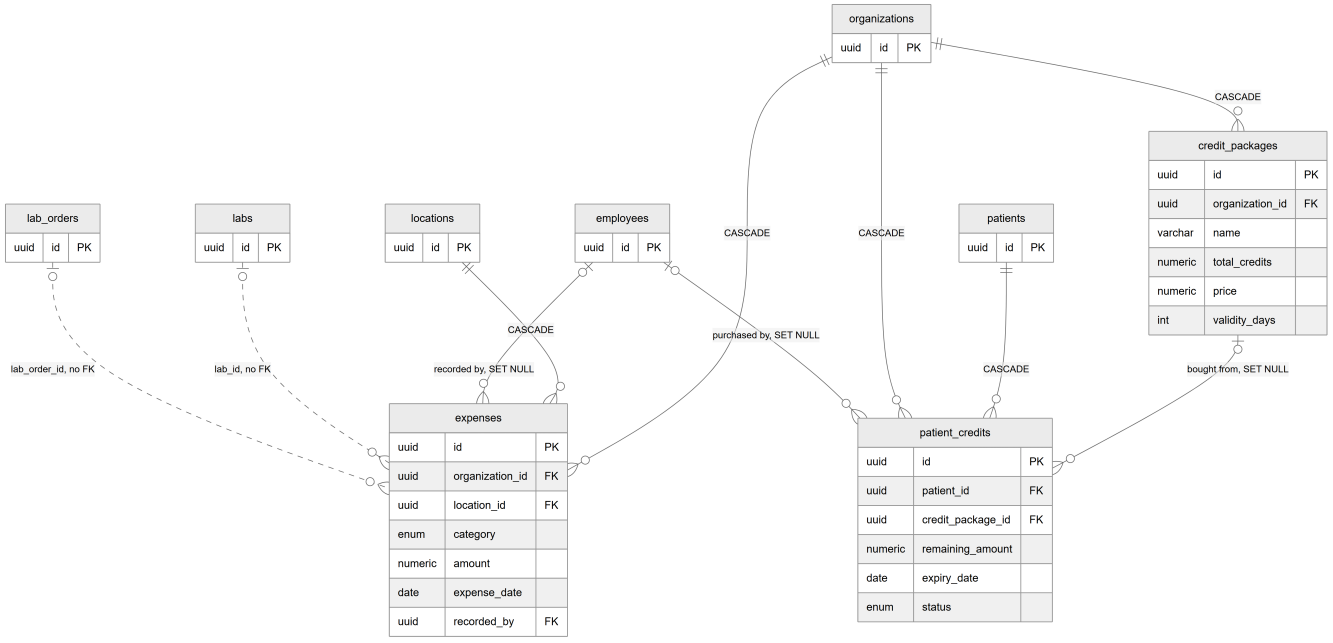

Figure 21.3 — Clinic expenses, credit packages and the credits patients buy.

The figure shows `expenses` with solid lines from `organizations` and `locations` (both CASCADE) and from `employees` ("recorded by, SET NULL"). Dotted lines from `labs` and `lab_orders` reach `expenses` (lab_id and lab_order_id, no FK). On the right, `organizations` connects to `credit_packages` (CASCADE). `credit_packages` connects to `patient_credits` with "bought from, SET NULL". `patients` and `organizations` both connect to `patient_credits` with CASCADE, and `employees` connects with "purchased by, SET NULL".

`expenses` — clinic running costs

Status: [Implemented]. Shown on the expenses page and used by finance reports.

One row is one clinic expense at one location: rent, utilities, supplies, salary, equipment, marketing, maintenance or other (`category`). Key columns: `description` , `amount` (`numeric(10,2)`), `expense_date` , `payment_method` , `reference_number` , `receipt_url` and `recorded_by` . The optional `lab_id` and `lab_order_id` mark an expense as the cost of a lab case. Timestamps in this table are stored without time zone, unlike the rest of billing.

`credit_packages` and `patient_credits` — prepaid credit

Status: [Partial]. The backend has endpoints to manage packages and to record a patient's purchase (feature `billing.credit_packages`), but no web page uses them, and credits are not a payment method at the billing desk.

`credit_packages` is an organization's offer: `name` , `total_credits` , `price` and `validity_days` , with `is_active` as a soft delete. `patient_credits` is one purchase by one patient: `total_amount` , `used_amount` , `remaining_amount` , `purchase_date` , `expiry_date` and `status` (ACTIVE, EXPIRED or EXHAUSTED). Money

columns are `numeric(10,2)` and timestamps have no time zone.

21.5 Relationships explained

Foreign-key relationships:

- **Cash close → cash close lines.** Each line belongs to exactly one cash close; a close has one line per counted account. A close with lines cannot be deleted (RESTRICT).
- **Journal → journal lines.** Each line belongs to exactly one journal; a journal has two or more lines. A journal with lines cannot be deleted (RESTRICT).
- **Organization → expenses** and **location → expenses.** Each expense belongs to one organization and one location. Deleting the organization or the location deletes its expenses (CASCADE).
- **Employee → expenses.** An expense may name the employee who recorded it; deleting that employee keeps the expense and clears the link (SET NULL).
- **Organization → credit packages.** Deleting the organization deletes its packages (CASCADE).
- **Credit package → patient credits.** A patient credit may come from one package; deleting the package keeps the credit and clears the link (SET NULL).
- **Patient → patient credits** and **organization → patient credits.** Deleting the patient or the organization deletes the credits (CASCADE).
- **Employee → patient credits.** The employee who sold the credit; deleting that employee clears the link (SET NULL).

Logical relationships (no foreign key):

- `ledger_entries.patient_id`, `invoice_id`, `payment_id` and `refund_id` point at the patient and at the billing record behind each line.
- `cash_close_lines.payment_account_id` → `payment_accounts`.
- `billing_audit_logs.entity_id` is polymorphic, discriminated by `entity_type`.
- `ledger_journals.source_id` is polymorphic, discriminated by `source_type`; `reversal_of_id` points at another journal.
- `ledger_journal_lines.patient_id`, `lab_id`, `doctor_id` (an employee) and `lab_order_id` are optional tags.
- `vendor_bills.journal_id` → `ledger_journals` (the Labs chapter, Chapter 25, covers vendor bills).
- `expenses.lab_id` → `labs` and `expenses.lab_order_id` → `lab_orders`.
- Actor columns (`created_by`, `closed_by`, `actor_id`) hold a user id with no foreign key.

21.6 Tenant scope

- **Organization and location:** `ledger_entries`, `cash_closes`, `document_sequences`, `billing_audit_logs`, `ledger_journals`, `ledger_journal_lines` (location is optional on the journal tables) and `expenses`.
- **Organization only:** `credit_packages`, `patient_credits`.
- **No tenant column:** `cash_close_lines`, which is scoped only through its cash close. Any query on it must join `cash_closes` to enforce isolation.
- Only `expenses`, `credit_packages` and `patient_credits` have foreign keys on their tenant columns; every other table here stores the organization and location ids without a constraint.

Chapter 22 — Compensation

This chapter describes the four tables that decide how a doctor or other employee is paid: compensation plans, commission rules, the commission ledger and commission settlements. It is for the product owner who needs to understand how commission is recorded, why its amounts can never be edited, and how it reaches payroll. The tables live in the billing module (`backend/src/billing/commissions/`) and connect billing charges, lab invoice lines and payroll records. The entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) remains the field-level reference.

Status: [Implemented]. Plans and rules are edited from the employee screens; commission entries are written automatically when money is collected; settlements are created and paid through payroll; the commission pages show earnings.

22.1 The idea in one paragraph

Pay is set per **assignment**, which is an employee's link to one organization and location, because the same doctor can be paid differently at two branches. A **compensation plan** says how the assignment is paid during a date range: salary, commission, or both. **Commission rules** override the plan's default rate for one service or one service category. Commission is earned only on money actually collected. Every time collection on a charge changes, the Platform writes a **commission entry** with only the difference, positive or negative. When payroll is generated, the unsettled entries of the assignment are grouped into a **settlement**, which is paid through one **payroll record**.

Money convention. All amounts here are `bigint` paisa (integer minor units). Rates are basis points: 4000 means 40.00%.

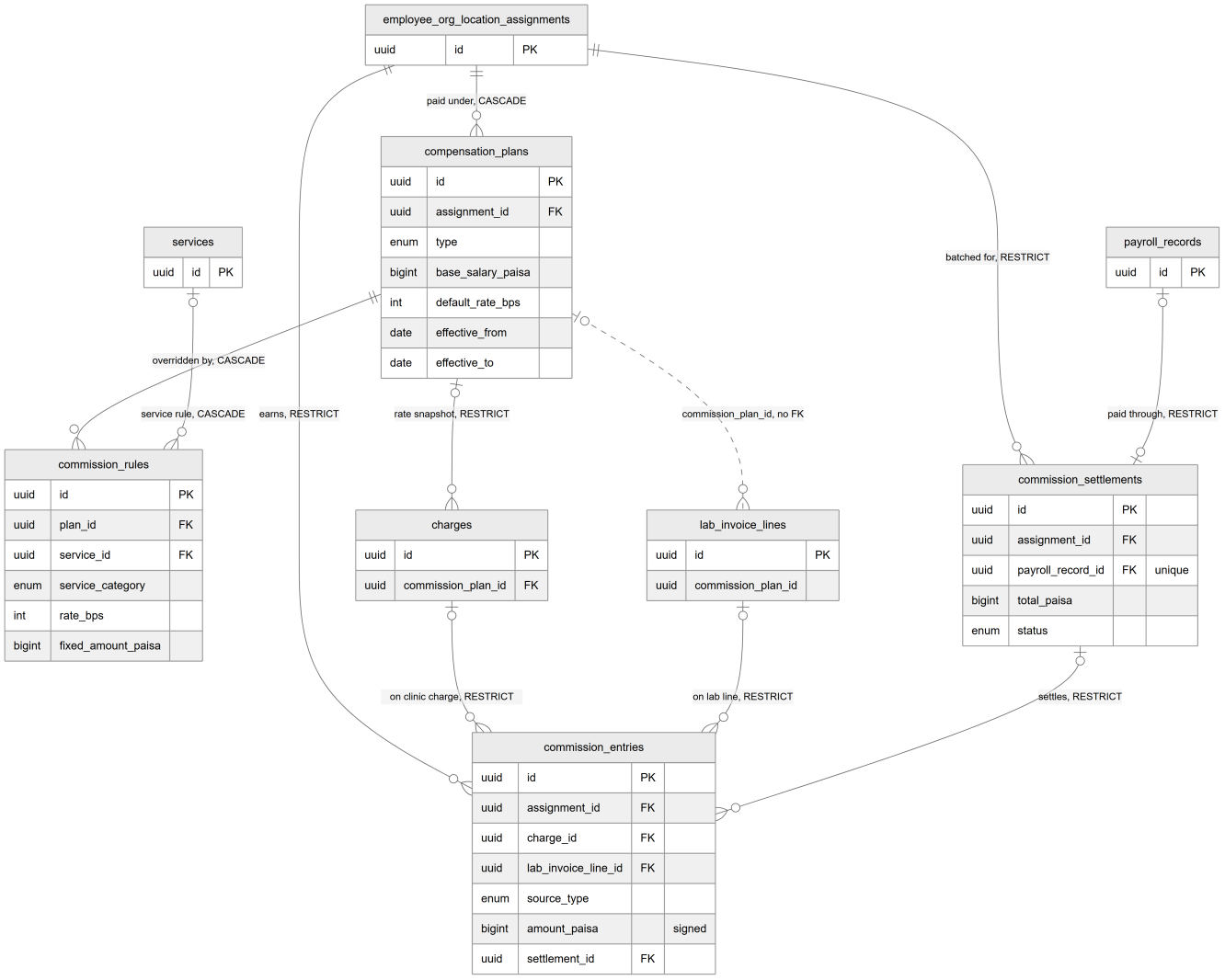

Figure 22.1 — Compensation plans, commission rules, the commission ledger and settlements, with their links to billing and payroll.

The figure shows `employee_org_location_assignments` with three solid lines: to `compensation_plans` ("paid under, CASCADE"), to `commission_entries` ("earns, RESTRICT") and to `commission_settlements` ("batched for, RESTRICT"). `compensation_plans` connects to `commission_rules` ("overridden by, CASCADE") and to `charges` ("rate snapshot, RESTRICT"), and has a dotted line to `lab_invoice_lines` (commission_plan_id, no FK). `services` connects to `commission_rules` ("service rule, CASCADE"). `commission_entries` receives solid lines from `charges` ("on clinic charge"), `lab_invoice_lines` ("on lab line") and `commission_settlements` ("settles"), all RESTRICT. `payroll_records` connects one-to-one with `commission_settlements` ("paid through, RESTRICT"). Each main box shows its key columns, and `amount_paisa` on the entry is marked "signed".

22.2 Tables

`compensation_plans` — how an assignment is paid

One row is one version of an assignment's pay, valid from `effective_from` to `effective_to` (both inclusive; an empty `effective_to` means "current"). `type` is SALARY, COMMISSION or SALARY_PLUS_COMMISSION. `base_salary_paisa` holds the salary, `default_rate_bps` the default commission rate, and `commission_basis` (NET by default, or GROSS) which charge amount the commission rate is applied to. A change of pay closes the current version and adds a new one, so old charges keep the rate that was copied onto them.

Constraints that matter:

- `excl_compensation_plans_overlap` (an EXCLUDE constraint using the `btree_gist` extension): two versions of the same assignment can never have overlapping date ranges.
- `chk_compensation_plans_fields`: a SALARY plan must have a salary and no rate; a COMMISSION plan must have a rate and no salary; SALARY_PLUS_COMMISSION must have both.
- `chk_compensation_plans_dates`, `chk_compensation_plans_salary`, `chk_compensation_plans_rate`: the end date is not before the start, the salary is not negative, and the rate is between 0 and 10,000 basis points.

`commission_rules` — per-service and per-category overrides

One row overrides the plan's default for either one catalog service (`service_id`) or one service category (`service_category`: CONSULTATION, PROCEDURE or OTHER), never both. It holds either a rate (`rate_bps`) or a fixed amount per fully paid charge (`fixed_amount_paisa`), never both. When the Platform looks up the rate for a charge, it tries the service rule first, then the category rule, then the plan default.

Constraints that matter:

- `chk_commission_rules_target` and `chk_commission_rules_amount`: exactly one target and exactly one kind of amount.
- `chk_commission_rules_rate` and `chk_commission_rules_fixed`: valid ranges.
- `uq_commission_rules_service` and `uq_commission_rules_category` (partial unique indexes): at most one rule per plan and service, and at most one per plan and category.

`commission_entries` — the commission ledger

One row is one change in what an assignment has earned on one source line. The source line is either a clinic charge (`charge_id`) or a lab invoice line (`lab_invoice_line_id`), never both. `source_type` says what caused the entry: ALLOCATION (new money), UNAPPLY, CHARGE_ADJUST, VOID or MANUAL; `source_id` is the payment or operation behind it. `base_paisa` is how much had been collected on the line (capped at its net amount) when the entry was written, `rate_bps` the rate used, and `amount_paisa` the signed difference. For each line, the sum of its entries always equals the commission due on what has been collected so far. `settlement_id` is empty until the entry is settled.

Constraints that matter:

- `chk_commission_entries_target`: exactly one of `charge_id` and `lab_invoice_line_id` is set.
- `uq_commission_entries_source` (source type, source id, charge) and `uq_commission_entries_lab_source` (partial, for lab lines): the same operation cannot write commission twice for the same line.
- `idx_commission_entries_unsettled`: a partial index over unsettled entries, used when payroll collects them.

The append-only trigger. This table is protected in the database by the trigger `trg_commission_entries_append_only`, installed by the hand-written migration `1792403456699-doctor-compensation`. The trigger:

- refuses every DELETE;
- refuses any UPDATE that changes a column other than `settlement_id`;
- allows `settlement_id` to be set once, from empty to a settlement;
- allows `settlement_id` to be cleared back to empty only while that settlement is still PENDING. Once the settlement is PAID, the entry can no longer move.

It is one of only three tables in the Platform whose append-only rule is enforced by the database itself (the others are `audit_logs` and `status_history`).

`commission_settlements` — a payout batch

One row groups unsettled commission entries of one assignment for a period (`period_start` , `period_end`) and links them to one payroll record. `total_paisa` is the sum of the entries; it is signed, because a period with only reversals can be negative. `status` is PENDING, then PAID, with `paid_at` and `paid_by` .

How it is used in the code:

- When payroll is generated, the unsettled entries created before the cut-off are locked (`SELECT … FOR UPDATE`), so one entry can never land in two settlements. A PENDING settlement is created and the entries are pointed at it.
- When the payroll record is marked paid, the settlement becomes PAID in the same transaction.
- If the payroll record is deleted before payment, the entries are released back to the unsettled pool and the settlement row is deleted. Once PAID, release is refused.

Constraint that matters: `payroll_record_id` is unique, so one payroll record has at most one settlement.

22.3 Relationships explained

- **Assignment → compensation plans.** Each plan belongs to exactly one assignment; an assignment has zero or more plan versions. Deleting an assignment deletes its plans (ON DELETE CASCADE). In practice this cascade is blocked as soon as any charge has copied one of those plans, because charges point at plans with RESTRICT.
- **Compensation plan → commission rules.** Each rule belongs to exactly one plan; deleting the plan deletes its rules (CASCADE).
- **Service → commission rules.** A rule may target one catalog service; deleting the service deletes its rules (CASCADE). Services are normally switched off with `is_active` rather than deleted.
- **Compensation plan → charges.** A charge may record the plan whose rate it copied; that plan cannot be deleted (RESTRICT). The rate and fixed amount themselves are also copied onto the charge.
- **Assignment → commission entries.** Each entry belongs to exactly one assignment; an assignment with entries cannot be deleted (RESTRICT).
- **Charge → commission entries.** An entry may belong to one charge; a charge has zero or more entries. A charge with entries cannot be deleted (RESTRICT).
- **Lab invoice line → commission entries.** An entry may belong to one lab invoice line instead; that line cannot be deleted (RESTRICT). Lab invoice lines are covered in the Labs chapter (Chapter 25).
- **Settlement → commission entries.** An entry belongs to zero or one settlement; a settlement groups many entries. A settlement that still has entries cannot be deleted (RESTRICT); the release step clears the entries first.
- **Assignment → commission settlements.** Each settlement belongs to exactly one assignment (RESTRICT).
- **Payroll record → commission settlement.** One-to-one: a settlement may be paid through one payroll record, and a payroll record has at most one settlement. A payroll record with a settlement cannot be deleted at database level (RESTRICT); the payroll service releases the settlement first, and only while it is PENDING.

Logical relationships (no foreign key):

- `lab_invoice_lines.commission_plan_id` → `compensation_plans` and `lab_invoice_lines.performing_assignment_id` → `employee_org_location_assignments` : the same snapshot as on charges, but without constraints.
- `commission_entries.source_id` is polymorphic, discriminated by `source_type` .
- Actor columns `compensation_plans.created_by` and `commission_settlements.paid_by` hold a user id.

22.4 Tenant scope

- **Organization and location:** `commission_entries`, `commission_settlements`.
- **Organization only:** `compensation_plans` (the location is implied by the assignment).
- **No tenant column:** `commission_rules`, scoped only through its plan. Queries must join `compensation_plans`.
- None of the tenant columns here has a foreign key; isolation is enforced by the application.

22.5 What an owner should remember

- Commission follows cash, not invoices: nothing is earned until money is allocated to a charge or lab line.
- Commission amounts cannot be edited or deleted by any ordinary SQL statement, because the trigger rejects it. Corrections are new signed entries.
- Tax is not part of the commission base. The base is the collected amount capped at the line's net amount.
- A rate change never rewrites history: old charges keep the rate copied onto them.

Chapter 23 — Inventory

This chapter describes the eleven tables of the inventory module: stock items and their categories, stock movements, suppliers, purchase orders, consumption templates and the record of materials used on a patient. It is for the product owner who wants to know how stock is tracked and where the module is incomplete. The module lives in `backend/src/inventory/` and is reached in the web app at `/inventory`. The entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) remains the field-level reference.

23.1 Two things to know first

camelCase column names. Unlike every other module, the inventory tables use camelCase column names in the database, for example `organizationId`, `locationId`, `currentStock` and `treatmentPlanId`. In SQL these names must be written in double quotes. The legacy `inventory` table is the only exception: it uses snake_case.

A note on the catalog. In the entity catalog, the "Notes" column of the inventory tables is shifted one row down: the comment shown next to a column belongs to the column listed above it. For example, the comment "NEVER update directly — always through StockMovement" belongs to `currentStock`, not to `unit`. This chapter was checked against the entity files in `backend/src/inventory/entities/`.

Money convention. Prices and totals are `numeric(10,2)` in rupees. Quantities are `numeric(10,2)`, so fractional quantities such as 1.5 ml are allowed. Timestamps are stored without time zone.

Status overview:

- Stock items, categories, suppliers, purchase orders, stock movements and consumption templates: [Partial]. The backend and the `/inventory` page (tabs for items, suppliers, purchase orders and templates) exist and are permission-gated, but `/inventory` has no sidebar entry, so users reach it only by URL.
- Recording materials used on a patient (`patient_consumptions`): [Partial]. The backend endpoints (`POST /patient-consumptions` and its read endpoints) exist, and the web app has a `ConsumptionRecorder` component, but no page renders that component.
- The legacy `inventory` table: [Not implemented]. It is not registered by any module.

23.2 Stock and purchasing

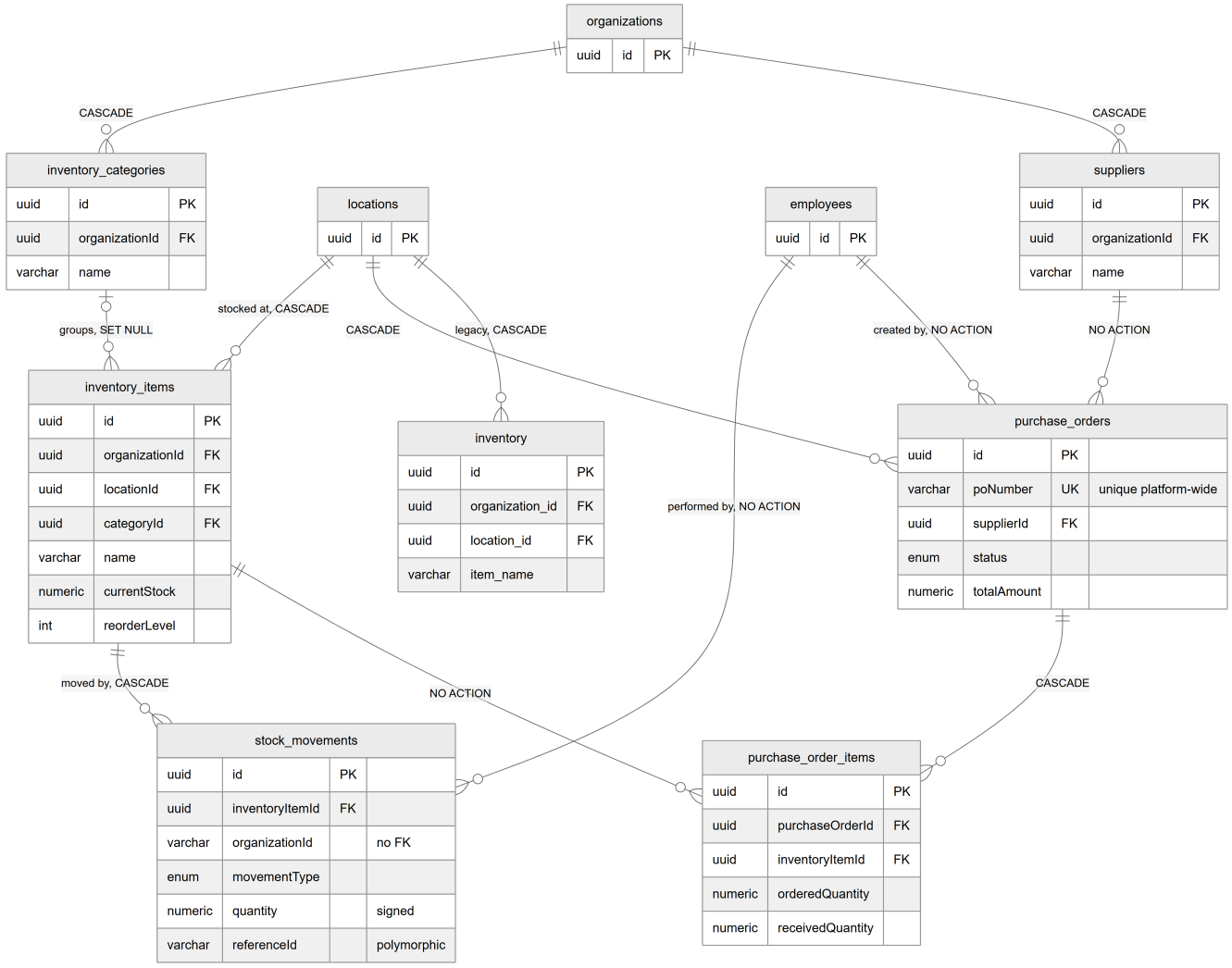

Figure 23.1 — Items, categories, stock movements, suppliers, purchase orders and the legacy inventory table.

The figure shows `organizations` connected to `inventory_categories` and `suppliers` (both CASCADE). `inventory_categories` connects to `inventory_items` with "groups, SET NULL", and `locations` connects to `inventory_items` with "stocked at, CASCADE". `inventory_items` connects to `stock_movements` ("moved by, CASCADE") and to `purchase_order_items` (NO ACTION). `employees` connects to `stock_movements` ("performed by") and to `purchase_orders` ("created by"), both NO ACTION. `suppliers` connects to `purchase_orders` (NO ACTION), `locations` to `purchase_orders` (CASCADE), and `purchase_orders` to `purchase_order_items` (CASCADE). A separate box, `inventory`, hangs from `locations` with "legacy, CASCADE". The `poNumber` column on `purchase_orders` carries the note "unique platform-wide".

`inventory_categories` — item groups

An organization's grouping of items, such as disposables, dental materials, medicines, instruments or office supplies. Key columns: `name`, `description`, `sortOrder`, and `isActive` as a soft delete.

`inventory_items` — one stocked item at one location

One row is one item kept at one location, with its current stock. Key columns:

- Identity: `name`, optional `sku` (an internal stock code) and `barcode` (kept for a future scanning feature).

- Stock: `currentStock` and `unit` (pieces, boxes, pairs, rolls, bottles, packets, cartridges, tubes, ml, gm, kg or liters). The code comment says `currentStock` must never be updated directly, only through a stock movement.
- Price: `unitPrice` is the last purchase cost per unit and is updated when purchase-order items are received; `sellingPrice` is used when the item is billed to a patient.
- Reorder: `reorderLevel` (alert when stock falls to or below it, default 10), `reorderQuantity` (suggested order quantity, default 50) and optional `maxStock`.
- Expiry: `expiryDate` (the nearest expiry among current batches) and `expiryAlertDays` (alert this many days before, default 30).
- Flags: `isBillable` (the item may be added to a patient invoice) and `isActive`.

There is no unique constraint on the SKU. The code comment says SKU uniqueness per organization is enforced in the service, not in the database.

`stock_movements` — the stock history

One row is one change to one item's stock. `movementType` is PURCHASE_IN, MANUAL_ADJUSTMENT, CONSUMPTION, WASTAGE, TRANSFER_IN, TRANSFER_OUT, RETURN_TO_SUPPLIER or INITIAL_STOCK. `quantity` is signed: positive for stock coming in, negative for stock going out. `previousStock` and `newStock` snapshot the item's stock before and after, so `newStock = previousStock + quantity`. `referenceType` names what triggered the movement ('purchase_order', 'appointment', 'treatment_plan', 'manual' or 'transfer') and `referenceId` holds that record's id. `batchNumber` and `expiryDate` describe the batch.

Two oddities: `organizationId` and `locationId` here are `varchar`, not `uuid`, and have no foreign key; `referenceId` is also `varchar`.

`suppliers` — vendors of stock

An organization's supplier: name, contact person, phone, email, address, city, website, notes, and `isActive`.

`purchase_orders` and `purchase_order_items` — buying stock

A purchase order is one order from one supplier for one location. `status` is DRAFT, ORDERED, PARTIALLY_RECEIVED, RECEIVED or CANCELLED. Money columns are `subtotal`, `taxRate`, `taxAmount` and `totalAmount`; dates are `orderDate`, `expectedDeliveryDate` and `receivedDate`. Each purchase-order item is one inventory item on the order, with `orderedQuantity`, `receivedQuantity`, `unitPrice`, `totalPrice`, and optional `batchNumber` and `expiryDate`.

Constraint that matters, and a risk. `poNumber` has a unique constraint on its own, across the whole Platform. The service generates numbers per organization in the form `PO-<year>-<five digits>`, starting from the organization's last order. Two organizations will therefore both try to create `PO-<year>-00001`, and the second insert will fail on the unique constraint. This is recorded as a defect for the owner; the module chapter for inventory (Chapter 39) covers its user impact.

`inventory` — legacy single table

Status: [Not implemented]. A simple item table (`item_name`, `quantity`, `unit_price`, `expiry_date`) for an organization and location. No module registers it, so the running application never creates or uses it in development. It exists only because the command-line schema tools load every entity file, so a production schema built with `schema:sync:prod` may contain it while a development database does not. It looks superseded by `inventory_items` and `stock_movements`.

23.3 Consumption

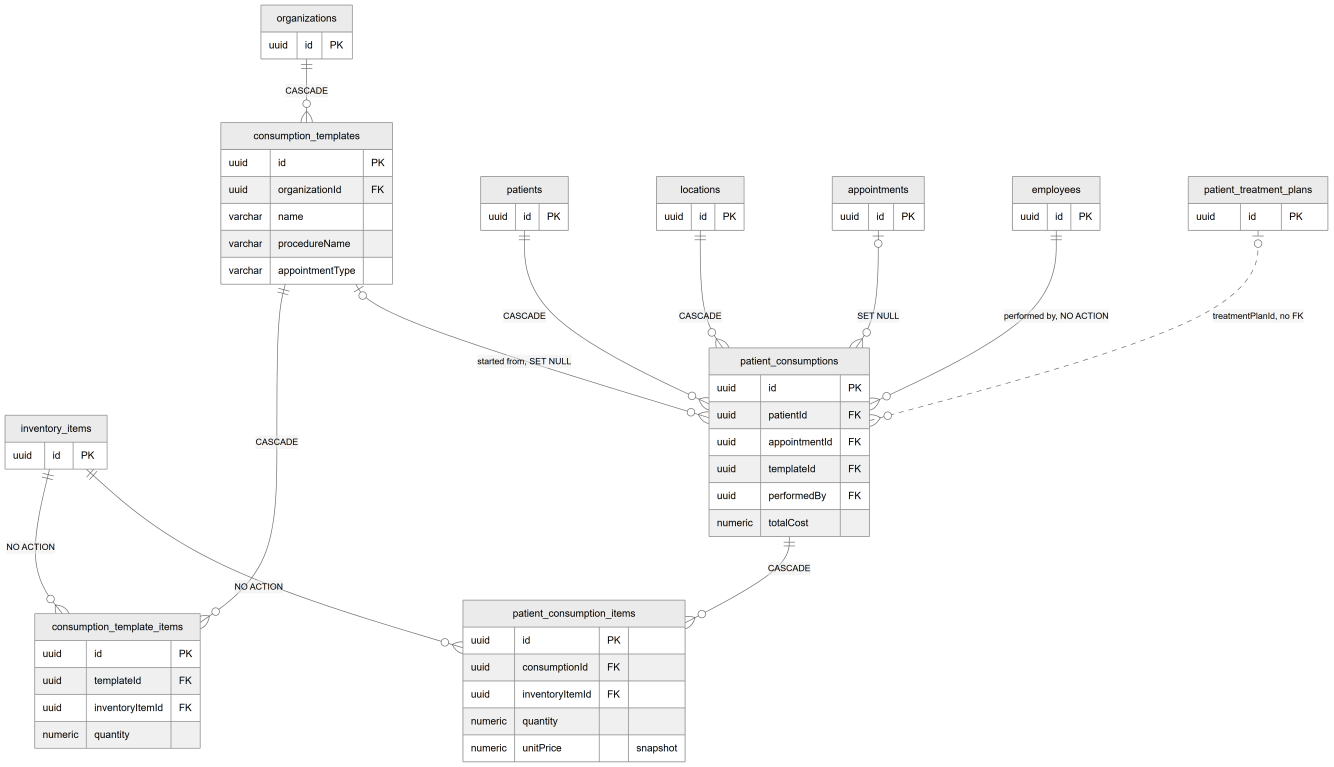

Figure 23.2 — Consumption templates and the record of materials used on a patient.

The figure shows `organizations` connected to `consumption_templates` (CASCADE). `consumption_templates` connects to `consumption_template_items` (CASCADE) and to `patient_consumptions` ("started from, SET NULL"). `inventory_items` connects to both `consumption_template_items` and `patient_consumption_items` with NO ACTION. `patients` and `locations` connect to `patient_consumptions` with CASCADE, `appointments` with SET NULL, and `employees` with "performed by, NO ACTION". `patient_consumptions` connects to `patient_consumption_items` (CASCADE). A dotted line from `patient_treatment_plans` to `patient_consumptions` is labelled "treatmentPlanId, no FK". The `unitPrice` column on the consumption item is marked "snapshot".

`consumption_templates` and `consumption_template_items` — standard kits

A consumption template is an organization's standard kit for a procedure, for example a root canal kit. Key columns: `name`, `procedureName`, optional `appointmentType` (used to suggest the template automatically when that type of visit starts) and `isActive`. Each template item names one inventory item and a default `quantity`, with optional `notes` as hints for the doctor and a `sortOrder`.

`patient_consumptions` and `patient_consumption_items` — what was used on a patient

A patient consumption records the materials used on one patient at one location, optionally for one appointment, and optionally based on a template. `performedBy` is the doctor (an employee). `totalCost` is the sum of the item totals, calculated by the service. Each consumption item records one inventory item, the `quantity` actually used, `unitPrice` copied from the item at the time of use, `totalPrice` (quantity × unit price, calculated by the service) and `wasFromTemplate` (true if the item was suggested by the template, false if added by hand).

`treatmentPlanId` has no foreign key. The code comment says this was done "to avoid circular" imports. The intended target is probably `patient_treatment_plans`, but the code does not say so.

23.4 Relationships explained

Foreign-key relationships:

- **Organization → categories, items, suppliers, purchase orders, consumption templates, patient consumptions.** Each belongs to one organization; deleting the organization deletes them (CASCADE).
- **Location → items, purchase orders, patient consumptions.** Each belongs to one location; deleting the location deletes them (CASCADE).
- **Category → items.** An item belongs to zero or one category; deleting the category keeps the items and clears the link (SET NULL).
- **Item → stock movements.** Each movement belongs to exactly one item; deleting the item deletes its whole stock history (CASCADE).
- **Employee → stock movements, purchase orders, patient consumptions.** Each names the employee who did it. Deleting that employee is blocked while such rows exist (NO ACTION, which behaves like RESTRICT at the end of the statement).
- **Supplier → purchase orders.** Each order belongs to exactly one supplier; a supplier with orders cannot be deleted (NO ACTION). Suppliers are deactivated with `isActive`.
- **Purchase order → purchase-order items.** Deleting an order deletes its items (CASCADE).
- **Item → purchase-order items, template items, consumption items.** Each refers to exactly one inventory item; an item used on any of these cannot be deleted (NO ACTION). Note the asymmetry: deleting an item would cascade its stock movements, but the delete is blocked first if the item appears on any order, template or consumption.
- **Template → template items.** Deleting a template deletes its items (CASCADE).
- **Template → patient consumptions.** A consumption may have started from one template; deleting the template keeps the consumption and clears the link (SET NULL).
- **Patient → patient consumptions.** Deleting the patient deletes the consumption records (CASCADE).
- **Appointment → patient consumptions.** Deleting the appointment keeps the consumption and clears the link (SET NULL).
- **Patient consumption → consumption items.** Deleting a consumption deletes its items (CASCADE).
- **Legacy `inventory`.** Belongs to one location and optionally one organization; both CASCADE.

Logical relationships (no foreign key):

- `patient_consumptions.treatmentPlanId` → probably `patient_treatment_plans` (not verified in the code).
- `stock_movements.referenceId` is polymorphic, discriminated by `referenceType`.
- `stock_movements.organizationId` and `locationId` → `organizations` and `locations`.

23.5 Tenant scope

- **Organization and location:** `inventory_items`, `stock_movements`, `purchase_orders`, `patient_consumptions`, and the legacy `inventory`.
- **Organization only:** `inventory_categories`, `suppliers`, `consumption_templates`.
- **No tenant column:** `purchase_order_items`, `consumption_template_items` and `patient_consumption_items`, scoped only through their parent order, template or consumption. Queries must join the parent to enforce isolation.
- Unlike the billing tables, most inventory tenant columns do have foreign keys. The exception is `stock_movements`, whose tenant columns are unconstrained `varchar`.

Chapter 24 — Notifications

This chapter describes the seven tables of the notification system: the platform catalog of events, each clinic's on/off rules and wording, the outbox of messages to send, the in-app inbox (the bell), and each patient's messaging preferences. It is for the product owner who needs to know what is stored when "something notifies someone" and which channels really work today. The module lives in `backend/src/notifications/`; the sending mechanics are covered in the Notification mechanics chapter (Chapter 59). The entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) remains the field-level reference.

24.1 The idea in one paragraph

The Platform defines a fixed list of **notification events** in code, such as an order being submitted or an item being ready. Each event can reach one or more **audiences**: patient, doctor, coordinator or lab. A clinic decides per event, audience and channel whether to send (**notification rules**), can switch a whole audience off or set quiet hours (**audience settings**), and can replace the default wording (**templates**). When something happens, the Platform writes one **outbox** row per recipient and channel in the same database transaction as the change itself, including rows it decided to skip, so the log can show why a message never went. A background relay then sends the queued rows. An in-app send also creates a row in the recipient's **bell**.

Channels in practice. The tables allow four channels: `in_app`, `whatsapp`, `sms` and `email`. In the code, only `in_app` is switched on for the doctor, coordinator and lab audiences; WhatsApp, SMS and email are commented out because no real provider is connected. The patient audience has no channel at all, so patients currently receive nothing.

Status:

- In-app notifications to staff and lab users (outbox, bell, settings, templates): [Implemented].
- WhatsApp, SMS and email delivery: [Not implemented]. The channel values exist in the tables and checks, but no provider sends them.
- Patient notifications: [Not implemented]. Patient preferences can be stored, but no channel reaches patients.

Keys. Every table here except `notification_events` and `patient_comm_prefs` uses a UUID version 7 primary key generated by the database function `uuid_generate_v7()`. Version 7 ids start with a timestamp, so they sort by creation time (the Labs chapter, Chapter 25, explains them in more detail).

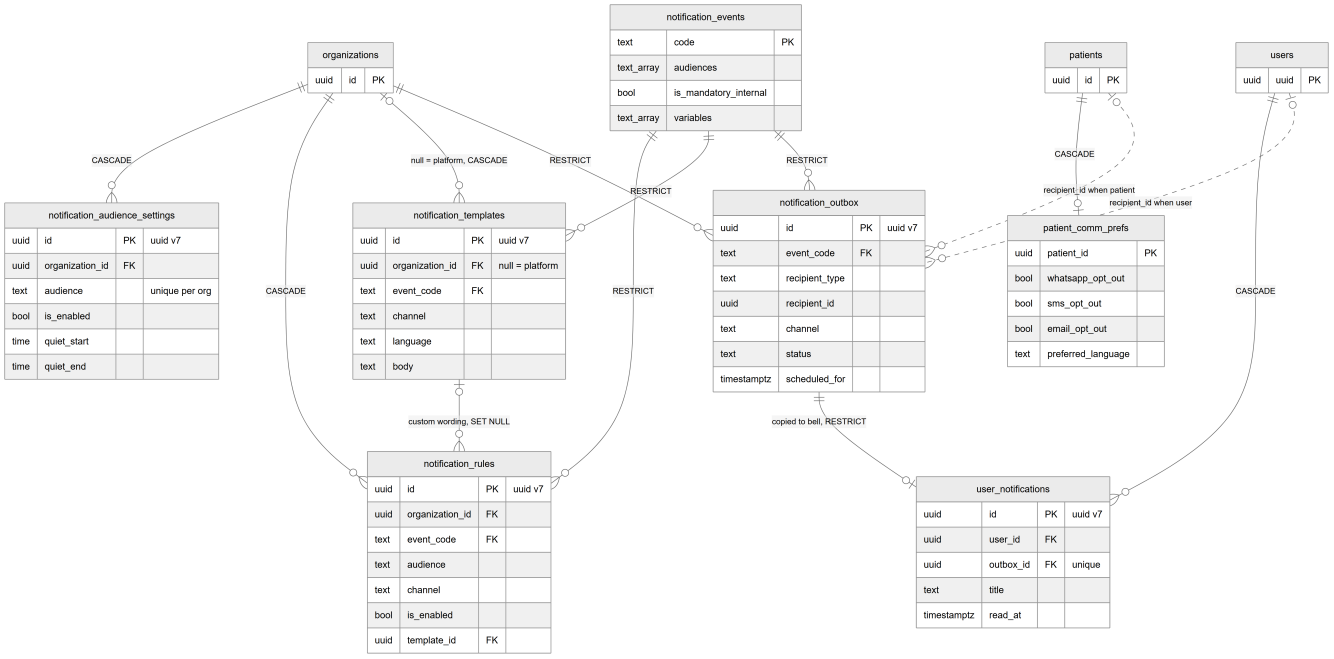

Figure 24.1 — The event catalog, clinic rules, templates, audience settings, the outbox, the bell and patient preferences.

The figure shows `notification_events` at the centre, keyed by a text `code`. It connects with RESTRICT to `notification_rules`, `notification_templates` and `notification_outbox`. `organizations` connects to `notification_audience_settings` and `notification_rules` (CASCADE), to `notification_templates` ("null = platform, CASCADE") and to `notification_outbox` (RESTRICT). `notification_templates` connects to `notification_rules` with "custom wording, SET NULL". `notification_outbox` connects one-to-one to `user_notifications` ("copied to bell, RESTRICT"), and `users` connects to `user_notifications` (CASCADE). `patients` connects one-to-one to `patient_comm_prefs` (CASCADE). Two dotted lines run from `users` and from `patients` to `notification_outbox`, labelled "recipient_id when user" and "recipient_id when patient", showing that the recipient column has no foreign key.

24.2 Tables

`notification_events` — the platform event catalog

One row is one kind of event that can notify someone. The primary key is a text `code`, for example `order.submitted` or `item.ready`. There is no organization column: this is a platform catalog, seeded from code on every start (missing rows are inserted, existing ones are kept). Key columns: `name`, `description`, `audiences` (the audiences it can reach), `is_mandatory_internal` (internal audiences always get it in-app and clinics cannot switch it off), `variables` (placeholders the event supplies) and `sort_order`.

`notification_rules` — per-clinic on/off switches

One row says whether one event is sent to one audience on one channel for one organization, and optionally which template to use. Rows are created only when a clinic changes something. A missing row means the default from the code: on for an internal audience (doctor, coordinator, lab) on the in-app channel, off for everything else. The table has an optimistic `version` column, so two admins saving at once cannot silently overwrite each other.

Constraints that matter:

- Unique per organization, event, audience and channel.

- CHECKs limit `channel` to in_app, whatsapp, sms or email, and `audience` to patient, doctor, coordinator or lab.

`notification_templates` — wording

One row is the wording of one event on one channel in one language (`en`, `ur` or `roman_ur`). If `organization_id` is empty, it is the platform default; a clinic's own row with the same event, channel and language overrides it. Key columns: `subject` (in-app title or email subject), `body` with `{placeholder}` markers, `variables` (computed on save) and `provider_template_ref` (for WhatsApp, the provider-approved template id).

Constraints that matter: two partial unique indexes allow exactly one platform template and one clinic template per event, channel and language.

`notification_audience_settings` — master switch and quiet hours

One row per organization and audience. `is_enabled` is the master switch for that audience. `quiet_start` and `quiet_end` define quiet hours as local wall-clock times, which may wrap past midnight (for example 22:00 to 08:00). `timezone` is the zone for quiet hours; if empty, the location's timezone is used. A missing row means the default: patients off, internal audiences on.

Constraints that matter: unique per organization and audience; quiet start and end must both be set or both be empty.

`notification_outbox` — one message to one recipient on one channel

One row is one message. It is written in the same transaction as the change that caused it. Key columns:

- Who: `audience`, `recipient_type` (user, patient or lab) and `recipient_id`.
- What: `event_code`, `channel`, `payload` (the rendered data), and `entity_type` with `entity_id` (the record that caused it, so a screen can show "messages about this order").
- State: `status`, `scheduled_for`, `attempts`, `provider_message_id`, `last_error` and `sent_at`.

`status` values: `queued`, `held_quiet_hours`, `sent`, `delivered`, `failed`, `skipped_disabled` and `skipped_opt_out`. The relay polls about every two seconds. When the worker claims a row, it increases `attempts` and moves `scheduled_for` five minutes ahead as a lease, so a crashed send is retried later.

Constraints that matter:

- CHECKs on `channel`, `audience` and `status`.
- A partial index over rows in `queued` or `held_quiet_hours`, which keeps the worker's search fast.
- The table is described as never deleted, but no database trigger enforces that.

`user_notifications` — the bell

One row is a user's in-app copy of one sent `in_app` outbox row. Key columns: `title`, `body`, `link`, `event_code`, `organization_id` (so a user who works for several clinics can filter) and `read_at`.

Constraints that matter: `outbox_id` is unique, so a re-delivered send never shows twice; a partial index over unread rows serves the unread count.

`patient_comm_prefs` — a patient's opt-outs

The primary key is the patient id, so each patient has at most one row. Key columns: `whatsapp_opt_out`, `sms_opt_out`, `email_opt_out`, `preferred_language` and `opted_out_at`. It has an optimistic `version` column.

24.3 Relationships explained

- **Organization → audience settings, rules, templates.** Each belongs to one organization (templates: zero or one, because platform defaults have none). Deleting the organization deletes them (CASCADE).
- **Organization → outbox.** Each outbox row belongs to one organization. An organization with outbox rows cannot be deleted (RESTRICT), which keeps the message log.
- **Event → rules, templates, outbox.** Each refers to exactly one event code. An event that is referenced cannot be deleted (RESTRICT). Events are seeded insert-if-missing and are never removed by the code.
- **Template → rules.** A rule may point at one template; deleting the template keeps the rule and clears the link (SET NULL), so the default wording is used again.
- **Outbox → bell entry.** One-to-one: a bell entry belongs to exactly one outbox row, and an outbox row has at most one bell entry. An outbox row with a bell entry cannot be deleted (RESTRICT).
- **User → bell entries.** Each bell entry belongs to one user; deleting the user deletes their bell entries (CASCADE).
- **Patient → preferences.** One-to-one; deleting the patient deletes the preferences (CASCADE).

Logical relationships (no foreign key):

- `notification_outbox.recipient_id` is polymorphic, discriminated by `recipient_type` (a user, a patient or a lab).
- `notification_outbox.entity_id` is polymorphic, discriminated by `entity_type`.
- `notification_outbox.location_id` → `locations`.
- `user_notifications.organization_id` and `patient_comm_prefs.organization_id` → `organizations`.
- Actor columns (`created_by`, `updated_by`) hold a user id.

24.4 Tenant scope

- **Global (platform catalog):** `notification_events`.
- **Organization and location:** `notification_outbox` (location optional).
- **Organization only:** `notification_audience_settings`, `notification_rules`, `notification_templates` (or platform-wide when empty), `user_notifications`, `patient_comm_prefs`.
- `user_notifications` belongs to a user, not a clinic, but carries the organization id so the bell can be filtered per clinic.

Chapter 25 — Labs

This chapter describes the data model of dental-lab work: the lab directory and how clinics connect to labs, the lab's service catalog and prices, lab orders and their items, production (try-in stages, technicians, shipments, remakes, warranties), and the money on both sides: what the patient pays the clinic for lab work, and what the clinic owes the lab. It is for the product owner who needs to know how the roughly fifty lab tables fit together. "Lab" here always means a dental laboratory that makes crowns, bridges and dentures, not a diagnostic test lab. The entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) remains the field-level reference.

25.1 Three ideas that run through every lab table

A lab is not a tenant. A clinic is an organization (the tenant). An external lab works for many clinics, so `labs` has no organization column. A clinic connects to a lab through a **lab link** (`lab_clinic_links`). Lab users are **lab members**: they have no clinic membership at all, so no clinic query can return their data. An internal lab, run by one clinic, records its owner in `owner_organization_id` .

Snapshots and versions. A lab order item copies the service name, options and prices when the order is submitted, and pins the exact form version and workflow version it runs on. Later catalog or workflow changes never alter work already ordered. Changes after submit become a new order revision, never an overwrite.

UUID version 7 keys. Every lab table, and the workflow engine, notification and general-ledger tables, uses a primary key generated by the database function `uuid_generate_v7()` , installed at start-up and by the hand-written migration `1792490000000-lab-management-foundation` . A version 7 UUID is still a normal 128-bit `uuid` , but its first 48 bits are the creation time in milliseconds. Ids therefore sort roughly by creation time, and indexes on append-heavy tables stay compact because new keys land at the end of the index instead of at random places. The remaining bits are still random. Older modules use random version 4 UUIDs.

Money convention. All lab money is `bigint` paisa. Tax rates are basis points (`tax_rate_bp` , `tax_bp`).

Status. The lab module is [Implemented] end to end: clinic screens (labs, lab price list, order templates, lab orders, lab billing, lab payables) and the lab portal for lab members. Exceptions are labelled where they occur.

25.2 Directory, links, members and magic links

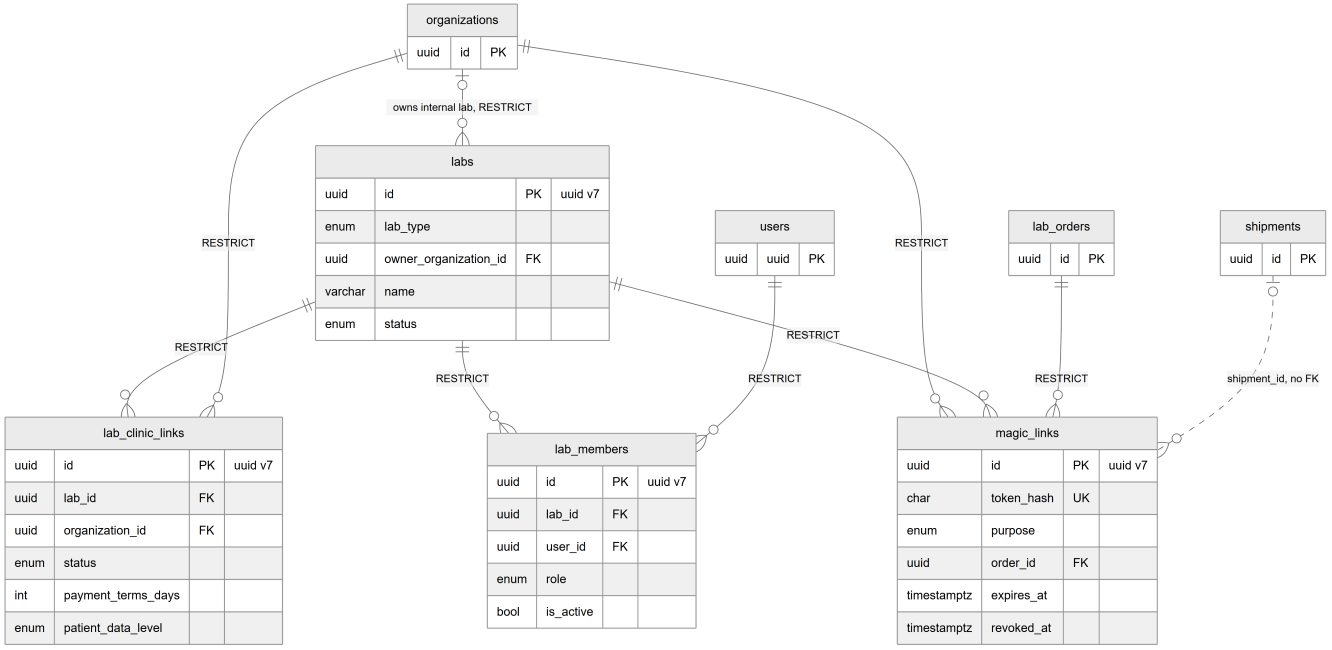

Figure 25.1 — Labs, the clinic-to-lab link, lab members and magic links.

The figure shows `labs` with solid lines to `lab_clinic_links`, `lab_members` and `magic_links`, all RESTRICT. `organizations` connects to `labs` ("owns internal lab, RESTRICT"), to `lab_clinic_links` and to `magic_links` (RESTRICT). `users` connects to `lab_members` (RESTRICT), and `lab_orders` connects to `magic_links` (RESTRICT). A dotted line from `shipments` to `magic_links` is labelled "shipment_id, no FK". All four main boxes note UUID version 7 keys, and `token_hash` on `magic_links` is marked unique.

`labs` — the lab directory

One row is one dental lab. `lab_type` is external or internal; `status` is active or suspended. Contact columns (`phone`, `whatsapp_phone`, `email`, `city`, `address`) hold the lab's business contact. Constraint that matters: `chk_labs_owner` requires `owner_organization_id` to be set exactly when the lab is internal.

`lab_clinic_links` — a clinic working with a lab

One row connects one organization to one lab. `status` is invited, active or suspended. `payment_terms_days` (0–365, default 30) sets the clinic's payment terms. `patient_data_level` decides how much patient identity the lab sees: `full_name` or `code_only`. `activated_at` records the first activation. Status changes go through one service and are logged in `status_history`. Constraint that matters: one link per lab and organization.

`lab_members` — people who work at a lab

One row makes a user a member of a lab, with `role` lab_admin or lab_technician, and `is_active` as a soft delete. Lab-side access is resolved from these rows only. Constraint that matters: one membership per lab and user.

`magic_links` — no-login links for a lab

A magic link lets a lab view or update one order without logging in. The token itself is never stored: the table keeps only `token_hash` (a SHA-256 hash, unique) and a random `nonce` from which the server can re-derive the token at send time. Key columns: `purpose` (lab_view_order, lab_update_order or receive_shipment), `lab_id`,

`order_id`, `delivery` (sent as a notification or copied by the user), `expires_at`, `max_uses` with `used_count`, `last_used_at`, `last_viewed_at`, and the revocation columns `revoked_at`, `revoked_reason` and `revoked_by`. Constraint that matters: `chk_magic_links_uses` keeps `used_count` within `max_uses`.

Status of shipment links: [Not implemented]. The `receive_shipment` purpose and the `shipment_id` column exist, but no code creates such links; clinics receive shipments by scanning the shipment's own QR token. The column comment says the foreign key would be added "until that table exists here"; the `shipments` table now exists but the foreign key was never added.

25.3 Lab catalog

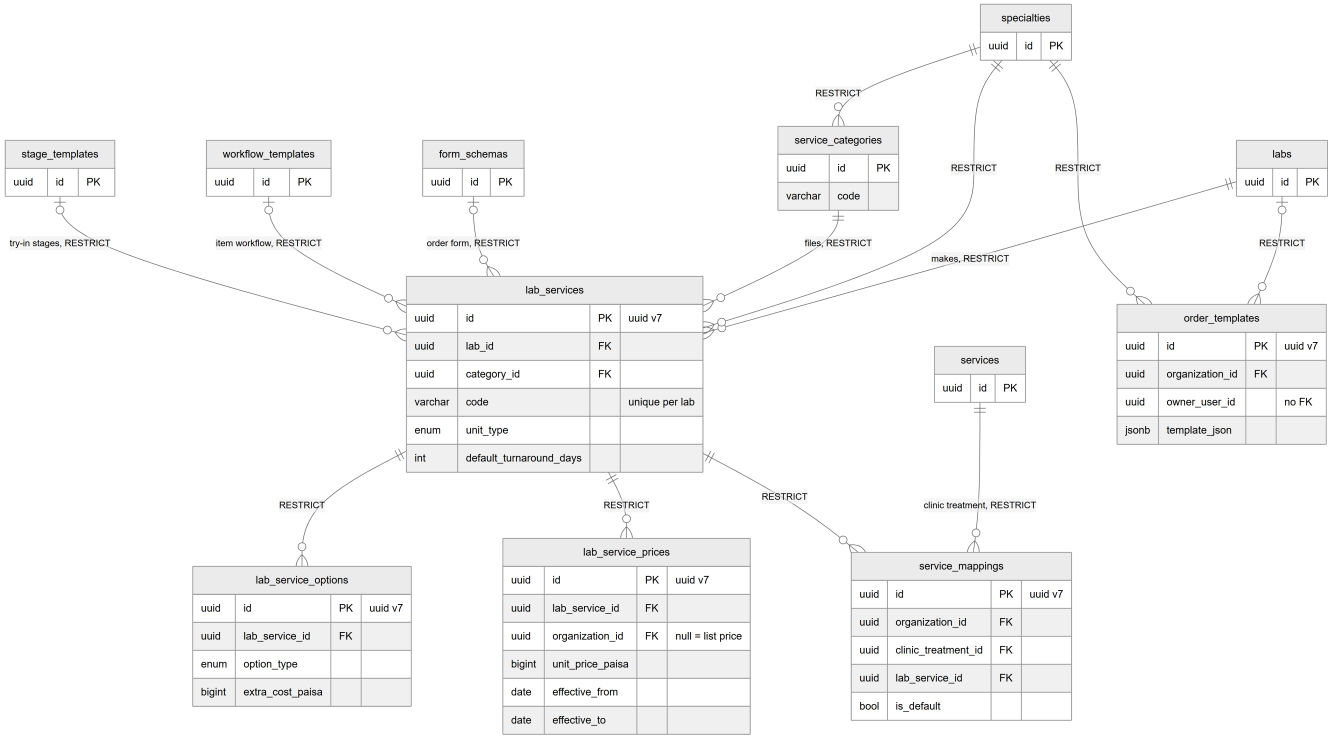

Figure 25.2 — Lab services, options, prices, categories, mappings to clinic services and order templates.

The figure shows `lab_services` at the centre. Solid RESTRICT lines run into it from `labs` ("makes"), `specialties`, `service_categories` ("files"), `form_schemas` ("order form"), `workflow_templates` ("item workflow") and `stage_templates` ("try-in stages"). `specialties` also connects to `service_categories` and to `order_templates`. `lab_services` connects down to `lab_service_options`, `lab_service_prices` and `service_mappings`. `services` (the clinic's billing catalog) connects to `service_mappings` with "clinic treatment, RESTRICT". `labs` connects to `order_templates`. Notes mark `lab_services.code` as "unique per lab", `lab_service_prices.organization_id` as "null = list price", and `order_templates.owner_user_id` as "no FK".

`service_categories` — shared categories

A platform-level grouping of lab services within a specialty, such as CROWN, BRIDGE or DENTURE. Every lab files its services under the same categories, so clinics can compare labs like for like. Unique per specialty and code.

`lab_services` — what a lab makes

One row is one product of one lab, for example a zirconia crown. It has no organization column; clinics see it through their lab link. Key columns: `code` (upper-case, unique per lab), `name`, `unit_type` (per_tooth, per_unit, per_arch or per_case), `default_turnaround_days` (0–365) and `is_active` (archived services are hidden from new

orders; old orders keep them). Three optional links point at the workflow engine: the order form the doctor fills (`form_schema_id`), the item workflow (`workflow_template_id`) and the try-in stage template (`stage_template_id`).

`lab_service_options` — materials and variants

A material or variant of a service, such as "Emax" on a crown, with `extra_cost_paisa` (not negative) added to the resolved price. Unique per service and code.

`lab_service_prices` — price windows

One row is a price for a service over a date window (`effective_from` to `effective_to` , inclusive; an empty end means open-ended). If `organization_id` is empty, it is the lab's list price; if set, it is a negotiated price for that clinic, which wins over the list price. Columns include `unit_price_paisa` , `rush_surcharge_paisa` and `currency` (default PKR). A price entered by mistake is voided (`voided_at` , `void_reason`), not deleted.

Constraint that matters: `excl_lab_service_prices_overlap` , an EXCLUDE constraint (using `btree_gist`), forbids two non-voided windows for the same service and clinic (or the same list price) from overlapping.

`service_mappings` — which lab service makes which clinic treatment

One row says that a lab service fulfils one of the clinic's billable treatments. The clinic treatment is a row of the billing catalog `services` , held in a column named `clinic_treatment_id` . Constraints that matter:

- One row per organization, treatment and lab service.
- At most one active default lab service per treatment (partial unique index).
- An active lab service can be mapped to only one treatment per organization (partial unique index).

`order_templates` — saved orders

A doctor's saved starting point for an order ("my usual zirconia crown"). `owner_user_id` empty means shared with every doctor in the clinic; `lab_id` empty means the doctor picks the lab when ordering. `template_json` holds the order body; `use_count` and `last_used_at` track use.

25.4 Lab orders

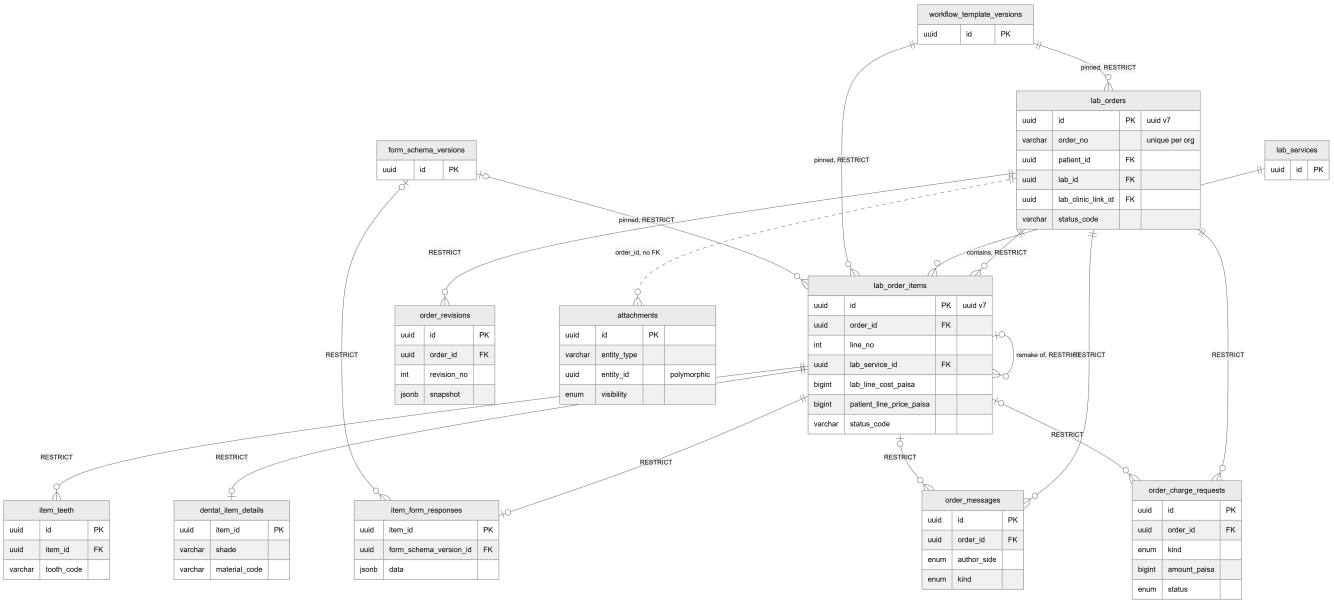

Figure 25.3 — Lab orders, items and their teeth, details, form answers, messages, revisions, charge requests and attachments.

The figure shows `lab_orders` connected to `lab_order_items` ("contains, RESTRICT"). `lab_order_items` has a loop to itself labelled "remake of, RESTRICT". `lab_services` connects to `lab_order_items`; `workflow_template_versions` connects to both `lab_orders` and `lab_order_items` ("pinned"); `form_schema_versions` connects to `lab_order_items` ("pinned") and to `item_form_responses`. From `lab_order_items`, lines run to `item_teeth` (many), `dental_item_details` (one-to-one) and `item_form_responses` (one-to-one). `lab_orders` connects to `order_messages`, `order_revisions` and `order_charge_requests`; items also connect optionally to messages and charge requests. A dotted line from `lab_orders` to `attachments` is labelled "order_id, no FK". All lines are RESTRICT.

`lab_orders` — a lab case

One row is one case: a container of items for one patient, one doctor and one lab, sent from one location. Key columns: `order_no` (for example LAB-0001, assigned at submit from the organization's number sequence), `status_code` (a state of the order workflow version pinned at creation, which never changes), `priority` (normal or urgent), `requested_on` and `lab_expected_on`, the time stamps `submitted_at`, `accepted_at`, `completed_at` and `cancelled_at`, `current_revision_no` (0 for drafts, 1 at submit, +1 per revision) and `idempotency_key`. Optional logical links point at the appointment, the patient's treatment record and the order template the draft started from.

Constraints that matter: `order_no` is unique per organization; the idempotency key is unique per organization, so repeating a create request returns the same draft.

`lab_order_items` — one piece of work

One row is one piece of work in an order, such as a crown or a bridge, numbered by `line_no` (unique per order). Key columns:

- Snapshots frozen at submit: `service_code_snap`, `service_name_snap`, `unit_type`, `options_snap`.
- Prices frozen at submit: `lab_unit_cost_paisa`, `lab_line_cost_paisa` (what the lab charges the clinic), `patient_unit_price_paisa`, `patient_line_price_paisa` (what the clinic charges the patient), `priced_on`,

`currency`.

- Draft inputs: `lab_service_id`, `option_ids`, `rush`, `quantity` (above zero), `clinic_treatment_id`.
- Engine: the pinned `form_schema_version_id` and `workflow_template_version_id`, the copied `stage_template_id`, `status_code` (empty while the order is a draft) and `current_stage_seq`.
- `remake_of_item_id`: set when this item remakes an earlier one.

Only a revision or an approved charge request may change the frozen columns. CHECKs keep all paisa columns non-negative and the quantity above zero.

Item parts: `item_teeth`, `dental_item_details`, `item_form_responses`

- `item_teeth`: one row per tooth of an item; a three-unit bridge has three rows. `numbering_system` is FDI (default), UNIVERSAL or PALMER; `tooth_code` is unique per item; `surfaces` lists tooth surfaces. Rows are written at submit from the form answers, which remain the source of truth.
- `dental_item_details`: one row per item (its primary key is the item id) with typed columns copied from the form answers at submit, such as shade, material and implant details, so reports read columns instead of JSON.
- `item_form_responses`: one row per item (primary key = item id) with the form answers in `data` (JSON) against the pinned form version. Drafts store whatever was typed; `validated_at` is set when submit validates them.

`order_messages`, `order_revisions`, `order_charge_requests`, `attachments`

- `order_messages`: the clinic-to-lab conversation on an order or one item. `author_side` is clinic or lab; `kind` is note, clarification_request or reply; a clarification is resolved by the clinic's reply (`resolved_at`).
- `order_revisions`: what the lab was asked to make, as of one revision. Revision 1 is written at submit; each later change adds a revision with a full `snapshot` and a `reason`. The lab acknowledges each one (`lab_acknowledged_at`). Unique per order and revision number.
- `order_charge_requests`: an extra charge the lab asks for after submit (`kind` rush, material, shipping or other) with `amount_paisa` above zero, a required `reason`, and `status` pending, approved or rejected. An approved item-level request is added to the item's lab line cost.
- `attachments`: a file on an order or item. The file itself is in private storage; the row keeps `storage_key`, `file_name`, `mime_type`, `size_bytes` and `sha256`. `entity_type` is limited by a CHECK to `lab_order` or `lab_order_item`. `visibility` is internal, lab_shared or patient_shared (the lab sees only lab_shared), and `uploaded_side` is clinic or lab. Downloads use short-lived signed URLs.

25.5 Production

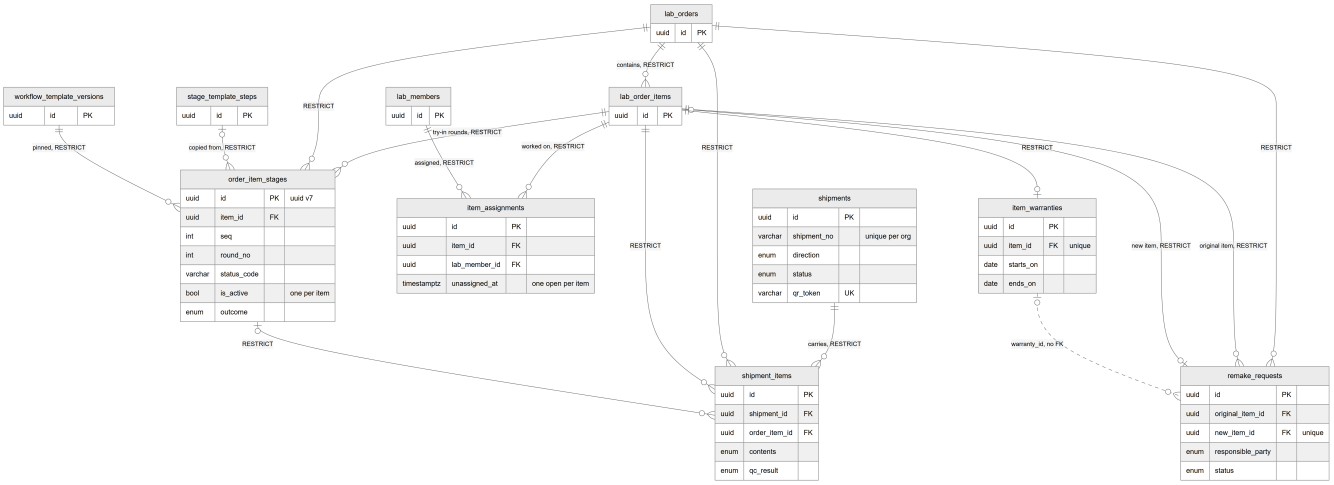

Figure 25.4 — Try-in stages, technician assignments, shipments, remakes and warranties.

The figure shows `lab_orders` and `lab_order_items` at the top. `lab_order_items` connects to `order_item_stages` ("try-in rounds"), `item_assignments` ("worked on"), `shipment_items`, `remake_requests` twice ("original item" many, and "new item" one-to-one) and `item_warranties` (one-to-one). `lab_orders` also connects to `order_item_stages`, `shipment_items` and `remake_requests`. `stage_template_steps` connects to `order_item_stages` ("copied from"), `workflow_template_versions` connects to it ("pinned"), and `lab_members` connects to `item_assignments` ("assigned"). `shipments` connects to `shipment_items` ("carries"), and `order_item_stages` connects optionally to `shipment_items`. A dotted line from `item_warranties` to `remake_requests` is labelled "warranty_id, no FK". All solid lines are RESTRICT. Notes mark `is_active` "one per item" on stages and `unassigned_at` "one open per item" on assignments.

`order_item_stages` — try-in rounds

One row is one round of one stage of an item, for example "metal try-in, round 2". Rows are created from the item's stage template when the lab accepts the order. A correction closes the round and adds a new row with `round_no + 1`; rounds are never overwritten. Key columns: `seq`, `round_no`, copied `code_snap` and `name_snap`, `is_final` (the last stage, finished work rather than a try-in), `requires_patient_visit`, `status_code`, `lab_expected_on`, an optional try-in `appointment_id`, `outcome` (approved, correction or rejected), notes, and who decided.

Constraints that matter: unique per item, sequence and round; at most one active stage per item (partial unique index); round and sequence above zero.

`item_assignments` — which technician works on an item

One row assigns a lab member to an item. Reassigning closes the open row (`unassigned_at`) and adds a new one. Constraint: at most one open assignment per item (partial unique index).

`shipments` and `shipment_items` — parcels between clinic and lab

A shipment is one parcel in either direction (`direction` to_lab or to_clinic) between one clinic and one lab; it can carry items of several orders. `shipment_no` (for example SHP-0001) is unique per organization. `status` is dispatched, partially_received, received or lost. `qr_token` is unique platform-wide and printed on the slip; scanning it opens the receive screen. A shipment item is one item in a parcel, with `contents` (impression, model, framework, tryin, final or other), `quantity`, `received_qty` (CHECK: not above quantity), `condition`, `qc_result` and `qc_notes`. An item appears at most once per shipment.

`remake_requests` — asking for work to be redone

One row asks to remake an item. `reason_code` is shade, fit, fracture, design, damaged or other. `responsible_party` (lab, clinic, patient or undetermined) decides pricing: if the lab is at fault, both the lab charge and the patient charge are zero. `status` is requested, agreed, disputed or closed. If agreed, a new item is created in the same order and linked through `new_item_id`. An active warranty marks the request as a warranty claim automatically.

Constraints that matter: `new_item_id` is unique; at most one open (requested or disputed) request per original item.

`item_warranties` — warranty after delivery

One row per item (the item id is unique) with `starts_on`, `ends_on` (CHECK: not before the start), `terms`, and void columns. The warranty starts when the item is delivered.

25.6 Lab finance: what the patient owes the clinic

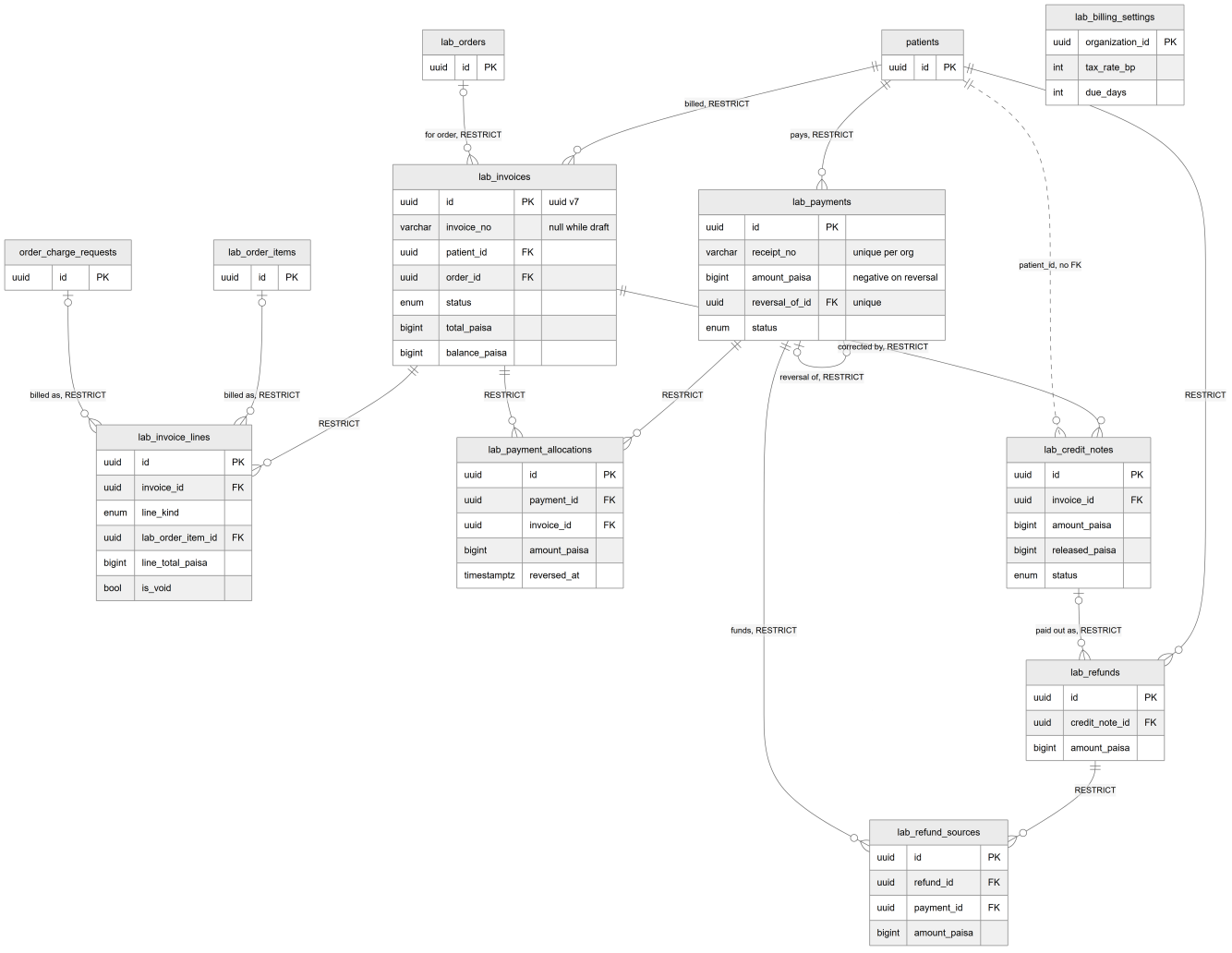

Figure 25.5 — Patient lab invoices, lines, payments, allocations, credit notes, refunds and settings.

The figure shows `patients` with solid lines to `lab_invoices` ("billed"), `lab_payments` ("pays") and `lab_refunds`, and a dotted line to `lab_credit_notes` (patient_id, no FK). `lab_orders` connects to `lab_invoices` ("for order"). `lab_invoices` connects to `lab_invoice_lines`, `lab_payment_allocations` and `lab_credit_notes` ("corrected by"). `lab_order_items` and `order_charge_requests` connect to `lab_invoice_lines` ("billed as"). `lab_payments`

has a one-to-one loop to itself ("reversal of") and connects to `lab_payment_allocations` and `lab_refund_sources` ("funds"). `lab_credit_notes` connects to `lab_refunds` ("paid out as"), and `lab_refunds` connects to `lab_refund_sources`. `lab_billing_settings` stands alone with its key `organization_id`. All solid lines are RESTRICT.

This side mirrors the clinic billing design (the Billing core chapter, Chapter 20) but with its own tables and in paisa. The UI calls these **patient lab invoices**.

- **`lab_invoices`.** A patient invoice for lab work, optionally for one order. A `draft` is an estimate: it has no number, can be edited and owes nothing. Issuing gives it an `invoice_no` (unique per organization) and freezes the lines. `status` is draft, issued, partially_paid, paid or void. Cached totals (`subtotal_paisa`, `discount_paisa`, `tax_paisa`, `total_paisa`, `credited_paisa`, `paid_paisa`, `balance_paisa`) are recomputed from source rows by one service. CHECK `chk_lab_invoices_math`: total = subtotal − discount + tax, and balance, paid and credited are not negative.
- **`lab_invoice_lines`.** One billed thing: `line_kind` lab_item, lab_charge or manual. CHECK `chk_lab_invoice_lines_math` enforces quantity × unit − discount + tax = line total. Two partial unique indexes make sure a lab order item or charge request sits on at most one non-void line, so it is never billed twice; voiding the invoice copies `is_void` onto the lines and frees them. Lines also carry a commission snapshot (`doctor_id`, `performing_assignment_id`, `commission_plan_id`, rate and fixed amount), all without foreign keys; the commission ledger links to these lines (the Compensation chapter, Chapter 22).
- **`lab_payments`.** Money received from a patient for lab work. It belongs to the patient; the unallocated, unrefunded part is the patient's advance. A wrong payment is never deleted: a reversal row with a negative amount points at it through `reversal_of_id` (unique). `received_on` is the location-local day. `receipt_no` and `idempotency_key` are each unique per organization. CHECK `chk_lab_payments_amounts` ties the sign of the amount to whether it is a reversal and keeps allocated + refunded within the amount.
- **`lab_payment_allocations`.** "This much of payment P settles invoice I", always positive. A reversal or credit note stops it counting by setting `reversed_at` and `release_reason`; rows are not deleted.
- **`lab_credit_notes`.** Corrects an issued invoice downwards. Money the patient already paid above the new total is released back to advance (`released_paisa`) and may then be refunded (`refunded_paisa`). `credit_note_no` is unique per organization. CHECK: refunded ≤ released ≤ amount.
- **`lab_refunds`** and **`lab_refund_sources`.** A refund pays money back out of advance; refund sources say which payments funded it. `credit_note_id` set means the refund pays out a credit note. `refund_no` and `idempotency_key` are unique per organization.
- **`lab_billing_settings`.** One row per organization (primary key = organization id): `tax_rate_bp` (0–10,000, default 0 until the accountant confirms) and `due_days`. A missing row means defaults.

25.7 Lab finance: what the clinic owes the lab

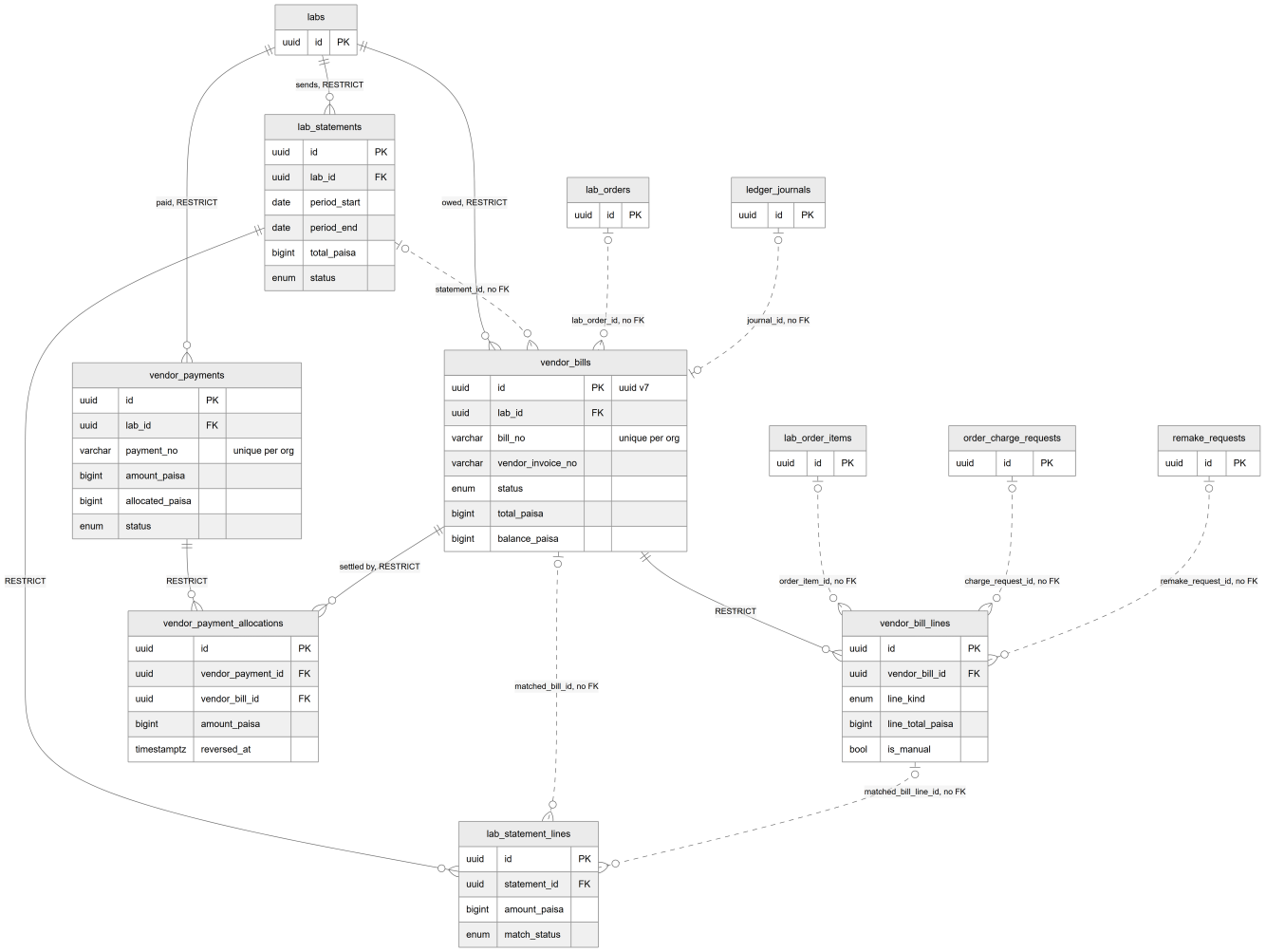

Figure 25.6 — Vendor bills, vendor payments, allocations and lab statements.

The figure shows `labs` with solid lines to `vendor_bills` ("owed"), `vendor_payments` ("paid") and `lab_statements` ("sends"). `vendor_bills` connects to `vendor_bill_lines` and `vendor_payment_allocations` ("settled by"); `vendor_payments` connects to `vendor_payment_allocations`; `lab_statements` connects to `lab_statement_lines`. Dotted lines (no foreign key) run from `lab_statements` to `vendor_bills` (statement_id), from `vendor_bills` and `vendor_bill_lines` to `lab_statement_lines` (matched bill and matched bill line), from `lab_orders` to `vendor_bills` (lab_order_id), from `ledger_journals` to `vendor_bills` (journal_id), and from `lab_order_items`, `order_charge_requests` and `remake_requests` to `vendor_bill_lines`. All solid lines are RESTRICT.

- **`vendor_bills`.** What the clinic owes a lab. One draft per lab order is created when the lab accepts the order, from the items' frozen lab costs (a partial unique index allows only one draft per order). Nothing reaches the general ledger until approval, which posts a Lab Cost / Lab Payable journal (`journal_id`). `status` is draft, approved, partially_paid, paid or void. `bill_no` is unique per organization; the lab's own invoice number (`vendor_invoice_no`), entered at approval, is unique per organization and lab among non-void bills. CHECK: total = subtotal + tax, and paid and balance are not negative. Bills are voided, never deleted.
- **`vendor_bill_lines`.** One cost on a bill: `line_kind` service, rush, shipping, material, remake or other. Lines of a draft follow the order and are re-synced; after approval they are frozen. `is_manual` marks lines added by hand. Links to the order, item, charge request and remake request have no foreign keys.
- **`vendor_payments`** and **`vendor_payment_allocations`.** Money the clinic paid a lab, split over approved bills by allocations. An unallocated rest is an advance to the lab. `payment_no` is unique per organization. A mistake is

reversed (`status` reversed, `reversed_at`), with a mirror journal; allocations are reversed with `reversed_at`, never deleted.

- **`lab_statements` and `lab_statement_lines`.** A lab's monthly statement, typed in by the clinic and reconciled against the bills of the period. `status` is uploaded, reconciling, reconciled or disputed. Each line has the lab's reference, date and amount, the clinic's `system_amount_paisa` for the same reference, and `match_status` (matched, amount_mismatch, missing_in_system or missing_in_statement).

Status of statement files: [Not implemented]. `lab_statements.attachment_id` exists, but the column comment says there is no storage for statement files yet, because attachments are order-scoped.

25.8 Relationships explained

Every foreign key in the lab tables uses ON DELETE RESTRICT: no lab record can be deleted while anything refers to it. Records are voided, reversed, archived or switched off instead.

Directory, links and magic links:

- An internal lab belongs to zero or one owner organization; an organization can own many labs.
- Each lab link belongs to exactly one lab and one organization; one link per pair.
- Each lab member belongs to exactly one lab and one user; one membership per pair.
- Each magic link belongs to exactly one organization, one lab and one lab order; an order can have many links.

Catalog:

- Each lab service belongs to exactly one lab, one specialty and one service category, and optionally to one form schema, one workflow template and one stage template.
- Each service category belongs to exactly one specialty.
- Each option and each price row belongs to exactly one lab service. A price row belongs to zero or one organization (empty = list price).
- Each service mapping belongs to exactly one organization, one clinic service (`services`) and one lab service.
- Each order template belongs to exactly one organization and one specialty, and optionally to one lab.

Orders:

- Each lab order belongs to exactly one organization, location, specialty, patient, lab, lab link and pinned workflow version, and optionally to one doctor (employee).
- Each order item belongs to exactly one order, one lab service and one pinned workflow version, and optionally to one form version and to one earlier item it remakes.
- Each tooth row belongs to exactly one item; each item has at most one details row and at most one form-response row (both keyed by the item id). A form response may point at one form version.
- Each message, revision and charge request belongs to exactly one order; messages and charge requests may also point at one item.

Production:

- Each stage row belongs to exactly one order, one item and one pinned workflow version, and optionally to one stage-template step.
- Each assignment belongs to exactly one item and one lab member.
- Each shipment item belongs to exactly one shipment, one order and one item, and optionally one stage.

- Each remake request belongs to exactly one order and one original item, and to zero or one new item (one-to-one).
- Each warranty belongs to exactly one item (one-to-one through the unique item id).

Finance:

- Each patient lab invoice belongs to exactly one patient and optionally one order; each line belongs to exactly one invoice and optionally one item or one charge request.
- Each lab payment belongs to exactly one patient and may reverse exactly one earlier payment (one-to-one).
- Each allocation belongs to exactly one payment and one invoice. Each credit note belongs to exactly one invoice.
- Each lab refund belongs to exactly one patient and optionally one credit note; each refund source belongs to exactly one refund and one payment.
- Each vendor bill and vendor payment belongs to exactly one lab; each bill line belongs to exactly one bill; each vendor allocation belongs to exactly one vendor payment and one bill.
- Each lab statement belongs to exactly one lab; each statement line belongs to exactly one statement.

Logical relationships (no foreign key):

- `magic_links.shipment_id` → `shipments` (unused, see 25.2).
- `order_templates.owner_user_id` → `users`.
- `attachments.entity_id` is polymorphic (order or item); `attachments.order_id` → `lab_orders`.
- `lab_order_items.clinic_treatment_id` → `services`, `option_ids` → `lab_service_options` (array), `lab_price_id` → `lab_service_prices`, `stage_template_id` → `stage_templates`. Note that `service_mappings.clinic_treatment_id` has a foreign key to the same `services` table, but the item's copy does not.
- `lab_orders.appointment_id` → `appointments`, `treatment_id` → `treatments`, `order_template_id` → `order_templates`.
- `order_messages.author_user_id` → `users`.
- `item_assignments.lab_id`, `order_item_stages.lab_id`, `remake_requests.lab_id`, `shipments.lab_id` → `labs`. These copies let the lab portal filter by lab quickly.
- `order_item_stages.appointment_id` → `appointments`; `remake_requests.warranty_id` → `item_warranties`.
- `lab_credit_notes.patient_id` → `patients`.
- `lab_invoice_lines.clinic_treatment_id` → `services`, `doctor_id` → `employees`, `performing_assignment_id` → `employee_org_location_assignments`, `commission_plan_id` → `compensation_plans`.
- `vendor_bills.lab_order_id` → `lab_orders`, `statement_id` → `lab_statements`, `journal_id` → `ledger_journals`.
- `vendor_bill_lines.lab_order_id`, `order_item_id`, `charge_request_id`, `remake_request_id` → the matching lab tables.
- `lab_statement_lines.matched_bill_id` → `vendor_bills` and `matched_bill_line_id` → `vendor_bill_lines`.
- `lab_statements.attachment_id` → `attachments` (unused).
- Actor columns (`created_by`, `updated_by`, `decided_by`, `received_by` and others) hold a user id.

25.9 Tenant scope

- **Global, lab-side:** `labs` (internal labs carry an owner organization), `lab_members`, `lab_services`, `lab_service_options`. These are scoped by `lab_id`, not by organization.

- **Global, platform catalog:** `service_categories`.
- **Organization and location:** `lab_orders`, `lab_invoices`, `lab_payments`, `lab_credit_notes`, `lab_refunds`, `vendor_bills`, `vendor_payments`.
- **Organization only:** every other lab table, including items, item parts, messages, revisions, charge requests, attachments, all production tables, invoice lines, allocations, refund sources, settings, bill lines, vendor allocations, statements and statement lines. `lab_service_prices` is organization-scoped when a negotiated price, global when a list price.
- Only `lab_clinic_links`, `magic_links`, `lab_service_prices`, `service_mappings`, `order_templates` and `lab_orders` have foreign keys on their tenant columns. Everywhere else the organization id is stored without a constraint and the application enforces isolation.

Chapter 26 — Workflow engine and common tables

This chapter describes two groups of tables. The first is the workflow engine: twelve tables that store specialties, versioned forms, try-in stage templates and versioned state machines, which the lab module runs on. The second is the shared infrastructure in `backend/src/common/`: the generic audit log, the status history and the per-organization number counters, plus the unused `chairs` table. It is written for the product owner who needs to know which behaviour is configured as data rather than code, and which history can never be changed. The entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) remains the field-level reference.

26.1 The workflow engine in one paragraph

A **specialty** (for example DENTAL) is the root. Under it, the Platform keeps **form schemas** (what the doctor fills in when ordering a lab item), **stage templates** (the try-in rounds a piece of work goes through) and **workflow templates** (state machines for lab orders, lab order items and stages). Forms and workflows are versioned: a version is a draft while it is edited, becomes published and then frozen, and is finally retired. Every lab order and item pins the exact version it started on, so editing a form or workflow never breaks work in progress. When a workflow version is published, its JSON definition is turned into rows of states, transitions and transition effects, which the runtime reads. Effects are code handlers listed in a registry; configuration can only choose among them, never add new code.

Status: [Implemented]. The super admin edits forms, workflows and stage templates in Config Studio (sidebar entries "Form schemas", "Workflows" and "Stage templates", super admin only). The lab order and production services run on the published versions.

Keys. All engine tables with an `id` use UUID version 7 keys (`uuid_generate_v7()`); the Labs chapter (Chapter 25) explains them. `organization_specialties`, `transition_effects` and `workflow_effects` have natural keys supplied by the application.

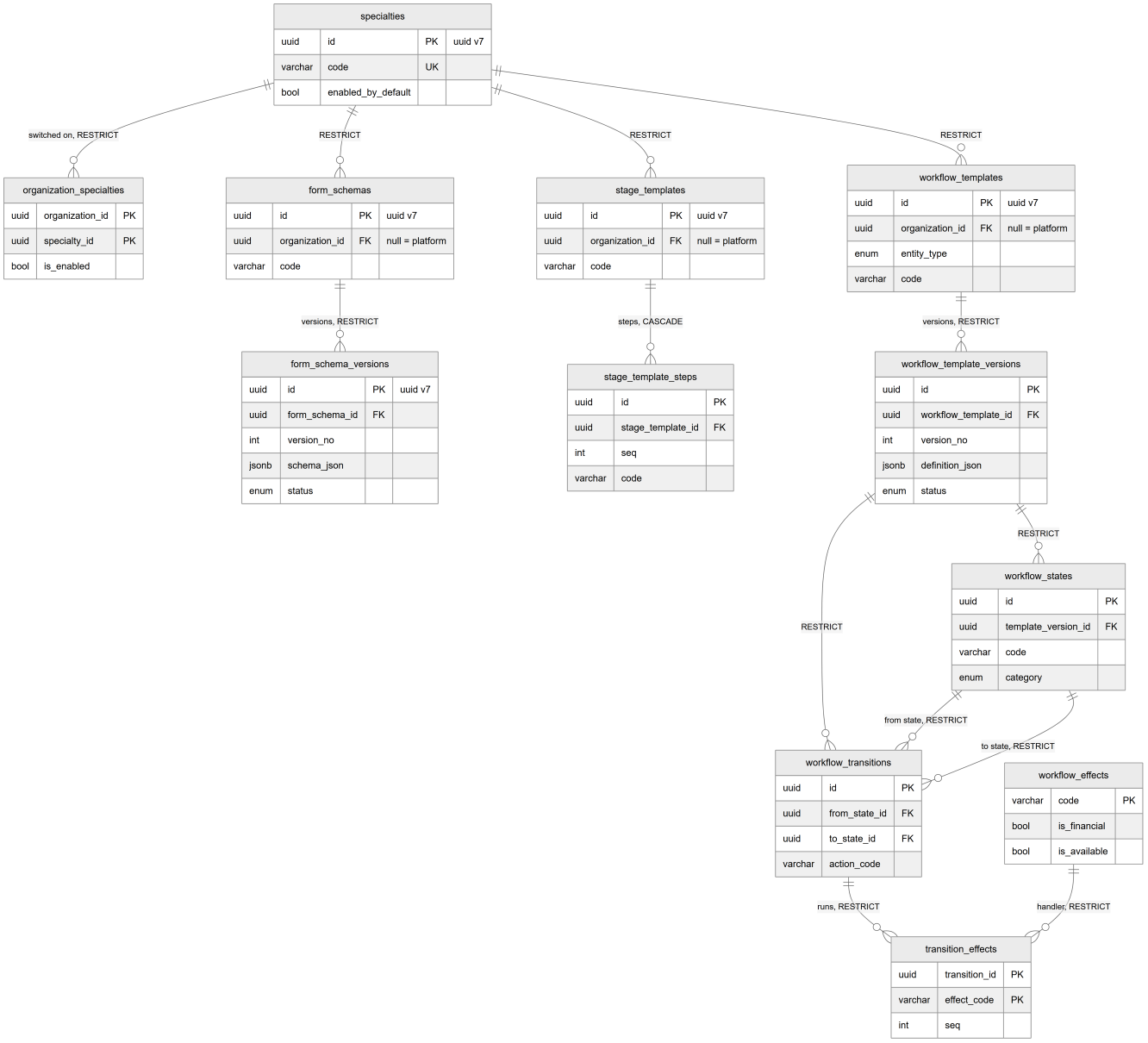

Figure 26.1 — Specialties, form schemas and versions, stage templates and steps, workflow templates, versions, states, transitions and effects.

The figure shows `specialties` at the top with four lines down: to `organization_specialties` ("switched on"), `form_schemas`, `stage_templates` and `workflow_templates`, all RESTRICT. `form_schemas` connects to `form_schema_versions` ("versions, RESTRICT"). `stage_templates` connects to `stage_template_steps` ("steps, CASCADE"). `workflow_templates` connects to `workflow_template_versions` ("versions, RESTRICT"). `workflow_template_versions` connects to `workflow_states` and to `workflow_transitions`. `workflow_states` connects twice to `workflow_transitions`, labelled "from state" and "to state". `workflow_transitions` and `workflow_effects` both connect to `transition_effects` ("runs" and "handler"). All lines except the stage steps are RESTRICT. The template boxes mark `organization_id` as "null = platform".

26.2 Engine tables

`specialties` and `organization_specialties`

`specialties` is a platform catalog of clinical specialties, each with a unique upper-case `code`, a `name`, `is_active`, and `enabled_by_default` (new organizations get it switched on; DENTAL does). `organization_specialties` records which specialties one organization works in, with `is_enabled`. Its primary key is the pair (organization, specialty). Rows are created lazily from `enabled_by_default` the first time an organization is read.

`form_schemas` and `form_schema_versions` — versioned order forms

A form schema is the stable identity of a form, such as CROWN or BRIDGE, within a specialty. If `organization_id` is empty, it is the platform default; an organization's own schema with the same code overrides the default for that organization. Partial unique indexes allow one platform schema and one per organization for each specialty and code.

A form schema version holds the content: `schema_json` (the field definitions), `ui_json` (layout, such as tooth-chart settings), `promotions` (which answers are copied into typed columns, for example into `dental_item_details`) and `status` (draft, published or retired) with `published_at` and `retired_at`.

Constraints that matter:

- `version_no` is unique per form and above zero.
- At most one draft and at most one published version per form (two partial unique indexes).
- `chk_form_schema_versions_published_at`: a draft has no publish date, and every non-draft has one.

Correction to the catalog. The entity's code comment, repeated in the catalog, says a database trigger makes published versions immutable. No such trigger exists. The rule is enforced by the form schemas service, which allows content changes only on drafts.

`stage_templates` and `stage_template_steps` — try-in rounds

A stage template lists the try-in rounds a kind of work goes through, for example metal, then bisque, then final for a porcelain-fused-to-metal crown. It follows the same platform-or-organization pattern as forms. Each step has a `seq` (1-based, unique per template), a `code` (unique per template), a `name`, `requires_patient_visit`, `requires_doctor_approval` and `default_lab_days` (0–365). Lab orders copy step names into their own stage rows, so editing a template never changes work in progress.

`workflow_templates`, `workflow_template_versions`, `workflow_states`, `workflow_transitions`

A workflow template is a state machine for one kind of record: `entity_type` is order, item or stage. It follows the platform-or-organization pattern. A workflow template version holds the draft definition as JSON (`definition_json`) and a `status` of draft, published or retired; as with forms, at most one draft and one published version exist per template, and `version_no` is unique and above zero. Published versions never change; this is enforced by the service, not the database.

Publishing a version writes its rows once:

- `workflow_states`: one row per state, with a `code` unique per version, a `name`, a `category` (initial, active, waiting, done or cancelled), `is_terminal` and `sort_order`.
- `workflow_transitions`: one row per allowed move from one state to another under an `action_code`, unique per version, from-state, action and to-state. `allowed_sides` and `allowed_roles` say who may perform it (an

empty role list means any role of an allowed side), `required_fields` lists data that must be present and `requires_reason` forces a reason.

`workflow_effects` and `transition_effects`

`workflow_effects` is a registry of effect handlers that exist in code, keyed by `code`. It is synced from the in-memory registry on every start. `is_financial` marks effects that validate money themselves; `is_available` is false when a handler was removed from code but old versions still mention it. `transition_effects` lists which effects a transition runs, in `seq` order (unique per transition), with optional JSON `params`; its primary key is the pair (transition, effect code). Effects run inside the transition's transaction.

26.3 Common tables

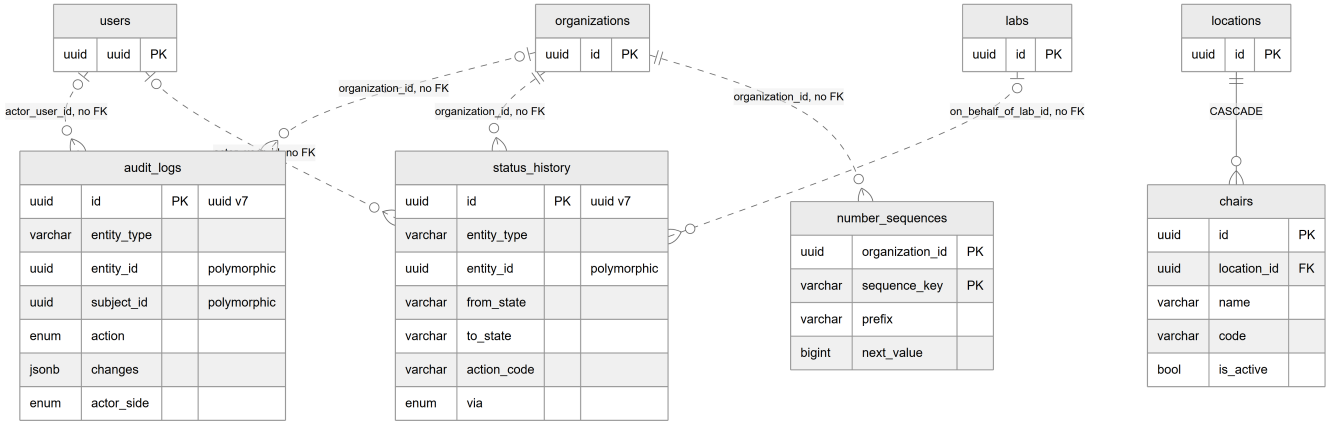

Figure 26.2 — The audit log, status history, number sequences and the unused chairs table.

The figure shows four main boxes. `audit_logs` receives dotted lines from `organizations` (organization_id) and `users` (actor_user_id). `status_history` receives dotted lines from `organizations`, `users` and `labs` (on_behalf_of_lab_id). `number_sequences` receives a dotted line from `organizations`. All dotted lines mean "no FK". `chairs` is the only box with a solid line, from `locations`, labelled CASCADE. `audit_logs` and `status_history` both show `entity_type` and an `entity_id` marked "polymorphic".

`audit_logs` — the generic audit trail

Status: [Implemented]. Used by about eighteen modules, including patients, appointments, staff, payroll, organizations, the engine and every lab module.

One row records who changed what. `entity_type` and `entity_id` name the changed record. `subject_id` names the record whose history this row joins when that is different, for example a patient for a change to the patient's vitals. `action` is create, update, void, delete, export or view_sensitive. `changes` holds `{ before, after }` for the changed fields only. `actor_user_id` and `actor_side` (clinic, lab, platform or system) say who did it, and `ip` and `request_id` record where it came from. `organization_id` is empty for platform-level records, such as a lab edited by a super admin.

Append-only trigger. The hand-written migration `1792490000000-lab-management-foundation` installs `trg_audit_logs_append_only`, which calls `reject_append_only_mutation()` before any UPDATE or DELETE and raises an error. Rows can only be inserted.

`status_history` — every status change

Status: [Implemented].

One row records one status change: `from_state` (empty for the creating transition), `to_state`, `action_code`, who did it (`actor_user_id`, `actor_side`), how (`via`: app, portal, magic_link, proxy or qr), an optional `reason` and `metadata`. `on_behalf_of_lab_id` is set when clinic staff updated something on the lab's behalf (`via = proxy`). States are free text, because workflow states are data, not code enums. The same migration installs `trg_status_history_append_only`, so rows cannot be updated or deleted.

`number_sequences` — human-readable numbers per organization

Status: [Implemented].

One row is a gapless counter for one organization and one `sequence_key`, with a `prefix` and `next_value` (CHECK: above zero). The primary key is the pair (organization, sequence key), supplied by the application. It is only touched through the numbering service inside a transaction, which increments the counter in a single statement. It produces numbers such as LAB-0001 for lab orders and SHP-0001 for shipments, and also numbers patient lab invoices, lab receipts, credit notes, lab refunds, vendor bills and vendor payments. Clinic invoice, receipt and refund numbers use a different mechanism, `document_sequences` (the Billing ledgers chapter, Chapter 21).

`chairs` — dental chairs

Status: [Not implemented]. Entity only.

The table would list the treatment chairs of a location: `name`, `code`, `description` and `is_active`. There is no module, service, controller or screen, and no module registers the entity, so the running application never creates the table in development. A production schema built with the command-line schema tools may contain it. An early link from appointments to chairs no longer exists.

26.4 Relationships explained

Engine:

- **Specialty → organization specialties, form schemas, stage templates, workflow templates.** Each belongs to exactly one specialty; a specialty in use cannot be deleted (RESTRICT).
- **Organization → organization specialties.** Deleting the organization deletes its specialty switches (CASCADE).
- **Organization → form schemas, stage templates, workflow templates.** Each belongs to zero or one organization (empty = platform default). An organization with its own definitions cannot be deleted (RESTRICT).
- **Form schema → versions.** Each version belongs to exactly one form; a form with versions cannot be deleted (RESTRICT).
- **Stage template → steps.** Each step belongs to exactly one template; deleting the template deletes its steps (CASCADE). Lab stage rows that point at a step block that delete in practice (RESTRICT from `order_item_stages`).
- **Workflow template → versions.** RESTRICT.
- **Workflow version → states and transitions.** Each belongs to exactly one version (RESTRICT).
- **State → transitions.** Each transition has exactly one from-state and one to-state (RESTRICT on both).
- **Transition → transition effects** and **effect → transition effects.** Each transition effect belongs to exactly one transition and one effect (RESTRICT on both).
- **Links from the lab tables.** Lab services point at form schemas, workflow templates and stage templates; lab orders, items and stages pin workflow versions; items and form responses pin form versions. All are RESTRICT

(the Labs chapter, Chapter 25, lists them).

Common:

- **Location → chairs.** Each chair belongs to one location; deleting the location deletes its chairs (CASCADE).
- `audit_logs` and `status_history` have no foreign keys at all. `entity_id` and `subject_id` are polymorphic; `organization_id`, `actor_user_id` and `on_behalf_of_lab_id` are logical references. This is deliberate for an audit table: history must survive even if the record it describes is gone.
- `number_sequences.organization_id` has no foreign key.
- Actor columns in the engine (`created_by`, `updated_by`, `published_by`) hold a user id with no foreign key.

26.5 Tenant scope

- **Global, platform catalog:** `specialties`, `workflow_effects`.
- **Platform or organization:** `form_schemas`, `stage_templates`, `workflow_templates` (empty organization = platform default). Their child tables (`form_schema_versions`, `stage_template_steps`, `workflow_template_versions`, `workflow_states`, `workflow_transitions`, `transition_effects`) have no organization column and inherit the parent's scope.
- **Organization only:** `organization_specialties`, `audit_logs` (empty for platform records), `status_history`, `number_sequences`.
- **Location only:** `chairs` (the organization is reachable through `locations`).

Chapter 27 — Overlaps and ambiguities

This chapter collects the places where the Platform's data model is confusing, duplicated or weaker than it looks. It is written for the product owner, who will meet these questions when planning features, writing reports or reviewing a migration. Each item says what it is, why it matters, the risk, and a status label. The facts come from the entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md` , sections 9 and 10) and were checked against the code; the other data-model chapters (Chapters 12 to 26) describe the tables themselves.

27.1 How to read this chapter

Each topic has the same four parts:

- **What it is.** The tables or columns involved.
- **Why it matters.** What goes wrong for a developer, a report writer or a user.
- **Risk.** Low, medium or high, with the reason.
- **Status.** One label. [Implemented] means both sides exist and work as designed; [Partial] means one side is incomplete; [Not implemented] means a planned link was never finished; [Unverified] means the code does not say which answer is right.

27.2 Two prescription tables

What it is. `prescriptions` (prescriptions module) is the current prescription record: draft and finalized states, medicines in their own table, change history, doctor signatures, templates and print layouts. `patient_prescriptions` (patients module) is the older, simpler record: one row with the medicines in a JSON array. The class comment of `prescriptions` says it "coexists with the legacy" table while consumers migrate.

Why it matters. The patient's Clinical tab shows both. The main card lists prescriptions from the new table, and a collapsible "Legacy quick-Rx history" section still reads the old table and still offers a form to add a legacy prescription. A report that counts prescriptions from only one table will be wrong.

Risk. Medium. Data is split, and new legacy rows can still be created.

Status. [Partial]. Migration to the new table is not finished; the legacy write path is still live.

27.3 Four "visit" concepts

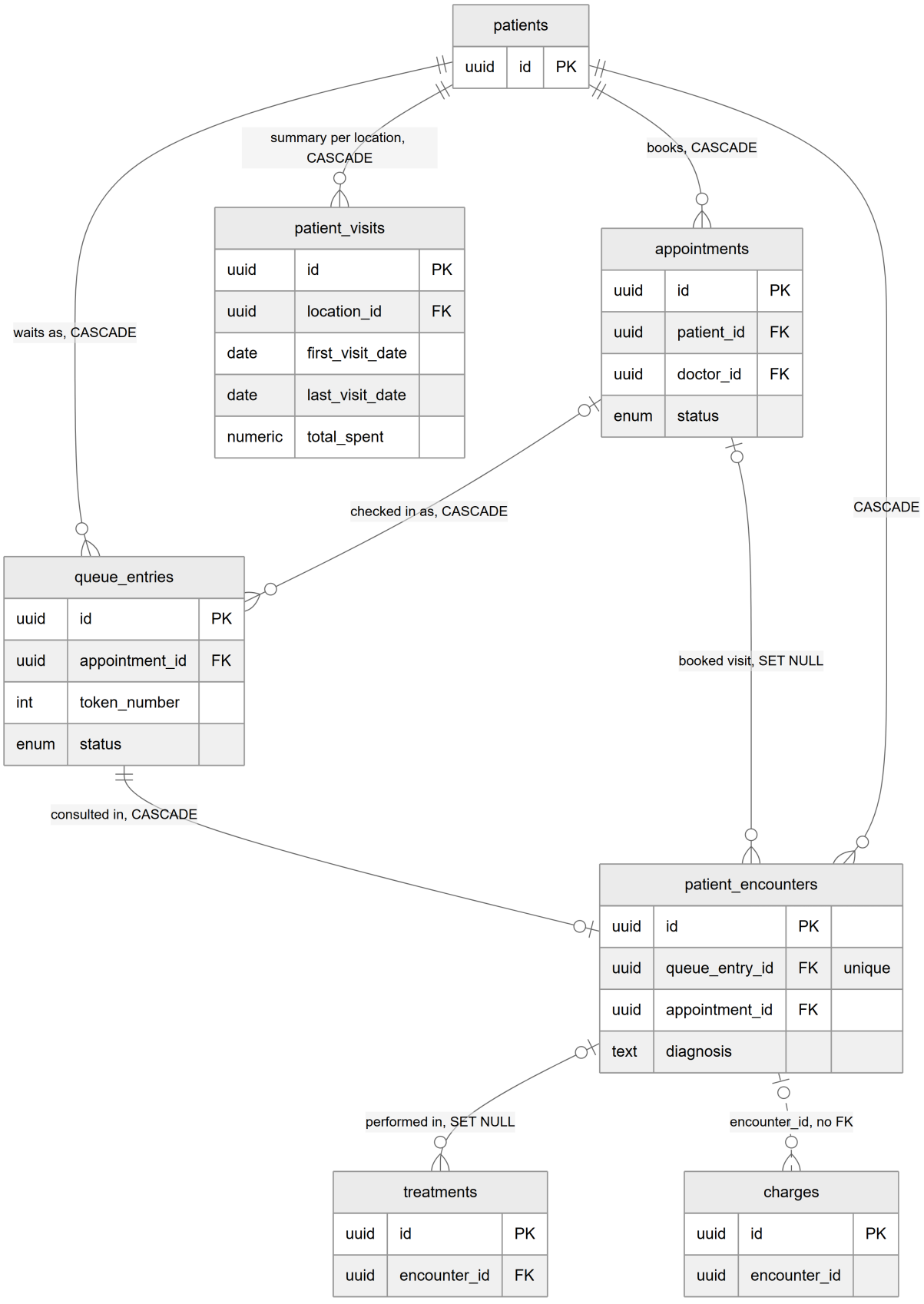

Figure 27.1 — Appointments, queue entries, encounters and the patient visit summary, with treatments and charges.

The figure shows `patients` at the top, connected to `appointments` ("books"), `queue_entries` ("waits as"), `patient_encounters` and `patient_visits` ("summary per location"), all CASCADE. `appointments` connects optionally to `queue_entries` ("checked in as, CASCADE") and to `patient_encounters` ("booked visit, SET NULL"). `queue_entries` connects one-to-one to `patient_encounters` ("consulted in, CASCADE"). `patient_encounters` connects to `treatments` ("performed in, SET NULL") and, by a dotted line with no foreign key, to `charges` (encounter_id). `patient_visits` has no link to any of the other visit tables.

What it is.

- `appointments` : a booking for a future time with a doctor.
- `queue_entries` : a patient's place in one doctor's queue on one day. Created only at check-in or for a walk-in, never at booking.
- `patient_encounters` : the clinical record of one consultation, created when the doctor starts it. Exactly one per queue entry; it points at the appointment for booked visits and has none for walk-ins.
- `patient_visits` : a per-location summary for a patient (first and last visit date, total spent, preferred location).

Why it matters. "How many visits did we have?" has four possible answers. Appointments include no-shows and cancellations; queue entries include walk-ins and people who left; encounters are consultations that actually started. `patient_visits` gives no answer at all: in the code, the patients service only reads it (`GET /patients/:id/visits`), and nothing writes rows into it.

Risk. Medium. Reports and dashboards can disagree depending on which table they count. The visit summary will be empty for every patient.

Status. [Partial]. Appointments, queue entries and encounters are [Implemented]; `patient_visits` is read but never written.

27.4 Treatment-plan table families

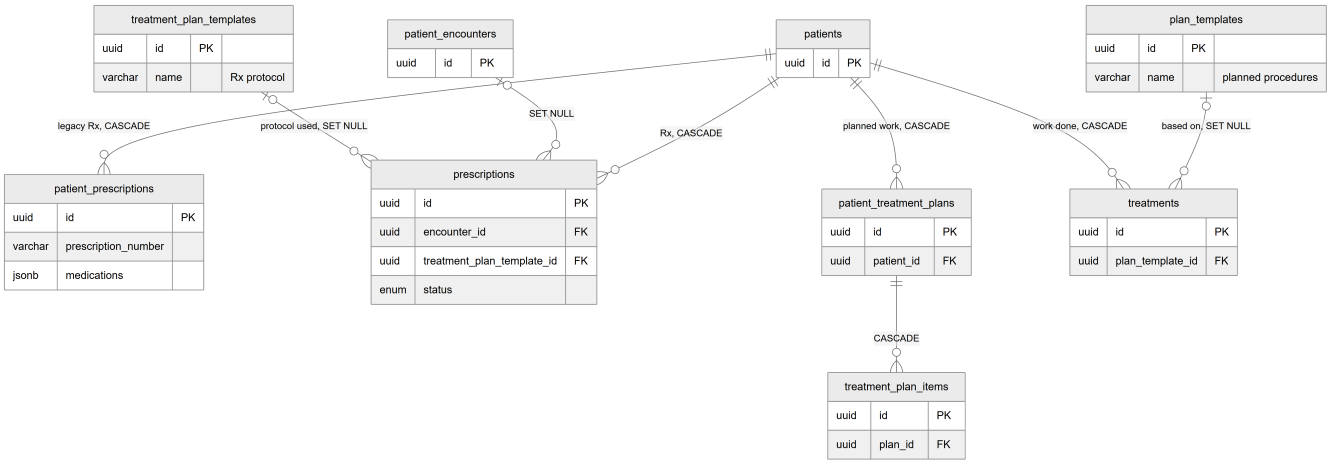

Figure 27.2 — The tables whose names suggest "plan" or "prescription", and what they really hold.

The figure shows `patients` connected to `patient_prescriptions` ("legacy Rx"), `prescriptions` ("Rx"), `patient_treatment_plans` ("planned work") and `treatments` ("work done"), all CASCADE. `patient_treatment_plans` connects to `treatment_plan_items` (CASCADE). `plan_templates` connects to `treatments` ("based on, SET NULL"). `treatment_plan_templates` connects to `prescriptions` ("protocol used, SET NULL"), and `patient_encounters` connects to `prescriptions` (SET NULL). Notes on the template boxes say "Rx protocol" for `treatment_plan_templates` and "planned procedures" for `plan_templates` .

What it is.

- `patient_treatment_plans` with `treatment_plan_items`: planned work for a patient.
- `treatments`: work actually performed, optionally in an encounter.
- `plan_templates`: predefined treatment plans ("Root canal treatment" with its usual steps), copied into a patient's plan.
- `treatment_plan_templates`: despite the name, prescription protocols ("Root canal aftercare"), used by the prescription builder to pre-fill medicines and instructions.

Why it matters. The names invite mistakes. A developer asked to "add a step to the treatment plan template" could easily edit the prescription protocol table. The code comment on `plan_templates` itself warns against confusing the two.

Risk. Low for data, medium for development mistakes.

Status. [Implemented]. All four families work; the issue is naming. In prose the handbook uses **plan template** for `plan_templates` and calls `treatment_plan_templates` "prescription protocol templates".

27.5 Several audit tables

What it is. Change history lives in five places: `audit_logs` (generic, changed fields only), `status_history` (every status change of lab and workflow records), `billing_audit_logs` (billing's own log with full before and after snapshots), `prescription_history` (prescription changes) and `queue_events` (queue actions). The comment on `billing_audit_logs` says it exists because "the repo has no general audit module", which is no longer true.

Why it matters. "Who changed this invoice?" is answered by `billing_audit_logs`; "who changed this patient?" by `audit_logs`. A single audit screen or export must read all five.

Risk. Low. Each log is consistent within its area.

Status. [Implemented]. Consolidation is a design choice not yet made.

27.6 Two ledgers

What it is. `ledger_entries` is the patient statement: one-sided, in rupees. `ledger_journals` with `ledger_journal_lines` is a double-entry general ledger in paisa. Both are described in the Billing ledgers chapter (Chapter 21).

Why it matters. The general ledger is not a complete set of books. In the code, only lab money posts journals: patient lab invoices, lab payments, credit notes, lab refunds, commission on lab lines, vendor bills and vendor payments. Ordinary clinic invoices and payments, and expenses, do not post journals, and no report reads the general ledger.

Risk. Medium. Anyone who builds accounting reports from the journals will miss most clinic revenue.

Status. [Partial]. The patient statement is [Implemented]; the general ledger covers lab money only.

27.7 Two outboxes

What it is. `outbox` (queue module) carries realtime queue events to the Socket.IO relay. `notification_outbox` (notifications module) holds messages for people: in-app today, other channels later.

Why it matters. The names are similar but the purposes are separate. A developer adding a notification should never write to `outbox`, and a realtime screen update should never go through `notification_outbox`.

Risk. Low.

Status. [Implemented]. Both work as designed.

27.8 Two sequence mechanisms

What it is. `document_sequences` (billing) counts clinic invoices, receipts and refunds per organization, location, document type and year, giving numbers like `<location code>-INV-<year>-00001`. `number_sequences` (common) counts per organization and key, giving numbers like LAB-0001 and SHP-0001 for the lab module. The inventory module uses neither: it builds purchase-order numbers by reading the organization's last order.

Why it matters. Numbering rules differ by document: clinic documents restart every year at each location, lab documents run on per organization. The inventory approach reads the last number without a lock, so two orders created at the same moment can get the same number; and its purchase-order number is unique across the whole Platform while it is generated per organization, so a second organization's first purchase order collides with the first organization's (the Inventory chapter, Chapter 23, explains it).

Risk. Low for the two sequence tables; high for inventory purchase-order numbering.

Status. [Implemented] for the two sequence tables. The purchase-order numbering defect is recorded for the owner.

27.9 `services` versus `lab_services`

What it is. `services` (code class `CatalogService`) is the clinic's billable catalog: what the patient is charged for. `lab_services` is a lab's catalog: what the lab makes and charges the clinic for. `service_mappings` connects them. Lab tables call a `services` id `clinic_treatment_id`.

Why it matters. The class name, table name and column name all differ, which makes code search and SQL harder. The clinic price and the lab cost of the same crown live in different tables.

Risk. Low.

Status. [Implemented].

27.10 Logical references with uncertain targets

These columns hold an id but nothing in the code says for certain which table it belongs to:

- `dentist_id` on `patient_teeth_chart`, `patient_treatment_plans`, `patient_recalls` and `patient_feedback`: an employee id or a user id. Only the request formats mention it, as a plain string.
- `doctor_signatures.employee_id`: its code comment says it currently stores the signed-in **user's** id, despite the column name.
- `locations.manager_id`: an employee or a user.
- `patient_consumptions.treatmentPlanId` (inventory): probably a patient treatment plan.
- `notification_outbox.recipient_id` (by `recipient_type`) and `config_values.scope_id` (by `scope`): polymorphic by design.
- `stock_movements.referenceId`: polymorphic and stored as text, not as a UUID.

Why it matters. A join to the wrong table returns nothing or, worse, the wrong person. The employee-versus-user confusion is the most dangerous, because both tables use UUIDs and the mistake is silent.

Risk. Medium.

Status. [Unverified] for the `dentist_id`, `manager_id` and `treatmentPlanId` targets; the `doctor_signatures` case is confirmed by the code comment.

27.11 Foreign keys that look missing

What it is.

- `magic_links.shipment_id` has no foreign key. Its comment says "no FK until that table exists here", but `shipments` now exists. No code creates shipment magic links.
- `lab_order_items.clinic_treatment_id` has no foreign key, while `service_mappings.clinic_treatment_id`, pointing at the same `services` table, has one.
- Almost every billing v2 table, and most lab tables, store `organization_id` and `location_id` without foreign keys, while older modules constrain them. No comment explains the choice.

Why it matters. Without a foreign key the database accepts ids that do not exist, and deletes are not blocked. Tenant isolation in these tables rests entirely on the application.

Risk. Medium. Low today because the application always fills these columns, but a bug or a manual SQL fix can create orphan rows.

Status. [Not implemented] for the shipment link; the others are as designed but undocumented.

27.12 "Append-only" that the database does not enforce

What it is. Class comments call `ledger_entries`, `ledger_journals`, `ledger_journal_lines`, `payment_allocations`, `billing_audit_logs` and `notification_outbox` append-only. Only three tables have a trigger that rejects changes: `commission_entries` (deletes and amount changes refused), `audit_logs` and `status_history` (all updates and deletes refused). A comment on `form_schema_versions` also claims a trigger, but none exists; the service enforces it.

Why it matters. For the unprotected tables, append-only is a promise of the application code only. A manual SQL statement or a future bug can change history without any error.

Risk. Medium for the financial tables.

Status. [Partial]. Enforced in the database for three tables, by the application for the rest.

27.13 Naming inconsistency

What it is. The ten registered inventory tables use camelCase column names (`organizationId`, `currentStock`, `treatmentPlanId`). Every other module, including the legacy `inventory` table, uses snake_case.

Why it matters. In SQL, camelCase names must be quoted ("organizationId"); unquoted names fail. Report writers who move between modules trip over it. In `stock_movements` the tenant columns are also text instead of UUID.

Risk. Low.

Status. [Implemented] as is; renaming would need a migration.

27.14 Employee phone and CNIC are unique across the whole Platform

What it is. `employees` is a person record shared by every clinic the person works for; the link to a clinic is the assignment. Its `phone` and `cnic` columns are unique across the entire Platform, not per organization.

Why it matters. If two clinics enter the same doctor separately, the second insert fails. The design intends the second clinic to reuse the existing person record, but the clinic may not know that record exists, and its contact details are shared with the other clinic.

Risk. Medium. Expect support questions when a doctor works at two unrelated clinics.

Status. [Implemented] by design.

27.15 Primary keys with no default

What it is. These tables have keys that the application must supply: `document_sequences`, `number_sequences`, `organization_specialties`, `transition_effects` (composite keys); `location_billing_settings` (location id), `lab_billing_settings` (organization id), `dental_item_details` and `item_form_responses` (item id), `patient_comm_prefs` (patient id); `workflow_effects` and `notification_events` (a code); and `idempotency_keys`.

Why it matters. Most of these are "one row per parent" tables, where using the parent's id as the key is the point: it makes a second row impossible. An insert that forgets the key fails at once, which is safe.

Risk. Low.

Status. [Implemented] by design.

27.16 Entities the running application does not create

What it is. `chairs` and the legacy `inventory` table have entity files, but no module registers them. The development database, built by the running application, does not contain them; a production database built by the command-line schema tool, which loads every entity file, may contain them.

Why it matters. Development and production schemas can differ by these two tables, which confuses schema comparisons.

Risk. Low.

Status. [Not implemented] for both tables.

Part 5 — Module chapters

Chapter 28 — Patients

The Patients module is the clinic's register of people it treats: who they are, how to reach them, and every clinical and administrative record attached to them. Front-desk staff use it to register and find patients; doctors and assistants use it to read and add clinical records; the owner uses it to import an old register, export data and review who changed what. Almost every other part of the Platform (appointments, the queue, the consultation workspace, prescriptions, billing, labs) points at a patient record from this module.

28.1 What this module is for

A **patient** is a person registered with one **organization** (one clinic business). The patient record belongs to the whole organization, not to one **location**: a patient registered at the Main location can be booked at the Second location without registering again. Each patient gets a patient number such as `P-00042` that is unique inside the organization.

The module covers:

- registration, editing, activation and deactivation, and permanent deletion;
- the patient list with card and list views, search and filters;
- bulk import from an Excel or CSV file, and export of the list to CSV;
- the patient detail page with its tabs: Overview, Appointments, Visits, Treatments, Vitals & Dental, Teeth Chart, Clinical & Rx, Consents, Billing and Files;
- clinical sub-records: dental info, teeth charts (FDI notation), vitals, consents with a drawn signature, uploaded documents, and the legacy quick prescription;
- CRM-style sub-records (communications log, recalls, referrals, feedback, insurance), which exist in the backend but have no screen today;
- a per-patient change history built from the audit log.

Who uses it (system roles, by default): the Receptionist registers, edits and finds patients at their location; the Doctor reads and adds clinical records for their own patients; the Assistant reads clinical records and edits teeth charts at their location; the Admin can do everything, including delete, export and change history; the Accountant has no patient-list access by default and sees patients only through billing screens.

Sidebar: the entry **Patients** (route `/patients`) sits in the "Clinic" section of the default sidebar and is shown to anyone holding `patients.view`. The detail page `/patients/$patientId` has no sidebar entry; it is reached from the list, from search pickers in other modules, or by link. The old route `/patients/new` only redirects to `/patients`, because registration happens in a dialog on the list page.

28.2 Screens

Figure 28.1 — Patient list (screenshot pending).

The patient list page has the title "Patients" with a subtitle giving the total count, for example "128 patients total". The top-right corner holds a card/table view toggle, an **Export** button (only for holders of `patients.export`), an **Import** button and a filled **Create** button (both only for holders of `patients.create`). Under the header is a filter row with a search box ("Search by name or phone...", debounced by 300 ms), a **Gender** select, a **Status** select (Active / Inactive) and a reset control. In table view the columns are Patient (initials avatar, name as a link, patient number underneath), Phone, Age, Gender, Blood, City, Status and an action column with a "View patient" eye button, an "Edit patient" pencil, an activate/deactivate switch and a red "Delete patient permanently" bin. The table has page-size and previous/next controls. Card view shows the same patients as cards in a responsive grid with the same actions. The page shows a loading skeleton while it loads, an error state with a retry button on failure, "No patients yet" with a **New Patient** button when the clinic has no patients, and a separate "no results" state with a clear-filters button when filters hide every patient.

Screenshot pending

New patient dialog

`C:/Program Files/Git/patients`

Figure 28.2 — New patient dialog (screenshot pending).

Pressing **Create** opens a wide dialog with a step indicator. Step 1, "Basic Info" (Personal, Guardian, Emergency & Medical), holds Full Name and Phone Number (both required), WhatsApp, Gender, Blood Group, Marital Status, Date of Birth, CNIC, Email, City, Address, a "minor" switch that reveals Guardian name, phone and relation, the emergency contact (Name, Phone, Relation), General Medical History and Known Allergies. Step 2, "Clinic Info" (Source & Clinic Notes), holds Source, Referral Notes, Preferred Language, Communication Preference, Occupation, "Patient Tags (VIP, Anxious, etc.)" and Internal Clinic Notes. The footer has **Cancel** / **Back**, **Next**, and on the last step **Add Patient**. Problems found when the user presses Next are listed at the top of the step. After a successful save the dialog moves to a third step, "Book Appointment", where the user can pick a doctor and time and press **Book Appointment**, or press **Skip**.

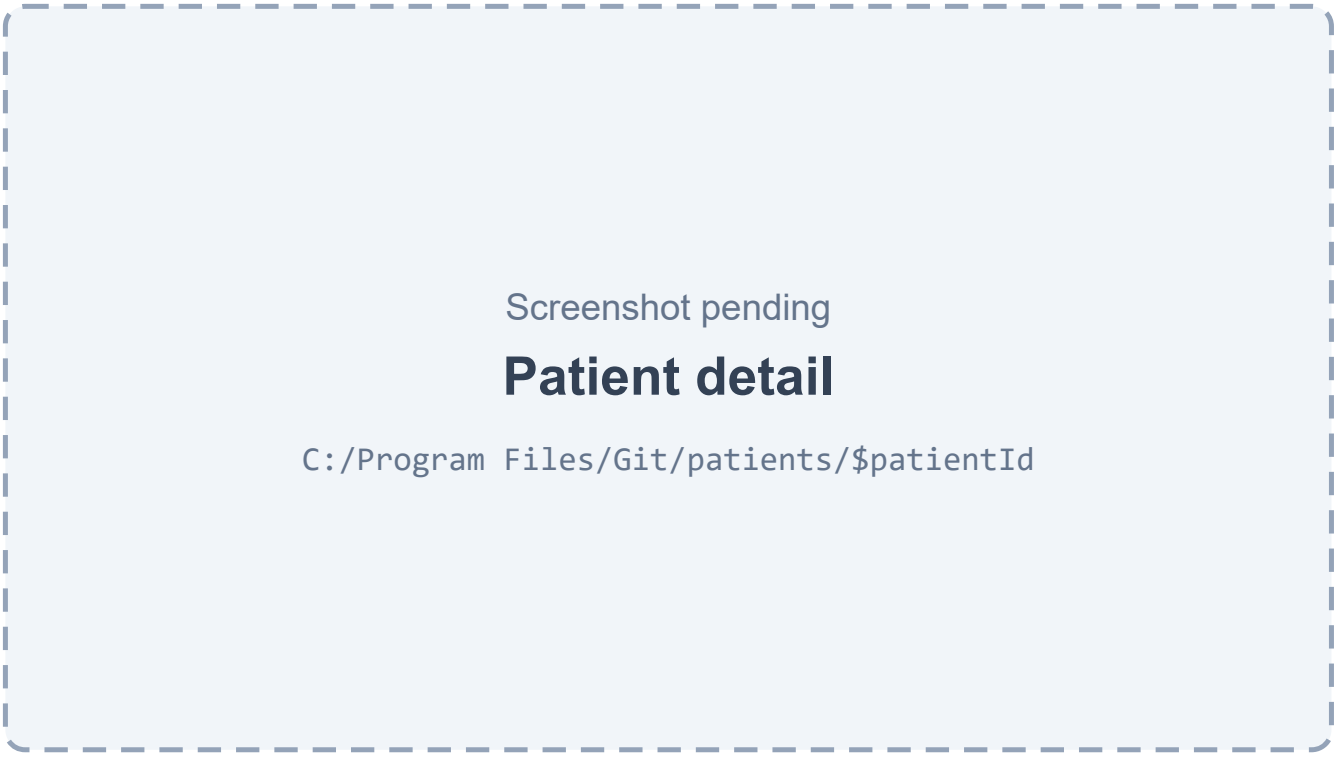

Figure 28.3 — Patient detail page with its tabs (screenshot pending).

The detail page header shows a back button ("Back to patients"), an initials avatar, the patient's name, an Active or Inactive badge, the patient number, an outstanding-dues badge (for holders of `billing.invoice.view`), age, gender, blood group, phone and email. A clock-style "Change history" button appears for holders of `admin.audit.view`. Under the header is a row of tabs; each tab appears only when the viewer holds the permission (and the feature) its data needs: **Overview** (always), **Appointments** (`appointments.view`), **Visits**, **Treatments**, **Vitals & Dental** and **Consents** (`patients.clinical.view`), **Teeth Chart** (`patients.clinical.view` and the `patients.teeth_chart` feature), **Clinical & Rx** (`patients.clinical.view` and the `patients.prescriptions` feature), **Billing** (`billing.invoice.view`) and **Files** (`patients.documents.manage`). The Overview tab has an **Edit Details** button (for `patients.edit`), the latest vitals card, "Medical Quick Access" (medical history and allergy chips), "Alerts & Tags" (tags plus a "Minor Patient" or "Blacklisted Patient" warning), "Emergency Contact" and "Demographics & Internal".

Screenshot pending

Teeth Chart tab

`C:/Program Files/Git/patients/$patientId`

Figure 28.4 — Teeth Chart tab (screenshot pending).

The Teeth Chart tab is titled "FDI Teeth Charting". It shows two rows of drawn teeth numbered in FDI two-digit notation (upper 18 to 28, lower 48 to 38, as seen facing the patient) and a "Chart History" side list of saved snapshots, with the first one marked as the initial chart. Holders of `patients.chart.edit` see **New Chart**, which opens the "Interactive FDI Chart": the user picks a condition (Healthy, Caries, Filled, Crown, Extraction, Missing), clicks teeth to mark them, can add a note per tooth, sees a "N teeth marked" counter with **Clear all**, writes chart notes and presses **Save Immutable Snapshot**. Clicking a saved snapshot opens a read-only viewer with a legend of the conditions it contains.

Screenshot pending

Import patients

`C:/Program Files/Git/patients`

Figure 28.5 — Import patients side panel (screenshot pending).

The **Import** button opens a side panel with five steps shown as a step indicator: Upload, Map columns, Check, Import and Summary. Upload accepts one `.xlsx` or `.csv` file and offers **Download sample template**. Map columns shows a table with "Column in your file", "Example value" and "Import as", plus a preview of the first rows; the **Check N rows** button stays disabled until Full name (or First name) and Phone are mapped. Check shows counts of rows that will be created, duplicates and errors, two duplicate-rule cards ("Skip the row" and "Create anyway"), and a table of "Rows that will not be imported" with row number, status and reason; the **Import N patients** button starts the real import. Import shows a progress bar. Summary shows created, skipped-as-duplicate and failed counts, a **Download error report** button when any row failed, and **Done**.

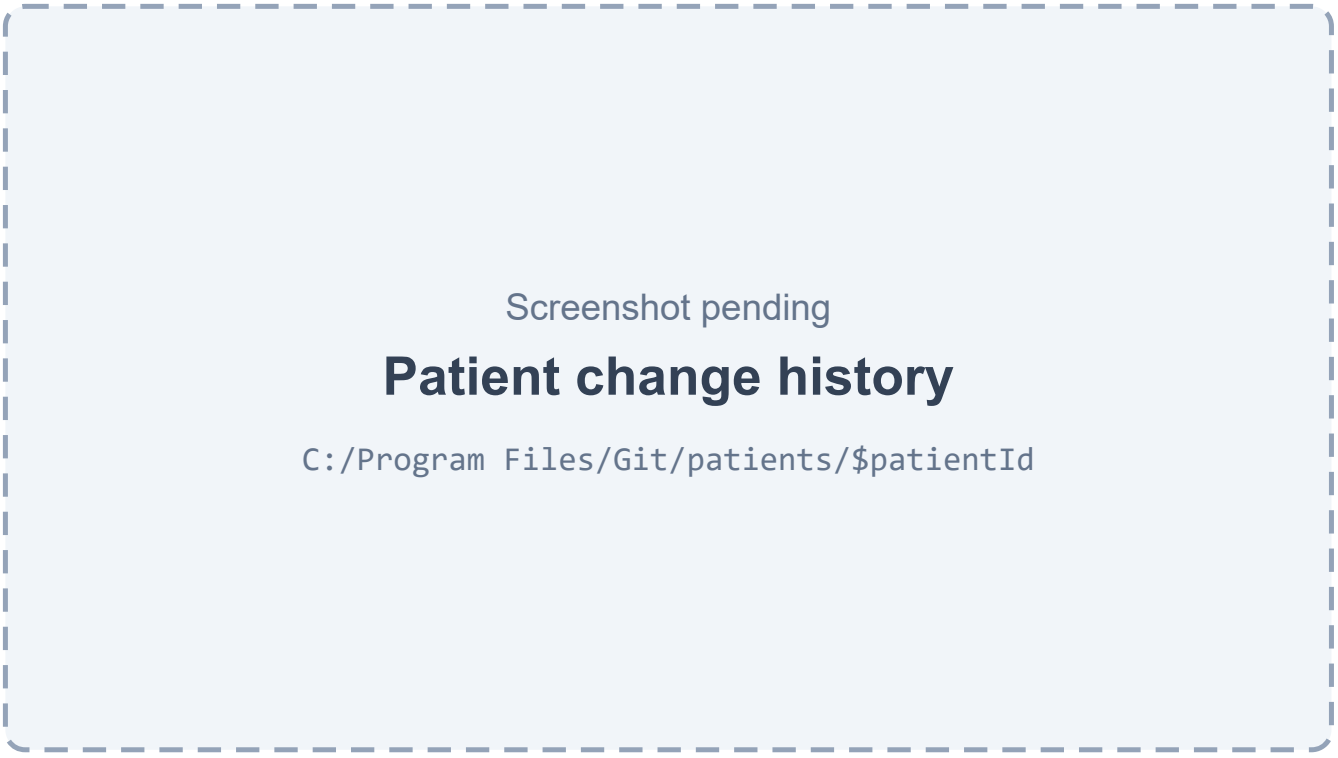

Figure 28.6 — Patient change history side panel (screenshot pending).

The change-history panel is titled " — change history" with the line "Every edit to the patient and their records, with who made it." It lists events newest first: who acted, when, what kind of record changed (patient, vitals, consent, document and so on), and field-by-field before and after values. Events on records the reader cannot open still appear, but without their values.

28.3 Features

28.3.1 Register a patient

Status: [Implemented]

What it does. Creates a new patient in the organization, gives them the next patient number and, optionally, books their first appointment straight away.

How the user does it.

1. On `/patients`, press **Create**.
2. Fill step "Basic Info": Full Name and Phone Number are required; for a minor, turn on the minor switch and enter the guardian.
3. Press **Next**, fill "Clinic Info", and press **Add Patient**.
4. On the "Book Appointment" step, pick a doctor and a time and press **Book Appointment**, or press **Skip**.

Behind the scenes — in simple terms. The Platform checks that no other patient in the clinic has the same phone number, counts the clinic's patients to make the next number, saves the patient with the current location as their home location, and writes a "created" entry in the audit log.

Behind the scenes — technically. `POST /patients` → `PatientsService.create`. It calls `validatePhoneUnique` (exact match of `phone` inside the organization), then `generatePatientNumber` (`P-` plus the count of the organization's patients plus one, padded to five digits). `primaryLocationId` defaults to the active location; `createdBy` is the caller's user id. The insert and the audit row (`recordPatientChange`, action `create`) run in one database transaction. While the user moves between steps, the web app keeps a draft of the form in the patient store so going back does not lose input. The appointment step calls the Appointments module (the Appointments chapter, Chapter 29, covers booking).

Rules and validations.

- Full name: required, at most 150 characters. Phone: required, at most 20 characters; the form also checks 7–20 digits and the characters `+ - ( )` and space.
- Phone must be unique inside the organization (server rule, exact text match).
- Email must be a valid address; CNIC must look like `12345-0000000-0` (form rule); date of birth cannot be in the future (form rule).
- A minor needs a guardian name (form rule).
- Gender, blood group, marital status, language, communication preference and source must be values from fixed lists. Defaults: language Urdu, communication preference WhatsApp, source walk-in.
- Tags are free text chosen in a multi-select that can create new tags.

Who can do it. `patients.create` (no scope; implies `patients.view`), feature `patients.core`. Default holders: Admin, Receptionist.

Data read and written. Reads `patients`; writes `patients` and `audit_logs`.

Effects on other modules. The new patient can be booked, checked in to the queue, billed and searched from every patient picker.

Error cases.

- "A patient with this phone number already exists in this organization" (409) → another patient already has this exact phone.
- "Full name is required" / "Phone number is required" (400) → missing fields.
- The form's own messages (for example "Phone Number looks invalid (7–20 digits)") → shown at the top of the step before anything is sent.
- "Failed to create patient" toast → any other server error.

Technical references: `backend/src/patients/patients.service.ts` · `PatientsService.create`, `CreatePatientDto` · `web-app/src/features/patients/components/CreatePatientModal.tsx`, `PatientForm.tsx`, `AddAppointmentStep.tsx`

28.3.2 Check a phone number before saving

Status: [Partial] — the endpoint and a web hook exist, but no screen calls them; the duplicate is caught only when the user saves.

What it does. Answers "does another patient in this clinic already have this phone number?" without saving anything.

How the user does it. No screen uses it. The registration form relies on the server's check at save time, which answers 409 for a duplicate phone.

Behind the scenes — in simple terms. The Platform looks for a patient with the same phone in the organization, optionally ignoring one patient (the one being edited), and answers yes or no.

Behind the scenes — technically. `GET /patients/check-phone/:phone?excludeId=` → `PatientsService.checkPhoneExists` returns `{ exists: boolean }`. The web hook `useCheckPhone` and API function `checkPhoneUnique` are exported but not used by any component.

Rules and validations.

- Exact text match on `phone` inside the organization; no normalisation.
- The answer ignores the caller's patient scope, but only says yes or no.

Who can do it. `patients.view` (any scope), feature `patients.core`. Default holders: Admin, Doctor, Receptionist, Assistant.

Data read and written. Reads `patients`.

Effects on other modules. None.

Error cases. None specific.

Technical references: `backend/src/patients/patients.controller.ts` · `checkPhone`, `checkPhoneExists` · `web-app/src/features/patients/hooks/useCheckPhone.ts`

28.3.3 Browse, search and filter the patient list

Status: [Implemented]

What it does. Shows the patients the viewer is allowed to see, one page at a time, in a table or as cards, with search and filters.

How the user does it.

1. Open **Patients** in the sidebar.
2. Type in the search box to match name, phone or patient number.
3. Narrow with **Gender** and **Status**; clear with the reset control or the "no results" state's clear button.
4. Switch between table and card view with the view toggle; in table view change the page size or page.

Behind the scenes — in simple terms. The list only ever contains patients inside the viewer's reach: everyone in the organization, everyone connected to the current location, or only the doctor's own patients (those with an appointment or queue entry with that doctor).

Behind the scenes — technically. `GET /patients` → `PatientsService.findAll` → shared `listQuery`, which applies `access.applyScope(qb, 'patients.view', patientScope('p.id'))`, then the filters, then sorting, then paging (`page`, `limit`, default 10, maximum 100). Search uses case-insensitive `ILIKE` on `full_name`, `phone` and `patient_number`.

Rules and validations.

- The API also accepts `bloodGroup`, `source`, `city`, `tag`, `isBlacklisted`, `isMinor`, `sortBy` (`createdAt`, `fullName`, `patientNumber`) and `sortOrder`; the screen offers only search, gender and status.
- Default order is newest registration first. Column sorting in table view re-orders the rows of the current page only.
- Card view shows the current page only and has no paging controls.

Who can do it. `patients.view` with scope `own`, `location` or `org`, feature `patients.core`. Default holders: Admin (org), Receptionist (location), Assistant (location), Doctor (own).

Data read and written. Reads `patients` and, for scoped viewers, `appointments`, `queue_entries` and `queue_sessions`.

Effects on other modules. The same endpoint feeds the type-ahead patient pickers of other modules (`usePatientSearch`, two or more characters, 300 ms debounce, eight results).

Error cases. Error state with **Retry** → the list request failed.

Technical references: `backend/src/patients/patients.service.ts` · `findAll`, `listQuery`, `PatientQueryDto` · `web-app/src/features/patients/pages/PatientsPage.tsx`, `PatientTable.tsx`, `PatientCard.tsx`, `FiltersBar.tsx`

28.3.4 Open the patient detail page

Status: [Implemented]

What it does. Shows one patient's profile and the tabs of records the viewer may open.

How the user does it.

1. Click a patient's name or the "View patient" eye button in the list.
2. Pick a tab. Use the ← button to go back to where you came from.

Behind the scenes — in simple terms. Before answering, the Platform checks that the patient is inside the viewer's reach. A patient outside it is reported as "not found", exactly like a patient of another clinic, so the viewer cannot even learn that the patient exists.

Behind the scenes — technically. `GET /patients/:id` → `PatientsService.findOne`. The controller carries `@PatientInScope('id')`, so `PatientScopeGuard` runs `isPatientInScope` for every route with an `:id` parameter (every sub-record route too) and throws `NotFoundException('Patient not found')` when the patient is outside the `patients.view` scope. Tabs are chosen by `visiblePatientTabs` from the viewer's permissions and features.

Rules and validations.

- A malformed id returns 400 (UUID check).
- A tab appears only when the viewer holds the permission its data needs; for example Teeth Chart needs `patients.clinical.view` and the `patients.teeth_chart` feature, Files needs `patients.documents.manage`.

Who can do it. `patients.view` (scoped), feature `patients.core`; the route declares `staticData.can: 'patients.view'`.

Data read and written. Reads `patients`.

Effects on other modules. The header's dues badge reads the Billing module.

Error cases. "Failed to load patient details. Please try again." → patient missing, out of scope, or request failed.

Technical references: `backend/src/patients/access/patient-scope.ts` · `PatientScopeGuard`, `PatientInScope` · `web-app/src/routes/patients/$patientId.index.tsx`, `features/patients/utils/patientTabs.ts`

28.3.5 Edit patient details

Status: [Implemented]

What it does. Changes any demographic, contact, medical quick-access or clinic field of a patient.

How the user does it.

1. In the list, press the "Edit patient" pencil, or on the detail Overview tab press **Edit Details**.

2. Change fields in the same two-step form, shown in a side sheet.
3. Press **Update Details**.

Behind the scenes — in simple terms. If the phone changed, the Platform re-checks that no other patient has it. It then saves the changes and records exactly which fields changed, from what to what.

Behind the scenes — technically. `PUT /patients/:id` → `PatientsService.update`. A snapshot of the row is taken first; `Object.assign` applies the DTO; `saveAudited` saves the row and the audit row (`update`, with a field diff) in one transaction. An update that changes nothing writes no audit row.

Rules and validations.

- All fields optional; same limits as registration.
- Phone uniqueness is checked only when the phone actually changes, ignoring this patient.
- The DTO also accepts `isActive`, `isBlacklisted` and `blacklistReason`; the form does not send the blacklist fields.

Who can do it. `patients.edit`, feature `patients.core`. Default holders: Admin, Receptionist.

Data read and written. `patients`, `audit_logs`.

Effects on other modules. Name, phone and allergies changes appear immediately in appointments, queue, consultation and billing screens, which read the patient row.

Error cases. 409 phone duplicate (same message as registration); "Patient not found" (404).

Technical references: `backend/src/patients/patients.service.ts` · `update`, `UpdatePatientDto` · `web-app/src/features/patients/components/EditPatientSheet.tsx`

28.3.6 Activate or deactivate a patient

Status: [Implemented]

What it does. Marks a patient inactive (hidden from "Active" filters, but every record kept) or active again. It is the recommended alternative to deletion.

How the user does it.

1. In the list (table or card), flip the switch in the patient's action area. Its tooltip reads "Deactivate patient" or "Activate patient".

Behind the scenes — in simple terms. The Platform flips the active flag and writes a status-change entry saying who moved the patient from active to inactive or back.

Behind the scenes — technically. `PATCH /patients/:id/status` with `{ isActive }` → `PatientsService.setActive`. When the value already matches, nothing is written. Otherwise the save and `AuditService.recordStatus` (table `status_history`, action code `activate` or `deactivate`, subject = the patient) run in one transaction.

Rules and validations.

- Only the `isActive` boolean is accepted.
- Inactive patients are not blocked anywhere else: they can still be booked, checked in and billed. The flag is informational and a list filter.

Who can do it. `patients.edit`, feature `patients.core`. Default holders: Admin, Receptionist.

Data read and written. `patients`, `status_history`.

Effects on other modules. The dashboard's patient count asks for active patients only.

Error cases. "Could not update patient status" toast → request failed.

Technical references: `backend/src/patients/patients.service.ts` · `setActive`, `UpdatePatientStatusDto` · `web-app/src/features/patients/hooks/useSetPatientStatus.ts`

28.3.7 Blacklist a patient

Status: [Partial] — stored, filterable and displayed, but no screen can set it; only an API client can.

What it does. Flags a patient the clinic does not want to serve, with a reason.

How the user does it. No screen sets the flag. When it is set, the Overview tab's "Alerts & Tags" card shows "Blacklisted Patient" and "Reason: …" ("Not specified" when empty).

Behind the scenes — in simple terms. It is a yes/no flag plus a reason text on the patient row. Nothing in booking, the queue or billing reads it.

Behind the scenes — technically. Set through `PUT /patients/:id` (the patient edit endpoint) with `isBlacklisted` and `blacklistReason`, audited as a field change. Listable with `GET /patients?isBlacklisted=true`.

Rules and validations.

- No rule blocks a blacklisted patient from being booked, queued or billed.

Who can do it. `patients.edit`, feature `patients.core`. Default holders: Admin, Receptionist.

Data read and written. `patients` (`is_blacklisted`, `blacklist_reason`), `audit_logs`.

Effects on other modules. None.

Error cases. As for editing.

Technical references: `backend/src/patients/entities/patient.entity.ts` · `isBlacklisted`, `blacklistReason` · `web-app/src/features/patients/components/tabs/OverviewTab.tsx`

28.3.8 Delete a patient permanently

Status: [Partial] — works for patients without billing or lab records; for a patient with charges, invoices, payments, refunds or lab records the database refuses and the user sees a generic failure.

What it does. Removes the patient and every record that hangs off them, for good.

How the user does it.

1. In the list, press the red bin ("Delete patient permanently").
2. Read the dialog "Delete patient permanently?", which says the patient and all records will be deleted and suggests deactivating instead.
3. Press **Delete permanently**.

Behind the scenes — in simple terms. Before deleting, the Platform counts how many linked records of each kind will disappear and stores the patient's last state and those counts in the audit log, so the deletion itself stays on record. Then it deletes the row, and the database deletes the dependent records.

Behind the scenes — technically. `DELETE /patients/:id` → `PatientsService.remove`, in one transaction: `cascadeCounts` reads the Postgres catalog for every foreign key that cascades from `patients` and counts rows per table; `recordPatientChange` writes an audit row with action `delete`, `before` = snapshot plus

`linkedRecords`; then a hard `DELETE`. Cascading tables include appointments, queue entries, encounters, treatments, prescriptions, inventory consumption, patient credits and every `patient_*` sub-record. Tables `charges`, `invoices`, `payments`, `refunds`, `lab_orders`, `lab_invoices`, `lab_payments` and `lab_credits` reference patients with `ON DELETE RESTRICT`, so the delete fails when any of them has a row, and the whole transaction (including the audit row) rolls back.

Rules and validations.

- Always behind a confirmation dialog.
- Hard delete: the entity has a `deleted_at` column but the service does not use soft delete.

Who can do it. `patients.delete` (high-risk permission), feature `patients.core`. Default holder: Admin.

Data read and written. Every table that cascades from `patients`; `audit_logs`.

Effects on other modules. The patient disappears from appointments, the queue history, consultation history and prescriptions. Patient numbering is count-based, so after a deletion the next new patient can be given a number that already exists, and that save then fails.

Error cases.

- "Could not delete patient" toast → the server failed, typically because of billing or lab records (the server answers with the generic "Something went wrong").
- "Patient not found" (404).

Technical references: `backend/src/patients/patients.service.ts` · `remove`, `cascadeCounts` · `web-app/src/features/patients/pages/PatientsPage.tsx` (ConfirmDialog)

28.3.9 Import patients from a spreadsheet

Status: [Implemented]

What it does. Brings an existing patient register (for example from an old system or an Excel sheet) into the Platform: read the file, match its columns to patient fields, check every row without saving, then create the valid rows.

How the user does it.

1. On `/patients`, press **Import**. Optionally press **Download sample template** to get a CSV with the recognised headers (Full Name, Phone, CNIC, Gender, Date of Birth, Age, Email, Address, City, Blood Group, Allergies, Medical History, Notes, Old File No).
2. Upload one `.xlsx` or `.csv` file.
3. In **Map columns**, check the suggested "Import as" field for each column; set unwanted columns to be ignored. Full name (or first name) and Phone must be mapped.
4. Press **Check N rows**. Review the counts and the "Rows that will not be imported" table. Choose **Skip the row** or **Create anyway** for duplicates.
5. Press **Import N patients**. Wait for the progress bar.
6. On **Summary**, press **Download error report** if any row failed, then **Done**.

Behind the scenes — in simple terms. The file is read in memory and never stored. Each row is cleaned up the way a careful receptionist would: extra phone numbers move into notes, an age becomes an estimated birth date, CNICs are reformatted, common spellings of blood groups and genders are understood. Every row is checked against the same rules as manual registration and against existing patients (same phone or CNIC). Each row is then created on its own, so one bad row never undoes the others. Every imported patient is tagged `imported`.

Behind the scenes — technically. Figure 28.7 shows the flow.

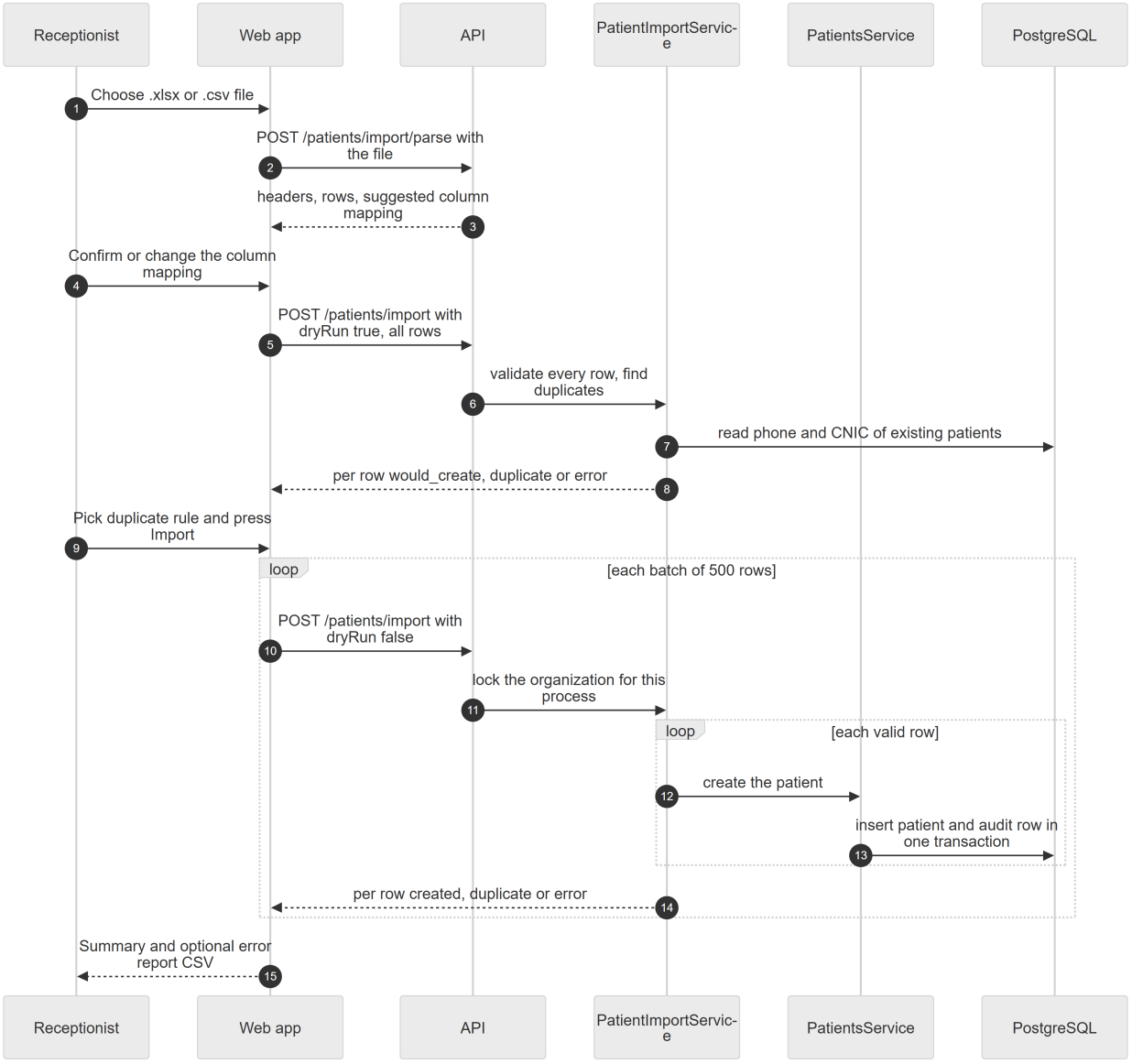

Figure 28.7 — Sequence of a patient import, from file upload to summary.

The diagram has six participants: Receptionist, Web app, API, PatientImportService, PatientsService and PostgreSQL. (1) The receptionist chooses a file. (2) The web app sends it to `POST /patients/import/parse`. (3) The API returns headers, rows and a suggested column mapping; nothing is stored. (4) The receptionist confirms the mapping. (5) The web app sends all mapped rows to `POST /patients/import` with `dryRun: true`. (6) The API hands them to `PatientImportService`, which (7) reads the phone and CNIC of every existing patient in the organization and (8) returns, per row, `would_create`, `duplicate` or `error`. (9) The receptionist picks the duplicate rule and presses Import. A loop then runs once per batch of 500 rows: (10) the web app posts the batch with `dryRun: false`; (11) the service marks the organization as "import running" in this server process; an inner loop runs per valid row, where (12) the service calls the normal patient create and (13) `PatientsService` inserts the patient and its audit row in one transaction; (14) the batch result comes back per row as `created`, `duplicate` or `error`. (15) The web app shows the summary and offers the error-report CSV.

Parsing uses `parseImportFile` (first non-empty worksheet of an `.xlsx`, or RFC-4180 CSV; the first non-empty row is the header; blank headers become "Column N"; repeated headers get "(2)", "(3)"). `suggestMapping` matches header names, including common local spellings, to the fields in `IMPORT_FIELDS`. `validateImportRow`

runs `mapImportRow` and then the `CreatePatientDto` validators. Duplicate detection normalises phones to digits with a leading zero (so `+92…`, `0092…` and `0…` match) and CNICs to digits. "Today" for date-of-birth checks is the location's date (`ClockService`).

Rules and validations.

- File: `.xlsx` (checked by its zip signature) or `.csv` / `.txt`; at most 10 MB; at most 5,000 data rows. Old `.xls` files are rejected with a "Save as .xlsx" hint.
- A dry run accepts up to 5,000 rows in one request; a real import at most 1,000 per request (the web app sends 500).
- Only one real import per organization at a time in one server process.
- Mapping fields: full name, first name, last name, phone, WhatsApp, CNIC, gender, date of birth, age, email, address, city, blood group, marital status, occupation, allergies, medical history, notes, old file number, guardian name and phone, emergency contact name and phone, referred by.
- First phone number becomes the patient's phone; further numbers go to internal notes as "Other phone: …".
- An age (for example "35", "35 years", "6 months") becomes 1 January of the estimated birth year, with a note "Date of birth estimated from age …". Dates accept day-first formats, ISO, month names and Excel serial numbers. A birth date in the future or before 1900 is an error.
- "Old File No" goes into internal notes as "Old file no: …" (the patient has no legacy-number column).
- Source becomes referral when "referred by" is filled, otherwise other.
- Duplicates: same phone or same CNIC as an existing patient or a previous row of the same file. With **Skip the row** the row is reported as a duplicate. With **Create anyway**, a same-CNIC row is created, but a same-phone row is always an error because phones are unique per clinic.

Who can do it. `patients.create`, feature `patients.core`. Default holders: Admin, Receptionist.

Data read and written. Reads `patients`; writes `patients` and `audit_logs` (one create entry per imported patient).

Effects on other modules. Imported patients behave exactly like hand-registered ones.

Error cases.

- "No file provided" → upload without a file.
- "Unsupported file. Upload an Excel (.xlsx) or CSV file — for old .xls files use "Save as" .xlsx first." → wrong type.
- "Could not read the Excel file. Is it a valid .xlsx?", "The workbook has no sheets", "The file is empty", "The file has no patient rows", "The file has more than 5,000 patient rows. Split it into smaller files." → unreadable or oversized file.
- "Send at most 1000 rows per import request" → oversized real-import request.
- "Another patient import is running for this clinic. Try again when it finishes." (409) → parallel import.
- Per-row reasons such as "Phone number "…" looks invalid (7–20 digits)", "CNIC "…" must have 13 digits (e.g. 12345-0000000-0)", "Date of birth cannot be in the future", "Same phone number as P-00012 (…)".
- "The server could not import this batch" → a batch request failed; all its rows are reported as errors and the import continues with the next batch.

Technical references: `backend/src/patients/import/patient-import.service.ts`, `patient-import.ts` · `PatientImportService.import`, `parseImportFile`, `mapImportRow`, `findDuplicate` · `web-app/src/features/patients/components/ImportPatientsSheet.tsx`, `hooks/usePatientImport.ts`, `utils/patientImport.ts`

28.3.10 Export the patient list

Status: [Implemented]

What it does. Downloads the patients matching the current search and filters as a CSV file that opens correctly in Excel, including Urdu names.

How the user does it.

1. On `/patients`, set the search and filters you want.
2. Press **Export**. A toast says "Export downloaded", or "Export downloaded (first 10,000 patients only)" when the result was cut.

Behind the scenes — in simple terms. The export uses the same reach and filters as the list but no paging, stops at 10,000 patients, and is always written to the audit log because patient data is leaving the system.

Behind the scenes — technically. `GET /patients/export` → `PatientsService.exportCsv` (same `listQuery`, `take(10001)` to detect truncation). The controller writes an audit row with `entity_type = 'patient_export'`, action `export`, and `after = { rows, truncated, filters }`, then streams the CSV with headers `Content-Disposition: attachment; filename="patients-YYYY-MM-DD.csv"`, `X-Export-Truncated: true|false` (exposed to the browser) and `Cache-Control: no-store`.

Rules and validations.

- Columns: Patient number, Name, Phone, Email, Gender, Date of birth, Blood group, City, Source, Active, Registered on.
- At most 10,000 rows (`EXPORT_MAX_ROWS`).
- Cells that a spreadsheet would run as a formula get a leading apostrophe; phone-like values such as `+92…` are left as they are.
- The file starts with a UTF-8 byte-order mark.
- The date in the file name and the "Registered on" column are UTC dates, not the location's date.
- The button is disabled when the list is empty.

Who can do it. `patients.export` (high-risk; implies `patients.view`), feature `patients.core` . Default holder: Admin. The rows are limited by the caller's `patients.view` scope.

Data read and written. Reads `patients` ; writes `audit_logs` .

Effects on other modules. None.

Error cases. "Could not export patients" toast → request failed.

Technical references: `backend/src/patients/patients.controller.ts` , `patients.service.ts` · `export` , `exportCsv` , `csvCell` , `EXPORT_MAX_ROWS` · `web-app/src/features/patients/services/patients.api.ts` (`exportPatientsCsv`)

28.3.11 See a patient's appointments and walk-in visits

Status: [Implemented]

What it does. The Appointments tab lists upcoming appointments and a history of past appointments and walk-in queue visits.

How the user does it.

1. Open the patient, then the **Appointments** tab.
2. Read "N upcoming · M past"; press **Book appointment** to open the scheduler.

Behind the scenes — in simple terms. Booked visits come from the Appointments module; walk-ins, which have no appointment, come from the patient's queue history.

Behind the scenes — technically. The tab calls the Appointments list with `patientId` (up to 100) and the Queue module's patient-visits query, keeping only `walk_in` entries. Future appointments that are not completed, cancelled or no-show count as upcoming. The Appointments chapter (Chapter 29) and the Queue chapter (Chapter 30) cover those endpoints.

Rules and validations. None specific to this module.

Who can do it. Tab shown with `appointments.view`. Default holders: Admin, Doctor (own), Receptionist and Assistant (location).

Data read and written. Reads `appointments`, `queue_entries`.

Effects on other modules. None.

Error cases. None specific.

Technical references: `web-app/src/features/patients/components/tabs/AppointmentsTab.tsx` · `useAppointments`, `usePatientVisits`

28.3.12 See visits and full clinical history

Status: [Implemented]

What it does. The Visits tab lists the patient's consultations (encounters), each with its treatments, and opens a "Visit Details" dialog. The same history feeds the patient snapshot in the consultation workspace.

How the user does it.

1. Open the **Visits** tab.
2. Each row shows the date, a summary (diagnosis, complaint or reason) and the treatments; press the "View visit details" arrow to open the visit.

Behind the scenes — in simple terms. The Platform gathers the patient, all their appointments, all their encounters with linked treatments, and treatments that were recorded without a visit. Prices are removed for viewers who may not see them.

Behind the scenes — technically. `GET /patients/:id/history` → `PatientsService.getHistory`: patient, `appointments` (newest first), `EncountersService.findByPatient`, and `TreatmentsService.findByPatient` filtered to `encounterId = null` as `standaloneTreatments`. `HideTreatmentPricesInterceptor` deletes `cost` from every treatment-shaped object when the caller lacks `patients.price.view`. Encounters are created when the doctor starts a consultation (the Consultation chapter, Chapter 31, covers this).

Rules and validations.

- No paging; everything is returned.
- Fees are hidden in the Visits tab when the viewer lacks `patients.price.view`.

Who can do it. `patients.clinical.view`, feature `patients.core`. Default holders: Admin, Doctor, Assistant.

Data read and written. Reads `patients`, `appointments`, `patient_encounters`, `treatments`.

Effects on other modules. None.

Error cases. "Patient not found" (404).

Technical references: `backend/src/patients/patients.service.ts` · `getHistory`, `HideTreatmentPricesInterceptor` · `web-app/src/features/patients/components/tabs/VisitsTab.tsx`, `EncounterDetailDialog.tsx`

28.3.13 Per-location visit statistics

Status: [Not implemented] — the table and a read endpoint exist, but no code ever writes to the table and no screen reads it.

What it does. Was meant to keep, per patient and location, the first and last visit date, the visit count, the total spent and a preferred-location flag.

How the user does it. Not available.

Behind the scenes — in simple terms. The endpoint always returns an empty list because nothing fills the table.

Behind the scenes — technically. `GET /patients/:id/visits` → `PatientsService.getVisits` reads `patient_visits` (unique per patient and location). The web function `fetchVisits` / `useVisits` exists but is not used. The encounter entity notes that `patient_visits` is "left untouched".

Rules and validations. None.

Who can do it. `patients.view`.

Data read and written. Reads `patient_visits`.

Effects on other modules. None.

Error cases. None.

Technical references: `backend/src/patients/entities/patient-visit.entity.ts` · `PatientVisit`, `getVisits` · `web-app/src/features/patients/hooks/usePatientSubEntities.ts` (`useVisits`)

28.3.14 View a patient's change history

Status: [Implemented]

What it does. Shows who created, changed, activated, deactivated or deleted the patient and any of their records, field by field.

How the user does it.

1. On the patient detail header, press the "Change history" button.
2. Read the timeline in the side panel.

Behind the scenes — in simple terms. Every write in this module also writes an audit entry in the same transaction, tagged with the patient as its subject, so one lookup reads the patient's whole history. File links (documents, X-rays, signatures, insurance cards, profile picture) are never copied into the log. When a reader lacks the right to open a kind of record, they still see that the event happened but not the values.

Behind the scenes — technically. `GET /patients/:id/audit-trail` → `PatientsService.getAuditTrail` → `AuditService.subjectTimeline` (merges `audit_logs` and `status_history` where `subject_id` = patient, newest first, at most 200 entries) → `redactPatientTimeline`. Values are removed for dental info, teeth charts, vitals, treatment plans, plan items, legacy prescriptions and consents unless the reader holds `patients.clinical.view`, and for documents unless they hold `patients.documents.manage`; medical history and allergies are removed from patient-row changes for readers without `patients.clinical.view`. What is logged:

Record	`entity_type`	Logged on
Patient	`patient`	create, update (diff), delete (last state and linked-record counts); activate/deactivate in `status_history`
Dental info	`patient_dental_info`	create, update
Teeth chart, vitals, document, consent, communication, referral, feedback, legacy prescription	`patient_teeth_chart`, `patient_vitals`, `patient_document`, `patient_consent`, `patient_communication`, `patient_referral`, `patient_feedback`, `patient_prescription`	create
Treatment plan, plan item, recall, insurance	`patient_treatment_plan`, `treatment_plan_item`, `patient_recall`, `patient_insurance`	create, update
Patient export	`patient_export`	every export (row count, truncated flag, filters); not tied to a patient subject

The table lists each kind of patient record, the `entity_type` value its audit rows carry, and which actions produce a row. Updates that change nothing are not logged. Identity and bookkeeping columns (ids, timestamps, created-by) are left out of every snapshot because the audit row already records who and when.

Rules and validations.

- The button and endpoint need `admin.audit.view` in addition to the patient being in the caller's scope.

Who can do it. `admin.audit.view` (feature `core`, always on). Default holder: Admin.

Data read and written. Reads `audit_logs`, `status_history`.

Effects on other modules. The Audit logging chapter (Chapter 60) describes the shared audit tables.

Error cases. Error state with retry in the panel.

Technical references: `backend/src/patients/patient-audit.ts` · `PATIENT_AUDIT_ENTITY`, `recordPatientChange`, `redactPatientTimeline` · `web-app/src/features/patients/components/PatientAuditSheet.tsx`

28.3.15 Record dental info

Status: [Implemented]

What it does. Keeps one "Dental Health & Habits" sheet per patient: last check-up, gum condition, sensitivity, brushing and flossing frequency, smoking, tobacco chewing, orthodontic history, prosthetics, previous extractions and surgeries, X-ray notes and dentist notes.

How the user does it.

1. Open **Vitals & Dental**.
2. Fill the "Dental Health & Habits" form (Gum Condition, Brushing Frequency, Flossing Frequency, checkboxes such as Teeth Sensitivity, Smoker, Tobacco Chewing, and notes fields).
3. Press **Save Dental Info** (first time) or **Update Dental Info**.

Behind the scenes — in simple terms. There is at most one sheet per patient; saving again overwrites it and the audit log keeps the old values.

Behind the scenes — technically. `GET /patients/:id/dental-info`; `POST /patients/:id/dental-info` → `upsertDentalInfo` (find, then update or create) → `saveAudited`.

Rules and validations.

- One row per patient (unique `patient_id`).
- Gum condition: healthy, mild gingivitis, moderate gingivitis, severe periodontitis.
- The form is read-only without `patients.clinical.edit`.

Who can do it. Read `patients.clinical.view`; write `patients.clinical.edit`; feature `patients.core`. Default writers: Admin, Doctor.

Data read and written. `patient_dental_info`, `audit_logs`.

Effects on other modules. None.

Error cases. "Patient not found" (404).

Technical references: `backend/src/patients/patients.service.ts` · `upsertDentalInfo`, `UpsertDentalInfoDto` · `web-app/src/features/patients/components/tabs/VitalsAndDentalTab.tsx`

28.3.16 Record vitals

Status: [Implemented]

What it does. Adds a dated set of vitals: blood pressure, pulse, temperature, SpO_2, weight, height and an automatically calculated BMI.

How the user does it.

1. Open **Vitals & Dental**.
2. Under "Record New Vitals", enter the values and notes.
3. Press **Save Vitals**. The new card appears in "Patient Vitals History", newest first.

Behind the scenes — in simple terms. Vitals are never edited; each save adds a new reading. The latest reading is shown on the Overview tab and in the consultation workspace.

Behind the scenes — technically. `GET /patients/:id/vitals` (newest first); `POST /patients/:id/vitals` → `createVitals`. BMI = weight ÷ (height in metres)^2, rounded to two decimals, when both are given. `recordedBy` is sent by the web app as the signed-in user's id; `locationId` is the active location.

Rules and validations.

- Form ranges: systolic 50–300, diastolic 30–200, pulse 20–250, temperature 30–45 °C, SpO_2 50–100 %, weight 1–500 kg, height 20–300 cm.
- The API also accepts flags (diabetic, hypertensive, pregnant, on blood thinners, on antibiotics) and anaesthesia details; the form does not show them.
- No update or delete endpoint.

Who can do it. Read `patients.clinical.view`; write `patients.clinical.edit`; feature `patients.core`. Default writers: Admin, Doctor.

Data read and written. `patient_vitals`, `audit_logs`.

Effects on other modules. The consultation workspace shows the latest vitals (Chapter 31).

Error cases. Field messages from the range checks; "Patient not found" (404).

Technical references: `backend/src/patients/patients.service.ts` · `createVitals`, `CreateVitalsDto` · `web-app/src/features/patients/components/tabs/VitalsAndDentalTab.tsx`

28.3.17 Chart teeth (FDI)

Status: [Implemented]

What it does. Saves a snapshot of the condition of each tooth, in FDI notation, and keeps every previous snapshot.

How the user does it.

1. Open **Teeth Chart** and press **New Chart**.
2. Pick a condition, click teeth to mark them (click again with the same condition to unmark), add tooth notes and chart notes.
3. Press **Save Immutable Snapshot**. Pick any entry in "Chart History" to view it.

Behind the scenes — in simple terms. Charts are append-only: a correction is a new snapshot, never an edit. The first chart of a patient is marked as the initial chart.

Behind the scenes — technically. `GET /patients/:id/teeth-charts` (newest first); `POST /patients/:id/teeth-charts` → `createTeethChart`. `chart_data` is JSON keyed by tooth number with `{ condition, note }`; `is_initial_chart` defaults to true when the patient has no chart yet. The web app sends the signed-in user's id as `dentistId`. A `periodontal_chart` JSON column exists but the screen does not fill it.

Rules and validations.

- `chartData` is required; there is no update or delete endpoint.
- Conditions offered by the screen: Healthy, Caries, Filled, Crown, Extraction, Missing.

Who can do it. Read `patients.clinical.view` plus feature `patients.teeth_chart`; write `patients.chart.edit` (feature `patients.teeth_chart`). Default writers: Admin, Doctor, Assistant. The feature is off by default in the catalog (dental and orthodontic clinics switch it on).

Data read and written. `patient_teeth_chart`, `audit_logs`.

Effects on other modules. None.

Error cases. Feature-off error → the organization does not have `patients.teeth_chart` active (the tab is hidden in that case).

Technical references: `backend/src/patients/patients.service.ts` · `createTeethChart`, `CreateTeethChartDto` · `web-app/src/features/patients/components/tabs/TeethChartTab.tsx`

28.3.18 Record a treatment from the patient page

Status: [Implemented]

What it does. The Treatments tab lists all treatments performed (linked to a visit or standalone) and lets a user record one, optionally starting from a plan template.

How the user does it.

1. Open **Treatments** and press **Add Treatment**.
2. In "Record Treatment", optionally pick a "Treatment plan" (a plan template) and a "Procedure", which fills the name and cost; enter the date, optional visit link and notes.
3. Press **Save Treatment**.

Behind the scenes — in simple terms. Treatments belong to the Treatments module; this tab is a window on them. The Treatment Plans chapter (Chapter 33) explains how plan templates fill the form.

Behind the scenes — technically. `GET /treatments?patientId=`, `POST /treatments`. The button shows only for viewers with both `patients.clinical.edit` and `patients.price.view`.

Rules and validations. Treatment name and date required; cost cannot be negative.

Who can do it. `patients.clinical.edit` and `patients.price.view`. Default holder with both: Admin.

Data read and written. `treatments`.

Effects on other modules. Treatments can create charges (Chapters 31 and 34).

Error cases. "Treatment plan not found" → the chosen template was deleted.

Technical references: `backend/src/treatments/treatments.service.ts` · `TreatmentsService.create` · `web-app/src/features/patients/components/tabs/TreatmentsTab.tsx`, `TreatmentFormDialog.tsx`

28.3.19 Patient treatment plans

Status: [Partial] — backend only; no screen lists, creates or edits a patient's treatment plan.

What it does. Stores multi-step plans (for example a root canal over several sessions) with items, estimated costs and a status for each patient.

How the user does it. No screen. The Treatment Plans chapter (Chapter 33) describes the endpoints `GET/POST /patients/:id/treatment-plans`, `PATCH …/:planId`, `GET/POST …/:planId/items` and `PATCH …/items/:itemId` in full.

Behind the scenes — in simple terms. Plans and items are stored, numbered `TP-00001` and audited like every other patient record.

Behind the scenes — technically. `PatientsService.createTreatmentPlan`, `updateTreatmentPlan`, `createPlanItem`, `updatePlanItem`, each through `saveAudited`.

Rules and validations. Covered in Chapter 33.

Who can do it. Read `patients.clinical.view` plus feature `patients.treatment_plans`; write `plans.manage`. Default writers: Admin, Doctor.

Data read and written. `patient_treatment_plans`, `treatment_plan_items`, `audit_logs`.

Effects on other modules. Billing can turn plan items into charges (backend only).

Error cases. "Treatment plan not found", "Plan item not found" (404).

Technical references: `backend/src/patients/patients.service.ts` · `createTreatmentPlan` · `web-app/src/features/patients/hooks/usePatientSubEntities.ts` (unused hooks)

28.3.20 Prescriptions tab and the legacy quick prescription

Status: [Implemented]

What it does. The **Clinical & Rx** tab shows the patient's prescriptions from the Prescriptions module and, in a collapsed "Legacy quick-Rx history" section, the older simple prescriptions stored with the patient.

How the user does it.

1. Open **Clinical & Rx**. Press **New Prescription** to open the full prescription builder for this patient.
2. To use the legacy form, open "Legacy quick-Rx history" and press **New Rx**: enter Chief Complaint, Diagnosis, Medications (Name, Dosage, Frequency, Instructions) and General Special Instructions. Saved legacy prescriptions can be printed.

Behind the scenes — in simple terms. Two prescription stores exist side by side. The full one belongs to the Prescriptions module (Chapter 32). The legacy one is a single table with the medicines as a JSON list, numbered `RX-00001` per organization.

Behind the scenes — technically. `GET /patients/:id/prescriptions` (feature `patients.prescriptions`, newest first) and `POST /patients/:id/prescriptions` → `createPrescription`, writing `patient_prescriptions`. The web app sends the signed-in user's id as `prescribedBy`. The number is a count of the organization's legacy prescriptions plus one.

Rules and validations.

- `prescribedBy` and a `medications` list are required.
- No update or delete endpoint.

Who can do it. Read `patients.clinical.view` plus feature `patients.prescriptions`; write `prescriptions.create`. Default writer: Doctor (the Admin template does not include `prescriptions.create`; an owner has every permission).

Data read and written. `patient_prescriptions`, `audit_logs`; the main list reads `prescriptions`.

Effects on other modules. None for the legacy table.

Error cases. "Patient not found" (404).

Technical references: `backend/src/patients/entities/patient-prescription.entity.ts` · `createPrescription` · `web-app/src/features/patients/components/tabs/ClinicalTab.tsx`, `PrescriptionFormDialog.tsx`

28.3.21 Capture a signed consent

Status: [Implemented]

What it does. Records that the patient (or guardian) agreed to a procedure, with the consent text, the signer's name and a hand-drawn signature, and lets staff print it.

How the user does it.

1. Open **Consents** and press **Capture Signature**.
2. In "Capture Consent", choose the Consent Type, edit the Consent Terms, enter the signer's name, pick Signatory Type (Patient or Guardian; Guardian is preselected for a minor), draw the signature on the pad and tick the agreement box.
3. Save. Each saved consent shows Status, Agreed Date, Signed By, Signatory Type, Witnessed By and the signature image, with a print button.

Behind the scenes — in simple terms. Consents cannot be changed or deleted once saved. The drawn signature is stored inside the record as an image, not as a separate upload.

Behind the scenes — technically. `GET /patients/:id/consents`; `POST /patients/:id/consents` → `createConsent`. The signature is a PNG data URL in `signature_url` (text). The web app sends the signed-in user's id as `witnessedBy`. `ip_address` exists but the screen does not send it. The audit snapshot omits `signature_url`.

Rules and validations.

- Consent type, consent text, agreement flag and signer name are required (signer name at most 150 characters).
- The form refuses to save without a signer name ("Signatory name is required.") or without a drawn signature ("Please capture a signature before saving.").

- Consent types: general treatment, extraction, implant, root canal, orthodontic, sedation, photo release.
- No update or delete endpoint.

Who can do it. Read `patients.clinical.view`; write `patients.clinical.edit`; feature `patients.core`. Default writers: Admin, Doctor.

Data read and written. `patient_consents`, `audit_logs`.

Effects on other modules. None.

Error cases. The two form messages listed; "Patient not found" (404).

Technical references: `backend/src/patients/entities/patient-consent.entity.ts` · `createConsent`, `CreateConsentDto` · `web-app/src/features/patients/components/tabs/ConsentsTab.tsx`, `ConsentFormDialog.tsx`

28.3.22 Upload and view documents

Status: [Implemented]

What it does. Stores X-rays, scans, reports, photos and ID copies against the patient, with type, tooth number, date taken and notes.

How the user does it.

1. Open **Files** and press the upload button.
2. In "Upload Document", choose an image or PDF, the Document Type, an optional Tooth #, a Title / Description, Clinical Notes and the other fields, and save.
3. Each document in the list opens its file in a new tab.

Behind the scenes — in simple terms. The file goes first to the Platform's file-storage service (the Cloudinary storage provider), which returns a link; the Platform then saves a document record holding that link.

Behind the scenes — technically. Step one: `POST /uploads` (multipart, optional `folder`) → `UploadsController` → the configured `StorageProvider`. Step two: `POST /patients/:id/documents` → `createDocument` with `fileUrl`, `fileName`, size and MIME type. `GET /patients/:id/documents` lists them newest first.

Rules and validations.

- Allowed file types: JPEG, PNG, GIF, WebP, HEIC and PDF; at most 15 MB.
- Document types: X-ray, CBCT scan, report, prescription, consent form, insurance card, CNIC copy, photo before, photo after, other.
- No update or delete endpoint for documents.

Who can do it. `patients.documents.manage` for both reading and writing (feature `patients.documents`). Default holders: Admin, Doctor, Receptionist. The upload endpoint itself is open to any member of the organization.

Data read and written. `patient_documents`, `audit_logs` (without the file link).

Effects on other modules. None.

Error cases. "Please choose a file to upload."; storage messages such as "Unsupported file type ..." or "File size exceeds the 15MB limit"; "File upload failed" toast.

Technical references: `backend/src/common/storage/uploads.controller.ts` · `createDocument`, `CreateDocumentDto` · `web-app/src/features/patients/components/tabs/FilesTab.tsx`, `DocumentFormDialog.tsx`

28.3.23 Communications log

Status: [Partial] — backend complete; the web component (`CrmTab` with a "log communication" dialog) exists but is not shown on any page.

What it does. Records calls, messages and visits with the patient: type (SMS, WhatsApp, call, email, in person), direction, purpose, subject, body and delivery status.

How the user does it. No reachable screen.

Behind the scenes — in simple terms. It is a manual log; the Platform does not send messages to patients (notifications are in-app only).

Behind the scenes — technically. `GET /patients/:id/communications` , `POST /patients/:id/communications` → `createCommunication` . Columns for automation and external message ids exist but nothing fills them.

Rules and validations. Type and direction required; subject at most 300 characters. No update endpoint, so the status set at creation never changes.

Who can do it. Read `patients.view` ; write `patients.edit` ; feature `patients.core` .

Data read and written. `patient_communications` , `audit_logs` .

Effects on other modules. None.

Error cases. "Patient not found" (404).

Technical references: `backend/src/patients/entities/patient-communication.entity.ts` · `createCommunication` · `web-app/src/features/patients/components/tabs/CrmTab.tsx` , `CommunicationFormDialog.tsx`

28.3.24 Recalls

Status: [Partial] — backend only; no component uses the recall hooks, and no job sends reminders or marks recalls overdue.

What it does. Schedules a future check-up for the patient (six-month check-up, annual cleaning, post-treatment follow-up, implant review, orthodontic review, custom) and tracks it through statuses.

How the user does it. No screen.

Behind the scenes — in simple terms. A recall is a due date plus a status that staff would update by hand. Columns for first, second and third reminder times exist but nothing fills them.

Behind the scenes — technically. `GET /patients/:id/recalls` (by due date), `POST /patients/:id/recalls` → `createRecall` (status `scheduled`), `PATCH /patients/:id/recalls/:recallId` → `updateRecall` (status, linked appointment, notes). Any status can be set at any time; there is no transition map.

Rules and validations. Recall type and due date required; any status can be set at any time.

Who can do it. Read `patients.view` ; write `patients.edit` ; feature `patients.core` .

Data read and written. `patient_recalls` , `audit_logs` .

Effects on other modules. None.

Error cases. "Recall not found" (404).

Technical references: `backend/src/patients/entities/patient-recall.entity.ts` · `createRecall` , `updateRecall` · `web-app/src/features/patients/hooks/usePatientSubEntities.ts` (`useRecalls` , unused)

28.3.25 Referrals

Status: [Partial] — backend create and list only; shown only in the unreachable `CrmTab`.

What it does. Records that this patient referred someone (an existing patient or a name and phone), with reward fields.

How the user does it. No reachable screen.

Behind the scenes — in simple terms. Each referral starts as pending; there is no endpoint to change its status or mark a reward given.

Behind the scenes — technically. `GET /patients/:id/referrals` (where the patient is the referrer), `POST /patients/:id/referrals` → `createReferral`. If the referred patient is deleted, the link is cleared (`ON DELETE SET NULL`).

Rules and validations. All fields optional; referred name at most 150 characters.

Who can do it. Read `patients.view`; write `patients.edit`; feature `patients.core`.

Data read and written. `patient_referrals`, `audit_logs`.

Effects on other modules. None.

Error cases. "Patient not found" (404).

Technical references: `backend/src/patients/entities/patient-referral.entity.ts` · `createReferral` · `web-app/src/features/patients/components/tabs/CrmTab.tsx`

28.3.26 Feedback

Status: [Partial] — backend create and list only; shown only in the unreachable `CrmTab`.

What it does. Stores a 1–5 rating with a category (doctor, service, facility, billing, waiting time, general), text and an anonymous flag.

How the user does it. No reachable screen.

Behind the scenes — in simple terms. Columns for publishing and for a clinic response exist, but no endpoint sets them.

Behind the scenes — technically. `GET /patients/:id/feedback`, `POST /patients/:id/feedback` → `createFeedback`.

Rules and validations. Rating required, whole number 1–5.

Who can do it. Read `patients.view`; write `patients.edit`; feature `patients.core`.

Data read and written. `patient_feedback`, `audit_logs`.

Effects on other modules. None.

Error cases. Rating out of range (400).

Technical references: `backend/src/patients/entities/patient-feedback.entity.ts` · `createFeedback` · `web-app/src/features/patients/components/tabs/CrmTab.tsx`

28.3.27 Insurance

Status: [Partial] — backend only; no component uses the insurance hooks.

What it does. Stores a patient's insurance policies: provider, plan, policy number, holder, coverage percentage, annual maximum, used benefit, validity dates, card images and an active flag.

How the user does it. No screen.

Behind the scenes — in simple terms. Policies are reference information only; billing does not read them and insurance claims are not built.

Behind the scenes — technically. `GET /patients/:id/insurance`, `POST /patients/:id/insurance`, `PATCH /patients/:id/insurance/:insuranceId`, all requiring feature `patients.insurance`.

Rules and validations. Provider name and policy number required; coverage 0–100 %.

Who can do it. Read `patients.view`; write `patients.edit`; feature `patients.insurance` (off by default).

Data read and written. `patient_insurance`, `audit_logs` (card links omitted).

Effects on other modules. None.

Error cases. "Insurance record not found" (404); feature-off error.

Technical references: `backend/src/patients/entities/patient-insurance.entity.ts` · `createInsurance`, `updateInsurance` · `web-app/src/features/patients/hooks/usePatientSubEntities.ts` (`useInsurance`, unused)

28.3.28 Billing tab

Status: [Implemented]

What it does. Shows the patient's money position (summary cards) and four sub-tabs: Open charges, Invoices, Payments and Statement, with collect-payment and, when the patient holds an advance, refund actions.

How the user does it. Open **Billing**. The Billing chapter (Chapter 34) describes every action.

Behind the scenes — in simple terms. This tab is the Billing module embedded in the patient page.

Behind the scenes — technically. `PatientBillingTab` from the billing feature.

Rules and validations. Covered in Chapter 34.

Who can do it. `billing.invoice.view`. Default holders: Admin, Receptionist, Accountant.

Data read and written. Billing tables.

Effects on other modules. Billing.

Error cases. Covered in Chapter 34.

Technical references: `web-app/src/features/billing/components/patient-billing/PatientBillingTab.tsx` · `PatientBillingTab`

28.3.29 Patient portal and the CRM feature key

Status: [Not implemented]

What it does. The feature keys `patients.portal` (patient self-service) and `patients.crm` (referrals, feedback and recalls as a switchable feature) are retired in the feature catalog: their entries are commented out and any existing row is deactivated so they cannot be switched on. The `patients.user_id` column reserved for a portal login is never filled.

How the user does it. Not available.

Behind the scenes — in simple terms. The communications, recall, referral and feedback endpoints are not gated by any CRM feature; they follow `patients.view` and `patients.edit`.

Behind the scenes — technically. `RETIRED_FEATURE_KEYS` in the feature catalog seed.

Rules and validations. None.

Who can do it. Nobody.

Data read and written. None.

Effects on other modules. None.

Error cases. None.

Technical references: `backend/src/feature-flags/feature-catalog.seed.ts` · `RETIRED_FEATURE_KEYS` · none in web-app

28.3.30 Who can see which patients (scope)

Status: [Implemented]

What it does. Limits every patient list, export and patient page to the viewer's reach, so a doctor sees only their own patients and front-desk staff only patients connected to their location.

How the user does it. Nothing to do; the owner sets the scope of `patients.view` per role on the roles screen (Chapter 11).

Behind the scenes — in simple terms. Figure 28.8 summarises the rules.

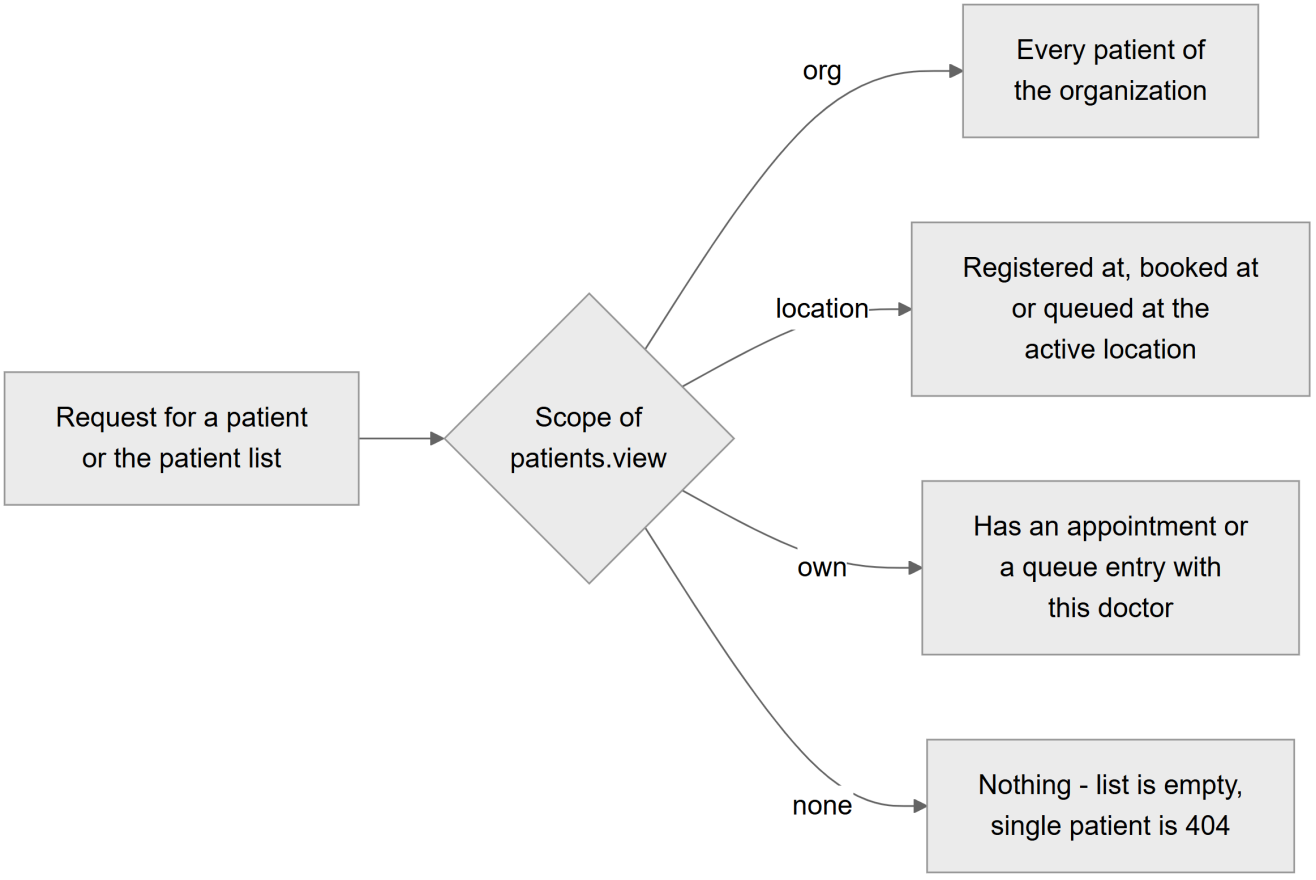

Figure 28.8 — How the scope of `patients.view` decides which patients a user can reach.

The flowchart starts from "Request for a patient or the patient list" and branches on the scope of `patients.view`. With `org`, the user reaches every patient of the organization. With `location`, the user reaches patients registered at the active location (their primary location), booked at it, or queued at it. With `own`, the user reaches patients who have an appointment or a queue entry with this doctor. With no grant, the list is empty and any single patient answers 404.

Behind the scenes — technically. `patientScope()` builds `EXISTS` sub-queries over `appointments`, `queue_entries` / `queue_sessions` and `patients.primary_location_id`; `AccessContext.applyScope` adds them to list and export queries; `PatientScopeGuard` checks single-patient routes. Prices are hidden separately: plan cost columns carry `@VisibleIf('patients.price.view')`, and the history endpoint strips treatment costs.

Rules and validations.

- A doctor with no employee record, or a user with no active location, gets an empty list under `own` or `location`.
- `GET /patients/check-phone` is not scope-limited (it only answers yes or no).

Who can do it. Applies to everyone except the owner and super admin, who hold every permission at `org` scope.

Data read and written. Reads `appointments`, `queue_entries`, `queue_sessions`, `patients`.

Effects on other modules. Other modules that list patients reuse `patientScope` or the patient-scope guard (encounters, treatments, prescriptions and inventory patient consumption).

Error cases. "Patient not found" (404) for an out-of-scope patient.

Technical references: `backend/src/patients/access/patient-scope.ts` · `patientScope`, `isPatientInScope`, `PatientScopeGuard` · `web-app/src/features/patients/utils/patientTabs.ts`

28.4 Entities and data model

The module owns fifteen tables. Two figures split them into clinical records and CRM-style records.

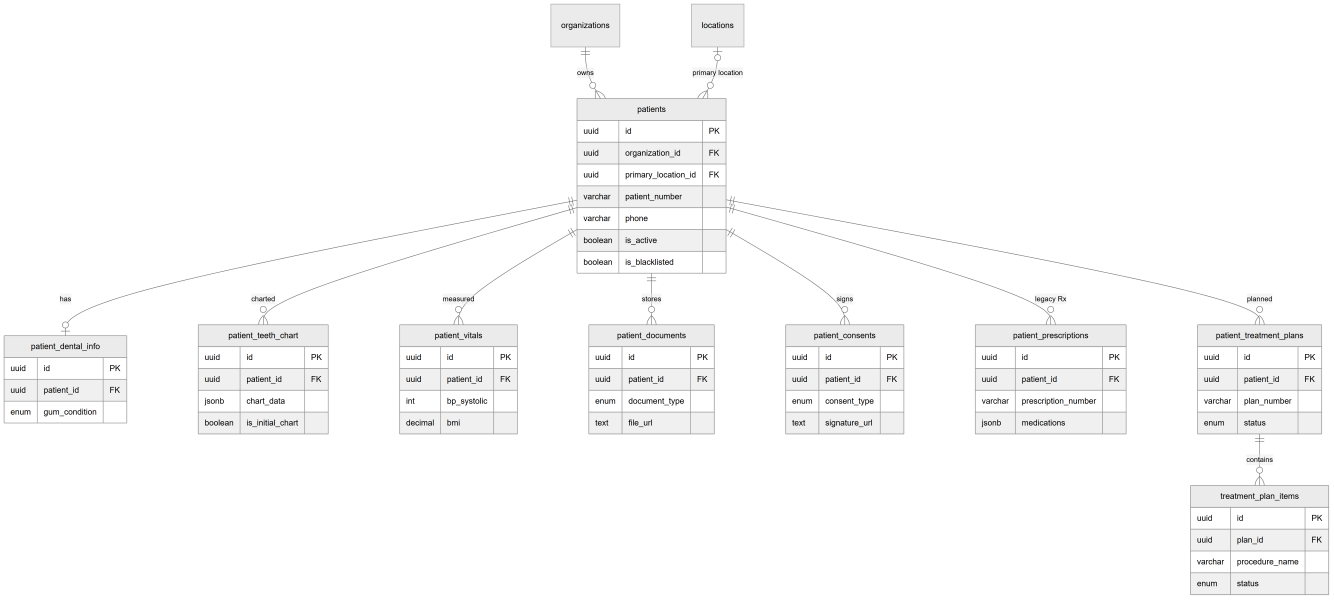

Figure 28.9 — Entity-relationship diagram of the patient and its clinical sub-records.

The diagram puts `patients` in the centre, with `organizations` and `locations` at the top and nine clinical tables along the bottom. Each patient belongs to exactly one organization; deleting the organization deletes its patients (ON DELETE CASCADE). A patient may have one primary location; deleting that location clears the link (ON DELETE SET NULL). A patient has at most one `patient_dental_info` row (unique `patient_id`). A patient has any number of `patient_teeth_chart`, `patient_vitals`, `patient_documents`, `patient_consents`, `patient_prescriptions` (legacy) and `patient_treatment_plans` rows; all of them are deleted with the patient (ON DELETE CASCADE), and most also carry `organization_id` and `location_id` foreign keys that cascade. Each treatment plan has any number of `treatment_plan_items`, deleted with the plan. Key columns shown: `patients.patient_number` (unique per organization), `phone`, `is_active`, `is_blacklisted`; `chart_data` and `is_initial_chart` on charts; `bp_systolic` and `bmi` on vitals; `document_type` and `file_url` on documents; `consent_type` and `signature_url` on consents; `prescription_number` and `medications` (JSON) on legacy prescriptions; `plan_number` and `status` on plans; `procedure_name` and `status` on items. Columns such as `dentist_id`, `recorded_by`, `uploaded_by`, `prescribed_by` and `witnessed_by` hold user or employee ids without a foreign key; the web app fills them with the signed-in user's id.

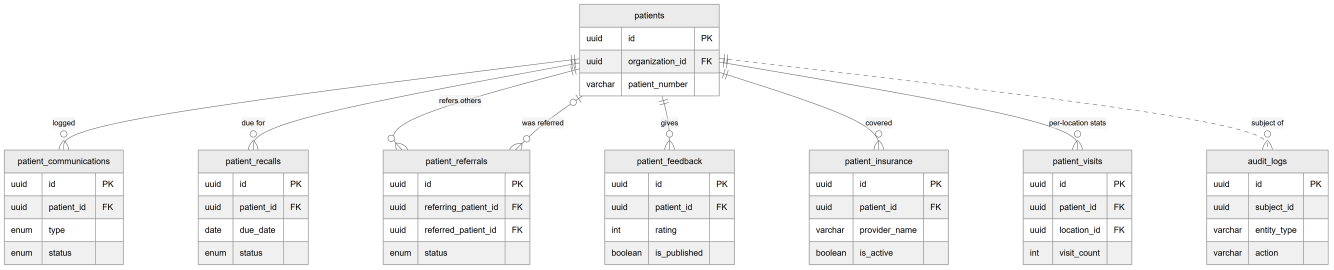

Figure 28.10 — Entity-relationship diagram of the patient's CRM records, visit statistics and audit link.

This diagram shows `patients` with seven related tables. A patient has any number of `patient_communications`, `patient_recalls`, `patient_feedback` and `patient_insurance` rows, all deleted with the patient. `patient_referrals` links to patients twice: `referring_patient_id` (the patient who referred; deleted with them) and an optional `referred_patient_id` (the person referred, if registered; cleared when that patient is deleted). `patient_visits` holds one row per patient and location (unique pair) with `visit_count`; nothing writes to it. The dotted line to `audit_logs` is a logical link without a foreign key: audit rows carry the patient id in `subject_id`, so they survive the patient's deletion.

Patients are also referenced from outside the module. Appointments, queue entries, encounters, treatments, prescriptions, inventory consumption and patient credits cascade on patient deletion; charges, invoices, payments, refunds, lab orders, lab invoices, lab payments and lab credits restrict it.

Money columns in this module (`patient_treatment_plans` costs, `treatment_plan_items.estimated_cost`, `patient_insurance` benefits, `patient_visits.total_spent`, referral reward) are decimal numbers in currency units (for example rupees), not paisa.

28.5 Status lifecycles

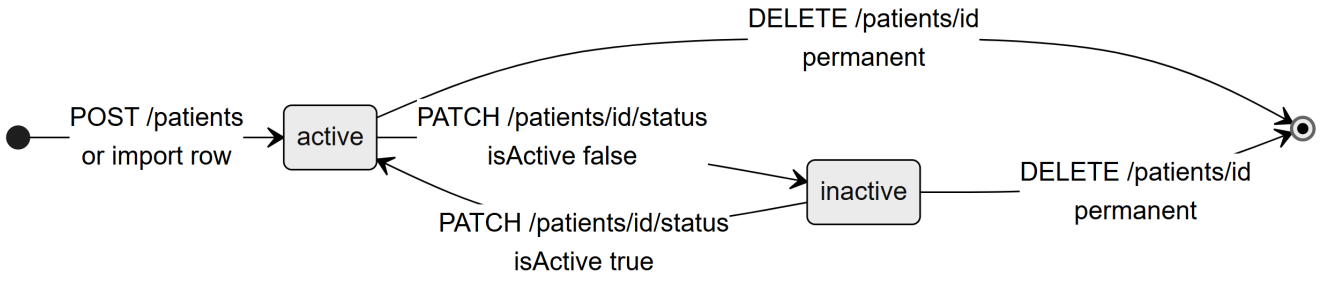

Figure 28.11 — Lifecycle of a patient's active status.

A patient is created **active** by `POST /patients` or by an import row. `PATCH /patients/:id/status` with `isActive: false` moves them to **inactive**, and with `isActive: true` back to **active**; each move writes a `status_history` row. From either state, `DELETE /patients/:id` removes the patient permanently (the end marker). Inactive patients keep all records and can still be booked and billed.

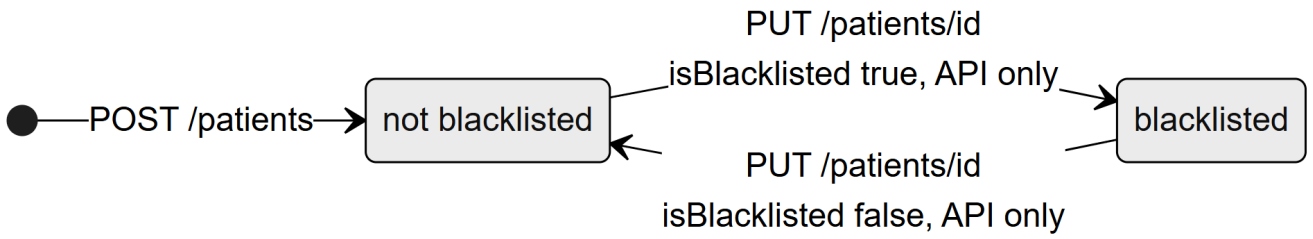

Figure 28.12 — Lifecycle of the blacklist flag.

The blacklist flag is independent of the active status. A new patient is **not blacklisted**. `PUT /patients/:id` with `isBlacklisted: true` (API only, no screen) makes them **blacklisted**, and `isBlacklisted: false` reverses it. The change is recorded as a normal field update in `audit_logs`, not in `status_history`.

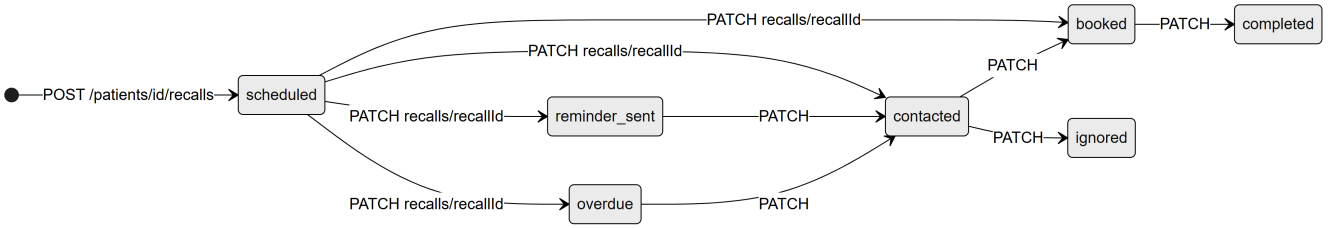

Figure 28.13 — Typical path of a recall through its statuses.

`POST /patients/:id/recalls` creates a recall in **scheduled**. The diagram shows the intended paths, each caused by `PATCH /patients/:id/recalls/:recallId`: from scheduled to **reminder_sent**, **contacted**, **booked** or **overdue**; reminder_sent to contacted; overdue to contacted; contacted to **booked** or **ignored**; booked to **completed**. The code has no transition map: the PATCH accepts any status from any status, and no background job sets reminder_sent or overdue. Recalls have no screen today.

Other status-like columns in the module have no transitions: referrals are always created as `pending` (no update endpoint), communications keep the status given at creation, and insurance `is_active` changes only through its PATCH. The treatment-plan and plan-item lifecycles are drawn in the Treatment Plans chapter (Chapter 33).

28.6 Connections to other modules

- **Appointments and Queue (Chapters 29–30):** booking and check-in pick a patient from this module; both appointment and queue rows define a doctor's `own` patient scope and a location's reach. Registration offers a "Book Appointment" step.
- **Consultation (Chapter 31):** the workspace shows the patient snapshot (history, latest vitals, allergies) from this module; encounters and treatments appear in the Visits and Treatments tabs.
- **Prescriptions (Chapter 32):** the Clinical & Rx tab links to the prescription builder; the legacy `patient_prescriptions` table lives here.
- **Treatment Plans (Chapter 33):** patient plans and items are stored by this module; plan templates fill the Record Treatment form.
- **Billing (Chapter 34):** the Billing tab and dues badge; billing tables block patient deletion.
- **Dental lab (Chapter 40):** lab orders reference a patient and block deletion.
- **Access control (Chapters 11 and 57):** permissions, scopes and features listed in each feature.

- **Audit logging (Chapter 60):** every write here produces an audit row tied to the patient.
- **Reports and dashboard (Chapter 36):** the dashboard counts active patients.

28.7 Limitations and known gaps

- **No screen for treatment plans, recalls, insurance, communications, referrals or feedback.** The backend endpoints work; the CRM tab component exists but is not mounted.
- **Blacklisting has no screen** and does not block booking, queueing or billing. Inactive patients are not blocked either.
- **Deleting a patient with billing or lab records fails** with a generic error, although the confirmation text says invoices and payments will be deleted.
- **Patient, plan and legacy prescription numbers come from a row count.** After a permanent deletion the count drops, so the next number can repeat an existing one; the unique index then makes the save fail with a server error. Two registrations at the same moment can also collide. Import guards against parallel imports only inside one server process.
- **The live phone check is not wired**, and phone uniqueness at registration is an exact text match, while import compares normalised phones. "0300 0000000" and "03000000000" are different to manual registration.
- `patient_visits` **is never filled**, so `GET /patients/:id/visits` is always empty.
- **No edit or delete for vitals, charts, consents, documents or legacy prescriptions** (by design for consents and charts; a mistaken upload cannot be removed from the screen).
- **The list screen exposes only search, gender and status filters**; card view has no paging; column sorting applies to the current page only.
- **Export uses UTC dates** for the file name and "Registered on" column.
- **Teeth charts store no periodontal data** from the screen, and vitals' medical flags are not in the form.

28.8 Owner's FAQ

Can a receptionist delete a patient? Not with the default Receptionist role: `patients.delete` is in the Admin template only. An owner can add it to a role on the roles screen; it is marked high-risk.

What happens to a patient's appointments and visits when I delete the patient? They are deleted with the patient, together with queue entries, encounters, treatments, prescriptions and all patient sub-records. If the patient has any charge, invoice, payment, refund or lab record, the delete is refused. Deactivating keeps everything and is the safer choice.

Why can a doctor not find a patient who is registered in the clinic? The Doctor role sees only `own` patients: those with an appointment or a queue entry with that doctor. Once the patient is booked with or queued for the doctor, they appear.

Can two patients share a phone number, for example a mother and child? No. The phone is unique per organization. Put the parent's number in the guardian phone and give the child another number, or leave the child's guardian details on a separate record.

Where are uploaded X-rays stored? In the external file-storage service behind the Platform's storage provider (Cloudinary today). The Platform stores only the link, file name, size and type in `patient_documents`.

Is every change to a patient recorded? Yes, for every write in this module: creation, edits with before/after values, activation changes, deletions and each sub-record. Exports are recorded too. Reads are not logged.

Can I bring in my old patient register? Yes, with **Import** on the Patients page: Excel `.xlsx` or CSV, up to 5,000 rows and 10 MB per file. Run the check first; rows with problems are listed and can be downloaded as an error report.

Who sees treatment prices on the patient page? Only holders of `patients.price.view` (Admin, Receptionist and Accountant by default). Doctors and assistants do not see fees in the Visits or Treatments tabs.

Does the Platform send recall reminders to patients? No. Recalls can be stored through the API, but there is no screen, no reminder job, and notifications are in-app only.

Chapter 29 — Appointments

This chapter describes the Appointments module of the Platform: how a clinic books a patient with a doctor for a future time, and how that booking is managed until the patient arrives. It is written for the product owner, who needs to know exactly what the module does today, what rules it enforces and where it stops. The module ends at the moment of arrival: check-in hands the patient over to the doctor-wise queue, which the Queue chapter (Chapter 30) covers in full.

29.1 What this module is for

An **appointment** is a booking: one patient, one doctor, one location, a start time and a duration. The Appointments module lets front-desk staff create, confirm, reschedule and cancel bookings, mark patients who never came as no-shows, and check arriving patients in. It does not track what happens after arrival. Once a patient is checked in, the **queue entry** (a place in one doctor's queue for the day) drives everything else, and the appointment only records the final outcome (completed or cancelled).

Who uses it:

- **Receptionist**: the main user. Books, reschedules, confirms, cancels, marks no-shows and checks patients in.
- **Assistant**: can view bookings at the location, book new ones and check patients in, but cannot reschedule or cancel.
- **Doctor**: sees only their own bookings (scope `own`), in the list and calendar. A doctor cannot book or change bookings with the default role.
- **Admin** and the **owner**: everything, including overriding booking rules.

Where it appears: the sidebar item **Appointments** (key `nav.appointments`, route `/appointments`, shown to users with `appointments.view`). The same page also hosts the reception queue board as its **Queue** tab, which is why the route opens for users who hold either `appointments.view` or `queue.board.view`. A custom role that holds only `queue.board.view` can open the page, but the sidebar item is hidden for it, because the sidebar entry checks `appointments.view` only.

The whole module sits behind the feature **`appointments.core`** ("Appointments", enabled by default). Turning that feature off also removes every queue permission, because the queue permissions belong to the same feature.

29.2 Screens

The Appointments page (`/appointments`) has a title "Appointments", the subtitle "Search, schedule, and update patient appointments.", a view switch with up to three tabs (**Queue**, **List**, **Calendar**) and a **Create** button for users with `appointments.book`. Users with `queue.board.view` land on the Queue tab; others land on the List tab. On top of the selected view sits a KPI strip, shown to users with `appointments.view`. The Queue tab is the reception board and is described in Chapter 30.

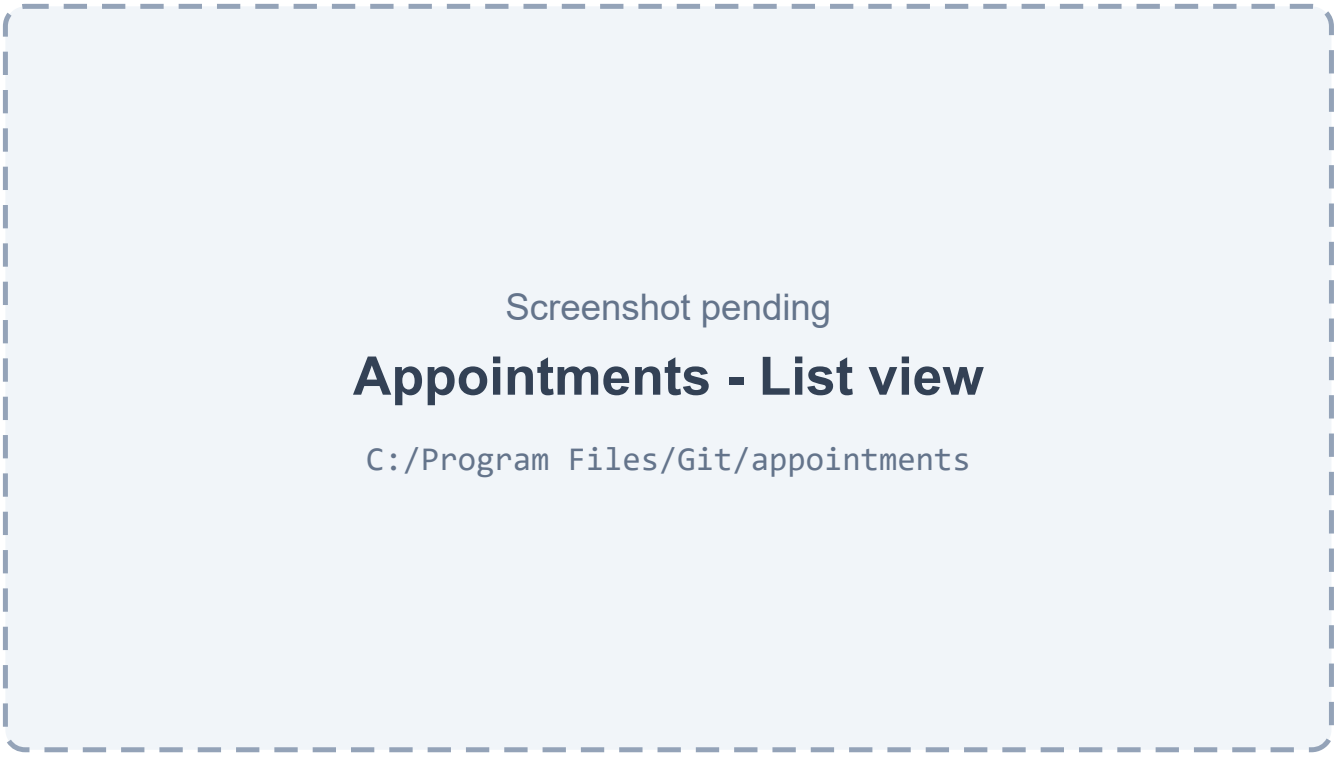

Figure 29.1 — Appointments, List view (screenshot pending).

The List view starts with the KPI strip: six small tiles labelled Total, Upcoming, Completed, Walk-ins, No-Show and Avg wait (in minutes), computed for the calendar month of the selected date. Under it is a toolbar card. Its first row has a "Jump to" date picker, a **Today** button when another day is selected, and a doctor filter. Its second row is a scrollable strip of 14 days; clicking a day narrows the list to that single day. Its third row holds the filters: a search box ("Search appointments...", matching patient name, phone or patient number), a status filter, a type filter that also offers "Walk-in", date presets (**Today**, **Upcoming**, **Past 30 days**, **All**), a "Custom range" picker, a **Reset** link and a count such as "12 visits". The table has the columns Token, Date & Time, Patient (name, phone and an "Emergency" badge), Doctor, Type, Status and an actions column. Walk-ins appear in the same table, with "Arrived" before their time and a "Walk-in" type badge. Rows are paged on the server, with page sizes 10, 25, 50 and 100. Clicking a booking opens the appointment detail dialog; clicking a walk-in opens the patient record. Cancelled rows are dimmed.

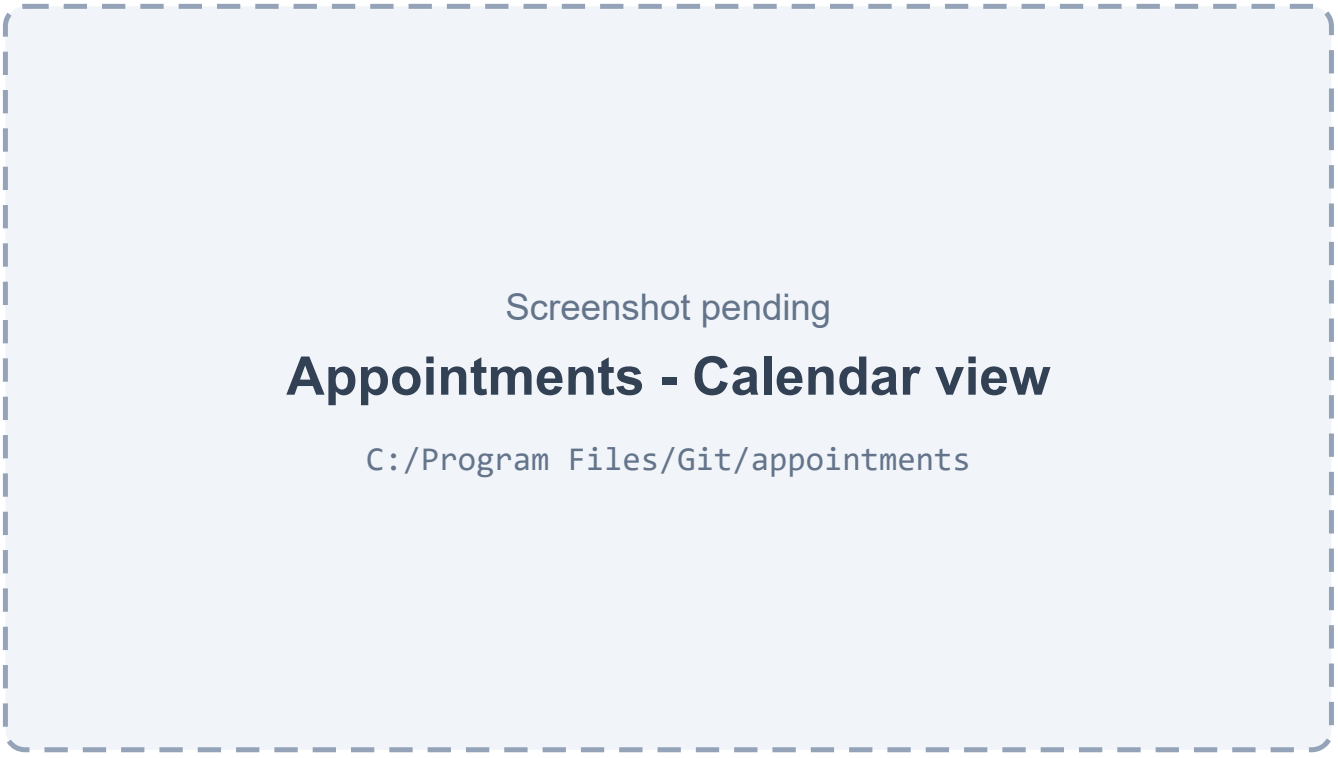

Figure 29.2 — Appointments, Calendar view (screenshot pending).

The Calendar view uses FullCalendar and its premium resource views (the resource time-grid and resource day-grid plugins), so each doctor at the location gets a column with initials, name and designation. A switch offers **Month**, **Week** and **Day**; Day by doctor is the default. The same toolbar (date picker, day strip, doctor filter) and the same filters as the List view appear over the grid. The time grid covers the full 24 hours in 30-minute rows and scrolls to 08:00 on opening. Each doctor's working hours for the visible days are drawn as a light green background. Bookings are coloured by status, with cancelled and emergency bookings styled differently. The user can select an empty time range to start a booking with that doctor and time pre-filled, click a booking to open its details, drag a booking to another time or another doctor's column, and drag its bottom edge to change its length. Only scheduled and confirmed bookings can be dragged or resized.

Screenshot pending

New Appointment dialog

C:/Program Files/Git/appointments

Figure 29.3 — The New Appointment dialog (screenshot pending).

The booking form opens as a centred dialog titled "New Appointment" (or "Edit Appointment" when rescheduling). It contains: **Patient *** (a search box, "Search by name or phone...", with a result list), **Doctor *** (a list of staff at the location shown as "name — designation"), **Date & Time *** (a browser date-and-time field), **Duration *** (15 min, 30 min, 45 min, 1 hour, 1.5 hours, 2 hours, plus the location's default slot length), a live validation message, **Appointment Type** (Consultation, Follow Up, Treatment, Checkup, Emergency, Cleaning), an **Emergency** switch and **Notes**. The live message is green ("Time slot is available") or red with the reason; when the doctor is busy it may offer "Suggested: 10:30 Apply". The buttons are **Cancel** and **Create Appointment** (or **Update Appointment**). The submit button is disabled while saving and while the live check reports a problem.

Screenshot pending

Appointment Details dialog

C:/Program Files/Git/appointments

Figure 29.4 — The Appointment Details dialog (screenshot pending).

The detail dialog shows the status badge, a large **Token** (or "Assigned at check-in" before arrival, because a booking has no token until the patient arrives), the queue status and doctor, the appointment type and an "Emergency" badge. Its body lists the patient (patient number and phone, clickable to open the patient record), the doctor and designation, the date, time and duration, notes and any cancellation reason. A "Quick actions" block offers **Confirm** (scheduled only), **Check in**, **No show** and **Cancel** for bookings not yet checked in. For a checked-in booking it shows the hint that calling, starting and finishing the visit happen on the Queue board or the doctor's screen. The footer has **Print ticket** (once a token exists), **Close** and **Edit**.

29.3 Features

29.3.1 See bookings and walk-ins in one list

Status: [Implemented]

What it does. Shows every booking at the active location, merged with that location's walk-ins, sorted by time and paged on the server. Staff can search and filter by date, doctor, status and type.

How the user does it.

1. Open **Appointments** in the sidebar and choose the **List** tab.
2. Pick a date preset (**Today**, **Upcoming**, **Past 30 days**, **All**), a custom range, or a day in the day strip.
3. Narrow with the doctor filter, the search box, the status filter or the type filter (choose "Walk-in" to see only walk-ins).
4. Click a row to open its details, or use the row menu (three dots).

Behind the scenes — in simple terms. Bookings and walk-ins live in two different tables. The server reads both, puts them in one time-ordered list and returns one page. A walk-in has no booking, so it is dressed up to look like a booking row, with its arrival time as its time. The doctor's own bookings are all a doctor sees.

Behind the scenes — technically. `GET /appointments` with `includeWalkIns=true` (the List view always sends it) runs `AppointmentsService.findAllWithWalkIns`: a SQL `UNION ALL` of `appointments` and walk-in `queue_entries` (joined to `queue_sessions` for doctor and location), counted and paged together, ordered by `sort_at` and id. Walk-in rows get `kind: 'walk_in'`; their status is mapped from the queue status (waiting, called, in consultation, on hold, skipped → `checked_in`; completed → `completed`; cancelled, aborted, transferred → `cancelled`). Every booking row is enriched with its latest queue entry (token such as "A-007", queue status, session id). Without `includeWalkIns`, a TypeORM query returns bookings only (the calendar uses this). The search box is debounced by 300 ms in the web app; the search runs as `ILIKE` on the patient's full name, phone and patient number.

Rules and validations.

- Always limited to the active location.
- Scope of `appointments.view`: `own` limits rows to the signed-in doctor's bookings and sessions; `location` and `org` show the whole location.
- A booking-only status filter (scheduled, confirmed, no-show) or a booking type filter leaves walk-ins out; the type filter "Walk-in" leaves bookings out.
- Default page size 50 on the server; the web app uses 25.

Who can do it. `appointments.view` (scoped `own` / `location` / `org`), feature `appointments.core`. Default: Admin (`org`), Receptionist (`location`), Assistant (`location`), Doctor (`own`). Accountant: no.

Data read and written. Reads `appointments`, `queue_entries`, `queue_sessions`, `patients`, `employees`, `locations`. Writes nothing.

Effects on other modules. None (read-only).

Error cases. 403 `NO_PERMISSION` without `appointments.view`. 400 on an invalid filter value (for example a malformed date).

Technical references: `backend/src/appointments/appointments.controller.ts` · `AppointmentsService.findAll` / `findAllWithWalkIns` · `web-app/src/features/appointments/components/AppointmentsList.tsx`

29.3.2 Browse the calendar by doctor

Status: [Implemented]

What it does. Shows bookings on a month, week or day grid, with one column per doctor and each doctor's weekly working hours shaded in the background.

How the user does it.

1. On **Appointments**, choose the **Calendar** tab.
2. Choose **Month**, **Week** or **Day**, and move with the arrows, the day strip or "Jump to".
3. Optionally filter by doctor, status, type or search text.

Behind the scenes — in simple terms. The page asks for the bookings in the visible range, the list of staff at the location (the columns) and the working-hours blocks for those days. The working hours come from each doctor's weekly schedule.

Behind the scenes — technically. Three calls: `GET /appointments` (bookings only, up to 500, for the visible range), `GET /appointments/doctors` (active staff assigned to the active location) and `GET /appointments/availability?startDate&endDate` (`AppointmentsService.getAvailability`). The availability call walks the location-local calendar days in the range (with `ClockService`) and, for each doctor and weekday, turns the matching `employee_schedules` row (working, active) into start and end instants in the location's timezone. The status, type and search filters on this tab are applied in the browser to the loaded bookings.

Rules and validations.

- Availability blocks come from the weekly schedule only. Date-specific availability overrides and approved leave are not shown (see 29.3.13).
- The weekly schedule lookup is by doctor and weekday; the schedule's own location is not considered, so a doctor who works Mondays at the second location shows Monday hours at the main location too.
- The "doctors" list is every active staff member with an active assignment at the location; the code does not filter by role.

Who can do it. `appointments.view` , feature `appointments.core` . Default: Admin, Receptionist, Assistant, Doctor (own bookings only).

Data read and written. Reads `appointments` , `employee_org_location_assignments` , `employees` , `employee_schedules` , `locations` . Writes nothing.

Effects on other modules. None.

Error cases. 403 `NO_PERMISSION` without `appointments.view` .

Technical references: `backend/src/appointments/appointments.service.ts` · `AppointmentsService.getAvailability` / `getDoctors` · `web-app/src/features/appointments/components/FullCalendarView.tsx`

29.3.3 See the KPI strip

Status: [Implemented]

What it does. Shows six numbers for the month of the selected date: Total bookings, Upcoming (scheduled plus confirmed), Completed, Walk-ins, No-Show and Avg wait in minutes.

How the user does it.

1. Open **Appointments**. The strip appears at the top for users who can view bookings.

Behind the scenes — in simple terms. Booking counts come from the appointments table. Walk-ins and waiting time come from the queue, because only the queue knows when a patient arrived and when the doctor started.

Behind the scenes — technically. `GET /appointments/stats?startDate&endDate` runs `AppointmentsService.getStats`: one grouped count over `appointments` (scoped by `appointments.view`) and one query over `queue_entries` joined to `queue_sessions`, giving walk-in count, waiting count, with-doctor count and the average of `consultation_started_at − checked_in_at` in minutes. The response also includes `cancelled`, `checkedIn`, `waiting`, `inProgress`, `completionRate` and `noShowRate`; the strip shows six of these.

Rules and validations.

- The web app sends the first and last moment of the selected month, computed in the browser's timezone.
- Average wait counts only entries whose consultation has started.

Who can do it. `appointments.view`, feature `appointments.core`. Default: Admin, Receptionist, Assistant, Doctor (own figures only).

Data read and written. Reads `appointments`, `queue_entries`, `queue_sessions`.

Effects on other modules. None.

Error cases. 403 `NO_PERMISSION`.

Technical references: `backend/src/appointments/appointments.service.ts` · `AppointmentsService.getStats` · `web-app/src/features/appointments/components/StatsCards.tsx`

29.3.4 Book an appointment

Status: [Implemented]

What it does. Creates a booking for a patient with a doctor at the active location. A new booking starts as `scheduled` and has no token; the token is issued only when the patient arrives.

How the user does it.

1. Click **Create** on the Appointments page, or select an empty time range in the calendar (doctor and time are pre-filled).
2. Search and pick the **Patient**, choose the **Doctor**, set **Date & Time** and **Duration**.
3. Wait for the live check to turn green, or click **Apply** on a suggested time.
4. Optionally set **Appointment Type**, **Emergency** and **Notes**.
5. Click **Create Appointment**. A toast confirms it and the calendar moves to the booked day.

Figure 29.5 shows the full exchange between the screen and the server.

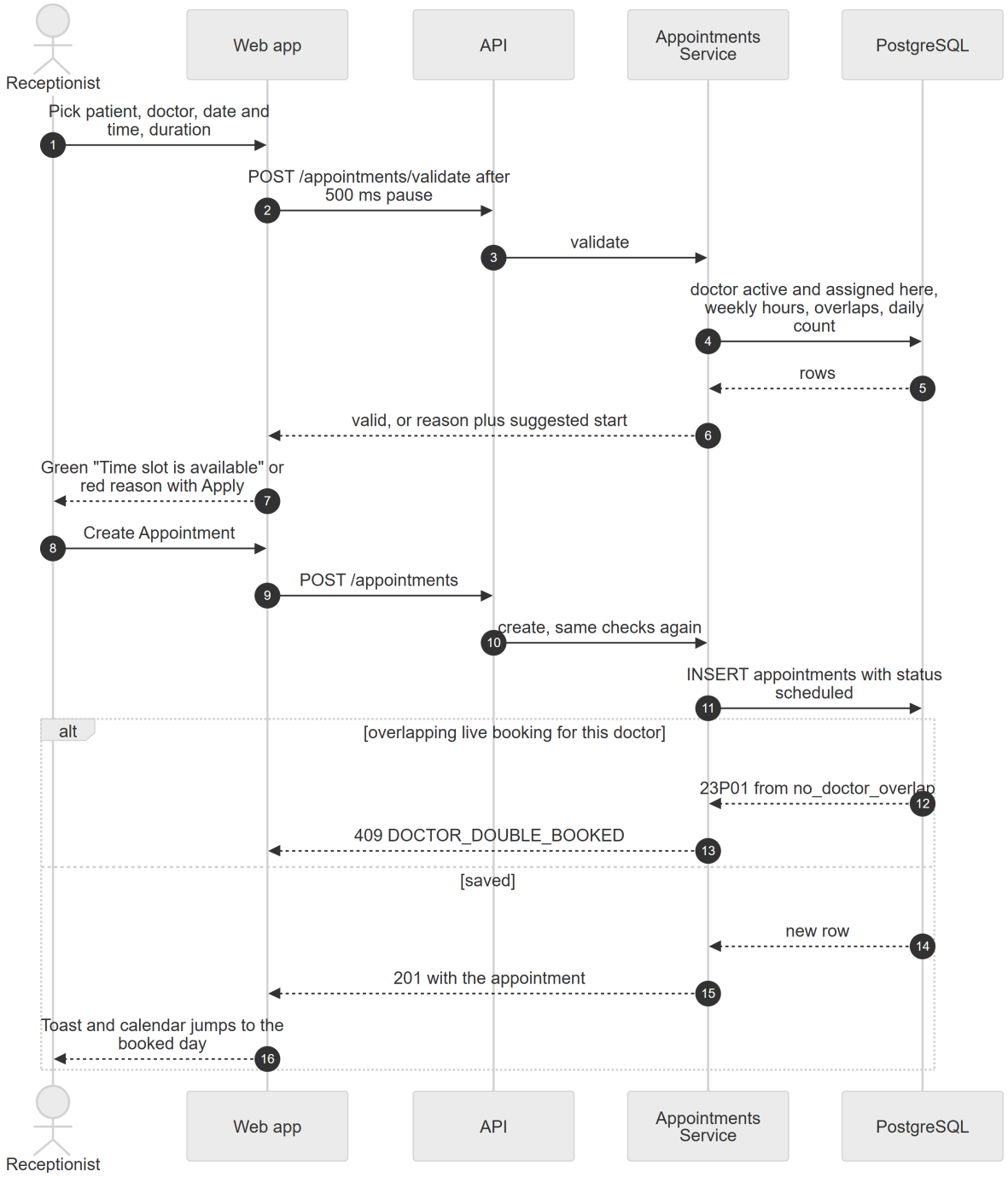

Figure 29.5 — Booking: live validation, then the save that the database has the final say on.

The diagram has five participants: the Receptionist, the Web app, the API, the AppointmentsService and PostgreSQL. In steps 1 to 7, the receptionist picks patient, doctor, date, time and duration; after a 500 ms pause the web app calls `POST /appointments/validate`; the service checks that the doctor is active and assigned to the location, that the slot fits the weekly hours, that there is no overlap and that the daily count is not reached; the answer (valid, or a reason with a suggested start) comes back and the form shows a green or red message with an Apply link. In steps 8 to 11, the receptionist clicks Create Appointment, the web app calls `POST /appointments`, the service runs the same checks again and inserts the row with status `scheduled`. An "alt" box then shows two outcomes. If another live booking for the same doctor overlaps (for example because two desks saved at the same moment), PostgreSQL rejects the insert with error 23P01 from the `no_doctor_overlap` constraint and the user receives 409 `DOCTOR_DOUBLE_BOOKED`. Otherwise the new row comes back, the API answers 201 and the screen shows a toast and jumps to the booked day.

Behind the scenes — in simple terms. The server runs a fixed list of checks in order: not in the past, the patient belongs to this organization, the doctor is active and works at this location, the slot fits inside one of the doctor's working blocks for that weekday, the doctor has no other live booking at that time at any location, and the location has not reached its daily booking limit. The database itself also refuses double bookings, so two people saving at the same second cannot both win.

Behind the scenes — technically. `POST /appointments` → `AppointmentsService.create`. Steps: resolve the location timezone (`ClockService.timezoneFor`); reject a start before `ClockService.now()` ; load the patient by id and organization; `assertDoctorAtLocation` (active `employees` row and an active `employee_org_location_assignments` row for this organization and location); `validateSchedule` (weekday and "HH:MM" read in the location's timezone; the slot must fit inside one active, working `employee_schedules` block); `checkGlobalConflict` (any appointment of the doctor whose time overlaps and whose status is not cancelled or no-show, at any location); `assertDailyLimit` (count of non-cancelled, non-no-show bookings at the location on that local day against `location_settings.max_appointments_per_day`). The row is saved by `saveBooking` , which maps PostgreSQL error 23P01 from the exclusion constraint `no_doctor_overlap` to 409 `DOCTOR_DOUBLE_BOOKED` . The constraint is `EXCLUDE USING gist (doctor_id WITH =, tstzrange(start_time, end_time, '[)') WITH &&) WHERE status NOT IN ('cancelled','no_show')` and needs the `btree_gist` extension, which the API creates on start-up. No queue, event or audit-log write happens at booking; `created_by` and `updated_by` hold the user id.

Rules and validations.

- Duration is a whole number of minutes from 5 to 480; patient, doctor and start time are required.
- `type` defaults to `consultation` ; `isEmergency` defaults to false. The emergency flag does not move the patient ahead in the queue on its own (Chapter 30).
- The start must not be in the past.
- The doctor must be active and actively assigned to the active location.
- The whole slot must fit inside one weekly working block for that weekday, read in the location's timezone. The lookup ignores which location the block belongs to (a known gap).
- No overlap with the doctor's other live bookings at any location of any organization the doctor works for. Cancelled and no-show bookings do not block. Bookings touching end-to-start do not overlap (`[)` range).
- Daily limit per location: `location_settings.max_appointments_per_day` (default 50 once the location's settings row exists; 0 or a missing row means no limit).
- The reason for visit can be stored (`reasonForVisit` in the API), but the booking dialog has no field for it.

Who can do it. `appointments.book` , feature `appointments.core` . Default: Admin, Receptionist, Assistant. Not Doctor or Accountant.

Data read and written. Reads `patients` , `employees` , `employee_org_location_assignments` , `employee_schedules` , `location_settings` , `locations` (timezone), `appointments` . Writes `appointments` .

Effects on other modules. The new booking appears in the calendar, the list, the KPI strip, the patient record's appointments tab and the clinic overview and visit reports (Chapter 36). A doctor with patient scope `own` can see a patient once that patient has an appointment with them.

Error cases.

- 400 "Cannot book appointments in the past".
- 404 "Patient not found" → the patient is not in this organization.
- 404 "Doctor not found or is inactive".
- 400 "Doctor is not assigned to this location".
- 400 "Doctor does not work on Monday" (or the weekday concerned).
- 400 "Requested time (10:00-10:30) is outside doctor's schedule. Available: 14:00 - 18:00".

- 409 "Doctor has a conflicting appointment during this time" → the read check found an overlap (no error code).
- 409 `DOCTOR_DOUBLE_BOOKED` → the database constraint caught an overlap.
- 409 `DAILY_LIMIT_REACHED` → "Daily limit reached: this location takes at most 50 bookings on 2026-10-26".
- 402 `SUBSCRIPTION_EXPIRED` → the clinic is in read-only mode.

Technical references: `backend/src/appointments/appointments.service.ts` · `AppointmentsService.create`, `saveBooking`, `Appointment` (`no_doctor_overlap`) · `web-app/src/features/appointments/components/AppointmentModal.tsx`

29.3.5 Check a time slot live and get a suggested time

Status: [Implemented]

What it does. While the booking form is open, the server checks the chosen doctor, time and duration and, if the doctor is busy, suggests the next free start time on the same day.

How the user does it.

1. Fill in Doctor, Date & Time and Duration in the booking dialog.
2. Read the green or red message. If a suggestion appears, click **Apply** to use it.

Behind the scenes — in simple terms. The same checks as a real booking run, but nothing is saved and problems come back as a message instead of an error. To suggest a time, the server walks forward through the doctor's bookings that day, jumping past each clash, until a free gap that still fits the working hours appears.

Behind the scenes — technically. `POST /appointments/validate` → `AppointmentsService.validate`, returning `{ valid, reason?, conflictLocation?, suggestedStartTime? }`. On a conflict, `suggestNextSlot` loads the doctor's live bookings for that location-local day, starts at the requested time and moves to the end of each clashing booking, up to 50 tries, stopping when the candidate no longer fits a working block. `excludeAppointmentId` lets a reschedule ignore the booking being moved. The web app calls it 500 ms after the last change to doctor, time or duration.

Rules and validations.

- Checks doctor, weekly schedule, overlaps and the daily limit. It does not check "in the past"; the save does.
- When the clash is at another location, the reason reads "Doctor busy at another location".
- The suggestion considers only the same day and the doctor's weekly hours.

Who can do it. `appointments.book`, feature `appointments.core`. Default: Admin, Receptionist, Assistant. A user with only `appointments.reschedule` (no `appointments.book`) gets 403 on this check when editing; the Receptionist template holds both.

Data read and written. Reads the same tables as booking. Writes nothing.

Effects on other modules. None.

Error cases. Problems come back as `valid: false` with a reason (same texts as 29.3.4). 400 on invalid input; 403 `NO_PERMISSION`.

Technical references: `backend/src/appointments/appointments.service.ts` · `AppointmentsService.validate`, `suggestNextSlot` · `web-app/src/features/appointments/hooks/useValidateAppointment.ts`

29.3.6 Override booking rules

Status: [Partial] — the API supports it; no screen sends it.

What it does. Lets an authorised user book (or move) a booking outside the doctor's working hours or beyond the location's daily limit.

How the user does it. No screen offers this. It is available only to an API client that sends `overrideRules: true`.

Behind the scenes — in simple terms. The working-hours check and the daily-limit check are skipped. The double-booking check is never skipped.

Behind the scenes — technically. `CreateAppointmentDto.overrideRules` and `UpdateAppointmentDto.overrideRules`. `AppointmentsService.overridesRules` checks `access.can('appointments.override_rules')` and throws 403 `NO_PERMISSION` otherwise. When allowed, `validateSchedule` and `assertDailyLimit` are skipped; `checkGlobalConflict` and the `no_doctor_overlap` constraint still apply.

Rules and validations.

- The live validation endpoint does not accept the flag, so it would still report "outside schedule" for such a slot.

Who can do it. `appointments.override_rules` (high-risk, implies `appointments.book`), feature `appointments.core`. Default: Admin only.

Data read and written. Same as booking and reschedule.

Effects on other modules. None beyond a normal booking.

Error cases. 403 `NO_PERMISSION` "Missing permission: appointments.override_rules".

Technical references: `backend/src/appointments/dto/create-appointment.dto.ts` · `AppointmentsService.overridesRules` · no web-app caller

29.3.7 Reschedule or edit a booking

Status: [Implemented]

What it does. Changes the time, duration, doctor, patient, type or notes of a booking that has not been checked in. A reschedule keeps the same row and the same status.

How the user does it.

1. Open the booking (row click or calendar click) and click **Edit**, or choose **Edit** in the row menu.
2. Change the fields and click **Update Appointment**.
3. Or, in the Calendar, drag the booking to another time or another doctor's column, or drag its bottom edge to change its length. A toast confirms ("Appointment rescheduled", "Reassigned to Dr. Example One", "Duration updated to 45 min"); on failure the booking jumps back.

Behind the scenes — in simple terms. Only scheduled and confirmed bookings can change. If the time or doctor really changes, all the booking checks run again (excluding this booking itself). Changing only the notes runs no checks.

Behind the scenes — technically. `PUT /appointments/:id` → `AppointmentsService.update`. It loads the booking at the active location and calls `assertEditable` (status must be `scheduled` or `confirmed`, else 422 `APPOINTMENT_LOCKED`). `timeChanged` and `doctorChanged` decide whether to re-run the past-time check, `validateSchedule`, `checkGlobalConflict` and `assertDailyLimit` (with this booking excluded). The status is not touched. The web form sends `startTime` and `duration` only when they changed, so editing notes never reschedules. The dialog loads the "Emergency" switch, but the update API has no `isEmergency` field, so a change to that switch on edit is not saved.

Rules and validations.

- Only `scheduled` or `confirmed` bookings are editable.
- A changed start time must not be in the past.
- A new doctor must be active and assigned to the location; a new patient must be in the organization.
- Same schedule, overlap and daily-limit rules as booking; `overrideRules` works as in 29.3.6.

Who can do it. `appointments.reschedule`, feature `appointments.core`. Default: Admin, Receptionist. Not Assistant, Doctor or Accountant.

Data read and written. Reads as booking. Writes `appointments` (time, duration, doctor, patient, type, notes, `updated_by`).

Effects on other modules. The calendar, list and KPI figures change. A booking moved to another day may change which day's queue it can be checked into.

Error cases. 404 "Appointment not found"; 422 `APPOINTMENT_LOCKED` "A checked_in appointment can no longer be edited; only scheduled or confirmed bookings can"; 400 "Cannot move an appointment into the past"; plus every booking error from 29.3.4.

Technical references: `backend/src/appointments/appointment-transitions.ts` · `AppointmentsService.update`, `assertEditable` · `web-app/src/features/appointments/components/FullCalendarView.tsx`

29.3.8 Confirm a booking

Status: [Implemented]

What it does. Marks a scheduled booking as confirmed (for example after the patient answered a call). It is a status marker only; it changes no rule.

How the user does it.

1. In the row menu or the detail dialog, click **Confirm** (shown only for scheduled bookings).

Behind the scenes — in simple terms. The status moves from scheduled to confirmed if the status map allows it.

Behind the scenes — technically. `PATCH /appointments/:id/confirm` → `AppointmentsService.confirm` → `assertTransition(status, CONFIRMED)` against `APPOINTMENT_TRANSITIONS`, then save.

Rules and validations.

- Allowed only from `scheduled`.

Who can do it. `appointments.book`, feature `appointments.core`. Default: Admin, Receptionist, Assistant.

Data read and written. Writes `appointments.status`, `updated_by`.

Effects on other modules. Counts as "Upcoming" in the KPI strip, like scheduled.

Error cases. 404 "Appointment not found"; 422 `INVALID_TRANSITION` "Cannot change an appointment from confirmed to confirmed".

Technical references: `backend/src/appointments/appointments.service.ts` · `AppointmentsService.confirm` · `web-app/src/features/appointments/hooks/useAppointmentActions.ts`

29.3.9 Cancel a booking (with reason codes)

Status: [Partial] — the API stores a reason code and text; the web app cancels with neither and asks no confirmation.

What it does. Ends a booking that has not been checked in. The API accepts a structured reason code (`patient_left` or `other`) and an optional free-text reason, for later analysis.

How the user does it.

1. In the row menu or the detail dialog, click **Cancel**. The booking is cancelled at once and a toast says "Appointment cancelled".

Behind the scenes — in simple terms. The booking becomes cancelled and stops blocking the doctor's time. A patient who has already checked in cannot be cancelled here; they must be removed from the queue instead, which also cancels the booking.

Behind the scenes — technically. `PATCH /appointments/:id/cancel` with `{ reasonCode?, reason? }` → `AppointmentsService.cancel`. A `checked_in` booking is refused with 422 `INVALID_TRANSITION`. Otherwise `assertTransition(status, CANCELLED)`, then `cancel_reason_code`, `cancel_reason` and `cancelled_at` are set. The DTO refuses `no_show` as a reason code ("use the no-show action for no-shows"). The web app's cancel helper can send a reason, and the enum and option list exist in `features/appointments/types.ts`, but no screen uses them, and no confirm dialog is shown.

Rules and validations.

- Allowed from `scheduled` or `confirmed`.
- `reasonCode` must be `patient_left` or `other` when given.
- After check-in, the queue's "Remove from queue" or "Stop consultation" closes the booking as cancelled with code `patient_left` or `other` (Chapter 30).

Who can do it. `appointments.cancel`, feature `appointments.core`. Default: Admin, Receptionist.

Data read and written. Writes `appointments` (`status`, `cancel_reason_code`, `cancel_reason`, `cancelled_at`, `updated_by`).

Effects on other modules. The slot becomes free for other bookings (the overlap check and the constraint ignore cancelled rows). The booking counts as cancelled in reports.

Error cases. 422 `INVALID_TRANSITION` "The patient has checked in — remove them from the queue instead"; 422 `INVALID_TRANSITION` for completed, cancelled or no-show bookings; 400 "reasonCode must be patient_left or other; use the no-show action for no-shows".

Technical references: `backend/src/appointments/enums/cancel-reason.enum.ts` · `AppointmentsService.cancel`, `CancelAppointmentDto` · `web-app/src/features/appointments/components/AppointmentDetailModal.tsx`

29.3.10 Mark a no-show (by hand or automatically)

Status: [Implemented]

What it does. Records that a booked patient never arrived. Staff can do it by hand; a background job also does it automatically once a patient is later than the clinic's "maximum late" setting.

How the user does it.

1. In the row menu or the detail dialog of a scheduled or confirmed booking, click **No show**.
2. Otherwise, do nothing: the job marks late bookings on its own.

Behind the scenes — in simple terms. The booking becomes a no-show and stops blocking the doctor's time. Every minute, a job looks for bookings that are still scheduled or confirmed and whose start time plus the "maximum late" minutes (45 by default) has passed, and marks them as no-shows.

Behind the scenes — technically. Manual: `PATCH /appointments/:id/no-show` → `AppointmentsService.noShow` → `assertTransition(status, NO_SHOW)`, sets `no_show_at`. Automatic: `QueueJobsService.markNoShows`, `@Interval` every 60 seconds in the worker process. It finds organization–location pairs with scheduled or confirmed bookings that started in the last 7 days, reads each location's effective queue policy, and runs one `UPDATE appointments SET status = 'no_show', no_show_at = now()` where `start_time + max_late_min < now()`. The job writes directly with SQL: no queue event, no audit row and no realtime message.

Rules and validations.

- Allowed only from `scheduled` or `confirmed`.
- The automatic threshold is the location's `max_late_min` (default 45 minutes; Chapter 30 covers queue policies).

Who can do it. Manual: `appointments.cancel` (a no-show ends the booking like a cancel does), feature `appointments.core`. Default: Admin, Receptionist. Automatic: the worker process, no user.

Data read and written. Reads `appointments`, `queue_policies`. Writes `appointments` (`status`, `no_show_at`, `updated_by` for the manual action).

Effects on other modules. No-shows count in the KPI strip and in the visit reports. A no-show booking no longer shows a **Check in** action in the web app (see 29.3.12).

Error cases. 422 `INVALID_TRANSITION` for a booking that is not scheduled or confirmed.

Technical references: `backend/src/queue/jobs/queue-jobs.service.ts` · `AppointmentsService.noShow`, `QueueJobsService.markNoShows` · `web-app/src/features/appointments/components/AppointmentsList.tsx`

29.3.11 Check a patient in (hand-over to the queue)

Status: [Implemented]

What it does. Records that a booked patient has arrived and gives them a doctor-wise **token** (for example "A-007") in that doctor's queue for today. From here on the queue runs the visit.

How the user does it.

1. In the row menu or the detail dialog, click **Check in**.
2. In the "Check in patient" dialog, optionally tick **Give urgent priority?** and type a reason.
3. Click **Check in & get token**. A toast shows "Checked in — token A-007" with a **Print ticket** button.

Behind the scenes — in simple terms. The queue checks that the booking is for today and that the patient is not too late, opens the doctor's queue for the day if needed, issues the next token, and marks the booking as checked in. If the patient is too late, the dialog switches to "Too late for the appointment" and offers to convert them to a walk-in.

Behind the scenes — technically. `PATCH /appointments/:id/check-in` (requires an `Idempotency-Key` header) is served by the Appointments controller but runs `QueueOrchestrator.checkInAppointment` in one transaction: lock the appointment row, refuse a second active entry (409 `ALREADY_CHECKED_IN`), check the status map, check the booking's service date equals today's (422 `NOT_TODAY`), check `isBeyondMaxLate` (422 `LATE_LIMIT_EXCEEDED`), ensure and lock the doctor's `queue_sessions` row, issue a token, insert a `queue_entries` row (`queue_type = appointment`, status `waiting`), and set the appointment to `checked_in` with `checked_in_at`. Chapter 30 explains tokens, idempotency and the queue rules in full.

Rules and validations.

- Only from `scheduled` or `confirmed`, only on the booking's own service day, and only up to `max_late_min` after the start.
- A retried request with the same key returns the same token.

Who can do it. `queue.checkin`, feature `appointments.core`. Default: Admin, Receptionist, Assistant.

Data read and written. Writes `appointments`, `queue_sessions`, `queue_entries`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. Creates the queue entry that the Queue module (Chapter 30) and the consultation workspace (Chapter 31) continue from.

Error cases. 409 `ALREADY_CHECKED_IN` (the dialog shows "Already checked in — token A-007"); 422 `LATE_LIMIT_EXCEEDED` (the dialog switches to the convert option); 422 `NOT_TODAY`; 422 `SESSION_NOT_OPEN` "This queue is closed for the day"; 400 `IDEMPOTENCY_KEY_REQUIRED`.

Technical references: `backend/src/appointments/appointments.controller.ts` · `QueueOrchestrator.checkInAppointment` · `web-app/src/features/queue/components/CheckInDialog.tsx`

29.3.12 Convert a late booking to a walk-in

Status: [Implemented] — with a practical gap: the web app reaches it only in the short time before the no-show job runs.

What it does. For a patient who arrives later than the clinic allows, marks the booking as a no-show and adds the patient to the same doctor's queue as a walk-in with a new token, linked to the old booking.

How the user does it.

1. Click **Check in** on the late booking.
2. The dialog shows "Too late for the appointment" and the server's message ("More than 45 minutes late — convert to a walk-in or reschedule").
3. Click **Convert to walk-in**. A toast shows "Added as walk-in — token A-012".

Behind the scenes — in simple terms. The booking is closed as a no-show, and a new walk-in entry is created for the patient in the doctor's queue, with a pointer back to the booking.

Behind the scenes — technically. `POST /appointments/:id/convert-to-walk-in` (with `Idempotency-Key`) → `QueueOrchestrator.convertToWalkIn`. Allowed when the booking is already `no_show`, or is `scheduled` / `confirmed` and beyond `max_late_min`. Sets the booking to `no_show` if needed, ensures today's session for the booking's doctor, refuses a duplicate active entry, and issues a token with `queue_type = walk_in`, `appointment_id = NULL` and `converted_from_appt_id` = the booking. The `CHECKED_IN` queue event carries reason code `CONVERTED_TO_WALK_IN`.

Rules and validations.

- Refused (422 `INVALID_TRANSITION`) when the booking is still within the late limit: "Only an appointment past the late limit (or a no-show) can be converted; check the patient in instead".
- The web app shows **Check in** only for scheduled or confirmed bookings. Because the no-show job marks a late booking within about a minute of the limit passing, a late patient's booking is usually already `no_show` and shows no check-in action; the desk then has to add an ordinary walk-in (Chapter 30), which is not linked to the booking. This is a deduction from the code, not an observed test.

Who can do it. `queue.checkin`, feature `appointments.core`. Default: Admin, Receptionist, Assistant.

Data read and written. Writes `appointments`, `queue_sessions`, `queue_entries`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. The visit is billed and reported as a walk-in.

Error cases. 422 `INVALID_TRANSITION` (not late enough); 409 `ALREADY_CHECKED_IN`; 422 `SESSION_NOT_OPEN`.

Technical references: `backend/src/queue/services/queue-orchestrator.service.ts` · `QueueOrchestrator.convertToWalkIn` · `web-app/src/features/queue/components/CheckInDialog.tsx`

29.3.13 Respect availability overrides and leave when booking

Status: [Not implemented]

What it does. Nothing today. The Employees module stores date-specific availability overrides (`doctor_availability_overrides`: a doctor is unavailable on a date, or has special hours) and leave (`employee_leaves`), but booking, live validation, slot suggestion and the calendar's availability shading never read them.

How the user does it. Not available. A receptionist can book a doctor on a day the doctor is on approved leave, as long as the weekly schedule allows it.

Behind the scenes — in simple terms. Only the weekly schedule is consulted.

Behind the scenes — technically. `AppointmentsService` reads `employee_schedules` only; there is no reference to `DoctorAvailabilityOverride` or `EmployeeLeave` in `backend/src/appointments/` or `backend/src/queue/`.

Rules and validations. None.

Who can do it. Not applicable.

Data read and written. None.

Effects on other modules. None.

Error cases. None.

Technical references: `backend/src/employees/entities/doctor-availability-override.entity.ts` · `DoctorAvailabilityOverride`, `EmployeeLeave` · no web-app use in appointments

29.3.14 Reminders, recurring bookings and online booking

Status: [Not implemented]

What it does. Nothing. The feature keys `appointments.reminders`, `appointments.recurring`, `appointments.online_booking` and `appointments.waiting_room` are commented out in the feature catalog. Notifications on the Platform are in-app only, and no notification event is raised for appointments.

How the user does it. Not available.

Behind the scenes — in simple terms. No reminder messages, no repeating bookings, no patient-facing booking page.

Behind the scenes — technically. Commented entries in `FEATURE_CATALOG_SEED`; `Appointment` has no recurrence fields; the notifications module has no appointment events.

Rules and validations. None.

Who can do it. Not applicable.

Data read and written. None.

Effects on other modules. None.

Error cases. None.

Technical references: `backend/src/feature-flags/feature-catalog.seed.ts` · `FEATURE_CATALOG_SEED` · none

29.4 Entities and data model

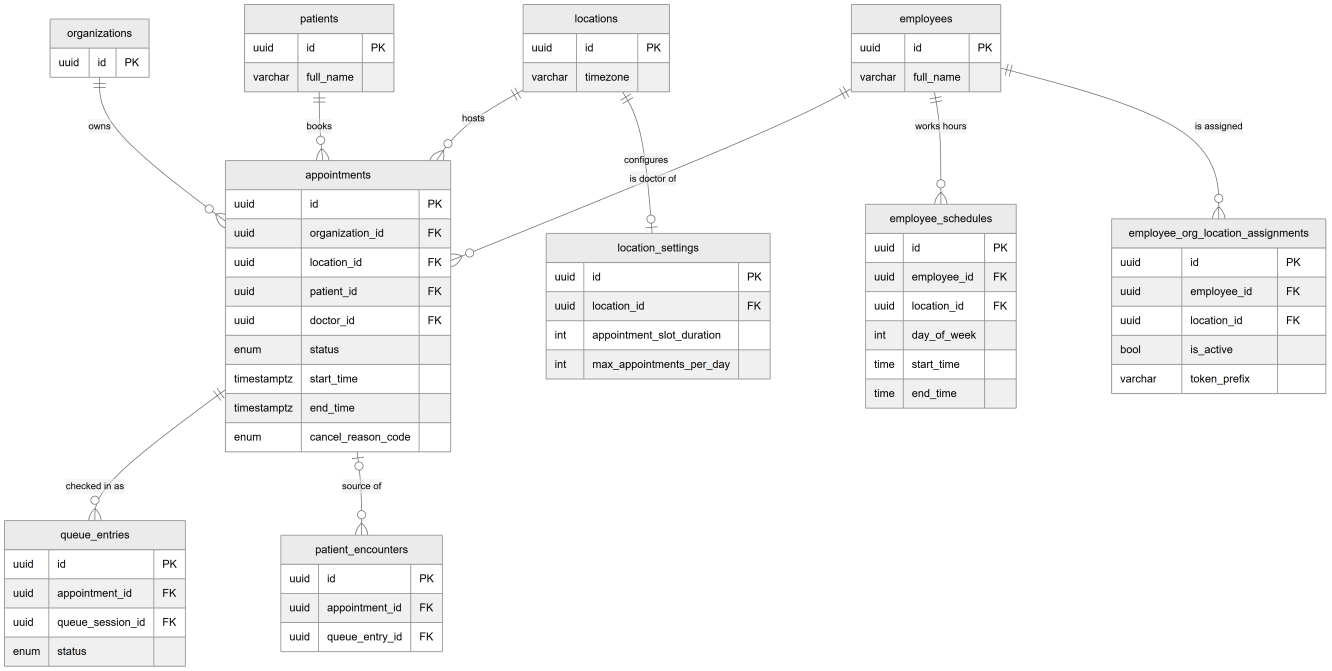

Figure 29.6 — The `appointments` table and the tables around it.

The diagram centres on `appointments`, with columns `id`, `organization_id`, `location_id`, `patient_id`, `doctor_id`, `status`, `start_time`, `end_time` and `cancel_reason_code`. Four tables point into it: `organizations` ("owns"), `locations` ("hosts"), `patients` ("books") and `employees` ("is doctor of"). Two tables hang off it: `queue_entries` ("checked in as", with `appointment_id`, `queue_session_id`, `status`) and `patient_encounters` ("source of", with `appointment_id` and `queue_entry_id`). On the right, `employees` also links to `employee_schedules` ("works hours": `employee_id`, `location_id`, `day_of_week`, `start_time`, `end_time`) and to `employee_org_location_assignments` ("is assigned": `employee_id`, `location_id`, `is_active`, `token_prefix`), and `locations` links to `location_settings` ("configures": `appointment_slot_duration`, `max_appointments_per_day`). `locations` shows its `timezone` column, which every clinic-time calculation uses.

The relationships in words:

- Each appointment belongs to exactly one organization and one location. Deleting the organization or the location deletes its appointments (ON DELETE CASCADE).
- Each appointment belongs to exactly one patient. Deleting the patient deletes the patient's appointments (CASCADE).
- Each appointment has exactly one doctor, an employee. An employee who still has appointments cannot be deleted (ON DELETE RESTRICT).
- An appointment can have several queue entries over time, but at most one active one (partial unique index `one_active_entry_per_appt`). Deleting an appointment deletes its queue entries (CASCADE), because a booked entry must always point to its appointment.
- A walk-in created by "Convert to walk-in" points back to the booking through `queue_entries.converted_from_appt_id`, a logical link with no foreign key.

- An encounter (the clinical record of one consultation) may point to the appointment it came from. Deleting the appointment sets that pointer to NULL (ON DELETE SET NULL).
- Weekly working hours (`employee_schedules`) belong to one employee and one location, unique per employee, location and weekday. Booking reads them by employee and weekday only.
- Each location has at most one settings row; `appointment_slot_duration` (default 30) is the booking form's default duration, and `max_appointments_per_day` (default 50) is the daily limit.

Key columns of `appointments` : `type` (consultation, follow_up, treatment, checkup, emergency, cleaning), `status` (six values, 29.5), `start_time` / `end_time` (timestamptz) and `duration_minutes` , `reason_for_visit` , `is_emergency` , `notes` , `cancel_reason_code` (no_show, patient_left, other) with `cancel_reason` , the lifecycle timestamps `checked_in_at` , `completed_at` , `cancelled_at` , `no_show_at` , and `created_by` / `updated_by` . Indexes cover location + start time, doctor + start + end, organization, patient, doctor and status. The schema comes from the entity (TypeORM synchronize in development; a reviewed schema sync in production); no migration file is specific to appointments.

29.5 Status lifecycles

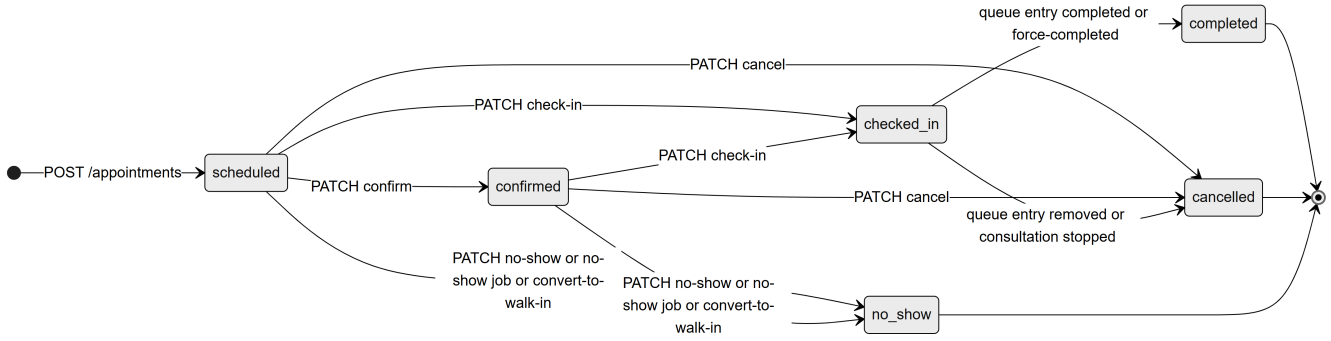

Figure 29.7 — The appointment status map from `appointment-transitions.ts` .

The diagram has six states. A booking enters at `scheduled` through `POST /appointments` . From `scheduled` , "PATCH confirm" leads to `confirmed` . Both `scheduled` and `confirmed` lead to `checked_in` through "PATCH check-in", to `cancelled` through "PATCH cancel", and to `no_show` through "PATCH no-show or no-show job or convert-to-walk-in". From `checked_in` there are two arrows: to `completed` when the queue entry is completed or force-completed, and to `cancelled` when the queue entry is removed from the queue or the consultation is stopped. `completed` , `cancelled` and `no_show` are final: each leads only to the end marker.

Walking through each state:

- **scheduled** — the starting state of every booking. It can be edited, confirmed, checked in, cancelled or marked no-show.
- **confirmed** — same as scheduled, but someone has confirmed it. It cannot go back to scheduled.
- **checked_in** — the patient arrived and has an active queue entry. The booking can no longer be edited (422 `APPOINTMENT_LOCKED`) or cancelled from the Appointments screen. The queue drives the rest.
- **completed** — the queue entry was completed (normally by the doctor, or force-completed by an admin). `completed_at` is set.
- **cancelled** — cancelled before arrival (with an optional reason code), or closed by the queue after arrival: code `patient_left` when reception removed the patient from the queue, code `other` with the stop reason when the doctor stopped the consultation.
- **no_show** — the patient never came. Set by hand, by the no-show job, or by "Convert to walk-in".

Any other change is refused with 422 `INVALID_TRANSITION`. There is no "rescheduled" or "in progress" status: a reschedule keeps the same row and status. A transferred queue entry keeps the appointment `checked_in`, because the new entry in the other doctor's queue carries the same `appointment_id`.

29.6 Connections to other modules

- **Queue (Chapter 30).** Check-in and convert-to-walk-in are queue operations exposed under `/appointments`. The queue later sets the appointment to `completed` or `cancelled`. The automatic no-show job and the `max_late_min` setting belong to the queue.
- **Patients (Chapter 28).** Bookings need an existing patient. The patient record shows the patient's appointments and visits. A doctor's `own` patient scope includes every patient who has an appointment or a queue visit with that doctor.
- **Employees (Chapter 37).** Doctors are employees with an active assignment at the location; weekly schedules come from the Employees module. Availability overrides and leave are not used (29.3.13).
- **Locations and settings.** The location's timezone drives every day and time calculation (`ClockService`); `location_settings` supplies the default duration and the daily limit.
- **Consultation (Chapter 31) and Billing (Chapter 34).** The encounter keeps the appointment id; automatic charges carry it too.
- **Dashboard and reports (Chapter 36).** The clinic overview and the visit reports read `appointments` and `queue_entries`.
- **Subscriptions (Chapter 45).** In read-only mode every write (book, edit, cancel, check-in) is refused with 402 `SUBSCRIPTION_EXPIRED`.

29.7 Limitations and known gaps

- Working hours are looked up by doctor and weekday and ignore the schedule's location, both for validation and for the calendar shading.
- Availability overrides and leave are ignored by booking, validation and the calendar.
- The "doctors" list (calendar columns, booking dropdown) contains every active staff member assigned to the location, not only doctors.
- Cancel sends no reason code and asks for no confirmation; the reason options exist in code but are unused.
- No screen can override booking rules, enter a reason for visit, or change the emergency flag on an existing booking.
- Convert-to-walk-in is hard to reach from the web app once the no-show job has run.
- The web app computes date ranges and shows times in the browser's timezone; the server applies the location's timezone. They agree only when the browser is set to the location's timezone.
- Booking and status changes are not written to the general audit log; only `created_by` / `updated_by` and timestamps record them. Queue events record what happens after check-in.
- There is no card view on the List tab, and the slot length (`appointment_slot_duration`) is only a form default, not an enforced grid.
- Reminders, recurring bookings and online booking are not implemented.

29.8 Owner's FAQ

Can two receptionists book the same doctor for the same time? No. The server checks for overlaps, and if two saves race, the database constraint `no_doctor_overlap` rejects the second with `DOCTOR_DOUBLE_BOOKED`. This holds across all locations.

Does a booking get a token? No. The token is issued at check-in by the queue. The booking shows "Assigned at check-in" until then.

What happens if a patient is very late? After `max_late_min` (45 minutes by default) check-in is refused with `LATE_LIMIT_EXCEEDED`, and within about a minute the no-show job marks the booking as a no-show. The patient can then be added as a walk-in.

Can a doctor see other doctors' bookings? With the default Doctor role, no: `appointments.view` has scope `own`. An owner can widen the scope in the role editor.

Can I book a doctor at a location where they are not assigned? No: "Doctor is not assigned to this location".

Does marking a booking "Emergency" put the patient first in the queue? No. Urgent priority is given at check-in or on the queue board, with a reason (Chapter 30).

Where is the daily booking limit set? In the location's settings (`location_settings.max_appointments_per_day`, default 50). Only users with `appointments.override_rules` can exceed it, and only through the API.

Can I undo a cancellation or a no-show? No. Both are final in the status map; create a new booking instead.

Where are bookings stored? In the `appointments` table, one row per booking, whatever happens to it.

Chapter 30 — Queue

This chapter describes the Queue module of the Platform: the doctor-wise waiting line that starts when a patient arrives at the clinic and ends when the doctor finishes the visit. It is written for the product owner, who needs to understand every queue action, the rules that decide who is called next, and the safety mechanisms behind them. Booking itself is covered by the Appointments chapter (Chapter 29); the clinical work done during the visit is covered by the Consultation chapter (Chapter 31).

30.1 What this module is for

Every doctor at a location has one **queue session** per clinic day: their own waiting line. When a patient arrives, either with a booking (check-in) or without one (**walk-in**), they get a **queue entry** in that doctor's session and a **token** such as "A-007". The doctor calls patients one at a time, starts the consultation, and completes it. Reception watches all doctors' lines at once, handles patients who did not come when called, patients sent for a test, transfers and emergencies. A TV screen in the waiting room shows which token is being served.

Who uses it:

- **Receptionist**: checks patients in, registers walk-ins, skips, rejoins, removes and transfers patients, returns patients from tests, prints tickets.
- **Assistant**: checks patients in, registers walk-ins and returns patients from tests.
- **Doctor**: runs their own queue on the doctor screen: start, pause, call next, select, consult, send for a test, transfer, stop, complete, close the day.
- **Admin** and the **owner**: everything, including running any doctor's queue, force-completing a visit, urgent priority, switching the queue order for the day, and editing queue settings.
- The **waiting room** itself: the public TV display, which needs no login.

Where it appears:

- The reception board is the **Queue** tab of the **Appointments** page (sidebar item **Appointments**, `/appointments`). Users with `queue.board.view` land on this tab.
- The doctor screen is the sidebar item **My Queue** (key `nav.queue.doctor`, route `/queue/doctor`, needs `queue.doctor.own`). It is the Doctor role's home page.
- The TV display (`/display/$locationId`) has no sidebar entry; it opens from a button on the reception board.
- Queue settings are a card on the location settings page (`/locations/$id/settings`), which has no sidebar entry of its own.

All queue permissions belong to the feature `appointments.core` ("Appointments", enabled by default). If that feature is switched off, the queue disappears with it.

30.2 Screens

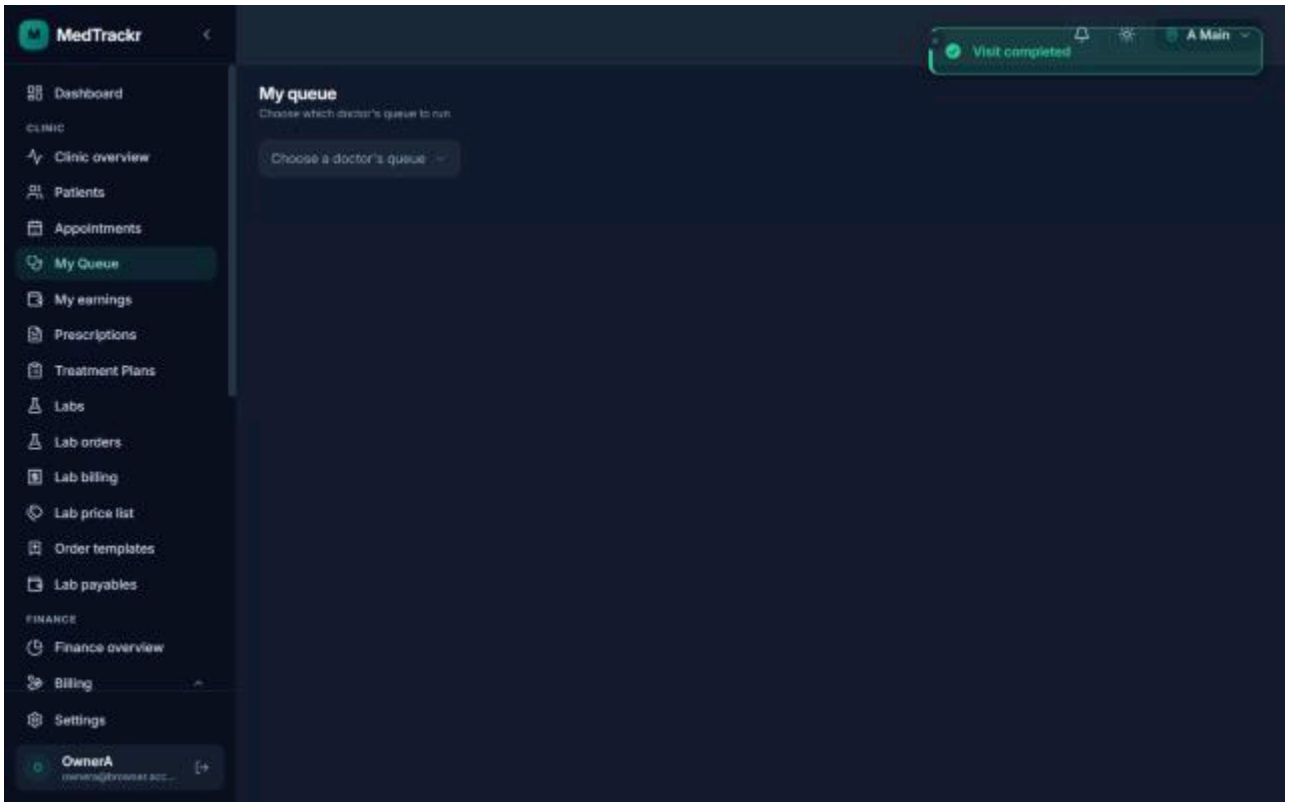

Figure 30.1 — The "My queue" page for a user who may run any doctor's queue but has no doctor profile at this location.

The screenshot was taken on 2026-10-23 in the dark theme and shows the former working name "MedTrackr" in the top-left corner. The left sidebar lists Dashboard, a "Clinic" group (Clinic overview, Patients, Appointments, My Queue highlighted, My earnings, Prescriptions, Treatment Plans, Labs, Lab orders, Lab billing, Lab price list, Order templates, Lab payables), a "Finance" group (Finance overview, Billing) and Settings, with a signed-in owner account at the bottom. The top bar shows the location switcher set to the main location, a theme toggle and a notifications bell. The page itself shows only the title "My queue", the subtitle "Choose which doctor's queue to run" and a dropdown "Choose a doctor's queue". A green toast "Visit completed" in the top-right corner is left over from finishing a consultation. This picker appears when the user holds `queue.doctor.any` but is not an employee at the location; after a doctor is chosen, the full doctor screen of Figure 30.2 opens for that doctor.

Screenshot pending

My queue - doctor screen

C:/Program Files/Git/queue/doctor

Figure 30.2 — The doctor screen, "My queue" (screenshot pending).

The header reads "My queue" with the doctor's name and the service date. On the right are a "Live" or "Reconnecting" indicator, a badge "Smart queue" or "First come, first served", the session status badge ("Doctor not started", "Open", "Paused", "Closed"), and the session buttons: **Start my queue**, **Pause** or **Resume**, and **Close day** (disabled while a patient is with the doctor). The left column has a "Current patient" card showing the token in large type, the patient's name and phone, the status, the call time, the reason for visit and badges. For a called patient it offers **Patient is here — start**, **Call again**, **Didn't come** and **Undo call**. For a patient in consultation it offers **Open consultation**, **Complete**, **Send for test**, **Transfer** and **Stop consultation**. Under the card is a large **Call next** button that shows the token and reason it would pick (for example "A-007 · Appointment due"), and a "completed today" count. The right column is the "Waiting" list, in the order Call next would pick them, each with badges and a **Select** button; the suggested patient is highlighted with "Suggested · reason". A small card lists patients "At a test — reception returns them" and "Skipped — reception can rejoin them".

Screenshot pending

Reception queue board - card view

`C:/Program Files/Git/appointments (Queue tab)`

Figure 30.3 — The reception queue board, card view (screenshot pending).

The board sits under the Appointments page header. Its filter bar has a search box ("Search token, name or phone…"), a doctor filter, a **Clear filters** button with a count when filters are active, a "Live"/"Reconnecting" indicator, a badge with the location's default queue order, a card/list switch, a refresh button, a button that opens the waiting-room TV display in a new tab, and **Walk-in**. In card view each doctor has a lane: the doctor's name, the token pattern ("Tokens A-xxx"), the number waiting and the queue order; the session status badge (with the pause reason); and, for users who may reprioritise, a settings menu "Today only" with "Switch to smart queue" or "Switch to first come, first served". The lane body shows "With doctor" (token, name, status, a **Force complete** button for admins when the consultation is running), the numbered waiting list in call order with badges (Emergency, Appointment 10:00 or Walk-in, Back from test, Transferred, minutes waited), an "At a test" group with **Back from test**, a "Skipped — didn't come when called" group with **Rejoin**, and a collapsible "Done today (n)". Each patient has an action menu: **Print ticket**, **Transfer to another doctor**, **Urgent priority**, **Remove from queue**. A called patient also has **Didn't come**. At the bottom is "Add walk-in for Dr. …". Loading, error ("Could not load the queue" with retry), empty ("No doctors at this location yet") and "No patients match your filters" states are handled.

Screenshot pending

Reception queue board - list view

`C:/Program Files/Git/appointments (Queue tab)`

Figure 30.4 — The reception queue board, list view (screenshot pending).

The list view shows every patient of every lane in one table with the columns Token, Patient, Doctor, Status, Arrived, an optional Billing column (for users with `billing.desk.use`, showing the patient's outstanding balance and a bill action once the visit is completed) and the same per-patient actions as the card view.

Screenshot pending

Check in patient dialog

`C:/Program Files/Git/appointments`

Figure 30.5 — The "Check in patient" dialog (screenshot pending).

The dialog shows the patient, the doctor and the booked time. If the booking is marked emergency, a note says that the emergency flag does not move the patient ahead on its own and that urgent priority can be given in this dialog. A checkbox **Give urgent priority?** reveals a required reason box. The buttons are **Cancel** and **Check in &**

get token. If the server answers that the patient is too late, the title changes to "Too late for the appointment", an amber box explains that the booking becomes a no-show and the patient joins the same doctor's queue as a walk-in with a new token, and the button becomes **Convert to walk-in**.

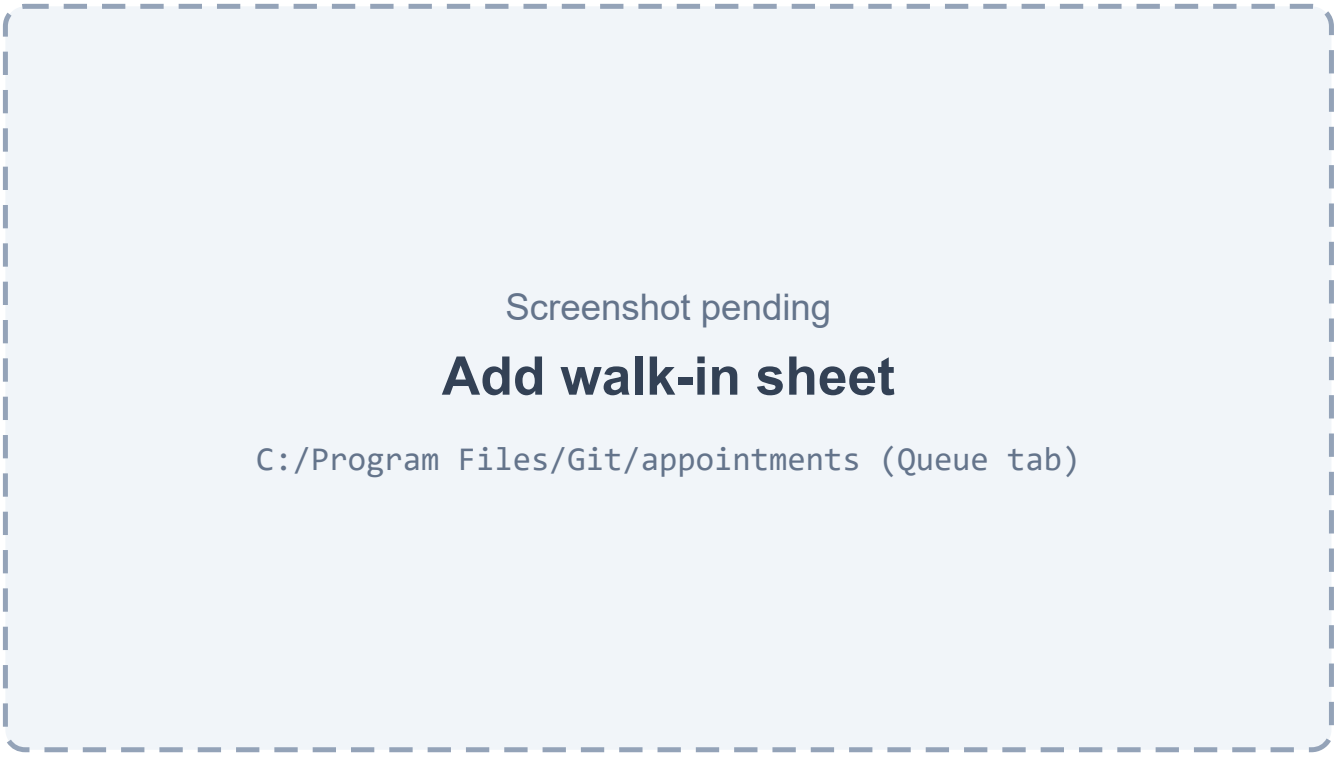

Figure 30.6 — The "Add walk-in" side sheet (screenshot pending).

The sheet slides in from the side with the title "Add walk-in" and the line "The patient gets the next token in the chosen doctor's queue." Two tabs choose **Existing patient** (search by name or phone, pick from results) or **New patient** (Patient name and Phone). Then come **Doctor** (required), **Reason for visit**, an **Urgent priority?** checkbox and, when ticked, a required **Urgent reason**. The button **Add to queue** is disabled until the form is valid. If the patient already has a token in this doctor's queue, an alert says so and nothing is added. If the patient is in another doctor's queue today, an alert lists those tokens and asks whether to add the patient to this doctor's queue as well, with an "add anyway" action.

Screenshot pending

Waiting-room TV display

`C:/Program Files/Git/display/$locationId`

Figure 30.7 — The public waiting-room TV display (screenshot pending).

A full-screen dark page with no menus. The header shows the location name, "Please wait for your token" and a large clock (with "Reconnecting…" when the live connection is down). Each doctor with a queue today that is not closed gets a panel: "Dr." and the doctor's name, a badge "On a break" (paused) or "Starting soon" (not started), "Now serving" with the token in very large type (pulsing green with "Please go to the doctor" while the patient is being called), "Next" with up to five tokens, and "n waiting". If no doctor has an open queue it shows "No doctors are seeing patients right now." No patient names appear anywhere.

Screenshot pending

Queue settings card

`C:/Program Files/Git/locations/$id/settings`

Figure 30.8 — The "Queue settings" card on the location settings page (screenshot pending).

The card says whether the location currently uses its own settings, the organization default or the built-in defaults. It holds the **Smart queue** switch, minute fields for the early window, late grace, maximum late and walk-in starvation limit, "Skipped patient rejoins" (end of line, original position, or manual), "Auto-skip after a call (seconds)" (empty means off), the "reason required for manual select" switch, "Service day starts at" (HH:MM), and "Save as" (**This location only** or **Organization default (all locations)**). A **Use organization default** button removes the location override. The card can be edited only while the location is the active one; otherwise it says "Switch to this location (top bar) to edit its queue settings."

30.3 Features

Every change to the queue in this section goes through one service, `QueueOrchestrator`, and follows the same pattern, so it is described once here and referred to as "the standard queue transaction". In one database transaction: set a 3-second lock timeout and the tenant context; claim the request's `Idempotency-Key` (a replay returns the saved response); lock the doctor's `queue_sessions` row (`SELECT … FOR UPDATE`); check the transition map; compare the client's `If-Match` version when sent; update the rows (each update bumps the entry's `version`); write one `queue_events` audit row and three `outbox` rows (rooms `session:<id>` , `location:<id>:board` , `location:<id>:display`) plus a `pg_notify` ; store the response under the key; commit. "Now" is the database clock of that transaction. Partial unique indexes on `queue_entries` (one active entry per appointment, one active entry per patient per session, at most one patient with the doctor per session) are the last line of defence.

30.3.1 Open a doctor's queue for the day

Status: [Implemented] — `POST /queue/sessions` has no web-app caller.

What it does. Makes sure a doctor has a queue session for today. A session is created automatically by the first check-in or walk-in, or when the doctor opens "My queue". A new session is in the state "Doctor not started" (`waiting_for_doctor`): reception can already add patients.

How the user does it.

1. A doctor opens **My Queue** in the sidebar; the session is created if missing.
2. A user who runs any doctor's queue without being a doctor at the location picks a doctor in "Choose a doctor's queue" (Figure 30.1).

Behind the scenes — in simple terms. One session per doctor, per location, per clinic day. The clinic day follows the location's timezone and the "service day starts at" setting, so a night clinic can keep 01:30 on the previous day.

Behind the scenes — technically. `GET /queue/my-session[?doctorId=]` (`DoctorQueueController.mySession`) and `POST /queue/sessions` (`QueueController.ensureSession` , `@AnyMember`) call `QueueOrchestrator.ensureTodaySession` , then return a snapshot (`QueueQueryService.snapshot`). `TokenEngine.ensureSession` inserts into `queue_sessions` with `ON CONFLICT ON CONSTRAINT uq_queue_session_doctor_day DO NOTHING` , so concurrent requests are safe. It copies the token prefix and the current `smart_queue_enabled` value into the session. The service date is `QueuePoliciesService.serviceDate` (location-local date of the time minus the cutoff). This path uses no idempotency key. Note that `GET /queue/my-session` writes a row, so it also works in read-only mode, where other writes are refused.

Rules and validations.

- The doctor must be active and actively assigned to the location.
- Reception (`queue.checkin`) may open any doctor's session; a doctor only their own unless they hold `queue.doctor.any` .

Who can do it. `GET /queue/my-session`: `queue.doctor.own` (with `queue.doctor.any` to name another doctor). `POST /queue/sessions`: any member, then `queue.checkin` or the right to run that session. Feature `appointments.core`. Default: Doctor (own), Admin (any); Receptionist and Assistant through check-in.

Data read and written. Reads `employee_org_location_assignments`, `employees`, `queue_policies`, `locations`. Writes `queue_sessions`.

Effects on other modules. None.

Error cases. 403 `FORBIDDEN` "You can only open your own queue (needs queue.doctor.any)"; 404 `NOT_FOUND` "Doctor profile for this user at this location not found" (the doctor screen then shows "You are not set up as a doctor at this location"); 404 "Doctor at this location not found".

Technical references: `backend/src/queue/controllers/doctor-queue.controller.ts` · `QueueOrchestrator.ensureTodaySession`, `TokenEngine.ensureSession` · `web-app/src/features/queue/pages/DoctorQueuePage.tsx`

30.3.2 Start, pause and resume the queue

Status: [Implemented]

What it does. The doctor starts their queue when ready, can pause it with a reason (break, prayer) and resume it. While a queue is not open, nobody can be called, but reception can keep checking patients in.

How the user does it.

1. On **My Queue**, click **Start my queue**.
2. Click **Pause**, type a reason in "Pause your queue" and confirm **Pause**.
3. Click **Resume** to continue.

Behind the scenes — in simple terms. The session moves between "Doctor not started", "Open" and "Paused". The pause reason is shown on the reception board, and the TV shows "On a break".

Behind the scenes — technically. `PATCH /queue/sessions/:id/start`, `/pause` (body `{ reason }`), `/resume` → `QueueOrchestrator.startSession` / `pauseSession` / `resumeSession` → `sessionChange`: lock the session, check the actor may run it, `assertSessionTransition`, update status (start sets `started_at` the first time; pause stores `pause_reason`; resume clears it), emit `SESSION_STARTED` / `SESSION_PAUSED` / `SESSION_RESUMED`. Standard queue transaction.

Rules and validations.

- Start only from `waiting_for_doctor`; pause only from `open`; resume only from `paused`.
- A pause reason is required (1–500 characters).

Who can do it. `queue.doctor.own` in one's own session, or `queue.doctor.any` for any session; feature `appointments.core`. Default: Doctor (own), Admin (any).

Data read and written. Writes `queue_sessions`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. The TV display and reception board update live.

Error cases. 422 `INVALID_TRANSITION` "Cannot move a open session to open"; 403 `FORBIDDEN` "Only this queue's doctor can do that"; 400 on a missing pause reason.

Technical references: `backend/src/queue/queue-transitions.ts` · `QueueOrchestrator.sessionChange` · `web-app/src/features/queue/pages/DoctorQueuePage.tsx`

30.3.3 Close the day

Status: [Implemented]

What it does. Ends the doctor's queue for the day. Every patient still waiting, skipped or at a test must get a decision first: cancel, or (for waiting patients) transfer to another doctor.

How the user does it.

1. Finish or undo the current patient (the **Close day** button is disabled while someone is with the doctor).
2. Click **Close day**. The "Close today's queue" dialog lists each remaining patient with a choice: **Cancel** (the default) or "Transfer → Dr. …".
3. Confirm. A toast says "Queue closed for today".

Behind the scenes — in simple terms. Each remaining patient is removed or moved, then the session is closed and accepts no more patients that day. A background job also closes past days' empty sessions on its own (30.3.25).

Behind the scenes — technically. `PATCH /queue/sessions/:id/close` with `{ decisions: [{ entryId, action: 'cancel' | 'transfer', toDoctorId? }] }` → `QueueOrchestrator.closeSession`: lock, check the actor, `assertSessionTransition(→ closed)`, `assertDoctorFree`, load remaining entries (waiting, skipped, on hold), refuse if any has no decision, then cancel (reason "Session closed") or transfer each one (target session ensured for the same service date), set `status = closed`, `closed_at`, emit `SESSION_CLOSED`. Standard queue transaction; everything succeeds or nothing does.

Rules and validations.

- Nobody may be called or in consultation.
- Every remaining entry needs a decision (up to 500 decisions).
- Only waiting entries can be transferred; skipped or at-test patients must be cancelled or brought back first.
- Cancelling a booked patient also closes the appointment as cancelled (`patient_left`).

Who can do it. `queue.doctor.own` in one's own session, or `queue.doctor.any`; feature `appointments.core`. Default: Doctor (own), Admin.

Data read and written. Writes `queue_sessions`, `queue_entries`, `appointments`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. Cancelled booked patients become cancelled appointments; transferred patients appear in another doctor's lane with new tokens.

Error cases. 409 `DOCTOR_BUSY` "The doctor already has a patient. Finish or undo that call first."; 422 `INVALID_TRANSITION` "Decide (cancel or transfer) for every remaining patient before closing" (with the undecided ids); 422 "Entry … is skipped; only waiting patients can be transferred".

Technical references: `backend/src/queue/dto/queue.dto.ts` · `QueueOrchestrator.closeSession` · `web-app/src/features/queue/components/CloseSessionDialog.tsx`

30.3.4 Switch between smart queue and first come, first served for today

Status: [Implemented]

What it does. Changes the calling order of one doctor's session for today only, without touching the clinic's settings.

How the user does it.

1. On the reception board, open the settings icon of a doctor's lane ("Queue options for Dr. …").

2. Choose "Switch to smart queue" or "Switch to first come, first served".

Behind the scenes — in simple terms. Each session copies the clinic's queue order when it is created. This switch changes only that copy.

Behind the scenes — technically. `PATCH /queue/sessions/:id/mode` with `{ smartQueueEnabled }` → `QueueOrchestrator.setMode`: lock, update `smart_queue_enabled`, emit `MODE_CHANGED`. No change if already in that mode.

Rules and validations.

- Not offered for closed sessions in the web app.

Who can do it. `queue.entry.reprioritize`; feature `appointments.core`. Default: Admin only.

Data read and written. Writes `queue_sessions`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. None beyond the call order.

Error cases. 403 `NO_PERMISSION` / `FORBIDDEN` "You cannot change the queue order".

Technical references: `backend/src/queue/controllers/doctor-queue.controller.ts` · `QueueOrchestrator.setMode` · `web-app/src/features/queue/components/QueueLaneCard.tsx`

30.3.5 Doctor-wise tokens

Status: [Implemented]

What it does. Gives each arriving patient a short number that belongs to one doctor's queue for one day, such as "A-007". Tokens are never reused and have no gaps.

How the user does it. Automatic at check-in, walk-in, convert-to-walk-in and transfer. The token appears in the toast, on the ticket, on the board, on the doctor screen and on the TV.

Behind the scenes — in simple terms. Each session keeps its own counter. The next token is the counter plus one. Because the counter lives in the same row that every change locks, two desks can never get the same number, and a failed check-in gives its number back.

Behind the scenes — technically. `TokenEngine.nextToken` runs `UPDATE queue_sessions SET last_token_number = last_token_number + 1 … WHERE id = $1 AND status <> 'closed' RETURNING last_token_number` inside the caller's transaction (no database sequence, so a rollback leaves no gap). `TokenEngine.format` gives prefix + "-" + the number padded to three digits (four digits after 999). The prefix is the doctor's `employee_org_location_assignments.token_prefix` at that location, otherwise the first letter of the doctor's name without a leading "Dr.", upper-cased, letters and digits only, at most 6 characters, else "T". It is copied into the session at creation. `queue_entries` has a unique constraint on (session, token number).

Rules and validations.

- A closed session issues no tokens.
- Two doctors whose names start with the same letter get the same prefix unless a prefix is set on their assignment.

Who can do it. Follows the arrival action (`queue.checkin`).

Data read and written. Reads `employee_org_location_assignments`, `employees`. Writes `queue_sessions.last_token_number`.

Effects on other modules. The token is shown on the appointment (list and detail) and in the patient's visit history.

Error cases. 422 `SESSION_NOT_OPEN` "This queue is closed for the day".

Technical references: `backend/src/queue/services/token-engine.service.ts` · `TokenEngine` · `web-app/src/features/queue/utils/printTicket.ts`

30.3.6 Check in a booked patient

Status: [Implemented]

What it does. Turns a booking into a queue entry with a token when the patient arrives. The Appointments chapter (Chapter 29, feature "Check a patient in") describes the screen steps; this feature covers the queue side.

How the user does it.

1. Click **Check in** on a booking (list row menu or detail dialog).
2. In "Check in patient" (Figure 30.5), optionally tick **Give urgent priority?** with a reason, and click **Check in & get token**.
3. If the patient is too late, click **Convert to walk-in** instead.

Behind the scenes — in simple terms. The patient joins the booked doctor's queue for today. A booked patient is not automatically first: the selection rules (30.3.8) decide when their scheduled time makes them "due".

Behind the scenes — technically. `PATCH /appointments/:id/check-in` → `QueueOrchestrator.checkInAppointment`, standard queue transaction. It locks the appointment row; returns 409 `ALREADY_CHECKED_IN` (with the existing entry) if an active entry exists; refuses a `no_show` booking with `LATE_LIMIT_EXCEEDED`; checks the appointment transition; checks the booking's service date is today (`NOT_TODAY`); checks `isBeyondMaxLate(start, now, max_late_min)`; opens and locks the session (`waiting_for_doctor`, `open` or `paused` accept arrivals); `issueToken` inserts the entry (`queue_type = appointment`, `waiting`, `checked_in_at = sort_anchor_at = now`, reason for visit copied from the booking) and emits `CHECKED_IN` (plus `PRIORITY_SET` if urgent); sets the appointment `checked_in`. `POST /appointments/:id/convert-to-walk-in` → `QueueOrchestrator.convertToWalkIn` is the late variant (Chapter 29).

Rules and validations.

- Booking must be scheduled or confirmed, for today's service date, and not more than `max_late_min` late (default 45).
- The session must not be closed.
- Urgent reason up to 500 characters; urgent priority sets `priority_class = authorized_urgent` and `urgent_since`.

Who can do it. `queue.checkin` (implies `queue.board.view`); feature `appointments.core`. Default: Admin, Receptionist, Assistant.

Data read and written. Writes `appointments`, `queue_sessions`, `queue_entries`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. The appointment becomes `checked_in` and locked for editing.

Error cases. 409 `ALREADY_CHECKED_IN`; 422 `LATE_LIMIT_EXCEEDED` "More than 45 minutes late — convert to a walk-in or reschedule"; 422 `NOT_TODAY`; 422 `SESSION_NOT_OPEN`; 503 `LOCK_TIMEOUT` "The queue is busy, please retry" (the web app retries automatically).

Technical references: `backend/src/queue/engines/check-in-rules.ts` · `QueueOrchestrator.checkInAppointment`, `issueToken` · `web-app/src/features/queue/components/CheckInDialog.tsx`

30.3.7 Register a walk-in

Status: [Implemented]

What it does. Adds a patient without a booking to a chosen doctor's queue, creating the patient record on the spot if needed.

How the user does it.

1. On the reception board click **Walk-in** (or "Add walk-in for Dr. …" in a lane).
2. In "Add walk-in" (Figure 30.6), choose **Existing patient** and search, or **New patient** and enter name and phone.
3. Choose the **Doctor**, optionally a **Reason for visit** and **Urgent priority?** with a reason.
4. Click **Add to queue**. A toast shows "Token A-008 — patient name" with **Print ticket**.

Behind the scenes — in simple terms. If the patient already waits for the same doctor, nothing is added and their token is shown. If they wait for another doctor today, the desk is warned and must confirm. A new patient gets the next patient number.

Behind the scenes — technically. `POST /queue/walk-ins` → `QueueOrchestrator.registerWalkIn`, standard queue transaction. `assertDoctorAtLocation`; `resolvePatient` either loads the patient (same organization) or creates one: it takes a transaction-level advisory lock per organization, computes the next number as `P-` plus five digits, saves the patient with the active location as primary location, and writes a patient audit record. It ensures and locks today's session, refuses a duplicate active entry in that session (`ALREADY_CHECKED_IN`), and unless `force` is true refuses when the patient has active entries in other sessions at the location today (`ACTIVE_WITH_OTHER_DOCTOR`, listing tokens and doctors). Then `issueToken` with `queue_type = walk_in`.

Rules and validations.

- Doctor required. Either `patientId`, or name (up to 150 characters) and phone (up to 20; the form accepts digits, spaces, `+`, `-` and parentheses, 7–20 characters).
- Reason for visit and urgent reason up to 500 characters; an urgent walk-in needs a reason.

Who can do it. `queue.checkin`; feature `appointments.core`. Default: Admin, Receptionist, Assistant.

Data read and written. Reads `patients`, `employee_org_location_assignments`. Writes `patients` (new patient), patient audit log, `queue_sessions`, `queue_entries`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. A new patient appears in Patients (Chapter 28). Walk-ins appear in the appointments list, the KPI strip and the reports.

Error cases. 409 `ALREADY_CHECKED_IN` (sheet: "Already in this doctor's queue with token … Nothing was added."); 409 `ACTIVE_WITH_OTHER_DOCTOR` "This patient is already in another doctor's queue today"; 404 "Patient not found"; 404 "Doctor at this location not found"; 422 `SESSION_NOT_OPEN`.

Technical references: `backend/src/queue/controllers/queue.controller.ts` · `QueueOrchestrator.registerWalkIn`, `resolvePatient` · `web-app/src/features/queue/components/WalkInSheet.tsx`

30.3.8 Call the next patient (smart and first-come rules)

Status: [Implemented]

What it does. With one button, the doctor calls whoever should be seen next. The patient's token starts flashing on the TV.

How the user does it.

1. On **My Queue**, with the queue open and nobody with the doctor, click **Call next**. The button already shows which token and why, for example "(A-007 · Appointment due)".

Behind the scenes — in simple terms. A small rule engine looks at everyone waiting and picks one. It has two modes.

In **first come, first served** mode: an urgent patient goes first (earliest urgent first); otherwise the patient with the earliest place in line goes first. The place in line is the arrival time, or the rejoin time after a "rejoin at end".

In **smart** mode the order is:

1. **Emergency** — an urgent patient, earliest first.
2. **Waited too long** (starvation guard) — if an appointment patient is about to overtake, and the patient who has waited longest in the ordinary line has waited at least the starvation limit (45 minutes by default), that patient goes first. The guard never fires twice in a row, so the due appointment goes right after.
3. **Appointment due** — a booked patient whose time has come: it is no earlier than 15 minutes (early window) before the booked time, and the patient arrived no later than 10 minutes (late grace) after it. Earliest booked time first.
4. **Back from test** — a patient returning from a test, earliest return first.
5. **Next in line** — everyone else (walk-ins, appointment patients who came very early, and those who arrived after the grace period) by place in line.

Ties are broken by the lower token number. "Due" depends on the clock, so priority is recalculated at every call; it is never stored.

The specification's 14 scenarios, each tested in both modes, show the effect. A 10:00 appointment who arrived at 9:58 is called at 10:01 before a walk-in waiting since 9:40 (smart), but after him in first-come mode. An appointment patient who arrived at 9:20 for 10:00 is not yet due at 9:35, so he queues by arrival. A walk-in who has waited 50 minutes is called before a due appointment; on the very next call the appointment goes. An appointment patient who arrives 25 minutes late has lost the priority and queues behind a walk-in who arrived at 10:15. A patient back from a test goes ahead of walk-ins in smart mode only. An urgent patient goes first in both modes. Two patients with the same time are ordered by token. With nobody waiting, nothing is called. Scenario 7 tests the separate check-in rule: a 10:00 patient arriving at 10:50 is beyond the 45-minute maximum and cannot check in.

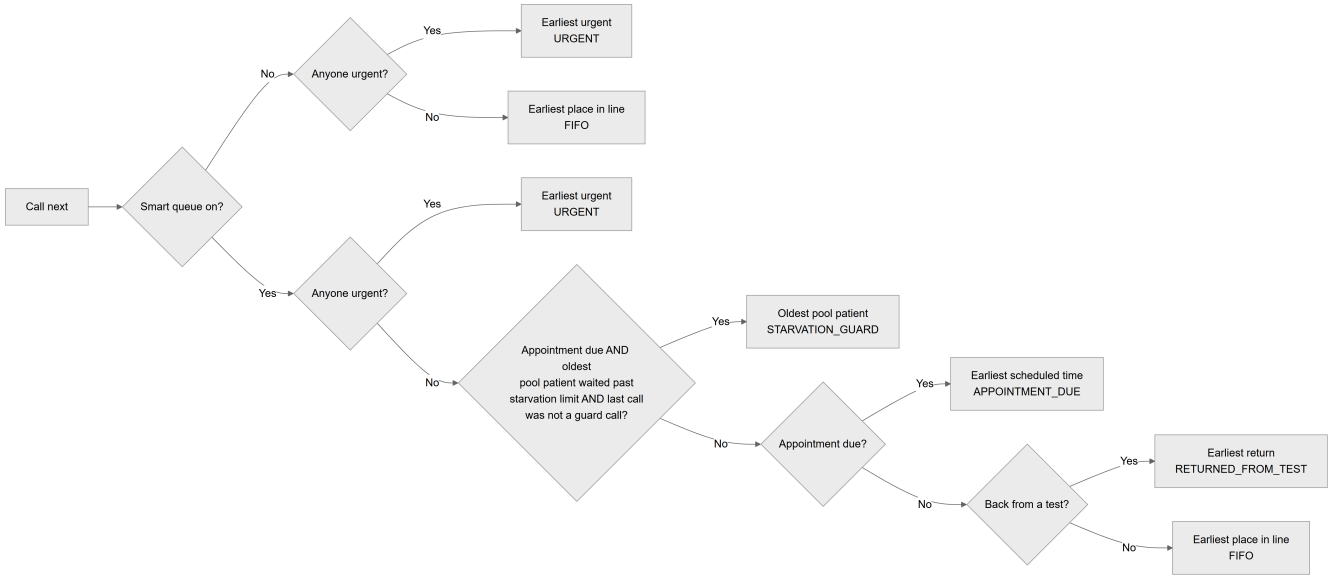

Figure 30.9 — The selection engine's decision path.

The flowchart starts at "Call next" and asks "Smart queue on?". On "No", it asks "Anyone urgent?": yes leads to "Earliest urgent, URGENT"; no leads to "Earliest place in line, FIFO". On "Yes", it asks "Anyone urgent?": yes leads to "Earliest urgent, URGENT". Otherwise it asks whether an appointment is due and the oldest pool patient has waited past the starvation limit and the last call was not a guard call: yes leads to "Oldest pool patient, STARVATION_GUARD". Otherwise "Appointment due?": yes leads to "Earliest scheduled time, APPOINTMENT_DUE". Otherwise "Back from a test?": yes leads to "Earliest return, RETURNED_FROM_TEST"; no leads to "Earliest place in line, FIFO". The capitalised words are the reason codes stored with each call and shown to the doctor as Emergency, Waited too long, Appointment due, Back from test and Next in line.

Behind the scenes — technically. `POST /queue/sessions/:id/call-next` → `QueueOrchestrator.callNext`, standard queue transaction: lock, check the actor, `assertOpen`, `assertDoctorFree`, load waiting entries, call the pure function `selectNext` (`engines/selection-engine.ts`) with the transaction's `now`, the session's mode, the policy's `early_window_min`, `late_grace_min`, `walkin_starvation_min` and the session's `last_call_reason`. It stores the new `last_call_reason`, moves the entry to `called` with `called_at`, and emits `CALLED` with the reason code. With nobody eligible it returns `entry: null` and reason `NO_ELIGIBLE_PATIENT` ("No patient is waiting"). The snapshot the screens load contains `waitingOrder` (the engine applied repeatedly) and `suggestion`.

Rules and validations.

- The session must be `open`; nobody may already be called or in consultation (also guaranteed by the index `one_patient_with_doctor`).
- Only `waiting` entries are candidates; skipped and at-test patients are not.

Who can do it. `queue.doctor.own` in one's own session, or `queue.doctor.any`; feature `appointments.core`. Default: Doctor (own), Admin.

Data read and written. Reads `queue_entries`, `appointments` (booked time), `queue_policies`. Writes `queue_sessions`, `queue_entries`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. The TV shows the called token; the board moves the patient to "With doctor".

Error cases. 422 `SESSION_NOT_OPEN` "The doctor has paused this queue" / "The doctor has not started this queue yet"; 409 `DOCTOR_BUSY`; 403 `FORBIDDEN`.

Technical references: `backend/src/queue/engines/selection-engine.ts` (+ `selection-engine.spec.ts`) · `selectNext`, `QueueOrchestrator.callNext` · `web-app/src/features/queue/pages/DoctorQueuePage.tsx`

30.3.9 Select a specific patient

Status: [Implemented]

What it does. Lets the doctor call a patient out of the engine's order. The engine's own suggestion is recorded next to the choice.

How the user does it.

1. In the "Waiting" list, click **Select** next to a patient.
2. If the patient is not the suggestion (or a reason is required by settings), the dialog "Call A-009 now?" explains whom the queue would have called and accepts a reason; confirm **Call this patient**.

Behind the scenes — in simple terms. The chosen patient is called, and the queue history notes who the engine would have picked.

Behind the scenes — technically. `PATCH /queue/entries/:id/select` with `{ reason? }` and `If-Match` → `QueueOrchestrator.selectPatient`: lock via the entry, `assertOpen`, `assertDoctorFree`, enforce `manual_select_reason_required`, compute the engine's suggestion, set `last_call_reason = MANUAL_SELECT`,

transition `waiting → called`, emit `CALLED` with `MANUAL_SELECT`, the reason text and `suggested_entry_id`.

Rules and validations.

- As Call next, plus a reason when the location requires one.
- The entry must still be at the version the screen showed.

Who can do it. `queue.doctor.own` (own session) or `queue.doctor.any`; feature `appointments.core`. Default: Doctor (own), Admin.

Data read and written. As Call next.

Effects on other modules. None beyond the queue.

Error cases. 400 `REASON_REQUIRED` "A reason is required to select a patient out of order"; 409 `VERSION_CONFLICT` "This was changed by someone else. The latest state is shown."; 409 `DOCTOR_BUSY`; 422 `INVALID_TRANSITION`.

Technical references: `backend/src/queue/services/queue-orchestrator.service.ts` · `QueueOrchestrator.selectPatient` · `web-app/src/features/queue/pages/DoctorQueuePage.tsx`

30.3.10 Call again (recall)

Status: [Implemented]

What it does. Announces a called patient once more without changing anything.

How the user does it.

1. With a called patient, click **Call again**.

Behind the scenes — in simple terms. Only a history entry and a live nudge are written, so screens refresh.

Behind the scenes — technically. `PATCH /queue/entries/:id/recall` → `QueueOrchestrator.recall`: the entry must be `called`; emits `RECALLED` (from and to `called`). No If-Match.

Rules and validations. Only for a `called` entry.

Who can do it. `queue.doctor.own` (own session) or `queue.doctor.any`; feature `appointments.core`. Default: Doctor (own), Admin.

Data read and written. Writes `queue_sessions.event_seq`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. None. The TV has no sound or extra animation for a recall; it refreshes.

Error cases. 422 `INVALID_TRANSITION` "Only a called patient can be recalled".

Technical references: `backend/src/queue/controllers/doctor-queue.controller.ts` · `QueueOrchestrator.recall` · `web-app/src/features/queue/pages/DoctorQueuePage.tsx`

30.3.11 Undo a call

Status: [Implemented]

What it does. Reverses a wrong call: the patient goes back to waiting in the same place.

How the user does it.

1. With a called patient, click **Undo call**.

Behind the scenes — in simple terms. The patient's place in line is unchanged, so they are not penalised.

Behind the scenes — technically. `PATCH /queue/entries/:id/undo-call` with `If-Match` → `QueueOrchestrator.undoCall`: transition `called → waiting`, clear `called_at`, emit `CALL_UNDONE`. `sort_anchor_at` is untouched.

Rules and validations. Only from `called`.

Who can do it. `queue.doctor.own` (own session) or `queue.doctor.any`; feature `appointments.core`. Default: Doctor (own), Admin.

Data read and written. Writes `queue_entries`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. None.

Error cases. 422 `INVALID_TRANSITION`; 409 `VERSION_CONFLICT`.

Technical references: `backend/src/queue/queue-transitions.ts` · `QueueOrchestrator.undoCall` · `web-app/src/features/queue/pages/DoctorQueuePage.tsx`

30.3.12 Skip a patient who did not come

Status: [Implemented]

What it does. Marks a called patient who did not come to the room as skipped. They leave the waiting list until reception rejoins them. The skip can also happen automatically after a set number of seconds (30.3.25).

How the user does it.

1. The doctor clicks **Didn't come** on the current patient, or reception clicks **Didn't come** in the lane.

Behind the scenes — in simple terms. The patient moves to the "Skipped" group, and the doctor is free to call the next one.

Behind the scenes — technically. `PATCH /queue/entries/:id/skip` with `If-Match` (`@AnyMember`) → `QueueOrchestrator.skip`: `assertHandleEntry` (entry managers anywhere, doctors in their own session), transition `called → skipped`, set `skipped_at`, emit `SKIPPED` (reason code `CALL_TIMEOUT` when the job did it).

Rules and validations. Only from `called`.

Who can do it. `queue.entry.manage`, or the session's doctor (`queue.doctor.own` / `queue.doctor.any`); feature `appointments.core`. Default: Receptionist, Admin, Doctor (own session).

Data read and written. Writes `queue_entries`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. None.

Error cases. 403 `FORBIDDEN` "Only reception or this queue's doctor can do that"; 422 `INVALID_TRANSITION`; 409 `VERSION_CONFLICT`.

Technical references: `backend/src/queue/queue-actor.ts` · `QueueOrchestrator.skip`, `canHandleEntry` · `web-app/src/features/queue/components/QueueEntryActions.tsx`

30.3.13 Rejoin a skipped patient

Status: [Implemented]

What it does. Puts a skipped patient back into the waiting list, at a place set by the clinic's "Skipped patient rejoins" setting.

How the user does it.

1. In the lane's "Skipped" group, click **Rejoin**.

2. If the setting is "manual" and others are waiting, first pick "Place after…" a token, then click **Rejoin**.

Behind the scenes — in simple terms. "End" puts the patient at the back of the line (default). "Original position" keeps their original arrival place. "Manual" places them right after a chosen patient.

Behind the scenes — technically. `PATCH /queue/entries/:id/rejoin` with `{ placeAfterEntryId? }` and `If-Match` → `QueueOrchestrator.rejoin`: `assertManageEntries`; compute the new `sort_anchor_at` (now for `end`; unchanged for `original_position`; the chosen entry's anchor plus one millisecond for `manual`); transition to `waiting`, clear `called_at`, emit `REJOINED` with the policy as reason code. The transition map also allows this from `called` and `on_hold`, but the web app offers it only for skipped patients.

Rules and validations.

- Manual policy needs `placeAfterEntryId` when anyone else is waiting; the target must be a waiting entry of the same session.

Who can do it. `queue.entry.manage`; feature `appointments.core`. Default: Receptionist, Admin.

Data read and written. Reads `queue_policies`. Writes `queue_entries`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. None.

Error cases. 422 `INVALID_TRANSITION` "Choose where to place the patient (placeAfterEntryId)"; 404 "Waiting entry to place after not found"; 403 `FORBIDDEN` "You cannot manage queue entries".

Technical references: `backend/src/queue/enums/queue.enums.ts` (`RejoinPolicy`) · `QueueOrchestrator.rejoin` · `web-app/src/features/queue/components/QueueEntryActions.tsx`

30.3.14 Send for a test and return from the test

Status: [Implemented]

What it does. During a consultation the doctor can send the patient for a test (an X-ray, a blood test). The patient keeps their token. When they come back, reception returns them to the waiting list, where in smart mode they go ahead of walk-ins.

How the user does it.

1. Doctor: with the patient in consultation, click **Send for test**, enter the test (for example "X-ray") and confirm **Send for test**.
2. Reception: in the lane's "At a test" group, click **Back from test**.

Behind the scenes — in simple terms. The doctor becomes free for the next patient while this one is away. On return the patient waits again, with a "Back from test" badge.

Behind the scenes — technically. `PATCH /queue/entries/:id/send-for-test` with `{ reason? }` → `QueueOrchestrator.sendForTest`: transition `in_consultation → on_hold`, set `hold_started_at`, emit `SENT_FOR_TEST`. `PATCH /queue/entries/:id/return-from-test` → `QueueOrchestrator.returnFromTest`: transition `on_hold → waiting`, set `returned_from_hold_at`, emit `RETURNED_FROM_TEST`. The selection engine uses `returned_from_hold_at` for the "Back from test" rule. The encounter stays open throughout.

Rules and validations.

- Send only from `in_consultation`; return only from `on_hold`.
- The return action has no per-session check beyond `queue.checkin`.

Who can do it. Send: `queue.doctor.own` (own session) or `queue.doctor.any`. Return: `queue.checkin`. Feature `appointments.core`. Default: send — Doctor (own), Admin; return — Receptionist, Assistant, Admin.

Data read and written. Writes `queue_entries`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. None.

Error cases. 422 `INVALID_TRANSITION`; 409 `VERSION_CONFLICT`.

Technical references: `backend/src/queue/controllers/queue.controller.ts` · `QueueOrchestrator.sendForTest`, `returnFromTest` · `web-app/src/features/queue/components/QueueEntryActions.tsx`

30.3.15 Transfer a patient to another doctor

Status: [Implemented]

What it does. Moves a waiting patient, or a patient in consultation, to another doctor's queue at the same location. The patient gets a new token there but keeps their original place in line by arrival time.

How the user does it.

1. Reception: in a waiting patient's menu choose **Transfer to another doctor**. Doctor: with the patient in consultation click **Transfer**.
2. Pick the doctor, add an optional reason, confirm. A toast shows "Transferred — new token B-004".

Behind the scenes — in simple terms. The old entry is closed as "transferred" and a new entry is created in the other doctor's queue for the same day, carrying the booking link, the urgent flag and the arrival time.

Behind the scenes — technically. `PATCH /queue/entries/:id/transfer` with `{ toDoctorId, reason? }` and `If-Match` (`@AnyMember`) → `QueueOrchestrator.transfer` → `transferEntry` : `assertHandleEntry`; refuse the same doctor; check the target doctor is at the location; ensure the target session for the same service date; lock both sessions in id order (avoids deadlocks); refuse a closed target; transition the old entry to `transferred` (`TRANSFERRED_OUT` with the target session); allocate a token in the target; insert a new entry with `transferred_from_id`, the same `appointment_id`, `converted_from_appt_id`, priority, urgent fields, reason, `checked_in_at` and `sort_anchor_at`; emit `CHECKED_IN` with reason code `TRANSFERRED_IN` in the target session.

Rules and validations.

- From `waiting` or `in_consultation` only (transition map). The web app offers it to reception for waiting patients and to the doctor for the patient in consultation.
- An encounter already started stays with the old entry; the new doctor's consultation starts a new encounter.

Who can do it. `queue.entry.manage`, or the doctor of the entry's session; feature `appointments.core`. Default: Receptionist, Admin, Doctor (own session).

Data read and written. Writes `queue_sessions` (both), `queue_entries` (old and new), `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. A booked appointment stays `checked_in` and follows the new entry.

Error cases. 422 `INVALID_TRANSITION` "The patient is already in this doctor's queue"; 404 "Doctor at this location not found"; 422 `SESSION_NOT_OPEN`; 409 `VERSION_CONFLICT`.

Technical references: `backend/src/queue/services/queue-orchestrator.service.ts` · `QueueOrchestrator.transfer`, `transferEntry` · `web-app/src/features/queue/components/TransferDialog.tsx`

30.3.16 Give urgent priority

Status: [Implemented]

What it does. Moves a patient ahead of everyone else waiting for the same doctor, for a stated reason (for example "Severe bleeding"). It works in both queue modes.

How the user does it.

1. In a patient's menu choose **Urgent priority**, enter the reason and click **Give urgent priority**.
2. Or tick **Give urgent priority?** / **Urgent priority?** at check-in or walk-in.

Behind the scenes — in simple terms. The patient gets an "Emergency" badge and is called first. Several urgent patients are called in the order they were made urgent.

Behind the scenes — technically. `PATCH /queue/entries/:id/priority` with `{ reason }` and `If-Match` → `QueueOrchestrator.setPriority` : `assertReprioritize` ; entry must be `waiting` , `skipped` or `on_hold` ; set `priority_class = authorized_urgent` , keep an existing `urgent_since` or set it to now, set `urgent_reason` , bump the version, emit `PRIORITY_SET` . A database check requires `urgent_since` and `urgent_reason` whenever the class is urgent. There is no endpoint to remove urgent priority.

Rules and validations.

- Reason required, up to 500 characters.
- At check-in and walk-in the urgent option needs only `queue.checkin` ; on an existing entry it needs `queue.entry.reprioritize` .

Who can do it. `queue.entry.reprioritize` ; feature `appointments.core` . Default: Admin only.

Data read and written. Writes `queue_entries` , `queue_events` , `outbox` , `idempotency_keys` .

Effects on other modules. None.

Error cases. 422 `INVALID_TRANSITION` "Priority can only be given to a patient who is still waiting"; 403 "You cannot change the queue order"; 409 `VERSION_CONFLICT` .

Technical references: `backend/src/queue/entities/queue-entry.entity.ts` · `QueueOrchestrator.setPriority` · `web-app/src/features/queue/components/QueueBoard.tsx`

30.3.17 Start the consultation

Status: [Implemented]

What it does. Confirms the called patient is in the room, starts the consultation and creates the **encounter** (the clinical record of this visit). The web app then opens the consultation workspace.

How the user does it.

1. With a called patient, click **Patient is here — start**. The consultation workspace opens.
2. Later, **Open consultation** returns to the workspace.

Behind the scenes — in simple terms. The clinical record begins only when the doctor actually starts, never at booking or check-in.

Behind the scenes — technically. `PATCH /queue/entries/:id/start-consultation` with `If-Match` → `QueueOrchestrator.startConsultation` : transition `called → in_consultation` , set `consultation_started_at` , emit `CONSULTATION_STARTED` , then `EncountersService.ensureForQueueEntry` in the same transaction creates a

`patient_encounters` row (unique per queue entry; copies `appointment_id`, patient, organization, location and reason) or returns the existing one. The response includes `encounterId`; the web app navigates to `/consultation/$encounterId`.

Rules and validations. Only from `called`.

Who can do it. `queue.doctor.own` (own session) or `queue.doctor.any`; feature `appointments.core`. Default: Doctor (own), Admin.

Data read and written. Writes `queue_entries`, `patient_encounters`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. Opens the consultation workspace (Chapter 31). `consultation_started_at` feeds the "Avg wait" figure.

Error cases. 422 `INVALID_TRANSITION`; 409 `VERSION_CONFLICT`; 403 `FORBIDDEN`.

Technical references: `backend/src/encounters/encounters.service.ts` · `QueueOrchestrator.startConsultation`, `EncountersService.ensureForQueueEntry` · `web-app/src/features/queue/pages/DoctorQueuePage.tsx`

30.3.18 Complete the visit (and bill it automatically)

Status: [Implemented]

What it does. Ends the consultation. A booked appointment becomes completed, and the visit's charges (consultation fee and recorded treatments) are created for billing at the same moment.

How the user does it.

1. On the doctor screen click **Complete**, or press complete in the consultation workspace (the toast says "Visit completed" and the doctor returns to **My Queue**).

Behind the scenes — in simple terms. The patient leaves the queue as completed. The billing module receives a consultation charge and one charge per treatment recorded in the encounter, priced from the price list, with the doctor's commission noted. Nothing about money is sent back to the doctor's screen.

Behind the scenes — technically. `PATCH /queue/entries/:id/complete` with `If-Match` → `QueueOrchestrator.complete`: transition `in_consultation → completed`, set `finished_at`, emit `COMPLETED`; if booked, update the appointment to `completed` with `completed_at`; then `ChargeSourcesService.syncForEncounter` inside the same transaction. That service does nothing if the `billing.invoices` feature is off or if no encounter exists. Otherwise it adds a consultation charge if none is active, adds a treatment charge for each treatment of the encounter not yet billed, snapshots the doctor's commission in force on the clinic date, voids unbilled charges whose treatment was removed, and writes billing audit rows. Inserts use `ON CONFLICT DO NOTHING`, so a repeat never duplicates charges. The Billing chapter (Chapter 34) and the Commissions chapter (Chapter 35) cover the details.

Rules and validations. Only from `in_consultation`; the actor must run this session.

Who can do it. `queue.doctor.own` (own session) or `queue.doctor.any`; feature `appointments.core` (charges also need `billing.invoices`). Default: Doctor (own), Admin.

Data read and written. Writes `queue_entries`, `appointments`, `charges`, `billing_audit_logs`, `queue_events`, `outbox`, `idempotency_keys`; reads `patient_encounters`, `treatments`, prices and compensation plans.

Effects on other modules. Charges appear on the billing desk and as outstanding balance; commission entries follow; the visit counts as completed in reports.

Error cases. 422 `INVALID_TRANSITION`; 409 `VERSION_CONFLICT`; 403 `FORBIDDEN`.

Technical references: `backend/src/billing/charges/charge-sources.service.ts` · `QueueOrchestrator.complete`, `ChargeSourcesService.syncForEncounter` · `web-app/src/features/consultation/pages/ConsultationPage.tsx`

30.3.19 Stop a consultation (abort)

Status: [Implemented]

What it does. Ends a consultation without completing it, for example when the patient has to leave. A booked appointment is closed as cancelled.

How the user does it.

1. Click **Stop consultation**, optionally enter a reason, confirm **Stop consultation**.

Behind the scenes — in simple terms. The visit ends as "Stopped". No charges are created by this action.

Behind the scenes — technically. `PATCH /queue/entries/:id/abort` with `{ reason? }` and `If-Match` → `QueueOrchestrator.abort`: transition `in_consultation → aborted`, set `finished_at`, emit `ABORTED`; if booked, `closeAppointment` sets the appointment (only if still `checked_in`) to `cancelled` with code `other` and the reason or "Consultation aborted".

Rules and validations. Only from `in_consultation`.

Who can do it. `queue.doctor.own` (own session) or `queue.doctor.any`; feature `appointments.core`. Default: Doctor (own), Admin.

Data read and written. Writes `queue_entries`, `appointments`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. The encounter remains but is not billed by this action.

Error cases. 422 `INVALID_TRANSITION`; 409 `VERSION_CONFLICT`.

Technical references: `backend/src/queue/services/queue-orchestrator.service.ts` · `QueueOrchestrator.abort`, `closeAppointment` · `web-app/src/features/queue/pages/DoctorQueuePage.tsx`

30.3.20 Remove a patient from the queue (cancel)

Status: [Implemented]

What it does. Removes a patient who left before being seen. Their token is never given to anyone else. A booked appointment is closed as cancelled with the reason "patient left".

How the user does it.

1. In the patient's menu choose **Remove from queue**.
2. In "Remove A-007 from the queue?" optionally enter a reason (for example "Left without being seen") and click **Remove from queue**.

Behind the scenes — in simple terms. The entry ends as cancelled and moves to "Done today".

Behind the scenes — technically. `PATCH /queue/entries/:id/cancel` with `{ reason? }` and `If-Match` → `QueueOrchestrator.cancel` → `cancelEntry`: `assertManageEntries`, transition to `cancelled`, set `finished_at`, emit `CANCELLED`; if booked, `closeAppointment` with code `patient_left`.

Rules and validations. From `waiting`, `skipped` or `on_hold`.

Who can do it. `queue.entry.manage`; feature `appointments.core`. Default: Receptionist, Admin.

Data read and written. Writes `queue_entries`, `appointments`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. The appointment becomes cancelled ("Left" in the appointment's queue label).

Error cases. 422 `INVALID_TRANSITION`; 403 `FORBIDDEN` "You cannot manage queue entries"; 409 `VERSION_CONFLICT`.

Technical references: `backend/src/queue/controllers/queue.controller.ts` · `QueueOrchestrator.cancel`, `cancelEntry` · `web-app/src/features/queue/components/QueueBoard.tsx`

30.3.21 Force-complete a visit for the doctor

Status: [Implemented]

What it does. Lets an admin complete a consultation the doctor forgot to close. It has exactly the same effects as Complete, including automatic charges, and is recorded separately in the history.

How the user does it.

1. On the reception board, in the lane's "With doctor" box, click **Force complete**.
2. Confirm "Complete A-007 for the doctor?" with **Mark completed**.

Behind the scenes — in simple terms. Same as Complete, done by someone else.

Behind the scenes — technically. `PATCH /queue/entries/:id/force-complete` → `QueueOrchestrator.complete(…, force = true)`: requires `anyDoctor`, emits `FORCE_COMPLETED`, otherwise identical to 30.3.18.

Rules and validations. Only from `in_consultation`.

Who can do it. `queue.doctor.any`; feature `appointments.core`. Default: Admin only.

Data read and written. As Complete.

Effects on other modules. As Complete.

Error cases. 403 "Only staff who run any doctor's queue can do that"; 422 `INVALID_TRANSITION`.

Technical references: `backend/src/queue/controllers/queue.controller.ts` · `QueueOrchestrator.complete` · `web-app/src/features/queue/components/QueueBoard.tsx`

30.3.22 Print a token ticket

Status: [Implemented]

What it does. Prints a small ticket for the patient: clinic name, location, the token in large type, the doctor, the date, the patient's name, "Appointment 10:00" or "Walk-in", and roughly how many people are ahead.

How the user does it.

1. Click **Print ticket** in the success toast after check-in or walk-in, in a patient's menu on the board, or in the appointment detail dialog.
2. The browser's print dialog opens in a small pop-up window.

Behind the scenes — in simple terms. "People ahead" is how many waiting patients the engine would call first right now; it can change.

Behind the scenes — technically. `GET /queue/entries/:id/ticket` (`@AnyMember`, then `queue.board.view` or `queue.doctor.own`) → `QueueQueryService.ticket`, limited to the active location. `printQueueTicket` opens the pop-up synchronously (to pass pop-up blockers), escapes all text, and calls `window.print()`.

Rules and validations. Pop-ups must be allowed ("Allow pop-ups to print the ticket").

Who can do it. `queue.board.view` or `queue.doctor.own`; feature `appointments.core`. Default: Admin, Receptionist, Assistant, Doctor.

Data read and written. Reads `queue_entries`, `queue_sessions`, `patients`, `appointments`, `organizations`, `locations`.

Effects on other modules. None.

Error cases. 404 "Queue entry not found"; toast "Could not load the ticket".

Technical references: `backend/src/queue/services/queue-query.service.ts` · `QueueQueryService.ticket` · `web-app/src/features/queue/utils/printTicket.ts`

30.3.23 Reception board, doctor screen and TV display (reads)

Status: [Implemented]

What it does. Provides the three live views: all lanes for reception, one session for the doctor, and the anonymous TV state.

How the user does it. Open the **Queue** tab of Appointments, **My Queue**, or the TV button on the board (which opens `/display/<location id>` in a new tab, to be shown full-screen on a waiting-room screen).

Behind the scenes — in simple terms. Each screen loads a full picture once and refreshes it whenever the server says something changed. The TV page needs no login and shows only tokens, doctor names, statuses and counts.

Behind the scenes — technically.

- `GET /queue/sessions?date=` (`queue.board.view`) → `QueueQueryService.lanes`: one lane per active staff member assigned to the location, with that doctor's snapshot for the service date (null before the first arrival), plus the effective policy.
- `GET /queue/sessions/:id` (`@AnyMember`; board viewers or the session's doctor) → `snapshot`: session view, all entries, `waitingOrder` and `suggestion` (null while someone is with the doctor).
- `GET /queue/displays/:locationId/state` (`@Public`, listed in the public routes) → `displayState`: for an active location, its name, service date and, per session of the day, the doctor's name, session status, the token being served and its status, the next five tokens in engine order, and the waiting count. No patient data. Anyone who knows a location's id can read it.
- `GET /queue/me` (`@AnyMember`) returns legacy role names and the employee id; the web app does not call it and uses `/me/access` instead.

Rules and validations.

- The web app hides buttons the user may not use (`useQueuePermissions`); the API enforces the same rules.
- The TV page hides closed sessions.

Who can do it. Board: `queue.board.view` (Admin, Receptionist, Assistant). Doctor screen: `queue.doctor.own` (Doctor, Admin through `queue.doctor.any`). TV: anyone, no login. Feature `appointments.core`.

Data read and written. Reads `queue_sessions`, `queue_entries`, `patients`, `appointments`, `employees`, `employee_org_location_assignments`, `queue_policies`, `locations`.

Effects on other modules. The list view shows billing status from Billing for users with `billing.desk.use`.

Error cases. 403 `NO_PERMISSION` "You can't see this doctor's queue"; 404 "Location not found" (TV shows "Display not available for this location").

Technical references: `backend/src/queue/controllers/display.controller.ts` · `QueueQueryService.lanes`, `snapshot`, `displayState` · `web-app/src/features/queue/pages/DisplayPage.tsx`

30.3.24 Live updates on every screen

Status: [Implemented]

What it does. Every change appears on the board, the doctor screen and the TV within about a second, without reloading. If the live connection drops, screens refresh every 10 seconds and show "Reconnecting".

How the user does it. Nothing to do. The green "Live" or amber "Reconnecting" label shows the state.

Behind the scenes — in simple terms. Each change writes small notes to an "outbox" table in the same transaction. A background process reads the outbox and broadcasts the notes to the screens that listen. A screen that receives a note newer than what it shows reloads its picture. Notes carry ids, statuses and tokens, never patient names.

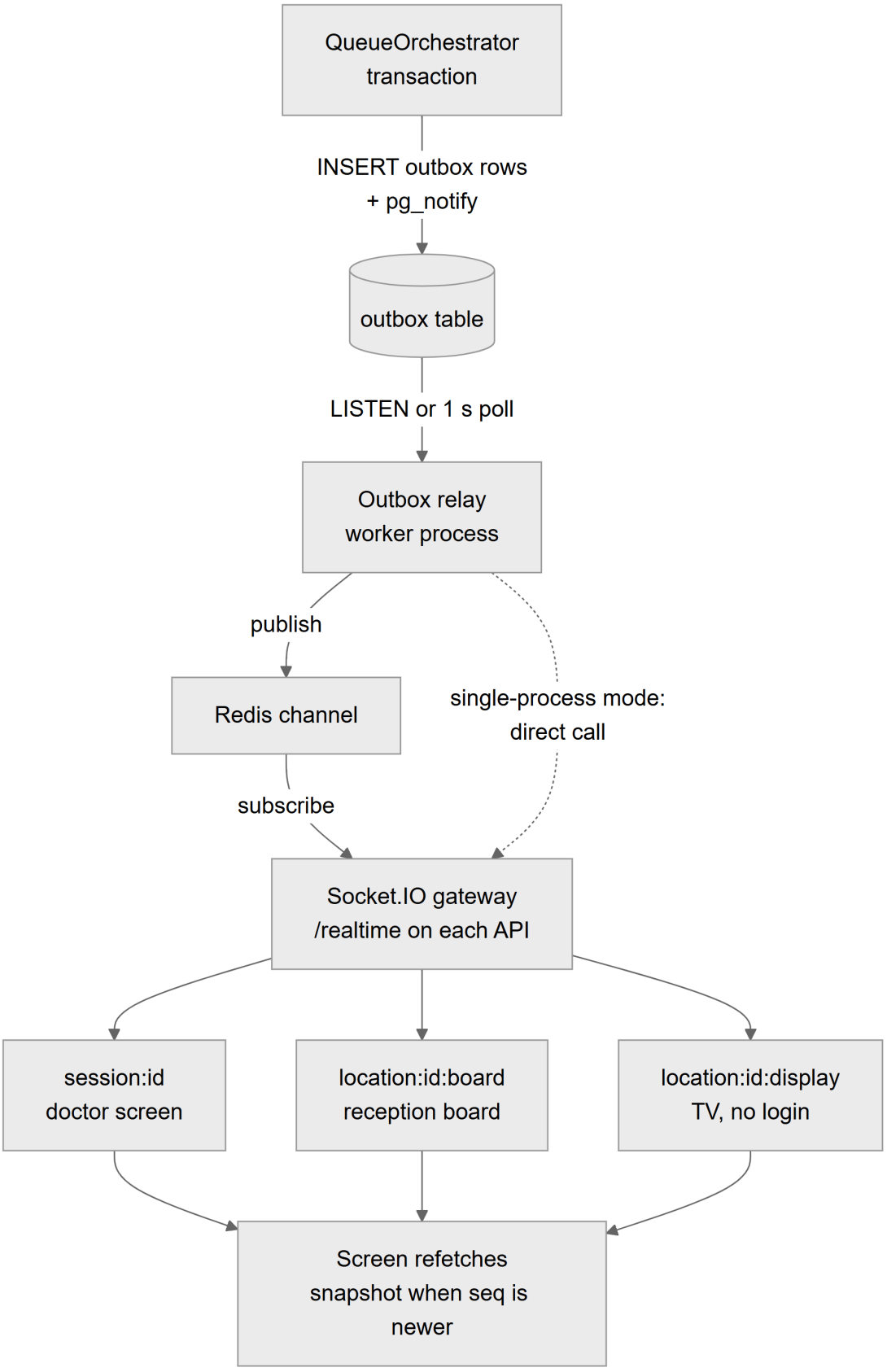

Figure 30.10 — The realtime path from a queue change to the screens.

The flowchart runs top to bottom. "QueueOrchestrator transaction" writes "INSERT outbox rows + pg_notify" into the "outbox table". The "Outbox relay worker process" picks rows up by "LISTEN or 1 s poll" and "publish"es them to a "Redis channel". The "Socket.IO gateway /realtime on each API" instance "subscribe"s to that channel. A

dotted arrow from the relay straight to the gateway is labelled "single-process mode: direct call". The gateway sends to three rooms: "session:id, doctor screen", "location:id:board, reception board" and "location:id:display, TV, no login". All three lead to "Screen refetches snapshot when seq is newer".

Behind the scenes — technically. `QueueTx.emit` bumps `queue_sessions.event_seq`, inserts the `queue_events` row and three `outbox` rows, and calls `pg_notify('queue_outbox')`, all in one statement inside the transaction, so screens never see a change that rolled back. `OutboxRelayService` (worker role) drains unpublished rows in batches of 100 with `FOR UPDATE SKIP LOCKED`, publishes them, and stamps `published_at` only after publishing succeeded (at-least-once delivery). In `api` + `worker` deployments it publishes to Redis and `RedisRealtimeSubscriber` on every API instance relays to its sockets; the gateway refuses sockets while Redis or the worker heartbeat is missing, so clients fall back to polling. In the single-process `all` role it calls the gateway directly. `QueueGateway` (namespace `/realtime`) accepts a JWT in the handshake; `subscribe` to `session:` and `location:…:board` rooms requires access to that organization and location; `location:…:display` needs no login. Clients (`useQueueSocket`, `useQueueData`) compare `seq` with the snapshot's `eventSeq`, coalesce refetches (120 ms) and poll every 10 s while disconnected. The worker process must run as exactly one instance.

Rules and validations. Payloads never contain patient names.

Who can do it. Any signed-in member with access to the location (board and session rooms); anyone (display room).

Data read and written. Writes `outbox`, `queue_events`, `queue_sessions.event_seq`; the relay updates `outbox.published_at`.

Effects on other modules. The same gateway also carries `access.changed` to the room `org:<id>` (Chapter 57).

Error cases. Subscribe answers "Sign in required for this room", "No access to this room" or "Unknown room"; the screen then stays on polling.

Technical references: `backend/src/queue/realtime/outbox-relay.service.ts` · `OutboxRelayService`, `QueueGateway`, `QueueTx.emit` · `web-app/src/features/queue/hooks/useQueueSocket.ts`

30.3.25 Safe retries and stale screens (Idempotency-Key and If-Match)

Status: [Implemented]

What it does. Protects against double clicks, lost network replies and two people acting on the same patient at once. A retried check-in returns the same token, never a second one; an action on a patient someone else has already moved is refused and the screen refreshes.

How the user does it. Nothing to do. On a conflict the screen shows a warning toast that someone else changed this patient, and reloads the latest state.

Behind the scenes — in simple terms. Each button press carries a unique key; the server remembers the answer for that key for 24 hours. Each patient row carries a version number; the screen sends the version it saw, and the server refuses if it changed.

Behind the scenes — technically. Every queue mutation (except opening a session) requires an `Idempotency-Key` header (1–200 characters). The `@Idempotency` decorator hashes method, path and body. `claimIdempotencyKey` inserts `(organization_id, key, request_hash)` with `ON CONFLICT DO NOTHING`; a later request with the same key gets the saved response, a different body gets 422 `IDEMPOTENCY_KEY_REUSED`, and one still running gets 409 `REQUEST_IN_FLIGHT`. Entry actions accept `If-Match: <version>`; a mismatch gives 409 `VERSION_CONFLICT` with the current state. The web app (`queue.api.ts`) creates one key per user action and retries the same key up to 5 times on 503 `LOCK_TIMEOUT` or a lost connection. Database errors are mapped: lock timeout → `LOCK_TIMEOUT`; `one_patient_with_doctor` → `DOCTOR_BUSY`; `one_active_entry_per_appt` / `one_active_entry_per_patient` → `ALREADY_CHECKED_IN`.

Rules and validations. Keys are per organization and kept 24 hours.

Who can do it. Applies to everyone.

Data read and written. `idempotency_keys`; `version` columns on `queue_entries` and `queue_sessions`.

Effects on other modules. None.

Error cases. 400 `IDEMPOTENCY_KEY_REQUIRED`; 422 `IDEMPOTENCY_KEY_REUSED`; 409 `REQUEST_IN_FLIGHT`; 409 `VERSION_CONFLICT`; 503 `LOCK_TIMEOUT`.

Technical references: `backend/src/queue/decorators/queue-request.decorators.ts` · `Idempotency`, `IfMatch`, `QueueOrchestrator.mutate` · `web-app/src/features/queue/services/queue.api.ts`

30.3.26 Background jobs

Status: [Implemented]

What it does. Four jobs keep the queue tidy without anyone pressing a button: marking no-shows, auto-skipping patients who do not answer a call, closing yesterday's empty queues, and nightly clean-up.

How the user does it. Nothing to do; they run in the worker process.

Behind the scenes — in simple terms.

- **No-show marker**, every 60 seconds: bookings still scheduled or confirmed whose start plus "maximum late" minutes has passed become no-shows.
- **Call timeout**, every 15 seconds: if the location has "Auto-skip after a call" set, a patient called longer ago than that is skipped.
- **Day-end close**, every 5 minutes: sessions from a past service day are closed if nobody is left in them. Sessions that still have patients are only logged as an alert and are never cancelled automatically.
- **Clean-up**, every day at 03:15 server time: idempotency keys older than 24 hours and published outbox rows older than 7 days are deleted.

Behind the scenes — technically. `QueueJobsService` (`@Interval('queue-no-show-marker', 60_000)`, `@Interval('queue-call-timeout', 15_000)`, `@Interval('queue-session-day-end', 300_000)`, `@Cron('15 3 * * *', { name: 'queue-cleanup' })`, no timezone option, so server time). Each run is skipped if the previous run of the same job in this process is still busy. The call-timeout job selects entries called more than 30 seconds ago, reads each location's `call_timeout_sec`, and calls `QueueOrchestrator.timeoutCall`, which runs `skip` as the system actor with the expected version, ignoring the case where the state already moved on. The day-end job compares each open session's date with the location's current service date and calls `closeIfEmpty` (event `SESSION_CLOSED`, reason `DAY_END`). The no-show marker writes with plain SQL (no queue event, no realtime message). Jobs that go through the orchestrator re-check state under the session lock, so a double run is harmless.

Rules and validations. `call_timeout_sec` is null (off) by default and at least 30 when set.

Who can do it. The system (actor label `system`).

Data read and written. `appointments`, `queue_entries`, `queue_sessions`, `queue_policies`, `queue_events`, `outbox`, `idempotency_keys`.

Effects on other modules. No-shows change appointment status and figures (Chapter 29).

Error cases. Failures are logged ("Queue job … failed") and retried at the next tick.

Technical references: `backend/src/queue/jobs/queue-jobs.service.ts` · `QueueJobsService` · none (no screen)

30.3.27 Queue settings (organization default and location override)

Status: [Implemented]

What it does. Lets an admin set how queues behave: smart or first-come order, the early window, late grace, maximum lateness, the walk-in starvation limit, how skipped patients rejoin, auto-skip after a call, whether a manual select needs a reason, and when the service day starts. Settings can be saved for the whole organization or for one location.

How the user does it.

1. Open the location's settings page (from the locations admin) and find the **Queue settings** card (Figure 30.8). Make that location the active one first.
2. Change the values, choose **Save as** "This location only" or "Organization default (all locations)", and save. The message says "Queue settings saved — they apply from tomorrow's queues".
3. To go back to the organization default, click **Use organization default**.

Behind the scenes — in simple terms. The location's own settings win; otherwise the organization default applies; otherwise the built-in defaults: first come, first served; early window 15 min; late grace 10 min; maximum late 45 min; starvation 45 min; rejoin at the end; auto-skip off; no reason required for select; day starts at 00:00.

Behind the scenes — technically. `GET /queue/policies` (`@AnyMember`) → `QueuePoliciesService.getForLocation` (organization default, location override, effective values with `source`). `PATCH /queue/policies` with `UpdateQueuePolicyDto` (`scope`, optional `version` for optimistic locking) → `upsert`; a new location override starts from the currently effective values. `DELETE /queue/policies/location` → `removeOverride`. `getEffective` is cached for 60 seconds per location in each API process; saving clears the cache in that process only. Only `smart_queue_enabled` is copied into a session at creation; every other value is read live at each action, so those changes take effect within about a minute, not only from the next day as the message says.

Rules and validations.

- Early window 0–240; late grace 0–240; maximum late 0–480 and at least the late grace; starvation 1–480; auto-skip empty or 30–3600 seconds; service day start "HH:MM". The same limits are database checks.
- One organization default and at most one override per location (partial unique indexes).

Who can do it. Read: any member. Save and remove: `admin.settings.manage`; the page itself needs `admin.locations.manage`. Default: Admin.

Data read and written. Reads and writes `queue_policies`; reads `locations.timezone`.

Effects on other modules. `max_late_min` drives the appointment no-show marker and check-in (Chapter 29).

Error cases. 409 `VERSION_CONFLICT` (someone saved in between); 422 `INVALID_TRANSITION` "max_late_min must be at least late_grace_min"; 400 on out-of-range values.

Technical references: `backend/src/queue/services/queue-policies.service.ts` · `QueuePoliciesService`, `POLICY_DEFAULTS` · `web-app/src/features/queue/components/QueueSettingsSection.tsx`

30.3.28 Patient visit history

Status: [Implemented]

What it does. Lists a patient's queue visits, booked and walk-in, newest first, with token, doctor, date and outcome, on the patient record.

How the user does it. Open a patient and look at the appointments and visits tab.

Behind the scenes — in simple terms. Every queue entry of the patient in the organization, up to 200.

Behind the scenes — technically. `GET /queue/patients/:patientId/visits` (`patients.view`) → `QueueQueryService.patientVisits` . It is organization-wide and does not apply the scope of `patients.view` to the visits.

Rules and validations. At most 200 rows.

Who can do it. `patients.view` ; feature `patients.core` . Default: Admin, Receptionist, Assistant, Doctor.

Data read and written. Reads `queue_entries` , `queue_sessions` , `employees` , `appointments` .

Effects on other modules. Shown in Patients (Chapter 28).

Error cases. 403 `NO_PERMISSION` .

Technical references: `backend/src/queue/services/queue-query.service.ts` · `QueueQueryService.patientVisits` · `web-app/src/features/patients/components/tabs/AppointmentsTab.tsx`

30.3.29 Next-patient preview

Status: [Partial] — backend only; no screen calls it.

What it does. Returns whom Call next would pick right now and the full waiting order, without reserving anyone.

How the user does it. Not from a screen. The doctor screen shows the same suggestion from its snapshot instead.

Behind the scenes — in simple terms. A read-only question to the selection engine.

Behind the scenes — technically. `GET /queue/sessions/:id/next-preview` (`@AnyMember` ; board viewers or the session's doctor) → snapshot's `suggestion` , `waitingOrder` and `eventSeq` .

Rules and validations. The suggestion is empty while someone is with the doctor.

Who can do it. `queue.board.view` , or the right to run that session; feature `appointments.core` .

Data read and written. Reads the session's entries and policy.

Effects on other modules. None.

Error cases. 403 `NO_PERMISSION` ; 404 "Queue session not found".

Technical references: `backend/src/queue/controllers/queue.controller.ts` · `QueueController.nextPreview` · no web-app caller

30.3.30 Queue history (events log)

Status: [Partial] — backend only; no screen shows it.

What it does. Returns the complete, append-only history of one session: every check-in, call (with the engine's reason and, for manual selects, its suggestion), skip, transfer, priority change, mode change, pause and close, with actor and time.

How the user does it. Not from a screen.

Behind the scenes — in simple terms. Every queue change writes one history row in the same transaction; rows are never edited.

Behind the scenes — technically. `GET /queue/sessions/:id/events?afterSeq=` (`admin.audit.view`) → `QueueQueryService.events` , ordered by `seq` . Rows hold `event_type` , `from_status` , `to_status` , `reason_code` , `reason_text` , `suggested_entry_id` , `actor_id` , `actor_role` (owner, super_admin, the role template keys, or

`system`), `metadata` and `occurred_at`.

Rules and validations. Limited to the active location.

Who can do it. `admin.audit.view`; feature core. Default: Admin.

Data read and written. Reads `queue_events`.

Effects on other modules. None.

Error cases. 404 "Queue session not found".

Technical references: `backend/src/queue/entities/queue-event.entity.ts` · `QueueQueryService.events` · no web-app caller

30.3.31 Manage the TV display

Status: [Not implemented]

What it does. Nothing. The permission `queue.display.manage` ("Manage TV display") exists in the catalog and in the Receptionist template, but no endpoint or screen checks it. The TV display has no settings.

How the user does it. Not available.

Behind the scenes — in simple terms. Any screen can show `/display/<location id>` without setup.

Behind the scenes — technically. `queue.display.manage` appears only in `permissions.ts`, `templates.ts` and the catalog test.

Rules and validations. None.

Who can do it. Not applicable.

Data read and written. None.

Effects on other modules. None.

Error cases. None.

Technical references: `backend/src/access/catalog/permissions.ts` · `queue.display.manage` · none

30.4 Entities and data model

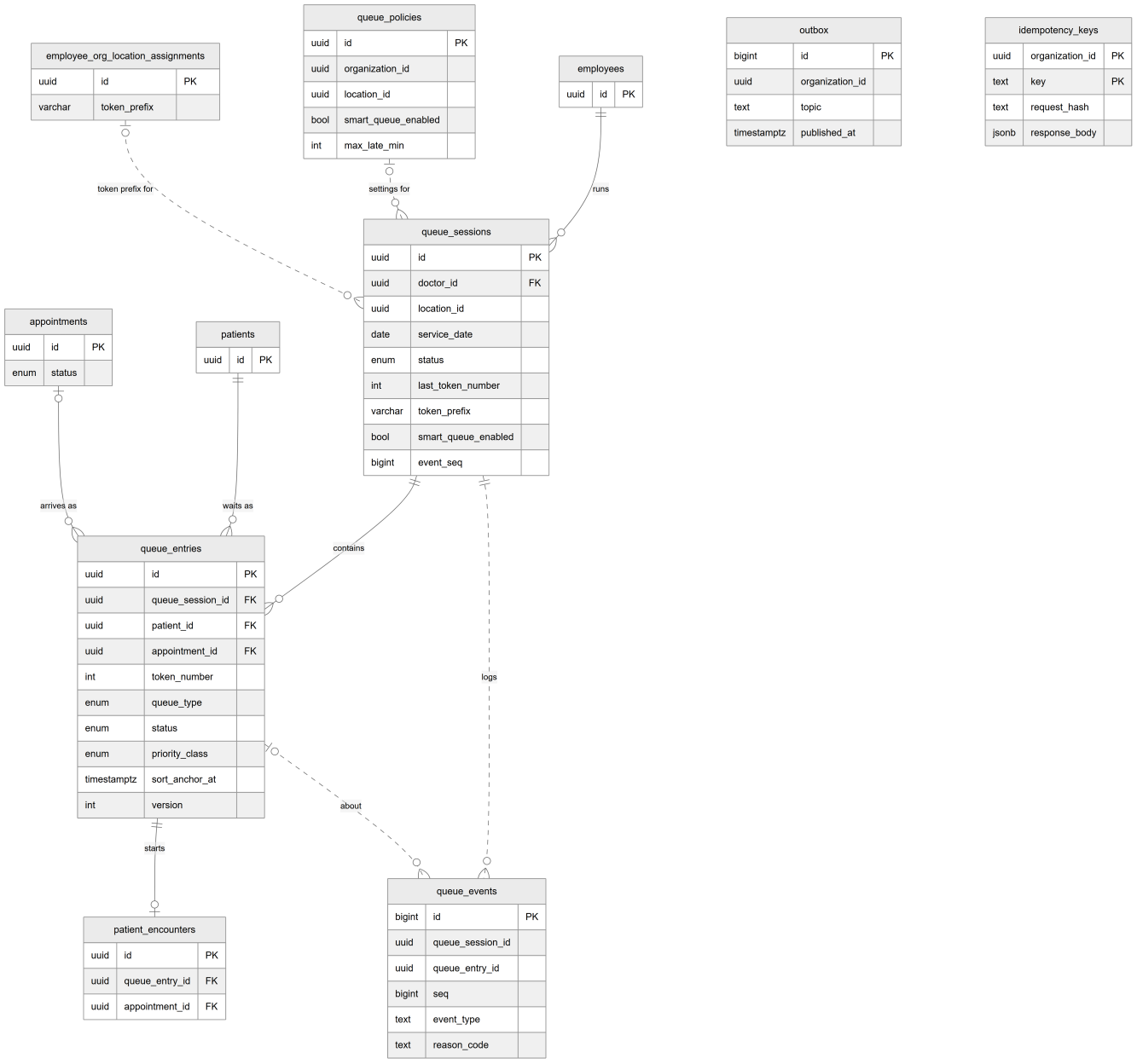

Figure 30.11 — The queue tables and their links. Dotted lines are logical links without a foreign key.

The diagram centres on `queue_sessions` (`id` , `doctor_id` , `location_id` , `service_date` , `status` , `last_token_number` , `token_prefix` , `smart_queue_enabled` , `event_seq`). Solid lines: `employees` "runs" sessions; a session "contains" `queue_entries` (`id` , `queue_session_id` , `patient_id` , `appointment_id` , `token_number` , `queue_type` , `status` , `priority_class` , `sort_anchor_at` , `version`); `patients` "waits as" entries; `appointments` "arrives as" entries; an entry "starts" at most one `patient_encounters` row. Dotted lines: a session "logs" `queue_events` , and an entry is what an event is "about"; `queue_policies` are "settings for" sessions; `employee_org_location_assignments` supply the "token prefix for" sessions. Two tables stand alone, with no links: `outbox` (`id` , `organization_id` , `topic` , `published_at`) and `idempotency_keys` (`organization_id` and `key` as the composite key, `request_hash` , `response_body`).

The relationships in words:

- Each queue session belongs to exactly one doctor (an employee). A doctor with sessions cannot be deleted (ON DELETE RESTRICT). There is one session per organization, location, doctor and service date (unique `uq_queue_session_doctor_day`). The session's `organization_id` and `location_id` have no foreign key.

- Each queue entry belongs to exactly one session; a session with entries cannot be deleted (RESTRICT). Token numbers are unique per session.
- Each entry belongs to one patient; deleting the patient deletes the entries (CASCADE).
- A booked entry points to its appointment, and a check constraint says `queue_type = 'appointment'` exactly when `appointment_id` is set. Deleting the appointment deletes the entry (CASCADE). At most one active entry per appointment and per patient per session, and at most one entry with the doctor per session (partial unique indexes).
- `converted_from_appt_id` (walk-in made from a late booking) and `transferred_from_id` (the entry a transfer replaced) are logical links without foreign keys.
- Each entry has at most one encounter (`patient_encounters.queue_entry_id` is unique); deleting the entry deletes the encounter (CASCADE).
- `queue_events` rows refer to a session and optionally an entry by id only, with `seq` unique per session. They are append-only.
- `queue_policies` : one row with `location_id` NULL per organization (the default) and at most one row per location (the override). No foreign keys.
- `outbox` rows are realtime messages addressed to a room (`topic`); `idempotency_keys` store the saved response per organization and key. Neither has foreign keys.

These tables come from the entities (TypeORM synchronize in development, a reviewed schema sync in production); none of the three migration files concerns the queue. The queue transaction sets `app.organization_id` and `app.location_id` for row-level security, but no row-level security policies exist, so tenant isolation is enforced in application code.

30.5 Status lifecycles

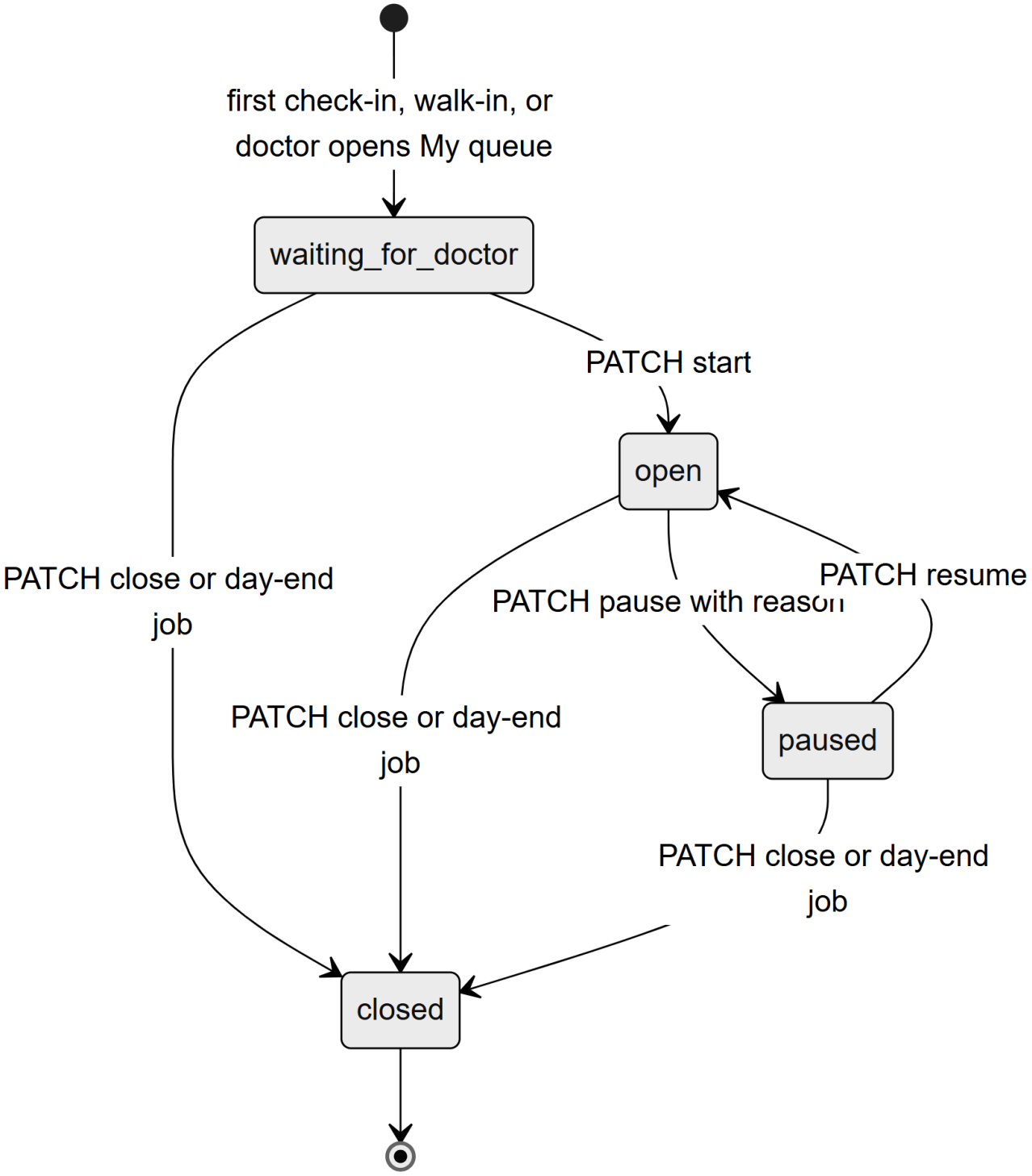

Figure 30.12 — The queue session lifecycle from `SESSION_TRANSITIONS`.

The diagram has four states. A session is created in `waiting_for_doctor` by the first check-in or walk-in, or when the doctor opens My queue. "PATCH start" leads to `open`. From `open`, "PATCH pause with reason" leads to `paused`, and "PATCH resume" leads back to `open`. From each of `waiting_for_doctor`, `open` and `paused`, "PATCH close or day-end job" leads to `closed`, the final state.

- **waiting_for_doctor** ("Doctor not started"): patients can arrive and get tokens; nobody can be called. The TV shows "Starting soon".
- **open**: the doctor can call, select and consult.

- **paused**: arrivals continue; calls are blocked; the TV shows "On a break" and the board shows the reason.
- **closed**: no arrivals, no tokens, no calls. Closing by hand needs a decision for each remaining patient; the day-end job closes only empty sessions of past days.

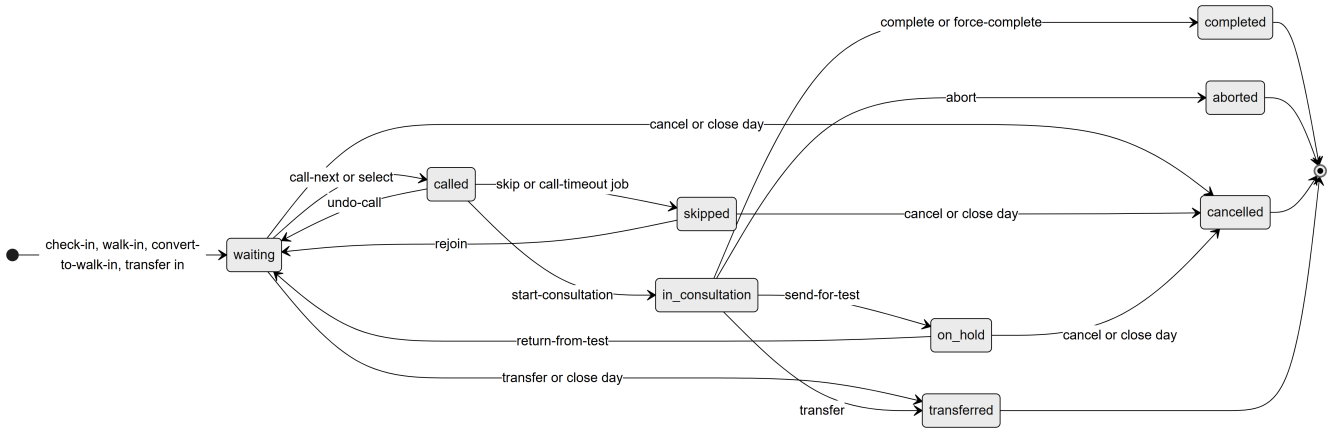

Figure 30.13 — The queue entry lifecycle from `ENTRY_TRANSITIONS`.

The diagram has nine states. Entries start in `waiting` through check-in, walk-in, convert-to-walk-in or "transfer in". From `waiting`: "call-next or select" to `called`; "cancel or close day" to `cancelled`; "transfer or close day" to `transferred`. From `called`: "undo-call" back to `waiting`; "skip or call-timeout job" to `skipped`; "start-consultation" to `in_consultation`. From `skipped`: "rejoin" back to `waiting`; "cancel or close day" to `cancelled`. From `in_consultation`: "send-for-test" to `on_hold`; "complete or force-complete" to `completed`; "abort" to `aborted`; "transfer" to `transferred`. From `on_hold`: "return-from-test" back to `waiting`; "cancel or close day" to `cancelled`. `completed`, `aborted`, `cancelled` and `transferred` are final.

- **waiting** ("Waiting"): in line; the only state the engine picks from.
- **called** ("Called"): the doctor called the token; it flashes on the TV. At most one entry per session is called or in consultation.
- **skipped** ("Skipped"): did not come when called; waits for reception to rejoin or remove.
- **in_consultation** ("In room"): with the doctor; the encounter exists.
- **on_hold** ("At test"): sent for a test; keeps the token; the doctor is free.
- **completed**: visit finished; the appointment is completed and charges are created.
- **aborted** ("Stopped"): consultation stopped; the appointment is cancelled with code `other`.
- **cancelled**: patient left or the day was closed; the appointment is cancelled with code `patient_left`.
- **transferred**: replaced by a new entry in another doctor's queue.

Any other change is refused with 422 `INVALID_TRANSITION`. The map also allows rejoin from `called` and `on_hold`, which the web app does not offer.

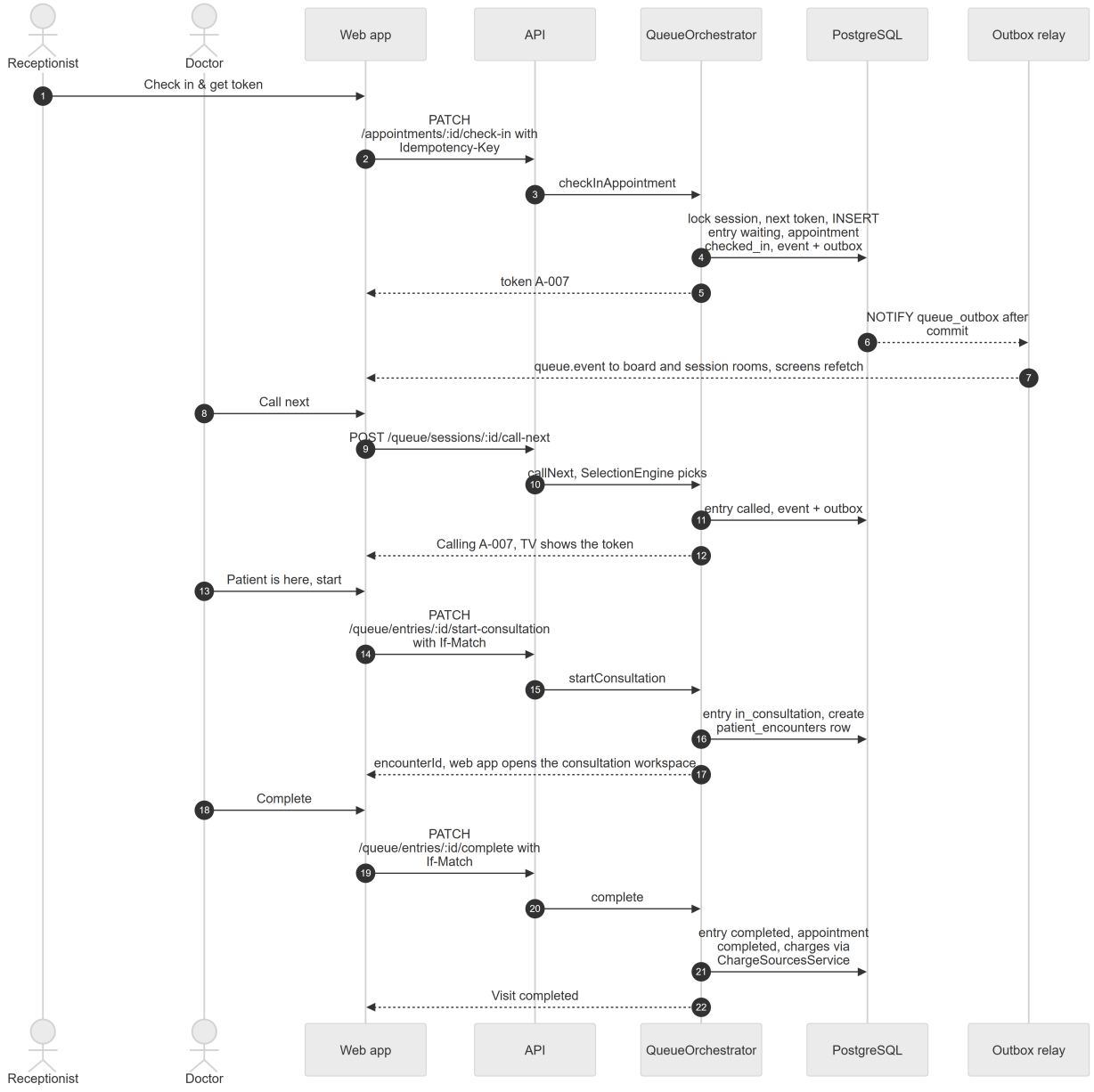

Figure 30.14 — Check-in, call, consultation and completion of one booked patient.

The sequence has seven participants: Receptionist, Doctor, Web app, API, QueueOrchestrator, PostgreSQL and Outbox relay, and 22 numbered steps. Steps 1–5: the receptionist clicks "Check in & get token"; the web app sends `PATCH /appointments/:id/check-in` with an Idempotency-Key; the API calls `checkInAppointment`; the orchestrator locks the session, takes the next token, inserts the entry as waiting, sets the appointment to checked_in and writes the event and outbox rows; the web app receives token A-007. Steps 6–7: after commit PostgreSQL notifies the relay, which sends `queue.event` to the board and session rooms, and the screens refetch. Steps 8–12: the doctor clicks "Call next"; the web app sends `POST /queue/sessions/:id/call-next`; the orchestrator lets the SelectionEngine pick, sets the entry to called and writes the event and outbox; the screen shows "Calling A-007" and the TV shows the token. Steps 13–17: the doctor clicks "Patient is here, start"; the web app sends `PATCH /queue/entries/:id/start-consultation` with If-Match; the orchestrator sets the entry to in_consultation and creates the `patient_encounters` row; the encounter id comes back and the web app opens the consultation workspace. Steps 18–22: the doctor clicks "Complete"; the web app sends `PATCH /queue/entries/:id/complete` with If-Match; the orchestrator sets the entry and the appointment to completed and creates the charges through ChargeSourcesService; the screen shows "Visit completed".

30.6 Connections to other modules

- **Appointments (Chapter 29).** Check-in and convert-to-walk-in start here; the queue sets the appointment to completed or cancelled; the no-show marker and `max_late_min` live here.
- **Patients (Chapter 28).** Walk-ins can create patients; the patient record shows visits; a doctor's `own` patient scope includes patients with a queue visit with that doctor.
- **Consultation (Chapter 31).** Start consultation creates the encounter and opens the workspace; the workspace's complete button calls the queue's complete.
- **Billing and Commissions (Chapters 34 and 35).** Completion creates charges and commission snapshots through `ChargeSourcesService`; the board's list view shows outstanding balances.
- **Employees (Chapter 37).** Doctors and token prefixes come from assignments.
- **Access control (Chapter 57).** All permissions are queue permissions of feature `appointments.core`; the realtime gateway also carries `access.changed`.
- **Realtime and jobs (Chapter 63).** The outbox relay, Redis bus and jobs run in the single worker process.
- **Subscriptions (Chapter 45).** In read-only mode queue writes are refused with 402 `SUBSCRIPTION_EXPIRED`; opening "My queue" still works because it is a GET.

30.7 Limitations and known gaps

- Next-patient preview and the queue history log have no screen.
- `queue.display.manage` is never checked; the TV display has no management.
- The TV state is public for any active location id and shows doctor names and tokens.
- Lanes, doctor pickers and transfer targets list every active staff member at the location, not only doctors.
- Urgent priority cannot be removed once set.
- The settings message says changes apply from tomorrow, but only the smart/FIFO choice is fixed per session; other values apply within about a minute (per-process 60-second cache).
- No row-level security policies exist, despite the tenant settings the transaction prepares.
- The no-show marker writes no queue event or realtime message.
- `GET /queue/me` and `POST /queue/sessions` have no web-app caller.
- Sound announcements, SMS or WhatsApp calls, and waiting-time estimates are not implemented.

30.8 Owner's FAQ

Can two receptionists give out the same token? No. Tokens come from a counter in the locked session row; a failed check-in rolls the counter back, so there are no gaps and no duplicates.

What if the network drops right after "Check in & get token"? The web app retries with the same Idempotency-Key and receives the same token. No second entry is created.

Does a booked patient always go before walk-ins? Only in smart mode and only when "due" (from 15 minutes before to 10 minutes after the booked time). A walk-in who has waited 45 minutes is called first once. In first-come mode, arrival order decides.

Can a receptionist call patients? Not with the default role. Calling and consulting need `queue.doctor.own` (own queue) or `queue.doctor.any` (Admin).

What happens to patients still waiting when the doctor leaves? The doctor must cancel or transfer each one to close the day. If nobody closes the day, the session stays open and the day-end job only logs an alert.

Can I see who skipped or reordered a patient? Yes in the database: every change is in `queue_events` with actor and reason. There is an API for it (`admin.audit.view`) but no screen yet.

Does the TV show patient names? No. Only tokens, doctor names, session status and counts.

Where are queue settings stored? In `queue_policies` : one organization default row and optional per-location override rows.

What does "Complete" do to billing? It creates the consultation and treatment charges in the same transaction, if the billing feature is enabled and an encounter exists.

Chapter 31 — Consultation

This chapter describes the consultation module of the Platform: the single screen a doctor works on while a patient is in the chair. It is written for the product owner and explains what the screen does, how the clinical record of one visit (the **encounter**) is created and saved, and how finishing the visit hands work over to billing. The consultation sits between the queue (which brings the patient to the doctor) and billing (which charges for the visit), so it touches both.

31.1 What this module is for

When a doctor starts seeing a patient from their queue, the Platform opens the **consultation workspace**. On one screen the doctor sees who the patient is and anything dangerous to know (allergies, blood thinners, pregnancy), writes the visit notes (complaint, examination, diagnosis, advice, next visit), records the **treatments** performed today, writes and signs the **prescription**, and finally presses "Complete visit". Everything saves itself while the doctor types.

Behind the screen, every consultation is stored as one **encounter**: the clinical record of one consultation. The Platform creates the encounter automatically when the doctor starts the consultation from the queue. There is no separate "new encounter" button.

Who uses it. Doctors, by default. The Doctor system role holds the permission `consultation.conduct` at scope `own`, so a doctor can only open consultations that came from their own queue. The Admin system role does not get this permission by default (Admins clone the Doctor role if they also see patients). The owner of the organization holds every permission.

Where it appears. The consultation workspace has **no sidebar entry**. The doctor reaches it from the "My Queue" sidebar item (`/queue/doctor`): the button "Patient is here — start" starts the consultation and opens the workspace, and the button "Open consultation" re-opens it later. The workspace route is `/consultation/$encounterId` .

31.2 Screens

The module has one page with two overlays. There are no usable screenshots of these screens yet, so each figure is a placeholder and the paragraph after it describes the screen from the web-app code.

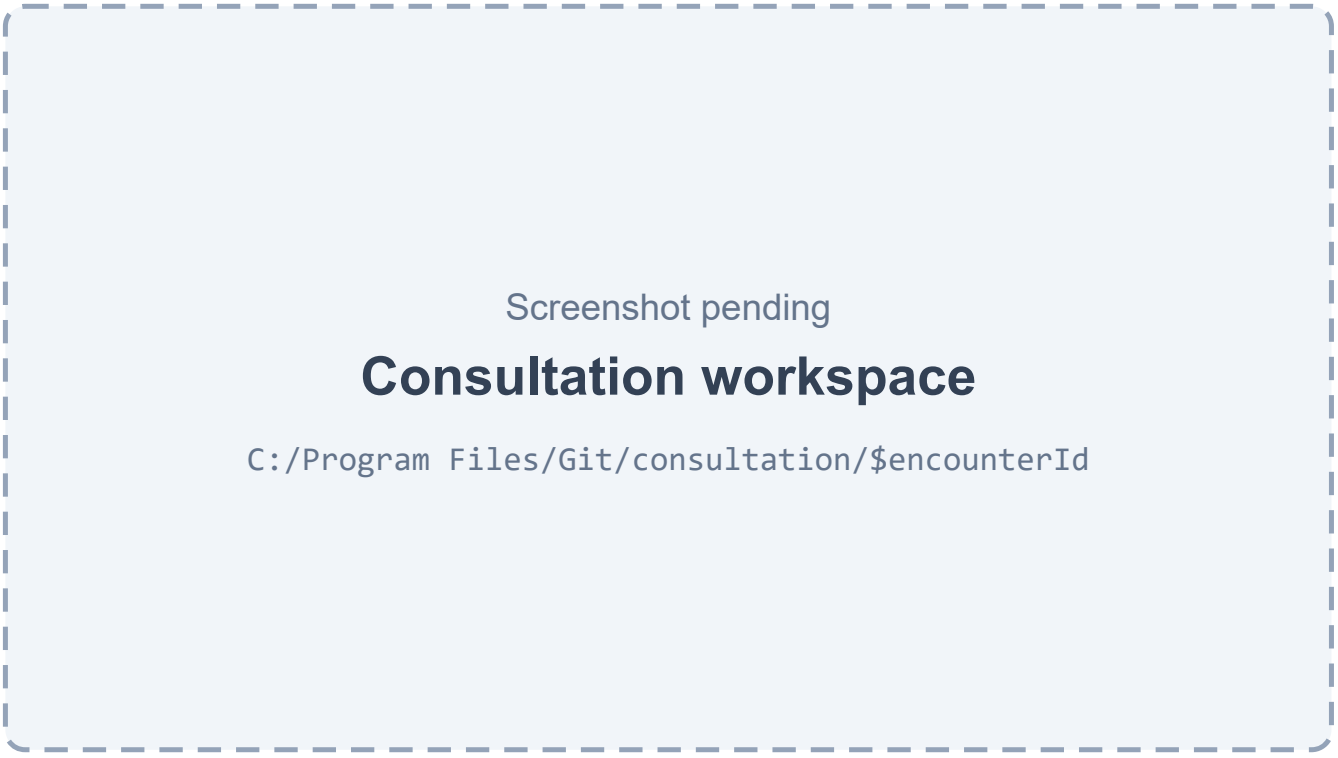

Figure 31.1 — Consultation workspace (screenshot pending).

The consultation workspace is a full-screen page without the normal sidebar. A sticky header runs across the top. On its left is a "Back to my queue" button, the patient's initials avatar, the patient's full name and a sub-line with age, gender, patient number and the doctor's name. Next to it is the queue **token** (for example "A-007") and a timer showing how many minutes the consultation has been running; once the queue entry is no longer in consultation, the timer is replaced by "Visit ended". A row of shortcut buttons opens the patient record: "History", "Teeth" (only with permission `patients.chart.edit`), "Vitals" and "Files" (only with `patients.documents.manage`). On the right is the green "Complete visit" button. Under the header, a strip of coloured alert chips appears when the patient has allergies or other risks. The body has three columns on a wide screen. The left column, "Patient at a glance", shows the latest vitals, up to four previous visits and up to three previous signed prescriptions, each with a "Use these medicines again" icon button. The middle column, "Visit notes", has text boxes for Complaint, Examination, Diagnosis and "Advice to patient", each with one-tap phrase chips, and a "Next visit" row with chips (3 days, 1 week, 2 weeks, 1 month, 3 months, 6 months) and a date picker. The right column holds two panels: "Treatment done today" and "Prescription". On a phone or tablet the columns stack, and the patient summary folds behind a "Patient summary & history" button.

Screenshot pending

Patient record slide-over

C:/Program Files/Git/consultation/$encounterId

Figure 31.2 — Patient record slide-over (screenshot pending).

The patient record slide-over opens from the right on top of the workspace when the doctor presses one of the header shortcuts or a "See all", "Record" or "Update" link in the patient summary. Its title is the patient's name followed by "patient record", with the line "Look anything up or add to it. Closing this returns to the consultation." It has tabs: "History" (past visits and treatments, with amounts hidden), "Teeth chart" (needs `patients.chart.edit`), "Vitals & dental", "Prescriptions" (needs `prescriptions.create`), "X-rays & files" (needs `patients.documents.manage`) and "Consents". The tabs reuse the same components as the patient detail page, so anything the doctor adds here is saved to the patient record. Closing the slide-over returns the doctor to exactly where they were.

Screenshot pending

Complete this visit? dialog

C:/Program Files/Git/consultation/$encounterId

Figure 31.3 — "Complete this visit?" dialog (screenshot pending).

The "Complete this visit?" dialog opens when the doctor presses "Complete visit". Its description says "The patient leaves your queue and reception is told you are done." It lists four reminder lines, each with a green tick or an amber warning icon: whether visit notes are written (or "No complaint or diagnosis written"), how many treatments were recorded (or "No treatment recorded"), the prescription state ("Prescription signed", "Prescription is still a draft — not signed", or "No prescription written") and the next visit date (or "No next visit set"). These are reminders only: the doctor can always finish. The buttons are "Keep working" and "Complete visit".

31.3 Features

31.3.1 Start a consultation (the encounter is created)

Status: [Implemented]

What it does. When the doctor accepts the called patient, the queue entry moves to "in consultation" and the Platform creates the encounter for that visit. The doctor lands straight in the consultation workspace.

How the user does it.

1. On "My Queue", call the next patient (Chapter 30 covers calling).
2. When the patient walks in, press "Patient is here — start".
3. The consultation workspace opens for that patient.

Behind the scenes — in simple terms. Starting the consultation and creating the clinical record happen together, in one database transaction. Either both happen or neither does. If the patient was sent for a test and comes back, starting again re-uses the same encounter instead of making a second one.

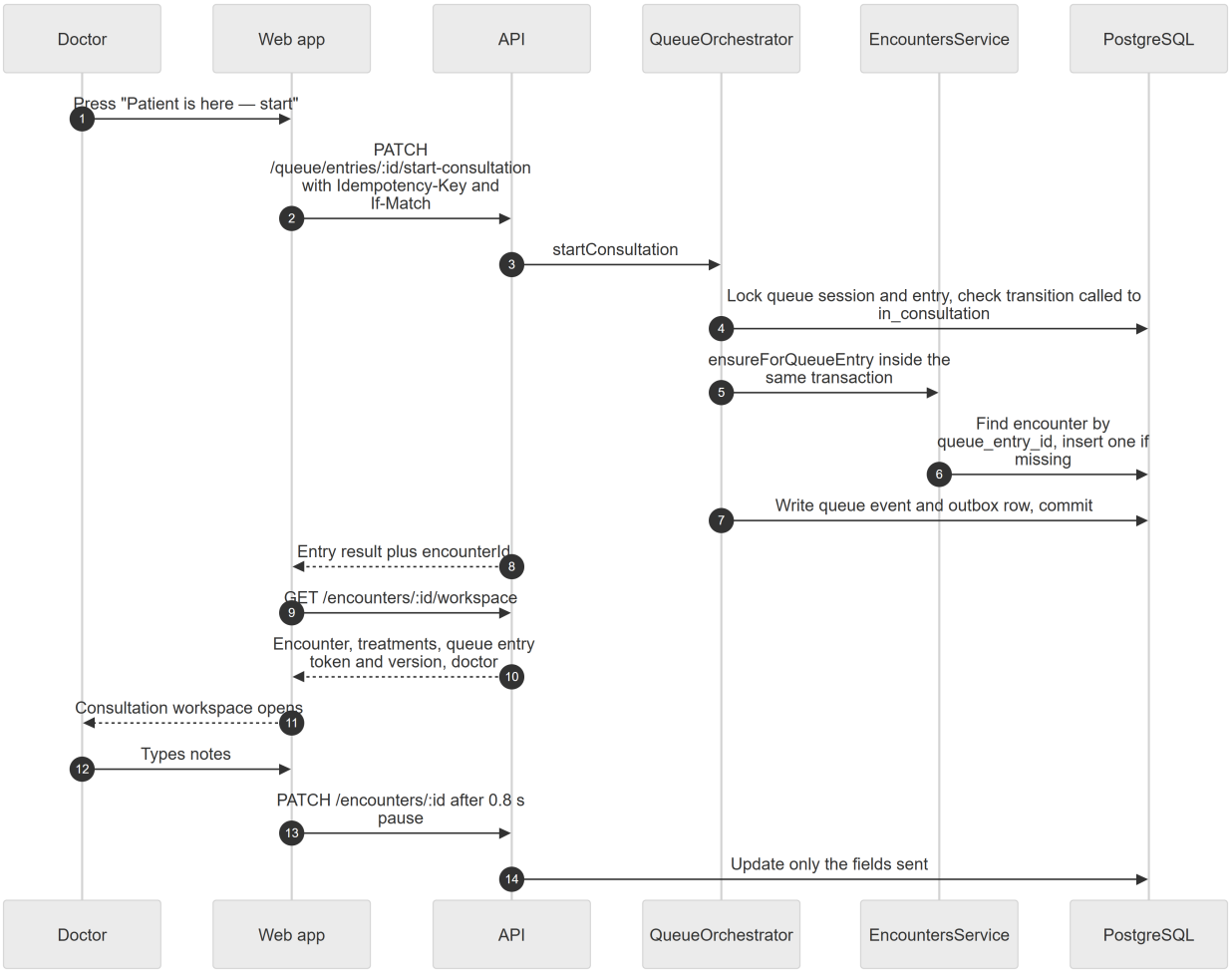

Figure 31.4 — Sequence from "Patient is here — start" to the first auto-saved note.

Figure 31.4 is a sequence diagram with six participants: Doctor, Web app, API, QueueOrchestrator, EncountersService and PostgreSQL. In step 1 the doctor presses "Patient is here — start". In step 2 the web app sends `PATCH /queue/entries/:id/start-consultation` with an Idempotency-Key header and an If-Match header carrying the entry's version. In step 3 the API calls `startConsultation` on the QueueOrchestrator. In step 4 the orchestrator locks the queue session and entry in PostgreSQL and checks that the move from `called` to `in_consultation` is allowed. In step 5 it calls `ensureForQueueEntry` on the EncountersService inside the same transaction, and in step 6 that service looks for an encounter with this queue entry id and inserts one only if none exists. In step 7 the orchestrator writes the queue event and outbox row and commits. In step 8 the API returns the entry result plus the new `encounterId`. In steps 9 and 10 the web app loads `GET /encounters/:id/workspace` and receives the encounter with its treatments, the queue entry's token and version, and the doctor. In step 11 the workspace opens. In steps 12 to 14 the doctor types notes, the web app sends `PATCH /encounters/:id` after a short pause, and the API updates only the fields that were sent.

Behind the scenes — technically. `PATCH /queue/entries/:id/start-consultation` (doctor queue controller) calls `QueueOrchestrator.startConsultation`. Inside the orchestrator's `mutate` transaction it locks the entry, checks the caller runs this doctor's session (`assertSessionDoctor`), applies the transition `called → in_consultation` with `consultation_started_at`, and records the queue event `CONSULTATION_STARTED`. It then calls `EncountersService.ensureForQueueEntry(tx, entry, locationId, userId)`. That method returns the existing encounter for `queue_entry_id` if there is one; otherwise it inserts a row with the entry's appointment id (null for a **walk-in**), patient, organization, location, `encounter_date = now`, `reason` copied from the entry's reason for visit, and `created_by` / `updated_by` set to the user. The response adds `encounterId`, and the doctor queue page navigates to `/consultation/$encounterId`.

Rules and validations.

- Only the doctor of that queue session can start it, or staff holding `queue.doctor.any`.
- The entry must be in status `called`; anything else answers 422 `INVALID_TRANSITION`.
- The request needs an Idempotency-Key header; If-Match must match the entry version when sent.
- At most one encounter per queue entry (unique `queue_entry_id`).

Who can do it. Permission `queue.doctor.own` (feature `appointments.core`), held by the Doctor template. The Admin template holds `queue.doctor.any`, which implies it. Opening the workspace afterwards also needs `consultation.conduct` (feature `patients.core`), which only the Doctor template holds by default.

Data read and written. `queue_entries`, `queue_sessions`, `queue_events`, outbox, `patient_encounters`.

Effects on other modules. The queue board and TV display update through the realtime outbox (Chapter 30). The encounter becomes visible in the patient's visit history (Chapter 28).

Error cases.

- "Only this queue's doctor can do that" (403 `FORBIDDEN`) → another doctor's entry without `queue.doctor.any`.
- 422 `INVALID_TRANSITION` → the entry is not in `called`.
- 409 `VERSION_CONFLICT`, "This was changed by someone else. The latest state is shown." → reception changed the entry meanwhile.

Technical references: `backend/src/queue/services/queue-orchestrator.service.ts` · `QueueOrchestrator.startConsultation`, `EncountersService.ensureForQueueEntry` · `web-app/src/features/queue/pages/DoctorQueuePage.tsx`

31.3.2 Re-open a consultation that is already running

Status: [Implemented]

What it does. If the doctor reloads the page, closes the tab, or goes back to their queue, they can return to the same consultation without starting anything again.

How the user does it.

1. On "My Queue", the current patient shows the button "Open consultation".
2. Press it. The workspace opens with everything saved so far.

Behind the scenes — in simple terms. The Platform looks up the encounter that belongs to the queue entry and opens it. If the consultation was never started, it says so.

Behind the scenes — technically. `GET /encounters/by-queue-entry/:queueEntryId` → `EncountersService.getWorkspaceByQueueEntry` finds the encounter by `queue_entry_id` within the active organization and location, checks the conduct scope (`assertConductScope`), and returns the same workspace object as `GET /encounters/:id/workspace` (`getWorkspace`): the encounter with its appointment, patient and treatments (newest first), the queue entry (id, display token built from the session's token prefix and a three-digit number, status, version, session id) and the doctor (id, full name). The web hook `useOpenConsultation` then navigates to `/consultation/$encounterId`.

Rules and validations.

- The encounter must belong to the active organization and location.
- With scope `own`, the caller must be the doctor of the queue session the encounter came from. With scope `location` or `org`, any encounter at this location is reachable.
- Outside that reach the encounter reads as "not found", so its existence is not revealed.

Who can do it. `consultation.conduct` (feature `patients.core`, scopes own / location / org). Doctor template: `own`. Not in the Admin, Receptionist, Assistant or Accountant templates. The route itself is gated by `staticData.can = 'consultation.conduct'`.

Data read and written. Reads `patient_encounters`, `queue_entries`, `queue_sessions`, `employees`, `treatments`, `appointments`, `patients`.

Effects on other modules. None; read only.

Error cases.

- "This patient has not been started yet" (404) → no encounter exists for that queue entry.
- "Encounter not found" (404) → wrong organization or location, or outside the caller's scope.
- The workspace page shows "Couldn't open this consultation" with a retry button when loading fails.

Technical references: `backend/src/encounters/encounters.service.ts` · `EncountersService.getWorkspace`, `assertConductScope` · `web-app/src/features/consultation/hooks/useConsultation.ts`

31.3.3 See the patient at a glance

Status: [Implemented]

What it does. The left column, "Patient at a glance", gives the doctor the latest vitals, the last few visits and the last few signed prescriptions, and lets them reuse an old prescription with one tap.

How the user does it.

1. Read "Latest vitals" (BP, Pulse, Temp °C, SpO_2 %, Weight kg, Height cm, with the date recorded). Press "Record" or "Update" to open the Vitals tab of the record slide-over.
2. Read "Earlier visits": up to four previous encounters, each showing the diagnosis (or complaint, or reason) and date, with the treatments under it. "See all" opens the History tab.
3. Under "Earlier prescriptions", press the repeat icon ("Use these medicines again") next to an old prescription to copy its medicines into today's prescription.

Behind the scenes — in simple terms. The screen pulls the patient's history, vitals and recent prescriptions. Amounts are never shown here.

Behind the scenes — technically. The page calls `GET /patients/:id` (patient), `GET /patients/:id/vitals` (the newest record is picked on the client), `GET /patients/:id/history` (encounters with treatments; the current encounter is filtered out) and `GET /prescriptions/patient/:patientId?page=1&limit=5` (draft and empty prescriptions, and those of this encounter, are filtered out; three are shown). The history endpoint runs through `HideTreatmentPricesInterceptor`, so treatment costs are removed for callers without `patients.price.view`.

Rules and validations.

- Only finalized or amended prescriptions with at least one medicine appear under "Earlier prescriptions".
- Reusing medicines skips any medicine already on today's prescription (matched by name, case-insensitive).

Who can do it. `patients.clinical.view` for history and vitals; reading prescriptions also needs the feature `patients.prescriptions`. The Doctor template holds both.

Data read and written. Reads `patients`, `patient_vitals`, `patient_encounters`, `treatments`, `prescriptions`, `prescription_medicines`.

Effects on other modules. Reusing medicines feeds the Prescription panel (Chapter 32).

Error cases. Each section shows an empty message instead of an error ("No vitals recorded yet.", "First visit — nothing earlier on record.", "No earlier prescriptions.").

Technical references: `web-app/src/features/consultation/components/PatientSnapshot.tsx` · `PatientSnapshot` · `backend/src/patients/patients.controller.ts`

31.3.4 Allergy and medical alerts

Status: [Partial] — the alert strip works, but it is built from free-text fields. There is no structured allergy list and no automatic check of a prescribed medicine against allergies.

What it does. Before treating, the doctor sees coloured chips under the header for anything that can hurt the patient today.

How the user does it. Nothing to do; the chips appear automatically. Red chips are dangers, amber chips are context.

Behind the scenes — in simple terms. The screen reads the patient's allergy and medical-history text and the latest vitals record, and turns them into labels.

Behind the scenes — technically. The web function `patientAlerts(patient, vitals)` builds the list on the client:

- red "Allergy: " when `patients.allergies` is not blank;
- red "Blood thinners" and "Pregnant" from the latest vitals flags `isOnBloodThinners` and `isPregnant`;
- amber "Diabetic", "Hypertensive", "On antibiotics" from the vitals flags;
- amber "Medical history: " from `patients.medical_history`;
- amber "Minor" (with "Guardian: …" when a guardian name exists). No backend endpoint is involved beyond loading the patient and vitals.

Rules and validations. Alerts follow the most recent vitals record only; older flags are ignored.

Who can do it. Anyone who can open the workspace (`consultation.conduct`).

Data read and written. Reads `patients` and `patient_vitals`.

Effects on other modules. None.

Error cases. None; with no data there is no alert strip.

Technical references: `web-app/src/features/consultation/utils/patientInfo.ts` · `patientAlerts` · `web-app/src/features/consultation/components/ConsultationHeader.tsx`

31.3.5 Write the visit notes (auto-saved)

Status: [Implemented]

What it does. The doctor records the chief complaint, examination findings, diagnosis, advice to the patient and the next-visit date. There is no Save button: the notes save themselves.

How the user does it.

1. Type into Complaint, Examination, Diagnosis and "Advice to patient", or tap the phrase chips under them (for example "Toothache", "Dental caries", "Warm salt-water rinse"). The first five chips show, the rest behind "+N more". Tapping a chip appends the phrase once.
2. Under "Next visit", tap a chip such as "1 week" or pick a date (dates before today are blocked). Tap the active chip again to clear it.
3. Watch the save badge in the panel header; it shows saving, saved, or an error.

Behind the scenes — in simple terms. About a second after the doctor stops typing, the screen sends the notes to the server. If a save fails, it tries again on the next change. Before the visit is completed, anything still unsaved is sent first. The Complaint box starts with the reason for visit from check-in when no complaint has been written yet.

Behind the scenes — technically. The `useAutosave` hook waits 800 ms after the last change, then calls `PATCH /encounters/:id` with the five note fields. `EncountersService.updateNotes` loads the encounter for the active organization and location and touches only fields present in the body: an empty string or `null` clears a field, a missing field is left alone. It sets `updated_by` to the user and saves. When the doctor leaves the screen with an unsaved change, the hook sends it once more. The one-tap phrases are fixed lists in the web app (`quickPicks.ts`); clinics cannot edit them.

Rules and validations.

- Each text field is at most 4,000 characters.
- `followUpDate` must be a valid date string, or empty / null to clear.
- Scope check `assertConductScope` runs before every save.
- The server does not refuse edits after the visit is completed.

Who can do it. `consultation.conduct` (scoped). Doctor template: `own`.

Data read and written. Writes `patient_encounters` (`chief_complaint`, `examination`, `diagnosis`, `advice`, `follow_up_date`, `updated_by`).

Effects on other modules. The prescription written in the same workspace copies the complaint, diagnosis, advice and follow-up date into its own fields so they print on the prescription (Chapter 32). The follow-up date does not create an appointment.

Error cases.

- 400 validation message → a field longer than 4,000 characters or an invalid date.
- "Encounter not found" (404) → outside the caller's scope.
- The save badge shows an error state; the next edit retries.

Technical references: `backend/src/encounters/encounters.controller.ts` · `EncountersService.updateNotes`, `UpdateEncounterNotesDto` · `web-app/src/features/consultation/components/VisitNotes.tsx`

31.3.6 Record the treatments done today

Status: [Implemented]

What it does. The doctor records each procedure performed in this visit. Each one becomes a **treatment** in the patient's history and, when the visit is completed, a **charge** for billing. The doctor never sees or types a price.

How the user does it.

1. In "Treatment done today", tap a procedure chip. The chips come from the clinic's service catalog (eight shown; a "Search procedures" box appears when there are more than ten). If the clinic has no catalog yet, the chips are a fixed list of common procedures and the doctor types the name.
2. Tap "Other…" to type a procedure that is not in the catalog.
3. Optionally type the tooth in "Tooth #".
4. Press "Add". The treatment appears in the list with a green tick.

Behind the scenes — in simple terms. A picked catalog procedure is stored with a link to the catalog service, so billing can price it later. A typed name is stored without a price, and reception prices the charge at the billing desk. The tooth number is saved as a note "Tooth: …".

Behind the scenes — technically.

- `GET /treatments/catalog` → `TreatmentsService.listProcedureCatalog` returns id, name and code of active services of kind `PROCEDURE` in the organization, ordered by sort order and name. No prices are returned.
- `POST /encounters/:encounterId/treatments` → `EncountersService.assertPatientInScope`, then `TreatmentsService.createForEncounter`. It takes the patient from the encounter (so the patient can never mismatch) and calls `create`, which:
  - checks the patient exists in the organization and, if given, the plan template;
  - checks `serviceId` points to an active `PROCEDURE` service, else 422;
  - uses the typed name, or the service name when none is typed;
  - stores `cost = null` when a service is linked (billing prices it), else the optional legacy cost;
  - stores `date = now` unless given.
- The web app refreshes the workspace, the patient's treatments and the patient's history.

Rules and validations.

- Either a service or a treatment name is required ("Pick a service or enter a treatment name").
- Name at most 200 characters.
- A linked service must be an active procedure (`TREATMENT_SERVICE_INVALID`).
- The encounter must be at the active location.
- There is no remove or edit button for a treatment in the workspace, and no API endpoint to delete one.

Who can do it. `patients.clinical.edit` (feature `patients.core`), plus the patient must be inside the caller's `patients.view` scope. Doctor and Admin templates hold `patients.clinical.edit`; `consultation.conduct` also implies it.

Data read and written. Reads `services`, `patient_encounters`, `patients`, `plan_templates`; writes `treatments`.

Effects on other modules. Billing turns each treatment into a charge when the visit is completed (31.3.10). Commissions are snapshotted on those charges (Chapter 35). The treatment shows in the patient's history (Chapter 28).

Error cases.

- "Pick a service or enter a treatment name" (400) → nothing chosen.
- "Pick an active procedure from the service catalog" (422 `TREATMENT_SERVICE_INVALID`) → service inactive or not a procedure.
- "Service not found" / "Encounter not found" / "Treatment plan not found" (404).

Technical references: `backend/src/treatments/treatments.service.ts` · `TreatmentsService.createForEncounter`, `listProcedureCatalog` · `web-app/src/features/consultation/components/TreatmentsToday.tsx`

31.3.7 Look up or add to the patient record without leaving

Status: [Implemented]

What it does. The doctor can open the full patient record in a slide-over (Figure 31.2), use it, and close it to return to the consultation.

How the user does it.

1. Press "History", "Teeth", "Vitals" or "Files" in the header, or a link in the patient summary.
2. Switch tabs inside the slide-over as needed and add data.
3. Close the slide-over.

Behind the scenes — in simple terms. The slide-over shows the same tabs as the patient detail page. History is shown without amounts.

Behind the scenes — technically. `RecordDrawer` renders the patients feature components `VisitsTab` and `TreatmentsTab` (with `hideAmounts`), `TeethChartTab`, `VitalsAndDentalTab`, `ClinicalTab`, `FilesTab` and `ConsentsTab`. Their endpoints are described in the Patients chapter (Chapter 28).

Rules and validations. Tabs are hidden without their permission: Teeth chart needs `patients.chart.edit`, Prescriptions needs `prescriptions.create`, X-rays & files needs `patients.documents.manage`.

Who can do it. As for each tab; the Doctor template holds all of them.

Data read and written. As for each patient tab.

Effects on other modules. New vitals immediately refresh the alert strip and the vitals shown in the summary.

Error cases. As for each patient tab.

Technical references: `web-app/src/features/consultation/components/RecordDrawer.tsx` · `RecordDrawer` · `web-app/src/features/patients/components/tabs/`

31.3.8 Write the prescription inside the consultation

Status: [Implemented]

What it does. The "Prescription" panel lets the doctor write, sign and print the prescription for this visit without leaving the workspace. The Prescriptions chapter (Chapter 32) covers prescriptions in full; this section only describes how the panel fits the consultation.

How the user does it.

1. Search a medicine ("Search medicine…"), tap a suggestion chip, press "Add “name”" for a medicine not in the directory, press "Repeat last", or choose from "Use a preset".
2. Open each medicine line to set Dose, How often, For how long, With food and an optional note.
3. Press "Sign & print". If the doctor has no saved signature, a "Sign the prescription" dialog asks them to sign once; it is saved for next time.
4. The PDF opens in an in-app preview with a Print button. Afterwards the panel offers "Print" and "Amend".

Behind the scenes — in simple terms. The prescription is created as a draft the moment the first complete medicine line exists, and saves itself after that. The complaint, diagnosis, advice, follow-up date and today's vitals from the visit notes are copied onto the prescription automatically, so nothing is typed twice. A preset's English instructions are offered to the "Advice" note when it is still empty.

Behind the scenes — technically. The panel uses `GET /prescriptions/encounter/:encounterId` to find this visit's prescription, then `POST /prescriptions` (with `patientId`, `encounterId` and the queue session's doctor as `doctorId`) for the first save and `PATCH /prescriptions/:id` with the full medicine list for later saves (autosave after 1 second). "Sign & print" flushes the autosave, then calls `POST /prescriptions/:id/finalize` with the signature image and `GET /prescriptions/:id/pdf`. Vitals are copied only when the latest vitals record was taken on the encounter's date.

Rules and validations.

- Only medicine lines with name, dose, frequency and duration are saved.
- "Sign & print" refuses with "Choose how often and for how long for every medicine" or "Add at least one medicine".
- The panel is shown whether or not the user holds `prescriptions.create` or the feature `patients.prescriptions` is on; the server refuses the writes in that case.

Who can do it. `prescriptions.create` (feature `patients.prescriptions`). Doctor template only, by default.

Data read and written. `prescriptions`, `prescription_medicines`, `prescription_history`, `doctor_signatures`.

Effects on other modules. The signed prescription appears in the patient's record and the Prescriptions list (Chapter 32).

Error cases. "Could not sign the prescription — try again", "Signed, but the PDF could not be opened — tap Print", "Could not save the prescription — try again".

Technical references: `web-app/src/features/consultation/components/PrescriptionPanel.tsx` · `PrescriptionPanel` · `backend/src/prescriptions/controllers/prescription.controller.ts`

31.3.9 Order lab work from the consultation

Status: [Not implemented] — the workspace has no lab-order entry point, and a **lab order** has no link to an encounter.

What it does. Nothing in the workspace today. A doctor who wants to send a case to a dental lab must leave the consultation and use the Labs module (Chapter 40), which is reached from the sidebar.

How the user does it. Not available in the workspace.

Behind the scenes — in simple terms. The consultation feature code contains no lab component, the record slide-over has no lab tab, and the lab-order tables carry no encounter reference.

Behind the scenes — technically. No `lab` reference exists in `web-app/src/features/consultation/`, and `backend/src/lab-orders/` has no `encounter` field.

Rules and validations. Not applicable.

Who can do it. Not applicable (the Doctor template holds `lab.orders.create` for the Labs module).

Data read and written. None.

Effects on other modules. None.

Error cases. None.

Technical references: `web-app/src/features/consultation/` · `RecordDrawer` · `backend/src/lab-orders/`

31.3.10 Complete the visit

Status: [Implemented]

What it does. Finishing the visit removes the patient from the doctor's queue, closes the booked appointment, tells reception, and creates the charges for billing.

How the user does it.

1. Press "Complete visit" in the header.
2. Read the reminders in the "Complete this visit?" dialog (Figure 31.3).
3. Press "Complete visit" again, or "Keep working" to go back.

4. A "Visit completed" message appears and the doctor returns to "My Queue".

The doctor can also press "Complete" directly on the "My Queue" page without opening the workspace. A user holding `queue.doctor.any` can force-complete a visit the doctor forgot to close (Chapter 30).

Behind the scenes — in simple terms. First the screen makes sure the latest notes and prescription are saved; if that fails, it stops and says so. Then, in one transaction, the queue entry becomes completed, the appointment (if any) becomes completed, and billing creates the charges. The doctor's screen receives no prices or charge data.

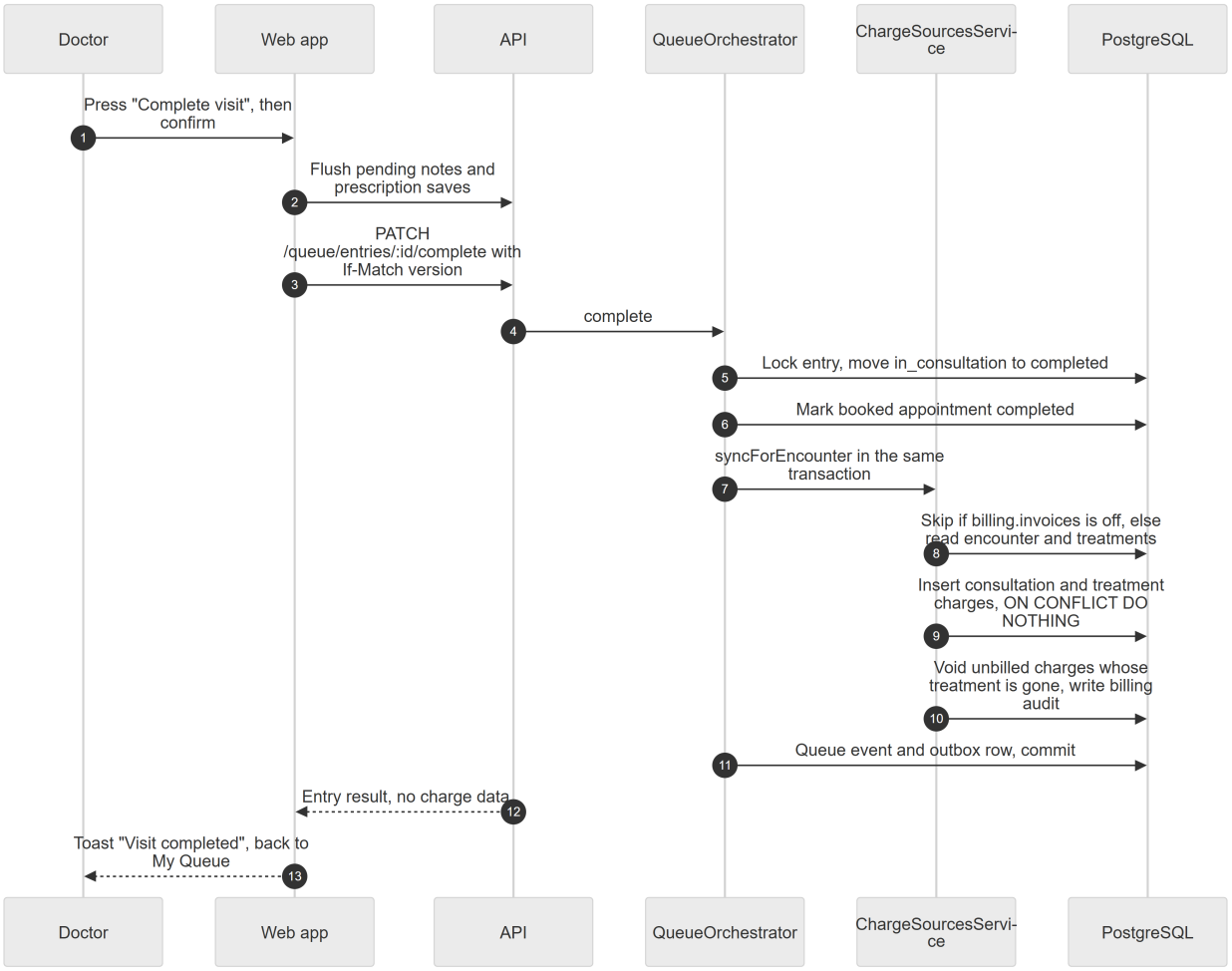

Figure 31.5 — Sequence from "Complete visit" to the automatic charges.

Figure 31.5 is a sequence diagram with six participants: Doctor, Web app, API, QueueOrchestrator, ChargeSourcesService and PostgreSQL. In step 1 the doctor presses "Complete visit" and confirms. In step 2 the web app flushes any pending notes and prescription saves to the API. In step 3 it sends `PATCH /queue/entries/:id/complete` with the entry version in If-Match. In step 4 the API calls `complete` on the QueueOrchestrator. In step 5 the orchestrator locks the entry and moves it from `in_consultation` to `completed`. In step 6 it marks the booked appointment as completed. In step 7 it calls `syncForEncounter` on the ChargeSourcesService in the same transaction. In step 8 that service stops if the feature `billing.invoices` is off, otherwise it reads the encounter and its treatments. In step 9 it inserts the consultation and treatment charges with "ON CONFLICT DO NOTHING". In step 10 it voids unbilled charges whose treatment has disappeared and writes billing audit rows. In step 11 the orchestrator writes the queue event and outbox row and commits. In step 12 the API returns the entry result without any charge data, and in step 13 the web app shows "Visit completed" and returns the doctor to My Queue.

Behind the scenes — technically. `PATCH /queue/entries/:id/complete` → `QueueOrchestrator.complete`: lock, `assertSessionDoctor` (or `assertAnyDoctor` for force-complete), transition to `completed` with `finished_at`, queue event `COMPLETED` (or `FORCE_COMPLETED`), update the appointment to `COMPLETED` with `completed_at`, then

`ChargeSourcesService.syncForEncounter`, all inside one transaction. The automatic charges are described in 31.3.11.

Rules and validations.

- The entry must be `in_consultation`; otherwise 422 `INVALID_TRANSITION`.
- Only the session's doctor, or `queue.doctor.any` for force-complete.
- The web "Complete visit" button is disabled while saving, once the visit has ended, or when the encounter has no queue entry.
- Nothing is mandatory: a visit can be completed with no notes, no treatment and no prescription.

Who can do it. `queue.doctor.own` (complete), `queue.doctor.any` (force-complete). Doctor template: complete; Admin template: force-complete.

Data read and written. `queue_entries`, `queue_events`, outbox, `appointments`, and the billing tables in 31.3.11.

Effects on other modules. Queue board, TV display and reception see the change in real time (Chapter 30). Billing receives the charges (Chapter 34). Commission entries follow the charges (Chapter 35).

Error cases.

- "Your latest changes could not be saved. Check the connection and try again." → the pre-completion save failed; nothing is completed.
- 409 `VERSION_CONFLICT` → someone changed the entry; the dialog closes and the workspace reloads.
- 403 "Only this queue's doctor can do that".

Technical references: `backend/src/queue/services/queue-orchestrator.service.ts` · `QueueOrchestrator.complete` · `web-app/src/features/consultation/components/CompleteDialog.tsx`

31.3.11 Automatic charges when the visit is completed

Status: [Implemented] — runs only when the organization has the feature `billing.invoices` enabled.

What it does. Every completed consultation produces one consultation **charge** plus one charge per treatment, ready for reception to collect. Nobody has to re-enter what the doctor did.

How the user does it. Nothing; it happens on completion. Reception sees the charges at the billing desk (Chapter 34).

Behind the scenes — in simple terms. The consultation fee comes from the clinic's "default consultation service" in billing settings. Each treatment linked to a catalog procedure is priced from the catalog, using the doctor-specific price when one exists. A treatment typed by name uses its old-style fee if it has one; otherwise the charge is created at zero and marked "needs pricing" for reception. A pricing problem never blocks the doctor: the charge is only marked "needs pricing". If the same visit is completed twice, no duplicate charges appear.

Behind the scenes — technically. `ChargeSourcesService.syncForEncounter(manager, { tenant, queueEntryId, doctorId, completedAt, actorUserId })`:

- Returns at once when `billing.invoices` is not enabled for the organization, or when no encounter exists for the queue entry.
- Loads the encounter's treatments and the existing active charges with source `CONSULTATION` or `TREATMENT` for this encounter.
- Builds drafts with `ChargeDraftFactory`: one `CONSULTATION` draft (source id = encounter id) if none exists, and one `TREATMENT` draft (source id = treatment id) for each treatment not yet billed.

- Prices them through `ChargePricingService.resolve`. A `BillingException` is caught, so the draft gets `unit_price = 0` and `needs_pricing = true`.
- Sets the charge date to the completion moment converted to a date in the **location's** timezone.
- Takes a commission snapshot for the doctor's compensation plan on that date (`CommissionService.snapshot`).
- Inserts with `.orIgnore()` against the partial unique index `uq_charges_source` (organization, source, source id, active only).
- Writes `CREATED` rows to the billing audit log.

Rules and validations.

- One active charge per source (enforced by `uq_charges_source`).
- The function returns nothing, so no charge data reaches the doctor's response.
- Runs inside the queue completion transaction: if it fails, the completion is rolled back.

Who can do it. Runs as a system billing actor on behalf of the doctor who completed the visit; no billing permission is needed by the doctor.

Data read and written. Reads `patient_encounters`, `treatments`, `location_billing_settings`, `services`, `service_prices` and the compensation tables (`compensation_plans`, `commission_rules`); writes `charges` and `billing_audit_logs`.

Effects on other modules. Billing desk and ready-to-bill list (Chapter 34); commission entries (Chapter 35); finance reports (Chapter 36).

Error cases. None shown to the doctor; pricing failures are logged as warnings and the charge is flagged "needs pricing".

Technical references: `backend/src/billing/charges/charge-sources.service.ts` · `ChargeSourcesService.syncForEncounter`, `ChargeDraftFactory` · (no screen; called by `QueueOrchestrator.complete`)

31.3.12 Void the charge of a treatment the doctor removed

Status: [Partial] — the logic exists in billing, but no current flow reaches it: there is no endpoint or button that removes a treatment from an encounter, and the charge sync runs only once, at completion.

What it does. If a treatment that was already charged disappears from the encounter, its unbilled charge is voided with the reason "Treatment removed by doctor". If the charge is already on an invoice, the money is kept and the charge is only flagged for review.

How the user does it. No user action leads here today.

Behind the scenes — in simple terms. When billing compares the encounter's treatments with the charges it already created, a charge without a matching treatment is either voided (not invoiced yet) or flagged (already invoiced).

Behind the scenes — technically. `ChargeSourcesService.handleRemoved` finds active `TREATMENT` charges whose `source_id` is no longer among the encounter's treatments. Charges without `invoice_id` are voided (`voidCharge`, reason "Treatment removed by doctor") and audited as `VOIDED`. Invoiced charges get `source_removed_at` set to the completion time.

Rules and validations. Invoiced charges are never voided by this path.

Who can do it. System, inside the completion transaction.

Data read and written. `charges`, `billing_audit_logs`.

Effects on other modules. Billing review of flagged charges (Chapter 34).

Error cases. None.

Technical references: `backend/src/billing/charges/charge-sources.service.ts` · `ChargeSourcesService.handleRemoved` · (no screen)

31.3.13 Keep prices away from doctors

Status: [Implemented]

What it does. Doctors never see treatment prices in the consultation, the procedure picker or the patient history.

How the user does it. Nothing; it is automatic.

Behind the scenes — in simple terms. Any response from the consultation or treatment endpoints has the `cost` field removed unless the caller may see prices.

Behind the scenes — technically. `HideTreatmentPricesInterceptor` is applied to the encounters and treatments controllers (and the patient history endpoint). It deletes `cost` from every object that has a `treatmentName`, anywhere in the response, unless the caller holds `patients.price.view`. With no access context it strips (fail-closed). The `Treatment.cost` column also carries `@VisibleIf('patients.price.view')`. The procedure catalog returns only id, name and code.

Rules and validations. Prices are visible only with `patients.price.view`.

Who can do it. `patients.price.view` is held by the Receptionist, Accountant and Admin templates, not by Doctor.

Data read and written. None (response filtering only).

Effects on other modules. None.

Error cases. None.

Technical references: `backend/src/common/interceptors/hide-treatment-prices.interceptor.ts` · `HideTreatmentPricesInterceptor` · `web-app/src/features/consultation/utils/treatmentInput.ts`

31.3.14 Encounter and treatment records outside the consultation

Status: [Implemented]

What it does. The same backend serves the patient record: reading one encounter in full, listing treatments of a patient or an encounter, and recording a treatment without a consultation.

How the user does it. From the patient detail page (Chapter 28), for example the visits and treatments tabs.

Behind the scenes — in simple terms. These endpoints follow the patient's visibility rules: a patient outside the caller's reach looks like it does not exist.

Behind the scenes — technically.

- `GET /encounters/:id` → `assertPatientInScope`, then `findOne`: the encounter with appointment, queue entry, patient and treatments (newest first), limited to the active location.
- `GET /treatments?patientId=` or `?encounterId=` → treatments with their plan template, newest first. One of the two parameters is required.
- `POST /treatments` → the standalone create path of `TreatmentsService.create`. Here the caller may pass `encounterId`, `planTemplateId`, `serviceId` and a legacy `cost` (0 or more, two decimals).

Rules and validations.

- A treatment linked to an encounter must belong to the encounter's patient ("Treatment patient does not match the encounter patient").
- "Provide either patientId or encounterId" when listing without either.
- Patient scope enforced on every path.

Who can do it. `patients.clinical.view` (reads), `patients.clinical.edit` (create). Reads: Doctor, Assistant, Admin. Create: Doctor, Admin.

Data read and written. `patient_encounters`, `treatments`.

Effects on other modules. A standalone treatment with no encounter is not picked up by the automatic charges of 31.3.11.

Error cases. 404 "Patient not found" / "Encounter not found"; 400 for mismatched patient or missing parameter.

Technical references: `backend/src/treatments/treatments.controller.ts` · `TreatmentsService.create`, `EncountersService.findOne` · `web-app/src/features/patients/services/patients.api.ts`

31.3.15 SOAP notes and note templates

Status: [Not implemented]

What it does. There is no separate clinical-note entity (for example structured SOAP notes with versions) and no clinic-defined note templates. The encounter's own fields (complaint, examination, diagnosis, advice, follow-up date) are the only visit notes, and the one-tap phrases are fixed in the web-app code.

How the user does it. Not available.

Behind the scenes — in simple terms. Not built.

Behind the scenes — technically. No entity, controller or route for clinical notes or note templates exists in `backend/src/encounters/` or `web-app/src/features/consultation/`.

Rules and validations. Not applicable.

Who can do it. Not applicable.

Data read and written. None.

Effects on other modules. None.

Error cases. None.

Technical references: `backend/src/encounters/entities/patient-encounter.entity.ts` · `PatientEncounter` · `web-app/src/features/consultation/constants/quickPicks.ts`

31.4 Entities and data model

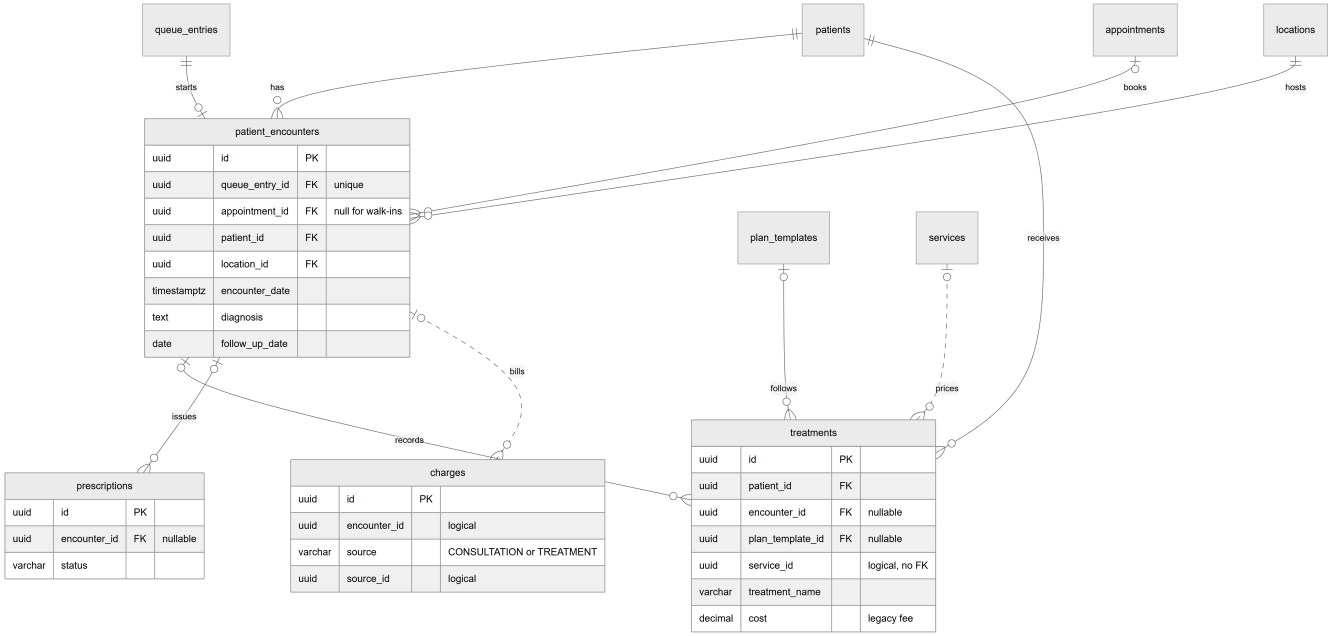

Figure 31.6 — Entity-relationship diagram of encounters, treatments and their neighbours.

Figure 31.6 shows the table `patient_encounters` in the centre with its key columns: `id` (primary key), `queue_entry_id` (foreign key, unique), `appointment_id` (foreign key, null for walk-ins), `patient_id`, `location_id`, `encounter_date`, `diagnosis` and `follow_up_date`. In the top row are `queue_entries` ("starts"), `patients` ("has"), `appointments` ("books") and `locations` ("hosts"). Under it, `treatments` ("records") shows `patient_id`, the nullable `encounter_id` and `plan_template_id`, a logical `service_id` without a foreign key, `treatment_name` and the legacy `cost`. `treatments` also links to `patients` ("receives"), to `plan_templates` ("follows") and, with a dotted line, to `services` ("prices"). `prescriptions` hangs off the encounter ("issues") through a nullable `encounter_id`. `charges` is connected to the encounter by a dotted line ("bills") through the logical columns `encounter_id`, `source` (CONSULTATION or TREATMENT) and `source_id`.

The relationships in words:

- **Queue entry → encounter.** Each encounter belongs to exactly one queue entry, and a queue entry has at most one encounter (unique `queue_entry_id`). Deleting the queue entry deletes the encounter (ON DELETE CASCADE).
- **Appointment → encounter.** An encounter has an appointment only for booked visits; walk-ins have none. Deleting the appointment keeps the encounter and sets `appointment_id` to null (ON DELETE SET NULL).
- **Patient, organization, location → encounter.** Each encounter belongs to one patient, one organization and one location. Deleting any of them deletes the encounter (ON DELETE CASCADE). The patient id is stored on the encounter for fast lookup even though the queue entry also has it.
- **Encounter → treatments.** An encounter can have many treatments. A treatment always belongs to a patient (ON DELETE CASCADE from patients) and may belong to an encounter; deleting the encounter keeps the treatment and clears `encounter_id` (ON DELETE SET NULL).
- **Plan template → treatments.** A treatment may follow a clinic **plan template**; deleting the template clears the link (SET NULL).
- **Service → treatment.** `treatments.service_id` points to a row in `services` (the billing catalog) but has no database foreign key; it is a logical link checked by the service code.
- **Encounter → prescriptions.** A prescription may record the encounter it was written in; deleting the encounter clears the link (SET NULL).

- **Encounter → charges.** `charges.encounter_id` and `charges.source_id` are logical links (no foreign keys). For the consultation charge, `source_id` is the encounter id; for a treatment charge, it is the treatment id.

Indexes exist on `appointment_id`, `organization_id`, `location_id`, `patient_id` and `encounter_date` of `patient_encounters`, and on organization, location, patient and encounter of `treatments`. The encounter is a different thing from `patient_visits`, which is a per-location visit-count aggregate (Chapter 18 explains both).

31.5 Status lifecycles

The encounter itself has **no status column**. Whether a consultation is running, paused or finished is read from the status of its queue entry. Figure 31.7 therefore shows the queue-entry states that matter to an encounter, using the code's state names.

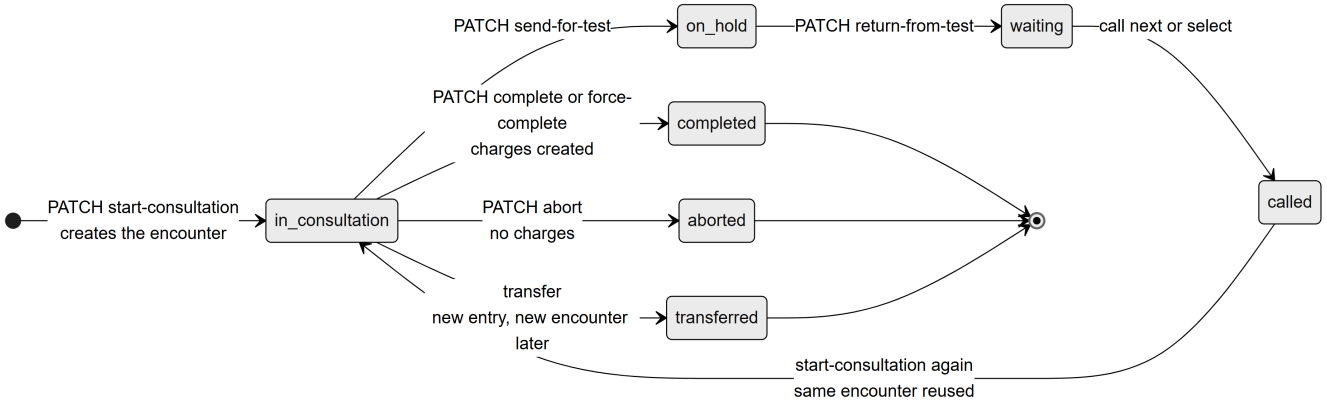

Figure 31.7 — Life of an encounter, read from the status of its queue entry.

Figure 31.7 is a left-to-right state diagram. From the start point, the transition "PATCH start-consultation, creates the encounter" leads to `in_consultation`. From `in_consultation` four transitions leave:

- "PATCH send-for-test" goes to `on_hold`. From there "PATCH return-from-test" goes to `waiting`, "call next or select" goes to `called`, and "start-consultation again, same encounter reused" returns to `in_consultation`.
- "PATCH complete or force-complete, charges created" goes to `completed`.
- "PATCH abort, no charges" goes to `aborted`.
- "transfer, new entry, new encounter later" goes to `transferred`.

`completed`, `aborted` and `transferred` each lead to the end point.

Walking through each state:

- **in_consultation.** The encounter exists and the doctor is working. Notes, treatments and the prescription are saved continuously.
- **on_hold.** The doctor pressed "Send for test" (for example an X-ray). The encounter and everything written so far stay. When reception marks the patient back, the entry returns to `waiting`; the doctor calls the patient again and starts again, and `ensureForQueueEntry` returns the same encounter.
- **completed.** The visit is finished. The appointment is marked completed and the automatic charges are created (31.3.11). The workspace shows "Visit ended" and disables "Complete visit".
- **aborted.** The doctor pressed "Stop consultation" on "My Queue". The appointment is closed as cancelled. The encounter row and its notes and treatments remain, but no charges are created because the charge sync runs only on completion.
- **transferred.** The patient was moved to another doctor. The encounter stays with the original queue entry. The new queue entry in the other doctor's queue gets its own encounter when that doctor starts.

The server does not lock an encounter after completion: `PATCH /encounters/:id` and `POST /encounters/:id/treatments` still accept changes. A treatment added after completion is not charged automatically.

31.6 Connections to other modules

- **Queue (Chapter 30).** Starting, sending for test, completing, aborting and transferring are queue actions run by `QueueOrchestrator`. The encounter is created inside the start transaction and billing runs inside the complete transaction.
- **Appointments (Chapter 29).** A booked visit's appointment is linked to the encounter and is marked completed (or cancelled on abort).
- **Patients (Chapter 28).** The workspace reads the patient, vitals and history, and the record slide-over reuses the patient tabs. Patient visibility (`patients.view` scope) governs treatment and encounter reads.
- **Prescriptions (Chapter 32).** The Prescription panel writes a prescription linked to the encounter and copies the visit notes onto it.
- **Treatment plans (Chapter 33).** A treatment can reference a plan template; planned work is separate from treatments performed.
- **Billing (Chapter 34).** Completion creates consultation and treatment charges, priced from the service catalog.
- **Commissions (Chapter 35).** Each automatic charge carries a commission snapshot for the doctor.
- **Access control (Chapter 57).** `consultation.conduct` , `patients.clinical.*` , `patients.price.view` , `queue.doctor.*` .

31.7 Limitations and known gaps

- No lab-order entry point in the workspace, and lab orders do not reference an encounter.
- No structured clinical notes, SOAP note entity or note templates; the one-tap phrases are fixed in code and dental-oriented.
- Allergies are free text; there is no allergy list and no check of prescribed medicines against allergies.
- Treatments cannot be edited or removed from the workspace, and no API deletes them. The "Treatment removed by doctor" voiding therefore has no current trigger.
- The encounter is not locked after completion; later edits are saved but later treatments are not charged.
- The follow-up date is only a note; it does not book an appointment or remind anyone.
- The Prescription panel is shown even when the user lacks `prescriptions.create` or the prescriptions feature is off; the server then refuses the save.
- Automatic charges depend on the feature `billing.invoices` being enabled for the organization; with it off, completion creates no charges and nothing is created later.

31.8 Owner's FAQ

When exactly is the encounter created? When the doctor presses "Patient is here — start", in the same transaction that moves the queue entry to `in_consultation` . Check-in does not create it, so waiting patients can still be cancelled cleanly.

Can a receptionist open the consultation screen? Not with the default templates. The route and the workspace endpoints need `consultation.conduct`, which only the Doctor template holds. A custom role can be given it at scope `location` or `org`.

Can one doctor open another doctor's consultation? Not with scope `own`, which the Doctor template uses: the encounter must come from the doctor's own queue session, otherwise it reads as "not found".

What happens if the doctor's connection drops while typing? The notes save about a second after typing stops. A failed save shows an error badge and is retried on the next change. Before completion the screen saves everything first and refuses to complete if that fails.

What if the doctor completes a visit without writing anything? It is allowed. The dialog only warns. A consultation charge is still created (if billing is enabled), priced from the default consultation service or marked "needs pricing".

Do doctors ever see prices? No. The treatment cost is stripped from responses for anyone without `patients.price.view`, and the procedure picker returns names only.

Where are the visit notes stored? In `patient_encounters`, columns `chief_complaint`, `examination`, `diagnosis`, `advice` and `follow_up_date`, one row per consultation.

What happens to the record if the visit is stopped half-way? The encounter and anything written stay in the patient's history, the appointment is cancelled, and no charges are created.

If the patient goes for an X-ray and comes back, is a second record created? No. Starting the consultation again re-uses the same encounter for that queue entry.

Chapter 32 — Prescriptions

This chapter describes the prescriptions module of the Platform: how doctors write, sign, amend and print prescriptions, and how the clinic maintains the library behind them (medicines, ready-made medicine sets and print layouts). It is written for the product owner. Prescriptions are part of the clinical record of a patient and are usually written inside the consultation workspace, so this module sits next to Patients (Chapter 28) and Consultation (Chapter 31).

32.1 What this module is for

A **prescription** is the signed list of medicines a doctor gives a patient, with dose, frequency, duration and instructions in English and Urdu, printed on the clinic's letterhead. In the Platform a prescription starts as a draft, becomes final when the doctor signs it, and can be amended later; every change is logged.

The module has three parts:

- **Prescriptions themselves**: list, builder, detail page, signing, PDF printing.
- **The prescription library**, shared by the whole organization:
  - the **medicine directory** (the drug catalogue doctors search);
  - **prescription presets** (ready-made sets of medicines for common situations);
  - **print layouts** (letterhead, footer and paper size).
- **Doctor signatures**: each doctor's saved signature image.

Who uses it.

- Doctors write and sign prescriptions (`prescriptions.create` , held only by the Doctor system role by default).
- Staff with `patients.clinical.view` (Doctor, Assistant and Admin by default) can read and print them.
- The Admin role maintains the library (`prescriptions.library.manage`).
- The owner holds every permission.

The whole module sits behind the **feature** `patients.prescriptions` , which is enabled by default for new organizations.

Where it appears. The sidebar item "Prescriptions" (`/prescriptions`) sits in the "Clinic" section and is shown to users holding `prescriptions.create` . The library pages have no sidebar entries of their own: they are reached through the "Library" button on the Prescriptions page and through links in the builder, and they share one tabbed header, "Prescription Library". Inside the consultation workspace, the "Prescription" panel is the doctor's main way to write prescriptions.

32.2 Screens

There are no usable screenshots of these screens yet, so each figure is a placeholder followed by a description taken from the web-app code.

Figure 32.1 — Prescriptions list (screenshot pending).

The Prescriptions page (`/prescriptions`) has the title "Prescriptions" and the subtitle "Create, review, and manage patient prescriptions." In the top-right are a card/table view toggle, a "Library" button that opens the prescription library, and "New Prescription" (only with `prescriptions.create`). A strip of four counters shows Total, This month, Drafts and Finalized. Under the counters, a filter row has status tabs "All", "Drafts", "Finalized" and "Amended", each with a count, a "Clear filter" button when a status is chosen, and the number of prescriptions found. The table view lists Patient, Doctor, Medicines, Status (a coloured badge: Draft, Finalized or Amended), Created, and an "Open prescription" icon button; clicking a row also opens it. The card view shows the same prescriptions as cards. Both views have pagination with a page-size selector (10, 20, 50, 100). Empty, error and "No prescriptions match this status." states are shown when relevant.

Screenshot pending

Prescription builder

C:/Program Files/Git/prescriptions/new

Figure 32.2 — Prescription builder (screenshot pending).

The builder (`/prescriptions/new` , title "New Prescription", with a back button to the list) has an editor on the left and a live preview of the printed page on the right. The toolbar of the editor has a "Presets" menu ("Apply prescription preset", with a link "Manage prescription presets"), a layout selector listing the organization's print layouts (or a "Set up layout" link when there are none), and, for a saved prescription, "History", a "Finalized" badge and "Print". The editor then has cards for:

- **Patient**: search box "Search patient by name or phone…", with "Change" once a patient is chosen.
- **Prescribing Doctor**: "Select a doctor…".
- **Chief Complaint** and **Diagnosis**.
- **Vitals (optional)**: BP, Pulse, Temp, SpO_2, Weight, Height.
- **Medicines**: numbered rows, each with a medicine search box, Dose, Frequency (free text or a pick list), a meal-relation selector, Duration, and English and Urdu instruction boxes, plus a delete icon. "Add medicine" adds a row.
- **Further Plan / Comment**.
- **Instructions (English)** and a right-to-left Urdu instructions box set in the Nastaliq font.

At the bottom are "Save Draft" and "Finalize & Sign" ("Amend & Sign" for a finalized prescription).

Screenshot pending

Prescription detail and print

`C:/Program Files/Git/prescriptions/$prescriptionId`

Figure 32.3 — Prescription detail and print (screenshot pending).

The detail page (`/prescriptions/$prescriptionId`) has the title "Prescription" and a back button "Back to prescriptions". It shows the same builder, loaded with the saved prescription, so the doctor can keep editing a draft or amend a finalized one. "Print" in the toolbar fetches the PDF and opens it in a new browser tab. The live preview on the right shows the prescription number and date.

Screenshot pending

Prescription Library: Presets

`C:/Program Files/Git/prescription-presets`

Figure 32.4 — Prescription Library: Presets (screenshot pending).

All three library pages share the title "Prescription Library", a back button to the prescriptions list and a tab bar with "Presets" (hint "Ready-made medicine sets"), "Medicines" ("Drug catalogue") and "Print layouts" ("Letterhead and paper"). The Presets tab (`/prescription-presets`) has the subtitle "Ready-made medicine sets. One tap adds

a whole set to a prescription." It offers a search box "Search presets or medicines…", a category filter ("All categories" and about forty categories such as Toothache, Post extraction, Root canal, Pregnancy safe), a card/table toggle, and "New preset" for library managers. The table shows Preset, Category, Medicines and row actions "Edit preset" and "Deactivate preset" (library managers only). The empty state suggests saving a set such as "Toothache after extraction". The "New prescription preset" side sheet has sections "Preset" (Name, Category, Description), "Medicines" (each with Dose, How often, With food, For how long) and "Instructions for the patient" (English, Urdu, Further plan).

Figure 32.5 — Prescription Library: Medicines (screenshot pending).

The Medicines tab (`/medicine-directory`) has the subtitle "The drug catalogue doctors search while writing a prescription." It has a search box "Search by brand or generic name…" (300 ms debounce), a type filter ("All types", Tablet, Capsule, Syrup and so on), a card/table toggle and "Add medicine" for library managers. The table shows Medicine (with the generic name under it), Type, Strength, "Usual prescription" (the default dose, frequency, duration and meal relation), and Edit / Deactivate icon buttons. When the directory is empty, the empty state offers "Add common medicines" and "Add medicine". The "Add medicine" side sheet has "Medicine" (Name, Generic name, Urdu name, Type, Strength) and "Defaults on a prescription" (Dose, With food, How often, For how long). Deactivation asks "Deactivate this medicine?" and explains that existing prescriptions are not changed.

Screenshot pending

Prescription Library: Print layouts

C:/Program Files/Git/prescription-templates

Figure 32.6 — Prescription Library: Print layouts (screenshot pending).

The Print layouts tab (`/prescription-templates`) has the subtitle "Letterhead, footer and paper size for printed prescriptions." It lists layouts with Layout, Paper and Default columns and row actions, and offers "New layout" for library managers. The empty state reads "No print layouts yet". The layout side sheet shows a live preview of the printed page beside these sections:

- **Layout**: Name, Paper size (A4, A5, Letter), Accent colour, "Use as the default layout".
- **Letterhead (top of the page)**: Clinic name, Clinic name in Urdu, Qualification, Registration no., Doctor phone.
- **Footer (bottom of the page)**: Address, Timings, Clinic phone, Small print.
- **Show on the prescription**: switches for Vitals, Medicine names in Urdu, Patient gender, Date.

32.3 Features

32.3.1 Browse and filter prescriptions

Status: [Implemented]

What it does. Shows every prescription at the active location, with counters and a status filter.

How the user does it.

1. Open "Prescriptions" in the sidebar.
2. Pick a status tab ("All", "Drafts", "Finalized", "Amended"); "Clear filter" resets it.
3. Switch between table and cards; change the page size or page.
4. Click a row or the "Open prescription" icon.

Behind the scenes — in simple terms. The list only contains prescriptions of patients the user is allowed to see, at the location the user is working in.

Behind the scenes — technically.

- `GET /prescriptions` → `PrescriptionService.findAll` : filters by organization and active location, applies the caller's `patients.view` scope to `patient_id` (`patientScope`), and accepts optional filters `patientId` , `doctorId` , `status` , `dateFrom` , `dateTo` , `page` , `limit` . It returns rows with patient, doctor and medicines, newest first.
- `GET /prescriptions/stats` → `getStats` returns `total` , `thisMonth` , `finalized` and `draft` under the same scope.
- The "Amended" count is computed in the browser as total minus drafts minus finalized.

Rules and validations.

- `status` must be DRAFT, FINALIZED or AMENDED.
- `page` and `limit` are at least 1.
- "This month" starts on the first day of the month in the server's clock, not the location's timezone.

Who can do it. `patients.clinical.view` plus feature `patients.prescriptions` (controller-level `@RequireFeature`). Route: any of `prescriptions.create` or `patients.clinical.view` . Doctor, Assistant and Admin templates can open it; only the Doctor sees the sidebar item.

Data read and written. Reads `prescriptions` , `prescription_medicines` , `patients` , `employees` .

Effects on other modules. None.

Error cases. Error state with retry when the list fails; "No prescriptions match this status." for an empty filter.

Technical references: `backend/src/prescriptions/services/prescription.service.ts` · `PrescriptionService.findAll` , `getStats` · `web-app/src/features/prescriptions/pages/PrescriptionsPage.tsx`

32.3.2 Write a prescription and save it as a draft

Status: [Implemented]

What it does. Creates a new prescription for a patient, with medicines, vitals and instructions, stored as a draft until it is signed.

How the user does it.

1. Press "New Prescription" on the list, or "New Prescription" on the patient's prescriptions tab (which pre-selects the patient).
2. Choose the patient and the prescribing doctor.
3. Fill Chief Complaint, Diagnosis and optional vitals.
4. Add medicines. Typing two or more letters searches the medicine directory; picking a result fills its defaults. Set dose, frequency, meal relation, duration and instructions.
5. Optionally fill Further Plan and the English and Urdu instructions.
6. Press "Save Draft". The page moves to the saved prescription.

In the consultation workspace the same thing happens automatically: the draft is created as soon as the first complete medicine line exists and then saves itself every second (Chapter 31 describes the panel).

Behind the scenes — in simple terms. The Platform gives the prescription a number such as "RX-2026-00042", links it to the patient (and the visit or appointment when known), chooses the print layout, and logs a "created" entry in its history.

Behind the scenes — technically. `POST /prescriptions` → `PrescriptionService.create(ctx, dto, dto.doctorId)` :

1. If `treatmentPlanTemplateId` (a preset) is given, the preset's medicines, further plan and instructions fill any fields the caller left empty.
2. The print layout is the one given in the request; otherwise the location's default; otherwise the organization-wide default; otherwise none.
3. The number is generated per organization as `RX-{current year}-{5 digits}`: it reads the organization's most recently created prescription and adds one to its last number part.
4. The prescription is saved with status `DRAFT`, version 1 and its medicines (cascade). Missing meal relation becomes `NONE`, missing sort order becomes the row position.
5. A history row `CREATED` is written.

Later saves use `PATCH /prescriptions/:id`, covered in 32.3.4.

Rules and validations.

- `patientId` is required (UUID).
- Each medicine needs a name (at most 200 characters), dosage (50), frequency (100) and duration (50). Urdu name is at most 200 characters, type at most 30. Meal relation must be BEFORE_MEAL, AFTER_MEAL, WITH_MEAL, EMPTY_STOMACH or NONE.
- The builder refuses to save without a patient ("Please select a patient first") or without a named medicine ("Add at least one medicine").
- The prescription number column is unique across the whole Platform.

Who can do it. `prescriptions.create` (feature `patients.prescriptions`). Doctor template only by default; the Admin template deliberately leaves it out.

Data read and written. Reads `treatment_plan_templates`, `prescription_templates`; writes `prescriptions`, `prescription_medicines`, `prescription_history`.

Effects on other modules. The prescription appears on the patient record (Chapter 28) and, when linked to an encounter, in the consultation workspace and the patient's "Earlier prescriptions".

Error cases.

- 400 with the field message → a medicine line with an empty dose, frequency or duration (the builder sends any row with a name).
- "Treatment plan template not found" (404) → the preset id does not belong to the organization.

Technical references: `backend/src/prescriptions/services/prescription.service.ts` · `PrescriptionService.create`, `CreatePrescriptionDto` · `web-app/src/features/prescriptions/components/PrescriptionBuilder.tsx`

32.3.3 Fill a prescription from a preset

Status: [Implemented]

What it does. A **prescription preset** (for example "Root canal aftercare") adds a whole set of medicines and instructions in one tap.

How the user does it.

- In the builder, open "Presets" and choose one. The medicine list is replaced by the preset's medicines, and empty Further Plan and instruction boxes are filled from the preset.
- In the consultation workspace, open "Use a preset" in the Prescription panel. The preset's medicines are added (duplicates by name are skipped), and its English instructions go into the visit's "Advice" note if that is empty.

Behind the scenes — in simple terms. Presets live in the prescription library and belong to the whole organization. Choosing one only copies its contents; later changes to the preset do not change prescriptions already written.

Behind the scenes — technically. The web app loads `GET /treatment-plan-templates` (active presets, optional `category` filter, ordered by name, medicines eager-loaded) and copies the medicines into the editor. The builder also sends the preset id as `treatmentPlanTemplateId`, which is stored on the prescription.

Rules and validations. Only active presets are listed.

Who can do it. Reading presets is open to any member of the organization (`@AnyMember`) when the feature is on. Writing the prescription needs `prescriptions.create`.

Data read and written. Reads `treatment_plan_templates`, `treatment_plan_template_medicines`.

Effects on other modules. None beyond the prescription.

Error cases. None specific; an empty preset list hides the menu in the workspace and shows a "Manage prescription presets" link in the builder.

Technical references: `backend/src/prescriptions/controllers/treatment-plan-template.controller.ts` · `TreatmentPlanTemplateService.findAll` · `web-app/src/features/prescriptions/components/PrescriptionBuilder.tsx`

32.3.4 Sign (finalize) a prescription

Status: [Implemented]

What it does. Signing makes the prescription final: it is stamped with the doctor's signature and time, and a PDF is produced.

How the user does it.

- In the builder: press "Finalize & Sign". The prescription is saved first. The dialog "Sign & finalize prescription" shows the saved signature (if any) with "Tap Confirm to use this — or draw a new one below to override it", and a "Draw new signature" pad. Press "Confirm & Sign".
- In the consultation workspace: press "Sign & print". If no signature is saved, the "Sign the prescription" dialog asks once ("Sign with your finger or mouse. It is saved, so you only do this once."). The PDF opens in an in-app preview with "Print".

Behind the scenes — in simple terms. A newly drawn signature is saved for next time. The prescription is checked (it must have medicines), marked final, and a PDF is generated and stored in the clinic's file storage if storage is available. If PDF storage fails, signing still succeeds and the PDF is built when someone prints.

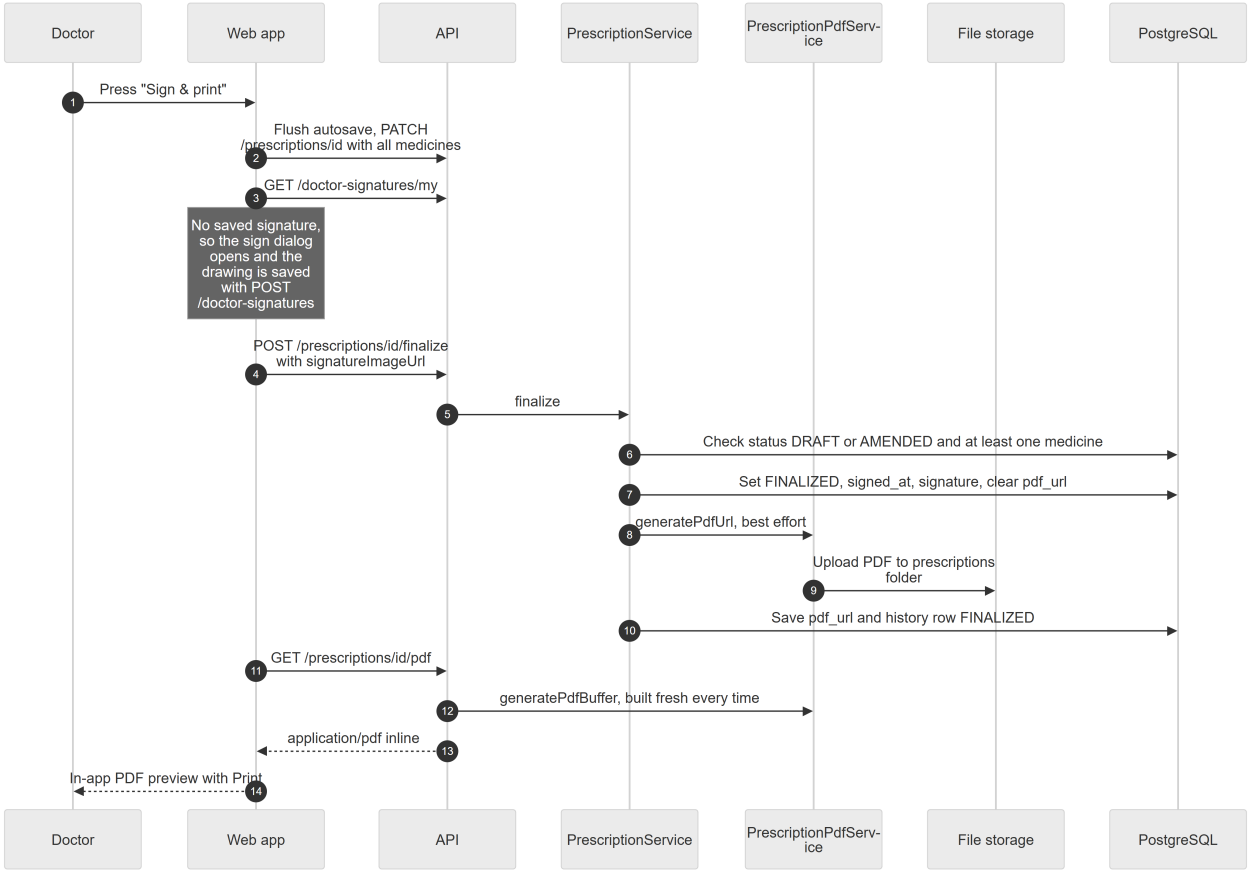

Figure 32.7 — Sequence of "Sign & print" in the consultation workspace.

Figure 32.7 is a sequence diagram with seven participants: Doctor, Web app, API, PrescriptionService, PrescriptionPdfService, File storage and PostgreSQL. In step 1 the doctor presses "Sign & print". In step 2 the web app flushes the autosave with `PATCH /prescriptions/id` carrying all medicines. In step 3 it asks `GET /doctor-signatures/my`. A note explains that when no signature is saved, the sign dialog opens and the drawing is saved with `POST /doctor-signatures`. In step 4 the web app sends `POST /prescriptions/id/finalize` with the signature image. In step 5 the API calls `finalize` on the PrescriptionService. In step 6 the service checks that the status is DRAFT or AMENDED and that there is at least one medicine. In step 7 it sets FINALIZED, the signing time and the signature, and clears the stored PDF address. In step 8 it asks the PrescriptionPdfService to generate the PDF address, best effort. In step 9 that service uploads the PDF to the "prescriptions" folder in file storage. In step 10 the PrescriptionService saves the PDF address and a FINALIZED history row. In steps 11 to 13 the web app calls `GET /prescriptions/id/pdf`, the API builds the PDF fresh with `generatePdfBuffer` and returns it inline as `application/pdf`. In step 14 the doctor sees the in-app preview with Print.

Behind the scenes — technically. `POST /prescriptions/:id/finalize` → `PrescriptionService.finalize`:

- Refuses unless the status is `DRAFT` or `AMENDED` and at least one medicine exists.
- Takes the signature image from the request, otherwise the saved signature found by the caller's user id (`DoctorSignatureService.getByEmployee`).
- Sets `status = FINALIZED`, `signed_at = now`, `signature_image_url`, `pdf_url = null`, and saves.
- Calls `PrescriptionPdfService.generatePdfUrl`, which builds the PDF and uploads it through the `StorageProvider` (folder "prescriptions"); any error is swallowed and the URL stays empty.
- Logs history `FINALIZED` with optional `changeNotes`.

Rules and validations.

- Only DRAFT or AMENDED can be finalized.
- At least one medicine is required.

- The workspace also refuses when a medicine line is incomplete ("Choose how often and for how long for every medicine").
- The builder refuses without a signature ("Please draw your signature or use a saved one").

Who can do it. `prescriptions.create`. Doctor template only by default.

Data read and written. `prescriptions`, `prescription_history`, `doctor_signatures`; the PDF file in external storage.

Effects on other modules. The signed status shows in the consultation's "Complete this visit?" dialog ("Prescription signed") and in the patient's "Earlier prescriptions".

Error cases.

- "Only DRAFT or AMENDED prescriptions can be finalized" (400).
- "Cannot finalize a prescription with no medicines" (400).
- Workspace toasts "Could not sign the prescription — try again" and "Signed, but the PDF could not be opened — tap Print".

Technical references: `backend/src/prescriptions/services/prescription.service.ts` · `PrescriptionService.finalize` · `web-app/src/features/consultation/components/PrescriptionPanel.tsx`

32.3.5 Amend a signed prescription

Status: [Implemented]

What it does. A signed prescription can still be corrected. Editing it turns it into an **amended** prescription with a higher version number; signing again makes it final once more. The state before each change is kept in the history.

How the user does it.

- Builder: open the prescription, edit, then press "Amend & Sign" (or "Save Draft" to save the amendment unsigned).
- Workspace: press "Amend", edit the medicines, then "Sign amendment & print".

Behind the scenes — in simple terms. Every save takes a full copy of the prescription before the change and stores it in the history, so any older version can be rebuilt. The cached PDF is thrown away so the next print shows the new content.

Behind the scenes — technically. `PATCH /prescriptions/:id` → `PrescriptionService.update`:

1. Takes a JSON snapshot of the prescription and its medicines (`PrescriptionHistoryService.snapshot`).
2. In a transaction:
   - if `medicines` is sent, deletes all medicine rows and inserts the new list (replace-all);
   - merges the other fields;
   - when the prescription was `FINALIZED`, sets `status = AMENDED`, increments `version` and clears `pdf_url`.
3. Logs a history row `AMENDED` with the snapshot and optional `changeNotes`.

A draft edit is also logged as `AMENDED`, with the same version.

Rules and validations.

- Any status can be patched: DRAFT stays DRAFT, FINALIZED becomes AMENDED, AMENDED stays AMENDED.
- Medicine lines follow the rules in 32.3.2.
- The patient, encounter, appointment and doctor cannot be changed through this endpoint.

Who can do it. `prescriptions.create`, and the patient must be inside the caller's `patients.view` scope (otherwise "Prescription not found").

Data read and written. `prescriptions`, `prescription_medicines`, `prescription_history`.

Effects on other modules. An amended (unsigned) prescription shows as "Prescription is still a draft — not signed" in the consultation completion dialog.

Error cases. "Prescription not found" (404) → wrong organization or patient out of scope.

Technical references: `backend/src/prescriptions/services/prescription.service.ts` · `PrescriptionService.update`, `PrescriptionHistoryService.snapshot` · `web-app/src/features/prescriptions/components/PrescriptionBuilder.tsx`

32.3.6 Add, edit or remove one medicine line

Status: [Partial] — backend only. No screen calls these endpoints; both the builder and the workspace send the whole medicine list with `PATCH /prescriptions/:id`.

What it does. Changes a single medicine on a draft or amended prescription without resending the rest.

How the user does it. Not available in the web app.

Behind the scenes — in simple terms. The change is allowed only while the prescription is not signed, and each change is noted in the history.

Behind the scenes — technically.

- `POST /prescriptions/:id/medicines` adds a line at the end of the sort order.
- `PATCH /prescriptions/:id/medicines/:medicineId` updates a line.
- `DELETE /prescriptions/:id/medicines/:medicineId` removes a line.

Each runs `assertEditable` (status DRAFT or AMENDED) and logs history `AMENDED` with a note such as "Added medicine: …". The version does not change.

Rules and validations. Finalized prescriptions refuse with "Finalized prescriptions cannot be edited — amend instead". Medicine field rules as in 32.3.2.

Who can do it. `prescriptions.create` with patient scope.

Data read and written. `prescription_medicines`, `prescription_history`.

Effects on other modules. None.

Error cases. "Medicine line not found" (404); the edit-refusal message (400).

Technical references: `backend/src/prescriptions/controllers/prescription.controller.ts` · `PrescriptionService.addMedicine`, `updateMedicine`, `removeMedicine` · `web-app/src/features/prescriptions/hooks/usePrescriptionMedicines.ts` (unused hooks)

32.3.7 Duplicate a prescription

Status: [Partial] — backend only; no screen calls it. It also stores the caller's user id in the doctor column, which normally holds an employee id; whether that save succeeds was not verified.

What it does. Creates a new draft prescription for the same patient with the same layout, notes, vitals and medicines.

How the user does it. Not available in the web app. The consultation's "Repeat last" and "Use these medicines again" copy medicines in the browser instead.

Behind the scenes — in simple terms. It is a copy that starts again as a draft with a new number, without the appointment link.

Behind the scenes — technically. `POST /prescriptions/:id/duplicate` → `PrescriptionService.duplicate` reads the source and calls `create` with its fields and medicines. The encounter and appointment are not copied. `doctorId` is `req.user.uuid`.

Rules and validations. As for create.

Who can do it. `prescriptions.create` with patient scope.

Data read and written. `prescriptions`, `prescription_medicines`, `prescription_history`.

Effects on other modules. None.

Error cases. "Prescription not found" (404).

Technical references: `backend/src/prescriptions/services/prescription.service.ts` · `PrescriptionService.duplicate` · (no screen)

32.3.8 Change history of a prescription

Status: [Partial] — history rows are written on every create, edit, line change and signing, but no screen shows them, and the read endpoint looks broken.

What it does. Keeps an audit trail of a prescription: who changed it, when, the type of change, the version and a full copy of the prescription before an edit.

How the user does it. The builder has a "History" button, but it only reloads the same page with a `tab=history` parameter that nothing reads, so no history view appears.

Behind the scenes — in simple terms. Each change adds one row; rows are never edited.

Behind the scenes — technically.

- Writing: `PrescriptionHistoryService.log` stores `prescription_id`, `version`, `changed_by`, `change_type` (CREATED, AMENDED or FINALIZED are used; PRINTED and SHARED exist in the enum but are never written), `previous_data` (JSON snapshot) and `change_notes`.
- `changed_by` holds the employee id given at creation, but the user id for later changes.
- Reading: `GET /prescriptions/:id/history` → `listFor` asks the database layer to load a relation named `changedByEmployee`. The history entity does not declare that relation, and TypeORM normally refuses such a query, so the endpoint is expected to fail at run time (found by reading the code, not tested).

Rules and validations. Rows are append-only by code convention; the database does not enforce it.

Who can do it. Written by the system; reading needs `patients.clinical.view` with patient scope.

Data read and written. `prescription_history`.

Effects on other modules. None.

Error cases. See the read problem described in the technical paragraph.

Technical references: `backend/src/prescriptions/services/prescription-history.service.ts` · `PrescriptionHistoryService.log`, `listFor` · `web-app/src/features/prescriptions/hooks/usePrescriptionHistory.ts` (unused)

32.3.9 Print a prescription as PDF, with Urdu

Status: [Implemented]

What it does. Produces a one-page PDF of the prescription on the clinic's layout. Urdu text (medicine names, notes, instructions, clinic name, disclaimer) is printed in proper Nastaliq script, right to left, with numbers and English words kept the right way round.

How the user does it.

- Workspace: "Sign & print" or "Print" opens an in-app PDF preview with a "Print" button.
- Builder or detail page: "Print" opens the PDF in a new browser tab.

Behind the scenes — in simple terms. The PDF is built fresh each time it is requested, from the saved prescription and its print layout. Any staff member who can read the prescription can print it, including drafts.

Figure 32.8 — Printed prescription PDF (screenshot pending).

Figure 32.8 stands for the generated PDF. From the PDF code, the page is laid out from top to bottom as follows:

- **Header.** "Dr. " on the left in the layout's accent colour, with the qualification under it. The clinic name is on the right (the layout's clinic name, else the location name, else "Clinic"), the Urdu clinic name under it, then the doctor phone. A coloured line closes the header.
- **Patient bar.** A grey bar with Name, Age, Gender, Reg#, Date and the RX number.
- **Vitals row.** BP, Pulse, Temp, SpO2, Wt and the other vitals present, unless the layout hides vitals.
- **Clinical text.** Labelled blocks "Chief Complaint" and "Diagnosis".
- **Medicines.** A bold "Rx", then numbered medicines. Each shows the name in bold; under it dose, frequency, "× duration" and the meal relation in brackets; then a bullet with the special instruction. When a medicine has an Urdu name or Urdu note, they are printed in a right-hand column taking five-twelfths of the width.
- **Instructions.** Labelled blocks "Instructions", the Urdu instructions block (right-aligned, headed in Urdu) and "Further Plan".
- **Signature.** In the bottom right: the signature image, a line and the doctor's name.
- **Footer.** Address, timings, phone and website, with the English small print on the left and the Urdu small print on the right.
- **Watermark.** An optional watermark text drawn diagonally across the page.

Behind the scenes — technically.

- `GET /prescriptions/:id/pdf` → `PrescriptionPdfService.generatePdfBuffer` loads the prescription with patient, doctor and layout, falling back to the location or organization default layout.
- It draws one page with pdf-lib, sized from the layout (A4 595×842 pt, A5, Letter, or custom millimetres). Margins and colours come from the layout's `styling`, with defaults of 15 mm top and bottom, 12 mm left and right, and a teal accent.
- Latin text uses the standard Helvetica fonts.
- Arabic-script runs go through `PdfTextWriter`:
  - text is split into direction runs by a small bidirectional algorithm (`bidi.ts`);
  - each Urdu run is shaped with fontkit in Noto Nastaliq Urdu (from the `@fontsource/noto-nastaliq-urdu` package), giving joined letter forms and right-to-left order;
  - each glyph is drawn as a vector outline, about 15% larger than the Latin text.
- Urdu printed this way cannot be selected or searched in the PDF; the code accepts that trade-off.
- The signature image is fetched (a `data:` URL is decoded, an http(s) URL is downloaded) and embedded as PNG or JPEG; if embedding fails, the page is printed without it.
- The response is streamed inline with `Content-Type: application/pdf`.

Rules and validations.

- The PDF can be generated for any status, including DRAFT.
- The date printed is the creation date, formatted day/month/year by the server.

Who can do it. `patients.clinical.view` with patient scope and the feature `patients.prescriptions`. Doctor, Assistant and Admin templates.

Data read and written. Reads `prescriptions`, `prescription_medicines`, `patients`, `employees`, `prescription_templates`.

Effects on other modules. None.

Error cases.

- "Prescription not found" (404).
- Web toasts "Failed to load PDF" (builder) or "Could not open the prescription PDF" (workspace).

Technical references: `backend/src/prescriptions/services/prescription-pdf.service.ts` · `PrescriptionPdfService.generatePdfBuffer`, `PdfTextWriter` (`backend/src/prescriptions/pdf/pdf-text.ts`, `bidi.ts`) · `web-app/src/features/consultation/components/PdfPreviewDialog.tsx`

32.3.10 Print layouts (prescription templates)

Status: [Implemented]

What it does. A print layout defines how printed prescriptions look: paper size, accent colour, letterhead, footer and which items to show. One layout can be the default. In code a print layout is a "prescription template"; the screens call it a print layout.

How the user does it.

1. Open "Prescriptions" → "Library" → "Print layouts".
2. Press "New layout", or the edit icon on a row.
3. Fill the sheet (Figure 32.6) and watch the live preview. Switch on "Use as the default layout" if wanted.
4. Save. To retire a layout, use "Deactivate" and confirm "Deactivate this layout?".

5. In the builder, pick a layout in the toolbar selector for one prescription.

Behind the scenes — in simple terms. Layouts can belong to one location or to the whole organization. When a prescription is created without a chosen layout, it takes the location's default, else the organization's default. Layouts made from the web screen are always organization-wide.

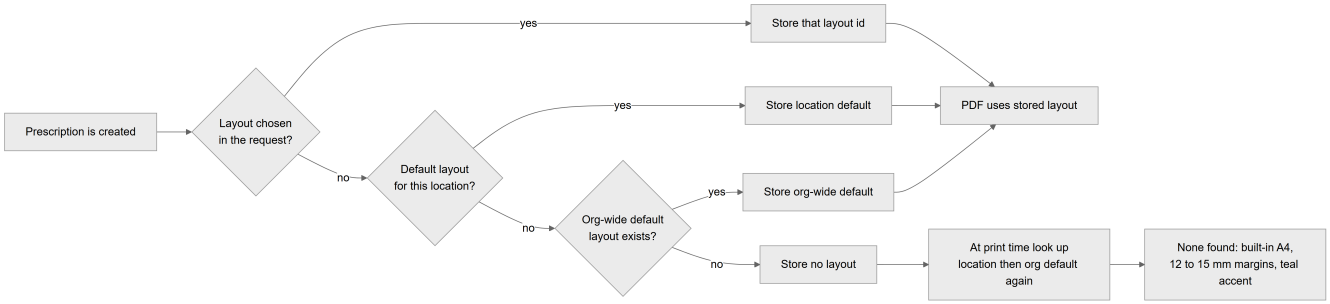

Figure 32.9 — How the print layout of a prescription is chosen.

Figure 32.9 is a left-to-right flowchart. It starts at "Prescription is created" and asks "Layout chosen in the request?". If yes, the next box is "Store that layout id". If no, it asks "Default layout for this location?"; yes leads to "Store location default". No leads to "Org-wide default layout exists?"; yes leads to "Store org-wide default" and no leads to "Store no layout". The three "store" boxes lead to "PDF uses stored layout". "Store no layout" leads to "At print time look up location then org default again", and from there to "None found: built-in A4, 12 to 15 mm margins, teal accent".

Behind the scenes — technically.

- `POST /prescription-templates` : when `isDefault` is set, it first clears the default flag of the other layouts with the same location (or the other organization-wide layouts). It then merges the given `header` , `footer` and `styling` objects over built-in defaults (Helvetica, Noto Nastaliq Urdu, show vitals, show Urdu medicine names, margins and so on).
- `PATCH /prescription-templates/:id` merges the JSON objects field by field and moves the default flag the same way.
- `DELETE /prescription-templates/:id` sets `is_active = false` and `is_default = false` .
- `GET /prescription-templates?locationId=` lists active layouts for that location plus organization-wide ones, default first.
- `GET /prescription-templates/default?locationId=` resolves location default, then organization default.
- `GET /prescription-templates/:id` reads one layout.

Rules and validations.

- At most one default per location and one organization-wide default, kept by the service rather than a database constraint.
- A deactivated layout already stored on a prescription is still used for that prescription's PDF.
- The PDF ignores `orientation` (always portrait), the logo flag and the registration number, even though the web preview shows the registration number.

Who can do it. Reading: any member (`@AnyMember`). Writing: `prescriptions.library.manage` (feature `patients.prescriptions`), held by the Admin template. Route: any of `prescriptions.create` or `prescriptions.library.manage` .

Data read and written. `prescription_templates` .

Effects on other modules. Every printed prescription.

Error cases. "Prescription template not found" (404).

Technical references: `backend/src/prescriptions/services/prescription-template.service.ts` · `PrescriptionTemplateService.getDefaultForLocation` · `web-app/src/features/prescriptions/pages/PrintLayoutsPage.tsx` , `components/library/LayoutSheet.tsx`

32.3.11 Doctor signatures

Status: [Implemented] — saving and reading one's own signature work from the screens. Deleting a signature and reading another doctor's signature exist only in the backend.

What it does. Each doctor keeps one saved signature image, which is placed on every prescription they sign unless they draw a new one at signing time.

How the user does it. The first time a doctor signs, they draw in the signature pad; the drawing is saved automatically. Drawing a new one later replaces it.

Behind the scenes — in simple terms. The signature is stored once per login account. It can be an image link or the drawing itself embedded as data.

Behind the scenes — technically.

- `POST /doctor-signatures` → `DoctorSignatureService.save` upserts the row keyed by the caller's **user id** (stored in the column `employee_id` ; the entity comment says this becomes a real employee link later). It sets `signature_image_url` , optional `signature_data` and `is_active = true` .
- `GET /doctor-signatures/my` returns the caller's active signature.
- `DELETE /doctor-signatures` removes it (no screen calls this).
- `GET /doctor-signatures/doctor/:employeeId` returns a given owner's active signature (no screen calls this).

Rules and validations.

- One signature per owner (unique index on `employee_id`).
- `signatureImageUrl` is required and must not be empty.

Who can do it. Save, read own and delete: `prescriptions.create` . Read by id: any member (`@AnyMember`). Feature `patients.prescriptions` .

Data read and written. `doctor_signatures` .

Effects on other modules. Finalize falls back to the saved signature when the request has none.

Error cases. "Signature not found" (404) on delete.

Technical references: `backend/src/prescriptions/services/doctor-signature.service.ts` · `DoctorSignatureService.save` · `web-app/src/features/prescriptions/hooks/useDoctorSignature.ts`

32.3.12 Medicine directory

Status: [Implemented]

What it does. The organization's drug catalogue. Doctors search it while writing; picking a medicine fills its usual dose, frequency, duration, meal relation and Urdu name. A new clinic can load a starter list with one button.

How the user does it.

1. Open "Prescriptions" → "Library" → "Medicines".
2. With an empty directory, press "Add common medicines" to load the starter list, or "Add medicine".
3. Fill the "Add medicine" sheet: name, generic name, Urdu name, type, strength, and the defaults.
4. Use the edit icon to change a medicine, or "Deactivate" (with confirmation) to hide it from search.

5. Doctors search by typing at least two letters in a medicine box in the builder or the workspace.

Behind the scenes — in simple terms. Medicines are never deleted, only deactivated, so old prescriptions keep their link. Each prescription stores its own copy of the medicine name and details, so renaming a directory entry does not change prescriptions already written. The starter list adds ten common medicines (for example paracetamol, ibuprofen and amoxicillin brands) only when the directory is completely empty.

Behind the scenes — technically.

- `GET /medicine-directory` lists active medicines by name.
- `GET /medicine-directory/search?query=&type=&limit=` does a case-insensitive match on name, generic name or Urdu name among active rows. Limit defaults to 10, maximum 50.
- `POST /medicine-directory` creates; `PATCH /medicine-directory/:id` updates; `DELETE /medicine-directory/:id` sets `is_active = false`.
- `POST /medicine-directory/seed-common` → `seedCommonMedicines` returns `{ seeded, skipped }`. It is skipped when any row (active or not) exists for the organization.
- The Medicines page filters the full list in the browser with a 300 ms debounced search and a type filter.

Rules and validations.

- Name is required (at most 200 characters).
- Type must be one of the 15 medicine types (Tablet, Capsule, Syrup, Injection, Cream, Drops, Inhaler, Sachet, Suppository, Ointment, Gel, Patch, Spray, Powder, Other).
- Default meal relation from the meal-relation list.
- Names are not unique; duplicates are possible.

Who can do it. Reading and search: any member (`@AnyMember`). Create, edit, deactivate and seed: `prescriptions.library.manage` (Admin template). Feature `patients.prescriptions`.

Data read and written. `medicine_directory`.

Effects on other modules. Prescriptions and presets keep a nullable link to the directory row (`medicine_directory_id`).

Error cases. "Medicine not found" (404). The seed answers "Directory already populated" when skipped.

Technical references: `backend/src/prescriptions/services/medicine-directory.service.ts` · `MedicineDirectoryService.search`, `seedCommonMedicines` · `web-app/src/features/prescriptions/pages/MedicinesPage.tsx`

32.3.13 Prescription presets

Status: [Implemented]

What it does. A **prescription preset** is a named, reusable set of medicines with instructions, filed under a category such as "Post extraction" or "Pregnancy safe". In the backend it is called a "treatment plan template" (`/treatment-plan-templates`, table `treatment_plan_templates`). Despite the name, it has nothing to do with clinical treatment plans or the **plan templates** at `/plan-templates`, which describe planned dental work (Chapter 33).

How the user does it.

1. Open "Prescriptions" → "Library" (the Presets tab opens first).
2. Press "New preset". In the sheet, enter Name, Category and Description, add medicines with Dose, How often, With food and For how long, and fill the instructions (English, Urdu, Further plan).

3. Save. Edit or deactivate from the row icons.
4. Use presets from the builder ("Presets") or the workspace ("Use a preset"), as described in 32.3.3.

Behind the scenes — in simple terms. A preset belongs to the whole organization. Deactivating hides it from the lists but keeps it.

Behind the scenes — technically.

- `POST /treatment-plan-templates` creates the preset with its medicines in one save (cascade). `specialty_tags` defaults to `["general"]`.
- `PATCH /treatment-plan-templates/:id` replaces all medicines in a transaction when a medicine list is sent; otherwise it updates fields only.
- `DELETE /treatment-plan-templates/:id` sets `is_active = false`.
- `GET /treatment-plan-templates?category=` lists active presets; `GET /treatment-plan-templates/:id` reads one.

Rules and validations.

- Name required (at most 200 characters).
- At least one medicine ("Template must include at least one medicine").
- Category must be one of the `PresetCategory` values (about forty, from GENERAL to OTHER).
- Medicine line rules as for prescriptions.

Who can do it. Reading: any member. Writing: `prescriptions.library.manage` (Admin template). Feature `patients.prescriptions`.

Data read and written. `treatment_plan_templates`, `treatment_plan_template_medicines`.

Effects on other modules. Prescriptions created from a preset store its id in `treatment_plan_template_id`.

Error cases. "Treatment plan template not found" (404); validation message for an empty medicine list (400).

Technical references: `backend/src/prescriptions/services/treatment-plan-template.service.ts` · `TreatmentPlanTemplateService`, `PresetCategory` · `web-app/src/features/prescriptions/pages/PresetsPage.tsx`, `components/library/PresetSheet.tsx`

32.3.14 Prescriptions of one patient, one visit or one appointment

Status: [Partial] — the patient and visit lookups are used by screens; the appointment lookup is backend only.

What it does. Returns the prescriptions written for a patient, for a consultation (encounter) or for a booked appointment.

How the user does it.

- The patient record's prescriptions tab and the consultation's "Earlier prescriptions" use the patient lookup.
- The consultation's Prescription panel uses the visit lookup to find today's prescription.
- No screen uses the appointment lookup.

Behind the scenes — in simple terms. All three follow the patient visibility rules.

Behind the scenes — technically.

- `GET /prescriptions/patient/:patientId?page=&limit=` is guarded by `@PatientInScope`. It is paged (default 20), with doctor and medicines, newest first.

- `GET /prescriptions/encounter/:encounterId` and `GET /prescriptions/appointment/:appointmentId` drop rows whose patient is outside the caller's scope (`visibleOnly`).
- `GET /prescriptions/:id` reads one prescription with patient, appointment, doctor and both template links, after `ensureVisible` .

Rules and validations. Organization filter on all; location filter is not applied to these lookups (unlike the main list).

Who can do it. `patients.clinical.view` plus the feature.

Data read and written. Reads `prescriptions` , `prescription_medicines` .

Effects on other modules. Feeds the consultation workspace (Chapter 31) and the patient record (Chapter 28).

Error cases. "Prescription not found" (404) for a single read out of scope.

Technical references: `backend/src/prescriptions/controllers/prescription.controller.ts` · `PrescriptionService.listByPatient` , `listByEncounter` , `listByAppointment` · `web-app/src/features/prescriptions/hooks/usePrescriptions.ts`

32.3.15 Legacy quick prescriptions (`patient_prescriptions`)

Status: [Implemented] — kept for older data; still readable and writable from the patient record.

What it does. Before the current module, prescriptions were stored as a simple record on the patient, with the medicines in one JSON list. That older table, `patient_prescriptions` , still exists next to the new `prescriptions` table. The patient's prescriptions tab shows it in a collapsed section "Legacy quick-Rx history" ("Older prescriptions saved in the simple form. Kept for reference."), with a button to add one in the old simple form.

How the user does it.

1. Open a patient, then the prescriptions tab.
2. Expand "Legacy quick-Rx history" to see old entries (with "Printed (n)" counters).
3. Users with `prescriptions.create` can add one through the old dialog.

Behind the scenes — in simple terms. The two tables are separate: legacy entries do not appear in the Prescriptions list, the consultation or the PDF printing of the new module, and the new prescriptions do not appear in the legacy list. The old entries print through the browser, not through the PDF service.

Behind the scenes — technically.

- `GET /patients/:id/prescriptions` and `POST /patients/:id/prescriptions` are in the patients controller (Chapter 28), with patient scope.
- Legacy numbers are `RX-{5 digits}` counted per organization, unique per organization.
- Legacy writes are audited through the patients audit trail.
- Columns include `prescribed_by` , `appointment_id` , `prescription_date` , `medications` (JSON), `special_instructions` , `follow_up_date` , `is_printed` , `printed_at` and `print_count` .

Rules and validations. Read: `patients.clinical.view` and feature `patients.prescriptions` . Create: `prescriptions.create` .

Who can do it. As in the rules; Doctor writes, Doctor, Assistant and Admin read.

Data read and written. `patient_prescriptions` .

Effects on other modules. None on the new module.

Error cases. As for patient sub-records (Chapter 28).

Technical references: `backend/src/patients/patients.controller.ts` · `PatientPrescription` · `web-app/src/features/patients/components/tabs/ClinicalTab.tsx`

32.4 Entities and data model

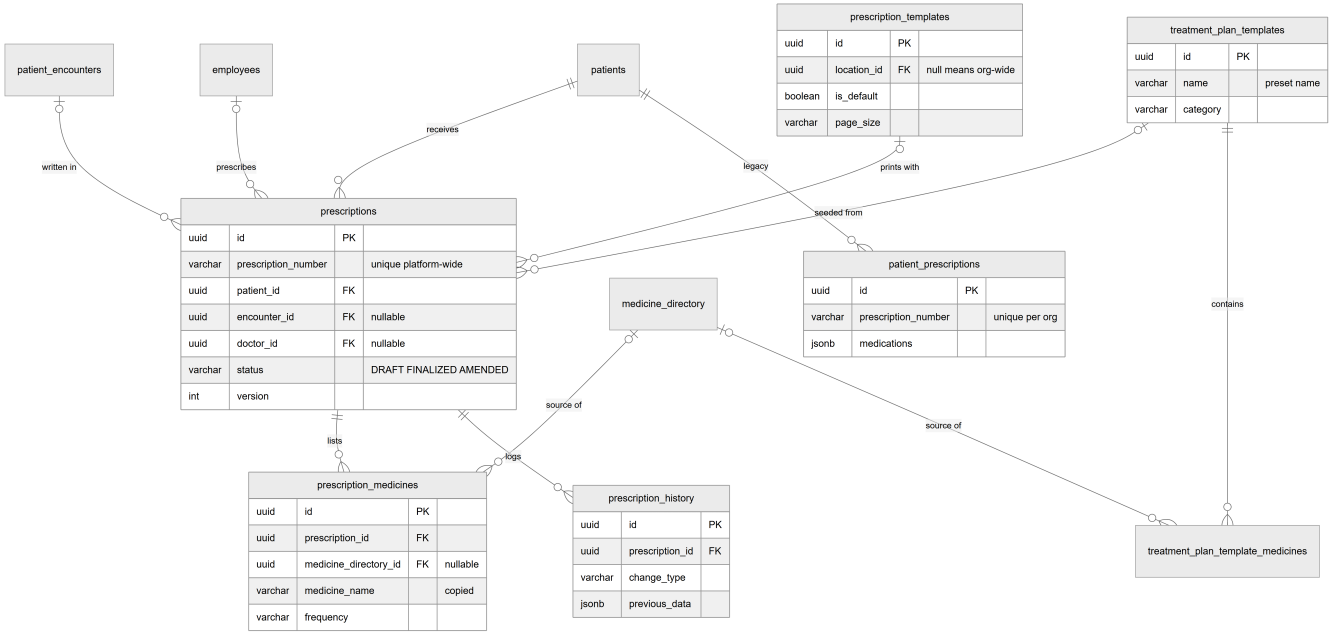

Figure 32.10 — Entity-relationship diagram of prescriptions and the prescription library.

Figure 32.10 shows `prescriptions` with its key columns:

- `id` (primary key) and `prescription_number` (marked "unique platform-wide");
- `patient_id`, plus the nullable foreign keys `encounter_id` and `doctor_id`;
- `status` (DRAFT, FINALIZED, AMENDED) and `version`.

Around it:

- **Top of the figure.** `patient_encounters` ("written in"), `employees` ("prescribes"), `patients` ("receives"), `prescription_templates` ("prints with": `location_id` where null means organization-wide, `is_default`, `page_size`) and `treatment_plan_templates` ("seeded from": `name` as preset name and `category`).
- **Bottom of the figure.** `prescription_medicines` ("lists": `prescription_id`, nullable `medicine_directory_id`, `medicine_name` marked "copied", `frequency`) and `prescription_history` ("logs": `prescription_id`, `change_type`, `previous_data`).
- **Library tables.** `medicine_directory` is a "source of" both `prescription_medicines` and `treatment_plan_template_medicines`, and `treatment_plan_templates` "contains" `treatment_plan_template_medicines`.
- **Legacy.** `patients` also links to the separate `patient_prescriptions` table ("legacy", with `prescription_number` unique per organization and `medications` as JSON).

The relationships in words:

- **Patient → prescriptions.** Each prescription belongs to exactly one patient, organization and location; deleting any of them deletes the prescription (ON DELETE CASCADE).
- **Encounter, appointment, doctor → prescription.** Optional links. Deleting the encounter, appointment or employee keeps the prescription and clears the link (ON DELETE SET NULL). The appointment link is not drawn in the figure.

- **Print layout and preset → prescription.** Optional links to `prescription_templates` and `treatment_plan_templates`; deleting either clears the link (SET NULL). In practice layouts and presets are only deactivated, never deleted.
- **Prescription → medicines.** A prescription has many medicine lines; deleting the prescription deletes its lines (CASCADE). Lines copy the medicine's name and details, and keep an optional link to `medicine_directory` that is cleared if the directory row is deleted (SET NULL).
- **Prescription → history.** Many history rows per prescription; deleted with it (CASCADE). `changed_by` is a plain id with no foreign key.
- **Preset → preset medicines.** `treatment_plan_template_medicines.template_id` is required; deleting the preset deletes its medicines (CASCADE); the optional directory link is SET NULL.
- **Doctor signatures** (not drawn). `doctor_signatures` has one row per owner (unique `employee_id`, which currently holds a user id, without a foreign key) and a foreign key to the organization (CASCADE).
- **Legacy table.** `patient_prescriptions` belongs to a patient, location and organization (CASCADE). Its `prescribed_by` and `appointment_id` are plain ids without foreign keys. It has no link to the new tables.

Indexes: `prescriptions` on (organization, location), patient, appointment, encounter, doctor and a unique index on `prescription_number`; `medicine_directory` on (organization, is_active) and (organization, name); `prescription_templates` on (organization, is_active) and (location, is_default); `treatment_plan_templates` on (organization, is_active) and (organization, category); `prescription_history` on (prescription, created_at). Chapter 19 covers these tables in the full data model.

32.5 Status lifecycles

A prescription has one status column with three values: `DRAFT`, `FINALIZED` and `AMENDED`.

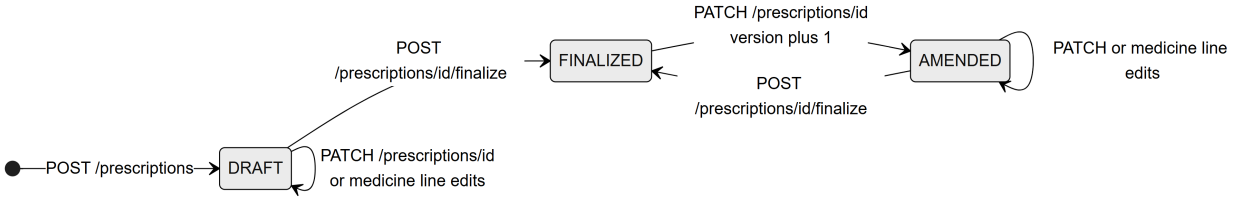

Figure 32.11 — Prescription status lifecycle.

Figure 32.11 is a left-to-right state diagram:

- From the start point, "POST /prescriptions" leads to DRAFT.
- DRAFT has a loop "PATCH /prescriptions/id or medicine line edits".
- "POST /prescriptions/id/finalize" leads from DRAFT to FINALIZED.
- "PATCH /prescriptions/id, version plus 1" leads from FINALIZED to AMENDED.
- AMENDED has a loop "PATCH or medicine line edits".
- "POST /prescriptions/id/finalize" leads from AMENDED back to FINALIZED.

There is no end state: a prescription can always be amended again.

Walking through each state:

- **DRAFT.** Created by the builder or the consultation panel, version 1. It can be edited freely, printed (the PDF works for drafts) and finalized once it has at least one medicine.
- **FINALIZED.** Signed: `signed_at` and the signature are set and a PDF may be stored. Single-line medicine edits are refused, but a full `PATCH` is accepted and turns it into AMENDED.

- **AMENDED.** A signed prescription that has been edited and not yet signed again. Its version is one higher, the stored PDF address is cleared, and the state before the edit is saved in the history. Signing again returns it to FINALIZED with the same version.

The enum of history change types also contains PRINTED and SHARED, but nothing writes them; printing does not change the status or the history. There is no cancelled or deleted status, and no endpoint deletes a prescription.

32.6 Connections to other modules

- **Consultation (Chapter 31).** The workspace's Prescription panel writes the prescription linked to the encounter, copies complaint, diagnosis, advice, follow-up and same-day vitals onto it, and signs and prints it. The completion dialog reports whether it is signed.
- **Patients (Chapter 28).** The patient's prescriptions tab lists new prescriptions, links to the builder with the patient pre-selected, and shows the legacy quick prescriptions. Patient visibility (`patients.view` scope) limits every read and write.
- **Treatment plans (Chapter 33).** Unrelated despite the backend name "treatment plan templates" for presets; real plan templates live at `/plan-templates`.
- **Employees (Chapter 37).** The prescribing doctor is an employee; the builder's doctor list comes from the employees module.
- **Features and access (Chapters 10 and 57).** Feature `patients.prescriptions`; permissions `prescriptions.create`, `prescriptions.library.manage`, `patients.clinical.view`.
- **File storage.** Finalized PDFs are uploaded through the Platform's shared storage provider (the same one used for patient files) when it is configured.

32.7 Limitations and known gaps

- **Numbering can collide.**
  - Numbers are counted per organization (`RX-{year}-{5 digits}`), but the database requires the number to be unique across the whole Platform. Two organizations will produce the same first number, and the second insert would be refused by the database (found by reading the code, not tested).
  - The counter reads the latest prescription, so two simultaneous creates in one organization can also clash.
  - The counter does not restart with a new year.
- **History is incomplete.** History is written but cannot be viewed: the "History" button does nothing visible, and the read endpoint requests a relation the entity does not have.
- **Backend-only endpoints.** Duplicate, single-line medicine edits, signature delete, signature read by doctor and the by-appointment lookup have no screen.
- **PDF limits.**
  - The PDF is a single page; no page-break logic exists, so a very long prescription can run off the page.
  - Orientation, logo and registration number from the layout are not printed.
  - Urdu text in the PDF is drawn as shapes and cannot be selected or searched.
  - The PDF date is the creation date formatted by the server, and drafts can be printed.
- **Clock.** "This month" on the list uses the server's clock rather than the location's timezone.
- **Layouts from the screen.** The web screen creates organization-wide layouts only; per-location layouts exist in the API (`locationId`) but have no field in the sheet.

- **Signatures.** Signatures are stored per login account, not per employee, so one person working in two organizations has a single signature row.
- **Two prescription tables.** The legacy `patient_prescriptions` table still accepts new entries, which keeps two separate prescription histories per patient.
- **Ordering.** Medicine order is kept, but there is no way to reorder lines except removing and re-adding them.

32.8 Owner's FAQ

Can an Admin write prescriptions? Not with the default templates. The Admin role is given every permission except a few doctor-only ones, and `prescriptions.create` is one of them. Admins can read, print and manage the library. An owner, or a custom role with `prescriptions.create`, can write.

Can a receptionist print a prescription? Not by default. Printing needs `patients.clinical.view`, which the Receptionist template does not hold. The Assistant template does, so an assistant can open the list by URL and print, even without a sidebar entry.

What happens if the clinic switches off the Prescriptions feature? The sidebar entry disappears (the permissions tied to the feature are dropped), every prescription endpoint answers with a feature error, and the legacy list read is blocked too. Existing data stays in the database.

Is a signed prescription locked? It cannot be edited line by line, but it can be amended. Amending raises the version, marks it AMENDED, and saves the previous version in `prescription_history`. It must be signed again to become final.

If we rename a medicine in the directory, do old prescriptions change? No. Each prescription line stores its own copy of the medicine name and details.

Where is the signature stored? In `doctor_signatures`, one row per login account. The image is either a link to file storage or the drawing embedded as a data URL. A copy of the image reference is also stored on each prescription at signing time.

Where are PDFs kept? The PDF is built on demand every time someone prints. At signing, the Platform also tries to upload a copy to the clinic's file storage and keeps its address in `pdf_url`; the web app does not use that stored copy.

How does Urdu print correctly? The server shapes Urdu with the Noto Nastaliq Urdu font and draws each letter as a vector shape in the right order, because the PDF library cannot join Urdu letters by itself. Numbers and English words inside Urdu text keep left-to-right order.

What is the difference between a preset and a plan template? A preset (backend name "treatment plan template") is a ready-made list of medicines for a prescription. A plan template (`/plan-templates`) describes planned dental work and its costs. Chapter 33 covers plan templates.

Are old prescriptions from the first version lost? No. They stay in `patient_prescriptions` and appear under "Legacy quick-Rx history" on the patient's prescriptions tab.

Chapter 33 — Treatment Plans

A **treatment plan** is the planned work for one patient, such as a root canal spread over several sessions, with its steps, estimated cost and progress. A **plan template** is a clinic's ready-made list of steps (Root Canal, Implant, Braces …) that staff reuse. This chapter is for the product owner who needs to know what the Platform already does with plans, what exists only in the backend, and how plans connect to treatments and billing.

33.1 What this module is for

The module has two halves that are built to very different depths:

- **Plan templates** are complete end to end. The clinic keeps its own list of templates on the **Treatment Plans** page (`/plan-templates`). Each template has a name, a category (endodontics, orthodontics …), a description and an ordered list of procedures with sessions and an optional usual fee. A new clinic receives twelve ready-made templates the first time anyone opens the list. Staff use a template when they record a treatment: picking it fills the treatment name and cost, and the treatment is tagged with the template.
- **Patient treatment plans** (one plan per course of treatment, with items) are stored and audited by the backend, and Billing can read a plan's money position or turn its items into charges. No web screen shows, creates or edits a patient plan today, and nothing creates a patient plan from a template.

Who uses it (system roles, by default): the Admin manages templates (`plans.templates.manage` is in the Admin template only) and can record treatments from templates. The Doctor holds `plans.manage` , so a doctor could create and update patient plans through the API, but has no screen for it. Any member of the organization can read the template list when the feature is on.

Sidebar: the entry **Treatment Plans** (route `/plan-templates` , sidebar key `nav.plan_templates`) sits in the "Clinic" section of the default sidebar and is shown only to holders of `plans.templates.manage` . The page itself is titled "Treatment Plan Templates". There is no sidebar entry or route for patient plans.

Feature key: everything in this chapter belongs to the feature `patients.treatment_plans` ("Treatment Plans — multi-step treatment plans with cost estimates"), which is enabled by default in the feature catalog. When the organization turns it off, the template page, the template list, the plan endpoints and the permissions `plans.manage` and `plans.templates.manage` all stop working.

Plan templates versus prescription presets. The prescriptions module has a table called `treatment_plan_templates` . Despite the name, those rows are not plan templates: each one is a set of **medicines** and instructions for a prescription (for example "Root Canal Aftercare"), shown in the web app as **Prescription Presets** at `/prescription-presets` and used by the prescription builder. Figure 33.1 shows the two side by side.

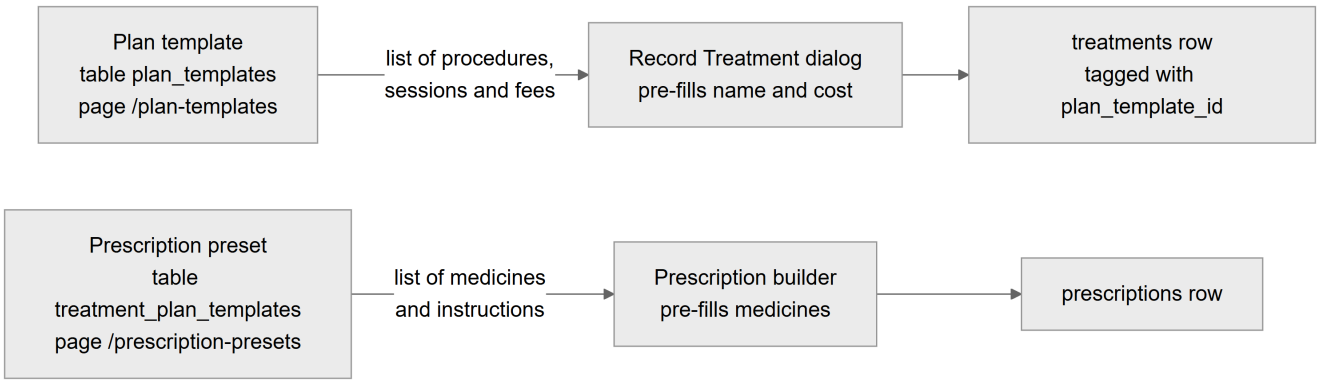

Figure 33.1 — The two kinds of "template" and where each one is used.

The top row starts at "Plan template" (table `plan_templates`, page `/plan-templates`). An arrow labelled "list of procedures, sessions and fees" leads to the "Record Treatment dialog", which pre-fills the treatment name and cost, and an arrow from there leads to a "treatments row tagged with plan_template_id". The bottom row starts at "Prescription preset" (table `treatment_plan_templates`, page `/prescription-presets`). An arrow labelled "list of medicines and instructions" leads to the "Prescription builder", which pre-fills the medicines, and then to a "prescriptions row". The two rows never touch: a plan template has no medicines and a prescription preset has no procedures. The Prescriptions chapter (Chapter 32) covers presets.

33.2 Screens

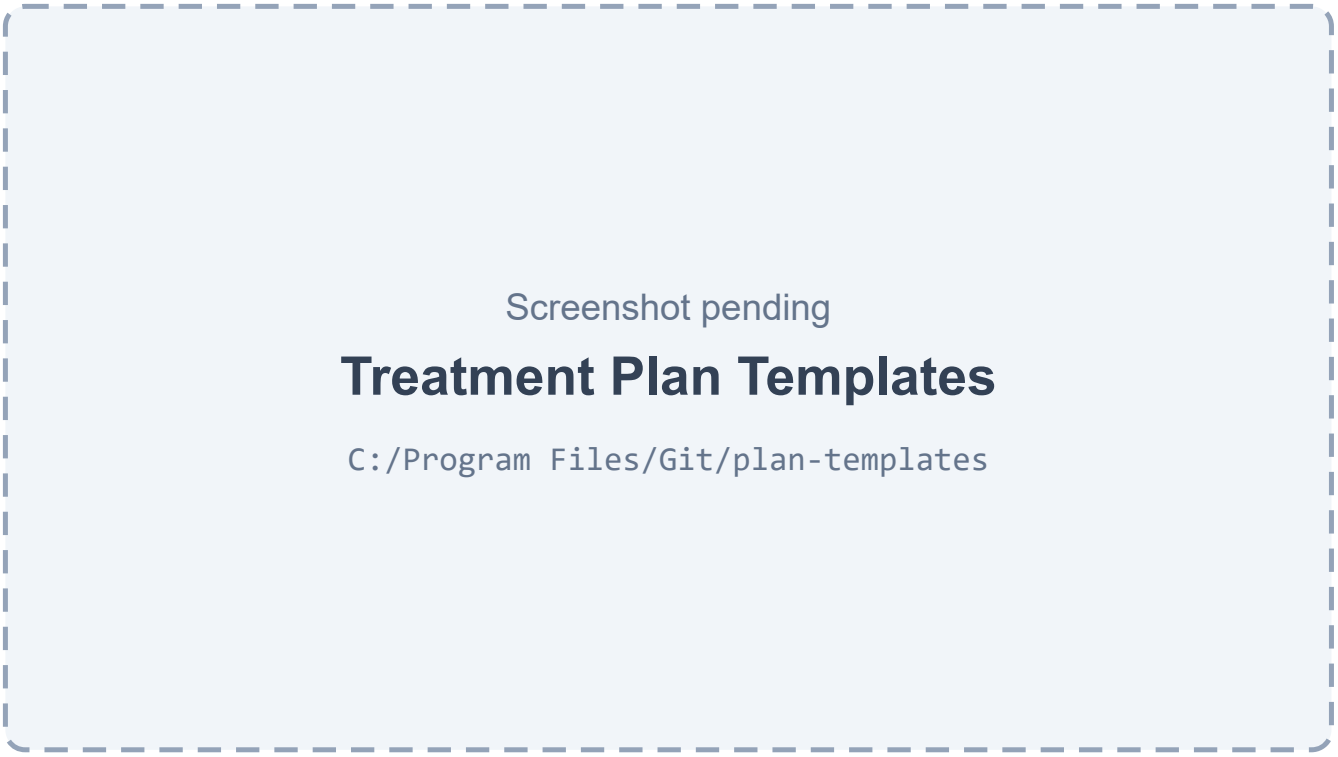

Figure 33.2 — Treatment Plan Templates page (screenshot pending).

The page has the title "Treatment Plan Templates" and the subtitle "Ready-made plans like Root Canal or Implant. Doctors pick one when starting a plan for a patient." A filled **New template** button sits top right. The filter row has a search box ("Search templates or procedures…", debounced by 300 ms) that matches template names and procedure names, a category select ("All categories" plus every category), an active-filter count badge and **Clear all filters**. Templates appear as cards in a grid. Each card shows the template name and category, the description, the numbered list of procedures, a line such as "6 procedures · 8 visits · PKR 45,000" (the fee total adds up the procedures that have a fee and is left out when none has one), an Active/Hidden switch with an explanation, and two icon buttons with tooltips: "Edit template" and "Delete template". Hidden templates are shown dimmed. When the clinic has no template the page shows "No treatment plan templates" with a **New template** button; when filters hide everything it shows a separate no-results state. There is no table view on this page.

Screenshot pending

Template editor

C:/Program Files/Git/plan-templates

Figure 33.3 — Template editor dialog (screenshot pending).

New template and "Edit template" open a dialog titled "New treatment plan template" or "Edit template", with the note "The procedures are copied into a patient's plan when a doctor uses this template." The form has a Name field (placeholder "e.g. Root Canal Treatment (RCT)"), a Category select, a Description field ("What this plan covers") and a "Procedures, in order" list. Each procedure row has a Procedure name, a Sessions number ("Sessions (visits)"), a "Fee (Rs.)" number and three icon buttons: "Move up", "Move down" and "Remove procedure". **Add procedure** appends a row. The footer has **Cancel** and **Create template** or **Save changes**; the save button is disabled while the form is invalid or saving.

Screenshot pending

Record Treatment dialog

C:/Program Files/Git/patients/$patientId

Figure 33.4 — Record Treatment dialog on the patient's Treatments tab (screenshot pending).

The **Add Treatment** button on a patient's Treatments tab (and "Add treatment" inside a visit's details) opens "Record Treatment" (or "Add Treatment to Visit"). Its first row holds two selects: "Treatment plan" (No plan, or any active plan template) and "Procedure" ("Whole plan" or one procedure of the chosen template; disabled until a template is chosen). The following fields are Treatment Name (required), "Cost (Rs.)", Date (required), an optional "Link to Visit" select listing the patient's visits, Notes, **Cancel** and **Save Treatment**.

No screen exists for patient treatment plans, so no figure is shown for them.

33.3 Features

33.3.1 Browse and filter plan templates

Status: [Implemented]

What it does. Lists all of the clinic's plan templates, active and hidden, with search and a category filter.

How the user does it.

1. Open **Treatment Plans** in the sidebar.
2. Type in "Search templates or procedures…" or pick a category; press **Clear all filters** to reset.

Behind the scenes — in simple terms. The list is loaded once and filtered in the browser. Fees are removed from the answer for anyone who may not see prices.

Behind the scenes — technically. `GET /plan-templates?includeInactive=true` → `PlanTemplatesService.findAll`, ordered by category then name. Because item fees live inside a JSON column, the controller deletes `estimatedCost` from each item itself when the caller lacks `patients.price.view`.

Rules and validations.

- Without `includeInactive=true` only active templates are returned (the Record Treatment dialog uses this).
- If the organization has no template rows at all, the twelve defaults are inserted first (Root Canal, Scaling & Polishing, Dental Implant and nine more).

Who can do it. The endpoint is open to any member (`@AnyMember`) with the feature `patients.treatment_plans` active. The page requires `plans.templates.manage` (route `staticData.can`). Default page holder: Admin.

Data read and written. Reads `plan_templates` (writes it on first read for a new clinic).

Effects on other modules. The same list feeds the Record Treatment dialog.

Error cases. Error state with retry → request failed; feature-off error when the organization switched the feature off.

Technical references: `backend/src/plan-templates/plan-templates.controller.ts` · `PlanTemplatesController.findAll`, `PlanTemplatesService.findAll` · `web-app/src/features/plan-templates/pages/PlanTemplatesPage.tsx`

33.3.2 Create a plan template

Status: [Implemented]

What it does. Adds a new reusable plan with ordered procedures.

How the user does it.

1. Press **New template**.

2. Enter Name, Category and Description; add procedures with **Add procedure**, set sessions and fees, reorder with "Move up" / "Move down".
3. Press **Create template**. A "Template created" toast confirms.

Behind the scenes — in simple terms. The template and its procedures are stored as one row; the procedures are kept in order inside it.

Behind the scenes — technically. `POST /plan-templates` → `PlanTemplatesService.create`. Names and notes are trimmed; empty notes become null; category defaults to `GENERAL`; `createdBy` is the user id. The web form drops rows with an empty procedure name before sending. No audit row is written.

Rules and validations.

- Name required, at most 200 characters; description at most 1,000.
- At most 50 procedures. Each needs a name (at most 200), sessions as a whole number of at least 1, an optional fee of 0 or more, and optional notes (at most 500).
- Category must be one of: general, preventive, restorative, endodontics, periodontics, oral surgery, prosthodontics, implantology, orthodontics, pediatric, cosmetic, geriatric, oral medicine, TMJ and orofacial pain, full mouth rehabilitation, emergency, other.
- Names do not have to be unique.

Who can do it. `plans.templates.manage`, feature `patients.treatment_plans`. Default holder: Admin.

Data read and written. Writes `plan_templates`.

Effects on other modules. The new template appears in the Record Treatment dialog when active.

Error cases. "Failed to create template" toast with the server message (for example a validation error).

Technical references: `backend/src/plan-templates/plan-templates.service.ts` · `create`, `CreatePlanTemplateDto`, `PlanTemplateItemDto` · `web-app/src/features/plan-templates/components/PlanTemplateForm.tsx`

33.3.3 Edit a plan template

Status: [Implemented]

What it does. Changes a template's name, category, description or procedures.

How the user does it.

1. Press the "Edit template" pencil on the card.
2. Change fields and press **Save changes**.

Behind the scenes — in simple terms. Only the fields sent are changed; sending procedures replaces the whole list. Treatments already recorded from the template keep their own name and cost.

Behind the scenes — technically. `PATCH /plan-templates/:id` → `PlanTemplatesService.update`. No audit row.

Rules and validations. Same limits as creation; the template must belong to the organization.

Who can do it. `plans.templates.manage`, feature `patients.treatment_plans`. Default holder: Admin.

Data read and written. `plan_templates`.

Effects on other modules. None on existing treatments.

Error cases. "Treatment plan template not found" (404); "Failed to update template" toast.

Technical references: `backend/src/plan-templates/plan-templates.service.ts` · `update`, `UpdatePlanTemplateDto` · `web-app/src/features/plan-templates/hooks/usePlanTemplates.ts`

33.3.4 Hide or show a plan template

Status: [Implemented]

What it does. Switches a template off so doctors no longer see it in the picker, without deleting it.

How the user does it.

1. Flip the switch on the template card. The card then reads "Hidden" (or "Active").

Behind the scenes — in simple terms. A hidden template stays on the management page but disappears from the Record Treatment dialog.

Behind the scenes — technically. `PATCH /plan-templates/:id` with `{ isActive }`.

Rules and validations. None beyond editing rules.

Who can do it. `plans.templates.manage`. Default holder: Admin.

Data read and written. `plan_templates.is_active`.

Effects on other modules. Hidden templates are not offered when recording a treatment.

Error cases. "Failed to update template" toast.

Technical references: `backend/src/plan-templates/entities/plan-template.entity.ts` · `isActive` · `web-app/src/features/plan-templates/pages/PlanTemplatesPage.tsx` (`TemplateCard`)

33.3.5 Delete a plan template

Status: [Implemented]

What it does. Removes a template for good.

How the user does it.

1. Press the "Delete template" bin on the card.
2. Read "Delete this template?" ("… will be deleted for good. Plans already made from it are not affected. To just hide it from doctors, switch it off instead.") and press **Delete**.

Behind the scenes — in simple terms. The template row is deleted. Treatments that were tagged with it lose the tag but keep their name and cost.

Behind the scenes — technically. `DELETE /plan-templates/:id` → `PlanTemplatesService.remove` (hard delete). `treatments.plan_template_id` has `ON DELETE SET NULL`, so tagged treatments show "—" in the Plan column afterwards.

Rules and validations.

- Always behind a confirmation dialog.
- If the clinic deletes every template, the next time anyone reads the list the twelve defaults are inserted again.

Who can do it. `plans.templates.manage`. Default holder: Admin.

Data read and written. `plan_templates`; `treatments.plan_template_id` (cleared by the database).

Effects on other modules. Tagged treatments lose their plan tag.

Error cases. "Treatment plan template not found" (404); "Failed to delete template" toast.

Technical references: `backend/src/plan-templates/plan-templates.service.ts` · `remove` · `web-app/src/features/plan-templates/pages/PlanTemplatesPage.tsx`

33.3.6 Starter templates for a new clinic

Status: [Implemented] (backend-only by design)

What it does. Gives every clinic a starting set so the picker is never empty.

How the user does it. Nothing to do; the templates appear the first time the list is opened.

Behind the scenes — in simple terms. Twelve common dental plans are added without fees, because fees belong to each clinic. The clinic sets fees once on the templates page.

Behind the scenes — technically. `PlanTemplatesService.findAll` counts the organization's templates (active or not); when the count is zero it inserts `DEFAULT_PLAN_TEMPLATES` with `estimatedCost: null` and `createdBy: null`. The defaults are: Root Canal Treatment (RCT), Scaling & Polishing, Gum Treatment (Periodontal), Composite Filling, Simple Extraction, Wisdom Tooth Surgical Extraction, Crown / Bridge, Dental Implant, Complete / Partial Denture, Braces (Orthodontics), Teeth Whitening and Pulpotomy (Child). For example, Root Canal Treatment has six procedures: diagnosis and X-ray, access opening, cleaning and shaping (two sessions), obturation, post-endodontic restoration, and crown (two sessions).

Rules and validations. Seeding happens only when the organization has zero template rows.

Who can do it. Triggered by any read of the list.

Data read and written. `plan_templates`.

Effects on other modules. None.

Error cases. None specific.

Technical references: `backend/src/plan-templates/default-plan-templates.ts` · `DEFAULT_PLAN_TEMPLATES`, `seedDefaults` · none in web-app

33.3.7 Use a template when recording a treatment

Status: [Implemented]

What it does. Lets staff record a treatment by picking a template (the whole plan or one of its procedures), which fills the name and cost and tags the treatment with the template.

How the user does it.

1. On a patient's **Treatments** tab press **Add Treatment** (or, in a visit's details, **Add treatment**).
2. Pick a "Treatment plan"; optionally pick a "Procedure".
3. Adjust Treatment Name, Cost and Date and press **Save Treatment**. The Treatments table shows the template name as a badge in its Plan column.

Behind the scenes — in simple terms. Choosing the whole plan fills the template name and the sum of the fees that are set (empty when no procedure has a fee). Choosing one procedure fills that procedure's name and fee. The template link is a label; it does not create or update a patient plan.

Behind the scenes — technically. `POST /treatments` with `planTemplateId` → `TreatmentsService.create` checks that the template belongs to the organization, then stores `treatments.plan_template_id`. The template list comes from `GET /plan-templates` (active only).

Rules and validations.

- Treatment name and date required; cost cannot be negative.
- The button is shown only to users with both `patients.clinical.edit` and `patients.price.view`.

Who can do it. `patients.clinical.edit` (treatments endpoint) plus `patients.price.view` (button visibility). Default holder with both: Admin. The Doctor template lacks `patients.price.view`, so doctors do not see this button.

Data read and written. Reads `plan_templates`; writes `treatments`.

Effects on other modules. Treatments can produce charges (the Consultation and Billing chapters, Chapters 31 and 34).

Error cases. "Treatment plan not found" (404) → the template was deleted meanwhile.

Technical references: `backend/src/treatments/treatments.service.ts` · `TreatmentsService.create`, `planTemplateId` · `web-app/src/features/patients/components/tabs/TreatmentFormDialog.tsx`

33.3.8 Create a patient treatment plan

Status: [Partial] — backend only; web API functions and hooks exist but no component uses them.

What it does. Opens a plan for a patient with a name, the responsible dentist, estimated cost, discount, dates and notes.

How the user does it. No screen. An API client calls the endpoint.

Behind the scenes — in simple terms. The plan gets the next plan number for the clinic (`TP-00001`, `TP-00002` ...), starts as a draft unless another status is given, and its final estimate is the estimate minus the discount. A "created" entry goes into the patient's audit history.

Behind the scenes — technically. `POST /patients/:id/treatment-plans` → `PatientsService.createTreatmentPlan`. `generatePlanNumber` counts the organization's plans (including soft-deleted ones) plus one. `finalEstimatedCost = estimatedCost − discountAmount` when an estimate is given. `locationId` is the active location; `createdBy` the user id. Save and audit row (`patient_treatment_plan`, `create`) in one transaction via `saveAudited`. The patient must be inside the caller's `patients.view` scope (`PatientScopeGuard`).

Rules and validations.

- `dentistId` (a UUID) and `planName` (at most 200) are required. The dentist id is not checked against employees.
- Optional `estimatedCost`, `discountAmount` (numbers), `status`, `startDate`, `expectedEndDate`, `notes`, `description`.
- The create request cannot carry items; items are added one by one with the items endpoint.

Who can do it. `plans.manage`, feature `patients.treatment_plans`. Default holders: Admin, Doctor.

Data read and written. `patient_treatment_plans`, `audit_logs`.

Effects on other modules. None until items are billed.

Error cases. "Patient not found" (404); validation errors (400); a duplicate plan number (possible after a patient with plans is deleted) fails with a server error.

Technical references: `backend/src/patients/patients.service.ts` · `createTreatmentPlan`, `CreateTreatmentPlanDto` · `web-app/src/features/patients/hooks/usePatientSubEntities.ts` (`useCreateTreatmentPlan`, unused)

33.3.9 Update a plan: status, acceptance and costs

Status: [Partial] — backend only.

What it does. Changes a plan's name, description, costs, status, dates, notes, and records that the patient accepted it.

How the user does it. No screen.

Behind the scenes — in simple terms. When the patient's acceptance is set for the first time, the time of acceptance is stamped. The final estimate is recalculated from the current estimate and discount.

Behind the scenes — technically. `PATCH /patients/:id/treatment-plans/:planId` → `updateTreatmentPlan`: loads the plan of this patient and organization; if `patientAccepted` turns true, sets `patientAcceptedAt` to now; recalculates `finalEstimatedCost`; saves with an audited diff.

Rules and validations.

- Any status may be set from any status; there is no transition map (the intended paths are drawn in Figure 33.7).
- `actualEndDate` is set only when sent; completing the plan does not set it.
- Turning `patientAccepted` back to false keeps the old acceptance time.

Who can do it. `plans.manage`. Default holders: Admin, Doctor.

Data read and written. `patient_treatment_plans`, `audit_logs`.

Effects on other modules. None.

Error cases. "Treatment plan not found" (404).

Technical references: `backend/src/patients/patients.service.ts` · `updateTreatmentPlan`, `UpdateTreatmentPlanDto` · `web-app/src/features/patients/services/patients.api.ts`

33.3.10 Add items to a plan

Status: [Partial] — backend only.

What it does. Adds one procedure step to a plan: name, optional code, tooth number and surface, estimated sessions, estimated cost, order, assigned doctor and notes.

How the user does it. No screen.

Behind the scenes — in simple terms. Each item starts as pending with zero completed sessions.

Behind the scenes — technically. `POST /patients/:id/treatment-plans/:planId/items` → `createPlanItem` (checks the plan belongs to the patient and organization) → `saveAudited` (`treatment_plan_item`, `create`). `GET …/items` lists items by `sequence_order`.

Rules and validations.

- Procedure name required (at most 200); code at most 20; tooth number at most 10; surface at most 50.
- Estimated sessions and sequence order are whole numbers of at least 1 (defaults 1).
- Item costs are not summed into the plan's estimate automatically.

Who can do it. `plans.manage` to write; `patients.clinical.view` plus the feature to read. Default writers: Admin, Doctor.

Data read and written. `treatment_plan_items`, `audit_logs`.

Effects on other modules. Billing can turn items into charges (backend only).

Error cases. "Treatment plan not found" (404).

Technical references: `backend/src/patients/patients.service.ts` · `createPlanItem`, `CreatePlanItemDto` · `web-app/src/features/patients/hooks/usePatientSubEntities.ts` (`useCreateTreatmentPlanItem`, unused)

33.3.11 Track progress of plan items

Status: [Partial] — backend only; nothing updates progress automatically.

What it does. Updates an item's status, completed sessions and any other field.

How the user does it. No screen.

Behind the scenes — in simple terms. Progress is entered by hand. Recording a treatment or completing a consultation does not move a plan item forward.

Behind the scenes — technically. `PATCH /patients/:id/treatment-plans/:planId/items/:itemId` → `updatePlanItem`. After applying the change, completed sessions are capped at the estimated sessions. Audited diff.

Rules and validations.

- Completed sessions: whole number of 0 or more, never more than the estimate after saving.
- Any status may be set from any status.
- Marking every item completed does not complete the plan.

Who can do it. `plans.manage`. Default holders: Admin, Doctor.

Data read and written. `treatment_plan_items`, `audit_logs`.

Effects on other modules. None.

Error cases. "Treatment plan not found", "Plan item not found" (404).

Technical references: `backend/src/patients/patients.service.ts` · `updatePlanItem`, `UpdatePlanItemDto` · none used in web-app

33.3.12 Start a patient plan from a template

Status: [Not implemented] — mentioned in code comments and UI text only.

What it does. The template page says "The procedures are copied into a patient's plan when a doctor uses this template", and the entity comment says the same. No endpoint, service method or screen does this: there is no "create plan from template" route, the create-plan request cannot carry items, and no screen creates a patient plan.

How the user does it. Not available.

Behind the scenes — in simple terms. Templates are used only to fill the Record Treatment form.

Behind the scenes — technically. None.

Rules and validations. None.

Who can do it. Nobody.

Data read and written. None.

Effects on other modules. None.

Error cases. None.

Technical references: `backend/src/plan-templates/entities/plan-template.entity.ts` (comment) · `PlanTemplate` · `web-app/src/features/plan-templates/pages/PlanTemplatesPage.tsx` (dialog description)

33.3.13 Turn plan items into charges

Status: [Partial] — backend only; no web code calls the endpoint.

What it does. Creates one unbilled **charge** per chosen plan item, ready for the billing desk.

How the user does it. No screen. An API client calls the endpoint.

Behind the scenes — in simple terms. For each chosen item the Platform creates a charge for the item's procedure, priced at the item's estimate. If the item has no estimate, the charge is created at zero and flagged as needing a price. An item that already has an active charge is skipped silently, so the same step cannot be billed twice. Figure 33.5 shows the flow.

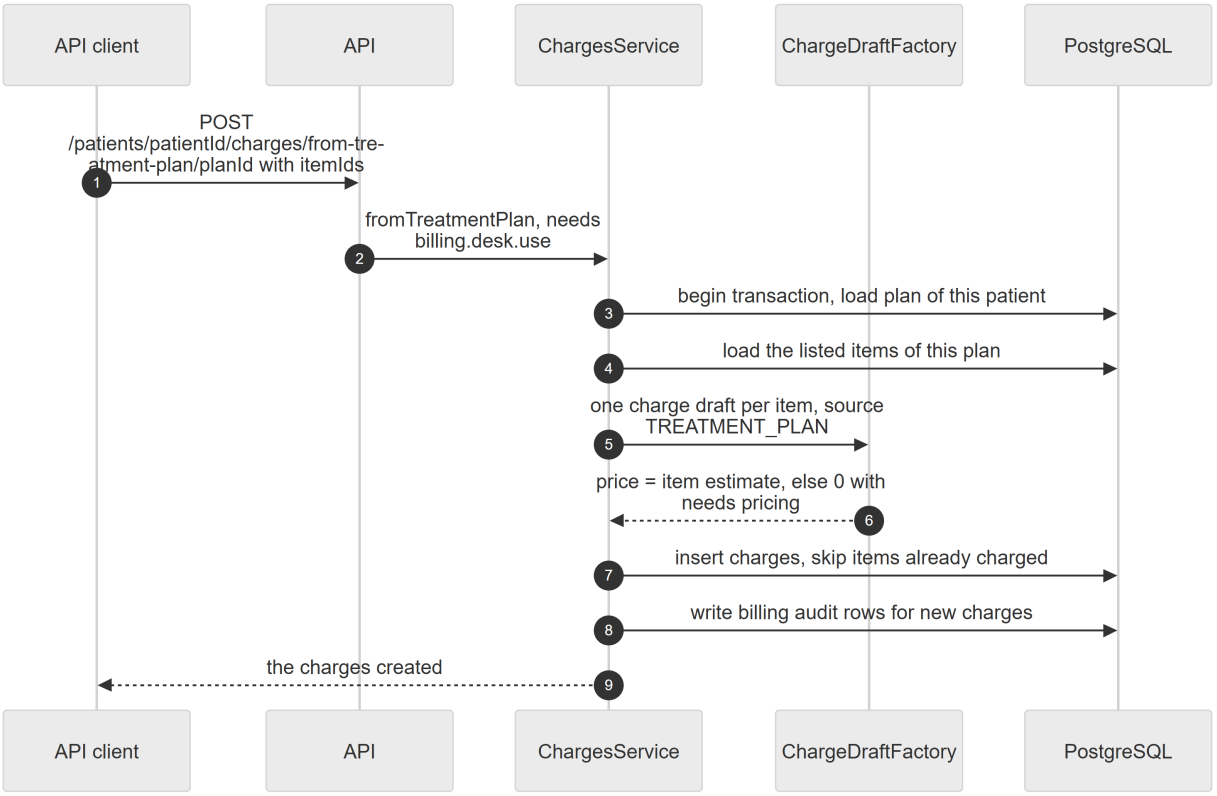

Figure 33.5 — Sequence of turning plan items into charges.

The diagram has five participants: API client, API, ChargesService, ChargeDraftFactory and PostgreSQL. (1) The client posts `POST /patients/patientId/charges/from-treatment-plan/planId` with a list of item ids. (2) The API, after checking `billing.desk.use`, calls `fromTreatmentPlan`. (3) The service opens a transaction and loads the plan, which must belong to this patient and organization. (4) It loads the listed items of that plan. (5) It asks the factory for one charge draft per item with source `TREATMENT_PLAN`. (6) The factory prices each draft at the item's estimate, or at zero with "needs pricing" set. (7) The service inserts the charges, skipping items that already have an active charge. (8) It writes billing audit rows for the charges actually created. (9) It returns the created charges.

Behind the scenes — technically. `POST /patients/:patientId/charges/from-treatment-plan/:planId` → `ChargesService.fromTreatmentPlan` (one transaction). `ChargeDraftFactory.fromPlanItem` sets `source = TREATMENT_PLAN`, `sourceId = item id`, description = procedure name (at most 200 characters), quantity 1, `unitPrice` = item estimate when above zero else 0, `needsPricing` = true when there is no estimate, and the doctor = the item's `assignedTo`, else the plan's `dentistId`. The charge date is today in the location's timezone. Duplicates are prevented by the partial unique index `uq_charges_source` (organization, source, source id, for active charges) with `INSERT … ON CONFLICT DO NOTHING`. Each new charge gets a billing audit row (`CREATED`).

Rules and validations.

- 1 to 100 item ids, each a UUID; every id must belong to the plan, otherwise nothing is created.
- Plan and item statuses are not checked: cancelled plans and skipped items can be charged.
- The catalog price list is not consulted (the factory receives no catalog price).

Who can do it. `billing.desk.use` (feature `billing.invoices`). Default holders: Admin, Receptionist, Accountant. The endpoint does not require `plans.manage`.

Data read and written. Reads `patient_treatment_plans`, `treatment_plan_items`; writes `charges` and the billing audit log.

Effects on other modules. The charges appear in the patient's open charges and on the billing desk (the Billing chapter, Chapter 34, covers what follows).

Error cases. "Treatment plan not found" or "Plan item not found" (404); validation errors for the item list (400).

Technical references: `backend/src/billing/charges/charges.service.ts` · `fromTreatmentPlan`, `ChargeDraftFactory.fromPlanItem`, `ChargesFromPlanDto` · none in web-app

33.3.14 See a plan's billing summary

Status: [Partial] — the endpoint, a web API function and a hook (`usePlanBillingSummary`) exist, but no component renders it.

What it does. Answers, for one plan: the estimate, how much has been billed on issued invoices, how much is charged but not yet invoiced, how much is not yet charged, how much has been paid, and the balance owed.

How the user does it. No screen.

Behind the scenes — in simple terms. Only charges created from the plan's items count; treatments recorded separately are not linked to plan items and do not count.

Behind the scenes — technically. `GET /treatment-plans/:planId/billing-summary` → `BillingReportsService.planSummary`. It sums active `TREATMENT_PLAN` charges whose source is one of the plan's items, at the active location only: billed (net amount on `ISSUED` invoices), tax on those, unbilled (no invoice yet) and paid (allocated amount). Estimate = the plan's final estimate, else its estimate, else the sum of item estimates. `notYetCharged = max(0, estimate − billed − unbilled)`; `balance = billed + tax − paid` (billed is before tax, what is owed includes tax). Arithmetic is done in paisa and returned in currency units.

Rules and validations.

- The plan must belong to the organization; the patient's scope is not checked on this route.
- Charges made at another location of the same organization are left out.

Who can do it. `billing.invoice.view` (feature `billing.invoices`). Default holders: Admin (org), Receptionist and Accountant (location).

Data read and written. Reads `patient_treatment_plans`, `treatment_plan_items`, `charges`, `invoices`.

Effects on other modules. None.

Error cases. "Treatment plan not found" (404).

Technical references: `backend/src/billing/reports/billing-reports.service.ts` · `planSummary`, `PlanBillingSummary` · `web-app/src/features/billing/hooks/use-billing-queries.ts` (`usePlanBillingSummary`, unused)

33.4 Entities and data model

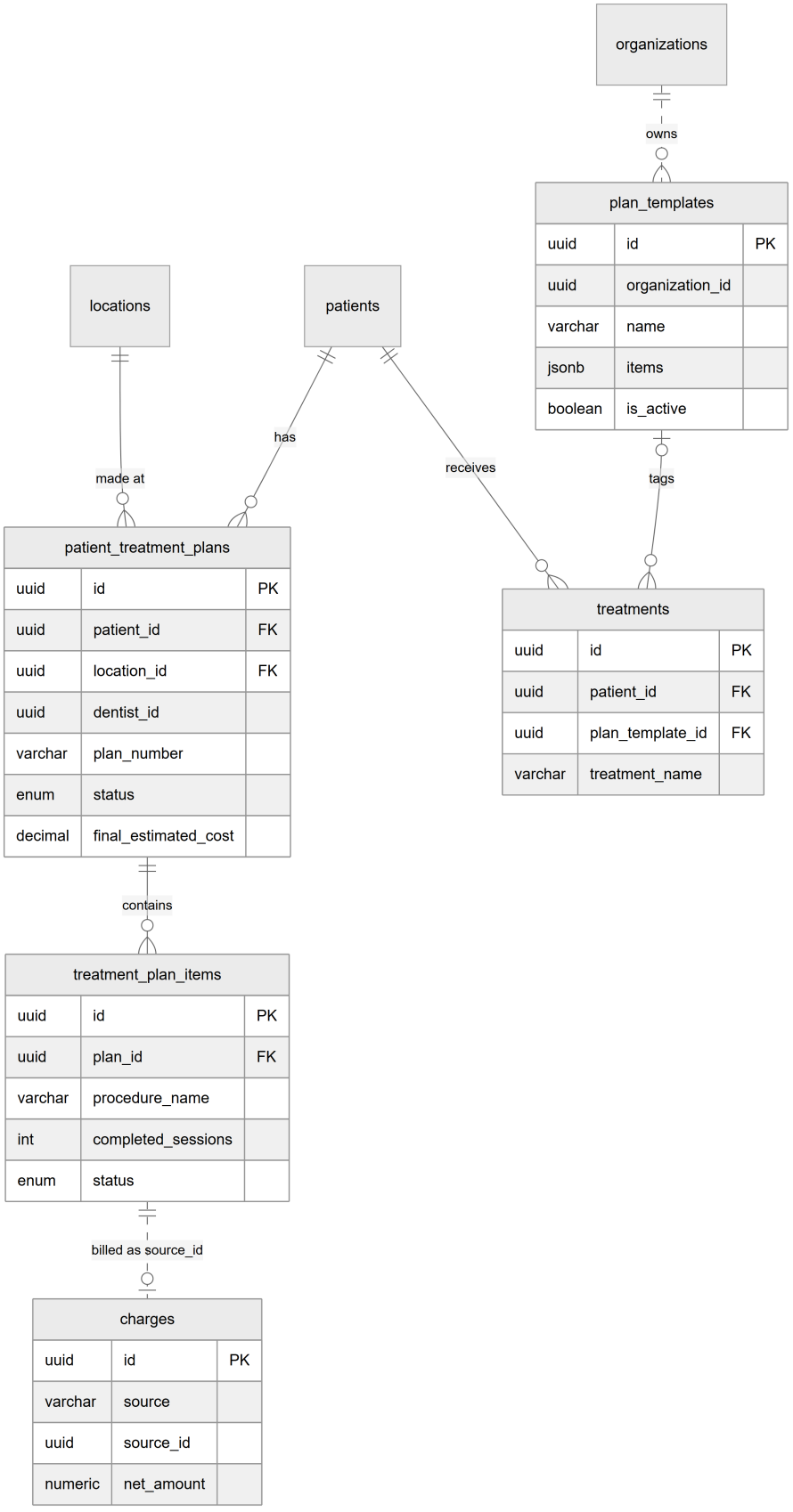

Figure 33.6 — Entity-relationship diagram of treatment plans, plan items, plan templates and their links to treatments and charges.

The diagram shows seven tables. Each `patient_treatment_plans` row belongs to exactly one patient and one location; deleting the patient or the location deletes the plan (ON DELETE CASCADE). The plan also carries `organization_id` (cascading), `dentist_id` (a plain UUID with no foreign key), `plan_number` (unique per organization), `status` and the three cost columns `estimated_cost`, `discount_amount` and `final_estimated_cost` (decimal, currency units, hidden from users without `patients.price.view` by `@VisibleIf`). Each plan contains any number of `treatment_plan_items`; deleting a plan deletes its items. Items have no `organization_id` of their own; they reach the organization through their plan. Items carry `procedure_name`, `estimated_sessions`, `completed_sessions`, `estimated_cost` (decimal, also price-hidden), `status`, `sequence_order` and `assigned_to` (a plain UUID). The dotted line from an item to `charges` is a logical link without a foreign key: a charge created from an item stores the item id in `source_id` with `source = 'TREATMENT_PLAN'`, and at most one active charge can exist per item. `plan_templates` belongs to an organization through `organization_id`, drawn dotted because there is no foreign key. Templates keep their procedures inside the JSON column `items` (each with `procedureName`, `estimatedSessions`, `estimatedCost`, `notes`), so template procedures are not rows. Each `treatments` row belongs to one patient (cascading) and may point to one plan template through `plan_template_id`; deleting the template clears that pointer (ON DELETE SET NULL). There is no link between a treatment and a plan item, and none between a patient plan and the template it might have come from.

Plans and templates are separate from the prescriptions module's `treatment_plan_templates` and `treatment_plan_template_medicines` tables, which hold prescription presets.

33.5 Status lifecycles

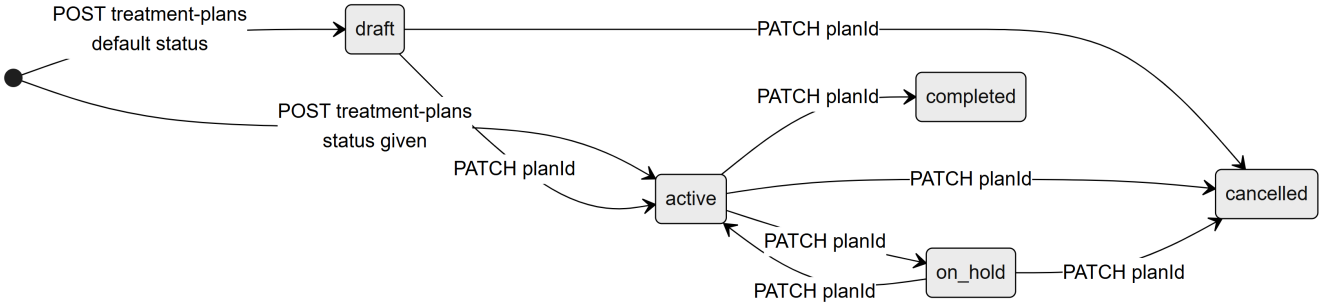

Figure 33.7 — Intended lifecycle of a patient treatment plan.

A plan is created by `POST /patients/:id/treatment-plans` in **draft** by default, or directly in **active** (or any other status) when the request gives one. `PATCH /patients/:id/treatment-plans/:planId` moves it: draft to active when work starts; active to **on_hold** and on_hold back to active; active to **completed**; and draft, active or on_hold to **cancelled**. The diagram shows the intended paths only. The code accepts any status from any status, never changes the status by itself, and does not stamp `actual_end_date` on completion. No screen shows the status today.

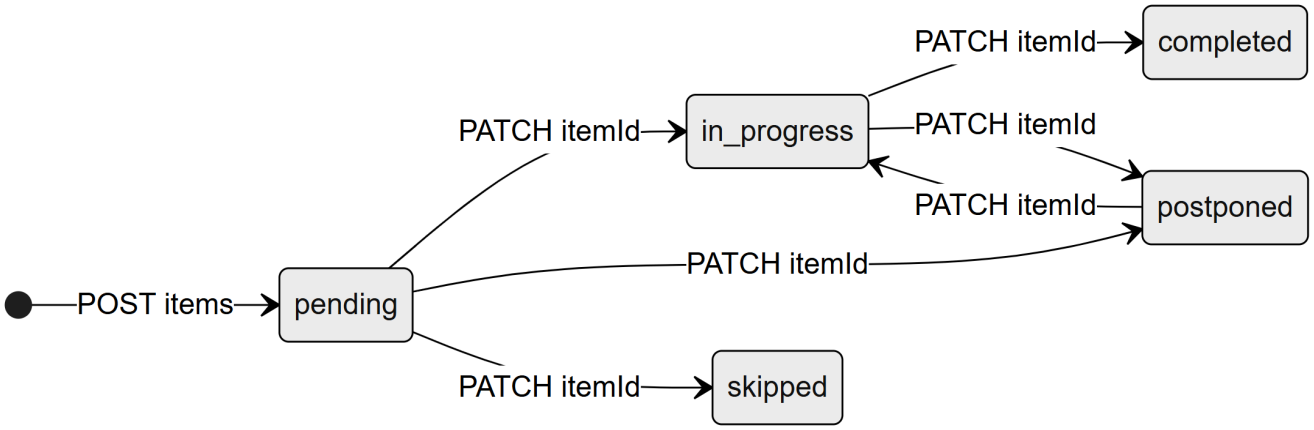

Figure 33.8 — Intended lifecycle of a treatment plan item.

An item is created by `POST …/items` in **pending**. `PATCH …/items/:itemId` moves it: pending to **in_progress**, in_progress to **completed**, pending or in_progress to **postponed**, postponed back to in_progress, and pending to **skipped**. As with plans, any status can be set from any status, and nothing changes item status automatically: recording a treatment, completing a consultation or billing the item leaves it as it is. Completed sessions are stored separately and capped at the estimated sessions.

Plan templates have no status lifecycle; they only have the `is_active` switch (Active or Hidden).

33.6 Connections to other modules

- **Patients (Chapter 28):** patient plans and items are stored and audited by the Patients module; plan writes appear in the patient's change history (values hidden from readers without `patients.clinical.view`). Deleting a patient deletes their plans.
- **Treatments and consultation (Chapter 31):** the Record Treatment dialog, on the patient page and inside a visit, uses active plan templates and tags treatments with them.
- **Billing (Chapter 34):** charges from plan items and the plan billing summary (both backend only); the billing desk then prices, invoices and collects those charges.
- **Prescriptions (Chapter 32):** prescription presets are a different kind of template, stored in `treatment_plan_templates`.
- **Features and access (Chapters 10 and 11):** feature `patients.treatment_plans`; permissions `plans.manage`, `plans.templates.manage`, `patients.clinical.view` and `patients.price.view`.

33.7 Limitations and known gaps

- **No screen for patient treatment plans.** Plans and items can only be created, read and updated through the API; the web hooks exist but are unused.
- **Templates cannot start a patient plan**, although the template page text says the procedures are copied into a patient's plan.
- **Plan billing exists only in the backend.** Neither "charges from plan" nor the billing summary is called by any screen.
- **No automatic progress.** Treatments are not linked to plan items, so completed sessions, item status, plan status and the actual end date are all manual.
- **Free status changes.** There is no transition map for plan or item status.
- **Plan numbers come from a row count** and can repeat after a patient with plans is deleted, which makes the next save fail.

- **No delete endpoints** for plans or items; a mistaken plan can only be cancelled.
- **The plan estimate is entered by hand**; item costs are not summed into it (the billing summary falls back to the item sum only when the plan has no estimate).
- **Template changes are not audited**, unlike patient plans.
- **Deleting every template brings the twelve defaults back** on the next read.
- **Doctors cannot use templates from the patient page by default**, because the Add Treatment button also needs `patients.price.view`, which the Doctor template lacks.

33.8 Owner's FAQ

Is "Treatment Plans" in the sidebar the place where a doctor builds a patient's plan? No. That page (`/plan-templates`) manages the clinic's reusable templates. There is no screen for a patient's own plan today; patient plans exist only in the backend.

What is the difference between a plan template and a prescription preset? A plan template is a list of procedures with sessions and fees, used when recording treatments. A prescription preset is a list of medicines with instructions, used in the prescription builder. The preset table is confusingly named `treatment_plan_templates` in the database.

If I change a template's fee, do old treatments change? No. The Record Treatment dialog copies the name and cost into the treatment when it is saved; later template edits do not touch existing treatments.

What happens to treatments if I delete a template? They keep their name, cost and date; only the template badge in their Plan column disappears.

Who can edit templates? Holders of `plans.templates.manage`, which only the Admin system role has by default. Every member of the clinic can read the template list when the feature is on, but fees are removed for users without `patients.price.view`.

Can a plan item be billed twice? Not while its first charge is active: the database allows only one active charge per plan item, and a repeated request skips that item silently. If the first charge is voided, the item can be charged again.

Does completing a consultation update the patient's plan? No. Plans, items and treatments are not linked, so progress must be entered by hand through the API.

What happens if I switch off the Treatment Plans feature? The Treatment Plans page and sidebar entry disappear, the template list endpoint and all patient plan endpoints refuse requests, and the permissions `plans.manage` and `plans.templates.manage` are dropped for every role until the feature is switched back on. Treatments already tagged with a template keep the tag.

Chapter 34 — Billing

This chapter describes the Billing module of the Platform: how a clinic turns finished visits and extra work into charges, collects money, issues invoices and receipts, gives refunds and closes the cash counter at the end of a shift. It is written for the product owner, who needs to know exactly what the module does today, which rules protect the money and where the module stops. Billing is the largest module of the Platform (84 endpoints under `backend/src/billing`, six of which belong to doctor commissions, covered in Chapter 35).

34.1 What this module is for

Billing on the Platform is **charge-based**. A **charge** is one billable line for one patient: a consultation fee, a treatment, or an extra item that reception adds. Charges pile up "unbilled" until the patient pays or reception issues an invoice. An **invoice** groups unbilled charges under one number and fixes the sales tax. A **payment** is money received into a **payment account** (Cash, a bank account, a card machine); it is split over charges as **allocations**. Money that is received but not needed for any charge stays with the patient as **advance**. A **refund** always pays out of advance. At the end of a shift, a **cash close** compares the money the system expects in each account with what was counted.

Who uses it:

- **Receptionist**: the main user at the desk. Sees the billing desk, adds charges, applies preset discounts, collects payments and prints receipts and invoices. Cannot edit or cancel invoices, refund or close the counter.
- **Accountant**: everything a receptionist can do, plus invoice edits and cancellations, refunds, cash close, billing activity, expenses and financial reports. Cannot apply discounts or override prices with the default role.
- **Admin** and the **owner**: every billing permission, including custom discounts, price overrides and billing settings.
- **Doctor** and **Assistant**: no billing permissions. A doctor's completed visit creates charges in the background, but the doctor never sees an amount.

Where it appears in the sidebar (`web-app/src/features/access/nav/nav.registry.ts`): the **Finance** section has a **Billing** group with **Billing desk** (`/billing`, needs `billing.desk.use`), **Invoices** (`/invoices`, `billing.invoice.view`), **Payments** (`/payments`, the sidebar checks `billing.payment.collect`) and **Expenses** (`/expenses`, `expenses.view`). The **Reports** group of the same section holds **Billing activity** (`/billing/activity`, `billing.activity.view`), **Doctor revenue** and **Doctor earnings**. Billing settings have no sidebar entry; they open from the Settings page as **Billing** (`/settings/billing`). The patient record has a **Billing** tab.

The module sits behind the feature `billing.invoices` ("Invoices"). Its seed sets `enabledByDefault: false`, so a clinic has no billing until the feature is entitled and enabled (Chapter 10). Discounts need the extra feature `billing.discounts`, expenses need `billing.expenses`, the financial summary needs `billing.financial_summary` and credit packages need `billing.credit_packages`.

34.2 Screens

All screens in this chapter are shown as placeholders; an older billing screenshot exists but is excluded because it shows realistic personal data.

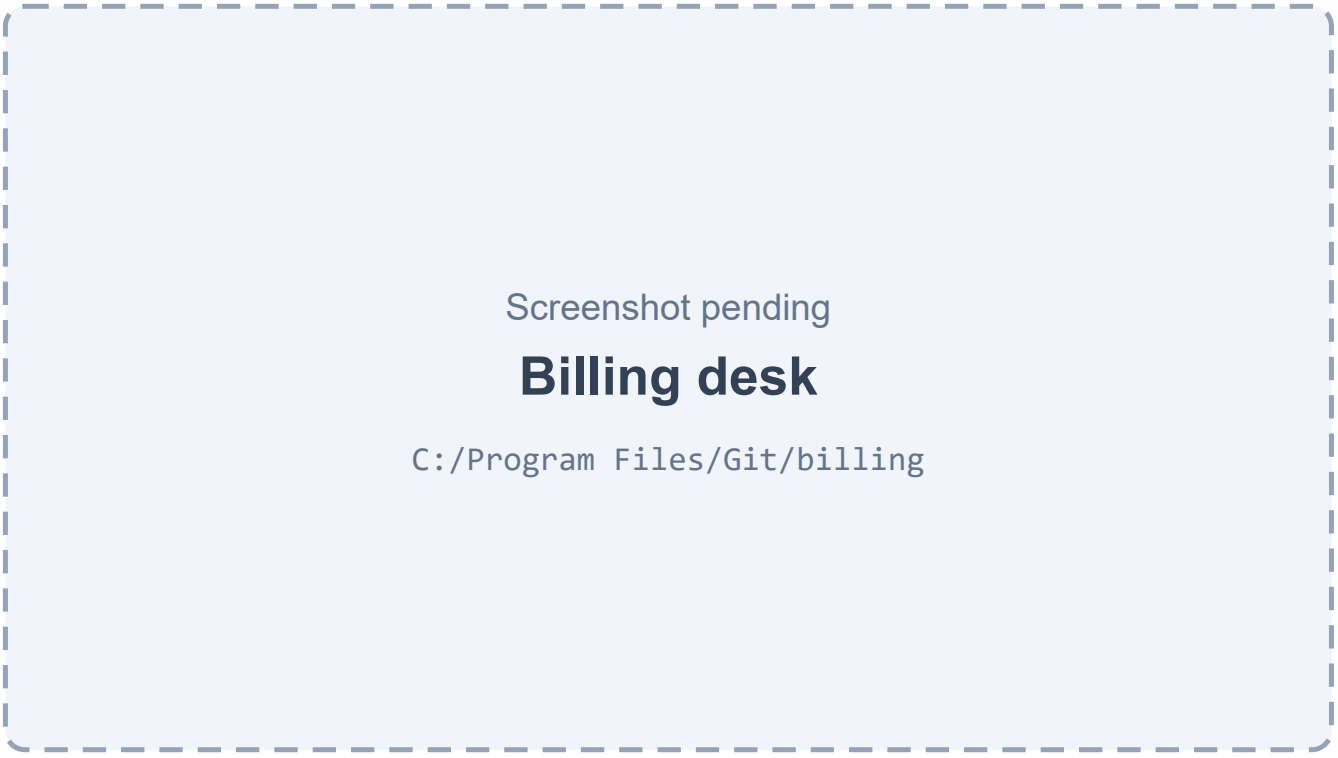

Figure 34.1 — Billing desk (screenshot pending).

The Billing desk (`/billing`) has the title "Billing desk" and the subtitle "Bill finished visits and follow up on dues". A date picker in the header ("Today" by default, clearable) chooses the clinic day. If the location has no document code, an amber notice says "This location has no document code, so invoices and receipts cannot be numbered yet.", with a link "Set it in billing settings" for users who manage settings. The first section, **Ready to bill** ("Patients with charges not invoiced yet"), lists patients with columns Patient (name, then patient number or phone), Charges (a count and a "Needs price" badge when one charge has no price), Amount (the unbilled total) and a **Bill** button. Clicking **Bill** opens the Collect payment sheet for that patient; there is no patient search on the desk. The list pages in the browser, 10 rows at a time; when empty it says "Nobody is waiting to be billed. Completed consultations appear here." The second section, **Outstanding** ("Overdue and unpaid invoices"), lists issued invoices whose payment status is Overdue or Unpaid (partly paid invoices are not shown), with Patient, Invoice (a link), Due (with a red "Overdue" badge), Balance and a **Collect** button. When empty it says "No dues — every issued invoice is paid."

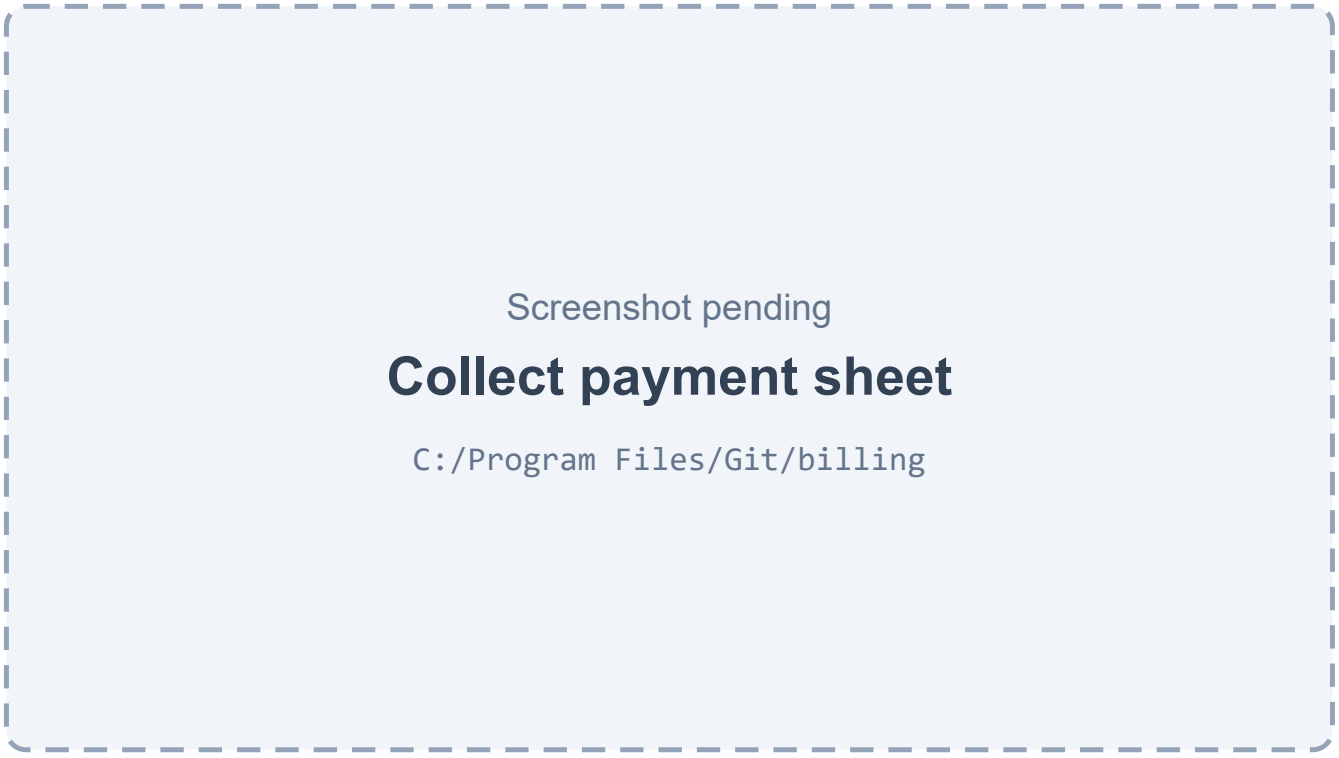

Figure 34.2 — The Collect payment sheet (screenshot pending).

The Collect payment sheet slides in from the right with the title "Collect payment — " and the patient's name. From top to bottom it shows: four summary figures ("Not invoiced", "Outstanding", "Overdue", "Advance"); the patient's open charges with a "Select all charges" box, a **Discount** button and an **Add charge** button, grouped as "Not invoiced yet" and then one group per invoice ("Invoice " plus its number); each line has a checkbox, the description, quantity, date, doctor, any discount, the balance and a three-dot menu. After the charges come the payment fields: **Amount received (Rs)** (filled automatically with what clears the selection until the user types), **Allocation** with the choices **Oldest first** and **Manual** (Manual adds a "Pay now for …" amount on each selected line), **Received in** (one chip per active payment account, the default account pre-selected), **Reference number** (only for accounts that require one), a **Use advance** switch when the patient holds advance, and **Notes**. A live summary shows "Selected balance", "Advance used", "Paying now" and "Going to advance", a line such as "An invoice will be issued for 2 charge(s) (incl. Tax (17%) …)", any blocker ("Set a price for: …") and, only for users who can see all earnings, an internal "Doctor share" line. The main button reads **Receive Rs …**, or **Apply advance Rs …** when the amount is zero and advance covers the selection. After success the sheet shows "Received …" with the receipt number, "Invoice … issued", "Still outstanding", "Advance held", a **Print receipt** button with a format choice and a **Print invoice** button.

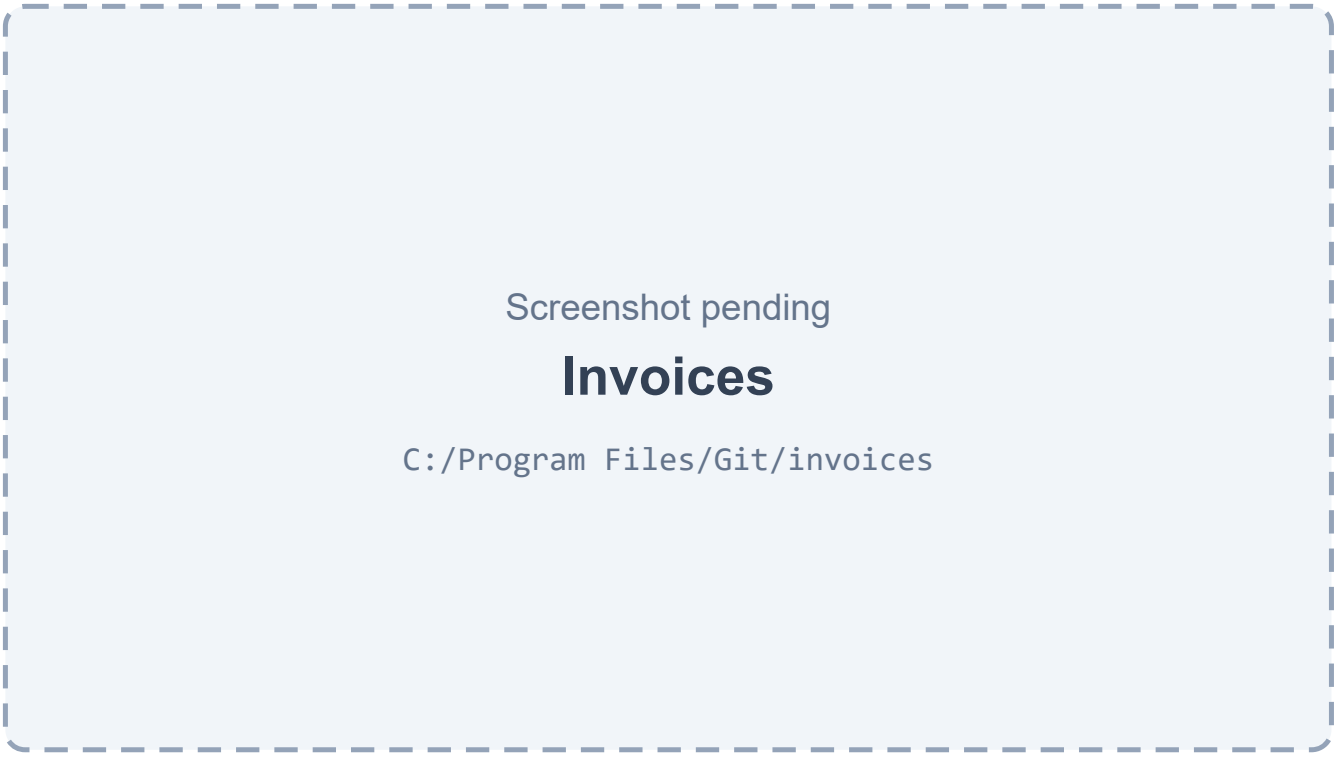

Figure 34.3 — Invoices list (screenshot pending).

The Invoices page (`/invoices`) has the subtitle "Search issued invoices — status follows payments and due dates.", a **List / Cards** view switch and a **Billing desk** button. A strip of six figures shows Invoices (count), Invoiced, Collected, Outstanding, Overdue and Refunded. Status tabs with counts read All, Unpaid, Partial, Overdue, Paid, Refunded and Cancelled. A search box ("Invoice no., patient or phone...") is followed by date presets (Today, This month, Past 30 days, All), a custom range and a **Reset** button. The table columns are Date (with "Due …" under it), Invoice (with "Rev n" after an edit), Patient, Total, Paid, Balance, Status and a menu with **View details**, **Collect payment** (issued invoices with a balance) and **Print**. Cancelled rows are dimmed. The card view shows the same data per invoice. Paging is on the server with 10, 25, 50 or 100 rows. Without filters and data it says "No invoices yet — they are issued when reception collects a payment." with an **Open billing desk** button; with filters and no match it says "No invoices match these filters."

Screenshot pending

Invoice detail

`C:/Program Files/Git/invoices/$invoiceId`

Figure 34.4 — Invoice detail (screenshot pending).

The invoice detail page (`/invoices/$invoiceId`) has the title "Invoice" plus the number and a back button "Back to invoices". Its action buttons are **Print** (A4 or Thermal 80 mm), **Edit**, **Refund**, **Cancel** (red) and **Collect payment**, each shown only when the invoice state and the user's permissions allow it. A cancelled invoice shows a red banner "Cancelled … — " and the reason. The header card shows the patient, a status badge (Paid, Partially paid, Unpaid, Overdue, Cancelled or Refunded, plus "Refunded …" for a partial refund), "Rev n", the issue date, due date and issuer. A totals box lists Subtotal, Discount, Tax (only when the rate is above zero), Total, Paid and Balance due. The page then has the sections **Charges** (Charge, Qty, Price, Discount, Net, Paid; void charges struck through), **Payments on this invoice**, **Refunds** ("Refunded … · still refundable …") and, after edits, **Revision history** with a **Print rev n** button per older revision.

Screenshot pending

Edit invoice sheet

`C:/Program Files/Git/invoices/$invoiceId`

Figure 34.5 — The Edit invoice sheet (screenshot pending).

The Edit sheet is titled "Edit" plus the invoice number, with the line "Revision r → r+1. The invoice number never changes." Each active charge has a Quantity field, a Unit price field (editable only with the price-override permission or for non-catalog charges), a Discount choice (keep, "Remove discount" or another discount type) and a **Void** box. A section "Add unbilled charges" lists the patient's unbilled charges with checkboxes. A **Reason** field is required (at least 5 characters). A live preview shows the tax, "New total", "Change", "New balance" and, when money is freed, "… will move to the patient's advance." If someone else changed the invoice meanwhile, a banner says "This invoice was changed by someone else." with **Reload**. The buttons are **Cancel**, **Save & refund** (only when money moves to advance; it opens the refund sheet) and **Save changes**.

Screenshot pending

Payments

`C:/Program Files/Git/payments`

Figure 34.6 — Payments page (screenshot pending).

The Payments page (`/payments`) has the subtitle "The day's collection per payment account, refunds subtracted." and a **Close counter** button for users who may close the counter. Four figures show Receipts, Collected, Refunded and Net in hand for the chosen day. A toolbar has a day picker, previous and next arrows, **Today** and **Yesterday** buttons and an account filter. The section **By payment account** lists each account with its payments, refunds and net. The section **Receipts** lists the day's payments (Time with receipt number, Patient, Account with reference, Received by, Amount with "−… refunded"); clicking a row prints the receipt, and the row menu offers **Print receipt** and **Open patient**. The last section, **Counter closes**, is the cash-close history. There is no card view on this page.

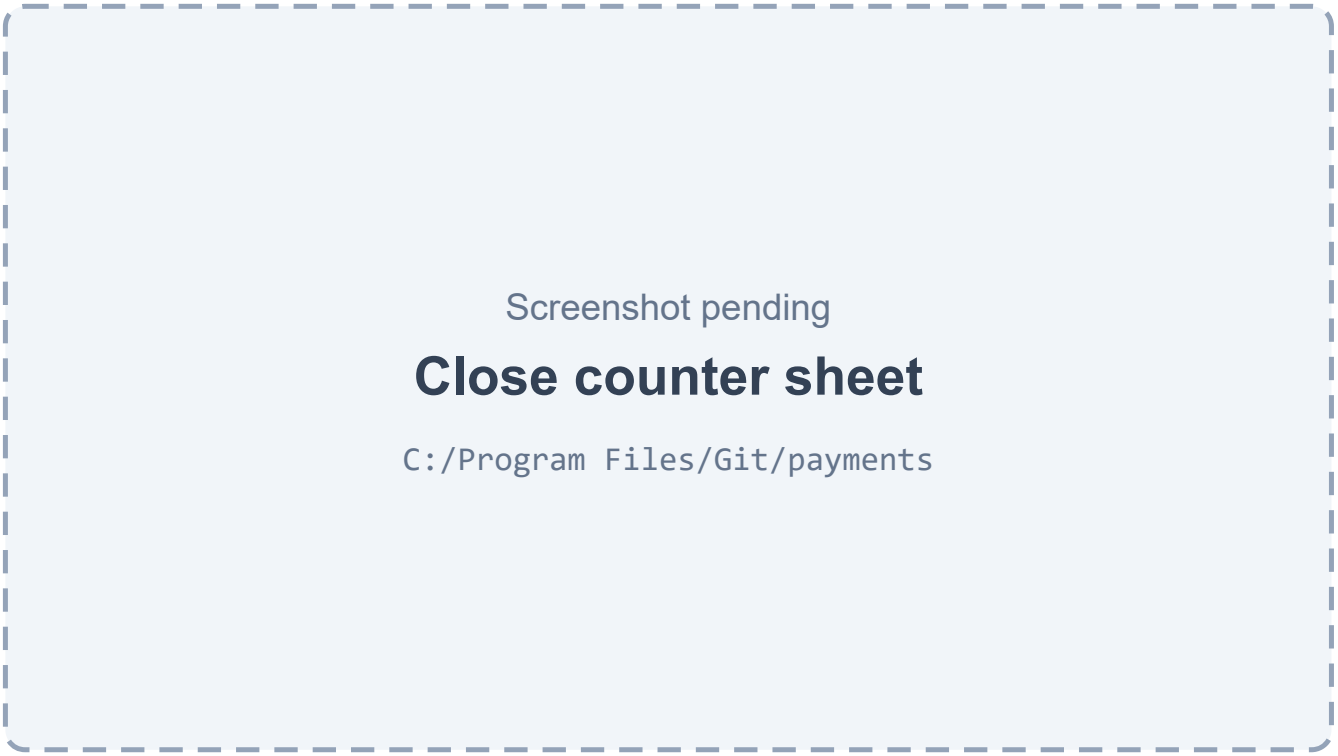

Figure 34.7 — The Close counter sheet (screenshot pending).

The Close counter sheet says "Since" and the end of the previous close (or "Since the first payment at this location"). It shows one card per payment account with "Expected" and the counts and totals of payments and refunds. Accounts marked "Count physically at cash close" have a required **Counted (Rs)** field with a live green or red difference. A **Note** is required when any difference exists ("Explain the difference"). The button reads **Close counter (Rs … expected)**. After success it says "Counter closed. Print the slip to hand over the cash." with **Print slip** and **Done**.

Screenshot pending

Patient Billing tab

C:/Program Files/Git/patients/$patientId

Figure 34.8 — The Billing tab on the patient record (screenshot pending).

The patient record's **Billing** tab shows the four figures Not invoiced, Outstanding, Overdue and Advance, a **Refund advance** button when advance exists, and four sub-tabs. **Open charges** lists unbilled and unpaid charges (Charge, Date, Invoice or "Not invoiced", Net with tax, Paid, Balance, row menu) with **Add charge**, **Issue invoice (n)** for selected unbilled charges and **Collect payment**. **Invoices** and **Payments** list the patient's documents with print buttons. **Statement** shows a date range, a **Print statement** button and a running ledger (Date, Description, Debit, Credit, Balance) between "Opening balance" and "Closing balance", with the note "Positive balance = patient owes; negative = clinic holds advance."

Screenshot pending

Billing settings

C:/Program Files/Git/settings/billing

Figure 34.9 — Billing settings (screenshot pending).

Billing settings (`/settings/billing`) have the subtitle "Services, prices, discounts, payment accounts and printing", a back button "Back to settings" and four tabs. **Services and prices** holds the "Service catalog" table (Service, Kind, Default price, an Active switch, Edit) with **Add service**, and "Price overrides for this location" with an inline row (Service, doctor or "All doctors", Price, **Add override**) and a remove button per override. **Discount types** (only with the discounts feature) lists presets (Name, Value, Applies to, Reception range or "Fixed", Active, Edit) with **Add discount type**. **Payments and printing** holds the location's document code, due days, print formats, consultation fee service, default account, variance alert, footer note and three switches. **Payment accounts** lists accounts (Account with a "Default" badge, Kind, Bank, Rules, Active, Edit) with **Add account**.

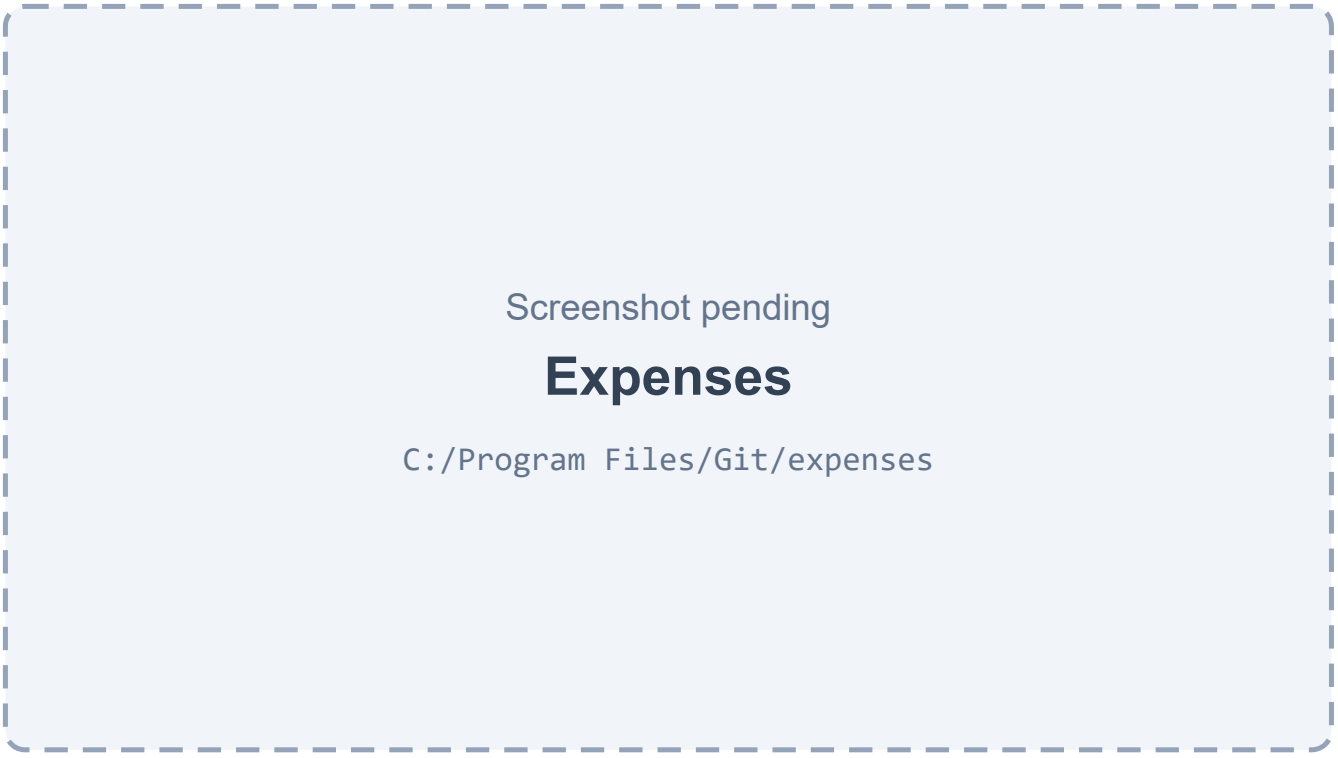

Figure 34.10 — Expenses page (screenshot pending).

The Expenses page (`/expenses`) has the subtitle "Track and categorise clinic spending" and an **Add Expense** button. A "This Month Total" card shows the current month's total and the three largest categories. Category buttons (All, Rent, Utilities, Supplies, Salary, Equipment, Marketing, Maintenance, Other) filter a table of Date, Category, Description, Method and Amount, with a delete icon per row. The page uses an older style: it shows the latest 30 rows with no paging, no search, no date filter, no card view and no edit action.

Screenshot pending

Billing activity

C:/Program Files/Git/billing/activity

Figure 34.11 — Billing activity (screenshot pending).

Billing activity (`/billing/activity`) has the subtitle "Every billing change with who, when and why." Date presets (Today by default, This month, Past 30 days, custom range) choose the period. A table "Per user per day" shows Date, User, Edits, Cancels, Refunds (count and amount) and Discounts (count and amount). The "Activity feed" lists Time, User, Action (for example Created, Voided, Discount, Price changed, Issued, Edited, Cancelled, Advance applied, Unapplied, Refunded, Counter closed), Document, Amount and Reason, 20 rows per page.

Screenshot pending

Print layouts A4 and 80 mm

browser print

Figure 34.12 — Printed documents in A4 and 80 mm thermal layout (screenshot pending).

Printing happens in the browser's print dialog. Every printed document has a header with the location logo (if enabled), clinic and location name, address and phone, and a footer with the footer note and "Computer-generated document" with the time. The invoice ("INVOICE") lists the lines, totals, the total in words, refunds and "Issued by", and carries a large "CANCELLED" or "REFUNDED" watermark when applicable. The receipt ("PAYMENT RECEIPT") shows the amount received, the amount in words, the account, reference, the charges it was applied to, the part kept as advance and what is still outstanding. The refund voucher, the patient statement and the cash-close slip have signature lines where a person hands over money. The A4 layout uses 12 mm margins; the thermal layout prints a 72 mm wide column on 80 mm paper.

34.3 Features

34.3.1 Keep a service catalog

Status: [Implemented]

What it does. Keeps the organization's list of billable services (consultations, procedures and other items) with a default price each. The catalog is shared by all locations of the organization.

How the user does it.

1. Open Settings → **Billing** → **Services and prices**.
2. Click **Add service**, fill **Name**, optional **Code**, **Kind** (Consultation, Procedure or Other) and **Default price (Rs)**, then **Save**.
3. Click the Edit icon to change a service, or flip the **Active** switch to stop offering it.

Behind the scenes — in simple terms. A price change never touches charges that already exist: each charge keeps a copy of its price. A service that is switched off can no longer be priced for new charges.

Behind the scenes — technically. `GET /billing/services` (optional `activeOnly`, `kind`), `POST /billing/services`, `PATCH /billing/services/:serviceId` → `CatalogServicesService` (class name of the entity: `CatalogService`, table `services`). Each write runs in a transaction and writes a `billing_audit_logs` row (entity `SERVICE`, action `CREATED` / `UPDATED`, organization level with no location). A unique violation becomes 409 `BILLING_DUPLICATE`.

Rules and validations.

- Name 2–120 characters, unique per organization; code up to 30 characters, stored in upper case, unique per organization when given.
- Default price 0 or more, at most 2 decimals.
- Services are never deleted; they are deactivated.

Who can do it. Read: `billing.desk.use`. Write: `admin.settings.manage`. Feature `billing.invoices`. Default: Admin (write and read); Receptionist and Accountant (read).

Data read and written. `services`, `billing_audit_logs`.

Effects on other modules. The consultation workspace's treatments can point to a service, which decides the automatic charge's price (Chapter 31).

Error cases. 409 `BILLING_DUPLICATE` "A service with this name or code already exists"; 404 "Service not found".

Technical references: `backend/src/billing/catalog/catalog.service.ts` · `CatalogServicesService`, `CatalogService` · `web-app/src/features/billing/components/settings/ServicesTab.tsx`

34.3.2 Set location and doctor price overrides

Status: [Implemented] — editing an existing override's price exists in the API but not on screen (remove and add again instead).

What it does. Lets a location charge a different price than the catalog default, either for all doctors or for one doctor. The **resolved price** of a service is the first that exists of: the doctor's price at this location, the location price, the default price.

How the user does it.

1. In **Services and prices**, under "Price overrides for this location", choose a Service, a doctor or "All doctors", and type a **Price (Rs)**.
2. Click **Add override**. Remove one with its remove button and confirm "Remove price override?".

Behind the scenes — in simple terms. When a charge is created, the Platform looks up the most specific price and copies it onto the charge.

Behind the scenes — technically. `GET/POST /billing/service-prices`, `PATCH /billing/service-prices/:id` (not used by the web app), `DELETE /billing/service-prices/:id`, `GET /billing/services/:serviceId/resolved-price?doctorId` (not used by the web app). `ChargePricingService.resolve` reads `service_prices` for (service, location, doctor) and (service, location, no doctor) and falls back to `services.default_price`; it returns the price level (`DOCTOR`, `LOCATION`, `DEFAULT`). An inactive service throws 422 `BILLING_SERVICE_INACTIVE`. Writes are audited (`SERVICE_PRICE`, `CREATED` / `PRICE_CHANGED` / `DELETED`); deleting is a hard delete.

Rules and validations.

- One location-wide override per service and location; one per service, location and doctor (partial unique indexes).
- The doctor must be staff of the active location.
- Price 0 or more, 2 decimals.

Who can do it. `admin.settings.manage`, feature `billing.invoices`. Default: Admin.

Data read and written. `service_prices`, `services`, `billing_audit_logs`.

Effects on other modules. Automatic charges after a consultation use the doctor's price.

Error cases. 409 `BILLING_DUPLICATE` "An override for this service and doctor already exists"; 422 `BILLING_SERVICE_INACTIVE` "... is no longer offered".

Technical references: `backend/src/billing/catalog/charge-pricing.service.ts` · `ChargePricingService.resolve`, `ServicePrice` · `web-app/src/features/billing/components/settings/ServicesTab.tsx`

34.3.3 Get automatic charges from a completed consultation

Status: [Implemented]

What it does. When a doctor completes a visit, the Platform creates a consultation charge and one charge per treatment recorded in the encounter, without anyone at the desk typing them. The patient then appears in **Ready to bill**.

How the user does it.

1. The doctor completes the visit in the queue or the consultation workspace (Chapter 30, Chapter 31).
2. Reception opens the Billing desk; the patient is listed under **Ready to bill**.

Behind the scenes — in simple terms. Figure 34.13 shows the decision. The consultation fee comes from the location's "Consultation fee service". If no price can be found, the charge is created at zero with the flag **Needs price**, so the doctor is never blocked by a missing catalog entry. If the doctor removes a treatment and completes again, its unbilled charge is voided with the reason "Treatment removed by doctor"; if that charge is already on an invoice, the money is kept and the charge is marked for review.

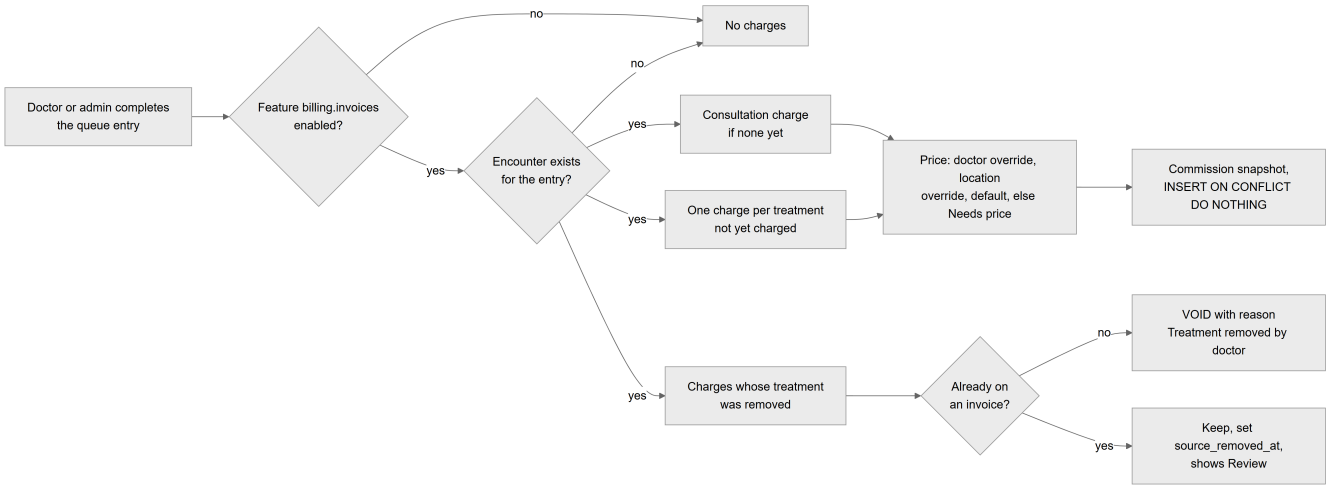

Figure 34.13 — How completing a visit creates and voids charges.

The flowchart starts at "Doctor or admin completes the queue entry". A first diamond asks whether the feature `billing.invoices` is enabled; "no" leads to "No charges". A second diamond asks whether an encounter exists for the entry; "no" again leads to "No charges". On "yes" three branches start. The first creates a "Consultation charge if none yet", the second "One charge per treatment not yet charged"; both go to the pricing box "Price: doctor override, location override, default, else Needs price" and then to "Commission snapshot, INSERT ON CONFLICT DO NOTHING". The third branch takes "Charges whose treatment was removed" to a diamond "Already on an invoice?": "no" leads to "VOID with reason Treatment removed by doctor", "yes" leads to "Keep, set source_removed_at, shows Review".

Behind the scenes — technically. `QueueOrchestrator.complete` (normal and force completion) calls `ChargeSourcesService.syncForEncounter` inside the queue's completion transaction, after the queue session lock. It reads the encounter by `queue_entry_id`, the encounter's `treatments` and its existing ACTIVE `CONSULTATION` / `TREATMENT` charges. `ChargeDraftFactory` builds drafts: `fromConsultation` (source id = encounter id, description from the service or "Consultation") and `fromTreatment` (source id = treatment id; price from the treatment's service, else the treatment's legacy cost, else zero with `needs_pricing`). The charge date is the completion instant converted to the location's date. `CommissionService.snapshot` adds the commission plan and rate in force on that date (Chapter 35). Rows are inserted with `ON CONFLICT DO NOTHING` on the partial unique index `uq_charges_source` (organization, source, source id, ACTIVE only), so repeating the completion never doubles a charge. Created and voided charges are audited with the system role.

Rules and validations.

- Runs only if the feature is enabled for the organization and the visit has an encounter.
- A catalog error (inactive service, missing service) never fails the completion; it yields a Needs price charge.
- Removed treatments: unbilled charge → `VOID`; billed charge → `source_removed_at` set, money untouched.

Who can do it. No user permission; it runs as part of `queue.doctor.own` / `queue.doctor.any` completion. Feature `billing.invoices`.

Data read and written. Reads `patient_encounters`, `treatments`, `charges`, `location_billing_settings`, `services`, `service_prices`, compensation tables. Writes `charges`, `billing_audit_logs`.

Effects on other modules. The doctor's response contains no charge data. Charges carry the encounter and appointment ids, which reports use.

Error cases. None shown to the doctor; pricing problems appear as "Needs price" at the desk.

Technical references: `backend/src/billing/charges/charge-sources.service.ts` · `ChargeSourcesService.syncForEncounter`, `ChargeDraftFactory` · `web-app/src/features/billing/pages/BillingDeskPage.tsx`

34.3.4 Add a manual charge

Status: [Implemented]

What it does. Adds a charge by hand: a catalog service priced automatically, or an "Other" item with free text and a price.

How the user does it.

1. In the Collect payment sheet or the patient's **Open charges**, click **Add charge**.
2. Choose **Catalog service** and pick a **Service**, or choose **Other** and type **Description** and **Price (Rs)**.
3. Set **Quantity** (1–999) and click **Add charge**. A toast says "Charge added".

Behind the scenes — in simple terms. A catalog service gets its resolved price for this location. A receptionist cannot type a different price for a catalog service.

Behind the scenes — technically. `POST /patients/:patientId/charges` → `ChargesService.addManual` in one transaction: checks the patient and optional doctor, resolves the price, builds the draft with `ChargeDraftFactory.manual` (source `MANUAL`, charge date = location's today unless given), applies `assertCanSetPrice` when a different price is sent for a catalog service, saves and audits `CHARGE` / `CREATED`. `GET /patients/:patientId/charges?state=unbilled|open|all` lists charges.

Rules and validations.

- "Other" needs a description of 2–200 characters and a price.
- Quantity 1–999; price 0 or more with 2 decimals.
- Manual charges carry no commission snapshot.

Who can do it. `billing.desk.use`, feature `billing.invoices`. Default: Admin, Receptionist, Accountant. A different catalog price needs `billing.price.override` (Admin).

Data read and written. `charges`, `services`, `service_prices`, `billing_audit_logs`.

Effects on other modules. None beyond billing.

Error cases. 422 `BILLING_CHARGE_INVALID` "Enter a price for this charge"; 403 `BILLING_PRICE_CHANGE_NOT_ALLOWED`; 422 `BILLING_SERVICE_INACTIVE`.

Technical references: `backend/src/billing/charges/charges.service.ts` · `ChargesService.addManual` · `web-app/src/features/billing/components/charges/AddChargeSheet.tsx`

34.3.5 Create charges from a treatment plan

Status: [Partial] — backend only; no screen calls the endpoint.

What it does. Turns selected items of a patient's treatment plan into unbilled charges, priced at each item's estimated cost.

How the user does it. Not available in the web app.

Behind the scenes — in simple terms. One charge per plan item; the same item can never be charged twice while its charge is active.

Behind the scenes — technically. `POST /patients/:patientId/charges/from-treatment-plan/:planId` with `itemIds` (1–100) → `ChargesService.fromTreatmentPlan`. Builds drafts with source `TREATMENT_PLAN` (doctor = item's assignee or the plan's dentist; price = estimated cost, else zero with Needs price) and inserts with `ON CONFLICT DO NOTHING` on `uq_charges_source`. `GET /treatment-plans/:planId/billing-summary` reports these charges per plan; its web hook exists but no component uses it.

Rules and validations. Items must belong to the plan, the plan to the patient and organization.

Who can do it. `billing.desk.use`, feature `billing.invoices`.

Data read and written. Reads `patient_treatment_plans`, `treatment_plan_items`; writes `charges`, `billing_audit_logs`.

Effects on other modules. Treatment plans (Chapter 33) are unchanged by it.

Error cases. 404 "Treatment plan not found", "Plan item not found".

Technical references: `backend/src/billing/charges/charges.controller.ts` · `ChargesService.fromTreatmentPlan` · none in the web app

34.3.6 Edit an unbilled charge (and the price policy)

Status: [Implemented]

What it does. Changes the quantity or price of a charge that is not yet on an invoice. The **price policy** says: without `billing.price.override`, only "Other" charges and charges that still need a price can be priced; for a catalog charge, give a discount instead.

How the user does it.

1. In the charge's three-dot menu, choose **Edit quantity / price**.
2. Change **Quantity** or **Unit price (Rs)** and click **Save**. If the price field is disabled, the hint says "Catalog price — give a discount instead."

Behind the scenes — in simple terms. Gross, discount and net are recalculated. A percentage discount follows the new amount; a fixed one stays, capped at the gross.

Behind the scenes — technically. `PATCH /charges/:chargeId` → `ChargesService.update` via `mutateUnbilled`: one transaction, locks the charge (and its invoice if any) with `BillingLocks.chargesWithInvoices`, asserts it is ACTIVE and unbilled, applies changes, runs `recalcCharge` (integer paisa math), saves, audits `UPDATED` with before and after snapshots (commission fields stripped). Setting a price clears `needs_pricing`. Changing the doctor re-takes the commission snapshot.

Rules and validations.

- Only ACTIVE, unbilled charges; billed charges change only through an invoice edit.
- Only "Other" charges can be renamed.
- Quantity 1–999.

Who can do it. `billing.desk.use` (+ `billing.price.override` for catalog prices), feature `billing.invoices`. Default: Admin, Receptionist, Accountant; price override Admin only.

Data read and written. `charges`, `billing_audit_logs`.

Effects on other modules. None.

Error cases. 409 `BILLING_CHARGE_ALREADY_BILLED` "This charge is on an invoice — edit the invoice instead"; 409 `BILLING_CHARGE_NOT_OPEN` "This charge is void"; 403 `BILLING_PRICE_CHANGE_NOT_ALLOWED` "You are not allowed to change a catalog price — give a discount instead".

Technical references: `backend/src/billing/charges/price-policy.ts` · `assertCanSetPrice`, `ChargesService.update` · `web-app/src/features/billing/components/charges/EditChargeSheet.tsx`

34.3.7 Void an unbilled charge

Status: [Implemented]

What it does. Cancels a charge that should not be billed, keeping it for the record.

How the user does it.

1. In the charge's menu, choose **Void**.
2. In "Void this charge?", type a **Reason** and click **Void charge**. A toast says "Charge voided".

Behind the scenes — in simple terms. The charge becomes VOID, with who, when and why. It no longer counts anywhere.

Behind the scenes — technically. `POST /charges/:chargeId/void` with `reason` → `ChargesService.void` through `mutateUnbilled`; sets `status = VOID`, `voided_at`, `voided_by`, `void_reason`; audits `VOIDED` with the reason.

Rules and validations. Reason 5–300 characters (screen and server). Only ACTIVE, unbilled charges. A void cannot be undone.

Who can do it. `billing.desk.use`, feature `billing.invoices`. Default: Admin, Receptionist, Accountant.

Data read and written. `charges`, `billing_audit_logs`.

Effects on other modules. A voided treatment charge does not change the clinical record.

Error cases. 400 validation for a short reason; 409 `BILLING_CHARGE_ALREADY_BILLED`; 409 `BILLING_CHARGE_NOT_OPEN`.

Technical references: `backend/src/billing/charges/charge-math.ts` · `voidCharge`, `ChargesService.void` · `web-app/src/features/billing/components/charges/ChargeRowActions.tsx`

34.3.8 Manage discount types

Status: [Implemented] — the optional Code field exists in the API but not on screen.

What it does. Lets the clinic define preset discounts ("Staff family 20%", "Senior citizen 10%", a fixed amount off) that reception may apply. A **discount type** has a kind (percentage or fixed), a value, an optional maximum amount per use, what it applies to (everything, some service kinds, some services), an optional range within which reception may change the value, an optional "note required" rule and optional validity dates.

How the user does it.

1. Open Settings → **Billing** → **Discount types**, click **Add discount type**.
2. Fill **Name**, **Kind**, **Value**, optionally **Max amount per use (Rs)**, **Applies to**, the switch **Reception may change the value** (then **Min value** and **Max value**), the switch **Note required when applied**, **Valid from** and **Valid to**.
3. Save. Use the **Active** switch to retire a type.

Behind the scenes — in simple terms. Charges keep a copy of the discount they got; changing a type later never changes them.

Behind the scenes — technically. `GET /billing/discount-types?usableToday` (list, `billing.discount.apply`), `POST` and `PATCH /billing/discount-types/:id` → `DiscountTypesService`, transactional and audited (`DISCOUNT_TYPE`). `assertShape` checks the merged row so a PATCH cannot break the rules. "Usable today" means active and the location's today within the validity dates.

Rules and validations.

- Name 2–80 characters, unique per organization; value at least 0.01; a percentage at most 100.
- "Service kinds" needs at least one kind; "Specific services" at least one service.
- Reception range must satisfy min ≤ value ≤ max; valid from ≤ valid to.
- Types are never deleted.

Who can do it. Write: `admin.settings.manage`; read: `billing.discount.apply`. Features `billing.invoices` and `billing.discounts`. Default: Admin.

Data read and written. `discount_types`, `billing_audit_logs`.

Effects on other modules. None.

Error cases. 422 `BILLING_DISCOUNT_INVALID` (for example "Percentage must be between 0 and 100", "Reception range must satisfy min ≤ value ≤ max"); 409 `BILLING_DUPLICATE`.

Technical references: `backend/src/billing/discounts/discount-types.service.ts` · `DiscountTypesService.assertShape` · `web-app/src/features/billing/components/settings/DiscountTypesTab.tsx`

34.3.9 Give a discount on one charge, many charges, or a custom discount

Status: [Implemented]

What it does. Applies one discount to an unbilled charge (each charge carries at most one; a new one replaces the old), applies one type to several charges at once, removes a discount, or gives a **custom discount** (any percentage or amount, not from a preset).

How the user does it.

1. One charge: menu → **Discount**. Several: tick unbilled charges and click **Discount** in the Collect payment sheet ("Discount n charges").
2. Choose a **Discount** type. If the type allows it, change the value within "Value (min–max)".
3. Or switch on **Custom discount** (admins) and choose **Kind** and **Value**.
4. Add a **Note** if required and click **Apply discount**. Remove with **Remove discount**.

Behind the scenes — in simple terms. The amount is computed, capped by the type's maximum and by the charge's gross. For many charges, a percentage applies to each charge and the maximum is shared; a fixed amount is one total spread over the charges in proportion to their size. Charges the type does not apply to are skipped and named in a warning ("Not applied to: …").

Behind the scenes — technically. `POST /charges/:chargeId/discount`, `DELETE /charges/:chargeId/discount`, `POST /charges/discount/bulk` (1–100 charge ids). `DiscountApplier.resolve` loads the type and calls `resolveDiscount`, which enforces the permission rules: a preset needs `billing.discount.apply`; a value different from the type's own goes through `AccessContext.decide('billing.discount.apply', { value, min, max })`, allowed inside the reception range, otherwise only with the escalation permission `billing.discount.custom`.

`discountForCharge` / `discountsForCharges` compute paisa amounts (`splitProportionally` gives leftover paisa to the largest share). The bulk call locks all charges in id order in one transaction and writes one `DISCOUNT_APPLIED` audit row per charge with the amount and note.

Rules and validations.

- Only ACTIVE, unbilled charges.
- The type must be active and valid on the location's today, and must apply to the charge (single charge: error; bulk: skipped).
- Value inside the type's range for reception; percentage 0–100 for anyone.
- Custom discount: needs `billing.discount.custom` and a note of at least 3 characters; it is named "Custom".
- Types marked "note required" need a note of at least 3 characters.

Who can do it. `billing.discount.apply` (presets), `billing.discount.custom` (custom or out-of-range values; implies apply). Feature `billing.discounts`. Default: Admin (both), Receptionist (apply). The Accountant template holds neither.

Data read and written. `charges`, `discount_types`, `billing_audit_logs`.

Effects on other modules. Discounts lower the commission base (Chapter 35) and appear in Doctor revenue.

Error cases. 403 `BILLING_CUSTOM_DISCOUNT_ADMIN_ONLY`; 403 `BILLING_DISCOUNT_VALUE_NOT_ALLOWED` "Value must be between … and …"; 422 `BILLING_DISCOUNT_NOTE_REQUIRED`; 422 `BILLING_DISCOUNT_INVALID` "… does not apply to …", "… is not available today".

Technical references: `backend/src/billing/discounts/discount-policy.ts` · `resolveDiscount`, `DiscountApplier`, `discountsForCharges` · `web-app/src/features/billing/components/charges/DiscountSheet.tsx`

34.3.10 Work the ready-to-bill list and the billing desk

Status: [Implemented]

What it does. Gives reception one screen with everyone to bill today and everyone who owes money.

How the user does it.

1. Open **Billing desk**. Optionally pick another day.
2. Click **Bill** on a ready-to-bill patient, or **Collect** on an outstanding invoice.

Behind the scenes — in simple terms. A patient is "ready to bill" on a day when they have at least one unbilled charge dated that day; their total includes older unbilled charges too.

Behind the scenes — technically. `GET /billing/ready-to-bill?date` → `ChargesService.readyToBill`: one SQL query over ACTIVE, unbilled charges with `charge_date ≤ day`, grouped by patient, `HAVING BOOL_OR(charge_date = day)`, newest first; the default day is the location's today from `BillingClock`. The Outstanding section calls `GET /invoices?paymentStatus=OVERDUE` and `?paymentStatus=UNPAID` (50 each).

Rules and validations. Active location only.

Who can do it. `billing.desk.use`, feature `billing.invoices`. Default: Admin, Receptionist, Accountant.

Data read and written. Reads `charges`, `patients`, `invoices`.

Effects on other modules. None.

Error cases. 403 `NO_PERMISSION`.

Technical references: `backend/src/billing/charges/charges.service.ts` · `ChargesService.readyToBill` · `web-app/src/features/billing/pages/BillingDeskPage.tsx`

34.3.11 Collect a payment (with preview and allocation)

Status: [Implemented]

What it does. Records money received from a patient, issues an invoice for any unbilled charges in the selection, splits the money over the selected charges, keeps any rest as advance and gives a receipt number.

How the user does it.

1. Open the Collect payment sheet (Billing desk, patient Billing tab, queue **Bill** button, invoice detail or invoice list menu).
2. Tick the charges to pay, check **Amount received (Rs)**, pick **Oldest first** or **Manual**, choose the account under **Received in**, add a **Reference number** if asked.
3. Read the live summary, then click **Receive Rs ...**.
4. Print the receipt and the invoice from the result screen.

Behind the scenes — in simple terms. Figure 34.14 shows the full exchange. The preview runs the same calculation without saving, so the screen never does money math itself. On save, everything happens in one database transaction: either all of it (invoice, receipt, allocations, totals, ledger, audit) is stored, or nothing is. Sending the same click twice returns the first result instead of charging twice.

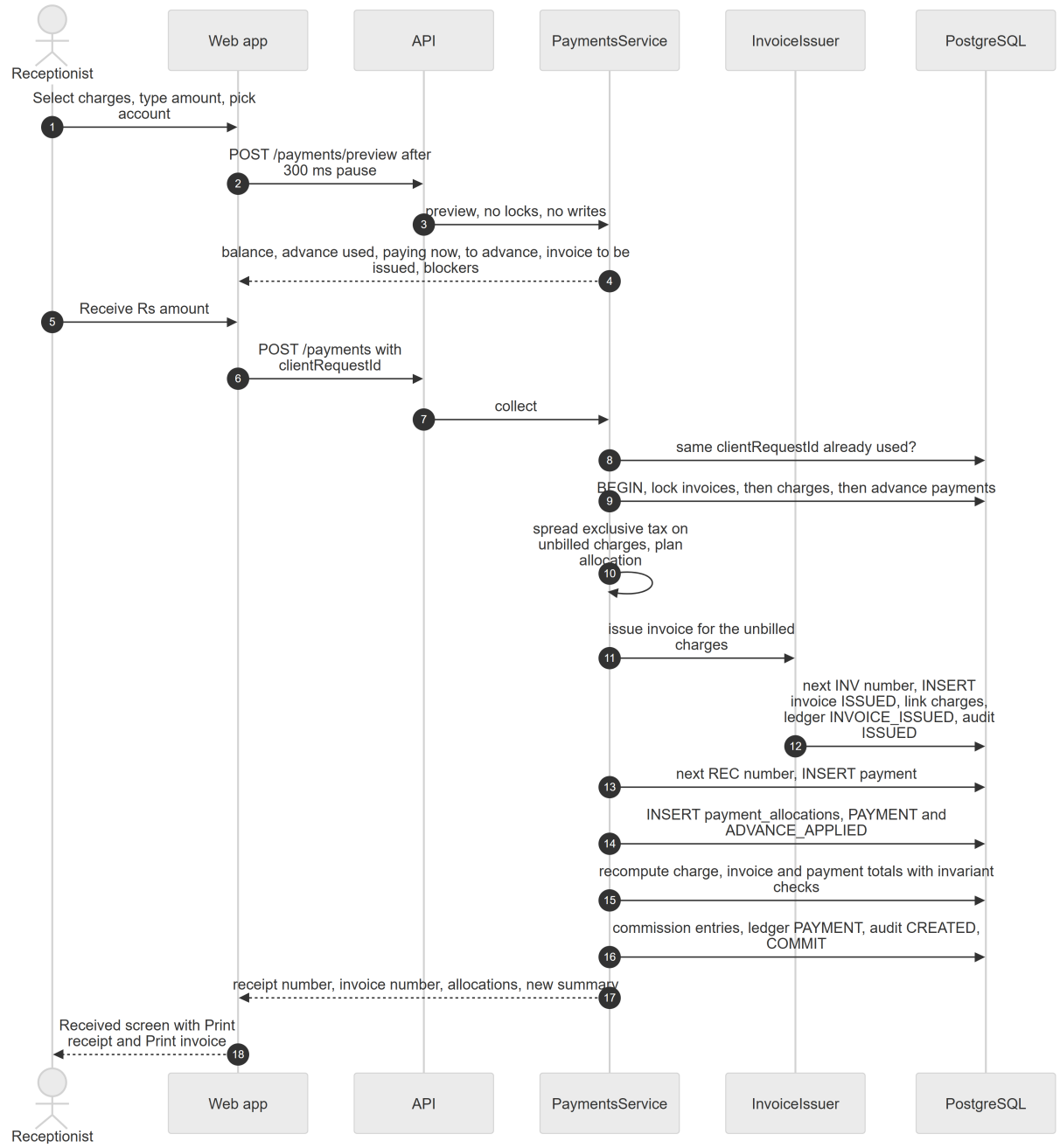

Figure 34.14 — Collect payment: preview, then one transaction that issues the invoice, records the payment and allocates it.

The diagram has six participants: the Receptionist, the Web app, the API, the PaymentsService, the InvoiceIssuer and PostgreSQL. In steps 1 to 4 the receptionist selects charges, types an amount and picks an account; after a 300 ms pause the web app calls `POST /payments/preview`; the service computes without locks or writes and returns the balance, advance used, paying now, the part going to advance, the invoice that will be issued and any blockers. In steps 5 to 7 the receptionist clicks "Receive Rs amount", the web app sends `POST /payments` with a `clientRequestId`, and the API calls collect. Step 8 checks whether that request id was already used. Step 9 opens the transaction and locks invoices, then charges, then the patient's advance payments. Step 10 spreads the exclusive tax on the unbilled charges and plans the allocation. Steps 11 and 12 hand the unbilled charges to the InvoiceIssuer, which takes the next INV number, inserts the invoice as ISSUED, links the charges, posts an INVOICE_ISSUED ledger row and an ISSUED audit row. Step 13 takes the next REC number and inserts the payment. Step 14 inserts the allocation rows with reasons PAYMENT and ADVANCE_APPLIED. Step 15 recomputes charge, invoice and payment totals with invariant checks. Step 16 records commission entries, posts the

PAYMENT ledger row, writes the CREATED audit row and commits. Steps 17 and 18 return the receipt number, invoice number, allocations and the new summary, and the screen shows the result with Print receipt and Print invoice.

Behind the scenes — technically. `POST /payments/preview` → `PaymentsService.preview`; `POST /payments` → `PaymentsService.collect` → `runCollect`. Locks follow `BillingLocks` order invoices → charges → payments (each by id) so two operations cannot deadlock. `assertCollectable` requires every selected charge to belong to the patient, be ACTIVE, have a balance (or await a price) and not sit on a cancelled invoice. `planAllocation` (see 34.3.12) draws first from old advance (oldest payment first), then from the new money. `InvoiceIssuer.issue` runs only when unbilled charges are selected. `AllocationWriterService.insert` appends rows and asserts each allocated charge is on an invoice. `BillingTotalsService` recomputes cached columns from source rows with SQL `SUM` (never increments) and throws `BILLING_INVARIANT_VIOLATION` (rolling everything back) if a charge is over-allocated, a payment over-used or an invoice balance negative. `CommissionService.recordForCharges` records commission on collection. Money crosses the API as rupees with two decimals, is stored as `numeric(12,2)`, and every calculation runs in integer paisa. Idempotency: the unique `client_request_id` per organization; a losing parallel retry returns the winner's result.

Rules and validations.

- New money: amount at least 0.01, an active account of this location, a reference when the account requires one, notes up to 300 characters.
- A charge with Needs price blocks the invoice ("Set a price for: …").
- Manual mode: every selected charge needs an amount, no amount above its balance, total not above the funds.
- Up to 200 selections.

Who can do it. `billing.payment.collect` (preview and collect), feature `billing.invoices`. Default: Admin, Receptionist, Accountant. The doctor-share line in the preview needs `earnings.all.view` and the commissions feature.

Data read and written. Writes `payments`, `payment_allocations`, `invoices`, `charges`, `document_sequences`, `ledger_entries`, `billing_audit_logs`, `commission_entries`.

Effects on other modules. Commission entries (Chapter 35); daily collection, dashboards and reports (Chapter 36).

Error cases. 422 `BILLING_LOCATION_CODE_MISSING`; 422 `BILLING_REFERENCE_REQUIRED`; 422 `BILLING_PAYMENT_ACCOUNT_INVALID`; 422 `BILLING_CHARGE_NEEDS_PRICING`; 409 `BILLING_CHARGE_NOT_OPEN` "… has nothing left to pay"; 409 `BILLING_INVOICE_CANCELLED`; 422 `BILLING_ALLOCATION_EXCEEDS_BALANCE` / `_EXCEEDS_FUNDS`; 500 `BILLING_INVARIANT_VIOLATION`.

Technical references: `backend/src/billing/payments/payments.service.ts` · `PaymentsService.collect`, `BillingLocks`, `BillingTotalsService` · `web-app/src/features/billing/components/collect-payment/CollectPaymentSheet.tsx`

34.3.12 Choose an allocation strategy

Status: [Implemented]

What it does. Decides how much of the available money each selected charge receives. **Oldest first** (code `AUTO`, FIFO) pays the oldest charge first; **Manual** pays exactly what the user typed per charge.

How the user does it. In the Collect payment sheet, choose **Oldest first** or **Manual** under **Allocation**.

Behind the scenes — in simple terms. Oldest first walks the charges from the oldest charge date (then creation time) and gives each the smaller of its balance and the money left. Whatever is not used becomes advance.

Behind the scenes — technically. `planAllocation` (pure function) uses `ALLOCATION_STRATEGIES`: `FifoAllocationStrategy` and `ManualAllocationStrategy`. Each strategy returns a target per charge; the planner then draws each target from the funding sources in order (old advance payments, oldest received first, then `NEW`). It returns the planned lines, the advance used and the unused part of the new payment.

Rules and validations. Manual amounts may not exceed a charge's balance or the total funds.

Who can do it. As for collecting a payment.

Data read and written. None by itself; the plan is written by collect.

Effects on other modules. None.

Error cases. 422 `BILLING_ALLOCATION_EXCEEDS_BALANCE` "An amount is larger than the charge balance"; 422 `BILLING_ALLOCATION_EXCEEDS_FUNDS` "The amounts add up to more than the money available".

Technical references: `backend/src/billing/payments/allocation/allocation-planner.ts` · `planAllocation`, `FifoAllocationStrategy`, `ManualAllocationStrategy` · `web-app/src/features/billing/components/collect-payment/CollectPaymentFields.tsx`

34.3.13 Hold advance and apply it later

Status: [Implemented]

What it does. Money received beyond what the selected charges need stays as the patient's **advance**. Advance is used on later charges, automatically when "Use advance" is on, or alone with **Apply advance** when no new money is received.

How the user does it.

1. In the Collect payment sheet, leave **Use advance (... available)** on (the default).
2. To use only advance, set the amount to 0; the button becomes **Apply advance Rs ...**.

Behind the scenes — in simple terms. Advance is not a separate balance: it is the unused part of the patient's previous payments. Applying it issues an invoice for unbilled charges but no new receipt.

Behind the scenes — technically. Advance of a payment = `amount − allocated_amount − refunded_amount`. `POST /patients/:patientId/advance/apply` → `PaymentsService.applyAdvance` runs `runCollect` with amount 0; it writes `ADVANCE_APPLIED` allocations, audits `ALLOCATED`, and posts no PAYMENT ledger row.

Rules and validations. Fails when there is no advance to apply.

Who can do it. `billing.payment.collect`, feature `billing.invoices`. Default: Admin, Receptionist, Accountant.

Data read and written. `payments`, `payment_allocations`, `charges`, `invoices`, `billing_audit_logs`.

Effects on other modules. Commission is recorded when advance is applied.

Error cases. 422 `BILLING_NOTHING_TO_APPLY` "There is no advance to apply to these charges".

Technical references: `backend/src/billing/payments/payments.service.ts` · `PaymentsService.applyAdvance` · `web-app/src/features/billing/components/collect-payment/CollectSummary.tsx`

34.3.14 Unapply a payment

Status: [Partial] — backend only; the web app defines the call but no screen uses it.

What it does. Takes back all or part of a payment's allocations; the money returns to the patient's advance. It is needed before an invoice with payments can be cancelled.

How the user does it. Not available in the web app.

Behind the scenes — in simple terms. Negative allocation rows are added; nothing is deleted. The receipt stays.

Behind the scenes — technically. `POST /payments/:paymentId/unapply` with `reason` (5–300) and optional `lines` → `PaymentsService.unapply`: locks charges with invoices, then the payment; `AllocationWriterService.releaseFromPayment` inserts negative `UNAPPLY` rows (never more than the current net); totals are recomputed; commission is reversed with a negative entry; audit `UNAPPLIED`. No ledger entry is posted, because the patient's net position does not change.

Rules and validations. Each line at most the payment's current net on that charge.

Who can do it. `billing.invoice.edit`, feature `billing.invoices`. Default: Admin, Accountant.

Data read and written. `payment_allocations`, `charges`, `invoices`, `payments`, `commission_entries`, `billing_audit_logs`.

Effects on other modules. Commission reversal (Chapter 35).

Error cases. 422 `BILLING_UNAPPLY_EXCEEDS_ALLOCATION` "Cannot release more than was applied"; 404 "Payment not found".

Technical references: `backend/src/billing/payments/allocation-writer.service.ts` · `PaymentsService.unapply` · none in the web app

34.3.15 Manage payment accounts and payment methods

Status: [Implemented]

What it does. Defines where the location receives money. Each **payment account** has a kind (the payment method: Cash, Bank transfer, Card, Online wallet, Cheque), optional bank details, and two rules: "Reference number required" and "Count physically at cash close". Every location gets a default account named **Cash** automatically.

How the user does it.

1. Open Settings → **Billing** → **Payment accounts**, click **Add account**.
2. Fill **Name**, **Kind** (fixed after creation), optional **Bank name**, **Account title**, **Account number**, and the two switches. For example, "Example Bank current account" with kind Bank transfer.
3. Use the **Active** switch to retire an account. Choose the default account in **Payments and printing**.

Behind the scenes — in simple terms. Payments and refunds copy the account's name and method, so old receipts keep the name they were taken with.

Behind the scenes — technically. `GET/POST /billing/payment-accounts`, `PATCH /billing/payment-accounts/:id` → `PaymentAccountsService`. Listing first calls `BillingSettingsService.getOrCreate`, which inserts the settings row and the `Cash` account (`ON CONFLICT DO NOTHING`) and makes it the default. New Cash accounts default to "no reference, counted"; other kinds to "reference, not counted". `resolveForUse` checks active, same location and reference.

Rules and validations.

- Name 2–60 characters, unique per location.
- The default account cannot be deactivated; the last active account cannot be deactivated.
- Deactivated accounts cannot receive payments or pay refunds.

Who can do it. Read: `billing.desk.use`; write: `admin.settings.manage`. Feature `billing.invoices`. Default: Admin (write); Receptionist and Accountant (read).

Data read and written. `payment_accounts`, `location_billing_settings`, `billing_audit_logs`.

Effects on other modules. Cash close groups by account.

Error cases. 409 `BILLING_PAYMENT_ACCOUNT_IS_DEFAULT` "Choose another default account before deactivating this one"; 422 `BILLING_PAYMENT_ACCOUNT_INVALID` "A location needs at least one active account"; 409 `BILLING_DUPLICATE`.

Technical references: `backend/src/billing/payment-accounts/payment-accounts.service.ts` · `PaymentAccountsService.assertCanDeactivate` · `web-app/src/features/billing/components/settings/PaymentAccountsTab.tsx`

34.3.16 Number invoices, receipts and refunds

Status: [Implemented]

What it does. Gives every invoice, receipt and refund a gapless number of the form `CODE-INV-YYYY-NNNNN` (receipts `REC`, refunds `RFD`), for example `KHI01-INV-2026-00001`. The counter is per location, per document type and per year (the year of the location's date).

How the user does it. Nothing to do once the **Document code** is set in billing settings.

Behind the scenes — in simple terms. If the transaction that took a number fails, the number goes back, so there are no gaps.

Behind the scenes — technically. `DocumentSequenceService.next` runs an `INSERT … ON CONFLICT … DO UPDATE SET last_value = last_value + 1 RETURNING` on `document_sequences` (primary key organization, location, type, year). The upsert row-locks the counter until the transaction ends, so parallel collections queue up. The code comes from `location_billing_settings.document_code`; when the settings row is first created the Platform tries to derive it from the location's code. Numbers are unique per organization (`uq_invoices_org_number`, `uq_payments_org_receipt`, `uq_refunds_org_number`). Applying advance issues no receipt number.

Rules and validations. Without a document code nothing can be numbered.

Who can do it. Runs inside collect, issue and refund.

Data read and written. `document_sequences`, `location_billing_settings`.

Effects on other modules. None.

Error cases. 422 `BILLING_LOCATION_CODE_MISSING` "Set a document code for this location in billing settings first".

Technical references: `backend/src/billing/numbering/document-sequence.service.ts` · `DocumentSequenceService.next` · `web-app/src/features/billing/components/settings/BillingSettingsForm.tsx`

34.3.17 Issue an invoice (at payment or on its own)

Status: [Implemented]

What it does. Creates an invoice for unbilled charges. This happens automatically when payment is collected or advance applied, and by hand with **Issue invoice (n)** when a patient leaves without paying.

How the user does it.

1. Usually nothing: collecting payment issues the invoice.

2. For a patient who pays later: patient **Billing** tab → **Open charges**, tick unbilled charges, click **Issue invoice (n)**. A toast says "Invoice … issued".

Behind the scenes — in simple terms. The invoice takes a number, fixes the tax rate of the location at that moment, gets a due date (issue date plus the location's due days, 7 by default) and posts a debit on the patient's statement.

Behind the scenes — technically. `POST /invoices` (`patientId` , 1–100 `chargeIds` , optional notes) → `InvoicesService.create` → `InvoiceIssuer.issue` inside a transaction: asserts charges are ACTIVE, unbilled, of one patient, priced; reads the tax rate with `locationTaxRate` ; `applyInvoiceTax` ; next INV number; inserts the invoice (`status ISSUED` , `revision 1` , `issue_date` , `due_date = today + due_days` , `tax_rate`); links charges and writes their tax shares; recomputes totals; ledger `INVOICE_ISSUED` ; audit `ISSUED` with a snapshot.

Rules and validations. At least one charge; Needs price charges cannot be invoiced.

Who can do it. `billing.desk.use` for the manual issue; feature `billing.invoices` . Default: Admin, Receptionist, Accountant.

Data read and written. `invoices` , `charges` , `document_sequences` , `location_settings` (tax rate), `ledger_entries` , `billing_audit_logs` .

Effects on other modules. The invoice counts in Outstanding until paid.

Error cases. 422 `BILLING_CHARGE_NEEDS_PRICING` "… needs a price first"; 409 `BILLING_CHARGE_NOT_OPEN` "… cannot be invoiced".

Technical references: `backend/src/billing/invoices/invoice-issuer.service.ts` · `InvoiceIssuer.issue` · `web-app/src/features/billing/components/patient-billing/OpenChargesPanel.tsx`

34.3.18 Apply exclusive sales tax

Status: [Implemented]

What it does. Adds the location's sales tax on top of the invoice. Tax is **exclusive**: prices are before tax.

How the user does it. Set the rate in the location settings ("Tax Rate (%)" on the location's settings page, or "Sales tax on invoices (%)" in the setup wizard). Billing settings do not contain the rate.

Behind the scenes — in simple terms. Tax = rate × (sum of the active charges' nets), rounded once. Because payments go to charges, the tax is shared out over the charges. The rate is frozen on the invoice; later edits use the frozen rate, not the current one.

Behind the scenes — technically. `locationTaxRate` reads `location_settings.tax_rate` (no row → 0), normalised to basis points. `splitInvoiceTax` gives each charge the tax on its own net and puts the rounding difference on the largest charge; `basisPointsOf` uses BigInt half-up rounding. A charge's payable = net + tax. Tax is not revenue: commission and revenue reports use the pre-tax part (`preTaxPaisa`).

Rules and validations. Rate 0–100 with 2 decimals. A cancelled invoice keeps its frozen totals; its charges return to unbilled with tax 0.

Who can do it. Setting the rate: `admin.locations.manage` (location settings).

Data read and written. `location_settings` , `invoices.tax_rate` , `invoices.tax_amount` , `charges.tax_amount` .

Effects on other modules. The finance reports subtract tax collected from revenue (Chapter 36).

Error cases. None specific.

Technical references: `backend/src/billing/tax/invoice-tax.ts` · `applyInvoiceTax` , `locationTaxRate` · `web-app/src/features/locations/LocationSettingsFeature.tsx`

34.3.19 Browse invoices and see their payment status

Status: [Implemented]

What it does. Lists and shows invoices with a status that follows payments and due dates. The stored status is only `ISSUED` or `CANCELLED`; the **payment status** (Unpaid, Partially paid, Paid, Overdue) and the **refund status** are computed each time.

How the user does it.

1. Open **Invoices**; use the status tabs, search, dates, list or cards.
2. Click a row to open the detail page.

Behind the scenes — in simple terms. An invoice is Paid when nothing is left to pay, Overdue when money is still owed after the due date, Partially paid when some was paid, otherwise Unpaid.

Behind the scenes — technically. `GET /invoices` (paging up to 100, `paymentStatus`, `patientId`, `from`, `to`, `search`), `GET /invoices/stats`, `GET /invoices/:invoiceId`, `GET /invoices/:invoiceId/revisions/:revision` (not used by the web app). `invoicePaymentStatus` and the SQL twin `paymentStatusSql` use the location's today. Row scope of `billing.invoice.view`: `location` / `org` show the active location's invoices; `own` shows invoices that carry one of the signed-in doctor's charges.

Rules and validations. Always the active location.

Who can do it. `billing.invoice.view` (scoped), feature `billing.invoices`. Default: Admin (`org`), Receptionist and Accountant (`location`).

Data read and written. Reads `invoices`, `charges`, `patients`, `payment_allocations`, `refunds`.

Effects on other modules. None.

Error cases. 404 "Invoice not found".

Technical references: `backend/src/billing/invoices/invoice-status.ts` · `invoicePaymentStatus`, `InvoiceQueries` · `web-app/src/features/billing/pages/InvoicesPage.tsx`

34.3.20 Edit an issued invoice (revisions)

Status: [Implemented]

What it does. Changes an issued invoice: quantities, prices, discounts, void lines, add unbilled charges, add a new manual line. Each save raises the revision number; the invoice number never changes. Money that is no longer needed moves to the patient's advance.

How the user does it.

1. On the invoice detail page click **Edit**.
2. Change lines, tick **Void** or add unbilled charges, type a **Reason**.
3. Read the preview, then click **Save changes** (or **Save & refund** to pay back freed money). A toast says "Saved — revision n".

Behind the scenes — in simple terms. The preview really performs the edit in a transaction that is then thrown away, so the preview is exactly what saving will do. If two people edit at once, the second gets "changed by someone else".

Behind the scenes — technically. `POST /invoices/:invoiceId/edit/preview` and `POST /invoices/:invoiceId/edit` (`expectedRevision`, 1–50 `operations`, `reason` 5–300) → `InvoicesService.runEdit`: lock the invoice, compare `revision` (optimistic check), lock its charges plus added ones, apply typed commands

through `InvoiceEditHandlers` (`updateQuantity`, `setUnitPrice`, `setDiscount`, `removeDiscount`, `voidCharge`, `addCharges`, `addManualCharge`), re-spread tax at the snapshot rate, release any allocation above a charge's new payable (newest allocation first, reason `INVOICE_EDIT_RELEASE`), bump `revision`, recompute totals, adjust commission, post ledger `INVOICE_ADJUSTED` with the difference, audit `EDITED` with full before and after snapshots. Older revisions are printed from these audit snapshots.

Rules and validations.

- Only ISSUED invoices.
- `setUnitPrice` follows the price policy; `voidCharge` needs `billing.invoice.void`; discount operations follow the discount rules.
- Added charges must be unbilled, of the same patient and priced.

Who can do it. `billing.invoice.edit` (+ `billing.invoice.void` to void lines, + `billing.price.override` for catalog prices), feature `billing.invoices`. Default: Admin, Accountant.

Data read and written. `invoices`, `charges`, `payment_allocations`, `payments`, `ledger_entries`, `commission_entries`, `billing_audit_logs`.

Effects on other modules. Commission adjusts to the new amounts.

Error cases. 409 `BILLING_INVOICE_REVISION_CONFLICT` "This invoice was changed by someone else. Reload and try again."; 409 `BILLING_INVOICE_CANCELLED` "This invoice is cancelled and read-only"; 404 "Charge is not on this invoice".

Technical references: `backend/src/billing/invoices/invoices.service.ts` · `InvoicesService.runEdit`, `InvoiceEditHandlers` · `web-app/src/features/billing/components/invoices/EditInvoiceSheet.tsx`

34.3.21 Cancel an invoice

Status: [Implemented] — an invoice with payments must first be unapplied, which only the API can do.

What it does. Cancels an unpaid invoice. Its charges go back to "not invoiced"; the number stays used.

How the user does it.

1. On the invoice detail page click **Cancel** (shown only when nothing is paid).
2. In "Cancel … ?" type a **Reason** and click **Cancel invoice**.

Behind the scenes — in simple terms. The invoice keeps its frozen totals for the record and is printed with a CANCELLED watermark. A credit is posted on the patient's statement.

Behind the scenes — technically. `POST /invoices/:invoiceId/cancel` with `reason` → `InvoicesService.cancel`: lock the invoice and its charges, refuse if any allocation or `paid_amount` exists, set `CANCELLED` with `cancelled_at/by` and `cancel_reason`, set active charges' `invoice_id` to null and tax to 0, ledger `INVOICE_CANCELLED` (minus the total), audit `CANCELLED`.

Rules and validations. ISSUED and nothing paid. Final: a cancelled invoice cannot be reopened.

Who can do it. `billing.invoice.void` (high risk), feature `billing.invoices`. Default: Admin, Accountant.

Data read and written. `invoices`, `charges`, `ledger_entries`, `billing_audit_logs`.

Effects on other modules. The charges reappear in Ready to bill.

Error cases. 409 `BILLING_INVOICE_HAS_PAYMENTS` "Unapply its payments before cancelling this invoice".

Technical references: `backend/src/billing/invoices/invoices.service.ts` · `InvoicesService.cancel` · `web-app/src/features/billing/pages/InvoiceDetailPage.tsx`

34.3.22 Refund money (from advance or from an invoice)

Status: [Implemented]

What it does. Pays money back to a patient. A refund always comes out of advance. An **advance refund** returns unused money; an **invoice refund** returns money released from an invoice, optionally together with the edit that released it, in one step.

How the user does it.

1. Advance: patient **Billing** tab → **Refund advance**. Invoice: invoice detail → **Refund**, or **Save & refund** in the Edit sheet.
2. Fill **Amount (Rs)**, **Paid out from**, **Reference number** if asked, and a **Reason**.
3. Click **Refund…**, confirm "Return … via … to …?". A toast says "Refund … recorded"; print the voucher for the patient to sign.

Behind the scenes — in simple terms. The Platform decides which previous payments the refund comes from (its **refund sources**): for an invoice refund, first the payments released from that invoice (latest first), then any other advance, newest first.

Behind the scenes — technically. `POST /refunds` (`patientId`) → `RefundsService.createAdvanceRefund` ; `POST /invoices/:invoiceId/refund` with optional `edit` and `refund` → `createInvoiceRefund` , which runs `InvoicesService.runEdit` and the refund in the same transaction. `runRefund` : account check, invoice refundable check (`released_amount − refunded_amount`), lock the patient's advance payments, check available advance, pick sources, take the next RFD number, insert `refunds` and `refund_sources` , recompute payment and invoice totals, ledger `REFUND` (a debit, because the clinic now holds less of the patient's money), audit `REFUNDED` on the refund and, for invoice refunds, on the invoice. Idempotent by `clientRequestId` . `GET /refunds` and `GET /refunds/:refundId` exist but the web app does not call them.

Rules and validations.

- Amount at least 0.01, at most the advance; for an invoice refund, at most the invoice's refundable amount.
- Reason 5–300 characters; reference when the account requires one.
- Refunds cannot be undone.

Who can do it. `billing.refund.issue` (high risk), feature `billing.invoices` . Default: Admin, Accountant.

Data read and written. `refunds` , `refund_sources` , `payments` , `invoices` , `document_sequences` , `ledger_entries` , `billing_audit_logs` .

Effects on other modules. Payments page and cash close subtract refunds per account.

Error cases. 422 `BILLING_REFUND_EXCEEDS_ADVANCE` "The patient's advance is only Rs …"; 422 `BILLING_REFUND_EXCEEDS_INVOICE_REFUNDABLE` "Only Rs … can be refunded from this invoice"; 422 `BILLING_REFERENCE_REQUIRED` .

Technical references: `backend/src/billing/refunds/refunds.service.ts` · `RefundsService.runRefund` , `sourceOrder` · `web-app/src/features/billing/components/refunds/RefundSheet.tsx`

34.3.23 See a patient's billing: summary, open charges, statement

Status: [Implemented]

What it does. Shows one patient's money position: not invoiced, outstanding, overdue and advance; the open charges; and a statement (a running ledger of invoices, adjustments, cancellations, payments and refunds).

How the user does it. Open the patient → **Billing** tab; switch sub-tabs; on **Statement** choose a range and **Print statement**.

Behind the scenes — in simple terms. The statement comes from an append-only patient ledger: an invoice adds a debit, a payment a credit. Its balance equals outstanding minus advance.

Behind the scenes — technically. `GET /patients/:patientId/billing/summary`, `/open-charges`, `/statement?from&to` → `PatientBillingService`. The summary query sums unbilled nets, issued balances, overdue balances (due date before the location's today), payment advance and the ledger. The statement reads `ledger_entries` with an opening balance before `from`. `LedgerService.post` only inserts rows (debit for positive, credit for negative, zero skipped).

Rules and validations. Active location only.

Who can do it. `billing.invoice.view`, feature `billing.invoices`. Default: Admin, Receptionist, Accountant.

Data read and written. Reads `charges`, `invoices`, `payments`, `ledger_entries`.

Effects on other modules. The Billing tab appears on the patient record (Chapter 28).

Error cases. 404 when the patient is not in the organization.

Technical references: `backend/src/billing/patient-billing/patient-billing.queries.ts` · `PatientBillingQueries.summary`, `LedgerService.post` · `web-app/src/features/billing/components/patient-billing/PatientBillingTab.tsx`

34.3.24 Show dues at check-in

Status: [Implemented]

What it does. Shows a red "Rs … due" badge (with a dot when overdue) or a green "Advance Rs …" badge on queue rows and on the patient header, so reception can ask for old dues.

How the user does it. Keep **Show dues at check-in** on in billing settings (default on). The queue gains a **Billing** column with the badge and a **Bill** button on completed entries.

Behind the scenes — in simple terms. The screen asks the server for the dues of the patients it shows.

Behind the scenes — technically. `GET /billing/outstanding?patientIds=` (at most 100 ids) → `PatientBillingService.outstanding`. The setting `show_outstanding_at_check_in` is checked only by the web app; the endpoint itself answers regardless.

Rules and validations. Queue column needs `billing.desk.use`; patient header badge needs `billing.invoice.view`.

Who can do it. `billing.invoice.view`, feature `billing.invoices`. Default: Admin, Receptionist, Accountant. Doctors never see it.

Data read and written. Reads `charges`, `invoices`, `payments`, `ledger_entries`, `location_billing_settings`.

Effects on other modules. Queue board (Chapter 30), patient header (Chapter 28).

Error cases. 400 for more than 100 ids.

Technical references: `backend/src/billing/patient-billing/billing-reports.controller.ts` · `PatientBillingService.outstanding` · `web-app/src/features/billing/components/queue/QueueBillingCell.tsx`

34.3.25 See the day's collection

Status: [Implemented]

What it does. Shows, for one clinic day, receipts and refunds per payment account and the net in hand, plus the list of receipts.

How the user does it. Open **Payments**, choose a day and optionally an account.

Behind the scenes — in simple terms. Payments by their payment date, refunds by their refund date, both in the location's calendar.

Behind the scenes — technically. `GET /payments/daily-collection?date` (`billing.desk.use`) and `GET /payments?from&to&paymentAccountId&patientId&method` (`billing.invoice.view` , paged). `GET /payments/:paymentId` exists but is unused.

Rules and validations. The day picker does not allow future days.

Who can do it. Daily collection: `billing.desk.use` ; list: `billing.invoice.view` . Default: Admin, Receptionist, Accountant. The sidebar item checks `billing.payment.collect` , the route checks `billing.invoice.view` .

Data read and written. Reads `payments` , `refunds` , `payment_accounts` .

Effects on other modules. None.

Error cases. 403 `NO_PERMISSION` .

Technical references: `backend/src/billing/payments/payments.queries.ts` · `PaymentQueries.dailyCollection` · `web-app/src/features/billing/pages/PaymentsPage.tsx`

34.3.26 Close the counter (cash close)

Status: [Implemented]

What it does. Ends a cash period at the location: for each payment account it computes the expected money (payments minus refunds since the last close), takes the counted amount for physically counted accounts, records the difference (variance) and keeps an immutable record with a printable slip. Closing never blocks taking payments.

How the user does it.

1. On **Payments** click **Close counter**.
2. Enter **Counted (Rs)** for each counted account; add a **Note** if there is a difference.
3. Click **Close counter (Rs … expected)**, then **Print slip**. Past closes appear under **Counter closes**, with a red "Flagged" badge when the difference is above the alert amount.

Behind the scenes — in simple terms. Each close starts where the previous one ended, so no payment is counted twice and none is missed.

Behind the scenes — technically. `GET /billing/cash-close/preview` , `POST /billing/cash-close` (`periodEnd` , `lines` , `note`), `GET /billing/cash-close?from&to` (latest 200), `GET /billing/cash-close/:id` (unused by the web app) → `CashCloseService` . Create takes a PostgreSQL advisory transaction lock per location, rejects a `periodEnd` in the future or not after the last close, sums payments by `received_at` and refunds by `refunded_at` in (start, end] per account (`calculateCashClose` , paisa math), requires counts and a note, inserts `cash_closes` and `cash_close_lines` (account name copied) and audits `CLOSED` with the variance. The flag compares the absolute variance with `cash_variance_alert_amount` .

Rules and validations. Counted amount required for accounts with "Count physically at cash close"; note required when any variance exists; closes cannot be edited or deleted.

Who can do it. `billing.cash_close` , feature `billing.invoices` . Default: Admin, Accountant.

Data read and written. Reads `payments`, `refunds`, `payment_accounts`, `location_billing_settings`; writes `cash_closes`, `cash_close_lines`, `billing_audit_logs`.

Effects on other modules. None.

Error cases. 422 `BILLING_CASH_CLOSE_COUNT_REQUIRED` "Enter the counted amount for …"; 422 `BILLING_CASH_CLOSE_NOTE_REQUIRED` "Explain the difference in a note"; 422 `BILLING_CASH_CLOSE_PERIOD_INVALID` "This counter was already closed — open a fresh preview".

Technical references: `backend/src/billing/cash-close/cash-close.service.ts` · `CashCloseService.create`, `calculateCashClose` · `web-app/src/features/billing/components/cash-close/CashCloseSheet.tsx`

34.3.27 Print invoices, receipts, refund vouchers, statements and cash-close slips

Status: [Implemented] — Urdu is supported for typed text only; printed labels and amount in words are English.

What it does. Prints the five billing documents in A4 or 80 mm thermal format from the browser.

How the user does it. Click **Print** (invoice and receipt offer **A4** or **Thermal 80 mm**), **Print receipt**, **Print voucher**, **Print statement** or **Print slip**; the browser's print dialog opens. The file name defaults to the document number.

Behind the scenes — in simple terms. The server sends a frozen set of numbers; the browser only lays them out. A cancelled invoice prints as it was when cancelled, with a CANCELLED watermark; an older revision prints from its saved snapshot.

Behind the scenes — technically. `GET /print/invoices/:invoiceId?revision`, `/print/payments/:paymentId`, `/print/refunds/:refundId`, `/print/patients/:patientId/statement?from&to`, `/print/cash-close/:id` → `PrintDataService`. Payloads include the location header, logo (if `show_logo`), the default format from settings, amount in words (English thousand/million wording, `amount-in-words.ts`) and, when `show_previous_balance` is on, the patient's other dues. The web app renders React templates into a print root with an `@page` rule (A4 with 12 mm margins, or 80 mm width), calls `window.print()` and cleans up afterwards. Typed text (names, notes, reasons, footer) is marked `dir="auto"` and the font stack includes an Arabic-script font, so Urdu text prints right to left. Refund vouchers and statements always print A4.

Rules and validations. No server PDF.

Who can do it. `billing.invoice.view` (cash-close slip: `billing.cash_close`), feature `billing.invoices`.

Data read and written. Reads the billing tables and `location_billing_settings`, `billing_audit_logs` (revision snapshots).

Effects on other modules. None.

Error cases. 404 for unknown documents.

Technical references: `backend/src/billing/print/print-data.service.ts` · `PrintDataService` · `web-app/src/features/billing/components/print/templates.tsx`

34.3.28 Configure billing settings

Status: [Implemented]

What it does. Sets, per location: the **Document code** (2–6 capital letters or digits), **Invoice due after (days)** (0–365, default 7), invoice and receipt print formats (A4 default), the **Consultation fee service** used for automatic charges, the **Default payment account**, the cash-variance alert amount, the printed footer note, and the switches **Print logo**, **Print previous balance** and **Show dues at check-in** (all on by default).

How the user does it. Settings → **Billing** → **Payments and printing**, change fields, click **Save settings** ("Billing settings saved").

Behind the scenes — in simple terms. The document code locks once any invoice, receipt or refund has been numbered at the location.

Behind the scenes — technically. `GET /billing/settings` (`billing.desk.use`), `PATCH /billing/settings` (`admin.settings.manage`) → `BillingSettingsService` ; one row per location in `location_billing_settings` , created lazily by `getOrCreate` . The lock checks for any `document_sequences` row of the location. Updates are audited (`SETTINGS`).

Rules and validations. Consultation fee service must be of kind Consultation; default account must be active; footer up to 300 characters.

Who can do it. `admin.settings.manage` , feature `billing.invoices` . Default: Admin. The page also requires the feature.

Data read and written. `location_billing_settings` , `payment_accounts` , `services` , `document_sequences` , `billing_audit_logs` .

Effects on other modules. Automatic charges, numbering, due dates, printing and queue badges.

Error cases. 409 `BILLING_LOCATION_CODE_LOCKED` "The document code cannot change after the first invoice, receipt or refund"; 422 `BILLING_SERVICE_KIND_INVALID` "Choose a consultation service"; 422 `BILLING_PAYMENT_ACCOUNT_INVALID` "The default account must be active".

Technical references: `backend/src/billing/settings/billing-settings.service.ts` · `BillingSettingsService` · `web-app/src/features/billing/components/settings/BillingSettingsForm.tsx`

34.3.29 Record expenses

Status: [Implemented] — editing exists in the API only; delete has no confirmation dialog.

What it does. Records clinic spending (rent, utilities, supplies and so on) per location, with a monthly summary by category. An expense can be linked to a lab order (for example a courier cost).

How the user does it.

1. Open **Expenses**, click **Add Expense**.
2. In "Record Expense" fill **Category**, **Amount**, **Description**, **Date**, **Payment Method**, optional **Reference** and **Notes**; click **Save Expense**.
3. Filter by category; delete with the trash icon.

Behind the scenes — in simple terms. Expenses are separate from patient billing; they feed the profit figures of the finance reports.

Behind the scenes — technically. `POST /expenses` , `GET /expenses` (category, `dateFrom` , `dateTo` , paging), `GET /expenses/summary?month&year` , `PATCH /expenses/:id` (no screen), `DELETE /expenses/:id` → `ExpensesService` (repository calls, no billing audit). Amounts are `decimal(10,2)` . Delete is a hard delete. Update and delete look up the row by organization, not location.

Rules and validations. Description up to 300 characters; amount 0 or more; payment methods Cash, Card, Bank transfer, Online, Insurance, Cheque, Credit package (a separate list from payment accounts).

Who can do it. `expenses.view` (read), `expenses.manage` (write; implies view). Feature `billing.expenses` . Default: Admin, Accountant.

Data read and written. `expenses` , `lab_orders` (lookup).

Effects on other modules. Finance overview and profit and loss (Chapter 36); lab case costs (Chapter 42).

Error cases. 404 "Expense not found", "Lab order not found"; the screen shows "Failed to record expense".

Technical references: `backend/src/billing/expenses.service.ts` · `ExpensesService` · `web-app/src/features/billing/pages/ExpensesPage.tsx`

34.3.30 Sell credit packages and track patient credits

Status: [Partial] — backend only; no screen uses it (web hooks exist but no component calls them).

What it does. Lets a clinic define prepaid packages (name, credits, price, validity days), record a package sold to a patient and draw credit from it. A nightly job marks expired credits.

How the user does it. Not available in the web app.

Behind the scenes — in simple terms. A sold credit starts ACTIVE, becomes EXHAUSTED when used up, or EXPIRED after its expiry date. Selling a package records no payment, no receipt and no charge, and using credit does not pay a charge.

Behind the scenes — technically. `POST/GET/PATCH/DELETE /credit-packages` (DELETE deactivates) → `CreditPackagesService`; `POST/GET /patients/:patientId/credits` and `POST /patients/:patientId/credits/:creditId/use` → `PatientCreditsService`. `checkExpiredCredits` runs `@Cron('0 1 * * *')`, every day at 01:00 server time, and compares `expiry_date` with the server's date (not the location's). No transactions, no locks, no billing audit.

Rules and validations. Use amount above 0 and at most the remaining amount; credit must be ACTIVE and not past expiry.

Who can do it. Packages: `admin.settings.manage` (write), `billing.desk.use` (read); credits: `billing.payment.collect` (sell, use), `billing.desk.use` (read). Feature `billing.credit_packages`.

Data read and written. `credit_packages`, `patient_credits`.

Effects on other modules. None.

Error cases. 400 "This credit package is not active", "This credit package has expired", "Amount exceeds the remaining balance (...)"; 404 "Credit package not found".

Technical references: `backend/src/billing/patient-credits.service.ts` · `PatientCreditsService.checkExpiredCredits` · `web-app/src/features/billing/hooks/useCreditPackages.ts` (unused)

34.3.31 Review billing activity (the billing audit log)

Status: [Implemented] — the API's filters by user, document type and action are not offered on screen.

What it does. Shows every billing change with who, when and why, and per-user daily totals of edits, cancellations, refunds and discounts.

How the user does it. Open **Billing activity**, choose the period.

Behind the scenes — in simple terms. Every billing write adds an audit row in the same transaction; if the audit row fails, the change fails too.

Behind the scenes — technically. `BillingAuditService.log/logMany` insert into `billing_audit_logs` (entity type, entity id, action, before and after as JSON, amount, reason, actor and role). Organization-level entities (services, discount types) have no location. `GET /billing/activity?`

`from&to&actorId&entityType&action&page&limit` → `BillingActivityService.list`: the feed for the location (plus organization-level rows) and day totals by the location's timezone. Commission fields are removed from charge snapshots.

Rules and validations. Default period is the location's today.

Who can do it. `billing.activity.view`, feature `billing.invoices`. Default: Admin, Accountant.

Data read and written. Reads `billing_audit_logs`.

Effects on other modules. The general audit log (Chapter 60) is separate.

Error cases. 403 `NO_PERMISSION`.

Technical references: `backend/src/billing/audit/billing-audit.service.ts` · `BillingAuditService`, `BillingActivityService` · `web-app/src/features/billing/pages/BillingReportsPages.tsx`

34.3.32 Run the nightly money-invariant check

Status: [Implemented] — backend job by design; it only logs.

What it does. Every night, re-checks the money rules for every patient with charges or payments and writes any violation to the server log. It repairs nothing.

How the user does it. Nothing; operations staff read the logs.

Behind the scenes — in simple terms. It checks that cached totals match the detailed rows and that no money is counted twice.

Behind the scenes — technically. `BillingInvariantJob.run`, `@Cron('30 2 * * *')`, every day at 02:30 server time, in the process that runs scheduled jobs. For each (organization, location, patient) it calls `BillingInvariantChecker.checkPatient` with raw SQL (deliberately not sharing code with `BillingTotalsService`). The invariants:

- **I1** a charge's allocated amount equals the sum of its allocations and lies between 0 and net + tax (0 for VOID);
- **I2** a payment's cached allocated and refunded amounts match the rows, and together do not exceed the payment;
- **I3** an issued invoice's subtotal, discount, tax (rate × net), total, paid and balance match its active charges;
- **I4** every allocation points to a charge on an ISSUED invoice and to that same invoice, and no net allocation is negative;
- **I5** the patient's ledger balance equals outstanding minus advance;
- **I6** invoice, receipt and refund numbers of the location are unique, consecutive from 1 per year, and the counter equals the highest;
- **I7** each refund equals the sum of its refund sources;
- **I8** allocations, refunds and ledger rows agree with their charge, payment and invoice on organization, location and patient;
- **I9** an invoice's refunded amount is between 0 and its released amount and matches the rows;
- **I10** each charge's commission entries add up to its commission target (Chapter 35).

Rules and validations. Read-only; one failing patient does not stop the run.

Who can do it. System job.

Data read and written. Reads all billing tables; writes logs only.

Effects on other modules. None.

Error cases. Log lines "[I3] org … loc … patient …: …".

Technical references: `backend/src/billing/invariants/billing-invariant-checker.ts` · `BillingInvariantJob`, `BillingInvariantChecker` · none

34.3.33 Doctor revenue and financial summary

Status: [Implemented] — the daily financial summary has a hook but no screen; `/financial-summary` redirects to the dashboard.

What it does. Doctor revenue lists per doctor what was billed (by invoice date), discounts and what was collected (by payment date, releases subtracted). The **financial summary** endpoints give daily and monthly collection, refunds, tax-free revenue, expenses and a trend; the dashboard uses the monthly and trend ones. Chapter 36 covers dashboards and reports.

How the user does it. Open **Doctor revenue** in the Reports group; choose a period.

Behind the scenes — in simple terms. Revenue excludes sales tax.

Behind the scenes — technically. `GET /billing/reports/doctor-revenue?from&to` → `BillingReportsService.doctorRevenue`; `GET /financial-summary/daily|monthly|trend` → `FinancialSummaryService` (class-level `@RequireFeature('billing.financial_summary')`).

Rules and validations. Active location.

Who can do it. `reports.financial.view`, features `reports.financial` and, for the summary, `billing.financial_summary`. Default: Admin, Accountant (`location`).

Data read and written. Reads `charges`, `invoices`, `payment_allocations`, `payments`, `refunds`, `expenses`.

Effects on other modules. Dashboard (Chapter 36).

Error cases. 403 `NO_PERMISSION` or feature disabled.

Technical references: `backend/src/billing/reports/billing-reports.service.ts` · `BillingReportsService.doctorRevenue`, `FinancialSummaryService` · `web-app/src/features/billing/pages/BillingReportsPages.tsx`

34.3.34 Snapshot the doctor's commission on a charge

Status: [Implemented] — cross-reference only; Chapter 35 covers compensation.

What it does. When a charge is created from a visit (or its doctor is changed while unbilled), it stores the performing assignment, the compensation plan and the commission rate (basis points) or fixed amount in force on the charge date. Commission is earned when money is allocated. These fields never appear on invoices, prints or audit snapshots.

How the user does it. Nothing at the desk.

Behind the scenes — in simple terms. Later changes to a doctor's plan do not change old charges.

Behind the scenes — technically. `CommissionService.snapshot` in `syncForEncounter` and `ChargesService.update`; columns `performing_assignment_id`, `commission_plan_id`, `commission_rate_bps` (0–10 000, checked), `commission_fixed_paisa` (bigint paisa).

Rules and validations. Manual charges have no snapshot.

Who can do it. System. Feature `billing.commissions`.

Data read and written. `charges`, compensation tables.

Effects on other modules. Commissions (Chapter 35).

Error cases. None.

Technical references: `backend/src/billing/commissions/commission.service.ts` · `CommissionService.snapshot` · none

34.4 Entities and data model

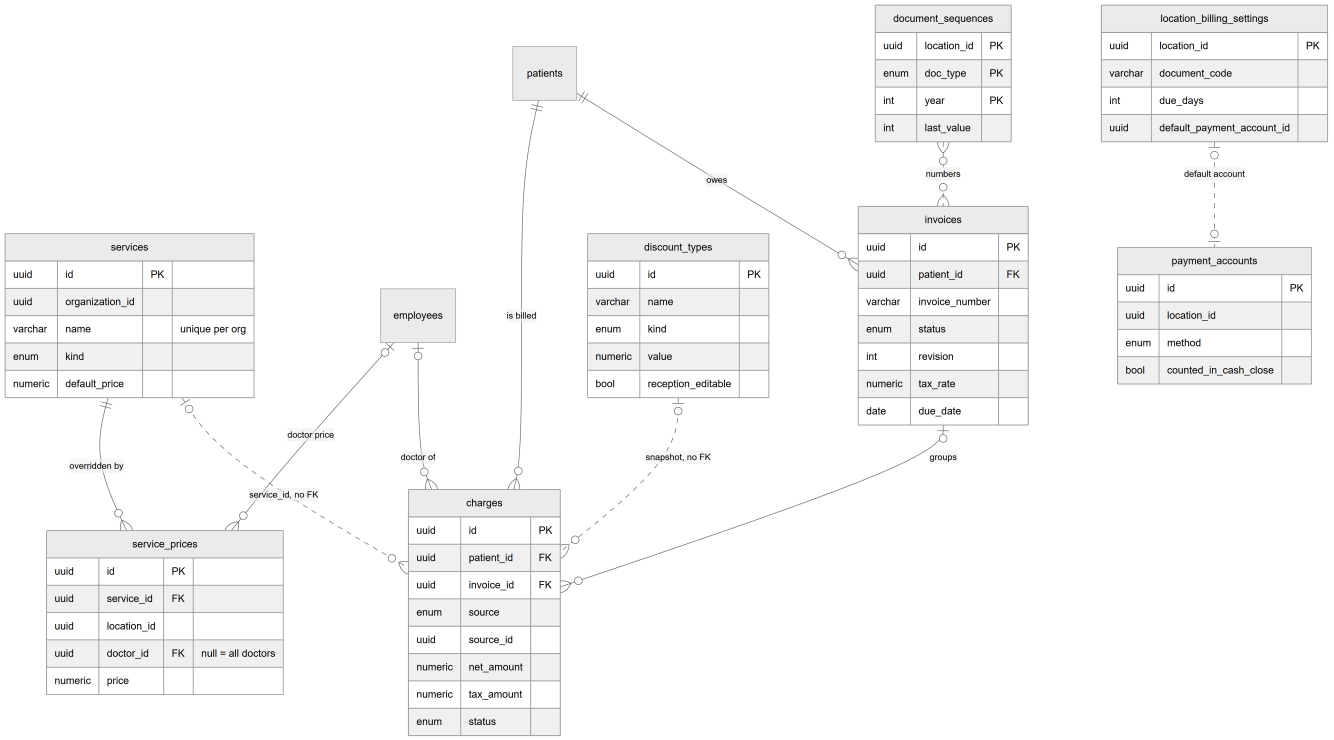

Figure 34.15 — The pricing, charge and invoice tables.

The diagram shows ten tables. `services` (id, organization_id, name unique per org, kind, default_price) is "overridden by" `service_prices` (id, service_id, location_id, doctor_id where empty means all doctors, price); `employees` links to `service_prices` as "doctor price". `charges` (id, patient_id, invoice_id, source, source_id, net_amount, tax_amount, status) is linked from `patients` ("is billed"), `employees` ("doctor of") and `invoices` ("groups"). Dotted lines with "no FK" connect `services` to `charges` (service_id) and `discount_types` (id, name, kind, value, reception_editable) to `charges` (snapshot). `patients` "owes" `invoices` (id, patient_id, invoice_number, status, revision, tax_rate, due_date). `document_sequences` (location_id, doc_type, year as primary key, last_value) is linked to `invoices` with a dotted "numbers" line. `location_billing_settings` (location_id, document_code, due_days, default_payment_account_id) has a dotted "default account" line to `payment_accounts` (id, location_id, method, counted_in_cash_close).

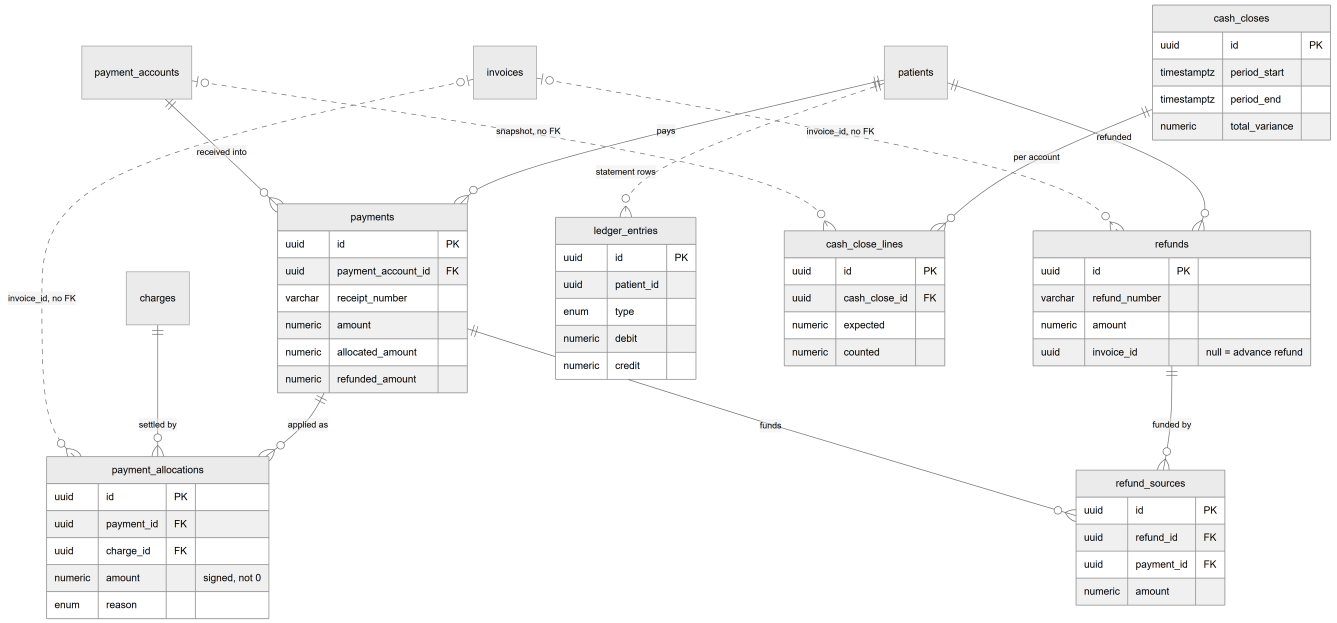

Figure 34.16 — The money-movement tables.

This diagram shows `payments` (id, payment_account_id, receipt_number, amount, allocated_amount, refunded_amount) linked from `patients` ("pays") and `payment_accounts` ("received into"). `payment_allocations` (id, payment_id, charge_id, amount signed and never zero, reason) hangs from `payments` ("applied as") and `charges` ("settled by"), with a dotted "invoice_id, no FK" line to `invoices`. `refunds` (id, refund_number, amount, invoice_id, empty for an advance refund) comes from `patients` and has a dotted line to `invoices`; `refund_sources` (id, refund_id, payment_id, amount) links each refund to the payments it drew from. `ledger_entries` (id, patient_id, type, debit, credit) is linked to `patients` by a dotted line. `cash_closes` (id, period_start, period_end, total_variance) owns `cash_close_lines` (id, cash_close_id, expected, counted), which reference `payment_accounts` by a dotted, no-FK line.

The relationships in words:

- Each service belongs to one organization; a service with price overrides cannot be deleted (`service_prices.service_id` ON DELETE RESTRICT). Overrides belong to one location and optionally one doctor (employee, RESTRICT).
- Each charge belongs to exactly one patient (RESTRICT), optionally one doctor (RESTRICT) and at most one invoice (RESTRICT). `invoice_id` empty means unbilled. `service_id`, `discount_type_id`, `source_id`, `encounter_id` and `appointment_id` are logical links without foreign keys; the charge keeps copies of description, price, kind and discount. One ACTIVE charge per (organization, source, source id) (`uq_charges_source`).
- Each invoice belongs to one patient (RESTRICT); its number is unique per organization.
- Each payment belongs to one patient and one payment account (RESTRICT); receipt number and `client_request_id` are unique per organization. The account name and method are copied onto the payment.
- Each allocation belongs to one payment and one charge (RESTRICT) and copies the charge's invoice id. Allocations are append-only: corrections are negative rows (`amount <> 0` check).
- Each refund belongs to one patient; `invoice_id` (no FK) is set only for invoice refunds. Each refund source belongs to one refund and one payment.
- `ledger_entries` is the append-only patient statement (types INVOICE_ISSUED, INVOICE_ADJUSTED, INVOICE_CANCELLED, PAYMENT, REFUND); it has no foreign keys.
- Each cash close belongs to one location and has one line per payment account, copying the account name.
- `document_sequences` has one row per organization, location, document type and year.

- `location_billing_settings` has one row per location (the location id is the key). `billing_audit_logs` stores every change; `expenses`, `credit_packages` and `patient_credits` are separate tables.

Money storage: billing amounts are `numeric(12,2)` rupee columns, read through a transformer that rounds via paisa; all arithmetic is integer paisa; commission amounts are `bigint` paisa. Expenses, credit packages and patient credits use `decimal(10,2)`. The general ledger journals in `billing/ledger/journal` are used by the lab finance modules, not by patient billing.

34.5 Status lifecycles

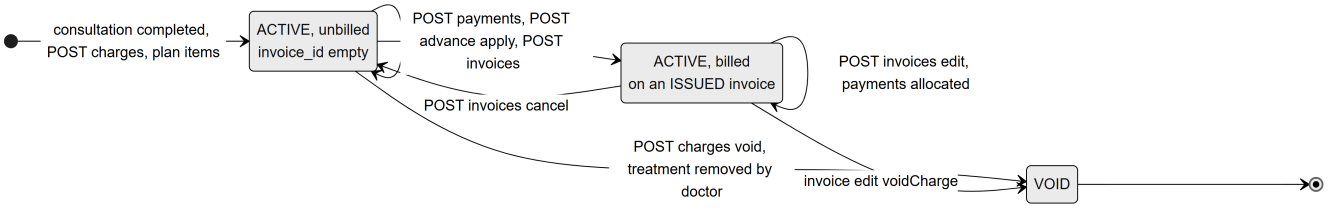

Figure 34.17 — A charge: stored status ACTIVE or VOID, and whether it is on an invoice.

The diagram has three boxes. A charge enters "ACTIVE, unbilled, invoice_id empty" when a consultation is completed, through `POST` charges, or from plan items. A self-loop on it says "PATCH charge, discount, needs price set". An arrow labelled "POST payments, POST advance apply, POST invoices" leads to "ACTIVE, billed on an ISSUED invoice", which has its own loop "POST invoices edit, payments allocated". An arrow "POST invoices cancel" goes back to unbilled. "POST charges void, treatment removed by doctor" leads from unbilled to "VOID", and "invoice edit voidCharge" leads from billed to "VOID", which ends.

The stored `status` has only two values, `ACTIVE` and `VOID`. VOID is final. Whether a charge is billed is not a status but the presence of `invoice_id`. Unbilled charges are changed directly; billed charges only through an invoice edit. A charge is "paid" when its allocated amount reaches net + tax; this too is computed.

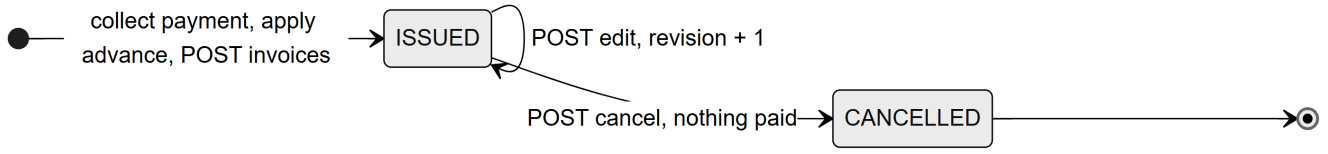

Figure 34.18 — The stored invoice status.

An invoice is born `ISSUED` by collecting a payment, applying advance or `POST /invoices`. "POST edit" keeps it ISSUED and raises the revision by one. "POST cancel, nothing paid" makes it `CANCELLED`, which is final. There is no draft state.

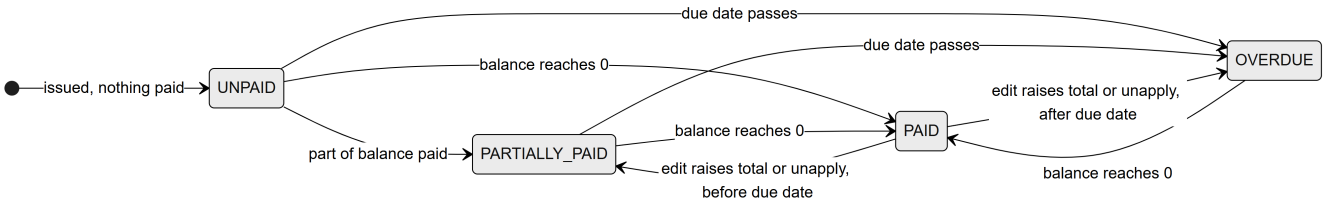

Figure 34.19 — The computed payment status of an issued invoice.

The diagram starts at UNPAID ("issued, nothing paid"). UNPAID goes to PARTIALLY_PAID when part of the balance is paid, to PAID when the balance reaches 0, and to OVERDUE when the due date passes. PARTIALLY_PAID goes to PAID or OVERDUE the same way. OVERDUE goes to PAID when the balance reaches 0. PAID goes back to PARTIALLY_PAID when an edit raises the total or an unapply happens before the due date, and to OVERDUE when that happens after the due date. The status is never stored; it is computed on each read from `balance_due`, `paid_amount`, `due_date` and the location's today. A cancelled invoice has no payment status. The due date never changes after issue.

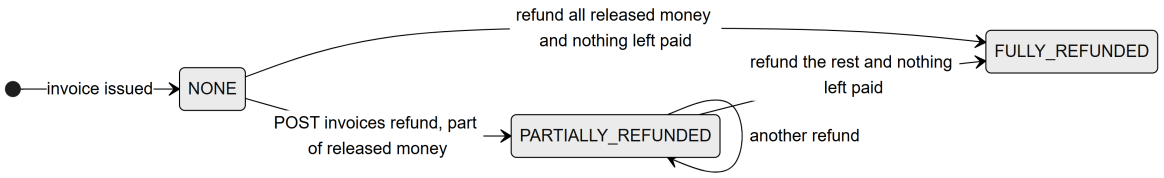

Figure 34.20 — The computed refund status of an invoice.

An invoice starts at NONE. "POST invoices refund, part of released money" moves it to PARTIALLY_REFUNDED, which loops on "another refund". It reaches FULLY_REFUNDED when all money released from the invoice has been refunded and nothing is left paid on it. Refunds themselves have no status column; a refund row is final once written.

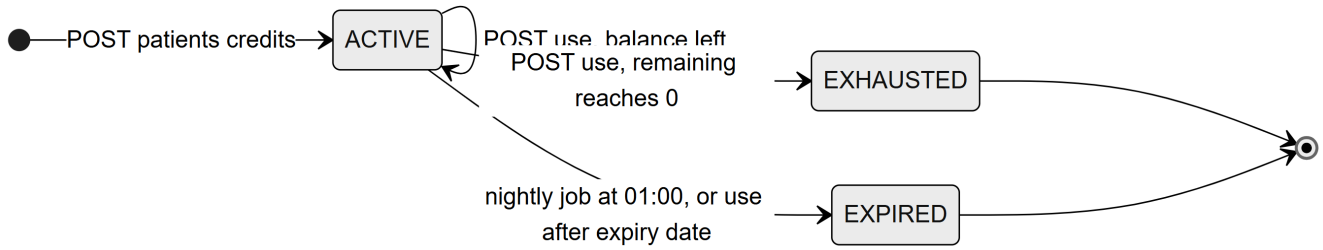

Figure 34.21 — The lifecycle of a sold patient credit.

A credit starts ACTIVE through `POST` patient credits. "POST use, balance left" keeps it ACTIVE; "POST use, remaining reaches 0" makes it EXHAUSTED; "nightly job at 01:00, or use after expiry date" makes it EXPIRED. Both end states are final.

Other objects have no lifecycle: payments, allocations, refunds, ledger entries and cash closes are written once and never change state (allocations are corrected by adding negative rows).

34.6 Connections to other modules

- **Queue and consultation (Chapters 30, 31).** Completing a visit creates charges in the same transaction. The queue board shows dues badges and a **Bill** button.
- **Patients (Chapter 28).** The Billing tab and the dues badge in the patient header. Patients with charges cannot be deleted (RESTRICT).
- **Treatment plans (Chapter 33).** Plan-to-charges and the plan billing summary exist in the API only.
- **Commissions (Chapter 35).** Charges snapshot the rate; collection, unapply and invoice edits write commission entries.
- **Dashboard and reports (Chapter 36).** Finance overview, profit and loss, doctor revenue and the dashboard read billing tables and expenses.
- **Locations (Chapter 9).** The location's timezone sets every clinic date; the location's tax rate is snapshotted on invoices.

- **Features and roles (Chapters 10, 11).** Five billing features; permissions listed per feature in 34.3.
- **Lab finance (Chapters 40, 42).** Expenses can point to lab orders; the general ledger journals are shared code.
- **Subscriptions (Chapter 45).** In read-only mode every billing write is refused.

34.7 Limitations and known gaps

- `billing.invoices` is off by default in the feature seed; a new clinic has no billing until it is enabled.
- No screen for: unapply, refund list, payment detail, resolved price, editing a price override, charges from a treatment plan, plan billing summary, credit packages and patient credits, editing an expense, daily financial summary.
- Because unapply has no screen, an invoice that has received any payment cannot be cancelled from the web app; it can only be edited (voiding lines moves the money to advance).
- The Payments sidebar item checks `billing.payment.collect` but the page checks `billing.invoice.view`.
- The billing desk's Outstanding section omits partly paid invoices.
- Expenses: no confirmation before delete, hard delete, no paging beyond 30 rows, no billing audit, update and delete not limited to the active location.
- Credit expiry uses the server's date, not the location's; selling or using credit is not linked to payments or charges.
- The "Show dues at check-in" switch is enforced only in the web app.
- Printed labels and amounts in words are English only; there is no server PDF.
- The invariant job only logs; nobody is alerted.
- The Accountant template has no discount permission; the Receptionist template cannot edit invoices or refund.

34.8 Owner's FAQ

Can a receptionist change the price of a consultation? No, not for a catalog service. The receptionist can apply a preset discount, within the range the type allows. Only a role with `billing.price.override` (Admin by default) can change a catalog price. "Other" charges and charges marked Needs price can be priced by anyone at the desk.

What happens if a doctor completes a visit and the consultation fee service is not set? The consultation charge is still created, at zero, with a "Needs price" badge. Reception must set a price before an invoice can be issued.

What if a patient pays more than they owe? The extra stays as advance. It is used on the next bill when "Use advance" is on, or can be refunded with **Refund advance**.

Can an invoice be deleted? No. It can be edited (each edit is a new revision with a reason) or cancelled while nothing is paid. Its number is never reused.

Why does an invoice say Overdue? It still has a balance and its due date (issue date plus the location's due days, 7 by default) is before the location's today. The status is computed, not stored.

Where is the tax rate set, and does changing it change old invoices? In the location's settings. Each invoice freezes the rate it was issued with; later edits keep using that frozen rate.

What protects against double charging when the network retries? Collect payment and refunds carry a client request id; a repeat with the same id returns the first result. Automatic charges use a unique index per source, so completing a visit twice never doubles them.

Can two receptionists take the next invoice number at the same time? Yes, safely. The counter row is locked until each transaction ends, so numbers stay unique and gapless.

Where is the history of who changed what? In `billing_audit_logs`, shown on the **Billing activity** page, with before and after snapshots and the reason.

Does cash close stop the desk from taking payments? No. A close only records a period; payments after it fall into the next close.

Chapter 35 — Commissions

This chapter describes the Commissions module of the Platform: how a clinic decides how each staff member is paid (a monthly salary, a share of what patients pay for that person's work, or both), how the Platform works out that share automatically from billing, and how the share is paid out through payroll. It is written for the product owner, who needs to know exactly which rules the code enforces, where the money figures are stored and what is still missing. Commissions sit between Billing (Chapter 34), which produces the patient money, and Payroll (Chapter 38), which pays the staff.

35.1 What this module is for

A **compensation plan** says how one **assignment** (one employee at one location) is paid. It has one of three types: `SALARY` (shown as "Salaried"), `COMMISSION` ("Commission") or `SALARY_PLUS_COMMISSION` ("Both"). A plan with commission has a default rate and may have **commission rules**: overrides for one catalog service or one service category. Plans are versioned by date, so a change never alters what was already earned.

Commission is earned **on collection**, not on billing. When patient money is applied to a doctor's charge, the Platform writes a **commission entry** in an append-only ledger. At month end, payroll gathers the unsettled entries into a **settlement** and adds them to the payroll record. Commission is an internal clinic cost: it never appears on invoices, receipts or prints.

Who uses it:

- **Admin** and the **owner**: set and change compensation plans and overrides, see every doctor's earnings and the "Doctor share" line when collecting payment.
- **Accountant**: sees every doctor's earnings and the doctor share line, but cannot change plans.
- **Doctor**: sees only their own earnings on "My earnings".
- **Receptionist** and **Assistant**: no access by default. A receptionist collecting payment does not see the doctor share line.

Where it appears in the sidebar (labels from `nav.registry.ts`):

- **My earnings** (`nav.my_earnings`, route `/my-earnings`, needs `earnings.own.view`) in the Clinic section.
- **Doctor earnings** (`nav.reports.doctor_earnings`, route `/billing/doctor-earnings`, needs `earnings.all.view`) in Finance → Reports.
- **Doctor revenue** (`nav.reports.doctor_revenue`, route `/billing/doctor-revenue`, needs `reports.financial.view`) in Finance → Reports. This report is about billed and collected money per doctor, not commission, but it sits next to the earnings report and owners read them together.
- The compensation screens have no sidebar entry: they are the **Compensation** tab inside the employee detail page (`/employees/$employeeId`).

The module is controlled by the feature **`billing.commissions`** ("Doctor Commissions", off by default). All three commission permissions belong to it, so with the feature off nobody holds them. A second feature key, **`hr.commissions`** ("Commissions", off by default), exists in the feature catalog, but no permission, route or code checks it: it gates nothing. Salaries keep working with the feature off, because a salary is stored as a salary-only compensation plan.

35.2 Screens

Figure 35.1 — Employee detail, Compensation tab (screenshot pending).

The tab is shown on the employee detail page to users who hold `compensation.manage`. It has two cards. The first card, "Current compensation", shows the plan in force today at the location: Type (Salaried, Commission or Both), Monthly salary, Default commission as a percentage, "Commission on" ("Amount after discount" or "Amount before discount") and "Since" (the start date). Under these it lists "Service-wise overrides" as small badges such as "All procedures: 35%" or "Root canal: Rs 500 per charge", or the line "None — every service earns the default rate.". The card header has an **Edit overrides** button (only when the current plan earns commission) and a **Change plan** button (labelled **Set up compensation** when no plan is in force). With no plan in force, the card shows an empty state "No compensation in force". The second card, "Version history", is a table of every version, newest first, with columns Period (for example "1 Oct 2026 – now", with a "Current" badge), Type, Salary, Commission (for example "40% (net)") and Overrides (a count or "None").

Screenshot pending

Change compensation sheet

`C:/Program Files/Git/employees/$employeeId`

Figure 35.2 — The "Change compensation" slide-over sheet (screenshot pending).

The sheet opens from the right with the title "Change compensation" and the explanation "Saves a new version from the chosen date. Charges already created keep the rate they were made with." It contains: "How is this person paid?" as a three-way switch (Salaried, Commission, Both; the two commission options are disabled with the hint "Commission needs a role that can consult patients." when the person's role cannot consult); "Monthly salary (Rs)" (hidden for Commission); "Default commission (%)" and "Commission on" ("After discount" / "Before discount"), both hidden for Salaried; "Service-wise overrides", an editor of rows where each row picks a service or one of "All consultations", "All procedures", "All other services", a percent-or-fixed switch and a value, with the hint "A service rule beats a category rule, which beats the default."; and "Effective from", a date picker that defaults to today at the location and cannot go before the current version's start. The footer shows "Unsaved changes" when the form is dirty, **Cancel** and **Save version** (disabled while invalid or saving). A second, smaller sheet, "Commission overrides", edits only the overrides of the current version and has a **Save overrides** button.

Figure 35.3 — Doctor earnings report (screenshot pending).

The page title is "Doctor earnings" with the subtitle "Commission is earned when patient money is applied to the doctor's charges. Internal — never shown to patients." A strip of five tiles shows Doctors, Collected, Commission earned, Settled and Pending for the chosen period. A toolbar offers the date presets Today, This month, Last month, Past 30 days and a custom range, shows the range as "from → to" and a small live dot while refreshing. The table has one row per doctor with the columns Doctor ("Dr." and the name), Plan (a badge), Collected, Earned, Settled and Pending. Negative amounts are shown in red. With no data the table says "No commission-earning doctors in this period."; a failed load shows an error state with a retry button.

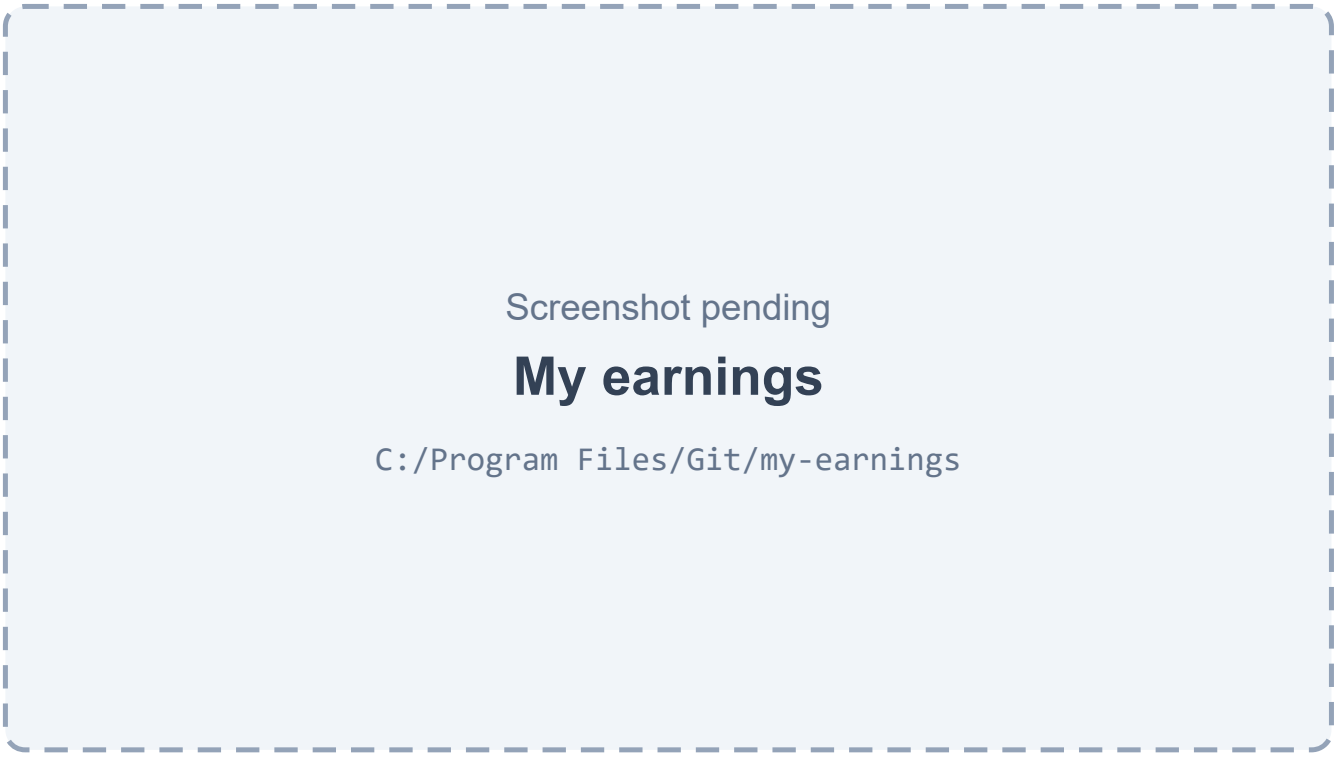

Figure 35.4 — A doctor's own earnings page (screenshot pending).

"My earnings" has the subtitle "Your commission from patient payments on your work." It uses the same four money tiles (without the doctor count) and the same date presets. Its first table has one row per location where the doctor works, with the columns Location, Plan, Collected, Earned, Settled and Pending (empty message "You have no commission plan yet."). Under the heading "Commission lines" a second, paged table lists every ledger line in the period: Date, Patient (with the charge description and invoice number under the name), Why ("Payment", "Unapplied", "Invoice edited", "Charge voided" or "Adjustment"), Rate (a percentage or "Fixed"), Amount and Status ("Settled" or "Pending").

Figure 35.5 — Doctor revenue report (screenshot pending).

The page title is "Doctor revenue" with the subtitle "Billed by invoice date; collected by payment date (releases subtract)." Five tiles show Doctors, Billed, Discounts, Collected and Collected %. The same date toolbar is used. The table has the columns Doctor (or "No doctor" in italics for charges without a doctor), Billed, Discounts, Collected and Collected % (a coloured pill: green from 90%, amber from 50%, red below).

A sixth place shows commission without a screen of its own: in the billing desk's **Collect payment** panel, users who hold `earnings.all.view` see a "Doctor share" line with the commission this payment would record, split per doctor when more than one doctor is involved. The Billing chapter (Chapter 34) describes that panel.

35.3 Features

35.3.1 Set up or change a compensation plan

Status: [Implemented]

What it does. Creates a new version of how one assignment is paid, starting on a chosen clinic date. The version in force until then is closed on the day before.

How the user does it.

1. Open **Employees**, click an employee, and choose the **Compensation** tab.
2. Click **Change plan** (or **Set up compensation**).
3. Pick Salaried, Commission or Both; fill in the salary and/or default commission, the basis and any overrides.

4. Choose **Effective from** and click **Save version**.

Behind the scenes — in simple terms. Plans are kept as a dated history per assignment. A new version starts on its date and the old one ends the day before, so charges made under the old version keep the old rate. If the new version starts on the same day as the latest one, it replaces it, but only if no charge has used that version yet. Commission is allowed only for staff whose role can consult patients.

Behind the scenes — technically. `POST /assignments/:assignmentId/compensation-plans` → `CompensationPlansService.create` → `CompensationPlanVersions.add`, in one transaction. The service loads the assignment for the active organization and location (with its role), runs `validateFields` (type/field combination, rate 0–10000 basis points, salary whole paisa, date `YYYY-MM-DD`), checks `canConsult` (the role's permissions, expanded through implies, contain `consultation.conduct`) for commission types, and `validateRules`. It then locks the assignment row (`SELECT … FOR UPDATE`), reads the latest version and either refuses, deletes it (same start day, unused), or sets its `effective_to` to the day before. The new row is inserted with `created_by`; the PostgreSQL error 23P01 from the exclusion constraint `excl_compensation_plans_overlap` (GiST over `assignment_id` and `daterange(effective_from, effective_to, '[]')`) is mapped to `COMPENSATION_PLAN_OVERLAP`. The default start date is the location's clinic date (`BillingClock`). The response is the full overview (`GET` shape).

Rules and validations.

- Salaried: salary required, no rate, no overrides. Commission: rate required, no salary. Both: salary and rate required. The database repeats this rule as a check constraint.
- Rates are integer basis points (4000 = 40.00%), 0 to 10000. The web form accepts percentages with at most two decimals and converts them with integer maths.
- Money is integer paisa; the form accepts rupees with at most two decimals.
- The basis defaults to `NET` (after discount).
- A new version must start on or after the latest version's start. Same day replaces only an unused version.
- At most 200 overrides per version.
- The web app opens the Compensation tab only when the employee record carries an assignment at the active location.

Who can do it. `compensation.manage` (marked high-risk in the role editor; it implies `earnings.all.view`), feature `billing.commissions`. Default: Admin only.

Data read and written. Reads `employee_org_location_assignments`, `employment_roles`, `services`, `charges` (to test whether a version was used). Writes `compensation_plans`, `commission_rules`.

Effects on other modules. Charges created after the start date take their snapshot from the new version (35.3.4). Payroll reads the salary from the version in force at month end (Chapter 38). The employee list and detail show the salary from the current version.

Error cases.

- 422 `COMPENSATION_PLAN_INVALID` → wrong field combination, for example "A commission plan needs a default rate and no salary".
- 422 `COMMISSION_ROLE_CANNOT_CONSULT` → "Commission is only for staff whose role can consult patients".
- 422 `COMMISSION_RULE_INVALID` → a rule with no target or two targets, both or neither amount, a duplicate target, or "Unknown service in rules".
- 409 `COMPENSATION_PLAN_OVERLAP` → "A new version must start after the current one (2026-10-01)".
- 409 `COMPENSATION_PLAN_IN_USE` → "The current version already priced charges; start the new one on a later date".
- 404 `BILLING_NOT_FOUND` "Assignment not found" → the assignment is not at the active location.

- 403 `NO_PERMISSION`; 402 `SUBSCRIPTION_EXPIRED` in read-only mode.

Technical references: `backend/src/billing/commissions/compensation-plan-versions.ts` · `CompensationPlanVersions.add`, `CompensationPlansService.create` · `web-app/src/features/billing/components/compensation/CompensationSheet.tsx`

35.3.2 See the current plan and its version history

Status: [Implemented]

What it does. Shows the plan in force today, all earlier and future versions, and whether the person's role allows commission.

How the user does it.

1. Open the employee and choose the **Compensation** tab.

Behind the scenes — in simple terms. The server returns every version of the assignment, marks the one whose dates include today at the location as current, and adds service names to the overrides.

Behind the scenes — technically. `GET /assignments/:assignmentId/compensation-plans` → `CompensationPlansService.list`. Returns `today` (the location's clinic date), `canConsult`, `current` and `versions` (newest first, with rules and service names).

Rules and validations.

- `isCurrent` = `effective_from ≤ today` and (`effective_to` is empty or `≥ today`).

Who can do it. `compensation.manage`, feature `billing.commissions`. Default: Admin.

Data read and written. Reads `employee_org_location_assignments`, `employment_roles`, `compensation_plans`, `commission_rules`, `services`.

Effects on other modules. None.

Error cases. 404 "Assignment not found"; 403 `NO_PERMISSION`. The tab shows "Could not load compensation." with a retry button.

Technical references: `backend/src/billing/commissions/compensation-plans.controller.ts` · `CompensationPlansService.list` · `web-app/src/features/billing/components/compensation/CompensationTab.tsx`

35.3.3 Edit service-wise overrides (commission rules)

Status: [Implemented]

What it does. Replaces the overrides of one version without starting a new version. Each override names one catalog service or one service category (`CONSULTATION`, `PROCEDURE`, `OTHER`) and gives either a percentage or a fixed amount per charge.

How the user does it.

1. On the Compensation tab click **Edit overrides**.
2. Add, change or remove rows in "Commission overrides".
3. Click **Save overrides**.

Behind the scenes — in simple terms. The overrides decide the rate for each service: a rule for the exact service wins, then a rule for its category, then the plan's default rate. New overrides apply only to charges created from now on; charges already made keep their stored rate.

Behind the scenes — technically. `PUT /compensation-plans/:planId/rules` → `CompensationPlansService.replaceRules`: in one transaction it locks the plan row (`FOR UPDATE`, scoped to the organization), checks the assignment belongs to the active location, refuses salary-only plans and versions that ended before today, validates the rules, deletes all rules of the plan and inserts the new list.

Rules and validations.

- Exactly one target per rule (service or category) and exactly one amount (rate or fixed); database check constraints enforce both.
- One rule per service and one per category within a plan (partial unique indexes).
- Fixed amounts are paisa ≥ 0; rates 0–10000 basis points.
- A fixed amount is earned in full only when the charge is fully paid; partial payment earns the same fraction.
- Services must belong to the organization.

Who can do it. `compensation.manage`, feature `billing.commissions`. Default: Admin.

Data read and written. Reads `compensation_plans`, `services`. Writes `commission_rules` (delete + insert).

Effects on other modules. Only later charges' snapshots.

Error cases. 404 "Compensation plan not found"; 422 `COMPENSATION_PLAN_INVALID` "A salaried plan cannot have commission rules"; 409 `COMPENSATION_PLAN_CLOSED` "This version has ended; add a new version instead"; 422 `COMMISSION_RULE_INVALID`.

Technical references: `backend/src/billing/commissions/compensation-plans.service.ts` · `CompensationPlansService.replaceRules`, `resolveCommissionRate` · `web-app/src/features/billing/components/compensation/CommissionRulesEditor.tsx`

35.3.4 Snapshot the rate on each visit charge

Status: [Implemented] (backend only by design; no screen)

What it does. When a visit produces charges, each consultation or treatment charge records which doctor assignment performed it, which plan was in force on the visit's clinic date and the resolved rate or fixed amount. Later plan changes do not touch these stored values.

How the user does it. Nothing to do. The snapshot is taken automatically when the visit's charges are created at the end of the consultation, and again when an unbilled charge is moved to a different doctor.

Behind the scenes — in simple terms. The Platform writes the doctor's deal onto the charge at the moment the charge is born, like a price tag. Manual charges and charges without a doctor carry no deal and never earn.

Behind the scenes — technically. `CommissionService.snapshot` runs inside the charge-creation transaction of `ChargeSourcesService` (visit charges) and inside `ChargesService` when the doctor of an unbilled charge changes. It returns empty snapshots when there is no doctor or when `billing.commissions` is not enabled for the organization. Otherwise it finds the doctor's assignment at the active location, loads the version in force on the charge's clinic date (`CompensationPlanVersions.activeOn`) and resolves per charge with `resolveCommissionRate` (service → category → default; a `SALARY` plan resolves to nothing). Only charges whose source is `CONSULTATION` or `TREATMENT` get a snapshot. It sets four columns on `charges`: `performing_assignment_id`, `commission_plan_id`, `commission_rate_bps`, `commission_fixed_paisa`.

Rules and validations.

- Charges with a salary-only plan keep `performing_assignment_id` and `commission_plan_id` but no rate, so they never earn.

- If the doctor has no assignment at the location, or no version covers the date, the charge has no plan and never earns.
- Turning the feature off stops new snapshots; charges that already carry one keep recording commission when paid.

Who can do it. Whoever creates the charges (the consultation and billing flows); no commission permission is needed.

Data read and written. Reads `employee_org_location_assignments`, `compensation_plans`, `commission_rules`, `org_feature_flags` (through the feature service). Writes the four snapshot columns of `charges`.

Effects on other modules. Billing (Chapter 34) carries the columns but never shows them on invoices; audit snapshots strip them.

Error cases. None of its own.

Technical references: `backend/src/billing/commissions/commission.service.ts` · `CommissionService.snapshot` · no web-app use

35.3.5 Earn commission when patient money is applied

Status: [Implemented] (backend only by design)

What it does. Every time money on a doctor's charge changes, the Platform brings the charge's total commission up (or down) to the correct amount by inserting one signed entry in `commission_entries`.

How the user does it. Nothing to do. It happens when staff collect a payment, apply a patient's advance, unapply a payment, or edit or void an invoice.

Figure 35.6 shows the main path, a payment collection.

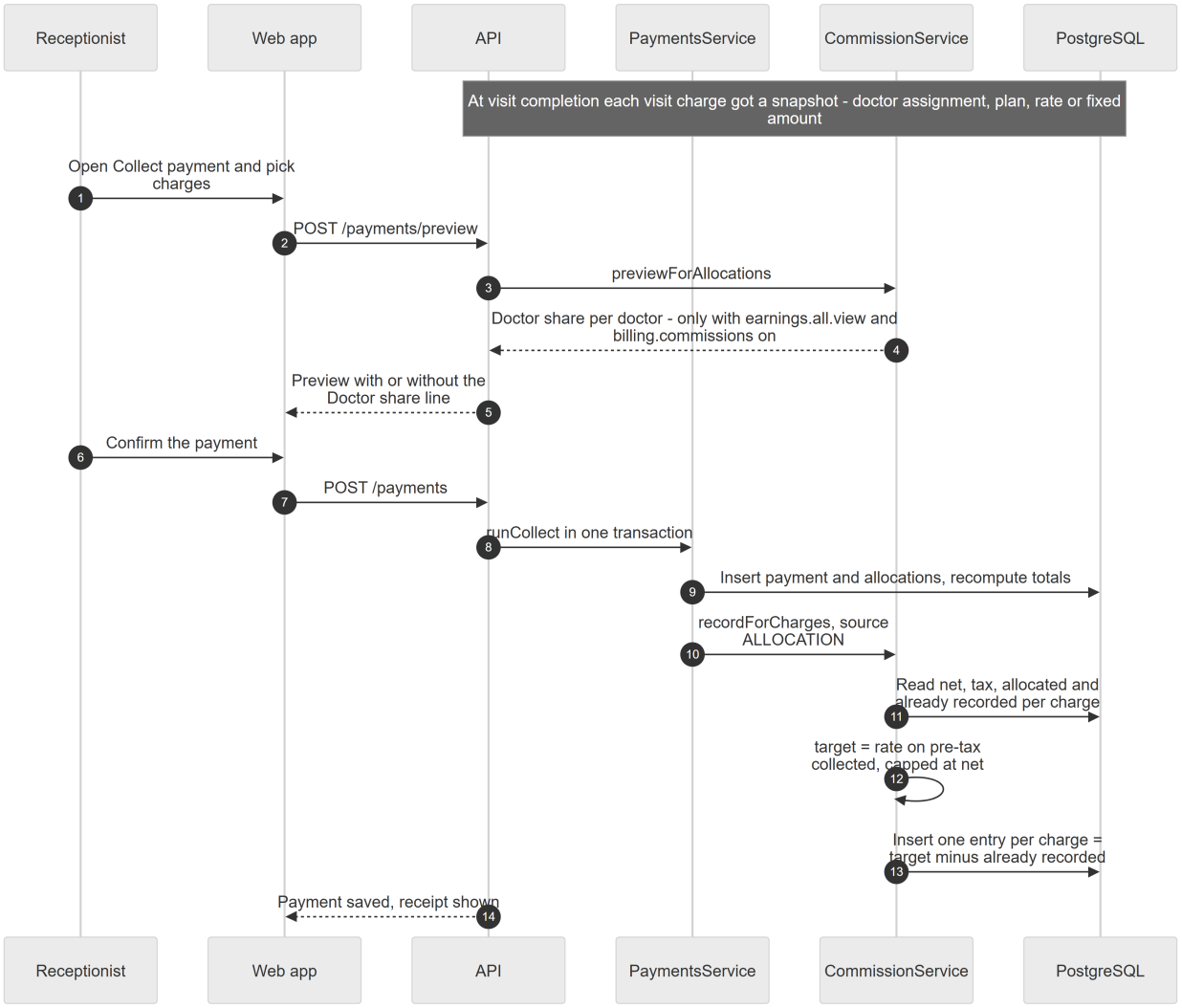

Figure 35.6 — How collecting a payment writes commission entries.

The diagram has six participants: Receptionist, Web app, API, PaymentsService, CommissionService and PostgreSQL. A note at the top says that, at visit completion, each visit charge received a snapshot with the doctor's assignment, plan and rate or fixed amount. In steps 1 to 5 the receptionist opens Collect payment and picks charges; the web app calls `POST /payments/preview`; the API asks CommissionService for `previewForAllocations`, which returns a doctor share per doctor only when the user holds `earnings.all.view` and `billing.commissions` is on; the web app shows the preview with or without the Doctor share line. In steps 6 to 9 the receptionist confirms, the web app calls `POST /payments`, the API runs `runCollect` in one transaction and PaymentsService inserts the payment and allocations and recomputes totals. In steps 10 to 13 PaymentsService calls `recordForCharges` with source `ALLOCATION`; CommissionService reads net, tax, allocated and already-recorded amounts for each charge, computes the target as the rate on the pre-tax collected amount capped at net, and inserts one entry per charge equal to the target minus what is already recorded. Step 14 returns the saved payment and receipt to the web app.

Behind the scenes — in simple terms. For each charge the Platform asks: "Given everything collected so far, how much commission should this charge have earned in total?" It then writes only the difference. That makes the ledger self-correcting: a second instalment adds a little more, an unapplied payment or a discount added later produces a negative entry, a voided charge brings the total back to zero, and running the same operation twice writes nothing the second time. Tax is never doctor revenue, so only the pre-tax part of what was paid counts.

Behind the scenes — technically. `CommissionService.recordForCharges(manager, chargeIds, source)` runs in the caller's money transaction under the caller's charge locks. It loads, per charge with a snapshot, `gross`, `net`, `tax` (all as paisa from `numeric(12,2)` columns), the plan's basis, the sum of `payment_allocations` and the sum of existing entries. The target comes from the pure function `commissionTarget` in `commission-math.ts`, using BigInt and half-away-from-zero rounding:

- collected = allocated (no tax) or round(allocated × net ÷ (net + tax)), capped at net; a `VOID` charge has net 0.
- Rate, `NET` basis: round(collected × rate ÷ 10000).
- Rate, `GROSS` basis: round(collected × gross × rate ÷ (net × 10000)), so the clinic absorbs the discount.
- Fixed amount: round(fixed × collected ÷ net).

The callers and source types are: `PaymentsService` collect and apply-advance (`ALLOCATION`, source id = the new payment, else an operation id), `PaymentsService.unapply` (`UNAPPLY`), and `InvoicesService.runEdit` (`CHARGE_ADJUST`, turned into `VOID` for a voided charge). Each entry stores `base_paisa` (collected so far), `rate_bps` and the signed `amount_paisa`. Invoice cancel needs no call because it is refused while the invoice has payments.

Rules and validations.

- One entry per (source type, source id, charge) (unique index `uq_commission_entries_source`).
- Commission is never added to charge or invoice totals.
- Entries already settled are never changed: a reversal after settlement simply lands, as a negative entry, in the next settlement.
- Overpayment beyond net goes to the patient's advance and never counts as collected.
- The `labCostPaisa` input of `commissionTarget` is not used for ordinary charges; only lab invoice lines use it (35.3.6).

Who can do it. Whoever performs the money operation (`billing.payment.collect`, `billing.invoice.edit`, and so on); the ledger itself needs no permission.

Data read and written. Reads `charges`, `compensation_plans`, `payment_allocations`, `commission_entries`. Writes `commission_entries` (insert only).

Effects on other modules. Feeds the earnings reports, the "Doctor commissions" report and the commission columns of other reports (Chapter 36), the finance overview, and payroll settlement (Chapter 38).

Error cases. None user-visible; an error rolls back the whole money operation.

Technical references: `backend/src/billing/commissions/commission-math.ts` · `CommissionService.recordForCharges`, `commissionTarget` · no web-app use

35.3.6 Earn commission on lab work (lab-line commissions)

Status: [Implemented] (backend only by design)

What it does. When lab work is billed to a patient on a **patient lab invoice**, the doctor named on each lab invoice line earns commission on what the patient pays for it, minus what the lab charged the clinic.

How the user does it. Nothing to do. It happens when a patient lab invoice is issued, paid, has advance applied, has a payment reversed, or receives a credit note.

Behind the scenes — in simple terms. For lab work, the doctor's share is calculated on the clinic's margin, not on the full price: if the patient pays for a crown and the lab charged the clinic part of that, only the remainder earns commission. The same "write only the difference" rule applies, so instalments and reversals correct themselves.

Behind the scenes — technically. `LabFinanceListeners` subscribe to lab-billing events inside the money transaction. On `invoiceIssued`, `LabCommissionService.snapshot` sets the same four snapshot columns on each non-void `lab_invoice_lines` row that has a doctor (plan in force on the issue date; a line without a mapped service counts as a `PROCEDURE`). On payment collected and advance applied (`ALLOCATION`), payment reversed (`UNAPPLY`) and credit note issued (`CHARGE_ADJUST`), `recordForInvoices` computes per line with `labLineCommissionTarget`: the invoice's net is its total minus credit notes (0 if void); each line's share of net and of collected money is proportional to its pre-tax amount; then `commissionTarget` runs with `labCostPaisa`. The lab cost is the item's cost from approved vendor bill lines (else its frozen snapshot) for a lab item line, the charge amount for an order-level charge passed on to the patient, and 0 for manual lines. With a lab cost, the rate applies to max(net − lab cost, 0) (or gross − lab cost on the `GROSS` basis); a fixed amount is not reduced. Entries go into the same `commission_entries` table with `lab_invoice_line_id` instead of `charge_id`, and the lab receivables ledger posts a journal "Doctor commission on lab work" (Commission Expense / Commission Payable).

Rules and validations.

- Each entry targets exactly one of a charge or a lab invoice line (check constraint `chk_commission_entries_target`).
- One entry per (source type, source id, lab line) (partial unique index).
- Nothing is snapshotted while `billing.commissions` is off; such lines never earn.

Who can do it. Whoever runs the lab billing operation (`lab.billing.manage` and related permissions, Chapter 42).

Data read and written. Reads `lab_invoice_lines`, `lab_invoices`, `lab_payment_allocations`, `order_charge_requests`, vendor bill and lab order item costs, `compensation_plans`. Writes `lab_invoice_lines` (snapshot), `commission_entries`, lab ledger journal rows.

Effects on other modules. Lab commission appears on My earnings, in settlements and payroll exactly like charge commission. The "Collected" column of the earnings reports counts only ordinary charges, not lab invoices.

Error cases. None user-visible.

Technical references: `backend/src/lab-finance/lab-commission.service.ts` · `LabCommissionService.recordForInvoices`, `labLineCommissionTarget` · no web-app use

35.3.7 Preview the doctor share while collecting payment

Status: [Implemented]

What it does. Shows, before a payment is saved, how much commission it would record, per doctor.

How the user does it.

1. On the billing desk, open **Collect payment** for a patient and choose what to pay.
2. Read the "Doctor share" line in the summary (only for users who may see earnings).

Behind the scenes — in simple terms. The server runs the same calculation as the real recording, on a "what if" basis, without saving anything.

Behind the scenes — technically. `POST /payments/preview` fills `doctorShare` only when the caller's access context can `earnings.all.view` and `billing.commissions` is enabled; otherwise `doctorShare` is `null`. `CommissionService.previewForAllocations` adds the planned allocation amounts to each charge, uses the tax share of charges that will be invoiced by this payment, and groups the differences per assignment with the doctor's name.

Rules and validations. Read-only; the frontend never computes commission itself.

Who can do it. `earnings.all.view`, feature `billing.commissions`. Default: Admin, Accountant. A Receptionist collects payment without seeing it.

Data read and written. Reads `charges`, `compensation_plans`, `payment_allocations`, `commission_entries`, `employees`.

Effects on other modules. None.

Error cases. None of its own.

Technical references: `backend/src/billing/payments/payments.service.ts` · `CommissionService.previewForAllocations` · `web-app/src/features/billing/components/collect-payment/CollectSummary.tsx`

35.3.8 See every doctor's earnings

Status: [Implemented]

What it does. Lists, per doctor at a location, the money collected on their charges, the commission earned, the part already settled and the part pending, for a date range.

How the user does it.

1. Open **Finance** → **Reports** → **Doctor earnings**.
2. Pick a date preset or a custom range.

Behind the scenes — in simple terms. For each assignment that has a commission plan or any commission entry, the server adds up collections and ledger entries written in the period, and splits the earned total into settled and pending.

Behind the scenes — technically. `GET /reports/doctor-earnings?from&to&locationId` → `CommissionReportsService.doctorEarnings`. The range is clinic dates at the location, converted to UTC instants; the default is the first of the current month to today. *Collected* = sum of `payment_allocations` created in the range on charges whose `performing_assignment_id` is the assignment; *earned* = sum of entries created in the range; *settled* = the part with a `settlement_id`; *pending* = earned − settled. The plan type shown is the version in force today. Another location of the organization can be requested with `locationId`, but only if the caller also holds `earnings.all.view` there (resolved like any request at that location). `GET /commissions?assignmentId&locationId&from&to` returns the raw ledger lines of a location (at most 1000, newest first); no screen calls it.

Rules and validations.

- Only assignments with a non-salary plan or with entries are listed.
- The web page does not offer the location switch.

Who can do it. `earnings.all.view`, feature `billing.commissions`. Default: Admin, Accountant.

Data read and written. Reads `employee_org_location_assignments`, `employees`, `locations`, `compensation_plans`, `payment_allocations`, `charges`, `commission_entries`.

Effects on other modules. None.

Error cases. 404 "Location not found" (another organization's location); 403 `NO_PERMISSION` "You cannot see earnings at that location".

Technical references: `backend/src/billing/commissions/commission-reports.service.ts` · `CommissionReportsService.doctorEarnings`, `entries` · `web-app/src/features/billing/pages/CommissionPages.tsx`

35.3.9 See my own earnings

Status: [Implemented]

What it does. Shows a doctor their own collected, earned, settled and pending amounts per location in the organization, and every ledger line behind them.

How the user does it.

1. Open **My earnings** in the sidebar.
2. Pick a date preset or a custom range.

Behind the scenes — in simple terms. The server filters by the signed-in user's own assignments. A doctor can never see another doctor's figures through this page.

Behind the scenes — technically. `GET /me/earnings?from&to&locationId` → `CommissionReportsService.myEarnings`: the same earnings query filtered on `a.user_id = signed-in user` across the organization, plus up to 1000 ledger lines (`entryRows`), which join both charges and lab invoice lines to show description, patient and invoice number. The date range uses the active location's timezone.

Rules and validations. Only the caller's assignments; optional filter by location.

Who can do it. `earnings.own.view`, feature `billing.commissions`. Default: Doctor; Admin and Accountant also hold it through `earnings.all.view`.

Data read and written. Reads the same tables as 35.3.8 plus `patients`, `invoices`, `lab_invoice_lines`, `lab_invoices`.

Effects on other modules. None.

Error cases. 403 `NO_PERMISSION`.

Technical references: `backend/src/billing/commissions/commissions.controller.ts` · `CommissionReportsService.myEarnings` · `web-app/src/features/billing/pages/CommissionPages.tsx`

35.3.10 See doctor revenue (billed and collected per doctor)

Status: [Implemented]

What it does. Shows, per doctor, the net amount billed on issued invoices, the discounts given and the pre-tax money collected in a period.

How the user does it.

1. Open **Finance** → **Reports** → **Doctor revenue** and pick a period.

Behind the scenes — in simple terms. Billed counts invoices by their issue date; collected counts payments by payment date, and money released back to advance subtracts.

Behind the scenes — technically. `GET /billing/reports/doctor-revenue?from&to` → `BillingReportsService.doctorRevenue`: one SQL with a `billed` part (active charges on `ISSUED` invoices, `issue_date` in range, grouped by `charges.doctor_id`) and a `collected` part (pre-tax share of `payment_allocations` created in the range), joined to `employees`. It does not read commission tables.

Rules and validations. Active location only; amounts in rupees (numeric columns).

Who can do it. `reports.financial.view`, feature `reports.financial` (off by default). Default: Accountant (scope `location`), Admin.

Data read and written. Reads `charges`, `invoices`, `payment_allocations`, `employees`.

Effects on other modules. None.

Error cases. 403 `NO_PERMISSION`.

Technical references: `backend/src/billing/reports/billing-reports.service.ts` · `BillingReportsService.doctorRevenue` · `web-app/src/features/billing/pages/BillingReportsPages.tsx`

35.3.11 Settle commission through payroll

Status: [Implemented]

What it does. At payroll generation, collects all unsettled commission entries of an assignment up to the end of the month into one settlement linked to that month's payroll record. Paying the payroll record pays the settlement; deleting an unpaid payroll record releases the entries.

How the user does it.

1. Generate payroll (on **Payroll**, or automatically on the 1st of the month).
2. Mark the payroll record as paid with **Mark as Paid**.

Behind the scenes — in simple terms. Payroll locks the doctor's unsettled ledger lines so they cannot be paid twice, adds their total to the payroll record and ties them to a settlement. If the doctor's unsettled lines add up to a negative number (only reversals), nothing is settled; the negative amount waits and reduces the next month's commission.

Behind the scenes — technically. `CommissionSettlements.lockUnsettled` selects entries of the assignment with `settlement_id IS NULL` and `created_at` before the end of the month's last clinic day (location timezone), `FOR UPDATE`. If at least one entry exists and the total is ≥ 0, `CommissionSettlements.create` inserts a `PENDING` row in `commission_settlements` (period start and end, total, payroll record id) and sets `settlement_id` on the entries. `PayrollService.markPaid` calls `CommissionSettlements.markPaid` in the same transaction (status `PAID`, `paid_at`, `paid_by`). `PayrollService.remove` calls `CommissionSettlements.release`: for a `PENDING` settlement it clears `settlement_id` on its entries and deletes the settlement; for a `PAID` one deletion of the payroll record is refused. The month-end flow is drawn in Chapter 38 (Figure 38.6).

Rules and validations.

- An entry can be in only one settlement (row locks plus `settlement_id IS NULL` on update).
- Settlement amounts are signed paisa; entries keep their amounts.
- Only active assignments at the location are processed by payroll.

Who can do it. `payroll.manage`, feature `hr.payroll`. Default: Admin.

Data read and written. Writes `commission_settlements`, `commission_entries.settlement_id`, `payroll_records`.

Effects on other modules. Settled and Pending columns of the earnings pages; payroll record commission amount, payslip and CSV.

Error cases. 400 "This payroll paid out commission and cannot be deleted".

Technical references: `backend/src/billing/commissions/commission-settlements.ts` · `CommissionSettlements.lockUnsettled`, `create`, `markPaid`, `release` · `web-app/src/features/employees/pages/PayrollPage.tsx`

35.3.12 Protect the ledger (append-only trigger and invariant I10)

Status: [Implemented] (backend only by design)

What it does. Makes sure commission history cannot be rewritten and that the ledger always matches the money.

How the user does it. Nothing to do. Violations appear only in the server logs.

Behind the scenes — in simple terms. The database itself refuses to delete commission lines or change their amounts. Every night a separate check recomputes, for every charge with a snapshot, what the commission should be and compares it with the sum in the ledger.

Behind the scenes — technically. The migration `DoctorCompensation1792403456699` creates the function `commission_entries_append_only()` and the trigger `trg_commission_entries_append_only` (BEFORE UPDATE OR DELETE): deletes raise "commission_entries is append-only: delete refused"; any change to the listed columns raises "only settlement_id may change"; `settlement_id` may move from a value back to NULL only while that settlement is `PENDING`. Invariant **I10** in `BillingInvariantChecker` recomputes the target in SQL numeric (deliberately not reusing `commission-math.ts`) and reports each charge whose recorded sum differs. `BillingInvariantJob` runs all invariants (I1–I10) every day at 02:30 server time, where scheduled jobs run, and only logs; nothing is repaired automatically. The same check is part of the per-patient invariant run.

Rules and validations.

- I10 covers charges only, not lab invoice lines.

Who can do it. Not applicable.

Data read and written. Reads `charges`, `compensation_plans`, `payment_allocations`, `commission_entries`.

Effects on other modules. None.

Error cases. A refused update raises a database error inside the operation, which rolls back.

Technical references: `backend/src/migrations/1792403456699-doctor-compensation.ts` · `BillingInvariantChecker.checkCommissions`, `BillingInvariantJob.run` · no web-app use

35.3.13 Choose a commission plan when adding an employee

Status: [Implemented]

What it does. Lets the first compensation version of a new employee be a commission or salary-plus-commission plan, instead of the default salaried plan.

How the user does it.

1. On **Employees**, click **Add Employee**.
2. In the "Compensation" section (shown only to users with `compensation.manage`), pick Salaried, Commission or Both and enter the rate and basis.
3. Save the employee.

Behind the scenes — in simple terms. Every new assignment gets a first plan version starting on the joining date. Without the compensation step it is a salaried plan with the entered salary (or zero).

Behind the scenes — technically. `POST /employees` with an optional `compensation` object; `EmployeesService.initialPlan` validates it before anything is saved, refuses commission types with 403 `COMMISSIONS_DISABLED` when `billing.commissions` is off and checks the role can consult. The plan is added in the same transaction as the assignment. The Employees chapter (Chapter 37) covers the rest of the create flow.

Rules and validations. As in 35.3.1; Both requires a salary.

Who can do it. `employees.manage` plus `compensation.manage` for the commission options. Default: Admin.

Data read and written. Writes `compensation_plans`, `commission_rules`.

Effects on other modules. As in 35.3.1.

Error cases. 403 `COMMISSIONS_DISABLED` "Doctor commissions are not enabled"; 422 `COMMISSION_ROLE_CANNOT_CONSULT`.

Technical references: `backend/src/employees/employees.service.ts` · `EmployeesService.initialPlan` · `web-app/src/features/employees/components/EmployeeForm.tsx`

35.3.14 Manual commission adjustments

Status: [Not implemented]

What it does. Nothing today. The source type `MANUAL` ("Reserved for an admin correction") exists in the enum and the web app has a label "Adjustment" for it, but no endpoint or service writes a `MANUAL` entry.

How the user does it. Not available.

Behind the scenes — in simple terms. Corrections happen only through the money operations themselves.

Behind the scenes — technically. `CommissionSourceType.MANUAL` is never used as a write source.

Rules and validations. None.

Who can do it. Not applicable.

Data read and written. None.

Effects on other modules. None.

Error cases. None.

Technical references: `backend/src/billing/commissions/commission.enums.ts` · `CommissionSourceType.MANUAL` · `web-app/src/features/billing/pages/CommissionPages.tsx`

35.4 Entities and data model

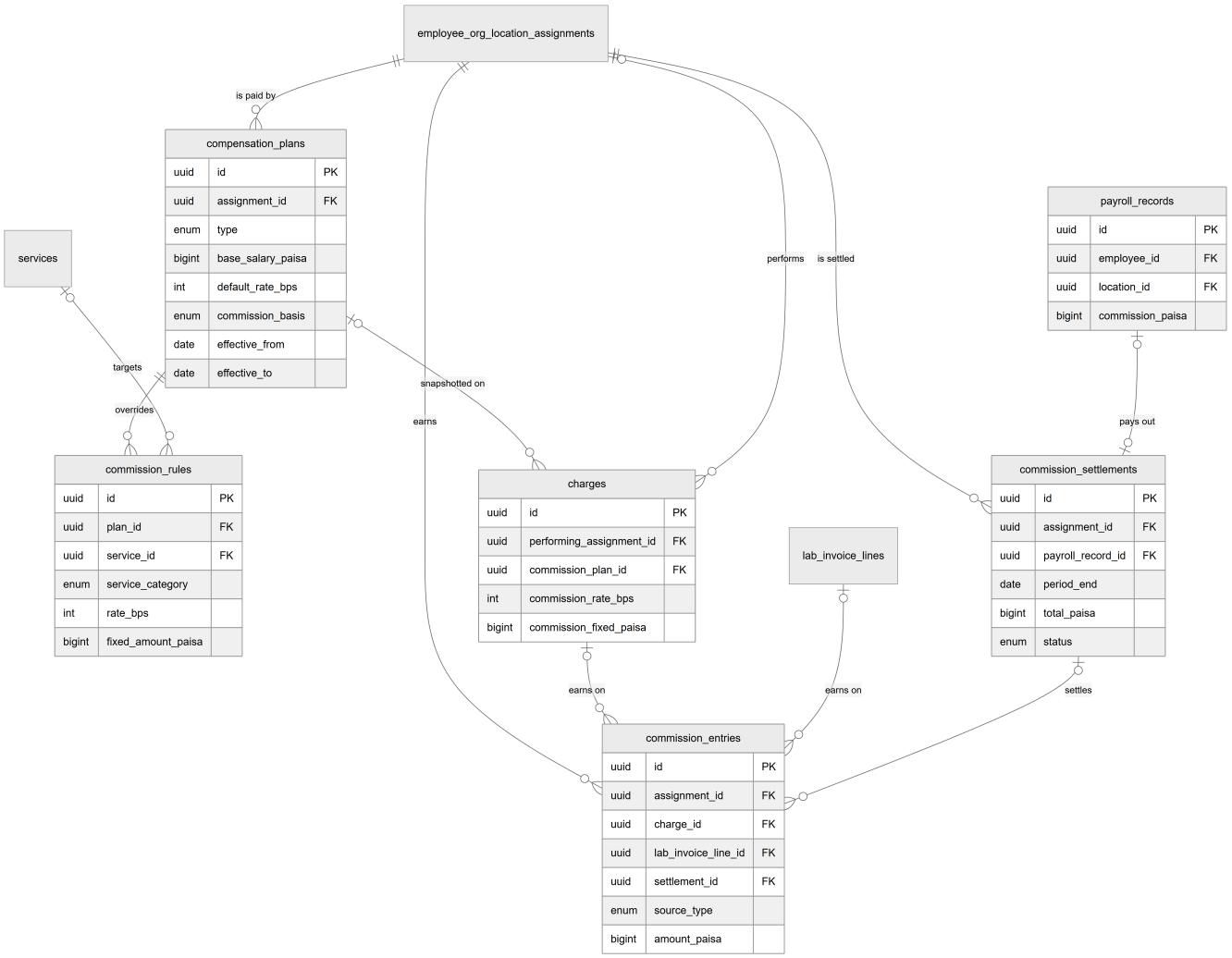

Figure 35.7 — Compensation plans, rules, the commission ledger and settlements.

The diagram has `employee_org_location_assignments` at the top. From it, solid lines go to `compensation_plans` ("is paid by"), `charges` ("performs", optional), `commission_entries` ("earns") and `commission_settlements` ("is settled"). `compensation_plans` shows `id`, `assignment_id`, `type`, `base_salary_paisa`, `default_rate_bps`, `commission_basis`, `effective_from` and `effective_to`; it links to `commission_rules` ("overrides") and, optionally, to `charges` ("snapshotted on"). `services` links optionally to `commission_rules` ("targets"). `commission_rules` shows `plan_id`, `service_id`, `service_category`, `rate_bps` and `fixed_amount_paisa`. `charges` shows its four snapshot columns. At the bottom, `commission_entries` shows `assignment_id`, `charge_id`, `lab_invoice_line_id`, `settlement_id`, `source_type` and `amount_paisa`, and receives optional links from `charges` and `lab_invoice_lines` (both "earns on") and from `commission_settlements` ("settles"). On the right, `payroll_records` (with `commission_paisa`) is linked one-to-one, optionally, with `commission_settlements` ("pays out"), which shows `assignment_id`, `payroll_record_id`, `period_end`, `total_paisa` and `status`.

The relationships in words:

- Each compensation plan belongs to exactly one assignment. Deleting the assignment deletes its plans (ON DELETE CASCADE). Versions of one assignment never overlap (GiST exclusion constraint, which needs the `btree_gist` extension the migration creates).
- Each commission rule belongs to one plan (CASCADE) and targets either one service (CASCADE when the service is deleted) or one category.

- A charge points to the performing assignment and the plan it was snapshotted with. These are plain columns on `charges`; the snapshot does not change when the plan changes.
- Each commission entry belongs to one assignment and earns on exactly one charge or one lab invoice line. The assignment, charge, lab line and settlement links are ON DELETE RESTRICT, so none of them can be deleted while entries point to them.
- Each settlement belongs to one assignment (RESTRICT) and to at most one payroll record (one-to-one, RESTRICT); a payroll record has at most one settlement.
- Money columns are integer paisa (`bigint`): `base_salary_paisa`, `fixed_amount_paisa`, `amount_paisa`, `base_paisa`, `total_paisa`. Rates are integer basis points. `charges` amounts themselves are numeric rupee columns, converted exactly to paisa in SQL.
- Indexes: plans by assignment and start; entries by assignment and time, by charge, by lab line, and a partial index on unsettled entries per assignment; settlements by assignment and period end.

The migration also backfilled one `SALARY` plan per existing assignment from the deprecated `gross_salary` column, effective from the joining date.

35.5 Status lifecycles

35.5.1 Commission entry and settlement

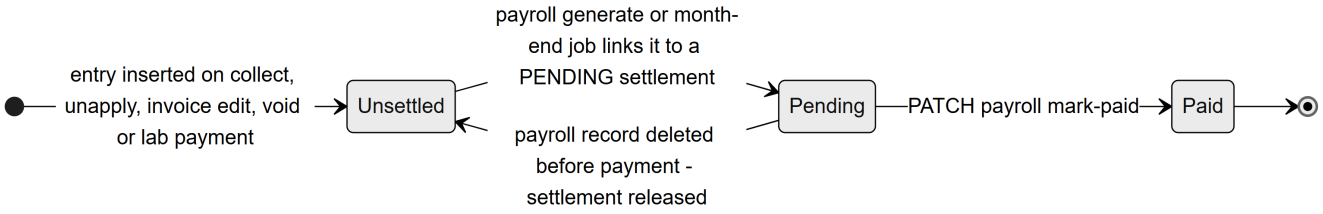

Figure 35.8 — The life of a commission entry, from earning to payout.

The diagram has three states. An entry starts as **Unsettled** when it is inserted on a collection, unapply, invoice edit, void or lab payment. It moves to **Pending** when payroll generation or the month-end job links it to a `PENDING` settlement. From Pending, two arrows leave: back to Unsettled when the payroll record is deleted before payment and the settlement is released, and forward to **Paid** through `PATCH /payroll/:id/mark-paid`. Paid leads to the end marker.

Walking through each state:

- **Unsettled** — `settlement_id` is NULL. The entry counts as "Pending" on the earnings pages. A net-negative group of unsettled entries stays here and offsets the next month.
- **Pending** — the entry points to a settlement whose status is `PENDING`; the earnings pages count it as "Settled" already, although the money is not yet paid.
- **Paid** — the settlement is `PAID`; the entry can never be released again (the trigger refuses).

The entry's amount never changes in any state. The two stored settlement statuses are `PENDING` and `PAID`.

35.5.2 Compensation plan versions

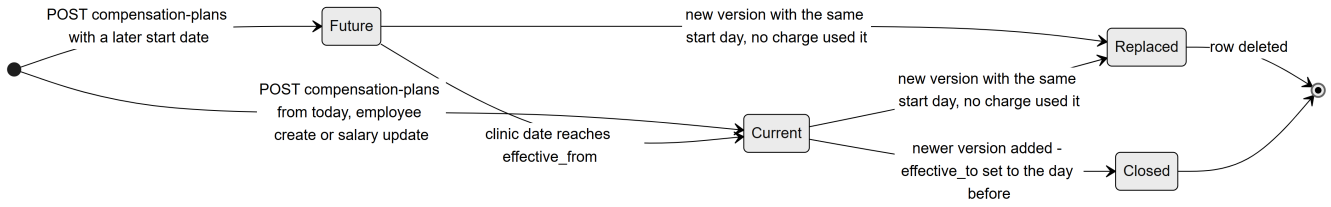

Figure 35.9 — How a compensation plan version is created, becomes current, and ends.

The diagram shows four states. A version enters as **Future** when it is created with a later start date, or as **Current** when it starts today (from the Compensation tab, from employee creation, or from a salary update). Future becomes Current when the clinic date reaches `effective_from`. Current becomes **Closed** when a newer version is added and its `effective_to` is set to the day before. Future or Current becomes **Replaced** when a new version with the same start day is saved and no charge has used it; the replaced row is deleted. Closed and Replaced lead to the end marker.

These states are not stored: the code derives "current" from the dates and the clinic date (`isCurrent`). Overrides of a Closed version cannot be edited (`COMPENSATION_PLAN_CLOSED`).

35.6 Connections to other modules

- **Employees (Chapter 37).** Plans hang from assignments. Creating an employee creates the first version; "Update Salary" on the employee page adds a new version from today and carries over the commission settings. The role decides whether commission is allowed (`consultation.conduct`).
- **Consultation (Chapter 31) and Billing (Chapter 34).** Visit charges take the snapshot when created; collect, apply advance, unapply and invoice edit write ledger entries in the same transaction. The billing desk shows the doctor share to authorised users.
- **Lab finance (Chapter 42).** Patient lab invoice lines earn commission on the margin over lab cost and post a commission journal.
- **Payroll (Chapter 38).** Settles entries monthly, pays settlements, releases them on delete.
- **Dashboard and reports (Chapter 36).** The "Doctor commissions" report (staff category) and commission columns in revenue and cost reports and the finance overview read `commission_entries`.
- **Features and roles (Chapters 10 and 11).** `billing.commissions` must be entitled and enabled; the three permissions are dropped when it is off.

35.7 Limitations and known gaps

- `hr.commissions` is a feature key that gates nothing; only `billing.commissions` matters.
- No `MANUAL` adjustment path exists, although the enum and UI label do.
- `GET /commissions` (raw ledger of a location) has no screen; an admin can see per-doctor totals on "Doctor earnings" but not the individual ledger lines of other doctors.
- The earnings reports' "Collected" column counts ordinary charges only; lab invoice collections are not included, while lab commission is.
- Invariant I10 checks ordinary charges only; lab-line commission has no nightly check.
- The append-only trigger was written before lab lines existed; its list of frozen columns does not include `lab_invoice_line_id`.

- Settlement covers only active assignments: commission of a deactivated doctor stays unsettled until the assignment is reactivated.
- Turning `billing.commissions` off stops new snapshots, but charges that already have one keep earning.
- The Compensation tab trigger is shown whenever the user holds `compensation.manage`, but its content appears only if the employee record has an assignment at the active location.
- Plan and rule changes are not written to the general audit log; versions keep `created_by` only.

35.8 Owner's FAQ

If I raise a doctor's rate today, does last month's commission change? No. Each charge keeps the rate stored when it was created. Only charges created after the new version starts use the new rate.

When exactly does a doctor earn? When patient money is applied to the doctor's charge, not when the invoice is issued. Half paid means half the commission.

What happens if a payment is reversed after the commission was paid out in payroll? The reversal writes a negative entry. It is settled with next month's payroll and reduces that month's commission. If the doctor has only negative entries, nothing is settled and the balance waits.

Does tax count towards commission? No. Only the pre-tax part of what the patient paid counts.

Can a receptionist see how much the doctor gets from a payment? Not with the default role. The doctor share appears only to users with `earnings.all.view` (Admin, Accountant by default).

Can a doctor see another doctor's earnings? No. "My earnings" is filtered on the signed-in user's own assignments on the server.

Can an assistant or hygienist be on commission? Only if their role holds `consultation.conduct`. Otherwise the commission options are disabled and the server refuses with `COMMISSION_ROLE_CANNOT_CONSULT`.

Where is commission stored? In `commission_entries`, one signed row per change, in paisa. The database refuses deletes and amount changes.

What is the difference between "after discount" and "before discount"? After discount (`NET`) means the doctor shares the discount. Before discount (`GROSS`) means the clinic absorbs it and the doctor earns as if no discount had been given.

Chapter 36 — Dashboard and Reports

This chapter describes how the Platform shows a clinic its numbers: the home dashboard, the finance overview (profit and loss), the clinic overview (patients, appointments and waiting), and a library of 37 reports that can be read on screen or downloaded as Excel or PDF. It is written for the product owner, who needs to know where every figure comes from, who may see it and where the numbers can disagree. The module only reads data; it owns no tables and changes nothing except one audit row per download.

36.1 What this module is for

The module answers two questions for the people who run a clinic: "How is the clinic doing?" and "Where is the money?". It has five screens:

- **Dashboard** (`/dashboard`): the home page after sign-in. Sidebar item **Dashboard** (key `nav.dashboard`). It has no page permission; each tile appears only when the user holds the permission behind it.
- **Clinic overview** (`/clinic-overview`): patients, appointments, waiting and doctors for a period. Sidebar item **Clinic overview** (key `nav.clinic_overview` , section Clinic, needs `reports.clinical.view`).
- **Finance overview** (`/finance`): the owner's profit and loss dashboard. Sidebar item **Finance overview** (key `nav.finance` , section Finance, needs `finance.overview.view`).
- **All reports** (`/reports`): the report library. Sidebar item **All reports** (key `nav.reports` , in the Finance → Reports group, needs `reports.financial.view` or `reports.clinical.view`).
- **Report viewer** (`/reports/<report key>`): one report on screen, opened from the library or from a link on an overview card. No sidebar entry of its own.

The old address `/financial-summary` has no screen any more: it redirects to `/dashboard` . The same Reports sidebar group also holds **Doctor revenue**, **Doctor earnings** and **Billing activity**; those screens belong to the Billing and Commissions chapters (Chapters 34 and 35).

Who uses it:

- The **owner** and the **Admin** role: everything, for one location or for all locations together.
- **Accountant**: financial reports, the finance overview and downloads, for the active location only.
- **Doctor**: clinical reports with scope `own` , so only the eight reports that can be narrowed to one doctor, showing only their own rows.
- **Receptionist** and **Assistant**: no report permission; they see only the dashboard tiles their other permissions allow.

Three features gate the module, all off by default for a new organization: `reports.financial` ("Financial Reports"), `reports.clinical` ("Clinical Reports") and `reports.exports` ("Data Exports"). The dashboard's money tiles also need the older feature `billing.financial_summary` ("Financial Dashboard", off by default).

36.2 Screens

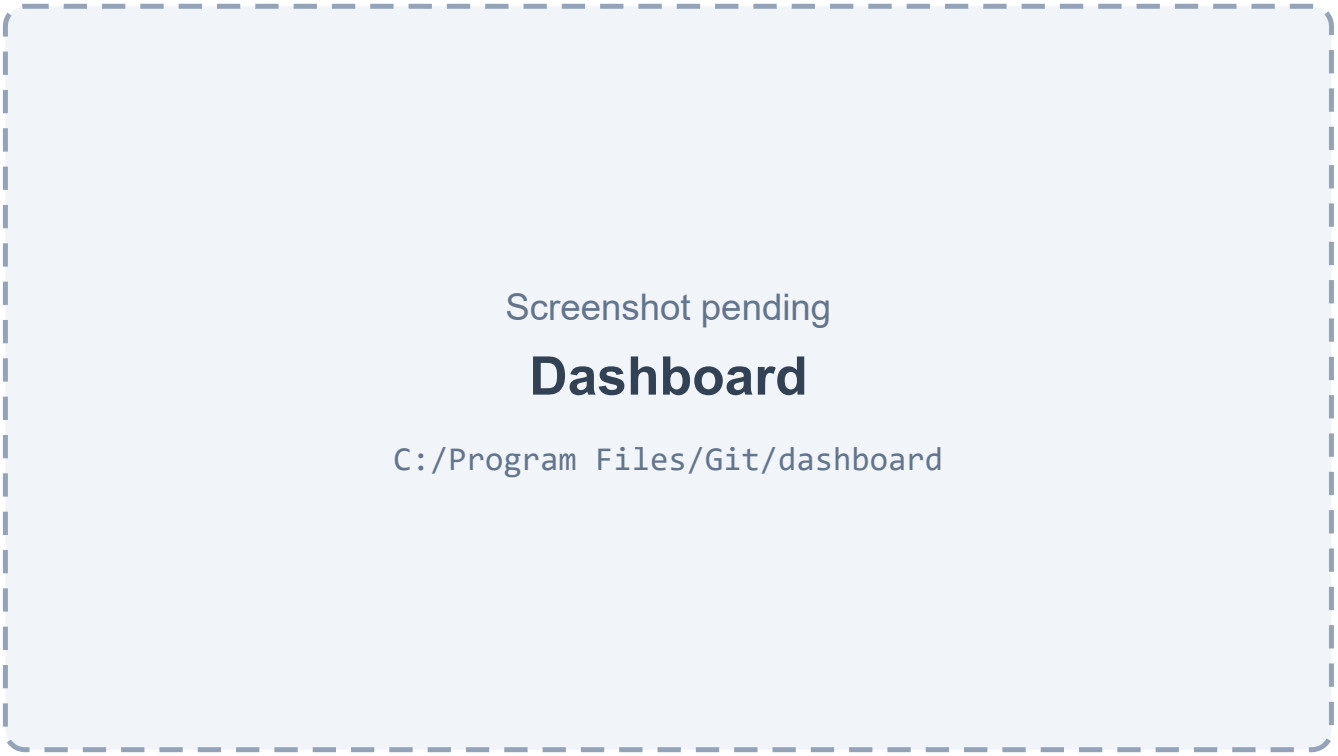

Figure 36.1 — Dashboard (screenshot pending).

The dashboard was redesigned on 2026-10-23; this describes the current code. The header shows today's date, a greeting such as "Good morning, Dr. Example One", and the organization and location name. On the right are a month picker (the last 12 months), a **Refresh** icon button, **Collect payment** (for users with `billing.desk.use`) and **New appointment** (for users with `appointments.book`). Below come up to four KPI tiles: **Active patients** (needs `patients.view`), **Visits today** with a progress bar of "done" and "in clinic" (needs `appointments.view`), **Revenue · Oct** with a daily sparkline and the change against last month (needs `reports.financial.view` and the `billing.financial_summary` feature) and **Outstanding** with an overdue count (needs `billing.invoice.view`). Each tile links to its module. The next row holds the **Revenue overview** card (month figures Revenue, Expenses, Net profit, Avg. invoice and a six-month revenue-versus-expenses bar chart) and the **Visits** donut (Completed, Upcoming, In clinic, No-show, Cancelled, plus Completion rate and Avg. wait). The last row holds **Today's schedule** (up to 8 visits with a **View all** link) and **Top procedures** (the month's six best-earning procedures). A user who lacks a permission simply does not see that tile or card.

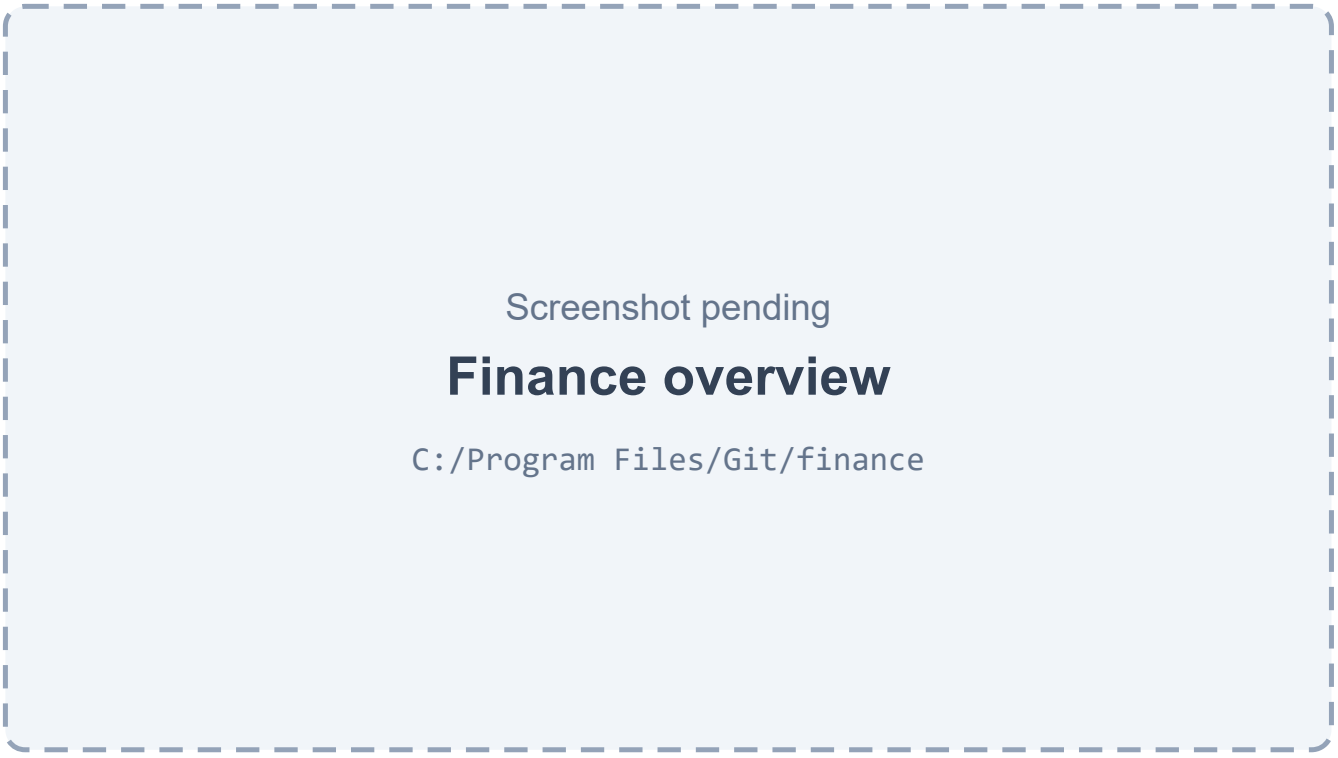

Figure 36.2 — Finance overview (screenshot pending).

The finance overview has the title "Finance overview" and a subtitle naming the location scope, the period and the period it is compared with. The top-right buttons are **Download P&L** (Excel or PDF, for users with `reports.export`) and **All reports**. The filter bar offers period presets (**This month**, **Last month**, **This quarter**, **This year**, **Last year**), a **Custom range** picker, a **This location / All locations** switch (only for org-wide access in an organization with more than one location) and a **Collected / Billed** switch with a short hint. Four KPI tiles show revenue, **Total costs**, **Net profit** (or Net loss) with margin, and **Patients owe**. Cards follow: **Last 12 months** (revenue, costs and profit chart, link "Cash book"), **Needs attention** (overdue patient balances, overdue lab bills, lab bills to pay, patient advances held, plus data warnings), **Profit & loss** (the statement, link "Full statement"), **Where the money goes** (cost breakdown), **Billing** (Invoices, Average invoice, Patients billed, Discounts given), **Revenue by doctor**, **Top services** and **Cash and receivables** (money in by payment account and unpaid balances by age). Each card link opens the matching report.

Screenshot pending

Clinic overview

`C:/Program Files/Git/clinic-overview`

Figure 36.3 — Clinic overview (screenshot pending).

The clinic overview has the same filter bar without the revenue basis, and an **All reports** button. Its KPI tiles are **Patients seen** (with visits and returning patients), **New patients** (with the walk-in share), **Show-up rate** (with appointments booked) and **Average wait** (with average consultation length). Cards follow: **Visits per day / week / month** (first visits and returning patients), **Needs attention** (missed follow-ups in the last 30 days, no-shows to call back, follow-ups due this week, patients who stopped coming, late lab orders), **Appointment outcomes**, **Busiest times** (check-ins by weekday and hour), **Doctors** (visits and waiting per doctor), **Patient mix** (tabs Source, Gender, Age) and **What we treat** (tabs Diagnoses, Treatments). Card links open the related report. This screen has no download button.

Screenshot pending

Reports

`C:/Program Files/Git/reports`

Figure 36.4 — The report library (screenshot pending).

The library has the title "Reports" and the subtitle "Every report in one place — view on screen or download as Excel / PDF.", with buttons to the clinic and finance overviews. The shared filter bar sets the period and location scope used by the quick downloads. Extra filters are a search box ("Search reports"), category chips (All plus each category that has at least one visible report) and **Clear filters** with a count. A card / table view switch is on the right. In card view the reports are grouped under category headings; each card shows an icon, the title, a one-line description, an **Open** button and two icon buttons, **Download Excel** and **Download PDF**. Table view lists Report, Category, What it shows and the same actions. Only reports the user may open are listed.

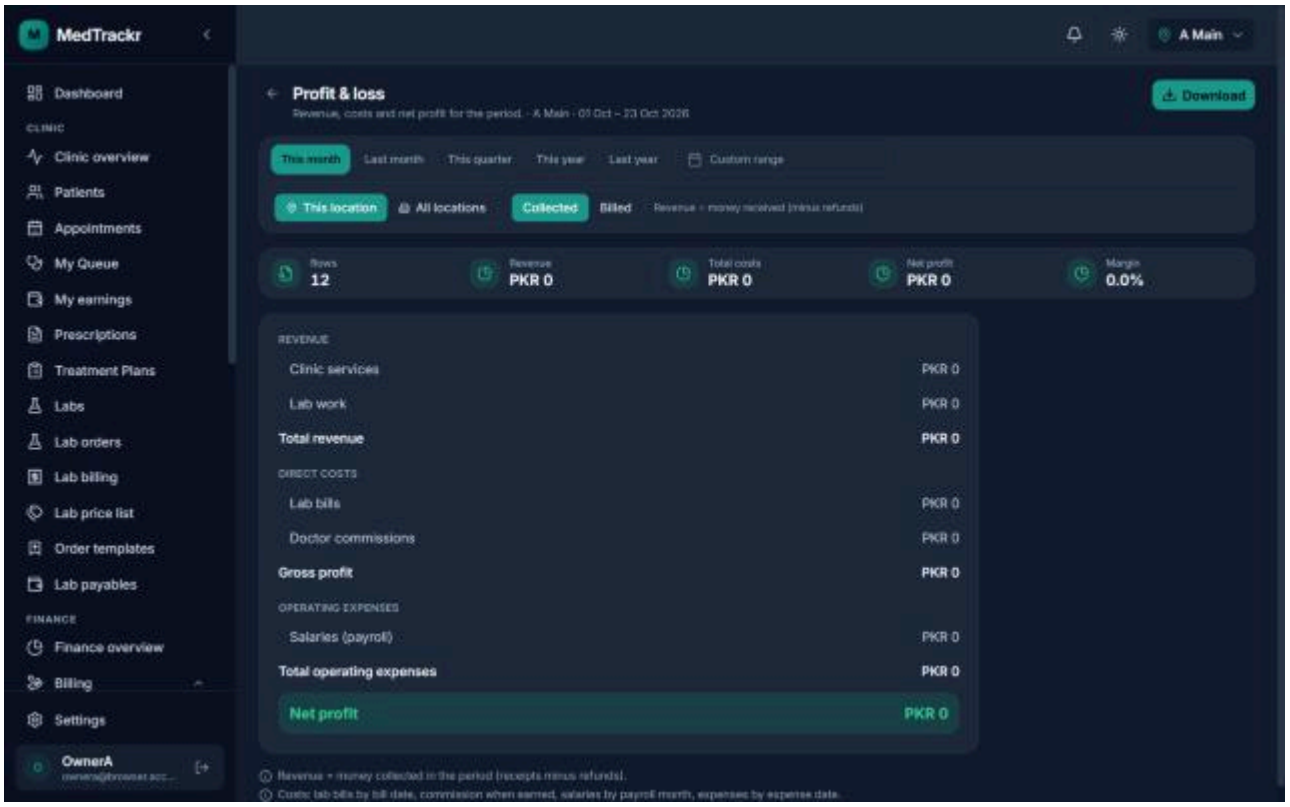

Figure 36.5 — The Profit & loss report in the report viewer.

This screenshot was captured on 2026-10-23 in dark theme and shows the former working name "MedTrackr" in the top-left corner; the product is called the Platform. On the left is the owner's full sidebar: Dashboard, then the Clinic section (Clinic overview, Patients, Appointments, My Queue, My earnings, Prescriptions, Treatment Plans, Labs, Lab orders, Lab billing, Lab price list, Order templates, Lab payables), then the Finance section (Finance overview, Billing) and Settings. The page has a back arrow, the title "Profit & loss" and the line "Revenue, costs and net profit for the period. · A Main · 01 Oct – 23 Oct 2026". A **Download** button sits top right. The filter bar shows the presets with **This month** selected, **Custom range**, the switch **This location / All locations** and the switch **Collected / Billed** with the hint "Revenue = money received (minus refunds)". A strip of five tiles shows Rows 12, Revenue PKR 0, Total costs PKR 0, Net profit PKR 0 and Margin 0.0%. The statement lists REVENUE (Clinic services, Lab work, Total revenue), DIRECT COSTS (Lab bills, Doctor commissions, Gross profit), OPERATING EXPENSES (Salaries (payroll), Total operating expenses) and a highlighted Net profit line, all PKR 0 because the test clinic had no money movement. Two notes at the bottom explain the revenue basis and when each cost counts.

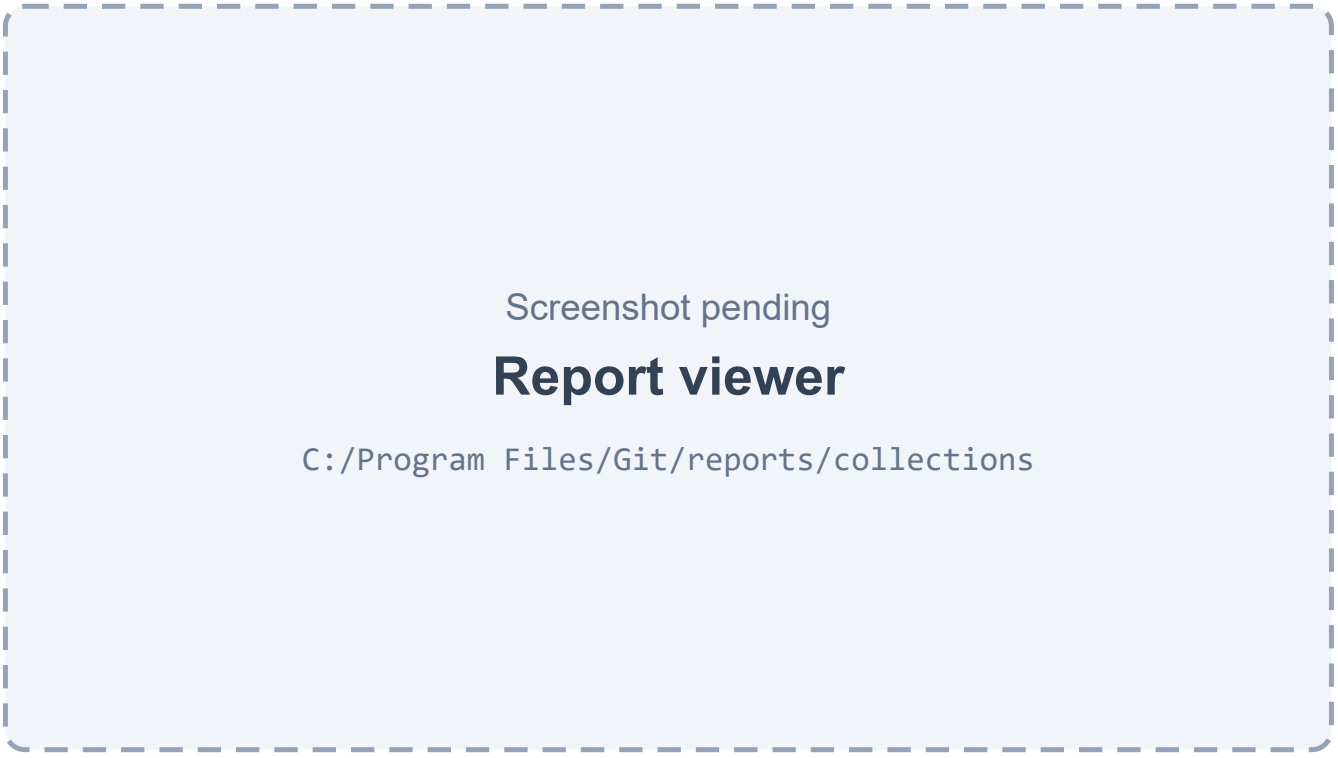

Figure 36.6 — A list report, such as "Payments received", in the report viewer (screenshot pending).

Every other report uses the same viewer. The header has a back button ("Back to reports"), the report title, its description, the location scope and the period, and a **Download** menu with **Excel (.xlsx)** and **PDF**. The filter bar adds a doctor picker ("All doctors") for reports that can be narrowed to one doctor, and an expense category picker for the expense register. Snapshot reports show "Balances as of today" instead of the period. Below come summary tiles (Rows plus the report's own summary or its column totals), an amber warning when more than 5,000 rows exist ("Showing the first 5,000 rows. Download Excel for up to 50,000, or pick a shorter period."), the sortable, paged table, and the report's notes. Empty results say "No records in this period."; empty results with a doctor or category filter say "No results for your filters".

36.3 Features

36.3.1 See the home dashboard

Status: [Partial] — the rendered dashboard uses real data; the money tiles need a second, older feature, and a mock data module with unused widgets is still in the code.

What it does. Gives each signed-in user a one-page view of today and the selected month, limited to what their role may see.

How the user does it.

1. Sign in, or click **Dashboard** in the sidebar.
2. Optionally pick another month in the month picker, or click **Refresh**.
3. Click a tile to jump to Patients, Appointments, Finance overview or Invoices.

Behind the scenes — in simple terms. The page asks several existing services for their numbers: the patient list for a count, the appointment statistics for today and for the month, the invoice statistics for what is owed, and the financial summary for the month's revenue, expenses and top procedures. It only asks for what the user may see, so a receptionist never gets an empty "PKR 0" money tile.

Behind the scenes — technically. Calls: `GET /patients?page=1&limit=1&isActive=true` (count), `GET /appointments/stats` twice (today and month), `GET /appointments` (today, 8 rows), `GET /invoices/stats`, `GET /financial-summary/monthly?month&year` and `GET /financial-summary/trend?months=6`. The financial-summary endpoints run `FinancialSummaryService`: monthly revenue is payments in the month minus refunds minus the tax part of allocations to taxed charges; expenses are the `expenses` table only; net profit is revenue minus those expenses; average invoice, paid-invoice count, revenue by day, expenses by category, payment-account split and the top 5 procedures (by `charges.net_amount` on issued invoices) come with it. The trend loops month by month for revenue and expenses. Each widget query uses `retry: false`, so one refused call blanks one widget only. `features/dashboard/services/dashboard.api.ts` is a mock module (fixed numbers such as "1630" patients and a simulated delay). Its four hooks (`useDashboardStats`, `useDashboardChart`, `useSpendingBreakdown`, `useDashboardAppointments`) are exported from the dashboard barrel, but nothing imports that barrel, and the widgets built for it (`AnalyticsChartCard`, `AppointmentsTable`, `SpendingBreakdownCard`, the dashboard `StatCard`, `KpiCard`) and several older cards (`AppointmentStatusCard`, `ExpenseCategoryChart`, `RecentAppointmentsCard`, `TopProceduresCard`, `DashboardSkeletons`) are not rendered anywhere. No mock number reaches a user.

Rules and validations.

- The money tile, Revenue overview and Top procedures need `reports.financial.view` and the feature `billing.financial_summary`.
- Dashboard "Net profit" is collected revenue minus recorded expenses only. It does not subtract salaries, lab bills or commissions, so it differs from the P&L (36.3.3).
- Today's and the month's ranges are built from the browser's date and sent as UTC midnight to UTC midnight; the financial summary uses calendar months of the payment date. Figures near midnight can differ from the location's clinic day.
- Only the active location is counted.

Who can do it. Any signed-in member. Tiles: `patients.view`, `appointments.view`, `billing.invoice.view`, `reports.financial.view` (+ feature `billing.financial_summary`). Admin sees everything; Receptionist sees patients, visits and outstanding; Assistant sees patients and visits; Doctor sees visits (own); Accountant sees revenue and outstanding.

Data read and written. Reads `patients`, `appointments`, `queue_entries`, `queue_sessions`, `invoices`, `payments`, `refunds`, `payment_allocations`, `charges`, `expenses`. Writes nothing.

Effects on other modules. None.

Error cases. A refused or failed call leaves that one widget empty or at zero; the rest of the page still loads. 403 `NO_PERMISSION` or a feature refusal on `/financial-summary/*` when the feature is off.

Technical references: `backend/src/billing/financial-summary.service.ts` · `FinancialSummaryService.getMonthlySummary`, `getRevenueComparison`, `DashboardPage` · `web-app/src/features/dashboard/pages/DashboardPage.tsx`

36.3.2 Read the finance overview

Status: [Implemented]

What it does. Shows the owner the period's revenue, costs, net profit and margin, compared with the period before, with a 12-month trend, a cost breakdown, revenue per doctor and service, cash by account, money owed both ways and warnings about incomplete data.

How the user does it.

1. Click **Finance overview** in the sidebar.

2. Pick a period preset or a custom range; choose **Collected** or **Billed**; optionally choose **All locations**.
3. Read the cards; click a card link (for example "Full statement" or "Who owes") to open the matching report.
4. Click **Download P&L** for an Excel or PDF statement.

Behind the scenes — in simple terms. The server builds the profit and loss for the chosen period and for the comparison period, the monthly figures for the last 12 months, and several side lists, all in one request. It also checks for three common data problems: a month whose payroll was not generated, salaries entered both as payroll and as "Salary" expenses, and lab bills still in draft.

Behind the scenes — technically. `GET /reports/finance/overview` → `FinanceOverviewService.overview`. It runs nine queries in parallel: P&L of the period and of `previousRange` (a period starting on the 1st compares with the same days of the previous month; a whole month with the whole previous month; any other range with the same number of days just before it), `FinanceDataService.monthlyComponents` for 12 trailing months, `balances` (open patient balances on issued invoices and live lab invoices, aged Not due / 1–30 / 31–60 / 61–90 / 90+ days against the due date; payable and overdue lab bills; patient advances = payments minus allocated minus refunded), `topDoctors` (billed: `charges.net_amount` on issued invoices; collected: pre-tax part of allocations), `topServices` (top 6 by net billed), `collectionsByAccount` (clinic payments minus refunds per `payment_account_name`, plus lab payments per method), `counts` and `warnings`. Lists keep the top 5 and fold the rest into "Others". Money is summed in paisa and returned in rupees. The scope comes from `ReportScopeService` with the rule "gate on `finance.overview.view`, reach from `reports.financial.view`".

Rules and validations.

- Dates are clinic-local `YYYY-MM-DD`; default period is the first of this month to today; "from" must not be after "to"; at most 3 years.
- "All locations" needs `reports.financial.view` with scope `org`.
- Change percentages are null when the earlier value is 0.

Who can do it. `finance.overview.view` (unscoped), feature `reports.financial`; all-locations reach needs `reports.financial.view` at `org`. Default: Admin (all locations), Accountant (active location). Not Doctor, Receptionist or Assistant.

Data read and written. Reads `payments`, `refunds`, `payment_allocations`, `charges`, `invoices`, `services`, `lab_payments`, `lab_refunds`, `lab_invoices`, `vendor_bills`, `commission_entries`, `payroll_records`, `expenses`, `employees`, `locations`, `organizations`. Writes nothing.

Effects on other modules. None.

Error cases. 403 `NO_PERMISSION`; 403 "Your access does not cover all locations together."; 400 "Dates must be YYYY-MM-DD.", ""From" must be on or before "to".", "A report can cover at most 3 years."; the screen shows "Couldn't load the finance overview" with a retry button.

Technical references: `backend/src/reports/finance/finance-overview.service.ts` · `FinanceOverviewService.overview`, `ReportScopeService.resolve` · `web-app/src/features/reports/pages/FinanceDashboardPage.tsx`

36.3.3 Read the profit and loss statement

Status: [Implemented]

What it does. Produces a classic profit and loss statement: revenue (clinic services, lab work), direct costs (lab bills, doctor commissions), gross profit, operating expenses (salaries from payroll and each expense category) and net profit or net loss, each with its share of revenue.

How the user does it.

1. Open **All reports** and click **Open** on "Profit & loss", or click "Full statement" on the finance overview.
2. Pick the period, location scope and **Collected** or **Billed**.
3. Optionally click **Download → Excel (.xlsx)** or **PDF**.

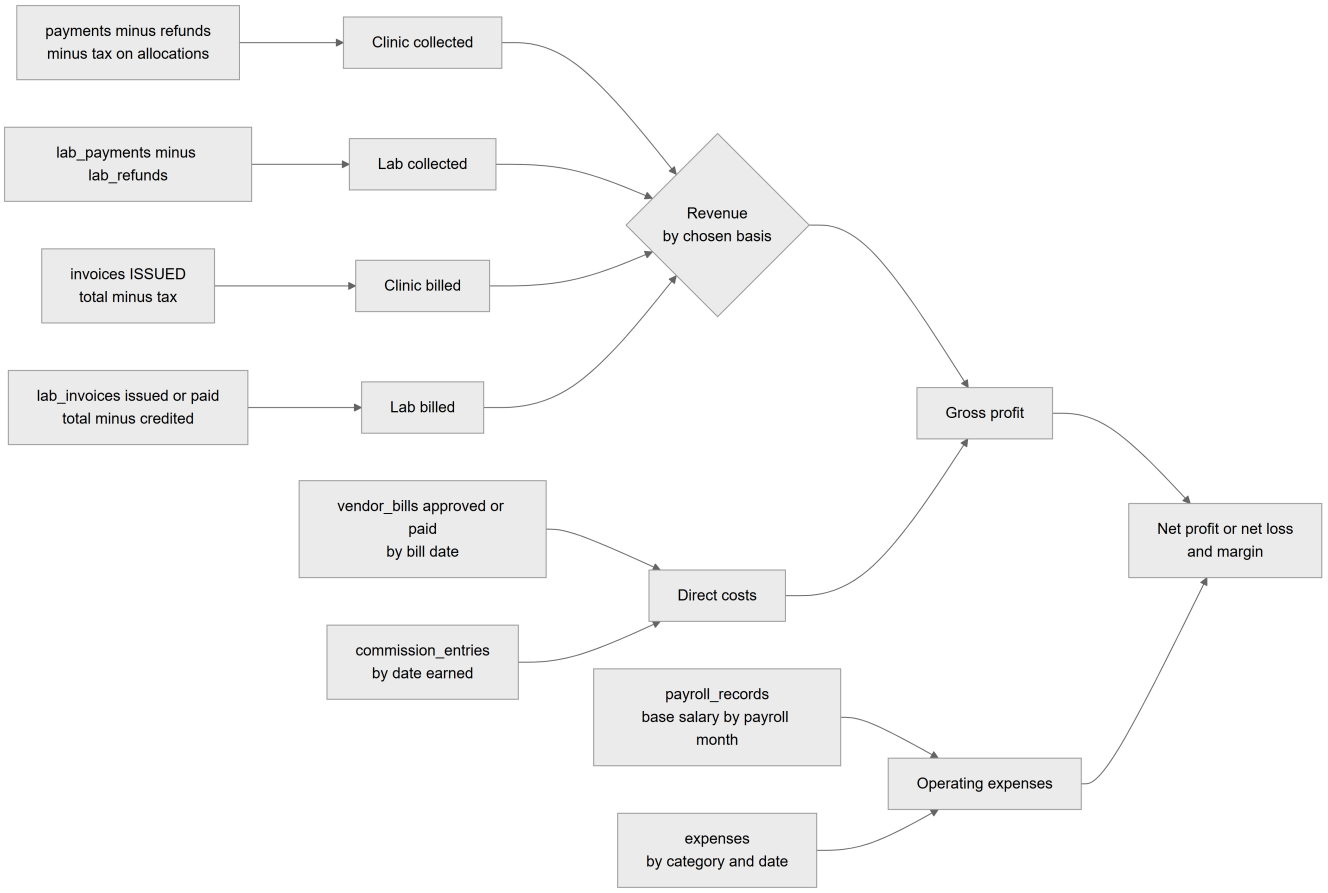

Figure 36.7 — Where each line of the profit and loss comes from.

The diagram reads from left to right. Four source boxes feed four revenue figures: "payments minus refunds minus tax on allocations" gives Clinic collected; "lab_payments minus lab_refunds" gives Lab collected; "invoices ISSUED total minus tax" gives Clinic billed; "lab_invoices issued or paid total minus credited" gives Lab billed. All four flow into a diamond "Revenue by chosen basis": the Collected basis uses the two collected figures, the Billed basis the two billed figures. Below, "vendor_bills approved or paid by bill date" and "commission_entries by date earned" flow into Direct costs. Revenue and Direct costs together give Gross profit. "payroll_records base salary by payroll month" and "expenses by category and date" flow into Operating expenses. Gross profit and Operating expenses together give the final box "Net profit or net loss and margin".

Behind the scenes — in simple terms. Revenue can be counted two ways. **Collected** counts money that actually came in during the period (clinic receipts and lab receipts, minus refunds, with sales tax taken out because tax is not the clinic's income). **Billed** counts work invoiced in the period (issued clinic invoices without tax, plus lab invoices minus credits), whether paid or not. Costs are always counted in the period they belong to, whatever basis is chosen: what labs billed the clinic by the bill date, doctor commission when it was earned, salaries by payroll month, and expenses by their expense date. Gross profit is revenue minus lab bills and commissions; net profit also subtracts salaries and expenses.

Behind the scenes — technically. `FinanceDataService.monthlyComponents` runs one SQL `UNION ALL` that returns, per month, `clinicCollected` (`payments.amount` by `payment_date`, minus `refunds.amount` by `refund_date`, minus `amount - preTaxAllocationSql` for `payment_allocations` on charges with `tax_amount > 0`, by allocation date in the location's timezone), `labCollected` (`lab_payments.amount_paisa` by `received_on` minus `lab_refunds` by `refunded_at`), `clinicBilled` (`invoices.total_amount - tax_amount`, status `ISSUED`, by

`issue_date`), `labBilled` (`lab_invoices.total_paisa − credited_paisa`, status issued, partially paid or paid, by `issued_on`), `labCost` (`vendor_bills.total_paisa`, status approved, partially paid or paid, by `bill_date`), `commission` (`commission_entries.amount_paisa` by `created_at`), `salaries` (`payroll_records.base_salary_paisa`, or `salary_amount` for older rows without a breakdown, by payroll year and month) and `expense` per category (`expenses.amount` by `expense_date`). Clinic billing tables store numeric rupees and are multiplied by 100; lab, commission and payroll tables store integer paisa. `buildProfitLoss` then computes revenue, direct costs, gross profit, operating total, net profit and margin (one decimal, null when revenue is 0). The report definition `profit-loss` turns this into section, line, subtotal and total rows, with summary tiles Revenue, Total costs, Net profit/Net loss and Margin, and two notes.

Rules and validations.

- Payroll counts base salary only; the commission part of payroll is already counted when earned, so it is not counted twice.
- Draft lab bills and cancelled clinic invoices are not counted.
- Expense category "Salary" is shown as "Salaries (expenses)"; if the clinic also runs payroll, the finance overview warns that salaries may be counted twice.
- Zero-value expense categories are hidden; expense lines are sorted largest first.

Who can do it. `reports.financial.view` (scope `location` or `org`), feature `reports.financial`. Download also needs `reports.export` (feature `reports.exports`). Default: Admin, Accountant.

Data read and written. As 36.3.2. Writes an audit row only on download (36.3.7).

Effects on other modules. None.

Error cases. As 36.3.2; 403 `NO_PERMISSION` without `reports.financial.view`.

Technical references: `backend/src/reports/finance/profit-loss.ts` · `buildProfitLoss`, `FinanceDataService.monthlyComponents`, `financeReports` · `web-app/src/features/reports/components/PnlStatement.tsx`

36.3.4 Read the clinic overview

Status: [Partial] — works for org- and location-scoped users; a Doctor with the default role sees the sidebar item but the page is refused (deduced from code, not observed).

What it does. Shows how the clinic is running: patients seen, new patients, appointments and show-up rate, waiting and consultation times, trends, busiest hours, doctors, patient mix, top diagnoses and treatments, and "Needs attention" lists.

How the user does it.

1. Click **Clinic overview** in the sidebar.
2. Pick a period and, if allowed, **All locations**.
3. Read the cards; use the tabs on Patient mix and What we treat; click a card link to open the detailed report.

Behind the scenes — in simple terms. The server counts encounters (consultations), new patient registrations, appointments and queue entries for the period and the comparison period. The trend is grouped by day for up to 14 days, by week up to 120 days and by month beyond. "Needs attention" ignores the period and looks at today: follow-ups missed in the last 30 days with no later visit, follow-ups due in the next 7 days, patients not seen for 180 days or more, no-shows of the last 30 days with no later booking or visit, and open lab orders past their expected date.

Behind the scenes — technically. `GET /reports/clinic/overview` → `ClinicOverviewService.overview`, eleven queries in parallel. Patients seen and visits come from `patient_encounters`; new patients from `patients.created_at` (deleted patients excluded); appointments, no-show and show-up rate from `appointments` (show-up = due minus no-show over due, where due = completed, checked in or no-show); average wait = check-in to call (or to consultation start), average consultation = start to finish of completed entries, walk-in share from `queue_entries` (transferred entries excluded); heatmap by ISO weekday and hour of check-in in the location's timezone; demographics by gender and age band (Under 18, 18–29, 30–44, 45–59, 60+, Not recorded); top 6 diagnoses (free text, grouped ignoring case and spaces) and treatments. Scope rule: `reports.clinical.view` without a doctor filter, so a grant with scope `own` is refused.

Rules and validations.

- Same date rules as 36.3.2.
- The Doctor template holds `reports.clinical.view` with scope `own`. The sidebar item checks only that the permission exists, so it is shown, but `ReportScopeService` answers 403 for `own` scope here.

Who can do it. `reports.clinical.view` at `location` or `org`, feature `reports.clinical`. Default: Admin. Accountant, Receptionist and Assistant: no. Doctor: refused (see rules).

Data read and written. Reads `patient_encounters`, `patients`, `appointments`, `queue_entries`, `queue_sessions`, `treatments`, `lab_orders`, `employees`. Writes nothing.

Effects on other modules. None.

Error cases. 403 "Your access covers only your own data; this report has no per-doctor view."; the date errors of 36.3.2; the screen shows "Couldn't load the clinic overview" with a retry button.

Technical references: `backend/src/reports/clinic/clinic-overview.service.ts` · `ClinicOverviewService.overview` · `web-app/src/features/reports/pages/ClinicOverviewPage.tsx`

36.3.5 Browse the report library

Status: [Implemented]

What it does. Lists every report the user may open, grouped by category, with search, category filters and quick downloads.

How the user does it.

1. Click **All reports** in the sidebar (Finance → Reports).
2. Search, or click a category chip; switch between card and table view.
3. Click **Open**, or use **Download Excel** / **Download PDF** for the current period.

Behind the scenes — in simple terms. The server holds a fixed list of 37 report definitions and returns only those the user's permissions allow. A doctor with own-only access gets only the reports that can be narrowed to one doctor.

Behind the scenes — technically. `GET /reports` (`@AnyMember`) → `ReportsService.catalog(access)`: keeps a definition when `access.scope(permissionOf(def))` is set, and, for scope `own`, only when the definition has `filters.doctor`. Each entry carries key, title, description, category, filter spec and permission. `permissionOf` maps feature `reports.clinical` to `reports.clinical.view` and everything else to `reports.financial.view`. The catalog is cached for 10 minutes in the web app. Categories are shown in the order Profit & cash, Revenue & collections, Expenses, Staff & payroll, Patients, Appointments, Waiting & queue, Clinical, Lab.

Rules and validations.

- Search is debounced by 300 ms and matches title and description.

- Quick downloads use "All locations" only when the user's grant for that report reaches the whole organization.

Who can do it. Page: `reports.financial.view` or `reports.clinical.view`. Default: Admin (37 reports), Accountant (15 financial reports), Doctor (8 doctor-filterable clinical reports).

Data read and written. Nothing from the database for the list.

Effects on other modules. None.

Error cases. "Couldn't load reports" with a retry button; "No reports available" when the list is empty; "No reports match your filters."

Technical references: `backend/src/reports/reports.service.ts` · `ReportsService.catalog`, `permissionOf` · `web-app/src/features/reports/pages/ReportsHubPage.tsx`

36.3.6 Run a report on screen

Status: [Implemented]

What it does. Runs one report for a period, a location scope and optional filters, and shows summary tiles, a sortable table and notes.

How the user does it.

1. Open a report from the library or an overview card.
2. Set the period (snapshot reports have none), the location scope, and the doctor or expense category filter if the report has one.
3. Read the tiles, table and notes; click ← to go back.

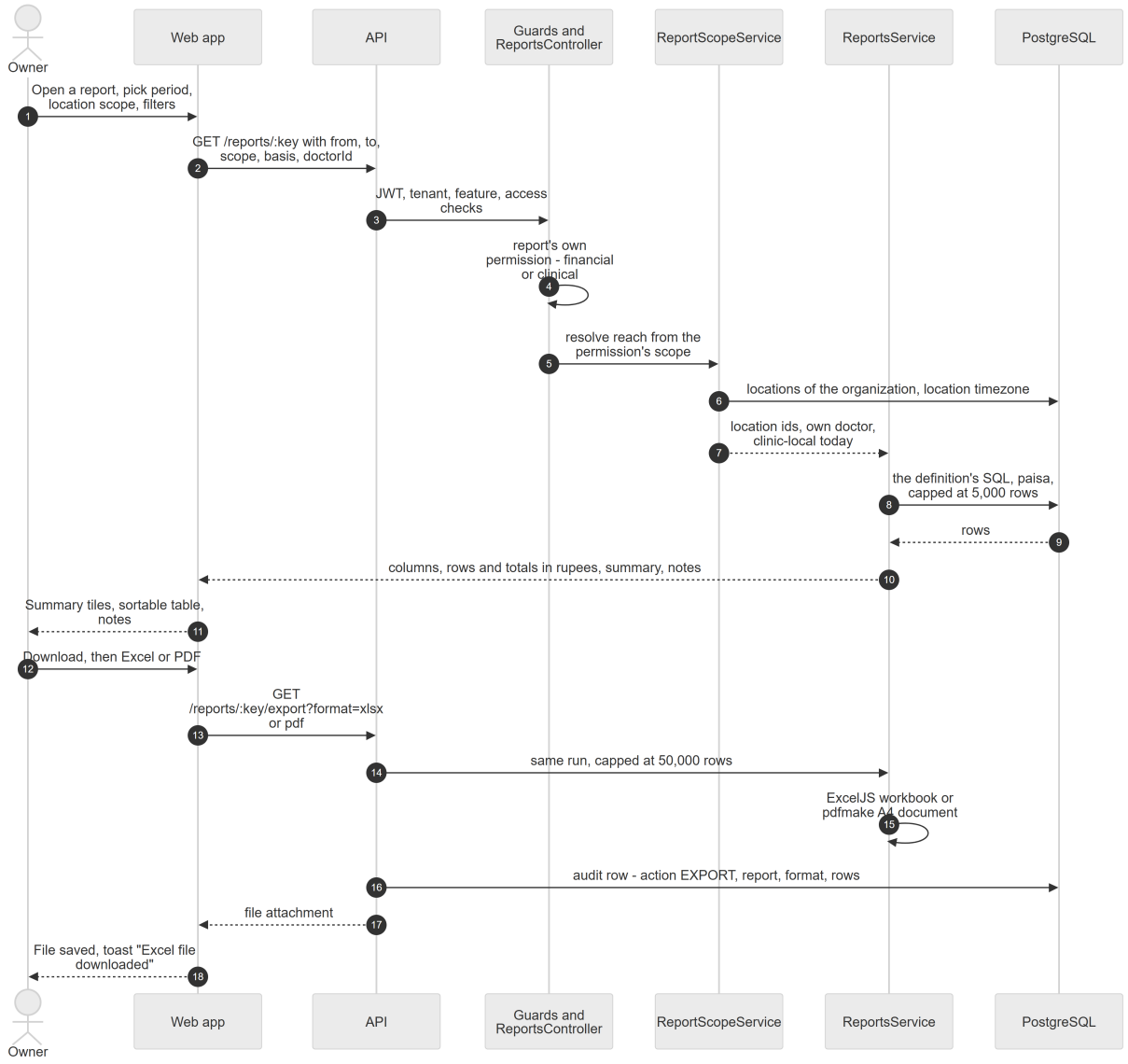

Figure 36.8 — A report request, then an Excel or PDF download.

The diagram has seven participants: the Owner, the Web app, the API, "Guards and ReportsController", ReportScopeService, ReportsService and PostgreSQL. In steps 1 to 11, the owner opens a report and picks period, scope and filters; the web app calls `GET /reports/:key` with from, to, scope, basis and doctorId; the global guards check the JWT, tenant, feature and access; the controller checks the report's own permission (financial or clinical); the scope service reads the organization's locations and the location's timezone and hands the engine the location ids, the doctor (for own-only users) and the clinic-local today; the engine runs the definition's SQL in paisa with a cap of 5,000 rows; rows come back and the engine returns columns, rows and totals in rupees, the summary and notes; the screen shows tiles, a sortable table and notes. In steps 12 to 18, the owner clicks Download and picks Excel or PDF; the web app calls `GET /reports/:key/export?format=xlsx` or `pdf`; the engine repeats the same run capped at 50,000 rows, builds an ExcelJS workbook or a pdfmake A4 document; the API writes an audit row with action EXPORT, report, format and row count; the file comes back as an attachment and the screen shows the toast "Excel file downloaded".

Behind the scenes — in simple terms. Every report is a definition: a title, a category, the filters it supports, its columns and one database query. The same rows feed the screen, the Excel file and the PDF, so they always match. Money is calculated in paisa (whole numbers, so no rounding drift) and shown in rupees with two decimals. Dates are clinic days in the active location's timezone.

Behind the scenes — technically. `GET /reports/:key` (`@AnyMember`; the handler checks `permissionOf(def)`) → `ReportScopeService.resolve` → `ReportsService.view`. Snapshot ("as of") reports run with from = to = today. The engine fetches the cap plus one row to detect truncation, adds the note "Totals cover only the rows shown." when truncated, computes a totals row from columns marked `total` (in paisa), converts money to rupees and adds meta: organization name, scope label, period, basis and a clinic-local "generated" time. `doctorId` from an own-only user is forced to their employee id whatever the request says. The viewer refuses to run until the catalog has loaded, so it knows the report's filter spec.

Rules and validations.

- Unknown key → 404 "Report not found"; the screen says "Report not found — This report doesn't exist or isn't available to you."
- On screen at most 5,000 rows; dates and range as 36.3.2.
- The P&L is drawn as a statement; all other reports as a table (25 rows per page by default).

Who can do it. The report's permission: `reports.financial.view` or `reports.clinical.view` (scope `own` / `location` / `org`); feature `reports.financial` or `reports.clinical`. Default as in 36.3.5.

Data read and written. Per report (table in 36.3.8). Writes nothing.

Effects on other modules. None.

Error cases. 403 `NO_PERMISSION` "Missing permission: reports.clinical.view" (or financial); 403 for own-only users on a report without a doctor filter; 404; 400 date errors; screen "Couldn't load this report" with retry.

Technical references: `backend/src/reports/reports.controller.ts` · `ReportsService.view`, `ReportDefinition` · `web-app/src/features/reports/pages/ReportViewerPage.tsx`

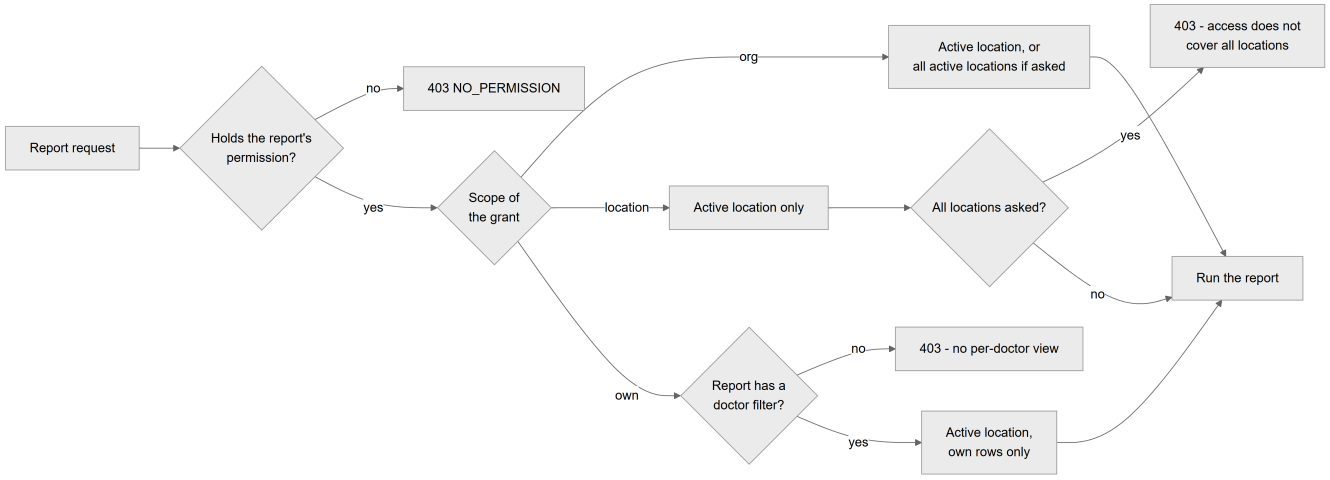

Figure 36.9 — How the permission's scope decides which locations and rows a report covers.

The flowchart starts at "Report request" and asks "Holds the report's permission?"; "no" ends at "403 NO_PERMISSION". On "yes" it branches on "Scope of the grant". With "org", the report runs for the active location, or for all active locations if asked. With "location", it covers the active location only; if all locations were asked for, it ends at "403 - access does not cover all locations"; otherwise it runs. With "own", it asks "Report has a doctor filter?"; "no" ends at "403 - no per-doctor view", "yes" runs the report for the active location with the user's own rows only. All successful paths end at "Run the report".

36.3.7 Download a report as Excel or PDF

Status: [Implemented]

What it does. Saves the report with the current filters as an Excel workbook or an A4 PDF, and records who downloaded what.

How the user does it.

1. In the viewer click **Download** → **Excel (.xlsx)** or **PDF**; on the finance overview click **Download P&L**; or use the icon buttons in the library.
2. The button shows "Preparing…"; the file is saved, named like `profit-loss_2026-10-01_to_2026-10-23.xlsx`, and a toast confirms.

Behind the scenes — in simple terms. The server runs the report again with a larger limit, turns it into a file and writes an audit entry before sending it, because clinic data is leaving the system.

Behind the scenes — technically. `GET /reports/:key/export?format=xlsx|pdf` (`@Can('reports.export')` plus the report's own permission). Up to 50,000 rows. Excel (ExcelJS): one sheet with title, organization and scope line, period line, summary block, a frozen and filterable header, typed numbers and dates with formats (money `#,##0.00` with red negatives, percent, dd-mmm-yyyy), a bold totals row and the notes. PDF (pdfmake): A4, landscape for wide registers, title block, summary strip, zebra table with a repeating header, bold totals, "Page x of y" footer; Urdu text is set in the bundled Noto Naskh Arabic font; the PDF engine may read only its bundled fonts and no URLs. `AuditService.record` writes `entityType: 'report'`, action `EXPORT`, with report key, format, period, scope label and row count. Headers: `Content-Disposition: attachment`, `Cache-Control: no-store`.

Rules and validations.

- PDF is refused above 5,000 rows.
- The download control is hidden for users without `reports.export`.

Who can do it. `reports.export` (high-risk, feature `reports.exports`) plus the report's permission. Default: Admin, Accountant. Not Doctor.

Data read and written. Reads as the report. Writes one audit-log row (Chapter 60).

Effects on other modules. The export appears in the audit trail.

Error cases. 400 "This report has 6,200 rows — too many for a PDF (max 5000). Download Excel or pick a shorter period."; 403 `NO_PERMISSION`; the web app shows the server message as an error toast, or "Could not download the report.".

Technical references: `backend/src/reports/exporters/xlsx.exporter.ts` · `ReportsService.export`, `exportXlsx`, `exportPdf` · `web-app/src/features/reports/components/ExportMenu.tsx`

36.3.8 The 37 reports

Status: [Implemented]

What it does. Provides 37 ready-made reports in 9 categories. 35 are written out individually; 2 ("Top diagnoses", "Top complaints") are built by one shared helper over free-text encounter fields.

Key	Title	Category	Feature	Purpose
`profit-loss`	Profit & loss	finance	reports.financial	Revenue, costs and net profit for the period (Collected / Billed basis).
`cash-book`	Daily cash book	finance	reports.financial	Per day: clinic and lab receipts, refunds, expenses, paid to labs, salaries paid, net cash.
`collections`	Payments received	revenue	reports.financial	Every receipt, clinic and lab, with patient and account.

Key	Title	Category	Feature	Purpose
`collections-by-account`	Collections by account	revenue	reports.financial	Cash, bank and wallet totals: what should be in each account.
`invoices`	Invoice register	revenue	reports.financial	Issued invoices (clinic and lab) with paid and balance.
`outstanding`	Outstanding balances	revenue	reports.financial	Who owes money, aged by due date (as of today).
`doctor-revenue`	Doctor performance	revenue	reports.financial	Billed, collected and commission per doctor.
`service-revenue`	Revenue by service	revenue	reports.financial	Which treatments earn the most, with discounts given.
`discounts`	Discounts given	revenue	reports.financial	Every discount on issued invoices and who gave it.
`refunds`	Refunds	revenue	reports.financial	Money returned to patients, with the reason.
`expenses`	Expense register	expenses	reports.financial	Every expense with category, method and who recorded it (category filter).
`expense-summary`	Expenses by category	expenses	reports.financial	Where operating money goes, with each category's share.
`payroll`	Payroll register	staff	reports.financial	Salary and commission paid out per employee per month.
`commissions`	Doctor commissions	staff	reports.financial	Commission earned on collections, settled and pending (doctor filter).
`lab-bills`	Lab bills	lab	reports.financial	What labs billed the clinic, what is paid and what is still owed.
`new-patients`	New patients	patients	reports.clinical	Everyone registered in the period, with source, age and city.
`patient-sources`	Where patients come from	patients	reports.clinical	New patients by source, and how many came and paid.
`patient-demographics`	Patient demographics	patients	reports.clinical	Patients seen by visit type, gender, age group and city.
`lapsed-patients`	Patients who stopped coming	patients	reports.clinical	Not seen for 6+ months, a call list, most valuable first (as of today).
`top-patients`	Top patients	patients	reports.clinical	Patients with the most billing and visits in the period.
`appointments-daily`	Appointments by day	appointments	reports.clinical	Booked, completed, cancelled and no-show appointments per day (doctor filter).
`appointments-by-doctor`	Appointments by doctor	appointments	reports.clinical	Each doctor's bookings, completions and no-show rate.
`no-shows`	No-shows & cancellations	appointments	reports.clinical	Who missed or cancelled, why, and how often (doctor filter).
`visit-mix`	Walk-ins vs appointments	appointments	reports.clinical	Arrivals with and without an appointment, per day (doctor filter).
`busy-hours`	Rush hours	appointments	reports.clinical	Arrivals by hour and weekday (doctor filter).

Key	Title	Category	Feature	Purpose
`queue-by-doctor`	Waiting time by doctor	operations	reports.clinical	How long patients wait for each doctor and how long consultations take.
`queue-daily`	Waiting time by day	operations	reports.clinical	Arrivals, patients seen and waiting time per clinic day (doctor filter).
`left-unseen`	Left without being seen	operations	reports.clinical	Patients who checked in but were skipped, cancelled or left (doctor filter).
`doctor-activity`	Doctor activity	clinical	reports.clinical	Patients seen, treatments, prescriptions, plans and lab orders per doctor.
`treatment-plans`	Treatment plan conversion	clinical	reports.clinical	Plans proposed per doctor, how many were accepted and finished.
`diagnoses`	Top diagnoses	clinical	reports.clinical	The most common diagnoses doctors recorded.
`complaints`	Top complaints	clinical	reports.clinical	The chief complaints recorded at visits.
`treatments-done`	Treatments done	clinical	reports.clinical	How often each treatment was performed and for how many patients.
`medicines`	Most prescribed medicines	clinical	reports.clinical	Medicines on finalized prescriptions (doctor filter).
`follow-ups`	Follow-ups	clinical	reports.clinical	Follow-ups due in the period: who came back and whom to call (doctor filter).
`lab-turnaround`	Lab turnaround	lab	reports.clinical	How fast each lab returns work, on-time rate, late orders and remakes.
`lab-delays`	Late lab orders	lab	reports.clinical	Open orders past the lab's promised date (as of today).

The table lists all 37 reports by their code key, the title users see, the category that groups them in the library, the feature (and therefore the permission) that gates them, and a one-line purpose taken from the report's own description. Fifteen reports are financial (`reports.financial` → permission `reports.financial.view`) and twenty-two are clinical or operational (`reports.clinical` → `reports.clinical.view`). The category and the feature are independent: "Lab bills" is a lab report gated as financial, while "Lab turnaround" and "Late lab orders" are lab reports gated as clinical. Three reports are snapshots "as of today" with no period: Outstanding balances, Patients who stopped coming and Late lab orders. Nine reports offer a doctor filter (the eight clinical ones marked plus Doctor commissions); only those can be run by an own-only user. The third feature, `reports.exports` , gates downloads of any report, not a report of its own.

How the user does it. Open the report from the library (36.3.5) and run it (36.3.6).

Behind the scenes — in simple terms. Each report is one read-only query over the tables of the module it describes: billing, lab billing, lab payables, payroll, commissions, expenses, patients, appointments, the queue, encounters, treatments, prescriptions and plans.

Behind the scenes — technically. Definitions live in `definitions/finance.reports.ts` (2), `revenue.reports.ts` (8), `cost.reports.ts` (5), `patient.reports.ts` (5), `visit.reports.ts` (8) and `clinical.reports.ts` (9, two of them via `textFrequency`). `ReportsService` registers them in that order and indexes them by key. Free-text reports note that the same thing spelled differently shows as separate lines. Wide registers are marked landscape for PDF.

Rules and validations. "Lapsed" means not seen for 180 days. Free-text grouping ignores case and repeated spaces.

Who can do it. As 36.3.6.

Data read and written. Read-only, per report. Writes nothing.

Effects on other modules. None.

Error cases. As 36.3.6.

Technical references: `backend/src/reports/definitions/` · `financeReports`, `revenueReports`, `costReports`, `patientReports`, `visitReports`, `clinicalReports`, `textFrequency` · `web-app/src/features/reports/lib/report-categories.ts`

36.3.9 Daily financial summary

Status: [Partial] — backend only; no screen calls it since `/financial-summary` was folded into the dashboard.

What it does. Returns one day's collections (net of refunds) per payment account, invoices issued that day, the outstanding amount and the overdue invoice count.

How the user does it. Not available in the web app. The address `/financial-summary` redirects to `/dashboard`. The day's takings per account are shown on the Payments page instead (Chapter 34).

Behind the scenes — in simple terms. A small summary query for one date at the active location.

Behind the scenes — technically. `GET /financial-summary/daily?date` → `FinancialSummaryService.getDailySummary`. With no date the server uses today's UTC date, not the location's. A web hook (`useDailySummary`) exists but no component uses it. The Payments page uses a different endpoint, `GET /payments/daily-collection` (permission `billing.desk.use`, clinic-local today), which belongs to Billing.

Rules and validations. Active location only.

Who can do it. `reports.financial.view`, feature `billing.financial_summary`. Default: Admin, Accountant.

Data read and written. Reads `payments`, `refunds`, `invoices`.

Effects on other modules. None.

Error cases. 403 when the feature is off or the permission is missing.

Technical references: `backend/src/billing/financial-summary.controller.ts` · `FinancialSummaryController.daily` · `web-app/src/routes/financial-summary.tsx`

36.4 Entities and data model

The Reports module owns no tables and has no entities. It reads, with raw SQL, the tables of other modules and writes only one row per download into the general audit log. Its "model" is in code:

- **Report definition** (`ReportDefinition`): key, title, description, category, optional feature (default `reports.financial`), filter spec (`dates`: range or asOf; `doctor`; `expenseCategory`; `basis`), columns (type text, money, number, decimal, date or percent; a `total` flag; a PDF width weight), a landscape flag and a `run` function.
- **Report scope** (`ReportScope`): organization, the list of location ids, a label such as "Main location" or "All locations (3)", whether it is the whole organization, an optional doctor (for own-only users), the active location's timezone and the clinic-local today.
- **Report view** (`ReportView`): what the API returns: columns, rows, totals, summary, notes, truncated flag and meta.

The relationships to stored data, in words:

- Financial reports and the P&L read billing (`payments` , `refunds` , `payment_allocations` , `charges` , `invoices` , `expenses` , `services`), lab billing (`lab_invoices` , `lab_payments` , `lab_refunds`), lab payables (`vendor_bills` , `vendor_payments`), commissions (`commission_entries`) and payroll (`payroll_records`). There are no foreign keys from reports to these tables; the link is the SQL.
- Clinical reports and the clinic overview read `patients` , `appointments` , `queue_sessions` , `queue_entries` , `patient_encounters` , `treatments` , prescriptions, treatment plans and `lab_orders` .
- Every query filters by `organization_id` and the scope's location ids.
- Money: clinic billing columns are numeric rupees, lab, payroll and commission columns are integer paisa; the engine converts everything to paisa, sums, and returns rupees. "PKR" is the default currency, not a limit.

36.5 Status lifecycles

The module has no lifecycle of its own: reports and dashboards are computed on request and stored nowhere. The statuses it filters on belong to other modules and are described there: invoice status `ISSUED` versus `CANCELLED` (Chapter 34), lab bill status draft, approved, partially paid, paid (Chapter 42), appointment and queue statuses (Chapters 29 and 30) and lab order status (Chapter 40).

36.6 Connections to other modules

- **Billing (Chapter 34).** Source of clinic revenue, receipts, refunds, discounts, expenses and outstanding balances. Dashboard money tiles use the billing financial-summary endpoints.
- **Commissions (Chapter 35).** Commission entries are a direct cost in the P&L and feed "Doctor commissions" and "Doctor performance". The "Doctor revenue" and "Doctor earnings" screens in the same sidebar group belong there.
- **Payroll (Chapter 38).** Base salary by payroll month is an operating cost; salaries paid appear in the cash book on the paid date.
- **Lab chapters (Chapters 40–42).** Lab invoices and lab receipts are revenue; vendor bills are a direct cost; lab orders feed turnaround and late-order reports.
- **Appointments, Queue, Consultation, Patients (Chapters 28–31).** Source of every clinical and operational figure.
- **Access control (Chapter 57).** Four permissions and three features decide what each user sees and how far it reaches.
- **Audit logging (Chapter 60).** Each download writes an EXPORT audit row.

36.7 Limitations and known gaps

- The dashboard's "Net profit" subtracts only recorded expenses, not payroll, lab bills or commissions, so it does not match the P&L.
- Dashboard money tiles need the separate feature `billing.financial_summary` ; an organization with `reports.financial` on but that feature off sees no money on the dashboard.
- The dashboard and the report period presets use the browser's date; the financial summary uses the server's UTC date when no date is given. The reports themselves use the location's timezone.
- A Doctor with the default role sees "Clinic overview" in the sidebar but gets an error on the page (own scope is refused there).
- The daily financial summary endpoint has no screen.

- `features/dashboard/services/dashboard.api.ts` is a mock module, and about ten dashboard components are unused code.
- On-screen reports stop at 5,000 rows, Excel at 50,000, PDF refuses more than 5,000 rows; ranges are limited to 3 years.
- The clinic overview has no download; report filters are kept in memory only and reset on reload.
- "Doctor performance" and "Revenue by service" accept a doctor id in the API but offer no doctor filter on screen.
- Free-text reports (diagnoses, complaints) split spelling variants into separate lines.

36.8 Owner's FAQ

Why does the dashboard show a different profit from the Finance overview? The dashboard subtracts only expenses from collected revenue. The P&L also subtracts lab bills, doctor commissions and payroll salaries.

What is the difference between Collected and Billed? Collected counts money received in the period (net of refunds and without sales tax). Billed counts invoices issued in the period, paid or not. Costs are counted the same way under both.

Are salaries counted twice if I also enter them as expenses? They can be. The finance overview warns when a period has both payroll records and "Salary" expenses.

Can an Accountant see all branches together? Not with the default role: their financial permission has scope `location`. Give the role `org` scope in the role editor.

Can a doctor see other doctors' figures? No. With scope `own` a doctor sees only the reports that can be narrowed to one doctor, and only their own rows.

Who downloaded our data? Every Excel or PDF download writes an audit entry with the user, report, format, period, scope and row count.

Why is a report cut off? More than 5,000 rows were found. Download Excel (up to 50,000 rows) or choose a shorter period.

Where are reports stored? Nowhere. Each one is calculated from live data when opened or downloaded.

Chapter 37 — Employees

This chapter describes the Employees module of the Platform: the staff directory of a clinic, where each person's details, workplace, role, login, salary, weekly hours, day exceptions and leave are kept. It is written for the product owner, who needs to know what the module really does today and where it stops. Other modules lean on it: appointments and the queue need doctors with an assignment and a schedule, access control reads the role on the assignment, and payroll and commissions pay the assignment (Chapters 38 and 35).

37.1 What this module is for

The module separates the **person** from the **job**. An **employee** is the person record, kept once for the whole Platform: name, phone, CNIC, gender, date of birth and photo. Phone and CNIC are unique across the whole Platform, not just within one clinic. An **assignment** links the employee to one organization and one location, with a **role** (employment role), a designation, an employment type, an employee code, a joining date, an active flag and portal access. The **staff login** (a user account) is attached to the assignment.

From the assignment hang the weekly schedule, date-specific availability overrides, leave requests, the compensation plan (Chapter 35) and payroll records (Chapter 38).

Who uses it:

- **Admin** and the **owner**: everything — add, edit, deactivate and delete employees, set salary, schedules and overrides, approve leave, give logins.
- **Doctor**, **Receptionist**, **Assistant**, **Accountant**: no access by default. None of these system roles holds `employees.view`.

Where it appears: the sidebar item **Employees** (key `nav.employees`, route `/employees`, needs `employees.view`) in the **Team** section, next to **Payroll**. Staff logins are also managed from **User Admin** (`nav.users`, route `/users`, needs `admin.users.manage`) in the Admin section.

The core employee permissions (`employees.view`, `employees.manage`) belong to the always-on `core` feature. Leave belongs to the feature `hr.leave` ("Leave Management", off by default), through the permission `leave.approve`. Attendance is mentioned only in that feature's description and is not built.

37.2 Screens

Figure 37.1 — The Employees page (screenshot pending).

The page title is "Employees" with the subtitle "N employees total". Top right are a card/table view toggle and an **Add Employee** button. A filter bar holds a search box (name, phone or employee code), a role filter ("All Roles", filled from the clinic's roles), an employment-type filter ("All Types": full time, part time, visiting, contract), a status filter ("All Status": active or inactive) and a reset action. The table view has the columns Employee (avatar, name, designation), Code, Phone, Type, Salary, Status (Active / Inactive badge) and an actions column with icon buttons with tooltips: "View employee", "Edit employee", an Activate/Deactivate switch and "Delete employee permanently". Rows are paged on the server (page sizes 10, 20, 50, 100). The card view shows the same people as a grid of cards with the same actions. With no rows the page shows "No employees found" and a hint to add the first employee; a failed load shows "Failed to load employees. Please try again." Without an active location the page asks "Select a location first".

Screenshot pending

Add Employee dialog

C:/Program Files/Git/employees

Figure 37.2 — The Add Employee dialog (screenshot pending).

Add Employee opens a wide dialog, "Add Employee — Fill in the details to add a new employee to this location." The form has: Full Name *, Phone *, CNIC, Designation *, Role * (the clinic's roles), Employment Type *, Joining Date * (date picker), Gender, Date of Birth (date picker) and Gross Salary (optional). Users who can manage compensation also see a "Compensation" section ("Salaried by default…") with Salaried / Commission / Both, a commission percentage and "Commission on". A "Portal Access" switch ("Allow this employee to log in to the system") reveals login email and password fields (password at least 6 characters). Errors appear under each field and as a red banner; the buttons are Cancel and a submit button. The same form also exists as a full page at `/employees/create`, which no button links to.

Screenshot pending

Employee detail - Details tab

C:/Program Files/Git/employees/$employeeId

Figure 37.3 — Employee detail page, Details tab (screenshot pending).

The detail page title is the employee's name with the designation as subtitle, a ← back button ("Back to employees"), an Active/Inactive badge and an **Edit** button for users with `employees.manage`. Tabs: **Details**, **Compensation** (with `compensation.manage`, Chapter 35), **Schedule**, **Leave** (with `leave.approve`) and **Payroll** (with `payroll.manage`, Chapter 38). The Details tab has a "Personal Information" card (Phone, CNIC, Date of Birth, Gender, Employment Type, Joining Date, Location, Employee Code, Portal Access as "Has Login Access" or "No Portal Access") and a "Current Salary" card with the monthly amount, "Effective since" and an **Update** (or **Set Salary**) button, or "No salary configured yet." **Edit** opens a slide-over sheet with the same form as Add Employee, without the compensation section.

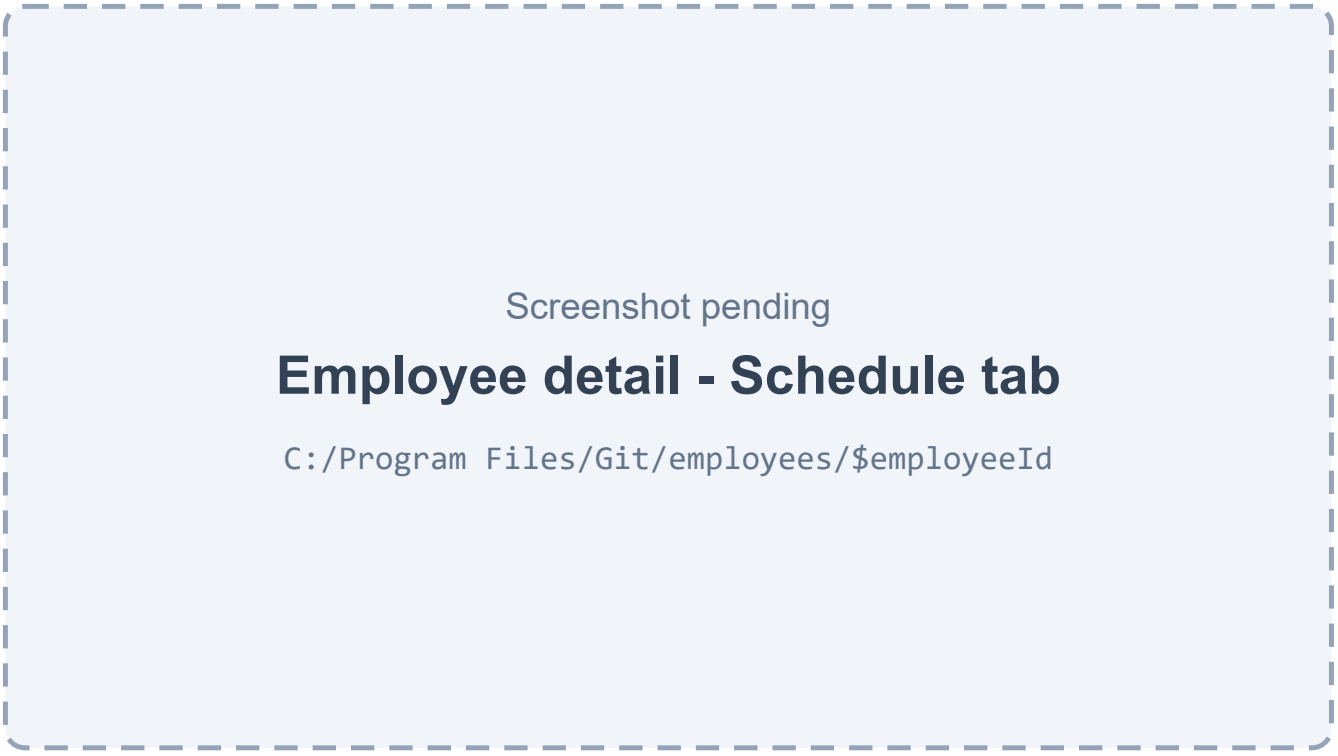

Figure 37.4 — Weekly schedule and availability overrides (screenshot pending).

The Schedule tab has two cards. "Weekly Schedule" lists the seven days, each with a switch (working or off) and, for working days, start and end times; the card header has a "Slot:" selector (15, 30, 45 or 60 minutes) and a **Save** button. Switching a day on fills 09:00–17:00. "Availability Overrides" has an **Add Override** button and a table with Date, Status ("Available" or "Unavailable"), Hours, Reason and edit and delete icon buttons; empty, it says "No overrides set — Add an override for a day off or special working hours". The override dialog has Date, "Available this day", "Working Hours" (one or more time slots) and "Reason (optional)" ("e.g. Conference, public holiday...").

Screenshot pending

Employee detail - Leave tab

`C:/Program Files/Git/employees/$employeeId`

Figure 37.5 — Leave requests and the leave summary (screenshot pending).

Four small cards show "Approved Days (2026)", "Pending Requests", "Annual" and "Sick". Under them a "Leave Requests" card has a **New Request** button and a table with Type, From, To, Days, Status (badge), Reason and a three-dot menu: **Approve**, **Reject** and **Edit** for pending requests and **Cancel Request** for pending or approved ones. The request dialog has Leave Type, Leave dates (a range picker) and Reason (optional).

Screenshot pending

User Admin - staff login dialog

`C:/Program Files/Git/users`

Figure 37.6 — Giving an existing employee a login from User Admin (screenshot pending).

On **User Admin**, the user dialog's role list offers "Staff (existing employee)" with the explanation "Gives an employee a login. What they can do comes from their role in the Employee directory." An "Employee" list shows employees of the organization without a login, as "name · location, location"; if none remain it says "Every

employee already has a login. Add new staff from the Employee directory first." The rest of the dialog asks for name, email, optional phone and password.

37.3 Features

37.3.1 Browse and filter the employee list

Status: [Implemented]

What it does. Lists the employees assigned to the active location, with search, filters, paging and card or table view.

How the user does it.

1. Open **Employees** in the sidebar.
2. Type in the search box or pick a role, type or status filter; switch between card and table view.

Behind the scenes — in simple terms. The list is really a list of assignments at this location. Each row shows the person's details plus the job details at this location and the salary from the current compensation plan.

Behind the scenes — technically. `GET /employees?search&role&employmentType&isActive&page&limit` → `EmployeesService.findAll` : query on `employee_org_location_assignments` for the active organization and location, joined to `employees` , `employment_roles` and `locations` ; search uses `ILIKE` on name, phone and employee code; the role filter compares the role's slug. Ordered by assignment creation, newest first. The salary and `compensationType` come from `CompensationPlanVersions.activeForMany` on the location's clinic date, falling back to the deprecated `gross_salary` column.

Rules and validations. Default page size 20 on the server (the web app starts at 10); `isActive` accepts `true` / `false` .

Who can do it. `employees.view` , feature `core` . Default: Admin.

Data read and written. Reads `employee_org_location_assignments` , `employees` , `employment_roles` , `locations` , `compensation_plans` .

Effects on other modules. None.

Error cases. 403 `NO_PERMISSION` .

Technical references: `backend/src/employees/employees.service.ts` · `EmployeesService.findAll` · `web-app/src/features/employees/pages/EmployeesPage.tsx`

37.3.2 Add an employee

Status: [Implemented]

What it does. Creates the person record, an assignment at the active location with a role and an employee code, a first compensation plan version and, optionally, a staff login.

How the user does it.

1. On **Employees**, click **Add Employee**.
2. Fill in the required fields (Full Name, Phone, Designation, Role, Employment Type, Joining Date) and any optional ones.
3. Optionally turn on **Portal Access** and enter the login email and password.
4. Submit. A toast "Employee created successfully" appears.

Behind the scenes — in simple terms. The server first checks that no employee anywhere on the Platform already has this phone number. It validates the pay terms, saves the person, generates the next employee code for the clinic, creates the login if asked, and saves the assignment together with the first pay version.

Behind the scenes — technically. `POST /employees` → `EmployeesService.create`:

1. Refuse when an `employees` row already has the phone (409).
2. Load the role (organization-scoped) and build the first plan with `initialPlan`: the optional `compensation` object, else a `SALARY` plan with the entered salary in paisa (0 when empty), effective from the joining date (or the location's clinic date).
3. Insert `employees`.
4. Generate `employee_code` as `EMP-` plus (number of assignments in the organization + 1), zero-padded to three digits.
5. If portal access is on and no `userId` was given, create a `users` row (email unique, password hashed with bcrypt, local provider).
6. In one transaction, insert the assignment (`is_primary` true by default, `gross_salary` left empty) and add the compensation version.

The access-version subscriber bumps the organization's access version when an assignment is saved, so sessions refresh their permissions.

Rules and validations.

- Phone is unique across the Platform (checked first, and a unique database index). CNIC is optional and also unique across the Platform (database index only).
- Full name 2–255 characters; designation required; role must be a UUID (the form requires it; the API allows it to be empty).
- Portal access needs an email and a password of at least 6 characters; the email must not belong to an existing user.
- Commission plans need `billing.commissions` and a role that can consult (Chapter 35).
- The person is saved before the login and the assignment, outside the final transaction.

Who can do it. `employees.manage`, feature `core`. Default: Admin. The compensation section is shown only with `compensation.manage`.

Data read and written. Reads `employees`, `employment_roles`, `users`. Writes `employees`, `users` (optional), `employee_org_location_assignments`, `compensation_plans`, `commission_rules`, `organizations.access_version`.

Effects on other modules. An active assignment makes the person appear in the appointments doctor list and queue lanes (Chapters 29 and 30); a login with portal access lets them sign in with the role's permissions.

Error cases.

- 409 "An employee with this phone number already exists".
- 409 "A user with email … already exists".
- 400 "portalEmail and portalPassword are required when hasPortalAccess is true".
- 400 "Salary must be a positive amount with at most 2 decimals".
- 403 `COMMISSIONS_DISABLED`; 422 `COMPENSATION_PLAN_INVALID`, `COMMISSION_ROLE_CANNOT_CONSULT`.
- A duplicate CNIC reaches the database unique index and is returned as a generic 500 "Something went wrong".
- 402 `SUBSCRIPTION_EXPIRED` in read-only mode.

Technical references: `backend/src/employees/employees.service.ts` · `EmployeesService.create`, `initialPlan` · `web-app/src/features/employees/components/CreateEmployeeModal.tsx`

37.3.3 View an employee

Status: [Implemented]

What it does. Shows one employee's details at the active location, with the tabs the viewer's permissions allow.

How the user does it.

1. Click "View employee" (eye icon) on a row or card.

Behind the scenes — in simple terms. The server finds the person's assignment at this location; a person who works only at another location is "not found" here.

Behind the scenes — technically. `GET /employees/:id` → `EmployeesService.findOne`: the assignment for (employee, active organization, active location) with employee, role, location and user; the current plan gives `grossSalary`, `compensationType` and `currentSalary.effectiveFrom`.

Rules and validations. Scoped to the active location.

Who can do it. `employees.view`, feature `core`. Default: Admin.

Data read and written. Reads `employee_org_location_assignments`, `employees`, `employment_roles`, `locations`, `users`, `compensation_plans`.

Effects on other modules. None.

Error cases. 404 "Employee not found at this location" (the page shows "Employee Not Found" with **Back to List**).

Technical references: `backend/src/employees/employees.controller.ts` · `EmployeesService.findOne` · `web-app/src/routes/employees/$employeeId.index.tsx`

37.3.4 Edit an employee

Status: [Implemented]

What it does. Changes the person's details and the assignment's details at this location, and can turn portal access on (creating a login) or off.

How the user does it.

1. Click **Edit** on the detail page, or "Edit employee" on the list.
2. Change fields in the "Edit Employee" sheet and save. A toast "Employee updated successfully" appears.

Behind the scenes — in simple terms. Personal fields go to the person record, which every clinic the person works for sees. Job fields go to this location's assignment only.

Behind the scenes — technically. `PUT /employees/:id` → `EmployeesService.update`: sets name, phone, CNIC, gender, date of birth and photo on `employees`; designation, role, employment type, joining date and primary flag on the assignment. Turning portal access on without a linked user creates a login (same rules as create); turning it off only clears the flag. A `grossSalary` value adds a new salary version from today (37.3.8).

Rules and validations.

- Same field rules as create; all fields optional.
- The phone uniqueness check of create is not repeated; a duplicate phone or CNIC is caught only by the database index (generic 500).

- Changing the role changes the person's permissions at this location; the access version is bumped.

Who can do it. `employees.manage`, feature `core`. Default: Admin.

Data read and written. Writes `employees`, `employee_org_location_assignments`, `users` (optional), `compensation_plans` (optional).

Effects on other modules. Role and portal changes take effect on the person's next access refresh (Chapter 57).

Error cases. 404 "Employee not found at this location"; 400 "portalEmail and portalPassword are required when enabling portal access"; 409 duplicate login email.

Technical references: `backend/src/employees/employees.service.ts` · `EmployeesService.update` · `web-app/src/features/employees/components/EditEmployeeSheet.tsx`

37.3.5 Activate or deactivate an employee

Status: [Implemented]

What it does. Switches the employee's assignment at this location on or off without deleting anything.

How the user does it.

1. On the list, use the switch with the tooltip "Deactivate employee" / "Activate employee".

Behind the scenes — in simple terms. An inactive assignment keeps all history. The person cannot sign in to this location, does not appear as a doctor for booking, gets no payroll, and their schedule and leave cannot be edited until reactivated.

Behind the scenes — technically. `PATCH /employees/:id/status` with `{ isActive }` → `EmployeesService.setActive` sets `employee_org_location_assignments.is_active`. The `employees.is_active` column is not changed by any endpoint. Login (`AuthorizationService`) and access resolution require an active assignment with portal access.

Rules and validations. Affects only the active location's assignment.

Who can do it. `employees.manage`, feature `core`. Default: Admin.

Data read and written. Writes `employee_org_location_assignments`, `organizations.access_version`.

Effects on other modules. Appointments doctor list, queue lanes, payroll generation, staff-login candidates and sign-in all ignore inactive assignments.

Error cases. 404 "Employee not found at this location".

Technical references: `backend/src/employees/employees.service.ts` · `EmployeesService.setActive` · `web-app/src/features/employees/components/EmployeeTable.tsx`

37.3.6 Delete an employee permanently

Status: [Implemented]

What it does. Deletes the person record and, by database cascade, their assignments, schedules, leave, overrides, payroll records and compensation plans.

How the user does it.

1. Click "Delete employee permanently" (bin icon).
2. Confirm in the dialog "Delete employee permanently?" with **Delete permanently**.

Behind the scenes — in simple terms. Deletion is refused when the person also works at another location, or when other records still need them. The dialog advises deactivating instead.

Behind the scenes — technically. `DELETE /employees/:id` → `EmployeesService.remove`: confirms the assignment at this location, refuses if the employee has an assignment at any other location, then deletes the `employees` row and bumps the access version. A PostgreSQL foreign-key error (23503) is turned into 409 with an inventory message. Appointments refuse deletion of a doctor who still has appointments (ON DELETE RESTRICT, Chapter 29), and commission entries and settlements refuse deletion of their assignment (RESTRICT, Chapter 35); both surface as the same 409.

Rules and validations. Only when this is the person's only assignment.

Who can do it. `employees.manage`, feature `core`. Default: Admin.

Data read and written. Deletes `employees` and cascaded rows.

Effects on other modules. Payroll history and plans of the person are gone.

Error cases. 409 "This employee is also assigned to other locations and cannot be deleted. Deactivate them here instead."; 409 "This employee has linked inventory records and cannot be deleted. Deactivate them instead." (shown for any blocking reference).

Technical references: `backend/src/employees/employees.service.ts` · `EmployeesService.remove` · `web-app/src/features/employees/pages/EmployeesPage.tsx`

37.3.7 Give an employee a staff login (portal access)

Status: [Implemented]

What it does. Lets an employee sign in. There are two ways: the Portal Access switch on the employee form, and the "Staff (existing employee)" option on **User Admin**.

How the user does it.

1. Either turn on **Portal Access** on Add or Edit Employee and enter email and password,
2. or open **User Admin**, add a user, choose "Staff (existing employee)", pick the employee and enter name, email and password.

Behind the scenes — in simple terms. A login alone grants nothing. What the person may do comes from the role on each assignment. The User Admin path links one login to all the person's active assignments in the organization at once.

Behind the scenes — technically. Employee form: `createPortalUser` inside `EmployeesService.create` / `update`, linking only the active location's assignment. User Admin: `GET /user/staff-candidates` lists employees with an active assignment and no linked login in the organization; `POST /user` with `type: 'employee'` and `employeeId` → `UserService.createStaffLogin`: after the subscription user-limit check (`assertCanAddUser`), in one transaction it locks the active assignments, refuses if any is already linked, creates the user and sets `user_id` and `has_portal_access = true` on every active assignment. The endpoint `PATCH /employees/:id/portal-access` (`togglePortalAccess`) flips the flag only; no screen calls it. Sign-in and location access require `has_portal_access = true` and an active assignment.

Rules and validations.

- User Admin: password at least 8 characters, valid email, email unique. Employee form: password at least 6 characters.
- The employee-form path does not check the subscription's user limit; the User Admin path does.

Who can do it. Employee form: `employees.manage`. User Admin: `admin.users.manage` (or super admin). Default: Admin.

Data read and written. Writes `users`, `employee_org_location_assignments`.

Effects on other modules. Authentication and access control (Chapters 56 and 57).

Error cases. 409 "This employee already has a login"; 404 "Employee not found in this organization"; 400 "User with email already exists"; 400 "employeeId is required for staff users"; the user-limit error from Subscriptions (Chapter 45).

Technical references: `backend/src/user/user.service.ts` · `UserService.createStaffLogin`, `staffCandidates`, `EmployeesService.togglePortalAccess` · `web-app/src/features/users/components/UserFormDialog.tsx`

37.3.8 Set or update the salary

Status: [Implemented]

What it does. Changes the monthly salary of the assignment from today.

How the user does it.

1. On the Details tab, click **Update** (or **Set Salary**) on the "Current Salary" card.
2. Enter the amount and save. A toast "Salary updated" appears.

Behind the scenes — in simple terms. Salary lives in the compensation plan. A change adds a new plan version from today; a commission rate and overrides in force are carried over, and a commission-only plan becomes "Both".

Behind the scenes — technically. `PATCH /employees/:id/salary` with `{ grossSalary }` → `EmployeesService.updateSalary` → `addSalaryVersion`: converts rupees to paisa, reads the version in force on the location's clinic date, does nothing if the salary is unchanged, otherwise calls `CompensationPlanVersions.add` with `effectiveFrom` = today. The deprecated `gross_salary` column is no longer written.

Rules and validations. Salary at least 1, at most 2 decimals. A second change on the same day replaces the first only if no charge used it (else 409 `COMPENSATION_PLAN_IN_USE`, Chapter 35).

Who can do it. `employees.manage`, feature `core`. Default: Admin. Works with `billing.commissions` off.

Data read and written. Reads and writes `compensation_plans`, `commission_rules`.

Effects on other modules. Payroll uses the version in force on the month's last day (Chapter 38).

Error cases. 400 "Salary must be a positive amount with at most 2 decimals"; 409 `COMPENSATION_PLAN_IN_USE`; toast "Failed to update salary".

Technical references: `backend/src/employees/employees.service.ts` · `EmployeesService.addSalaryVersion` · `web-app/src/features/employees/components/UpdateSalaryModal.tsx`

37.3.9 Set the weekly schedule

Status: [Implemented]

What it does. Stores, per weekday, whether the person works at this location and from when to when, plus a slot length.

How the user does it.

1. Open the **Schedule** tab.
2. Switch days on or off, set times, pick "Slot:", and click **Save**.

Behind the scenes — in simple terms. The week is saved as seven rows for this location. Booking reads these hours to decide whether a doctor works at a given time (Chapter 29).

Behind the scenes — technically. `GET /employees/:employeeId/schedule` returns seven days (missing days filled as not working, slot 30). `POST /employees/:employeeId/schedule` → `ScheduleService.setSchedule` validates and upserts each day; `PATCH /employees/:employeeId/schedule/:dayOfWeek` updates one day (no screen calls it). Days are 0–6; times are "HH:MM" strings with no timezone (clinic local time).

Rules and validations.

- Exactly seven unique days 0–6; slot 15, 30, 45 or 60 minutes.
- A working day needs start and end; end after start.
- The person must have an active assignment at the location.
- One row per employee, location and weekday (unique index).
- A helper that checks overlap with the same person's hours at other locations exists but is never called; overlapping weekly hours at two locations are accepted.

Who can do it. Read: `employees.view`; save: `employees.manage`; feature `core`. Default: Admin. The Save button is visible without `employees.manage`, but the server refuses.

Data read and written. Reads `employee_org_location_assignments`. Writes `employee_schedules`.

Effects on other modules. Appointment validation, slot suggestions and the calendar's working-hours shading (Chapter 29).

Error cases. 400 "Employee is not assigned to this location"; 400 "Schedule must contain exactly 7 unique days (0-6)"; 400 "Day 1: End time must be after start time".

Technical references: `backend/src/employees/schedule.service.ts` · `ScheduleService.setSchedule` · `web-app/src/features/employees/components/ScheduleTab.tsx`

37.3.10 Record availability overrides

Status: [Partial] — recorded and edited, but booking, slot suggestions and the calendar ignore them.

What it does. Records a one-day exception to the weekly schedule: unavailable that day, or available with special hours.

How the user does it.

1. On the **Schedule** tab, click **Add Override**.
2. Pick the date, set "Available this day" and, if available, the working hours; add a reason.
3. Save. Edit or delete with the icons in the table.

Behind the scenes — in simple terms. The exception is stored for this person, location and date. Nothing else in the Platform reads it today.

Behind the scenes — technically. `POST`, `GET ?from&to`, `PATCH /:id`, `DELETE /:id` on `/employees/:employeeId/availability-overrides` → `AvailabilityOverrideService`. Hours are stored as JSON slots (`override_hours`); when unavailable they are cleared.

Rules and validations.

- One override per employee, location and date (checked, and a unique index).
- Each slot "HH:MM", end after start, slots must not overlap.
- Active assignment at the location required for create and list.

- Delete has no confirmation dialog and the icon buttons have no tooltips.

Who can do it. Read: `employees.view`; write: `employees.manage`; feature `core`. Default: Admin.

Data read and written. Writes `doctor_availability_overrides`.

Effects on other modules. None today. Booking a doctor on an "Unavailable" date is still possible (Chapter 29, 29.3.13).

Error cases. 409 "An override for this employee on this date already exists"; 400 "Override hour slots must not overlap"; 404 "Availability override not found".

Technical references: `backend/src/employees/availability-override.service.ts` · `AvailabilityOverrideService` · `web-app/src/features/employees/components/AvailabilityOverrideTab.tsx`

37.3.11 Manage leave requests

Status: [Partial] — the full request and approval cycle works for managers, but employees cannot request their own leave, and approved leave does not block bookings.

What it does. Records leave for an employee, then approves, rejects, edits or cancels it.

How the user does it.

1. Open the **Leave** tab (visible with `leave.approve`) and click **New Request**.
2. Pick Leave Type, the date range and an optional reason; submit.
3. Use the three-dot menu: **Approve**, **Reject**, **Edit** (pending only) or **Cancel Request** (pending or approved).

Behind the scenes — in simple terms. A request starts pending. It counts the calendar days from start to end. Two pending or approved requests of the same person may not overlap.

Behind the scenes — technically. All routes are on `LeaveController` (`/leaves`), which requires `leave.approve` for every endpoint: `POST /leaves`, `GET /leaves?employeeId&status&leaveType`, `GET /leaves/:id`, `PATCH /leaves/:id`, `PATCH /leaves/:id/approve`, `PATCH /leaves/:id/reject` (both take an optional `reviewNote`), `PATCH /leaves/:id/cancel`. `LeaveService.review` sets `status`, `review_note`, `approved_by` (also for rejections) and `reviewed_at`. Types: annual, sick, unpaid, maternity, paternity, other.

Rules and validations.

- End date on or after start date; `total_days` = calendar days inclusive (weekends and holidays count).
- No overlap with the person's other pending or approved leave (any location).
- Only pending requests can be edited, approved or rejected; pending or approved can be cancelled.
- The person must have an active assignment at the location to file.
- The web app sends no review note.

Who can do it. `leave.approve`, feature `hr.leave` (off by default). Default: Admin (when the feature is on).

Data read and written. Writes `employee_leaves`.

Effects on other modules. None. Appointments and the queue do not read leave, and payroll does not deduct unpaid leave.

Error cases. 400 "Employee already has a pending or approved leave overlapping these dates"; 400 "Only pending leave requests can be reviewed"; 400 "Only pending or approved leave requests can be cancelled"; 400 "End date must be on or after start date"; 404 "Leave request not found".

Technical references: `backend/src/employees/leave.service.ts` · `LeaveService.create`, `review`, `cancel` · `web-app/src/features/employees/components/LeaveTable.tsx`

37.3.12 See the leave summary ("balance")

Status: [Partial] — a usage tally, not a balance: there are no leave entitlements to subtract from.

What it does. Shows the approved leave days this calendar year, the number of pending requests and approved days per type.

How the user does it.

1. Open the **Leave** tab; the four cards appear at the top.

Behind the scenes — in simple terms. The server adds up the person's approved leave that touches the current year and counts pending requests.

Behind the scenes — technically. `GET /leaves/balance/:employeeId` → `LeaveService.getBalance`. The year is the server's calendar year; a leave that crosses New Year counts all its days in both years.

Rules and validations. Counts leave in the whole organization, not only this location.

Who can do it. `leave.approve`, feature `hr.leave`. Default: Admin.

Data read and written. Reads `employee_leaves`.

Effects on other modules. None.

Error cases. 400 "Employee is not assigned to this location".

Technical references: `backend/src/employees/leave.service.ts` · `LeaveService.getBalance` · `web-app/src/features/employees/components/LeaveTable.tsx`

37.3.13 List employment roles

Status: [Implemented]

What it does. Supplies the role list for the employee form and the role filter.

How the user does it. Open the Role field or filter; the clinic's roles appear.

Behind the scenes — in simple terms. The roles are the clinic's system and custom roles. Creating and editing roles happens in the role editor (Chapter 11).

Behind the scenes — technically. `GET /employment-roles` and `GET /employment-roles/:id` need `employees.view`. The same controller also has `POST`, `PUT` and `DELETE` guarded by `admin.roles.manage`; the role editor uses `/admin/roles`.

Rules and validations. Organization-scoped.

Who can do it. `employees.view`. Default: Admin.

Data read and written. Reads `employment_roles`.

Effects on other modules. None.

Error cases. 403 `NO_PERMISSION`.

Technical references: `backend/src/employees/employment-roles.controller.ts` · `EmploymentRolesService` · `web-app/src/features/employees/components/EmployeeForm.tsx`

37.3.14 Assign an existing employee to another location

Status: [Not implemented]

What it does. Nothing today. The data model allows one person to have assignments at several locations and organizations, and a `CreateAssignmentDto` exists, but no endpoint uses it. Adding the same person at a second location through **Add Employee** fails, because the phone number is already taken.

How the user does it. Not available.

Behind the scenes — in simple terms. Every assignment is created together with a new person record.

Behind the scenes — technically. `EmployeesService.create` is the only code that inserts assignments; `create-assignment.dto.ts` and `update-assignment.dto.ts` are unused; `getAssignmentsByEmployee` is not exposed.

Rules and validations. None.

Who can do it. Not applicable.

Data read and written. None.

Effects on other modules. Multi-location doctors (Chapters 29, 35, 38) can exist only in data created another way.

Error cases. 409 "An employee with this phone number already exists" when trying via Add Employee.

Technical references: `backend/src/employees/dto/create-assignment.dto.ts` · `CreateAssignmentDto` · none

37.3.15 Attendance

Status: [Not implemented]

What it does. Nothing. Attendance appears only in the description of the `hr.leave` feature ("Track employee leave and attendance."). There is no table, endpoint or screen.

How the user does it. Not available.

Behind the scenes — in simple terms. Not built.

Behind the scenes — technically. No attendance entity or controller exists.

Rules and validations. None.

Who can do it. Not applicable.

Data read and written. None.

Effects on other modules. None.

Error cases. None.

Technical references: `backend/src/feature-flags/feature-catalog.seed.ts` · `hr.leave` · none

37.4 Entities and data model

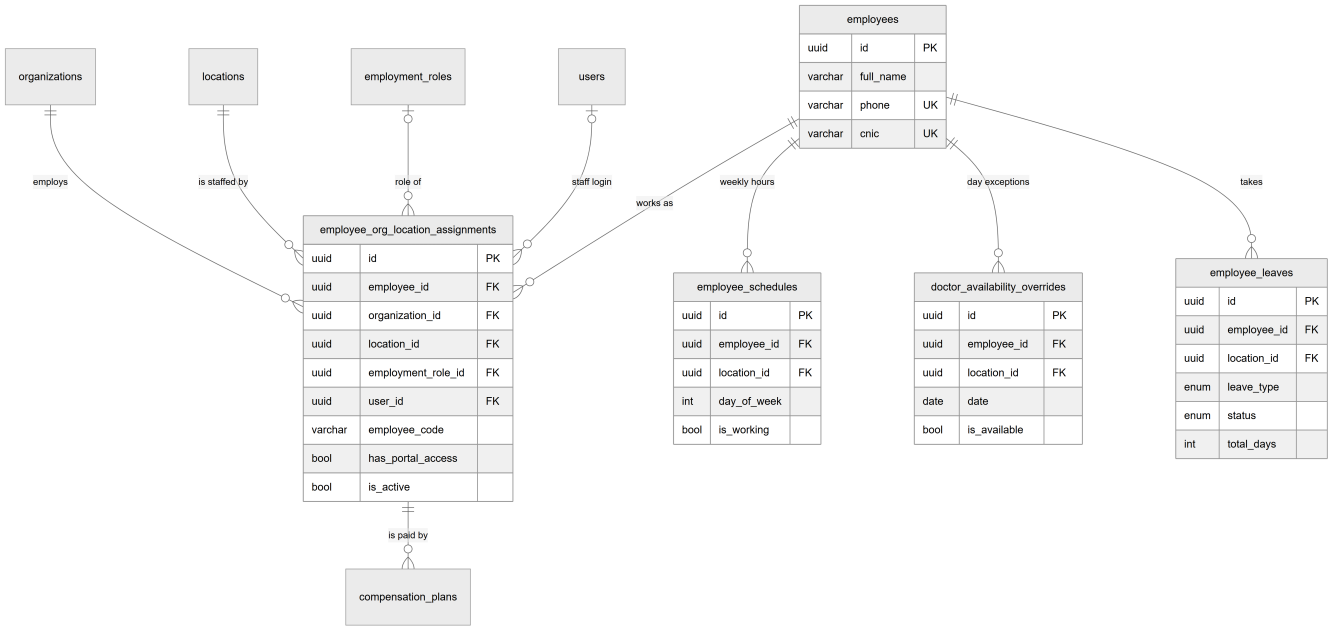

Figure 37.7 — The person, the assignment and the tables that hang from them.

The diagram has `employees` at the top right with `id`, `full_name`, and `phone` and `cnic` marked UK (unique). A "works as" line goes from `employees` to `employee_org_location_assignments`, which shows `id`, `employee_id`, `organization_id`, `location_id`, `employment_role_id`, `user_id`, `employee_code`, `has_portal_access` and `is_active`. Over the assignment, four boxes link to it: `organizations` ("employs"), `locations` ("is staffed by"), and, as optional links, `employment_roles` ("role of") and `users` ("staff login"). Under the assignment, `compensation_plans` hangs off it ("is paid by"). From `employees` three more lines go to `employee_schedules` ("weekly hours": `employee_id`, `location_id`, `day_of_week`, `is_working`), `doctor_availability_overrides` ("day exceptions": `date`, `is_available`) and `employee_leaves` ("takes": `leave_type`, `status`, `total_days`).

The relationships in words:

- Each employee can have several assignments; each assignment belongs to exactly one employee, organization and location. Deleting any of these deletes the assignment (CASCADE). One person has at most one assignment per location (unique `employee_id`, `location_id`); the employee code is unique within an organization.
- An assignment may have one role; deleting the role leaves the assignment without one (SET NULL), which means no permissions. An assignment may have one login; deleting the user clears the link (SET NULL).
- Schedules, overrides and leave belong to one employee and name one organization and location; deleting the employee or the location deletes them (CASCADE). Schedules are unique per employee, location and weekday; overrides per employee, location and date.
- Compensation plans belong to the assignment (CASCADE); payroll records belong to the employee and location (Chapter 38).
- `employees.is_active` exists but nothing writes it; the active flag that matters is on the assignment. `gross_salary` on the assignment is deprecated; salary lives in `compensation_plans`.

The Data model chapter (Chapter 16) lists every column.

37.5 Status lifecycles

37.5.1 Leave request

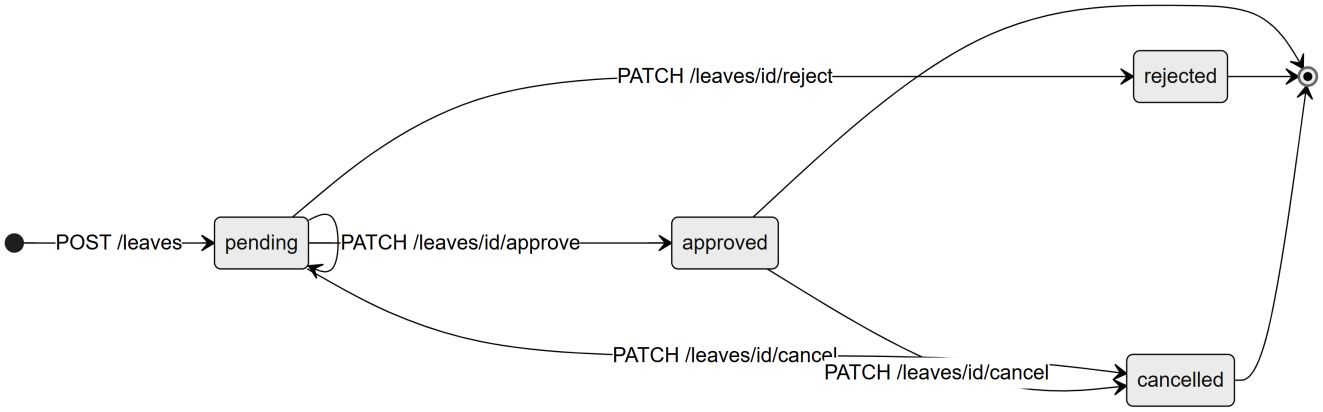

Figure 37.8 — The states of a leave request.

The diagram has four states. A request enters at **pending** through `POST /leaves`. A loop on pending shows `PATCH /leaves/:id` (edit). From pending, `PATCH …/approve` leads to **approved**, `PATCH …/reject` to **rejected** and `PATCH …/cancel` to **cancelled**. From approved, `PATCH …/cancel` leads to cancelled. Approved, rejected and cancelled all lead to the end marker.

- **pending** — filed, can be edited, approved, rejected or cancelled; blocks overlapping requests.
- **approved** — counts in the summary and blocks overlapping requests; can still be cancelled.
- **rejected** — final; does not block.
- **cancelled** — final; does not block.

Any other change is refused with 400.

37.5.2 Employee (assignment) status

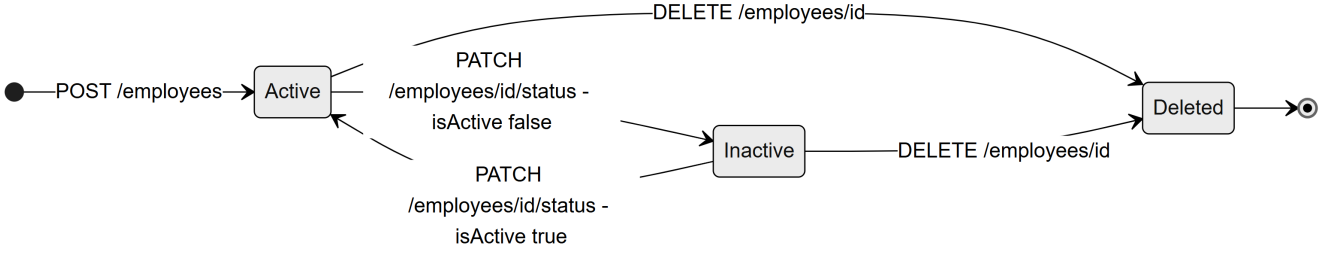

Figure 37.9 — Active, inactive and deleted.

The diagram shows an employee entering **Active** through `POST /employees`. `PATCH /employees/:id/status` with `isActive false` moves Active to **Inactive**; with `isActive true` it moves back. From either state, `DELETE /employees/:id` leads to **Deleted** and the end marker.

The status is the assignment's `is_active` flag at one location; the same person can be active at one location and inactive at another. Deleted is not a stored state: the rows are gone.

37.6 Connections to other modules

- **Appointments and queue (Chapters 29 and 30).** Doctors are active assignments at the location; weekly schedules drive booking. Overrides and leave are not read.
- **Access control (Chapters 11 and 57).** The role and portal flag on the assignment decide what a staff login may do; changes bump the access version.
- **Authentication (Chapter 56).** Sign-in lists organizations and locations from active assignments with portal access.
- **Commissions (Chapter 35).** Compensation plans per assignment; salary updates create plan versions.
- **Payroll (Chapter 38).** Generated for active assignments; reads the plan.
- **Subscriptions (Chapter 45).** The User Admin path counts logins against the plan's limit; writes are refused in read-only mode.

37.7 Limitations and known gaps

- No way to add an existing person at a second location or organization; the phone check blocks it.
- Employee creation is not one transaction: if the login email is already taken, the person record is already saved, and a retry fails on the phone number.
- A duplicate CNIC (and a duplicate phone on edit) returns a generic 500 instead of a clear message.
- The employee code is "number of assignments + 1"; after deletions it can repeat an existing code and the save fails.
- Availability overrides and approved leave do not affect booking.
- Leave can only be filed by managers (`leave.approve`); there is no self-service, no entitlement and no attendance.
- Payslips and CSV show the designation as empty, because it is read from the person record instead of the assignment (Chapter 38).
- Overlapping weekly hours of one person at two locations are not checked.
- Employee changes are not written to the general audit log.

37.8 Owner's FAQ

Can the same doctor work at two of our branches? The data model supports it (one assignment per location), but there is no screen or endpoint to add the second assignment today.

Why does "Add Employee" say the phone number already exists when the person is new to my clinic? Phone numbers are unique across the whole Platform. The person may already be registered by another clinic.

What is the difference between deactivating and deleting? Deactivating keeps everything and blocks login, booking and payroll at this location. Deleting removes the person and their schedules, leave, overrides, plans and payroll records, and is refused if they work elsewhere or are still referenced.

Can a receptionist see staff salaries? Not with the default role; only the Admin holds `employees.view` .

If I mark a doctor on leave, will reception be stopped from booking them? No. Leave and availability overrides are recorded but not checked by booking.

How does a staff member get a login? Turn on Portal Access when adding or editing them, or use User Admin → "Staff (existing employee)". Their permissions come from their role.

Where is a salary stored? In `compensation_plans` , as a dated version per assignment, in paisa.

Chapter 38 — Payroll

This chapter describes the Payroll module of the Platform: how a clinic turns each staff member's pay terms into one monthly payroll record, tracks it until it is paid, and exports it as a payslip or a spreadsheet. It is written for the product owner, who needs to know exactly how base salary and doctor commission are combined and which gaps remain. Payroll reads pay terms from the Employees module (Chapter 37) and commission from the Commissions module (Chapter 35).

38.1 What this module is for

A **payroll record** is one month's pay for one employee at one location. It stores a base salary, a commission amount and their total, all in integer paisa, plus a status (pending, paid, on hold), the payment method and reference once paid, and notes. There is at most one record per employee, location and month.

Records come from three sources: the automatic month-end job, the **Generate Payroll** action (all employees of the active location for a chosen month), and **Create Payroll** (one record typed in by hand).

Who uses it: the **Admin** and the **owner**. No other system role (Doctor, Receptionist, Assistant, Accountant) holds the permission by default. The permission `payroll.manage` is marked high-risk in the role editor.

Where it appears: the sidebar item **Payroll** (key `nav.payroll`, route `/payroll`, needs `payroll.manage`) in the **Team** section. The employee detail page also has a **Payroll** tab for the same permission. The whole module sits behind the feature `hr.payroll` ("Payroll", enabled by default).

38.2 Screens

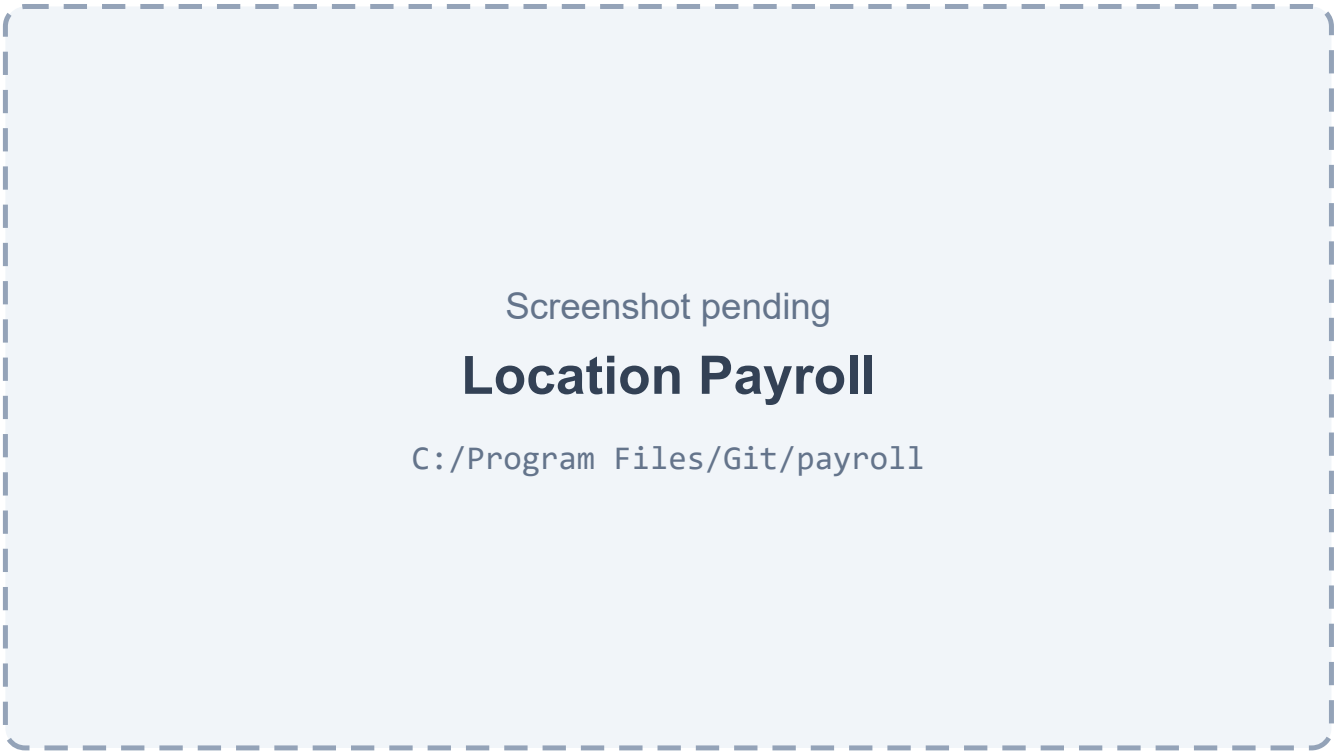

Figure 38.1 — The Location Payroll page (screenshot pending).

The page title is "Location Payroll" with the subtitle "Manage monthly payroll for this location". In the header are a month selector, a year selector (current year minus two to plus two), a **Create Payroll** button and a **Generate Payroll** button. Under the header, four cards summarise the chosen month: Total Payroll (amount and "N employees"), Paid Amount ("N paid records"), Pending Amount ("N pending records") and On Hold ("Records on hold"). A heading "Records for October 2026" introduces a paged table with the columns Employee, Period, Amount (with a small "commission …" line when the record includes commission), Status (Pending, Paid or On Hold badge), Paid On, Method and an actions column. Actions are **Mark as Paid** (hidden once paid), an edit icon button ("Edit payroll") and a delete icon button ("Delete payroll"). An empty month shows "No payroll records found."

Figure 38.2 — The Generate Payroll dialog (screenshot pending).

The dialog says "Generate payroll records for all active employees at this location" and offers a Month and a Year selector and a note that existing records for the same month are skipped. After it runs, a toast lists how many records were created, how many already existed and the names of employees with no salary set.

Screenshot pending

Create or edit payroll record

C:/Program Files/Git/payroll

Figure 38.3 — The dialog for one payroll record (screenshot pending).

The dialog has Employee (a searchable list when creating, read-only when editing), Month, Year, Amount, Status (Pending, Paid, On Hold) and Notes ("Reason for this payroll (e.g. mid-month joiner, bonus adjustment)..."). Choosing an employee pre-fills the amount with their current salary.

Screenshot pending

Mark Salary as Paid dialog

C:/Program Files/Git/payroll

Figure 38.4 — The Mark Salary as Paid dialog (screenshot pending).

The dialog "Mark Salary as Paid" asks for Payment Method (cash, bank transfer or cheque), Reference Number ("Transaction ID or reference") and Notes (optional). On success a toast confirms the payment.

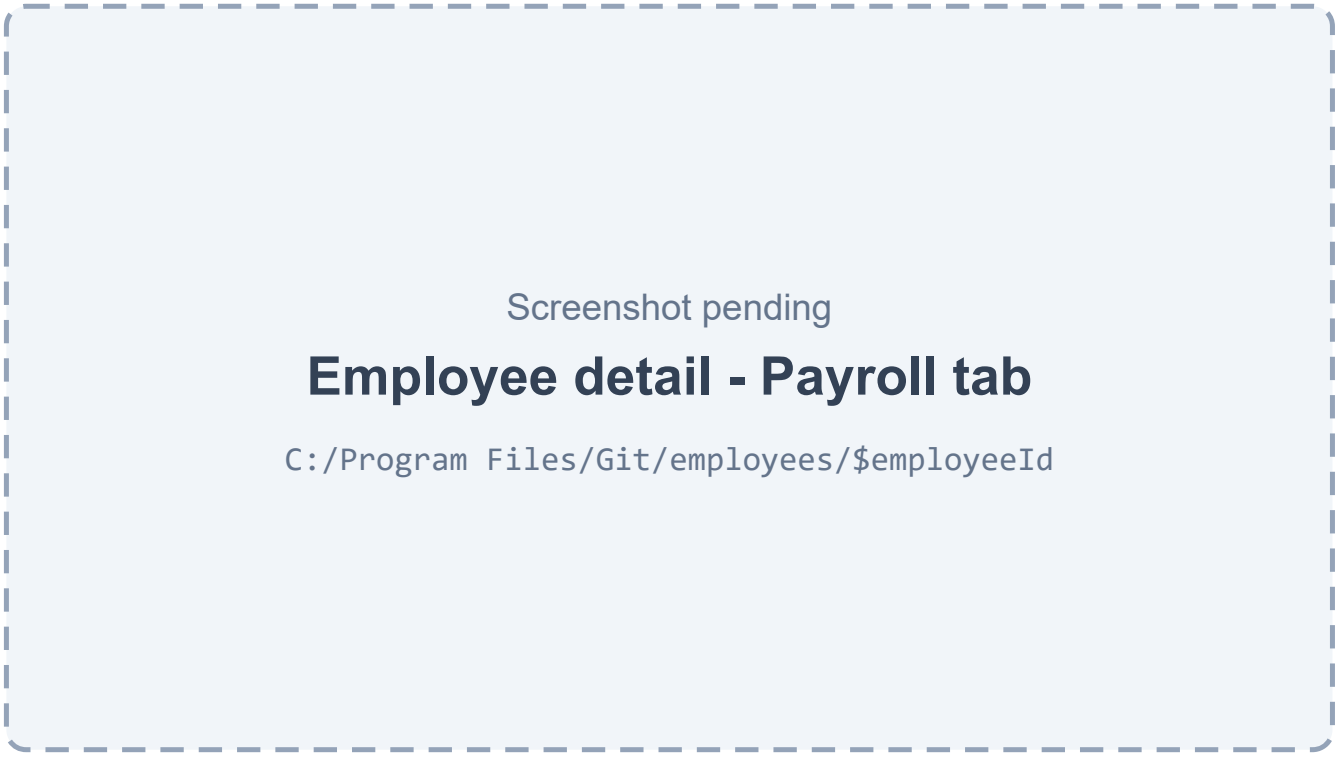

Figure 38.5 — The Payroll tab of an employee (screenshot pending).

The tab shows "Payroll History" with a **Generate This Month** button (when this month has no record and the person has a salary or a commission plan), the same table without the Employee column, or an empty state "No payroll records yet — Generate payroll to track monthly salaries" with a **Generate Payroll** button. Its Generate button opens the same dialog as the main page, which generates for the whole location, not only this person.

38.3 Features

38.3.1 Generate payroll for a month

Status: [Implemented]

What it does. Creates one pending payroll record for every active employee of the active location who has something to be paid for the chosen month, and settles that employee's unsettled commission into it.

How the user does it.

1. Open **Payroll** and choose the month and year.
2. Click **Generate Payroll**, check the month and year, and confirm.
3. Read the toast: records created, records skipped, employees with no salary.

Behind the scenes — in simple terms. For each active employee the Platform skips anyone who already has a record that month. Otherwise it takes the base salary from the pay plan in force at the end of the month, adds all the doctor's commission earned but not yet paid out, and saves one record. An employee with neither salary nor commission is listed as "no salary".

Behind the scenes — technically. `POST /payroll/generate` with `{ month, year }` → `PayrollService.generate` → `generateForLocation`. The month may not be in the future in the location's clinic calendar (400 "Cannot generate payroll for future months"). All work for the location runs in one transaction on a query runner; any error rolls back the whole location. Per active assignment (`EmployeesService.getActiveAssignmentsByLocation`) the steps are the same as in the month-end job drawn in Figure 38.6 and described in 38.3.2.

Rules and validations.

- Month 1–12, year at least 2020 (body); not in the future.
- Unique per employee, location and month (database constraint and a prior read).
- Base salary = `base_salary_paisa` of the compensation plan in force on the last day of the month, else on the first day; if the assignment has no plan at all, the deprecated `gross_salary` is used.
- Commission = sum of the assignment's unsettled `commission_entries` created before the end of the month's last clinic day (location timezone), locked `FOR UPDATE`. A net-negative sum is not settled and is carried to a later month; the record then has commission 0.
- Total = base + commission; `salary_amount` (rupees) mirrors the total.
- Nothing to pay (both zero) → no record; the name goes into `noSalary`.
- Records are created `pending`, with `created_by` = the user.

Who can do it. `payroll.manage`, feature `hr.payroll`. Default: Admin.

Data read and written. Reads `employee_org_location_assignments`, `employees`, `compensation_plans`, `commission_entries`. Writes `payroll_records`, `commission_settlements`, `commission_entries.settlement_id`.

Effects on other modules. Commission becomes "settled" on the earnings pages (Chapter 35).

Error cases. 400 "Cannot generate payroll for future months"; 400 validation messages for month or year; 403 `NO_PERMISSION`; 402 `SUBSCRIPTION_EXPIRED` in read-only mode.

Technical references: `backend/src/payroll/payroll.service.ts` · `PayrollService.generate`, `generateForLocation` · `web-app/src/features/employees/components/GeneratePayrollModal.tsx`

38.3.2 Month-end automatic generation

Status: [Implemented]

What it does. On the first day of every month the Platform generates payroll for the month that just ended, for every active location of every organization.

How the user does it. Nothing to do. Open Payroll afterwards and select the previous month. Records made this way have no `created_by`.

Figure 38.6 shows the job, including the commission settlement.

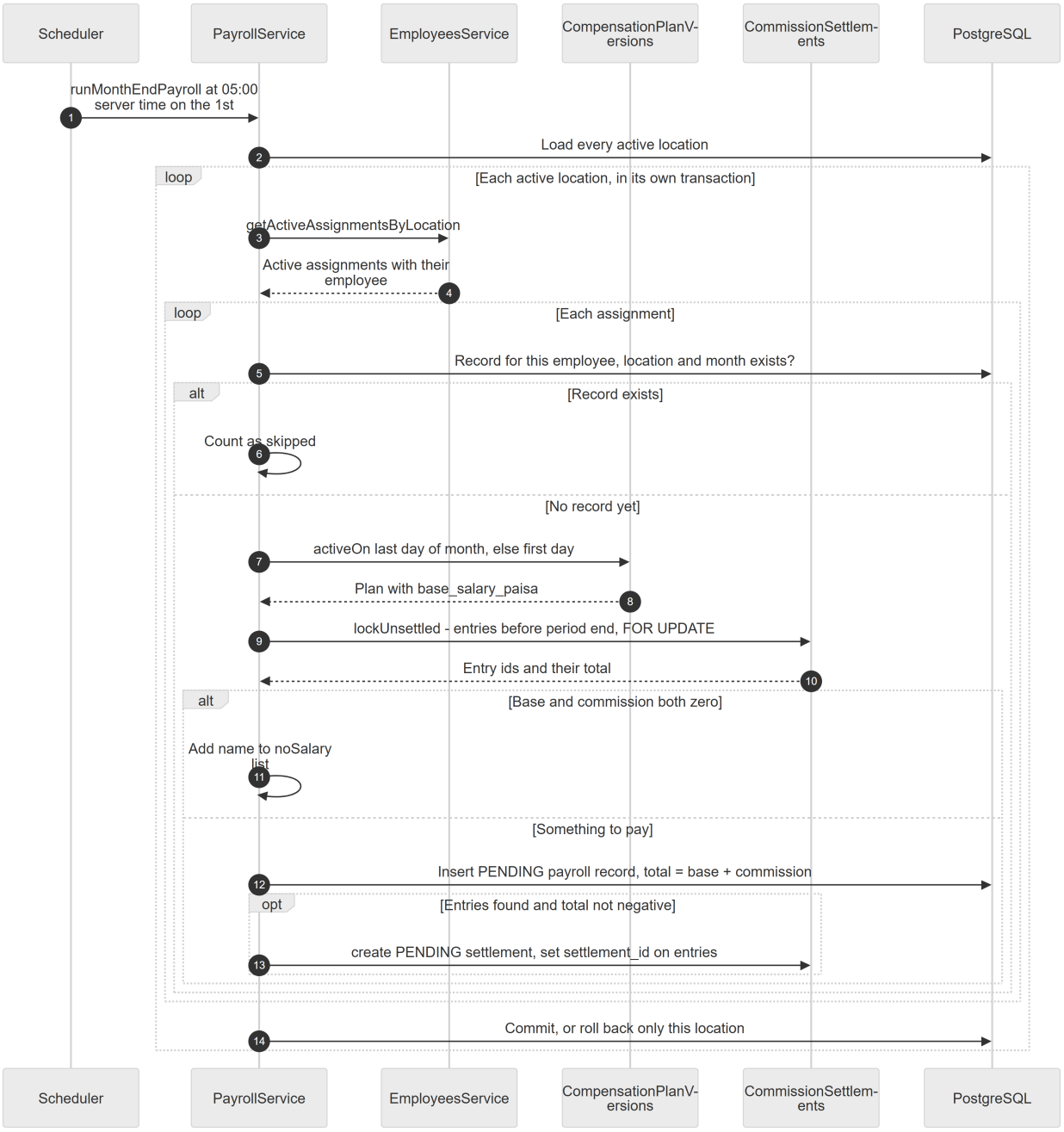

Figure 38.6 — The month-end job, with commission settlement, per location and per assignment.

The diagram has six participants: Scheduler, PayrollService, EmployeesService, CompensationPlanVersions, CommissionSettlements and PostgreSQL. In step 1 the scheduler calls `runMonthEndPayroll` at 05:00 server time on the 1st. In step 2 the service loads every active location. A first box, "loop: each active location, in its own transaction", contains the rest. Step 3 asks EmployeesService for the active assignments of the location and step 4 returns them with their employees. A second box, "loop: each assignment", opens with step 5, a check whether a record for this employee, location and month exists. An "alt" box has two branches. If a record exists (step 6) it is counted as skipped. Otherwise step 7 asks CompensationPlanVersions for the plan in force on the last day of the month, else the first day, and step 8 returns the plan with `base_salary_paisa`. Step 9 calls `lockUnsettled` on CommissionSettlements for entries before the period end with a row lock and step 10 returns the entry ids and their total. A nested "alt" box then decides: if base and commission are both zero (step 11) the name is added to the no-salary list; otherwise step 12 inserts a PENDING payroll record whose total is base plus commission, and an optional step 13, taken only when entries were found and their total is not negative, creates a PENDING settlement and sets `settlement_id` on the entries. Step 14, after the loops, commits the location, or rolls back only that location.

Behind the scenes — in simple terms. It is the same work as manual generation, run for every clinic in one go. If one clinic fails, the others still run. Running it twice, or after a manual generation, is safe because existing records are skipped.

Behind the scenes — technically. `PayrollService.runMonthEndPayroll`, decorated `@Cron('0 5 1 * *', { name: 'month-end-payroll' })` with no timezone option, so it fires at 05:00 server time on the 1st. The code comment above it says "00:05", which is wrong; the expression means 05:00 (the same conflict is noted in the scheduled-jobs appendix). The month is computed from the server's own `new Date()` (previous month, December of the previous year in January), not from each location's timezone. It loads `locations` with `is_active = true` (without checking that the organization is active or that `hr.payroll` is enabled), calls `generateForLocation(…, userId = null)` for each in a try/catch, and logs totals generated and skipped. It runs where scheduled jobs run (the worker or all-in-one process).

Rules and validations. Same as 38.3.1, except there is no future-month check and no user.

Who can do it. The scheduler; no permission.

Data read and written. As in 38.3.1.

Effects on other modules. As in 38.3.1.

Error cases. A location error is logged ("Month-end payroll failed for location …") and skipped; there is no in-app notification.

Technical references: `backend/src/payroll/payroll.service.ts` · `PayrollService.runMonthEndPayroll` · no web-app use

38.3.3 Create a payroll record by hand

Status: [Implemented]

What it does. Adds one record for a chosen employee and month, for example for a mid-month joiner or a bonus.

How the user does it.

1. Click **Create Payroll**.
2. Pick the employee, month, year, amount, status and notes; submit.

Behind the scenes — in simple terms. The amount you type is stored as the base salary with zero commission. This record does not settle any commission.

Behind the scenes — technically. `POST /payroll` → `PayrollService.createManual`: refuses a duplicate (400 "A payroll record already exists for this employee and period"), then inserts with `base_salary_paisa = total_paisa` = rupee amount converted to paisa (at most 2 decimals), `commission_paisa = 0`, status from the request (default pending) and the notes.

Rules and validations.

- Month 1–12, year at least 2020, amount at least 0.
- The server does not check that the employee has an assignment at this location, nor that the month is not in the future.
- Choosing status "Paid" here records no payment method, no paid date and no settlement change.

Who can do it. `payroll.manage`, feature `hr.payroll`. Default: Admin.

Data read and written. Reads `payroll_records`. Writes `payroll_records`.

Effects on other modules. None.

Error cases. 400 "A payroll record already exists for this employee and period"; 400 "Amount must have at most 2 decimals".

Technical references: `backend/src/payroll/payroll.service.ts` · `PayrollService.createManual` · `web-app/src/features/employees/components/CreateEditPayrollModal.tsx`

38.3.4 View payroll for a month or for one employee

Status: [Implemented]

What it does. Lists the month's records of the location with a summary, or all records of one employee.

How the user does it.

1. Open **Payroll** and pick a month and year, or open an employee's **Payroll** tab.

Behind the scenes — in simple terms. The four summary cards add up the month's records by status.

Behind the scenes — technically. `GET /payroll/location?month&year` → `PayrollService.findByLocation` returns the records (with the employee) and a summary: total employees, total amount, paid, pending and on-hold counts, paid and pending amounts and the paisa split into base and commission. A month outside 1–12 or a year outside 2000–2100, or a non-number, returns 400 "month must be 1–12" / "year must be a four-digit year". `GET /payroll/employee/:id` → `findByEmployee` returns the person's records in the organization, newest first.

Rules and validations.

- The summary uses the rupee amounts; records created before the paisa breakdown fall back to those.
- The location list is scoped to the active location; the per-employee list covers every location of the organization.

Who can do it. `payroll.manage`, feature `hr.payroll`. Default: Admin.

Data read and written. Reads `payroll_records`, `employees`.

Effects on other modules. None.

Error cases. 400 "month must be 1–12"; 400 "year must be a four-digit year".

Technical references: `backend/src/payroll/payroll.controller.ts` · `payrollPeriod`, `PayrollService.findByLocation` · `web-app/src/features/employees/pages/PayrollPage.tsx`

38.3.5 Mark a record as paid

Status: [Implemented]

What it does. Records that the salary was paid, how, and with what reference, and marks the linked commission settlement as paid.

How the user does it.

1. In the table, click **Mark as Paid**.
2. Choose Payment Method, enter a Reference Number and optional Notes, and confirm.

Behind the scenes — in simple terms. The record becomes paid with the time, the method and who paid it. In the same step the commission that was bundled into it is marked paid, so it can never be released again.

Behind the scenes — technically. `PATCH /payroll/:id/mark-paid` → `PayrollService.markPaid`, one transaction: loads the record in the organization, refuses a paid one (400 "Payroll already marked as paid"), sets status `paid`, `paid_at` = now, `payment_method` (cash, bank_transfer or cheque), `payment_reference`, `paid_by` and `notes` (the

notes field is replaced, also by empty), then `CommissionSettlements.markPaid` sets the settlement to `PAID`.

Rules and validations.

- Payment method is required and must be one of the three values; reference and notes are optional text.
- An on-hold record can be paid.
- No money moves: the Platform records the payout, it does not make it, and it posts nothing to the general ledger or expenses.

Who can do it. `payroll.manage`, feature `hr.payroll`. Default: Admin.

Data read and written. Writes `payroll_records`, `commission_settlements`.

Effects on other modules. Settled commission appears as settled on the earnings pages (Chapter 35). The payroll cost report counts records by month (Chapter 36).

Error cases. 400 "Payroll already marked as paid"; 404 "Payroll record not found"; toast "Failed to mark as paid".

Technical references: `backend/src/payroll/payroll.service.ts` · `PayrollService.markPaid` · `web-app/src/features/employees/components/MarkPaidModal.tsx`

38.3.6 Put a record on hold

Status: [Partial] — the endpoint and the edit dialog's status field work, but no button calls `mark-on-hold`.

What it does. Marks a record as on hold, for example while a dispute is resolved.

How the user does it.

1. Click the edit icon on a record, set Status to **On Hold** and save.

Behind the scenes — in simple terms. On hold changes only the label and the summary counts. The commission stays tied to the record.

Behind the scenes — technically. `PATCH /payroll/:id/mark-on-hold` → `PayrollService.markOnHold` refuses a paid record (400 "Cannot put a paid payroll record on hold"). A client function for it exists in `payroll.api.ts`, but no hook or screen uses it. The edit dialog uses `PATCH /payroll/:id` (38.3.7).

Rules and validations. Not allowed on paid records.

Who can do it. `payroll.manage`, feature `hr.payroll`. Default: Admin.

Data read and written. Writes `payroll_records`.

Effects on other modules. None.

Error cases. 400 "Cannot put a paid payroll record on hold"; 404.

Technical references: `backend/src/payroll/payroll.service.ts` · `PayrollService.markOnHold` · `web-app/src/features/employees/services/payroll.api.ts`

38.3.7 Edit a payroll record

Status: [Implemented]

What it does. Changes the amount, month, year, status or notes of a record.

How the user does it.

1. Click the edit icon ("Edit payroll"), change fields and save.

Behind the scenes — in simple terms. The amount you edit is the total. Commission already inside the record stays as it is, and the difference is treated as base salary.

Behind the scenes — technically. `PATCH /payroll/:id` → `PayrollService.update`: for a new amount, total = paisa(amount); the amount may not be below `commission_paisa`; `base_salary_paisa = total − commission`. Month and year changes rebuild the period label. The save maps a unique-key clash (23505) to 400.

Rules and validations.

- Amount at least 0, at most 2 decimals; month 1–12; year at least 2020.
- Editing is allowed in any status, including paid.
- Setting the status to "paid" here does not set the paid date, method or reference and does not mark the settlement paid (see 38.7).

Who can do it. `payroll.manage`, feature `hr.payroll`. Default: Admin.

Data read and written. Writes `payroll_records`.

Effects on other modules. None.

Error cases. 400 "The amount cannot be below the commission already in this payroll"; 400 "A payroll record already exists for this employee and period"; 404 "Payroll record not found".

Technical references: `backend/src/payroll/payroll.service.ts` · `PayrollService.update` · `web-app/src/features/employees/components/CreateEditPayrollModal.tsx`

38.3.8 Delete a payroll record

Status: [Implemented]

What it does. Deletes a record and returns its commission to the unsettled pool, unless commission was already paid out.

How the user does it.

1. Click the delete icon ("Delete payroll") and accept the browser confirmation "Delete the payroll record for … ? This cannot be undone."

Behind the scenes — in simple terms. Deleting an unpaid record frees its commission so the next generation picks it up again. A record that paid out commission cannot be deleted.

Behind the scenes — technically. `DELETE /payroll/:id` → `PayrollService.remove`, one transaction: `CommissionSettlements.release` locks the settlement; `PENDING` → clears `settlement_id` on its entries and deletes the settlement; `PAID` → 400; none → nothing. Then the record is removed. The confirmation uses the browser's `window.confirm`, not the project's confirmation dialog.

Rules and validations. A paid record with no commission settlement (zero commission) can be deleted.

Who can do it. `payroll.manage`, feature `hr.payroll`. Default: Admin.

Data read and written. Deletes `payroll_records`, `commission_settlements`; updates `commission_entries`.

Effects on other modules. The commission appears as pending again on the earnings pages.

Error cases. 400 "This payroll paid out commission and cannot be deleted"; 404 "Payroll record not found".

Technical references: `backend/src/payroll/payroll.service.ts` · `PayrollService.remove` · `web-app/src/features/employees/components/PayrollTable.tsx`

38.3.9 Export a month as CSV

Status: [Partial] — the endpoint works; no button in the web app calls it.

What it does. Returns a CSV file for one month at the active location.

How the user does it. Not available in the web app. The API can be called directly.

Behind the scenes — in simple terms. One row per record with base salary, commission, gross, deductions, net, status, method and paid date.

Behind the scenes — technically. `GET /payroll/export?month&year` → `PayrollExportService.generatePayrollSummaryCsv` with the same month and year validation (400). Columns: Employee Name, Designation, Base Salary, Commission, Gross Pay, Deductions, Net Pay, Status, Payment Method, Paid Date. Deductions are always 0.00 and net equals gross. Designation reads a field the employee record does not have, so the column is empty. Response type `text/csv`, file name `payroll-<year>-<month>.csv`.

Rules and validations. Active location only. Amounts come from paisa.

Who can do it. `payroll.manage`, feature `hr.payroll`. Default: Admin.

Data read and written. Reads `payroll_records`, `employees`.

Effects on other modules. None.

Error cases. 400 "month must be 1–12" / "year must be a four-digit year".

Technical references: `backend/src/payroll/payroll-export.service.ts` · `PayrollExportService.generatePayrollSummaryCsv` · none

38.3.10 Payslip PDF

Status: [Partial] — the endpoint works; no button in the web app calls it.

What it does. Returns a one-page A4 PDF payslip for one record.

How the user does it. Not available in the web app.

Behind the scenes — in simple terms. The slip shows the period, the employee, base salary, commission, gross pay, deductions (always zero), net pay, status, payment method and paid date.

Behind the scenes — technically. `GET /payroll/:id/payslip-pdf` → `PayrollExportService.generatePayslipPdf`, built with `pdf-lib` and Helvetica (Latin text only). Returned inline as `application/pdf`, file name `payslip-<period>.pdf`. Records from before the paisa breakdown show their whole amount as base salary. The designation line is skipped because the employee record has none.

Rules and validations. The record must belong to the organization.

Who can do it. `payroll.manage`, feature `hr.payroll`. Default: Admin.

Data read and written. Reads `payroll_records`, `employees`.

Effects on other modules. None.

Error cases. 404 "Payroll record not found".

Technical references: `backend/src/payroll/payroll-export.service.ts` · `PayrollExportService.generatePayslipPdf` · none

38.3.11 Deductions, tax and bank files

Status: [Not implemented]

What it does. Nothing. There are no deductions, advances, loans, taxes, overtime or bank payment files; the "Deductions" column is a fixed zero, and unpaid leave does not reduce pay.

How the user does it. Not available.

Behind the scenes — in simple terms. Pay is base salary plus commission, nothing else.

Behind the scenes — technically. `PayrollRecord` has no deduction columns; the export functions print `0.00`.

Rules and validations. None.

Who can do it. Not applicable.

Data read and written. None.

Effects on other modules. None.

Error cases. None.

Technical references: `backend/src/payroll/payroll-export.service.ts` · `PayrollExportService` · none

38.4 Entities and data model

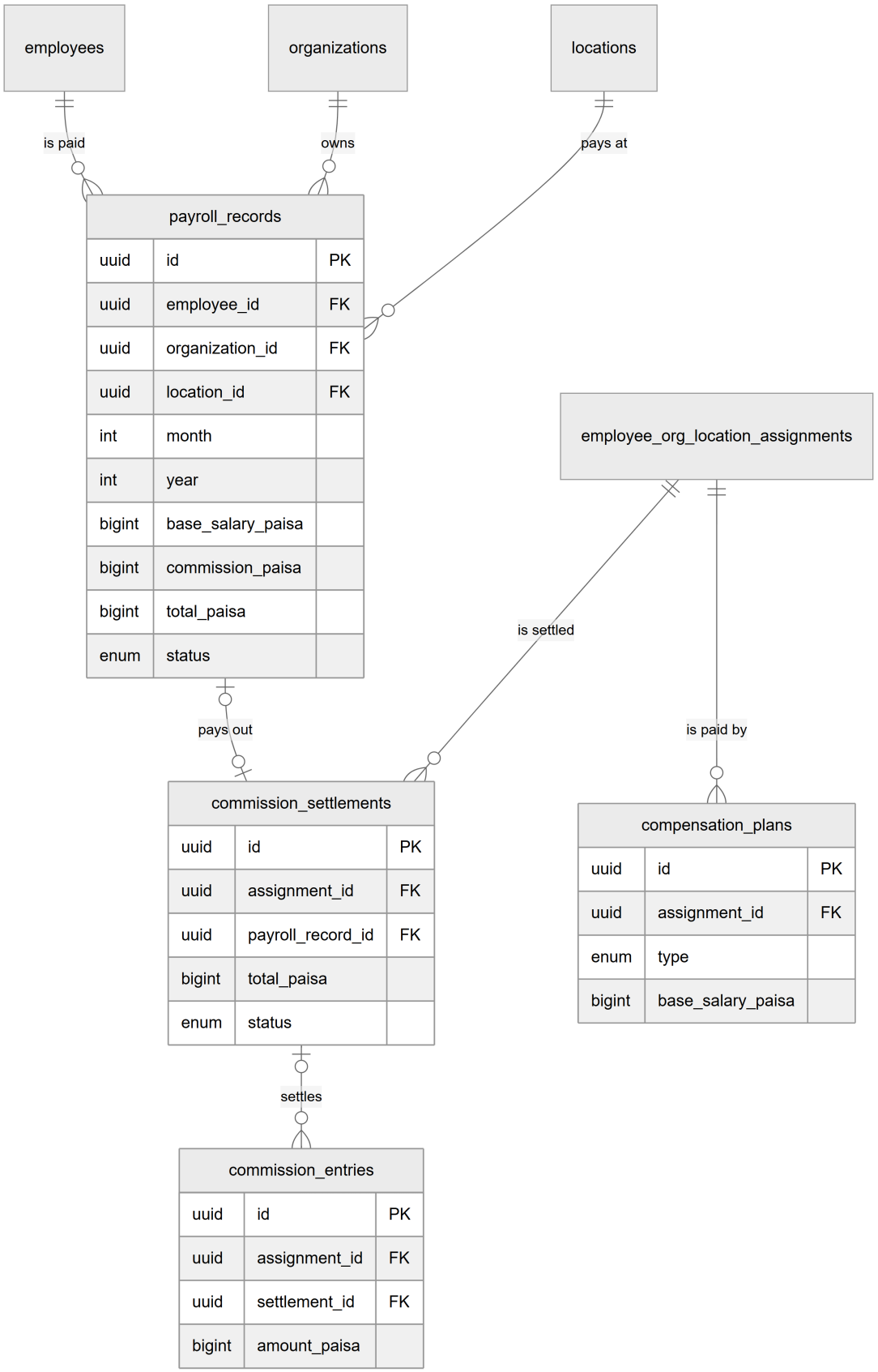

Figure 38.7 — The payroll record and the tables that feed it.

The diagram has `payroll_records` at the top, showing `id`, `employee_id`, `organization_id`, `location_id`, `month`, `year`, `base_salary_paisa`, `commission_paisa`, `total_paisa` and `status`. Three lines come into it from `employees` ("is paid"), `organizations` ("owns") and `locations` ("pays at"). A one-to-one optional line labelled

"pays out" connects it to `commission_settlements` (`assignment_id`, `payroll_record_id`, `total_paisa`, `status`). `commission_settlements` has an optional one-to-many line "settles" to `commission_entries` (`assignment_id`, `settlement_id`, `amount_paisa`), and a line "is settled" from `employee_org_location_assignments` (shown only as a relationship). The same assignment has a line "is paid by" to `compensation_plans` (`type`, `base_salary_paisa`).

The relationships in words:

- Each payroll record belongs to one employee, one organization and one location. Deleting any of them deletes the record (ON DELETE CASCADE).
- A record is unique per employee, location, month and year.
- A record has at most one commission settlement; the settlement's link to the record is ON DELETE RESTRICT, so a record that has one cannot be removed by a plain database delete. The service releases and deletes the settlement first.
- A settlement groups entries of one assignment; entries only point to it and are never changed otherwise.
- The base salary is copied from the compensation plan at generation time. Later plan changes do not change existing records.
- Columns: `period_label` (for example "October 2026"), `salary_amount` (rupee decimal mirror of the total), the three paisa columns, `status`, `paid_at`, `payment_method`, `payment_reference`, `paid_by`, `notes`, `created_by` (empty for the job).

38.5 Status lifecycles

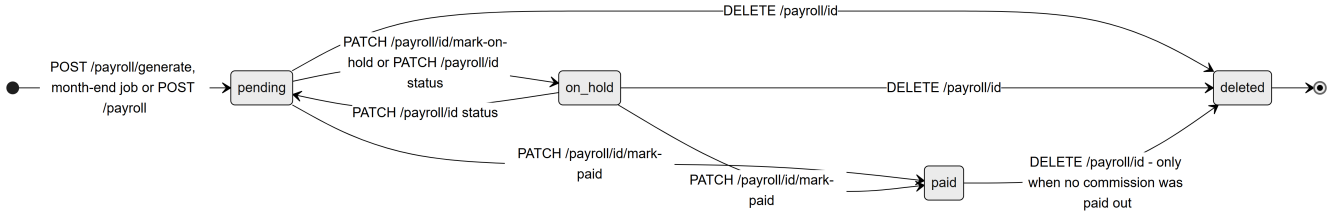

Figure 38.8 — The states of a payroll record.

The diagram has three stored states and a "deleted" end state. A record enters **pending** through `POST /payroll/generate`, the month-end job or `POST /payroll`. From pending, `PATCH …/mark-paid` leads to **paid**, and `PATCH …/mark-on-hold` or a `PATCH /payroll/:id` with a status change leads to **on_hold**. From on_hold, `PATCH …/mark-paid` leads to paid, and a status edit leads back to pending. Pending and on_hold lead to deleted through `DELETE /payroll/:id`; paid leads to deleted only when no commission was paid out. Deleted leads to the end marker.

- **pending** — generated and waiting for payment; its commission settlement, if any, is `PENDING`.
- **on_hold** — set aside; nothing else changes.
- **paid** — payment method, reference, date and payer recorded; the settlement is `PAID`. Editing is still possible.

38.6 Connections to other modules

- **Employees (Chapter 37).** Payroll runs over active assignments and reads their plans; the employee page has a Payroll tab.
- **Commissions (Chapter 35).** Settlement lock, mark paid and release.
- **Dashboard and reports (Chapter 36).** The payroll and staff-cost reports and the finance overview read `payroll_records` by month.
- **Subscriptions (Chapter 45).** Read-only mode refuses generation, edits and payments with 402.

- **Scheduled jobs (Chapter 63).** The month-end job is one of the timed jobs.

38.7 Limitations and known gaps

- The month-end job's code comment says 00:05; the schedule is 05:00 server time, and the month is taken from server time, not from each location's timezone.
- The job does not check that the organization is active or that `hr.payroll` is on.
- Marking a record paid through the edit dialog's Status field does not pay the commission settlement; a later delete would then release the commission back to the pool.
- Manual creation does not check the assignment or the month, and a hand-made record carries no commission.
- Payslip PDF and CSV export have no button.
- On-hold has no button; the delete confirmation is a browser dialog.
- Designation is empty on exports.
- No deductions, tax, overtime or unpaid-leave handling; no general-ledger posting for salary payment.
- Payroll changes are not written to the general audit log.

38.8 Owner's FAQ

When does payroll run by itself? At 05:00 server time on the 1st of each month, for the month that just ended, at every active location.

What if I generate manually first? Safe: existing records are skipped, so the job and manual runs never double up.

A doctor is paid only commission. Will they get a record? Yes, if they have unsettled commission of zero or more. A commission-only doctor with no earnings gets none and appears in the "no salary" list.

What happens to commission reversed after payroll was paid? It becomes a negative entry that is settled with the next payroll, reducing it.

Can I delete a paid record? Only if it paid out no commission.

Can I change the amount after generating? Yes, with the edit icon; it may not go below the commission in the record.

Can I get a payslip? The endpoint exists, but the web app has no button yet.

Is the unique rule per person or per branch? Per person per branch per month: a doctor at two branches gets two records.

Chapter 39 — Inventory

This chapter describes the Inventory module of the Platform: how a clinic keeps a list of its supplies, tracks how much of each it holds, buys more from suppliers through purchase orders, and defines kits of materials used per procedure. It is written for the product owner, who needs to know exactly what works today and where the module stops. Inventory is a per-location module; it has a complete set of screens, but no sidebar entry, and several of its write paths carry an unverified risk that is explained in full.

39.1 What this module is for

The module keeps a stock list for each location. Every change of quantity is recorded as a **stock movement**, so the clinic can always see who changed what, when and why. Staff can:

- add items (gloves, composite, anaesthetic cartridges…) with a unit, cost, reorder level and expiry date;
- correct stock by hand, write off wastage, record transfers and returns;
- keep a list of suppliers and raise purchase orders, then receive the goods into stock;
- define **consumption templates** (kits of items used for a procedure).

Who uses it: the **Admin** role (all actions) and the **Assistant** role (view only). Doctor, Receptionist and Accountant have no inventory permission by default.

Where it appears: **no sidebar entry**. The page `/inventory` exists and is protected by `inventory.view`, but `web-app/src/features/access/nav/nav.registry.ts` has no item for it, and no other screen links to it. Users reach it only by typing the address. The component that would record materials used on a patient (`ConsumptionRecorder`) is exported but not placed on any screen, so the consultation and patient screens offer no link either.

Two features exist: `inventory.core` ("Inventory", off by default) gates every permission of the module; `inventory.alerts` ("Inventory Alerts", off by default) only switches on the daily alert job.

39.2 Screens

The page title is "Inventory Management" with the subtitle "Track supplies, suppliers, purchase orders and consumption". Without an active location it shows "Select a location first". Above four tabs (**Items**, **Suppliers**, **Purchase Orders**, **Templates**) sits a strip of five tiles: Total Items, Total Value (stock × cost price), Low Stock, Expiring (within 30 days) and Out of Stock.

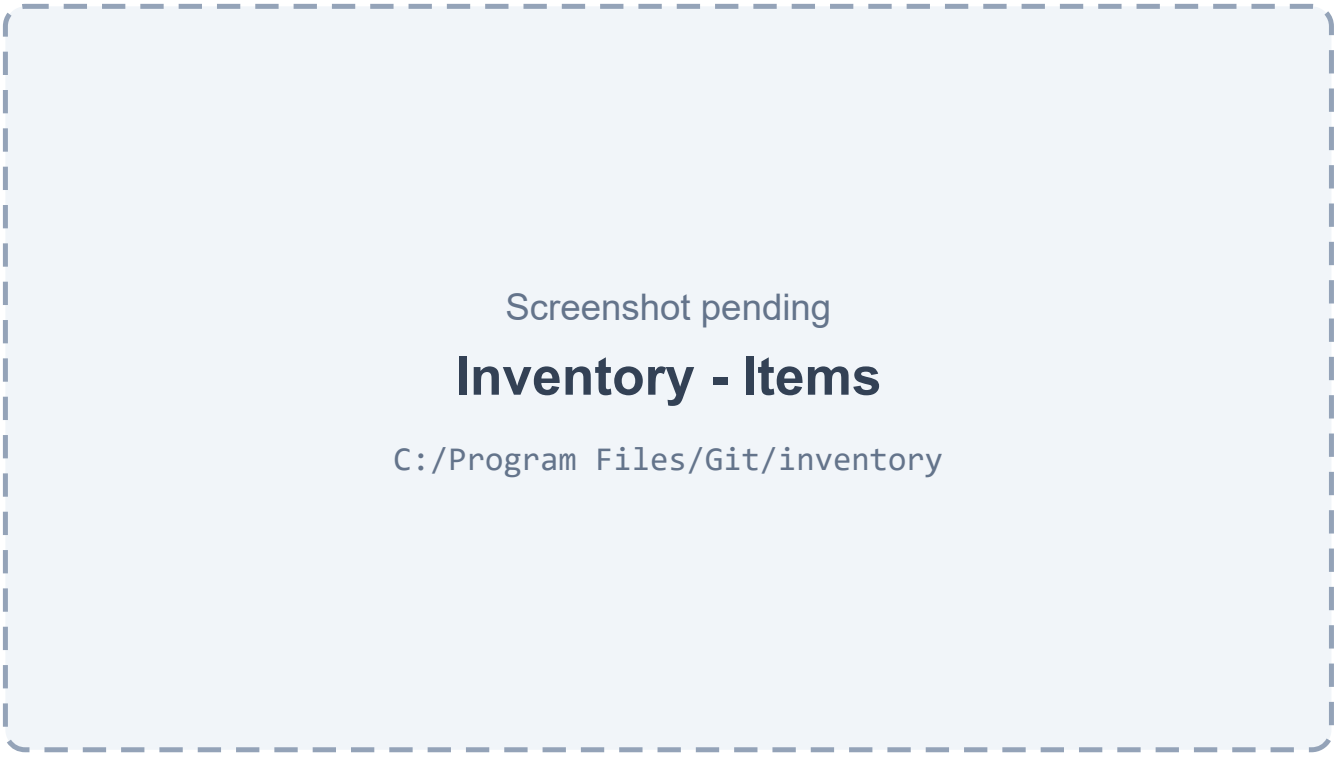

Figure 39.1 — Inventory, Items tab (screenshot pending).

The Items tab has a search box ("Search name or SKU…"), a category filter ("All categories"), two toggle buttons **Low Stock** and **Expiring**, and an **Add Item** button for users with `inventory.stock.adjust`. The table shows Name, SKU, Category, Stock (for example "12 / 10 pieces", coloured red when at or below the reorder level, amber up to twice that level, green above, or "Out of stock"), Unit Price and Expiry (a red "Expired" badge or a warning when within 30 days). A three-dot menu per row offers **Edit item**, **Adjust stock**, **View movements** and **Deactivate** (the last asks a browser confirmation "Deactivate "…"?"). Pages of 20 with **Previous** / **Next**. The **Add Item** / edit form is a dialog with Name, SKU, Category, Initial Stock (new items only), Unit, Cost Price, Reorder Level, Expiry Date, a **Billable to patient** switch with Selling Price, and Description.

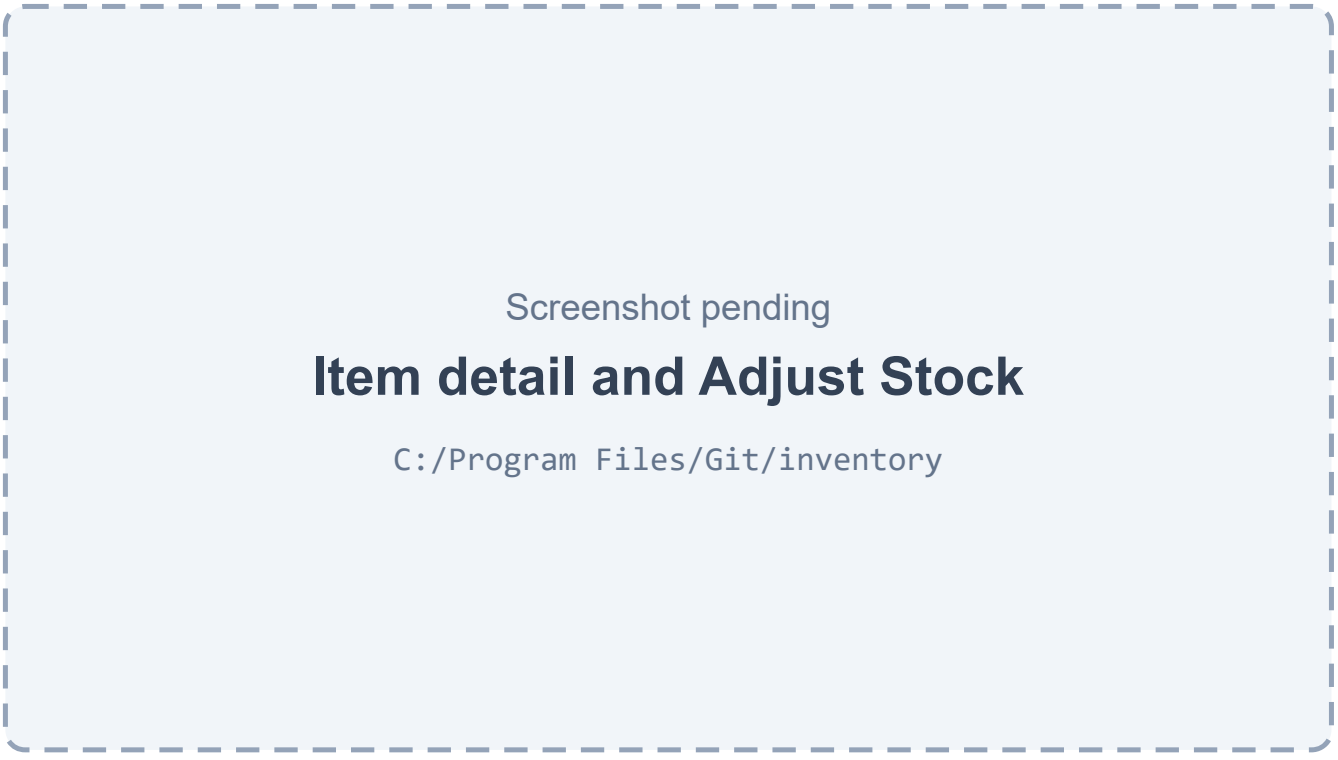

Figure 39.2 — Item detail sheet and the Adjust Stock dialog (screenshot pending).

Clicking a row opens a side sheet with the item name, SKU, Category, Unit, Cost Price, Selling Price, Expiry, Current Stock against the reorder level, an **Adjust Stock** button and a "Stock Movements" table (Date, Type, Qty, After, Notes) of the ten most recent movements. The Adjust Stock dialog, titled "Adjust Stock — ", offers Reason (Manual Adjustment, Wastage, Return to Supplier, Transfer In, Transfer Out), Quantity, Direction (Increase or Decrease, for manual adjustments), Batch Number and Expiry Date (for incoming stock), Notes (required for manual adjustment and wastage), and a preview "Current: 12 → New: 9".

Figure 39.3 — Inventory, Suppliers tab (screenshot pending).

The Suppliers tab lists Name, Contact Person, Phone, Email and City, with **Add Supplier** for users with `inventory.po.manage` and a row menu to edit or deactivate. The supplier dialog has Name (required), Contact Person, Phone, Email, City, Address, Website and Notes ("Payment terms, delivery schedule…").

Screenshot pending

Inventory - Purchase Orders

C:/Program Files/Git/inventory

Figure 39.4 — Inventory, Purchase Orders tab with its dialogs (screenshot pending).

The Purchase Orders tab has status buttons All, Draft, Ordered, Partial and Received (no Cancelled filter) and **Create PO**. The table shows PO #, Supplier, Order Date, Expected, Items, Total and Status. The "Create Purchase Order" dialog takes a supplier, Expected Delivery, items with Qty and Unit Price, Tax Rate (%) and Notes, and shows Subtotal, Tax and Total. Clicking a PO opens a sheet with its status badge, the lines (Item, Ordered, Received, Unit Price, Total) and buttons by status: **Mark as Ordered** and **Cancel** for a draft, **Receive Items** (or **Receive More Items**) for an ordered or partly received PO. The "Receive Items — PO-..." dialog asks per line for "Receiving now", Batch # and Expiry.

Screenshot pending

Inventory - Templates

C:/Program Files/Git/inventory

Figure 39.5 — Inventory, Templates tab (screenshot pending).

The Templates tab lists Template, Procedure, Appointment Type and Items, with **Add Template**, edit and deactivate actions for users with `inventory.stock.adjust`. The template dialog has a name ("Root Canal Kit"), a procedure ("Root Canal"), "Appointment Type (for auto-matching)" and "Add Items to Kit" with quantity and notes per item.

39.3 Features

39.3.1 Browse stock and see stock health

Status: [Partial] — works; reachable only by typing `/inventory` (no sidebar entry).

What it does. Lists the active items at the active location with search, category, low-stock and expiring filters, and shows five stock-health tiles.

How the user does it.

1. Type the address `/inventory`.
2. Search, pick a category, or toggle **Low Stock** / **Expiring**.

Behind the scenes — in simple terms. Low stock means more than zero but at or below the reorder level. Expiring means the item's nearest expiry date is within 30 days (already expired items count too). Total value is quantity times the last cost price.

Behind the scenes — technically. `GET /inventory/items` (`InventoryService.findAll`, paged, default 20, `ILIKE` on name and SKU) and `GET /inventory/items/stats` (`getStats`). Separate endpoints `GET /inventory/items/low-stock`, `/expiring?daysAhead=30` and `/out-of-stock` exist; the web app has hooks for two of them but no component calls them. `GET /inventory/categories` fills the category filter.

Rules and validations.

- Active location and active items only.
- The 30-day window uses the server's date, not the location's; the per-item `expiryAlertDays` setting is stored but not used.
- The search box is not debounced; each keystroke queries.

Who can do it. `inventory.view`, feature `inventory.core`. Default: Admin, Assistant.

Data read and written. Reads `inventory_items`, `inventory_categories`.

Effects on other modules. None.

Error cases. 403 `NO_PERMISSION`; the page shows "No items found" for an empty result (no separate filtered-empty message).

Technical references: `backend/src/inventory/controllers/inventory.controller.ts` · `InventoryService.findAll`, `getStats` · `web-app/src/features/inventory/components/ItemsTab.tsx`

39.3.2 Add, edit and deactivate an item

Status: [Partial] — adding without stock, editing and deactivating work; adding with an initial stock goes through the stock-movement path whose foreign-key risk is described in 39.3.4 ([Unverified]).

What it does. Creates a stock item at the active location, edits its details, or hides it (soft delete).

How the user does it.

1. Click **Add Item**, fill the dialog and save; or choose **Edit item** in the row menu.

2. To remove, choose **Deactivate** and confirm.

Behind the scenes — in simple terms. A new item starts at zero. If an initial stock is entered, an "initial stock" movement is recorded right after. Editing can never change the quantity; only movements can.

Behind the scenes — technically. `POST /inventory/items` → `InventoryService.create` (SKU uniqueness check per organization, save with `currentStock = 0`, then `adjustStock` with `INITIAL_STOCK` when `initialStock > 0`; these are two separate steps, not one transaction). `PATCH /inventory/items/:id` → `update` (drops `currentStock` from the payload). `DELETE /inventory/items/:id` → `deactivate` (sets `isActive = false`).

Rules and validations.

- Name, unit, cost price and initial stock (0 or more) are required by the API; reorder level defaults to 10, reorder quantity to 50, expiry alert days to 30.
- SKU must be unique within the organization (checked in code, not by a database constraint).
- Units: pieces, boxes, pairs, rolls, bottles, packets, cartridges, tubes, ml, gm, kg, liters.
- "Billable to patient" and the selling price are stored, but no billing screen uses them.

Who can do it. `inventory.stock.adjust` (implies `inventory.view`), feature `inventory.core`. Default: Admin.

Data read and written. Writes `inventory_items` (and `stock_movements` for initial stock).

Effects on other modules. None.

Error cases. 409 "An item with SKU "GLV-M-001" already exists"; 404 "Inventory item not found"; in the dialog "Name is required".

Technical references: `backend/src/inventory/services/inventory.service.ts` · `InventoryService.create`, `update`, `deactivate` · `web-app/src/features/inventory/components/ItemFormDialog.tsx`

39.3.3 Manage categories

Status: [Partial] — the API supports create, rename and deactivate; the web app only lists categories, so there is no way to create one from a screen.

What it does. Groups items (for example "Consumables", "Anaesthetics") for filtering.

How the user does it. Only pick an existing category in the item form or the filter. No screen creates categories, and the default-category seeding function is never called, so a new organization has none.

Behind the scenes — in simple terms. Categories are shared by all locations of an organization.

Behind the scenes — technically. `POST /inventory/categories`, `GET`, `PATCH /inventory/categories/:id`, `DELETE` (deactivate) → `InventoryCategoryService`. `seedDefaults` exists but has no caller. The web hooks `useCreateCategory`, `useUpdateCategory`, `useDeleteCategory` are not used by any component.

Rules and validations. Organization-wide; deactivated categories are hidden. Deleting a category row in the database sets items' category to null.

Who can do it. View: `inventory.view`; change: `inventory.stock.adjust`. Feature `inventory.core`.

Data read and written. `inventory_categories`.

Effects on other modules. None.

Error cases. 404 "Category not found".

Technical references: `backend/src/inventory/services/inventory-category.service.ts` · `InventoryCategoryService` · `web-app/src/features/inventory/hooks/useCategories.ts`

39.3.4 Adjust stock and keep the movement history

Status: [Unverified] — the logic is complete and the screen calls it, but the code stores the signed-in user's id in `stock_movements.performedBy`, a column with a foreign key to `employees`. User ids and employee ids are different records on the Platform, so the insert is expected to fail with a database error. Not run or tested.

What it does. Changes an item's quantity for a stated reason and records the change as a stock movement with the stock before and after.

How the user does it.

1. Choose **Adjust stock** in the row menu, or **Adjust Stock** in the item sheet.
2. Pick the Reason, enter the Quantity (and Direction for a manual adjustment), optional Batch Number and Expiry Date, and Notes when required.
3. Check the "Current → New" preview and save.
4. Open the item (or **View movements**) to see the last ten movements.

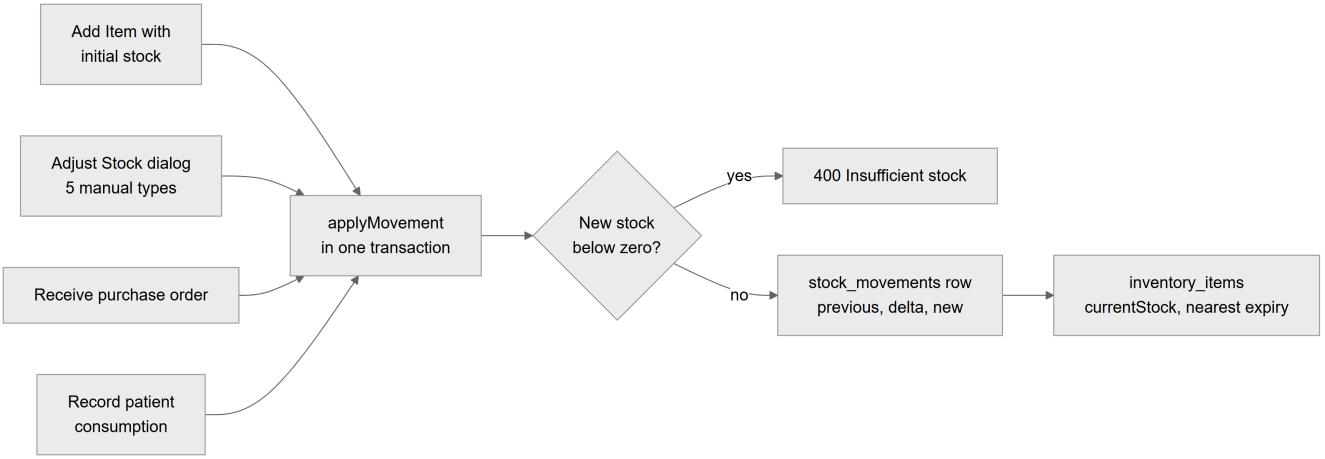

Figure 39.6 — All four ways stock changes go through one method.

On the left are four sources: "Add Item with initial stock", "Adjust Stock dialog 5 manual types", "Receive purchase order" and "Record patient consumption". All four arrows meet at "applyMovement in one transaction". From there a diamond asks "New stock below zero?". "Yes" leads to "400 Insufficient stock". "No" leads to "stock_movements row previous, delta, new" and then to "inventory_items currentStock, nearest expiry", meaning the movement is saved first and the item's quantity and nearest expiry date are then updated.

Behind the scenes — in simple terms. Incoming types (transfer in, purchase, initial stock) add; outgoing types (wastage, transfer out, return to supplier, consumption) subtract; a manual adjustment can go either way. Stock may never go below zero. When a batch arrives with an expiry date earlier than the item's current one, the item's expiry date moves to that earlier date. The movement list is the audit trail of stock; there is no other log.

Behind the scenes — technically. `POST /inventory/items/:id/adjust-stock` → `InventoryService.adjustStock` → `applyMovement` inside `dataSource.transaction`. It loads the item (organization and location), computes the signed delta, refuses a negative result, inserts the `stock_movements` row (`movementType`, signed `quantity`, `previousStock`, `newStock`, `referenceType`, `referenceId`, batch, expiry, notes, `performedBy`) and saves the item. The item row is read without a row lock, so two adjustments at the same instant could both read the same starting stock. The controller passes `req.user.uuid` (the user id) as `performedBy`. Movement history: the item detail returns the last 10 movements; `GET /inventory/items/:id/movements` (paged) exists but no screen calls it.

Rules and validations.

- The dialog allows only MANUAL_ADJUSTMENT, WASTAGE, RETURN_TO_SUPPLIER, TRANSFER_IN, TRANSFER_OUT.

- Notes are required for manual adjustment and wastage.
- Quantity must be greater than zero in the dialog; resulting stock must not be negative.
- Transfers are recorded at one location only; nothing is added at the other location.

Who can do it. `inventory.stock.adjust`, feature `inventory.core`. Default: Admin.

Data read and written. Writes `stock_movements`, `inventory_items`.

Effects on other modules. None (stock is not linked to billing or the consultation).

Error cases. 400 "Notes are required for wastage and manual adjustments"; 400 "Insufficient stock for "Gloves". Available: 3 pieces"; 400 on a disallowed movement type; in the dialog "Enter a quantity greater than zero", "Adjustment would take stock below zero".

Technical references: `backend/src/inventory/entities/stock-movement.entity.ts` · `InventoryService.adjustStock`, `applyMovement`, `StockMovementType` · `web-app/src/features/inventory/components/AdjustStockDialog.tsx`

39.3.5 Manage suppliers

Status: [Partial] — works; reachable only through the unlisted `/inventory` page.

What it does. Keeps the organization's supplier list for purchase orders.

How the user does it.

1. On the **Suppliers** tab click **Add Supplier**, fill the dialog and save.
2. Use the row menu to edit or deactivate (with a browser confirmation).

Behind the scenes — in simple terms. Suppliers are shared by all locations of the organization; deactivation hides them.

Behind the scenes — technically. `POST /suppliers`, `GET /suppliers`, `GET /suppliers/:id`, `PATCH /suppliers/:id`, `DELETE /suppliers/:id` (sets `isActive = false`) → `SupplierService`.

Rules and validations. Name required; other fields free text with no format checks.

Who can do it. View: `inventory.view`; change: `inventory.po.manage`. Feature `inventory.core`. Default: Admin manages; Assistant views.

Data read and written. `suppliers`.

Effects on other modules. None.

Error cases. 404 "Supplier not found"; "Name is required" in the dialog.

Technical references: `backend/src/inventory/controllers/supplier.controller.ts` · `SupplierService` · `web-app/src/features/inventory/components/SuppliersTab.tsx`

39.3.6 Raise and track a purchase order

Status: [Unverified] — screens and logic exist; creation stores the user id in `purchase_orders.createdBy`, which has a foreign key to `employees` (same risk as 39.3.4). Also, PO numbers are unique across the whole database but numbered per organization, so the second organization's first order in a year would collide (deduced from code).

What it does. Records an order to a supplier with lines, quantities, prices and tax, and moves it through Draft → Ordered.

How the user does it.

1. On **Purchase Orders** click **Create PO**, choose a supplier, add items with Qty and Unit Price, optional Expected Delivery, Tax Rate and Notes, and save. The PO starts as Draft.
2. Open it and click **Mark as Ordered**, or **Cancel** while still a draft.

Behind the scenes — in simple terms. The server numbers the order (for example "PO-2026-00001"), computes each line total, the subtotal, the tax and the total.

Behind the scenes — technically. `POST /purchase-orders` → `PurchaseOrderService.create` : `generatePoNumber` takes the organization's latest PO and adds one (`PO-<year>-<5 digits>`), lines get `totalPrice = qty × price` rounded to 2 decimals, `taxAmount = subtotal × rate / 100` . `GET /purchase-orders` (status and supplier filters, paged), `GET /purchase-orders/:id` . `PATCH /purchase-orders/:id/status` checks `VALID_PO_TRANSITIONS` . `DELETE /purchase-orders/:id` cancels a draft. `poNumber` has a unique index over the whole table.

Rules and validations.

- At least one line in the dialog; ordered quantity ≥ 1; unit price ≥ 0.
- The sequence continues across years (the counter is not reset), and two orders created at the same moment could get the same number.
- Line items are not checked to belong to the organization.

Who can do it. View: `inventory.view` ; create and change: `inventory.po.manage` (implies `inventory.view`). Feature `inventory.core` . Default: Admin.

Data read and written. Writes `purchase_orders` , `purchase_order_items` .

Effects on other modules. None; purchase orders are not linked to expenses or lab payables, and do not appear in the P&L.

Error cases. 400 "Cannot transition purchase order from RECEIVED to ORDERED"; 400 "Only draft purchase orders can be cancelled"; 404 "Purchase order not found"; dialog "Select a supplier", "Add at least one item".

Technical references: `backend/src/inventory/services/purchase-order.service.ts` · `PurchaseOrderService.create` , `updateStatus` , `VALID_PO_TRANSITIONS` · `web-app/src/features/inventory/components/PurchaseOrdersTab.tsx`

39.3.7 Receive goods into stock

Status: [Unverified] — goes through the stock-movement path of 39.3.4 with the same foreign-key risk.

What it does. Records quantities that arrived against an ordered PO, adds them to stock with batch and expiry, updates the item's cost price, and sets the PO to Partially Received or Received.

How the user does it.

1. Open an Ordered or Partial PO and click **Receive Items** (or **Receive More Items**).
2. Enter "Receiving now", Batch # and Expiry per line, and save.

Behind the scenes — in simple terms. Each line can be received in several deliveries, but never more than ordered. Each received line adds a "purchase in" movement. When every line is complete the order becomes Received.

Behind the scenes — technically. `POST /purchase-orders/:id/receive` → `PurchaseOrderService.receiveItems` in one transaction: per line with quantity > 0, check `received + new ≤ ordered` , update the PO line, call `applyMovement` with `PURCHASE_IN` and reference `purchase_order` /PO id, and set `inventory_items.unitPrice` to the line price. Then it recomputes the status from all lines.

Rules and validations.

- Only from ORDERED or PARTIALLY_RECEIVED.
- No over-receiving; a zero line is skipped.

Who can do it. `inventory.po.manage`, feature `inventory.core`. Default: Admin.

Data read and written. Writes `purchase_order_items`, `purchase_orders`, `stock_movements`, `inventory_items`.

Effects on other modules. None.

Error cases. 400 "Cannot receive items for a purchase order in DRAFT status"; 400 "Cannot receive more than ordered for "...". Ordered: 10, already received: 8"; 404 "Purchase order item ... not found"; dialog "Enter at least one quantity to receive".

Technical references: `backend/src/inventory/services/purchase-order.service.ts` · `PurchaseOrderService.receiveItems` · `web-app/src/features/inventory/components/PurchaseOrdersTab.tsx`

39.3.8 Define consumption templates (procedure kits)

Status: [Partial] — templates can be created, edited and deactivated, but nothing on any screen uses them, because the consumption recorder is not placed anywhere.

What it does. Defines a named kit of items and quantities for a procedure, optionally tied to an appointment type for automatic matching.

How the user does it.

1. On **Templates** click **Add Template**, enter name and procedure, optionally an appointment type, add items with quantities and notes, and save.
2. Edit or deactivate from the row actions.

Behind the scenes — in simple terms. A template is a list, not stock; saving it changes no quantities.

Behind the scenes — technically. `POST /consumption-templates`, `GET`, `GET /consumption-templates/:id`, `PATCH` (replaces all items in one transaction), `DELETE` (deactivate), `GET /consumption-templates/by-type?appointmentType=` (first active template for that type) → `ConsumptionTemplateService`. Items load the inventory item eagerly.

Rules and validations. Name and procedure required; at least one item in the dialog; item quantity ≥ 0.01. Templates are organization-wide, but items belong to one location, so a template made at one location lists that location's items.

Who can do it. View: `inventory.view`; change: `inventory.stock.adjust`. Feature `inventory.core`. Default: Admin.

Data read and written. `consumption_templates`, `consumption_template_items`.

Effects on other modules. None.

Error cases. 404 "Consumption template not found"; dialog "Name and procedure are required", "Add at least one item".

Technical references: `backend/src/inventory/services/consumption-template.service.ts` · `ConsumptionTemplateService` · `web-app/src/features/inventory/components/TemplatesTab.tsx`

39.3.9 Record materials used on a patient

Status: [Not implemented] in the web app — the API and a ready component exist, but no screen shows the component. The API path also carries the foreign-key risk of 39.3.4.

What it does. Would record which items were used for a patient (optionally for an appointment or treatment plan), price them at cost, and deduct them from stock.

How the user does it. Not available. `ConsumptionRecorder` (template picker, "Items Used", "Add Item", Notes) is exported from the inventory feature but imported by no page, including the consultation workspace and the patient record.

Behind the scenes — in simple terms. One record per use, with lines priced at the item's current cost; stock goes down in the same transaction, so either everything is saved or nothing is.

Behind the scenes — technically. `POST /patient-consumptions` → `PatientConsumptionService.recordConsumption`: per line, load the item at the location, snapshot `unitPrice`, compute totals, save `patient_consumptions` with items, then `applyMovement` with `CONSUMPTION` and reference `appointment`, `treatment_plan` or `manual`. Read endpoints: `GET /patient-consumptions/appointment/:appointmentId`, `GET /patient-consumptions/patient/:patientId` (checks the patient is in the user's scope), `GET /patient-consumptions/report?dateFrom&dateTo` (top 10 items, total cost, per performer, template versus manual). None of them is called by a screen.

Rules and validations. Quantity ≥ 0.01 per line; stock must not go negative; the treatment-plan id is stored without a foreign key.

Who can do it. Record: `inventory.stock.adjust`; read: `inventory.view`. Feature `inventory.core`.

Data read and written. `patient_consumptions`, `patient_consumption_items`, `stock_movements`, `inventory_items`.

Effects on other modules. None today (not billed, not shown on the encounter).

Error cases. 404 "Inventory item … not found"; 400 "Insufficient stock…".

Technical references: `backend/src/inventory/services/patient-consumption.service.ts` · `PatientConsumptionService.recordConsumption` · `web-app/src/features/inventory/components/ConsumptionRecorder.tsx`

39.3.10 Daily low-stock and expiry alerts

Status: [Partial] — the job runs and counts, but only writes a warning to the server log; no user is notified.

What it does. Once a day finds items that are low or expiring at each location where both inventory features are on.

How the user does it. Switch on `inventory.alerts` (and `inventory.core`) in **Features**. Nothing appears for users; the counts go to the server log only.

Behind the scenes — in simple terms. Every morning the server checks each location's stock. A notification for users is planned in a code comment but not built.

Behind the scenes — technically. `InventoryAlertService.checkAlerts`, `@Cron('0 8 * * *')`, every day at 08:00 server time (not the location's timezone), in the process that runs scheduled jobs (worker or all-in-one). For each organization–location pair with active items it checks both features, calls `getLowStockItems` and `getExpiringItems` (30 days) and logs "[Inventory Alerts] org=… location=… lowStock=… expiringSoon=…". A TODO notes that in-app notifications should go to members holding `inventory.view`.

Rules and validations. Fixed 30-day window; out-of-stock items are not counted as low.

Who can do it. The job, no user. Feature `inventory.alerts` (no permission belongs to it).

Data read and written. Reads `inventory_items`. Writes nothing.

Effects on other modules. None; the notifications module (Chapter 44) receives nothing.

Error cases. None visible to users.

Technical references: `backend/src/inventory/services/inventory-alerts.service.ts` · `InventoryAlertService.checkAlerts` · none

39.4 Entities and data model

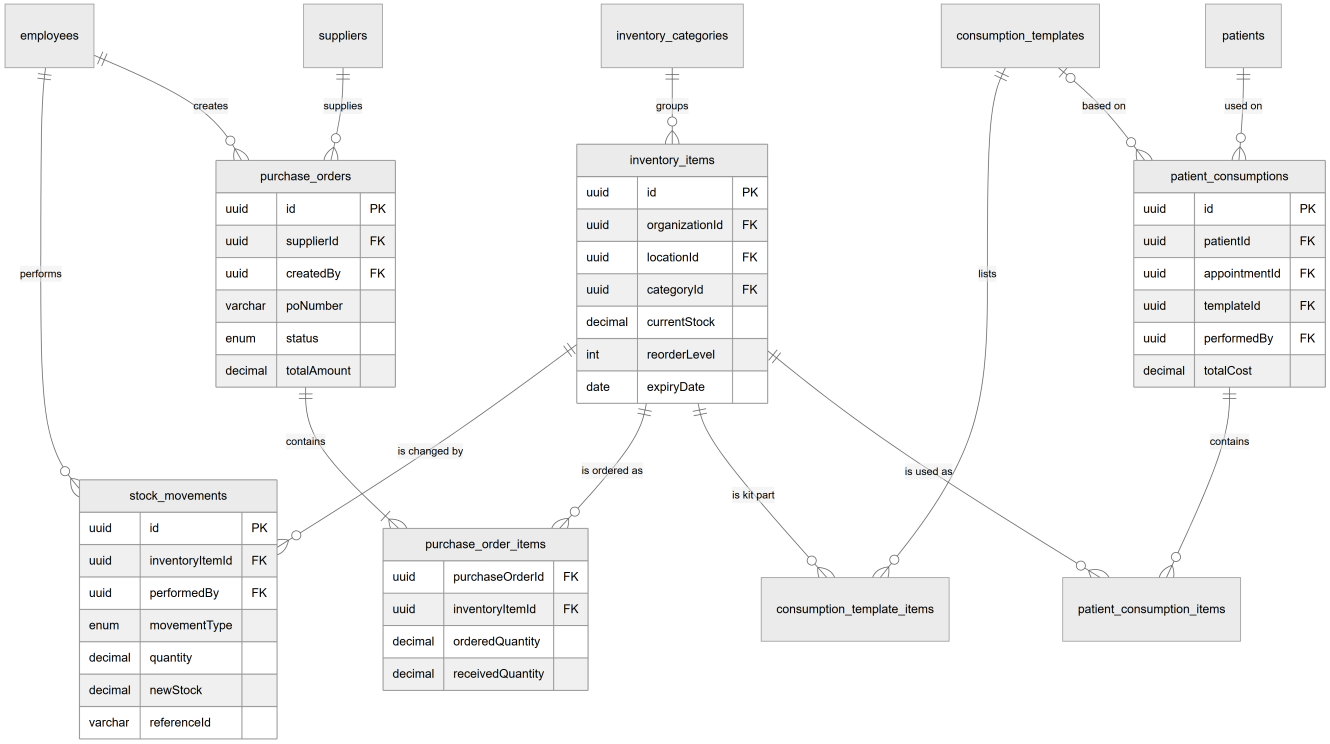

Figure 39.7 — The inventory tables and their links.

The diagram centres on `inventory_items` (`id` , `organizationId` , `locationId` , `categoryId` , `currentStock` , `reorderLevel` , `expiryDate`). Above it, `inventory_categories` "groups" items. Left, `stock_movements` (`inventoryItemId` , `performedBy` , `movementType` , `quantity` , `newStock` , `referenceId`) records how items "are changed"; `employees` "performs" movements and "creates" `purchase_orders` (`supplierId` , `createdBy` , `poNumber` , `status` , `totalAmount`), which `suppliers` "supplies". `purchase_orders` "contains" `purchase_order_items` (`purchaseOrderId` , `inventoryItemId` , `orderedQuantity` , `receivedQuantity`), each pointing to an item. Right, `consumption_templates` "lists" `consumption_template_items` (each a kit part pointing to an item); `patients` are "used on" `patient_consumptions` (`patientId` , `appointmentId` , `templateId` , `performedBy` , `totalCost`), which may be "based on" a template and "contains" `patient_consumption_items` , each pointing to an item.

The relationships in words (columns are camelCase in this module, unlike most of the Platform):

- Each item belongs to one organization and one location; deleting either deletes its items (CASCADE). An item may have a category; deleting the category sets it to NULL.
- Each stock movement belongs to one item (deleting the item deletes its movements, CASCADE) and names one employee in `performedBy` (FK to `employees` , NO ACTION). `referenceType` + `referenceId` point to the cause (purchase order, appointment, treatment plan, manual) without a foreign key; `referenceId` is text.
- Suppliers and categories belong to the organization (CASCADE), not to a location.
- Each purchase order belongs to one organization and location (CASCADE), one supplier and one creating employee (both NO ACTION). Its lines are deleted with it (CASCADE); each line points to one item.

- Each template belongs to the organization (CASCADE); its items are deleted with it (CASCADE) and point to items.
- Each patient consumption belongs to an organization, location and patient (CASCADE), optionally to an appointment (SET NULL) and a template (SET NULL); `treatmentPlanId` has no foreign key. Its lines are deleted with it (CASCADE).
- Quantities and prices are `decimal(10,2)` numeric columns in rupees, not paisa.
- A legacy entity `inventory` (snake_case columns `item_name`, `quantity`, `unit_price`) exists in the code but is registered by no module, so the running API never creates or uses it.

39.5 Status lifecycles

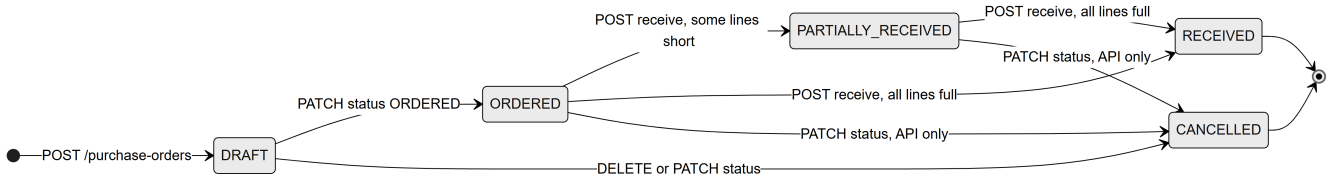

Figure 39.8 — The purchase order status map from `VALID_PO_TRANSITIONS`.

A purchase order enters at `DRAFT` through `POST /purchase-orders`. From `DRAFT`, "PATCH status ORDERED" leads to `ORDERED` (setting the order date), and "DELETE or PATCH status" leads to `CANCELLED`. From `ORDERED`, "POST receive, some lines short" leads to `PARTIALLY_RECEIVED`, "POST receive, all lines full" leads to `RECEIVED`, and "PATCH status, API only" leads to `CANCELLED`. From `PARTIALLY_RECEIVED`, "POST receive, all lines full" leads to `RECEIVED` and "PATCH status, API only" to `CANCELLED`. `RECEIVED` and `CANCELLED` are final. The screen offers cancel only for drafts; cancelling an ordered PO is possible only through the API, and cancelling a partly received PO does not reverse stock already received. A manual status change to `RECEIVED` is also allowed by the map without receiving any goods.

Stock movements have no lifecycle; they are written once and never changed. Their eight types are: `INITIAL_STOCK`, `PURCHASE_IN`, `TRANSFER_IN` (add), `CONSUMPTION`, `WASTAGE`, `TRANSFER_OUT`, `RETURN_TO_SUPPLIER` (subtract) and `MANUAL_ADJUSTMENT` (either way).

39.6 Connections to other modules

- **Access control (Chapter 57).** Three permissions (`inventory.view`, `inventory.stock.adjust`, `inventory.po.manage`) belong to the feature `inventory.core`.
- **Patients and Appointments (Chapters 28–29).** Patient consumptions point to patients and appointments; no screen uses this yet.
- **Consultation (Chapter 31).** Not connected: the consultation workspace does not record materials.
- **Billing and Reports (Chapters 34, 36).** Not connected: billable items, purchase orders and consumption costs appear in no invoice, expense or report.
- **Notifications (Chapter 44).** Not connected: the alert job only logs.

39.7 Limitations and known gaps

- No sidebar entry; `/inventory` is reachable only by address.
- Stock movements, purchase orders and patient consumptions store the user id in columns that reference employees; this is expected to fail at the database (not tested).

- PO numbers are globally unique but generated per organization, so they can collide between organizations; the counter does not reset each year and is not safe against simultaneous creation.
- No category management screen and no default categories.
- Consumption recording, consumption reports and the full movement history have no screen.
- Alerts are log-only and use the server's time and date.
- Stock changes do not lock the item row.
- Transfers do not move stock to the other location.
- Inventory costs do not reach the P&L, expenses or patient invoices.
- The tabs use plain tables without card view, and confirmations use the browser's dialog.

39.8 Owner's FAQ

How do staff open Inventory? Only by typing `/inventory`; there is no sidebar item. Adding one means adding an entry to the navigation registry.

Can stock go negative? No. Any movement that would take stock below zero is refused with "Insufficient stock".

Can I see who changed a quantity? Yes, in principle: every change is a stock movement with the before and after quantity, type, notes and the person. The item sheet shows the last ten.

Does receiving a purchase order create an expense or a lab bill? No. Purchase orders are not linked to billing or the P&L.

Will the system warn me about low stock? Not on screen. The daily 08:00 job writes counts to the server log only. The Low Stock tile and filter on the page show the current state.

Are suppliers per branch? No. Suppliers, categories and templates are shared by the whole organization; items, stock and purchase orders are per location.

Can an Assistant change stock? Not with the default role: the Assistant can only view.

Where is stock stored? In `inventory_items.currentStock`, kept equal to the `newStock` of the latest movement in `stock_movements`.

Chapter 40 — Dental Lab

This chapter describes the clinic side of dental-lab work on the Platform: how a clinic keeps a list of the dental labs it works with, how it orders crowns, bridges and dentures from them, follows the work through try-ins and shipments, and approves or sends back what comes home. It is written for the product owner, who needs to know exactly what the module does today and where it stops. "Lab" here always means a dental laboratory that makes prosthetics, never a diagnostic test lab. The lab's own screens (the lab portal and the no-login magic link page) are in Chapter 41, and the money side (patient lab invoices, vendor bills, payments, statements) is in Chapter 42.

40.1 What this module is for

A dental clinic rarely makes crowns, bridges and dentures itself. It sends a **lab order** to a **lab**, the lab makes the work in stages, ships it back, and the clinic fits it on the patient. This module covers that whole case from the clinic's chair:

- **Lab list.** The clinic chooses which labs it works with. Each connection is a **lab link** that carries payment terms and how much of the patient's identity the lab may see.
- **Lab price list.** For every service a linked lab offers, the clinic sees what the lab charges it and what it charges the patient, and maps each lab service to one of its own billable treatments.
- **Order templates.** A doctor saves "my usual zirconia crown" once and starts later orders from it.
- **Lab orders.** One order is one case for one patient with one or more **lab order items** (a crown, a bridge). The order is a draft until submitted. At submit, prices and form answers are frozen. After submit, changes are revisions, never overwrites.
- **Lab access.** The lab can answer through the lab portal, through a **magic link** (a no-login web page the clinic sends by message) or, when the lab does not answer at all, a lab coordinator at the clinic can record the lab's answer on its behalf.
- **Production.** Work moves through the lab's stages. Try-in rounds go to the clinic and back. The clinic receives parcels, approves or rejects them, delivers the finished item and, if needed, asks for a remake.

Who uses it, by canonical role:

- **Doctor.** Creates and submits orders, answers the lab's questions, decides try-ins, approves or rejects work, delivers it. Sees orders they ordered or created (scope `own`).
- **Receptionist** and **Assistant.** Can view the lab orders of their location and receive shipments at the front desk. They cannot create or change orders.
- **Admin** and the **owner.** Everything, including the lab list, the lab price list, shared templates and acting for a lab.
- **Accountant.** No lab-order access; the Accountant holds the lab billing and payables permissions that Chapter 42 covers.
- **Super admin.** Maintains the platform lab directory, the lab members and each lab's services and prices.

Where it appears in the sidebar (labels from `nav.registry.ts`, all inside the **Clinic** section by default): **Labs** (`/labs`, needs `lab.links.manage`), **Lab orders** (`/lab-orders`, needs `lab.orders.view`), **Lab price list** (`/lab-catalog`, needs `lab.catalog.manage`) and **Order templates** (`/lab-order-templates`, needs `lab.orders.create`). **Lab billing** and **Lab payables** also sit there and belong to Chapter 42. A **Lab portal** entry appears only for users who are lab members. For the super admin there is **Lab Directory** (`/lab-directory`) in the **Admin** section.

The clinic side sits behind the feature `dental.lab_orders` (display name "Lab Management"). It is not enabled by default: the super admin entitles it to an organization and the owner switches it on. The platform routes (directory, catalog) ignore it, and so do the lab portal routes, because a lab is not a tenant.

How the lab order runs on the workflow engine

A lab order, each lab order item and each try-in stage round have a status. These statuses are **not fixed in code**. Each one is a state of a **workflow template** (a versioned definition of states and transitions that the super admin edits in Config Studio; Chapter 43 covers the engine). When a draft order is created it pins the newest published order workflow of its specialty, and every item it contains pins an item workflow. Later edits to a template never change cases already in flight. Every status change goes through one engine call, which locks the row, finds the (state, action) pair in the pinned version, checks the actor's side (clinic, lab or system) and role, checks a required reason or input, writes the new state, writes the status history and then runs the transition's effects.

The default dental templates are seeded as published version 1 (`DENTAL_ORDER` , `DENTAL_ITEM` , `DENTAL_STAGE`). The state diagrams in section 40.5 show those seeded definitions. An organization that has published its own version would see different states, and nothing in this chapter would describe them. The labels in the web app are a lookup table for the seeded codes; an unknown code is shown with its underscores replaced by spaces.

One rule is in code, not in data: **the order's status follows its items.** After every item change the engine recomputes the order status in the same transaction, using fixed rules (first match wins): all items cancelled gives cancelled; a manual hold gives on hold; ignoring items that left the work (cancelled, rejected, remake required), all items approved or delivered gives completed; all received or later gives under review; some received gives partially received; all ready or dispatched gives ready; some ready gives partially ready; any in production, at clinic try-in or in QC gives in progress; all accepted gives accepted; otherwise submitted. The statuses draft, clarification needed, rejected by lab, cancelled and completed are never overwritten by this derivation, except that a remake reopens a completed order.

40.2 Screens

Figure 40.1 — Labs, the clinic's lab list (screenshot pending).

The **Labs** page (`/labs`) has the title "Labs", the subtitle "The dental labs your clinic sends work to." and an **Add lab** button. A filter row holds a search box ("Search by lab, city or phone…") and a status filter ("All statuses", Awaiting confirmation, Active, Suspended). A view toggle switches between a table and a grid of cards. The table has the columns Lab, Contact, Status, Payment terms, "Patient details shared" and an actions column. The row menu offers **Edit terms**, **Activate** (for a link awaiting confirmation), **Reactivate** (for a suspended link), **History** and **Suspend**. With no links the page shows "No labs yet" and a short explanation; with filters that match nothing it shows "No labs match your filters." and a clear-filters action. **Add lab** opens a side sheet that lists the platform directory with a search box ("Search labs by name or city…"), then asks for the payment terms in days, the patient-details setting and notes.

Figure 40.2 — The platform Lab Directory (screenshot pending).

The **Lab Directory** page (`/lab-directory` , super admin only; other users are sent to the dashboard) is titled "Lab Directory" with the subtitle "Dental labs on the platform. Clinics add labs from this list." Its table has the columns Lab, Type (External or In-house), Contact, Clinics, Staff and Status, plus an edit button on each row. Filters are a search box (name, city, phone or email), a status filter and a type filter. The **Add lab** sheet asks for the name, the type (an In-house lab also asks for the owner clinic), and the contact details and address.

Screenshot pending

Lab detail (super admin)

`C:/Program Files/Git/lab-directory/$labId`

Figure 40.3 — One lab in the directory (screenshot pending).

The lab detail page (`/lab-directory/$labId` , super admin only) shows the lab's phone, WhatsApp, email, city, address and, for an in-house lab, the clinic it belongs to. Under that are four tabs: **Services & prices** (the lab's catalog with price windows and options), **Staff** (the lab members, with a role selector and an active switch on each row and an **Add lab member** sheet), **Clinics** (every clinic linked to the lab, with link status, payment terms, patient-details setting and active-since date) and **Activity** (the audit timeline). A switch suspends or reactivates the whole lab after a confirmation.

Screenshot pending

Lab price list

`C:/Program Files/Git/lab-catalog`

Figure 40.4 — The clinic's Lab price list (screenshot pending).

The **Lab price list** page (`/lab-catalog`) has the title "Lab price list" and the subtitle "Lab costs, patient prices and which lab makes each treatment." It has two tabs. **Lab services** lists every service of every linked lab with the columns Service, Lab, Charged (per tooth, unit, arch or case), Lab cost, Patient price, Margin and an action to bill the service as one of the clinic's treatments; filters are search, lab, category and "Mapped or not". **Treatment mapping** lists the existing mappings with the columns "Your treatment", "Lab service" and Default, a star button to make a lab the default for a treatment and an unlink button that removes a mapping after a confirmation.

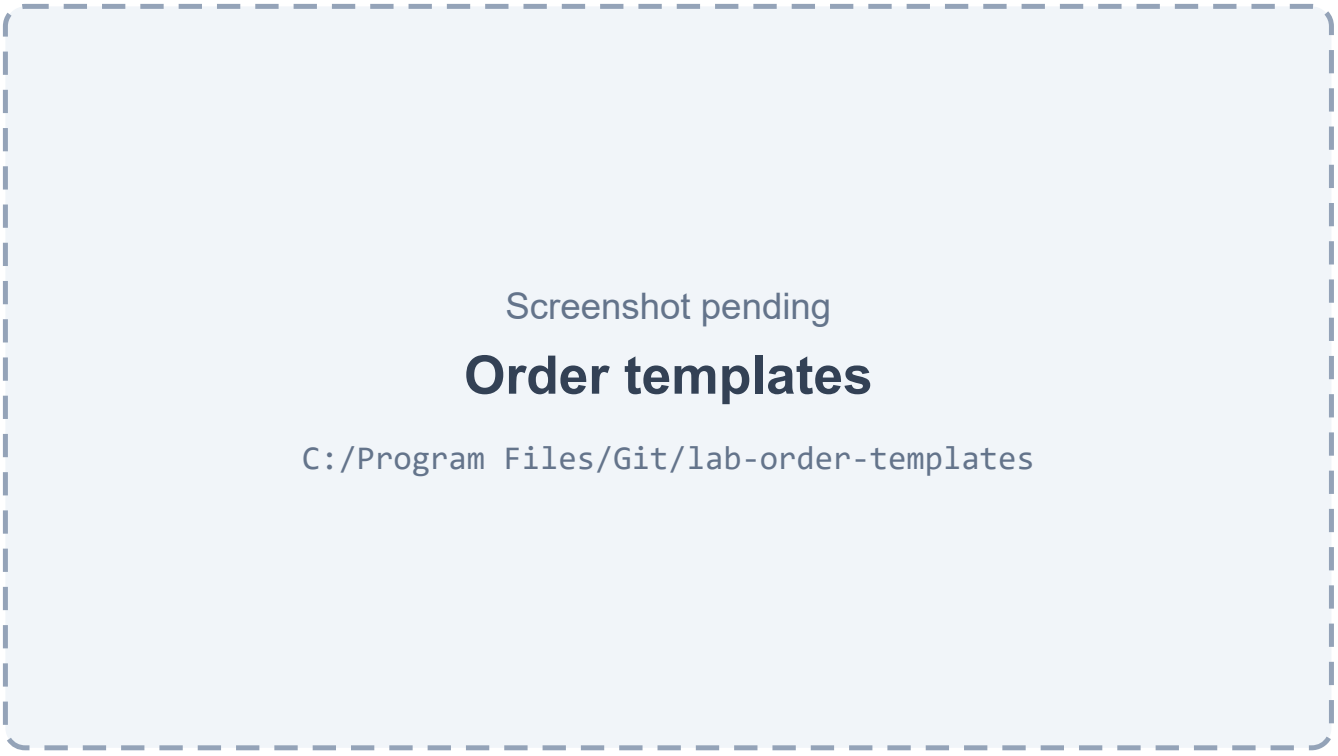

Figure 40.5 — Order templates (screenshot pending).

The **Order templates** page (`/lab-order-templates`) has the title "Order templates" and the subtitle "Your usual lab orders, ready to send in seconds." Its table has the columns Template, Lab, Owner, Used and an edit button; filters are search, owner (mine or shared), lab and "Include archived". The **New order template** sheet asks for a name, a "Share with the whole clinic" switch (shown only to users who may share), one or more services with their options, quantity, rush flag and any form answers, and "Notes for the lab". All services of a template must belong to one lab.

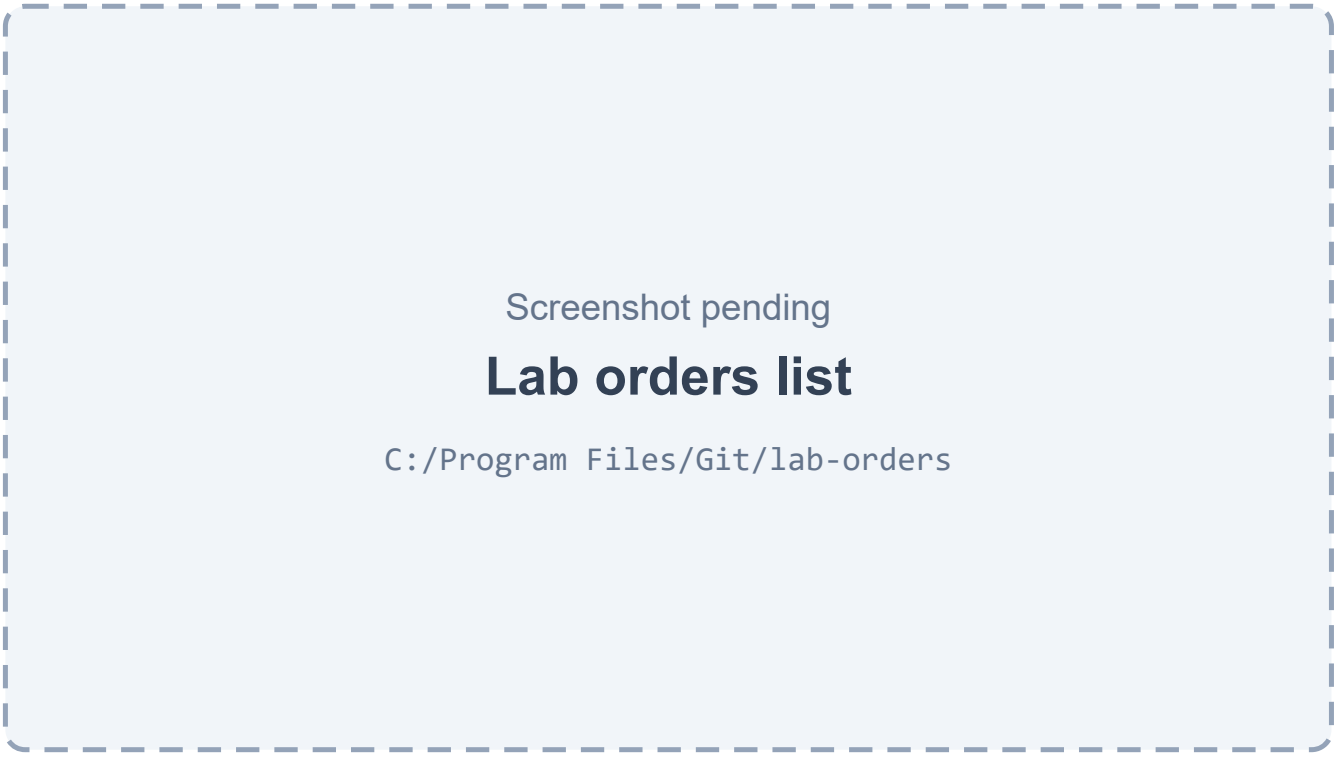

Figure 40.6 — The Lab orders list (screenshot pending).

The **Lab orders** page (`/lab-orders`) has the title "Lab orders" and the subtitle "Crowns, bridges, dentures — from draft to delivery." A **New lab order** button appears for users with `lab.orders.create` . Filters are a debounced search ("Search order no. or patient…"), status, lab, doctor and a From and To date. The table has the columns Order, Patient, Lab, Doctor, Status, "Due back", "Lab cost", Created and an actions column; a card view is the alternative. The lab cost is hidden for drafts. With no orders the page shows "No lab orders yet" and explains that prices are fixed at submit.

Figure 40.7 — The New lab order sheet, step 1 of 3 (screenshot pending).

The **New lab order** sheet is a three-step form: "Patient & lab", "Items" and "Prices & send". Step 1 has a patient search ("Search by name, number or phone…"), the lab, the ordering doctor (default: the signed-in user if they are a doctor), the priority, "Needed back by" and "Instructions for the lab". Once a patient is chosen, a "Quick start" box offers **From a template** and **Repeat last order**. Step 2 lists the items; each has the service, its options, quantity, a Rush switch and the form fields of that service (shade, material, teeth and so on). Step 3 shows the price quote per line (lab cost and patient price) and a total. Buttons are **Cancel** or **Back**, **Next**, **Save draft** and **Submit to lab**.

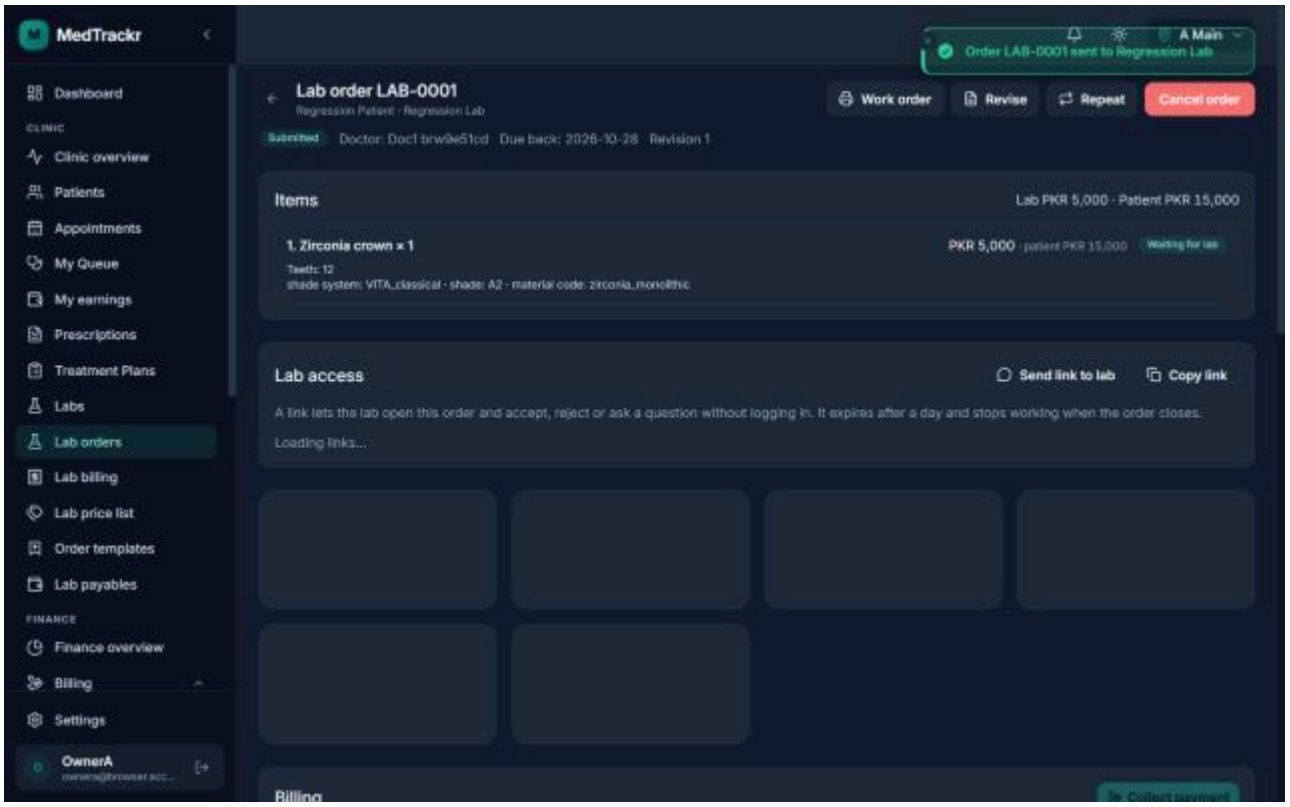

Figure 40.8 — Lab order LAB-0001 in status Submitted, with one zirconia crown item and the Lab access panel.

This screenshot was captured on 2026-10-23 in dark theme and shows the former working name "MedTrackr" in the sidebar corner. The page header reads "Lab order LAB-0001" with the line "Regression Patient · Regression Lab" under it and a back arrow to the lab orders list. A green toast at the top right confirms that the order was sent to the lab. Buttons on the right are **Work order**, **Revise**, **Repeat** and a red **Cancel order**. A status line shows the badge "Submitted", the doctor, the due-back date and "Revision 1". The **Items** section shows the total "Lab PKR 5,000 · Patient PKR 15,000" and one item, "1. Zirconia crown × 1", with tooth 12, the shade system, shade and material, the lab price PKR 5,000 with the patient price PKR 15,000, and the item badge "Waiting for lab". Under it the **Lab access** panel explains that a link lets the lab open the order and answer without logging in, that it expires after a day and stops working when the order closes, and offers **Send link to lab** and **Copy link**; its list of links was still loading when the picture was taken. The start of the Billing section is visible at the bottom. All names in the picture are test data.

Besides these, the order detail page has sections that the screenshot does not reach: **Production**, **Billing** (Chapter 42), **Messages**, **Extra charges**, **Revisions**, **History** and **Files**.

Screenshot pending

Production panel on a lab order

C:/Program Files/Git/lab-orders/$orderId

Figure 40.9 — The Production section of a lab order (screenshot pending).

The **Production** section lists "Incoming from the lab" shipments with their lines and a button to mark a shipment lost, then one block per item with its status, its try-in rounds as an ordered list ("Stages"), its warranty dates once delivered, and the next action for the clinic. Depending on the state these are **Record try-in**, **Approve** (the stage name is added), **Needs correction**, **Approve work**, **Reject — remake**, **Fitted — mark delivered** and **Request remake** (after delivery). A "Remake requests" block lists each request with its reason, responsible party, status and a decision button.

Screenshot pending

Receive shipment

C:/Program Files/Git/lab-orders/receive/$token

Figure 40.10 — The Receive shipment page opened from a slip's QR code (screenshot pending).

The receive page (`/lab-orders/receive/$token`) is titled "Receive SHP-0001" (the shipment number) with the subtitle "From" and the lab's name. Each line of the parcel has a tick box to receive it, a Condition selector (the first choice is OK, then Damaged and Missing), a received quantity, an "Incoming QC" selector with "Not checked" as its first choice, and a notes box ("e.g. box cracked, shade looks off"). The button is **Receive**. If nothing is left to receive the page says "Nothing left to receive" with the shipment status.

40.3 Features

40.3.1 Keep the platform lab directory

Status: [Implemented]

What it does. The super admin keeps one list of dental labs for the whole Platform. A lab is either **external** (serves many clinics) or **in-house** (belongs to one clinic and serves only it). A lab can be suspended; suspending blocks new links and new activations, while links that already exist stay as they are.

How the user does it.

1. As super admin, open **Lab Directory** in the sidebar.
2. Press **Add lab**. Enter the name, choose External or In-house (for In-house choose the owner clinic) and fill the contact details.
3. To change contact details press the edit button on the row. To suspend, use the status switch on the lab detail page and confirm.

Behind the scenes — in simple terms. Labs belong to the Platform, not to a clinic, so there is no clinic filter. When an in-house lab is created, the Platform also creates its link to the owner clinic, already active, in the same step.

Behind the scenes — technically. `GET/POST /platform/labs` , `GET/PATCH /platform/labs/:id` , `POST /platform/labs/:id/status` , `GET /platform/labs/:id/links` , `GET /platform/labs/:id/history` in `PlatformLabsController` (`@RequireSuperAdmin` , no tenant context) over `LabsDirectoryService` . Create runs in one transaction: owner check, insert into `labs` (UUID version 7 key), an `audit_logs` row, and for an in-house lab a call to `LabLinksService.createInTx` that inserts the active link plus a `status_history` row. Update and status change lock the lab row (`pessimistic_write`) and compare the sent `version` with the stored one (409 `STALE_VERSION` when they differ).

Rules and validations.

- Only in-house labs have an owner organization, and every in-house lab must have one (`LAB_OWNER_INVALID` , `LAB_OWNER_ORG_NOT_FOUND`).
- The lab name is not editable from the lab portal; only contact fields are (Chapter 41).
- Suspending a lab does not close existing links, but a magic link of a suspended lab stops working.

Who can do it. Super admin only (`@RequireSuperAdmin`). No feature key applies.

Data read and written. Reads and writes `labs` , `lab_clinic_links` , `audit_logs` , `status_history` ; reads `organizations` .

Effects on other modules. Clinics can add only active external labs and their own in-house labs.

Error cases. 404 `LAB_NOT_FOUND` ; 422 `LAB_OWNER_INVALID` or `LAB_OWNER_ORG_NOT_FOUND` ; 409 `STALE_VERSION` .

Technical references: `backend/src/labs/directory/platform-labs.controller.ts` · `LabsDirectoryService` · `web-app/src/features/labs/pages/LabDirectoryPage.tsx`

40.3.2 Manage the staff of a lab

Status: [Implemented]

What it does. The super admin adds people to a lab as **lab members**, with the role lab admin or lab technician, changes a role, and switches a member on or off. These accounts are how lab staff log in to the lab portal (Chapter 41).

How the user does it.

1. As super admin open a lab from **Lab Directory** and choose the **Staff** tab.
2. Press **Add lab member**, enter the account email and the role. The account must already exist: create it first under **User Admin**.
3. Change the role in the role selector, or flip the Active switch to deactivate.

Behind the scenes — in simple terms. A lab member is a normal user account with one extra row that says "works at this lab". Lab staff have no clinic membership at all, so no clinic screen can ever show their data and no lab screen can show a clinic's.

Behind the scenes — technically. `GET/POST /platform/labs/:id/members`, `PATCH /platform/labs/:id/members/:memberId` over `LabMembersService`. Add runs in a transaction: finds the user by lower-cased email (not deleted), runs the "may join" check, inserts into `lab_members`, writes an audit row. Update locks the member row and checks the version. Members are deactivated, never deleted.

Rules and validations.

- A super admin account cannot be a lab member (`LAB_MEMBER_NOT_ALLOWED`).
- A user who owns an organization or has an active staff assignment cannot join an external lab. For an in-house lab the only clinic account allowed is one of its owner clinic.
- One membership per lab and user (`LAB_MEMBER_EXISTS`).
- If no account uses the email, the message is "No account uses this email. Create the user first, then add them here."

Who can do it. Super admin. Lab admins can also add and change members of their own lab from the portal, but not themselves (Chapter 41).

Data read and written. Writes `lab_members`, `audit_logs`; reads `users`, `organization_ownership`, `employee_org_location_assignments`.

Effects on other modules. Active members receive in-app notifications about the lab's orders.

Error cases. 404 `LAB_MEMBER_USER_NOT_FOUND`; 409 `LAB_MEMBER_EXISTS`, `LAB_MEMBER_NOT_ALLOWED`; 409 `STALE_VERSION`.

Technical references: `backend/src/labs/members/lab-members.service.ts` · `LabMembersService` · `web-app/src/features/labs/components/LabMembersPanel.tsx`

40.3.3 Manage a lab's services, options and prices

Status: [Implemented]

What it does. For each lab, the super admin keeps the list of what the lab makes (a **lab service**, for example a zirconia crown), the **options** on it (a material or variant with an extra cost per unit) and its **prices**. A price is a window of dates. A lab's **list price** applies to every clinic; a **negotiated price** applies to one clinic and wins over the list price.

How the user does it.

1. As super admin open the lab and choose **Services & prices**.
2. Add a service: category, code, name, how it is charged (per tooth, per unit, per arch or per case), turnaround in days, description, and the form, item workflow and try-in stage template an order line will use.
3. Open the pricing sheet of a service to add prices (list or for one clinic, with "From" and "Until" dates and an optional rush surcharge) and options.
4. A price that has not started yet can be edited or voided. A running price can only be ended.

Behind the scenes — in simple terms. Prices are never overwritten. A new open-ended price closes the one running before it the day before. A price that is being used can only get an end date, so history stays true. A mistaken future price is voided, not deleted.

Behind the scenes — technically. Routes under `/platform/labs/:labId/services` in `PlatformLabServicesController` (`@RequireSuperAdmin`): list, get, create, patch, history, options (add, patch) and prices (list, add, patch, `POST …/void` , history). Services run in transactions; price writes lock the service row so two edits queue up, then pure functions (`planNewWindow` , `checkWindowEdit`) check dates, and the database exclusion constraint `excl_lab_service_prices_overlap` is the last line of defence. "Today" for a price window is the platform's day in the Asia/Karachi timezone (`CATALOG_TIMEZONE`), not a location's. Every change writes an `audit_logs` row.

Rules and validations.

- A new price cannot start in the past (`LAB_PRICE_INVALID`); the end date cannot precede the start date.
- Two non-void windows of the same service and clinic cannot overlap (`LAB_PRICE_OVERLAP`) unless the new one is open-ended and supersedes an open-ended one that began before it.
- A started price cannot change its amounts or start date (`LAB_PRICE_LOCKED`) and can end yesterday at the earliest.
- A negotiated price needs the clinic to be linked to the lab.
- The form, workflow and stage template must be active platform definitions of the service's specialty (`LAB_SERVICE_DEFINITION_INVALID`); the category must be of the same specialty.
- Amounts are `bigint` paisa, at most PKR 1,000,000,000 (`MAX_PAISA`).
- Archived services are hidden from new orders; old orders keep them.

Who can do it. Super admin. The lab itself cannot edit its catalog: no lab portal screen or route exists for it (the code comments say the portal "will add" one).

Data read and written. Writes `lab_services` , `lab_service_options` , `lab_service_prices` , `audit_logs` ; reads `service_categories` , `form_schemas` , `workflow_templates` , `stage_templates` .

Effects on other modules. Clinics see these services and prices in the Lab price list and in the order sheet.

Error cases. 409 `LAB_PRICE_OVERLAP` , `LAB_PRICE_LOCKED` ; 422 `LAB_PRICE_INVALID` , `LAB_SERVICE_DEFINITION_INVALID` , `SERVICE_CATEGORY_MISMATCH` .

Technical references: `backend/src/lab-catalog/categories/platform-catalog.controller.ts` · `LabServicesService` , `LabServicePricesService` , `price-resolver.ts` · `web-app/src/features/lab-catalog/components/LabServicesPanel.tsx`

40.3.4 Service categories

Status: [Partial]

What it does. Categories such as Crowns, Bridges, Implant restorations, Dentures, Orthodontic appliances and "Repairs & relines" group lab services, so clinics can compare labs like for like. They are seeded for the dental specialty.

How the user does it. The category list appears as a filter and in the service form. There is no screen to create or rename a category.

Behind the scenes — in simple terms. The six default categories exist after start-up. Adding more needs an API call.

Behind the scenes — technically. `GET/POST /platform/service-categories` and `PATCH …/:id` (super admin) exist; the web app calls only the list. `CatalogSeedService` inserts missing default categories and never changes existing ones.

Rules and validations. Category code is unique per specialty (`SERVICE_CATEGORY_EXISTS`).

Who can do it. Super admin.

Data read and written. `service_categories` , `audit_logs` .

Effects on other modules. None beyond grouping.

Error cases. 409 `SERVICE_CATEGORY_EXISTS` ; 404 `SERVICE_CATEGORY_NOT_FOUND` .

Technical references: `backend/src/lab-catalog/categories/service-categories.service.ts` · `ServiceCategoriesService` · `web-app/src/features/lab-catalog/hooks/usePlatformCatalog.ts` (read only)

40.3.5 Add a lab to the clinic's lab list

Status: [Implemented]

What it does. A clinic chooses a lab from the platform directory and adds it to its own list. The new link starts as **invited** ("Awaiting confirmation") until it is activated.

How the user does it.

1. Open **Labs** and press **Add lab**.
2. Search the directory and pick a lab. Labs already linked show their link status.
3. Set "Payment terms (days)", "Patient details shared" and optional notes, then press **Add lab**.
4. The message says "Lab added — activate it once the lab confirms". Activate it as in 40.3.7.

Behind the scenes — in simple terms. The directory offers only active external labs plus the clinic's own in-house labs; another clinic's in-house lab never appears and answers "not found" even if its id is guessed.

Behind the scenes — technically. `GET /lab-links/directory` (up to 25 labs, optional search) and `POST /lab-links` in `LabLinksController` (`lab.links.manage`) over `LabLinksService.create` : one transaction that read-locks the lab, inserts into `lab_clinic_links` with status invited (unique per lab and organization), writes an `audit_logs` row and a `status_history` row (action `invite`).

Rules and validations.

- The lab must be active (`LAB_SUSPENDED` otherwise) and visible to the clinic (`LAB_NOT_FOUND`).
- One link per lab and clinic (`LAB_LINK_EXISTS` : "This lab is already in your lab list").
- Payment terms are 0 to 365 days, default 30.

Who can do it. `lab.links.manage` , feature `dental.lab_orders` . Default: Admin. Reading the lab list (`GET /lab-links` , `GET /lab-links/:id`) is open to every member of the clinic (needed so doctors can pick a lab), still behind the feature.

Data read and written. Writes `lab_clinic_links` , `audit_logs` , `status_history` ; reads `labs` .

Effects on other modules. Only an active link allows ordering, pricing quotes and price negotiation.

Error cases. 404 `LAB_NOT_FOUND`; 409 `LAB_LINK_EXISTS`; 422 `LAB_SUSPENDED`.

Technical references: `backend/src/labs/links/lab-links.controller.ts` · `LabLinksService` · `web-app/src/features/labs/components/LabLinkSheet.tsx`

40.3.6 Set payment terms and patient data level

Status: [Implemented]

What it does. On each lab link the clinic sets how many days it takes to pay the lab, and how much of the patient's identity the lab sees: **Full patient name** or **Patient code only**. The change applies to new orders; the sheet says "Orders already sent keep their terms."

How the user does it.

1. In **Labs** open the row menu of the lab and choose **Edit terms** (or click the row).
2. Change "Payment terms (days)", "Patient details shared" or "Notes" and press **Save terms**.

Behind the scenes — in simple terms. With "Patient code only" the lab sees the clinic's patient number instead of the name on its order list, the order page, the magic link page, the printed work order and notifications. The patient number is still sent in a separate field in lab responses.

Behind the scenes — technically. `PATCH /lab-links/:id` → `LabLinksService.update`: lock, version check, copy changed fields, audit diff, save only if something changed. `patient_data_level` is read through the order's `lab_clinic_link_id` by `patientForLab` (order mapper) and by `orderRecipients` (notification variable `patient_ref`).

Rules and validations. Payment terms 0 to 365. A stale version gives 409.

Who can do it. `lab.links.manage`; Admin by default.

Data read and written. `lab_clinic_links`, `audit_logs`.

Effects on other modules. Payment terms are used by lab payables (Chapter 42).

Error cases. 404 `LAB_LINK_NOT_FOUND`; 409 `STALE_VERSION`.

Technical references: `backend/src/labs/links/lab-links.service.ts` · `LabLinksService.update`, `patientForLab` · `web-app/src/features/labs/components/LabLinkSheet.tsx`

40.3.7 Activate, suspend and reactivate a lab link

Status: [Implemented]

What it does. A link moves between three states: invited, active and suspended. Either side can stop working with the other and a reason is kept. A suspension can be lifted only by the side that made it.

How the user does it.

1. In **Labs** open the row menu.
2. Choose **Activate** on a link awaiting confirmation, or **Reactivate** on a suspended one.
3. To stop, choose **Suspend**, type a reason ("e.g. Repeated late deliveries") and confirm.
4. Choose **History** to see the timeline of changes.

Behind the scenes — in simple terms. Today the clinic can confirm a lab on the lab's behalf, so most links are activated by the clinic. A lab that has a portal account can also accept an invitation itself from its portal (Chapter 41). If the lab suspended the link, the clinic cannot undo it, and the reverse.

Behind the scenes — technically. `POST /lab-links/:id/transitions` with `action` and optional `reason`, `version`; `GET /lab-links/:id/history`. `LabLinksService.applyTransition` is the single place a link's status changes, shared with the lab portal. It checks the transition table (`activate`: from invited; `suspend`: from invited or active, reason required; `reactivate`: from suspended), refuses activation when the lab itself is suspended, and on reactivation reads the last suspension actor from `status_history` (`canLiftSuspension`). Writes `lab_clinic_links`, `status_history` (with `via`) and `audit_logs` in one transaction under a row lock.

Rules and validations.

- A reason is required to suspend (`LAB_LINK_REASON_REQUIRED`).
- A wrong from-state gives `INVALID_TRANSITION`.
- Only the suspending side (or the platform) reactivates (`LAB_LINK_SUSPENDED_BY_OTHER_SIDE`).
- `activated_at` is set on the first activation only.

Who can do it. `lab.links.manage`; Admin by default.

Data read and written. `lab_clinic_links`, `status_history`, `audit_logs`.

Effects on other modules. Ordering requires an active link (`LAB_NOT_LINKED`). Orders already sent are not touched by a suspension.

Error cases. 422 `INVALID_TRANSITION`, `LAB_LINK_REASON_REQUIRED`, `LAB_SUSPENDED`; 403 `LAB_LINK_SUSPENDED_BY_OTHER_SIDE`; 409 `STALE_VERSION`.

Technical references: `backend/src/labs/links/lab-link-transitions.ts` · `resolveLinkTransition`, `canLiftSuspension` · `web-app/src/features/labs/components/LabLinkActions.tsx`

40.3.8 Read the lab price list

Status: [Implemented]

What it does. Shows the clinic every active service of every lab on its list, with the price this clinic pays (its negotiated price, else the list price), the patient price of the mapped treatment at the active location and the margin. A clinic never sees another clinic's negotiated price.

How the user does it.

1. Open **Lab price list** and stay on **Lab services**.
2. Search, or filter by lab, category and "Mapped or not".
3. Open a service for its options and details.

Behind the scenes — in simple terms. The list joins the lab's catalog with the clinic's own billing catalog through the mapping. Services without a mapping show "Not mapped" for the patient price.

Behind the scenes — technically. `GET /lab-catalog/categories`, `/lab-catalog/services` and `/lab-catalog/services/:id` (`@AnyMember`, `@RequireFeature('dental.lab_orders')`, so every member of the clinic can read them) in `ClinicCatalogController` over `ClinicCatalogService`, tenant-filtered by the clinic's links.

Rules and validations. Only labs with a link on the clinic's list contribute services; unlinked ids answer 404.

Who can do it. Every clinic member for the reads, behind `dental.lab_orders`. The page itself opens with `lab.catalog.manage` (Admin by default).

Data read and written. Reads `lab_services`, `lab_service_options`, `lab_service_prices`, `service_mappings`, `services`, `lab_clinic_links`.

Effects on other modules. Feeds the order sheet's service pickers.

Error cases. 404 `LAB_SERVICE_NOT_FOUND`.

Technical references: `backend/src/lab-catalog/clinic/clinic-catalog.controller.ts` · `ClinicCatalogService` · `web-app/src/features/lab-catalog/components/ClinicServicesTab.tsx`

40.3.9 Get a price quote

Status: [Implemented]

What it does. For a lab service, chosen options, a rush flag and a date, the quote returns the lab cost per unit (base price + option extras + rush surcharge) and the patient price per unit, and says where each came from.

How the user does it. The user does not ask for it directly. Step 3 of the New lab order sheet requests one quote per line and shows "Prices" and a "Total".

Behind the scenes — in simple terms. The patient price follows the same precedence billing uses (doctor at location, then location, then the treatment's default), so the order line and the charge later billed for it agree.

Behind the scenes — technically. `GET /lab-catalog/price-quote` (`@AnyMember`, feature-gated) → `LabPriceResolver.quote`: loads the service, requires an active link and an active service, validates the options, picks a lab price (`pickLabPrice`: negotiated window active that day, else list window), computes `labUnitCost`, then resolves the patient price through the active service mapping or a given treatment (`pickPatientPrice`). Read only. The same function runs again at submit inside the order transaction.

Rules and validations.

- Without a price window on the day: 422 `LAB_PRICE_MISSING` ("The lab has no price for … on …").
- Without a mapped treatment the patient price is null; at submit this blocks the line.
- The quote endpoint defaults to the platform day (Asia/Karachi) when no date is sent; submit prices on the location's day.

Who can do it. Every clinic member, feature `dental.lab_orders`.

Data read and written. Reads only.

Effects on other modules. The numbers become the order's frozen price snapshot at submit.

Error cases. 404 `LAB_SERVICE_NOT_FOUND`; 422 `LAB_NOT_LINKED`, `LAB_SERVICE_INACTIVE`, `LAB_PRICE_MISSING`, `LAB_SERVICE_OPTION_NOT_FOUND`.

Technical references: `backend/src/lab-catalog/pricing/lab-price-resolver.service.ts` · `LabPriceResolver.quote` · `web-app/src/features/lab-orders/components/LabOrderSheet.tsx`

40.3.10 Map a lab service to a clinic treatment

Status: [Implemented]

What it does. A **mapping** says "when we order this lab service, we bill the patient for this treatment from our own catalog". The treatment's price is what the patient pays; the lab's price is the clinic's cost. One treatment can have several lab services, one of them the **default**.

How the user does it.

1. In **Lab price list**, press the bill button on a service row (tooltip "Bill … as one of your treatments"), or open the **Treatment mapping** tab.
2. In "Map to a treatment" choose the "Lab service" and "Your treatment", and tick the default option if wanted.
3. Make another lab the default with the star button; remove a mapping with the unlink button and a confirmation.

Behind the scenes — in simple terms. If a mapping was removed before, adding the same pair brings the old row back instead of creating a duplicate. The first mapping of a treatment becomes its default automatically.

Behind the scenes — technically. `GET /lab-catalog/treatments`, `GET/POST /lab-catalog/mappings`, `PATCH /lab-catalog/mappings/:id` (`lab.catalog.manage`) over `ServiceMappingsService`. Writes lock the treatment row so "one default per treatment" cannot race; unique indexes enforce one row per organization, treatment and lab service and one active mapping per lab service.

Rules and validations.

- The treatment and the lab service must both be active (`CLINIC_TREATMENT_INACTIVE`, `LAB_SERVICE_INACTIVE`).
- An active lab service maps to only one treatment (`LAB_SERVICE_ALREADY_MAPPED`); the same pair twice gives `SERVICE_MAPPING_EXISTS`.

Who can do it. `lab.catalog.manage`; Admin by default.

Data read and written. `service_mappings`, `audit_logs`; reads `services` (the billing catalog), `lab_services`.

Effects on other modules. The mapping chooses the treatment, and so the patient price, that a lab order line carries into lab billing (Chapter 42) and commissions (Chapter 35).

Error cases. 409 `SERVICE_MAPPING_EXISTS`, `LAB_SERVICE_ALREADY_MAPPED`; 422 `CLINIC_TREATMENT_INACTIVE`, `LAB_SERVICE_INACTIVE`.

Technical references: `backend/src/lab-catalog/mappings/service-mappings.service.ts` · `ServiceMappingsService` · `web-app/src/features/lab-catalog/components/MappingsTab.tsx`

40.3.11 Save and reuse order templates

Status: [Implemented]

What it does. A template stores the lines of a lab order (services, options, quantity, rush, answers that do not depend on the patient) and notes. A template is private to its owner or shared with the whole clinic. Templates can be archived.

How the user does it.

1. Open **Order templates** and press the add button.
2. Name the template, add one or more services (all from one lab), set options, quantity and rush, and add notes.
3. Switch on "Share with the whole clinic" if you hold the right to share.
4. Edit later with the edit button, or archive it by switching it off.

Behind the scenes — in simple terms. Nobody sees another user's private templates. The use counter rises when an order started from the template is submitted, so the list shows the most used first.

Behind the scenes — technically. `GET/POST /lab-catalog/order-templates`, `GET/PATCH …/:id`, `POST …/:id/use` (`lab.orders.create`) over `OrderTemplatesService`. The body is validated and stored in `order_templates.template_json` (at most 20 lines). Create and update run in a transaction with an audit row. `markUsed` is a raw `UPDATE` of `use_count` and `last_used_at` (not an edit, so no version bump and no audit); `OrderSubmitService` calls it inside the submit transaction. The explicit `POST …/:id/use` endpoint exists, but no web screen calls it.

Rules and validations.

- Every service must be active, of one lab on the clinic's list, and of one specialty; options must belong to their service; answers must fit the service's form (partial answers allowed).

- Sharing or editing a shared template needs `lab.templates.share` (`ORDER_TEMPLATE_FORBIDDEN`).

Who can do it. `lab.orders.create` to use and keep private templates (Doctor, Admin); `lab.templates.share` to share (Admin only by default).

Data read and written. `order_templates`, `audit_logs`; reads `lab_services`, `lab_service_options`, `lab_clinic_links`, form definitions.

Effects on other modules. A draft started from a template records `order_template_id`.

Error cases. 403 `ORDER_TEMPLATE_FORBIDDEN`; 422 `ORDER_TEMPLATE_INVALID`.

Technical references: `backend/src/lab-catalog/templates/order-templates.service.ts` · `OrderTemplatesService` · `web-app/src/features/lab-catalog/pages/OrderTemplatesPage.tsx`

40.3.12 Create a lab order draft

Status: [Implemented]

What it does. Starts a case for a patient with a chosen lab and one or more items. A draft is a working copy: no order number, no frozen prices, free to edit or abandon.

How the user does it.

1. Press **New lab order** on **Lab orders**.
2. Step 1: choose the patient, lab, ordering doctor, priority and "Needed back by"; add instructions.
3. Step 2: add items. For each choose the service, options, quantity, Rush and fill its form (shade, material, teeth, files).
4. Press **Save draft** (or continue to step 3 and submit). A draft can be reopened from the order page with **Edit draft**.

Behind the scenes — in simple terms. The draft already remembers which version of each service's form and workflow it will use, so changes made by the super admin later do not alter it. Saving twice with the same key returns the same draft instead of making a second one.

Behind the scenes — technically. `POST /lab-orders` (`lab.orders.create`, optional `Idempotency-Key`) → `OrderDraftsService.create`: one transaction that checks the patient, the active link, the doctor and the optional appointment and treatment, loads the services, requires one specialty, inserts `lab_orders` (UUID version 7 key, status `draft`, pinned order workflow version) and the items with their pinned form and item workflow versions (`pin`), saves a form-response row per item, and writes a `status_history` row (`create`) and an `audit_logs` row. A racing duplicate key is caught on the unique constraint `uq_lab_orders_org_idempotency` and returns the winner. `PATCH /lab-orders/:id` updates a draft under a row lock with a version check; it can rewrite the lines (kept items are re-pinned only if the service changed; removed lines are deleted, which is allowed only for drafts).

Rules and validations.

- 1 to 20 items per order (`MAX_ORDER_ITEMS`); all items must be services of the chosen lab and of one specialty (`LAB_ORDER_LAB_MISMATCH`, `LAB_ORDER_INVALID`).
- The lab link must be active (`LAB_NOT_LINKED`: "Activate this lab in your lab list before ordering from it").
- The doctor defaults to the caller when the caller can conduct consultations.
- Drafts are not validated beyond coherence; form answers are checked at submit.
- Only drafts can be edited (`LAB_ORDER_NOT_DRAFT`).

Who can do it. `lab.orders.create`, feature `dental.lab_orders`. Default: Doctor, Admin.

Data read and written. Writes `lab_orders`, `lab_order_items`, `item_form_responses`, `status_history`, `audit_logs`; reads `patients`, `appointments`, `treatments`, `lab_clinic_links`, `lab_services`, workflow and form versions.

Effects on other modules. None until submit.

Error cases. 404 `LAB_ORDER_PATIENT_NOT_FOUND`, `LAB_ORDER_DOCTOR_NOT_FOUND`; 409 `LAB_ORDER_NOT_DRAFT`; 422 `LAB_ORDER_INVALID`, `LAB_ORDER_WORKFLOW_MISSING`, `LAB_SERVICE_INACTIVE`.

Technical references: `backend/src/lab-orders/services/order-drafts.service.ts` · `OrderDraftsService.create`, `update` · `web-app/src/features/lab-orders/components/LabOrderSheet.tsx`

40.3.13 Start from a template or repeat a past order

Status: [Implemented]

What it does. Two shortcuts fill a new draft. **From a template** copies the template's lines. **Repeat last order** (and the **Repeat** button on an order) copies the lines and answers of a past order of the same patient onto the current form versions.

How the user does it.

1. In step 1 of the sheet, after choosing the patient, open "Quick start".
2. Choose "From a template" and start it, or press **Repeat last order**.
3. Or, on a submitted order, press **Repeat**; a new draft opens.

Behind the scenes — in simple terms. Files never carry over. Teeth carry over only when the new order is for the same patient. Cancelled items are skipped.

Behind the scenes — technically. `POST /lab-orders/from-template` and `POST /lab-orders/:id/repeat` (`lab.orders.create`) → `OrderDraftsService.fromTemplate` / `repeat`, both ending in `create`. "Repeat last order" first lists the patient's latest submitted order, then repeats it; a patient without one gives the message "This patient has no past lab order" in the web app. Repeating returns a draft with the same lab, priority and notes.

Rules and validations. A template must be active and have a lab (`LAB_ORDER_INVALID`: "This template can't be used"). Private templates of other users stay invisible.

Who can do it. `lab.orders.create`. Default: Doctor, Admin.

Data read and written. As 40.3.12; reads `order_templates`, `item_form_responses`.

Effects on other modules. Submitting a draft that came from a template raises its use count.

Error cases. 404 `LAB_ORDER_NOT_FOUND`; 422 `LAB_ORDER_INVALID`.

Technical references: `backend/src/lab-orders/services/order-drafts.service.ts` · `OrderDraftsService.fromTemplate`, `repeat` · `web-app/src/features/lab-orders/components/LabOrderSheet.tsx`

40.3.14 Submit an order to the lab

Status: [Implemented]

What it does. Sends the case. The order gets its number (for example LAB-0001), the prices of every line are frozen, the form answers are checked, the lab is told, and a magic link is sent to the lab if it has a contact address.

How the user does it.

1. In the sheet press **Submit to lab**, or on a draft's page press **Submit to lab**.

2. If lines need attention, the message lists them (for example a per-tooth service without teeth, or a service not mapped to a treatment); fix and submit again.
3. On success the toast says "Order … sent to …".

Behind the scenes — in simple terms. Everything happens in one step: if anything fails, nothing happened, not even the order number. Pressing the button twice is safe: the second call returns the first result.

Behind the scenes — technically. `POST /lab-orders/:id/submit` with a required `Idempotency-Key` header and the draft's `version` → `OrderSubmitService.submit`, one transaction: claim the idempotency key (`IdempotencyService`, scope `lab_order.submit`, replay returns the stored result with `replayed: true`) → engine `transitionIn` of the order with action `submit` (state, role and version checks) → require an ordering doctor → per line: validate answers against the pinned form version (`validateFormData`), derive the quantity from the teeth for per-tooth services, call `LabPriceResolver.quote` on the location's clinic day and write snapshot columns (`service_name_snap`, `options_snap`, lab and patient unit and line prices in paisa, `priced_on`) → order number from the organization's number sequence (`NumberSequenceService`, prefix `LAB`) → `lab_expected_on` = the requested date or submit day plus the longest default turnaround → per line the initial item state (`pending_acceptance`), form answers, teeth rows and promoted detail columns (`item_teeth`, `dental_item_details`), a `status_history` row → revision 1 in `order_revisions` with a snapshot that carries lab cost but not the patient price → template use count → `audit_logs` row → outbox row `order.submitted` for the lab's members → lifecycle hooks (the magic-link module issues a link when the lab has a WhatsApp number, phone or email). After submit, a guard on `LabOrderItem` saves (`snapshot-freeze.ts`) refuses to change frozen columns except through a revision or an approved charge (`LAB_ORDER_SNAPSHOT_FROZEN`).

Rules and validations.

- `Idempotency-Key` is mandatory (`IDEMPOTENCY_KEY_REQUIRED`, 400).
- The status must allow `submit` (draft only) for the caller's role: in the seeded workflow, roles `doctor` or `clinic_admin`, which the code derives from permissions (`consultation.conduct` gives doctor, any `admin.*` gives clinic admin; the owner has every role).
- An ordering doctor is required; at least one line.
- Every line must price on the submit day: a lab price window and a mapped treatment must exist.

Who can do it. `lab.orders.create` plus the seeded workflow role (doctor or clinic admin). Default: Doctor, Admin.

Data read and written. Writes `lab_orders`, `lab_order_items`, `item_form_responses`, `item_teeth`, `dental_item_details`, `order_revisions`, `order_templates`, `idempotency_keys`, `status_history`, `audit_logs`, `magic_links`, `notification_outbox`; reads price, mapping and form tables, `number_sequences`.

Effects on other modules. Lab members are notified in the app. Billing and payables read the frozen prices later.

Error cases. 400 `IDEMPOTENCY_KEY_REQUIRED`; 403 `TRANSITION_FORBIDDEN`; 409 `STALE_VERSION`; 422 `INVALID_TRANSITION`, `LAB_ORDER_INVALID` (with a list of line issues), `LAB_PRICE_MISSING`.

Technical references: `backend/src/lab-orders/services/order-submit.service.ts` · `OrderSubmitService.submit` · `web-app/src/features/lab-orders/hooks/useLabOrderForm.ts`

40.3.15 List and search lab orders

Status: [Implemented]

What it does. Lists the clinic's lab orders with item count, lab cost, patient price, dates and status, newest first, as a table or cards.

How the user does it.

1. Open **Lab orders**.
2. Search by order number, patient name or patient number; filter by status, lab, doctor and dates.
3. Click a row to open the order; "Clear all filters" appears when filters are active.

Behind the scenes — in simple terms. What a user sees depends on their permission scope: a doctor sees orders they ordered or created, the front desk sees their location.

Behind the scenes — technically. `GET /lab-orders` (`lab.orders.view`) → `LabOrdersService.list` with `scopeOrders` : scope `own` filters on `created_by` or `doctor_id` , `location` on the active location, `org` is unfiltered, no grant gives no rows. Sub-selects sum the lab line cost and the patient line price of items that are not cancelled; both are null for drafts.

Rules and validations. Paged on the server. An order outside the caller's scope answers 404, not 403.

Who can do it. `lab.orders.view` , scope own, location or org. Default: Doctor (own), Receptionist and Assistant (location), Admin (org).

Data read and written. Reads `lab_orders` , `lab_order_items` , `patients` , `labs` , `employees` .

Effects on other modules. None.

Error cases. 403 `NO_PERMISSION` ; 400 on an invalid filter.

Technical references: `backend/src/lab-orders/services/lab-orders.service.ts` · `LabOrdersService.list` , `scopeOrders` · `web-app/src/features/lab-orders/pages/LabOrdersPage.tsx`

40.3.16 Open an order and print the work order

Status: [Implemented]

What it does. The order page shows the status, the doctor, the due-back date, the revision number, warnings, every item with its teeth and details and both prices, and the sections listed in 40.2. **Work order** prints a sheet with a QR code and the instructions, without prices, to travel with the case.

How the user does it.

1. Open an order from the list. A back arrow returns to the list.
2. Press **Work order** to print it (not available for drafts).

Behind the scenes — in simple terms. The QR on the printed sheet points at one address that works for both sides: a lab member who scans it signed in lands on the lab portal order, a clinic user lands on the clinic order, and anyone else gets "not found".

Behind the scenes — technically. `GET /lab-orders/:id` and `GET /lab-orders/:id/work-order` (`lab.orders.view`). The detail includes `availableActions` for the caller from the pinned workflow (minus actions with their own endpoint) and warnings such as `LAB_DUE_AFTER_NEXT_APPOINTMENT` . The work order carries `qrValue` = the front-end base address (environment variable `FRONTEND_URL`) plus `/o/:orderId` and a patient reference that follows the link's patient data level. `GET /order-links/:orderId` (`@ContextFree` , signed in) resolves the QR for the web route `/o/$orderId` .

Rules and validations. A work order needs an order number (submit first).

Who can do it. `lab.orders.view` . Default: Doctor, Receptionist, Assistant, Admin.

Data read and written. Reads orders, items, parts, `lab_clinic_links` , `organizations` .

Effects on other modules. None.

Error cases. 404 `LAB_ORDER_NOT_FOUND` ; 422 `LAB_ORDER_INVALID` for a draft work order.

Technical references: `backend/src/lab-orders/controllers/lab-orders.controller.ts` · `LabOrdersService.workOrder`, `OrderLinkController.resolve` · `web-app/src/features/lab-orders/pages/LabOrderDetailPage.tsx`

40.3.17 Cancel, hold, resume or reopen an order

Status: [Implemented]

What it does. The buttons next to Work order come from the workflow: **Cancel order** (with a reason, from draft, submitted or accepted), **Put on hold** (from in progress, with a reason), **Resume** and **Reopen as draft** (after the lab rejected the order).

How the user does it.

1. Press the action button.
2. Type the reason in the dialog (the reason is kept in the history) and confirm.

Behind the scenes — in simple terms. Only actions that the pinned workflow allows from the current state, for the caller's role, are offered. In the seeded workflow, cancelling the order does not itself cancel its items, and an order that is already in progress cannot be cancelled as a whole; work in progress is stopped item by item (see 40.3.26).

Behind the scenes — technically. `POST /lab-orders/:id/transitions` (`lab.orders.create`, body `action`, `reason`, `version`) → `LabOrdersService.transition` → engine `transition`. The actions `submit`, `reply` and `request_clarification` have their own endpoints and are refused here. `resume` runs the `recompute_order_status` effect. Closing an order (cancelled, completed, rejected by lab) revokes its magic links in the same transaction through a lifecycle hook.

Rules and validations. Reason required for cancel, hold, reject and similar (`TRANSITION_REASON_REQUIRED`). A state/action pair that the workflow lacks gives `INVALID_TRANSITION`.

Who can do it. `lab.orders.create` and the seeded roles doctor or clinic admin. Default: Doctor, Admin.

Data read and written. `lab_orders`, `status_history`, `audit_logs`, `magic_links`.

Effects on other modules. Closed orders stop being open to the lab through links.

Error cases. 422 `INVALID_TRANSITION`, `TRANSITION_REASON_REQUIRED`; 403 `TRANSITION_FORBIDDEN`; 409 `STALE_VERSION`.

Technical references: `backend/src/engine/runtime/workflow-engine.service.ts` · `WorkflowEngine.transition` · `web-app/src/features/lab-orders/pages/LabOrderDetailPage.tsx`

40.3.18 Revise a submitted order

Status: [Implemented]

What it does. Changes the instructions of an order that was already sent: options, rush, quantity, answers and notes. The lab receives the new revision and must acknowledge it. Prices stay as agreed at submit unless the work itself changed.

How the user does it.

1. On an order in submitted, clarification needed, accepted, in progress or on hold press **Revise**.
2. In "Revise ..." edit the lines and fill "What changed".
3. Press the send button; the message says "Revision n sent to the lab". The Revisions section shows "Not read by lab yet" until the lab acknowledges it.

Behind the scenes — in simple terms. A revision never overwrites: it adds a numbered snapshot. Changed lines are re-checked against their pinned form and re-priced on the original price day, so a price change since submit does not leak in. Approved extra charges stay on top.

Behind the scenes — technically. `POST /lab-orders/:id/revisions` (`lab.orders.create`) → `LabOrdersService.revise`: one transaction under a row lock; validates and re-prices each changed line, saves it with `snapshotChange: 'revision'`, rewrites answers, teeth and promoted details, re-applies approved item charges, increments `current_revision_no`, inserts `order_revisions` (snapshot, reason), writes audit. If the order was in clarification needed, open clarification messages are marked resolved and the `reply` transition runs.

Rules and validations. Allowed statuses are submitted, clarification needed, accepted, in progress and on hold (`LAB_ORDER_REVISION_NOT_ALLOWED` otherwise); the reason is required; a cancelled line cannot be revised.

Who can do it. `lab.orders.create`. Default: Doctor, Admin.

Data read and written. `lab_order_items`, `item_form_responses`, `item_teeth`, `dental_item_details`, `order_revisions`, `order_messages`, `audit_logs`.

Effects on other modules. The lab's open work now follows the new instructions; the vendor bill draft follows the items' frozen costs.

Error cases. 409 `LAB_ORDER_REVISION_NOT_ALLOWED`, `STALE_VERSION`; 422 `LAB_ORDER_INVALID`.

Technical references: `backend/src/lab-orders/services/lab-orders.service.ts` · `LabOrdersService.revise` · `web-app/src/features/lab-orders/components/RevisionSheet.tsx`

40.3.19 Talk to the lab: messages and clarifications

Status: [Implemented]

What it does. A message thread on the order (optionally on one item). The lab can ask a clarification question, which moves the order to "Clarification needed"; the clinic answers with a reply, which moves it back to "Submitted".

How the user does it.

1. In **Messages** type a note ("Message the lab"), or when the lab asked a question type the answer ("Reply to the lab").
2. Send. The thread shows each message with its side and time.

Behind the scenes — in simple terms. The clinic cannot ask for clarification; only the lab does. A reply is accepted only while a question is open.

Behind the scenes — technically. `POST /lab-orders/:id/messages` (`lab.orders.create`) → `LabOrdersService.clinicMessage`: locks the order, refuses closed orders, resolves open clarification messages for a reply, inserts `order_messages` (author side clinic), runs the `reply` transition. The lab's counterpart is `labMessage` and the lab actions of Chapter 41.

Rules and validations. `LAB_ORDER_MESSAGE_INVALID` for a clinic clarification request or a reply without a question. Closed orders (draft, cancelled, completed, rejected by lab) take no messages.

Who can do it. `lab.orders.create`. Default: Doctor, Admin. Reading: `lab.orders.view`.

Data read and written. `order_messages`, `status_history`, `audit_logs`.

Effects on other modules. The doctor receives `order.clarification_requested` when the lab asks (40.3.30).

Error cases. 422 `LAB_ORDER_MESSAGE_INVALID`, `LAB_ORDER_INVALID`.

Technical references: `backend/src/lab-orders/services/lab-orders.service.ts` · `LabOrdersService.clinicMessage` · `web-app/src/features/lab-orders/pages/LabOrderDetailPage.tsx`

40.3.20 Decide the lab's extra charge requests

Status: [Implemented]

What it does. After submit, the lab can ask for an extra charge (rush, material, shipping, other) with an amount and a reason. The clinic approves or rejects each request. An approved charge on an item is added to that item's lab cost.

How the user does it.

1. If requests exist, the **Extra charges** section lists them with the kind, amount and reason.
2. Press the approve or reject icon button (tooltips "Approve charge", "Reject charge").

Behind the scenes — in simple terms. This is, together with a revision, the only way a frozen price can move. An order-level charge (for example shipping) does not touch an item; it stays a separate row and later lands on the vendor bill.

Behind the scenes — technically. `POST /lab-orders/:id/charge-requests/:chargeId/decision` (`lab.orders.create` , body decision approve or reject, note, version) → `LabOrdersService.decideCharge` : row lock, version check, only pending requests, sets status, `decided_by` , `decided_at` , and for an approved item charge adds the amount to `lab_line_cost_paisa` with `snapshotChange: 'charge'` . Audit row written. The request itself is created by the lab; see Chapter 41 for why that has no screen yet.

Rules and validations. `LAB_ORDER_CHARGE_DECIDED` if already decided; the amount is greater than zero and a reason is required (set when the lab asks).

Who can do it. `lab.orders.create` . Default: Doctor, Admin.

Data read and written. `order_charge_requests` , `lab_order_items` , `audit_logs` .

Effects on other modules. The patient price is not changed; only the lab cost. Lab payables (Chapter 42) read the new cost.

Error cases. 404 `LAB_ORDER_CHARGE_NOT_FOUND` ; 409 `LAB_ORDER_CHARGE_DECIDED` , `STALE_VERSION` ; 403 `LAB_ORDER_FORBIDDEN` .

Technical references: `backend/src/lab-orders/services/lab-orders.service.ts` · `LabOrdersService.decideCharge` · `web-app/src/features/lab-orders/pages/LabOrderDetailPage.tsx`

40.3.21 Attach and download files

Status: [Implemented]

What it does. Staff attach files (scans, photos, documents) to an order or to one item and choose who may see them: internal (clinic only), shared with the lab, or shared with the patient. Files are downloaded through short-lived links.

How the user does it.

1. In **Files** choose "Who can see it" and upload.
2. Press the download button of a file (tooltip "Download …"). Files can also be added in the form of an item that has a file field.

Behind the scenes — in simple terms. The file is stored privately; the database keeps only its name, type, size and a checksum. A download link works for ten minutes. The lab can only ever see files marked as shared with the lab.

Behind the scenes — technically. `POST /lab-orders/:id/attachments` (multipart, `lab.orders.create`; default visibility `lab_shared`) and `GET /lab-orders/:id/attachments/:attachmentId/url` (`lab.orders.view`) over `AttachmentsService` and an `ATTACHMENT_STORAGE` binding (the Cloudinary implementation uploads as an authenticated, non-public object and signs the download URL with a 600-second lifetime; the credentials are environment variables of the file-upload service). One row in `attachments` per file with a SHA-256 checksum, and an audit row.

Rules and validations. At most 50 MB per file (`ATTACHMENT_INVALID`); a file must be chosen; the item, if given, must belong to the order. Download of an unknown or (for the lab) not lab-shared file answers `ATTACHMENT_NOT_FOUND`. The `patient_shared` visibility is stored, but no patient-facing screen reads it.

Who can do it. Upload: `lab.orders.create`; download: `lab.orders.view`.

Data read and written. `attachments`, `audit_logs`, private file storage.

Effects on other modules. A file field of an order form refers to an attachment id and is checked at submit.

Error cases. 422 `ATTACHMENT_INVALID`; 404 `ATTACHMENT_NOT_FOUND`, `LAB_ORDER_ITEM_NOT_FOUND`.

Technical references: `backend/src/lab-orders/attachments/attachments.service.ts` · `AttachmentsService`, `CloudinaryAttachmentStorage` · `web-app/src/features/lab-orders/components/OrderItemEditor.tsx`

40.3.22 Send, copy and revoke magic links

Status: [Implemented]

What it does. A **magic link** lets one lab contact open one order and answer it without logging in. The clinic can send it to the lab by message, copy it to paste into a chat app, and revoke it. A link is created automatically at submit when the lab has a WhatsApp number, phone or email.

How the user does it.

1. On a submitted order open **Lab access**.
2. Press **Send link to lab** (the message goes to the lab's contact) or **Copy link** ("Link copied — paste it into WhatsApp").
3. The list shows each link's state (Active, Expired, Used up, Revoked), how it was delivered and when it was last used.
4. To stop a link press its revoke button (tooltip "Revoke link") and confirm "Revoke this link?".

Behind the scenes — in simple terms. The link's secret part is never stored. The database keeps a one-way hash and a random value from which the server can rebuild the secret only at the moment of sending, so a database leak gives nobody a working link. A link dies by itself after 24 hours (configurable), when it is revoked, when the order closes or when the lab is suspended.

Behind the scenes — technically. `GET/POST /lab-orders/:id/magic-links`, `POST …/:linkId/revoke` in `ClinicLabAccessController` (read: `lab.orders.view`; issue and revoke: `lab.orders.create`) over `MagicLinksService`. Issue: random 32-byte nonce; token = HMAC-SHA256 of the nonce with the signing secret (environment variable `MAGIC_LINK_SECRET`, with a fallback to the access-token secret); table stores `token_hash` (SHA-256, unique) and the nonce; expiry = now + hours from the request (1 to 168) or `LAB_MAGIC_LINK_TTL_HOURS` (default 24); `max_uses` null. For delivery `notification` the outbox row `order.magic_link` carries a placeholder; a payload finalizer swaps in the real URL just before sending and drops

the message if the link was revoked or expired meanwhile. For `copy` the URL is returned once in the response. Revoke locks the link, stamps `revoked_at`, `revoked_reason` `revoked_by` and writes a `void` audit row. Purposes: `lab_view_order` (read only) and `lab_update_order`; the default is update.

Rules and validations.

- The order must be submitted (`LAB_ORDER_INVALID`: "Submit the order before sending the lab a link").
- Sending needs a lab contact; otherwise "copy the link instead".
- `receive_shipment` links exist as a purpose but nothing creates them (clinics receive by the shipment's own QR token).

Who can do it. Send, copy, revoke: `lab.orders.create`. List: `lab.orders.view`. Default: Doctor, Admin.

Data read and written. `magic_links`, `notification_outbox`, `audit_logs`; reads `labs`, `lab_orders`.

Effects on other modules. The lab can accept, reject or ask a question through the link (Chapter 41).

Error cases. 422 `LAB_ORDER_INVALID`; 404 `MAGIC_LINK_NOT_FOUND`.

Technical references: `backend/src/lab-access/magic-links/magic-links.service.ts` · `MagicLinksService.issue`, `revoke` · `web-app/src/features/lab-orders/components/LabAccessPanel.tsx`

40.3.23 Update on behalf of a lab that does not answer

Status: [Implemented]

What it does. When a lab does not use the portal or the link, a **lab coordinator** at the clinic can record the lab's answer: accept (with the expected date), reject, record a clarification question, or change the expected date. The history marks such changes as "entered by clinic for the lab".

How the user does it.

1. On a submitted order, find "Update for the lab" in **Lab access** (visible only to coordinators).
2. Choose the action offered and fill the dialog (for Accept, the "Ready on" date and a note).

Behind the scenes — in simple terms. The change is checked as if the lab had made it, but it is stamped as a proxy action with the lab's id, so nobody can mistake it for the lab's own click.

Behind the scenes — technically. `GET /lab-orders/:id/proxy/actions`, `POST …/proxy/accept`, `…/reject`, `…/clarification`, `…/expected-date` (`@Can('lab.orders.create','lab.links.manage')`) → `LabOrderActionsService` with a `LabSideCall` whose actor has side clinic, `via = proxy` and `onBehalfOfLabId`. The engine's `authorize` accepts the proxy only if the caller holds both permissions and the lab id matches the order's lab; it then checks the transition as side lab without lab roles. History rows carry `via` and `on_behalf_of_lab_id`. Accept, reject, clarify and date change are described in 40.3.30 and Chapter 41.

Rules and validations. As for the lab's own actions: accept needs an expected date not in the past in the location's day and a submitted order; reject needs a reason; a date change needs a reason and an accepted, in-progress, partly ready or on-hold order.

Who can do it. `lab.orders.create` and `lab.links.manage` together, which only Admin and the owner hold by default.

Data read and written. As the lab's own actions: `lab_orders`, `lab_order_items`, `order_messages`, `status_history`, `audit_logs`, `notification_outbox`.

Effects on other modules. Accepting creates the item stages and the vendor bill draft (40.3.24, Chapter 42).

Error cases. 403 `TRANSITION_FORBIDDEN` ("You cannot update on behalf of this case's lab"); 422 `LAB_ACTION_NOT_ALLOWED`, `LAB_EXPECTED_DATE_INVALID`.

Technical references: `backend/src/lab-access/controllers/clinic-lab-access.controller.ts` · `ClinicLabAccessController`, `WorkflowEngine.authorize` · `web-app/src/features/lab-orders/components/LabAccessPanel.tsx`

40.3.24 Follow production and try-in rounds

Status: [Implemented]

What it does. Once the lab accepts, the order page shows each item moving through the lab: accepted, in production, at clinic for try-in, quality check, ready, dispatched, received. Items with a **stage template** (for example metal try-in, bisque try-in, final) show each try-in round as a stage.

How the user does it. Open the order and read **Production**. For a try-in stage that has arrived, press **Record try-in** after the patient visit (an appointment of the patient can be linked), then **Approve** the stage or press **Needs correction** and type what the lab must fix.

Behind the scenes — in simple terms. When the lab accepts, the Platform copies the item's stage template into real stage rows (names are copied, so editing the template later changes nothing). Only one round per item is active at a time. A correction closes the round and opens the same stage again as the next round; rounds are never overwritten.

Behind the scenes — technically. `GET /lab-orders/:id/production`, `POST …/stages/:stageId/tryin`, `POST …/stages/:stageId/decision` (`lab.orders.view` to read, `lab.orders.create` to act) in `ClinicProductionController` over `ClinicProductionService`, one transaction each. `ProductionWorkflowService` listens to every item transition: at accepted it runs `initStages` (inserts `order_item_stages` round 1 for each template step, initial state from the pinned stage workflow, expected dates from default lab days, last step `is_final`); at in production it starts the next unstarted round (`ensureActiveStage`); at delivered it starts the warranty; at remake required it opens the remake request. The stage adapter keeps `is_active` true while a stage's state category is active or waiting. `record_tryin` moves received to try-in done; the decision `approve` approves the stage and, for a try-in, moves the item back to production (`resume`) so the next round starts, while for the final stage it approves the item; `correction` needs notes, runs `request_correction`, closes the round, adds `round_no + 1` and resumes the item. Notifications `stage.approved` and `stage.correction` go to the lab.

Rules and validations.

- A final stage cannot be "corrected": final work is approved or rejected (`PRODUCTION_NOT_ALLOWED`).
- Correction notes are required (`PRODUCTION_INVALID`: "Say what the lab must correct").
- A try-in appointment must be one of the patient's (`PRODUCTION_INVALID`).
- A stage that needs no patient visit goes straight from received to the decision.

Who can do it. Read: `lab.orders.view`. Act: `lab.orders.create` plus the workflow role (seeded: doctor or clinic admin). Default: Doctor, Admin.

Data read and written. `order_item_stages`, `lab_order_items`, `lab_orders`, `status_history`, `audit_logs`, `notification_outbox`; reads `stage_template_steps`, `appointments`.

Effects on other modules. Try-in appointments are linked by id; no appointment is created or reminded automatically (the event `tryin.appointment_due` is not emitted; see 40.3.30).

Error cases. 404 `PRODUCTION_STAGE_NOT_FOUND`; 409 `PRODUCTION_NOT_ALLOWED`, `STALE_VERSION`; 422 `PRODUCTION_INVALID`, `INVALID_TRANSITION`.

Technical references: `backend/src/lab-production/services/clinic-production.service.ts` · `ClinicProductionService.recordTryin` , `decideStage` · `web-app/src/features/lab-orders/components/production/ProductionPanel.tsx`

40.3.25 Approve, reject or deliver finished work

Status: [Implemented]

What it does. When the finished item has been received, the doctor either approves it (then marks it delivered once fitted on the patient) or rejects it and asks for a remake.

How the user does it.

1. On a received item press **Approve work**, or **Reject — remake** (choose the reason and say what is wrong).
2. After approval press **Fitted — mark delivered**. The toast says "Delivered — warranty started".

Behind the scenes — in simple terms. Delivery starts a warranty of 12 months by default (terms "Standard lab warranty"). When all items are approved or delivered, the order becomes completed, and the patient and doctor are told.

Behind the scenes — technically. `POST /lab-orders/:id/items/:itemId/approve` , `…/reject` , `…/deliver` (`lab.orders.create`) → `ClinicProductionService.approveWork` , `rejectWork` , `deliver` , each in a transaction, each through the engine. Approve goes through the final stage's approval when stages exist, otherwise approves the item directly. Reject allows only received items, closes the final round with outcome rejected, moves the item to remake required (`reject_work` , reason required) and opens a remake request with the warranty check. Deliver moves approved to delivered and `startWarranty` inserts `item_warranties` (starts on the location's day, ends the day before the anniversary). After each item change the order is re-derived; `order.completed` is queued when the order reaches completed.

Rules and validations. Only received work can be rejected (`PRODUCTION_NOT_ALLOWED`); reason required; version checks (409). Roles per the seeded workflow: doctor or clinic admin.

Who can do it. `lab.orders.create` . Default: Doctor, Admin.

Data read and written. `lab_order_items` , `order_item_stages` , `item_warranties` , `remake_requests` , `lab_orders` , `status_history` , `audit_logs` , `notification_outbox` .

Effects on other modules. Delivered work is what lab billing and commissions later settle on (Chapters 42 and 35).

Error cases. 409 `PRODUCTION_NOT_ALLOWED` , `STALE_VERSION` ; 422 `INVALID_TRANSITION` , `TRANSITION_REASON_REQUIRED` .

Technical references: `backend/src/lab-production/services/clinic-production.service.ts` · `approveWork` , `rejectWork` , `deliver` · `web-app/src/features/lab-orders/components/production/ProductionPanel.tsx`

40.3.26 Cancel one item

Status: [Partial]

What it does. Cancels a single item of an order while it is accepted or in production, with a reason.

How the user does it. No screen offers it. The order's own **Cancel order** button works at order level only, and only from draft, submitted or accepted.

Behind the scenes — in simple terms. The capability exists in the API, so work in progress can be stopped item by item, but the web app has no button for it.

Behind the scenes — technically. `POST /lab-orders/:id/items/:itemId/cancel` (`lab.orders.create`) → `ClinicProductionService.cancelItem` → item transition `cancel` (seeded from accepted or in production, reason required). `productionApi` in the web app has no matching function.

Rules and validations. Reason required; the workflow decides from which states.

Who can do it. `lab.orders.create` plus the seeded role (doctor or clinic admin).

Data read and written. `lab_order_items`, `lab_orders` (re-derived), `status_history`, `audit_logs`.

Effects on other modules. A cancelled item no longer counts for the order's progress.

Error cases. 422 `INVALID_TRANSITION`, `TRANSITION_REASON_REQUIRED`.

Technical references: `backend/src/lab-production/controllers/clinic-production.controller.ts` · `ClinicProductionService.cancelItem` · no web-app caller

40.3.27 Ask for a remake and decide who pays

Status: [Implemented]

What it does. A **remake request** asks the lab to make an item again. It opens when received work is rejected, or on request for delivered work (for example a fracture months later). If the item's warranty is active, the request is marked as a warranty claim automatically. The lab may dispute it. The clinic then agrees or closes it and says who is responsible, which decides the prices of the new item.

How the user does it.

1. After delivery press **Request remake**, choose the reason (shade, fit, fracture, design, damaged, other) and write what is wrong.
2. In **Remake requests** press the decision button. In "Remake decision" choose "Agree — make it again" or "Close — no remake", pick "Who is responsible" (lab, clinic, patient) and add a note.

Behind the scenes — in simple terms. If the lab is at fault, the new item costs nobody anything. If the clinic is at fault, the clinic pays the lab and the patient pays nothing. If the patient is at fault, both pay as originally priced. The new item sits in the same order with a copy of the original's answers and teeth, starts again from "Waiting for lab", and the lab accepts it like any new work. A completed order reopens.

Behind the scenes — technically. `POST /lab-orders/:id/items/:itemId/remake-requests` and `POST /lab-orders/:id/remake-requests/:remakeId/decision` (`lab.orders.create`). `requestRemake` accepts delivered items (or remake required items without an agreed remake) and calls `openRemakeRequest`, which is idempotent per item (one open request at a time, enforced by a partial unique index) and queues `remake.requested` to the lab. `decideRemake` runs in one transaction: lock the request, check version and that it is requested or disputed, then either closes it, or computes charges (`remakeCharges`), creates the new item through `createRemakeItem` (copy of snapshot with `remake_of_item_id`, charges as line prices, initial state), sets the request to agreed with the new item id, re-derives the order with `reopen`, queues `remake.agreed` and writes audit. Agreeing needs a responsible party other than undetermined.

Rules and validations.

- "Decide who is responsible before agreeing a remake" (`PRODUCTION_INVALID`).
- An item already remade cannot be remade again; received work must be rejected, not "requested" (`PRODUCTION_NOT_ALLOWED`).
- A decided request (agreed or closed) cannot be decided again.

Who can do it. `lab.orders.create`. Default: Doctor, Admin. The lab's dispute is in Chapter 41.

Data read and written. `remake_requests`, `lab_order_items`, `item_form_responses`, `item_teeth`, `dental_item_details`, `item_warranties`, `lab_orders`, `status_history`, `audit_logs`, `notification_outbox`.

Effects on other modules. The new item's costs feed the vendor bill (a remake line, free when the lab is at fault) and the patient lab invoice (Chapter 42).

Error cases. 404 `PRODUCTION_REMAKE_NOT_FOUND`; 409 `PRODUCTION_NOT_ALLOWED`; 422 `PRODUCTION_INVALID`.

Technical references: `backend/src/lab-production/services/clinic-production.service.ts` · `requestRemake`, `decideRemake`, `remakeCharges` · `web-app/src/features/lab-orders/components/production/RemakeSheets.tsx`

40.3.28 Receive a shipment and report a lost one

Status: [Implemented]

What it does. When the lab ships try-in or finished work, the clinic sees an incoming shipment. The front desk records each line (quantity, condition, quality check) and the work moves to "Received". A parcel that never arrives can be marked lost.

How the user does it.

1. Scan the QR code on the slip, or use the incoming list in **Production**. The page "Receive SHP-..." opens.
2. For each line set Condition (OK, Damaged, Missing), the received quantity, the incoming QC choice and notes. Untick lines that have not arrived.
3. Press **Receive**. Or on a parcel that did not come press the lost button (tooltip "Mark this shipment lost") and give a reason.

Behind the scenes — in simple terms. Lines left out stay open, so a parcel can be received in parts. "Missing" closes a line without receiving work, and the work stays owed so the lab can ship it again. Marking a parcel lost turns every open line into missing and tells both the lab and the doctor.

Behind the scenes — technically. `GET /lab-shipments`, `GET /lab-shipments/by-token/:token`, `GET /lab-shipments/:id`, `POST /lab-shipments/:id/receive`, `POST …/:id/lost` (`lab.orders.view`; there is no separate shipments permission in the catalog, so the front desk can receive). `ShipmentsService.receive` runs in one transaction under a shipment row lock: direction must be to clinic, status open; each line is validated (quantity between 1 and the shipped quantity, or 0 for missing), saved, and the stage (`receive`) and the item (`receive` from dispatched) are moved through the engine with `via = qr` when scanned; the shipment becomes partially received or received; audit row; notification `shipment.received` to the lab. `markLost` sets the lost fields, marks open lines missing, and queues `shipment.lost` to the lab and the doctor of each order. The slip's QR value is the front-end address plus `/lab-orders/receive/` and a random 16-byte hex token, unique platform-wide, stored in `shipments.qr_token`. Shipment numbers (SHP-0001 style) come from the organization's number sequence with prefix `SHP`.

Rules and validations.

- A shipment to the lab cannot be received from the clinic side.
- Received quantity 1 to shipped quantity; a missing line has quantity 0; a line already received cannot be changed.
- Workflow roles for `receive` include receptionist, doctor and clinic admin.

Who can do it. `lab.orders.view` (Doctor, Receptionist, Assistant, Admin).

Data read and written. `shipments`, `shipment_items`, `order_item_stages`, `lab_order_items`, `lab_orders`, `status_history`, `audit_logs`, `notification_outbox`.

Effects on other modules. Received work is what the doctor approves next.

Error cases. 404 `PRODUCTION_SHIPMENT_NOT_FOUND`; 409 `PRODUCTION_NOT_ALLOWED`; 422 `PRODUCTION_INVALID`.

Technical references: `backend/src/lab-production/services/shipments.service.ts` · `ShipmentsService.receive`, `markLost` · `web-app/src/features/lab-orders/pages/ReceiveShipmentPage.tsx`

40.3.29 Send impressions and models to the lab

Status: [Partial]

What it does. Creates a shipment from the clinic to the lab (impressions, models, work going back). It changes no status; the lab confirms receipt in its portal.

How the user does it. No clinic screen offers it. The web app's API layer has a `sendToLab` function that no component uses.

Behind the scenes — in simple terms. The backend can record that the clinic posted something to the lab, with a courier and tracking number, and print-able QR data; no button exists.

Behind the scenes — technically. `POST /lab-shipments` (`lab.orders.create`) → `ShipmentsService.dispatchToLab`: all lines must belong to one lab and to items that are not rejected or cancelled; a shipment row and lines are inserted with direction `to_lab`, number from the `SHP` sequence and a QR token; an audit row is written.

Rules and validations. An item cannot be listed twice; one shipment goes to one lab.

Who can do it. `lab.orders.create`.

Data read and written. `shipments`, `shipment_items`, `audit_logs`, `number_sequences`.

Effects on other modules. The lab can receive it in its portal (Chapter 41).

Error cases. 409 `PRODUCTION_NOT_ALLOWED`; 422 `PRODUCTION_INVALID`.

Technical references: `backend/src/lab-production/controllers/clinic-production.controller.ts` · `ShipmentsService.dispatchToLab` · `web-app/src/features/lab-orders/services/production.api.ts` (`sendToLab`, unused)

40.3.30 Notifications about lab work

Status: [Partial]

What it does. The doctor, the lab's members, the lab contact and the patient are told about key moments. Doctor and lab-member messages are in-app. Messages to a lab contact or a patient have a channel (WhatsApp, SMS or email), but the Platform has no real provider for those channels yet: the only provider registered for them writes the message to the server log.

How the user does it. Nothing to do. In-app messages appear in the notification inbox of the doctor or lab member.

Behind the scenes — in simple terms. Each message is written to an outbox in the same transaction as the business change, so a message exists exactly when the change committed. A worker sends it later. Which channels and templates apply is set by notification rules (Chapter 44).

Behind the scenes — technically. Events are seeded in `notification-events.seed.ts` and queued with `NotificationService.enqueue`. The code that emits each one:

Event	Emitted by	Audience
`order.submitted`	`OrderSubmitService.submit`	lab members

Event	Emitted by	Audience
`order.magic_link`	`MagicLinksService.issue`	lab contact
`order.accepted`, `order.rejected`, `order.clarification_requested`, `order.expected_date_changed`	`LabOrderActionsService`	ordering doctor
`item.ready`	`LabWorkService.qc` (on QC pass)	doctor and patient
`stage.sent_to_clinic`	`ShipmentsService.dispatchToClinic`	doctor
`shipment.received`	`ShipmentsService.receive` (shipments to the clinic)	lab members
`shipment.lost`	`ShipmentsService.markLost`	lab members and doctor
`stage.approved`, `stage.correction`	`ClinicProductionService.decideStage`	lab members
`remake.requested`	`ProductionWorkflowService.openRemakeRequest`	lab members
`remake.agreed`	`ClinicProductionService.decideRemake`	lab members
`order.completed`	`LabPatientNotifier` (on order state completed)	doctor and patient

The table lists, for each seeded event that code actually emits, the emitting class and who receives it. Four seeded events have no emitter in the code: `order.overdue`, `shipment.dispatched`, `tryin.appointment_due` and `appointment.rescheduled_lab_delay`. No scheduled job checks for overdue orders or due try-ins (the only `@Cron` jobs belong to other modules). A workflow transition could emit an event through a `notify` effect, but the seeded dental workflows list none. The patient is addressed in notifications only as the patient's number or name according to the link's patient data level.

Rules and validations. A message with no recipient is skipped. Lab members get a link into the portal; the doctor gets a link to the clinic order.

Who can do it. Automatic.

Data read and written. `notification_outbox`, then `user_notifications` for in-app delivery.

Effects on other modules. Chapter 44 covers rules, quiet hours and delivery.

Error cases. None are shown to the user; a failed send is retried by the notification worker.

Technical references: `backend/src/lab-production/workflow/production-notifier.ts` · `ProductionNotifier`, `NotificationService.enqueue` · `backend/src/notifications/seeds/notification-events.seed.ts`

40.4 Entities and data model

The lab module has about fifty tables. This section covers the ones the clinic side uses; Chapter 25 describes every table and Chapter 42 the money tables. Every foreign key in the lab tables uses ON DELETE RESTRICT: no lab record can be deleted while anything refers to it, so records are voided, reversed, archived or switched off instead. All lab tables use UUID version 7 keys, whose first 48 bits are the creation time.

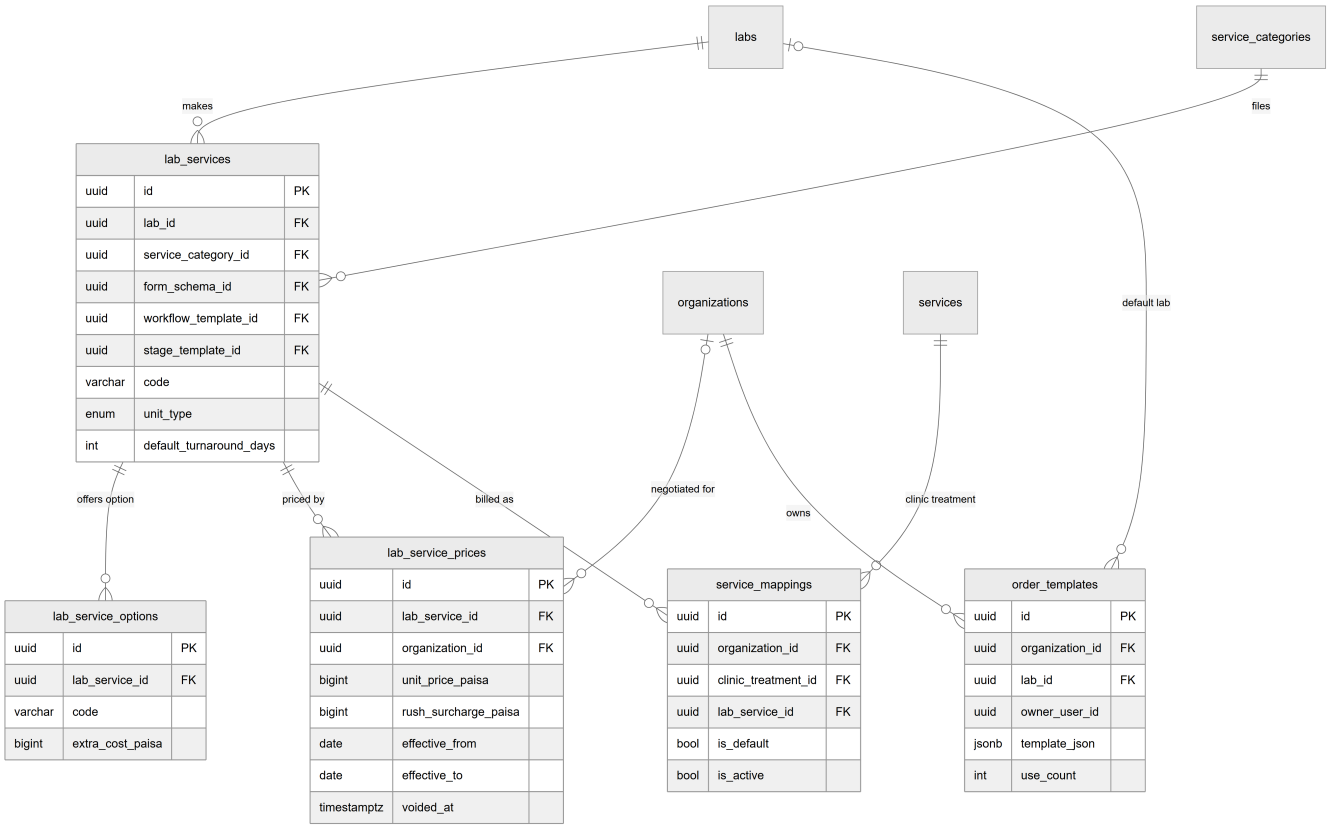

Figure 40.11 — Lab services, options, prices, mappings and order templates.

The diagram centres on `lab_services`, with columns `lab_id`, `service_category_id`, `form_schema_id`, `workflow_template_id`, `stage_template_id`, `code`, `unit_type` and `default_turnaround_days`. `labs` points to it ("makes") and `service_categories` points to it ("files"). Three tables hang off it: `lab_service_options` ("offers option", with `extra_cost_paisa`), `lab_service_prices` ("priced by", with `unit_price_paisa`, `rush_surcharge_paisa`, `effective_from`, `effective_to`, `voided_at`) and `service_mappings` ("billed as", with `is_default` and `is_active`) hang off it. `organizations` connects to `lab_service_prices` with an optional line ("negotiated for"), because an empty organization means the list price. `services` (the clinic's billing catalog) connects to `service_mappings` ("clinic treatment"). `order_templates` belongs to an organization ("owns") and optionally to a lab ("default lab"), and holds `owner_user_id` (a logical link to a user, no foreign key), `template_json` and `use_count`.

The relationships in words:

- Each lab service belongs to exactly one lab and one service category; its form, workflow and stage template are optional.
- Each option and each price row belongs to exactly one lab service; a price row belongs to zero or one organization.
- Two non-void windows of one service and one clinic (or both list prices) cannot overlap: the exclusion constraint `excl_lab_service_prices_overlap` guarantees it.
- Each mapping belongs to one organization, one clinic treatment and one lab service. There is at most one active default per treatment, and an active lab service maps to one treatment per organization.
- Each template belongs to one organization; its lab, if set, is one lab.

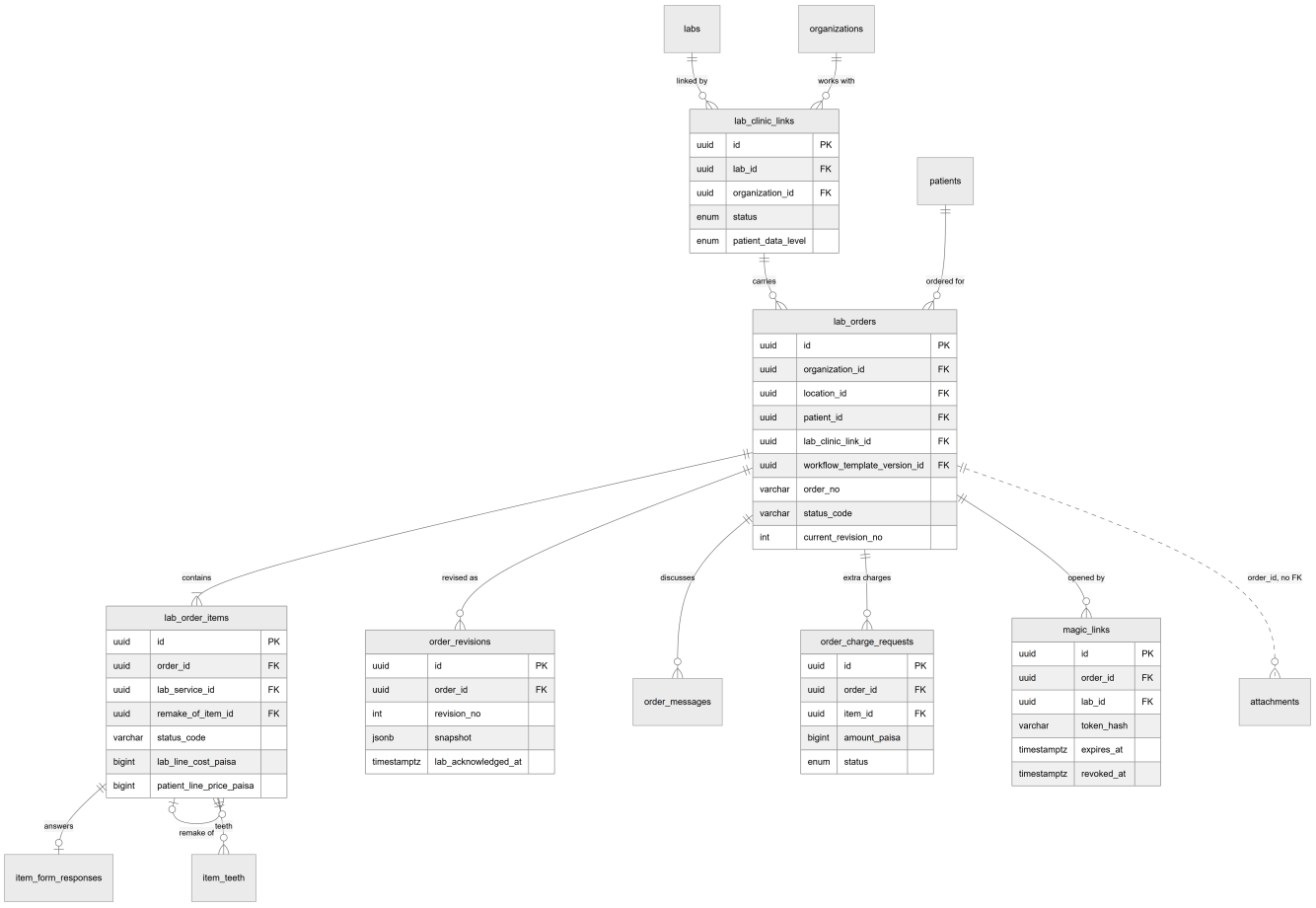

Figure 40.12 — Lab orders, items, revisions, messages, charge requests and magic links.

The diagram shows `lab_orders` with `organization_id`, `location_id`, `patient_id`, `lab_clinic_link_id`, `workflow_template_version_id`, `order_no`, `status_code` and `current_revision_no`. It points down to `lab_order_items` ("contains": `order_id`, `lab_service_id`, `remake_of_item_id`, `status_code`, `lab_line_cost_paisa`, `patient_line_price_paisa`), which has a loop to itself ("remake of"), one `item_form_responses` row and many `item_teeth`. `lab_orders` also reaches `order_revisions` ("revised as", with a `snapshot` and `lab_acknowledged_at`), `order_messages` ("discusses"), `order_charge_requests` ("extra charges") and `magic_links` ("opened by", with `token_hash`, `expires_at`, `revoked_at`). `lab_clinic_links` ("carries") links a lab and an organization to the orders, and `patients` ("ordered for") points to the order. A dotted line from `lab_orders` to `attachments` is labelled "order_id, no FK".

The relationships in words:

- Each order belongs to exactly one organization, location, patient, lab and lab link, and pins one workflow version that never changes; the doctor is optional.
- Each order contains at least one item. Deleting is blocked (RESTRICT); only draft items may be removed, by the draft editor.
- An item points to at most one original item it remakes; a request and a new item are linked one to one.
- Each item has at most one form-response row and any number of tooth rows.
- Each revision, message and charge request belongs to one order; messages and charge requests may point at one item.
- `order_no` is unique per organization; `idempotency_key` is unique per organization.
- Attachments point to an order by `order_id` without a foreign key and are polymorphic by `entity_type` (order or item).

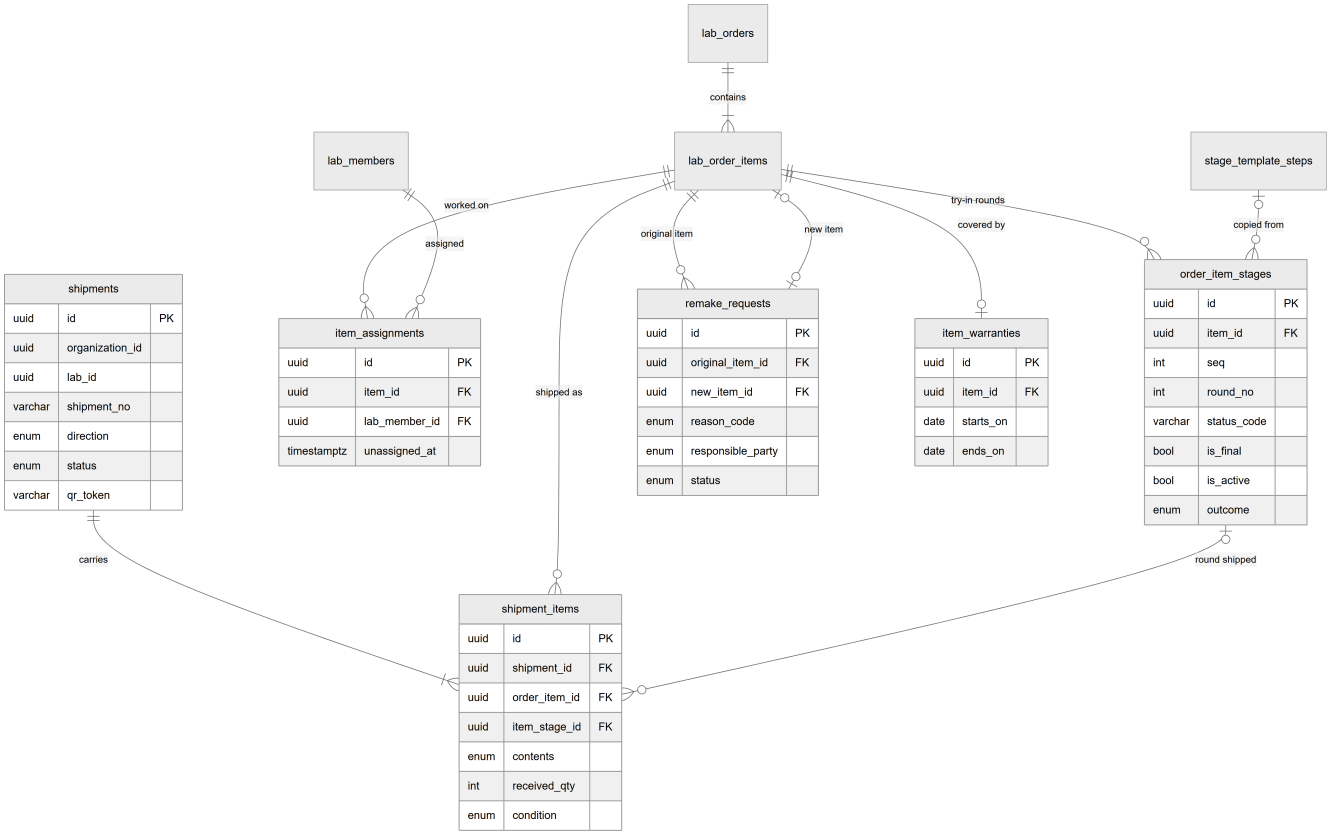

Figure 40.13 — Stages, technician assignments, shipments, remake requests and warranties.

The diagram shows `lab_order_items` at the centre. `order_item_stages` hangs off it ("try-in rounds") with `seq`, `round_no`, `status_code`, `is_final`, `is_active` and `outcome`, and is "copied from" `stage_template_steps`. `item_assignments` ("worked on") connects an item to `lab_members` ("assigned") with `unassigned_at`. `shipments` ("carries") has `shipment_items`, each of which points to an item ("shipped as") and optionally a stage ("round shipped"); a shipment has `direction`, `status` and `qr_token`. `remake_requests` connects twice to the item ("original item" many, "new item" one to one) and carries `reason_code`, `responsible_party` and `status`. `item_warranties` is one to one with the item, with `starts_on` and `ends_on`.

The relationships in words:

- At most one stage round per item is active at a time (partial unique index), and rounds are unique by item, sequence and round number.
- At most one open assignment per item (partial unique index); reassigning closes the old row.
- A shipment carries lines for items of one clinic and one lab; an item appears at most once in a shipment; received quantity cannot exceed shipped quantity.
- `new_item_id` is unique, and at most one open (requested or disputed) remake request exists per original item.
- A warranty is one per item.

The schema comes from the entity files; there is no migration file for these tables, because the lab module relies on the development schema sync and a reviewed schema sync in production.

40.5 Status lifecycles

The statuses in this section are the **seeded defaults** (version 1 of the `DENTAL_ORDER`, `DENTAL_ITEM` and `DENTAL_STAGE` workflows). Because states and transitions are data, a custom published version can differ; the order derivation rules described in 40.1 are fixed in code.

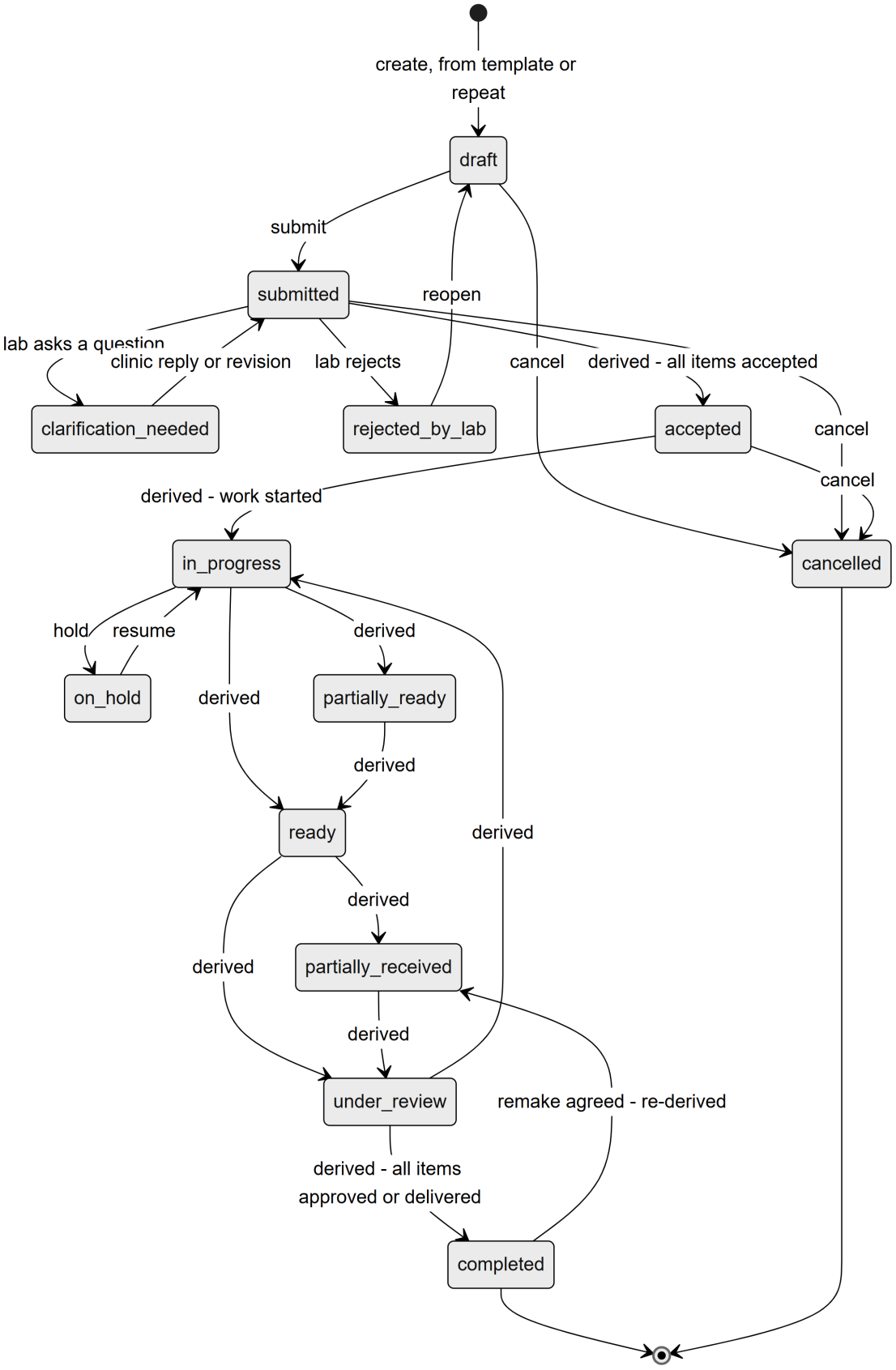

Figure 40.14 — The seeded lab order workflow.

The diagram has thirteen states. A new order enters at `draft` (by create, from a template, or by repeat). `draft` leads to `submitted` on "submit" and to `cancelled` on "cancel". From `submitted`: the lab asks a question and the order goes to `clarification_needed`; the clinic's reply or a revision returns it to `submitted`; the lab rejects and the order goes to `rejected_by_lab`, from which "reopen" returns it to `draft`; the clinic cancels and it goes to `cancelled`; and when every item is accepted it is derived to `accepted`. `accepted` can be cancelled, or derived

to `in_progress` when work starts. `in_progress` can be put on hold (`on_hold` , with a reason) and resumed, or derived to `partially_ready` or `ready` . `partially_ready` is derived to `ready` . `ready` is derived to `partially_received` or `under_review` ; `partially_received` to `under_review` . `under_review` is derived back to `in_progress` (for example after a rejection) or onward to `completed` once all items are approved or delivered. `completed` can return to `partially_received` when a remake adds new work. `completed` and `cancelled` end the diagram.

Walking through the states:

- **draft** — editable working copy, no number. Anyone with the right to order may edit, submit, cancel.
- **submitted** — number and prices frozen, the lab has been told. Changes are revisions.
- **clarification_needed** — the lab asked something. A reply or a revision answers it.
- **rejected_by_lab** — the lab refused the order, with a reason. The clinic may reopen it as a draft. Its items keep the status "Waiting for lab" in the database, which is an oddity recorded in the conflicts file.
- **accepted** — every item accepted with an expected date.
- **in_progress**, **partially_ready**, **ready** — derived from the items' production states.
- **on_hold** — a manual hold by the clinic. Derivation does not override it; **Resume** re-derives.
- **partially_received**, **under_review** — derived from receipt and approval.
- **completed**, **cancelled** — closed; magic links are revoked.

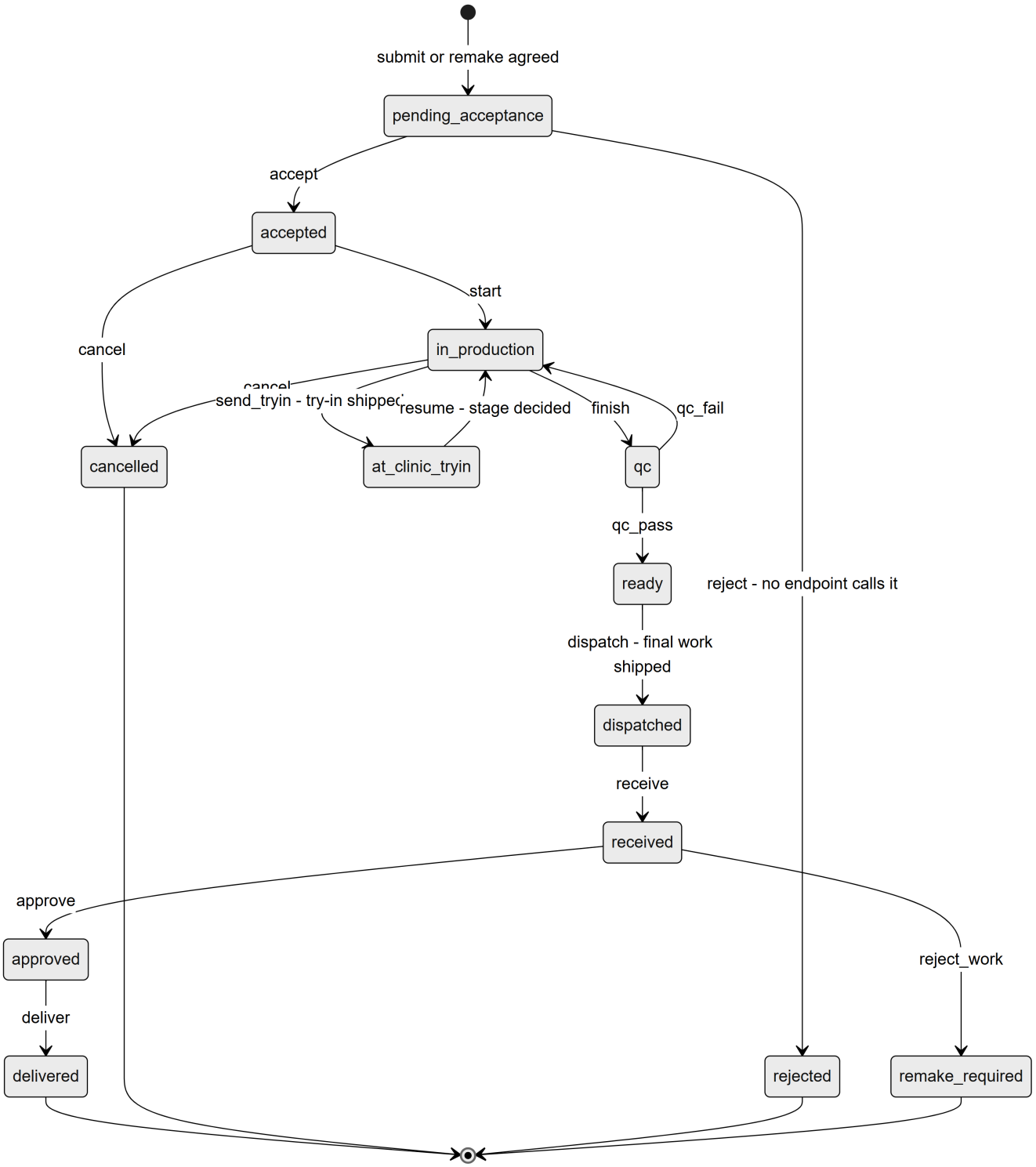

Figure 40.15 — The seeded item workflow.

The diagram runs from `pending_acceptance` (entry at submit, or when a remake is agreed) to `accepted` on "accept". An unused branch leads to `rejected` ("reject", no endpoint calls it). `accepted` goes to `in_production` on "start" or to `cancelled`. `in_production` goes to `at_clinic_tryin` ("send_tryin", when a try-in round is shipped), back from there on "resume", to `qc` on "finish", or to `cancelled`. `qc` returns to `in_production` on "qc_fail" or advances to `ready` on "qc_pass". `ready` goes to `dispatched` on "dispatch" (final work shipped). `dispatched` goes to `received` on "receive". `received` goes to `approved` on "approve" or to `remake_required` on "reject_work". `approved` goes to `delivered` on "deliver". `delivered`, `cancelled`, `rejected` and `remake_required` are final. The last state is final for this item only: the remake is a new item.

Walking through the states: **pending_acceptance** (shown as "Waiting for lab") waits for the lab's accept with an expected date; **accepted** has its stages created; **in_production** is bench work; **at_clinic_tryin** means a try-in round is with the clinic; **qc** is the lab's own quality check; **ready** means passed QC; **dispatched** is on the way to the clinic; **received** has been logged by the front desk; **approved** has been accepted by the doctor; **delivered** was fitted and has a warranty; **remake_required** was rejected and has an open remake request. The `send_tryin`, `resume` and `dispatch` transitions are done by the system, driven by shipments and decisions, not by a user button. The `rejected` state has no caller in the code that this review found (the order-level reject changes only the order); this is listed as unverified in the hand-back.

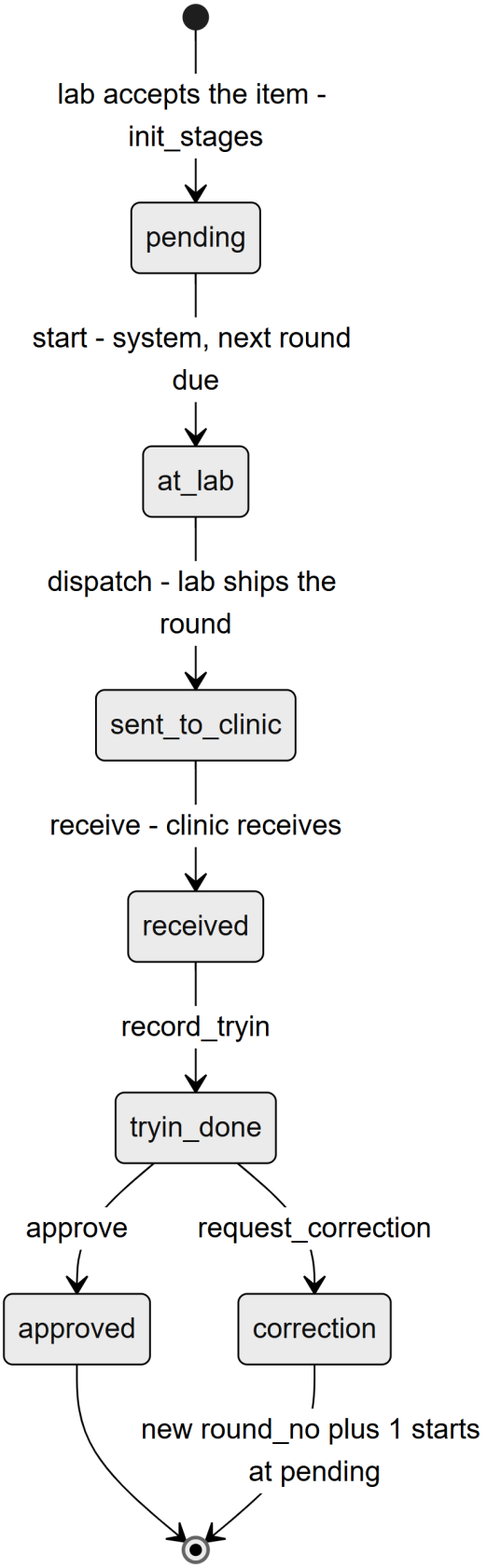

Figure 40.16 — One try-in round.

The diagram shows one round: `pending` (created when the lab accepts) goes to `at_lab` when it is started by the system, to `sent_to_clinic` when the lab ships it, to `received` on receipt, to `tryin_done` when the doctor records the try-in, and then either to `approved` (end) or to `correction` (end). A correction closes the round and the same stage starts again as `round_no` plus one, beginning at `pending`.

Walking through the states: **pending** — created, not yet started; **at_lab** — the active round, being worked; **sent_to_clinic** — on its way; **received** — logged at the clinic; **tryin_done** — the patient has tried it; **approved** — the stage is complete and the next stage starts; **correction** — closed with notes, a new round follows. A stage that needs no patient visit goes straight from received to the decision.

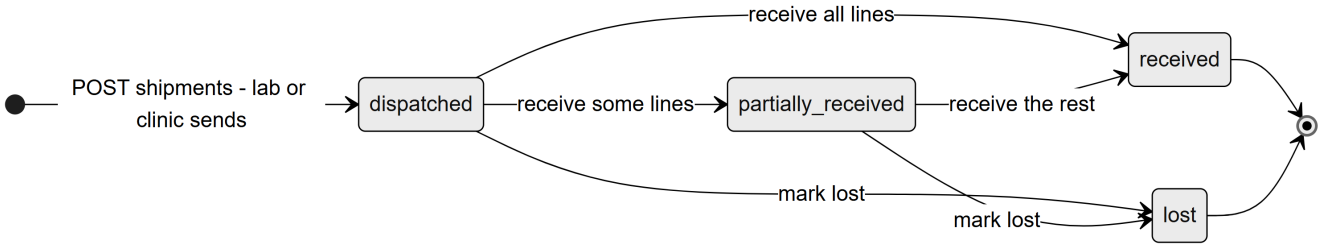

Figure 40.17 — The shipment status values.

The diagram shows a shipment entering at `dispatched` when either side posts it. It goes to `received` when every line is recorded, or to `partially_received` when some lines are recorded and others remain; `partially_received` goes to `received` when the rest arrive. From `dispatched` or `partially_received` a shipment can be marked `lost`. `received` and `lost` are final. These four values are fixed in code, not in a workflow.

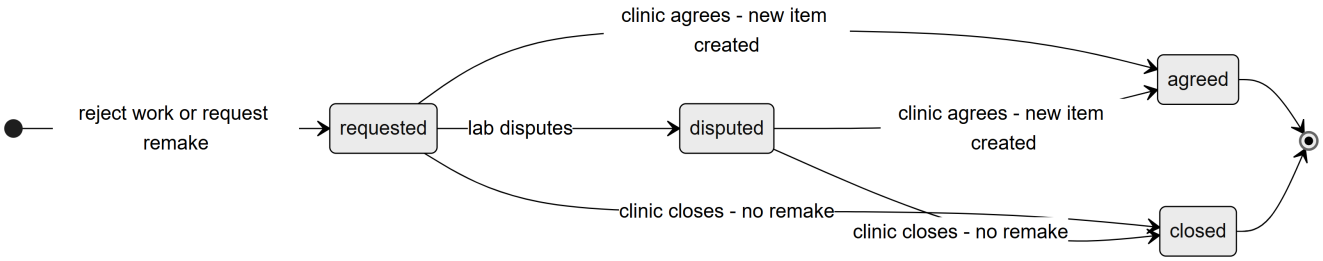

Figure 40.18 — The remake request status values.

The diagram shows a request entering at `requested` (a rejection or a remake request). The lab may dispute it (`disputed`). From `requested` or `disputed` the clinic either agrees (`agreed`, a new item is created) or closes it (`closed`, no remake). `agreed` and `closed` are final. These values are fixed in code.

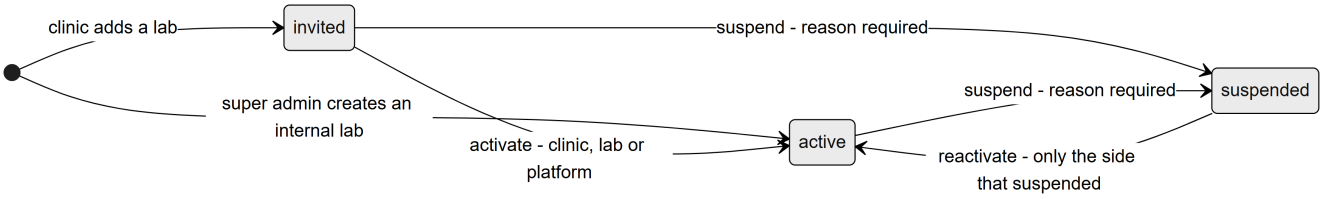

Figure 40.19 — The lab link status values and who may move them.

The diagram shows a link entering at `invited` (a clinic adds a lab) or directly at `active` (the super admin creates an in-house lab). `invited` goes to `active` on "activate" or to `suspended` on "suspend" (reason required). `active` goes to `suspended` on "suspend". `suspended` goes back to `active` on "reactivate", only by the side that suspended it. These values and rules are fixed in code (`lab-link-transitions.ts`).

40.6 Connections to other modules

- **Workflow engine (Chapter 43).** Order, item and stage statuses and their transitions are workflow template data; forms are form schemas. The super admin edits them in Config Studio.
- **Lab portal (Chapter 41).** The lab's side of every step: accept, reject, ask, produce, ship, dispute, read bills.

- **Lab finance (Chapter 42).** Accepting an order creates the vendor bill draft from the items' frozen lab costs; the patient lab invoice, payments and statements read the same snapshots.
- **Billing and treatments (Chapters 33 and 34).** A lab service is billed through a mapped treatment from the billing catalog; the patient price follows the billing price precedence.
- **Commissions (Chapter 35).** Lab-line commissions read the lab cost and the treatment's doctor.
- **Patients and appointments (Chapters 28 and 29).** An order belongs to a patient; a draft may link an appointment and a treatment; a try-in may link the patient's appointment. A warning appears when the lab's date is after the patient's next appointment.
- **Notifications (Chapter 44).** The events in 40.3.30.
- **Access control and features (Chapters 10 and 11).** Permissions `lab.*` and the feature `dental.lab_orders`.
- **Subscriptions (Chapter 45).** In read-only mode every write is refused with 402 `SUBSCRIPTION_EXPIRED`.

40.7 Limitations and known gaps

- The lab cannot manage its own catalog or prices; only the super admin does, through the Lab Directory.
- Service categories can be created only through the API.
- No screen cancels a single item or sends impressions and models to the lab, although the API supports both.
- A whole order that is already in progress cannot be cancelled in the seeded workflow, and cancelling an order does not cancel its items.
- Four seeded notification events are never emitted: `order.overdue`, `shipment.dispatched`, `tryin.appointment_due` and `appointment.rescheduled_lab_delay`. There is no overdue job.
- Patient and lab-contact messages go to a development logger until a real provider exists.
- Attachment visibility "patient shared" is stored but nothing shows it to a patient.
- Lab price windows use the platform's Asia/Karachi day, while order submit prices on the location's day.
- The warranty length is fixed at 12 months by default; no screen or endpoint changes or voids it.
- Magic-link rate limiting is kept in the memory of each API process, so it is per server instance.
- A suspended link stops new orders but does not stop orders already sent.
- A rejected order that is reopened keeps its items in the status "Waiting for lab" while the order is a draft again.

40.8 Owner's FAQ

Can a receptionist create a lab order? No. The Receptionist and Assistant templates have only `lab.orders.view` at location scope, so they can read orders and receive shipments but not create or change them. Doctors and Admin hold `lab.orders.create`.

Who can see which orders? Scope of `lab.orders.view`: a doctor sees the orders they are the doctor of or created; the front desk sees their location's orders; Admin sees all. Orders outside the scope answer "not found".

What happens to the price if the lab changes its price after I submit? Nothing. Prices of every line are copied into the item at submit and cannot change except by a revision or an approved extra charge.

Why did my submit fail with a message about a treatment? The line's lab service is not mapped to one of your treatments, so the patient price is unknown. Map it in **Lab price list** under **Treatment mapping**, or choose a treatment on the line.

What if the lab never answers? A coordinator holding both `lab.orders.create` and `lab.links.manage` (Admin by default) can record the lab's accept, reject, question or date change as a proxy action. It is marked in the history as entered by the clinic for the lab.

How long does a magic link last, and can I stop it? 24 hours by default (the environment variable `LAB_MAGIC_LINK_TTL_HOURS` changes it, up to 168 hours). You can revoke it at any time; it also stops when the order is completed, cancelled or rejected, or the lab is suspended.

What does "Patient code only" do? The lab sees the patient number instead of the name in its portal, its link page, the printed work order and notifications.

Where are lab orders stored? In `lab_orders` with their items in `lab_order_items`; every status change is also written to `status_history` and every edit to `audit_logs`.

Can I delete a lab order? No. There is no delete. Drafts can be cancelled; submitted orders are cancelled or completed, and every foreign key blocks deletion.

Can a lab see what I charge the patient? No. Lab-facing responses and the revision snapshots carry the lab cost only, never the patient price, the clinic's treatment or internal files.

Chapter 41 — Lab Portal

This chapter describes the lab side of dental-lab work on the Platform: the lab portal where lab staff sign in to receive clinic orders, accept and produce them, ship them and see what they are owed, and the no-login magic link page that lets a lab contact answer one order from a phone. It is written for the product owner, who needs to know what a lab can and cannot do today. The clinic side of the same cases (ordering, try-ins, approvals) is in Chapter 40, and the money the lab sees read-only is covered by Chapter 42.

41.1 What this module is for

A **lab** is a dental laboratory that makes crowns, bridges and dentures for clinics. It is not a tenant of the Platform and has no organization of its own. Its people are **lab members**: user accounts with the role **lab admin** or **lab technician** on one or more labs. They have no clinic membership, so a clinic can never see their data and they can never see a clinic's.

The lab portal gives them a workspace that is separate from the clinic workspace:

- **Orders.** An inbox of the work that clinics sent, where a lab admin accepts an order with a ready date, rejects it, asks a question or moves the date.
- **Production.** A work queue per item: start work, send to quality check (QC), pass or fail QC, assign a technician, ship try-in rounds and finished work to the clinic, receive parcels the clinic sent, report a lost parcel and dispute a remake request.
- **Clinics.** The clinics that linked this lab, where a lab admin accepts an invitation or suspends and reactivates a clinic.
- **Team** and **Lab profile.** The lab's members and its contact details.
- **Billing.** A read-only view of the bills clinics approved for the lab, the payments, and the statements.
- **Magic link page.** One order, no login, for a lab contact who received a link.

Who uses it, by canonical role:

- **Lab admin.** Everything in the portal: answers orders, assigns work, manages the team, accepts or suspends clinics, edits contact details.
- **Lab technician.** Reads orders, starts work, sends to QC, passes or fails QC, ships and receives parcels. Cannot accept or reject orders, assign technicians, manage the team or change a clinic's link.
- **Lab contact acting through a magic link.** Not a user. Sees one order and can accept, reject or ask a question on it.
- **Clinic staff** act on the same orders from the clinic side (Chapter 40) and, with the right permissions, can record answers for a lab that does not respond.

Where it appears: there is no clinic sidebar for lab-only users. They land in their own layout with the entries **Dashboard**, **Orders**, **Lab profile**, **Team** (lab admins only), **Clinics**, **Production** and **Billing**. A user who is both a lab member and clinic staff also sees a **Lab portal** entry in the clinic sidebar (`nav.lab_portal` , route `/lab-portal` , shown to anyone with at least one lab membership), and the portal offers a **Clinic workspace** link back.

The lab portal does not depend on a feature switch. Access is decided only by an active lab membership of an active lab. The clinic's feature `dental.lab_orders` gates the clinic side (Chapter 40), not the portal.

Patient data level

Each clinic decides per lab link how much of the patient's identity the lab sees: **Full patient name** or **Patient code only**. In the portal, with "code only" the order list, the order page, the magic link page, the printed work order and notification texts show the clinic's patient number instead of the name. The patient number is always sent in a separate field as well. Prices are never shown: lab-facing responses carry the lab's own cost, never the patient price, the clinic's treatment or internal files, and the clinic list shows a clinic only by name and city, status and payment terms.

41.2 Screens

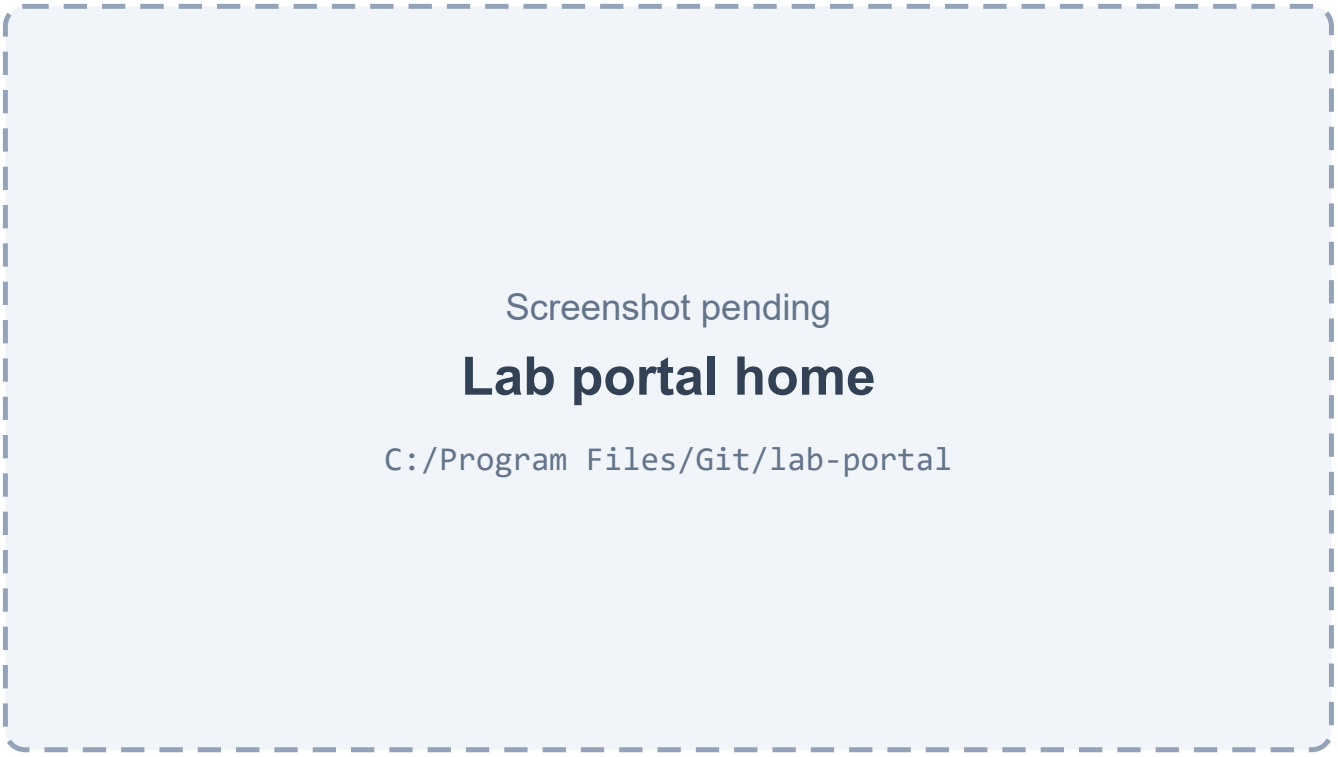

Figure 41.1 — The lab portal Dashboard (screenshot pending).

The portal has a left navigation (Dashboard, Orders, Lab profile, Team, Clinics, Production, Billing), a lab switcher at the top (a selector "Switch lab" with the placeholder "Choose a lab", for users who work for several labs), a user block with a **Sign out** icon and, for users who also work for a clinic, a **Clinic workspace** link. The Dashboard is titled "Welcome to" followed by the lab's name, with the subtitle "Your lab's workspace." It shows three cards: "Orders waiting for you" (the number of orders in status Submitted, a link to the order list), "Active clinics" (a link to the clinic list) and "Team members".

Screenshot pending

Lab portal orders

`C:/Program Files/Git/lab-portal/orders`

Figure 41.2 — The lab's order inbox (screenshot pending).

The **Orders** page is titled "Orders" with the subtitle "Work sent to your lab. Accept new orders with the date they'll be ready." It has a search box ("Search order no. or clinic…") and a status filter. The table has the columns Order (with an "Urgent" badge), Clinic, Patient (a name or a code, by the link's setting), Doctor, Status, Due, Received. With no orders the page says "No orders yet. When a clinic sends your lab an order, it shows up here — and on WhatsApp if the clinic sends you a link."

Screenshot pending

Lab portal order detail

`C:/Program Files/Git/lab-portal/orders/$orderId`

Figure 41.3 — One order as the lab sees it (screenshot pending).

The order page is titled "Order" and the order number, with the clinic and the patient reference as subtitle, a back button to the list, the status badge, an "Urgent" badge and the dates "Wanted by" and "Ready on". The buttons at the top right are the actions the lab may take now: **Accept**, **Ask a question**, **Reject**, **Change date**. The body has a **Work** section (each item with its service, quantity, rush, options, teeth, and the form answers such as shade and material), a **Revisions** section (each revision with "Read" and a time, or a button to acknowledge it, or "New"), **Messages** with a box "Write to the clinic…" and **Files** (only files the clinic shared with the lab, with a download button).

Figure 41.4 — The Production desk (screenshot pending).

The **Production** page is titled "Production" with the subtitle "Work in progress, try-in rounds, QC and shipments to clinics." It has three tabs: **Work queue**, **Shipments** and **Remakes**. The work queue table has the columns Order, Work, Stage, Status, Due, Technician and an action column. The action on each row changes with the item's step: **Start work**, **Ship for try-in**, **Send to QC**, **QC pass** with a cross button ("QC fail — back to production"), or **Ship**; a button "Assign technician" appears for lab admins. The Shipments tab lists shipment number, contents, status and dispatch time with buttons to print the slip with its QR code, to mark a shipment lost, and **Receive** for parcels the clinic sent. The Remakes tab lists order, reason, status and a dispute button.

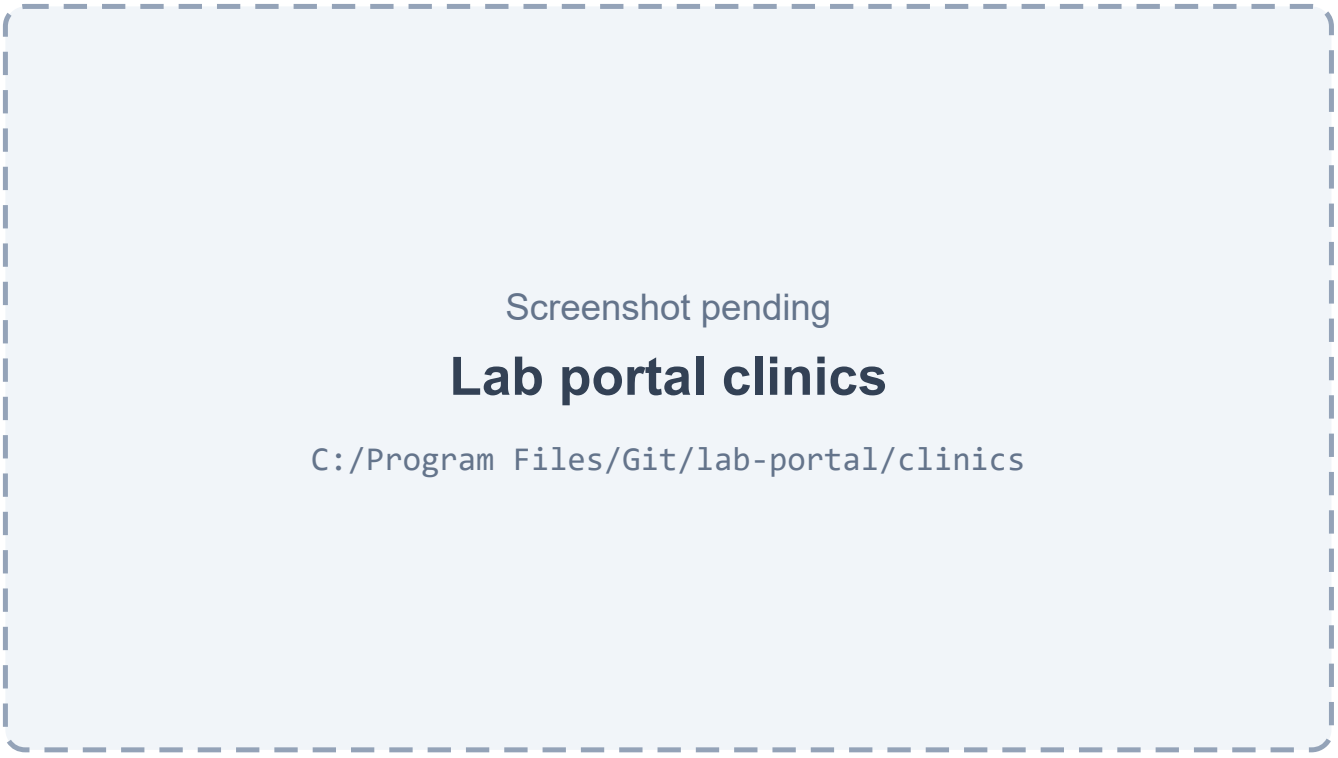

Figure 41.5 — The clinics that work with the lab (screenshot pending).

The **Clinics** page is titled "Clinics" with the subtitle "Clinics that send work to your lab. Accept new invitations here." The table has the columns Clinic, City, Status, Payment terms and an actions column; the filters are a search box ("Search by clinic name…") and a status filter. For a lab admin each row shows an accept button (tooltip "Accept" followed by the clinic name), a reactivate button or a suspend action that asks for a reason ("e.g. Outstanding balance"). With no clinics the page says "No clinics yet" and explains that an invitation appears here when a clinic adds the lab to its list.

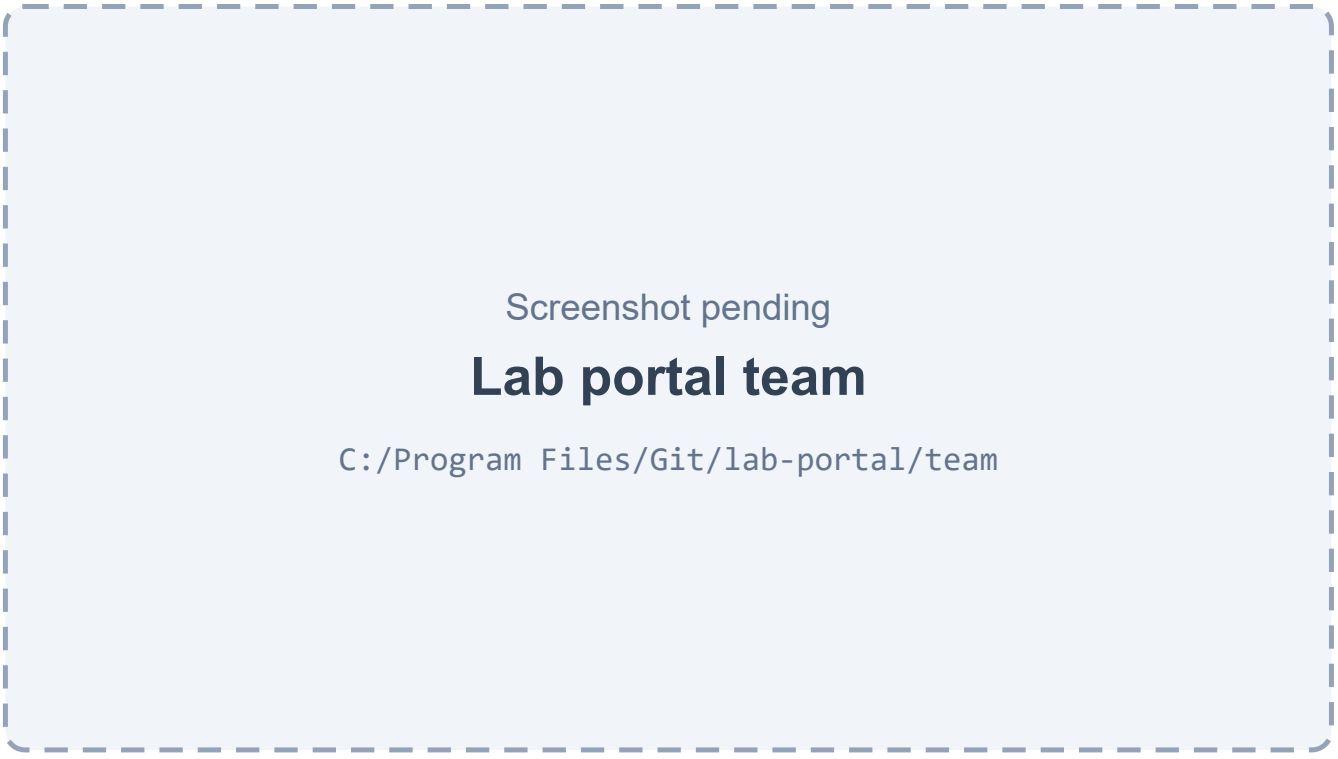

Figure 41.6 — The lab's team (screenshot pending).

The **Team** page is titled "Team" with the subtitle "People who work at your lab." For a technician it shows "Lab admins only — Ask your lab admin to add or change team members." For lab admins the table has the columns Member, Status and "Role & access" (a role selector and an active switch per row) and the filters search ("Search by name or email...") and role. The add sheet is "Add team member": "The person needs their own account — clinic staff accounts can't join an outside lab." It asks for "Account email" and "Role".

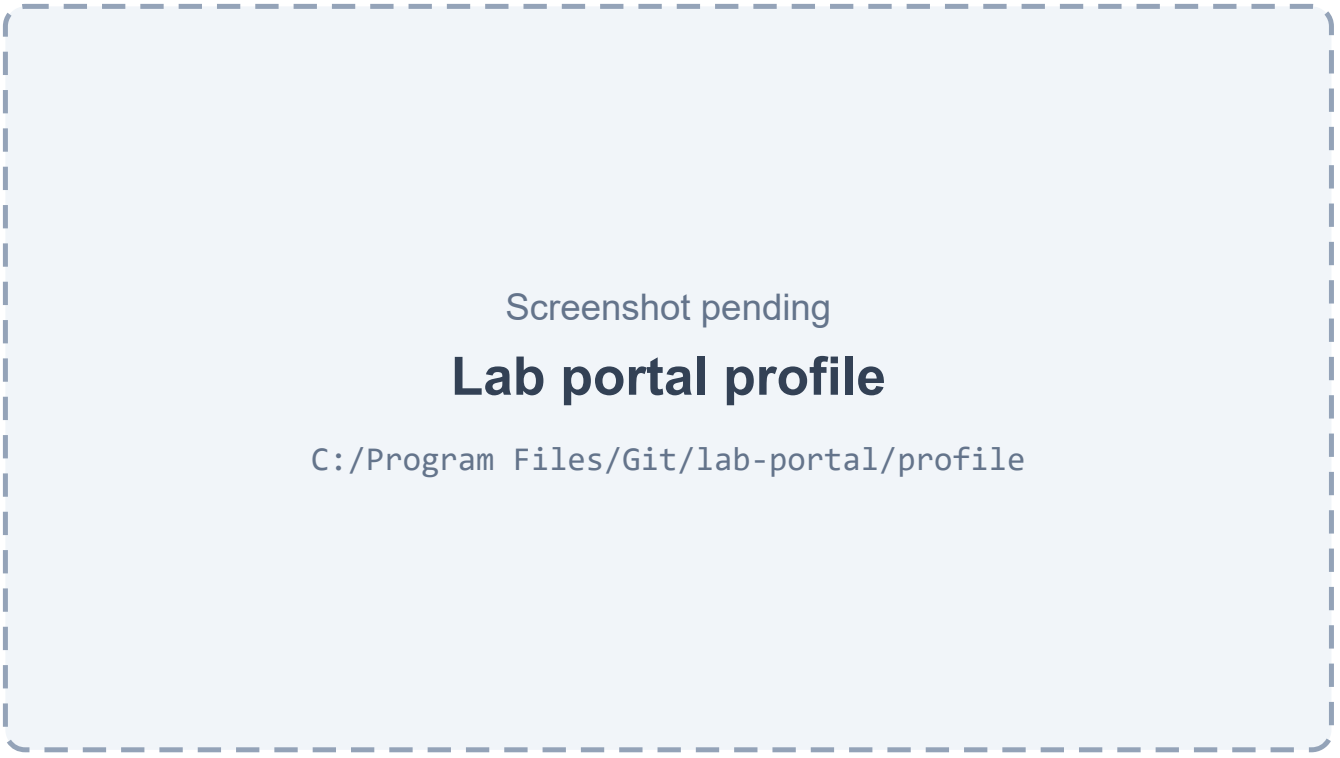

Figure 41.7 — The lab profile (screenshot pending).

The **Lab profile** page is titled "Lab profile" with the subtitle "How clinics and couriers reach your lab." It lists phone, WhatsApp, email, city and address. A lab admin can press an edit button that opens "Edit contact details": "Clinics see these details on their lab list. The lab's name is managed by platform support." The form has Phone, WhatsApp, Email, City and "Pickup / delivery address", with an "unsaved changes" hint.

Screenshot pending

Lab portal billing

`C:/Program Files/Git/lab-portal/billing`

Figure 41.8 — The lab's read-only Billing page (screenshot pending).

The **Billing** page is titled "Billing" with the subtitle "Bills clinics approved for your work, and what they paid." It has three tabs. **Bills** has the columns Bill, Clinic, Status, Due, Total and Balance, and opens a bill with its lines and the totals Total, Paid, Balance. **Payments** has Payment, Clinic, "Paid on", Method, Amount and Status. **Statements** has Period, Clinic, "Ref.", Total and Status. Nothing can be edited here; an empty tab says "Nothing here yet".

Screenshot pending

Magic link order page

`C:/Program Files/Git/lab-link/$token`

Figure 41.9 — The no-login order page opened from a magic link (screenshot pending).

The magic link page (`/lab-link/$token`) is a single page with a small header "Lab order" and, on the right, "For" the lab's name and "link valid until" a date and time. The body shows "Order" with its number, the clinic, the patient reference and the doctor, the status badge, an "Urgent" badge, "Wanted by" and "Ready on" dates, the

same action buttons as the portal (but only **Accept**, **Reject** and **Ask a question**) and the same Work, Revisions, Messages and Files sections. A dead link shows "This link is not valid" or "This link no longer works" with "Ask the clinic to send a new link, or sign in to the lab portal."

41.3 Features

How one case flows between the two sides

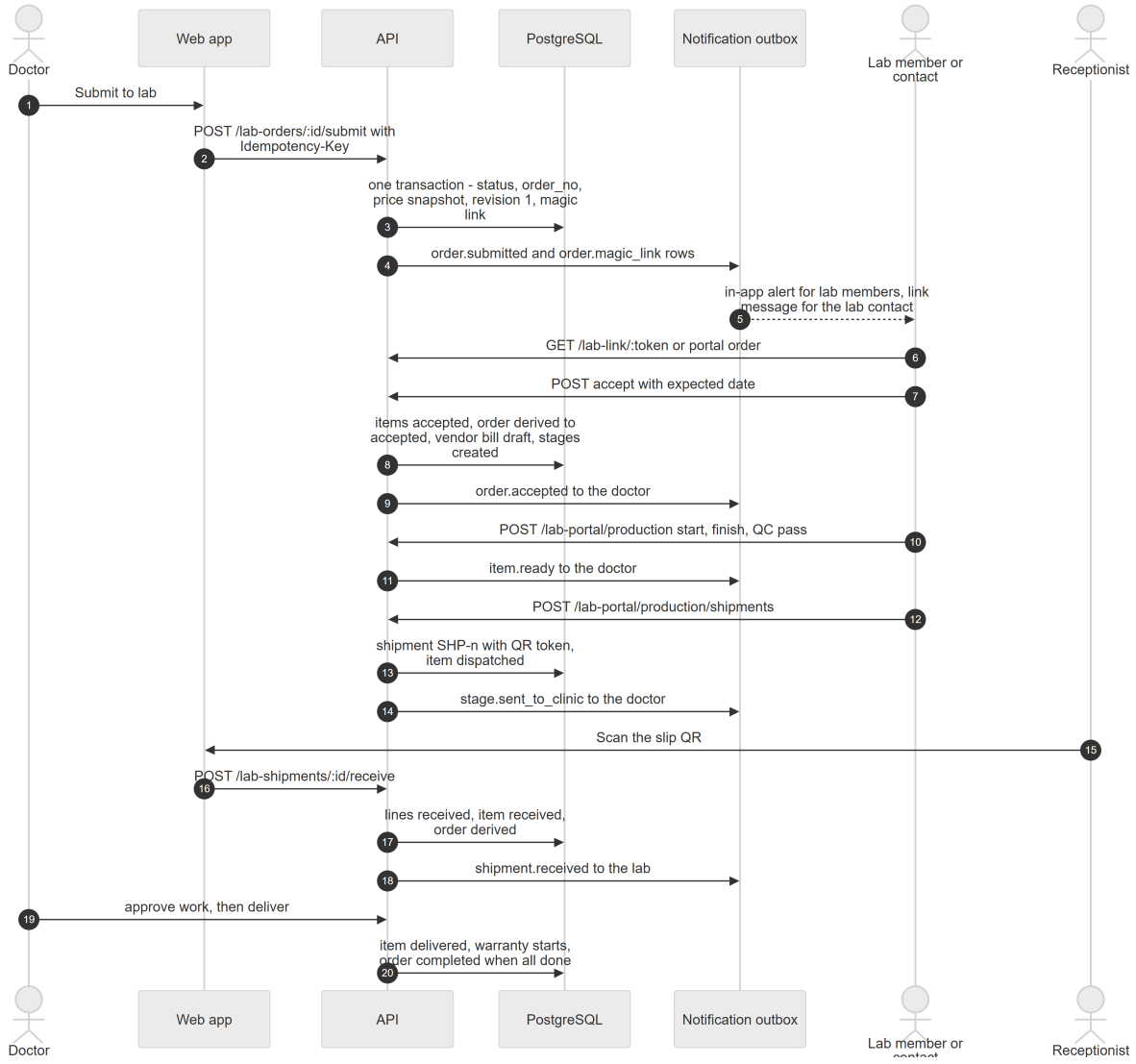

Figure 41.10 — Submit, magic link, accept, production, shipment and receipt.

The diagram has seven participants: Doctor, Web app, API, PostgreSQL, Notification outbox, Lab member or contact and Receptionist. The Doctor presses "Submit to lab"; the Web app calls `POST /lab-orders/:id/submit` with an Idempotency-Key. The API does one database transaction (status, order number, price snapshot, revision 1, magic link) and puts `order.submitted` and `order.magic_link` rows in the outbox. The lab members get an in-app alert and the lab contact gets the link message. The Lab opens the link or the portal order and posts "accept" with an expected date; the API marks the items accepted, derives the order to accepted, creates the vendor bill draft and the stages, and queues `order.accepted` to the doctor. The Lab then posts start, finish and QC pass on the portal production routes, which queues `item.ready` to the doctor. The Lab creates a shipment; the API stores a shipment with a QR token, marks the item dispatched and queues `stage.sent_to_clinic`. The Receptionist

scans the slip, the Web app calls `POST /lab-shipments/:id/receive`, the API records the lines, marks the item received, re-derives the order and queues `shipment.received` to the lab. Finally the Doctor approves the work and delivers it, which starts the warranty and completes the order once every item is done.

41.3.1 Sign in as a lab member

Status: [Implemented]

What it does. A lab member signs in on the same login page as everybody else. The Platform recognises active lab memberships, and a user with no clinic access goes straight to the lab portal.

How the user does it.

1. A super admin creates the lab member's account under **User Admin** and adds it to the lab (Chapter 40, feature "Manage the staff of a lab"), or a lab admin adds an existing account from **Team**.
2. The lab member opens the login page, enters email and password and signs in.
3. A user with only lab memberships lands on `/lab-portal`. A user who also works at a clinic goes through the usual clinic flow and opens the portal from the **Lab portal** sidebar entry.

Behind the scenes — in simple terms. The login answer carries the list of labs the user may open. The app stores it, and from then on a lab-only user is kept inside the portal: any other address redirects back to it. Signing in again after a page reload restores the same list.

Behind the scenes — technically. `POST /auth/login`, the silent refresh and the other session responses include `labMemberships`, computed by `findLabMemberships` (active `lab_members` rows joined to active labs). In the web app `LoginForm` calls `isLabOnlyUser` (not super admin, no accessible organization, at least one membership) and navigates to `/lab-portal`; `__root.tsx` keeps lab-only users inside `/lab-portal`, sends users without a membership away from it, and lists `/lab-link/` among the paths open to everybody. `GET /lab-portal/me` (any membership, no selected lab) returns the labs for the switcher.

Rules and validations.

- A lab member's account cannot be a clinic staff or owner account of another clinic, and a super admin cannot be a lab member.
- A deactivated member or a suspended lab drops out of the list and is refused (`LAB_ACCESS_DENIED`).

Who can do it. Any user with an active membership. No permission key or feature key applies.

Data read and written. Reads `users`, `lab_members`, `labs`.

Effects on other modules. Authentication (Chapter 56) issues the session.

Error cases. 403 `LAB_ACCESS_DENIED` ("You are not a member of any lab").

Technical references: `backend/src/labs/portal/lab-memberships.ts` · `findLabMemberships` · `web-app/src/features/auth/components/LoginForm.tsx`

41.3.2 Work in one lab or switch between labs

Status: [Implemented]

What it does. Every portal call is tied to one lab. A member of several labs chooses the active lab with the lab switcher; a member of one lab needs no choice.

How the user does it. Use the "Switch lab" selector at the top of the portal.

Behind the scenes — in simple terms. The portal sends the chosen lab with every request. The server checks that the caller really is an active member of that lab and treats any other lab's id exactly like a lab that does not exist.

Behind the scenes — technically. `LabPortalGuard` (applied to every `@LabPortalController` route, which is `@ContextFree` so the clinic tenant guard skips it): rejects requests that carry clinic headers (400 `LAB_PORTAL_CLINIC_CONTEXT`), loads memberships, takes the lab from the `x-lab-id` header (or the only lab; 400 `LAB_SELECTION_REQUIRED` when there are several and none is given), checks `@LabRoles` against the membership's role (403 `FORBIDDEN`) and sets `request.labActor`. The web client adds `x-lab-id` only on `/lab-portal` calls and never sends the clinic headers there. Lab-owned reads start from `LabScopedService.scoped()`, already filtered by `lab_id`, so another lab's ids answer 404.

Rules and validations. Super admins hold no membership and are refused like anyone else. A malformed lab id is refused as access denied.

Who can do it. Any lab member.

Data read and written. Reads `lab_members`, `labs`.

Effects on other modules. None.

Error cases. 400 `LAB_SELECTION_REQUIRED`, `LAB_PORTAL_CLINIC_CONTEXT`; 403 `LAB_ACCESS_DENIED`, `FORBIDDEN`.

Technical references: `backend/src/labs/portal/lab-portal.guard.ts` · `LabPortalGuard`, `LabScopedService` · `web-app/src/features/lab-portal/components/LabSwitcher.tsx`

41.3.3 See the lab dashboard

Status: [Implemented]

What it does. Shows how many orders wait for an answer, how many clinics are active and how many people are on the team.

How the user does it. Open **Dashboard**. Click a card to go to the order list or the clinic list.

Behind the scenes — in simple terms. "Orders waiting for you" counts orders in the status Submitted.

Behind the scenes — technically. `GET /lab-portal/lab` returns the lab profile with `memberCount` and `activeLinkCount`; `GET /lab-portal/orders?status=submitted&limit=1` supplies the waiting count.

Rules and validations. Counts are for the active lab only.

Who can do it. Any lab member.

Data read and written. Reads `labs`, `lab_members`, `lab_clinic_links`, `lab_orders`.

Effects on other modules. None.

Error cases. Shows an error state with retry.

Technical references: `backend/src/labs/portal/lab-portal.controller.ts` · `PortalLabController`, `LabPortalService.profile` · `web-app/src/features/lab-portal/pages/LabPortalDashboardPage.tsx`

41.3.4 Read the order inbox

Status: [Implemented]

What it does. Lists every non-draft order sent to the lab, newest first, with clinic, patient reference, doctor, status, expected date and revision number.

How the user does it. Open **Orders**, search by order number or clinic name, filter by status and click a row.

Behind the scenes — in simple terms. Drafts never show: the lab sees an order only once the clinic submitted it.

Behind the scenes — technically. `GET /lab-portal/orders` → `LabOrdersPortalService.list`, which starts from the lab-scoped query and adds `status_code <> 'draft'`, joins the patient, the link (for the patient data level), the organization and the doctor, and maps the patient through `patientForLab`. Paged on the server.

Rules and validations. Search matches order number or clinic name.

Who can do it. Any lab member (lab admin and technician).

Data read and written. Reads `lab_orders`, `patients`, `lab_clinic_links`, `organizations`, `employees`.

Effects on other modules. None.

Error cases. 403 `LAB_ACCESS_DENIED`.

Technical references: `backend/src/lab-orders/portal/lab-orders-portal.service.ts` · `LabOrdersPortalService.list` · `web-app/src/features/lab-portal/pages/LabPortalOrdersPage.tsx`

41.3.5 Open an order

Status: [Implemented]

What it does. Shows one order the way the lab needs it: items, teeth, form answers, revisions, messages and shared files, plus the actions the lab may take.

How the user does it. Click an order in the list. The back button returns to where the user came from.

Behind the scenes — in simple terms. The lab sees the lab cost, never the patient price. Internal files of the clinic and files marked only for the patient are hidden.

Behind the scenes — technically. `GET /lab-portal/orders/:id` → `toLabOrder` (the lab mapper) over the loaded order bundle; `GET /lab-portal/orders/:id/actions` lists the actions the lab may take now (accept, reject, request clarification, change expected date) computed from the pinned workflow and the member's role.

Rules and validations. An order of another lab, or a draft, answers 404 `LAB_ORDER_NOT_FOUND`.

Who can do it. Any lab member.

Data read and written. Reads the order tables, `attachments`, `order_revisions`, `order_messages`.

Effects on other modules. None.

Error cases. 404 `LAB_ORDER_NOT_FOUND`.

Technical references: `backend/src/lab-orders/mappers/order-response.ts` · `toLabOrder` · `web-app/src/features/lab-portal/pages/LabPortalOrderDetailPage.tsx`

41.3.6 Accept an order with an expected date

Status: [Implemented]

What it does. The lab accepts a submitted order and promises a ready date. The clinic's doctor is told. The case now enters production.

How the user does it.

1. Open the order and press **Accept**.
2. Enter "Ready on" and optionally "Note for the clinic", then confirm.

Behind the scenes — in simple terms. Every item waiting for the lab is accepted in one step. The Platform creates the try-in stages of each item from its stage template, builds the draft bill the clinic will later approve (from the items' frozen lab costs), and re-derives the order to Accepted.

Behind the scenes — technically. `POST /lab-portal/orders/:id/accept` (body `labExpectedOn`, optional `note`, optional `version`) → `LabOrderActionsService.accept`, one transaction: row-lock the order and check the version; the order must be submitted (`LAB_ACTION_NOT_ALLOWED` otherwise); the date must not be past in the location's clinic day (`ClockService`); `UPDATE lab_orders SET lab_expected_on`; for each item in `pending_acceptance`, engine `transitionIn` with action `accept`, required input `lab_expected_on`, actor side lab, `via = portal`; item listeners create `order_item_stages` (`initStages`) and the vendor bill draft (`create_vendor_bill_draft` logic in the payables module), and the order is re-derived; an `audit_logs` row; outbox `order.accepted` to the doctor. Roles come from the pinned workflow (seeded: lab admin only).

Rules and validations.

- Date format YYYY-MM-DD, not in the past (`LAB_EXPECTED_DATE_INVALID`).
- Only a lab admin may accept in the seeded workflow; a technician does not see the button.
- "No line is waiting for acceptance" if nothing is pending.

Who can do it. Lab admin (seeded workflow role). No permission key.

Data read and written. `lab_orders`, `lab_order_items`, `order_item_stages`, `vendor_bills`, `vendor_bill_lines`, `status_history`, `audit_logs`, `notification_outbox`.

Effects on other modules. The clinic sees the order as Accepted with the date; a vendor bill draft appears in the clinic's lab payables (Chapter 42).

Error cases. 422 `LAB_ACTION_NOT_ALLOWED`, `LAB_EXPECTED_DATE_INVALID`, `TRANSITION_INPUT_REQUIRED`; 403 `TRANSITION_FORBIDDEN`; 409 `STALE_VERSION`.

Technical references: `backend/src/lab-access/actions/lab-order-actions.service.ts` · `LabOrderActionsService.accept` · `web-app/src/features/lab-portal/components/LabOrderActions.tsx`

41.3.7 Reject an order

Status: [Implemented]

What it does. The lab refuses the whole order with a reason. The order becomes "Rejected by lab" and the doctor is told; the clinic may reopen it as a draft.

How the user does it. Press **Reject**, type the "Reason" and confirm.

Behind the scenes — in simple terms. Only the order changes status. Its items stay as they are.

Behind the scenes — technically. `POST /lab-portal/orders/:id/reject` → `LabOrderActionsService.reject`: engine transition `reject` on the order (reason required), then outbox `order.rejected`. Closing the order revokes its magic links.

Rules and validations. Reason required, at most 1,000 characters; the order must be submitted.

Who can do it. Lab admin (seeded workflow role).

Data read and written. `lab_orders`, `status_history`, `audit_logs`, `magic_links`, `notification_outbox`.

Effects on other modules. The clinic can reopen the order (Chapter 40).

Error cases. 422 `INVALID_TRANSITION`, `TRANSITION_REASON_REQUIRED`; 403 `TRANSITION_FORBIDDEN`.

Technical references: `backend/src/lab-access/actions/lab-order-actions.service.ts` · `LabOrderActionsService.reject` · `web-app/src/features/lab-portal/components/LabOrderActions.tsx`

41.3.8 Ask the clinic a question

Status: [Implemented]

What it does. The lab asks for clarification. The order moves to "Clarification needed" until the clinic replies or revises it.

How the user does it. Press **Ask a question**, type the "Question" (optionally about one item) and send.

Behind the scenes — in simple terms. The question is stored as a message in the order's thread and the doctor gets a notice with the first part of the text.

Behind the scenes — technically. `POST /lab-portal/orders/:id/clarification` → `LabOrderActionsService.clarify`: the status transition `request_clarification` first (so a wrong state writes no message), then `order_messages` (kind clarification request, author side lab), audit, and outbox `order.clarification_requested` with the question shortened to 300 characters. A message with kind `clarification_request` through `POST /lab-portal/orders/:id/messages` also moves the status.

Rules and validations. Body at most 4,000 characters; the order must be submitted; the item, if given, must belong to the order.

Who can do it. Lab admin (seeded workflow role).

Data read and written. `order_messages`, `lab_orders`, `status_history`, `audit_logs`, `notification_outbox`.

Effects on other modules. None.

Error cases. 404 `LAB_ORDER_ITEM_NOT_FOUND`; 422 `INVALID_TRANSITION`.

Technical references: `backend/src/lab-access/actions/lab-order-actions.service.ts` · `LabOrderActionsService.clarify` · `web-app/src/features/lab-portal/components/LabOrderActions.tsx`

41.3.9 Change the expected date

Status: [Implemented]

What it does. When the work will be late, the lab moves the expected date and says why. The doctor is told the old date, the new date and the reason.

How the user does it. Press **Change date**, choose the new date, type the reason and confirm.

Behind the scenes — in simple terms. Only orders that are already accepted and not yet finished can change date.

Behind the scenes — technically. `POST /lab-portal/orders/:id/expected-date` (`@LabRoles(lab_admin)`) → `LabOrderActionsService.changeExpectedDate`: order lock and version check, allowed statuses accepted, in progress, partially ready, on hold; refuses the same date; refuses a past date; updates `lab_expected_on`; audit; outbox `order.expected_date_changed`.

Rules and validations. Reason required (at most 1,000 characters). Date in `YYYY-MM-DD`, not in the past. Not allowed in other statuses (`LAB_ACTION_NOT_ALLOWED`).

Who can do it. Lab admin.

Data read and written. `lab_orders`, `audit_logs`, `notification_outbox`.

Effects on other modules. The clinic may see a warning when the date falls after the patient's next appointment.

Error cases. 422 `LAB_ACTION_NOT_ALLOWED`, `LAB_EXPECTED_DATE_INVALID`; 403 `FORBIDDEN`.

Technical references: `backend/src/lab-access/controllers/lab-portal-order-actions.controller.ts` · `LabOrderActionsService.changeExpectedDate` · `web-app/src/features/lab-portal/components/LabOrderActions.tsx`

41.3.10 Write a note to the clinic

Status: [Implemented]

What it does. Adds a plain note to the order thread that the clinic can read.

How the user does it. In **Messages** type in "Write to the clinic..." and send.

Behind the scenes — in simple terms. A note does not change the order's status; a question does.

Behind the scenes — technically. `POST /lab-portal/orders/:id/messages` (kind `note`) → `LabOrdersService.labMessage`: lock, closed orders refused, optional item check, insert into `order_messages` (author side lab), audit.

Rules and validations. Replies come only from the clinic (`LAB_ORDER_MESSAGE_INVALID`). Closed orders take no messages.

Who can do it. Any lab member.

Data read and written. `order_messages`, `audit_logs`.

Effects on other modules. The doctor sees it in the order's Messages section.

Error cases. 422 `LAB_ORDER_MESSAGE_INVALID`, `LAB_ORDER_INVALID`.

Technical references: `backend/src/lab-orders/services/lab-orders.service.ts` · `LabOrdersService.labMessage` · `web-app/src/features/lab-portal/pages/LabPortalOrderDetailPage.tsx`

41.3.11 Acknowledge a revision

Status: [Implemented]

What it does. When the clinic revises a sent order, the lab confirms that it read the new version. The clinic sees "Read by lab" or "Not read by lab yet".

How the user does it. In **Revisions** press the acknowledge button next to a revision marked "New".

Behind the scenes — in simple terms. Pressing it twice changes nothing the second time.

Behind the scenes — technically. `POST /lab-portal/orders/:id/revisions/:revisionNo/acknowledge` → `LabOrdersService.acknowledgeRevision`: row-lock the revision, set `lab_acknowledged_at` and `lab_acknowledged_by` once, audit.

Rules and validations. Revision must exist (`LAB_ORDER_REVISION_NOT_FOUND`).

Who can do it. Any lab member.

Data read and written. `order_revisions`, `audit_logs`.

Effects on other modules. Shown on the clinic order page.

Error cases. 404 `LAB_ORDER_REVISION_NOT_FOUND`.

Technical references: `backend/src/lab-orders/services/lab-orders.service.ts` · `LabOrdersService.acknowledgeRevision` · `web-app/src/features/lab-portal/components/LabOrderView.tsx`

41.3.12 Ask for an extra charge

Status: [Partial]

What it does. The lab asks the clinic to approve an extra charge (rush, material, shipping or other) with an amount and a reason. The portal's order page has no form for it.

How the user does it. Not available in the web app. The clinic can already decide such requests (Chapter 40), but the lab has no button to create one.

Behind the scenes — in simple terms. The API accepts the request; nothing in the portal calls it.

Behind the scenes — technically. `POST /lab-portal/orders/:id/charge-requests` (`@LabRoles(lab_admin)`) → `LabOrdersService.requestCharge` : lock, closed orders refused, optional item check, insert into `order_charge_requests` with status pending, audit. No portal component or API function uses it.

Rules and validations. Amount greater than zero (paisa), reason required, kind one of rush, material, shipping, other.

Who can do it. Lab admin.

Data read and written. `order_charge_requests` , `audit_logs` .

Effects on other modules. An approved item charge raises the item's lab cost and so the vendor bill.

Error cases. 422 `LAB_ORDER_INVALID` ; 404 `LAB_ORDER_ITEM_NOT_FOUND` .

Technical references: `backend/src/lab-orders/controllers/lab-orders-portal.controller.ts` · `LabOrdersService.requestCharge` · none in `web-app/src/features/lab-portal/`

41.3.13 Upload files to an order

Status: [Partial]

What it does. Lets the lab attach a file (for example a photo of a problem) that the clinic can see. There is no upload control in the portal.

How the user does it. Not available in the web app.

Behind the scenes — in simple terms. Anything a lab uploads is shared with the clinic automatically.

Behind the scenes — technically. `POST /lab-portal/orders/:id/attachments` (multipart) → `AttachmentsService.upload` with visibility `lab_shared` , side lab; at most 50 MB.

Rules and validations. `ATTACHMENT_INVALID` for no file or a file over 50 MB.

Who can do it. Any lab member.

Data read and written. `attachments` , private file storage, `audit_logs` .

Effects on other modules. The clinic sees the file in the order's Files section.

Error cases. 422 `ATTACHMENT_INVALID` .

Technical references: `backend/src/lab-orders/controllers/lab-orders-portal.controller.ts` · `AttachmentsService.upload` · none in the portal web code

41.3.14 Download files the clinic shared

Status: [Implemented]

What it does. Opens a scan, photo or document that the clinic marked "shared with the lab".

How the user does it. In **Files**, press the download button (tooltip "Download" and the file name).

Behind the scenes — in simple terms. The download link works for ten minutes. A file that is internal to the clinic does not exist for the lab.

Behind the scenes — technically. `GET /lab-portal/orders/:id/attachments/:attachmentId/url` and `GET /lab-link/:token/attachments/:attachmentId/url` → `AttachmentsService.signedUrl(order, id, labOnly = true)`, which answers `ATTACHMENT_NOT_FOUND` for anything not `lab_shared`.

Rules and validations. Lab-shared files only.

Who can do it. Any lab member, or the holder of a valid magic link.

Data read and written. Reads `attachments`.

Effects on other modules. None.

Error cases. 404 `ATTACHMENT_NOT_FOUND`.

Technical references: `backend/src/lab-orders/attachments/attachments.service.ts` · `AttachmentsService.signedUrl` · `web-app/src/features/lab-portal/components/LabOrderView.tsx`

41.3.15 Run the work queue: start, QC pass, QC fail

Status: [Implemented]

What it does. The lab moves each accepted item through its bench steps. **Start work** begins production; **Send to QC** finishes the work for inspection; **QC pass** marks it ready and tells the doctor (and the patient, if a patient channel exists); **QC fail** sends it back to production with notes.

How the user does it.

1. Open **Production**, tab **Work queue**. Search by order number, clinic or work; filter by status.
2. On a row press the next step: **Start work**, then (for items with try-in stages) **Ship for try-in**, and later **Send to QC**, **QC pass** or the cross button "QC fail — back to production" (then describe what needs fixing in "QC failed").

Behind the scenes — in simple terms. If an item has try-in stages, the lab must ship each try-in and the clinic must approve it before the final stage is at the lab; only then can the item go to QC. The portal shows only the step that is allowed.

Behind the scenes — technically. `GET /lab-portal/production/items`, `POST …/items/:itemId/start`, `…/finish`, `…/qc` (body `result` pass or fail, notes) in `LabProductionController` over `LabWorkService`, each in a transaction through the engine (actor side lab, `via = portal`, optional `version`). `finish` is refused unless the final stage is at the lab (`PRODUCTION_NOT_ALLOWED`: "Send … for try-in first"). `qc` runs `qc_pass` (then notification `item.ready` to the doctor and the patient) or `qc_fail` (reason). Starting moves the item to in production; the item listener starts the first stage round.

Rules and validations. Roles from the pinned item workflow (seeded: lab admin and lab technician). The next-step button follows `nextLabStep` in the web app. Optimistic version checks (409).

Who can do it. Lab admin and lab technician.

Data read and written. `lab_order_items`, `order_item_stages`, `lab_orders` (re-derived), `status_history`, `audit_logs`, `notification_outbox`.

Effects on other modules. The clinic sees the order's progress change; the order status is re-derived on every item change.

Error cases. 409 `PRODUCTION_NOT_ALLOWED`, `STALE_VERSION`; 422 `INVALID_TRANSITION`; 404 `PRODUCTION_ITEM_NOT_FOUND`.

Technical references: `backend/src/lab-production/services/lab-work.service.ts` · `LabWorkService.start`, `finish`, `qc` · `web-app/src/features/lab-portal/production/LabProductionPage.tsx`

41.3.16 Assign a technician

Status: [Implemented]

What it does. A lab admin assigns an item to a technician of the lab. Reassigning closes the old assignment and opens a new one, so the history is kept. A call to remove the assignment exists in the API, but the portal has no button for it.

How the user does it. On a work row press the button "Assign technician" and choose a member.

Behind the scenes — in simple terms. The list of choices is the active members of this lab only.

Behind the scenes — technically. `GET /lab-portal/production/technicians`, `POST …/items/:itemId/assign` and `…/unassign` (`@LabRoles(lab_admin)`) → `LabWorkService.assign`, `unassign`: item row lock, closed items refused, member must be active in this lab, previous open `item_assignments` row closed (`unassigned_at`), new row inserted, audit. A partial unique index allows one open assignment per item.

Rules and validations. "Technician not found in this lab" for a member of another lab; closed items (delivered, cancelled, rejected, remake required) cannot be assigned.

Who can do it. Lab admin.

Data read and written. `item_assignments`, `lab_members`, `audit_logs`.

Effects on other modules. None.

Error cases. 404 `PRODUCTION_MEMBER_NOT_FOUND`; 409 `PRODUCTION_NOT_ALLOWED`.

Technical references: `backend/src/lab-production/services/lab-work.service.ts` · `LabWorkService.assign` · `web-app/src/features/lab-portal/production/LabProductionPage.tsx`

41.3.17 Ship a try-in or finished work to the clinic

Status: [Implemented]

What it does. Creates a shipment to the clinic for one or several items of one clinic, prints a slip with a QR code, and moves the work to the clinic's side.

How the user does it.

1. On a work row press **Ship for try-in** or **Ship**.
2. In the dialog add the courier, the tracking number and notes and confirm. A shipment number (for example SHP-0001) is created.
3. In the **Shipments** tab press the print button (tooltip "Print slip with QR") to print the slip.

Behind the scenes — in simple terms. A try-in round is shipped while the item is in production; finished work can only be shipped after it passed QC. The clinic is told "stage sent to clinic" for every line.

Behind the scenes — technically. `POST /lab-portal/production/shipments`, `GET …/shipments`, `GET …/shipments/:id` → `ShipmentsService.dispatchToClinic`, one transaction: items must belong to this lab and one clinic and not repeat; per line `planToClinic` decides the stage and item actions (stage at lab, final stage requires

item ready; item `send_tryin` or `dispatch`, stage `dispatch`); a shipment row with number from the `SHP` sequence, courier and tracking, a random 16-byte hex QR token (unique platform-wide), and `shipment_items`; audit row; outbox `stage.sent_to_clinic` per line to the doctor. Up to 100 lines, quantity 1 to 64.

Rules and validations. "finished work must pass QC before it ships"; "no stage is in progress"; re-shipping work whose first shipment was lost or reported missing needs no status change.

Who can do it. Lab admin and lab technician (the status changes are made by the system on the lab's behalf).

Data read and written. `shipments`, `shipment_items`, `order_item_stages`, `lab_order_items`, `status_history`, `audit_logs`, `number_sequences`, `notification_outbox`.

Effects on other modules. The clinic receives the parcel (Chapter 40).

Error cases. 409 `PRODUCTION_NOT_ALLOWED`; 422 `PRODUCTION_INVALID`.

Technical references: `backend/src/lab-production/services/shipments.service.ts` · `ShipmentsService.dispatchToClinic` · `web-app/src/features/lab-portal/production/print-shipment-slip.tsx`

41.3.18 Receive what the clinic sent

Status: [Implemented]

What it does. Confirms that an impression, model or returned work sent by the clinic arrived.

How the user does it. In **Shipments**, on a shipment from a clinic that is dispatched or partially received, press **Receive**. The toast says "Received from the clinic".

Behind the scenes — in simple terms. The portal records every line as received in full and in good condition. A finer per-line receive (damage, partial quantities) exists in the API but not in the portal screen.

Behind the scenes — technically. `POST /lab-portal/production/shipments/:id/receive` → `ShipmentsService.receive` with a lab caller (direction must be to the lab). The portal sends each line with its full quantity and condition OK. No item or stage status changes for shipments to the lab.

Rules and validations. Only the receiving side can receive; lines already received cannot change.

Who can do it. Lab admin and lab technician.

Data read and written. `shipments`, `shipment_items`, `audit_logs`.

Effects on other modules. The clinic's record shows the shipment as received.

Error cases. 409 `PRODUCTION_NOT_ALLOWED`; 404 `PRODUCTION_SHIPMENT_NOT_FOUND`.

Technical references: `backend/src/lab-production/services/shipments.service.ts` · `ShipmentsService.receive` · `web-app/src/features/lab-portal/production/LabProductionPage.tsx`

41.3.19 Report a shipment lost

Status: [Implemented]

What it does. Marks an open shipment lost with a reason. Open lines become missing, the work stays owed so the lab can ship it again, and the doctor is told.

How the user does it. In **Shipments** press the lost button (tooltip "Mark lost"), type the reason and confirm "Mark … lost".

Behind the scenes — in simple terms. Either side may report a loss. A new shipment can then carry the same item.

Behind the scenes — technically. `POST /lab-portal/production/shipments/:id/lost` → `ShipmentsService.markLost`: lock, open status only, open lines set to missing with quantity 0, status lost with time and reason, audit, outbox `shipment.lost` for the lab and for the doctor of each order.

Rules and validations. Reason required (at most 2,000 characters); only dispatched or partially received shipments.

Who can do it. Lab admin and lab technician.

Data read and written. `shipments`, `shipment_items`, `audit_logs`, `notification_outbox`.

Effects on other modules. None beyond notifications.

Error cases. 409 `PRODUCTION_NOT_ALLOWED`; 404 `PRODUCTION_SHIPMENT_NOT_FOUND`.

Technical references: `backend/src/lab-production/services/shipments.service.ts` · `ShipmentsService.markLost` · `web-app/src/features/lab-portal/production/LabProductionPage.tsx`

41.3.20 Dispute a remake request

Status: [Implemented]

What it does. When the clinic asks for a remake and the lab disagrees, the lab states why. The clinic then decides who is responsible and who pays (Chapter 40).

How the user does it. In **Remakes** press the dispute button, write the reason in "Dispute remake request" ("Tell the clinic why the lab isn't responsible. They decide who pays.") and confirm.

Behind the scenes — in simple terms. A dispute does not block the clinic: it only records the lab's note and changes the request's status to disputed.

Behind the scenes — technically. `GET /lab-portal/production/remake-requests`, `POST …/remake-requests/:id/dispute` (`@LabRoles(lab_admin)`) → `LabWorkService.dispute`: lock, version check, only a request in status requested, set disputed and `dispute_note`, audit.

Rules and validations. Only requested requests can be disputed (`PRODUCTION_NOT_ALLOWED`).

Who can do it. Lab admin.

Data read and written. `remake_requests`, `audit_logs`.

Effects on other modules. The clinic sees the request as disputed.

Error cases. 404 `PRODUCTION_REMAKE_NOT_FOUND`; 409 `PRODUCTION_NOT_ALLOWED`, `STALE_VERSION`.

Technical references: `backend/src/lab-production/services/lab-work.service.ts` · `LabWorkService.dispute` · `web-app/src/features/lab-portal/production/LabProductionPage.tsx`

41.3.21 Accept, suspend or reactivate a clinic

Status: [Implemented]

What it does. The **Clinics** page lists the clinics that linked the lab. A lab admin can accept an invitation, suspend a clinic with a reason, or reactivate a clinic the lab itself suspended.

How the user does it.

1. Open **Clinics**, filter by status or search by name.
2. Press the accept button for a clinic that is "Awaiting confirmation".
3. To stop working with a clinic, choose suspend, type the reason and confirm. To resume, press reactivate.

Behind the scenes — in simple terms. If the clinic suspended the link, the lab cannot undo it, and the reverse. A lab never sees the clinic's other data.

Behind the scenes — technically. `GET /lab-portal/clinics`, `POST /lab-portal/clinics/:linkId/transitions` (`@LabRoles(lab_admin)`; actions activate, suspend, reactivate) → `LabPortalService.transitionClinic`: locks the link in the lab's scope and calls the shared `LabLinksService.applyTransition` with `via = portal`. The clinic list query returns only the link's id, status, payment terms and version plus the clinic's name and the city of its main active location.

Rules and validations. Same transition table as Chapter 40 (reason required to suspend; only the suspending side reactivates).

Who can do it. Read: any lab member. Change: lab admin.

Data read and written. `lab_clinic_links`, `status_history`, `audit_logs`; reads `organizations`, `locations`.

Effects on other modules. An active link is required for the clinic to order from the lab.

Error cases. 404 `LAB_LINK_NOT_FOUND`; 403 `LAB_LINK_SUSPENDED_BY_OTHER_SIDE`; 422 `INVALID_TRANSITION`, `LAB_LINK_REASON_REQUIRED`.

Technical references: `backend/src/labs/portal/lab-portal.service.ts` · `LabPortalService.transitionClinic` · `web-app/src/features/lab-portal/pages/LabClinicsPage.tsx`

41.3.22 Manage the lab's team

Status: [Implemented]

What it does. A lab admin adds members by account email, changes a member's role and switches members on or off.

How the user does it.

1. Open **Team** (visible to lab admins; technicians see "Lab admins only").
2. Press the add button. In "Add team member" enter "Account email" and "Role" (Lab admin or Technician).
3. Change a role in the selector or flip the active switch.

Behind the scenes — in simple terms. A lab admin cannot change their own membership, so the lab cannot lock itself out by demoting or deactivating itself. The account must already exist; clinic staff accounts cannot join an outside lab.

Behind the scenes — technically. `GET/POST /lab-portal/members`, `PATCH /lab-portal/members/:memberId` (`@LabRoles(lab_admin)`) → `LabPortalService` over the same `LabMembersService` that the super admin uses. `updateMember` refuses the caller's own row (`LAB_MEMBER_SELF_CHANGE`). Add checks the user and the "may join" rules; update locks the member row and checks the version.

Rules and validations. As Chapter 40 (feature "Manage the staff of a lab"): no super admins, no clinic staff or owners for external labs, one membership per lab and user. The last lab admin could still be demoted by another admin; no rule keeps one admin.

Who can do it. Lab admin.

Data read and written. `lab_members`, `audit_logs`; reads `users`.

Effects on other modules. The member list decides who receives the lab's in-app notifications.

Error cases. 404 `LAB_MEMBER_USER_NOT_FOUND`, `LAB_MEMBER_NOT_FOUND`; 409 `LAB_MEMBER_EXISTS`, `LAB_MEMBER_NOT_ALLOWED`, `LAB_MEMBER_SELF_CHANGE`.

Technical references: `backend/src/labs/portal/lab-portal.controller.ts` · `LabPortalService.addMember`, `updateMember` · `web-app/src/features/lab-portal/pages/LabTeamPage.tsx`

41.3.23 View and edit the lab profile

Status: [Implemented]

What it does. Shows the lab's contact details and lets a lab admin change them. The lab's name and type are managed by the platform.

How the user does it. Open **Lab profile**. A lab admin presses the edit button, changes Phone, WhatsApp, Email, City or "Pickup / delivery address" and saves.

Behind the scenes — in simple terms. Clinics see these details on their lab list, and the WhatsApp number, phone and email decide whether a magic link can be sent automatically at submit.

Behind the scenes — technically. `GET /lab-portal/lab`, `PATCH /lab-portal/lab` (`@LabRoles(lab_admin)`) → `LabPortalService.updateContact` → `LabsDirectoryService.update` with a lab-side audit actor (lock, version check, audit diff).

Rules and validations. Only contact fields are accepted; stale versions give 409.

Who can do it. Read: any lab member. Edit: lab admin.

Data read and written. `labs`, `audit_logs`.

Effects on other modules. The contact fields are what clinics and the magic link sender use.

Error cases. 409 `STALE_VERSION`.

Technical references: `backend/src/labs/portal/lab-portal.service.ts` · `LabPortalService.updateContact` · `web-app/src/features/lab-portal/components/LabContactSheet.tsx`

41.3.24 See bills, payments and statements

Status: [Implemented]

What it does. A read-only view of the money the clinics owe or paid the lab: approved vendor bills, vendor payments, and statements. Draft bills are hidden, because a draft is the clinic's working copy.

How the user does it. Open **Billing** and switch between **Bills**, **Payments** and **Statements**; open a bill to see its lines and the totals Total, Paid and Balance.

Behind the scenes — in simple terms. The lab sees only its own bills. It cannot approve, pay, dispute or upload anything here; those actions belong to the clinic (Chapter 42).

Behind the scenes — technically. `GET /lab-portal/payables/bills`, `…/bills/:id`, `…/payments`, `…/statements`, `…/statements/:id` in `LabPortalPayablesController` over `LabPortalPayablesService`, all scoped to the lab; bills with status draft are excluded in the query.

Rules and validations. Read-only; status filters supported.

Who can do it. Any lab member.

Data read and written. Reads `vendor_bills`, `vendor_bill_lines`, `vendor_payments`, `vendor_payment_allocations`, `lab_statements`, `lab_statement_lines`.

Effects on other modules. None.

Error cases. 404 for another lab's record.

Technical references: `backend/src/lab-payables/controllers/lab-portal-payables.controller.ts` · `LabPortalPayablesService` · `web-app/src/features/lab-portal/payables/LabBillingPage.tsx`

41.3.25 Answer an order from a magic link, without logging in

Status: [Implemented]

What it does. A lab contact who received a link opens one order on any phone or computer and can view it, accept it with a ready date, reject it with a reason or ask a question. It needs no account.

How the user does it.

1. Open the link from the message. The page shows the order and the actions the link allows.
2. Press **Accept** (enter "Ready on"), **Reject** (enter a reason) or **Ask a question**. Download a shared file with its button.
3. If the link is dead, the page explains "This link no longer works" and suggests asking the clinic for a new one or signing in to the portal.

Behind the scenes — in simple terms. The long, random address is the only key. It opens exactly one order of one lab, shows what the lab may see and lets the lab do three things. It does not offer the date-change action. The server records every action as made through a magic link, with no user, and counts each use. A link stops by itself after 24 hours (default), when the clinic revokes it, when the order closes or when the lab is suspended.

Behind the scenes — technically. Public routes (`@Public()` and listed in `PUBLIC_ROUTES`) under `/lab-link` : `GET /lab-link/:token` , `POST …/accept` , `…/reject` , `…/clarification` , `GET …/attachments/:attachmentId/url` in `MagicLinkController` , behind `MagicLinkThrottleGuard` (a fixed one-minute window per client IP, limit from the environment variable `LAB_LINK_RATE_LIMIT` , default 120, kept in process memory; over the limit gives 429 `MAGIC_LINK_RATE_LIMITED`). `MagicLinksService.resolve` checks the token shape (43 base64url characters), finds the row by `token_hash` (SHA-256 of the token; the token itself is an HMAC of a random nonce under the secret from `MAGIC_LINK_SECRET`), rejects revoked, expired or used-up links, a closed order or an inactive lab. The view marks `last_viewed_at` and returns the lab mapper's order and the actions from `LabOrderActionsService.available` , limited to accept, reject and request clarification and only for a `lab_update_order` link. Each action runs in one transaction that locks the link, runs the action as a lab admin without a user (`via = magic_link` , `magic_link_id` stored with the status change) and increments `used_count` and `last_used_at` . Default lifetime is 24 hours, or the value of `LAB_MAGIC_LINK_TTL_HOURS` (1 to 168).

Rules and validations.

- Unknown or badly shaped token: 404 `MAGIC_LINK_INVALID` . Expired or used up: 410 `MAGIC_LINK_EXPIRED` . Revoked, order closed or lab inactive: 410 `MAGIC_LINK_REVOKED` .
- A view-only link (`lab_view_order`) cannot act: 403 `MAGIC_LINK_FORBIDDEN` .
- The same business rules as in the portal apply (accept needs a submitted order and a date that is not in the past; reject needs a reason).
- `max_uses` is never set by the code, so links are not limited by count.

Who can do it. Anyone holding a valid link. No permission or feature key applies.

Data read and written. `magic_links` , `lab_orders` , `lab_order_items` , `order_messages` , `status_history` , `audit_logs` , `notification_outbox` .

Effects on other modules. Same effects as the portal: the doctor is notified, stages and the vendor bill draft are created on accept.

Error cases. 404 `MAGIC_LINK_INVALID` ; 410 `MAGIC_LINK_EXPIRED` , `MAGIC_LINK_REVOKED` ; 403 `MAGIC_LINK_FORBIDDEN` ; 429 `MAGIC_LINK_RATE_LIMITED` .

Technical references: `backend/src/lab-access/magic-links/magic-link.controller.ts` · `MagicLinkController`, `MagicLinksService.resolve` · `web-app/src/features/lab-portal/pages/MagicLinkOrderPage.tsx` · route `web-app/src/routes/lab-link.$token.tsx`

41.3.26 Open an order from the printed QR code

Status: [Implemented]

What it does. The QR code on the printed work order opens the right page for whoever scans it while signed in: a lab member lands on the portal order (and the active lab is switched to the order's lab), clinic staff land on the clinic order (and the organization and location are switched if needed). Anyone else sees "Order not found".

How the user does it. Scan the QR code with a phone that is signed in, or type the address.

Behind the scenes — in simple terms. The answer for an outsider is exactly the same as for an order that does not exist, so the code leaks nothing.

Behind the scenes — technically. Route `/o/$orderId` renders `OrderQrPage`, which calls `GET /order-links/:orderId` (`@ContextFree`, signed-in users). The controller answers `target: lab` (for a lab member of the order's lab when the order is not a draft) or `target: clinic` (owner, or active assignment at the order's organization), else 404 `ORDER_LINK_NOT_FOUND`. The page then calls `setActiveLab` or `POST /auth/switch-location` and navigates.

Rules and validations. Needs a signed-in user (the route is not among the open paths, so the app asks for login first).

Who can do it. Any signed-in lab member of the order's lab, or clinic staff of the ordering clinic.

Data read and written. Reads `lab_orders`, `lab_members`, `organization_ownership`, `employee_org_location_assignments`.

Effects on other modules. None.

Error cases. 404 `ORDER_LINK_NOT_FOUND`.

Technical references: `backend/src/lab-access/controllers/order-link.controller.ts` · `OrderLinkController.resolve` · `web-app/src/features/lab-portal/pages/OrderQrPage.tsx` · route `web-app/src/routes/o.$orderId.tsx`

41.3.27 Be told about new work

Status: [Partial]

What it does. Lab members get in-app notifications for `order.submitted`, `shipment.received`, `shipment.lost`, `stage.approved`, `stage.correction`, `remake.requested` and `remake.agreed`. The lab contact gets the magic link message (`order.magic_link`).

How the user does it. Nothing to do. This review found no notification inbox inside the portal layout (a user who is also clinic staff sees the inbox in the clinic header).

Behind the scenes — in simple terms. The notification exists in the database for the lab member's user account, with a link into the portal order, but the portal itself does not show a bell or a list. The message to a lab contact has the channels WhatsApp, SMS or email, and the Platform has no real provider for them yet, so those are written to a development log.

Behind the scenes — technically. `orderRecipients` finds the lab's active members and the ordering doctor and builds the link `/lab-portal/orders/:id`; messages are queued with `NotificationService.enqueue` in the business transaction. The in-app provider writes `user_notifications`; the provider registered for WhatsApp, SMS

and email is `DevLogChannelProvider`. The full list of events and emitters is in Chapter 40 (feature "Notifications about lab work").

Rules and validations. A message with no recipient is skipped.

Who can do it. Automatic.

Data read and written. `notification_outbox`, `user_notifications`.

Effects on other modules. Chapter 44.

Error cases. None shown.

Technical references: `backend/src/lab-orders/workflow/order-recipients.ts` · `orderRecipients` · `web-app/src/features/lab-portal/components/LabPortalLayout.tsx` (no inbox)

41.4 Entities and data model

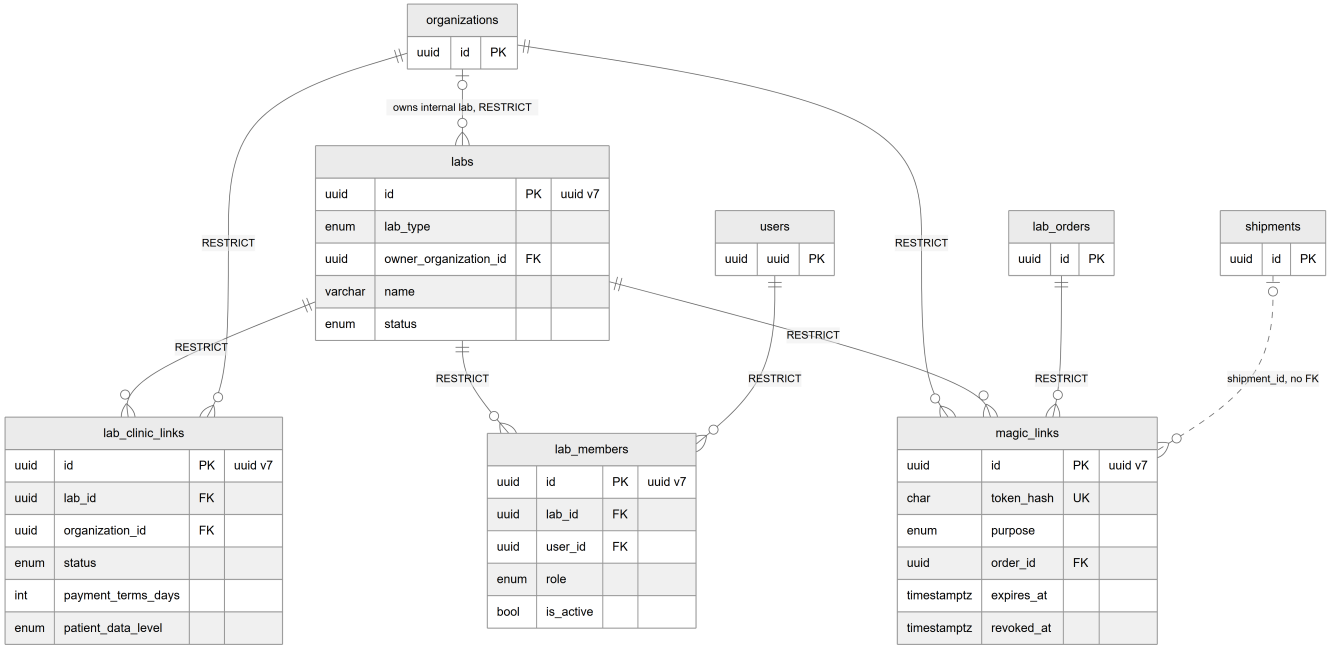

Figure 41.11 — The tables that identify the lab side: labs, the clinic link, lab members and magic links.

The diagram shows `labs` with solid lines to `lab_clinic_links`, `lab_members` and `magic_links`, all ON DELETE RESTRICT. `organizations` connects to `labs` ("owns internal lab"), to `lab_clinic_links` and to `magic_links`. `users` connects to `lab_members`, and `lab_orders` connects to `magic_links`. A dotted line from `shipments` to `magic_links` is labelled "shipment_id, no FK". All four main boxes use UUID version 7 keys, and `token_hash` on `magic_links` is unique.

The relationships in words:

- A lab is not a tenant and has no organization column. An in-house lab records its owner organization; an external lab has none.
- Each lab link belongs to exactly one lab and one organization, one link per pair. It carries `status` (invited, active, suspended), `payment_terms_days` and `patient_data_level` (full name or code only).
- Each lab member belongs to exactly one lab and one user, one membership per pair, with `role` lab admin or lab technician and `is_active` as a soft delete.
- Each magic link belongs to one organization, one lab and one lab order and stores `token_hash` (never the token), a random `nonce`, `purpose`, `delivery`, `expires_at`, `max_uses`, `used_count` and the revocation

columns.

- Lab-side data is read through columns that copy the lab id (`lab_orders.lab_id` , `order_item_stages.lab_id` , `shipments.lab_id` , `remake_requests.lab_id` , `item_assignments.lab_id`) so the portal can filter every query by the caller's lab.
- The shipment link on magic links is unused: clinics receive by the shipment's QR token, so the `receive_shipment` purpose has no creator.

The order, item, stage, shipment and remake tables are shown in Chapter 40 and the vendor bill tables in Chapter 42. Every one of them uses ON DELETE RESTRICT, and no migration file exists for these tables (development schema sync, reviewed schema sync in production).

41.5 Status lifecycles

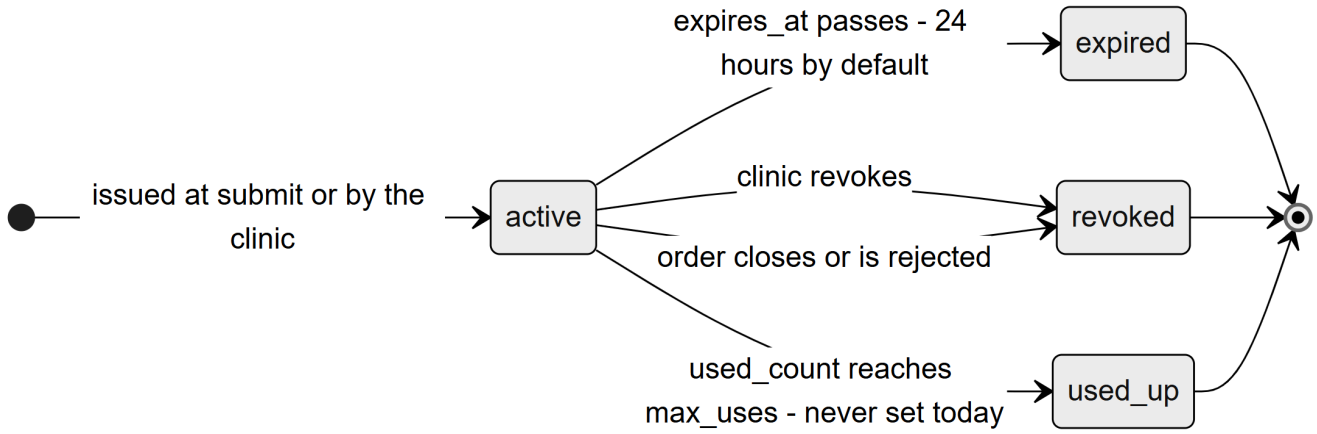

Figure 41.12 — The states of a magic link, which are computed from its columns and not stored.

The diagram has four states. A link enters `active` when it is issued, either at order submit or on demand by the clinic. From `active` it goes to `expired` when `expires_at` passes (24 hours by default), to `revoked` when the clinic revokes it, to `revoked` as well when the order is closed or rejected, and to `used_up` when `used_count` reaches `max_uses` . `expired` , `revoked` and `used_up` are final. No code sets `max_uses` , so `used_up` never happens today.

Walking through the states: **active** — opens the order and allows the actions of its purpose; **expired** — the time ran out, the request answers 410 "This link has expired"; **revoked** — withdrawn by the clinic or closed by the order's closing, 410 "This link was withdrawn"; **used up** — reserved for a use limit. The state is computed when the link is read (`linkState`), except revocation, which is stored in `revoked_at` and `revoked_reason` .

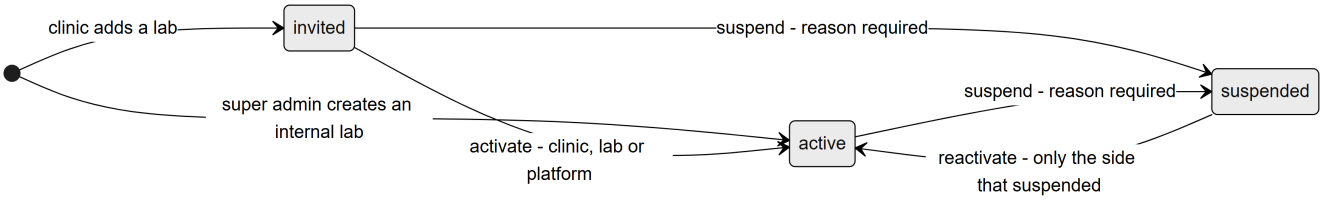

Figure 41.13 — The lab link status values (the same diagram as in Chapter 40).

The diagram shows a link entering at `invited` (a clinic adds a lab) or directly at `active` (an in-house lab). `invited` goes to `active` on "activate" or to `suspended` on "suspend" with a reason. `active` goes to `suspended` on "suspend". `suspended` returns to `active` on "reactivate", only by the side that suspended it. In the portal a lab admin uses the accept, suspend and reactivate buttons of the **Clinics** page for the lab's side of these changes.

The lab sees the same item, stage, shipment and remake lifecycles as the clinic, from the other side. The table lists what the lab can do in each seeded item state; the clinic's view of the same states is in Chapter 40.

Item state (label)	What the lab can do	Who moves it next
pending_acceptance ("Waiting for lab")	Accept with a date, reject the order, ask a question	Lab admin
accepted	Start work	Lab admin or technician
in_production	Ship a try-in round, or Send to QC	Lab, then the clinic's approval
at_clinic_tryin	Wait; the round is with the clinic	Clinic (try-in decision)
qc	QC pass or QC fail	Lab admin or technician
ready	Ship finished work	Lab admin or technician
dispatched	Wait for receipt	Clinic front desk
received, approved, delivered	Nothing; clinic decisions	Clinic doctor
remake_required	Dispute the remake request	Lab admin, then the clinic

The table shows, for each seeded item state, the lab's available action and who moves the item next. The states are workflow data and a custom published workflow may differ.

41.6 Connections to other modules

- **Dental lab, clinic side (Chapter 40).** Every portal action changes a clinic order; every clinic action changes what the portal offers.
- **Lab finance (Chapter 42).** Accepting an order creates the vendor bill draft; the lab later sees approved bills, payments and statements read-only.
- **Workflow engine (Chapter 43).** All status changes go through the engine; the lab roles (lab admin, lab technician) are matched against the workflow transitions' allowed roles, and a magic link acts as lab admin.
- **Notifications (Chapter 44).** In-app alerts to lab members and link messages to the lab contact.
- **Authentication (Chapter 56).** Lab members use the normal login and session; the session response carries their lab memberships.
- **Access control (Chapter 57).** Lab roles are separate from clinic permissions. The portal uses `@LabRoles` on routes, never `@Can`.
- **Audit (Chapter 60).** Lab-side writes are audited with the actor side "lab" and the route (`portal`, `magic_link`, `proxy`, `qr`) in the status history.

41.7 Limitations and known gaps

- The lab cannot edit its own catalog or prices; platform staff do that.
- No form for extra charges and no upload control in the portal, although the API supports both.
- The portal's receive button always records a full, good-condition receipt; per-line condition and partial quantities are API-only.
- No remove-assignment button, although the API has one.
- No notification inbox inside the portal layout.
- Messages to the lab contact and the patient go to a development logger until a real provider is added.
- A technician cannot accept, reject or ask; those transitions are lab admin only in the seeded workflow.
- A magic link has no use limit and cannot change the expected date.

- The magic link rate limit is per server process.
- The first lab account must be created in User Admin by someone with the right to do so; the portal cannot invite a person who has no account.
- A lab admin can demote or deactivate another admin; nothing keeps at least one admin.
- A lab that serves several clinics sees each clinic's orders in one list; there is no per-clinic filter other than search.

41.8 Owner's FAQ

How does a lab person log in? With the same login page as everybody else, with an account that a super admin or a lab admin added to the lab. A user with only lab memberships goes straight to the lab portal.

Can a lab see our patients' names? Only if the lab link's "Patient details shared" is "Full patient name". With "Patient code only" the lab sees the patient number. The setting is per clinic and lab.

Can a lab see what we charge patients? No. Lab responses carry the lab cost only and omit the patient price, the clinic's treatment and internal files.

Is a magic link safe to forward? Anyone holding the link can view and answer that one order until it expires, is revoked, or the order closes. The default life is 24 hours, and the clinic can revoke it. The secret part is never stored in the database.

What if the lab does not answer a submitted order? A clinic coordinator can record the lab's answer on its behalf (Chapter 40). Otherwise the order stays Submitted.

Can a technician accept an order? No. In the seeded workflow only a lab admin accepts, rejects or asks. A technician starts work, sends it to QC, passes or fails QC and ships.

Can a lab work for several clinics? Yes. A lab is shared by clinics; each clinic has its own lab link and its own prices. A person can also be a member of several labs and switch with the lab switcher.

What happens when a lab is suspended on the platform? Its members lose portal access, its magic links stop working, new links and activations are blocked, and orders already sent are not changed.

Where is lab activity recorded? Every status change goes to `status_history` with the actor side and route, and every edit to `audit_logs`.

Chapter 42 — Lab Finance

This chapter describes the money side of lab work in the Platform: what a patient owes the clinic for lab work, and what the clinic owes the lab. It is written for the product owner, who needs to know what is built, which rules protect the money and where the gaps are. Lab Finance sits on top of the Lab Management module (Chapter 40), which creates lab orders and freezes their prices, and it feeds the commission feature (Chapter 35) and the double-entry journals.

42.1 What this module is for

Lab Finance is three backend modules that work as one flow.

- **Lab billing** (`lab-billing`) handles the **patient lab invoice**: an estimate that becomes an invoice, payments, advances, credit notes, refunds and reminders. It is kept apart from the clinic's own billing (Chapter 34), with its own tables and numbers.
- **Lab payables** (`lab-payables`) handles what the clinic owes each **lab**: the **vendor bill**, payments to the lab, the monthly **lab statement** and three reports.
- **Lab finance glue** (`lab-finance`) has no screen and no endpoint. It listens to lab billing events and, in the same database transaction, records the doctor's lab commission and the double-entry journal lines.

Who uses it:

- **Accountant**: the main user. The default Accountant system role holds both lab permissions and the two billing permissions needed to undo money.
- **Admin** and the **owner**: everything.
- **Receptionist, Assistant, Doctor**: no lab finance permissions by default. A doctor can still see the lab order, but not the Billing panel's actions.
- **Lab admin** (a **lab member**): reads the bills, payments and statements that clinics created for their lab, on the lab portal (Chapter 41).

Sidebar: **Lab billing** (`nav.lab.billing` , `/lab-billing` , needs `lab.billing.manage`) and **Lab payables** (`nav.lab.payables` , `/lab-payables` , needs `lab.payables.manage`), both in the Clinic group. The payables sub-pages (payments, statements, reports) have no sidebar entries of their own; they are reached through tabs inside Lab payables. All lab permissions belong to the feature `dental.lab_orders` ("Lab Management", off by default).

42.2 Screens

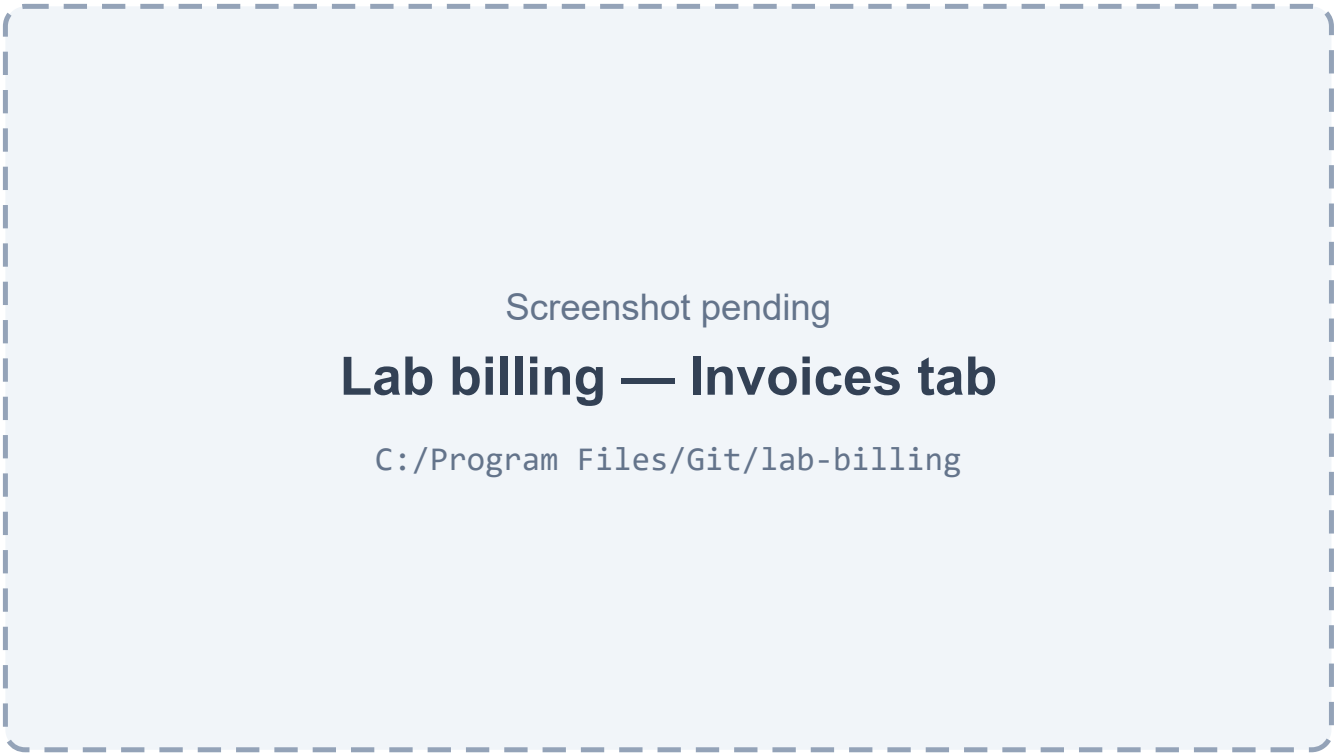

Figure 42.1 — Lab billing, Invoices tab (screenshot pending).

The Lab billing page has the title "Lab billing", the subtitle "What patients are billed and pay for lab work", a **Settings** button (only for users who may edit settings) and two tabs, **Invoices** and **Payments**. Without `lab.billing.manage` the page shows "No access to lab billing". The Invoices tab has a search box ("Search invoice, order or patient…"), a status filter, a From and To date range and a table with the columns Invoice, Patient, Status, Issued, Total and Balance. Drafts show "Estimate" and "Not issued". Overdue invoices carry an "Overdue" badge. The empty state says that work is billed from the Billing panel of a submitted lab order.

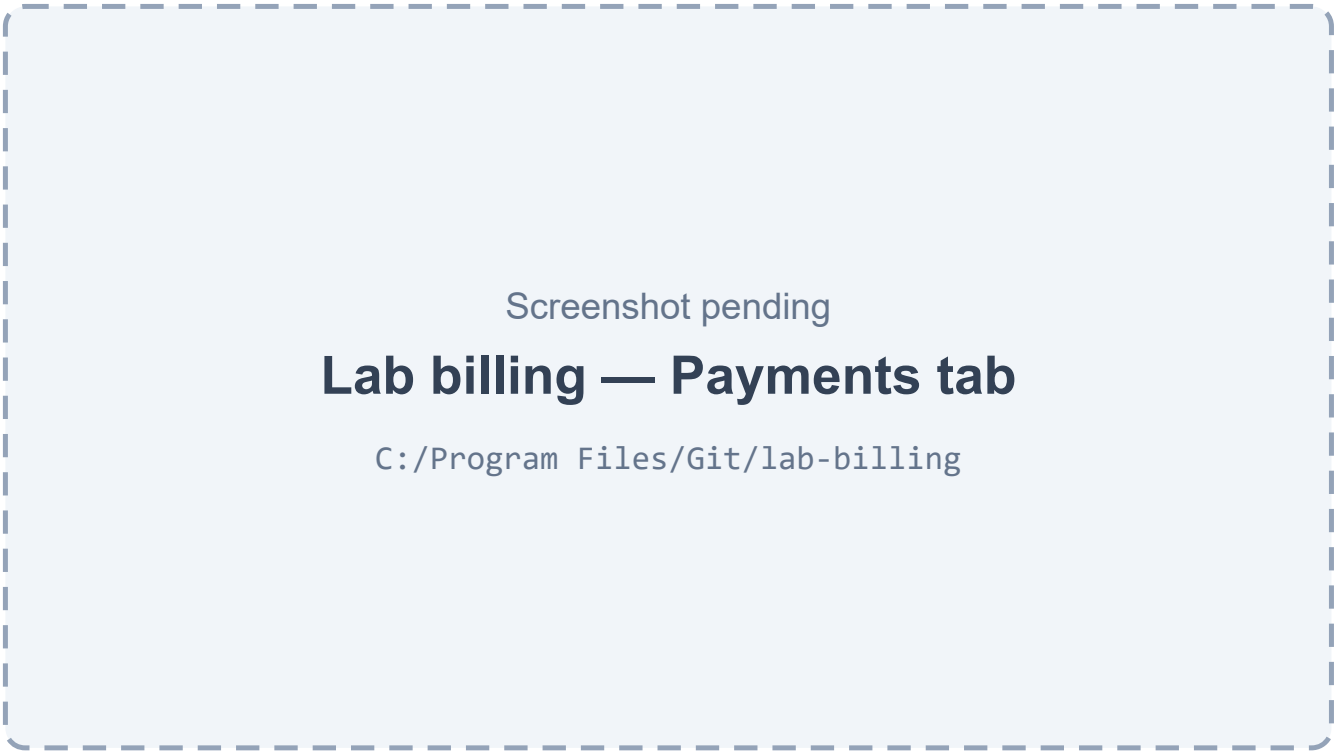

Figure 42.2 — Lab billing, Payments tab (screenshot pending).

The Payments tab lists payments collected on lab invoices, including advances. It has a search box (receipt, patient or reference), method and status filters, a date range and the columns Receipt, Patient, Amount and Status (Posted or Reversed). Reversal rows are marked "Reversal". Row actions are icon buttons with tooltips: "Reverse payment" and "Refund advance".

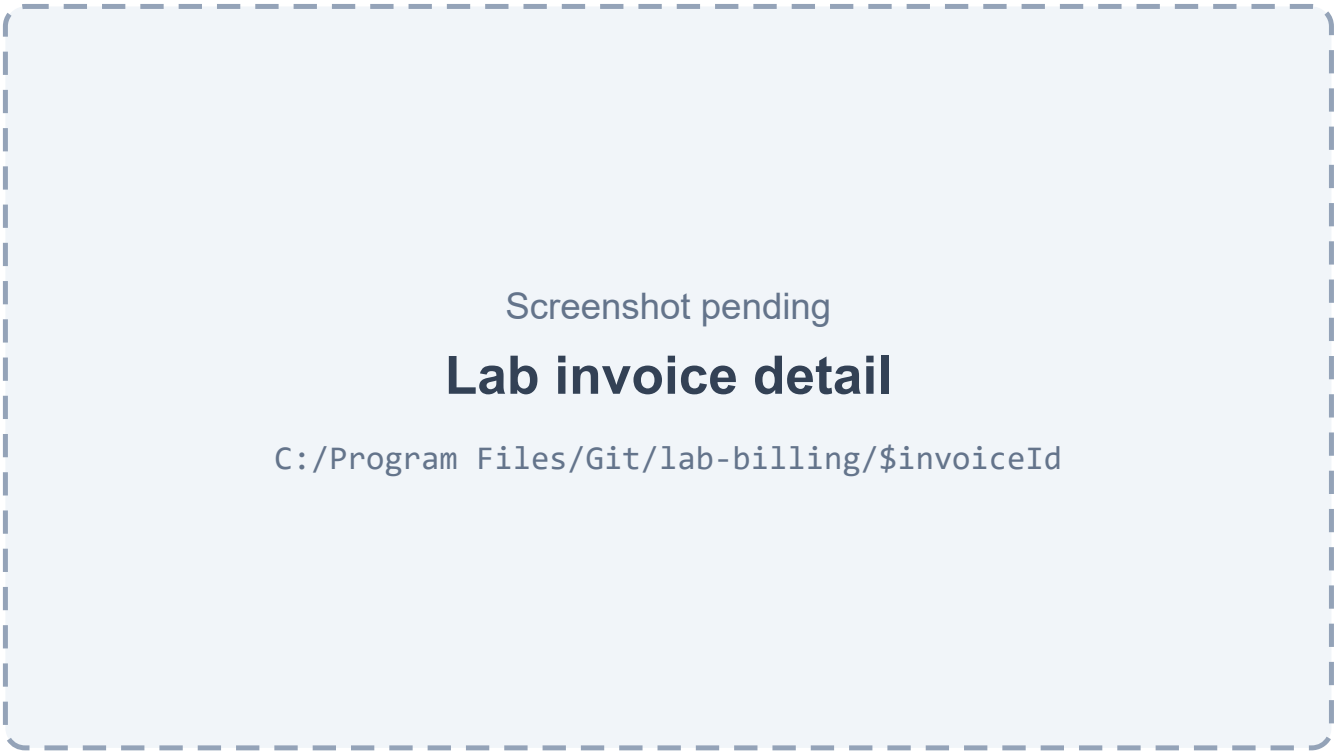

Figure 42.3 — Lab invoice detail (screenshot pending).

The invoice page shows a Summary block (Subtotal, Discount, Tax, Total, Credited, Paid, Advance, Owes), a Lines table (Description, Qty, Price, Tax, Total), Payments, Credit notes and a History timeline. The action bar changes with the status. A draft shows **Edit** and **Issue invoice**. An open invoice shows **Use advance (amount)**, **Send reminder**, **Credit note**, **Void** and **Collect payment**. Credit note and Void need extra permissions.

Screenshot pending

Billing panel on a lab order

`C:/Program Files/Git/lab-orders/$orderId`

Figure 42.4 — Billing panel on a lab order (screenshot pending).

On the lab order detail page (Chapter 40) the Billing panel shows Billed, Paid, Credited, Balance and the patient's advance, lists the order's invoices, and offers **Bill N item(s)** (opens the invoice sheet) and **Collect payment**. This is where most lab invoices start.

Screenshot pending

Lab payables — Bills

`C:/Program Files/Git/lab-payables`

Figure 42.5 — Lab payables, Bills (screenshot pending).

Lab payables has the subtitle "Vendor bills from your labs — review, approve, pay." and a tab strip Bills, Payments, Statements and Reports with a lab filter. The Bills tab has a search box (bill, order or lab invoice number), a status filter, a date range and a table with Bill, Lab, Bill date, Status, Total and Balance. Manual bills are marked "Manual".

Screenshot pending

Vendor bill detail

`C:/Program Files/Git/lab-payables/bills/$billId`

Figure 42.6 — Vendor bill detail (screenshot pending).

The bill page shows Total, Paid and Balance, the lines (Description, Kind, Qty, Unit, Total) and the Payments made against the bill. A draft shows a **Re-read costs from the order** icon button, **Approve** and **Void**. An approved or partly paid bill shows **Pay**.

Screenshot pending

Lab payables — Payments

`C:/Program Files/Git/lab-payables/payments`

Figure 42.7 — Payments to labs (screenshot pending).

The Payments tab ("Payments to labs and advances not yet used.") lists payments with Payment, Amount, Advance left, Paid on and Status, and a **Record payment** button. The button is disabled until a lab is chosen in the filter. Each row has a "Reverse payment" action.

Screenshot pending

Lab payables — Statements

`C:/Program Files/Git/lab-payables/statements`

Figure 42.8 — Lab statements (screenshot pending).

The Statements tab lists statements with Period, Lab says, To check and Status, and an **Add statement** button.

Screenshot pending

Lab statement detail

`C:/Program Files/Git/lab-payables/statements/$statementId`

Figure 42.9 — Reconcile one lab statement (screenshot pending).

The statement page lists each line with Lab reference, Lab says, Our bill, Our bills say and Result (Matched, Amount differs, Not in our bills, Not on statement). Buttons **Dispute** and **Mark reconciled** appear depending on the current status.

Figure 42.10 — Payables reports (screenshot pending).

The Reports tab has three sub-tabs: Payables aging (owed per lab by days overdue), Lab cost by month (with a Year selector) and Case profit (Order, Patient, Invoiced, Collected, Lab cost, Expenses, Commission, Profit).

42.3 Features

42.3.1 Estimate and invoice lab work for a patient

Status: [Implemented]

What it does. Creates a lab invoice from a submitted lab order, or from manual lines. A new invoice starts as a draft, called an estimate: it is editable, has no number and nothing is owed. Issuing it gives it a number and a due date.

How the user does it.

1. Open a submitted lab order and click **Bill N item(s)** in the Billing panel (or create from Lab billing).
2. In the "Bill lab work" sheet, tick the items and charges to bill. Prices are the patient prices fixed when the order was submitted. Optionally add manual lines, a discount and a tax percentage.
3. Click **Save estimate**, or switch on **Issue now** and click **Issue invoice**.
4. Later, open the estimate, click **Edit** to change it, or **Issue invoice**.

Behind the scenes — in simple terms. Each line copies the price the order froze when it was submitted, so a later change to the lab's price list never changes what the patient is billed. A piece of work can be on only one live invoice. Issuing takes the next number in a gapless counter, sets the due date from the clinic's setting and then uses the patient's advance if there is any.

Behind the scenes — technically. `POST /lab-billing/invoices` → `LabInvoicesService.create`; `PATCH /lab-billing/invoices/:id` → `update`; `POST /lab-billing/invoices/:id/issue` → `issue` / `issueIn`. Each runs in one transaction under a row lock on the invoice (`LabBillingLocks`). Three line kinds exist: `lab_item` (an order item at its frozen patient line price; if quantity times unit price differs from the frozen total, one line with quantity

1 is used), `lab_charge` (an approved charge request of the order) and `manual`. Line math (`lineMath`, `spreadDiscount`) is integer paisa with the tax rate in basis points; an order-wide discount is spread over the lines. Issue sets `invoice_no` (sequence `lab_invoice`, prefix `LINV`), `issued_on` (the location's today via `ClockService`), `due_on`, then calls `LabBillingTotals.recomputeInvoices`, writes a status-history row and emits `invoiceIssued`, then `LabAllocations.applyAdvance` and `advanceApplied`. The estimate's update replaces all its lines and bumps `version`. Optimistic locking uses `version`.

Rules and validations.

- A patient or an order is required; an order's patient must match the given patient.
- The order must be submitted (a draft order cannot be billed).
- Items in state cancelled or rejected cannot be billed.
- An item or charge already on another non-void invoice cannot be billed again. The partial unique indexes `uq_lab_invoice_lines_item` and `uq_lab_invoice_lines_charge` decide races.
- Only an approved charge request can be billed.
- A manual line needs a description (1 to 200 characters) and a price.
- At least one line before issue; at most 100 lines; quantity 1 to 1000; tax 0 to 10,000 basis points.
- Only a draft can be edited or issued, and the `version` sent must match.

Who can do it. `lab.billing.manage` (no scope), feature `dental.lab_orders`. Default: Admin and Accountant.

Data read and written. Reads `lab_orders`, `lab_order_items`, `order_charge_requests`, `patients`, `lab_billing_settings`, `number_sequences`. Writes `lab_invoices`, `lab_invoice_lines`, `status_history`, `audit_logs`; on issue also journal and commission rows (42.3.12, 42.3.13).

Effects on other modules. Issuing creates the receivable journal and the commission snapshot. The order's Billing panel updates.

Error cases. `LAB_BILLING_INVALID` (missing patient, empty invoice, draft order); `LAB_BILLING_ITEM_NOT_BILLABLE`; `LAB_BILLING_ALREADY_BILLED`; `LAB_BILLING_INVOICE_NOT_DRAFT` ("Only an estimate can be changed"); `LAB_BILLING_VERSION_CONFLICT` ("Someone else changed this invoice — reload and try again"); `LAB_BILLING_NOT_FOUND`.

Technical references: `backend/src/lab-billing/services/lab-invoices.service.ts` · `LabInvoicesService` · `backend/src/lab-billing/common/lab-billing-math.ts` · `web-app/src/features/lab-billing/components/InvoiceSheet.tsx`

42.3.2 Browse lab invoices and see one in full

Status: [Implemented]

What it does. Lists lab invoices with filters and opens one invoice with its lines, payments, credit notes, refunds and history.

How the user does it.

1. Open **Lab billing**, tab **Invoices**.
2. Search, filter by status (including Overdue), or set a date range.
3. Click a row to open the detail page.

Behind the scenes — in simple terms. The server reads the invoices and works out "overdue" on the fly: an issued or partly paid invoice whose due date is before today in the location's timezone.

Behind the scenes — technically. `GET /lab-billing/invoices` → `LabBillingQueries.listInvoices` (raw SQL, page default 1, limit default 20, maximum 100); `GET /lab-billing/invoices/:id` → `invoice`. The `status=overdue` filter is not a stored status; it expands to `status IN ('issued','partially_paid') AND due_on < today`. Search uses `ILIKE` on invoice number, patient name, patient number and order number.

Rules and validations. Always limited to the active organization. The list is not limited by location.

Who can do it. `lab.billing.manage`. Default: Admin, Accountant.

Data read and written. Reads `lab_invoices`, `lab_invoice_lines`, `lab_payment_allocations`, `lab_payments`, `lab_credit_notes`, `lab_refunds`, `status_history`, `patients`, `lab_orders`. Writes nothing.

Effects on other modules. None.

Error cases. 403 `NO_PERMISSION`; `LAB_BILLING_NOT_FOUND` for an unknown id.

Technical references: `backend/src/lab-billing/services/lab-billing-queries.service.ts` · `LabBillingQueries.listInvoices` / `invoice` · `web-app/src/features/lab-billing/components/InvoicesList.tsx`

42.3.3 Void an estimate or an unpaid invoice

Status: [Implemented]

What it does. Cancels a draft, or an issued invoice that has no payment and no credit note. The invoice stays on record as void and its lab work can be billed again.

How the user does it.

1. Open the invoice and click **Void**.
2. Enter a reason (3 to 500 characters) in the "Void" dialog and confirm.

Behind the scenes — in simple terms. A void is a cancellation, not a deletion. All its lines are marked void so the items are free again. If the invoice had been issued, the receivable entry is reversed.

Behind the scenes — technically. `POST /lab-billing/invoices/:id/void` → `LabInvoicesService.void`: locks the invoice, checks `version`, sets status `void` with reason, time and user, marks all lines `is_void`, writes a status-history row and emits `invoiceVoided` (with `wasIssued`). `LabFinanceListeners` mirrors the receivable journal for an issued invoice.

Rules and validations.

- Refused when payments (`paid_paisa > 0`) or credit notes exist; use a payment reversal or a credit note instead.
- Refused if already void.

Who can do it. `lab.billing.manage` **and** `billing.invoice.void` (both required; the second belongs to feature `billing.invoices`). Default: Admin, Accountant.

Data read and written. Writes `lab_invoices`, `lab_invoice_lines`, `status_history`, `audit_logs`, `ledger_journals`.

Effects on other modules. Receivable reversed in the journals; billed items become billable again.

Error cases. `LAB_BILLING_INVOICE_HAS_MONEY`; `LAB_BILLING_INVOICE_NOT_OPEN` ("This invoice is already void"); `LAB_BILLING_VERSION_CONFLICT`.

Technical references: `backend/src/lab-billing/lab-billing.controller.ts` · `LabInvoicesService.void` · `web-app/src/features/lab-billing/pages/LabInvoiceDetailPage.tsx`

42.3.4 Collect a payment for lab work

Status: [Implemented]

What it does. Records money received from a patient, and splits it over one or more open lab invoices. Whatever is not allocated stays as the patient's **advance**.

How the user does it.

1. Open an invoice (or the order's Billing panel) and click **Collect payment**.
2. Enter Amount, Method (cash, card, bank transfer or wallet), Reference and Notes.
3. Choose "Put it on" (which invoices), or keep "Oldest unpaid invoices first", or keep the money as advance.
4. Confirm. A toast says "Payment received" and a receipt number appears.

Behind the scenes — in simple terms. One click creates one payment with a receipt number. The system locks the invoices so two desks cannot pay the same balance twice, allocates the amounts, recomputes each invoice's paid amount and status, and records the cash in the journals. The page sends a unique key with each click, so a retry after a network error never creates a second payment.

Behind the scenes — technically. `POST /lab-billing/payments` → `LabPaymentsService.collect` (header `Idempotency-Key` required). A replay is found first by `byKey`; the unique index `uq_lab_payments_org_idempotency` settles parallel retries. `runCollect` does, in one transaction: patient check → lock invoices → validate → receipt number (sequence `lab_receipt`, prefix `LRC`) → insert payment → insert `lab_payment_allocations` → `LabBillingTotals.recomputeInvoices` / `recomputePayments` → audit → `paymentCollected` event → `payment.receipt` notification to the outbox. Event listeners post the cash journal and the commission (42.3.12, 42.3.13). Invoice status is written only by `LabBillingTotals` (partially_paid when something is allocated, paid when the balance is zero).

Rules and validations.

- Amount 1 to `MAX_PAISA`, in paisa; at most 50 allocations.
- An invoice may appear once; it must belong to the same patient and be open (issued or partly paid, not a draft or void).
- Each allocation is at most the invoice's balance; the allocations together are at most the payment.
- Without allocations, the payment is spread over the patient's open invoices, oldest first (`autoAllocate`, default true); with `autoAllocate` false the whole amount is advance. Any rest is advance.

Who can do it. `lab.billing.manage`. Default: Admin, Accountant.

Data read and written. Writes `lab_payments`, `lab_payment_allocations`, `lab_invoices` (cached totals and status), `number_sequences`, `audit_logs`, `idempotency` records, `ledger_journals`, `commission_entries`, notification outbox rows. Reads `patients`.

Effects on other modules. Doctor commission (Chapter 35); the patient notification `payment.receipt` is queued but the patient audience is switched off by default, so it is recorded as skipped (Chapter 44).

Error cases. `IDEMPOTENCY_KEY_REQUIRED`; `IDEMPOTENCY_KEY_REUSED` (same key, different body; 422); `LAB_BILLING_INVOICE_NOT_OPEN`; `LAB_BILLING_ALLOCATION_EXCEEDS_BALANCE`; `LAB_BILLING_ALLOCATION_EXCEEDS_PAYMENT`; `LAB_BILLING_INVALID`.

Technical references: `backend/src/lab-billing/services/lab-payments.service.ts` · `LabPaymentsService.collect` / `runCollect` · `web-app/src/features/lab-billing/components/MoneySheets.tsx`

42.3.5 Use a patient's advance on an invoice

Status: [Implemented]

What it does. Puts money the patient already paid in advance onto an open invoice. No new money arrives and no receipt is made.

How the user does it.

1. Open an open invoice. If the patient has advance, the button **Use advance (amount)** appears.
2. Click it. A toast says "Advance applied".

Behind the scenes — in simple terms. The system takes from the patient's oldest advance first, never more than the invoice balance. This also happens automatically when an invoice is issued.

Behind the scenes — technically. `POST /lab-billing/invoices/:id/apply-advance` → `LabPaymentsService.applyAdvance` → `LabAllocations.applyAdvance` (header `Idempotency-Key` required; optional `amountPaisa`). It locks the invoice, then the advance payments (lock order: invoices before payments), inserts allocations, recomputes totals and emits `advanceApplied`.

Rules and validations. The invoice must be open with a balance; the patient must have advance.

Who can do it. `lab.billing.manage`. Default: Admin, Accountant.

Data read and written. Writes `lab_payment_allocations`, `lab_invoices`, `lab_payments` (cached allocated amount), journals and commission.

Effects on other modules. Commission is accrued on the allocated amount; the journal moves Patient Advances to Patient Receivable.

Error cases. `LAB_BILLING_INVOICE_NOT_OPEN`; `LAB_BILLING_NOTHING_TO_APPLY`; `IDEMPOTENCY_KEY_REQUIRED`.

Technical references: `backend/src/lab-billing/services/lab-allocations.service.ts` · `LabAllocations.applyAdvance` · `web-app/src/features/lab-billing/pages/LabInvoiceDetailPage.tsx`

42.3.6 Reverse a lab payment

Status: [Implemented]

What it does. Undoes a wrong payment. The payment stays on record as reversed, a mirror row with a negative amount is added, and the money comes off every invoice it paid.

How the user does it.

1. On the Payments tab, click the "Reverse payment" icon of a posted payment.
2. Enter a reason and confirm. A toast says "Payment reversed".

Behind the scenes — in simple terms. Nothing is deleted. The original is flagged, a negative twin is created with its own number, and the invoices' paid amounts are recalculated. Once any part of a payment was refunded, it cannot be reversed.

Behind the scenes — technically. `POST /lab-billing/payments/:id/reverse` → `LabPaymentsService.reverse` / `runReverse` (header `Idempotency-Key`). The service reads the touched invoice ids, locks them, then the payment; sets the original to `reversed`; inserts the mirror row (`reversal_of_id`, sequence `lab_payment_reversal`, prefix `LRV`); sets `reversed_at` on the allocations with reason `payment_reversed`; recomputes totals; emits `paymentReversed`. The ledger posts the exact opposite of what the payment still holds, not a plain mirror, because a credit note may have moved part of it into advance.

Rules and validations. A reversal cannot be reversed; an already reversed payment cannot be reversed again; a payment with `refunded_paisa > 0` cannot be reversed; reason 3 to 500 characters.

Who can do it. `lab.billing.manage` **and** `billing.refund.issue`. Default: Admin, Accountant.

Data read and written. Writes `lab_payments`, `lab_payment_allocations`, `lab_invoices`, journals, commission entries (negative), audit.

Effects on other modules. Commission earned on the payment is taken back by a negative entry.

Error cases. `LAB_BILLING_PAYMENT_REVERSED`; `LAB_BILLING_PAYMENT_REFUNDED`; `LAB_BILLING_INVALID`; `LAB_BILLING_VERSION_CONFLICT`; `IDEMPOTENCY_KEY_REUSED`.

Technical references: `backend/src/lab-billing/services/lab-payments.service.ts` · `LabPaymentsService.reverse` · `web-app/src/features/lab-billing/components/PaymentsList.tsx`

42.3.7 Issue a credit note

Status: [Implemented]

What it does. Lowers an issued invoice by an amount. If the patient had already paid more than the new total, the excess becomes advance again.

How the user does it.

1. On an issued invoice click **Credit note**.
2. Enter Amount and Reason and confirm. A toast says "Credit note issued".
3. If money was released, the invoice shows "Moved to advance", and a refund can follow (42.3.8).

Behind the scenes — in simple terms. The credit note reduces what the invoice is worth. If the invoice was fully paid, the newest payments are released first and go back into the patient's advance. The credit note remembers how much was released, so it can later be paid out.

Behind the scenes — technically. `POST /lab-billing/invoices/:id/credit-notes` → `LabCreditsService.creditNote` (header `Idempotency-Key`, claim by request hash). Locks the invoice; `creditable = total − credited`; `excess = max(0, amount − balance)`; releases allocations newest first with reason `credit_note`; a row only partly needed stays allocated for the rest; creates the note (sequence `lab_credit_note`, prefix `LCN`) with status `applied` (fully absorbed) or `issued` (waiting for refund); emits `creditNoteIssued`.

Rules and validations. Only issued or partly paid or paid invoices can be credited; the amount is at most what is left to credit; reason 3 to 500 characters.

Who can do it. `lab.billing.manage` **and** `billing.invoice.void`. Default: Admin, Accountant.

Data read and written. Writes `lab_credit_notes`, `lab_payment_allocations`, `lab_invoices`, `lab_payments`, journals, commission entries.

Effects on other modules. Commission is adjusted on the invoice's lines.

Error cases. `LAB_BILLING_INVOICE_NOT_OPEN` ("Only an issued invoice can be credited"); `LAB_BILLING_CREDIT_EXCEEDS_INVOICE`.

Technical references: `backend/src/lab-billing/services/lab-credits.service.ts` · `LabCreditsService.creditNote` · `web-app/src/features/lab-billing/components/MoneySheets.tsx`

42.3.8 Refund a patient's advance

Status: [Implemented]

What it does. Pays money back to the patient out of their advance, optionally as the payout of a credit note.

How the user does it.

1. Click "Refund advance" on a payment row, or choose the refund option next to a credit note.

2. Enter Amount, Method, Reason, Reference and "Paid back by".
3. Confirm "Pay this refund?". A toast says "Refund recorded".

Behind the scenes — in simple terms. The refund is taken from the patient's advance payments, oldest first. A payment that was partly refunded can no longer be reversed.

Behind the scenes — technically. `POST /lab-billing/refunds` → `LabCreditsService.refund` (header `Idempotency-Key`, unique on the refund's own key). It locks the credit note (if any) and the advance payments, writes one `lab_refunds` row (sequence `lab_refund`, prefix `LRF`) and one `lab_refund_sources` row per payment, raises each payment's `refunded_paisa`, updates the credit note's refunded total and status (`refunded` when fully paid out), and emits `refundPaid`. The response carries the patient's updated billing summary.

Rules and validations. The credit note must belong to the patient; the amount is at most what is left on the credit note, and at most the patient's advance.

Who can do it. `lab.billing.manage` **and** `billing.refund.issue`. Default: Admin, Accountant.

Data read and written. Writes `lab_refunds`, `lab_refund_sources`, `lab_payments`, `lab_credit_notes`, journals, audit.

Effects on other modules. Cash or bank goes down in the journals.

Error cases. `LAB_BILLING_REFUND_EXCEEDS_CREDIT`; `LAB_BILLING_REFUND_EXCEEDS_ADVANCE`; `LAB_BILLING_INVALID`; `IDEMPOTENCY_KEY_REUSED`.

Technical references: `backend/src/lab-billing/services/lab-credits.service.ts` · `LabCreditsService.refund` · `web-app/src/features/lab-billing/components/MoneySheets.tsx` (`RefundSheet`)

42.3.9 Send a payment reminder

Status: [Partial]

What it does. Queues a reminder message for the patient about the balance of an invoice.

How the user does it.

1. On an open invoice click **Send reminder**.
2. The toast says "Reminder queued", or "Reminder logged — patient messages are off for this event".

Behind the scenes — in simple terms. The reminder is written to the notification outbox. The Platform sends in-app notifications only, and the patient audience is off by default, so in practice the reminder is recorded as skipped.

Behind the scenes — technically. `POST /lab-billing/invoices/:id/remind` → `LabInvoicesService.remind` → `LabPatientNotifier.reminder` → `NotificationService.enqueue` with event `payment.outstanding_reminder`. The decision order is: patient audience off → `skipped_disabled`; patient opt-out → `skipped_opt_out`; quiet hours → held. The result rows are returned (channel, status, reason). Nothing is sent from lab billing itself.

Rules and validations. The invoice must be open with a balance greater than zero.

Who can do it. `lab.billing.manage`. Default: Admin, Accountant.

Data read and written. Writes notification outbox rows and `audit_logs`.

Effects on other modules. Notifications (Chapter 44).

Error cases. `LAB_BILLING_INVOICE_NOT_OPEN` ("Nothing is owed on this invoice").

Technical references: `backend/src/lab-billing/notifications/patient-notifier.service.ts` · `LabPatientNotifier` · `web-app/src/features/lab-billing/pages/LabInvoiceDetailPage.tsx`

42.3.10 See billing for one order and one patient

Status: [Implemented]

What it does. Shows the Billing panel on a lab order (billed, paid, credited, balance, advance, invoice list) and gives a per-patient summary used by the sheets.

How the user does it. Open a lab order; the panel is at the bottom of the detail page.

Behind the scenes — in simple terms. The numbers are read from the cached amounts on the invoices and payments, which only the billing totals service writes.

Behind the scenes — technically. `GET /lab-billing/orders/:orderId` → `LabBillingQueries.order` (includes which items are still unbilled); `GET /lab-billing/patients/:patientId/summary` → `patientSummary` (invoiced, credited, collected, refunded, advance).

Rules and validations. Organization-scoped.

Who can do it. `lab.billing.manage`.

Data read and written. Reads `lab_orders`, `lab_order_items`, `lab_invoices`, `lab_payments`, `lab_credit_notes`, `lab_refunds`.

Effects on other modules. Shown inside Lab Management (Chapter 40).

Error cases. `LAB_BILLING_NOT_FOUND`.

Technical references: `backend/src/lab-billing/services/lab-billing-queries.service.ts` · `LabBillingQueries.order` / `patientSummary` · `web-app/src/features/lab-billing/components/LabBillingPanel.tsx`

42.3.11 Lab billing settings

Status: [Implemented]

What it does. Sets the default tax rate and the number of days until payment is due for new lab invoices. Issued invoices keep the values they were issued with.

How the user does it.

1. On Lab billing click **Settings**.
2. Set "Tax on lab work (%)" and "Payment due after (days)" and save. A toast says "Settings saved".

Behind the scenes — in simple terms. One settings row per organization. With no row, tax is 0 and due days are 0.

Behind the scenes — technically. `GET /lab-billing/settings` and `PATCH /lab-billing/settings` → `LabInvoicesService.readSettings` / `updateSettings`; saves `lab_billing_settings` and writes an audit row with before and after. Tax is stored in basis points.

Rules and validations. Tax 0 to 10,000 basis points; due days an integer from 0.

Who can do it. Read: `lab.billing.manage`. Change: `lab.billing.manage` **and** `admin.settings.manage`. Default: Admin both; Accountant read only unless the settings permission is added.

Data read and written. `lab_billing_settings`, `audit_logs`.

Effects on other modules. None.

Error cases. 403 `NO_PERMISSION`; 400 for an out-of-range value.

Technical references: `backend/src/lab-billing/lab-billing.controller.ts` · `LabInvoicesService.updateSettings` · `web-app/src/features/lab-billing/components/SettingsSheet.tsx`

42.3.12 Doctor commission on lab work

Status: [Implemented]

What it does. When lab work is invoiced and collected, the ordering doctor earns commission according to their compensation plan. It is recorded as `commission_entries` that settlements and payroll pick up unchanged.

How the user does it. Nothing to click. It runs when an invoice is issued, a payment is collected, an advance is applied, a payment is reversed or a credit note is issued.

Behind the scenes — in simple terms. At issue, each line remembers which doctor, which plan and which rate applied that day. As money is collected, the doctor earns a share proportional to the money collected, after subtracting the lab's cost (so the doctor is paid on the clinic's margin). Reversals and credit notes give commission back.

Behind the scenes — technically. `LabFinanceListeners` (an `OnModuleInit` hook on `LabBillingEvents`) calls `LabCommissionService.snapshot` on `invoiceIssued` and `recordForInvoices` on the money events. The snapshot is skipped when the feature `billing.commissions` is off for the organization. Per line it stores `performing_assignment_id`, `commission_plan_id` and the rate or fixed amount, resolved with `resolveCommissionRate` (lab work is treated as a procedure unless the mapped service says otherwise). `labLineCommissionTarget` (pure function) computes a cumulative target per line; because lab allocations are per invoice, each line's share is proportional to its pre-tax amount. The service inserts the difference between target and what is already recorded, so installments never drift. Lab cost per line comes from approved vendor bill lines, otherwise the order item's frozen snapshot. Source types used: allocation, unapply, charge adjust.

Rules and validations. Only lines with a doctor earn; a doctor with no assignment at the invoice's location or no plan in force on the issue date earns nothing. The commission basis is the "collected" basis net of lab cost.

Who can do it. Automatic; no permission. Commission screens follow Chapter 35.

Data read and written. Reads `employee_org_location_assignments`, compensation plans, `vendor_bill_lines`, `lab_order_items`. Writes `lab_invoice_lines` (snapshot columns), `commission_entries` (with `lab_invoice_line_id`), journals.

Effects on other modules. Commissions (Chapter 35), payroll (Chapter 38).

Error cases. None shown to the user; a failure rolls back the whole money operation.

Technical references: `backend/src/lab-finance/lab-finance.listeners.ts` · `LabFinanceListeners` · `backend/src/lab-finance/lab-commission.service.ts` · `backend/src/lab-finance/lab-commission-math.ts`

42.3.13 Receivables journals (general ledger)

Status: [Partial]

What it does. Posts balanced double-entry journals for every lab money event. Backend only: no screen or report reads these journals.

How the user does it. Nothing; it is automatic.

Behind the scenes — in simple terms. Each event produces a journal with equal debits and credits. A correction is a new mirror journal, never an edit.

Behind the scenes — technically. `LabReceivablesLedger` posts through `GeneralLedgerService` in the operation's own transaction. Patient side: invoice issue debits Patient Receivable and credits Lab Revenue (and Tax Payable); payment debits Cash or Bank and credits Receivable (allocated part) and Patient Advances (rest); advance applied moves Advances to Receivable; credit note debits Lab Revenue and credits Receivable, and moves released money from Receivable to Advances; refund debits Advances and credits Cash or Bank; commission debits Commission Expense and credits Commission Payable; void and reversal post the opposite. Payables side (42.3.15, 42.3.19) uses Lab Cost, Lab Payable and Lab Advances. Tables: `ledger_journals`, `ledger_journal_lines`. Account codes are constants in code, not a table.

Rules and validations. Journals balance; a reversal journal points at the original (`reversal_of_id`, unique); journal lines are never updated or deleted.

Who can do it. Automatic.

Data read and written. Writes `ledger_journals`, `ledger_journal_lines`.

Effects on other modules. None visible. The clinic profit-and-loss report (Chapter 36) does not read these lab journals.

Error cases. An unbalanced journal fails the whole operation.

Technical references: `backend/src/lab-finance/lab-receivables-ledger.service.ts` · `LabReceivablesLedger` · `backend/src/billing/ledger/journal/general-ledger.service.ts`

42.3.14 A vendor bill drafts itself when the lab accepts work

Status: [Implemented]

What it does. As soon as a lab accepts an item of an order, the Platform creates, or updates, one draft vendor bill for that order, priced at the frozen lab costs.

How the user does it. Nothing. The draft appears in **Lab payables** → **Bills**. The empty-state text says a draft appears as soon as a lab accepts an order.

Behind the scenes — in simple terms. The bill is a mirror of what the order says the clinic owes: each accepted item at its lab cost (approved item charges are already inside), plus approved order-level charges. Running it again changes nothing. If a cost rises after approval, the difference goes onto a new draft.

Behind the scenes — technically. `CreateVendorBillDraftEffect` (effect code `create_vendor_bill_draft`, financial) registers an item-transition listener: on every item reaching state `accepted` it calls `BillDraftsService.sync(m, orderId)` inside the transition's transaction. The effect can also be listed in a workflow version; the sync is idempotent. `sync` locks the order, reads expected costs, reads what approved, partly paid or paid bills already hold, locks the open draft, and `planDraftLines` returns inserts and updates. A zero cost (a lab-fault remake) is listed on an open draft for the record but never opens a draft alone. A cost that went down after approval is left to a manual correction. Numbers: sequence `vendor_bill`, prefix `VB`. A system audit row is written on creation. Bill date is the location's today.

Rules and validations. The order must be submitted (have an order number); otherwise the effect fails with `WORKFLOW_MISCONFIGURED`.

Who can do it. Automatic.

Data read and written. Reads `lab_orders`, `lab_order_items`, `order_charge_requests`, remake requests. Writes `vendor_bills`, `vendor_bill_lines`, `audit_logs`.

Effects on other modules. Lab Management's accept action triggers it (Chapters 40 and 41).

Error cases. `WORKFLOW_MISCONFIGURED` for an unsubmitted order.

Technical references: `backend/src/lab-payables/workflow/vendor-bill-draft.effect.ts` · `CreateVendorBillDraftEffect` · `backend/src/lab-payables/services/bill-drafts.service.ts` · `BillDraftsService.sync`

42.3.15 Review, edit, approve and void a vendor bill

Status: [Partial]

What it does. The accountant reviews a draft, re-reads costs from the order, approves it (this makes it a real debt to the lab) or voids it. Header edits and manual lines exist in the API but the screen does not offer them.

How the user does it.

1. Open **Lab payables** → **Bills**, click a draft.
2. Optionally click the **Re-read costs from the order** icon.
3. Click **Approve**; in the sheet set the lab's invoice number, due date and tax %, and see "Total owed".
4. Or click **Void** with a reason.

Behind the scenes — in simple terms. Before approval the bill is re-synced from the order. If anything changed, the approval is refused with a stale-version message, so the accountant approves exactly what they saw. Approval books the cost and the debt in one step. A voided approved bill gets a mirror journal.

Behind the scenes — technically. `GET /lab-payables/bills`, `GET /lab-payables/bills/:id`; `PATCH /lab-payables/bills/:id` (`VendorBillsService.update`: vendor invoice number, bill date, due date, tax in basis points, notes); `POST /lab-payables/bills/:id/lines` (`addLine`: kind, description, quantity, unit cost; creates a manual line); `POST /lab-payables/orders/:orderId/sync` (`syncOrder`); `POST /lab-payables/bills/:id/approve` (`approve`); `POST /lab-payables/bills/:id/void` (`void`). Approve: sync in its own transaction, then one transaction: lock draft and check `version`, recompute totals with `billTotals` (tax rounded half up on integer paisa), refuse total 0, set `due_on` (the given date, else bill date plus the lab link's `payment_terms_days`, default 30), post journal debit Lab Cost / credit Lab Payable, set status `approved`, balance = total, store `journal_id`, audit. Void: refused when `paid_paisa > 0` or already void; a draft is closed without a journal; an approved bill is reversed with a mirror journal. The web screen calls approve, void, sync, list and read; `updateBill` and `addLine` exist in `lab-payables.api.ts` but no page uses them.

Rules and validations.

- Only a draft can be edited, have lines added or be approved.
- A bill of 0 cannot be approved ("Void it instead").
- The lab's invoice number is unique per lab among bills (`uq_vendor_bills_vendor_invoice`).
- Line kinds: service, rush, shipping, material, remake, other.
- `version` must match on every write.

Who can do it. `lab.payables.manage` (class-level on the controller), feature `dental.lab_orders`. Default: Admin, Accountant.

Data read and written. Reads `vendor_bills`, `vendor_bill_lines`, `lab_orders`, `lab_clinic_links`. Writes `vendor_bills`, `vendor_bill_lines`, `ledger_journals`, `audit_logs`.

Effects on other modules. Lab cost enters the journals; the case-profit report reads bill totals.

Error cases. `PAYABLES_BILL_NOT_FOUND`; `PAYABLES_INVALID`; `PAYABLES_NOT_ALLOWED` ("Reverse the payments on this bill before voiding it"); `PAYABLES_DUPLICATE_INVOICE`; stale `version` error.

Technical references: `backend/src/lab-payables/services/vendor-bills.service.ts` · `VendorBillsService` · `web-app/src/features/lab-payables/pages/VendorBillDetailPage.tsx`

42.3.16 Pay a lab

Status: [Implemented]

What it does. Records a payment to a lab and splits it over the lab's approved bills. An unallocated rest is kept as an advance to the lab.

How the user does it.

1. On **Payments** (choose the lab first) click **Record payment**, or click **Pay** on an approved bill.
2. Enter Amount, Method (cash, bank transfer, cheque, other), Reference, Paid on and Notes.
3. Under "Split over bills" enter an amount per bill ("Never more than a bill's balance").
4. Save. A toast says "Payment recorded".

Behind the scenes — in simple terms. The payment is split over bills, never more than the payment and never more than a bill's balance. Bills are locked while this happens, and each bill's paid amount and status are recomputed.

Behind the scenes — technically. `POST /lab-payables/payments` → `VendorPaymentsService.create` (header `Idempotency-Key` optional; when given, replays return the stored response). One transaction: the lab must be linked to the clinic; allocations merged per bill; sum at most amount; `lockBills` (id order; same lab; status approved or partly paid; amount at most balance); number from sequence `vendor_payment` (prefix `VP`); insert payment and allocations; recompute each bill (`settledStatus`); journal debit Lab Payable (allocated), debit Lab Advances (rest), credit Cash or Bank by method; audit. The payment is recorded at the active location.

Rules and validations. Amount at least 1; at most 200 allocations; the bills must belong to the chosen lab.

Who can do it. `lab.payables.manage` . Default: Admin, Accountant.

Data read and written. Writes `vendor_payments` , `vendor_payment_allocations` , `vendor_bills` , `ledger_journals` , `audit_logs` , `number_sequences` .

Effects on other modules. Cash or bank reduces in the journals.

Error cases. `PAYABLES_LAB_NOT_FOUND` ("Lab not linked to this clinic"); `PAYABLES_ALLOCATION_LIMIT` ; `PAYABLES_BILL_NOT_FOUND` ; `PAYABLES_NOT_ALLOWED` .

Technical references: `backend/src/lab-payables/services/vendor-payments.service.ts` · `VendorPaymentsService.create` · `web-app/src/features/lab-payables/components/PayablesSheets.tsx` (`PayLabSheet`)

42.3.17 Use a lab advance on bills

Status: [Partial]

What it does. Applies the unallocated rest of a payment already made to approved bills of the same lab.

How the user does it. The Payments tab shows "Advance left", but no button on any page calls this action.

Behind the scenes — in simple terms. It moves money from the lab advance to the lab payable.

Behind the scenes — technically. `POST /lab-payables/payments/:id/allocations` → `VendorPaymentsService.allocate` : locks the payment, checks `version` , `adding ≤ amount − allocated` , locks bills, writes allocations, posts a journal (debit Lab Payable, credit Lab Advances). `labPayablesApi.allocate` exists in the web client; no component uses it.

Rules and validations. The payment must be posted; the sum must fit the unallocated rest.

Who can do it. `lab.payables.manage` .

Data read and written. Writes `vendor_payment_allocations`, `vendor_payments`, `vendor_bills`, journals.

Effects on other modules. Journals only.

Error cases. `PAYABLES_NOT_ALLOWED` ("This payment is reversed"); `PAYABLES_ALLOCATION_LIMIT`.

Technical references: `backend/src/lab-payables/services/vendor-payments.service.ts` · `VendorPaymentsService.allocate` · `web-app/src/features/lab-payables/services/lab-payables.api.ts`

42.3.18 Reverse a payment to a lab

Status: [Implemented]

What it does. Corrects a wrong entry: the payment becomes reversed, its allocations are cancelled, the bills get their balance back and every journal of the payment is mirrored. The payment stays on record.

How the user does it. Click "Reverse payment" on the row, enter a reason, confirm. A toast says "Payment reversed".

Behind the scenes — technically. `POST /lab-payables/payments/:id/reverse` → `VendorPaymentsService.reverse`: lock payment, check `version` and that it is posted, lock the touched bills in id order, set `reversed_at` on active allocations, set status `reversed`, recompute the bills, reverse the journals.

Rules and validations. Only a posted payment; `version` must match.

Who can do it. `lab.payables.manage`. Default: Admin, Accountant.

Data read and written. Writes `vendor_payments`, `vendor_payment_allocations`, `vendor_bills`, journals, audit.

Effects on other modules. Journals.

Error cases. `PAYABLES_NOT_ALLOWED` ("This payment is already reversed"); `PAYABLES_PAYMENT_NOT_FOUND`.

Technical references: `backend/src/lab-payables/services/vendor-payments.service.ts` · `VendorPaymentsService.reverse` · `web-app/src/features/lab-payables/pages/LabPaymentsPage.tsx`

42.3.19 Check a lab's monthly statement

Status: [Implemented]

What it does. The clinic types in the lines of the lab's monthly statement. The Platform matches them to the clinic's bills for the period and shows what differs.

How the user does it.

1. On **Statements** click **Add statement**.
2. Choose the lab, the period (From, To), the statement reference and enter the lines (lab reference, date, description, amount).
3. Save. If everything matches, the toast says "Statement saved and reconciled"; otherwise the statement is "Needs review".
4. Open the statement and click **Mark reconciled** or **Dispute** (add a note).

Behind the scenes — in simple terms. Each statement line is matched by reference (the lab's invoice number, the clinic's bill number or the order number) and amount. Each bill is used once. A bill the statement left out is added as a "Not on statement" row.

Behind the scenes — technically. `POST /lab-payables/statements` → `LabStatementsService.create`; `POST /lab-payables/statements/:id/status` → `setStatus`; `GET /lab-payables/statements`, `/:id`. `reconcile()` (pure) returns per line one of `matched`, `amount_mismatch`, `missing_in_system` and a list `missingInStatement`.

Period bills are those of the lab that are not void, have a total greater than 0 and a bill date inside the period, drafts included. When all lines match and none is missing, the statement starts as `reconciled`; otherwise `reconciling` (default column value is `uploaded`, which the code never writes on create). Matched bills get `statement_id`. The status endpoint accepts only `reconciled` or `disputed`, with `version`.

Rules and validations. The period end must not be before the start; the lab must be linked; the total defaults to the sum of the lines.

Who can do it. `lab.payables.manage`. Default: Admin, Accountant.

Data read and written. Reads `vendor_bills`, `lab_orders`, `lab_clinic_links`. Writes `lab_statements`, `lab_statement_lines`, `vendor_bills.statement_id`, `audit_logs`.

Effects on other modules. The lab admin sees statements in the lab portal.

Error cases. `PAYABLES_INVALID`; `PAYABLES_LAB_NOT_FOUND`; `PAYABLES_STATEMENT_NOT_FOUND`; stale `version`.

Technical references: `backend/src/lab-payables/services/lab-statements.service.ts` · `LabStatementsService` · `backend/src/lab-payables/common/payables-math.ts` · `web-app/src/features/lab-payables/pages/LabStatementDetailPage.tsx`

42.3.20 Payables reports

Status: [Implemented]

What it does. Three reports: what is owed to each lab by age, approved lab cost per lab per month, and profit per lab case.

How the user does it. Open **Lab payables** → **Reports**, choose Payables aging, Lab cost by month (and a Year) or Case profit; optionally filter by lab and dates.

Behind the scenes — in simple terms. Aging buckets are Not due, 1 to 30, 31 to 60, 61 to 90 and over 90 days past due. Case profit is revenue minus lab cost minus case expenses minus doctor commission. Invoiced and collected are shown apart.

Behind the scenes — technically. `GET /lab-payables/reports/aging` (`PayablesReportsService.aging`: approved and partly paid bills with a balance, bucketed by days past `due_on`, or bill date when no due date, in the active location's today), `GET /lab-payables/reports/monthly-cost` (`monthlyCost`: approved bills by bill month plus what was paid) and `GET /lab-payables/reports/case-profit` (`caseProfit`: revenue from issued lab invoices net of credit notes, or the frozen patient price while nothing is invoiced; lab cost is the frozen item costs plus approved order charges; commission is the signed lab commission entries). A helper `forItems` gives other modules an item's lab cost: approved bill lines, else the frozen snapshot.

Rules and validations. Organization-scoped; aging uses the active location's timezone.

Who can do it. `lab.payables.manage`.

Data read and written. Reads `vendor_bills`, `vendor_payments`, `lab_orders`, `lab_invoices`, `commission_entries`. Writes nothing.

Effects on other modules. None. These are not part of the 37 clinic reports (Chapter 36).

Error cases. 403 `NO_PERMISSION`.

Technical references: `backend/src/lab-payables/services/payables-reports.service.ts` · `PayablesReportsService` · `web-app/src/features/lab-payables/pages/PayablesReportsPage.tsx`

42.3.21 The lab sees its bills, payments and statements

Status: [Implemented]

What it does. A lab member can read, but not change, the approved bills, payments and statements that clinics recorded for their lab.

How the user does it. On the lab portal, open the lab billing page (`/lab-portal/billing`).

Behind the scenes — in simple terms. The lab never sees drafts. It sees only its own data.

Behind the scenes — technically. `LabPortalPayablesController` under the lab portal path `payables` : `GET bills` , `bills/:id` , `payments` , `statements` , `statements/:id` , guarded by `LabPortalGuard` and `@LabActor` . `LabPortalPayablesService` excludes bills in status `draft` .

Rules and validations. Read-only; scoped to the member's lab.

Who can do it. A lab member (lab admin or technician); no clinic permission.

Data read and written. Reads `vendor_bills` , `vendor_payments` , `lab_statements` .

Effects on other modules. Lab portal (Chapter 41).

Error cases. 403 for a non-member; 404 for another lab's record.

Technical references: `backend/src/lab-payables/controllers/lab-portal-payables.controller.ts` · `LabPortalPayablesService` · `web-app/src/features/lab-portal/payables/LabBillingPage.tsx`

42.4 Entities and data model

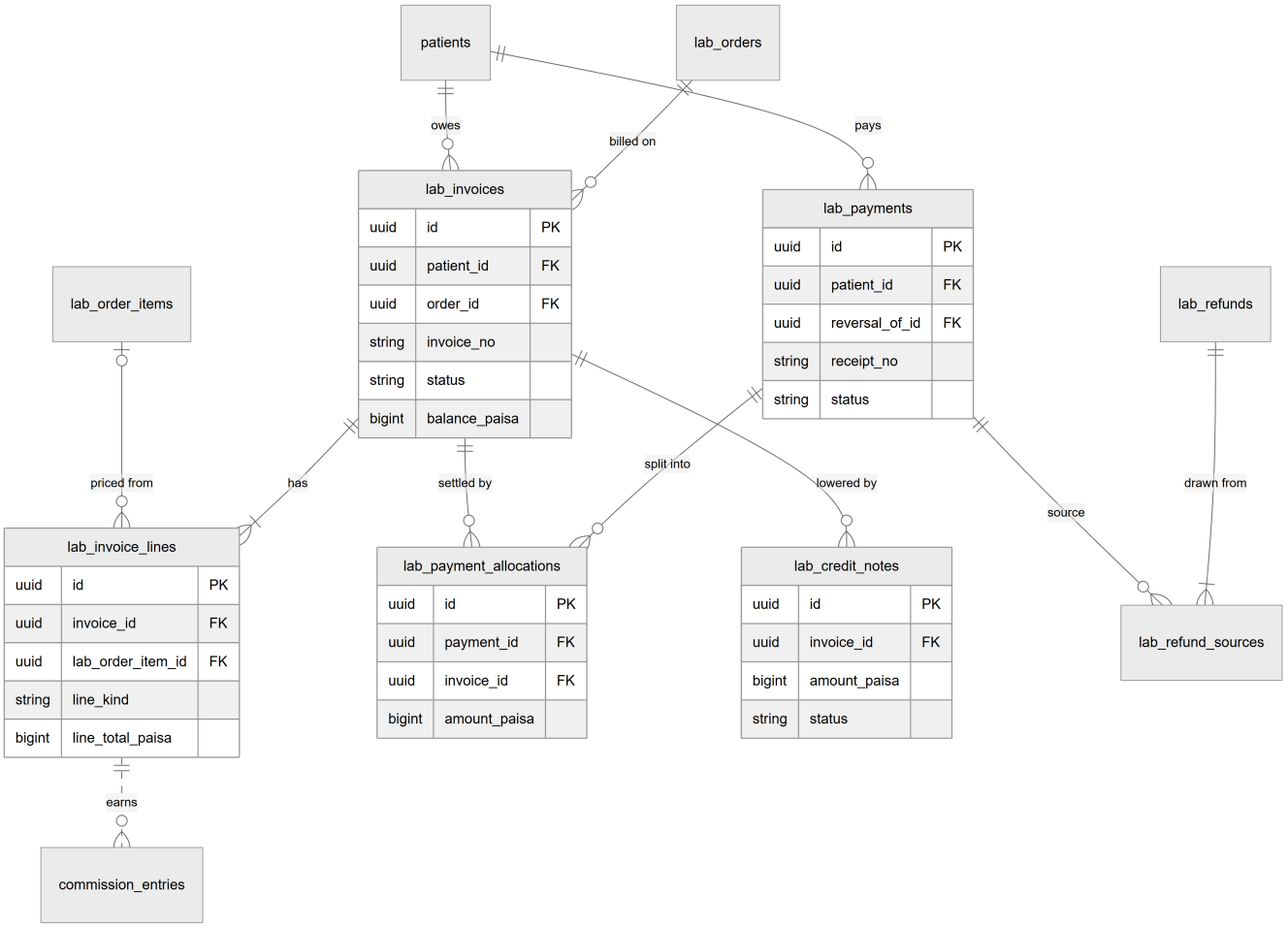

Figure 42.11 — Tables for patient lab billing.

The figure shows `lab_invoices` with a patient and an optional lab order, owning `lab_invoice_lines`. A `lab_invoice_lines` row may point at a `lab_order_items` row (dotted: optional). `lab_payments` belong to a patient and are split by `lab_payment_allocations` over invoices. `lab_credit_notes` belong to an invoice. `lab_refunds` draw from payments through `lab_refund_sources`. A dotted line shows that an invoice line earns `commission_entries`.

Each invoice belongs to exactly one patient and optionally to one lab order; deleting either is blocked (`ON DELETE RESTRICT`, as for every foreign key in these tables). An invoice has one or more lines. A payment belongs to one patient; a reversal row points at the payment it reverses (`reversal_of_id`). An allocation links one payment to one invoice with an amount and, once released, a `reversed_at` time and reason. Credit notes point at one invoice. A refund has one or more sources, each a payment. `lab_billing_settings` holds one row per organization. Amounts are integer paisa in `bigint` columns; tax rates are basis points. The Labs data chapter (Chapter 25) describes the columns in full.

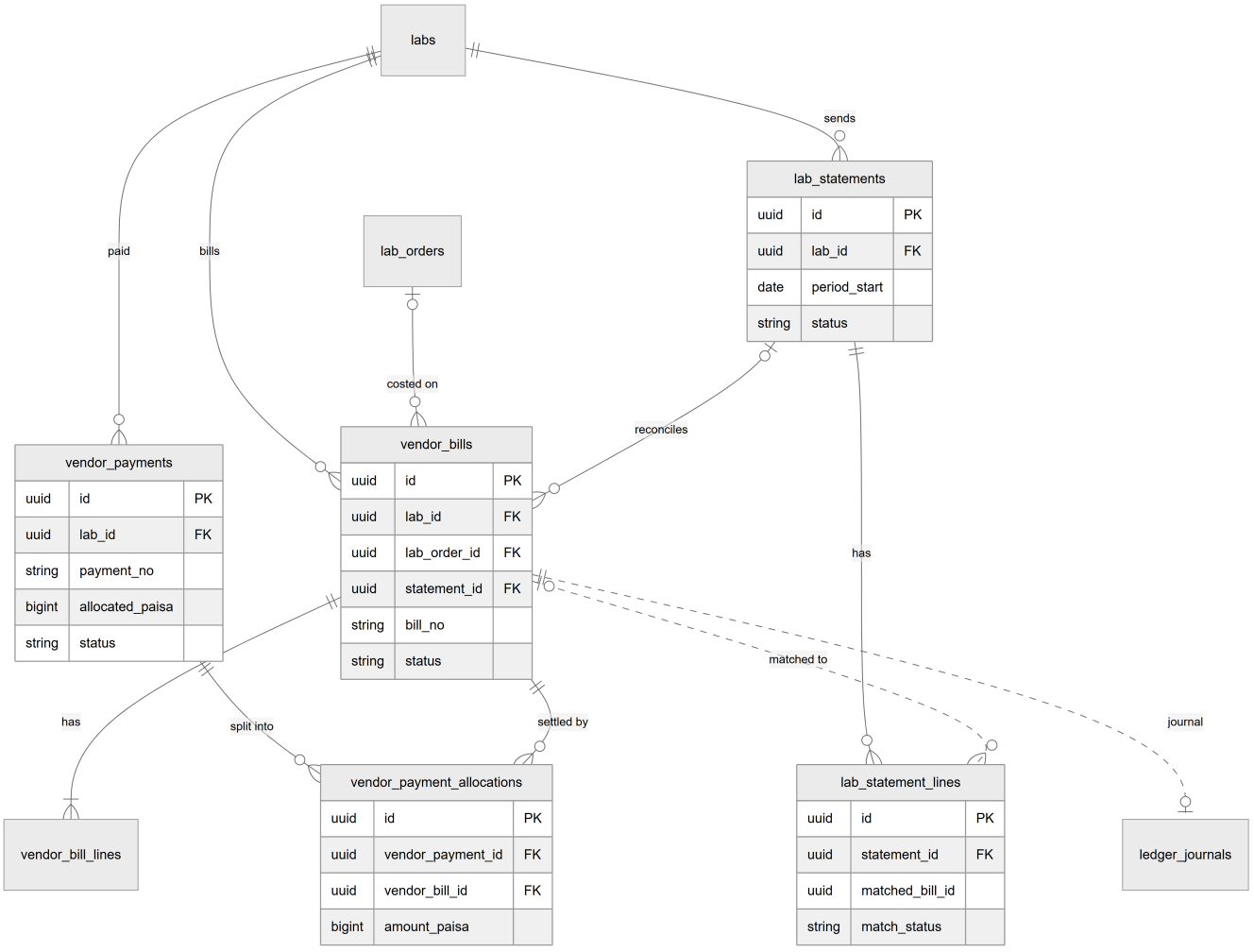

Figure 42.12 — Tables for lab payables.

The figure shows `labs` linked to `vendor_bills`, `vendor_payments` and `lab_statements`. A bill has lines, may point at a lab order and at a statement, and has an optional journal (dotted). Payments are split into `vendor_payment_allocations` over bills. A statement has `lab_statement_lines`; a line points at the bill it matched through `matched_bill_id` (dotted, logical link without a foreign key constraint in this diagram).

Each vendor bill belongs to one lab and usually to one lab order (a manual bill has none). A bill's `statement_id` is set when a statement matches it. A payment belongs to one lab; each allocation ties one payment to one bill. The lab's invoice number is unique per lab among bills. The journals live in `ledger_journals` and `ledger_journal_lines` (Chapter 21).

42.5 Status lifecycles

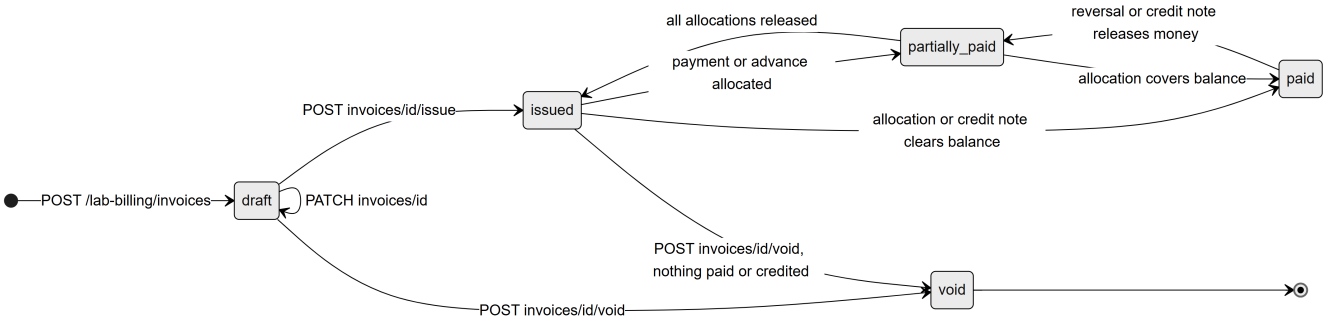

Figure 42.13 — Lab invoice: draft, issued, partially paid, paid, void.

A lab invoice is created as `draft` (an estimate) and can be edited any number of times. `POST invoices/id/issue`, or creating with `issue: true`, moves it to `issued`. Payments or advance allocated move it to `partially_paid`, and to `paid` when the balance is zero (a credit note can also clear the balance). A reversal or credit note that releases money can move `paid` back to `partially_paid`, and a `partially_paid` invoice whose allocations are all released goes back to `issued`. A draft, or an issued invoice with no payment and no credit note, can be voided. `void` is final. The status is written only by the totals service, never directly.

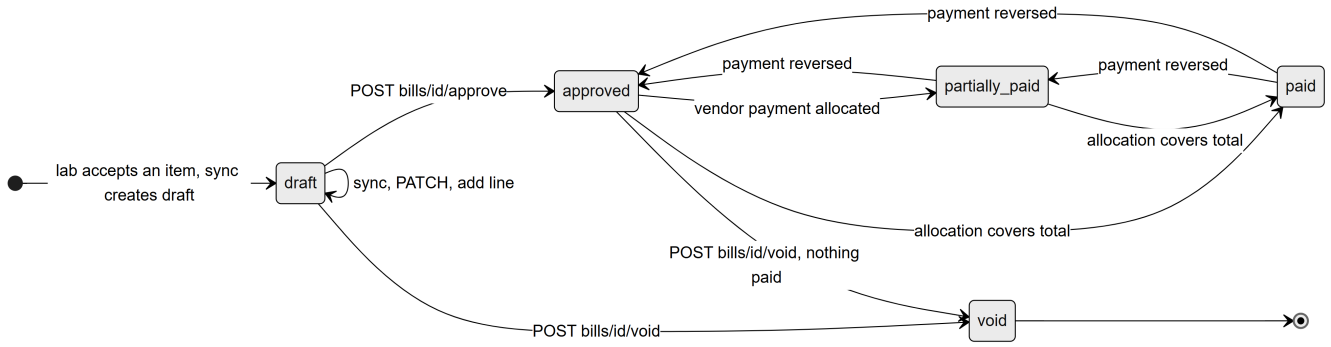

Figure 42.14 — Vendor bill: draft, approved, partly paid, paid, void.

A vendor bill starts as `draft` when a lab accepts work, and stays a draft while syncs, edits and manual lines change it. Approve makes it `approved`. Allocations move it to `partially_paid` and then `paid`. Reversing a payment moves it back. Void is allowed for a draft, or for an approved bill with nothing paid. `void` is final.

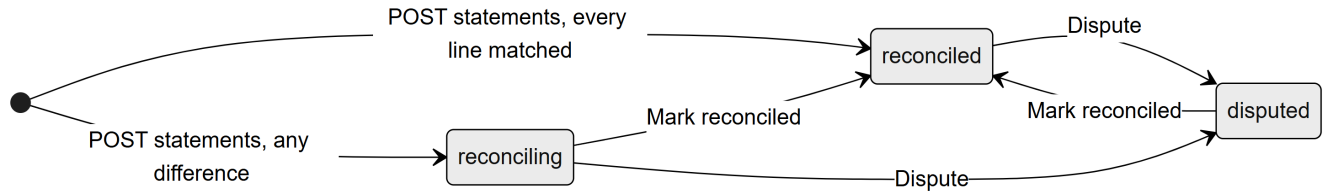

Figure 42.15 — Lab statement: reconciling, reconciled, disputed.

A statement is created as `reconciled` when every line matched and no bill is missing, otherwise as `reconciling` (shown as "Needs review"). The accountant can mark it `reconciled` or `disputed` at any time, and move between the two. The enum also has `uploaded`, the column default, but no code path writes it.

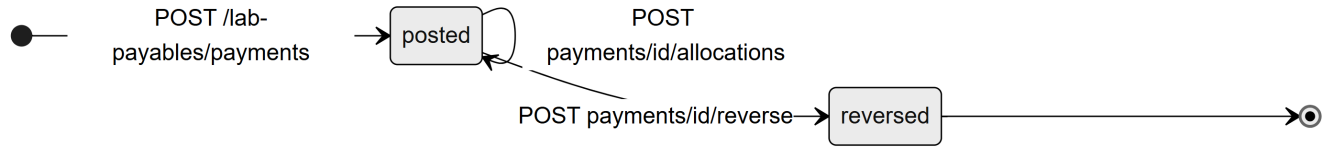

Figure 42.16 — Vendor payment: posted, reversed.

A vendor payment is `posted` on creation. More allocations can be added while it stays posted. Reversal makes it `reversed`, which is final.

Patient lab payments follow the same two states, `posted` and `reversed`. Credit notes have three: `issued` (money released, waiting for refund), `applied` (fully absorbed by the invoice balance) and `refunded`.

42.6 Connections to other modules

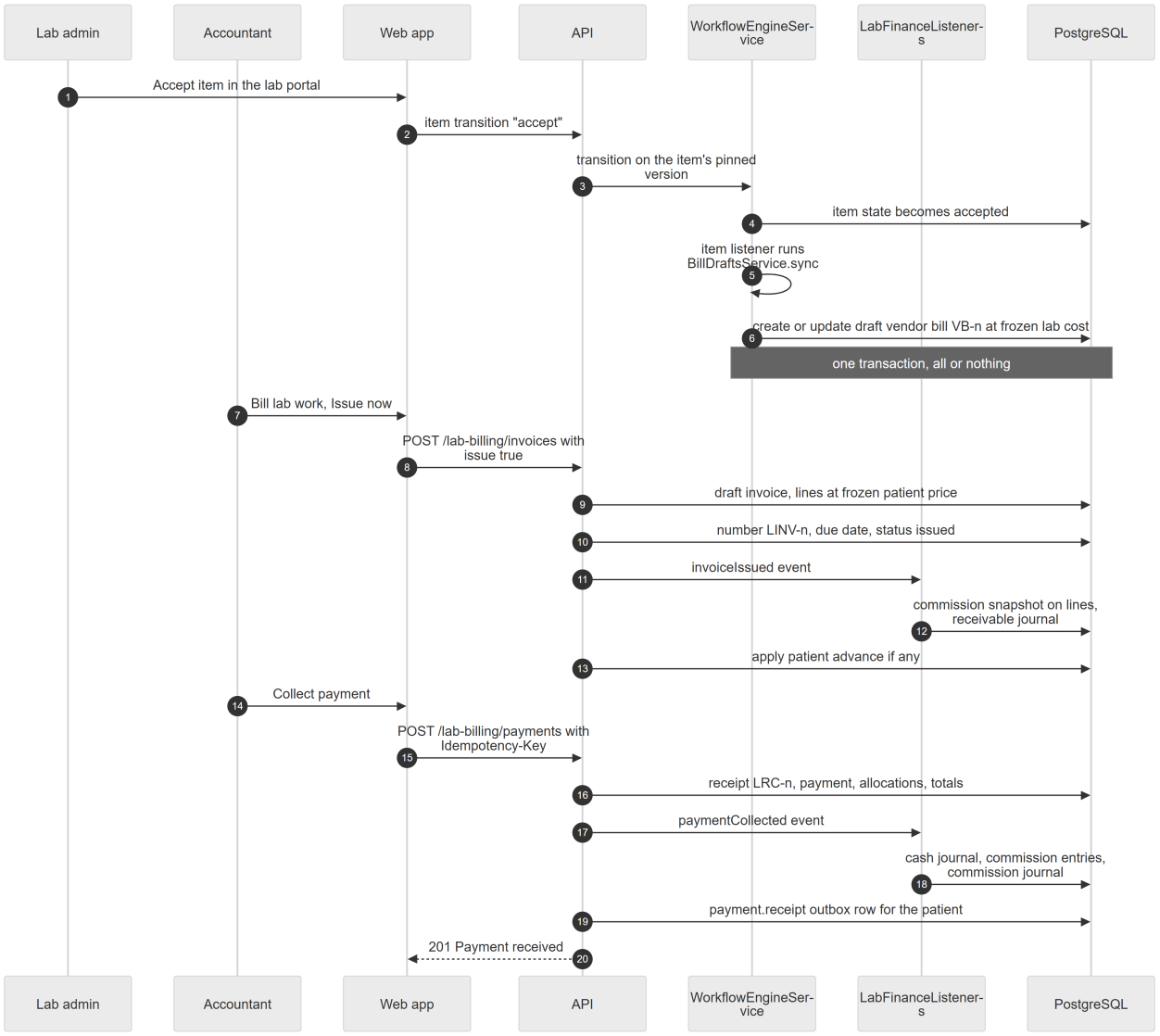

Figure 42.17 — A lab accepts work, a vendor bill drafts, the clinic invoices and collects, commission follows.

The diagram has seven participants: Lab admin, Accountant, Web app, API, WorkflowEngineService, LabFinanceListeners and PostgreSQL. In steps 1 to 6, the lab admin accepts an item in the lab portal; the web app calls the item transition `accept`; the API asks the workflow engine to run it on the item's pinned workflow version; the engine writes the new state, runs the item listener `BillDraftsService.sync` and creates or updates the draft vendor bill at the frozen lab cost. A note marks that all of this is one transaction. In steps 7 to 13, the accountant bills the lab work with Issue now; the API creates the draft with lines at the frozen patient price, numbers it and sets the due date, emits `invoiceIssued`, the listeners snapshot the commission on the lines and post the receivable journal, then any patient advance is applied. In steps 14 to 20, the accountant collects a payment with an `Idempotency-Key`; the API writes receipt, payment, allocations and totals, emits `paymentCollected`, the listeners post the cash journal, commission entries and the commission journal, a receipt notification is queued and the API answers 201.

Links to other modules:

- **Lab Management (Chapter 40)** creates and submits orders, freezes prices, and hosts the Billing panel.
- **Lab Portal (Chapter 41)** lets the lab accept work and read its bills.
- **Workflow Engine (Chapter 43)** runs the accept transition and the vendor bill effect.
- **Commissions (Chapter 35)** and **Payroll (Chapter 38)** use the lab commission entries.

- **Billing (Chapter 34)**: lab invoices are separate from clinic invoices; only the number sequences, journals and the void and refund permissions are shared.
- **Notifications (Chapter 44)** receive the receipt and reminder events.

42.7 Limitations and known gaps

- The journals (`ledger_journals`) are written for every lab money event, but no screen or report reads them. The clinic profit-and-loss report does not include them.
- Vendor bill header edit (`PATCH bills/:id`) and manual lines (`POST bills/:id/lines`) exist in the API only. The screen has no form for them, so a lab's partial charge on a cancelled item cannot be entered from the UI.
- "Use advance" on a lab payment (`POST payments/:id/allocations`) has no button.
- Payment reminders and receipts are queued but not delivered to patients, because only in-app notifications exist and the patient audience is off by default.
- A cost that goes down after bill approval is not corrected automatically; it needs a manual credit that the UI cannot enter.
- Statement lines are typed in by hand; there is no file upload even though the initial status is called `uploaded` .
- Lab invoices are not limited by location in the list; the permissions have no scope.
- There is no print layout for lab invoices or receipts in the code reviewed. [Unverified]: whether a PDF exists elsewhere.

42.8 Owner's FAQ

Can a receptionist collect a lab payment? Not by default. `lab.billing.manage` is held by Admin and Accountant. You can add it to a custom role.

What if the patient paid more than the invoice? The rest stays as the patient's advance, usable on the next lab invoice or refundable.

Can an invoice be deleted? No. A draft or unpaid issued invoice can be voided; paid ones need a payment reversal or a credit note.

Why can the Accountant not change the tax rate? Settings need `admin.settings.manage` in addition to lab billing; add it to the role if wanted.

What happens if two people click Collect payment together? The invoices are row-locked and the unique key stops duplicates; the second sees an allocation error if the balance is gone.

Where do I see what we owe a lab? Lab payables → Reports → Payables aging, or Bills filtered by lab.

Does the lab's price change alter a patient's invoice? No. Prices are frozen when the order is submitted.

Is there a general ledger screen? No. The journals exist in the database only.

Does the doctor earn commission on lab work? Yes, when the commissions feature is on and the doctor has a plan; it is calculated on collected money net of lab cost.

Chapter 43 — Workflow Engine (Config Studio)

This chapter describes the workflow engine and its editing tool, Config Studio. The engine stores order forms, try-in stage templates and status state machines as versioned data, so that the Lab Management module can run on them. The chapter is written for the product owner, who needs to know what the super admin can configure without a code change, what the engine enforces at run time and what cannot be configured. The data tables are described in Chapter 26; the lab module that uses the engine is described in Chapters 40 and 41.

43.1 What this module is for

The engine answers three questions for lab work: what does the doctor fill in when ordering an item (a **form schema**), which try-in rounds does the item go through (a **stage template**), and which status changes are allowed, by whom and with what side effects (a **workflow template**). All three belong to a **specialty** (the seed has one, DENTAL). Forms and workflows are versioned. A version is a draft while edited, becomes published when it passes the checks, and is retired when replaced.

Who uses it: only the **super admin**. Every editing endpoint is restricted to super admins, and the web routes send everyone else to the dashboard. Clinic users never open Config Studio; they only meet its results (the order form, the status names, the allowed actions). Two read-only endpoints serve clinic members: the forms an organization currently uses, and the specialties it works in.

Where it appears: the **Admin** sidebar group, under **Config Studio**, with three items: **Form schemas** (`/studio/forms`), **Workflows** (`/studio/workflows`) and **Stage templates** (`/studio/stages`). All three are `superAdminOnly` entries. The route `/studio` redirects to `/studio/forms` . The studio routes declare no `staticData.can` ; they rely on the super admin check in the route component and on the API. Config Studio is not tied to a feature key; it is platform-level.

43.2 Screens

Figure 43.1 — Config Studio, Form schemas (screenshot pending).

The page is titled "Form schemas" with the subtitle "Order forms per specialty — JSON definition with a live preview. Platform admins only." and a **New form** button. It has a search box ("Search by code or name…"), filters by specialty and owner (All owners, Platform defaults, or one organization) and a list of cards or rows showing code, name, specialty, owner, number of versions, and an "Enter" action. An empty list shows "No form schemas yet".

Figure 43.2 — Form schema detail (screenshot pending).

The detail page has a "Back to form schemas" link, the code and name, an "Inactive" badge when switched off, and tabs **Editor** and **Activity**. The editor has a Versions panel (Version, Status, Created, Published, Retired; icon buttons to open, compare, make a new draft from, or retire a version) and, for a draft, three JSON areas: `schemaJson` (fields), `uiJson` (layout) and `promotions`, each with a format button. Beside it is a live "Form preview" that shows the form as a doctor would see it, with a button to reset answers, and a list "Fix before publishing" that names every problem. Buttons are **Save draft**, **Publish** and **Discard draft**. A compare view marks fields Added, Removed or Changed. The Activity tab lists every change of the form.

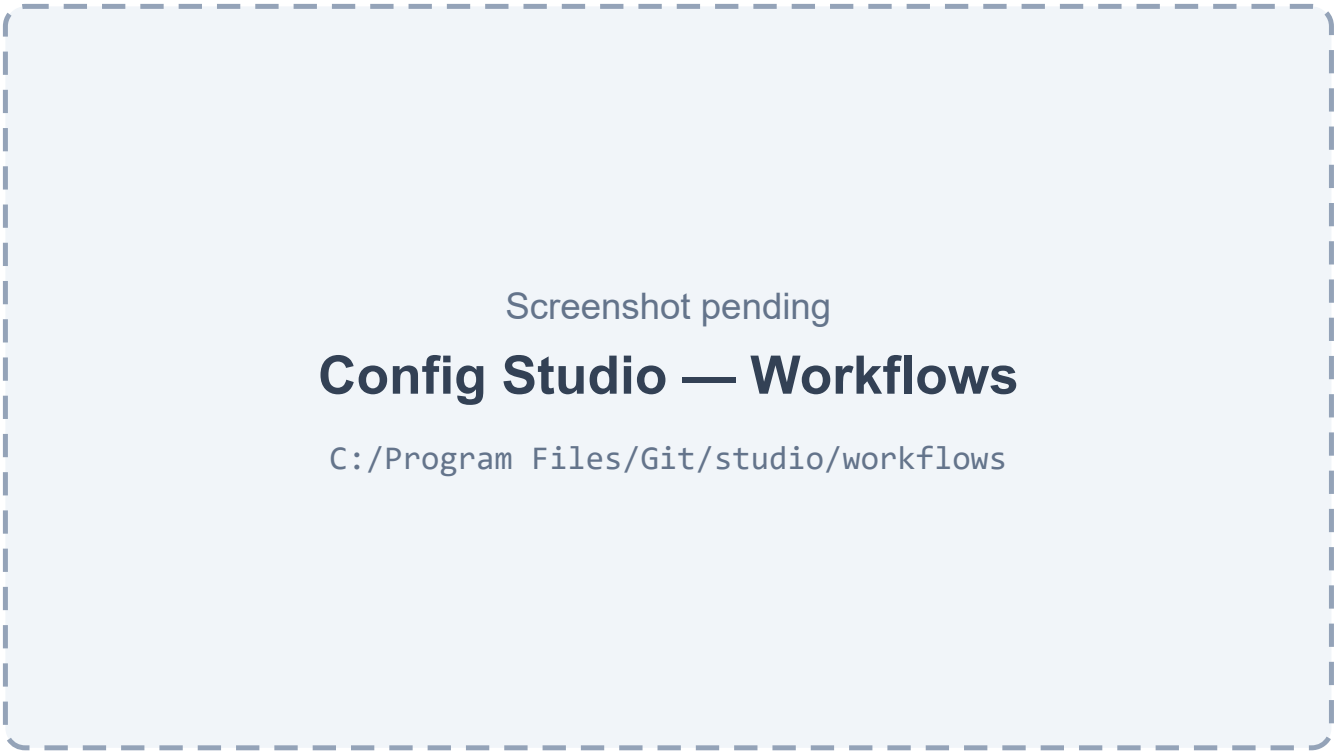

Figure 43.3 — Config Studio, Workflows (screenshot pending).

The Workflows list is titled "Workflows" with the subtitle "States and transitions for orders, items and stages. Platform admins only." It has a **New workflow** button, the same search and owner filters, and a filter by entity (order, item, stage). The empty state says workflows define which status changes are allowed, by whom, and what they trigger.

Screenshot pending

Config Studio — Workflow detail

`C:/Program Files/Git/studio/workflows/$workflowId`

Figure 43.4 — Workflow detail (screenshot pending).

The detail page has the same header and Versions panel. A draft is edited as tables: States (Code, Name, Category, Order) and Transitions (From, Action, To, Sides, Roles, Required input, Reason, Effects), with "Any role" and "None" placeholders. Beside the tables is a **Test run** panel that lists the shortest path from the initial state to each end state. Buttons are **Save draft**, **Publish** and **Discard draft**; a compare view shows States and Transitions that changed.

Screenshot pending

Config Studio — Stage templates

`C:/Program Files/Git/studio/stages`

Figure 43.5 — Config Studio, Stage templates (screenshot pending).

The page lists stage templates (Template, Specialty, Owner, Steps, Active) with an edit icon per row and a "New stage template" sheet. The sheet has Code, Name, Specialty, Owner, Active and a list of steps (code, name, lab days, whether a patient visit and doctor approval are needed), with move and remove icon buttons.

43.3 Features

43.3.1 Manage form schemas

Status: [Implemented]

What it does. Creates a form schema (for example CROWN), renames it, lists and searches them, and shows its history. A schema without an owner is the platform default, used by every organization. A schema owned by one organization overrides the default with the same code for that organization.

How the user does it.

1. Open **Config Studio** → **Form schemas**, click **New form**.
2. Fill Code, Name, Description, Specialty, Owner ("Platform default (everyone)" or an organization) and "Start from" (empty definition or a copy of an existing version).
3. Save, then open the card with "Enter".

Behind the scenes — in simple terms. A form schema is the stable identity; the actual fields live in its versions. Creating one makes the schema and its first draft.

Behind the scenes — technically. `GET /platform/form-schemas`, `POST /platform/form-schemas`, `GET /platform/form-schemas/:id`, `PATCH /platform/form-schemas/:id`, `GET /platform/form-schemas/:id/history`, all behind `@RequireSuperAdmin()` (`PlatformFormSchemasController`). `FormSchemasService.create` / `createInTx` / `update` / `history`. Updates carry `version` for optimistic locking. Changes are written to `audit_logs` (entity `form_schema`); history reads them as a timeline.

Rules and validations.

- Code is unique per specialty and per owner (one platform schema and one per organization for each code).
- The specialty must exist; the owner organization must exist.
- Copying a version needs a version of the same kind.

Who can do it. Super admin only. No permission key, no feature key. No system role holds it.

Data read and written. Reads and writes `form_schemas`, `form_schema_versions`, `audit_logs`; reads `specialties`, `organizations`.

Effects on other modules. New schemas appear in the order form of Lab Management once a version is published (43.3.3).

Error cases. `FORM_SCHEMA_EXISTS` (409); `FORM_SCHEMA_NOT_FOUND` (404); `SPECIALTY_NOT_FOUND`; `ORGANIZATION_NOT_FOUND`; 403 for a non-super-admin.

Technical references: `backend/src/engine/forms/form-schemas.controller.ts` · `FormSchemasService` · `web-app/src/features/engine-studio/pages/DefinitionsListPage.tsx`

43.3.2 Edit a form version and preview it

Status: [Implemented]

What it does. Creates a draft version, edits its JSON (fields, layout, promotions), saves it, previews it with sample answers and discards it.

How the user does it.

1. Open a form schema and, in the Versions panel, click "New draft" (or "New draft from vN").
2. Edit `schemaJson`, `uiJson` and `promotions`. The preview on the side updates, and "Fix before publishing" lists problems.
3. Click **Save draft**. An "Unsaved changes" marker shows pending edits.
4. Click **Discard draft** to throw it away.

Behind the scenes — in simple terms. Saving never needs a clean form: problems are shown but do not block saving. The preview runs the same server rules that real orders use, so it tells the truth about what would be accepted.

Behind the scenes — technically. `POST /platform/form-schemas/:id/versions` (`createDraft`, optionally from a version), `GET .../versions/:versionId`, `PATCH .../versions/:versionId` (`updateDraft`: saves, returns issues), `DELETE .../versions/:versionId` (`discardDraft`), `POST .../versions/:versionId/validate` (`validateData`: tries answers against any version, drafts included). The checker `checkDefinition` and the answer validator `FormDataValidator` are pure code. Field types: `tooth_chart`, `select`, `multi_select`, `text`, `textarea`, `number`, `date`, `boolean`, `file`. Promotions copy chosen answers into typed columns of `dental_item_details` (shade system, shade, material code, implant system, platform, diameter, margin type), and only to targets whose field type fits.

Rules and validations.

- Only a draft's content can change; a published or retired version cannot.
- At most one draft per form at a time (`FORM_DRAFT_EXISTS`).
- A discarded draft is deleted, and an audit row on the schema keeps its content.
- `version` must match on save.
- Answer validation rejects unknown keys, so a client typo never silently loses data; a field hidden by `visibleIf` is neither required nor stored.

Who can do it. Super admin only.

Data read and written. `form_schema_versions` (`schema_json`, `ui_json`, `promotions`), `audit_logs`.

Effects on other modules. None until published.

Error cases. `FORM_VERSION_NOT_FOUND`; `FORM_VERSION_NOT_DRAFT`; `FORM_DRAFT_EXISTS`; stale `version`.

Technical references: `backend/src/engine/forms/form-schemas.service.ts` · `FormSchemasService.updateDraft` · `backend/src/engine/forms/form-definition.checker.ts` · `web-app/src/features/engine-studio/components/FormVersionEditor.tsx`

43.3.3 Publish and retire a form version

Status: [Implemented]

What it does. Publishing makes a draft the live version that new lab orders use. Retiring stops offering a published version to new orders; orders already pinned to it keep working.

How the user does it.

1. In the editor with a clean draft, click **Publish**, confirm "Publish version N".
2. To retire, click the retire icon in the Versions panel and confirm "Retire vN".

Behind the scenes — in simple terms. The system checks the whole definition first. If anything is wrong, nothing is published. Publishing stores exactly what was checked, in cleaned form. The previous published version is retired by the same step, so there is only ever one live version.

Behind the scenes — technically. `POST .../versions/:versionId/publish` → `FormSchemasService.publish`: locks the schema and the draft, checks `version`, runs `checkDefinition` and throws `FORM_DEFINITION_INVALID` with an `issues` list on any problem, then `publishInTx`. `POST .../retire` → `retire`: only the published version can be retired. Status changes are written to `status_history` and `audit_logs`. A database check constraint requires a publish date on every non-draft version. Immutability of published versions is enforced by the service, not by a trigger.

Rules and validations.

- Valid JSON shape, unique field keys, known types and properties, consistent options and limits, conditions that point back to fields placed before them only (so no cycles), a layout that names only real fields, promotions to registered columns of a matching type.
- At most one published version per form.
- Retire needs a published version.

Who can do it. Super admin only.

Data read and written. `form_schema_versions`, `status_history`, `audit_logs`.

Effects on other modules. Clinics see the newly published form on their next lab order (43.3.4). Drafts are never shown to clinics.

Error cases. `FORM_DEFINITION_INVALID` (422, with issues); `FORM_VERSION_NOT_DRAFT`; `FORM_VERSION_NOT_PUBLISHED`; stale `version`.

Technical references: `backend/src/engine/forms/form-schemas.service.ts` · `FormSchemasService.publish` / `retire` · `web-app/src/features/engine-studio/components/VersionsPanel.tsx`

43.3.4 Clinics read the forms they use

Status: [Implemented]

What it does. Lets any clinic member read the current published form per code for their organization's specialties, and test answers against a pinned version.

How the user does it. Nothing directly. The lab order form and the lab price list load these forms.

Behind the scenes — in simple terms. An organization sees the platform defaults and its own forms. Its own form with the same code wins. It never sees drafts or another organization's forms.

Behind the scenes — technically. `GET /form-schemas` (`@AnyMember`), `GET /form-schemas/versions/:versionId`, `POST /form-schemas/versions/:versionId/validate` → `FormResolverService.current` / `version` / `validate`. A retired version still validates answers, so orders pinned to it keep working. Specialty enablement is respected (`SPECIALTY_NOT_ENABLED` when an organization does not work in the specialty).

Rules and validations. Another organization's version answers 404.

Who can do it. Any member of the active organization (`@AnyMember`).

Data read and written. Reads `form_schemas`, `form_schema_versions`, `organization_specialties`.

Effects on other modules. Lab orders (Chapter 40) and the lab catalog.

Error cases. `FORM_VERSION_NOT_FOUND`; `FORM_DATA_INVALID` (422); `SPECIALTY_NOT_ENABLED` (403).

Technical references: `backend/src/engine/forms/form-resolver.service.ts` · `FormResolverService` · `web-app/src/features/lab-catalog/services/lab-catalog.api.ts`

43.3.5 Manage workflow templates

Status: [Implemented]

What it does. Creates a workflow template for one kind of record (order, item or stage), lists and searches them, and renames them.

How the user does it.

1. Open **Config Studio** → **Workflows**, click **New workflow**.
2. Fill Code, Name, Description, Specialty, "Runs on" (order, item or stage), Owner, and "Start from" (empty or a copy of an existing version of the same kind).
3. Save and open the workflow.

Behind the scenes — in simple terms. A template is the identity; the state machine is in its versions. A copy keeps the definition but starts a new template.

Behind the scenes — technically. `GET/POST /platform/workflow-templates`, `GET/PATCH /platform/workflow-templates/:id` → `WorkflowTemplatesService.list / create / get / update`, all super admin only. Copying checks that the source's entity type equals the requested one (`WORKFLOW_DEFINITION_INVALID` otherwise). The new template gets a draft version 1.

Rules and validations. Code unique per specialty, entity type and owner (`WORKFLOW_TEMPLATE_EXISTS`); specialty and organization must exist.

Who can do it. Super admin only.

Data read and written. `workflow_templates`, `workflow_template_versions`, `audit_logs`.

Effects on other modules. None until a version is published.

Error cases. `WORKFLOW_TEMPLATE_EXISTS`; `WORKFLOW_TEMPLATE_NOT_FOUND`; `WORKFLOW_VERSION_NOT_FOUND`; `SPECIALTY_NOT_FOUND`; `ORGANIZATION_NOT_FOUND`.

Technical references: `backend/src/engine/workflows/workflow-templates.controller.ts` · `WorkflowTemplatesService` · `web-app/src/features/engine-studio/pages/DefinitionsListPage.tsx`

43.3.6 Edit a workflow draft and run a test

Status: [Implemented]

What it does. Edits the states and transitions of a draft, and shows a test run: the shortest route from the start to each end state, computed without touching data.

How the user does it.

1. In the Versions panel click "New draft" or "New draft from vN".
2. Edit States (code, name, category initial, active, waiting, done or cancelled) and Transitions (from, action, to, sides, roles, required input, reason, effects). The effect dropdown lists only handlers that exist in code.
3. Click **Save draft**. Problems appear in "Fix before publishing"; the Test run panel updates.

Behind the scenes — in simple terms. The definition is JSON while it is a draft. It is not used by any order. The effects list is a menu of code that already exists: configuration can pick among them but cannot add new behaviour.

Behind the scenes — technically. `POST /platform/workflow-templates/:id/versions`, `GET .../versions/:versionId`, `PATCH .../versions/:versionId` (`updateDraft`, returns `issues` and a `simulation`), `DELETE .../versions/:versionId` (`discardDraft`), `GET /platform/workflow-effects` (`PlatformWorkflowEffectsController`, lists `EffectRegistry` handlers). The simulation is `simulateWorkflow`. Registered effect codes in this build: `recompute_order_status`, `init_stages`, `notify`, `start_warranty`, `check_warranty_on_remake`, `create_vendor_bill_draft`.

Rules and validations.

- At most one draft per template (`WORKFLOW_DRAFT_EXISTS`); only a draft can be edited or discarded.
- Limits: 60 states, 300 transitions.
- Checks at publish time are in 43.3.7.

Who can do it. Super admin only.

Data read and written. `workflow_template_versions.definition_json`, `audit_logs`.

Effects on other modules. None until published.

Error cases. `WORKFLOW_VERSION_NOT_DRAFT`; `WORKFLOW_DRAFT_EXISTS`; `WORKFLOW_VERSION_NOT_FOUND`; stale `version`.

Technical references: `backend/src/engine/workflows/workflow-templates.service.ts` · `WorkflowTemplatesService.updateDraft` · `backend/src/engine/workflows/workflow-definition.checker.ts` · `web-app/src/features/engine-studio/components/WorkflowVersionEditor.tsx`

43.3.7 Publish and retire a workflow version

Status: [Implemented]

What it does. Publishing turns the JSON draft into real rows (states, transitions, effects) and makes it the version that new orders and items pin. The previous published version is retired in the same transaction.

How the user does it. Click **Publish** and confirm "Publish version N"; or use the retire icon and confirm "Retire vN".

Behind the scenes — in simple terms. The checker is strict: it refuses any machine that could trap work. Rows are written once and never change. Work already in progress stays on the version it started with.

Behind the scenes — technically. `POST .../versions/:versionId/publish` → `publishInTx`: runs `checkWorkflow(definition, effects.codes())`; on any issue throws `WORKFLOW_DEFINITION_INVALID` with `issues`. Otherwise inserts `workflow_states` (with `is_terminal` from the category and `sort_order`), `workflow_transitions` (from, to, action, `allowed_sides`, `allowed_roles`, `required_fields`, `requires_reason`) and `transition_effects` (in order, with params), retires the current published version, sets `published_at` and `published_by`, and records status history and audit. `POST .../retire` → `retire` (only a published version).

Rules and validations. The checks (lab ERD section 18):

- Exactly one initial state; at least one done and one cancelled end state.
- Every state reachable from the initial state; every non-terminal state has a way forward (no dead end).
- Nothing leaves a terminal state; no self-loop.
- Unique state codes; each (state, action) leads to one target (except derived changes).
- Only registered effects (no duplicates in one transition) and only existing roles.
- Valid sides (clinic, lab, system) and required fields.

Who can do it. Super admin only.

Data read and written. `workflow_template_versions`, `workflow_states`, `workflow_transitions`, `transition_effects`, `status_history`, `audit_logs`.

Effects on other modules. New lab orders and items pin the new version (43.3.9). Existing ones are unchanged.

Error cases. `WORKFLOW_DEFINITION_INVALID` (422, with issues); `WORKFLOW_VERSION_NOT_DRAFT`; `WORKFLOW_VERSION_NOT_PUBLISHED`; stale `version`.

Technical references: `backend/src/engine/workflows/workflow-templates.service.ts` · `WorkflowTemplatesService.publishInTx` · `web-app/src/features/engine-studio/components/VersionsPanel.tsx`

43.3.8 Manage stage templates

Status: [Implemented]

What it does. Defines the try-in rounds of a kind of work, for example metal, bisque, final. Not versioned: changing a template affects only work that starts later.

How the user does it.

1. Open **Config Studio** → **Stage templates**, click to create ("New stage template"), or the edit icon on a row.
2. Enter Code, Name, Specialty, Owner, Active, and steps: code, name, lab days, whether a patient visit and doctor approval are needed. Reorder or remove steps with the icon buttons.
3. Click **Create template** or **Save changes**.

Behind the scenes — in simple terms. When an order item starts, the system copies the steps into the item's own stage rows. Editing a template later cannot damage running work.

Behind the scenes — technically. `GET/POST /platform/stage-templates`, `GET/PATCH /platform/stage-templates/:id` → `StageTemplatesService` (`PlatformStageTemplatesController`, super admin only). Steps are in `stage_template_steps` (cascade delete with the template). Seeded templates: `SINGLE_STAGE`, `PFM_3_STAGE` and `COMPLETE_DENTURE`.

Rules and validations. Code unique per specialty and owner; step codes unique within a template; lab days 0 to 365; the last step must require doctor approval. There is no delete endpoint; a template is switched inactive.

Who can do it. Super admin only.

Data read and written. `stage_templates`, `stage_template_steps`, `audit_logs`.

Effects on other modules. The `init_stages` effect creates an item's stage rows from the template (Chapter 40).

Error cases. `STAGE_TEMPLATE_EXISTS`; `STAGE_TEMPLATE_NOT_FOUND`; `STAGE_TEMPLATE_INVALID` (422).

Technical references: `backend/src/engine/stages/stage-templates.service.ts` · `StageTemplatesService` · `web-app/src/features/engine-studio/components/StageTemplateSheet.tsx`

43.3.9 A lab order's state machine comes from a published workflow version

Status: [Implemented]

What it does. Every lab order, item and try-in stage stores the id of the workflow version it started on. Every status change is then checked against that version.

How the user does it. Nothing special: a doctor creates a lab order draft; later the doctor, the lab or the system performs actions such as submit, accept or approve.

Behind the scenes — in simple terms. Think of the workflow version as the rule book printed on the day the case opened. The case keeps its copy. If the super admin publishes a new rule book tomorrow, old cases still follow the old one. A referee (the engine) is the only way to change a status: a direct status update is not allowed. The order's status is not set by hand; it is derived from its items.

Behind the scenes — technically. `OrderWorkflowService.pinWorkflowVersion` chooses the newest published version of the given template, or, without one, of the specialty's default template for the entity type; the organization's own template beats the platform's (`ORDER BY organization_id IS NOT NULL DESC`). It is stored in `workflow_template_version_id` on `lab_orders` and `lab_order_items` . If no published workflow exists, the order fails with `LAB_ORDER_WORKFLOW_MISSING` . `WorkflowEngineService.transition` runs in one transaction: idempotency claim, lock the subject, find the (state, action) pair in its pinned version, check side and role, check required input and reason, write the state, write status history, run the effects in order, run the after-transition hook (derive the order status from its items), audit. `transitionIn` runs the same inside a larger transaction (for example order submit). `applyDerived` sets a status the engine computed, which is not looked up as a user transition. `compiled` loads and caches a version's states and transitions and refuses a draft version. A clinic user can act for a lab (proxy) only with the lab-coordinator capability, and the action is recorded as proxy.

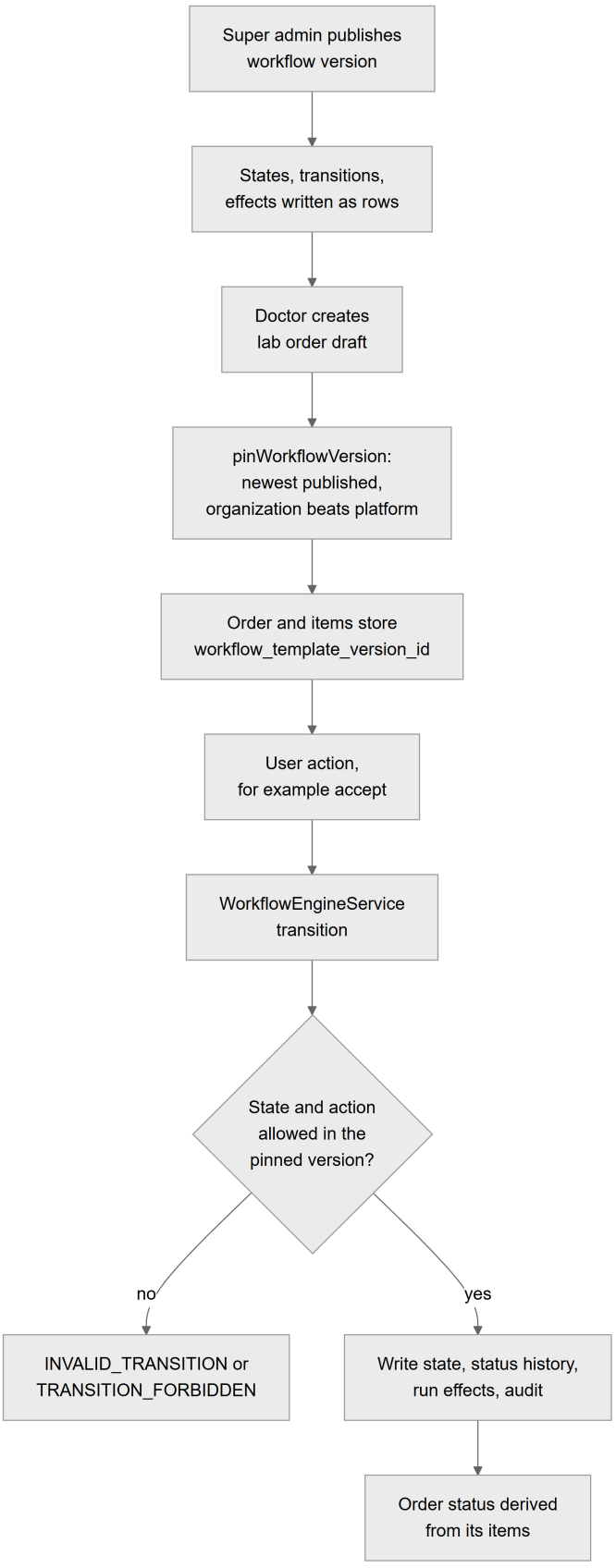

Figure 43.6 — From publishing a version to a checked transition.

The flowchart reads top to bottom. "Super admin publishes workflow version" leads to "States, transitions, effects written as rows". "Doctor creates lab order draft" leads to "pinWorkflowVersion: newest published, organization beats platform", then to "Order and items store workflow_template_version_id". A "User action, for example

accept" goes to "WorkflowEngineService transition", then to a decision "State and action allowed in the pinned version?". If no, the result is `INVALID_TRANSITION` or `TRANSITION_FORBIDDEN`. If yes, the engine writes state, status history, runs effects and audit, and the order status is derived from its items.

The seeded dental workflows show what a version holds. The item workflow (`DENTAL_ITEM`) has states such as Waiting for lab (initial), Accepted, In production, At clinic for try-in, Quality check, Ready, Dispatched, Received, Approved, Delivered (done), and Rejected by lab, Cancelled and Remake required (cancelled). Its transition `accept` is allowed on the lab side for the lab admin role and requires the field `lab_expected_on` ; `reject` and `qc_fail` require a reason; `send_tryin` and `dispatch` are system-driven. The order workflow (`DENTAL_ORDER`) runs Draft, Submitted, Clarification needed, Accepted, In progress, On hold, Partially ready, Ready, Completed and Cancelled, among others. The stage workflow (`DENTAL_STAGE`) covers one try-in round.

Rules and validations.

- A subject pinned to a draft version is refused (`WORKFLOW_MISCONFIGURED`).
- A user action needs the right side (clinic, lab or system) and, if the transition lists roles, a matching role.
- Required fields and reasons are enforced per transition.
- Derived order status never overrides terminal states.

Who can do it. The permissions are those of the lab order actions (Chapters 40 and 41), not Config Studio. Super admin configures the version only.

Data read and written. Reads `workflow_template_versions` , `workflow_states` , `workflow_transitions` , `transition_effects` . Writes the subject's `status_code` , `status_history` , `audit_logs` .

Effects on other modules. Effects fire in Lab Management (stages, warranty, notifications), Lab Finance (the `create_vendor_bill_draft` effect creates a vendor bill draft when a lab accepts an item, Chapter 42) and order status.

Error cases. `INVALID_TRANSITION` (422), `TRANSITION_FORBIDDEN` (403), `TRANSITION_INPUT_REQUIRED` (422), `TRANSITION_REASON_REQUIRED` (422), `WORKFLOW_SUBJECT_NOT_FOUND` (404), `WORKFLOW_MISCONFIGURED` (500), `REQUEST_IN_FLIGHT` (409), `IDEMPOTENCY_KEY_REUSED` (422), `LAB_ORDER_WORKFLOW_MISSING` .

Technical references: `backend/src/engine/runtime/workflow-engine.service.ts` · `WorkflowEngineService.transition` · `backend/src/lab-orders/workflow/order-workflow.service.ts` · `OrderWorkflowService.pinWorkflowVersion` · `backend/src/engine/workflows/order-status.ts`

43.3.10 Compare versions and read the activity log

Status: [Implemented]

What it does. Shows what changed between two versions of a form or workflow, and lists every change to a form.

How the user does it. In the Versions panel click the compare icon, pick the compare version; the diff marks Added, Removed, Changed. Open the Activity tab on a form for the timeline.

Behind the scenes — in simple terms. Comparison happens in the browser on two loaded versions. The activity log is a view of the audit trail.

Behind the scenes — technically. `FormCompare` , `WorkflowCompare` and `DiffView` in the web app. `GET /platform/form-schemas/:id/history` → `FormSchemasService.history` reads the audit timeline of the schema and of each of its versions. There is no history endpoint for workflows or stage templates; the Activity tab exists only on forms.

Rules and validations. Read-only.

Who can do it. Super admin only.

Data read and written. Reads `audit_logs`.

Effects on other modules. None.

Error cases. `FORM_SCHEMA_NOT_FOUND`.

Technical references: `backend/src/engine/forms/form-schemas.service.ts` · `FormSchemasService.history` · `web-app/src/features/engine-studio/components/StudioActivity.tsx`

43.3.11 Specialties and which organizations use them

Status: [Partial]

What it does. Lists specialties, creates or edits one, and switches a specialty on or off per organization. The seed creates DENTAL, enabled by default.

How the user does it. The Studio only reads the specialty list (to fill the Specialty filters and dropdowns). No screen creates a specialty or switches it per organization.

Behind the scenes — in simple terms. An organization's specialty rows are created on first use from the "enabled by default" flag. A super admin's later on/off choice always wins.

Behind the scenes — technically. `GET /platform/specialties`, `POST /platform/specialties`, `PATCH /platform/specialties/:id`, `GET /platform/specialties/:id/organizations`, `PUT /platform/specialties/:id/organizations/:organizationId` (`PlatformSpecialtiesController`, super admin only) and `GET /specialties` (`@AnyMember`, the organization's enabled specialties) → `SpecialtiesService`. The web client calls only `GET /platform/specialties`.

Rules and validations. Specialty code unique (`SPECIALTY_EXISTS`); an organization change is audited only when the value changes.

Who can do it. Super admin (platform endpoints); any member for `GET /specialties`.

Data read and written. `specialties`, `organization_specialties`, `audit_logs`.

Effects on other modules. A specialty that is off for an organization hides its forms (`SPECIALTY_NOT_ENABLED`).

Error cases. `SPECIALTY_NOT_FOUND`; `SPECIALTY_EXISTS`; `ORGANIZATION_NOT_FOUND`.

Technical references: `backend/src/engine/specialties/specialties.service.ts` · `SpecialtiesService` · `web-app/src/features/engine-studio/services/studio.api.ts`

43.3.12 Built-in seed of dental defaults

Status: [Implemented]

What it does. On every start, the engine registers the code effect handlers and creates the dental starter data if it is missing: the DENTAL specialty, five forms (CROWN, BRIDGE, IMPLANT_CROWN, DENTURE, REPAIR), the three dental workflows and three stage templates.

How the user does it. Nothing.

Behind the scenes — in simple terms. The seed only adds what is missing. It never edits existing rows, so published versions stay exactly as they were and a super admin's changes are not overwritten.

Behind the scenes — technically. `EngineSeedService` (`OnApplicationBootstrap`, after every effect handler has registered). One advisory lock stops an API and a worker that boot together from both inserting. It syncs `workflow_effects` from the in-memory registry (a handler removed from code is marked unavailable, old

versions that mention it remain).

Rules and validations. Insert-if-missing only.

Who can do it. Automatic.

Data read and written. `specialties`, `form_schemas`, `form_schema_versions`, `workflow_templates`, `workflow_template_versions`, `workflow_states`, `workflow_transitions`, `transition_effects`, `stage_templates`, `stage_template_steps`, `workflow_effects`.

Effects on other modules. Without this seed no lab order could be created.

Error cases. A failure is logged at start.

Technical references: `backend/src/engine/seeds/engine-seed.service.ts` · `EngineSeedService.seed` · `backend/src/engine/seeds/dental-workflows.seed.ts`

43.4 Entities and data model

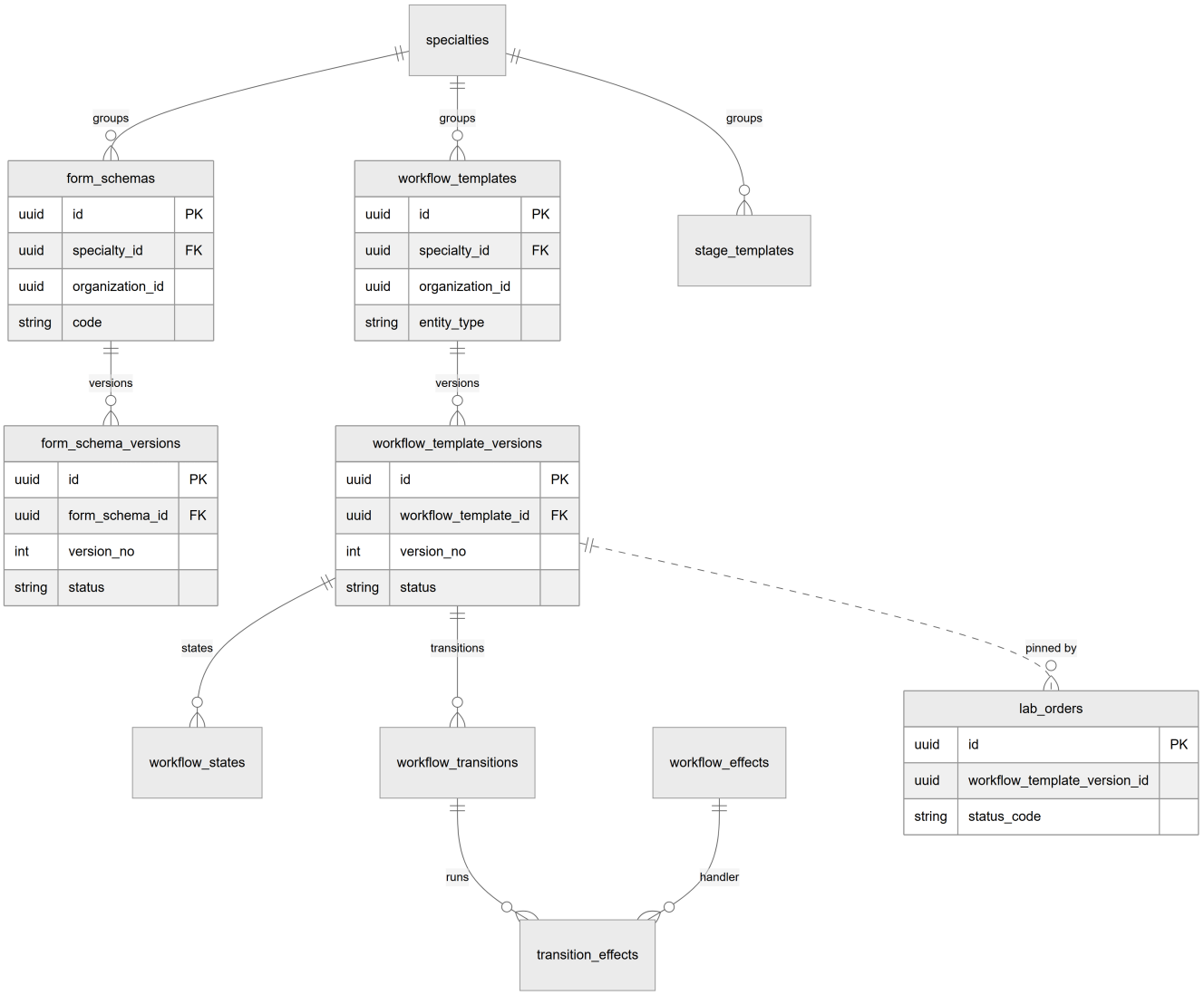

Figure 43.7 — Specialties, forms, workflows and the lab orders that pin versions.

The figure shows `specialties` at the top, with `form_schemas`, `workflow_templates` and `stage_templates` under it. `form_schemas` own `form_schema_versions`. `workflow_templates` own `workflow_template_versions`, which own `workflow_states` and `workflow_transitions`. `workflow_transitions` link to `workflow_effects` through

`transition_effects`. A dotted line goes from `workflow_template_versions` to `lab_orders`, labelled "pinned by".

Each form, workflow and stage template belongs to one specialty; deleting a specialty is blocked. Each version belongs to one template or schema. A published version's states and transitions are written once. A transition's effects are listed in order. The link from a lab order to its version is a stored id (`workflow_template_version_id`); it is drawn as a logical link. Stage steps are removed when their template is removed (cascade). `organization_id` empty on a schema or template means the platform default. Chapter 26 describes every column and constraint.

43.5 Status lifecycles

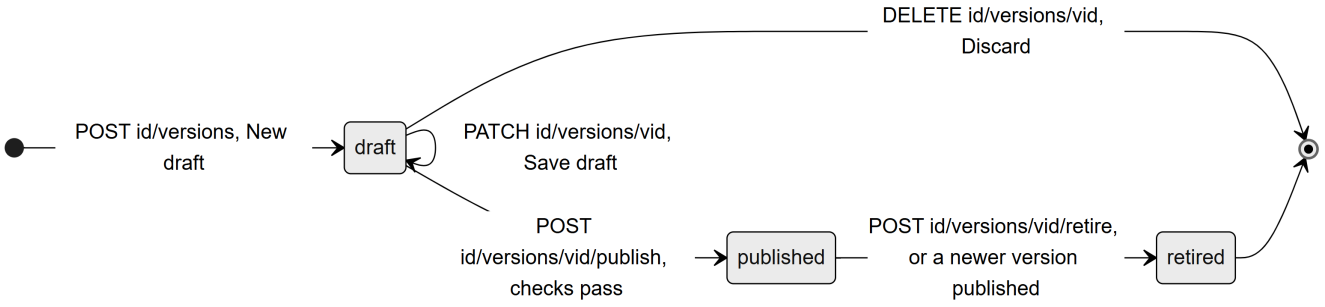

Figure 43.8 — A form or workflow version: draft, published, retired.

A version is created as `draft`. Saving keeps it a draft. Discarding deletes the row (an audit entry keeps its content). Publishing, which requires a clean definition, makes it `published` and retires the previously published version. Retiring a published version makes it `retired` with no replacement, so new orders find no published version until another is published (lab orders then fail with `LAB_ORDER_WORKFLOW_MISSING`). `retired` is final. At most one draft and one published version exist per template at any time. Stage templates have no versions, only an Active switch.

43.6 Connections to other modules

- **Lab Management (Chapter 40)**: order forms, try-in stages and every status change.
- **Lab Portal (Chapter 41)**: lab-side actions are transitions with side `lab`.
- **Lab Finance (Chapter 42)**: the `create_vendor_bill_draft` effect.
- **Notifications (Chapter 44)**: the `notify` effect.
- **Audit (Chapter 60)**: every definition change and every transition is recorded.
- **Access control (Chapter 57)**: Config Studio uses the super admin check, not permission keys.

43.7 Limitations and known gaps

- Config Studio is for super admins only. Clinics cannot create their own forms or workflows, although the data model supports organization-owned ones.
- Effects are fixed in code. Configuration can choose and order them, not create new ones.
- Specialty management and per-organization enablement have no screen.
- Stage templates are not versioned and have no delete.
- Workflows and stage templates have no activity tab.
- The role names in transitions (for example `lab_admin`, `doctor`, `clinic_admin`) are legacy role names derived from permissions; they do not match the five system role templates one to one.

- Studio routes declare no `staticData.can`; protection is the super admin check in the route component plus the API.
- Published versions are immutable by service logic only; no database trigger enforces it.

43.8 Owner's FAQ

Can a clinic owner change the lab order form? No. Only the super admin, in Config Studio. An organization-owned override exists in the data model.

If I publish a new workflow, what happens to open lab orders? Nothing. Each order keeps the version it started on.

What if I retire the only published workflow? New orders cannot be created until another version is published.

Can I add a new action, such as "send SMS to patient"? Only by choosing an existing effect; a new kind of effect needs code.

Can I undo a publish? Not directly. Create a draft from the old version and publish it; the engine retires the one in use.

Where do I see who changed a form? Form schemas → Activity tab.

Why does a workflow fail to publish? The checker lists each problem: a state that cannot be reached, a dead end, a missing end state, an unknown effect or role.

Is the Studio tied to a feature flag? No. It is platform-level and visible only to super admins.

Chapter 44 — Notifications

This chapter describes the Notifications module of the Platform: the in-app inbox that staff and lab members see behind the bell icon, and the clinic settings that decide who is told about which event. It is written for the product owner, who needs to know what is built, what is only prepared, and how a message travels from a business event to a screen. Today the module delivers in-app messages only. Messages to patients by WhatsApp, SMS or email are prepared in the code but switched off.

44.1 What this module is for

A **notification event** is a named business happening that someone should hear about, for example "New lab order" or "Payment receipt". The Platform ships 21 of them. When one happens, the module decides, for each person concerned, whether to tell them and on which channel, writes the message to an outbox table inside the same database transaction as the business change, and a background worker delivers it. For staff and lab members, delivery means a row in their personal inbox, shown behind the bell in the page header.

Who uses it:

- **Every signed-in user** (doctor, receptionist, lab member and others): sees the bell, the unread badge and the inbox list. No permission is needed, because the inbox is always the user's own.
- **Admin** (and the **owner**): opens the notification settings page. This needs the permission `notifications.settings.manage`, which only the Admin system role holds by default.
- **Receptionist**: in the backend, can read and change a patient's messaging opt-outs through `patients.view` and `patients.edit`. No screen shows this today (see 44.7).

Where it appears: there is no sidebar entry. The bell sits in the header on every dashboard page. The settings page is reached from Settings, through the link **Notifications**, which is shown only to users with `notifications.settings.manage`. The route is `/settings/notifications`.

Feature keys: `comms.in_app` ("In-App Notifications", enabled by default) and `comms.whatsapp` ("WhatsApp", disabled by default) exist in the feature catalog, but no code checks either of them. The keys `comms.sms` and `comms.email` are commented out. Notifications run regardless of feature switches, and the notification endpoints carry no feature requirement.

44.2 Screens

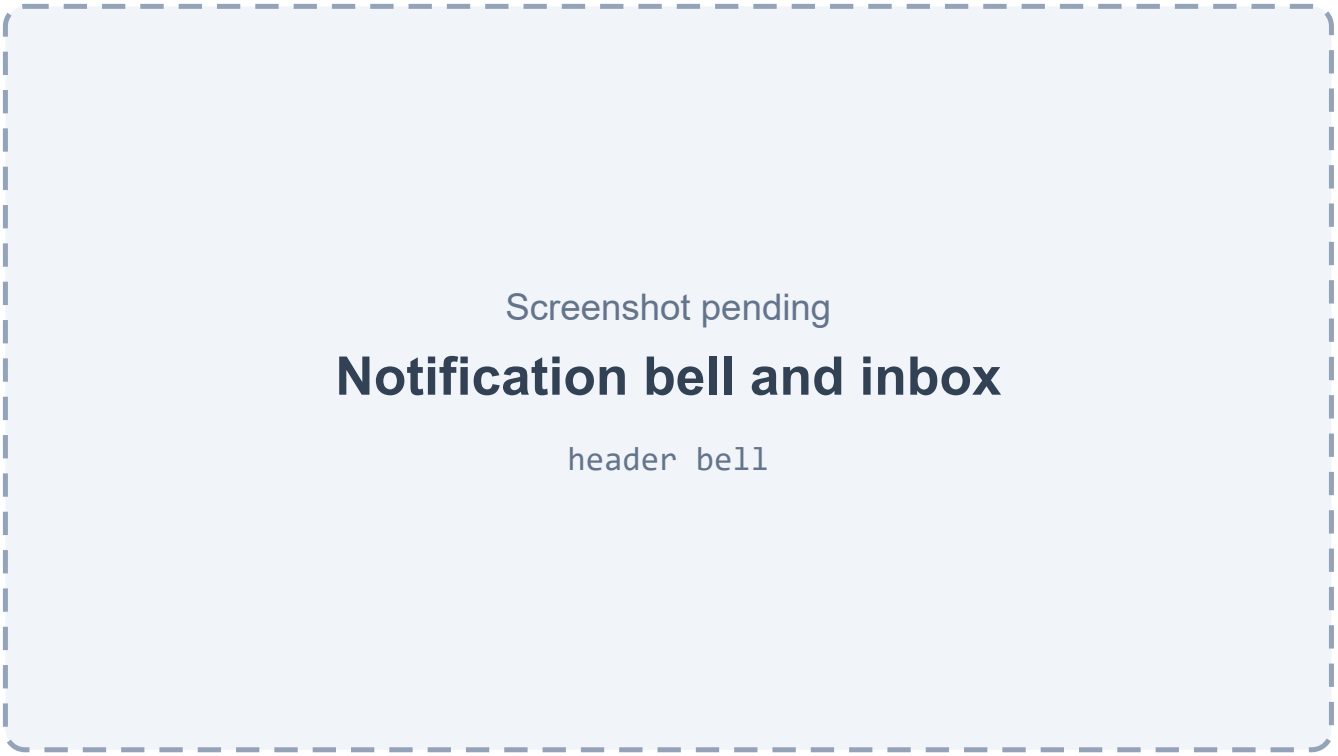

Figure 44.1 — Notification bell and inbox (screenshot pending).

The bell is an outlined icon button in the page header, next to the theme and location controls. When the user has unread items, a small badge shows the count (capped at "99+") and the tooltip reads "Notifications (N unread)". Clicking the bell opens a popover headed "Notifications" with a **Mark all read** button, disabled when nothing is unread or while saving. The list shows the 20 newest items. Each item is a clickable row with a title, a body and a time. Unread rows have a tinted background and a coloured dot. Clicking a row marks it read and, when the item has a link (for example a lab order), closes the popover and opens that page. While loading, three skeleton rows show. On failure the popover shows "Couldn’t load notifications." and a **Try again** button. When empty it shows "You’re all caught up" and "Updates about lab orders and your patients appear here."

Screenshot pending

Notification settings - Team and labs

C:/Program Files/Git/settings/notifications

Figure 44.2 — Notification settings, Team & labs tab (screenshot pending).

The settings page has the title "Notifications", the subtitle "In-app notifications for your team — and the log of every message.", and a back button to Settings. It has two tabs: **Team & labs** and **Message log**. A **Patients** tab exists in the code but is commented out, because patients have no working channel. The Team & labs tab has a "Master switches" section with one card per audience (doctor, coordinator, lab). Each card has an on/off switch, a description line such as "Updates about the doctor's own lab orders.", the current quiet-hours summary ("No quiet hours" or "Quiet hours 22:00 – 08:00") and a **Quiet hours** button. The "Events" section is a table with the columns Event and Sent to, one switch per audience and channel, and an action column. Events flagged mandatory show a lock and a "Mandatory" badge, and their in-app switch is locked on with the tooltip "Mandatory — always sent in-app". Each row has an icon button with the tooltip "Edit wording — ".

Screenshot pending

Quiet hours sheet

C:/Program Files/Git/settings/notifications

Figure 44.3 — Quiet hours sheet (screenshot pending).

The quiet-hours sheet slides in from the side. It explains: "Messages that fall inside quiet hours are held and sent when they end. In-app notifications are never held." It has a "Use quiet hours" switch, a start time, an "Until" time (the hint says it may be the next morning, for example 22:00 → 08:00) and a Timezone field with the placeholder "Asia/Karachi" and the hint "Leave empty to use each location's own timezone." Save is disabled until the form is valid and changed.

Screenshot pending

Edit wording sheet

C:/Program Files/Git/settings/notifications

Figure 44.4 — Edit wording sheet (screenshot pending).

The wording sheet lets the clinic replace the platform text of one event: "Your clinic's own message. Placeholders in {braces} are filled in for each recipient." It has a Channel selector, a Language selector (English, Urdu, Roman Urdu), a Subject or Title field, a Message field, and, for WhatsApp, a "WhatsApp template ID" field. It shows the platform default for comparison ("No default yet" when none exists). Buttons are Save and Reset. Reset asks "Reset to the default wording?" and, when confirmed, deletes the clinic's own wording. Closing with edits asks "Discard changes?".

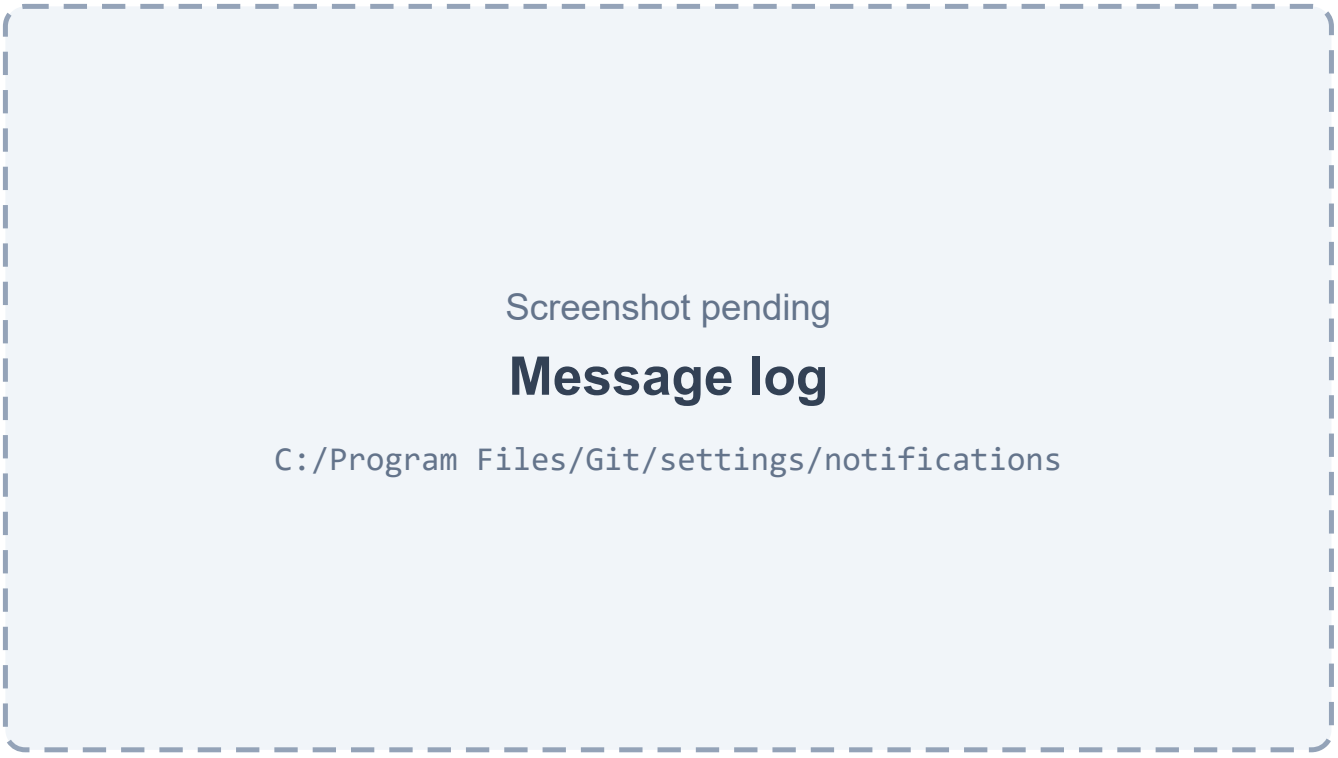

Figure 44.5 — Message log (screenshot pending).

The Message log tab lists every notification the clinic has produced, including the ones that were skipped or held. It has a search box ("Search by name, number or text…"), a status filter ("All statuses" and the seven statuses), a channel filter ("All channels"), and a table with the columns Recipient, Message, Channel and Status, paged with a page-size selector (default 20). Status badges read Queued, Held (quiet hours), Sent, Delivered, Failed, Skipped — off and Skipped — opted out. The empty state is "No messages yet" with "Every notification — sent, held or skipped — is listed here with the reason." A filter with no hits shows "No messages match your filters."

44.3 Features

44.3.1 See notifications in the header bell

Status: [Implemented]

What it does. Shows the signed-in user's in-app notifications, with an unread count that refreshes by itself.

How the user does it.

1. Look at the bell in the header. A badge shows the unread count.
2. Click the bell to open the popover.
3. Click an item to open the linked page.

Behind the scenes — in simple terms. Each person has their own inbox. It spans every clinic the person works at, because it is tied to the user and not to a location. The badge asks the server for the count every 30 seconds and when the browser window regains focus. There is no live push.

Behind the scenes — technically. `GET /me/notifications?page&limit&unreadOnly` (limit 1–50, default 20) and `GET /me/notifications/unread-count` in `MyNotificationsController`, which is `@ContextFree()`: no organization or location headers are needed, and lab users have none. `UserNotificationsService.list` reads `user_notifications` for the user, newest first. The web hook `useUnreadNotificationCount` polls with `refetchInterval` 30 000 ms; the popover loads `useMyNotifications({ limit: 20 })` only while open.

Rules and validations.

- Always scoped to the signed-in user; there is no way to read another user's inbox.
- The list is paged; the bell shows only the first 20.

Who can do it. Any signed-in user; no permission key and no feature key.

Data read and written. Reads `user_notifications`.

Effects on other modules. None. Links in items point to pages such as `/lab-orders/<id>`.

Error cases. A 401 when no user is resolved; the popover shows "Couldn't load notifications." on any load failure.

Technical references: `backend/src/notifications/controllers/my-notifications.controller.ts` · `MyNotificationsController`, `UserNotificationsService` · `web-app/src/features/notifications/components/NotificationBell.tsx`, `hooks/useMyNotifications.ts`

44.3.2 Mark notifications as read

Status: [Implemented]

What it does. Marks one notification, or all of them, as read.

How the user does it.

1. Click an item (marks it read and follows its link), or click **Mark all read**.

Behind the scenes — in simple terms. Reading sets a time stamp on the inbox row. Nothing is ever deleted from the inbox, and there is no "mark as unread".

Behind the scenes — technically. `POST /me/notifications/:id/read` and `POST /me/notifications/read-all` (declared first so the route does not clash). `markRead` loads the row by id and user and sets `read_at` if it is empty; it is idempotent. `markAllRead` updates every row of the user where `read_at IS NULL` and returns `{ updated }`. The web mutations invalidate the inbox queries on completion.

Rules and validations. `:id` must be a UUID. Only the owner's rows are touched.

Who can do it. Any signed-in user.

Data read and written. Writes `user_notifications.read_at`.

Effects on other modules. None.

Error cases. `NOTIFICATION_NOT_FOUND` (404) when the id does not exist for this user.

Technical references: `UserNotificationsService.markRead`, `markAllRead` · `web-app/src/features/notifications/hooks/useMyNotifications.ts`

44.3.3 Raise a notification event (how messages are created)

Status: [Implemented]

What it does. Any module can notify people about a business event. The module decides who is told, on which channel, with what wording and when, and records the result for every recipient and channel.

How the user does it. Users do not do this directly. It happens as a side effect of actions such as submitting a lab order or recording a lab payment (see 44.6).

Behind the scenes — in simple terms. For each person concerned and each channel their audience uses, the module asks six questions in a fixed order and the first match decides:

1. Is this a mandatory event for an internal audience, on the in-app channel? Then send, whatever the settings say.
2. Is the audience's master switch off? Then skip.
3. Is the rule for this event, audience and channel off? Then skip.
4. Has the patient opted out of this channel? Then skip.
5. Is it inside quiet hours (not in-app)? Then hold until quiet hours end.
6. Otherwise send.

A row is written for every outcome, so the message log can explain why something was not sent. Defaults when the clinic has changed nothing: internal audiences (doctor, coordinator, lab) are on with in-app on; the patient audience and every rule for it are off.

Behind the scenes — technically. `NotificationService.enqueue(manager, input)` runs inside the caller's transaction. It loads the event, de-duplicates recipients by type and id, rejects an audience the event does not allow (`NOTIFICATION_AUDIENCE_INVALID`), reads audience settings, rules, templates, contact details and patient preferences, and calls the pure function `decide()` for each recipient and channel in `channelsFor()` . A template is chosen by rule, then by channel and language (clinic override before platform default; language from the recipient, then the patient preference, then English), and `renderTemplate` fills `{placeholders}` ; unknown placeholders become empty. Rows are inserted into `notification_outbox` in one statement, then `pg_notify('notification_outbox')` is issued in the same transaction, so listeners wake only on commit. A recipient with no phone or email on an external channel gets a `failed` row with reason `no_address` . Because the work is inside the caller's transaction, a rollback removes the messages too.

Rules and validations.

- Unknown event code: `NOTIFICATION_EVENT_UNKNOWN` (422); a typo fails the business action instead of silently dropping messages.
- Mandatory in-app messages ignore master switches and rules.
- Quiet hours never hold in-app messages.
- The same person listed twice (or under two audiences) gets one row per channel.

Who can do it. Internal service call; no permission.

Data read and written. Reads `notification_events` , `notification_audience_settings` , `notification_rules` , `notification_templates` , `patient_comm_prefs` , `users` , `patients` , `organizations` ; writes `notification_outbox` .

Effects on other modules. 44.6 lists the emitters.

Error cases. `NOTIFICATION_EVENT_UNKNOWN` , `NOTIFICATION_AUDIENCE_INVALID` , `PATIENT_NOT_FOUND` (404). All abort the caller's transaction.

Technical references: `backend/src/notifications/services/notification.service.ts` · `NotificationService.enqueue` · `decision/notification-decision.ts` · `decide`, `quietHoursEnd` · `templates/template-render.ts`

44.3.4 Deliver messages in the background

Status: [Implemented] (in-app channel only)

What it does. A worker sends committed outbox rows, retries failures and releases rows held for quiet hours.

How the user does it. Nothing; it is automatic. The user sees the result in the bell, and the clinic admin sees it in the message log.

Behind the scenes — in simple terms. The worker wakes the moment a database message arrives, and also checks every two seconds in case it missed one. It takes a batch of due messages, sends each one and records the outcome. If a send fails it waits and tries again, up to five times, then marks the message failed.

Behind the scenes — technically. `NotificationRelayService` runs only where background jobs run (`NotificationsRuntimeModule.forRole`; never on a second API process). It opens a dedicated `LISTEN notification_outbox` connection and a 2 000 ms `setInterval`; if the listen connection fails it logs a warning and polls only. Each tick calls `NotificationDispatcher.drain()`, which coalesces concurrent calls. `claim()` is one `UPDATE … WHERE id IN (SELECT … status IN ('queued','held_quiet_hours') AND scheduled_for <= now() ORDER BY scheduled_for LIMIT 50 FOR UPDATE SKIP LOCKED) RETURNING …` that also increments `attempts` and pushes `scheduled_for` forward by a 5-minute lease, so a crashed worker's rows reappear. The provider is called with no transaction open. The result is written back only when `attempts` is unchanged. Failures wait 30 s, 2 min, 10 min, then 1 h; a `PermanentSendError` or the fifth attempt gives `failed`. Delivery is at-least-once. Providers are chosen per channel by `ChannelProviderRegistry`: `InAppChannelProvider` for in-app, and `DevLogChannelProvider` for WhatsApp, SMS and email, which only writes a log line and reports success. A `PayloadFinalizers` hook lets an event fill in data that must never be stored, used by the lab magic-link event.

Rules and validations. Batch size 50; at most 5 attempts; in-app copies are unique per outbox row (`ON CONFLICT (outbox_id) DO NOTHING`), so a retry never shows twice. An in-app message to a non-user recipient fails permanently.

Who can do it. Automatic; job role only.

Data read and written. Reads and updates `notification_outbox`; inserts `user_notifications`.

Effects on other modules. None. If the worker is not running, no message is delivered, although rows are still recorded.

Error cases. Failures are stored in `last_error` and shown in the message log. A lost listen connection is logged once and polling continues.

Technical references: `backend/src/notifications/delivery/notification-relay.service.ts` · `NotificationRelayService` · `notification-dispatcher.service.ts` · `NotificationDispatcher` · `in-app.provider.ts` · `dev-log.provider.ts` · `channel-provider.registry.ts` · `notifications-runtime.module.ts`

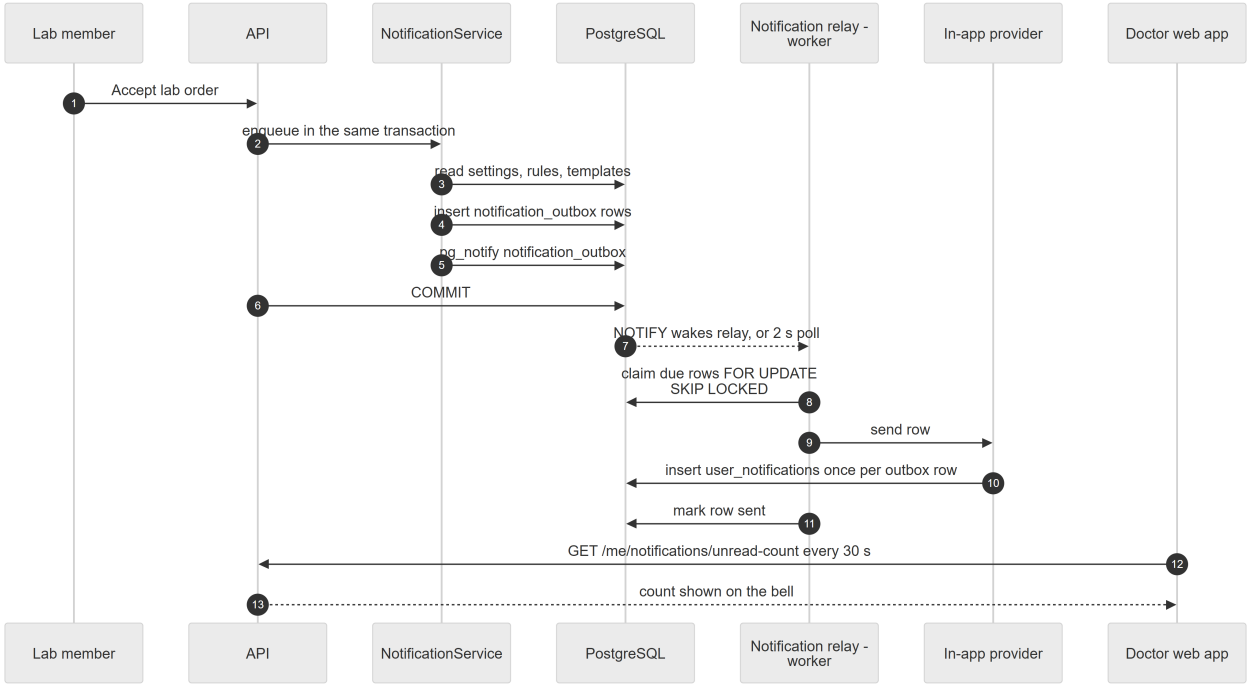

Figure 44.6 — A lab member accepts an order and the ordering doctor sees it behind the bell.

The diagram has seven participants: Lab member, API, NotificationService, PostgreSQL, Notification relay (worker), In-app provider and Doctor web app. The Lab member asks the API to accept a lab order. In the same transaction the API calls NotificationService, which reads settings, rules and templates, inserts outbox rows and issues a database notify. The API then commits. The commit wakes the relay (or its two-second poll), which claims due rows with a skip-locked update, hands each row to the In-app provider, which inserts one inbox row per outbox row, and the relay marks the outbox row sent. Separately, the Doctor web app asks the API for the unread count every 30 seconds and shows it on the bell.

44.3.5 Choose who is notified (master switches and rules)

Status: [Implemented]

What it does. The clinic admin switches an audience on or off and turns individual event rules on or off.

How the user does it.

1. Open **Settings** → **Notifications**, tab **Team & labs**.
2. Use the switch on an audience card to turn the whole audience on or off.
3. In the Events table, use the switch in the audience and channel column of an event.

Behind the scenes — in simple terms. A switch the admin never touched uses the default (internal on, patient off). Mandatory events keep their in-app switch locked on. Only in-app switches appear, because the other channels are not offered to any audience.

Behind the scenes — technically. `GET /notification-settings` returns audiences, events with their rules (defaults filled in, `isDefault`, `locked`), `channelsByAudience` and languages. `PATCH /notification-settings/audiences/:audience` (body `isEnabled`, `quietStart`, `quietEnd`, `timezone`) upserts `notification_audience_settings` under a `pessimistic_write` row lock. `PUT /notification-settings/rules` takes 1–500 `{eventCode, audience, channel, isEnabled}` changes and upserts `notification_rules`, skipping unchanged ones and returning `{changed}`. Both write an audit entry in the same transaction (`notification_audience_setting`, `notification_rule`).

Rules and validations.

- The event must exist, include the audience, and the audience must be reachable on the channel (`NOTIFICATION_EVENT_UNKNOWN`, `NOTIFICATION_AUDIENCE_INVALID`, `NOTIFICATION_CHANNEL_INVALID`).
- Switching a locked mandatory in-app rule off is refused with `NOTIFICATION_RULE_LOCKED`; switching it on is accepted and ignored.
- Audience must be one of patient, doctor, coordinator, lab.

Who can do it. `notifications.settings.manage` (feature `core`, no scope); Admin by default. No feature key.

Data read and written. `notification_events`, `notification_audience_settings`, `notification_rules`, `audit_logs`.

Effects on other modules. Changes apply to the next event raised by any module.

Error cases. The codes above, all HTTP 422; 403 without the permission.

Technical references: `backend/src/notifications/controllers/notification-settings.controller.ts` · `NotificationSettingsService.overview`, `updateAudience`, `updateRules` · `web-app/src/features/notifications/pages/NotificationSettingsPage.tsx`, `components/AudienceSwitchCard.tsx`, `EventRulesTable.tsx`, `RuleSwitch.tsx`

44.3.6 Set quiet hours

Status: [Partial]

What it does. Lets an audience have a daily window in which messages are held. The setting is saved and applied by the decision step, but it has no visible effect today, because quiet hours never hold in-app messages and in-app is the only channel in use.

How the user does it.

1. On an audience card, click **Quiet hours**.
2. Switch on "Use quiet hours", set the start and "Until" times, and optionally a timezone.
3. Save.

Behind the scenes — in simple terms. The window may cross midnight (22:00 to 08:00). Without a timezone, the location's own timezone is used. A held message is released when the window ends.

Behind the scenes — technically. Same endpoint as 44.3.5. `quietHoursEnd(now, quiet)` compares local `HH:MM` and returns the end instant, or null outside the window; start equal to end means no window. A held row has status `held_quiet_hours` and `scheduled_for` set to the end.

Rules and validations. Times are 24-hour `HH:MM`. Start and end are required together (`NOTIFICATION_QUIET_HOURS_INVALID`), they must differ, and a non-empty timezone must be a valid IANA zone. Without a location, UTC is used.

Who can do it. `notifications.settings.manage`.

Data read and written. `notification_audience_settings`.

Effects on other modules. None today.

Error cases. `NOTIFICATION_QUIET_HOURS_INVALID` (422): one time missing, equal times, or unknown timezone.

Technical references: `NotificationSettingsService.updateAudience` · `decision/notification-decision.ts` · `web-app/src/features/notifications/components/QuietHoursSheet.tsx`, `hooks/useQuietHoursForm.ts`

44.3.7 Edit message wording in several languages

Status: [Implemented]

What it does. Lets the clinic replace the platform text of an event, per channel and language (English, Urdu, Roman Urdu).

How the user does it.

1. In the Events table, click the edit-wording icon on a row.
2. Pick a channel and a language.
3. Edit the title and message, using placeholders such as `{clinic_name}`.
4. Save, or choose Reset to return to the platform text.

Behind the scenes — in simple terms. The platform keeps default wording for each event. A clinic's own wording overrides it for that clinic only. The page shows the placeholders the event allows. Resetting deletes the clinic's own text.

Behind the scenes — technically. `GET /notification-settings/templates?eventCode` returns platform rows (organization null) and the clinic's (`isOverride`). `PUT /notification-settings/templates` upserts by organization, event, channel and language under a row lock, and stores the placeholders used. `DELETE /notification-settings/templates/:id` clears `notification_rules.template_id` values that point to it, deletes the row and audits a void. Platform defaults are seeded on boot, insert-if-missing, under an advisory lock.

Rules and validations. Body required, at most 1 000 characters; subject at most 200; template reference at most 200. Every `{placeholder}` must be a common one (`clinic_name`, `recipient_name`) or one of the event's variables (`NOTIFICATION_TEMPLATE_INVALID` lists the unknown ones). The channel must be one the event's audiences can use.

Who can do it. `notifications.settings.manage`.

Data read and written. `notification_templates`, `notification_rules`, `audit_logs`.

Effects on other modules. Changes the text of future messages only; existing inbox rows keep the text they were created with.

Error cases. `NOTIFICATION_TEMPLATE_INVALID`, `NOTIFICATION_TEMPLATE_NOT_FOUND` (404 on reset), `NOTIFICATION_CHANNEL_INVALID`, `NOTIFICATION_EVENT_UNKNOWN`.

Technical references: `NotificationSettingsService.templates`, `upsertTemplate`, `resetTemplate` · `templates/template-render.ts` · `seeds/notification-seed.service.ts` · `web-app/src/features/notifications/components/TemplateSheet.tsx`, `hooks/useTemplateForm.ts`

44.3.8 Review the message log

Status: [Implemented]

What it does. Shows every notification the clinic generated, with status, reason and error.

How the user does it.

1. Open **Settings** → **Notifications** → **Message log**.
2. Search by recipient name, address or text, and filter by status or channel.
3. Page through results.

Behind the scenes — in simple terms. Skipped and held messages are listed too, so an admin can see why someone was not told.

Behind the scenes — technically. `GET /notification-settings/log?page&limit&status&eventCode&channel&audience&search` (limit 1–100, default 20) over `notification_outbox` for the active organization, newest first. Search matches recipient name, address or body inside the JSON payload. Rows expose status, reason, attempts, `last_error`, `sent_at` and the cause entity.

Rules and validations. Status, channel and audience must be valid enum values.

Who can do it. `notifications.settings.manage`.

Data read and written. Reads `notification_outbox`.

Effects on other modules. None.

Error cases. 403 without the permission; 400 on an invalid filter value.

Technical references: `NotificationSettingsService.log` · `web-app/src/features/notifications/components/MessageLogPanel.tsx`

44.3.9 Record a patient's messaging opt-outs and language

Status: [Partial]

What it does. Stores whether a patient has opted out of WhatsApp, SMS or email and their preferred message language. The backend works and the decision step honours it. No screen is reachable: the card that would show it is commented out in the patient page while notifications are in-app only.

How the user does it. Not available in the web app. The component `PatientMessagesCard` exists but is not rendered.

Behind the scenes — in simple terms. If a patient opts out of a channel, messages on that channel are recorded as skipped. The first time any opt-out is switched on, the time is stored.

Behind the scenes — technically. `GET` and `PUT /notification-settings/patients/:patientId/prefs`. `PUT` upserts `patient_comm_prefs` under a row lock and audits it. The patient must belong to the organization.

Rules and validations. `preferredLanguage` is `en`, `ur`, `roman_ur` or null.

Who can do it. `patients.view` to read, `patients.edit` to change; the Receptionist holds both by default.

Data read and written. `patient_comm_prefs`, `patients`, `audit_logs`.

Effects on other modules. Would suppress patient messages once patient channels exist.

Error cases. `PATIENT_NOT_FOUND` (404).

Technical references: `NotificationSettingsService.patientPrefs`, `updatePatientPrefs` · `web-app/src/features/notifications/components/PatientMessagesCard.tsx` (unused) · `web-app/src/features/patients/components/PatientDetailTabs.tsx`

44.3.10 Notify through a workflow transition

Status: [Partial]

What it does. A workflow transition can carry a `notify` effect that raises an event inside the transition's transaction. The effect and the recipient registry are built, but the shipped lab code paths raise their events directly through the service, and the repository contains no transition definition that uses the effect.

How the user does it. A workflow author adds the effect with the event code (and optionally a narrower list of audiences) in the Config Studio.

Behind the scenes — in simple terms. The engine does not know any table. Each module registers how to find the doctor, lab and patient of its own record type.

Behind the scenes — technically. `NotifyEffect` (code `notify`) validates `params.event` and `params.audiences`, looks up the resolver for `request.entityType` in `NotificationRecipientsRegistry`, and calls `enqueue` with the resolved recipients. A missing resolver or malformed params raises `WORKFLOW_MISCONFIGURED`. The lab-order workflow registers its resolver.

Rules and validations. Unknown event codes fail the transition.

Who can do it. Whoever may perform the transition.

Data read and written. `notification_outbox`.

Effects on other modules. The Workflow engine chapter (Chapter 43) describes effects.

Error cases. `WORKFLOW_MISCONFIGURED`; `NOTIFICATION_EVENT_UNKNOWN`.

Technical references: `backend/src/notifications/effects/notify.effect.ts` · `NotifyEffect` · `effects/notification-recipients.registry.ts` · `NotificationRecipientsRegistry` · `backend/src/lab-orders/workflow/order-workflow.service.ts`

44.3.11 Send to patients by WhatsApp, SMS or email

Status: [Not implemented]

What it does. Nothing reaches a patient. Patients have no in-app inbox, and the audience-to-channel table gives the patient audience no channels (WhatsApp and SMS are commented out). No patient row is ever queued. The WhatsApp, SMS and email code path exists only as a development stand-in that writes a log line. The seeded events addressed to patients (such as "Payment receipt") are recorded in the catalog, but produce no patient message.

How the user does it. Not available. The Patients tab on the settings page is commented out.

Behind the scenes — technically. `CHANNELS_BY_AUDIENCE[patient]` is empty; `DevLogChannelProvider` covers the three channels but would only log. No real provider is registered. The WhatsApp template ID field in the wording sheet is stored but never used.

Rules and validations. None active.

Who can do it. Not applicable.

Data read and written. None.

Effects on other modules. Lab events that list the patient audience produce no patient message.

Error cases. None.

Technical references: `backend/src/notifications/notifications.enums.ts` · `CHANNELS_BY_AUDIENCE`, `CHANNELS_BY_RECIPIENT` · `delivery/dev-log.provider.ts` · `backend/src/feature-flags/feature-catalog.seed.ts`

44.3.12 Appointment reminders

Status: [Not implemented]

What it does. Nothing. No notification event exists for appointment reminders. The location settings carry reminder fields, but no job reads them, and the location settings page hides its reminder card until delivery exists. The seeded events "Try-in appointment reminder" and "Appointment moved (lab delay)" have no emitter either (44.6).

How the user does it. Not available.

Behind the scenes — technically. No `@Cron` or listener in the appointments or notifications modules raises a reminder.

Rules and validations. None.

Who can do it. Not applicable.

Data read and written. None.

Effects on other modules. The Appointments chapter (Chapter 29) lists reminders as a gap.

Error cases. None.

Technical references: `backend/src/notifications/seeds/notification-events.seed.ts` · `web-app/src/features/locations/LocationSettingsFeature.tsx`

44.4 Entities and data model

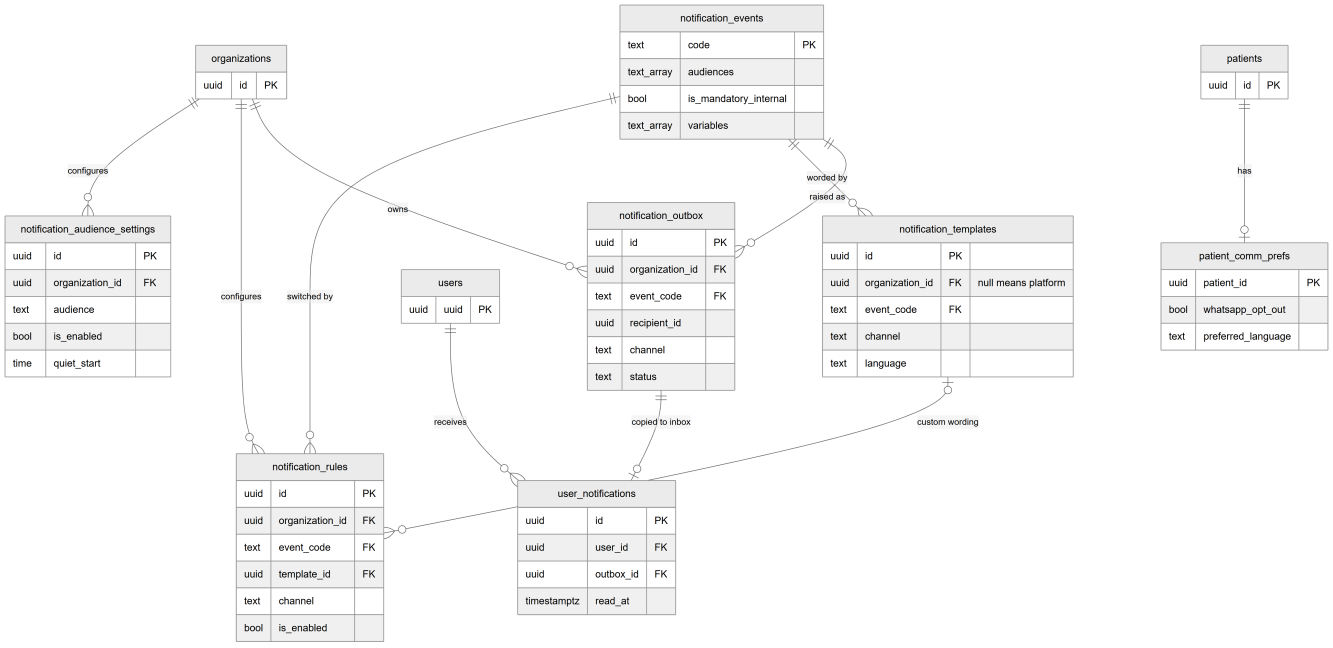

Figure 44.7 — The notification tables and the tables around them.

The diagram shows seven notification tables and three neighbours. `notification_events` (key `code`, `audiences`, `is_mandatory_internal`, `variables`) is the catalog. `notification_rules` (organization, event, audience, channel, `is_enabled`, optional template) holds one on/off decision per event, audience and channel. `notification_templates` holds wording per event, channel and language; `organization_id` is null for platform defaults. `notification_audience_settings` holds one row per organization and audience with the master switch and quiet hours. `notification_outbox` records every message decision per recipient and channel. `user_notifications` is the inbox copy. `patient_comm_prefs` holds a patient's opt-outs and language. `organizations`, `users` and `patients` are the owners.

The relationships in words:

- Audience settings and rules belong to one organization; deleting the organization deletes them (ON DELETE CASCADE). A rule is unique per organization, event, audience and channel; a setting is unique per organization and audience.
- Each rule and each template points to exactly one event; an event cannot be deleted while they exist (RESTRICT).

- A rule may point to one template for custom wording. Deleting the template sets the rule's pointer to NULL.
- A template belongs to an organization or to nobody (platform default). Deleting the organization deletes its templates.
- Each outbox row belongs to one organization and one event, both RESTRICT. Its recipient is a user, a patient or a lab, identified by `recipient_type` and `recipient_id`. These are logical links with no foreign key.
- An outbox row has at most one inbox row (`outbox_id` is unique, RESTRICT). Each inbox row belongs to one user; deleting the user deletes their inbox.
- Each patient has at most one preferences row; deleting the patient deletes it.

Key outbox columns: `audience`, `recipient_type`, `channel`, `status`, `scheduled_for`, `attempts` (5 at most), `provider_message_id`, `last_error`, `sent_at`, `entity_type` and `entity_id` (what caused it) and a `payload` JSON (title, body, link, language, address, reason). A partial index covers rows with status queued or held. Identifiers are time-ordered UUID v7. Settings, rules, templates and preferences use a version column.

44.5 Status lifecycles

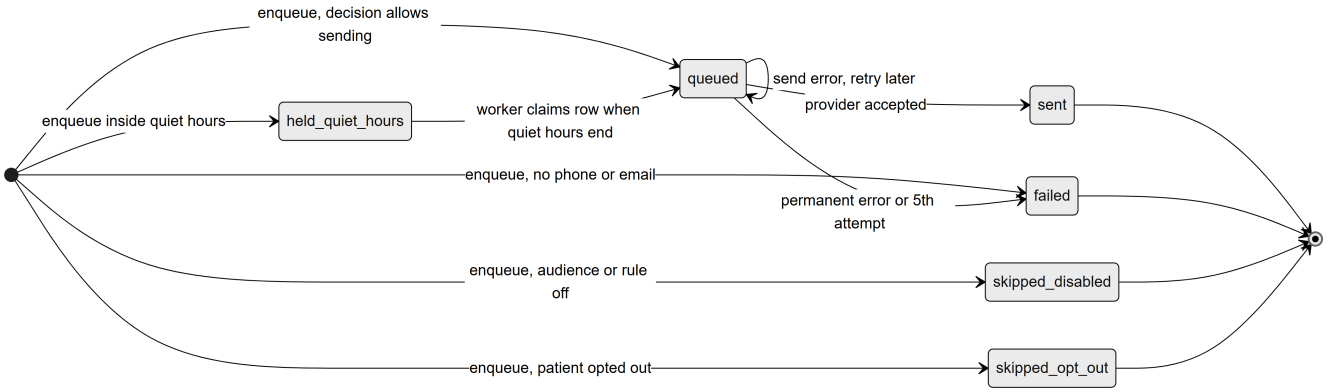

Figure 44.8 — Lifecycle of a `notification_outbox` row.

An outbox row starts in one of five states, chosen by the decision step at enqueue time. `queued` means it is due for the worker. `held_quiet_hours` means it must wait for the end of quiet hours (`scheduled_for` is set to that moment). `skipped_disabled` means the audience or the rule was off. `skipped_opt_out` means the patient opted out. `failed` at creation means the recipient had no phone number or email address. A skipped or failed row is final. A held row becomes eligible when its time arrives, and the worker claims it just as it claims a queued row. From `queued`, a successful send moves the row to `sent`. A transient error keeps it `queued` with a later `scheduled_for` (30 s, 2 min, 10 min, 1 h). A permanent error, or the fifth attempt, moves it to `failed`. The status `delivered` exists in the enum and in the web labels, but no code sets it, so it is never reached.

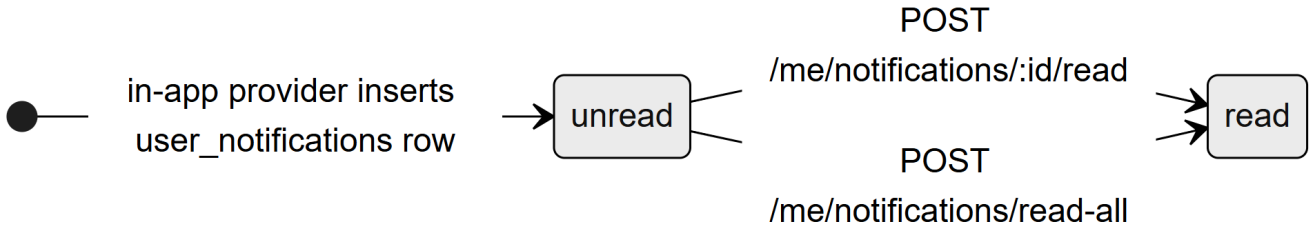

Figure 44.9 — Read state of a `user_notifications` row.

An inbox row is created unread when the in-app provider delivers an outbox row. It becomes read, by setting `read_at`, either when one notification is marked read or when all are marked read. There is no way back to unread and no deletion.

44.6 Connections to other modules

All current emitters are in the lab modules. The 21 seeded events and who raises them:

Event	Audiences	Raised by
order.submitted	lab (mandatory)	lab-orders, order submit
order.accepted	doctor, coordinator	lab-access, order actions
order.rejected, order.clarification_requested, order.expected_date_changed	doctor, coordinator (mandatory)	lab-access, order actions
order.magic_link	lab	lab-access, magic links
item.ready	doctor, coordinator, patient	lab-production, lab work
stage.sent_to_clinic	doctor, coordinator	lab-production, shipments
stage.approved, stage.correction	lab	lab-production, clinic production
shipment.received	lab	lab-production, shipments
shipment.lost	lab, doctor (mandatory)	lab-production, shipments
remake.requested	lab (mandatory)	lab-production, workflow
remake.agreed	lab (mandatory)	lab-production, clinic production
order.completed	doctor, patient	lab-billing, when an order reaches completed
payment.receipt	patient	lab-billing, lab payments
payment.outstanding_reminder	patient	lab-billing, lab invoices, on request
order.overdue, shipment.dispatched, tryin.appointment_due, appointment.rescheduled_lab_delay	various	no emitter found

The table groups the events by emitter. Every audience listed as patient produces no message today (44.3.11). The events with no emitter exist only in the catalog. Billing for clinic treatments (the Billing chapter, Chapter 34) raises no notification at all: the two "payment" events belong to lab patient invoices. Lab members receive messages as users in the audience `lab`, except the magic-link event, which is addressed to the lab itself. The Dental lab chapter (Chapter 40) and the Lab portal chapter (Chapter 41) describe the screens that trigger them. The Notification mechanics chapter (Chapter 59) covers the cross-cutting design. Notification settings changes are audited (the Audit logging chapter, Chapter 60). The Patients chapter (Chapter 28) owns the patient page where the messaging card is hidden.

44.7 Limitations and known gaps

- Only the in-app channel works. WhatsApp, SMS and email are not delivered, and the patient audience has no channel.
- Appointment reminders do not exist.
- Four seeded events have no emitter; billing for clinic treatments emits nothing.
- The status `delivered` is never set.
- The patient opt-out screen is hidden, so the opt-out backend cannot be used from the web app.
- Quiet hours are saved but cannot hold any message today.
- The inbox updates by 30-second polling, not by live push, and the bell shows only the latest 20.

- No sidebar entry; the settings page is reached only from Settings.
- Feature keys `comms.in_app` and `comms.whatsapp` do not control anything.
- Delivery needs the worker process. Without it, messages stay queued.
- The inbox has no delete and no mark-as-unread.

44.8 Owner's FAQ

Can I message patients about appointments? No. No patient channel is active and there is no reminder event.

A doctor says they never got a lab notification. Where do I look? Open Settings → Notifications → Message log, search for the doctor, and read the status and reason. Check that the doctor's audience is switched on and that the doctor has a staff login linked to the employee record.

Can I switch off a notification? Optional events yes, using the event table. Events marked Mandatory (for example a rejected order) cannot be switched off in-app.

Who can change notification settings? Users with `notifications.settings.manage`, which is the Admin role by default, and the owner.

What happens if a business action fails after a message was queued? The message disappears with the rolled-back transaction, because it is written in the same transaction.

Will a message be sent twice? Delivery is at-least-once, but the inbox copy is unique per outbox row, so the user sees it once.

Can I change the wording? Yes, per event, channel and language, using only the placeholders the editor allows. Reset returns the platform text.

Where are messages stored? In `notification_outbox` (the record of every decision) and `user_notifications` (the inbox).

Is it live? The bell updates every 30 seconds and on window focus, not instantly.

Chapter 45 — Subscriptions and Onboarding

This chapter describes how a new clinic joins the Platform and how its right to keep using the Platform is tracked. It covers self-signup, email verification, the one-time setup wizard, the 14-day trial, plan limits, the read-only mode that starts after a subscription runs out, and the super admin screens that record manual payments. It is written for the product owner, who needs to know what a clinic sees, what the backend enforces and where the gaps are.

45.1 What this module is for

A **subscription** is the clinic's permission to use the Platform for a period of time under a plan. Plans are defined in code, not in a table. Nothing charges the clinic automatically: the clinic pays by bank transfer, JazzCash or similar, and a **super admin** records the payment by hand, which extends the period.

Onboarding is the path from "I found the Platform" to "my clinic is set up". A visitor creates an account and a clinic in one step, starts a 14-day trial on the Professional plan, and then completes a three-step setup wizard.

Who uses it:

- **Visitor, then owner**: registers, verifies the email address, completes the wizard and watches the trial.
- **Owner**: opens "Plan & billing" to see the plan, usage and payment history. Only the owner (or a super admin) can use the setup and plan screens. Other staff see only the warning banners.
- **Super admin**: lists every clinic's subscription, changes plan, dates and limits, records payments and edits the payment instructions text.

Where it appears: the super admin sidebar item **Subscriptions** (key `nav.subscriptions`, route `/subscriptions`, super admin only). Owners reach **Plan & billing** from Settings (route `/settings/subscription`), where the link is shown to owners only. `/register`, `/verify-email` and `/setup` have no sidebar entry.

Permissions and feature keys: none. These endpoints are `@AnyMember()` with an owner check in code, `@Public()`, or `@RequireSuperAdmin()`. There is no feature key.

45.2 Screens

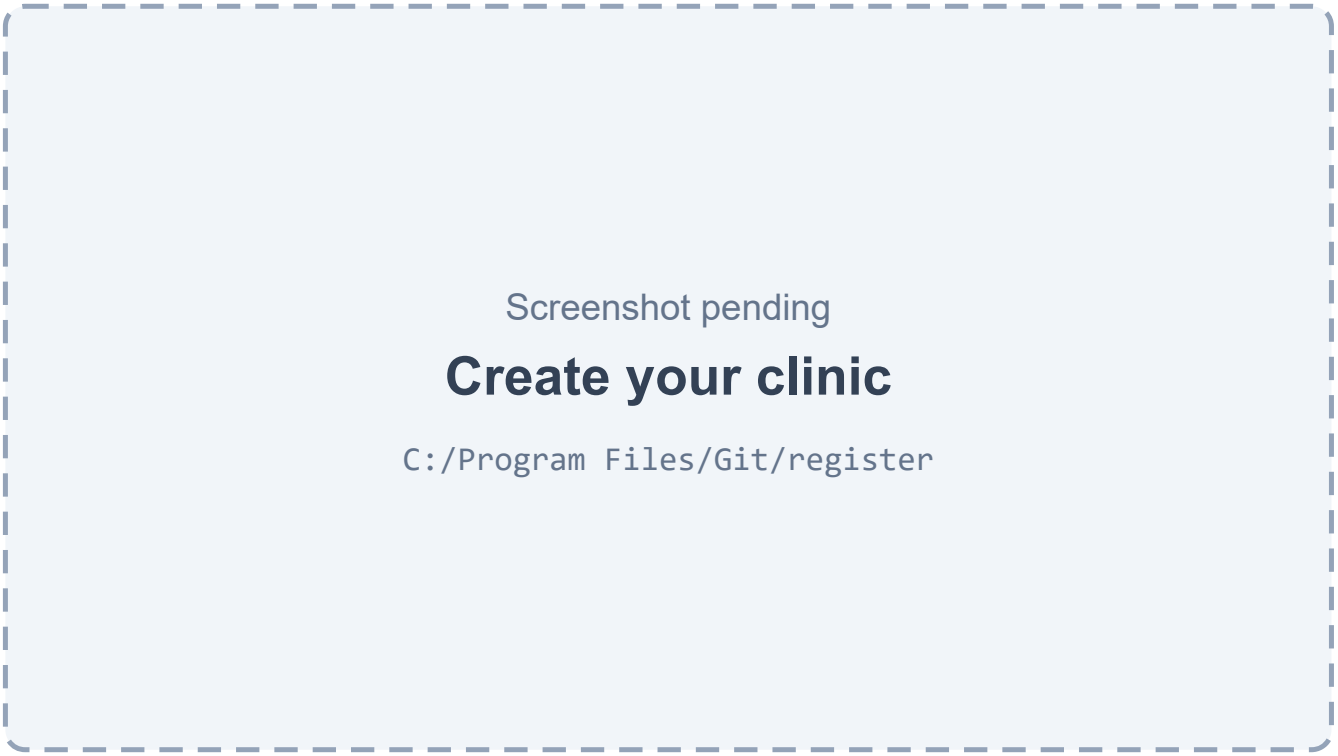

Figure 45.1 — Create your clinic (screenshot pending).

The registration page (`/register`) is headed "Create your clinic" with the line "Start your 14-day free trial. No card needed." Its fields are the clinic name (placeholder "City Care Clinic"), the owner's full name, an email address, a phone number and a password with a show or hide toggle. Messages say "Enter your clinic name", "Please enter a valid email", "Password must be at least 6 characters" and "At most 20 characters" for the phone. On success a toast says "Your clinic is ready" and "Your 14-day free trial has started.", and the app moves to the setup wizard. A failure shows "Registration failed. Please try again."

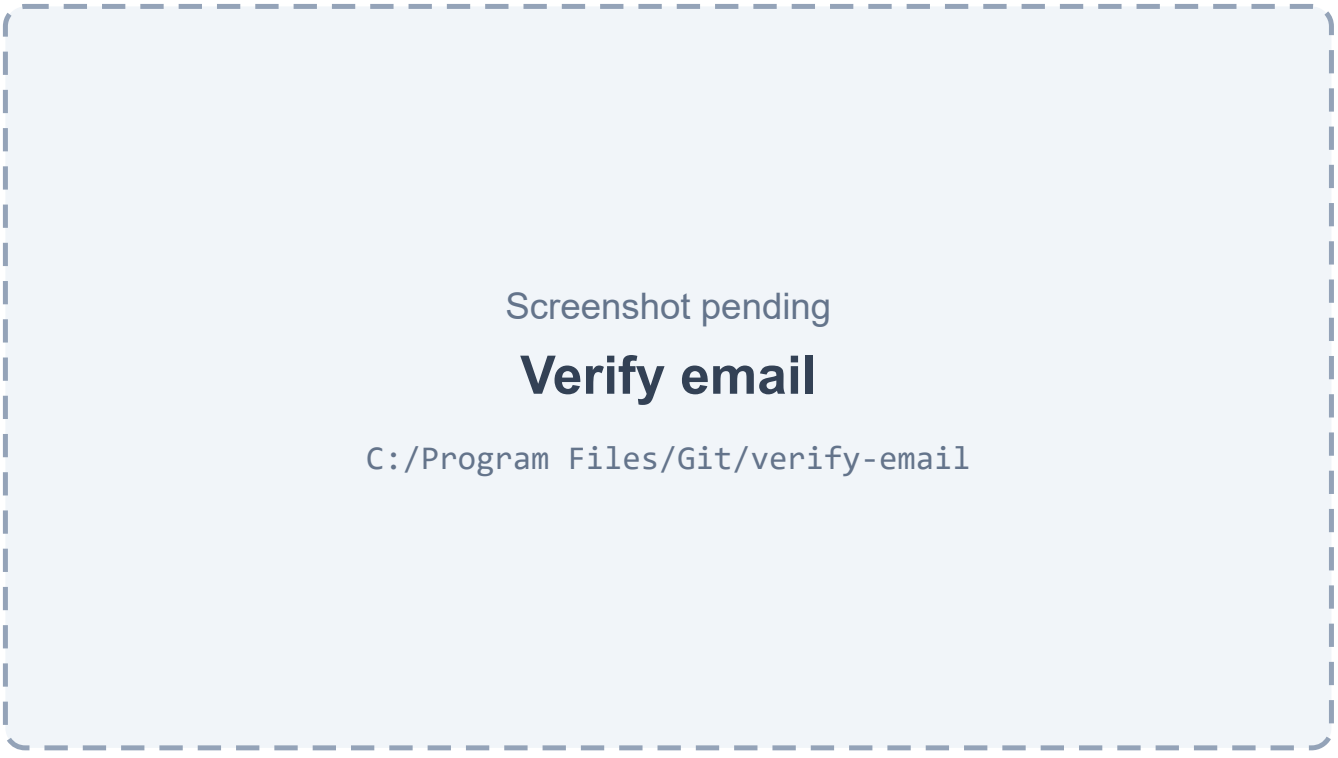

Figure 45.2 — Verify email (screenshot pending).

The page `/verify-email?token=…` is the landing page of the link in the verification email. It confirms the address and offers "Continue to the app" or "Go to sign in". If the link has no token it says "This link is missing its token." If the token is wrong or expired it says "This verification link is invalid or has expired." and "Use "Resend link" in the app to get a new one."

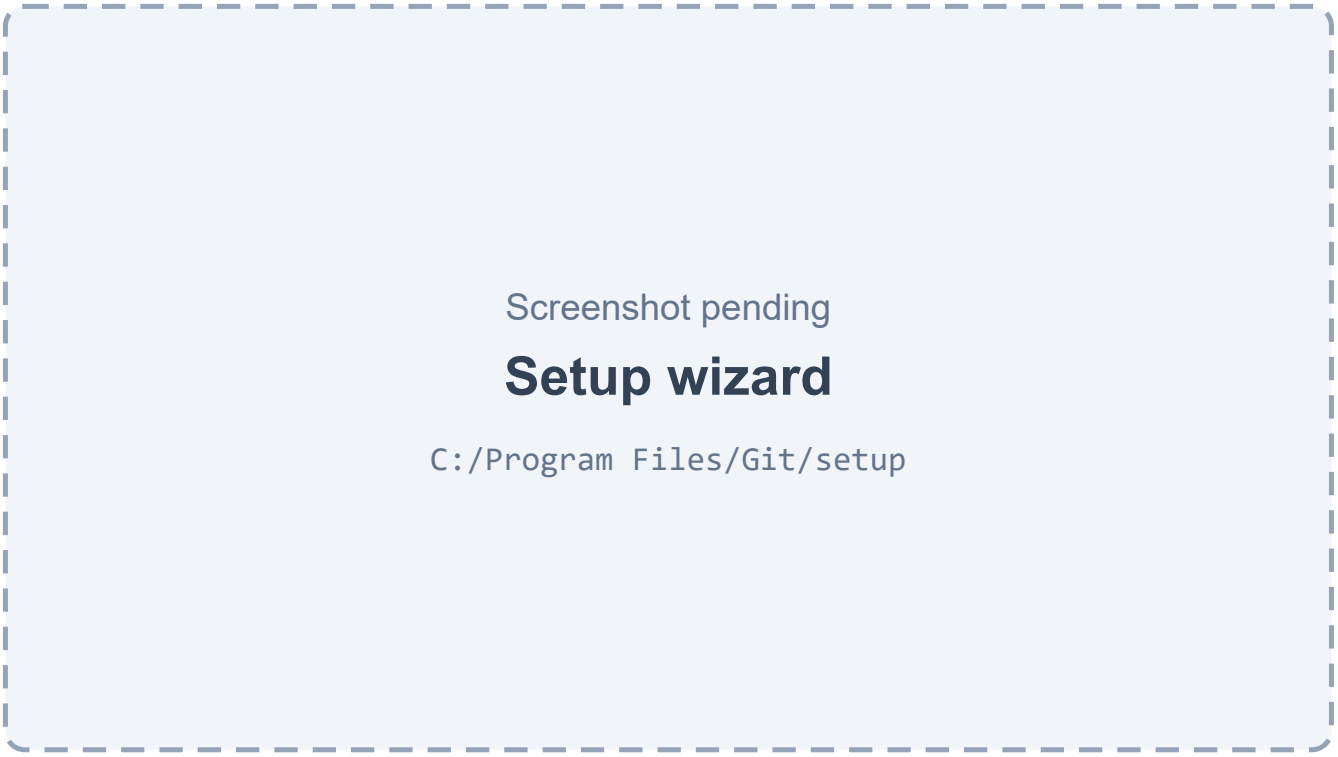

Figure 45.3 — Setup wizard (screenshot pending).

The wizard (`/setup`) shows three steps in a progress strip: **Clinic**, **Location & tax** and **Your team**. The Clinic step asks for the clinic name, "What kind of clinic is it?" (pick every type that applies, because it decides which tools the user sees), phone, clinic email, timezone (a list including Asia/Karachi, Asia/Dubai, Asia/Riyadh, Asia/Qatar, Asia/Kolkata, Asia/Dhaka, Europe/London and America/New_York) and currency. The Location & tax step asks for the location name ("Your first branch. You can add more under Locations later."), address, city, phone, email and "Sales tax on invoices (%)" with the hint "Added on top of each patient invoice. Leave 0 for no tax." The Your team step explains that doctors and front-desk staff can be added later, each with a role, and links to Employees, User Admin and Patients → Import. Its final button finishes setup, and a toast says "Your clinic is ready". Once setup is done, the page shows a **Back to the app** button.

Screenshot pending

Subscription and verification banners

`any dashboard page`

Figure 45.4 — Subscription and verification banners (screenshot pending).

A banner appears at the top of every dashboard page when something needs attention. For an unverified email: "Please verify your email — we sent a link to …" with a **Resend link** button. For a trial, owners see "Free trial — N days left." with a **See plans** link. For an active plan close to its end, owners see "Your plan renews in N days — send the payment to keep the clinic running without interruption." During the grace period everyone sees a warning that the clinic turns read-only in N days. After expiry: "Subscription expired. The clinic is read-only — renew your plan to make changes again." (owners) or "The clinic is read-only until the owner renews the plan." (others). An owner whose setup is pending is redirected to `/setup` from any other page.

Screenshot pending

Plan and billing

`C:/Program Files/Git/settings/subscription`

Figure 45.5 — Plan and billing (screenshot pending).

"Plan & billing" has the subtitle "Your subscription, usage and payments." It shows the current plan and status with "Trial ends", "Paid until" or "Ended on", usage bars for Locations and Logins against the limits, a "Plans" section with Free, Starter, Professional and Enterprise (price as "PKR n / month" or "Custom pricing", "Unlimited" limits), a "How to pay" box with the instructions text, and a "Payment history" table (Paid on, Plan, Period, Method, Amount) with the empty state "No payments yet". Non-owners see "Only the clinic owner can see this".

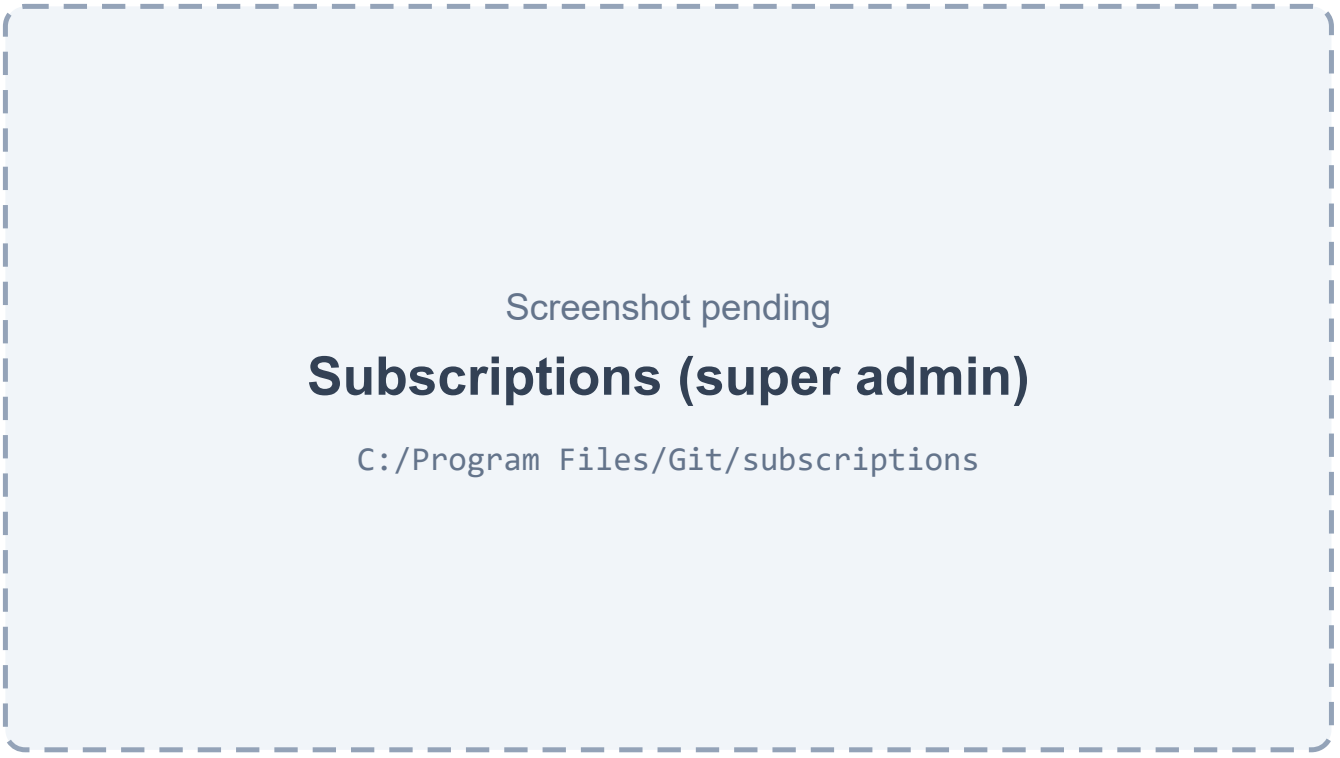

Figure 45.6 — Subscriptions (super admin) (screenshot pending).

The super admin page has the subtitle "Every clinic's plan and status. Record payments clinics send you." A "Payment instructions" card holds a text field (at most 2000 characters, shown to every owner) with "Unsaved changes" tracking. Below it, a table lists clinics with columns Clinic, Plan, Status, Period ends and Days left, a search box ("Search clinics…"), a status filter ("All statuses") and a row action with the tooltip "Manage subscription". The empty state is "No clinics yet".

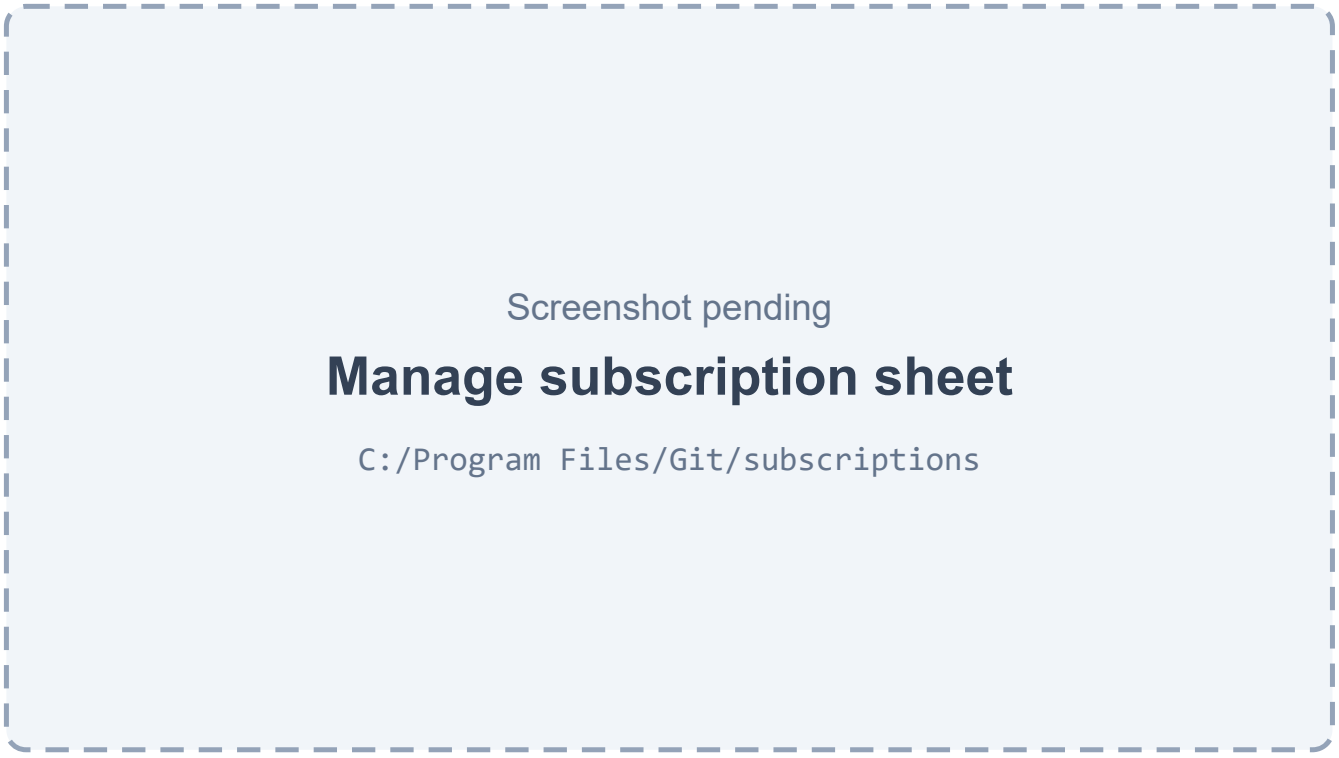

Figure 45.7 — Manage subscription sheet (screenshot pending).

The sheet has three tabs. **Record payment**: Plan, "Months paid", "Amount received (PKR)", Method, "Paid on", "Reference / transaction ID" and Notes. **Plan & limits**: Plan, "Trial ends" ("Extend a trial by moving this date."), "Paid until" ("Normally set by recording a payment."), "Location limit" and "Login limit". **History**: the clinic's payments, or "No payments recorded yet". Both forms warn about unsaved changes.

45.3 Features

45.3.1 Sign up and create a clinic

Status: [Implemented]

What it does. Creates the owner's login, the clinic (organization), its first location and the ownership link in one step, starts a 14-day trial and signs the owner in.

How the user does it.

1. Open `/register`.
2. Fill in clinic name, full name, email, phone (optional) and password.
3. Submit. The app signs the owner in and opens the setup wizard.

Behind the scenes — in simple terms. The clinic is ready to use at once. It starts on the Professional plan for 14 days, with PKR and the Asia/Karachi timezone as starting values, and is marked "onboarding pending" until the wizard is finished. If clinic creation fails, the new login is removed so no orphan account is left behind.

Behind the scenes — technically. `POST /auth/register` (`@Public`, rate limit 5 per hour) runs `AuthService.register`: `UserService.createLocalUser`, then `ClinicSetupService.createClinicForOwner` in one transaction. The transaction creates the organization (slug from the clinic name, with a random suffix on collision; plan `TRIAL_PLAN` = professional; `trial_ends_at` = now + `TRIAL_DAYS` (14); `onboarding_pending` = true), a location with code `MAIN` and `is_main_branch`, its settings row and the primary ownership row. After the commit,

`FeatureFlagsService.seedFlagsForOrg` and `AccessTemplatesService.seedForOrg` seed feature switches and the five system roles; a failure there is only logged. A verification email is then sent and the user is signed in through `login()` with a 30-day session.

Rules and validations. Email must be valid; name and clinic name are required (at most 255); password at least 6 characters; phone at most 20.

Who can do it. Anyone, no account needed (a public route, also listed in `PUBLIC_ROUTES`).

Data read and written. Writes `users`, `organizations`, `locations`, `location_settings`, `organization_ownership`, and, after commit, feature-flag and role tables.

Effects on other modules. The new clinic gets the default features and roles (Chapters 10 and 11).

Error cases. A duplicate email is rejected by user creation; more than 5 attempts per hour is rate limited; any clinic-creation error returns the error and deletes the new user.

Technical references: `backend/src/auth/auth.service.ts` · `AuthService.register` · `backend/src/subscriptions/clinic-setup.service.ts` · `ClinicSetupService.createClinicForOwner` · `web-app/src/routes/register.tsx`, `web-app/src/features/auth/components/RegisterForm.tsx`

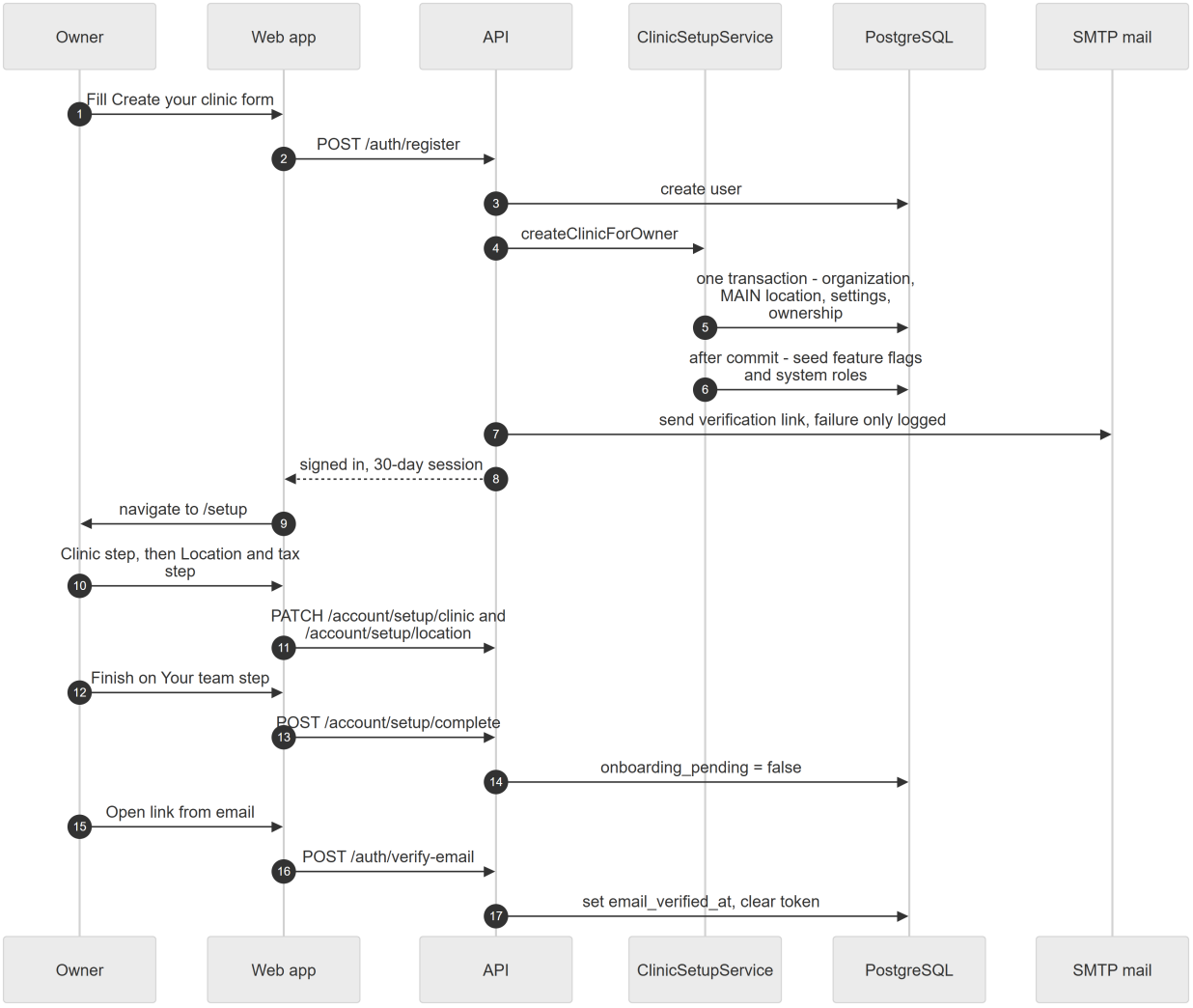

Figure 45.8 — From sign-up to a verified, set-up clinic.

The diagram has six participants: Owner, Web app, API, ClinicSetupService, PostgreSQL and SMTP mail. The Owner fills the "Create your clinic" form and the Web app posts to the API. The API creates the user, then asks ClinicSetupService to create the clinic, which writes the organization, the MAIN location, its settings and the ownership in one transaction and, after the commit, seeds feature flags and system roles. The API sends the

verification link by email (a failure is only logged) and returns a signed-in 30-day session. The Web app moves the Owner to `/setup`. The Owner completes the Clinic step and the Location and tax step, which the Web app saves with two PATCH calls, then finishes on the Your team step, which posts to `/account/setup/complete` and clears the pending flag. Separately, the Owner opens the link from the email; the Web app posts the token to the API, which stores the verification time and clears the token.

45.3.2 Verify the email address and resend the link

Status: [Partial]

What it does. Sends a link to the new owner's email and confirms the address when the link is opened. Verification is informational: nothing is blocked while the email is unverified.

How the user does it.

1. After signing up, read the banner "Please verify your email".
2. Open the link in the email, which lands on `/verify-email`.
3. If the link is lost or expired, click **Resend link** in the banner.

Behind the scenes — in simple terms. The email contains a one-time random token. The server stores only a fingerprint of it, valid for 48 hours. A new link replaces the old one. The banner shows while a token is still waiting.

Behind the scenes — technically. `sendVerificationEmail` creates 32 random bytes, stores the SHA-256 hash in `users.email_verification_token` with `email_verification_expires` = now + 48 h, and calls `MailService.sendEmailVerification`, which builds `<frontend URL>/verify-email?token=…` and sends through SMTP (environment variables `SMTP_USER`, `SMTP_PASS` and `FRONTEND_URL`). `POST /auth/verify-email` (`@Public`, 10 per 15 minutes) hashes the token, finds an unexpired match, sets `email_verified_at` and clears the token. `POST /auth/resend-verification` (signed in, `@ContextFree`, 3 per hour) sends a fresh link, or answers "Email already verified" when no token is pending. The user DTO exposes `emailVerificationPending` (token present), which drives the banner.

Rules and validations. Token valid for 48 hours; single use. Partial because no feature or screen is gated by `email_verified_at`, and a failed email send is only logged, so the user sees success.

Who can do it. Anyone with the link (verify); any signed-in user (resend).

Data read and written. `users` (`email_verification_token`, `email_verification_expires`, `email_verified_at`).

Effects on other modules. None; sign-in does not require verification.

Error cases. "This verification link is invalid or has expired" (validation error) when the token is unknown or expired; 429 when rate limited; toast "Could not send the email. Try again." when resend fails.

Technical references: `backend/src/auth/auth.service.ts` · `verifyEmail`, `resendVerification` · `backend/src/common/services/mail.service.ts` · `MailService` · `web-app/src/routes/verify-email.tsx`, `web-app/src/features/account/pages/VerifyEmailPage.tsx`, `components/AccountBanner.tsx`

45.3.3 Complete the setup wizard

Status: [Implemented]

What it does. Collects the clinic's real details in three steps and then clears the "onboarding pending" flag.

How the user does it.

1. After signing up, the app opens `/setup`. Until it is finished, an owner who opens any other page is sent back to `/setup`.
2. Step **Clinic**: name, clinic types, phone, email, timezone, currency. Save and continue.
3. Step **Location & tax**: location name, address, city, phone, email and the sales tax percentage. Save and continue.
4. Step **Your team**: read the hints, then finish. The app refreshes the session and opens the home page.

Behind the scenes — in simple terms. The first two steps save immediately. The finish button only flips the flag; it does not check the earlier steps were saved. Saving the clinic step copies the timezone and currency to every location of the clinic, because a new clinic's branches follow the clinic.

Behind the scenes — technically. `GET /account/setup` returns the main location (first active by `is_main_branch DESC, created_at ASC`). `PATCH /account/setup/clinic` (`SetupClinicDto`: name, phone, email, website, `specialtyTags` from `CLINIC_SPECIALTIES`, timezone, currency upper-cased) runs `saveClinic` in a transaction, updating the organization and every location. `PATCH /account/setup/location` runs `saveLocation` in a transaction, updating the main location and its settings, with `taxRate` 0–100. `POST /account/setup/complete` sets `onboarding_pending` to false. All four are `@AnyMember()` and call `assertOwner`. Setup writes are on the always-writable path list, so an expired clinic can still finish setup.

Rules and validations.

- Owner or super admin only.
- Email must be valid; tax 0–100; clinic types must come from the fixed list; timezone is free text and is not checked against the IANA list.
- Currency is upper-cased.

Who can do it. The owner or a super admin; no permission key, no feature key.

Data read and written. Reads and writes `organizations`, `locations`, `location_settings`; reads `organization_ownership`.

Effects on other modules. Timezone and currency drive clinic times and money display everywhere (Chapter 62). Tax rate is added to patient invoices (Chapter 34). Clinic types decide which tools show.

Error cases. 403 "Only the clinic owner can do this"; validation errors per field; toast "Could not save. Try again."

Technical references: `backend/src/subscriptions/account.controller.ts` · `AccountController` · `clinic-setup.service.ts` · `ClinicSetupService.saveClinic`, `saveLocation`, `complete` · `web-app/src/routes/setup.tsx`, `web-app/src/features/account/pages/SetupWizardPage.tsx`, `hooks/useSetupForms.ts`

45.3.4 Trial, plans and plan limits

Status: [Implemented]

What it does. Gives each managed clinic a plan with limits on locations and logins, and refuses to exceed them.

How the user does it. The owner opens **Settings** → **Plan & billing** to see the plan, the usage bars and the plan catalog. Reaching a limit shows the error when adding a location or a login.

Behind the scenes — in simple terms. Four plans exist. A clinic with no trial date and no paid date is called unmanaged: it has no limits and never expires (older clinics and clinics created by a super admin). A limit set on the clinic itself wins over the plan's limit.

Plan	Price per month (PKR)	Locations	Logins
Free	0	1	2

Plan	Price per month (PKR)	Locations	Logins
Starter	4 999	1	5
Professional	9 999	3	15
Enterprise	custom	unlimited	unlimited

The table lists the plan catalog from code. Prices are only what the clinic sees; nothing bills them. A "login" is an owner plus any employee with portal access.

Behind the scenes — technically. `PLANS` in `plans.ts`. `effectiveLimits(org)` returns `org.maxLocations ?? plan.maxLocations` and likewise for users, and none for unmanaged clinics. `assertCanAddLocation` and `assertCanAddUser` compare against `usage()` (active locations; owners plus active staff with a `user_id`) and throw `PlanLimitException` (code `PLAN_LIMIT`). `GET /account` returns the summary, usage and, for owners, the plans, payment instructions and payments. Location creation calls the check for non-super-admins; login creation calls it from the user service.

Rules and validations. Limits are skipped for super admins and unmanaged clinics.

Who can do it. Any member reads the summary; owners also get plans and payments. Writes happen when locations or logins are added.

Data read and written. Reads `organizations`, `locations`, `employee_org_location_assignments`, `organization_ownership`, `subscription_payments`, `config_values`.

Effects on other modules. Locations (Chapter 9) and Employees (Chapter 37) are blocked at the limit.

Error cases. `PLAN_LIMIT` (HTTP 403): "Your Professional plan allows 3 locations. Upgrade your plan to add more." or "... allows 15 logins. ...".

Technical references: `backend/src/subscriptions/plans.ts` · `PLANS`, `TRIAL_PLAN`, `TRIAL_DAYS` · `subscription-state.ts` · `effectiveLimits` · `subscriptions.service.ts` · `SubscriptionsService.usage`, `assertCanAddLocation`, `assertCanAddUser` · `web-app/src/routes/settings_.subscription.tsx`, `web-app/src/features/account/pages/SubscriptionPage.tsx`

45.3.5 Read-only mode after expiry

Status: [Implemented]

What it does. When a clinic's period and its grace days have passed, every write is refused until a payment is recorded. Reading still works, so the clinic keeps its data.

How the user does it. Staff see the red banner and a rejected save. The owner pays and the super admin records it (45.3.7).

Behind the scenes — in simple terms. The period end is the later of the trial end and the paid-until date. After that there are 3 grace days with a warning, then the clinic becomes read-only. The decision is cached for 30 seconds per server process, so a renewal can take up to 30 seconds to be seen.

Behind the scenes — technically. `SubscriptionGuard` is the last global guard. It lets GET, HEAD and OPTIONS pass; skips `@Public` routes and super admins; lets paths starting with `/auth/`, `/account`, `/me/` and `/admin/subscriptions` through; resolves the organization from the access context or the `x-organization-id` header; and calls `SubscriptionsService.isReadOnly`, a 30 s cache over `subscriptionState()`. When read-only it throws HTTP 402 with code `SUBSCRIPTION_EXPIRED`. Requests with no organization pass.

Rules and validations. `GRACE_DAYS` = 3. Unmanaged clinics are never read-only.

Who can do it. Applies to every user except super admins.

Data read and written. Reads `organizations` (`trial_ends_at` , `paid_until`).

Effects on other modules. Every module's write endpoints are blocked, including the queue and billing. Realtime reads are unaffected.

Error cases. HTTP 402 `SUBSCRIPTION_EXPIRED` : "Your subscription has expired — the clinic is read-only until it is renewed."

Technical references: `backend/src/subscriptions/subscription.guard.ts` · `SubscriptionGuard` · `subscriptions.service.ts` · `isReadOnly` · `web-app/src/features/account/components/AccountBanner.tsx`

45.3.6 See subscription status and warnings

Status: [Implemented]

What it does. Shows owners and staff where the subscription stands and warns before it runs out.

How the user does it. The banner appears automatically. Owners open Plan & billing for detail.

Behind the scenes — in simple terms. The status is never stored. It is worked out every time from the two dates and the current time.

Behind the scenes — technically. `subscriptionState(org, now)` returns `status` , `periodEndsAt` , `readOnlyFrom` , `daysLeft` and `readOnly` . Rules: no dates means `unmanaged` ; `paid_until` in the future means `active` ; else `trial_ends_at` in the future means `trialing` ; else before period end plus 3 days means `grace` ; else `expired` . `AccountBanner` reads `GET /account` and shows the matching message; owners see trial and renewal messages, everyone sees grace and expiry.

Rules and validations. The status rules are the ones in the technical paragraph.

Who can do it. Any signed-in member sees the banner for their clinic.

Data read and written. Reads `organizations` .

Effects on other modules. None.

Error cases. The banner is simply absent if the account request fails.

Technical references: `backend/src/subscriptions/subscription-state.ts` · `subscriptionState` · `web-app/src/features/account/components/AccountBanner.tsx` , `hooks/useAccount.ts`

45.3.7 Record a manual payment (super admin)

Status: [Implemented]

What it does. Records a payment the clinic made outside the Platform and extends the paid period.

How the user does it.

1. As a super admin, open **Subscriptions** in the sidebar.
2. Click **Manage subscription** on the clinic's row.
3. On the **Record payment** tab, pick the plan, months paid, amount, method, date, reference and notes.
4. Save. A toast says "Payment recorded — the plan is extended".

Behind the scenes — in simple terms. The new period starts where the current trial or paid period ends, so paying early loses no days. Recording a payment also sets the clinic's plan to the plan paid for, so paying for Starter during a Professional trial lowers the limits at once.

Behind the scenes — technically. `POST /admin/subscriptions/:orgId/payments` (`RecordSubscriptionPaymentDto`) runs `SubscriptionsService.recordPayment` in a transaction: locks the organization row, computes `nextPaidPeriod(org, months, now)`, inserts a `subscription_payments` row with `period_start` and `period_end`, sets `paid_until` to the period end and `subscription_plan` to the payment's plan, writes audit entries and clears the read-only cache.

Rules and validations. Plan in the enum; months 1–36; amount at least 0 with 2 decimals; method one of bank_transfer, jazzcash, easypaisa, cash, cheque, other; reference at most 120; notes at most 1 000; date optional.

Who can do it. Super admin only (`@RequireSuperAdmin()` on the controller). No feature key.

Data read and written. Writes `subscription_payments`, `organizations`, `audit_logs`.

Effects on other modules. Lifts read-only mode; changes the plan limits.

Error cases. 404 "Organization not found"; 400 validation errors; 403 for non-super-admins.

Technical references: `backend/src/subscriptions/admin-subscriptions.controller.ts` · `AdminSubscriptionsController.recordPayment` · `subscriptions.service.ts` · `recordPayment` · `entities/subscription-payment.entity.ts` · `web-app/src/routes/subscriptions.tsx`, `web-app/src/features/account/components/ManageSubscriptionSheet.tsx`

45.3.8 Change a clinic's plan, dates or limits (super admin)

Status: [Implemented]

What it does. Lets a super admin set the plan, trial end, paid-until date and per-clinic location and login limits directly.

How the user does it.

1. Open **Subscriptions**, click **Manage subscription** on a row, choose the **Plan & limits** tab.
2. Change fields and save. A toast says "Subscription updated".

Behind the scenes — in simple terms. Moving the trial date extends a trial. Clearing both dates makes the clinic unmanaged (no limits, no expiry).

Behind the scenes — technically. `PATCH /admin/subscriptions/:orgId` (`UpdateSubscriptionDto`) runs `update` under a `pessimistic_write` lock, records an audit entry with before and after values (`organization_subscription`) and clears the cache. `GET /admin/subscriptions` lists clinics (including inactive ones) with derived state, and `GET /admin/subscriptions/:orgId` returns one clinic with its payments.

Rules and validations. Dates are ISO strings or null; limits are integers of at least 1 or null.

Who can do it. Super admin only.

Data read and written. `organizations`, `audit_logs`; reads `subscription_payments`.

Effects on other modules. Same as 45.3.4 and 45.3.5.

Error cases. 404 for an unknown clinic; 400 on invalid values.

Technical references: `AdminSubscriptionsController.list`, `get`, `update` · `SubscriptionsService.update` · `web-app/src/features/account/pages/AdminSubscriptionsPage.tsx`, `hooks/useAdminSubscriptionForms.ts`

45.3.9 Edit the payment instructions (super admin)

Status: [Implemented]

What it does. Sets the text every clinic owner sees under "How to pay".

How the user does it. On the Subscriptions page, edit the "Payment instructions" card and save ("Payment instructions saved").

Behind the scenes — in simple terms. One text for the whole Platform, with a built-in default asking the clinic to pay by bank transfer or JazzCash and share the receipt with support.

Behind the scenes — technically. `GET` and `PUT /admin/subscriptions/payment-instructions` (`PaymentInstructionsDto` , at most 2 000 characters) through the settings registry key `subscription.payment_instructions` at platform scope. The route is declared before `:orgId` .

Rules and validations. At most 2 000 characters; platform scope only; not lockable.

Who can do it. Super admin only.

Data read and written. `config_values` .

Effects on other modules. None.

Error cases. Toast "Could not save. Try again."

Technical references: `backend/src/subscriptions/subscription.settings.ts` · `SUBSCRIPTION_SETTINGS` · `SubscriptionsService.setPaymentInstructions`

45.3.10 Create a clinic from the Organizations screen (older path)

Status: [Partial]

What it does. A super admin can create or edit an organization on the Organizations screen. The result is an unmanaged clinic with no owner, no location and no trial. The plan selector on that screen is broken: it sends uppercase values the API rejects.

How the user does it. Super admin, Organizations screen, create or edit dialog.

Behind the scenes — technically. The form offers FREE, BASIC, PREMIUM and ENTERPRISE and sends them as the `subscriptionPlan` . The DTO validates against the `SubscriptionPlan` enum, whose values are lowercase `free` , `starter` , `professional` and `enterprise` . So every uppercase value fails validation, and BASIC and PREMIUM are not plans at all. This was verified by reading both files; the app was not run. The plan is optional in the DTO, so a request that omits it would pass. Plan changes made here are not audited, unlike changes through the Subscriptions screen.

Rules and validations. `@IsEnum(SubscriptionPlan)` on create and update.

Who can do it. Super admin.

Data read and written. `organizations` .

Effects on other modules. None.

Error cases. HTTP 400 with a message that `subscriptionPlan` must be one of the enum values.

Technical references: `backend/src/organizations/dto/organization.dto.ts` · `subscriptionPlan` · `web-app/src/features/organizations/OrganizationManagementView.tsx` · `backend/docs/modules/01-organizations-locations.md`

45.4 Entities and data model

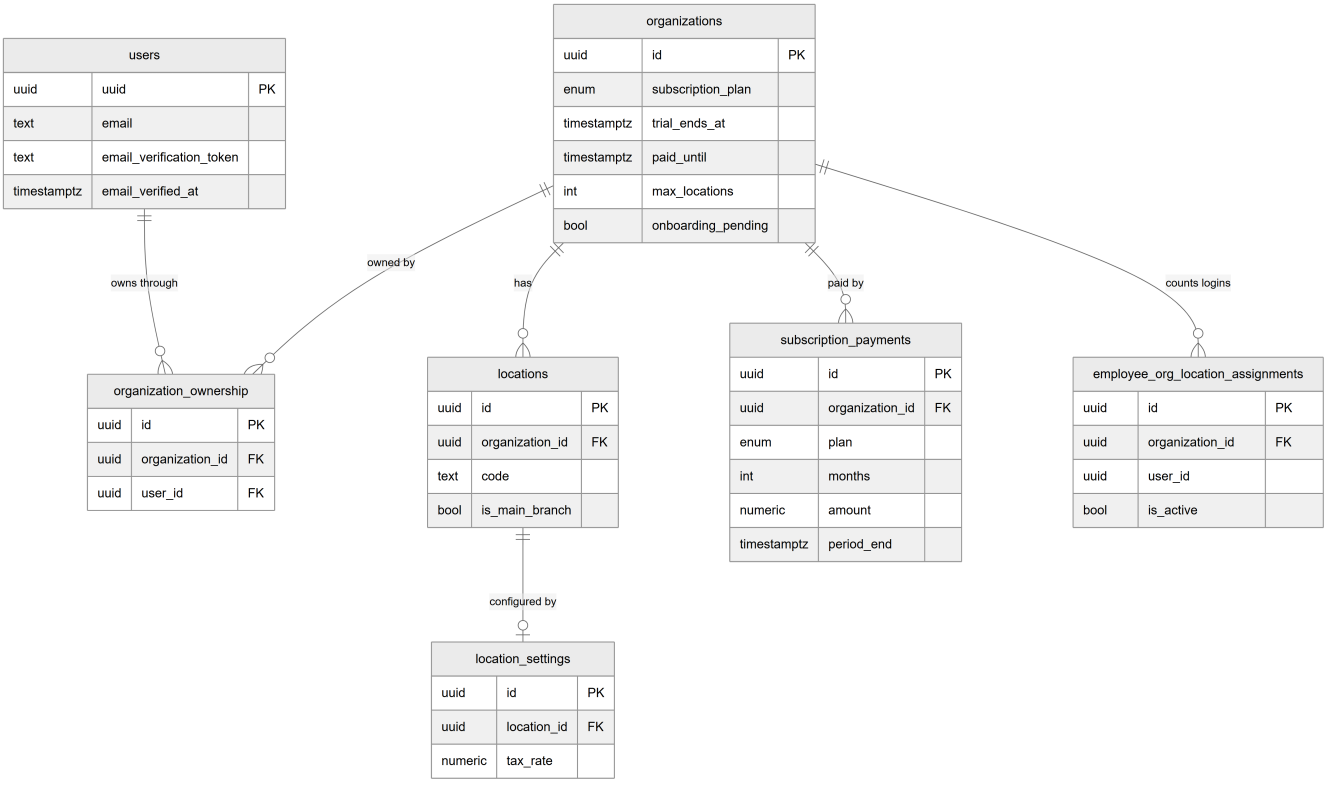

Figure 45.9 — The tables behind subscriptions and sign-up.

The diagram shows seven tables. `users` carries the email verification columns (`email_verification_token` , `email_verified_at`). `organizations` carries the subscription columns: `subscription_plan` , `trial_ends_at` , `paid_until` , `max_locations` (and a login limit) and `onboarding_pending` . `organization_ownership` links a user to an organization. `locations` has a code and an `is_main_branch` flag. `location_settings` holds the tax rate. `subscription_payments` records each manual payment. `employee_org_location_assignments` is read to count logins.

The relationships in words:

- Each ownership row links one user to one organization; deleting either deletes the row (ON DELETE CASCADE). A clinic created through signup has one primary owner.
- Each location belongs to one organization (CASCADE). Signup creates the first one with code `MAIN` .
- Each location has at most one settings row.
- Each payment belongs to one organization; deleting the organization deletes its payments (CASCADE). The organization has many payments over time.
- Login usage is counted from owners plus active assignments that have a user; this is a read, not a link.

Key `subscription_payments` columns: `plan` , `months` , `amount` (numeric, 2 decimals), `method` , `reference` , `paid_at` , `period_start` , `period_end` , `notes` and `recorded_by` . Plan definitions are not stored; they live in `plans.ts` . The subscription status is not stored either.

45.5 Status lifecycles

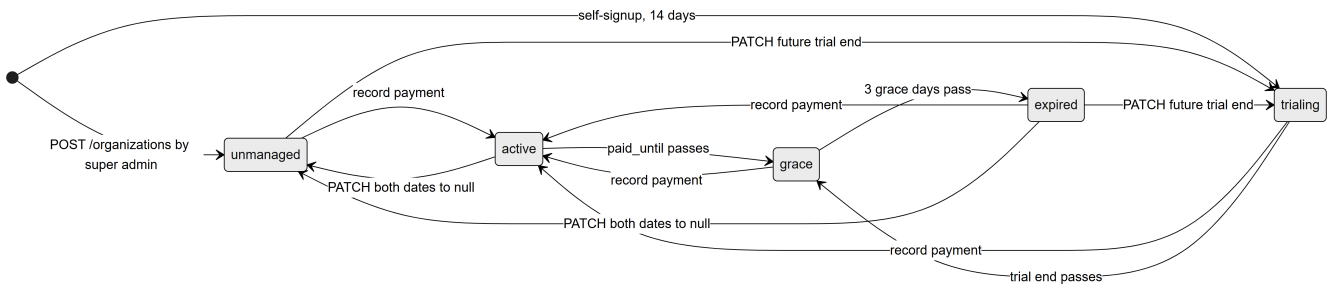

Figure 45.10 — Derived subscription status.

A clinic is **unmanaged** when created by a super admin (no dates), and **trialing** when created by signup. **Trialing** becomes **active** when a payment is recorded, or **grace** when the trial end passes. **Active** becomes **grace** when `paid_until` passes. **Grace** lasts 3 days and then becomes **expired**; a recorded payment returns it to active. **Expired** is read-only, and a recorded payment returns it to active. A super admin can move an unmanaged clinic to active (payment) or trialing (set a future trial end), and can send an active or expired clinic back to unmanaged by clearing both dates. A future trial end also moves an expired clinic back to trialing. Status is computed on read, so these changes happen as time passes, not through a job.

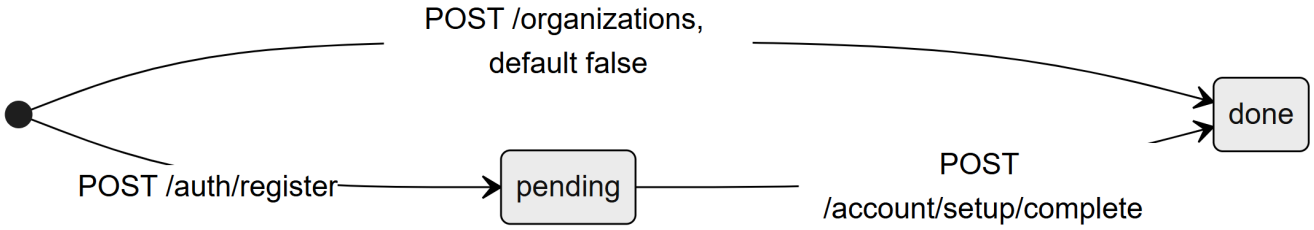

Figure 45.11 — The `onboarding_pending` flag.

A signup creates the organization as pending. `POST /account/setup/complete` sets it to done. An organization created by a super admin starts as done. There is no way back to pending.

45.6 Connections to other modules

- **Authentication (Chapter 56)** owns register, verify and resend, and the 30-day session after signup.
- **Organizations and locations (Chapter 9)** and **Features (Chapter 10)**: signup seeds feature switches and the five system roles (Chapter 11). Plan limits block adding locations and logins.
- **Employees (Chapter 37)**: adding a login counts against the login limit.
- **Billing (Chapter 34)**: the tax rate saved in the wizard applies to patient invoices.
- **Request lifecycle (Chapter 5)** lists the five global guards; the subscription guard is the last.
- **Audit logging (Chapter 60)**: plan changes and payments are audited.
- **Time, money and numbering (Chapter 62)**: timezone and currency from the wizard.

45.7 Limitations and known gaps

- Nothing charges clinics. Payments are recorded by hand, and there is no reminder email for renewals.
- Email verification is not required for anything; a failed send is only logged.
- A free-text timezone is accepted in the wizard without checking it against the IANA list.
- The wizard's finish button does not check the earlier steps were saved.

- Saving the clinic step overwrites timezone and currency on every location.
- Recording a payment overwrites the plan, which can lower the limits.
- The location limit is skipped when a super admin acts; unmanaged clinics have no limits.
- Read-only state is cached for 30 seconds per server process.
- The Organizations screen sends plan values the API rejects (45.3.10), and changes through it are not audited.

45.8 Owner's FAQ

What does a new clinic get? A 14-day trial on Professional (3 locations, 15 logins), one location called `MAIN`, default features and the five system roles.

What happens when the trial ends? The clinic has 3 grace days with a warning banner, then becomes read-only. Reads work, all saves are refused with `SUBSCRIPTION_EXPIRED`.

How does a clinic pay? Outside the Platform, by the method in the instructions text. A super admin then records the payment, which extends the period and lifts read-only mode.

Can an expired clinic still finish setup? Yes. Sign-in, `/account` (including setup), `/me/` and `/admin/subscriptions` paths stay writable.

Can a clinic add a fourth branch on Professional? No. It gets `PLAN_LIMIT` unless a super admin raises its location limit.

Do users have to verify their email? No. The banner nags, but nothing is blocked.

Where is the status stored? Nowhere. It is derived from `trial_ends_at` and `paid_until`.

Can a super admin give a clinic unlimited access? Yes, by clearing both dates (unmanaged) or by setting Enterprise limits.

What if the verification email never arrives? The owner clicks Resend link (3 per hour). If SMTP is not configured, sending fails silently on the server and only a log line records it.

Part 6 — End-to-end workflows

Chapter 46 — Workflow: Onboarding a New Clinic

This chapter follows one new clinic from nothing to a clinic that is ready to receive patients. There are two ways in: the owner signs up alone, or a super admin creates the organization by hand. It is written for the product owner, who needs to know which steps are automatic, which need a super admin, and where the two paths differ. Each step names the module, its status and the chapter that explains it in full.

46.1 Who is involved

- **Owner**: the person who runs the clinic business. On the self-signup path this person creates the clinic. On the other path an owner login is created for them.
- **Super admin**: the Platform operator. Creates organizations by hand, entitles features, records payments and can act inside any clinic.
- **Example Clinic**: the fictional organization used in this chapter, with a Main location.
- **Sample staff**: employees added at the end (Chapter 47 covers staff onboarding).

46.2 The steps

Figure 46.1 shows both paths and the shared steps that follow them.

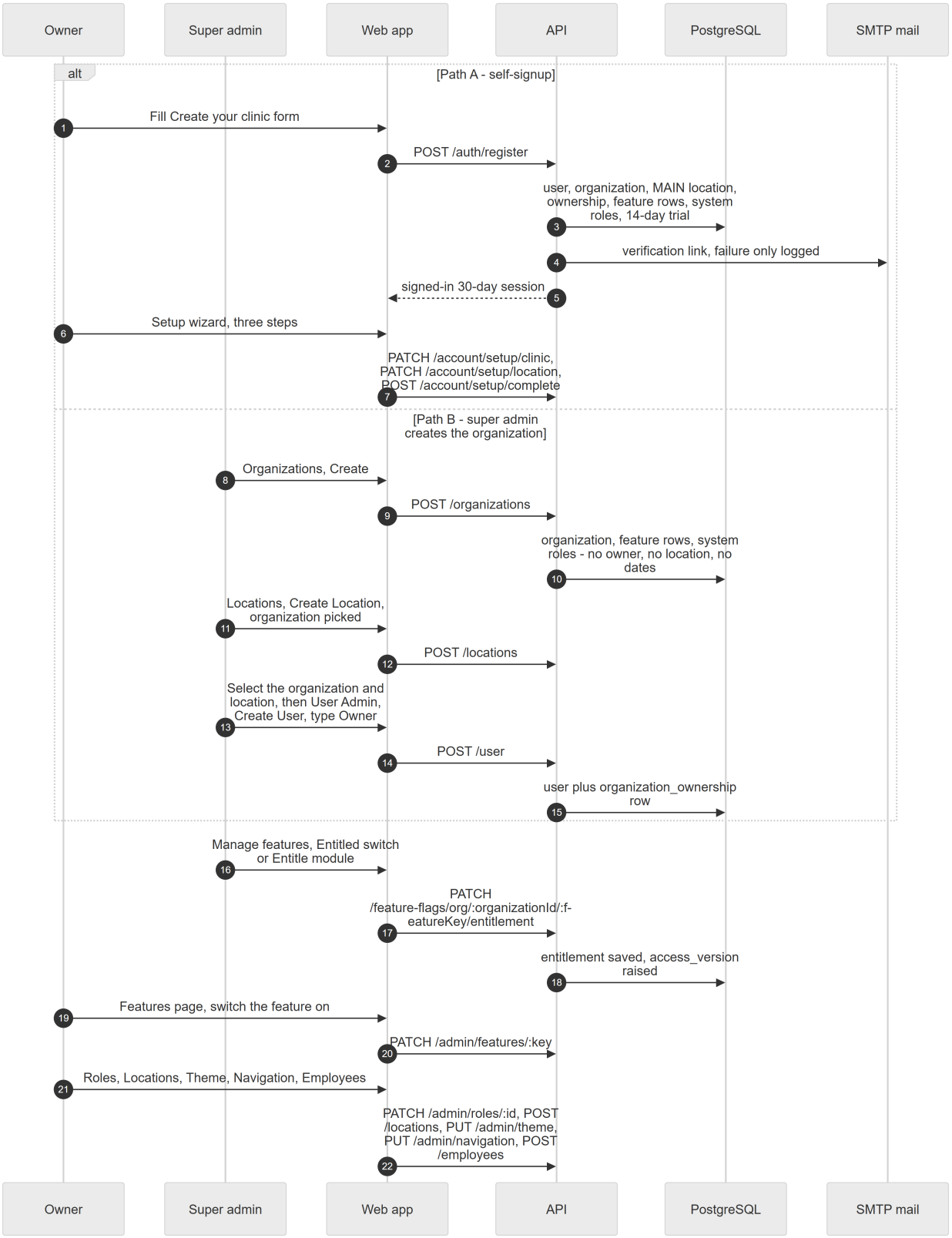

Figure 46.1 — Two ways to create a clinic, then the shared set-up steps.

The diagram has six participants from left to right: Owner, Super admin, Web app, API, PostgreSQL and SMTP mail. It has 22 numbered messages. A box labelled "alt" holds the two paths. In Path A (messages 1 to 7) the Owner fills the "Create your clinic" form, the Web app posts to `POST /auth/register`, and the API writes the user, the organization, the MAIN location, the ownership row, the feature rows, the system roles and a 14-day trial to PostgreSQL. The API sends a verification link through SMTP mail (a failure is only logged) and returns a signed-in 30-day session. The Owner then goes through the three-step setup wizard, and the Web app saves it with two PATCH calls and one POST that completes setup. In Path B (messages 8 to 15) the Super admin creates the

organization from the Organizations screen; the API stores the organization, its feature rows and system roles but no owner, no location and no dates. The Super admin then creates a location, selects the organization and location, and creates a user of type Owner in User Admin, which writes the user and an ownership row. After the box, both paths continue: the Super admin entitles features (messages 16 to 18), the Owner switches features on (19 and 20), and the Owner sets up roles, locations, theme, navigation and employees (21 and 22).

Step 1a — Self-signup (Path A)

Module: Subscriptions and Onboarding. **Status:** [Implemented]. The visitor opens `/register`, enters clinic name, full name, email, an optional phone and a password. The Platform creates the owner login, the organization, one location with code `MAIN`, the ownership link, a 14-day trial on the Professional plan, the feature rows and the five system roles, then signs the owner in. Chapter 45 describes this in full: a visitor registers, and a trial starts at once.

Step 1b — Verify the email address

Module: Subscriptions and Onboarding. **Status:** [Partial]. The owner receives a link that is valid for 48 hours and opens `/verify-email`. Verification is informational: nothing is blocked while the address is unverified, and a failed email send is only logged. Chapter 45 and the Authentication chapter (Chapter 56) cover it.

Step 1c — Setup wizard

Module: Subscriptions and Onboarding. **Status:** [Implemented]. An owner whose setup is pending is sent to `/setup` from any other page. Step "Clinic" collects name, clinic types, phone, email, timezone and currency. Step "Location & tax" collects the main location's details and the sales tax percentage. Step "Your team" only explains that staff are added later; its finish button clears the pending flag without checking that the first two steps were saved. Saving the clinic step copies timezone and currency to every location of the clinic (Chapter 45).

Step 2 — Super admin creates the organization (Path B)

Module: Organizations and locations. **Status:** [Partial]. `POST /organizations` works, and it seeds the feature rows and the five system roles. The Organizations screen's form offers plan, clinic type, timezone and currency values that the API rejects, so a form filled with those fields fails with HTTP 400 (found by reading both files, not by running the app). The result is an unmanaged clinic: no owner, no location, no trial and no plan limits (Chapter 9 and Chapter 45).

Step 3 — Give the clinic a location and an owner (Path B)

Module: Organizations and locations, and User Admin. **Status:** [Partial]. A super admin adds a location from the Locations screen by choosing the organization. There is no field on the organization screen to name an owner. The code creates an owner login through User Admin with the account type "Owner", which uses the active organization from the request header. A super admin therefore has to select that organization and a location before creating the owner. This sequence is deduced from the code and was not run. Chapter 8 explains ownership; Chapter 56 explains User Admin.

Step 4 — Entitle features

Module: Features. **Status:** [Implemented]. A super admin opens "Manage features" for the clinic. Every feature has an **Entitled** switch, an optional trial end date and an **Enabled** switch; "Entitle module" acts on a whole module. The seeding at clinic creation sets entitled and enabled equal to each feature's default. Several features that clinics rely on are off by default and need this step: among them `billing.invoices`, `billing.commissions`, `inventory.core`, `dental.lab_orders`, `reports.financial` and `reports.clinical`. Without `billing.invoices`, completing a consultation creates no charges (Chapter 31). Chapter 10 covers the three switches.

Step 5 — Owner enables features

Module: Features. **Status:** [Implemented]. The owner opens **Features** (`/admin/features`) and switches on the features in the plan. A locked feature, which is not entitled or whose trial ended, cannot be switched on. Turning a feature off shows an impact list of the permissions and roles affected, and a location can switch a feature off for itself only (Chapter 10).

Step 6 — Roles

Module: Roles and permissions. **Status:** [Implemented]. The five system roles (Admin, Doctor, Receptionist, Assistant, Accountant) already exist. The owner may clone one or build a custom role at **Roles & permissions** (`/admin/roles`) with None, Basic or Full per module. System roles are read-only (Chapter 11).

Step 7 — Locations

Module: Organizations and locations. **Status:** [Implemented]. The owner adds further branches from Locations. The plan's location limit applies (Professional allows 3) and an expired subscription refuses the write. The edit screen for locations sends fields the API refuses, so changing a location afterwards may fail [Partial] (Chapter 9).

Step 8 — Theme and navigation

Module: Customization. **Status:** [Implemented]. At `/admin/theme` the owner sets the display name (default "Clinic Portal"), logos, favicon, brand colour (default a blue), sidebar style and default light, dark or system mode. A location may override the name and logos. At `/admin/navigation` the owner arranges the sidebar for the whole clinic. Both need their own permission (`admin.theme.manage` , `admin.nav.manage`). Changing the sidebar layout raises the access version; the theme save does not appear to (Chapter 58 covers customization; Chapter 54 describes how access changes spread).

Step 9 — Staff

Module: Employees. **Status:** [Implemented]. The owner or an Admin adds employees with roles and logins. Chapter 47 follows that workflow.

46.3 What can go wrong

- **Duplicate email at sign-up.** User creation rejects it. If clinic creation fails after the user is created, the new user is deleted so no orphan account remains.
- **Rate limit.** Registration is limited to 5 per hour, verification to 10 per 15 minutes and resend to 3 per hour.
- **Feature set after signup is smaller than expected.** Features that are off by default must be entitled by a super admin. Clinics that have not been entitled for billing see no automatic charges.
- **Seeding failure.** After the clinic is committed, seeding the feature rows and roles can fail; the failure is only logged, so a clinic can exist with no roles (Chapter 45).
- **Plan limit.** Adding a fourth location on Professional gives `PLAN_LIMIT` .
- **Super admin path cannot be finished in one screen.** Owner creation needs an organization context, which needs a location first.
- **Expired trial.** After the trial and 3 grace days the clinic is read-only; setup and account paths stay writable (Chapter 53).

46.4 Data left behind

- `users` : the owner login, with the email verification columns.
- `organizations` : plan, `trial_ends_at` , `onboarding_pending` , `access_version` .
- `organization_ownership` : one primary owner on Path A; one non-primary owner on Path B.
- `locations` and `location_settings` : at least one location with the tax rate.
- `org_feature_flags` (entitled and enabled per feature) and `employment_roles` (the five system roles).
- `config_values` : theme settings. `nav_layouts` : the sidebar layout, if the owner saved one.
- `audit_logs` : feature entitlement changes and nav layout changes.
- Employees and assignments after Step 9.

Chapter 47 — Workflow: Onboarding a Staff Member

This chapter follows one new staff member, Dr. Example One, from the first employee record to the first sign-in. It is written for the product owner, who needs to know how a person, a role, a login, working hours and pay terms fit together, and what the Platform cannot do. The workflow touches Employees, Roles, User Admin, Authentication and Commissions.

47.1 Who is involved

- **Admin** (or the owner): holds `employees.manage`, `admin.users.manage` and `compensation.manage`. Only the Admin system role holds them by default.
- **Dr. Example One**: the new employee. Becomes a **staff login** once a user account is linked.
- **Example Clinic**, Main location.

47.2 The steps

Figure 47.1 shows the whole flow.

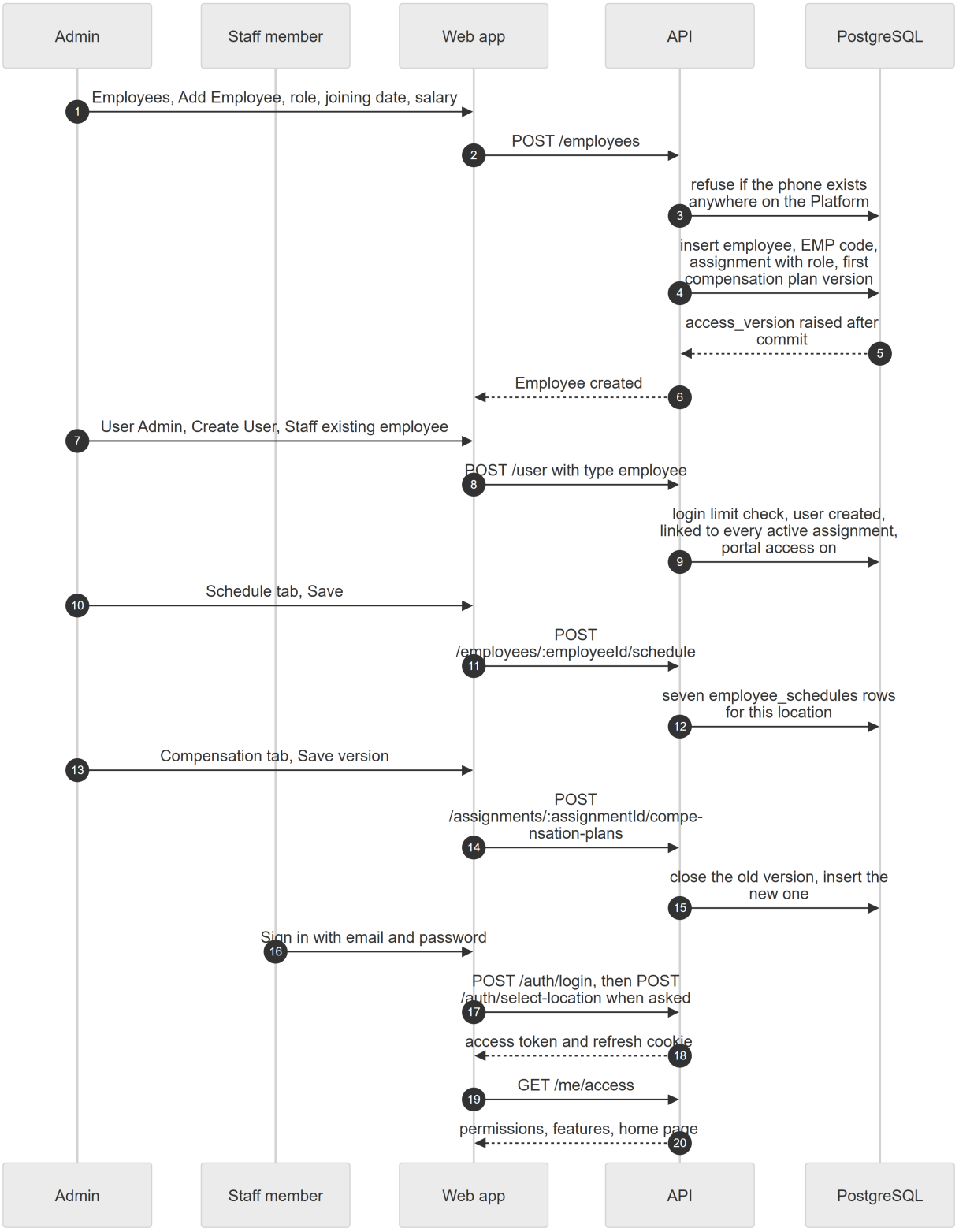

Figure 47.1 — From the Add Employee form to the first sign-in.

The diagram has five participants: Admin, Staff member, Web app, API and PostgreSQL, and 20 messages. The Admin fills the Add Employee form and the Web app calls `POST /employees`. The API first refuses the request if the phone number exists anywhere on the Platform, then inserts the employee, an `EMP-` code, the assignment with its role and the first compensation plan version. PostgreSQL raises the organization's access version after the commit, and the API answers "Employee created". Next the Admin opens User Admin and creates a staff login (`POST /user` with type employee); the API checks the login limit, creates the user and links it to every active assignment with portal access on. The Admin saves the weekly schedule (`POST /employees/:employeeId/schedule`, seven rows for this location) and, if needed, a new compensation plan version

(`POST /assignments/:assignmentId/compensation-plans`). Finally the Staff member signs in, the Web app calls `POST /auth/login` and, when asked, `POST /auth/select-location`, and then `GET /me/access`, which returns permissions, features and the home page.

Step 1 — Employee record, assignment and role

Module: Employees. **Status:** [Implemented]. On **Employees**, **Add Employee** asks for Full Name, Phone, Designation, Role, Employment Type and Joining Date. The server rejects a phone that already exists on the Platform (the employee record is shared across clinics). It generates the employee code (`EMP-001` style, counting the organization's assignments), inserts the assignment at the active location with the chosen role, and adds the first compensation plan version starting on the joining date: salaried with the entered salary, unless a commission plan was chosen (Chapter 37, Chapter 35). The role decides what the person may do at that location; a login alone grants nothing.

Step 2 — Staff login

Module: Employees and User Admin. **Status:** [Implemented]. There are two routes. The **Portal Access** switch on the employee form creates a login with an email and a password of at least 6 characters, linked only to the active location's assignment. **User Admin** → **Create User** → **Staff (existing employee)** needs a password of at least 8 characters, checks the plan's login limit and links the login to all the person's active assignments in the organization. Only the second route checks the limit (Chapter 37). The user receives no verification email when created by an administrator (Chapter 56).

Step 3 — Weekly schedule

Module: Employees. **Status:** [Implemented]. On the **Schedule** tab the Admin switches days on or off, sets start and end times and a slot length of 15, 30, 45 or 60 minutes, and saves seven rows. Booking checks these hours: a doctor with no working block that day cannot be booked (Chapter 29). The Save button is visible to people without `employees.manage`, but the server refuses them. Availability overrides and approved leave are stored but do not affect booking [Partial] (Chapter 37).

Step 4 — Compensation plan

Module: Commissions. **Status:** [Implemented]. On the **Compensation** tab the Admin picks Salaried, Commission or Both and an effective date. A new version closes the old one on the previous day. Commission is allowed only for roles that can conduct consultations, and the feature `billing.commissions` must be on. A second change on the same day replaces the first only if no charge used it (Chapter 35).

Step 5 — First sign-in

Module: Authentication and Access control. **Status:** [Implemented]. The staff member signs in with the email and password. If they have one location, it becomes active at once; otherwise the app shows the location picker. `GET /me/access` returns what they can do and the role's home page, for example `/queue/doctor` for a Doctor (Chapter 8, Chapter 11, Chapter 56).

What is not possible: a second location

Module: Employees. **Status:** [Not implemented]. There is no endpoint that assigns an existing employee to another location or organization. `CreateAssignmentDto` exists but nothing uses it. Add Employee at a second location fails because the phone number is already taken. A doctor who works at two locations therefore cannot be set up through the Platform (Chapter 37).

47.3 What can go wrong

- **Duplicate phone** gives 409 "An employee with this phone number already exists". A duplicate CNIC is caught only by a database index and returns a generic 500 error.
- **Duplicate login email** gives 409.
- **Commission plan refused**: 403 `COMMISSIONS_DISABLED` when the feature is off, 422 `COMMISSION_ROLE_CANNOT_CONSULT` for a non-consulting role.
- **Login limit** reached gives `PLAN_LIMIT` on the User Admin route.
- **Wrong or missing role**: the person signs in but sees a restricted sidebar or the no-access page.
- **Read-only mode**: every step is refused with 402.
- The person is saved before the login and the assignment, outside the final transaction, so a failure part-way can leave a person record with no assignment.

47.4 Data left behind

- `employees` (person), `employee_org_location_assignments` (role, location, `user_id`, `has_portal_access`).
- `users` (login), `employee_schedules` (seven rows per location).
- `compensation_plans` and `commission_rules`.
- `organizations.access_version` raised by the assignment save.

Chapter 48 — Workflow: The Patient Journey

This chapter follows Sample Patient A through one visit at Example Clinic, from registration to a printed receipt. It is written for the product owner. It follows the order the code really uses, which differs from a naive reading in two places: the invoice is created when payment is collected, and a lab case is a separate flow that is billed separately.

48.1 Who is involved

- **Receptionist R1**: registers, books, checks in and collects payment.
- **Dr. Example One**: calls the patient, consults, prescribes, completes the visit.
- **Sample Patient A**: the patient.
- **Accountant** and **Admin**: may also collect payments and correct invoices.

48.2 The steps

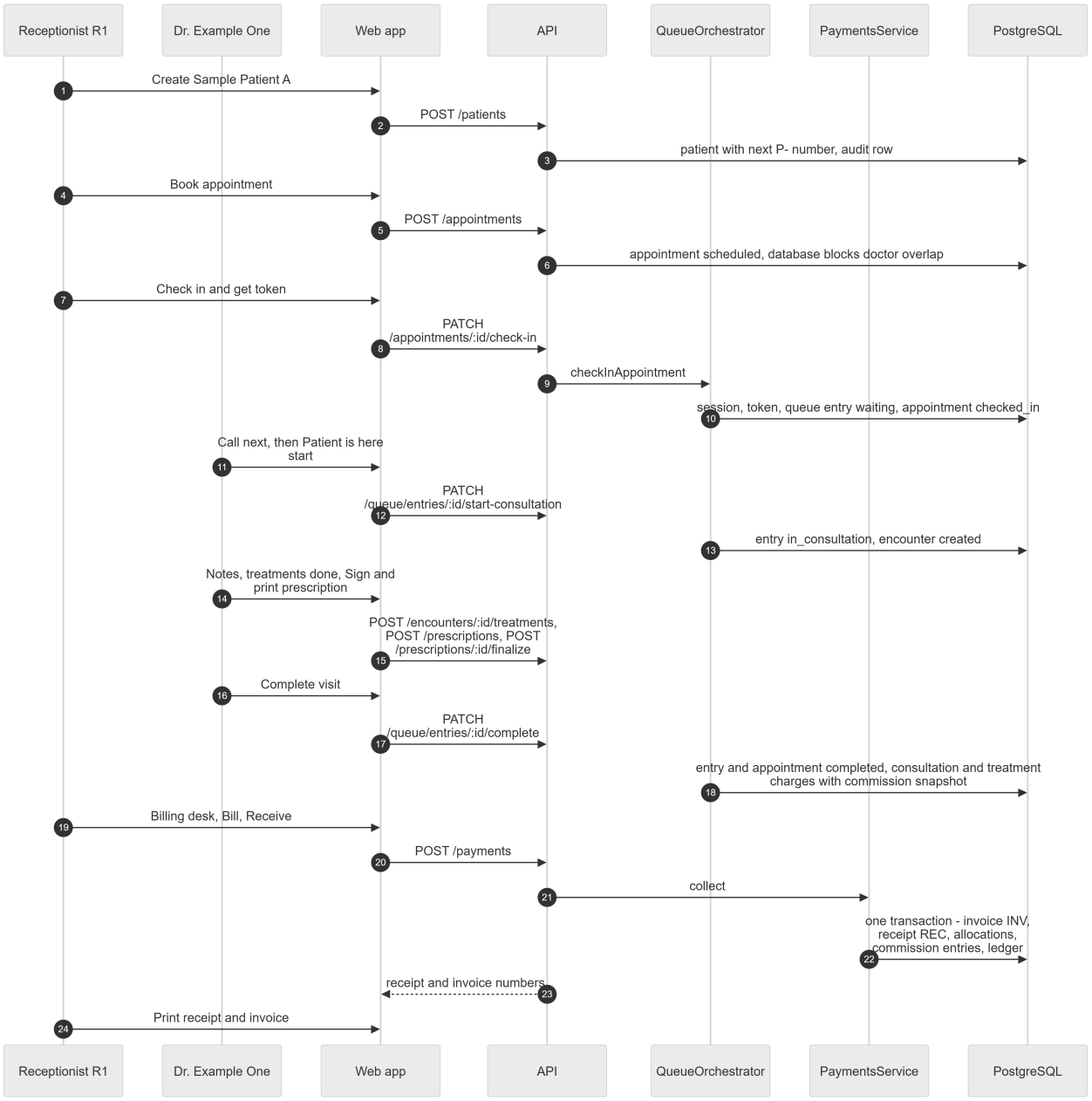

Figure 48.1 — One visit from registration to receipt.

The diagram has seven participants: Receptionist R1, Dr. Example One, Web app, API, QueueOrchestrator, PaymentsService and PostgreSQL, and 24 messages. R1 creates the patient (`POST /patients`) and the API stores the patient with the next `P-` number and an audit row. R1 books an appointment (`POST /appointments`); it is saved as scheduled and the database blocks a doctor overlap. On arrival R1 clicks "Check in and get token" (`PATCH /appointments/:id/check-in`), the API hands over to QueueOrchestrator, which writes the session, the token, a queue entry in state waiting and sets the appointment to checked in. Dr. Example One calls the next patient and presses "Patient is here, start" (`PATCH /queue/entries/:id/start-consultation`); the entry becomes in consultation and the encounter is created. The doctor records notes and treatments, signs the prescription (`POST /encounters/:id/treatments` , `POST /prescriptions` , `POST /prescriptions/:id/finalize`) and presses "Complete visit" (`PATCH /queue/entries/:id/complete`); in one transaction the entry and appointment become completed and the consultation and treatment charges are created with the commission snapshot. R1 then opens the billing desk, presses Bill and Receive (`POST /payments`); PaymentsService runs one transaction that issues the invoice, records the receipt, allocates the money, writes commission entries and the ledger row. The API returns the receipt and invoice numbers and R1 prints them.

Step 1 — Registration

Module: Patients. **Status:** [Implemented]. **Create** on `/patients` takes Full Name and Phone. The phone must be unique inside the organization. The server assigns `P-` plus a five-digit number and writes an audit row. The check-before-save endpoint exists, but no screen calls it [Partial] (Chapter 28).

Step 2 — Appointment

Module: Appointments. **Status:** [Implemented]. The booking checks the doctor works at that location, that the slot fits a working block, that the doctor has no overlapping booking and the daily limit. A booking has no token yet. A walk-in skips this step (Chapter 29).

Step 3 — Check-in

Module: Appointments and Queue. **Status:** [Implemented]. Check-in is allowed only on the booking's own day and up to the location's maximum lateness (45 minutes by default). It opens the doctor's queue session if needed and issues a token such as A-007. A patient who is too late is offered "Convert to walk-in" (Chapter 29, Chapter 30).

Step 4 — Queue

Module: Queue. **Status:** [Implemented]. The doctor starts the queue, then **Call next** picks the next patient by the smart or first-come rules. Reception and the TV display update live (Chapter 30).

Step 5 — Consultation

Module: Consultation. **Status:** [Implemented]. Starting the consultation creates the encounter, the clinical record of this visit, in the same transaction. The workspace shows the patient summary and allergy strip, and auto-saves notes (Chapter 31).

Step 6 — Treatment

Module: Consultation. **Status:** [Implemented]. The doctor taps procedure chips from the service catalog. Each becomes a treatment. The doctor never sees prices. A treatment typed by name has no price and is later priced by reception (Chapter 31).

Step 7 — Prescription

Module: Prescriptions. **Status:** [Implemented]. In the workspace panel the doctor adds medicines and presses "Sign & print". The prescription is optional and has no effect on billing (Chapter 32).

Step 8 — Lab case (separate flow)

Module: Dental Lab. **Status:** [Not implemented] from the consultation workspace. The workspace has no lab-order button, and a lab order has no link to an encounter in the data model (Chapter 31). A lab order is actually created from the sidebar: **Lab orders** → **New lab order**, where the user picks the patient, the lab, the ordering doctor and the items, then presses **Submit to lab**. The API accepts an optional appointment and treatment on the order, but the order form in the web app does not appear to send them [Unverified]. Chapter 49 follows the lab case. Lab work is billed by its own invoice, not by the clinic's charges.

Step 9 — Automatic charges

Module: Billing. **Status:** [Implemented] with a condition. When the doctor completes the visit, the Platform creates one consultation charge and one charge per treatment, priced from the service catalog with any doctor-specific price. This runs only when the feature `billing.invoices` is enabled; that feature is off by default (Chapter 31, Chapter 34). A charge with no price is marked "needs pricing" and blocks invoicing until someone sets a price. Each charge stores a snapshot of the doctor's commission terms (Chapter 35).

Step 10 — Payment and invoice

Module: Billing. **Status:** [Implemented]. On the **Billing desk**, "ready to bill" lists patients with unbilled charges. **Collect payment** previews the split and then saves. The invoice is issued at this moment, taking the next `INV` number, with the location's sales tax added on top. Surplus money becomes the patient's advance. A patient who leaves without paying can still be invoiced with **Issue invoice** from the patient's Billing tab (Chapter 34).

Step 11 — Receipt

Module: Billing. **Status:** [Implemented]. The payment takes the next `REC` number. The result screen prints the receipt and the invoice in A4 or 80 mm thermal layout. There is no server-generated PDF receipt [Not implemented] (Chapter 34).

48.3 What can go wrong

- Duplicate phone on registration gives 409.
- Booking conflicts: `DOCTOR_DOUBLE_BOOKED`, `DAILY_LIMIT_REACHED`, outside working hours.
- Check-in too late (`LATE_LIMIT_EXCEEDED`), wrong day (`NOT_TODAY`), queue closed (`SESSION_NOT_OPEN`).
- A stale queue screen gives 409 `VERSION_CONFLICT`.
- With `billing.invoices` off, completion succeeds and no charge appears.
- A charge needing a price blocks the invoice ("Set a price for ...").
- Payment errors: missing reference for an account that requires one, allocation above balance.
- In read-only mode every write gives 402.

48.4 Data left behind

`patients`, `audit_logs`, `appointments`, `queue_sessions`, `queue_entries`, `queue_events`, `outbox`, `idempotency_keys`, `patient_encounters`, `treatments`, `prescriptions` and children, `charges`, `billing_audit_logs`, `invoices`, `payments`, `payment_allocations`, `document_sequences`, `ledger_entries`, `commission_entries`.

Chapter 49 — Workflow: A Dental Lab Case

This chapter follows one lab case, a zirconia crown for Sample Patient A, from the order to the delivered work, and then follows the money: what the patient is billed, what the clinic owes the lab and what the doctor earns. It is written for the product owner. The case involves three modules (Dental Lab, Lab Finance, Commissions) and two sides: the clinic and the lab, which is not a tenant of the Platform. The lab feature `dental.lab_orders` is off by default; a super admin entitles it and the owner switches it on.

49.1 Who is involved

- **Dr. Example One**: creates and submits the order, decides try-ins, approves and delivers.
- **Receptionist R1**: receives shipments at the front desk.
- **Accountant**: bills the patient for lab work, approves vendor bills, pays the lab.
- **Example Dental Lab**: the **lab**. Its people are **lab members** (lab admin, lab technician) who use the **lab portal**, or a contact who acts through a **magic link**.
- **Admin**: may act on behalf of a lab that does not answer.

49.2 The steps

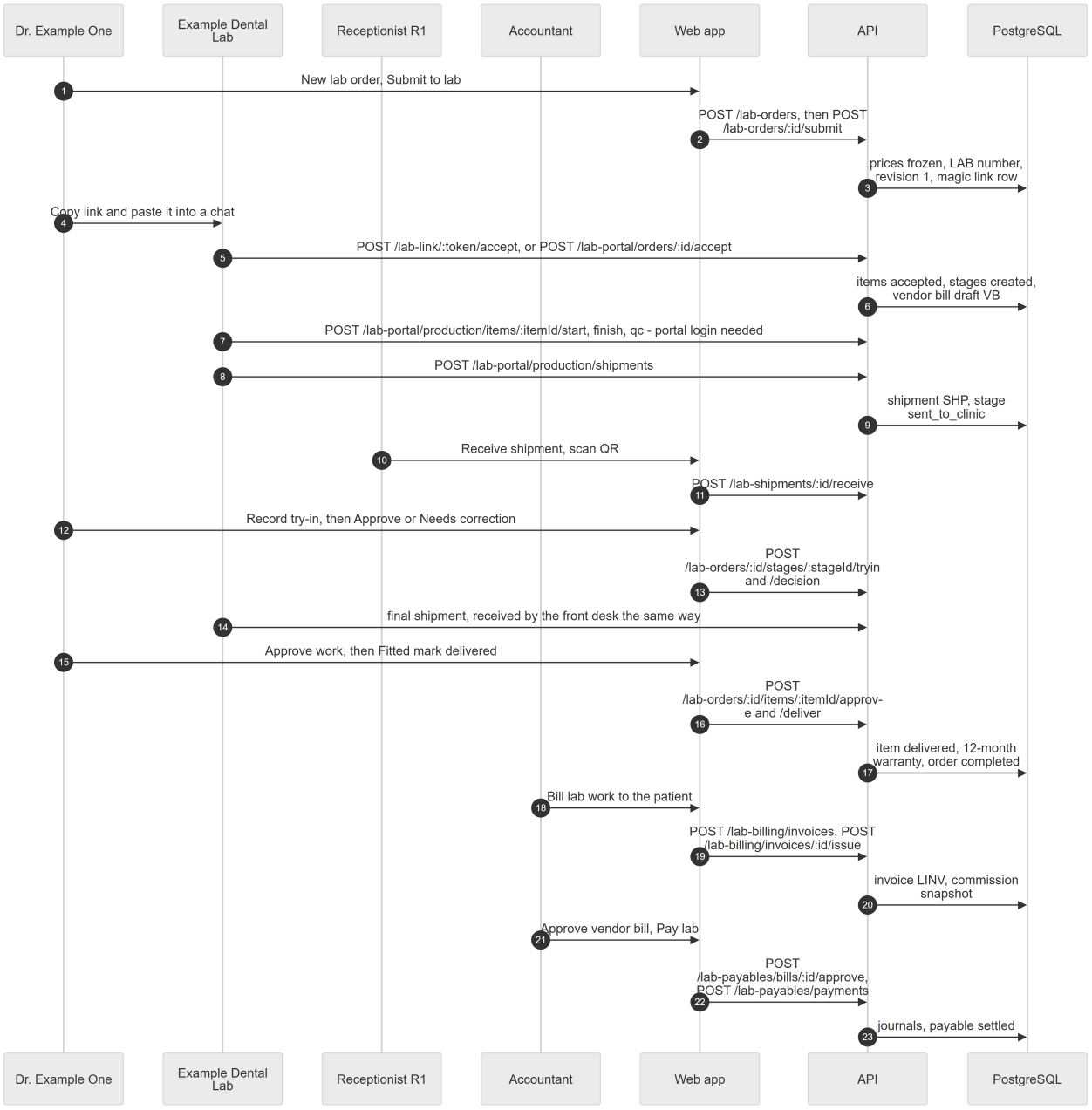

Figure 49.1 — Order to delivery, then the money.

The diagram has seven participants: Dr. Example One, Example Dental Lab, Receptionist R1, Accountant, Web app, API and PostgreSQL, and 23 messages. The doctor submits the order (`POST /lab-orders` , then `POST /lab-orders/:id/submit`); the API freezes the prices, assigns a LAB number, stores revision 1 and a magic link row. The doctor copies the link and pastes it into a chat. The lab accepts, through the link (`POST /lab-link/:token/accept`) or the portal (`POST /lab-portal/orders/:id/accept`), and the API marks the items accepted, creates the stages and drafts a vendor bill. The lab then works through the portal: start, finish and QC (`POST /lab-portal/production/items/:itemId/start` and the following actions) and dispatches a shipment (`POST /lab-portal/production/shipments`). The front desk receives it by scanning the QR code (`POST /lab-shipments/:id/receive`). The doctor records the try-in and approves or asks for correction (`.../stages/:stageId/tryin` and `/decision`). After the final shipment is received, the doctor approves and marks the work delivered (`.../items/:itemId/approve` and `/deliver`), which starts a 12-month warranty and completes the order. The Accountant bills the patient (`POST /lab-billing/invoices` and `/issue`), approves the vendor bill and pays the lab (`POST /lab-payables/bills/:id/approve` , `POST /lab-payables/payments`).

Step 1 — Create and submit the order

Module: Dental Lab. **Status:** [Implemented]. **Lab orders** → **New lab order**: choose patient, lab, ordering doctor, priority and date needed; add items (service, options, quantity, teeth, form answers); **Submit to lab**. The lab must be an active **lab link** of the clinic. At submit the Platform checks the forms, freezes the lab and patient prices of every line, assigns a number such as LAB-0001 and queues a message for the lab members. Every line needs a lab price and a mapped clinic treatment on the submit day (Chapter 40).

Step 2 — Reach the lab

Module: Dental Lab. **Status:** [Partial]. Three routes exist. The lab portal gives members the order in their own inbox [Implemented] (Chapter 41 covers the portal). A **magic link** opens one order without a login and allows view, accept, reject and ask a question only. **Send link to lab** queues a message, but the only registered provider for WhatsApp, SMS and email writes the message to the server log, so nothing reaches the lab; **Copy link** returns the link once and the doctor pastes it into a chat by hand (Chapter 40, Chapter 44). A coordinator holding both `lab.orders.create` and `lab.links.manage` can record the lab's answer on its behalf [Implemented].

Step 3 — Lab accepts

Module: Dental Lab and Lab Finance. **Status:** [Implemented]. Accept needs an expected date. Items move to accepted, the stage rows are copied from the item's stage template, and a draft **vendor bill** is created or updated at the frozen lab cost (Chapter 42).

Step 4 — Production stages

Module: Lab Portal. **Status:** [Implemented]. Production actions (start, assign a technician, finish, QC, dispatch a shipment) are lab portal endpoints for members. A lab that has only a magic link and no portal login cannot perform them, and the clinic's proxy actions cover only accept, reject, clarification and expected date. The clinic follows progress on the order's Production panel (Chapter 40).

Step 5 — Try-in

Module: Dental Lab. **Status:** [Implemented]. For items with try-in stages the doctor presses **Record try-in** after the patient visit, then **Approve** or **Needs correction** with notes. A correction closes the round and opens the same stage as the next round. No appointment is created or reminded automatically; the event for a due try-in is never emitted (Chapter 40).

Step 6 — Shipment and receive

Module: Dental Lab. **Status:** [Implemented]. The front desk opens the incoming shipment or scans its QR code, sets condition and quantity per line and presses **Receive**. A parcel that never arrives can be marked lost with a reason.

Step 7 — Approve and deliver

Module: Dental Lab. **Status:** [Implemented]. The doctor presses **Approve work**, then **Fitted, mark delivered**, which starts a 12-month warranty. Rejecting received work opens a remake request that the lab may dispute. When all items are approved or delivered the order is completed.

Step 8 — Money: patient lab invoice

Module: Lab Finance. **Status:** [Implemented]. This is not the clinic's normal invoice: lab work is billed through its own tables and numbers (`LINV`). The Accountant opens the submitted order and presses **Bill N item(s)**, which starts an **estimate**; **Issue invoice** gives it a number and due date. The lines copy the patient prices frozen at submit. This can be done any time after the order is submitted, not only after delivery. Payments, advances and credit notes follow (Chapter 42).

Step 9 — Money: vendor bill and payment to the lab

Module: Lab Finance. **Status:** [Partial]. The draft vendor bill appears under **Lab payables** → **Bills**. The Accountant presses **Approve**, setting the lab's invoice number, due date and tax, which books the debt. **Pay lab** records a payment and splits it over approved bills. Editing the bill header and adding manual lines exist only in the API (Chapter 42).

Step 10 — Money: lab-line commission

Module: Commissions. **Status:** [Implemented] (backend only by design). When the lab invoice is issued, each line stores the doctor's plan and rate. As the patient pays, the doctor earns commission on the money collected minus the lab's cost, so the doctor is paid on the clinic's margin. Reversals and credit notes give it back. It needs `billing.commissions` on and feeds My earnings and payroll like any other commission (Chapter 35).

49.3 What can go wrong

- Submit fails when a line has no lab price, no mapped treatment, a per-tooth service without teeth or answers that fail the form (`LAB_ORDER_INVALID` , `LAB_PRICE_MISSING`).
- The lab link is not active: `LAB_NOT_LINKED` .
- A magic link dies after 24 hours by default, on revoke, when the order closes or when the lab is suspended.
- A lab that never answers: use the proxy actions.
- Parcels may arrive damaged, short or not at all.
- A rejected final piece needs a remake; who pays is decided in the remake request.
- Double billing is blocked: an item can be on only one live lab invoice.
- A bill of 0 cannot be approved; approval is refused if the draft changed since it was read.
- Four seeded notification events are never emitted, including overdue orders.

49.4 Data left behind

`lab_orders` , `lab_order_items` , `order_revisions` , `magic_links` , `order_item_stages` , `shipments` , `item_warranties` , `status_history` , `audit_logs` , `notification_outbox` , `vendor_bills` , `vendor_bill_lines` , `vendor_payments` and allocations, `lab_invoices` , `lab_invoice_lines` , `commission_entries` with a lab invoice line, and journal rows in `ledger_journals` .

Chapter 50 — Workflow: Closing the Clinic Day

This chapter describes how a clinic day ends at Example Clinic: the doctor closes the queue, late bookings become no-shows, reception reviews the day's collection, the accountant closes the counter, and the Platform's overnight jobs run. It is written for the product owner. Some of the work is done by people and some by background jobs in the worker process (Appendix E lists all jobs). Job times use the server clock, not each clinic's timezone.

50.1 Who is involved

- **Dr. Example One**: closes the queue.
- **Receptionist R1**: reads the day's collection.
- **Accountant**: closes the counter. Admin may also do this.
- **Worker process**: the one process that runs scheduled jobs.

50.2 The steps

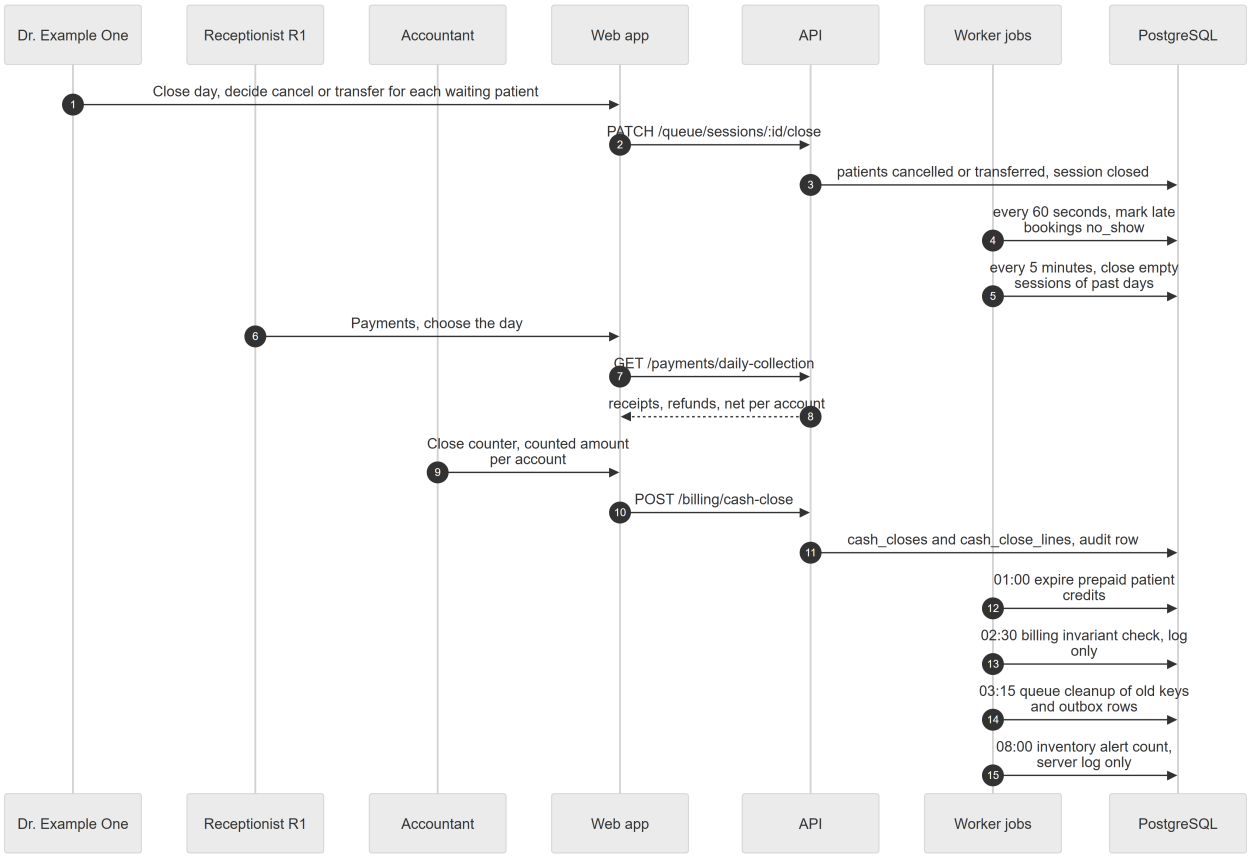

Figure 50.1 — People first, then the jobs that run on their own.

The diagram has seven participants: Dr. Example One, Receptionist R1, Accountant, Web app, API, Worker jobs and PostgreSQL, and 15 messages. The doctor closes the day and decides cancel or transfer for each waiting patient (`PATCH /queue/sessions/:id/close`); the API cancels or transfers them and closes the session. In parallel, the worker marks late bookings as no-shows every 60 seconds and closes empty sessions of past days every 5 minutes. Receptionist R1 opens Payments and picks the day (`GET /payments/daily-collection`) and the API returns receipts, refunds and net per account. The Accountant closes the counter with a counted amount per

account (`POST /billing/cash-close`) and the API stores the close and its lines with an audit row. The worker then runs four overnight jobs: expire prepaid credits at 01:00, billing invariant check at 02:30, queue cleanup at 03:15 and inventory alert count at 08:00.

Step 1 — Queue day-end

Module: Queue. **Status:** [Implemented]. In **My Queue**, **Close day** is disabled while a patient is with the doctor. The "Close today's queue" dialog lists every remaining patient (waiting, skipped or at a test) with **Cancel** as the default or "Transfer to Dr. ..." for waiting patients only. Confirming cancels or moves them and closes the session. Cancelling a booked patient closes the appointment as cancelled. Everything succeeds or nothing does (Chapter 30).

Step 2 — Automatic day-end

Module: Queue. **Status:** [Implemented]. Every 5 minutes the worker closes sessions of a past service day if they are empty. A session that still has patients is only logged as an alert and is never cancelled automatically, so someone must decide. The "past day" is judged by each location's timezone and day cut-off, not by the server date.

Step 3 — No-show marking

Module: Appointments and Queue. **Status:** [Implemented]. Staff can press **No show** on a scheduled or confirmed booking (`appointments.cancel`). Otherwise the worker does it every 60 seconds once start time plus maximum lateness (45 minutes by default) has passed. The job writes with plain SQL, so there is no queue event, no audit row and no realtime message. A no-show no longer shows Check in in the web app, so a late patient usually has to be added as an ordinary walk-in (Chapter 29).

Step 4 — Daily collection

Module: Billing. **Status:** [Implemented]. On **Payments**, the day picker (future days not allowed) shows receipts by their payment date and refunds by their refund date per payment account, with the net in hand (Chapter 34).

Step 5 — Cash close

Module: Billing. **Status:** [Implemented]. **Close counter** computes for each payment account the expected money since the previous close, asks for the counted amount on accounts marked "Count physically at cash close", and requires a note when there is a difference. A close is a permanent record with a printable slip and a red "Flagged" badge when the difference exceeds the alert amount. Closing never blocks taking payments, and a close cannot be edited or deleted. It needs `billing.cash_close` (Admin, Accountant).

Step 6 — Overnight jobs

Module: Several. Times are server time (Appendix E).

Time	Job	Status
01:00	Patient credit expiry: marks credits past their expiry date as expired	[Implemented] (credit packages have no screen, so little to expire in practice)
02:30	Billing invariant check: re-checks ten money rules and logs any break	[Implemented], log only
03:15	Queue cleanup: deletes idempotency keys older than 24 hours and published outbox rows older than 7 days	[Implemented]

Time	Job	Status
08:00	Inventory alerts: counts low and expiring items per location	[Partial], server log only

The table lists the four daily jobs with what each does and its status. The credit job reads the server date rather than the location's, so a credit can expire a few hours early or late for a clinic in another timezone. Nobody is notified by the invariant check or the inventory alert; an operator must read the logs (Chapter 34, Chapter 39).

50.3 What can go wrong

- Closing the queue while someone is with the doctor: 409 `DOCTOR_BUSY`.
- A remaining patient without a decision: 422 `INVALID_TRANSITION`.
- A skipped or at-test patient cannot be transferred, only cancelled or brought back.
- Counter close: 422 when a count or a note is missing, or when the close period is stale.
- Overnight times differ from clinic local time when the server is in another timezone.
- Jobs run only in a worker (or all-in-one) process; if no worker runs, none of the jobs run. Exactly one worker must run.
- The invariant check only logs; a violation is not repaired or announced.

50.4 Data left behind

`queue_sessions` (closed), `queue_entries`, `queue_events` and `outbox` (for closes done by users), `appointments` (cancelled or no-show), `cash_closes`, `cash_close_lines`, `billing_audit_logs`, `patient_credits` (expired), and server log lines for the invariant check and the inventory alerts.

Chapter 51 — Workflow: Month-End Payroll

This chapter follows payroll for one month at Example Clinic, from the automatic run on the 1st to the payment record. It is written for the product owner. Payroll combines two things: the base salary from each employee's compensation plan and the doctor commission earned but not yet paid out. The Platform records the payout; it does not move money.

51.1 Who is involved

- **Scheduler** (the worker process): runs the job.
- **Admin**: holds `payroll.manage` (feature `hr.payroll`, on by default). No other system role holds it.
- **Dr. Example One**: an employee with a salary and commission.

51.2 The steps

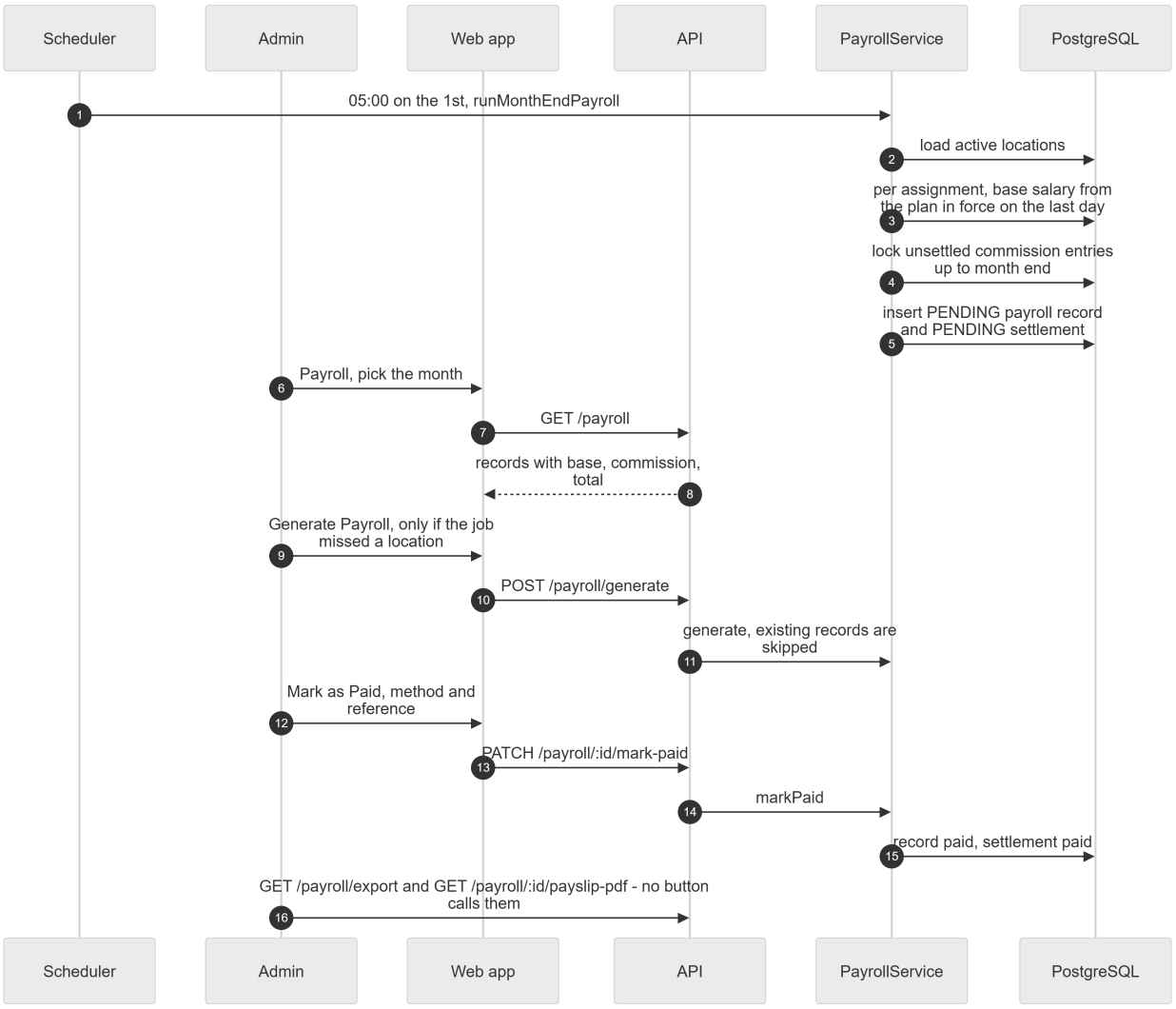

Figure 51.1 — The automatic run, the optional manual run and the payment.

The diagram has six participants: Scheduler, Admin, Web app, API, PayrollService and PostgreSQL, and 16 messages. At 05:00 on the 1st the Scheduler calls the month-end run on PayrollService, which loads the active locations. For each assignment it reads the base salary from the plan in force on the last day of the month, locks

the unsettled commission entries up to month end, and inserts a pending payroll record and a pending settlement. The Admin opens Payroll for the month (`GET /payroll`) and sees records with base, commission and total. If the job missed a location, the Admin presses Generate Payroll (`POST /payroll/generate`) and existing records are skipped. The Admin presses Mark as Paid with a method and reference (`PATCH /payroll/:id/mark-paid`); the service sets the record and the settlement to paid. The last message shows the CSV export and payslip PDF endpoints, which no button calls.

Step 1 — Automatic generation

Module: Payroll. **Status:** [Implemented]. The cron expression is `0 5 1 * *` with no timezone option, so it fires at 05:00 on the 1st in the server's timezone. A code comment says 00:05; the code wins. The month is taken from the server's own date (the previous month), not from each location's timezone. The job covers every active location of every organization, each in its own transaction, so one failing location does not stop the others. It does not check that the organization is active or that `hr.payroll` is on. It runs without a request, so the subscription read-only guard, which acts on web requests, does not apply to it (deduced from the guard being an HTTP guard) (Chapter 38, Appendix E).

Step 2 — Base plus commission

Module: Payroll and Commissions. **Status:** [Implemented]. For each active assignment: skip if a record already exists for that employee, location and month; otherwise base salary = the plan version in force on the month's last day (else the first day); commission = the sum of unsettled commission entries created before the end of the month's last clinic day. A net-negative sum is not settled and waits for a later month. Total = base + commission. An employee with neither is listed as "no salary" and gets no record (Chapter 38, Chapter 35).

Step 3 — Settlement lock

Module: Commissions. **Status:** [Implemented]. The entries are row-locked and tied to a **settlement** so they cannot be paid twice. Entries are never edited; a reversal after settlement lands as a negative entry in a later month's settlement (Chapter 35).

Step 4 — Manual generation

Module: Payroll. **Status:** [Implemented]. **Generate Payroll** on `/payroll` takes a month and year, refuses future months and generates for the whole active location, even when pressed from an employee tab. It is safe to run after the automatic job.

Step 5 — Review and edit

Module: Payroll. **Status:** [Implemented]. Records start as pending. Edit, delete and create-by-hand exist. A record that paid out commission cannot be deleted. Deleting an unpaid record releases its commission back to the pool. Putting a record on hold has an endpoint but no button; the edit dialog's status field can set it [Partial].

Step 6 — Mark paid

Module: Payroll. **Status:** [Implemented]. **Mark as Paid** needs a method (cash, bank transfer or cheque) and optional reference and notes. The record becomes paid with time and payer, and the settlement becomes paid in the same transaction. No money moves and nothing is posted to the general ledger or expenses. Setting status to paid through the edit dialog does not pay the settlement, which is a trap: a later delete would release the commission.

Step 7 — Payslip and export

Module: Payroll. **Status:** [Partial]. `GET /payroll/:id/payslip-pdf` returns an A4 PDF (Latin text only) and `GET /payroll/export?month&year` returns a CSV. Neither has a button in the web app, so they can be called only through the API. Deductions are always 0.00, net equals gross, and the designation column is empty.

51.3 What can go wrong

- Server timezone differs from the clinic's, so "the 1st at 05:00" falls at another local time, and the month boundary can shift near midnight.
- No worker process is running: nothing is generated.
- A location error is logged and skipped; no in-app notification tells anyone.
- Future month: 400 "Cannot generate payroll for future months".
- "Payroll already marked as paid" on a second payment.
- Paying through the edit dialog leaves the commission unpaid.
- No deductions, tax, overtime or unpaid-leave handling exist [Not implemented].
- Payroll changes are not written to the general audit log.

51.4 Data left behind

`payroll_records` (pending, then paid with `paid_at`, method, reference), `commission_settlements` (period, total, payroll record id, status), `commission_entries.settlement_id`. Records made by the job have no `created_by`.

Chapter 52 — Workflow: Correcting an Invoice and Refunding Money

This chapter follows what the Accountant does when an issued invoice turns out to be wrong: preview and save an edit that creates a new revision, cancel an unpaid invoice, refund money, or undo a payment first. It is written for the product owner, who needs to know what stays on record and how doctor commission follows each change. Invoices are never deleted; every change adds to the history.

52.1 Who is involved

- **Accountant** (or Admin): holds `billing.invoice.edit`, `billing.invoice.void` and `billing.refund.issue`. The last two are marked high risk.
- **Sample Patient A**: the patient whose invoice is corrected.
- **Dr. Example One**: the doctor whose commission adjusts.

52.2 The steps

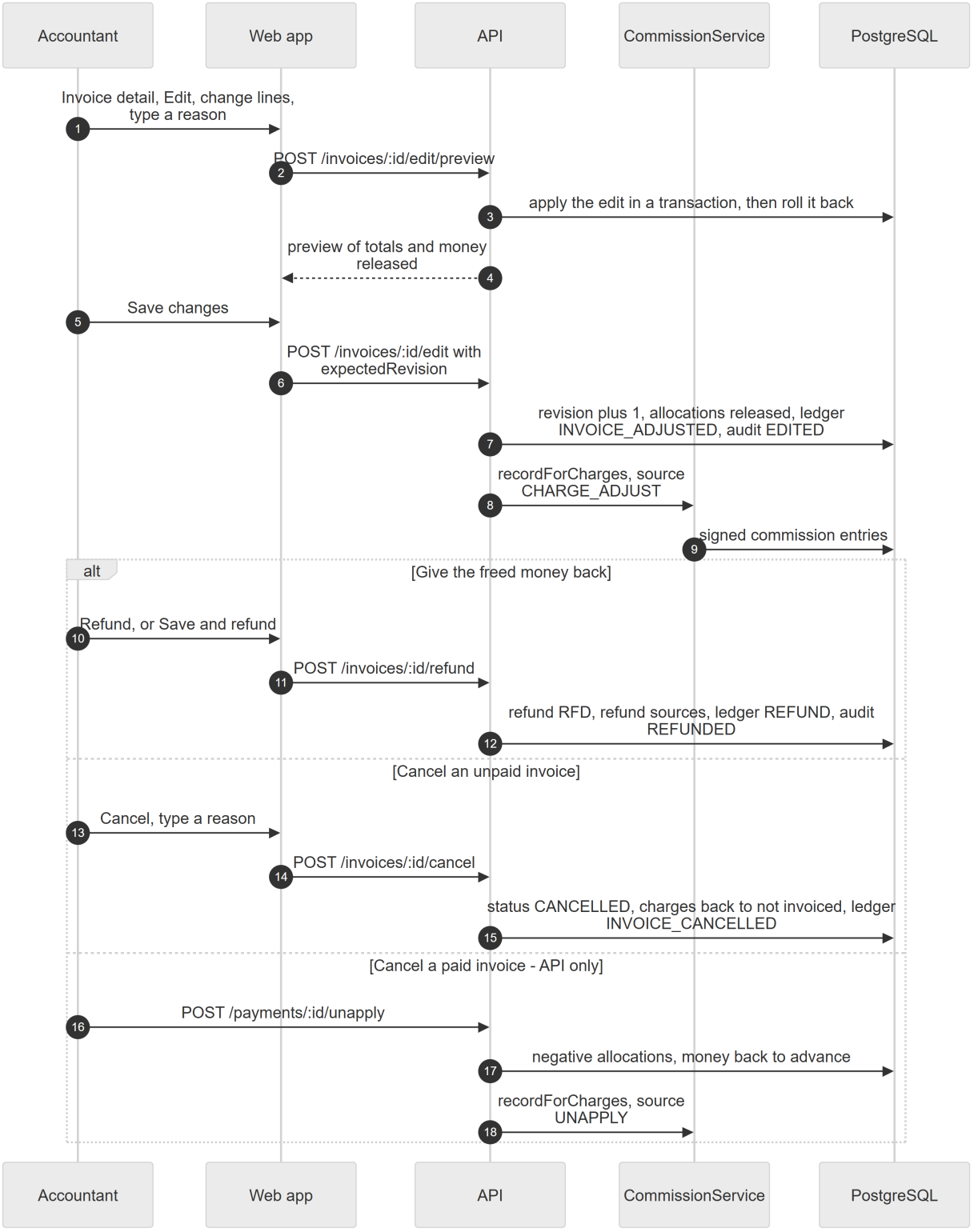

Figure 52.1 — Edit with preview, then refund or cancel.

The diagram has five participants: Accountant, Web app, API, CommissionService and PostgreSQL, and 18 messages. The Accountant opens the invoice, presses Edit, changes lines and types a reason; the Web app calls `POST /invoices/:id/edit/preview`, the API applies the edit inside a transaction and rolls it back, and returns the preview. The Accountant presses Save changes (`POST /invoices/:id/edit` with the expected revision); the API raises the revision, releases allocations, posts an adjustment to the ledger and writes an audit row, then asks CommissionService to record the signed commission entries. An "alt" box has three outcomes. To give freed

money back, the Accountant presses Refund or Save and refund (`POST /invoices/:id/refund`) and the API writes the refund, its sources, a ledger row and an audit row. To cancel an unpaid invoice, the Accountant presses Cancel with a reason (`POST /invoices/:id/cancel`), and the invoice becomes cancelled with its charges back to not invoiced. To cancel a paid invoice, an API client must first unapply the payment (`POST /payments/:id/unapply`), which writes negative allocations and reverses commission.

Step 1 — Preview the edit

Module: Billing. **Status:** [Implemented]. On the invoice detail, **Edit** lets the Accountant change quantities, prices and discounts, void lines, add unbilled charges or add a manual line, with a reason of 5 to 300 characters. The preview really runs the edit in a transaction and discards it, so the numbers match what saving will do (Chapter 34).

Step 2 — Save a revision

Module: Billing. **Status:** [Implemented]. Saving raises the revision number; the invoice number never changes. If a line's payable shrinks, the allocation above the new amount is released, newest first, and moves to the patient's advance. Older revisions can be printed from audit snapshots. If two people edit at once, the second gets `BILLING_INVOICE_REVISION_CONFLICT` . Voiding a line needs `billing.invoice.void` ; changing a catalog price needs `billing.price.override` .

Step 3 — Commission follows

Module: Commissions. **Status:** [Implemented] (backend only). On every edit the Platform recomputes the doctor's commission target per charge and inserts only the difference as a signed entry. A voided charge brings its commission to zero. Entries already settled in payroll are never changed; the correction lands in the next settlement (Chapter 35).

Step 4 — Cancel an unpaid invoice

Module: Billing. **Status:** [Implemented]. **Cancel** needs a reason and is shown only when nothing is paid. The invoice keeps its totals and prints with a CANCELLED watermark, the number stays used, the charges return to Ready to bill, and the patient's statement gets a credit. Cancelled is final.

Step 5 — Unapply a payment (backend only)

Module: Billing. **Status:** [Partial]. `POST /payments/:paymentId/unapply` takes back all or part of a payment's allocations and sends the money to advance. It adds negative rows and deletes nothing, and the receipt stays. No screen calls it, so cancelling an invoice that has payments cannot be done from the web app (`BILLING_INVOICE_HAS_PAYMENTS` : "Unapply its payments before cancelling this invoice").

Step 6 — Refund

Module: Billing. **Status:** [Implemented]. A refund always comes out of the patient's advance. An **advance refund** returns unused money from the patient's Billing tab. An **invoice refund** returns money released from an invoice, optionally in the same step as the edit that released it. The accountant chooses the account under "Paid out from"; it is not forced to be the account that received the payment. Sources are chosen automatically: payments released from that invoice first, latest first, then other advance, newest first. A refund takes the next `RFD` number, needs a reason of 5 to 300 characters, and cannot be undone. Cash close subtracts refunds per account (Chapter 34, Chapter 50).

52.3 What can go wrong

- `BILLING_INVOICE_REVISION_CONFLICT` : someone else changed the invoice; reload.
- `BILLING_INVOICE_CANCELLED` : a cancelled invoice is read-only.
- `BILLING_REFUND_EXCEEDS_ADVANCE` and `BILLING_REFUND_EXCEEDS_INVOICE_REFUNDABLE` : the amount is above what can be returned.
- `BILLING_REFERENCE_REQUIRED` for an account that needs a reference.
- `BILLING_UNAPPLY_EXCEEDS_ALLOCATION` : more than was applied.
- Wrong refunds cannot be reversed; the only remedy is a new payment.
- Read-only mode refuses all of these with 402.

52.4 Data left behind

`invoices` (revision, status), `charges` , `payment_allocations` (including negative and release rows), `payments` , `refunds` , `refund_sources` , `ledger_entries` (INVOICE_ADJUSTED, INVOICE_CANCELLED, REFUND), `billing_audit_logs` (EDITED, CANCELLED, REFUNDED, UNAPPLIED with snapshots), `commission_entries` (signed differences), `document_sequences` .

Chapter 53 — Workflow: Subscription Expiry and Recovery

This chapter follows Example Clinic when its trial runs out and the clinic is brought back by a manual payment. It is written for the product owner. The Platform never charges anyone: the clinic pays outside the Platform and a super admin records the payment, which extends the period. The subscription status is never stored; it is worked out from two dates every time it is read.

53.1 Who is involved

- **Owner**: sees the banners and pays.
- **Other staff**: see warning banners and rejected saves.
- **Super admin**: records the payment and may change plan, dates and limits.

53.2 The steps

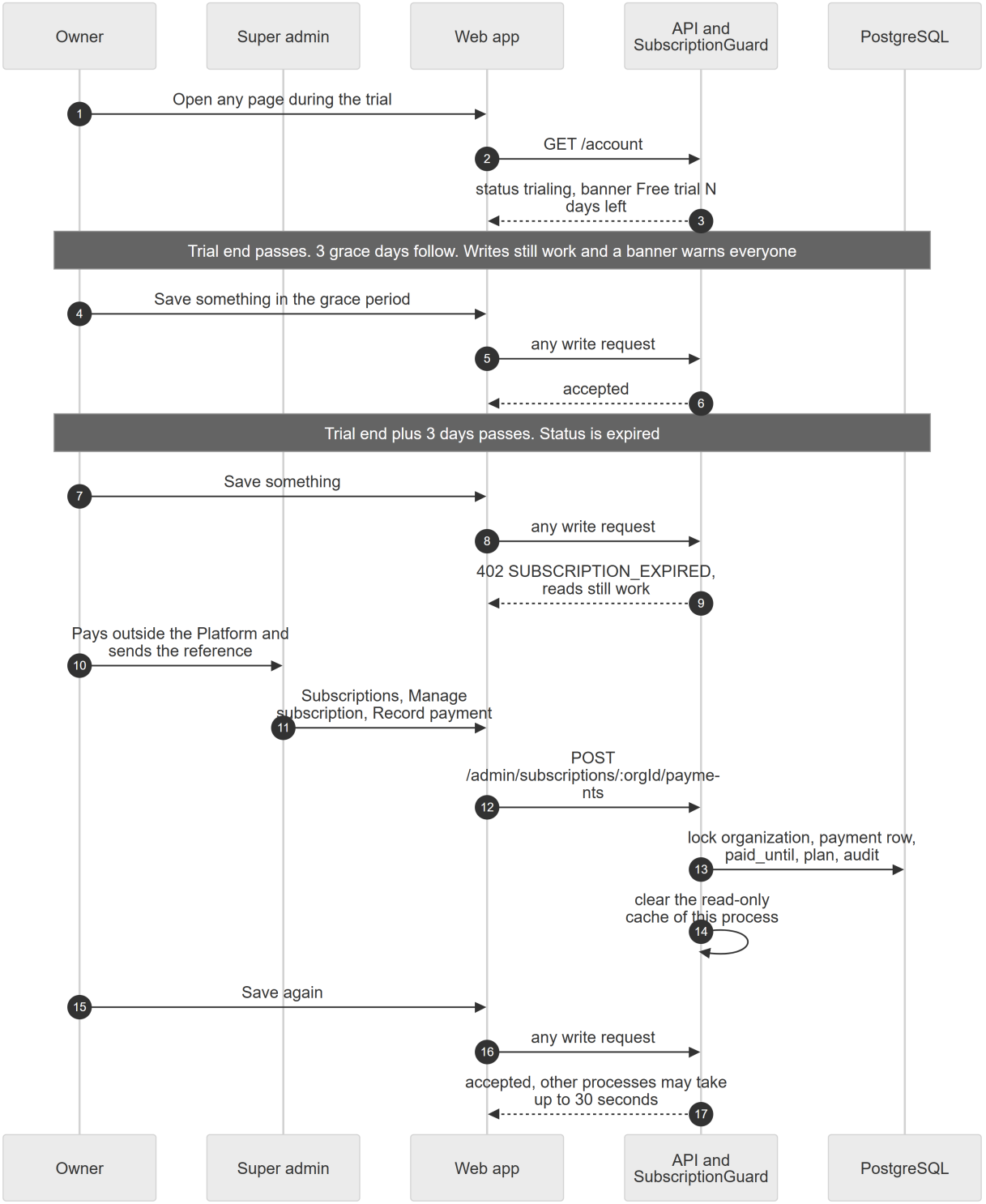

Figure 53.1 — Trial, grace, read-only, payment, back to active.

The diagram has five participants: Owner, Super admin, Web app, API and SubscriptionGuard (drawn as one box), and PostgreSQL, and 17 messages plus two notes. During the trial the Owner opens a page, the Web app calls `GET /account`, and the API answers status trialing with the banner "Free trial, N days left". A first note says that after the trial end passes there are 3 grace days in which writes still work and a banner warns everyone; a save in that period is accepted. A second note says that once the trial end plus 3 days has passed, the status is expired. A save then returns 402 `SUBSCRIPTION_EXPIRED`, while reads still work. The Owner pays outside the Platform and sends the reference to the Super admin, who opens Subscriptions, Manage subscription, Record payment (`POST`

`/admin/subscriptions/:orgId/payments`). The API locks the organization, writes the payment row, sets `paid_until` and the plan and writes an audit entry, then clears the read-only cache of its own process. The Owner saves again and the write is accepted; other server processes may take up to 30 seconds to notice.

Step 1 — Trial

Module: Subscriptions and Onboarding. **Status:** [Implemented]. Self-signup gives 14 days on the Professional plan (3 locations, 15 logins). Owners see "Free trial, N days left" with a **See plans** link. A clinic with no trial date and no paid date is "unmanaged": no limits, never expires (Chapter 45).

Step 2 — Grace period

Module: Subscriptions and Onboarding. **Status:** [Implemented]. The period end is the later of the trial end and the paid-until date. For 3 days after it (`GRACE_DAYS`) the clinic works normally; everyone sees a warning that the clinic turns read-only in N days. A plan close to its end shows owners "Your plan renews in N days".

Step 3 — Read-only mode (402)

Module: Subscriptions and Onboarding. **Status:** [Implemented]. After the grace days, the last global guard refuses every write with HTTP 402 and code `SUBSCRIPTION_EXPIRED`. Reads (GET, HEAD, OPTIONS) pass, public routes pass, super admins pass, and paths starting with `/auth/`, `/account`, `/me/` and `/admin/subscriptions` stay writable. The banner says "Subscription expired. The clinic is read-only" to owners and "read-only until the owner renews the plan" to others. Everything is affected, including the queue and billing. Jobs in the worker are not requests, so they keep running; this is deduced from the guard being an HTTP guard [Unverified].

Step 4 — Pay outside the Platform

Module: Subscriptions and Onboarding. **Status:** [Implemented]. The owner opens **Settings** → **Plan & billing** and reads the "How to pay" text, which the super admin edits on the Subscriptions page (up to 2,000 characters). Methods recorded are bank transfer, JazzCash, EasyPaisa, cash, cheque and other.

Step 5 — Super admin records the payment

Module: Subscriptions and Onboarding. **Status:** [Implemented]. On **Subscriptions**, **Manage subscription** → **Record payment** takes plan, months (1 to 36), amount, method, date, reference and notes. The new period starts where the current trial or paid period ends, or now if that is over, so early payment loses no days. The plan on the clinic becomes the plan paid for, so paying for Starter during a Professional trial lowers the limits at once.

Step 6 — Active again

Module: Subscriptions and Onboarding. **Status:** [Implemented]. With `paid_until` in the future the status is active and writes resume. The decision is cached for 30 seconds per server process, so a renewal can take up to 30 seconds to be seen by a process other than the one that took the payment. A super admin can also set a future trial end or clear both dates to make the clinic unmanaged from the **Plan & limits** tab.

53.3 What can go wrong

- The owner never sees a renewal reminder email; none is sent.
- The grace period is only 3 days.
- Recording a payment overwrites the plan and can lower the limits.

- Staff cannot read the cause of a rejected save unless the banner is visible; the API message is "Your subscription has expired. The clinic is read-only until it is renewed."
- The Organizations screen's plan selector sends values the API rejects, so use the Subscriptions screen for plan changes (Chapter 45).
- Nothing charges the clinic, so a payment the super admin forgets to record keeps the clinic read-only.

53.4 Data left behind

`subscription_payments` (plan, months, amount, method, reference, period start and end, `recorded_by`), `organizations` (`paid_until`, `subscription_plan`, limits), `audit_logs` (before and after values). The status itself is not stored.

Chapter 54 — Workflow: How an Access Change Reaches Every Screen

This chapter follows what happens after an owner edits a role, a feature switch or the sidebar layout: how the change reaches the server's caches and the staff members' open browser tabs without anyone signing out. It is written for the product owner. The key idea is the **access version**, a counter on the organization that goes up by one on every change to who can do what.

54.1 Who is involved

- **Owner** (or an Admin with `admin.roles.manage`): makes the change.
- **Staff member**: has the Platform open in a browser tab.
- **API and worker processes**: keep a cache of each user's access.

54.2 The steps

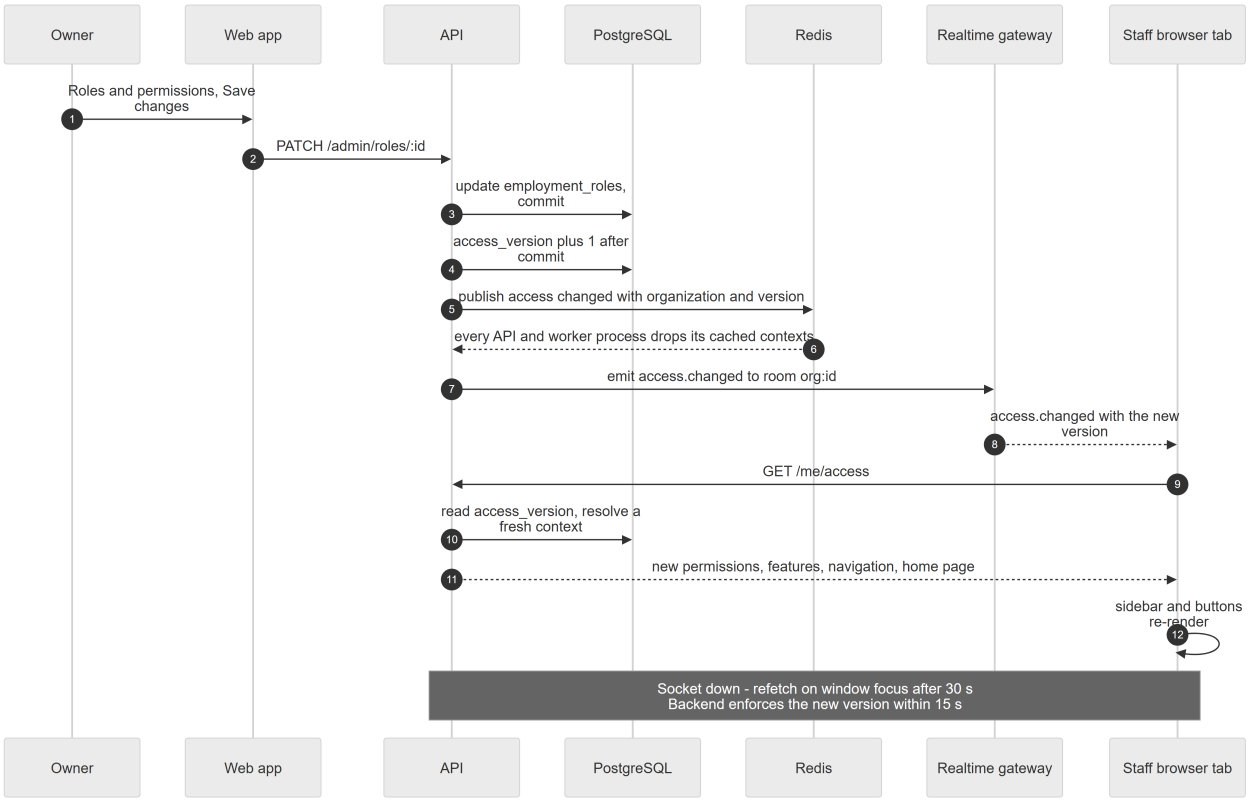

Figure 54.1 — From Save changes to a re-drawn sidebar.

The diagram has seven participants: Owner, Web app, API, PostgreSQL, Redis, Realtime gateway and Staff browser tab, and 12 messages plus one note. The Owner presses Save changes on the role (`PATCH /admin/roles/:id`). The API updates `employment_roles` and commits, then raises `access_version` by one after the commit. It publishes "access changed" with the organization and version to Redis, and every API and worker process drops its cached contexts. The API tells the realtime gateway to emit `access.changed` to the room for the organization, and the gateway sends it to the Staff browser tab with the new version. The tab calls `GET /me/access` ; the API reads the version, resolves a fresh context and returns new permissions, features, navigation

and home page, and the sidebar and buttons re-render. The note says that if the socket is down, the tab refetches on window focus once its data is 30 seconds old, and the backend still enforces the new version within 15 seconds.

Step 1 — The owner edits

Module: Roles and permissions, Features, Customization. **Status:** [Implemented]. Three kinds of edit start the chain: a role at **Roles & permissions**, a feature switch at **Features** or the entitlement page, and the clinic sidebar layout at **Navigation**. Granting a high-risk permission asks for a reason (Chapter 11, Chapter 10).

Step 2 — The version goes up

Module: Access control. **Status:** [Implemented]. `organizations.access_version` increases after any committed change to employment roles, assignments (role, location, active flag, portal access, login), organization feature flags or ownership. A database event subscriber does this automatically once per transaction, after the commit. Location feature overrides and nav layout changes bump it explicitly; saving the same nav layout again does not. The theme save has no bump in the code, so theme changes do not trigger this chain (Chapter 57).

Step 3 — Server caches drop

Module: Access control. **Status:** [Implemented]. The new version goes over Redis to every API and worker process, which drop cached contexts and feature sets. The resolver's cache is keyed on user, organization, location and access version, holds up to 10,000 entries and lasts 5 minutes; the version itself is trusted for 15 seconds without a bus message. If Redis is down, other processes catch up within 15 seconds. Because the key includes the version, a stale entry is never matched after a bump.

Step 4 — The browser is told

Module: Realtime. **Status:** [Implemented]. The gateway emits `access.changed` to the organization's room. The web app joins that room through the same socket used for the queue. A message whose version is not newer than the one the tab already has is ignored. After a reconnect the tab refetches in case messages were missed.

Step 5 — The tab refetches and redraws

Module: Access control (web app). **Status:** [Implemented]. The tab invalidates its access query, loads `GET /me/access` and redraws the sidebar, buttons and home page from `can(...)`. Route permission checks happen when the user navigates. If the room is refused or the socket is down, the fallback is refetch on window focus with a 30-second freshness window (Chapter 57).

Step 6 — Enforcement is already updated

Module: Access control (backend). **Status:** [Implemented]. The backend never trusts the screen. The access guard resolves access again on each request using the current version, so a removed permission is refused as soon as the version bump is visible to that process, even if the tab has not redrawn. A missing key means no access (fail-closed).

54.3 What can go wrong

- Redis is down: other server processes may take up to 15 seconds; tabs rely on window focus.
- Socket blocked or offline: the tab keeps old buttons until focus or the next reload, but the server still refuses actions.

- Raw SQL edits to roles do not bump the version unless done explicitly.
- A bump failure after commit is logged and the change itself stays saved, so caches may age out only on their own timers.
- Escalation protection: granting more than the editor holds is refused (`ESCALATION`).
- Super admin acting in the clinic is recorded as a platform action.

54.4 Data left behind

`organizations.access_version` , `employment_roles` (or `org_feature_flags` , `location_feature_overrides` , `nav_layouts`), `audit_logs` entries for the edit, and short-lived in-memory caches that hold no lasting data.

Chapter 55 — Workflow: From Purchase Order to Consumption

This chapter follows a box of gloves from a purchase order to stock, then to use on a patient and to a low-stock alert. It is written for the product owner. The honest summary is that the chain is only partly connected: the pieces exist in the backend, several have no screen, two steps carry a database risk that could not be tested, and the final alert reaches nobody. Each link in the chain carries its own label.

55.1 Who is involved

- **Admin**: the only system role with inventory write permissions (`inventory.po.manage` , `inventory.stock.adjust`). The Assistant role can only view.
- **Worker process**: runs the alert job.
- Features: `inventory.core` and `inventory.alerts` , both off by default.

55.2 The steps

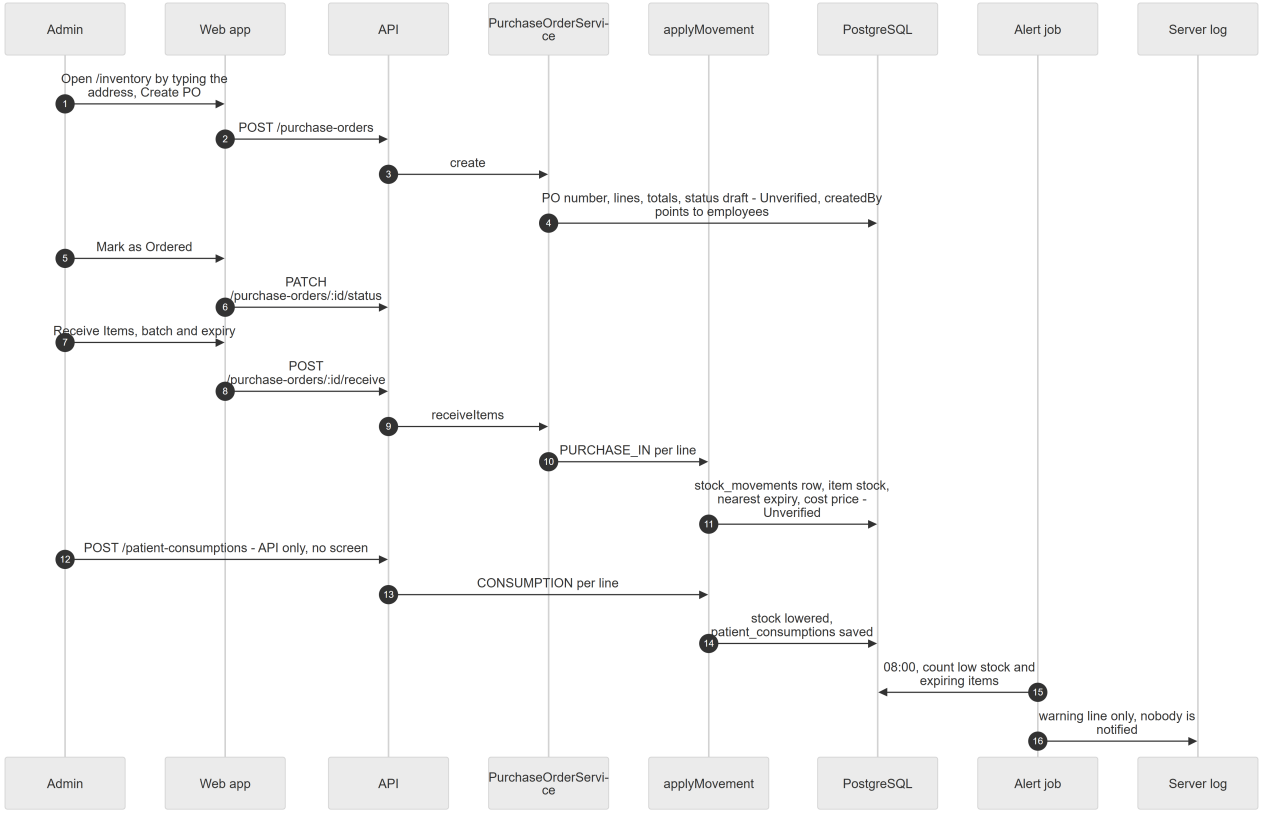

Figure 55.1 — The inventory chain and where it breaks.

The diagram has eight participants: Admin, Web app, API, PurchaseOrderService, applyMovement, PostgreSQL, Alert job and Server log, and 16 messages. The Admin opens `/inventory` by typing the address and creates a purchase order (`POST /purchase-orders`); the service stores the order number, lines, totals and status draft, with the Unverified note that the creator column points to employees. The Admin marks it ordered (`PATCH /purchase-orders/:id/status`), then receives items with batch and expiry (`POST /purchase-orders/:id/receive`). The service calls applyMovement with PURCHASE_IN per line, which writes a stock movement row and updates the item's

stock, nearest expiry and cost price, again marked Unverified. A call to `POST /patient-consumptions` has no screen; it lowers stock per line and saves the consumption. At 08:00 the alert job counts low and expiring items and writes only a warning line to the server log.

Link 1 — Reach the page

Module: Inventory. **Status:** [Partial]. The page `/inventory` works and needs `inventory.view`, but there is no sidebar entry and no screen links to it. Users must type the address. The feature `inventory.core` must be entitled and enabled first (Chapter 39).

Link 2 — Purchase order

Module: Inventory. **Status:** [Unverified]. Screens and logic exist: **Create PO** with supplier, lines, quantity, unit price and tax; the server numbers it like `PO-2026-00001` and computes totals; status goes Draft then Ordered (**Mark as Ordered**), and only a draft can be cancelled. The risk: creation stores the signed-in user's id in `purchase_orders.createdBy`, which has a foreign key to `employees`. A user id and an employee id are different records, so the insert is expected to fail. Reading the entity and the controller confirms that the user id is passed to that column; it was not run. PO numbers are also unique across the whole database but counted per organization, so a second organization's first order of a year can collide (deduced from code). Purchase orders are not linked to expenses or lab payables (Chapter 39).

Link 3 — Receive goods

Module: Inventory. **Status:** [Unverified]. **Receive Items** on an Ordered or Partially received PO takes quantities, batch and expiry per line; over-receiving is refused; the status becomes Partially received or Received. Each line adds a "purchase in" stock movement through the single `applyMovement` path, sets the item's cost price to the line price and moves the item's expiry date earlier if the batch expires sooner. It carries the same kind of foreign-key risk as the purchase order, through `stock_movements.performedBy`.

Link 4 — Stock movement and manual adjustment

Module: Inventory. **Status:** [Unverified]. Every quantity change is a row in `stock_movements` with previous and new stock; stock cannot go below zero. The Adjust stock dialog offers manual adjustment, wastage, return to supplier and transfers (notes required for wastage and manual). A transfer is recorded at one location only; nothing is added at the other. The item row is read without a row lock, so two simultaneous adjustments could start from the same number.

Link 5 — Patient consumption

Module: Inventory. **Status:** [Not implemented] in the web app. `POST /patient-consumptions` exists and prices each line at the item's current cost, then lowers stock in one transaction, but the component that would call it (`ConsumptionRecorder`) is placed on no screen, including the consultation workspace. Consumption templates (kits) can be created but nothing uses them. Consumed materials are not billed and do not show on the encounter (Chapter 39, Chapter 31).

Link 6 — Low-stock alert

Module: Inventory. **Status:** [Partial]. At 08:00 server time, for every location where both features are on, the job counts low-stock items and items expiring within 30 days and writes one warning line to the server log. No user is notified, no inbox entry is created and nothing appears on a screen; a code comment says notifications are planned (Chapter 44, Appendix E).

55.3 What can go wrong

- The first purchase order, receive or adjustment may fail with a database foreign-key error, which would stop the whole chain [Unverified].
- The page is not discoverable without a sidebar entry.
- "Insufficient stock" when an outgoing movement would go below zero.
- "Cannot receive more than ordered".
- Alerts are visible only to whoever reads server logs.
- Stock movement history is the only audit trail; no separate log exists.
- With `inventory.core` off, nothing in the module is permitted.

55.4 Data left behind

`purchase_orders`, `purchase_order_items`, `stock_movements`, `inventory_items` (stock, expiry, cost), `patient_consumptions` and `patient_consumption_items` (API only), and server log lines from the alert job. No billing, ledger or notification rows are written by any step.

Part 7 — Cross-cutting concerns

Chapter 56 — Authentication and sessions

This chapter explains how a person proves who they are to the Platform and stays signed in: registering a clinic, signing in, choosing an organization and location, the short-lived access token and the long-lived session cookie, managing sessions, password flows, rate limiting and the staff-login administration screen. It is written for the product owner, who needs to know exactly how long a sign-in lasts, what a user can revoke and which protections exist today. What a signed-in person may then do is a different question, answered by Chapter 11 (roles and permissions) and Chapter 57 (access control in depth).

56.1 The model in one paragraph

Status: [Implemented]

A **user** is a login: an email address, a password hash and an account status (`active` , `suspended` or `deleted`). A user is not the same as an **employee**, who is the person record that the Employees module keeps (Chapter 37). A **staff login** links one user to one employee. Signing in creates a **session** row in the database and gives the browser two things. The first is an **access token**, valid for 15 minutes, which the web app keeps in memory only and sends as a Bearer header. The second is a **session cookie** named `refresh_token` , which is `httpOnly` (scripts cannot read it) and carries the session id and a long random secret. When the access token expires, the web app calls `POST /auth/refresh` ; the cookie proves the session is still valid and a new access token is issued. Nothing about the login is stored in the browser's local storage.

Technical references: `backend/src/auth/auth.service.ts` · `AuthService` · `backend/src/session/session.service.ts` · `SessionService` · `backend/src/auth/jwt.service.ts` · `web-app/src/store/useAuthStore.ts`

56.2 Screens

The sign-in related screens are public pages (`/login` , `/register` , `/forgot-password` , `/reset-password` , `/verify-email`), the location selector (`/select-location`), the signed-in pages for sessions and security (`/sessions` and the Security and Sessions sections of Settings), and the staff-login administration page (`/users`).

Screenshot pending

Login page

`C:/Program Files/Git/login`

Figure 56.1 — Login page (screenshot pending).

The login page shows the clinic's own name and logo when the browser knows which clinic it last used (Chapter 58 explains the public theme). It has an email field, a password field with a show and hide button, a "Keep me signed in for 30 days" check box, a "Forgot password?" link, a sign-in button and a link to the sign-up page. The sign-up page asks for clinic name, the owner's name, email, an optional phone number and a password, with a five-segment strength meter (Weak, Fair, Good, Strong, Very strong) that is advice only and does not block the form.

Screenshot pending

Location selector

`C:/Program Files/Git/select-location`

Figure 56.2 — Location selector (screenshot pending).

The selector is a two-step screen. Step one lists the organizations the user can enter; choosing one with a single location selects that location and continues at once. Step two lists the organization's locations as cards; the user picks one and continues. If an organization has no location, the screen says that a location must be created to continue.

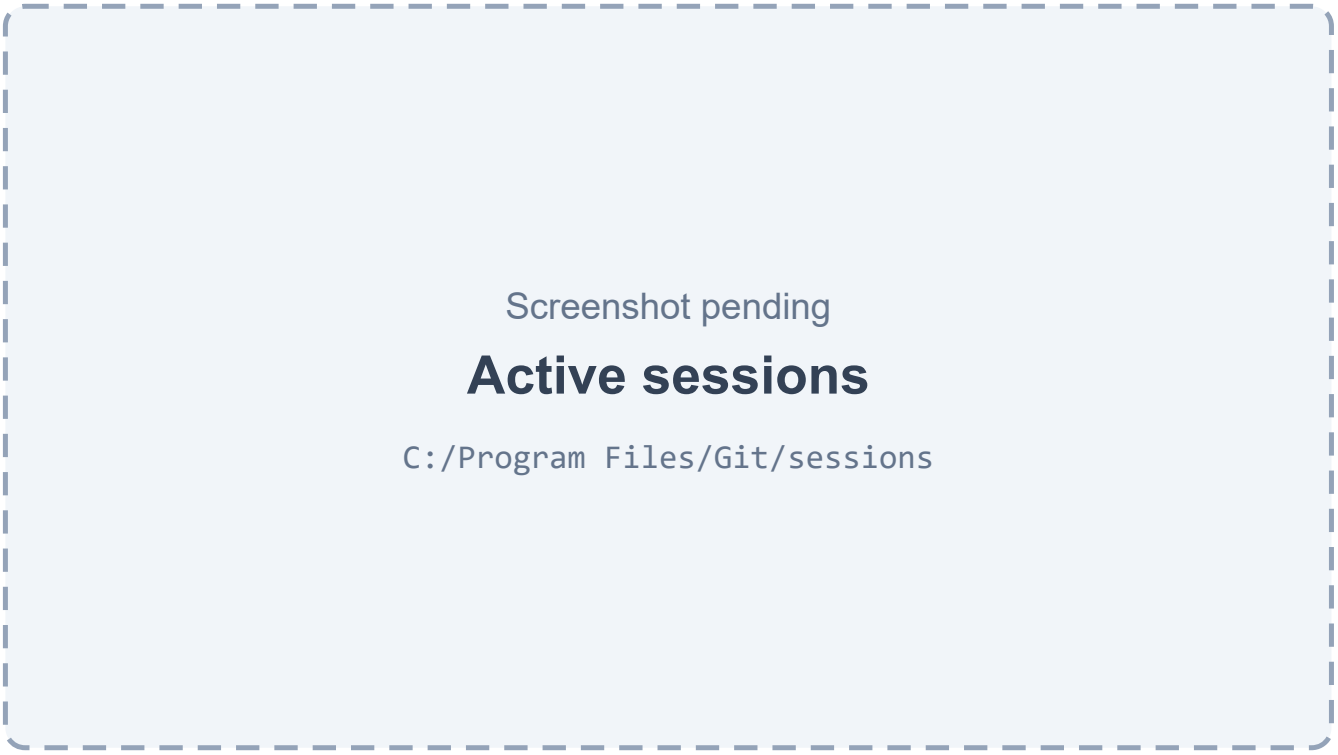

Figure 56.3 — Active sessions (screenshot pending).

The sessions list (page title "Active Sessions", also available as the Sessions section of Settings) shows one row per active session with browser, operating system, device, IP address, the time it started and the time it was last active. The current session is marked. Every other row has a **Revoke** button, the current row has **Sign out**, and a **Sign out all others** button sits at the top.

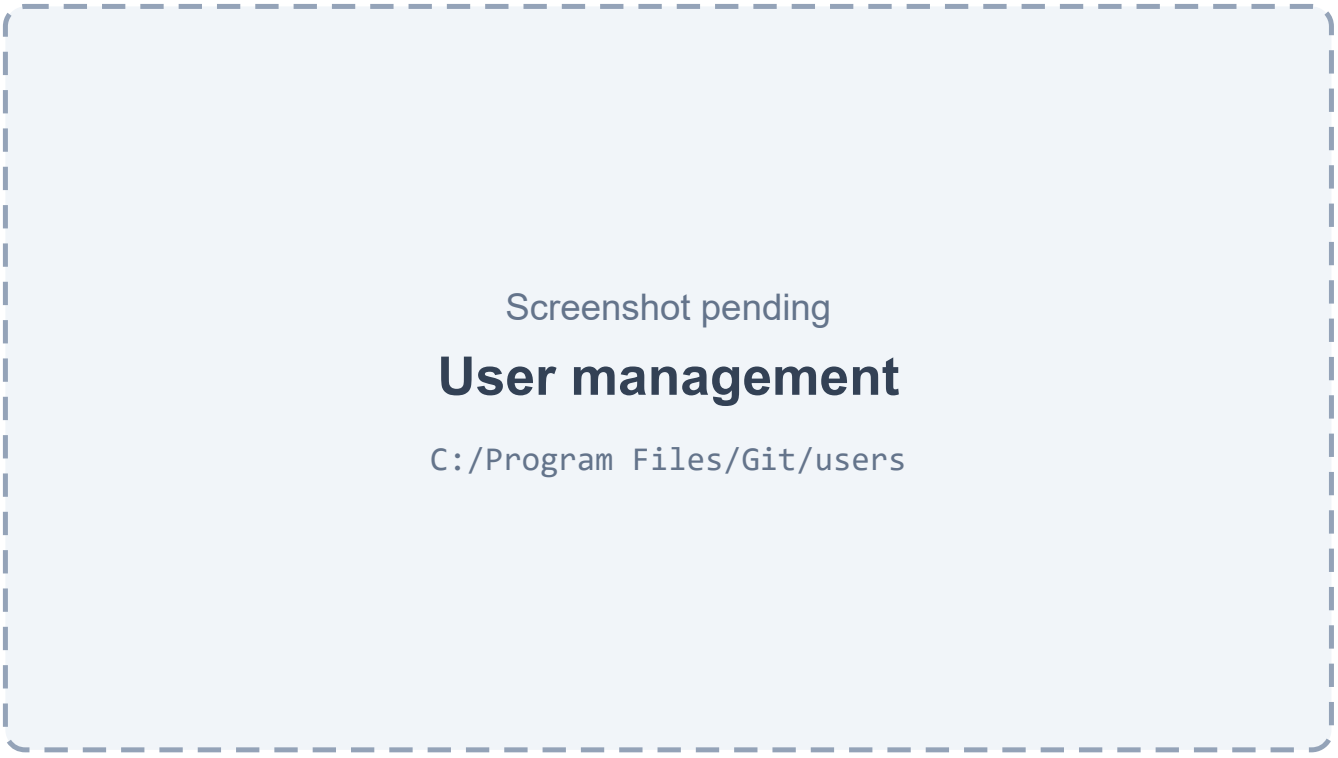

Figure 56.4 — User management (screenshot pending).

The user administration page (`/users` , title "User Management", subtitle "Manage system access across roles and organizations") lists users by type and has a **Create User** button. The create form asks for name, email, password, role type (Super Admin, Owner or Staff), and, for a staff user, which employee the login is for. Each row can be edited (including its status) or deleted.

56.3 Features

56.3.1 Register a clinic (self-signup)

Status: [Implemented]

What it does. Lets a new clinic owner create the login and the clinic in one step. The Platform creates the user, a new organization with a main location and a 14-day trial (Chapter 45 covers subscriptions and onboarding), sends a verification email and signs the owner straight in.

How the user does it.

1. Open the sign-up page and enter clinic name, name, email, optional phone and a password.
2. Submit. The owner lands in the application, kept signed in for 30 days.
3. Later, click the link in the verification email.

Behind the scenes — technically. `POST /auth/register` is public and rate limited. `AuthService.register` creates the user, calls the clinic set-up service to create the organization, and sends the verification email. If clinic creation fails, the new user row is deleted so no empty login remains. It then runs the normal login flow with "keep me signed in" on.

Rules and validations.

- Email must be a valid address; name and clinic name are required (at most 255 characters); phone at most 20 characters; password at least 6 characters.
- An email that already exists is refused.

- The verification email failing to send is logged but does not stop registration.

Who can do it. Anyone; the route is public.

Data read and written. `users`, `organizations`, `locations`, owner link, subscription rows, `user_sessions`, `security_logs`.

Error cases. 429 when more than 5 sign-ups come from one address in an hour; 400 for invalid fields or a duplicate email.

Technical references: `backend/src/auth/auth.controller.ts` · `register` · `backend/src/subscriptions/clinic-setup.service.ts` · `web-app/src/features/auth/components/RegisterForm.tsx`

56.3.2 Verify an email address

Status: [Implemented]

What it does. Confirms that the owner controls the email address. Verification is informational: an unverified user can still sign in and work. The web app shows a "verify your email" banner until verification is done.

How the user does it. Click the link in the email, which opens `/verify-email?token=…`. The page shows "Verifying your email…" and then a success or failure message. A signed-in user whose address is still unverified can ask for a fresh link.

Behind the scenes — technically. The email carries a random 32-byte token; the database stores only its SHA-256 hash, valid for 48 hours. `POST /auth/verify-email` (public, 10 attempts per 15 minutes) hashes the supplied token, finds an unexpired match and sets `email_verified_at`. `POST /auth/resend-verification` (signed in, 3 per hour) issues a new token; if the user has no pending token it answers "Email already verified". Sending depends on SMTP settings in the server environment (`SMTP_USER`, `SMTP_PASS`).

Rules and validations. An invalid or expired token gives 400 "This verification link is invalid or has expired". Users created by an administrator through user administration are not sent a verification email.

Who can do it. Anyone holding the token; resend needs a signed-in user.

Data read and written. `users` (`email_verification_token`, `email_verification_expires`, `email_verified_at`).

Technical references: `AuthService.verifyEmail` · `AuthService.resendVerification` · `backend/src/common/services/mail.service.ts` · `web-app/src/features/account/pages/VerifyEmailPage.tsx`

56.3.3 Sign in

Status: [Implemented]

What it does. Checks the email and password, creates a session and returns the information the web app needs to continue: the user, an access token, the organizations and locations the user may enter, and flags saying whether an organization or location still has to be chosen.

How the user does it. Enter email and password, optionally tick "Keep me signed in for 30 days", and click sign in.

Behind the scenes — in simple terms. The server looks up the user, compares the password against the stored hash, refuses suspended or deleted accounts, records a new session, and works out where the person is allowed to go. The session lasts one day, or 30 days if the box was ticked. Only five sessions can be active at once; a sixth sign-in quietly revokes the oldest.

Behind the scenes — technically. `POST /auth/login` (public; 5 attempts per 15 minutes per address and route). `UserService.validateUser` compares with bcrypt (12 rounds). `SessionService.createSession` stores a bcrypt hash of a 64-byte random refresh secret, the user agent, parsed browser, operating system and device, and the client IP. `AuthorizationService.resolveUserAccess` returns the accessible organizations. The response also lists any lab memberships, so a user who works only for a lab is sent to the lab portal (Chapter 41). The cookie is `httpOnly`, `sameSite=lax`, `secure` in production, with a maximum age equal to the session's remaining lifetime.

Rules and validations.

- Email valid; password at least 6 characters.
- A wrong email or password gives a plain 401 with no hint about which was wrong.
- A suspended account gets 403 "Your account has been suspended. Please contact support."; a deleted account gets 403 "This account has been deleted."
- A session's expiry is fixed at sign-in and is never extended by refreshing.

Who can do it. Anyone with an account.

Data read and written. `users`, `user_sessions`, `employee_org_location_assignments`, owner links, `organizations`, `locations`, `security_logs` (a success row).

Effects on other modules. The login result seeds the web app's auth store, which every later request reads for its tenant headers (Chapter 8).

Error cases. 401 wrong credentials; 403 suspended or deleted; 429 too many attempts (message "Too many requests. Please try again later.", with a `retryAfter` value in seconds).

Technical references: `AuthService.login` · `SessionService.createSession` · `AuthorizationService.resolveUserAccess` · `web-app/src/features/auth/components/LoginForm.tsx`

56.3.4 Choose and switch organization and location

Status: [Implemented]

What it does. Decides which clinic and branch a person is working in. Every later request carries that choice in two headers.

How the user does it. After sign-in the selector appears when a choice is needed. Later, the location switcher in the page header changes the active location.

Behind the scenes — in simple terms. A single organization with a single location is chosen automatically. A super admin always picks an organization first. Otherwise the person picks, and the Platform remembers the last choice for next time.

Behind the scenes — technically. The login response sets `requiresOrgSelection` and `requiresLocationSelection`. `POST /auth/select-location` and `POST /auth/switch-location` (both signed in, context free) validate access with `AuthorizationService.validateAccess` and save `last_accessed_organization_id` and `last_accessed_location_id` on the user. Switching also returns the legacy permission flags for the new location (deprecated; the web app reads `GET /me/access`). Chapter 8 describes the choice flow and what each type of user may enter in full.

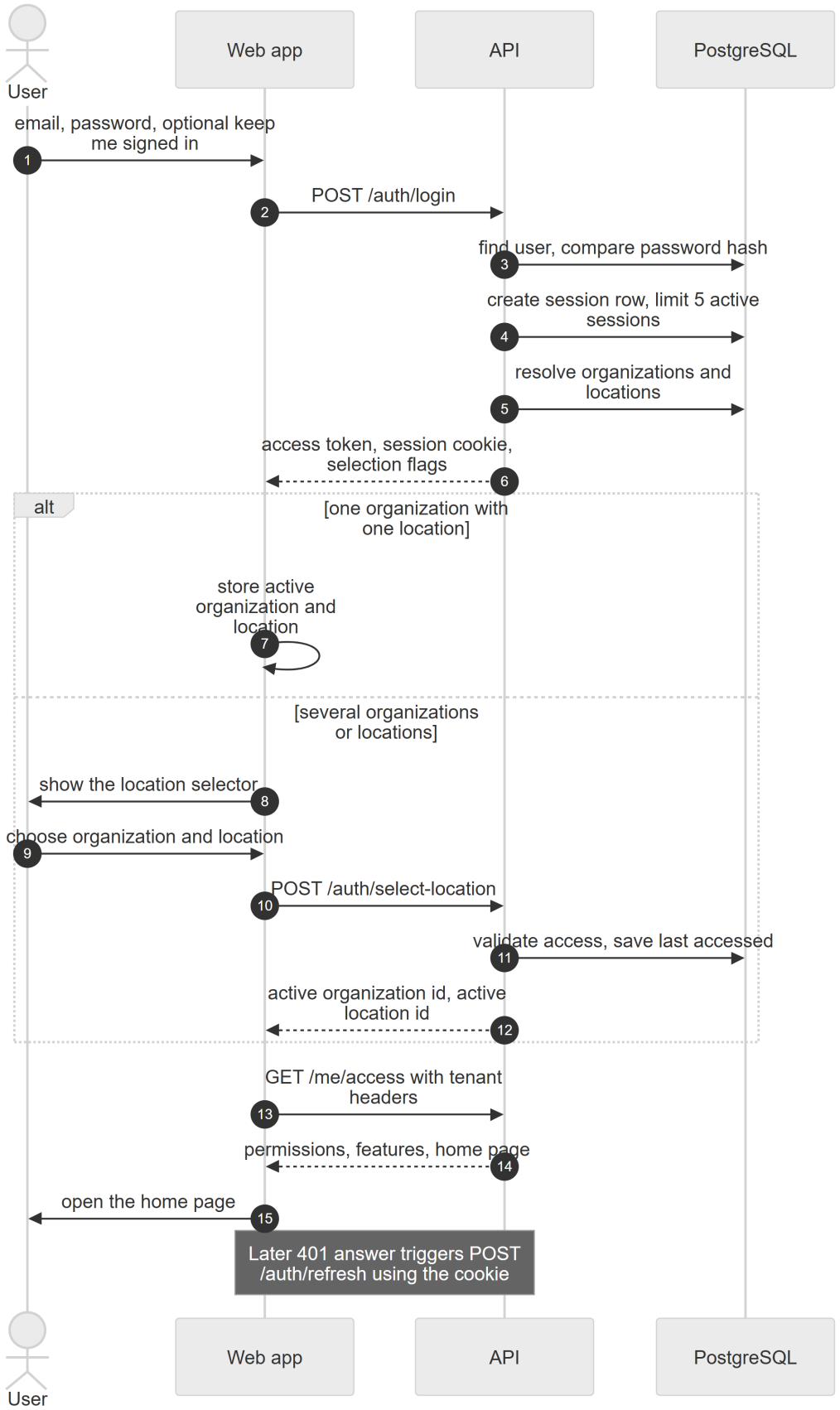

Figure 56.5 — Sequence of a sign-in, the optional location choice and the first access fetch.

The diagram has three participants besides the User: the Web app, the API and PostgreSQL. (1) The user enters email, password and the optional keep-me-signed-in choice. (2) The web app calls `POST /auth/login`. (3) The API finds the user and compares the password hash. (4) It creates the session row and enforces the limit of five active sessions. (5) It resolves the organizations and locations. (6) It answers with the access token, the session cookie and the selection flags. An "alt" box then shows two paths. If the user has one organization with one

location, the web app stores them as active. Otherwise the web app shows the selector, the user chooses, the web app calls `POST /auth/select-location`, the API validates access and saves the last accessed choice, and returns the two active ids. After the box, the web app calls `GET /me/access` with the tenant headers and receives permissions, features and the home page, and opens the home page. A note says that a later 401 answer triggers `POST /auth/refresh` with the cookie.

Rules and validations. Both organization id and location id are required UUIDs. A request for a location the user may not enter gets 403 "You do not have access to this location".

Who can do it. Any signed-in user.

Data read and written. `users.last_accessed_*`.

Error cases. 403 no access; the web app shows the toast "Failed to select location".

Technical references: `AuthService.selectLocation` · `AuthService.switchLocation` · `web-app/src/features/auth/components/LocationSelector.tsx` · `web-app/src/components/layout/LocationSwitcher.tsx` · `web-app/src/routes/__root.tsx`

56.3.5 Stay signed in: access token, refresh cookie and silent refresh

Status: [Implemented]

What it does. Keeps a person signed in across page reloads and across the 15-minute life of the access token, without storing credentials in the browser.

Behind the scenes — in simple terms. The access token is a short pass kept only in the page's memory. When the page is reloaded the memory is empty, so the web app quietly asks the server for a new pass using the cookie. When a pass expires in the middle of work, the same quiet request happens and the failed request is repeated.

Behind the scenes — technically.

- The access token is a JWT signed with the server's access secret, expiring in 15 minutes, holding the user id, email, global role and session id. There is no JWT refresh token; the refresh secret is a random value whose bcrypt hash lives on the session row.
- On every app start the root route calls `POST /auth/refresh` once ("silent refresh"). On success the auth store is filled; on failure the user is treated as signed out.
- `POST /auth/refresh` (public) reads the cookie, validates the session (not revoked, not expired, secret matches), re-checks the account status, **rotates** the refresh secret (the old one stops working) and returns a new access token and the full login payload. The cookie's expiry is the original session expiry, not a fresh window.
- When any API call returns 401, the axios interceptor refreshes and retries once (Chapter 61 shows the sequence).
- The auth store is memory only; it never uses local storage.

Rules and validations. A missing, revoked or expired session gives 403 "Invalid or expired session" and the cookie is cleared. A session whose user turned suspended or deleted is revoked on the spot.

Who can do it. Any browser holding a valid cookie.

Data read and written. `user_sessions` (`refresh_token_hash`, `last_active_at`).

Technical references: `AuthService.refreshTokens` · `SessionService.rotateRefreshToken` · `JwtTokenService` · `backend/src/session/session-lifetime.ts` · `web-app/src/lib/api.ts`

56.3.6 Sign out, list and revoke sessions

Status: [Implemented]

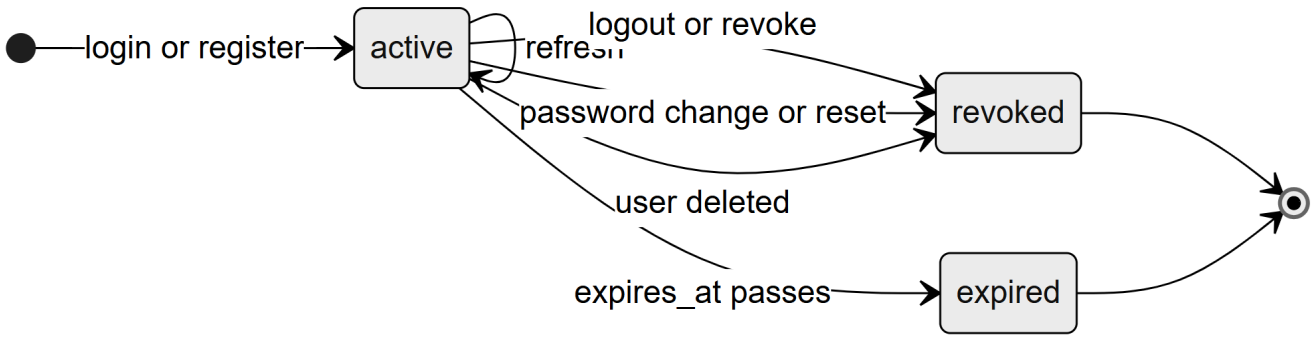

Figure 56.6 — Status lifecycle of a session.

The diagram shows a session starting at a dot and entering **active** when the user logs in or registers. A loop on active is labelled refresh: a refresh rotates the secret but keeps the same session and the same expiry. Active moves to **revoked** in three ways: the user logs out or revokes a session (single, all others or all), the user changes or resets the password, or the user is deleted. Active moves to **expired** when `expires_at` passes, which is one day or 30 days after sign-in. Revoked and expired are both final states. (A sixth sign-in also revokes the oldest session; the diagram leaves that case out to stay readable.)

What it does. Lets a user end the current session, see every active session with device details, and end any of them.

How the user does it. Open Settings, Sessions (or `/sessions`). Click **Revoke** on a row, **Sign out** on the current row, or **Sign out all others**. The toasts are "Session revoked", "Signed out successfully" and "All other sessions revoked".

Behind the scenes — technically.

Action	Endpoint	Notes
Sign out this session	`POST /auth/logout`	Public; reads the cookie, revokes that session, clears the cookie, writes a security log row
List active sessions	`GET /auth/sessions`	Newest activity first; marks the current one
Revoke one session	`DELETE /auth/sessions/:id`	Only the caller's own sessions; otherwise 404
Revoke all others	`POST /auth/sessions/revoke-all`	Keeps the current session
Sign out everywhere	`POST /auth/logout-all`	Revokes all sessions including the current one, clears the cookie

The table lists the five session endpoints. Revoking is a flag (`is_revoked`) on the row; rows are kept. There is no clean-up job for old session rows.

Rules and validations. A revoked session cannot refresh. An access token that was already issued stays valid until it expires, at most 15 minutes, because requests are checked by the token only.

Who can do it. Any signed-in user, on their own sessions.

Data read and written. `user_sessions`, `security_logs`.

Error cases. 404 "Session not found" for an id that is not the caller's.

Technical references: `backend/src/auth/auth.controller.ts` · `SessionService` · `web-app/src/features/auth/components/SessionManagement.tsx` · `web-app/src/routes/sessions.tsx`

56.3.7 Passwords: change, forgot and reset

Status: [Implemented] for change, forgot and reset. **Add password** is [Partial]: the endpoint exists but no screen calls it, because it is meant for accounts created through the disabled OAuth sign-in.

What it does. Lets a user change a known password, or recover from a forgotten one through an emailed link.

How the user does it.

- *Change:* Settings, Security, enter the current and new password. All other sessions are revoked, as the page says.
- *Forgot:* on the login page click "Forgot password?", enter the email and click "Send reset link". The page always says to check the inbox.
- *Reset:* open the link in the email, enter a new password.

Behind the scenes — technically.

- `POST /auth/change-password` (signed in) checks the current password with bcrypt, hashes the new one (12 rounds), and by default revokes every other session, keeping the current one.
- `POST /auth/forgot-password` (public, 3 per hour) always answers "If the email exists, a reset link will be sent", whether or not the address exists. For a real user it stores the SHA-256 hash of a random token, valid for 15 minutes, and emails the link.
- `PATCH /auth/reset-password` (public) hashes the supplied token, finds an unexpired match, sets the new password, clears the token and revokes **all** sessions of that user.
- `POST /auth/add-password` (signed in) sets a password only when the account has none; otherwise 400 "Account already has a password".
- All three paths write security log rows (change, reset requested, reset succeeded).

Rules and validations. Password rules are in section 56.5. A wrong current password gives 401 "Current password is incorrect". An invalid or expired reset token gives 400 "Invalid or expired token". Reset has no rate limit of its own; the token is single use and short lived.

Who can do it. Anyone for forgot and reset; signed-in users for change.

Data read and written. `users` (`password`, `password_reset_token`, `password_reset_expires`), `user_sessions`, `security_logs`.

Technical references: `AuthService.changePassword` · `AuthService.forgotPassword` · `AuthService.resetPassword` · `AuthService.addPassword` · `web-app/src/features/auth/components/ForgotPasswordForm.tsx` · `ResetPasswordForm.tsx` · `web-app/src/features/settings/pages/SettingsPage.tsx`

56.3.8 Rate limiting

Status: [Implemented], with a limit: the counters live in the memory of one API process.

What it does. Slows down guessing and abuse on the public and sensitive authentication routes.

Behind the scenes — technically. `RateLimiterGuard` counts requests per client address and route in a fixed window and answers 429 with `retryAfter` when the count is exceeded. The address comes from the first value of the forwarding header when present, otherwise the connection address.

Endpoint	Allowed	Window
`POST /auth/login`	5	15 minutes
`POST /auth/register`	5	1 hour
`POST /auth/verify-email`	10	15 minutes
`POST /auth/resend-verification`	3	1 hour
`POST /auth/forgot-password`	3	1 hour

The table lists the five throttled endpoints with their allowed attempts per window. Other routes, including refresh, reset-password and all clinical routes, have no limiter. A separate small throttle (120 requests per minute per address by default) protects the public lab magic-link routes (Chapter 41). Because the counters are in process memory, they reset when the process restarts and are not shared between API processes, so with several API processes the effective limit is higher. The code notes that a shared store such as Redis would fix this.

Who can do it. Not applicable; it applies to everyone.

Technical references: `backend/src/auth/guards/rate-limiter.guard.ts` · `RateLimiterGuard` · `RateLimit` · `backend/src/lab-access/magic-links/magic-link-throttle.guard.ts`

56.3.9 Security log

Status: [Partial]

What it does. Keeps a record of authentication events in the table `security_logs`: user, event type, IP address, user agent, session, success flag and a small metadata object.

What is written today. Successful sign-in, sign-out, sign-out everywhere, password change, password reset requested, password reset succeeded, and a failed sign-in only for suspended or deleted accounts. The event list in the code also names email verification, session, token, OAuth, account and suspicious-activity events, but nothing writes them yet. A wrong password does not produce a row.

What is missing. No endpoint or screen reads the security log, and no job deletes old rows, so it is a write-only record that can be queried in the database only.

Technical references: `backend/src/auth/security-log.service.ts` · `SecurityLogService` · `backend/src/auth/enum/SecurityEventType.ts` · `backend/src/auth/entities/SecurityLog.ts`

56.3.10 User administration and staff logins

Status: [Implemented]

What it does. Lets a super admin, an owner, or a user with the permission `admin.users.manage` create, edit, suspend and delete logins. The most common case is giving an existing employee a login.

How the user does it.

1. Open **User Admin** (`/users`) and click **Create User**.
2. Choose the type. For Staff, pick the employee from the list of staff who have no login yet.
3. Enter name, email and a password of at least 8 characters, and save.

Behind the scenes — technically.

Action	Endpoint
Profile of the caller	`GET /user/me`
List users by type (`super_admin` , `owner` , `employee`), paged	`GET /user`
Staff without a login	`GET /user/staff-candidates`
Create a user	`POST /user`
Edit a user (including status and password)	`PATCH /user/:id`
Delete a user	`DELETE /user/:id`

The table lists the user endpoints. The controller is context free and checks access itself: a super admin may act with or without an organization; anyone else must send an organization header and hold `admin.users.manage` at their location.

A **staff login** (type `employee`) runs in one transaction that locks the employee's active assignments in the organization. It refuses an employee who already has a login (409 "This employee already has a login") and an email already in use. It creates the user, then links the user to all of the employee's active assignments and switches portal access on for them. The login grants no permission by itself: what the person can do comes from the role of each assignment (Chapter 11). Changing assignments bumps the organization's access version, so the change reaches open tabs (Chapter 57).

Rules and validations.

- Only a super admin can create or edit a super admin. Only an owner can add, edit or delete another owner.
- A non-super-admin manager may touch only users of their own organization, never a super admin, and cannot elevate a user to super admin.
- Creating a user checks the plan's login limit for the organization (error code `PLAN_LIMIT`); super admins may exceed it.
- Delete is a soft delete: the status becomes `deleted` , a deletion time is set and every session of the user is revoked.
- Passwords created here need 8 characters, which is stricter than the 6 required elsewhere.

Who can do it. Super admin, owner, or `admin.users.manage` (high-risk; the Admin system role holds it).

Data read and written. `users` , `organization_ownership` , `employee_org_location_assignments` .

Error cases. 403 for the manager-scope rules in this list; 409 existing login; 400 duplicate email.

Technical references: `backend/src/user/user.controller.ts` · `UserService.createUser` · `createStaffLogin` · `staffCandidates` · `web-app/src/features/users/` · `web-app/src/routes/users.tsx`

56.3.11 Sign-in with Google or GitHub

Status: [Not implemented]

The code for Google and GitHub sign-in exists but is **disabled**: neither the controller nor the two strategies are registered in the authentication module, so the `/auth/google` and `/auth/github` routes do not exist. The user table has a `provider` field and an `oauth_providers` table, and the service has an OAuth login method, but none of it can be reached. To switch it on, a developer would register the three classes, read callback addresses from the environment and add the routes to the public route list. The Platform therefore supports email and password only.

Technical references: `backend/src/auth/oauth.controller.ts` · `backend/src/auth/auth.module.ts` · `AuthService.handleOAuthLogin`

56.3.12 The first super admin

Status: [Partial]

There is no screen, command or seed that creates a super admin. The usual path is to register an account and then set `users.global_role` to `super_admin` directly in the database (Chapter 7, section 7.5, gives the steps). After that, other super admins can be created from User Admin. A super admin signs in, chooses any active organization, and may open the setup pages (Organizations, Locations, Lab Directory, Config Studio, Subscriptions) even before choosing a location.

Technical references: `backend/src/common/enums/global-role.enum.ts` · `web-app/src/routes/__root.tsx` · `SUPER_ADMIN_SETUP_PATHS`

56.4 Entities

The tables involved are `users`, `user_sessions`, `security_logs` and the unused `oauth_providers`; Chapter 13 describes them column by column. In short: each session belongs to exactly one user and is deleted if the user row is deleted; a security log row points to a user but is kept (the link is set to null) if the user is removed; a user is linked to organizations as an owner through the ownership table and to employees through assignments.

56.5 Password rules

Status: [Implemented]

Where	Minimum length
Register, login, change password, reset password	6 characters
Create user in User Admin	8 characters

The table shows the lengths the API enforces. There is no rule about letters, digits or symbols, no list of banned passwords and no expiry. The sign-up and reset pages show a strength meter, but it only advises. Passwords are stored as bcrypt hashes with 12 rounds and are never returned by the API.

56.6 Limitations and known gaps

- Revoking a session does not cancel an access token that was already issued; it works until it expires, at most 15 minutes.
- Rate-limit counters are per process and lost on restart.
- The security log has no reader and no clean-up; failed password attempts are not recorded.
- No two-factor authentication, no password strength enforcement and no lockout of an account after repeated failures.
- Session rows are never deleted, only flagged.

56.7 Owner's FAQ

How long does a sign-in last? One day by default and 30 days when "Keep me signed in for 30 days" is ticked. The length is fixed at sign-in; using the application does not extend it.

If an employee leaves, how do I cut off access at once? Set the user's status to suspended or delete the user in User Admin. Deleting revokes every session; a suspended user is refused at the next refresh. An access token already issued may still work for up to 15 minutes.

Can one person have several logins on different devices? Yes, up to five active sessions. A sixth sign-in revokes the oldest.

What happens if someone forgets their password? They use "Forgot password?". The link lasts 15 minutes and a successful reset signs the user out of every device.

Can staff sign in with Google? No. That feature is built but disabled.

Where do I see failed sign-in attempts? Nowhere in the application today. Only the throttle protects the login route.

Chapter 57 — Access control in depth

This chapter follows the Platform's access control from the first guard that sees a request to the last button hidden in the browser. It is written for the product owner, who needs to know where a permission is checked, how quickly a change reaches users and which protections do not exist. Chapter 11 explains the permission catalog, the system roles and the roles screen; this chapter does not repeat them and goes deeper into the mechanics. Customization screens (features, roles, sidebar, theme) are in Chapter 58.

57.1 The idea in one minute

Status: [Implemented]

Access control answers four separate questions, in this order: Is this a real, signed-in user? Is this user allowed to work in this organization and location? Is the feature switched on for this clinic? Does the user's role hold the permission this action needs? A fifth check asks whether the clinic's subscription still allows writing. Each question is a global guard that runs before the controller. After the guards, two more mechanisms narrow what a person sees inside a permitted action: **row scope** (only own, location or organization rows) and **field hiding** (prices removed from a response). The browser repeats the same questions only to decide what to show; the server never trusts the browser.

Technical references: `backend/src/app.module.ts` · `backend/src/access/` · `web-app/src/features/access/`

57.2 The five guards in order

Status: [Implemented]

Five global guards run on every HTTP request, in a fixed order, and the first refusal stops the request (Chapter 5, section 5.4, lists them with their skip rules).

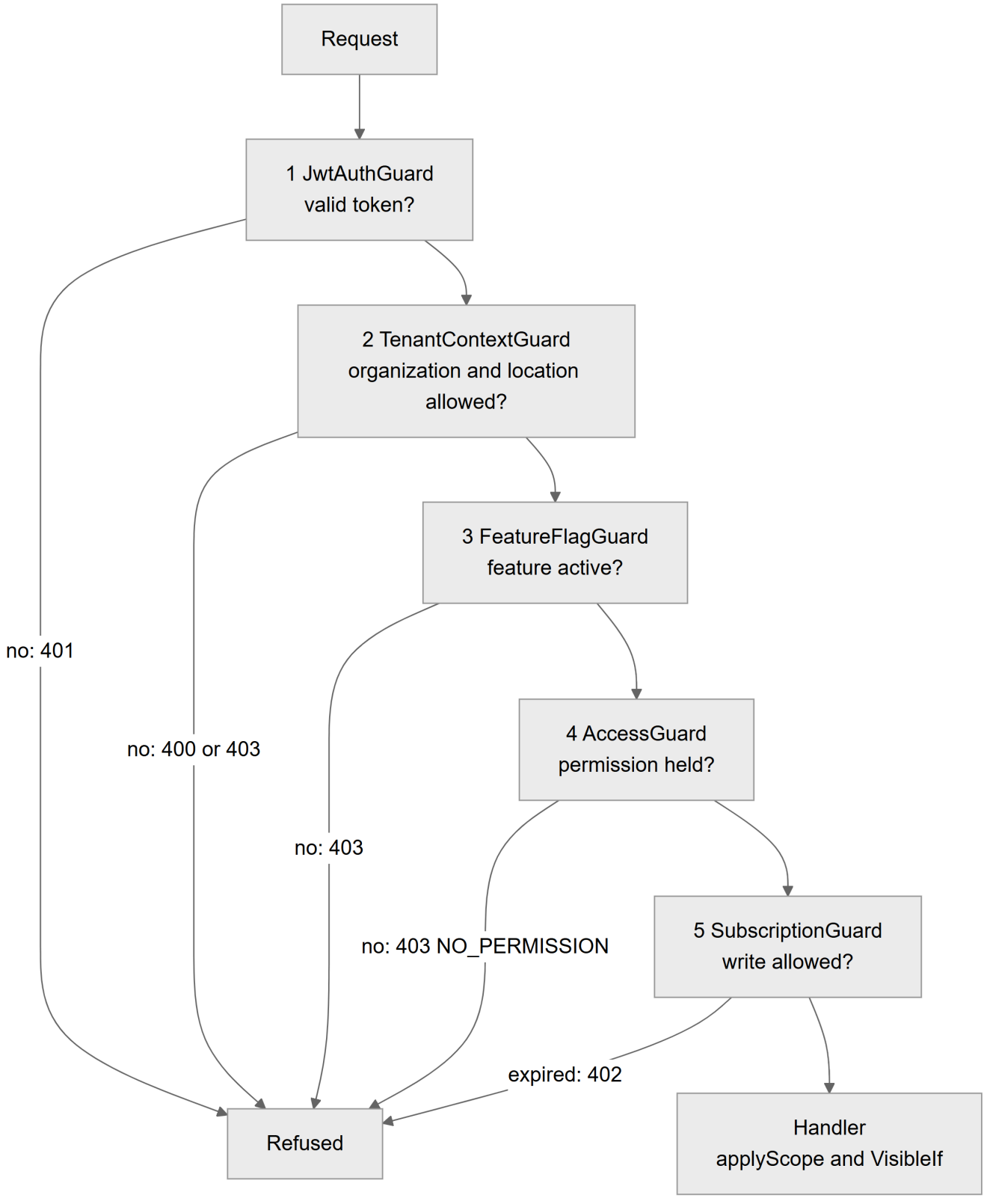

Figure 57.1 — The five global guards in the order they run, with the refusal each can give.

The flowchart reads from top to bottom. A Request enters guard 1, `JwtAuthGuard`, which asks whether the token is valid; if not, the answer is 401 and the request is refused. Guard 2, `TenantContextGuard`, asks whether the organization and location in the headers are allowed for this user; if not, the answer is 400 (headers missing) or 403. Guard 3, `FeatureFlagGuard`, asks whether the required feature is active; if not, 403. Guard 4, `AccessGuard`, asks whether the permission is held; if not, 403 with the code `NO_PERMISSION`. Guard 5, `SubscriptionGuard`, asks whether a write is allowed; if the subscription has expired the answer is 402. A request that passes all five reaches the handler, where row scope and field hiding apply. Every refusal leads to the same "Refused" box.

The details that matter to an owner:

- **Guard 1 and public routes.** `JwtAuthGuard` skips only the routes in a fixed public list (sign-in, register, refresh, password routes, email verification, logout, the public theme, the waiting-room display state and the lab magic links). Marking a handler `@Public()` is not enough on its own; the route must also be in that list.
- **Guard 2 and super admins.** A super admin passes without headers. `@RequireSuperAdmin()` routes are checked here: anyone else gets 403 "Super admin access required". `@ContextFree()` routes skip the tenant check, and the handler then checks access itself. The guard has no path-prefix exceptions. A user passes for an organization if they own it, or if they have an active assignment at that exact location with portal access on.
- **Guard 3 only runs for tagged routes.** `@RequireFeature('key')` is needed only on routes that use `@AnyMember()`, because a permission already implies its feature (the access context drops permissions of inactive features).
- **Guard 4 attaches the access context.** Whenever the request names an organization, it puts the resolved access context on the request for the handler, even on routes with no permission marker.
- **Guard 5 is read-only mode.** It refuses writes with 402 `SUBSCRIPTION_EXPIRED` for a clinic whose subscription has lapsed. Reads, super admins and the paths for signing in, the account, `/me/` and subscription administration stay open.

Technical references: `backend/src/auth/guards/jwt.guard.ts` · `backend/src/auth/guards/tenant-context.guard.ts` · `backend/src/feature-flags/guards/feature-flag.guard.ts` · `backend/src/access/access.guard.ts` · `backend/src/subscriptions/subscription.guard.ts`

57.3 The decorators

Status: [Implemented]

Decorator	Where	What it means	Enforced by
`@Can('a', 'b')`	handler or controller	Every listed permission is required; class-level and handler-level lists are merged	`AccessGuard`
`@AnyMember()`	handler or controller	Any active member of the location (owners and super admins too)	`AccessGuard`
`@Public()`	handler or controller	No sign-in required (plus the public route list)	all guards skip
`@ContextFree()`	handler or controller	No tenant headers needed; the handler checks access itself	`TenantContextGuard` skips
`@RequireSuperAdmin()`	handler or controller	Platform operator only	`TenantContextGuard`
`@RequireFeature('key')`	handler or controller	The feature must be active; class and handler lists are merged	`FeatureFlagGuard`
`@Access()`	handler parameter	Injects the `AccessContext` of the request	decorator
`@VisibleIf('perm')`	property of a response class	The field is removed from the response unless the caller holds the permission	global interceptor

The table lists the eight decorators with their placement and meaning. The first five say who may call a handler. `@RequireFeature` adds a feature condition. `@Access()` and `@VisibleIf` are tools for handlers and response classes.

The coverage test. The test `access-coverage.spec.ts` loads every controller and fails if any handler lacks one of the markers `@Can`, `@AnyMember`, `@Public`, `@RequireSuperAdmin`, `@ContextFree` or an explicit `@UseGuards(...)`, or if any handler still uses the removed role decorator. Controllers under the path prefixes for authentication, sessions and health, and the hello endpoint, are exempt. This test is the safety net, because at runtime a handler with no marker passes the access guard.

Technical references: `backend/src/access/decorators/access.decorators.ts` · `backend/src/common/decorators/` · `backend/src/access/access-coverage.spec.ts`

57.4 The access context

Status: [Implemented]

For each request that names an organization, `AccessResolver` builds an `AccessContext`: user id, organization, location, access version, super-admin flag, owner flag, member flag, employee id, active features, granted permissions (feature-filtered, with implied permissions expanded), the role and its home page. Owners and super admins receive every permission at its widest scope. Staff receive the permissions of the role on their assignment at the active location, and only if portal access is on for that assignment. A role that is inactive or belongs to another organization gives nothing. A missing key means no access.

The context offers a small set of questions that code uses instead of role names:

- `can(permission)` and `canAll([...])`: is it granted.
- `scope(permission)`: `own`, `location` or `org`, or nothing. An unscoped grant reads as `org`.
- `applyScope(queryBuilder, permission, columns)`: adds the row filter to a database query. With `own` it adds a condition on the caller's employee id (or a custom condition). With `location` it adds a condition on the active location. With `org` it adds nothing, because the organization filter is the caller's. With no grant, or when the caller has no employee record or no location to match, it adds a condition that matches no rows. If a developer forgets to name the column, it throws an error rather than silently showing nothing.
- `decide(permission, { value, min, max, approvalFlow })`: returns allow, deny or "needs approval". Inside the range the answer is allow; outside it, allow only when the caller holds the escalation permission (today: preset discounts are escalated by the custom-discount permission), otherwise needs approval when the clinic runs an approval flow, else deny.

Field hiding. A response class can mark a property `@VisibleIf('patients.price.view')`. A global interceptor removes that property from every instance in the response when the caller lacks the permission, and removes every guarded field when there is no access context. It works only on class instances, not on plain object literals.

Technical references: `backend/src/access/access.resolver.ts` · `backend/src/access/access-context.ts` · `AccessContext` · `backend/src/access/visible-if/visible-if.ts` · `backend/src/patients/access/patient-scope.ts`

57.5 What the web app receives: `GET /me/access`

Status: [Implemented]

`GET /me/access` is a context-free route that checks membership itself. It needs the organization header and, for staff, the location header; without a location, staff roles give no permissions because roles are per location. It answers 400 without an organization, 403 when the caller has no access there. The answer has these parts: the access version; the organization and location ids; the super-admin and owner flags; the list of active feature keys;

the granted permissions with their scope (or `true`); the roles (id, name and template key); the employee id; the home page; and the clinic's sidebar layout, which is null while the clinic uses the default. `GET /access/catalog` (needs `admin.roles.manage`) returns the catalog, bundles and role templates for the roles screen.

Technical references: `backend/src/access/access.controller.ts` · `MeAccessController` · `AccessCatalogController`

57.6 In the browser

Status: [Implemented]

- **`AccessProvider`** loads `/me/access` for the active organization and location. The data is fresh for 30 seconds and refetched when the window regains focus, when the realtime socket announces `access.changed` and after a reconnect.
- **`useAccess()`** returns `can(permission)`, which takes one permission or a list and means any-of; `canAll(list)` for all-of; `scope(permission)`; feature checks; the owner and super-admin flags; the home page and the sidebar layout. The `<Can perm="...">` component wraps content the same way. When no payload exists yet for a super admin on a setup page, a fallback allows everything; a loaded payload is always the source of truth.
- **Route gate.** Each route file declares `staticData: { can: ... }`. One component, `RouteAccessGate`, mounted in the root route (`__root.tsx`), reads the requirement of every matched route and shows the No access page when any requirement fails. While access is loading it shows a skeleton; if loading fails it shows "Couldn't load your access" with a retry button. Routes without a requirement pass.
- **Sidebar.** The sidebar comes from a developer-owned registry of 39 items. Each item declares its own requirement: a permission (any-of), `superAdminOnly` (six items: Organizations, Subscriptions, Lab Directory, Form schemas, Workflows, Stage templates), `labMember` (the Lab portal item) and `ownerOrSuperAdmin` (the flag exists but no item uses it today). The clinic's saved layout only arranges, renames and hides these items; items a user may not open are removed before display. Registry items that the layout forgot appear in a trailing "More" section so a new page never disappears silently.
- **Home page.** After sign-in, the role's home page is used, then the clinic's home page, and the page is opened only if the user passes the same sidebar and route checks; otherwise the Dashboard opens.

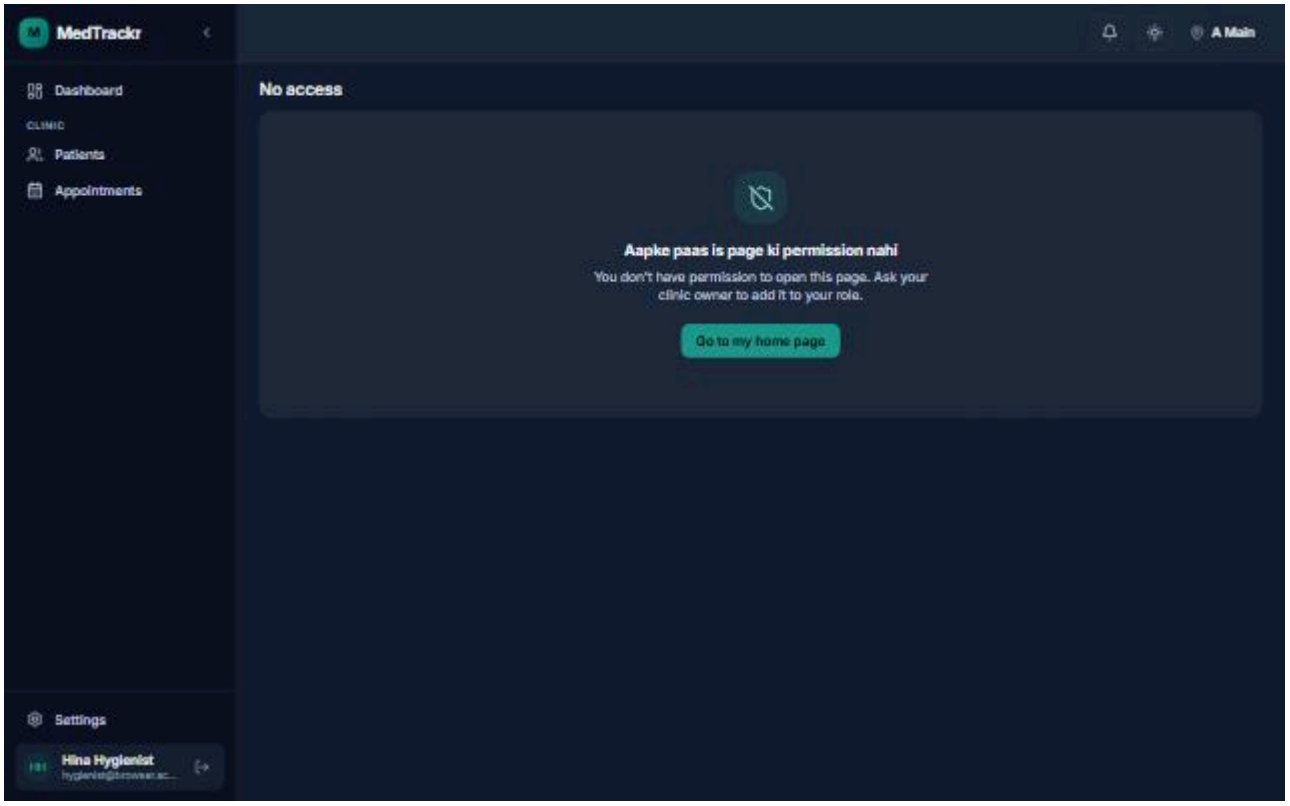

Figure 57.2 — The No access page shown when a role lacks the permission of the page.

The screenshot was taken on 2026-10-23 in dark theme and shows the former working name of the product in the corner; the product is now called the Platform and each clinic sets its own name. A test user in a custom role opened a page their role does not allow. The sidebar is short: Dashboard, and under "Clinic" only Patients and Appointments, with Settings and the user at the bottom. The main area is titled "No access" and shows a shield icon, a Roman-Urdu headline meaning that the user has no permission for this page, an English sentence saying the user does not have permission and should ask the clinic owner to add the page to their role, and a button, "Go to my home page". The user can click the button to go to the role's home page, or to the Dashboard if the role has none.

Technical references: `web-app/src/features/access/components/AccessProvider.tsx` · `RouteAccessGate.tsx` · `NoAccess.tsx` · `web-app/src/features/access/lib/access.ts` · `web-app/src/features/access/nav/nav.registry.ts` · `resolveNav.ts` · `homeRoute.ts` · `web-app/src/routes/__root.tsx`

57.7 How fast a change reaches users

Status: [Implemented]

Each organization has a counter, `organizations.access_version`. It is raised by one after any committed change that can alter someone's access: employment roles (any column), assignments (role, location, organization, active flag, portal access or linked login), organization feature flags, ownership, location feature overrides and sidebar layout changes. An entity subscriber does this automatically for writes made through the data layer, once per transaction after the commit and never on rollback. Code that uses raw SQL or database cascades must bump explicitly; the employee delete path does.

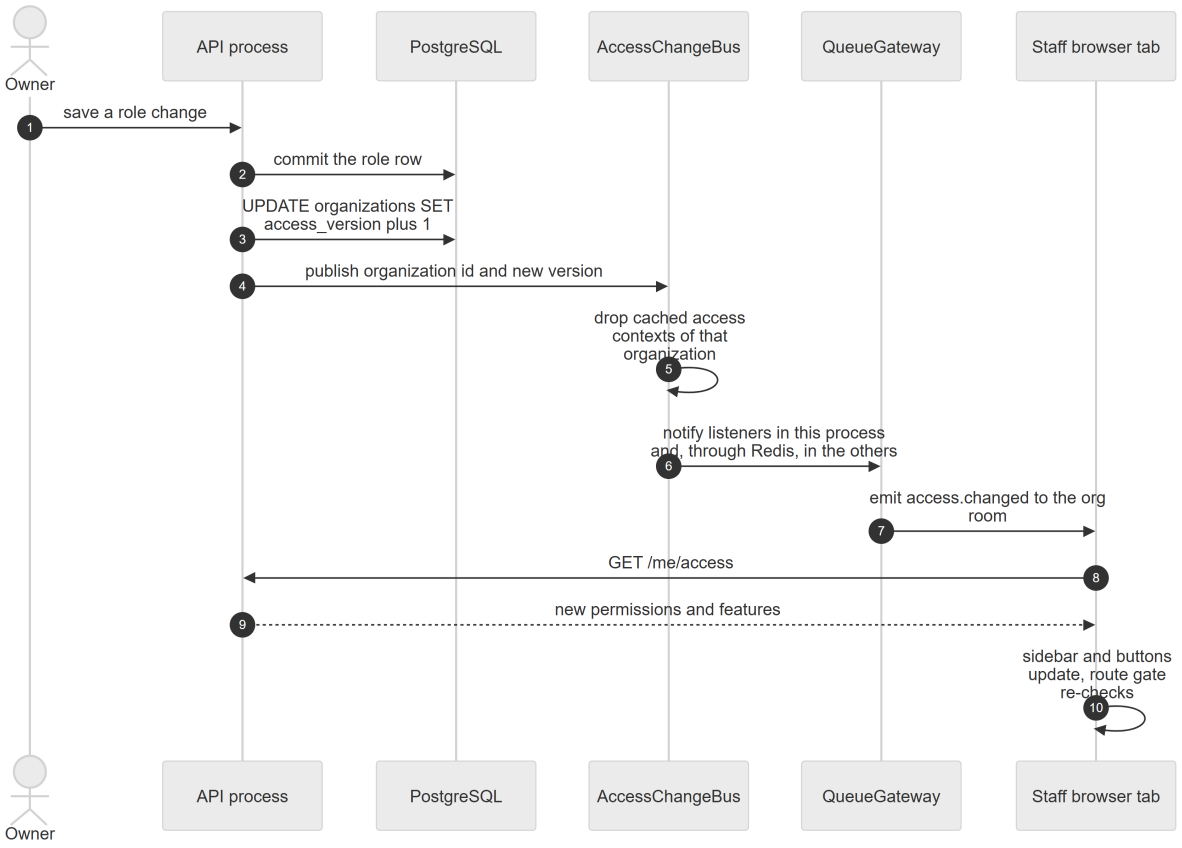

Figure 57.3 — Sequence from an owner's role change to the refreshed browser tabs.

The diagram has six participants: the Owner, the API process, PostgreSQL, the `AccessChangeBus`, the `QueueGateway` and a Staff browser tab. (1) The owner saves a role change. (2) The API commits the role row. (3) The API increments `access_version` of the organization. (4) The API publishes the organization id and new version to the bus. (5) The bus drops cached access contexts of that organization. (6) It notifies listeners in this process and, through Redis, in the other processes. (7) The gateway emits `access.changed` to the organization's room. (8) The browser tab calls `GET /me/access`. (9) The API answers with new permissions and features. (10) The sidebar and buttons update and the route gate checks again.

The timing rules are:

- **Version cache.** Each process caches the current version of an organization for 15 seconds. The bus updates it at once in every process that hears the change.
- **Context cache.** Access contexts are cached in process memory, at most 10,000, for at most 5 minutes, keyed by user, organization, location and version. A new version means a new key, so stale entries are never used.
- **Redis.** One Redis channel carries access-change messages between API and worker processes. If Redis is down, only the process that made the change learns at once; the others catch up within 15 seconds, when their cached version expires.
- **Feature cache.** The enabled-feature set is cached for a short time and dropped by the same bus.
- **Browser.** Open tabs refetch when told, on focus, and in any case after 30 seconds of staleness.

In practice, an owner's change reaches other staff within seconds, and within 15 seconds even when Redis is unavailable. A person who is already inside a permitted request is not interrupted.

Technical references: `backend/src/access/version/access-version.service.ts` · `AccessVersionService` · `access-version.subscriber.ts` · `access-change-bus.ts` · `web-app/src/features/access/hooks/useAccessSocket.ts`

57.8 The lab portal is a separate door

Status: [Implemented]

Lab staff do not use the clinic guards for their screens. Every lab portal controller uses its own `LabPortalGuard` after the global guards (which skip these routes as context free). It rejects any request carrying clinic headers (400 `LAB_PORTAL_CLINIC_CONTEXT`), so lab and clinic scopes cannot mix. It requires an active membership of an active lab; the lab is named by an `x-lab-id` header, or is the user's only lab (several labs without a header gives 400 `LAB_SELECTION_REQUIRED`). Another lab's id looks exactly like a lab that does not exist (403 `LAB_ACCESS_DENIED`). A lab role list on the handler (lab admin or technician) is checked against the membership. Super admins hold no lab membership and are refused. Chapter 41 describes the portal itself.

Technical references: `backend/src/labs/portal/lab-portal.guard.ts` · `LabPortalGuard` · `backend/src/labs/portal/lab-memberships.ts`

57.9 Row-level security does not exist

Status: [Not implemented]

The database has no row-level security policies. Tenant isolation is enforced by the application: handlers filter every query by the organization (and often the location) taken from the validated tenant context, and by `applyScope` for own, location and organization reach. A bug in a query that forgets the organization filter would therefore not be stopped by the database. The safeguards are the coverage test, the single `@ActiveTenant()` source of the organization, and code review. Chapter 8 describes the tenant model.

57.10 Limitations and known gaps

- Tenant isolation is application-level only (section 57.9).
- A handler without a marker passes the access guard at runtime; only the coverage test prevents it.
- Contexts and version numbers are cached in process memory; with Redis down, other processes may lag up to 15 seconds.
- An access token already issued is not re-checked against the session; permissions, however, are checked on every request from the access context, so a role change takes effect without a new token.
- The `ownerOrSuperAdmin` sidebar flag is unused today.

57.11 Owner's FAQ

I removed a permission from a role. When does it stop working? Within seconds for open screens (the browser is told over the realtime connection) and on the next request on the server. At worst 15 seconds if the message bus is down.

A user opens a page by typing its address. Can they bypass the sidebar? No. The route gate shows No access and the API refuses the data calls with 403 `NO_PERMISSION`. Hiding a sidebar item is cosmetic; the checks are the permission on the route and on the API.

Does the sidebar layout give anyone access? Never. The layout only arranges pages the registry knows; every item still needs the viewer's permission.

Can the database stop one clinic seeing another's data? Not by itself. The application filters every query by organization; there are no database row-level policies.

Who can see which patients? The scope of `patients.view`: a doctor sees own patients, front desk sees the location, an admin sees the organization (Chapter 11).

What does 402 mean? The clinic's subscription expired; the clinic is read-only until a payment is recorded (Chapter 45).

Chapter 58 — Clinic customization

This chapter describes how a clinic shapes the Platform to its own way of working: which features are on, which roles exist and what they may do, how the sidebar is arranged, how the application looks, and where each role starts. It is written for the product owner, who needs to know what an owner can change without a developer and what only the platform team can change. Four setup screens carry this work: Features, Roles and permissions, Sidebar and Branding and theme. The permission model behind them is in Chapters 10 and 11 and the access mechanics are in Chapter 57; this chapter restates what is needed and describes the screens.

58.1 The four setup screens

Status: [Implemented]

Screen	Route	Sidebar label (section "Clinic setup")	Permission to open
Features	`/admin/features`	Features	`admin.features.manage`
Roles	`/admin/roles`	Roles & permissions	`admin.roles.manage`
Sidebar builder	`/admin/navigation`	Sidebar	`admin.nav.manage`
Branding and theme	`/admin/theme`	Theme	`admin.theme.manage`

The table lists the four screens, their routes, the labels in the sidebar and the permission each route needs. The owner always has all four permissions. The Admin system role holds them by default. Every save on these screens increases the organization's access version or invalidates the settings cache, so changes reach other users quickly (Chapter 57, section 57.7).

Technical references: `web-app/src/routes/admin.features.tsx` · `admin.roles.index.tsx` · `admin.navigation.tsx` · `admin.theme.tsx` · `web-app/src/features/access/nav/nav.registry.ts`

58.2 Features screen

Status: [Implemented]

What it does. Lets the owner switch the features in the clinic's plan on or off for the whole clinic, or switch one off at a single location. A feature is only available if the platform team has **entitled** the clinic to it; the owner then decides whether it is **enabled**; and a **location override** can switch it off at one branch. Chapter 10 describes the three switches and the catalog in full.

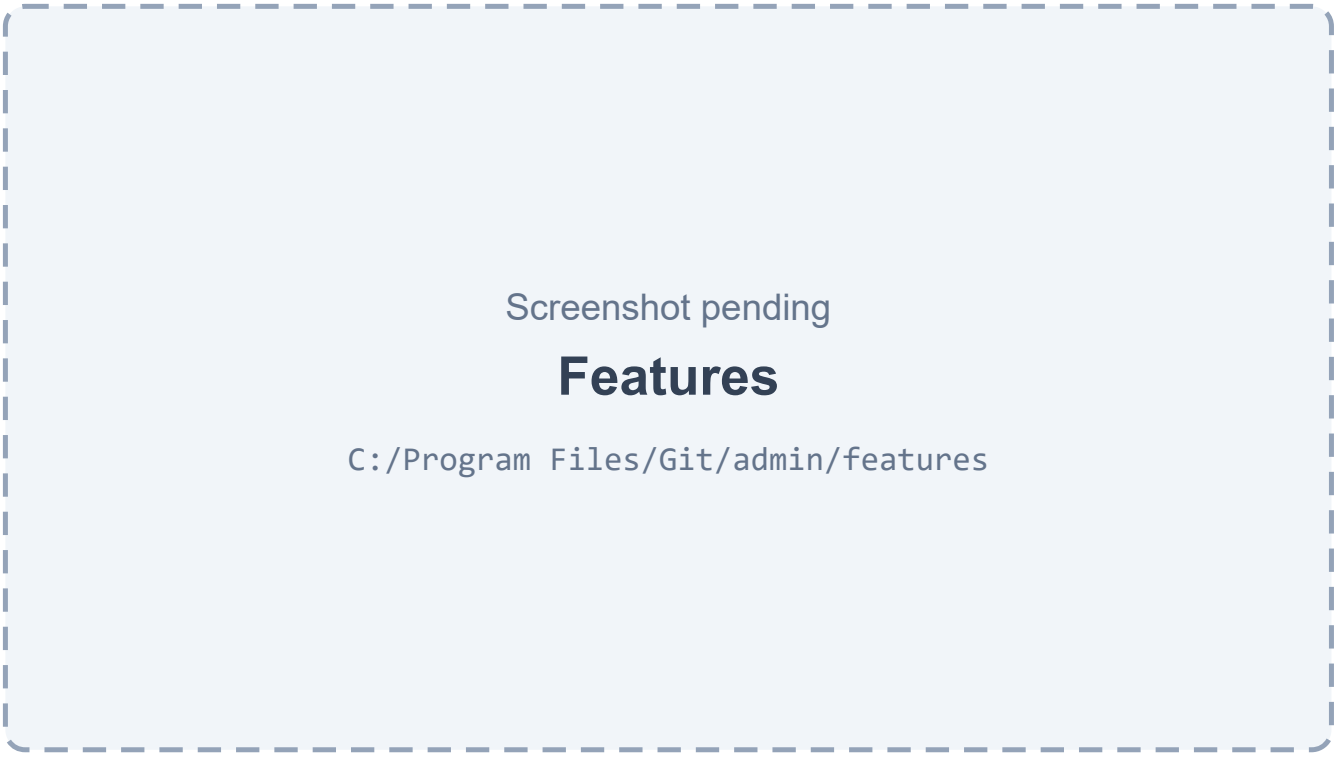

Figure 58.1 — Features screen (screenshot pending).

The page is titled "Features" with the subtitle "Turn the features in your plan on or off for the clinic, or for one location." Features are listed in rows, each with a name, a switch and a button that opens the locations sheet. A filter row narrows the list. Features the clinic is not entitled to cannot be switched on. If there are no features the page says "No features yet" and that the platform has not set up any features for the clinic.

How the user does it.

1. Open **Features** in the sidebar.
2. Move a switch to turn a feature on or off. Turning a feature off first opens a dialog titled "Turn off ?" that lists the permissions the feature carries and the roles that will lose them. This is the **impact preview**.
3. Click **Turn off** to confirm.
4. To switch a feature off for one branch only, open the locations sheet and move the switch of that location.

Behind the scenes — technically.

Action	Endpoint
List features for the clinic	`GET /admin/features`
Impact of turning one off	`GET /admin/features/:key/impact`
Switch on or off	`PATCH /admin/features/:key`
Switch off or on at one location	`PUT /admin/features/:key/locations/:locationId`

The table lists the four endpoints; all need `admin.features.manage`. The impact call returns the catalog permissions the feature carries and, for each active role of the organization that grants any of them, the permissions it would lose. Turning on a feature the clinic is not entitled to gives 403 `NOT_ENTITLED`. Every change writes an `audit_logs` row; a super admin acting inside a clinic is recorded as the platform.

Technical references: `backend/src/feature-flags/admin-features.controller.ts` · `FeatureAdminService.getImpact` · `web-app/src/features/feature-settings/`

58.3 Roles builder

Status: [Implemented]

What it does. Lets the owner (or anyone with `admin.roles.manage`) create custom roles, copy a system role, choose a level for each module, set a home page and see members. Chapter 11, section 11.5, describes the rules in detail: escalation protection, reason for high-risk grants, read-only system roles, unique names and the ban on deleting a role that has members.

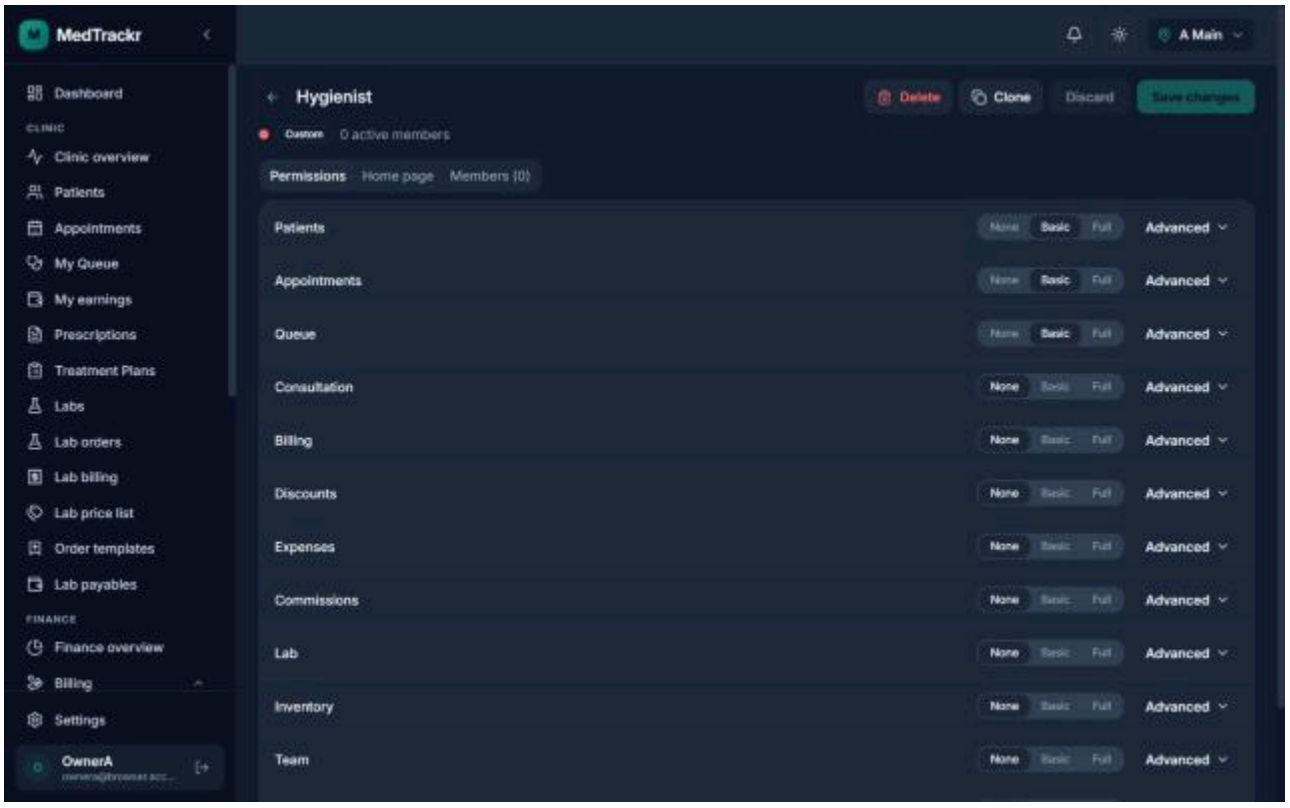

Figure 58.2 — The role editor for a custom role, showing None, Basic and Full per module.

The screenshot was taken on 2026-10-23 in dark theme and shows the former working name of the product in the corner. It shows a custom role open in the editor. The header holds the role name and the actions Delete, Clone, Discard and Save changes. Three tabs sit under it: Permissions, Home page and Members. On the Permissions tab each module has a row with three level buttons, None, Basic and Full, and an Advanced control that opens the module's full permission list with a check box and, for scoped permissions, a scope selector. A module whose permissions match no level exactly shows Custom. The left sidebar shows the full owner menu.

What the levels mean. None grants nothing from the module. **Basic** grants an everyday set. **Full** grants every permission of the module at its widest scope. **Advanced** lets the owner tick individual permissions. **Clone** copies any role, including a system role, with the same permissions and home page, and moves nobody to the copy. **Members** lists the employees who hold the role, per location; roles are assigned on each employee's page.

Technical references: `backend/src/employees/employment-roles.admin.controller.ts` · `backend/src/access/catalog/bundles.ts` · `web-app/src/features/roles/`

58.4 Per-role home page

Status: [Implemented]

What it does. Sets the page a member of the role lands on after signing in. The five system roles have defaults (Admin: dashboard; Doctor: the doctor's queue; Receptionist and Assistant: appointments; Accountant: billing).

How the user does it. Open a custom role, choose the **Home page** tab, pick a page in the "Page" list and save. The clinic as a whole can also set a **Clinic home page** on the Sidebar screen; it is used when the role sets none.

Rules and validations. The value must be an in-app path starting with a single slash, with no address or scheme. At sign-in the web app uses the role's page, then the clinic's page, then the Dashboard; the page opens only if the user passes the permission checks for it, otherwise the Dashboard opens. A home page never widens access.

Technical references: `web-app/src/features/roles/components/HomePageTab.tsx` · `web-app/src/features/access/nav/homeRoute.ts` · `backend/src/access/nav/nav-layout.dto.ts`

58.5 Sidebar builder

Status: [Implemented]

What it does. Lets the owner rename, hide, group and reorder the pages in the sidebar for the whole clinic. Each person still sees only the pages they may open, so the layout can never give access.

Figure 58.3 — Sidebar builder (screenshot pending).

The page is titled "Sidebar" with the subtitle "Rename, hide and reorder the pages in your clinic's sidebar. Each person still sees only the pages they have access to." A line at the top says whether the clinic uses the standard sidebar or a custom one, or that there are unsaved changes. The buttons are **Discard changes** (when dirty), **Reset to default** and **Save**. A "Clinic home page" selector (default: Dashboard) comes first. The editor is a drag-and-drop area of sections; each section has a label that can be renamed, can contain pages and collapsible groups, and has a remove button. An **Add section** button adds a section. Pages that are not in any section wait in a tray called "Not placed". Each page can be renamed inline, hidden with a toggle, or dragged to another place. On the right, a live preview shows the sidebar as the owner would see it, with a note that other roles see only their pages. Leaving with unsaved changes asks "Leave without saving?".

How the user does it.

1. Open **Sidebar** under Clinic setup.
2. Drag pages between sections or the tray, rename sections and pages, hide pages, add groups.

3. Choose a clinic home page.
4. Click **Save**. To undo everything, click **Reset to default** and confirm; sections, renames, hidden pages and the clinic home page are removed.

Behind the scenes — technically. The layout is stored as one JSON document per organization in `nav_layouts`; a column for a role exists but only the clinic default is written today. Endpoints (all need `admin.nav.manage`): `GET /admin/navigation` (null while the clinic uses the default), `PUT /admin/navigation` and `DELETE /admin/navigation`. Saving and resetting bump the access version and write an audit row.

Rules and validations.

- At most 20 sections; labels at most 40 characters; at most 100 items in a list.
- Item keys must look like `nav.something`; no key may appear twice anywhere.
- A section entry is either a link (a key) or a group (a label and items, no key).
- The home route must be an in-app path.
- Footer items (Settings) stay pinned and cannot be arranged.
- Pages the layout forgets appear in a "More" section, so nothing disappears by accident.

Data read and written. `nav_layouts`, `audit_logs`.

Technical references: `backend/src/access/nav/nav-layouts.controller.ts` · `NavLayoutsService` · `nav-layout.dto.ts` · `web-app/src/features/nav-builder/` · `web-app/src/features/access/nav/resolveNav.ts`

58.6 Sidebar registry flags

Status: [Implemented]

The list of pages that can appear is owned by developers in `nav.registry.ts`: 39 items. The clinic's layout only arranges them. Each item has one requirement: a permission or list of permissions (any-of), `superAdminOnly` (Organizations, Subscriptions, Lab Directory, Form schemas, Workflows and Stage templates), `labMember` (Lab portal) or `ownerOrSuperAdmin` (defined but unused). Section 57.6 describes how the sidebar is filtered. Adding a new page needs a code change; the owner cannot create sidebar items.

58.7 Theme and branding

Status: [Implemented]

What it does. Controls the clinic's name, logos, brand colour, sidebar style and default light or dark mode, for staff screens, the login page and printouts.

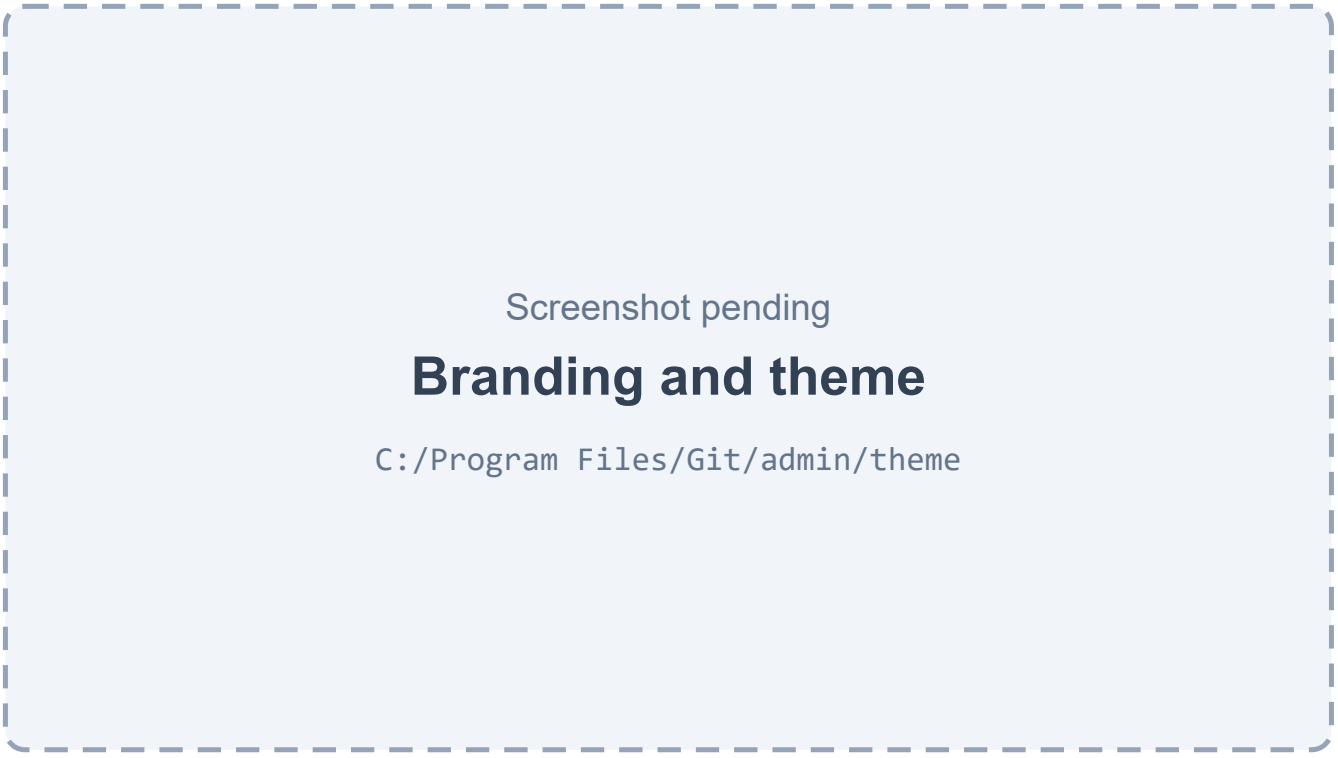

Figure 58.4 — Branding and theme (screenshot pending).

The page is titled "Branding & theme" with the subtitle "Your clinic's name, logo, colours and look — for staff screens, the login page and printouts." It has a back link to Settings and a button that opens the per-location sheet. The controls are on the left and a live preview is on the right. The controls are: Clinic name; Logo (light); Logo (dark); Browser tab icon; Brand colour; Sidebar style; Default mode. Each can carry a lock switch labelled "Locations can't override". Buttons are **Save theme** (enabled only when the form changed and is valid) and a reset button that asks "Reset to default?".

The seven keys.

Key	Meaning	Allowed values	Scopes that may set it
`theme.app_name`	Clinic name shown in the application (default "Clinic Portal")	1 to 60 characters	organization, location
`theme.logo_url`	Light-mode logo	http or https address	organization, location
`theme.logo_dark_url`	Dark-mode logo	http or https address	organization, location
`theme.favicon_url`	Browser tab icon	http or https address	organization
`theme.brand_color`	Brand colour (default blue)	`#rrggbb`	organization
`theme.sidebar_style`	Sidebar look	light, dark or brand	organization
`theme.default_mode`	Starting mode	light, dark or system	organization

The table lists the seven theme keys with their meaning, allowed values and the levels that may set them. Only the name and the two logos can differ per location. The platform level may set any key. Each key can be locked by the platform for organizations, or by the organization for its locations.

The cascade.

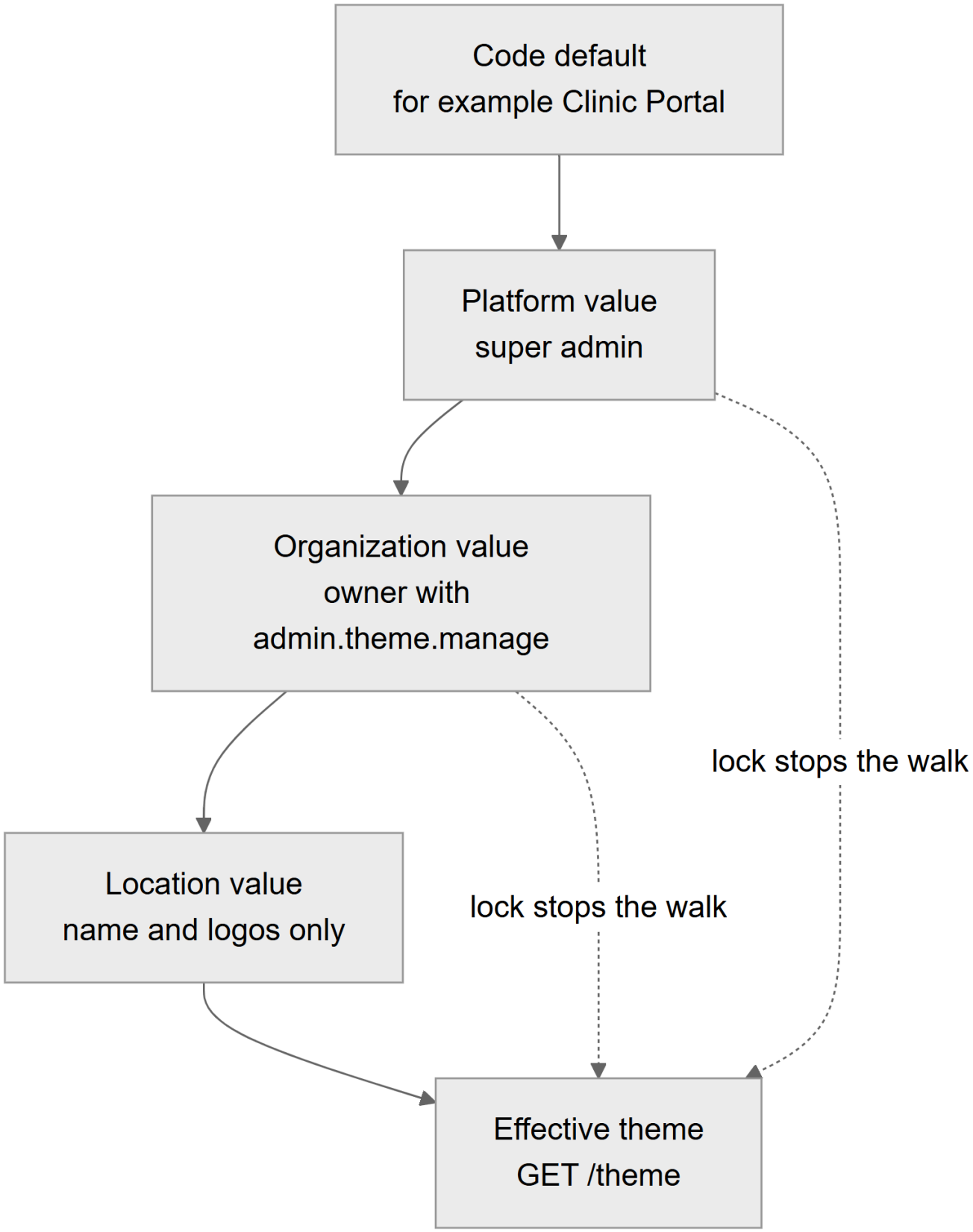

Figure 58.5 — The theme cascade from code default to the effective theme, with locks.

The flowchart reads from top to bottom. The code default (for example the name "Clinic Portal") feeds the Platform value set by the super admin. That feeds the Organization value set by an owner with `admin.theme.manage`. That feeds the Location value, which may set only the name and logos. The last box is the

effective theme returned by `GET /theme`. Two dotted arrows from the Platform and Organization boxes straight to the effective theme show that a lock stops the walk: a value locked at a higher level wins over anything set lower down.

Logos. A logo is uploaded with `POST /uploads`, which accepts one file up to 15 MB; the theme screen accepts images up to 2 MB and stores the returned address as the value.

Brand colour contrast check. When the owner picks a colour, the screen checks it against accessibility rules. It warns if the button text on the colour has less than 4.5 to 1 contrast, and warns if the colour on a white page has less than 3 to 1 (links and outlines will be hard to see, and a darker shade is used for brand text). When both pass, the screen says "Contrast OK" with the ratio. The warning is advice; it does not block saving.

Public theme for the login page. `GET /public/theme/:organizationId` (optionally with a location) is public and returns theme tokens only, nothing else about the clinic. An unknown organization gets the platform theme. The login page learns which clinic to show from an organization parameter in the address or from the last clinic used in that browser. The browser also keeps a copy of the last theme and its colours so the loading screen is on-brand before the server answers; local storage is used only for this convenience.

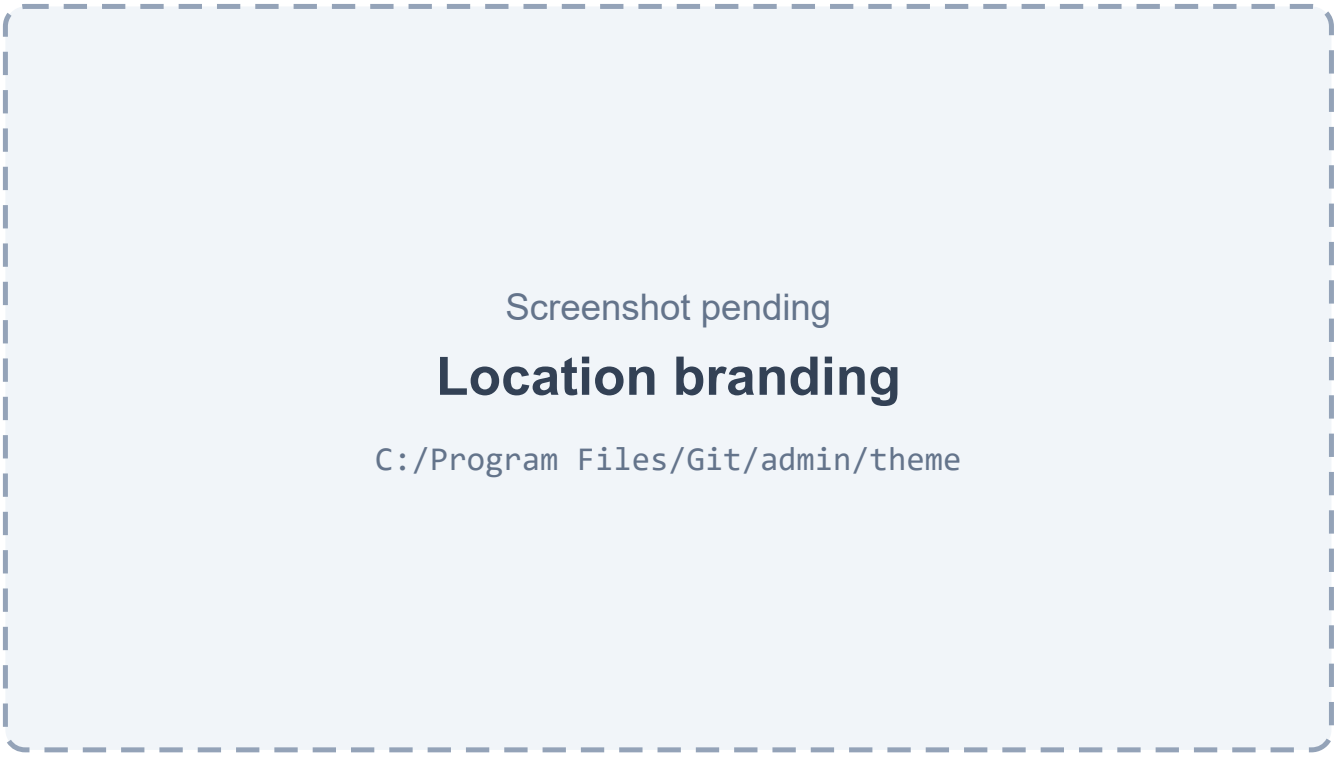

Figure 58.6 — Location branding (screenshot pending).

The sheet is titled "Location branding" with the text "A branch can show its own name and logo. Empty fields use the clinic's." It opens from the button on the theme page. A "Location" list chooses the branch, then an "Overrides" section holds "Name at this location", "Logo (light)" and "Logo (dark)". Saving applies the override; empty fields inherit.

Endpoints.

Action	Endpoint	Permission
Resolved theme for the signed-in clinic	`GET /theme`	any member
Organization view and save	`GET` and `PUT /admin/theme`	`admin.theme.manage`
Location view and save	`GET` and `PUT /admin/theme/location/:locationId`	`admin.theme.manage`
Platform view and save	`GET` and `PUT /platform/theme`	super admin

Action	Endpoint	Permission
Public theme	`GET /public/theme/:organizationId`	none

The table lists the endpoints. A write sends values by short key (a null or empty text clears the value, which inherits) and optional locks (organization and platform only; a location cannot lock). Values are validated by the registry's schema for each key. A write to a key locked higher up fails with 403 `SETTING_LOCKED`. The write takes a row lock, increases the row's version, writes an audit row and invalidates the settings cache in every process through Redis; without Redis the other processes catch up within the 60-second cache time.

Data read and written. `config_values` (one row per scope, scope id and key, with the value, a lock flag, a version and the editor), `audit_logs`.

Technical references: `backend/src/settings-registry/theme/theme.controller.ts` · `ThemeService` · `theme.settings.ts` · `backend/src/settings-registry/settings-registry.service.ts` · `SettingsRegistryService` · `web-app/src/features/theme/` · `web-app/src/features/theme/lib/color.ts`

58.8 Settings registry

Status: [Partial]

The settings registry (`config_values`) is the general store for new settings with the platform, organization and location cascade and locks. Today only the seven theme keys are defined in it. Older settings live in their own typed tables, for example the queue policies and the location billing settings, and keep working unchanged. The code does not move them into the registry.

58.9 Limitations and known gaps

- The sidebar layout is per clinic only; per-role layouts are not written yet (the column exists).
- The owner cannot add new sidebar pages or permissions; both are fixed in code.
- Only the name and logos can be customized per location.
- The brand colour check is advice, not a rule.
- Fonts, spacing and corner radius cannot be customized.

58.10 Owner's FAQ

Who can switch a feature on that my plan does not include? Only the platform team, by entitling it. The owner can only enable features the clinic is entitled to.

If I hide a page in the sidebar, can people still open it? Yes, by address, if their role allows it. Hiding is cosmetic; use roles to remove access.

Can each branch have its own logo? Yes. Each branch can have its own name and light and dark logos; the colour and mode are clinic wide.

Why can I not change the colour? The platform team may have locked that key for clinics. The screen marks such a key as locked at a higher level.

Where do I set what a role sees first? On the role's Home page tab. The clinic-wide fallback is on the Sidebar screen.

What happens to people when I turn a feature off? They lose the permissions that belong to the feature at once, and the dialog before you confirm lists the roles affected.

Chapter 59 — Notification mechanics

This chapter explains how a notification travels inside the Platform, from the business event that causes it to the bell icon where a staff member reads it. It is written for the product owner who wants to understand the pipeline and its guarantees. The Notifications module chapter (Chapter 44) describes the settings screen, the event catalog and what users see; this chapter covers only the mechanics. Today the Platform delivers notifications **in the application only**: WhatsApp, SMS and email are prepared in the code but have no real sender.

59.1 The pipeline

Status: [Implemented] for in-app delivery. [Not implemented] for WhatsApp, SMS and email.

A **notification event** is a named thing that happens, such as `order.submitted` or `item.ready` (21 are seeded, all about lab orders, shipments, stages, appointments linked to labs and payments). When a workflow step runs the `notify` effect, the Platform decides who should be told, whether each person wants it, what the message says and when it may be sent. The result is one row per person and channel in a table called `notification_outbox`. A separate loop in the worker process sends those rows later. This is the **transactional outbox** pattern: the notification row is written in the same database transaction as the business change, so it exists only if the change was committed.

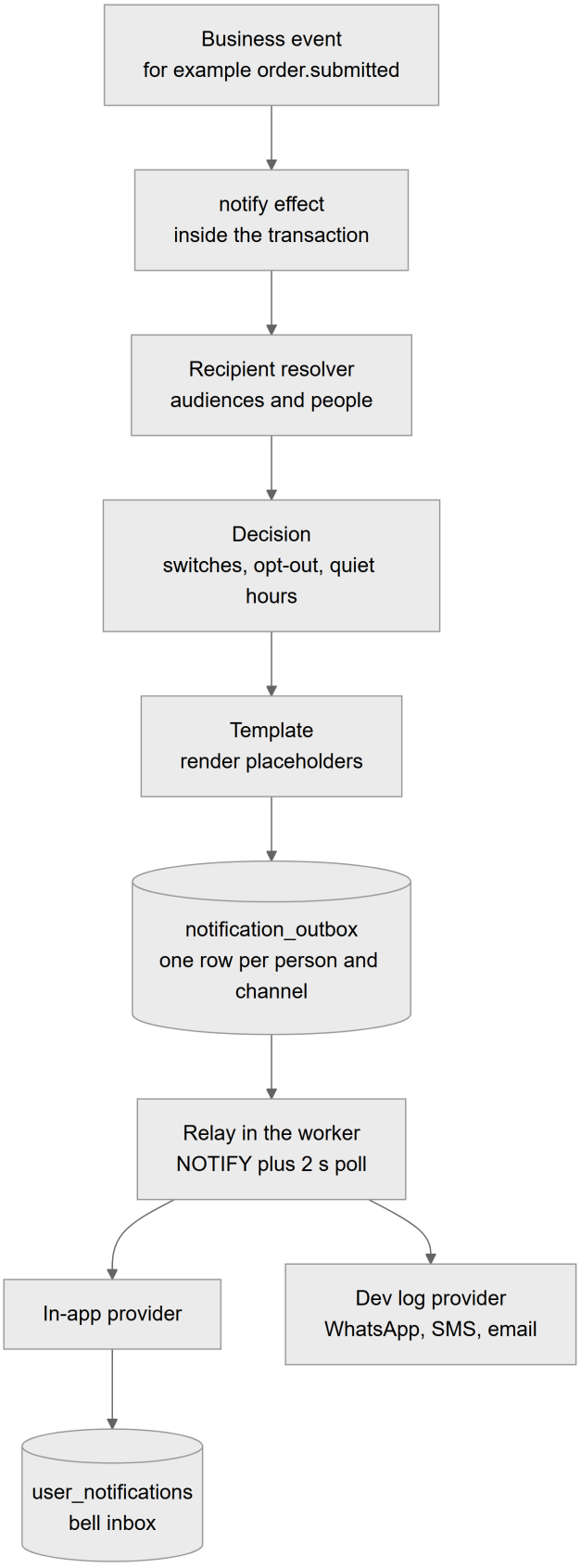

Figure 59.1 — The path of a notification from the business event to the inbox.

The flowchart reads from top to bottom. A "Business event" (for example `order.submitted`) triggers the "notify effect", which runs inside the transaction of the change. The effect asks the "Recipient resolver" for the audiences and people to tell. For each person and channel a "Decision" step applies the switches, the patient's opt-out and quiet hours. A "Template" step renders the message, filling placeholders. The result is stored in the database table `notification_outbox`, one row per person and channel. The "Relay in the worker", woken by a database

NOTIFY and by a two-second poll, claims due rows. The "In-app provider" writes the user's copy into `user_notifications`, which is the bell inbox. A second branch, the "Dev log provider", handles WhatsApp, SMS and email by writing the message to the server log instead of sending it.

Technical references: `backend/src/notifications/services/notification.service.ts` · `NotificationService.enqueue` · `backend/src/notifications/effects/notify.effect.ts` · `backend/src/notifications/delivery/`

59.2 Event, recipients and audiences

Status: [Implemented]

The `notify` effect names an event code (and may narrow the audiences). An unknown code fails the whole transition, so a typo can never silently drop messages. Who receives the message is decided by a **recipient resolver** registered per kind of subject: the module that owns the subject says how to find its doctor, coordinator, lab and patient. Today only the lab order workflow registers one, so lab events are the only ones that produce notifications.

There are four **audiences**: patient (the only external audience), doctor, coordinator and lab. A recipient is a user (a signed-in person), a patient or a lab contact. The same person listed twice, or under two audiences, gets one row per channel. If an event is not meant for an audience, enqueueing for it fails with `NOTIFICATION_AUDIENCE_INVALID`. A patient is looked up in the same organization, so a clinic can never message another clinic's patient.

Technical references: `backend/src/notifications/effects/notification-recipients.registry.ts` · `NotificationRecipientsRegistry` · `backend/src/notifications/notifications.enums.ts`

59.3 Rules and the decision order

Status: [Implemented]

For every recipient and channel the Platform makes one decision. The first check that matches wins:

1. An internal audience with a mandatory event on the in-app channel is queued, ignoring settings.
2. If the audience's master switch is off, the row is stored as `skipped_disabled`.
3. If the rule for this event and channel is off, it is `skipped_disabled`.
4. If the patient opted out of the channel, it is `skipped_opt_out`.
5. If the time is inside the audience's quiet hours (and the channel is not in-app), it is `held_quiet_hours` until the window ends.
6. Otherwise it is `queued` for now.

Skipped rows are still written, so the clinic can see in its message log why nothing was sent. Defaults apply when a clinic has set nothing: the patient audience and every patient rule are off; internal audiences are on, with in-app on and the external channels off. The channels offered per audience in code are in-app only for doctor, coordinator and lab, and none for patients, so patients receive nothing today.

Quiet hours are a start and end time per audience. The window can cross midnight (for example 22:00 to 08:00); equal start and end means no window. They use the audience setting's timezone if it names one, otherwise the timezone of the location the event belongs to (Chapter 62), otherwise UTC. In-app messages are never held, because they are silent.

Technical references: `backend/src/notifications/decision/notification-decision.ts` · `decide` · `quietHoursEnd`

59.4 Templates

Status: [Implemented]

A template has a subject, a body, a channel, a language (English, Urdu or Roman Urdu) and optional provider reference. The body uses placeholders such as `{clinic_name}` and `{recipient_name}`, plus the event's own. An unknown placeholder becomes empty text so no raw braces ever reach a person. The Platform picks the clinic's own template in the person's language, then the platform's template in that language, then the same two in English. A rule can name a specific template. If no template exists, the event's name and description are used.

Technical references: `backend/src/notifications/templates/template-render.ts` · `renderTemplate` · `pickTemplate`

59.5 Outbox, relay and sending

Status: [Implemented]

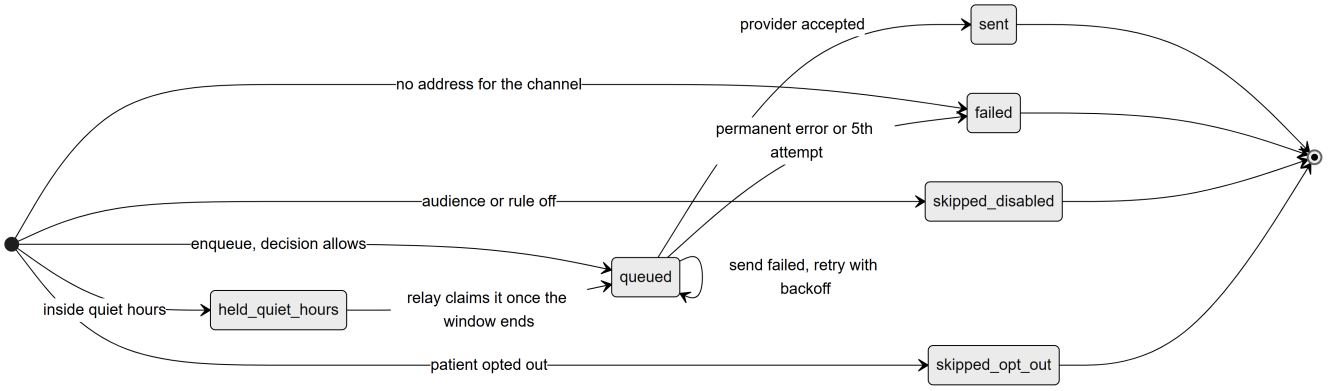

Figure 59.2 — Status lifecycle of a row in `notification_outbox`.

The diagram starts at a dot and shows several entry points. A row is created as `queued` when the decision allows sending, as `held_quiet_hours` when it falls inside quiet hours, as `skipped_disabled` when a switch or rule is off, as `skipped_opt_out` when the patient opted out, and directly as `failed` when the channel needs an address and the recipient has none. A held row becomes `queued` when the relay claims it once the quiet window has ended. A queued row becomes `sent` when the provider accepts it. A queued row whose send fails stays `queued` and is tried again after a pause. It becomes `failed` on a permanent error or at the fifth attempt. `sent`, `failed` and both skipped statuses are final. The status `delivered` exists in the code for providers that report delivery but nothing sets it today.

The relay (`NotificationRelayService`) runs only in processes that run jobs, which is the worker. It is woken at once by a database NOTIFY sent inside the enqueue transaction (so it fires only on commit) and also polls every two seconds, which picks up retries and quiet-hours rows that are due. If the NOTIFY connection fails, it logs a warning and relies on polling.

Sending works like this:

- A row is **claimed** with one short statement that locks it so no other worker takes it, adds one to its attempt count and pushes its due time out by a five-minute lease. If a worker dies mid-send, the row reappears after the lease.
- The provider is called with no transaction open; the result is written back only if this claim is still the latest.

- A failure is retried after 30 seconds, 2 minutes, 10 minutes and one hour; the fifth failure makes the row `failed`. A provider can mark an error permanent to skip retries.
- Delivery is at least once. The in-app copy is protected by a unique link to the outbox row, so a retry after a crash never shows the same message twice.
- Batches of 50 are processed until nothing is due.

Providers. The in-app provider writes `user_notifications` (user, event, title, body, link, read time). The development provider logs WhatsApp, SMS and email messages and marks them sent, so flows run end to end; it sends nothing to anyone. A real provider can take over a channel through the provider registry.

Technical references: `backend/src/notifications/delivery/notification-relay.service.ts` · `notification-dispatcher.service.ts` · `in-app.provider.ts` · `dev-log.provider.ts` · `channel-provider.registry.ts`

59.6 The inbox and the settings screen

Status: [Implemented]

The bell shows the signed-in user's inbox through `GET /me/notifications` (paged, with an unread-only filter), `GET /me/notifications/unread-count`, `POST /me/notifications/:id/read` and `POST /me/notifications/read-all`. These routes are context free: a user's inbox spans every clinic they work at, and lab users have no clinic location. Clinic settings are under `/notification-settings` and need `notifications.settings.manage`: the overview with master switches and quiet hours, audience updates, the rules matrix, templates (create, change, reset to platform wording) and the message log. A patient's channel preferences are read with `patients.view` and changed with `patients.edit`. Chapter 44 describes these screens.

59.7 Limitations and known gaps

- Only lab-related events exist; no appointment reminders or billing notices are produced.
- WhatsApp, SMS and email have no provider; patients cannot be reached.
- Only the lab workflow registers recipients.
- Inventory alerts are not notifications; they go to the server log only (Chapter 63).
- The status `delivered` is never set.

59.8 Owner's FAQ

If a lab order is rolled back, is a notification still sent? No. The notification row is part of the same transaction and disappears with it.

Why did a staff member not get a message? Check the log in notification settings: it shows each row with its status and reason, such as an audience switched off or a rule disabled.

Can I send reminders to patients by WhatsApp? Not yet. The decision logic and templates exist, but the channel is commented out and no provider is connected.

What happens if the worker is down? Rows stay queued in the database and are sent when the worker returns; nothing is lost.

Can quiet hours delay an in-app message? No. Quiet hours apply only to external channels.

Chapter 60 — Audit logging

This chapter explains how the Platform records who changed what and when: the history tables, the modules that write to them today, who may read them and what is kept for how long. It is written for the product owner, who must be able to answer an auditor's question honestly. The most important message is that audit coverage is **uneven**: patients and several administration areas are recorded in detail, while appointments, employees, payroll and organizations are not yet recorded in the general audit log.

60.1 The history tables

Status: [Implemented]

The Platform has several history tables, written by different modules at different times.

Table	Written by	What a row says
`audit_logs`	the shared audit service	One change: entity type and id, action (create, update, void, delete, export, view_sensitive), the **changed fields** before and after, actor, actor side (clinic, lab, platform, system), IP address and request id
`status_history`	the shared audit service	One state change: from state, to state, action code, actor, channel (app, portal, magic link, proxy, QR), reason
`billing_audit_logs`	billing services	One billing event with before and after, reason, amount, actor and the actor's role
`queue_events`	the queue orchestrator	One queue event with a running sequence number, status before and after
`prescription_history`	the prescription service	One prescription event: created, amended (with a full snapshot of the earlier version), finalized, printed, shared

The table lists the five history tables and what each row records. `audit_logs` and `status_history` are the general tables. The other three are older, module-specific histories that stay as they are.

Append only. `audit_logs` and `status_history` are protected by database triggers that reject any update or delete, so a row, once written, cannot be changed through the application or by accident. `commission_entries` has a similar trigger (Chapter 35). `billing_audit_logs`, `queue_events` and `prescription_history` are append-only by convention: the code only inserts, but no trigger enforces it.

Written in the same transaction. The shared service always writes in the caller's transaction, so the history commits or rolls back together with the change. An update that changed nothing writes no row.

Subject. Each general row can carry a `subject_id`, the record whose history it joins. For example, a patient's vitals row carries the patient as subject, so one query returns the patient's whole history.

Technical references: `backend/src/common/audit/audit.service.ts` · `AuditService` · `backend/src/common/audit/entities/` · `backend/src/migrations/1792490000000-lab-management-foundation.ts` · `backend/src/billing/audit/billing-audit.service.ts` · `backend/src/queue/entities/queue-event.entity.ts` · `backend/src/prescriptions/entities/prescription-history.entity.ts`

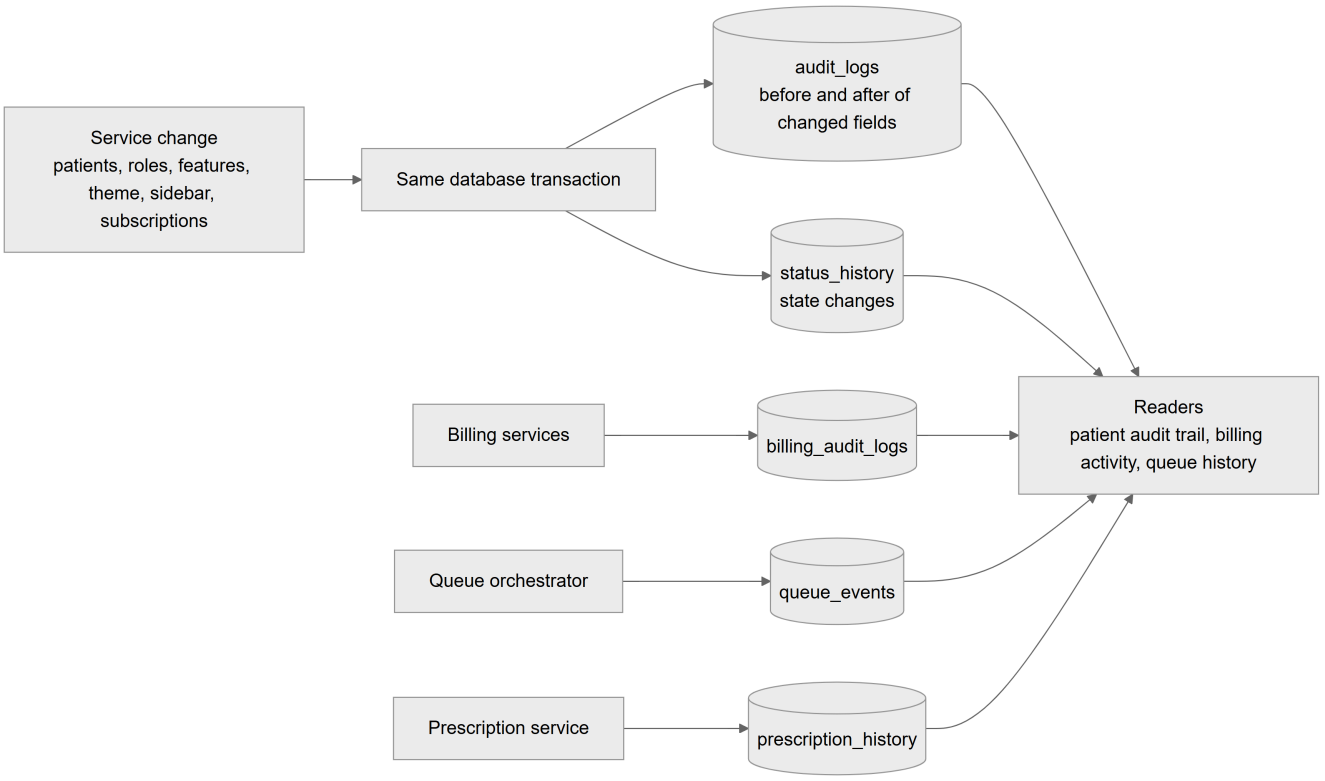

Figure 60.1 — Writers, history tables and readers.

The flowchart reads left to right. A box "Service change" (patients, roles, features, theme, sidebar, subscriptions) leads to "Same database transaction", which writes two tables: `audit_logs` (before and after of changed fields) and `status_history` (state changes). Three other writers stand alone: "Billing services" write `billing_audit_logs`, the "Queue orchestrator" writes `queue_events` and the "Prescription service" writes `prescription_history`. All five tables feed a single box, "Readers", which names the patient audit trail, the billing activity feed and the queue history.

60.2 What is recorded today

Status: [Partial] — coverage differs by module. The table states what the code does now.

Area	Written to the general audit log?	Detail
Patients	Yes, [Implemented]	Create, update and delete of the patient; activate and deactivate (as status history); 13 kinds of patient records (dental info, tooth chart, vitals, treatment plans and plan items, prescriptions as listed on the patient, documents, consents, communications, recalls, referrals, feedback and insurance). Delete rows keep the last snapshot and the counts of linked records. Patient import and queue walk-in patient creation are audited the same way. File addresses and signatures are never stored in the log.
Patient exports	Yes	Every patient CSV export writes an `export` row with the number of rows and the filters used
Report exports	Yes	Every Excel or PDF report export writes an `export` row with the report, format and period
Employment roles	Yes	Each create, edit, clone and delete, with before and after values and the reason for high-risk grants

Area	Written to the general audit log?	Detail
Organization features and location overrides	Yes	Each switch, with the platform recorded as actor when a super admin acts
Subscriptions and subscription payments	Yes	Payments and status changes
Theme and registry settings	Yes	One row per changed key
Sidebar layout	Yes	Save and reset
Billing	Own table	`billing_audit_logs` for charges, invoices, payments, refunds, discounts, cash close and settings
Lab modules	Yes	Lab orders, items, attachments, shipments, production, links, members, catalog, patient lab invoices, credits, vendor bills, statements and payments
Workflow engine	Yes	Transitions write status history; form schemas, workflow and stage templates write audit rows
Queue	Own table	`queue_events`, written for every queue change
Prescriptions	Own table	`prescription_history`
Appointments	**No** [Not implemented]	No audit row is written for booking, rescheduling, cancelling or status changes
Employees (staff records, salary, schedules)	**No** [Not implemented]	Only role definitions are audited, not employee data
Payroll	**No** [Not implemented]	No audit rows for payroll records or payments
Organizations and locations	**No** [Not implemented]	Settings and details of organizations and locations are not audited, apart from features, theme and subscriptions
Encounters, treatments, inventory	**No** [Not implemented]	Not audited
Sign-in events	Separate	`security_logs` (Chapter 56)

The table lists each area, whether it writes to the general audit log and what is recorded. It was built by searching the code for every writer of the audit service. The audit rollout plan names appointments, staff, payroll and organizations as later phases; their code is not written yet.

Technical references: `backend/src/patients/patient-audit.ts` · `recordPatientChange` · `backend/src/patients/patients.service.ts` · `backend/src/reports/reports.controller.ts` · `backend/src/employees/employment-roles.service.ts` · `backend/src/feature-flags/feature-admin.service.ts` ·

`backend/src/settings-registry/settings-registry.service.ts` · `backend/src/subscriptions/subscriptions.service.ts`

60.3 Reading the history

Status: [Partial]

Reader	Endpoint	Permission	Screen
Patient audit trail	`GET /patients/:id/audit-trail`	`admin.audit.view`	A history sheet on the patient page, shown only to holders of the permission
Queue history of a session	`GET /queue/sessions/:id/events`	`admin.audit.view`	Queue history
Billing activity feed	`GET /billing/activity`	`billing.activity.view`	Billing activity page
Entity timelines for labs	lab endpoints	lab permissions	Activity timelines on lab pages

The table lists the readers. The patient audit trail merges `audit_logs` and `status_history` for the patient and every record under them, newest first (up to 200 entries), and the values are redacted for readers who lack the clinical or document permissions. The permission `admin.audit.view` is "View the audit log" in the Admin group; the owner and super admin hold it, and the Admin system role holds the whole Admin group by default (Chapter 11).

There is **no organization-wide audit page**: no screen or endpoint lists all changes of a clinic, searches by user or date, or exports the log. The general tables can be read only through the entity or subject timelines in the table and, for developers, in the database.

60.4 Retention

Status: [Not implemented]

The Platform has no retention or purge policy for audit data. The audit tables are append-only, so nothing deletes them. The only clean-up job in the area removes queue idempotency keys older than 24 hours and published realtime outbox rows older than 7 days (Chapter 63), which are technical records, not history. `security_logs` and session rows are also never purged. A clinic with a legal retention limit has no built-in way to apply it.

60.5 Who is the actor

Each general row stores the user id, the **actor side** and, when available, the IP address and a request id taken from an `x-request-id` header if the caller sent one. The side is clinic for clinic staff, lab for lab members, platform for a super admin and system for background jobs. A super admin acting inside a clinic is recorded as platform.

60.6 Limitations and known gaps

- Appointments, staff data, payroll and organization settings have no audit history.

- No organization-wide audit viewer, filter or export.
- No retention policy.
- Three of the five history tables are append-only by convention only.
- Reading sensitive patient data is not logged; the action `view_sensitive` exists but is not written by any reader.
- Encounter notes in progress are not audited; the plan is one snapshot at completion (not built).

60.7 Owner's FAQ

Can I see who edited a patient's phone number? Yes, if you hold `admin.audit.view`: open the patient and the audit history. It shows the old and new value, who and when.

Can I see who cancelled an appointment? Not from an audit log today. Appointments are not audited. The appointment row itself shows its status and cancellation reason.

Can someone delete a history row to hide a change? Not through the application. For `audit_logs` and `status_history` the database itself rejects updates and deletes.

Who is told when patient data is exported? Nobody is notified, but each export writes a record with who, when, how many rows and the filters.

How long are audit records kept? Forever, because nothing deletes them.

Chapter 61 — Error handling

This chapter describes how the Platform reports problems: the shape of every API answer, the families of error codes, how validation and database conflicts become messages, how retries stay safe, and what the web app shows the user. It is written for the product owner and for support staff who read an error message and need to know what it means. Chapter 5 traces one request through the system and shows the main status codes; this chapter goes into the error side in more detail.

61.1 The envelope

Status: [Implemented]

Every controller answers inside one JSON envelope built by `ResponseWrapper`. On success the fields are `data` (the payload), `message` (for example "Patient created successfully"), `isSuccess` (true) and `httpStatusCode` (always 200 in the envelope, even when the HTTP status is 201). On failure the global exception filter `GlobalExceptionFilter` builds the same envelope with `isSuccess` false, the real HTTP status in `httpStatusCode`, a short `message` for people, a `devMessage` with a technical text for developers and an `error` field holding the exception body. The web app reads `res.data.data` on success and `err.response.data.message` (or `error.code`) on failure.

Technical references: `backend/src/common/dtos/ResponseWrapper.ts` · `backend/src/common/exceptions/HttpExceptionHandler.ts`

61.2 What the filter does with an error

Status: [Implemented]

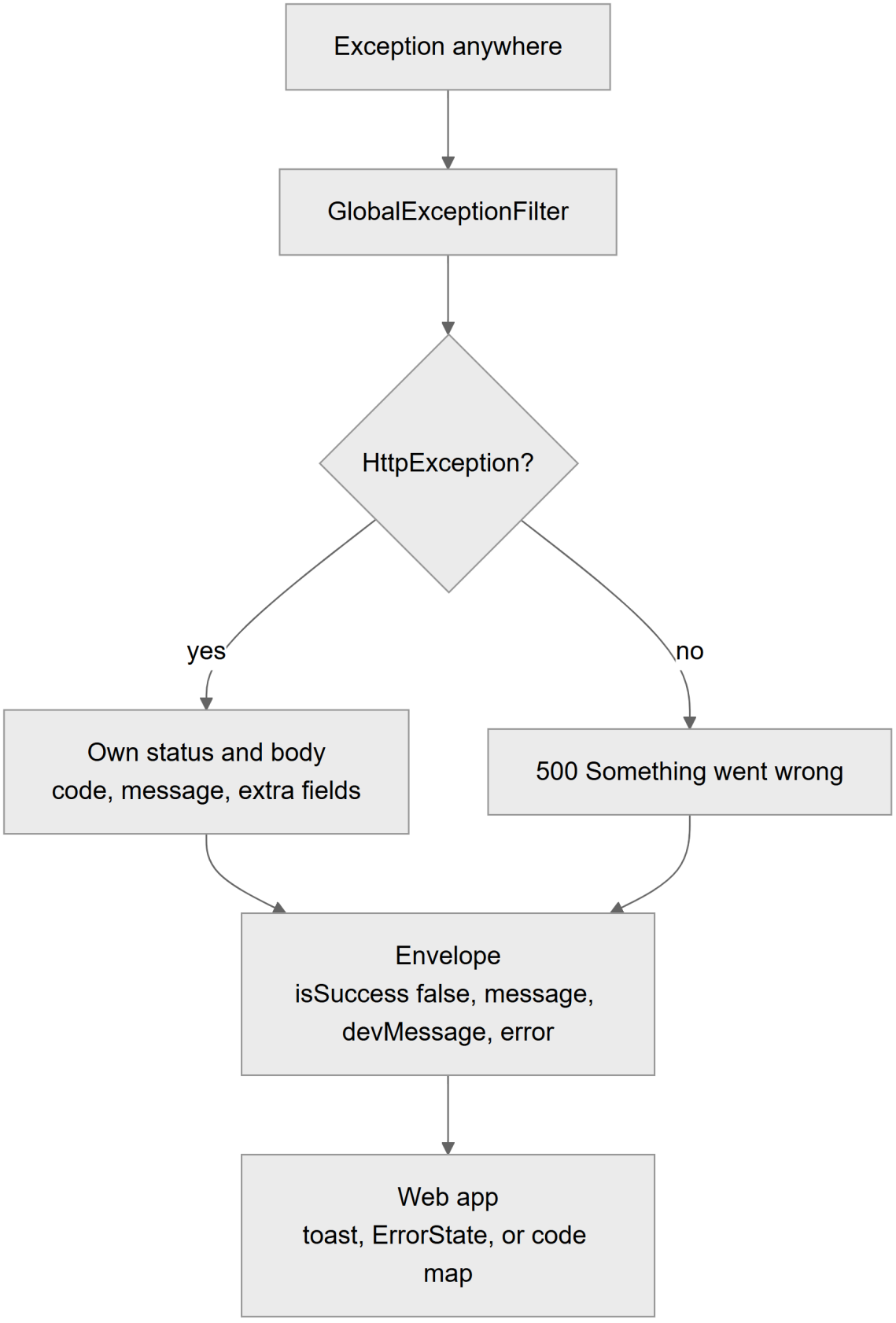

Figure 61.1 — How an exception becomes an envelope and a message on screen.

The flowchart reads from top to bottom. An "Exception anywhere" in the request reaches `GlobalExceptionFilter`. The filter asks whether it is an HTTP exception. If yes, the request keeps its own status and body, with a code, a message and any extra fields. If no, which means an unexpected failure, the answer is

500 with the message "Something went wrong". Both branches build the envelope: `isSuccess` false, `message`, `devMessage` and `error`. The web app then shows a toast, an `ErrorState` panel or a friendly text chosen by the error code.

Categories of error:

- **Expected HTTP errors.** Business rules and security refusals are thrown as typed exceptions with a status and, in most modules, a machine-readable `code`. The message is written for the user.
- **Validation errors.** The global validation pipe rejects bad input (section 61.3). Several messages are possible; the envelope's `message` shows the first one.
- **Unexpected errors.** Anything that is not an HTTP exception, such as an unhandled database failure, is logged with its stack on the server and answered as 500 "Something went wrong". The user-facing text never contains internals.
- **Rate limit.** A throttled request is answered 429 with "Too many requests. Please try again later." and a number of seconds to wait (Chapter 56).

Every error is logged by the application logger together with the request address.

61.3 Validation

Status: [Implemented]

The global validation pipe converts input to the types named in each request class and has `whitelist` and `forbidNonWhitelisted` on. A request that contains a field the API does not define is **rejected** with 400, not silently ignored, and a field with a wrong type or format is rejected with a message such as "email must be an email". Request classes use validation decorators for lengths, formats, enumerations and nested objects. The same rules appear in the web app as form schemas, so most mistakes are caught before the request leaves the browser; the server rule is the one that counts.

61.4 Error code families

Status: [Implemented]

Status	Code or family	Meaning
400	`IDEMPOTENCY_KEY_REQUIRED`, `REASON_REQUIRED`, `INVALID_PERMISSIONS`, validation messages	The request is incomplete or malformed
401	(message only)	No token, expired token or bad token; the web app tries to refresh
402	`SUBSCRIPTION_EXPIRED`	The clinic is read-only because its subscription lapsed
403	`NO_PERMISSION` with the missing permissions, `NOT_ENTITLED`, `ESCALATION`, `SYSTEM_ROLE_READ_ONLY`, `SETTING_LOCKED`, `FORBIDDEN`, `LAB_ACCESS_DENIED`	The caller is not allowed
404	`NOT_FOUND`, `BILLING_NOT_FOUND`, `LAB_NOT_FOUND`, and similar	The record does not exist in this organization
409	`DOCTOR_DOUBLE_BOOKED`, `VERSION_CONFLICT`, `STALE_VERSION`, `ALREADY_CHECKED_IN`, `ACTIVE_WITH_OTHER_DOCTOR`, `DOCTOR_BUSY`, `ROLE_NAME_TAKEN`, `ROLE_HAS_MEMBERS`, `REQUEST_IN_FLIGHT`, `BILLING_DUPLICATE`, `PLAN_LIMIT` and similar	The request conflicts with the current state or with another request

Status	Code or family	Meaning
422	`INVALID_TRANSITION`, `IDEMPOTENCY_KEY_REUSED`, `NOT_TODAY`, `LATE_LIMIT_EXCEEDED`, `SESSION_NOT_OPEN`, many `BILLING_*`, `FORM_*` and `WORKFLOW_*` codes	The request is well formed but the rules refuse it
429	`MAGIC_LINK_RATE_LIMITED` and the login throttle	Too many requests
500	`BILLING_INVARIANT_VIOLATION`, `WORKFLOW_MISCONFIGURED`, anything unexpected	A bug or a broken money rule
503	`LOCK_TIMEOUT`	The queue was busy; retry

The table groups the codes by HTTP status and shows what each family means. It lists examples, not every code. The codes are thrown by several module-specific error builders, each of which passes the code in `error.code`:

- **Billing** (`billing-errors.ts`): about 40 `BILLING_*` codes plus `COMPENSATION_*` and `COMMISSION_*` for the commission rules. Each maps to a fixed status, for example allocation larger than the balance is 422 and a duplicate is 409.
- **Queue** (`queue.errors.ts`): `NOT_FOUND`, `FORBIDDEN`, `INVALID_TRANSITION`, `SESSION_NOT_OPEN`, `DOCTOR_BUSY`, `VERSION_CONFLICT`, `ALREADY_CHECKED_IN`, `ACTIVE_WITH_OTHER_DOCTOR`, `LATE_LIMIT_EXCEEDED`, `NOT_TODAY`, `LOCK_TIMEOUT`, `IDEMPOTENCY_KEY_*`, `REQUEST_IN_FLIGHT`, `REASON_REQUIRED`.
- **Workflow engine** (`engine-errors.ts`): form, workflow and stage template codes, `INVALID_TRANSITION`, `TRANSITION_FORBIDDEN`, `TRANSITION_INPUT_REQUIRED`, `TRANSITION_REASON_REQUIRED`.
- **Labs** (`lab-errors.ts`, plus lab billing, catalog, orders and payables): `LAB_*` codes, including `LAB_PORTAL_CLINIC_CONTEXT` and `LAB_SELECTION_REQUIRED`.
- **Notifications** (`notification-errors.ts`): `NOTIFICATION_*` codes and `PATIENT_NOT_FOUND`.
- **Access and features**: `NO_PERMISSION`, `ESCALATION`, `INVALID_PERMISSIONS`, `NOT_ENTITLED`, `SETTING_LOCKED`, `PLAN_LIMIT`.

`DOCTOR_DOUBLE_BOOKED` is the appointment overlap error (Chapter 29): the database rejects two bookings of one doctor at overlapping times, and the service turns the failure into a 409 with that code.

Technical references: `backend/src/billing/common/billing-errors.ts` · `backend/src/queue/queue.errors.ts` · `backend/src/engine/common/engine-errors.ts` · `backend/src/labs/common/lab-errors.ts` · `backend/src/notifications/notification-errors.ts` · `backend/src/access/decorators/access.decorators.ts`

61.5 Safe retries: idempotency keys and If-Match

Status: [Implemented]

Two mechanisms protect against double clicks, flaky networks and two people editing the same record.

Idempotency-Key. A client sends a header `Idempotency-Key` (1 to 150 characters) on actions that must not run twice. The server stores the key, a hash of the request and, once finished, the response, inside the same transaction as the work. If the same key arrives again with the same request, the stored answer is replayed and nothing runs twice. The same key with a different request gives 422 `IDEMPOTENCY_KEY_REUSED`. The same key while the first request is still running gives 409 `REQUEST_IN_FLIGHT`. A missing key where one is required gives 400 `IDEMPOTENCY_KEY_REQUIRED`. The key is required on every queue change, on patient check-in from an appointment, on money-moving lab billing posts and on the workflow engine transitions; the web app creates a fresh key for each open sheet or action. Queue idempotency keys are deleted after 24 hours by a nightly job.

If-Match and versions. Records that two people may edit carry a version number. A client may send `If-Match: <version>` (queue entries, lab order actions) or the version in the body; the server checks it while holding a row lock and, if the record has moved on, answers 409 with `VERSION_CONFLICT` or `STALE_VERSION` and the current

state or version. The web app reloads and asks the user to try again, with texts such as "Someone else changed this just now. The latest version is loaded".

Technical references: `backend/src/common/idempotency/idempotency.service.ts` · `IdempotencyService` · `backend/src/common/errors/optimistic-lock.ts` · `assertVersion` · `backend/src/queue/decorators/queue-request.decorators.ts`

61.6 Database constraints become messages

Status: [Partial]

The database enforces uniqueness and exclusion rules, and several services translate the database error into a clear conflict. The translation is per service; there is no global mapping.

- **Exclusion constraint on appointments** (`no_doctor_overlap`): Postgres error 23P01 becomes 409 `DOCTOR_DOUBLE_BOOKED` . Compensation plan versions and lab prices use similar exclusion constraints with their own messages.
- **Unique violations** (23505) are translated in billing, lab billing, lab orders, lab catalog and lab payables into codes such as `BILLING_DUPLICATE` .
- **Foreign key violations** (23503) are translated when deleting an employee that is still referenced.
- **Lock timeouts** in the queue become 503 `LOCK_TIMEOUT` .

A database error that no service translates reaches the filter as an unexpected error and is answered as 500 "Something went wrong". A duplicate patient number or purchase order number, for instance, would currently surface this way (Chapter 62).

61.7 In the web app

Status: [Implemented]

- **401 and refresh.** The shared axios client refreshes the access token on a 401 and retries once (Chapter 56).
- **Toasts.** Mutations report failures with a toast (the library `sonner` , at the top right, closable, 4 seconds). Many hooks have a helper that reads `err.response.data.message` and falls back to a text such as "Failed to create item". About 180 places raise an error toast.
- **Friendly code maps.** Billing, labs, lab orders, lab catalog, lab payables, lab billing and the engine studio each have a small map from an error code to a plain sentence, for example the billing code for a missing document code becomes "Set a document code for this location in Billing settings first." If a code has no entry, the server's own message is shown, then a generic text.
- **Stale screens.** The queue hook shows "Someone else just changed this patient — showing the latest." as a warning on `VERSION_CONFLICT` and refreshes the screen.
- **Page states.** Every list or detail screen uses `LoadingSkeleton` while loading (table, cards, stats or detail shape), `EmptyState` when there is nothing (icon, title, message and an optional action) and `ErrorState` when a request failed (title "Something went wrong", a sentence and a **Try again** button). A route-level error boundary, `RouteError` , shows the error message with a retry button for render failures.
- **Access errors.** A missing route permission shows the No access page, not an error (Chapter 57). The access gate shows "Couldn't load your access" if `/me/access` fails.
- **Query settings.** Queries retry twice and mutations once, with 5 minutes of freshness.

Technical references: `web-app/src/lib/api.ts` · `web-app/src/lib/query-client.ts` · `web-app/src/features/billing/lib/billing-errors.ts` · `web-app/src/features/engine-studio/lib/studio-errors.ts` · `web-app/src/features/queue/hooks/useQueueAction.ts` · `web-app/src/components/feedback/ErrorState.tsx` · `EmptyState.tsx` · `LoadingSkeleton.tsx` · `web-app/src/components/shared/RouteError.tsx`

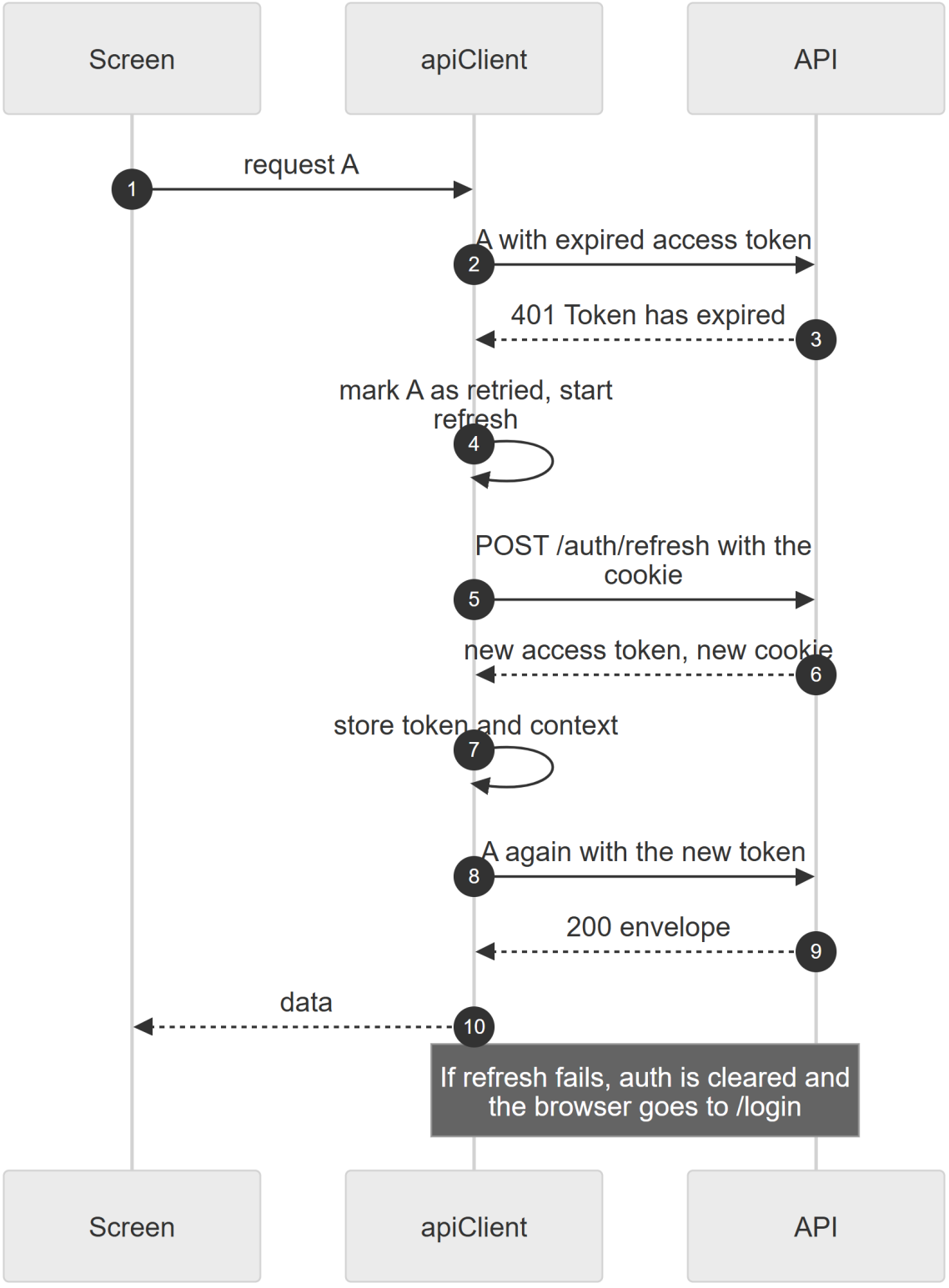

Figure 61.2 — How an expired access token is replaced without the user noticing.

The diagram has three participants: the Screen, the `apiClient` axios instance and the API. The screen sends request A. The client sends A with an expired access token. The API answers 401 "Token has expired". The client marks A as retried and starts a refresh. It sends `POST /auth/refresh` with the cookie. The API answers with a new

access token and a new cookie. The client stores the token and context, sends A again with the new token, gets a 200 envelope and returns the data to the screen. A note says that if the refresh fails, the stored login is cleared and the browser goes to `/login`. While one refresh is running, other requests that fail with 401 wait in a queue and are retried with the new token.

61.8 Limitations and known gaps

- Error messages are in English. Only the No access headline carries a Roman-Urdu line.
- Code-to-sentence maps exist for some modules only; others show the server's text.
- There is no central error-tracking service; errors are written to the server log.
- Database errors are translated service by service, so an untranslated one shows as a generic 500.
- The error body includes a developer message field in every failure.

61.9 Owner's FAQ

A user says "Something went wrong". What now? It is an unexpected server error. Ask for the time and the page; the server log holds the cause. Other failures give a specific message.

What does 402 mean? The subscription expired; the clinic is read-only until a payment is recorded.

What does "Someone else changed this" mean? Another person saved the same record first. The latest data is loaded; redo the change.

What happens if the receptionist double-clicks "Check in"? The second click carries the same idempotency key, so the first result is returned and only one token is issued.

Does the API ignore fields it does not know? No. It rejects them with 400.

Chapter 62 — Time, money and document numbers

This chapter explains three rules that run through every module: how the Platform decides what time it is for a clinic, how it stores and calculates money, and how it numbers invoices, patients, orders and other documents. It is written for the product owner, because mistakes in these three areas, a wrong day, a wrong rupee or a duplicate number, are the ones that clinics notice first. The chapter states the rules, where they are followed strictly, and where the code still departs from them.

62.1 Clinic time

Status: [Implemented] for the rule. [Partial] for coverage: some jobs and statistics still use the server clock.

The rule. Every clinic date and time follows the **location's** timezone (the `timezone` of the location, for example a Pakistan or a Gulf timezone). Working hours, appointment availability, the same-day duplicate check, the queue's day boundary, token days and quiet hours all use it. A clinic in one timezone and a server in another still agree on what "today" means for the clinic.

How it works. `ClockService` is the single source of "now" and "today" for clinic logic. Its main functions are:

- `now()` : the current instant (tests can replace it).
- `timezoneFor(locationId)` : the location's timezone name, defaulting to UTC when the stored name is not valid, cached for 60 seconds.
- `today(timezone)` : the location-local date as `YYYY-MM-DD` .
- `wallClock(timezone, instant)` : the local weekday and `HH:MM` .
- `dayRange(timezone, date)` : the start and end instants (in UTC) of a local day.
- `at(timezone, date, time)` : the instant when the clinic's wall clock reads a given date and time.

The code rule is: never use `new Date()` together with `setHours()` for a clinic day or time, because those use the server's timezone. Appointments, queue and notifications follow it (Chapters 29, 30 and 59).

Where the server clock is still used. These places do not use the location's timezone:

- **Scheduled jobs.** Daily and monthly jobs run on the worker's server timezone, for example the 01:00 credit expiry, the 02:30 billing check, the 08:00 inventory alert and the payroll run at 05:00 on the 1st (Chapter 63 and Appendix E). The queue's day-end job is an exception: it decides what a past clinic day is from each location's policy.
- **Credit expiry.** The check works out "today" from the server's local date, so a prepaid credit can expire a few hours early or late for a clinic in another timezone.
- **Statistics.** The financial summary takes its default date and its month list from the server date.
- **Numbering years.** The year inside purchase order and prescription numbers is the server's year.
- **Leave balance year** uses the server year.
- **The lab catalog calendar** fixes its "today" to one specific timezone (Asia/Karachi) regardless of the location.
- **Exports** that print a date (patient export file name) use the server date.

For a clinic whose timezone differs from the server's by a few hours, the differences matter only near midnight or at month boundaries.

Technical references: `backend/src/common/clock/clock.service.ts` · `ClockService` · `backend/src/common/clock/tz.util.ts` · `backend/src/billing/patient-credits.service.ts` · `backend/src/billing/financial-summary.service.ts` · `web-app/src/features/lab-catalog/lib/money.ts` · `catalogToday`

62.2 Money

Status: [Implemented]

What is stored. Money has two storage styles, depending on the module:

Style	Where	Meaning
`numeric(12,2)` rupees	Patient billing: charges, invoices, payments, refunds, ledger entries	Exact decimal with two places
`bigint` paisa	Commissions (rules, entries, settlements, compensation plans), the lab billing tables (invoice, credits, payments), and a few billing fields	Whole number of the smallest unit
`decimal(10,2)`	Inventory prices and quantities, payroll records, employee salary on assignments	Exact decimal

The table shows the three styles. A **paisa** is one hundredth of a rupee; "PKR" is the default currency and not a limit. Amounts cross the API as numbers with at most two decimals in the billing module, and as integer paisa in the lab and commission modules.

How it is calculated. All billing math is done in **integer paisa** so that no floating-point error can creep in. `money.ts` holds the helpers: `toPaisa` and `fromPaisa`, `sumPaisa`, `percentOf` and `basisPointsOf`. A percentage with up to two decimals is turned into basis points (17.5 percent becomes 1750) and applied with exact big-integer arithmetic.

- **Rounding is half up** (0.5 paisa rounds to the next whole paisa), the same everywhere, including tax.
- **Splitting.** When an amount is shared in proportion (for example tax spread over charges), each share is rounded down and the leftover paisa go to the largest share, so the shares always add up exactly to the total.
- **Database columns** are read through transformers: `numeric` comes back from the database as text and is rounded to two decimals; `bigint` comes back as text and is converted to a number, which is exact for any clinic amount.

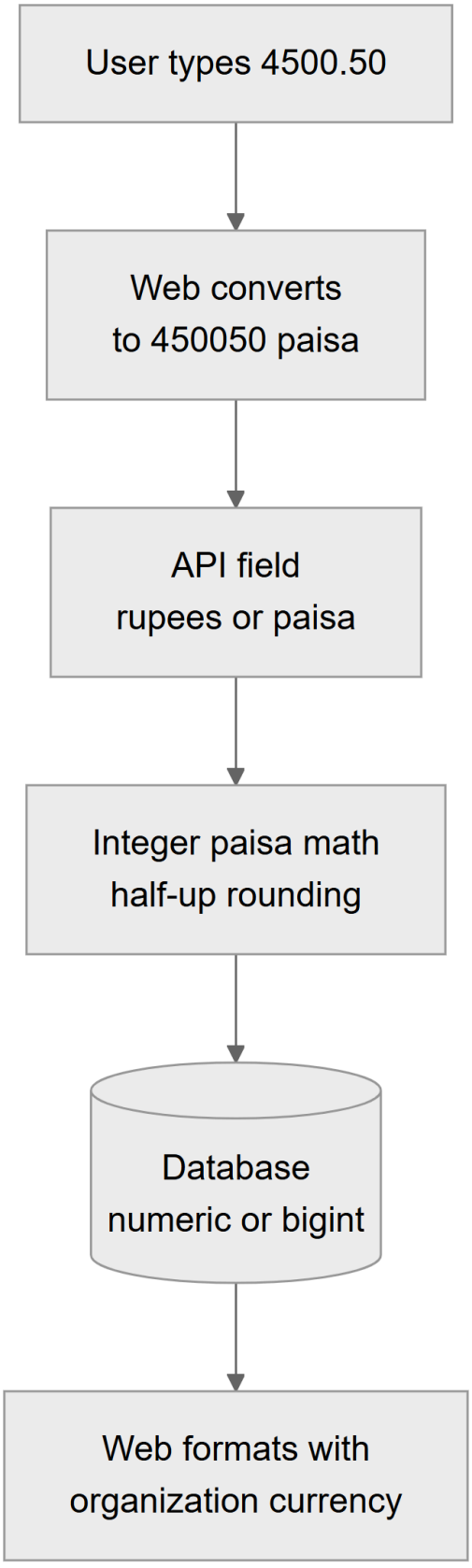

Figure 62.1 — How an amount typed by a person travels to the database and back.

The flowchart reads top to bottom. A user types 4500.50. The web app converts the text to 450050 paisa with integer-only code (no floating point). The API receives the amount as rupees or paisa depending on the field. Server code calculates in integer paisa with half-up rounding. The value is stored in the database as `numeric` or as `bigint` paisa. When the screen reads it back, the web app formats it with the organization's currency.

Tax is exclusive. Prices are before tax. Tax equals the taxable amount (the sum of active charge net amounts after discounts) times the rate, rounded half up once per invoice. The total is subtotal minus discount plus tax. The rate is copied onto the invoice when it is issued from the location's billing settings, and later edits use that copy, not the current setting. Tax is spread over the invoice's charges so that payments can be allocated charge by charge; tax is never revenue and never earns commission. Lab invoices use the same half-up rounding and keep tax in its own paisa column.

Currency. Each organization has a `currency` (default PKR). It is returned with the organization list after sign-in. The web app's `activeCurrency()` and `useCurrency()` read it, and `formatPaisa` writes amounts as "PKR 4,500.50". Number grouping uses a Pakistani locale. The currency is a display label: there is no conversion and no multi-currency ledger.

Technical references: `backend/src/billing/common/money.ts` · `money-column.ts` · `paisa-column.ts` · `backend/src/billing/tax/invoice-tax.ts` · `backend/src/organizations/entities/organization.entity.ts` · `web-app/src/lib/currency.ts` · `web-app/src/features/lab-catalog/lib/money.ts`

62.3 Document numbering

Status: [Partial]. Billing and lab documents are numbered safely; some older numbers are not.

There are four numbering families.

Family	Documents	Format	How it counts
Per location, per type, per year	Patient invoices (`INV`), receipts (`REC`), refunds (`RFD`)	location code, type, year, five digits, for example `MAIN-INV-2026-00001`	Table `document_sequences`
Per organization sequence	Lab order (`LAB`), shipment (`SHP`), lab invoice (`LINV`), lab receipt (`LRC`), payment reversal (`LRV`), credit note (`LCN`), lab refund (`LRF`), vendor bill (`VB`), vendor payment (`VP`)	prefix and at least four digits, for example `LAB-0001`	Table `number_sequences`
Count plus one in application code	Patient number, treatment plan number, patient-page prescription number	`P-00001` , `TP-00001` , `RX-00001`	Number of existing rows plus one
Last number plus one in application code	Purchase order number, prescription number from the prescriptions module	`PO-2026-00001` , `RX-2026-00001`	Reads the latest row and adds one

The table lists the four families with their formats. Invoices, receipts and refunds need the location's **document code** (2 to 6 letters or digits, set in billing settings and locked after the first invoice); without it the Platform refuses to issue the document.

The safe families. The first two families use a database counter row that is locked until the surrounding transaction ends. Concurrent requests wait their turn, no number repeats, and a rollback gives the number back, so numbers have no gaps. The service refuses to run outside a transaction. Invoice sequences restart each year, per location. Lab sequences never restart.

The risky families.

- **Patient, plan and prescription numbers (count plus one).** The next number is the count of rows for the organization plus one. Two registrations at the same moment can compute the same number, and the second save fails on the unique index. Patients are deleted permanently, so after a deletion the count drops and the next number can repeat one that is still in use (the patient count includes soft-deleted rows but not permanently deleted ones). The result in either case is a failed save that the user sees as a generic error, not a wrong document.

- **Purchase order and prescription numbers (last plus one).** The next number reads the most recent row of the organization and adds one, with the year taken from the server clock. Two simultaneous requests can read the same row. In addition, the unique index on the purchase order number and on the prescription number covers all organizations together, so two clinics that both reach the same sequence in the same year would collide.

These are reliability risks, not security ones. A collision stops the save; it does not issue a duplicate.

Technical references: `backend/src/billing/numbering/document-sequence.service.ts` · `DocumentSequenceService` · `backend/src/common/sequences/number-sequence.service.ts` · `NumberSequenceService` · `backend/src/patients/patients.service.ts` · `generatePatientNumber` · `backend/src/inventory/services/purchase-order.service.ts` · `generatePoNumber` · `backend/src/prescriptions/services/prescription.service.ts` · `generatePrescriptionNumber`

62.4 Limitations and known gaps

- Several jobs and statistics use the server timezone, not the clinic's.
- Two storage styles for money (decimal rupees and integer paisa) exist side by side; conversion happens at module boundaries.
- Count-based and last-number-based document numbers can collide under concurrency, and purchase order and prescription numbers are unique across all clinics together.
- No multi-currency support or conversion.
- Tax is exclusive only; there is no tax-inclusive pricing.

62.5 Owner's FAQ

What time does "today" mean for my branch? The branch's own timezone, set on the location.

Why can a daily job run at an odd hour for my clinic? Jobs follow the server's timezone. A job set for 01:00 runs at 01:00 server time.

Can an invoice number repeat? No. Invoice, receipt and refund numbers come from a locked counter per branch and year.

Can two patients get the same patient number? The database refuses a duplicate, so the second save fails and must be tried again.

Are my prices with or without tax? Without. Tax is added on top at the rate set for the branch when the invoice is issued.

Can I invoice in another currency? You can set the organization's currency label, but there is no conversion.

Chapter 63 — Realtime updates and background jobs

This chapter explains the two ways the Platform acts without a user clicking: it pushes live updates to open screens over a socket connection, and it runs timed jobs and background loops in a worker process. It is written for the product owner who must understand which processes to run and what happens when Redis or the worker is down. Appendix E holds the full job table; this chapter restates the essentials and explains how the pieces fit together. The queue module (Chapter 30) and notifications (Chapters 44 and 59) are the main users of these mechanisms.

63.1 Process roles

Status: [Implemented]

The backend is one program that starts in one of three roles, chosen by the start command.

Role	Start file	Serves web requests and sockets	Runs jobs and relays	Needs Redis
`api`	`main.api.ts`	Yes	No	Yes, to receive events
`worker`	`main.worker.ts`	No	Yes	Yes, to send events
`all`	`main.ts`	Yes	Yes	No

The table shows the three roles. In production, run two or more `api` processes behind a load balancer and **exactly one** `worker`. In development, the single `all` process does everything and needs no Redis, because the relay hands events directly to the local socket gateway. The job scheduler is enabled only in processes whose role is not `api`, so adding API processes never duplicates a job.

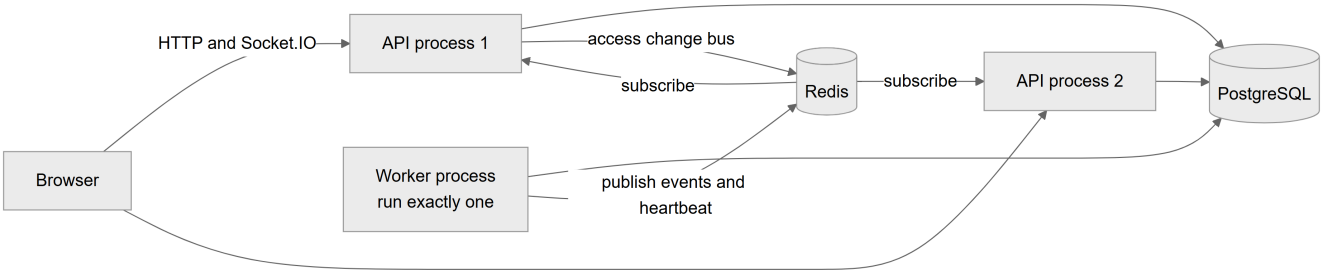

Figure 63.1 — The browser, two API processes, one worker, PostgreSQL and Redis.

The flowchart reads left to right. The Browser talks to API process 1 and API process 2 over HTTP and Socket.IO. Both API processes read and write PostgreSQL. The Worker process, marked "run exactly one", also uses PostgreSQL. The worker publishes events and a heartbeat to Redis. Redis delivers them to both API processes through a subscription. A further arrow shows an API process using Redis for the access change bus, which is how one API process tells the others that access changed (Chapter 57).

Technical references: `backend/src/common/runtime/process-role.ts` · `backend/src/main.ts` · `main.api.ts` · `main.worker.ts` · `backend/src/app.module.ts` · `AppModule.forRole`

63.2 The realtime gateway

Status: [Implemented]

What it does. Pushes small "something changed" messages to open browser screens so that the reception board, doctor screen, waiting-room display and permission-aware screens stay live without constant refreshing.

How it connects. A Socket.IO gateway listens at the path `/realtime` of the API. The web app connects with websocket transport and passes its access token in the handshake. The token is verified once, when the socket connects. A browser with no token may connect anonymously, but may join only the public display room. After connecting, the client sends a `subscribe` message naming a room; the server answers with success or a reason.

Rooms.

Room	Who may join	What it carries
`session:<id>`	Users with access to the queue session's location	Events for one doctor's queue (doctor screen, reception)
`location:<id>:board`	Users with access to the location	Events of every queue at the location (reception board)
`location:<id>:display`	Anyone, no sign-in	Waiting-room TV events; payloads carry token numbers and doctor names, never patient names
`org:<id>`	Users with access to the organization (super admin, owner, or an active staff assignment)	The `access.changed` event with the organization id and the new version

The table lists the four room types. Anything else gets "Unknown room"; a room the user may not enter gets "No access to this room".

Events are nudges. A message carries a sequence number (`seq`). The browser keeps the sequence of the snapshot it loaded, ignores older or duplicate messages and fetches a fresh snapshot when it sees a newer one. This makes delivery at least once safe, because a duplicate does nothing.

Technical references: `backend/src/queue/realtime/queue.gateway.ts` · `QueueGateway` · `web-app/src/features/access/hooks/useAccessSocket.ts`

63.3 From a commit to a screen: outbox, relay, Redis

Status: [Implemented]

When a queue action changes state, the same database transaction also writes an event row in the `outbox` table and sends a database NOTIFY, so an event exists only if the change was committed. The **outbox relay** in the worker takes unpublished rows (up to 100 at a time, locking them so a second relay would skip them), publishes each to one Redis channel, waits until Redis accepted the whole batch and only then marks the rows as published. If Redis is down, the batch is rolled back and tried again every second. It is woken by the NOTIFY and also polls every second as a fallback. Every API process subscribes to the channel and emits each message to its own local sockets, so a browser connected to any API process sees every event. Chapter 5, section 5.8, shows this as a sequence diagram.

Heartbeat and watchdog. The worker publishes a heartbeat to the same channel every 5 seconds. Each API process runs a watchdog: it considers the bus healthy only while it is subscribed and has seen a heartbeat within 15 seconds. When the bus is unhealthy (Redis or worker down), the API process **refuses new socket connections and drops existing ones**, using a transport close so that browsers keep trying to reconnect. Screens then fall back to polling about every 10 seconds and reload their data once the socket returns. At start-up the gateway stays closed until the first heartbeat arrives.

Access changes. The access change bus is a second, separate path on its own Redis channel. When an organization's access version is bumped, the bus notifies the local caches and the local gateway, and publishes to Redis so the other processes do the same. Each process then emits `access.changed` to its sockets in the `org:<id>` room. This path does not use the outbox or the worker (Chapter 57).

Single-process mode. In the `all` role, the relay publishes straight to the local gateway.

Technical references: `backend/src/queue/realtime/outbox-relay.service.ts` · `OutboxRelayService` · `redis-realtime.publisher.ts` · `redis-realtime.subscriber.ts` · `realtime-bus.ts` · `in-process-realtime.publisher.ts` · `backend/src/access/version/access-change-bus.ts`

63.4 Scheduled jobs

Status: [Implemented] (the inventory alert job is [Partial])

All jobs run in the worker (or the `all` process). The schedules are in **server time**; none sets a timezone.

Job	Schedule	What it does
Queue no-show marker	every 60 seconds	Marks a booked patient as no-show once they have not checked in by the start time plus the location's maximum lateness
Queue call timeout	every 15 seconds	Skips a called patient who never came, when the queue policy sets a timeout
Queue session day-end	every 5 minutes	Closes empty queue sessions of a past clinic day; leaves non-empty ones open and logs an alert
Queue cleanup	daily at 03:15	Deletes queue idempotency keys older than 24 hours and published outbox rows older than 7 days
Patient credit expiry	daily at 01:00	Marks prepaid credits past their expiry date as expired
Billing invariant check	daily at 02:30	Recomputes billing totals, checks money rules including the commission ledger and logs any break
Month-end payroll	05:00 on the 1st of each month	Generates payroll records for the month just ended for every active location; locations already generated are skipped
Inventory alerts	daily at 08:00	Counts low-stock and soon-expiring items and writes a warning to the server log only

The table lists the eight timed jobs. The first four keep the live queue consistent; the others handle daily and monthly housekeeping. Appendix E gives the same table with statuses and references. The queue day-end job is the only one that respects each location's clinic day, taken from its queue policy.

Background loops. Three loops run continuously in the worker. The **queue outbox relay** (section 63.3). The **notification relay**, which is woken by a database NOTIFY and polls every two seconds, sends notification rows to the inbox (Chapter 59). The **heartbeat** is published every 5 seconds. Both relays follow the outbox pattern: a change and its message commit together, and a separate loop delivers the message later. If the database NOTIFY connection drops, each relay logs a warning and keeps working by polling.

Other timers. The in-memory rate limit tables in the authentication guards and the magic-link throttle are swept every minute or when they grow large; they do not need the worker.

Technical references: `backend/src/queue/jobs/queue-jobs.service.ts` · `backend/src/billing/patient-credits.service.ts` · `backend/src/billing/invariants/billing-invariant.job.ts` · `backend/src/payroll/payroll.service.ts` · `backend/src/inventory/services/inventory-alerts.service.ts` · `backend/src/notifications/delivery/notification-relay.service.ts` · `docs/handbook/src/appendix/appE-scheduled-jobs.md`

63.5 Limitations and known gaps

- Exactly one worker must run; nothing in the code stops a second worker, and a second worker would run each timed job a second time (the relays are safe with several workers, the timed jobs are not guaranteed to be).
- Job times follow the server timezone, not the clinics'.
- Inventory alerts reach only the server log, not any user.
- Realtime depends on Redis in production; without it screens poll about every 10 seconds.
- The access token is checked once when a socket connects; the connection continues after the token expires.
- There is no clean-up job for session rows, security logs or the notification outbox.

63.6 Owner's FAQ

What happens if Redis is down? Realtime stops: the API refuses sockets and screens poll every ten seconds, so they update a little later. Access changes still reach the process that made them at once and the others within 15 seconds.

What happens if the worker is down? No timed jobs run, no notifications are sent and no queue events are published. Rows wait in the database and are processed when the worker returns. Screens fall back to polling.

Can I run two workers for safety? No. Run exactly one. A second worker would repeat the scheduled jobs.

Do I need Redis on my laptop? No. The single `all` process does not need it.

Can the waiting-room TV show patient names? No. The public display room carries token numbers and doctor names only.

Part 8 — Current status

Chapter 64 — Feature Status Matrix

This chapter lists every feature described in the module chapters (28 to 45), the workflow chapters (46 to 55) and the cross-cutting chapters (56 to 63), one row each, with its status label. It is for the owner who needs to know what is really built before promising anything to a clinic. The status of each row was merged from the notes of the chapters and checked against the chapter text; where a chapter and its notes differed, the chapter wins.

64.1 How to read the matrix

Each row has four columns: the chapter number, the feature as a user would name it, the status label and a one-line note. The four labels mean the following.

- **[Implemented]**: built end to end. The backend and logic exist and a web screen uses them, or it is a backend-only job by design.
- **[Partial]**: some of it exists. The note says what is missing, for example "backend only, no screen calls it".
- **[Not implemented]**: planned, seeded, commented out or only mentioned. It must not be described as working.
- **[Unverified]**: the code could not confirm it. The note says what could not be confirmed.

The workflow chapters (46 to 55) follow one business process across several modules, so many of their rows restate a feature that already has a row in a module chapter. They are listed separately so that each workflow can be judged on its own, and the totals in Section 64.5 report modules, workflows and cross-cutting chapters separately to avoid counting the same feature twice. No application was run while the handbook was written, so every label comes from reading the code.

64.2 Module chapters

64.2.1 Chapter 28 — Patients

Chapter	Feature	Status	One-line note
28	Register a patient	[Implemented]	Two-step dialog, phone unique per org, P-number, optional first appointment
28	Check a phone number before saving	[Partial]	Endpoint and hook exist; no component calls them
28	Browse, search and filter the patient list	[Implemented]	Card + table; UI offers only search, gender, status
28	Open the patient detail page	[Implemented]	Tabs gated by permission and feature; out-of-scope = 404
28	Edit patient details	[Implemented]	Audited diff
28	Activate or deactivate a patient	[Implemented]	status_history row; does not block booking
28	Blacklist a patient	[Partial]	API only (PUT); displayed on Overview; blocks nothing
28	Delete a patient permanently	[Partial]	Fails (500) when billing/lab rows exist (ON DELETE RESTRICT)
28	Import patients from a spreadsheet	[Implemented]	parse → dry run → batches of 500; error report CSV
28	Export the patient list	[Implemented]	10,000-row cap, X-Export-Truncated, audit row
28	Appointments tab	[Implemented]	Appointments + walk-in queue visits

Chapter	Feature	Status	One-line note
28	Visits and clinical history	[Implemented]	GET history; prices stripped without patients.price.view
28	Per-location visit statistics	[Not implemented]	patient_visits never written; endpoint always empty
28	Patient change history	[Implemented]	admin.audit.view; values redacted per permission
28	Dental info	[Implemented]	One row per patient, upsert
28	Vitals	[Implemented]	Append-only, BMI computed
28	Teeth chart (FDI)	[Implemented]	Append-only snapshots; feature off by default
28	Record a treatment from the patient page	[Implemented]	Needs clinical.edit + price.view to see button
28	Patient treatment plans	[Partial]	Backend only (see ch33)
28	Prescriptions tab and legacy quick-Rx	[Implemented]	Legacy patient_prescriptions table still writable
28	Signed consents	[Implemented]	Signature pad, PNG data URL stored in record
28	Documents (Files tab)	[Implemented]	Upload via storage provider, then record
28	Communications log	[Partial]	CrmTab component exists but not mounted
28	Recalls	[Partial]	Backend only; no reminder job
28	Referrals	[Partial]	Backend create/list only; CrmTab unmounted
28	Feedback	[Partial]	Backend create/list only; CrmTab unmounted
28	Insurance	[Partial]	Backend only; feature off by default
28	Billing tab	[Implemented]	Embedded billing module (ch34)
28	Patient portal / CRM feature keys	[Not implemented]	Retired feature keys
28	Patient scope (own/location/org)	[Implemented]	patientScope + PatientScopeGuard

30 rows: 19 Implemented, 9 Partial, 2 Not implemented, 0 Unverified. Patients is the largest module. The core record is built end to end: registration, list, detail tabs, edit, import, export, audit history, dental info, vitals, teeth chart, signed consents and documents. The Partial rows share one pattern: the backend exists but no screen reaches it (phone pre-check, blacklist, recalls, referrals, feedback, insurance, communications log, patient treatment plans). Permanent delete is Partial because it fails with a generic error once billing or lab rows exist. The two Not implemented rows are the per-location visit statistics, which nothing ever writes, and the retired patient portal and CRM feature keys.

64.2.2 Chapter 29 — Appointments

Chapter	Feature	Status	One-line note
29	See bookings and walk-ins in one list	[Implemented]	UNION of appointments and walk-in queue entries, server paging, scoped by appointments.view
29	Browse the calendar by doctor	[Implemented]	FullCalendar premium resource views; drag/resize only scheduled/confirmed; weekly hours shaded
29	See the KPI strip	[Implemented]	Month of selected date; walk-ins and avg wait come from the queue
29	Book an appointment	[Implemented]	Ordered checks plus DB exclusion constraint no_doctor_overlap → DOCTOR_DOUBLE_BOOKED; daily limit default 50

Chapter	Feature	Status	One-line note
29	Check a time slot live and get a suggested time	[Implemented]	POST /appointments/validate, 500 ms debounce, same-day suggestion, up to 50 tries
29	Override booking rules	[Partial]	API flag and appointments.override_rules exist; no screen sends it
29	Reschedule or edit a booking	[Implemented]	Only scheduled/confirmed (APPOINTMENT_LOCKED); emergency switch change on edit is not saved
29	Confirm a booking	[Implemented]	scheduled → confirmed
29	Cancel a booking (with reason codes)	[Partial]	API stores patient_left/other and text; UI sends neither and has no confirm dialog
29	Mark a no-show (by hand or automatically)	[Implemented]	Manual action plus 60 s job using max_late_min
29	Check a patient in (hand-over to the queue)	[Implemented]	QueueOrchestrator.checkInAppointment; detailed in Chapter 30
29	Convert a late booking to a walk-in	[Implemented]	Works; UI rarely reaches it because the no-show job removes the Check in action
29	Respect availability overrides and leave when booking	[Not implemented]	Tables exist in Employees; booking, validation and calendar ignore them
29	Reminders, recurring bookings and online booking	[Not implemented]	Feature keys commented out; notifications are in-app only

14 rows: 10 Implemented, 2 Partial, 2 Not implemented, 0 Unverified. Booking is solid: the list, calendar, live slot check, double-booking constraint and no-show job all work. The gaps are rules the booking engine does not consult (availability overrides and leave), the override-booking flag that no screen sends, and cancel reasons that the API supports but the screen never sends. Reminders, recurring and online booking are not built.

64.2.3 Chapter 30 — Queue

Chapter	Feature	Status	One-line note
30	Open a doctor's queue for the day	[Implemented]	Automatic on arrival or on opening My queue; POST /queue/sessions has no web caller
30	Start, pause and resume the queue	[Implemented]	Pause needs a reason
30	Close the day	[Implemented]	Every remaining patient needs a cancel/transfer decision
30	Switch between smart queue and first come, first served for today	[Implemented]	queue.entry.reprioritize; Admin only by default
30	Doctor-wise tokens	[Implemented]	Counter on the session row, gap-free; prefix from assignment or name
30	Check in a booked patient	[Implemented]	NOT_TODAY, LATE_LIMIT_EXCEEDED, ALREADY_CHECKED_IN guards
30	Register a walk-in	[Implemented]	Quick-creates patient P-xxxxx; ACTIVE_WITH_OTHER_DOCTOR needs force
30	Call the next patient (smart and first-come rules)	[Implemented]	Pure selection engine; 14-scenario spec in both modes

Chapter	Feature	Status	One-line note
30	Select a specific patient	[Implemented]	Engine suggestion recorded; optional required reason
30	Call again (recall)	[Implemented]	Event only, no state change
30	Undo a call	[Implemented]	Back to waiting, same place in line
30	Skip a patient who did not come	[Implemented]	Reception or the session's doctor; also the call-timeout job
30	Rejoin a skipped patient	[Implemented]	end / original_position / manual policy
30	Send for a test and return from the test	[Implemented]	on_hold keeps the token; returns ahead of walk-ins in smart mode
30	Transfer a patient to another doctor	[Implemented]	New token, keeps arrival place; from waiting or in consultation
30	Give urgent priority	[Implemented]	Reason required; no way to remove it
30	Start the consultation	[Implemented]	Creates the patient_encounters row and opens the workspace
30	Complete the visit (and bill it automatically)	[Implemented]	ChargeSourcesService.syncForEncounter in the same transaction (needs billing.invoices)
30	Stop a consultation (abort)	[Implemented]	Appointment cancelled with code other
30	Remove a patient from the queue (cancel)	[Implemented]	Appointment cancelled with code patient_left
30	Force-complete a visit for the doctor	[Implemented]	queue.doctor.any; FORCE_COMPLETED event
30	Print a token ticket	[Implemented]	Pop-up print with a people-ahead estimate
30	Reception board, doctor screen and TV display (reads)	[Implemented]	TV public, no patient names; GET /queue/me unused by the web app
30	Live updates on every screen	[Implemented]	Outbox, relay, Redis, Socket.IO rooms; 10 s polling fallback
30	Safe retries and stale screens (Idempotency-Key and If-Match)	[Implemented]	Keys kept 24 h; VERSION_CONFLICT on a stale version
30	Background jobs	[Implemented]	No-show 60 s, call timeout 15 s, day-end 5 min, clean-up 03:15 server time
30	Queue settings (organization default and location override)	[Implemented]	Only smart mode is fixed per session; other values apply within about 60 s
30	Patient visit history	[Implemented]	Organization-wide, up to 200 rows, patients.view without scope
30	Next-patient preview	[Partial]	Backend only, no screen
30	Queue history (events log)	[Partial]	Backend only (admin.audit.view), no screen
30	Manage the TV display	[Not implemented]	queue.display.manage defined but never checked

31 rows: 28 Implemented, 2 Partial, 1 Not implemented, 0 Unverified. The queue is the most complete module in the Platform: 28 of 31 rows are Implemented. Check-in, walk-ins, smart or first-come calling, tests, transfers, urgent priority, completion with automatic billing, live updates, safe retries and the background jobs all work. The Partial rows are two backend reads with no screen (next-patient preview and queue history). The one Not implemented row is managing the TV display, whose permission is defined but never checked.

64.2.4 Chapter 31 — Consultation

Chapter	Feature	Status	One-line note
31	Start a consultation (encounter created)	[Implemented]	ensureForQueueEntry inside the start-consultation transaction; idempotent per queue entry
31	Re-open a running consultation	[Implemented]	GET /encounters/by-queue-entry/:id; conduct scope own = session doctor
31	See the patient at a glance	[Implemented]	Vitals, 4 previous visits, 3 signed prescriptions with reuse
31	Allergy and medical alerts	[Partial]	Client-side chips from free-text allergies/medical history and latest vitals flags; no structured allergy list or drug check
31	Write the visit notes (auto-saved)	[Implemented]	PATCH /encounters/:id, 800 ms debounce, 4000-char fields; not locked after completion
31	Record the treatments done today	[Implemented]	POST /encounters/:id/treatments; catalog procedure or typed name; no edit/delete
31	Patient record slide-over	[Implemented]	Reuses patient tabs; amounts hidden
31	Prescription inside the consultation	[Implemented]	Draft on first complete line, autosave 1 s, Sign & print; panel not gated by permission/feature
31	Order lab work from the consultation	[Not implemented]	No lab entry point; lab orders have no encounter link
31	Complete the visit	[Implemented]	Flush saves, PATCH complete; appointment completed; charges in same transaction
31	Automatic charges on completion	[Implemented]	ChargeSourcesService.syncForEncounter; only when billing.invoices enabled; needsPricing fallback
31	Void charge of a removed treatment	[Partial]	handleRemoved exists but unreachable: no treatment delete, sync runs once at completion
31	Keep prices away from doctors	[Implemented]	HideTreatmentPricesInterceptor plus VisibleIf on patients.price.view
31	Encounter and treatment records outside the consultation	[Implemented]	GET /encounters/:id, GET/POST /treatments with patient scope
31	SOAP notes and note templates	[Not implemented]	Only encounter text fields; phrase chips fixed in code

15 rows: 11 Implemented, 2 Partial, 2 Not implemented, 0 Unverified. The consultation workspace covers notes with autosave, treatments, the patient slide-over, prescribing, completion and automatic charges. Allergy alerts are Partial because they are only text chips with no structured list or drug check. Voiding the charge of a removed treatment is Partial because no screen can remove a treatment. Ordering lab work from the consultation and structured SOAP notes are not built.

64.2.5 Chapter 32 — Prescriptions

Chapter	Feature	Status	One-line note
32	Browse and filter prescriptions	[Implemented]	GET /prescriptions and /stats, location + patient scope; "this month" uses server clock
32	Write a prescription and save as draft	[Implemented]	POST /prescriptions; RX-{year}-{5 digits} per org, unique platform-wide
32	Fill from a preset	[Implemented]	Builder replaces list; workspace appends and offers advice
32	Sign (finalize) a prescription	[Implemented]	DRAFT/AMENDED only, needs a medicine; best-effort PDF upload
32	Amend a signed prescription	[Implemented]	PATCH on FINALIZED gives AMENDED, version+1, snapshot in history
32	Add/edit/remove one medicine line	[Partial]	Backend only; screens send the full list
32	Duplicate a prescription	[Partial]	Backend only; stores user id as doctor_id (FK to employees), unverified
32	Change history	[Partial]	Rows written; no viewer; read requests undeclared relation changedByEmployee
32	PDF with Urdu (Nastaliq)	[Implemented]	pdf-lib + fontkit-shaped Noto Nastaliq outlines; single page; drafts printable
32	Print layouts (prescription templates)	[Implemented]	Default per location/org; UI creates org-wide only; orientation/logo/reg no. not printed
32	Doctor signatures	[Implemented]	Keyed by user id; delete and by-doctor read have no screen
32	Medicine directory incl. search and seed	[Implemented]	Soft deactivate; seed adds 10 medicines only when empty
32	Prescription presets (treatment-plan-templates)	[Implemented]	About 40 PresetCategory values; not clinical plan templates
32	Prescriptions by patient / visit / appointment	[Partial]	By-appointment lookup has no screen
32	Legacy quick prescriptions (patient_prescriptions)	[Implemented]	Still readable/writable from patient tab; separate numbering, no link to new table

15 rows: 11 Implemented, 4 Partial, 0 Not implemented, 0 Unverified. Prescriptions are largely built, including the Urdu PDF, presets, print layouts, signatures and the medicine directory. The Partial rows are backend-only operations (single-line edits, duplicate, change history, lookup by appointment). Two of them have likely defects that were read from code but not run: the history read and the duplicate action.

64.2.6 Chapter 33 — Treatment plans

Chapter	Feature	Status	One-line note
33	Browse and filter plan templates	[Implemented]	Card grid, search + category; fees stripped without price.view
33	Create a plan template	[Implemented]	Up to 50 procedures in jsonb; not audited
33	Edit a plan template	[Implemented]	Replaces procedure list
33	Hide or show a plan template	[Implemented]	is_active switch
33	Delete a plan template	[Implemented]	Hard delete; treatments.plan_template_id SET NULL

Chapter	Feature	Status	One-line note
33	Starter templates for a new clinic	[Implemented]	12 defaults seeded when org has zero templates (re-seeds if all deleted)
33	Use a template when recording a treatment	[Implemented]	Prefills name/cost, tags treatment; button needs price.view
33	Create a patient treatment plan	[Partial]	Backend only; web hooks unused
33	Update a plan (status, acceptance, costs)	[Partial]	Backend only; free status changes
33	Add items to a plan	[Partial]	Backend only
33	Track progress of plan items	[Partial]	Backend only; nothing automatic
33	Start a patient plan from a template	[Not implemented]	Only UI text/comment claim it
33	Turn plan items into charges	[Partial]	Backend only; no web caller
33	Plan billing summary	[Partial]	Endpoint + API fn + hook; no component renders it

14 rows: 7 Implemented, 6 Partial, 1 Not implemented, 0 Unverified. Plan templates are fully built. Patient treatment plans are the weak half: creating, updating and tracking a patient plan, turning its items into charges and the plan billing summary all exist only in the backend. Starting a patient plan from a template is not built, although some screen text says it is.

64.2.7 Chapter 34 — Billing

Chapter	Feature	Status	One-line note
34	Keep a service catalog	[Implemented]	Org-level services, audited, never deleted
34	Set location and doctor price overrides	[Implemented]	PATCH override and resolved-price endpoints unused by UI
34	Automatic charges from a completed consultation	[Implemented]	syncForEncounter in queue completion tx; removed treatment → VOID or Review
34	Add a manual charge	[Implemented]	Catalog or Other; price policy enforced
34	Create charges from a treatment plan	[Partial]	Backend only, no web call
34	Edit an unbilled charge (price policy)	[Implemented]	Catalog price needs billing.price.override
34	Void an unbilled charge	[Implemented]	Reason 5–300 chars
34	Manage discount types	[Implemented]	Code field not on screen
34	Discount one / many / custom	[Implemented]	Range via AccessContext.decide; custom needs note
34	Ready-to-bill list and billing desk	[Implemented]	Outstanding omits partly paid invoices
34	Collect a payment	[Implemented]	One tx, lock order, idempotent clientRequestId
34	Choose an allocation strategy	[Implemented]	Oldest first (FIFO) and Manual

Chapter	Feature	Status	One-line note
34	Hold and apply advance	[Implemented]	Advance = amount − allocated − refunded
34	Unapply a payment	[Partial]	Backend only
34	Payment accounts and methods	[Implemented]	Auto Cash default; default/last account cannot be deactivated
34	Number invoices, receipts and refunds	[Implemented]	CODE-INV/REC/RFD-YYYY-NNNNN, gapless
34	Issue an invoice	[Implemented]	At collection/advance apply or manual; due days default 7
34	Exclusive sales tax	[Implemented]	Snapshot of location_settings.tax_rate
34	Browse invoices and payment status	[Implemented]	Stored ISSUED/CANCELLED; computed payment status
34	Edit an issued invoice	[Implemented]	Preview in rolled-back tx; revision conflict check
34	Cancel an invoice	[Implemented]	Only when nothing paid; unapply has no UI
34	Refunds	[Implemented]	Always from advance; invoice refund may carry an edit
34	Patient billing summary and statement	[Implemented]	Ledger-based statement
34	Show dues at check-in	[Implemented]	Setting enforced only in web app
34	Daily collection	[Implemented]	Payments page
34	Cash close	[Implemented]	Advisory lock; immutable; variance flag
34	Printing	[Implemented]	Browser print A4/80 mm; English labels, Urdu typed text
34	Billing settings	[Implemented]	Document code locks after first number
34	Expenses	[Implemented]	Edit API-only; no delete confirm; hard delete
34	Credit packages and patient credits	[Partial]	Backend only; expiry cron 01:00 server date
34	Billing activity / audit log	[Implemented]	User/action filters not on screen
34	Nightly invariant check	[Implemented]	02:30 server time, I1–I10, log only
34	Doctor revenue and financial summary	[Implemented]	Daily summary unused; details in Ch 36
34	Commission snapshot on charges	[Implemented]	Cross-reference to Ch 35

34 rows: 31 Implemented, 3 Partial, 0 Not implemented, 0 Unverified. Billing is the deepest module and is almost entirely Implemented: catalog, automatic and manual charges, discounts, collection, advance, invoices, tax, edits, cancellation, refunds, cash close, printing, expenses and the nightly checks. Three Partial rows are backend-only: unapplying a payment, credit packages and plan-based charges. Because unapply has no screen, an invoice that has any payment cannot be cancelled from the web app.

64.2.8 Chapter 35 — Commissions

Chapter	Feature	Status	One-line note
35	Set up or change a compensation plan	[Implemented]	Versioned per assignment, GiST no-overlap, compensation.manage
35	See the current plan and version history	[Implemented]	Compensation tab in employee detail
35	Edit service-wise overrides (commission rules)	[Implemented]	Replace rules of current version; service > category > default
35	Snapshot the rate on each visit charge	[Implemented]	CONSULTATION and TREATMENT charges only; backend by design
35	Earn commission when patient money is applied	[Implemented]	Cumulative target, NET/GROSS, tax pro-rating, fixed amount; backend by design
35	Earn commission on lab work	[Implemented]	Lab-line commissions, net of lab cost; backend by design
35	Preview the doctor share while collecting payment	[Implemented]	doctorShare gated by earnings.all.view and billing.commissions
35	See every doctor's earnings	[Implemented]	/billing/doctor-earnings; GET /commissions has no screen
35	See my own earnings	[Implemented]	/my-earnings, filtered by signed-in user
35	See doctor revenue	[Implemented]	/billing/doctor-revenue, reports.financial.view, not commission
35	Settle commission through payroll	[Implemented]	lockUnsettled, create, markPaid, release
35	Protect the ledger (append-only trigger, I10)	[Implemented]	Trigger in migration; nightly I10 covers charges only
35	Choose a commission plan when adding an employee	[Implemented]	compensation object on POST /employees
35	Manual commission adjustments	[Not implemented]	MANUAL source type reserved, never written

14 rows: 13 Implemented, 0 Partial, 1 Not implemented, 0 Unverified. Commissions work end to end in the backend, which is by design: rates are snapshotted on charges, commission is earned as patient money is applied, lab lines are included and the ledger is protected by a trigger. Screens exist for plans, earnings and the doctor-share preview. Manual commission adjustments are the only gap: the source type is reserved and never written.

64.2.9 Chapter 36 — Dashboard and reports

Chapter	Feature	Status	One-line note
36	See the home dashboard	[Partial]	Real data; money tiles also need billing.financial_summary; mock dashboard.api.ts and ~10 widgets unused
36	Read the finance overview	[Implemented]	/reports/finance/overview, gated finance.overview.view, 12-month trend, warnings

Chapter	Feature	Status	One-line note
36	Read the profit and loss statement	[Implemented]	Collected vs billed basis; costs by own period; payroll base salary only
36	Read the clinic overview	[Partial]	Default Doctor role (own scope) sees sidebar item but API refuses
36	Browse the report library	[Implemented]	37 reports, filtered by permission and own-scope doctor filter
36	Run a report on screen	[Implemented]	5,000 row cap, scope resolution, rupees
36	Download a report as Excel or PDF	[Implemented]	50,000 rows, PDF max 5,000, audit EXPORT row
36	The 37 reports	[Implemented]	15 financial, 22 clinical; 9 categories
36	Daily financial summary	[Partial]	Backend only; /financial-summary redirects to /dashboard

9 rows: 6 Implemented, 3 Partial, 0 Not implemented, 0 Unverified. The finance overview, profit and loss, the 37-report library and Excel and PDF export are Implemented. The home dashboard is Partial because roughly ten unused widgets still read mock data and the money tiles need an extra feature. The clinic overview is Partial because the default Doctor role sees it in the sidebar but the API refuses that scope. The daily financial summary has no screen.

64.2.10 Chapter 37 — Employees

Chapter	Feature	Status	One-line note
37	Browse and filter the employee list	[Implemented]	Assignments at active location, server paging
37	Add an employee	[Implemented]	Platform-unique phone checked; CNIC only by DB index
37	View an employee	[Implemented]	Scoped to active location
37	Edit an employee	[Implemented]	Duplicate phone/CNIC on edit gives generic 500
37	Activate or deactivate an employee	[Implemented]	Assignment is_active; employees.is_active never written
37	Delete an employee permanently	[Implemented]	Refused if other locations or referenced rows
37	Give an employee a staff login	[Implemented]	Employee form and User Admin; PATCH portal-access has no screen
37	Set or update the salary	[Implemented]	Adds compensation plan version from today
37	Set the weekly schedule	[Implemented]	7 days, slot 15/30/45/60; cross-location check never called
37	Record availability overrides	[Partial]	CRUD works; booking ignores them
37	Manage leave requests	[Partial]	Manager-only; approved leave ignored by booking and payroll
37	See the leave summary	[Partial]	Usage tally, no entitlements
37	List employment roles	[Implemented]	GET /employment-roles, employees.view
37	Assign an existing employee to another location	[Not implemented]	DTOs exist, no endpoint
37	Attendance	[Not implemented]	Only in hr.leave description

15 rows: 10 Implemented, 3 Partial, 2 Not implemented, 0 Unverified. Employee records, schedules, salary, staff logins and deletion are built. Availability overrides, leave requests and the leave summary are Partial because they can be recorded but are never read by booking or payroll. Assigning an existing employee to another location and attendance tracking are not built.

64.2.11 Chapter 38 — Payroll

Chapter	Feature	Status	One-line note
38	Generate payroll for a month	[Implemented]	Base + commission, settlement lock, future months refused
38	Month-end automatic generation	[Implemented]	Cron 0 5 1 * * server time; comment says 00:05
38	Create a payroll record by hand	[Implemented]	No assignment or future-month check; commission 0
38	View payroll for a month or employee	[Implemented]	month/year validation 400
38	Mark a record as paid	[Implemented]	Also pays settlement; records payout only
38	Put a record on hold	[Partial]	Endpoint exists; no button, edit dialog status works
38	Edit a payroll record	[Implemented]	Amount is total; floor at commission
38	Delete a payroll record	[Implemented]	Releases pending settlement; refused if commission paid
38	Export a month as CSV	[Partial]	Endpoint only, no button
38	Payslip PDF	[Partial]	Endpoint only, no button
38	Deductions, tax and bank files	[Not implemented]	Deductions fixed at 0

11 rows: 7 Implemented, 3 Partial, 1 Not implemented, 0 Unverified. Payroll generation, month-end automation, editing, deleting and marking paid all work. Putting a record on hold, the CSV export and the payslip PDF are Partial: the endpoints exist but no button calls them. Deductions, tax and bank files are not built, so net pay always equals gross pay.

64.2.12 Chapter 39 — Inventory

Chapter	Feature	Status	One-line note
39	Browse stock and see stock health	[Partial]	Works; no sidebar entry
39	Add, edit and deactivate an item	[Partial]	Initial stock uses movement path (FK risk)
39	Manage categories	[Partial]	API only; no screen creates; seedDefaults never called
39	Adjust stock and keep the movement history	[Unverified]	performedBy gets user id but FK is to employees
39	Manage suppliers	[Partial]	Works; no sidebar entry
39	Raise and track a purchase order	[Unverified]	createdBy FK risk; poNumber global unique vs per-org sequence
39	Receive goods into stock	[Unverified]	Same FK risk via applyMovement
39	Define consumption templates	[Partial]	CRUD works; nothing uses templates
39	Record materials used on a patient	[Not implemented]	ConsumptionRecorder not placed on any screen
39	Daily low-stock and expiry alerts	[Partial]	08:00 server-time cron only logs

10 rows: 0 Implemented, 6 Partial, 1 Not implemented, 3 Unverified. Inventory has no Implemented row. Stock, suppliers, purchase orders and receiving work through the API and have screens, but the module has no sidebar entry. Three rows are Unverified because the code passes a login id into columns that are foreign keys to employees, and this was never run to see whether the writes fail. Recording materials used on a patient has no screen, and the daily alert job only writes to the log.

64.2.13 Chapter 40 — Dental lab (clinic side)

Chapter	Feature	Status	One-line note
40	Keep the platform lab directory	[Implemented]	Super admin; external and in-house labs; in-house lab auto-creates its active link
40	Manage the staff of a lab	[Implemented]	Super admin adds lab members by account email; deactivate, never delete
40	Manage a lab's services, options and prices	[Implemented]	Super admin only; price windows, void, history; lab cannot edit its own catalog
40	Service categories	[Partial]	Create/update endpoints exist, no screen; six dental categories seeded
40	Add a lab to the clinic's lab list	[Implemented]	Invited link; directory shows active external and own in-house labs
40	Set payment terms and patient data level	[Implemented]	full_name or code_only per link; 0-365 days
40	Activate, suspend and reactivate a lab link	[Implemented]	Reason to suspend; only suspending side reactivates
40	Read the lab price list	[Implemented]	Negotiated else list price; margin; AnyMember reads
40	Get a price quote	[Implemented]	LabPriceResolver; quote default day is Asia/Karachi
40	Map a lab service to a clinic treatment	[Implemented]	One default per treatment; needs lab.catalog.manage
40	Save and reuse order templates	[Implemented]	Private or shared; /use endpoint not called by web
40	Create a lab order draft	[Implemented]	Optional idempotency key; pins form and workflow versions
40	Start from a template or repeat a past order	[Implemented]	from-template and repeat endpoints
40	Submit an order to the lab	[Implemented]	One transaction; Idempotency-Key required; price snapshot; revision 1; magic link
40	List and search lab orders	[Implemented]	Scoped by lab.orders.view own/location/org
40	Open an order and print the work order	[Implemented]	QR /o/:orderId; no prices on work order
40	Cancel, hold, resume or reopen an order	[Implemented]	Workflow driven; cancel from draft/submitted/accepted only
40	Revise a submitted order	[Implemented]	New revision, re-priced on original day
40	Talk to the lab: messages and clarifications	[Implemented]	Clinic replies; only lab asks

Chapter	Feature	Status	One-line note
40	Decide the lab's extra charge requests	[Implemented]	Approved item charge adds to lab cost
40	Attach and download files	[Implemented]	50 MB; 10-minute signed URL; patient_shared not surfaced
40	Send, copy and revoke magic links	[Implemented]	Hash only in DB; 24 h default TTL
40	Update on behalf of a lab that does not answer	[Implemented]	Needs lab.orders.create and lab.links.manage; via proxy
40	Follow production and try-in rounds	[Implemented]	Stages from template; rounds; record try-in; decide
40	Approve, reject or deliver finished work	[Implemented]	Delivery starts 12-month warranty
40	Cancel one item	[Partial]	Endpoint only, no screen
40	Ask for a remake and decide who pays	[Implemented]	Warranty claim auto; charges by responsible party
40	Receive a shipment and report a lost one	[Implemented]	QR token; only lab.orders.view needed
40	Send impressions and models to the lab	[Partial]	Endpoint and unused web API function
40	Notifications about lab work	[Partial]	4 seeded events never emitted; patient/lab-contact channels dev-log only

30 rows: 26 Implemented, 4 Partial, 0 Not implemented, 0 Unverified. The clinic side of lab work is broad and mostly Implemented: directory, links, price lists, orders, revisions, messages, files, magic links, production rounds, delivery, warranties, remakes and shipments. The Partial rows are cancel-one-item, sending impressions to the lab, service categories and lab notifications (four seeded events are never emitted).

64.2.14 Chapter 41 — Lab portal

Chapter	Feature	Status	One-line note
41	Sign in as a lab member	[Implemented]	labMemberships in login response; lab-only users land on /lab-portal
41	Work in one lab or switch between labs	[Implemented]	x-lab-id header; LabPortalGuard
41	See the lab dashboard	[Implemented]	Three counters
41	Read the order inbox	[Implemented]	Drafts hidden; patient ref by data level
41	Open an order	[Implemented]	Lab mapper: no patient price
41	Accept an order with an expected date	[Implemented]	Lab admin; creates stages and vendor bill draft
41	Reject an order	[Implemented]	Order-level reject with reason
41	Ask the clinic a question	[Implemented]	Moves order to clarification_needed

Chapter	Feature	Status	One-line note
41	Change the expected date	[Implemented]	Lab admin; reason; accepted/in progress/partially ready/on hold
41	Write a note to the clinic	[Implemented]	Notes only; no status change
41	Acknowledge a revision	[Implemented]	Idempotent
41	Ask for an extra charge	[Partial]	Backend only; no portal form
41	Upload files to an order	[Partial]	Backend only; no portal control
41	Download files the clinic shared	[Implemented]	lab_shared only; 10-minute URL
41	Run the work queue: start, QC pass, QC fail	[Implemented]	Lab admin and technician
41	Assign a technician	[Implemented]	Lab admin; unassign endpoint has no button
41	Ship a try-in or finished work to the clinic	[Implemented]	SHP number, QR slip
41	Receive what the clinic sent	[Implemented]	Portal always records full qty, condition ok
41	Report a shipment lost	[Implemented]	Open lines become missing
41	Dispute a remake request	[Implemented]	Lab admin; only requested status
41	Accept, suspend or reactivate a clinic	[Implemented]	Lab admin; shared transition code
41	Manage the lab's team	[Implemented]	Lab admin; no self change
41	View and edit the lab profile	[Implemented]	Contact fields only
41	See bills, payments and statements	[Implemented]	Read only; drafts hidden
41	Answer an order from a magic link, without logging in	[Implemented]	Accept, reject, ask; public routes; throttled
41	Open an order from the printed QR code	[Implemented]	/o/:orderId resolves lab or clinic
41	Be told about new work	[Partial]	In-app rows exist; no inbox in portal layout found

27 rows: 24 Implemented, 3 Partial, 0 Not implemented, 0 Unverified. The lab portal covers sign-in, order inbox, accept, reject, clarify, production queue, shipping, team and profile, and no-login magic-link answers. Partial rows are charge requests and file upload (backend only) and the new-work notification (rows exist but the portal has no inbox).

64.2.15 Chapter 42 — Lab finance

Chapter	Feature	Status	One-line note
42	Estimate and invoice lab work	[Implemented]	draft to issued, LINV numbers, frozen patient prices
42	Browse lab invoices and see one	[Implemented]	overdue computed in location timezone
42	Void estimate or unpaid invoice	[Implemented]	needs billing.invoice.void too
42	Collect lab payment	[Implemented]	Idempotency-Key required, FIFO auto-allocation
42	Use patient advance	[Implemented]	also automatic at issue
42	Reverse lab payment	[Implemented]	needs billing.refund.issue; refused after refund
42	Credit note	[Implemented]	releases newest allocations to advance
42	Refund advance	[Implemented]	needs billing.refund.issue
42	Payment reminder	[Partial]	queued to outbox, patient audience off by default, nothing delivered
42	Order and patient billing summaries	[Implemented]	read panels
42	Lab billing settings	[Implemented]	tax bp and due days; needs admin.settings.manage
42	Doctor commission on lab work	[Implemented]	collected basis net of lab cost; skipped when billing.commissions off
42	Receivables journals (GL)	[Partial]	written for all lab money, no screen or report reads them
42	Vendor bill drafts at lab accept	[Implemented]	effect create_vendor_bill_draft plus item listener
42	Review, approve, void vendor bill	[Partial]	PATCH bill and add line exist in API only
42	Pay a lab	[Implemented]	allocations over approved bills, journals
42	Use lab advance on bills	[Partial]	endpoint and api client exist, no UI button
42	Reverse a payment to a lab	[Implemented]	mirrors journals
42	Lab statements and reconciliation	[Implemented]	manual line entry, no upload
42	Payables reports	[Implemented]	aging, monthly cost, case profit
42	Lab portal payables (read)	[Implemented]	drafts excluded

21 rows: 17 Implemented, 4 Partial, 0 Not implemented, 0 Unverified. Lab finance has full estimate-to-invoice, collection, advance, credit note, refund, vendor bill, payment and statement flows. Partial rows are backend-only vendor bill editing and lab advance use, payment reminders that are queued but never delivered, and journals that are written but never read by any screen.

64.2.16 Chapter 43 — Workflow engine (Config Studio)

Chapter	Feature	Status	One-line note
43	Manage form schemas	[Implemented]	super admin only
43	Edit form version and preview	[Implemented]	issues do not block saving
43	Publish and retire form version	[Implemented]	service-enforced immutability
43	Clinics read current forms	[Implemented]	AnyMember endpoints
43	Manage workflow templates	[Implemented]	
43	Edit workflow draft and test run	[Implemented]	60 states, 300 transitions
43	Publish and retire workflow version	[Implemented]	rows materialised at publish
43	Manage stage templates	[Implemented]	not versioned, no delete
43	Order state machine from pinned version	[Implemented]	pinWorkflowVersion, transition referee
43	Compare versions and activity	[Implemented]	activity tab on forms only
43	Specialties and organization enablement	[Partial]	endpoints exist, Studio only lists
43	Dental seed	[Implemented]	insert-if-missing at boot

12 rows: 11 Implemented, 1 Partial, 0 Not implemented, 0 Unverified. Config Studio is a super admin tool for form schemas, workflows and stage templates. Everything is Implemented except specialties and organization enablement, where endpoints exist but the Studio only lists.

64.2.17 Chapter 44 — Notifications

Chapter	Feature	Status	One-line note
44	See notifications in the header bell	[Implemented]	30 s polled badge, 20 newest items, own inbox only
44	Mark notifications as read	[Implemented]	one or all; no mark-unread or delete
44	Raise a notification event	[Implemented]	enqueue in caller transaction; 6-step decision order
44	Deliver messages in the background	[Implemented]	LISTEN + 2 s poll, skip-locked claim, 5 attempts; in-app only
44	Choose who is notified (switches and rules)	[Implemented]	audited; mandatory rules locked
44	Set quiet hours	[Partial]	saved but never holds in-app messages
44	Edit message wording in several languages	[Implemented]	en / ur / roman_ur; placeholder validation
44	Review the message log	[Implemented]	filters by status, channel, text
44	Patient messaging opt-outs and language	[Partial]	backend works; UI card commented out
44	Notify through a workflow transition	[Partial]	notify effect exists; no shipped transition found using it
44	Send to patients by WhatsApp, SMS or email	[Not implemented]	no patient channel; dev-log stand-in only
44	Appointment reminders	[Not implemented]	no event, no job

12 rows: 7 Implemented, 3 Partial, 2 Not implemented, 0 Unverified. The in-app bell, message rules, templates and the delivery pipeline work. Quiet hours, patient opt-outs and workflow-driven notification are Partial. Sending to patients by WhatsApp, SMS or email and appointment reminders are not built.

64.2.18 Chapter 45 — Subscriptions and onboarding

Chapter	Feature	Status	One-line note
45	Sign up and create a clinic	[Implemented]	one transaction; 14-day Professional trial
45	Verify email and resend link	[Partial]	works, but nothing is gated by verification
45	Complete the setup wizard	[Implemented]	3 steps; owner only; finish does not check the other steps
45	Trial, plans and plan limits	[Implemented]	plans in code; PLAN_LIMIT on locations and logins
45	Read-only mode after expiry	[Implemented]	402 SUBSCRIPTION_EXPIRED; 30 s cache
45	See subscription status and warnings	[Implemented]	derived status; AccountBanner
45	Record a manual payment	[Implemented]	super admin; sets plan to paid plan
45	Change plan, dates or limits	[Implemented]	super admin; audited
45	Edit the payment instructions	[Implemented]	platform setting, 2000 chars
45	Create a clinic from the Organizations screen	[Partial]	plan selector sends values the DTO rejects

10 rows: 8 Implemented, 2 Partial, 0 Not implemented, 0 Unverified. Self-signup with a 14-day trial, plan limits, read-only mode after expiry, warnings and manual payments work. Email verification is Partial because nothing is gated by it. Creating a clinic from the Organizations screen is Partial because the plan selector sends values the API rejects.

64.3 Workflow chapters

64.3.1 Chapter 46 — Clinic onboarding

Chapter	Feature	Status	One-line note
46	Self-signup and clinic creation	[Implemented]	POST /auth/register creates user, clinic, MAIN location, trial
46	Email verification	[Partial]	Informational; nothing gated by email_verified_at
46	Setup wizard	[Implemented]	finish does not check the other steps
46	Super admin creates organization	[Partial]	Endpoint works; screen form sends values the API rejects; no owner/location created
46	Create owner login for a super-admin-created clinic	[Partial]	Only through User Admin after selecting the org; sequence deduced, not run
46	Entitle features	[Implemented]	Several key features are off by default
46	Owner enables features	[Implemented]	Locked when not entitled

Chapter	Feature	Status	One-line note
46	Roles	[Implemented]	Five system roles seeded
46	Locations	[Implemented]	Edit screen sends fields the API refuses
46	Theme and navigation	[Implemented]	Nav save bumps access_version; theme save does not

10 rows: 7 Implemented, 3 Partial, 0 Not implemented, 0 Unverified. The two ways to start a clinic are both traced. The self-signup path is built. The super admin path is Partial because the form sends rejected values and no owner login or location is created with the organization.

64.3.2 Chapter 47 — Staff onboarding

Chapter	Feature	Status	One-line note
47	Employee record, assignment and role	[Implemented]	POST /employees, phone unique platform-wide
47	Staff login (employee form route)	[Implemented]	Does not check plan login limit
47	Staff login (User Admin route)	[Implemented]	Checks login limit; links all active assignments
47	Weekly schedule	[Implemented]	Overlap helper across locations never called
47	Compensation plan	[Implemented]	Commission needs billing.commissions and consulting role
47	Assign an existing employee to a second location	[Not implemented]	No endpoint; Add Employee fails on duplicate phone
47	First sign-in and access	[Implemented]	Home page from role

7 rows: 6 Implemented, 0 Partial, 1 Not implemented, 0 Unverified. Adding staff works, with one gap: an existing employee cannot be assigned to a second location, because the only creation path refuses an existing phone number. The employee-form login route does not check the plan login limit, unlike User Admin.

64.3.3 Chapter 48 — Patient journey

Chapter	Feature	Status	One-line note
48	Patient registration	[Implemented]	Phone unique per organization
48	Appointment booking	[Implemented]	Validate then insert; DB blocks overlap
48	Check-in and token	[Implemented]	Same-day, within max lateness
48	Queue and call next	[Implemented]	Smart or first-come
48	Consultation and encounter	[Implemented]	Encounter at start-consultation
48	Treatments in the workspace	[Implemented]	Doctor sees no prices
48	Prescription in the workspace	[Implemented]	Optional
48	Lab order from the consultation workspace	[Not implemented]	Created from Lab orders page instead
48	Automatic charges on completion	[Implemented]	Only with billing.invoices enabled (default off)
48	Payment and invoice issue	[Implemented]	Invoice issued at collection
48	Receipt printing	[Implemented]	Browser print layouts; no server PDF

11 rows: 10 Implemented, 0 Partial, 1 Not implemented, 0 Unverified. The patient journey from registration to receipt is built. The only break is that no lab order can be started from the consultation workspace. Automatic charges depend on the billing.invoices feature, which is off by default.

64.3.4 Chapter 49 — Dental lab case

Chapter	Feature	Status	One-line note
49	Create and submit lab order	[Implemented]	Prices frozen at submit
49	Send magic link by message	[Partial]	Provider for contact channels is a logger
49	Copy magic link	[Implemented]	Link shown once
49	Lab accepts via link or portal	[Implemented]	Creates stages and vendor bill draft
49	Production stages (lab side)	[Implemented]	Portal login required
49	Try-in rounds	[Implemented]	No appointment automation
49	Shipment receive and lost	[Implemented]	QR token
49	Approve and deliver	[Implemented]	12-month warranty
49	Patient lab invoice	[Implemented]	Separate tables and LINV numbers
49	Vendor bill review and pay	[Partial]	Header edit and manual lines API only
49	Lab-line commission	[Implemented]	Backend only; lab cost netted

11 rows: 9 Implemented, 2 Partial, 0 Not implemented, 0 Unverified. The lab case workflow is built from order to vendor payment. Sending the magic link by message is Partial because the contact channel is a log stand-in, and vendor bill header edits are backend only.

64.3.5 Chapter 50 — Daily clinic closing

Chapter	Feature	Status	One-line note
50	Queue close day	[Implemented]	Decisions required for remaining patients
50	Automatic day-end close	[Implemented]	Only empty sessions; others alert only
50	No-show marking	[Implemented]	Manual or 60 s job; job writes no event
50	Daily collection	[Implemented]	Per payment account
50	Cash close	[Implemented]	Immutable record
50	Credit expiry job 01:00	[Implemented]	Server date
50	Invariant check 02:30	[Implemented]	Log only
50	Queue cleanup 03:15	[Implemented]	
50	Inventory alerts 08:00	[Partial]	Log only

9 rows: 8 Implemented, 1 Partial, 0 Not implemented, 0 Unverified. Closing the day is built: queue close, automatic day-end close, no-show marking, daily collection, cash close and the nightly jobs. The inventory alert job is Partial because it only logs.

64.3.6 Chapter 51 — Month-end payroll

Chapter	Feature	Status	One-line note
51	Month-end automatic generation	[Implemented]	05:00 on the 1st, server time

Chapter	Feature	Status	One-line note
51	Base plus commission	[Implemented]	Negative commission carried forward
51	Settlement lock	[Implemented]	
51	Manual generation	[Implemented]	Whole location
51	Edit and delete records	[Implemented]	Paid-out commission blocks delete
51	Put on hold	[Partial]	No button
51	Mark paid	[Implemented]	No money moves
51	Payslip PDF and CSV export	[Partial]	No buttons
51	Deductions and tax	[Not implemented]	

9 rows: 6 Implemented, 2 Partial, 1 Not implemented, 0 Unverified. Month-end payroll is built from automatic generation to mark-paid. On hold, payslip PDF and CSV export have no buttons, and deductions and tax are not built.

64.3.7 Chapter 52 — Invoice correction and refund

Chapter	Feature	Status	One-line note
52	Edit preview	[Implemented]	
52	Edit and revision	[Implemented]	Optimistic revision check
52	Commission adjustment on edit	[Implemented]	Backend only by design
52	Cancel unpaid invoice	[Implemented]	
52	Unapply payment	[Partial]	Backend only; needed to cancel a paid invoice
52	Refund from advance or invoice	[Implemented]	Cannot be undone

6 rows: 5 Implemented, 1 Partial, 0 Not implemented, 0 Unverified. Invoice correction and refund work. Unapplying a payment is Partial because it has no screen.

64.3.8 Chapter 53 — Subscription expiry and recovery

Chapter	Feature	Status	One-line note
53	Trial and banners	[Implemented]	14 days
53	Grace period	[Implemented]	3 days
53	Read-only mode 402	[Implemented]	Cached 30 s per process
53	Super admin records payment	[Implemented]	Overwrites plan
53	Return to active	[Implemented]	
53	Renewal reminder email	[Not implemented]	None sent
53	Worker jobs while read-only	[Unverified]	Deduced: guard is HTTP only

7 rows: 5 Implemented, 0 Partial, 1 Not implemented, 1 Unverified. The subscription lifecycle (trial, grace, read-only, recovery) is built. Renewal reminder emails are not sent. Whether background jobs keep running while a clinic is read-only is Unverified, deduced from the guard being HTTP only.

64.3.9 Chapter 54 — Access change propagation

Chapter	Feature	Status	One-line note
54	Owner edit of role, feature or nav	[Implemented]	
54	access_version bump	[Implemented]	Subscriber plus explicit bumps
54	Redis bus to all processes	[Implemented]	15 s fallback
54	access.changed realtime event	[Implemented]	Room org:id
54	Web app refetch	[Implemented]	Focus fallback, 30 s stale
54	Theme save triggers propagation	[Not implemented]	No bump found

6 rows: 5 Implemented, 0 Partial, 1 Not implemented, 0 Unverified. Access changes propagate through the version counter, the Redis bus, the realtime event and the web refetch. Saving a theme does not trigger propagation.

64.3.10 Chapter 55 — Purchase to consumption

Chapter	Feature	Status	One-line note
55	Reach the inventory page	[Partial]	No sidebar entry
55	Create purchase order	[Unverified]	createdBy FK to employees receives a user id
55	Receive goods	[Unverified]	performedBy FK to employees receives a user id
55	Stock movement and adjustment	[Unverified]	Same FK risk
55	Patient consumption	[Not implemented]	API only, no screen
55	Low-stock alert	[Partial]	Server log only

6 rows: 0 Implemented, 2 Partial, 1 Not implemented, 3 Unverified. The purchase-to-consumption chain is the weakest workflow: the page has no sidebar entry, three steps are Unverified because of the login-id versus employee-id foreign keys, consumption has no screen and the alert only logs.

64.4 Cross-cutting chapters

64.4.1 Chapter 56 — Authentication

Chapter	Feature	Status	One-line note
56	Register a clinic (self-signup)	[Implemented]	Creates user, organization, main location and 14-day trial
56	Verify an email address	[Implemented]	Informational only; nothing is gated by it (Chapter 45 labels the same feature Partial)
56	Sign in	[Implemented]	Session of 1 or 30 days; at most 5 sessions per user
56	Choose and switch organization and location	[Implemented]	
56	Access token, refresh cookie and silent refresh	[Implemented]	Refresh secret rotates on every use; expiry fixed
56	Sign out, list and revoke sessions	[Implemented]	A revoked session's access token lives until it expires
56	Change, forgot and reset password	[Implemented]	Six-character minimum on these paths

Chapter	Feature	Status	One-line note
56	Add a password to an account without one	[Partial]	Endpoint only; no screen calls it
56	Rate limiting	[Implemented]	In memory of one API process; five endpoints
56	Security log	[Partial]	Write-only; some event types never written; no reader
56	User administration and staff logins	[Implemented]	Eight-character minimum on this path
56	Sign-in with Google or GitHub	[Not implemented]	Disabled; strategies not registered
56	The first super admin	[Partial]	Created by a manual database step

13 rows: 9 Implemented, 3 Partial, 1 Not implemented, 0 Unverified. Sign-in, sessions, refresh and rate limiting work. The gaps are the add-password screen, a write-only security log, a manual first super admin and disabled Google and GitHub sign-in.

64.4.2 Chapter 57 — Access control

Chapter	Feature	Status	One-line note
57	Five global guards	[Implemented]	JWT, TenantContext, FeatureFlag, Access, Subscription
57	Decorators and the coverage test	[Implemented]	Only a unit test enforces that every handler carries a marker
57	AccessContext, applyScope, decide and VisibleIf	[Implemented]	
57	GET /me/access	[Implemented]	
57	Frontend can(), route gate, sidebar filter and No access page	[Implemented]	
57	access_version, caches and the access.changed event	[Implemented]	5 min context cache, 15 s version cache
57	Lab portal guard	[Implemented]	Separate guard for lab members
57	Row-level security in the database	[Not implemented]	Isolation is application-level only

8 rows: 7 Implemented, 0 Partial, 1 Not implemented, 0 Unverified. The guard chain, decorators, scopes, the frontend gate and propagation are built. Row-level security in the database is not built.

64.4.3 Chapter 58 — Customization

Chapter	Feature	Status	One-line note
58	Features screen and impact preview	[Implemented]	
58	Roles builder	[Implemented]	Detail in Chapter 11
58	Per-role and clinic home page	[Implemented]	
58	Sidebar builder	[Implemented]	Clinic default layout only; per-role layouts are never written
58	Registry flags for sidebar items	[Implemented]	The ownerOrSuperAdmin flag is unused
58	Theme and branding (7 keys, per location)	[Implemented]	
58	Public login theme	[Implemented]	

Chapter	Feature	Status	One-line note
58	Settings registry beyond theme	[Partial]	Only theme keys are defined

8 rows: 7 Implemented, 1 Partial, 0 Not implemented, 0 Unverified. Features, roles, home pages, sidebar and theme customization are built. The settings registry only holds theme keys.

64.4.4 Chapter 59 — Notification mechanics

Chapter	Feature	Status	One-line note
59	Pipeline from event to in-app inbox	[Implemented]	
59	Delivery by WhatsApp, SMS and email	[Not implemented]	Dev-log stand-in provider only
59	Recipient resolvers	[Partial]	Only the lab order workflow registers one
59	Decision order and quiet hours	[Implemented]	Quiet hours never hold in-app messages
59	Message templates	[Implemented]	
59	Outbox, relay and retries	[Implemented]	Status delivered is never set

6 rows: 4 Implemented, 1 Partial, 1 Not implemented, 0 Unverified. The in-app pipeline is built; external channels are not. Only the lab workflow registers a recipient resolver.

64.4.5 Chapter 60 — Audit logging

Chapter	Feature	Status	One-line note
60	History tables (audit_logs, status_history)	[Implemented]	Trigger-protected against update and delete
60	Patients audit	[Implemented]	13 record kinds plus export and import
60	Appointments audit	[Not implemented]	
60	Employees audit	[Not implemented]	Only role definitions are audited
60	Payroll audit	[Not implemented]	
60	Organizations and locations audit	[Not implemented]	Features, theme and subscriptions are audited
60	Patient audit trail reader	[Implemented]	Needs admin.audit.view
60	Organization-wide audit viewer	[Not implemented]	
60	Retention policy	[Not implemented]	No purge or archive anywhere

9 rows: 3 Implemented, 0 Partial, 6 Not implemented, 0 Unverified. Audit coverage is the clearest cross-cutting gap. Patients are audited and a reader exists. Appointments, employees, payroll and organizations and locations are not audited, there is no organization-wide viewer and no retention policy.

64.4.6 Chapter 61 — Error handling

Chapter	Feature	Status	One-line note
61	Response envelope and global exception filter	[Implemented]	Developer message carries raw text for unexpected errors
61	Validation (whitelist, forbid non-whitelisted)	[Implemented]	
61	Error code families	[Implemented]	Two parallel version-conflict codes

Chapter	Feature	Status	One-line note
61	Idempotency keys and If-Match	[Implemented]	
61	Database constraint mapping	[Partial]	Per service; no global mapping, so some errors are generic
61	Frontend toasts, states and code maps	[Implemented]	Code maps exist only for some modules

6 rows: 5 Implemented, 1 Partial, 0 Not implemented, 0 Unverified. The envelope, validation, error codes, idempotency and frontend states are built. Database constraint errors are mapped per service, so some surface as generic errors.

64.4.7 Chapter 62 — Time, money and numbering

Chapter	Feature	Status	One-line note
62	ClockService and location timezone	[Implemented]	
62	Server-clock leftovers	[Partial]	Jobs, credit expiry, daily summary, purchase order and prescription year, leave year, lab catalog day
62	Money storage and half-up rounding	[Implemented]	Paisa integers or numeric columns
62	Exclusive tax	[Implemented]	
62	Currency label	[Implemented]	Label only; no conversion
62	document_sequences and number_sequences	[Implemented]	Safe for billing and lab documents
62	Count-plus-one and last-plus-one numbers	[Partial]	Collision risk; purchase order and prescription numbers unique across organizations

7 rows: 5 Implemented, 2 Partial, 0 Not implemented, 0 Unverified. The location clock, money rules and the two safe sequence mechanisms are built. Some jobs and older numbering still use the server clock or count-plus-one.

64.4.8 Chapter 63 — Real time and jobs

Chapter	Feature	Status	One-line note
63	Process roles api, worker and all	[Implemented]	Exactly one worker
63	Socket.IO gateway and rooms	[Implemented]	
63	Outbox relay, Redis bus, heartbeat and watchdog	[Implemented]	
63	Scheduled jobs (seven of eight)	[Implemented]	
63	Inventory alert job	[Partial]	Logs only; sends nothing

5 rows: 4 Implemented, 1 Partial, 0 Not implemented, 0 Unverified. Process roles, sockets, the relay and seven of the eight jobs are Implemented. The inventory alert job is the eighth.

64.5 Totals

The counts in this table were computed by counting the rows of the tables in this chapter with a script, not estimated.

Chapter	Subject	Implemented	Partial	Not implemented	Unverified	Total
28	Patients	19	9	2	0	30
29	Appointments	10	2	2	0	14
30	Queue	28	2	1	0	31
31	Consultation	11	2	2	0	15
32	Prescriptions	11	4	0	0	15
33	Treatment plans	7	6	1	0	14
34	Billing	31	3	0	0	34
35	Commissions	13	0	1	0	14
36	Dashboard and reports	6	3	0	0	9
37	Employees	10	3	2	0	15
38	Payroll	7	3	1	0	11
39	Inventory	0	6	1	3	10
40	Dental lab (clinic side)	26	4	0	0	30
41	Lab portal	24	3	0	0	27
42	Lab finance	17	4	0	0	21
43	Workflow engine (Config Studio)	11	1	0	0	12
44	Notifications	7	3	2	0	12
45	Subscriptions and onboarding	8	2	0	0	10
Modules (28 to 45)		**246**	**60**	**15**	**3**	**324**
46	Clinic onboarding	7	3	0	0	10
47	Staff onboarding	6	0	1	0	7
48	Patient journey	10	0	1	0	11
49	Dental lab case	9	2	0	0	11
50	Daily clinic closing	8	1	0	0	9
51	Month-end payroll	6	2	1	0	9
52	Invoice correction and refund	5	1	0	0	6
53	Subscription expiry and recovery	5	0	1	1	7
54	Access change propagation	5	0	1	0	6
55	Purchase to consumption	0	2	1	3	6
Workflows (46 to 55)		**61**	**11**	**6**	**4**	**82**
56	Authentication	9	3	1	0	13
57	Access control	7	0	1	0	8
58	Customization	7	1	0	0	8

Chapter	Subject	Implemented	Partial	Not implemented	Unverified	Total
59	Notification mechanics	4	1	1	0	6
60	Audit logging	3	0	6	0	9
61	Error handling	5	1	0	0	6
62	Time, money and numbering	5	2	0	0	7
63	Real time and jobs	4	1	0	0	5
Cross-cutting (56 to 63)		**44**	**9**	**9**	**0**	**62**
All rows		**351**	**80**	**30**	**7**	**468**

The module chapters hold 324 rows: 246 Implemented (76 percent), 60 Partial, 15 Not implemented and 3 Unverified. The workflow chapters hold 82 rows and the cross-cutting chapters 62. Adding modules and cross-cutting rows, which do not overlap, gives 386 distinct rows with 290 Implemented, 69 Partial, 24 Not implemented and 3 Unverified. The grand total of 468 includes the workflow rows, which repeat module features.

The pattern behind the numbers is consistent. The Partial label is almost always "the backend exists but no screen reaches it" or "the data is recorded but no rule reads it", so a Partial row is usually a small piece of web work or a decision, not a missing engine. The Not implemented rows are mostly features a clinic would expect from a mature product (patient messaging, reminders, attendance, deductions, audit coverage). The Unverified rows are concentrated in inventory, where a foreign-key mismatch could make writes fail at run time, and in one job-behaviour question during read-only mode. Chapter 65 lists each gap with its severity, and Chapter 67 lists the questions that decide what to build next.

Chapter 65 — Known Gaps

This chapter collects every gap found while the handbook was written: features that do not exist, features that exist in the backend but cannot be reached from the web app, settings that do nothing, data-integrity oddities and places where older documents describe something the code no longer does. It is for the owner who has to decide what to fix, build or promise before more clinics go live. Each gap states what it is, why it matters to an owner, which chapter holds the evidence, a severity and a suggested next step.

65.1 How severity is judged

Severity is judged by the effect on a live clinic, not by the effort to fix it.

- **High**: the problem can stop daily work (registering a patient, writing a prescription) or put the business at risk, and it can happen in normal use.
- **Medium**: a clinic would notice and need a workaround, or an expectation set by the product screens is not met, but daily work continues.
- **Low**: a convenience, a cleanup or a feature most clinics can live without.

No application was run while the handbook was written, so every gap was found by reading the code. Where a gap depends on behaviour that was deduced but not observed, the text says so.

65.2 Features that are not built

These features are absent. Some have a seeded feature key or a database table that suggests they exist; none of them works.

Gap	Why it matters to an owner	Evidence	Severity	Suggested next step
No messages to patients by SMS, WhatsApp or email, and no appointment reminders	Clinics cannot remind patients of visits, so no-shows are handled by phone calls only. The notification pipeline already exists, but the only active channel is the in-app inbox for staff and lab members.	Chapters 44, 59	Medium	Decide on a provider (Question 3 in Chapter 67), then switch on the patient audience and add a reminder event and job.
No renewal or trial-ending messages to clinic owners	An owner learns of an expiring trial only from the banner inside the app. Expiry puts the clinic in read-only mode.	Chapters 45, 53	Medium	Send an email to the owner at fixed days before expiry.
No patient portal	Patients cannot see their own records or invoices online. The feature key was retired.	Chapters 1, 28	Low	Treat as a roadmap item; no work needed for current clinics.
No online booking and no recurring appointments	Every booking is made by staff, and a course of visits must be booked one by one.	Chapter 29	Low	Roadmap items.
No insurance claims and no online payment gateway	Insurance details can be stored (backend only) but no claim is created. Payments are recorded by staff, never collected online.	Chapters 28, 34	Low	Roadmap items; confirm whether target clinics need them.
No server-generated PDF receipts or invoices	Receipts and invoices are printed from the browser (A4 and 80 mm). They cannot be emailed or stored as files.	Chapters 34, 48	Low	Add a PDF generator only if sending receipts to patients becomes a requirement.
No attendance tracking	Payroll uses salary and commission only; there is no record of who came to work. Leave exists but is not read by payroll.	Chapter 37	Medium	Decide whether attendance is wanted at all; if so, design it with payroll deductions.

Gap	Why it matters to an owner	Evidence	Severity	Suggested next step
No payroll deductions, tax or bank files	Net pay always equals gross pay, so deductions and withholding must be done outside the Platform.	Chapters 38, 51	Medium	Define the deduction rules the target market needs, then add them to payroll generation.
No deploy or continuous-integration configuration	The repository holds no Dockerfile, pipeline or hosting configuration. How the Platform is deployed lives in people's heads.	Chapter 6	High	Write down and automate the build, migration and process-role steps before more clinics depend on it.
Chairs exist as a table only	The `chairs` entity has no module, no endpoint and no screen.	Chapters 12, 26	Low	Remove the entity or build the module; the rest of the Platform does not need it.
No lab order can be started from the consultation workspace	Doctors must leave the consultation and use the Lab orders page. Lab orders carry no link to the encounter.	Chapters 31, 48	Low	Add a button that opens a lab order with the patient and doctor prefilled.
No structured allergy list and no drug-interaction check	Allergy alerts are text chips built from free-text fields. A doctor can prescribe a drug the patient is allergic to without any warning.	Chapter 31	Medium	Add a structured allergy list first; a drug check can follow.
No way to assign an existing employee to a second location	The only way to create an assignment is to create a new employee, and a phone number that already exists is refused. A doctor who works at two locations cannot be set up.	Chapters 37, 47	Medium	Add an endpoint and screen that create an assignment for an existing employee.
No SOAP note entity or note templates	Visit notes are four fixed text fields. Clinics that need structured notes cannot have them.	Chapter 31	Low	Defer until a clinic asks.
No manual commission adjustment	The source type exists in the ledger but nothing writes it, so a one-off correction to a doctor's commission needs a database change.	Chapter 35	Low	Add an adjustment screen with a reason and an audit row.
No per-location patient visit statistics	The `patient_visits` table is never written, so the endpoint always returns an empty list.	Chapters 17, 27, 28	Low	Drop the table and endpoint, or fill it from encounters.
No organization-wide audit viewer and no audit for appointments, employees, payroll, organizations or locations	The owner cannot answer "who changed this?" for bookings, staff records or payroll. Only patient records have a history screen.	Chapter 60	Medium	Extend the audit calls to the missing modules and add one viewer screen.

The most important rows are the missing deploy configuration, which is an operational risk, and the missing second-location assignment, which blocks a normal multi-branch clinic. Everything else is either a feature a clinic would ask for later or a gap that staff can work around manually.

65.3 Built but unreachable from the web app

In these cases the backend is finished, or nearly so, but no screen calls it, so the feature cannot be used by a clinic. Each one is usually a small piece of web work.

Gap	Why it matters to an owner	Evidence	Severity	Suggested next step
The inventory page has no sidebar entry and nothing links to it	Stock, suppliers and purchase orders work, but a user can only reach them by typing the address. Most staff will never find them.	Chapters 39, 55	Medium	Add a sidebar entry; resolve the foreign-key question in Section 65.6 first.
Unapply a payment has no screen	An invoice that has any payment on it cannot be cancelled from the web app, because cancelling needs the payments released first. The workaround is a refund.	Chapters 34, 52	Medium	Add an Unapply action to the payment detail.
Patient treatment plans have no screen	Creating, updating and tracking plans, turning plan items into charges and the plan billing summary exist only in the backend. Doctors cannot plan a course of treatment in the Platform.	Chapters 28, 33	Medium	Build the plan screens, or remove the backend if plans are not wanted.
Payroll: put on hold, CSV export and payslip PDF have no buttons	An owner cannot give staff a payslip or export a month for an accountant from the Platform.	Chapters 38, 51	Medium	Add the three buttons; the endpoints exist.
Lab portal: charge requests and file upload have no portal control	A lab cannot ask for an extra charge or attach a file from the portal, so these happen by message or phone.	Chapters 40, 41	Medium	Add the two forms to the portal order page.
Lab portal: no notification inbox	Lab members receive in-app notifications that link into the portal, but the portal layout has no bell to show them.	Chapters 40, 41	Medium	Add the bell to the portal header.
Cancel one lab item has no screen	Stopping one item of an order in progress is not possible from the web app.	Chapter 40	Low	Add a Cancel item action on the order detail.
Recalls, referrals, feedback, insurance and the communications log	The component that shows them is never mounted, and recalls have no component at all. The data model is ready.	Chapter 28	Low	Mount the tab or drop the claim; update the older documents that call it implemented.
Blacklist a patient	The flag can only be set through the API, and it blocks nothing when set.	Chapter 28	Low	Decide what blacklisting should do (see Section 65.4), then add the control.
Override booking rules	The API flag and permission exist, but no booking screen sends it, so no one can book outside working hours or past the daily limit.	Chapter 29	Low	Add an override option for users who hold the permission.
Cancel reasons	The API stores a reason code and note; the screen sends neither and shows no confirmation.	Chapter 29	Low	Add the reason dialog.
Phone pre-check at registration	The endpoint and hook exist but no component calls them. The server rejects duplicates at save with a conflict message.	Chapter 28	Low	Call the hook on the phone field.
Credit packages and patient credits	Backend only, and credits are not a payment method.	Chapter 34	Low	Decide whether prepaid packages are wanted.
Plan billing summary, daily financial summary and several billing helpers	Endpoints and client functions exist but no component renders them. `/financial-summary` redirects to the dashboard.	Chapters 33, 34, 36	Low	Remove unused code or build the screens.
About ten dashboard widgets that still use mock data	They are not on the live dashboard but remain in the code and could be mistaken for real features.	Chapter 36	Low	Delete the mock module.

Gap	Why it matters to an owner	Evidence	Severity	Suggested next step
Queue: next-patient preview, queue history, the doctor's own session read and session creation	Backend only. The queue works without them.	Chapter 30	Low	Add a history screen for supervisors if audit of the queue is needed.
Prescriptions: edit one medicine line, duplicate, change history viewer, lookup by appointment	Backend only. The history read and the duplicate action are also likely to fail (see Section 65.6).	Chapter 32	Low	Fix the two defects, then add the screens.
Employee portal-access switch and some schedule calls	The endpoint exists, but the switch is set through the employee form instead. `/employees/create` exists but nothing links to it.	Chapter 37	Low	Clean up.
Lab: vendor bill header edit, manual bill lines, using lab advance on bills, impressions to the lab, service categories, specialty enablement	Endpoints exist; no buttons call them.	Chapters 40, 42, 43	Low	Build when the workflow is needed.

The Medium rows share one theme: a flow is half-finished at the boundary between backend and web. Finishing the inventory entry, the unapply action and the payroll buttons would remove most of the daily friction an owner would see.

65.4 Seeded but inert

These things look real in the catalog, a table or a setting, but nothing reads them or acts on them.

Gap	Why it matters to an owner	Evidence	Severity	Suggested next step
Legacy organization settings (booking defaults, advance booking days, cancellation notice, late fee percentage and old feature switches)	An owner can save these values and nothing changes. Only a location settings screen reads the old feature switches to hide sections.	Chapters 9, 14	Medium	Remove the screen or wire the rules (Chapter 67, Question 5).
Blacklisted and inactive patient flags	A blacklisted or inactive patient can still be booked, queued and billed. Staff may assume the flag protects them.	Chapter 28	Medium	Decide the intended rule and enforce it at booking and check-in.
Availability overrides and approved leave	They can be recorded and edited, but booking, validation, the calendar, the queue and payroll never read them. A doctor on approved leave can still be booked.	Chapters 29, 37	Medium	Make booking and the calendar read them.
Feature keys that gate nothing (`comms.in_app`, `comms.whatsapp`, `hr.commissions`)	Switching these on or off has no effect. Twelve more keys for unbuilt features are deactivated at every start and are hidden.	Chapters 10, 35, 44	Low	Retire the three keys or attach checks.

Gap	Why it matters to an owner	Evidence	Severity	Suggested next step
Notification events that are never emitted (`order.overdue` , `shipment.dispatched` , `tryin.appointment_due` , `appointment.rescheduled_lab_delay`)	Clinics can edit the wording of messages that will never be sent. There is no overdue job.	Chapters 40, 44	Low	Add emitters or remove the events.
Quiet hours, patient opt-outs and the delivered status	Quiet hours are saved but never hold in-app messages. The outbox status `delivered` is never set. The opt-out screen is commented out.	Chapters 44, 59	Low	Fix or remove.
The `queue.display.manage` permission	It is in the catalog and in the Receptionist template but nothing checks it.	Chapter 30	Low	Attach it to display management or drop it.
Email verification	The verification state is stored but nothing is gated by it.	Chapters 45, 56	Low	Decide whether unverified owners should be limited.
Dead schema and code (the table for linked external sign-ins, the deprecated refresh-token column, unused queue and database packages, per-role sidebar layouts that are never written, unused audit and history values, unused magic-link limits)	They add noise for the next developer and imply features that are not there.	Chapters 4, 13, 15, 32, 40, 58, 60	Low	Clean up in one pass.
Settings registry beyond theme	The generic cascading settings store holds only theme keys.	Chapter 58	Low	Move new settings onto it or leave as is.

The Medium rows are the ones that mislead: a screen or flag that implies protection or behaviour the Platform does not provide. Cleaning them up costs little and removes false assurance.

65.5 Data-integrity oddities

These are places where the data model can produce a wrong or failing result in normal use. Several were deduced from code and constraints and not reproduced.

Gap	Why it matters to an owner	Evidence	Severity	Suggested next step
Patient numbers are "count plus one", but patients can be permanently deleted	After any patient is deleted, the next registration computes a number that already exists and fails on the unique index. Because the count does not change, every later registration fails the same way. This was confirmed by reading the code.	Chapters 28, 62	High	Use the safe sequence mechanism that billing uses, and stop hard-deleting patients that have history.
Prescription numbers are unique across the whole Platform, but are numbered per organization	The first prescription of a second clinic in the same year is numbered like the first clinic's first one and violates the unique key. This was confirmed by reading the code, but not run.	Chapters 19, 32, 62	High	Make the number unique per organization, or use a Platform-wide sequence.
Purchase order numbers have the same flaw	Same cause, in inventory. A second clinic's first purchase order collides.	Chapters 23, 39, 62	Medium	Same fix.

Gap	Why it matters to an owner	Evidence	Severity	Suggested next step
Other count-plus-one numbers (treatment plan, legacy prescription, employee code) and unlocked "last plus one" numbering	They can repeat after a delete or when two people save at the same moment.	Chapters 28, 33, 37, 62	Medium	Move to the sequence mechanism.
Login id stored where an employee id is expected	Several writes (inventory movements and purchase orders, prescription duplication, saved doctor signatures) put a login id into columns defined as links to employees. In some tables there is no foreign key, so the value is silently wrong; where there is one, the write may fail. Not run.	Chapters 28, 32, 39, 55	Medium	Decide which id each column should hold and make the code consistent.
Deleting a patient or an employee is blocked by `ON DELETE RESTRICT` rows and fails with a generic server error	The user sees "Something went wrong" instead of a reason. A code comment claims the rows cascade, but they do not.	Chapters 16, 17, 28, 37	Medium	Catch the foreign-key error and give a clear message, or soft-delete.
Duplicate CNIC or phone number on employee edit	Reaches the database unique index and returns a generic error.	Chapter 37	Low	Check before saving.
Employee phone and CNIC are unique across the whole Platform	By design one person record can serve several clinics, but the only creation path refuses an existing phone (see Section 65.2). A second clinic cannot hire someone another clinic already registered.	Chapters 8, 37	Medium	Add the assign-existing-employee flow.
Tenant columns without foreign keys on most billing, queue and lab tables, and logical references to unknown targets (`dentist_id` , `manager_id`)	The database does not stop a row pointing at a missing or wrong organization. Nothing in the code explains the choice.	Chapters 12, 27	Low	Document the decision or add the keys.
"Append-only" tables enforced by convention only	Only three tables are protected by triggers. The billing audit log, the queue log and the ledgers can be changed by anyone with database access.	Chapters 26, 27	Low	Add triggers to the money tables.
Jobs and statistics that use the server clock	The month-end payroll month, credit expiry, daily summary, "this month" prescription counts, the lab catalog price day (fixed to one time zone) and the leave year follow the server, not the location. Wrong around midnight for locations in other zones.	Chapters 38, 62	Medium	Move jobs to the location clock or document the server zone.
The web app computes some appointment ranges in the browser time zone	The server uses the location time zone, so a user in another zone sees shifted times.	Chapter 29	Low	Use the location zone in the web app.
Location defaults of UTC and USD while organizations default to Asia/Karachi and PKR	A new branch created without explicit values gets different defaults from its clinic.	Chapters 1, 9, 14	Low	Make new locations inherit the clinic's values.
Concurrent patient imports are blocked only inside one process	With several API instances two imports can run at once.	Chapter 28	Low	Use a database lock.

Gap	Why it matters to an owner	Evidence	Severity	Suggested next step
Registration compares phone numbers exactly, import compares normalised numbers	The same number typed two ways can create two patients.	Chapter 28	Low	Normalise in both.
Queue policy values apply within about a minute, and per process	The screen says changes apply tomorrow. Only the smart-queue switch is fixed per day. Another instance can use old values for up to a minute.	Chapter 30	Low	Correct the screen text.
Check-in lookup and no-show job: write through GET and silent SQL	One read endpoint creates a session. The no-show job writes no queue event.	Chapter 30	Low	Log the event.

The two High rows deserve attention first because they stop core work. The patient-number problem appears after the first deletion in any clinic. The prescription-number problem appears as soon as a second clinic writes a prescription in the same year.

65.6 Unverified behaviour

These items could not be confirmed from code reading, and a real run is needed.

- Whether the foreign-key mismatches in inventory (Section 65.5) make stock movements, purchase orders and goods receipts fail at run time. Chapter 39 labels three features Unverified for this reason.
- Whether the prescription change-history read fails because it asks for a relation that is not declared, and whether duplicating a prescription fails on the doctor foreign key. Chapter 32.
- Whether background jobs keep running while a clinic is read-only. The read-only rule is an HTTP guard, so the answer is probably yes. Chapter 53.
- Whether a local PostgreSQL without SSL works with the connection settings. Chapter 7.
- Which account type to create in User Admin for a person who works only for a lab, and the exact sequence a super admin must follow to create an owner login for a clinic created from the Organizations screen. Chapters 41 and 46.
- Whether storage settings make uploaded patient documents and prescription PDFs publicly readable by address. Chapters 28 and 32.

65.7 Documentation drift

Several older documents describe a Platform that no longer exists. The outdated ones should not be used. Chapter 68 lists each conflict with evidence. The gaps that matter for planning are these.

Gap	Why it matters to an owner	Evidence	Severity	Suggested next step
The root project instructions describe a frontend layout, a four-guard chain, "stub" modules and path-prefix skips that the code does not have	A new developer or an AI assistant that follows them will look in the wrong places.	Chapters 4, 5, 8	Medium	Update the instructions file to match the code.
Five historical documents (hand-off note, project overview, audit, plan and the old billing context pack) are outdated	They list fixed problems as open and built features as missing.	Chapter 68	Medium	Archive them or mark them superseded by this handbook.
The first inventory of the documents marked recall, feedback and referral as implemented through a CRM tab	The tab is never mounted. This handbook corrects that to Partial.	Chapters 28, 68	Low	Use the handbook as the source of truth.

Gap	Why it matters to an owner	Evidence	Severity	Suggested next step
Some screen texts claim behaviour that does not exist (starting a patient plan from a template, queue settings applying "from tomorrow", a lab access panel saying a link expires after a day)	Staff are told something false.	Chapters 30, 33, 40	Low	Correct the texts.
The generated data-model catalog has two known defects (shifted notes for ten inventory tables, a false trigger claim on form schema versions)	Anyone reading the catalog directly can be misled.	Chapters 23, 26	Low	Fix the generator.

Documentation drift is Medium at worst, but it compounds: every wrong document is a future wrong decision.

65.8 What to do first

The gaps fall into a clear order. First fix the two High data-integrity gaps (patient numbers after deletion and per-organization prescription numbers), because they break core work and will appear in normal use. Second, write down the deploy procedure, because it is the only High gap that is not a code defect. Third, finish the half-built flows that cost clinics time: the inventory sidebar entry, the unapply action, the payroll buttons and the lab portal forms. Fourth, remove or wire the settings that imply protection, such as the legacy organization settings and the blacklist flag. Everything else is roadmap work that depends on the answers in Chapter 67.

Chapter 66 — Risks

This chapter lists the risks an owner carries by running the Platform as it is today: risks in the data model, risks in how it is operated, observations that a regulator or a careful customer might ask about, and security risks described at category level only. It is for the product owner, who decides what to fix before more clinics rely on the Platform. Nothing in this chapter was tested by running the application; every item comes from reading the code and the generated data-model catalog.

66.1 How to read the risks

Each risk has a likelihood in normal use (Likely, Possible, Unlikely) and an impact on a live clinic (High, Medium, Low). Security items are grouped by category and deliberately contain no file paths, endpoint paths, secret values or exploit detail, because this text is shared outside the engineering team. The engineering team holds the detailed findings separately.

66.2 Data-model risks

Risk	What can happen	Likelihood	Impact	Evidence
Count-plus-one and last-plus-one numbering	After a permanent delete, or when two people save at the same moment, a patient, plan, employee or purchase order number can repeat and the save fails.	Likely for patients after any delete	High	Chapters 28, 62
Platform-wide unique keys with per-organization numbering	Prescription and purchase order numbers restart for each organization but are unique across all of them, so a second clinic collides with the first.	Likely in a multi-clinic deployment	High	Chapters 19, 32, 39, 62
Login id stored in employee-id columns	Some writes put a user id where an employee id is expected. Without a foreign key the value is silently wrong; with one the write may fail.	Possible	Mediu m	Chapters 28, 32, 39
Tenant columns without foreign keys	Most billing, queue and lab tables keep organization and location ids with no foreign key, so the database cannot reject a wrong tenant id.	Unlikely	Mediu m	Chapters 12, 27
"Append-only" promised by comments only	Only the commission ledger, the audit log and the status history are protected by triggers. The billing audit log, the queue log, the ledgers and the allocation table rely on application code.	Unlikely	Mediu m	Chapters 26, 27
Duplicate concepts	Two prescription tables, two audit mechanisms, two number-sequence mechanisms, two outbox tables and two feature-switch mechanisms coexist. A change to one can miss the other.	Possible	Mediu m	Chapter 27
Deletes blocked by RESTRICT keys	Deleting a patient or employee with history fails with a generic error.	Likely	Low	Chapters 16, 17
Schema drift between development and production	Development builds the schema from the entities; production applies a reviewed diff. Two migrations were written before later entity changes, so production needs the diff applied.	Possible	Mediu m	Chapters 6, 12
Mixed timestamp types	Older tables use timestamps without a time zone, newer ones with.	Unlikely	Low	Chapter 12

The two numbering rows are the most urgent because they fail in ordinary use. The rest are latent: they cost nothing until a clinic hits a particular edge.

66.3 Operational risks

Risk	What can happen	Likelihood	Impact	Evidence
Exactly one worker process	The worker runs the outbox relay, the notification relay and all eight scheduled jobs. Two workers would duplicate jobs; none would stop live queue updates, notifications and nightly checks. There is no supervision described.	Possible	High	Chapters 6, 63
In-memory rate limiting	Login and other limits are counted in one API process. With several instances the limit multiplies, and a restart clears it.	Likely when scaled out	Medium	Chapters 6, 56
Server time zone for jobs	No schedule sets a time zone, so jobs follow the server clock, not each location.	Likely	Medium	Chapters 6, 62, 63
No deploy or CI configuration	There is no pipeline, container file or hosting description in the repository. Builds and releases depend on individual knowledge. The built output path also differs from the one in the start script.	Likely	High	Chapter 6
Dependence on Redis	Realtime updates, access-change propagation and settings caches use Redis. Without it clinics see stale screens for up to the fallback interval.	Possible	Medium	Chapters 54, 63
Per-process caches	The subscription decision (30 seconds), access context (5 minutes), version (15 seconds) and queue policy (60 seconds) are cached per process.	Likely	Low	Chapters 30, 45, 57
Nightly jobs only log	The money invariant check and the inventory alert job write to the log. Nobody is told unless someone reads it.	Likely	Medium	Chapters 34, 39
Calendar licence	The appointment calendar uses premium resource views from a third-party library, and the licence key in use is a non-commercial one.	Likely	Medium	Chapter 29
Seeded data re-created at boot	Seeds run in the worker as well as the API and recreate a deleted system role, and re-seed starter plan templates when a clinic has none.	Possible	Low	Chapters 11, 33
Unrun test coverage of behaviour	The handbook was written by reading code. The access and billing suites exist, but the findings here were not reproduced.	Possible	Medium	Chapter 64

Of these, the single worker and the missing deploy configuration matter most, because a failure there is silent and affects every clinic at once.

66.4 Compliance-sounding observations

These are observations, not legal conclusions. A clinic or a regulator may ask about them; the Platform makes no compliance claim, and earlier screenshots that showed such claims were excluded from the handbook.

- **No retention policy.** Nothing purges or archives old sessions, security logs, audit rows, notification messages or consent signatures. Network addresses and browser strings are kept indefinitely.
- **Audit coverage is partial.** Patient records, billing, prescriptions and queue events keep history. Appointments, employees, payroll, organizations and locations do not, and there is no organization-wide viewer.
- **No row-level security.** Tenant isolation is application-level only. The queue sets database session values for tenant and location, but no database policy uses them.

- **Patient documents on external storage.** Documents, insurance images and prescription PDFs are stored with an external provider and referenced by address. Access to those addresses is not decided by the Platform's permissions.
- **Consent signatures inside the database.** Signature images are stored inline in the consent row, and immutability of consents is a convention, not a database rule.
- **Message text in logs.** The stand-in delivery provider writes message bodies and recipient addresses to the application log, which would be unsuitable once real patient channels are switched on. Outbox rows also keep rendered text.
- **No second factor or lockout.** Accounts rely on a password with a short minimum.

66.5 Security risks by category

These categories summarise findings from all chapters. They contain no locations, paths or exploit detail by design.

Category	Summary	Impact	Chapters
Tenant isolation checks missing on some endpoints	A few reads or writes do not filter by organization or location, or do not verify that referenced records belong to the caller's organization or patient scope. A few reads apply organization scope where the permission has a narrower scope.	High	19, 30, 32, 33, 34, 42
Application-level isolation only	No database row-level policies back up the application checks, so one missed check is a data leak.	Medium	8, 30, 57
Committed credentials in test scripts	Test scripts and a fixture in the repository hold a database connection and test passwords. They need rotation and removal from history.	High	Inventory of documents
Token handling and logging	A legacy realtime gateway logs token-related data and accepts any origin. Real-time connections check the token once and outlive its expiry. A revoked session's access token stays valid until it expires.	High	56, 57, 63
Input from clients trusted	The server fetches a signature image address supplied by the client. An upload route is open to any organization member. Upload size and type are only partly restricted. Some edit screens send whole rows.	Medium	28, 32, 58
Information exposure in errors	The error envelope carries raw internal messages for unexpected errors.	Medium	5, 61
Public routes by design	The waiting-room display and lab magic links need no login by design, using an identifier or a bearer token as the credential. They rely on expiry, revocation and throttling, and the throttle is in memory.	Medium	30, 40, 41
Secret reuse	The signing secret for magic links falls back to the access-token secret when its own is missing.	Medium	40, 41
Authentication hardening	Short minimum password, two different minimums on two paths, no lockout, no second factor, no logging of wrong passwords, no rate limit on two sensitive endpoints.	Medium	56, 45, 47
Authorization granularity	Some actions need only a broad read permission, and one payroll permission guards all payroll data.	Medium	37, 38, 40
Audit and retention	Missing audit for personnel and payroll data; no retention.	Medium	13, 60
Money-moving replay protection	Idempotency keys are required for most money actions but optional for one vendor payment path.	Low	42
Third-party licence	See the calendar licence in Section 66.3.	Medium	29

The high-impact rows are tenant-isolation gaps, committed credentials and token handling. The others are hardening work that lowers the chance and the blast radius of a mistake.

66.6 What to fix first

Fix the committed credentials first: rotate them and remove the files from history, because the exposure already exists. Then close the tenant-isolation gaps and the token logging, since a single missing filter lets one clinic read another's data. Third, fix the two numbering collisions, which break daily work. Fourth, document and automate deployment and decide how the single worker is supervised. After these, add audit coverage for staff and payroll and a retention policy, then harden authentication and the realtime token check. The rest can follow the roadmap.

Chapter 67 — Open Questions

This chapter lists the decisions and facts that only the product owner can supply. They come from the notes of every handbook chapter, from the questions in the generated organizations and locations chapter, from the ambiguities in the generated entity catalog and from items the code could not settle. Each question has the context and the reason the answer matters, so the answers can steer what is built, fixed or removed next.

67.1 Product direction

1. **Which patient channels will the Platform use, and who pays for them?** Context: the message pipeline is built, but only the in-app channel for staff and lab members is active. WhatsApp, SMS and email to patients are commented out, and a log stand-in takes their place. Why it matters: reminders, receipts and lab-delay messages all wait on this choice, and it also decides the privacy and logging work needed (Chapter 66).
2. **Should owners get trial-ending and expiry messages, and how many days ahead?** Context: only an in-app banner warns today; no renewal email is sent. Why it matters: expiry puts a clinic in read-only mode, so a missed warning is a lost day of work and a support call.
3. **Should the Platform have a patient portal, online booking and recurring appointments?** Context: all three were planned and their feature keys were retired. Why it matters: it decides whether the retired keys are deleted or the work is scheduled.
4. **Are insurance claims and an online payment gateway required for target clinics?** Context: insurance details can be stored (backend only) and no claim or gateway exists. Why it matters: these are large features, and the answer avoids building them early or promising them.
5. **Is attendance wanted, and should payroll have deductions, tax and bank files?** Context: payroll is salary plus commission; deductions are always zero; leave is recorded but not read. Why it matters: payroll is the most legally sensitive money flow, and clinics will ask.
6. **What should a blacklisted or inactive patient be prevented from doing?** Context: both flags are informational and block nothing. Why it matters: staff may rely on them, and the answer defines the rule to enforce at booking, check-in and billing.
7. **Should the product name be fixed?** Context: the handbook calls it "the Platform" and each clinic sets its display name; the default is "Clinic Portal". Older documents and some stored keys carry a former working name. Why it matters: it decides what is cleaned out of the code and which documents can be reused.

67.2 Clinics, locations and subscriptions

These come from the generated organizations and locations chapter.

8. **Should a clinic created by a super admin start on a trial, or stay unmanaged?** Context: unmanaged clinics have no expiry and no plan limits. Why it matters: it decides whether all clinics are subject to billing.
9. **When a clinic pays for a lower plan during a higher-plan trial, should the payment downgrade it at once?** Context: recording a payment overwrites the plan with the paid plan. Why it matters: it can silently remove features during a trial.
10. **Should deactivating a clinic also make it read-only and end sessions?** Context: today deactivation only hides the clinic at login. Why it matters: owners may still hold working sessions, and the subscription guard is not involved.
11. **Can a clinic have more than one main branch, and which becomes main when one is deactivated?** Context: the main-branch flag is not exclusive. Why it matters: reports and defaults may pick the wrong branch.

12. **Should new branches inherit the clinic's time zone and currency, and should changing the clinic's time zone update existing branches?** Context: locations default to UTC and USD while organizations default to Asia/Karachi and PKR; the setup wizard updates branches but later changes do not. Why it matters: it controls every date and amount a branch shows.
13. **Should the legacy organization settings be removed or wired?** Context: booking defaults, advance booking days, cancellation notice, late fee percentage and old feature switches are saved but read by no backend rule. Why it matters: owners may think these rules are in force.
14. **Who is the location manager meant to be, an employee or a login, and what powers should a manager have?** Context: the column's target is not stated anywhere. Why it matters: a branch-manager role needs a definite owner.
15. **Should staff without the locations permission read booking-related location settings?** Context: slot length and daily cap sit on a screen that needs the manage permission. Why it matters: receptionists may need to see the limits.
16. **Which account type should be created for a person who works only for a lab, and what is the exact sequence to create an owner login for a clinic made by a super admin?** Context: both were deduced, not run. Why it matters: onboarding instructions must be correct.

67.3 Data model and numbering

17. **Which id should the `dentist_id` columns hold (employee or login)?** Context: the teeth chart, plans, recalls and feedback store a value with no foreign key and no stated target. Why it matters: reports that join to the person need a defined meaning.
18. **Which id should doctor signatures, inventory movements and purchase orders store?** Context: code passes a login id into columns that are, or look like, employee links. Why it matters: the inventory writes may fail, and signatures are keyed per login across clinics.
19. **Should prescriptions and purchase orders be numbered per organization or across the Platform?** Context: the numbers restart per organization but are unique everywhere. Why it matters: it decides which side of the conflict to change.
20. **Should patients ever be permanently deleted?** Context: deletion breaks numbering and fails when billing exists. Why it matters: it decides between soft delete and a clear refusal.
21. **Is the missing foreign key on tenant columns deliberate?** Context: most billing, queue and lab tables omit it without comment. Why it matters: it decides whether to add keys or document the choice.
22. **Which duplicate concepts should be merged?** Context: legacy and new prescriptions, two audit mechanisms, two sequence mechanisms, per-visit tables with unclear roles. Why it matters: each duplicate doubles the work for every change.
23. **Should the append-only money tables get triggers like the commission ledger?** Context: three tables are protected; others are promised by comments. Why it matters: it is the cheapest strong guard for financial records.
24. **Should the unused tables and columns be dropped (linked sign-ins, per-visit summary, deprecated refresh token, chairs, legacy inventory table)?** Why it matters: they mislead readers and the generated catalog.

67.4 Operations and security

25. **Where and how is the Platform deployed, and who supervises the single worker?** Context: no deploy configuration exists. Why it matters: exactly one worker must run, and nothing here shows how that is ensured.

26. **Should the scheduled jobs use each location's time zone?** Context: jobs follow the server clock. Why it matters: payroll month, credit expiry and day-end alerts depend on it.
27. **What is the data retention policy for sessions, security logs, audit rows, messages and consent signatures?** Why it matters: it is the first question a regulator or clinic will ask.
28. **Is the calendar library licence suitable for a commercial product?** Why it matters: it is a legal exposure that can be solved by buying the licence or changing the component.
29. **Should audit extend to appointments, employees, payroll, organizations and locations, with one viewer?** Why it matters: owners cannot otherwise answer who changed a record.
30. **What should be done about the committed test credentials?** Why it matters: rotating them and cleaning history should be scheduled, not assumed.
31. **Should public lab links and the TV display remain unauthenticated by design?** Why it matters: it defines the acceptable exposure and the expiry rules.

67.5 Behaviour to confirm by running

32. **Do inventory writes fail because of the id mismatch?** Why it matters: it decides whether inventory can be opened to clinics at all.
33. **Do the prescription change history and duplicate actions fail?** Why it matters: they are small fixes if confirmed.
34. **Do background jobs continue while a clinic is read-only?** Why it matters: payroll and invariants could be skipped or not.
35. **Are uploaded patient documents and PDFs publicly readable by address?** Why it matters: it depends on the storage settings, not on the code.
36. **Which currency column do billing screens read, the location's or the organization's?** Why it matters: a mismatch shows the wrong currency label.
37. **Does the lab ever receive a magic-link message in practice?** Why it matters: today the contact channel is a stand-in, so links must be copied by hand.
38. **Should lab orders link to the appointment and treatment?** Context: the API accepts both, the web app never sends them. Why it matters: try-in scheduling and case reporting depend on it.
39. **Is a printable lab invoice or receipt needed?** Context: none was found. Why it matters: lab patient invoices cannot be handed to patients otherwise.

67.6 How the answers are used

The product questions (Section 67.1) set the roadmap. The clinic and numbering questions (Sections 67.2 and 67.3) decide small fixes that remove the High and Medium gaps in Chapter 65. The operations and security questions (Section 67.4) decide the order in Chapter 66. The questions in Section 67.5 need someone to run the application and watch, and they should be answered before the related work is planned.

Chapter 68 — Documentation Conflicts

This chapter records where existing documents, code comments and screen texts disagree with the code. In every case the handbook follows the code. It is for the owner and for new developers, who need to know which documents can be trusted and which should not be used. The table merges the document inventory made before the handbook was written with the conflicts that each chapter found.

68.1 Conflicts table

Document	Claim	Code reality	Evidence
Root project instructions	The frontend uses `app`, `shared` and feature `api` folders	The code uses `routes`, `components`, `lib`, `store` and feature `services`	Chapter 4
Root project instructions	There are four global guards	There are five; the subscription guard comes last	Chapters 5, 11
Root project instructions	The tenant guard skips path prefixes, and the JWT guard skips public markers	There are no prefixes; the guard uses decorators only, and the JWT guard checks a list of public routes	Chapters 5, 8
Root project instructions	Tokens carry organization context	Tokens carry user id, email, global role and session id only	Chapter 8
Root project instructions	Inventory, invoices, reports and notifications are stubs	They are implemented; only chairs is entity-only	Chapters 4, 12, 20, 23
Root project instructions	Soft deletes by an active flag in most modules	Patient delete is a hard delete	Chapter 28
Root project instructions	Some dashboard widgets use mock data	The live dashboard uses real data; the mock widgets are not imported	Chapter 36
Root project instructions	Organizations own employee role definitions	The role entity lives in employees, with logic in access	Chapter 15
Root readme	Bull queue and Google and GitHub sign-in are part of the stack	The queue has no producer or consumer, and OAuth is not registered	Chapter 4
Root readme	The architecture diagram shows four guards	Five guards	Chapter 5
Root readme	`/register` creates your account	It also creates a trial clinic, main location and ownership	Chapter 7
Root readme and backend readme	The start script runs `dist/main`	The build emits the entry file one folder deeper	Chapter 6
Environment example	Refresh, email and reset secrets are required	No code reads them; several variables the code reads are missing	Chapter 6
Access-control guide	`GET /me/catalog`	The endpoint is `GET /access/catalog`	Chapter 11
Access-control guide	Assistant template summary	The Assistant also holds the lab orders view permission at location scope	Chapter 11
Access-control test cases	FEAT-08 expects "refused"	The code returns a successful no-op	Chapter 10
Appointments and queue guide	No migration files; production must still switch from synchronize	There are three migrations, and production already runs with synchronize off plus a reviewed schema sync	Chapters 6, 12, 29, 30

Document	Claim	Code reality	Evidence
Appointments and queue guide	The worker runs one or more instances	Exactly one worker may run	Chapters 6, 30
Appointments and queue guide	Realtime fan-out uses the Socket.IO Redis adapter	It uses its own Redis publish and subscribe with a worker heartbeat	Chapter 30
Appointments and queue guide	Validate runs the same rules as create; overlap gives a coded conflict	Validate skips the past-time check and the override flag; the read-time overlap has no code, only the database path does	Chapter 29
Appointments and queue guide	API list is complete	It omits the session creation endpoint	Chapter 30
Commissions guide	Target is the smaller of allocations and net	The pre-tax part is pro-rated and capped, and a lab-cost basis reduces the base	Chapter 35
Commissions guide	One method writes commission entries	A lab commission service also writes entries for lab lines	Chapter 35
Commissions guide	Access by role	Access by the permissions `compensation.manage`, `earnings.all.view`, `earnings.own.view`	Chapter 35
Billing test cases	Truncate a coupons table; amounts assume no tax; role-based error texts	Coupons do not exist; exclusive tax applies; permission-based messages	Chapter 34
Old billing context pack	Describes pre-version-two billing	Outdated throughout	Chapter 34
Feature catalog seed	`billing.discounts` "Discounts & Coupons"; `reports.exports` as CSV and PDF	Coupons were removed; exports are Excel and PDF	Chapters 10, 36
Feature catalog seed	`comms.in_app`, `comms.whatsapp`, `hr.commissions` are active features	Nothing checks them	Chapter 10
Feature catalog seed	`inventory.alerts` gives low-stock and expiry alerts	The job only writes a log line	Chapters 10, 39
Entity catalog	Notes for ten inventory tables	Shifted one row down	Chapter 23
Entity catalog	A trigger protects published form versions	No such trigger; the service enforces it	Chapter 26
Entity catalog	`charges.source_id` is discriminated by `source_type`	The column is named `source`	Chapter 20
Entity catalog	Only the queue writes the idempotency table	Workflow, lab billing, lab orders and vendor payments also write it, and the queue clean-up deletes all rows older than a day	Chapter 18
Entity catalog	`patient_visits` is a per-location summary	Nothing writes it	Chapter 27
Entity catalog	Append-only list is complete	The allocation and journal-line tables are also called append-only without triggers	Chapter 27
Entity catalog	Migration creates the audit tables completely	It lacks the subject column and index present in the entity	Chapter 26
Migration for compensation	`charge_id` is not null	The entity allows null and adds a lab-line column and a check	Chapter 22

Document	Claim	Code reality	Evidence
Code comment on patient delete	Related rows cascade	Billing and lab rows block the delete with RESTRICT keys	Chapter 17
Code comment on employee delete	Related rows keep the row with the doctor set to null	They are RESTRICT, and the message names inventory even when an appointment blocks it	Chapter 16
Code comment on prescription numbers	Sequential per organization	Unique across the Platform	Chapter 19
Code comment on plan templates and screen text	Procedures are copied into a patient's plan	Nothing creates a patient plan from a template	Chapter 33
Code comment in billing audit entity	The repo has no general audit module	A general audit table exists	Chapter 21
Code comment on journal source	Expenses post journals	Only lab finance posts journals	Chapter 21
Code comment in payroll	Runs at 00:05 on the first	The schedule is 05:00 server time	Chapter 38
Code comments in lab modules	The portal will manage the lab catalog; the clinic activates labs until the portal ships	The portal ships and lets a lab admin act on links, but only a super admin manages the catalog	Chapter 40
Queue settings screen text	Changes apply from tomorrow	Only the smart-queue switch is fixed per day	Chapter 30
Access guard comment	A missing marker becomes a boot error	Only a unit test enforces markers	Chapter 11
First inventory document	Recall, feedback and referral are implemented through a CRM tab	The tab is never mounted; recalls have no component	Chapter 28
First inventory document	Leave and availability overrides are implemented	They can be recorded but are never read by booking or payroll	Chapter 37
First inventory document	Inventory is implemented	The module works through the API but has no sidebar entry and some writes are unverified	Chapter 39
First inventory document	Digital consent signing is unverified	It is implemented with a signature pad	Chapter 28
Outline	The booking form is a sheet; lab order comes before automatic charges	It is a dialog; lab work never creates clinic charges	Chapters 29, 48
Patients visit tab text	A visit is recorded at check-in	The encounter is created at start of consultation	Chapter 28

The table shows that most drift is in descriptions of structure (folders, guards, stubs) and in comments written before a redesign. The numbers and behaviours in the code are more stable than the words around them.

68.2 Documents and how far they can be trusted

The following documents are current or mostly current and may be used, with the corrections in the table: the generated entity catalog and the organizations and locations chapter, the access-control guide, the database migrations guide, the root readme (for the process split and counts) and the appointments and queue guide.

The following are outdated and should not be used for decisions: the hand-off note (it mentions an appointment token that no longer exists and fixed problems), the project overview (it predates access control, billing version two and the lab modules), the audit file (seven of its ten most serious findings are fixed), the plan file (its checklist is stale) and the old billing context pack (it describes replaced code). The two stock starter readmes carry no

project information. The design mock-up file in the repository root uses a former name and does not describe the current screens. The root project instructions are partly outdated and need an update before they are given to a new developer or an assistant.

68.3 The CRM tab correction

The first document inventory marked recall, feedback and referral as implemented through a CRM tab on the patient page. The code shows that backend endpoints exist but the tab component is never mounted in the patient detail page, so those features cannot be reached by a user, and recalls have no component at all. The handbook therefore labels them Partial. Any plan or sales material that repeated the earlier claim should be corrected.

68.4 Using this chapter

Treat the handbook as the source of truth and the code as the final authority. When a document in the table is edited, update the row, so it does not return as a source of confusion. Chapter 65 lists the same drift among the known gaps, and Chapter 67 asks which documents the owner wants archived.

Appendices

Appendix A — Glossary

This glossary defines every entity, role, feature, status label and key term used in the handbook, once, in alphabetical order. It is for anyone reading a single chapter on its own, including readers who start in the middle. Where the code uses a different name from the screen, the code name follows in parentheses. Terms that exist only in the database are shown with their table name.

A

Access context — the resolved permissions of one user at one organization and location, which answers "can", "scope" and "decide" for a request (`AccessContext`).

Access marker — one of `@Can` , `@AnyMember` , `@Public` , `@RequireSuperAdmin` or `@ContextFree` ; every endpoint handler must carry one, and a unit test fails when one is missing.

Access token — the 15-minute token kept in the browser's memory and sent with every request.

Access version — a counter on the organization (`access_version`) raised whenever roles, features or navigation change, so every process and screen knows to reload access.

Active feature — a feature that is in the active catalog, entitled, enabled and not switched off at the current location.

Active queue entry — a queue entry in status waiting, called, in consultation, on hold or skipped.

Advance — patient money received and not yet allocated or refunded; it is calculated (amount minus allocated minus refunded) and has no table of its own. A lab advance is the same idea for lab payments.

Allocation — a signed record of how much of a payment was applied to a charge or released back (`payment_allocations`).

Amended prescription — a signed prescription that was edited and not signed again (status `AMENDED` , version increased).

Append-only — a table whose rows are never changed or deleted. Only `commission_entries` , `audit_logs` and `status_history` are protected by a database trigger; others are append-only by convention.

Appointment — a booking: one patient, one doctor, one location, a start time and a duration (`appointments`). Also called a booking before arrival.

Assignment — an employee's link to an organization and a location with an employment role, a staff login and portal access (`employee_org_location_assignments`).

Attachment — a file on a lab order or item (`attachments`).

Audience — who a notification is for: patient, doctor, coordinator or lab.

Audit log — the generic append-only change history (`audit_logs`).

Autosave — saving shortly after the user stops typing, without a Save button.

Availability override — a one-date exception to a doctor's weekly schedule (`doctor_availability_overrides`); recorded but never read by booking.

B

Basis points (bps) — a rate stored as an integer where 10,000 means 100 percent and 4000 means 40 percent.

Bell — the header icon that opens a user's in-app inbox (`user_notifications`).

Billing audit log — billing's own audit trail with before and after snapshots (`billing_audit_logs`).

Billing v2 — the code's name for the current charge-based billing design.

Blacklisted patient — a patient flagged as not to be served; informational only, it blocks nothing.

Booking — see Appointment.

Bundle — a ready-made per-module level on the roles screen: None, Basic or Full.

C

Cancel reason code — the structured reason stored when a booking is cancelled (`patient_left` or `other`).

Case profit — revenue minus lab cost, case expenses and doctor commission for one lab order.

Cash close — an immutable end-of-period count of a location's counter (`cash_closes`); each payment account has a line with expected, counted and variance.

Chair — a treatment chair of a location; the table exists but no module uses it (`chairs`).

Change history — the per-patient audit timeline merged from the audit log and the status history.

Channel — how a notification is delivered: in-app, WhatsApp, SMS or email. Only in-app is active.

Charge — one billable line for a patient; the atomic unit of billing (`charges`).

Charge request — an extra charge a lab asks for after an order is submitted (`order_charge_requests`).

Clinic default layout — a sidebar layout with no role, applied to every role of a clinic (`nav_layouts`).

Clinic overview — the dashboard of patients, appointments and waiting (`/clinic-overview`).

Clinic time — dates and times computed in the location's time zone through `ClockService` .

Collected basis and billed basis — whether profit-and-loss revenue is counted from money received or from invoices issued.

Commission basis — whether a commission rate applies to the net or gross charge amount.

Commission entry — one signed change in commission earned on one charge or lab invoice line; append-only (`commission_entries`).

Commission rule — an override of a compensation plan for one service or service category (`commission_rules`).

Communications log — a manual record of calls and messages with a patient (`patient_communications`).

Compensation plan — one dated version of how an assignment is paid: salary, commission or both (`compensation_plans`).

Config Studio — the super admin area for form schemas, workflows and stage templates (routes under `/studio`).

Consent — a signed patient consent with the signature image stored in the row (`patient_consents`).

Consultation workspace — the doctor's one-screen visit page where notes, treatments and prescriptions are recorded.

Consumption template — a standard kit of materials for a procedure (`consumption_templates`).

Convert to walk-in — closing a too-late booking as a no-show and adding the patient to the same queue as a linked walk-in.

Credit note (lab) — a reduction of an issued lab invoice; money already paid above the new total returns to advance (`lab_credit_notes`).

Credit package — a prepaid credit offer of an organization (`credit_packages`); backend only.

Custom discount — a discount not taken from a discount type; it needs a permission and a note.

Custom role — a role created by a clinic, in contrast to the five system roles.

D

Daily booking limit — the most live bookings a location accepts on one local day (default 50).

Dental info — the single dental health and habits sheet of a patient.

Dev-log provider — the stand-in delivery provider that writes patient-channel messages to the application log instead of sending them.

Direct costs — lab bills plus doctor commissions in the profit-and-loss statement.

Discount type — an owner-defined discount preset (`discount_types`).

Doctor screen — the My queue page where a doctor runs their own queue.

Doctor share — the commission a payment would record, shown to authorised users while collecting payment.

Doctor signature — one saved signature image per login, placed on prescriptions (`doctor_signatures`).

Document code — the two- to six-character location code at the start of invoice, receipt and refund numbers.

Document sequence — the per-location, per-type, per-year counter behind billing numbers (`document_sequences`).

Double-booking constraint — the database rule that refuses overlapping live bookings of one doctor (`no_doctor_overlap`).

Dry run — an import request that checks rows without saving them.

E

Early window — minutes before the booked time from which a checked-in patient counts as due (default 15).

Effect — a code handler that a workflow transition may run (`workflow_effects`).

Employee — the person record of a staff member, shared across organizations (`employees`).

Employee code — a per-organization code such as EMP-001 on an assignment.

Employment role — see Role.

Encounter — the clinical record of one consultation (`patient_encounters`).

Entitled — a feature the super admin has granted to an organization, with an optional trial end date.

Entity catalog — the generated field-level reference of every table.

Envelope — the JSON shape every endpoint returns (`ResponseWrapper`): data, message, success flag, status code, error and developer message.

Escalation protection — the rule that a non-owner cannot grant a permission or scope they do not hold.

Estimate — a lab invoice in draft: editable, unnumbered, nothing owed.

Exclusive tax — tax added on top of the price at issue, using the location's rate.

Expense — one clinic running cost at a location (`expenses`).

External lab — a lab serving many clinics.

F

Feature — a feature key such as `billing.invoices` that an organization may be entitled to and enable.

Feature impact — the permissions and roles that stop working when a feature is turned off.

Finance overview — the owner's profit-and-loss dashboard (`/finance`).

First come, first served — the queue mode that calls patients in arrival order; the alternative is the smart queue.

Force complete — completing a consultation on the doctor's behalf.

Form schema — the stable identity of an order form (`form_schemas`); its versions are `form_schema_versions` .

G

General ledger — double-entry bookkeeping in paisa, written only for lab money (`ledger_journals` , `ledger_journal_lines`).

Global guard — a guard run on every request; there are five: JWT, tenant context, feature flag, access and subscription.

Global role — a user-level role; its only value is super admin.

Grace period — three days after a subscription ends during which the clinic still works, with warnings.

H

Heartbeat — a Redis message the worker sends every five seconds; API instances refuse sockets after fifteen seconds without one.

High-risk permission — one of 13 permissions marked high risk; granting it needs a reason.

Home page — the page a role's members open after sign-in.

I

Idempotency key — a request header that makes a retried action run once (`idempotency_keys`).

If-Match version — the entry version a screen sends so that a stale action is refused.

Impact preview — the dialog that lists what stops working when a feature is turned off.

Implemented — status label: built end to end, with a screen or by design a backend job.

Implies — permissions granted automatically with another (for example create implies view).

In-house lab — a lab owned by one clinic and serving only it.

Inactive patient — a patient hidden by the active filter with all records kept.

Inbox — a user's own notification rows shown behind the bell.

Invoice — an issued document grouping a patient's charges; there are no drafts (`invoices`).

Item stage — one try-in round of one stage of a lab item (`order_item_stages`).

J

Journal — one balanced double-entry posting; a journal line is one debit or credit side.

K

KPI strip — the six summary tiles at the top of the Appointments page.

L

Lab — a dental laboratory, external or in-house (`labs`).

Lab admin — a lab member with full portal rights for one lab.

Lab billing settings — per-organization lab tax rate and due days.

Lab coordinator — a clinic user holding both permissions needed to act for a lab that does not answer.

Lab directory — the platform list of labs kept by the super admin.

Lab link — a clinic's connection to a lab, with status, payment terms and patient data level (`lab_clinic_links`).

Lab member — a user who works at a lab as lab admin or lab technician (`lab_members`).

Lab order — a lab case: items for one patient, doctor and lab (`lab_orders`).

Lab order item — one piece of work with frozen price snapshots (`lab_order_items`).

Lab-only user — a user with lab memberships and no clinic access, kept inside the lab portal.

Lab payable — what the clinic owes a lab.

Lab portal — the area where lab members work on orders, production, billing and their team.

Lab service — one product a lab makes, such as a crown (`lab_services`).

Lab statement — a lab's monthly statement reconciled against bills (`lab_statements`).

Lab technician — a lab member who works the queue but cannot answer orders or manage the team.

Lane — one doctor's column on the reception board for a date.

Last accessed context — the organization and location saved on the user and pre-selected at next sign-in.

Late grace — minutes after the booked time within which an arriving patient keeps appointment priority (default 10).

Legacy baseline — the first migration, recorded as applied and never executed.

Legacy organization settings — the `organization_settings` table, editable but read by no backend rule.

Legacy quick prescription — a prescription in the older `patient_prescriptions` table.

Live slot check — the debounced server check the booking dialog runs while open.

Location — a branch of an organization (`locations`).

Location override — a switch that turns an enabled feature off at one location.

Lock (setting) — a flag freezing a setting's value for lower scopes.

Locked feature — a feature the owner cannot switch on because it is not entitled or the trial ended.

Logical reference — an id column that points to another row without a foreign key.

M

Magic link — a no-login link for a lab to one order; only a hash of the token is stored (`magic_links`).

Main branch — a location flagged as the clinic's main one; the flag is not exclusive.

Mandatory event — an event whose in-app message to internal audiences cannot be switched off.

Manual payment — a subscription payment recorded by a super admin.

Maximum late — minutes after the booked time after which check-in is refused and the booking becomes a no-show (default 45).

Medicine directory — the clinic's list of medicines (screen label "Medicines").

Modular monolith — one backend application split into domain modules.

Money invariants — ten rules checked every night by the billing job.

Monorepo — one repository holding both the backend and the web app.

N

Needs pricing — a flag on a charge created without a resolvable price; it blocks invoicing until priced.

Nightly job — a scheduled task such as the money invariant check.

No-show marker — the background job that marks unattended bookings as no-shows.

Not implemented — status label: planned, seeded, commented out or only mentioned.

Notification event — a platform-defined kind of event that can notify someone (`notification_events`).

Notification outbox — one message to one recipient on one channel, written in the same transaction as its cause (`notification_outbox`).

Notification rule — a clinic's on/off switch for one event, audience and channel.

Notification template — wording of one event on one channel in one language.

Number sequence — a per-organization counter for numbers such as LAB-0001.

O

Onboarding pending — a flag on the organization that is true from signup until the setup wizard is finished.

Order template — a saved starting point for a lab order.

Organization — one clinic business; the tenant (`organizations`).

Outbox (realtime) — table of realtime messages written in the same transaction as a queue change (`outbox`).

Outbox relay — the worker loop that publishes unpublished outbox rows.

Override booking rules — the permission to book outside weekly hours or past the daily limit; it never bypasses the double-booking check.

Owner — the user who owns an organization and has all permissions in it.

P

Paisa — the integer minor unit of the rupee; 100 paisa is one rupee.

Partial — status label: some of it exists; the note says what is missing.

Patient — the patient record, one per patient per organization (`patients`).

Patient data level — per lab link, whether the lab sees the full name or only the patient code.

Patient import — bulk creation of patients from a spreadsheet through parse, dry run and batches.

Patient lab invoice — a patient invoice for lab work (`lab_invoices`).

Patient ledger — one-sided statement rows per patient (`ledger_entries`).

Patient number — the per-organization patient identifier, P- plus five digits.

Patient scope — the reach of the patient view permission: own, location or org.

Payment — money received from a patient; it belongs to the patient, not to an invoice (`payments`).

Payment account — one way a location receives money: cash, bank or wallet.

Payment status — the computed invoice status unpaid, partially paid, paid or overdue; never stored.

Payroll record — one employee's pay for one month at one location (`payroll_records`).

Permission — a key such as `billing.invoice.void` granted to a role.

Pinned version — the workflow template version stored on a lab order or item when it is created.

Plan template — a predefined treatment plan offered when recording treatment (`plan_templates`).

Platform default — a form, stage or workflow template with no owner organization.

Polymorphic reference — a type plus id column pair that can point at different tables.

Portal access — the assignment flag that lets a linked staff login sign in at that location.

Prescription — the current prescription record (`prescriptions`).

Prescription preset — a reusable set of medicines for the builder; stored in `treatment_plan_templates` and unrelated to plan templates.

Price override — a location or doctor price for a catalog service (`service_prices`).

Price policy — the rule that only a holder of the price override permission may change a catalog charge's price; others give a discount.

Print layout — the look of a printed prescription (`prescription_templates`).

Process role — which half of the backend a process runs: all, api or worker.

Proxy update — clinic staff recording a lab's answer for a lab that does not respond.

Purchase order — an order to one supplier for one location (`purchase_orders`).

Q

QC — the lab's own quality check between finishing and ready.

Queue entry — a patient's place in a queue session (`queue_entries`).

Queue history — the append-only log of queue changes (`queue_events`).

Queue policy — queue settings at organization level with location override (`queue_policies`).

Queue session — one doctor's queue for one clinic day at one location (`queue_sessions`).

Quiet hours — a daily window in which non-in-app messages are held.

R

Read-only mode — the state of an expired clinic: reads work and writes are refused with a payment-required error.

Ready to bill — the desk list of patients with an unbilled charge on a chosen day.

Recall — a planned future check-up with a due date (`patient_recalls`).

Reception board — the Queue tab showing every doctor's lane.

Refund — money returned to a patient, always from advance (`refunds`).

Refund source — a previous payment from which a refund draws.

Rejoin policy — where a skipped patient goes when rejoined: end, original position or manual.

Remake request — a request to make an item again, with a responsible party.

Report definition — a registered report with key, columns, filters and one query.

Resolved price — the price a service gets: doctor override, then location override, then catalog default.

Retired feature key — a catalog key with no code behind it, deactivated at every start.

Revision — a numbered change of a submitted lab order, or the counter raised by each invoice edit.

Role — an employment role with permissions and a home page (`employment_roles`); "system role" for the five templates, "custom role" for clinic-made ones.

Room — a Socket.IO channel a screen subscribes to.

Route file — a thin web file declaring a URL, an optional permission and the page to render.

Row scope — the filter that limits rows to own, location or organization.

S

Scope — how far a permission reaches: `own`, `location` or `org`.

Selection engine — the pure function that decides whom Call next picks.

Service catalog — the organization's list of billable services (`services`).

Service day — the clinic day a queue session belongs to, using the location time zone and a cutoff.

Service mapping — which lab service fulfils which clinic billing service.

Session (login) — one signed-in device with a hashed refresh token (`user_sessions`).

Settings registry — the cascading settings store (`config_values`).

Settlement — a batch of unsettled commission entries paid through one payroll record.

Setup wizard — the three-step owner onboarding at `/setup`.

Shipment — a parcel between clinic and lab with a QR token (`shipments`).

Smart queue — the calling order that weighs emergencies, due appointments, starvation and returns from tests.

Snapshot (commission) — the doctor, plan and rate stored on a charge when it is created.

Specialty — a clinical specialty the engine is configured for.

Staff login — a user account linked to an employee through assignments.

Stage template — the try-in rounds a kind of lab work goes through.

Starter templates — the twelve default plan templates inserted for a clinic with none.

Starvation guard — the rule that calls the longest-waiting ordinary patient after a limit.

Status history — the append-only log of status changes (`status_history`).

Stock movement — one signed change to an item's stock (`stock_movements`).

Subscription status — the derived state of a clinic's plan: trial, active, grace, expired or unmanaged.

Super admin — the Platform operator with a global role, exempt from tenant checks.

Supplier — a vendor of stock.

synchronize — the ORM setting that builds the schema from entities; on in development, off in production.

T

Teeth chart snapshot — one append-only chart of a patient's teeth in FDI notation.

Tenant context — the active organization and location of a request.

Tenant headers — the two request headers that carry the active organization and location.

The Platform — the name used in this handbook for the whole multi-tenant clinic management product. Each clinic sets its own display name; the default is "Clinic Portal".

Token (queue) — the number a patient holds in a doctor's queue, with a letter prefix.

Transactional outbox — writing a message in the same transaction as the change and delivering it afterwards.

Treatment — work actually performed on a patient (`treatments`).

Treatment plan — planned work for a patient (`patient_treatment_plans`).

Trial — the 14-day Professional plan period given at self-signup.

TV display — the public waiting-room page showing tokens being served.

U

Unapply — releasing a payment's allocations back to advance; backend only.

Unbilled charge — an active charge not yet on an invoice.

Unmanaged clinic — an organization with no trial and no paid date: no limits and no expiry.

Unverified — status label: could not be confirmed from the code; the note says what.

Urgent priority — an authorised emergency flag with a reason that puts a patient first.

User — a login account for the whole Platform (`users`).

V

Variance — counted minus expected money for a payment account at cash close.

Vendor bill — what the clinic owes a lab for an order (`vendor_bills`).

Vendor payment — money the clinic paid a lab.

Visit notes — the doctor's free-text notes for an encounter.

W

Walk-in — a patient who arrives without a booking and is added directly to a queue.

Warranty — a period after delivery, 12 months by default (`item_warranties`).

Watchdog — the API check that refuses sockets when no heartbeat arrives.

Weekly working hours — a doctor's recurring working block for one weekday.

Work order — the printable sheet with a QR code and instructions, without prices.

Worker — the process running the outbox relay, notification relay and scheduled jobs; exactly one may run.

Workflow template — a state machine for orders, items or stages; its versions are draft, published or retired.

Z

Zero-priced charge — a charge created at zero because no price could be resolved; see Needs pricing.

Appendix B — Entity Index

This appendix lists all 153 database tables of the Platform, with the entity class, the owning backend module, the business purpose and the chapters that explain each table. It is for engineers and owners who meet a table name and need to find out what it is. The list was built from the generated entity catalog and checked against the data-model chapters (Chapters 12 to 27).

B.1 How to read the index

The owning module is the backend folder whose entities directory defines the class. The chapter columns give the data-model chapter that describes the table's structure (Part 4) and the module or cross-cutting chapter that explains its behaviour. Some class names differ from the table name; for example the class `CatalogService` is the table `services`. Table names are shown exactly as in the database.

#	Table	Entity class	Owning module	Business purpose	Data-model chapter	Behaviour chapter
1	`users`	`User`	user	One login account for the whole Platform (staff, owner, super admin, lab member).	13	56
2	`oauth_providers`	`OAuthProvider`	auth	Linked external sign-in identity; unused because OAuth is disabled.	13	56
3	`security_logs`	`SecurityLog`	auth	Write-only log of login, logout, password and session events.	13	56
4	`user_sessions`	`UserSession`	session	One signed-in device with its hashed refresh token.	13	56
5	`organization_ownership`	`OrganizationOwnership`	organizations	Links a user to an organization as its owner.	14	9
6	`organization_settings`	`OrganizationSettings`	organizations	Legacy organization-wide booking defaults and feature switches that no backend rule reads.	14	9
7	`organizations`	`Organization`	organizations	The tenant: one clinic business, with plan, trial and paid-until dates.	14	9
8	`location_settings`	`LocationSettings`	locations	Per-location booking, tax, reminder and branding settings.	14	9
9	`locations`	`Location`	locations	One branch of an organization, with time zone and currency.	14	9
10	`nav_layouts`	`NavLayout`	access	A clinic's arrangement of sidebar items.	15	58
11	`feature_catalog`	`FeatureCatalog`	feature-flags	Every feature key the Platform offers, with defaults.	15	10
12	`location_feature_overrides`	`LocationFeatureOverride`	feature-flags	Switches an enabled feature off at one location.	15	10
13	`org_feature_flags`	`OrgFeatureFlag`	feature-flags	Per-organization entitled and enabled state of each feature.	15	10

#	Table	Entity class	Owning module	Business purpose	Data-model chapter	Behaviour chapter
14	`config_values`	`ConfigValue`	settings-registry	Cascading settings store (platform, organization, location) with locks; holds theme values.	14	58
15	`subscription_payments`	`SubscriptionPayment`	subscriptions	Manual subscription payments recorded by a super admin.	14	45
16	`doctor_availability_overrides`	`DoctorAvailabilityOverride`	employees	One-date exceptions to a doctor's weekly schedule (not read by booking).	16	37
17	`employee_leaves`	`EmployeeLeave`	employees	Leave requests of employees (not read by booking or payroll).	16	37
18	`employee_org_location_assignments`	`EmployeeOrgLocationAssignment`	employees	An employee's job at one organization and location, with role, code and staff login.	16	37
19	`employee_schedules`	`EmployeeSchedule`	employees	Weekly working hours of an employee at a location.	16	37
20	`employees`	`Employee`	employees	The person record of a staff member, shared across organizations.	16	37
21	`employment_roles`	`EmploymentRole`	employees	System and custom roles with their permissions and home page.	15	11
22	`payroll_records`	`PayrollRecord`	payroll	One employee's pay for one month at one location.	16	38
23	`patient_communications`	`PatientCommunication`	patients	Manual log of calls and messages with a patient.	17	28
24	`patient_consents`	`PatientConsent`	patients	Signed consent forms with the signature image.	17	28
25	`patient_dental_info`	`PatientDentalInfo`	patients	One dental health and habits sheet per patient.	17	28
26	`patient_documents`	`PatientDocument`	patients	Uploaded files attached to a patient.	17	28
27	`patient_feedback`	`PatientFeedback`	patients	Patient rating and staff response (backend only).	17	28
28	`patient_insurance`	`PatientInsurance`	patients	Insurance policy details of a patient (backend only).	17	28
29	`patient_prescriptions`	`PatientPrescription`	patients	Legacy quick prescriptions kept beside the newer prescriptions module.	17	32
30	`patient_recalls`	`PatientRecall`	patients	Planned future check-ups (backend only).	17	28
31	`patient_referrals`	`PatientReferral`	patients	One patient referring another (backend only).	17	28
32	`patient_teeth_chart`	`PatientTeethChart`	patients	Append-only teeth chart snapshots in FDI notation.	17	28
33	`patient_treatment_plans`	`PatientTreatmentPlan`	patients	Planned treatment for a patient (backend only).	17	33

#	Table	Entity class	Owning module	Business purpose	Data-model chapter	Behaviour chapter
34	`patient_visits`	`PatientVisit`	patients	Per-location visit summary; nothing writes it.	17	27
35	`patient_vitals`	`PatientVitals`	patients	Append-only vital sign readings.	17	28
36	`patients`	`Patient`	patients	The patient record, one per patient per organization.	17	28
37	`treatment_plan_items`	`TreatmentPlanItem`	patients	One planned procedure inside a patient treatment plan.	17	33
38	`appointments`	`Appointment`	appointments	A booking: one patient, one doctor, one start time and duration.	18	29
39	`idempotency_keys`	`IdempotencyKey`	queue	Saved results of retried requests, kept 24 hours.	18	61
40	`outbox`	`OutboxMessage`	queue	Realtime messages written with a queue change and published to socket rooms.	18	63
41	`queue_entries`	`QueueEntry`	queue	A patient's place in a doctor's queue for a day.	18	30
42	`queue_events`	`QueueEvent`	queue	Audit record of every queue change.	18	30
43	`queue_policies`	`QueuePolicy`	queue	Queue settings at organization level with location overrides.	18	30
44	`queue_sessions`	`QueueSession`	queue	One doctor's queue for one clinic day at one location.	18	30
45	`patient_encounters`	`PatientEncounter`	encounters	The clinical record of one consultation, with visit notes.	18	31
46	`treatments`	`Treatment`	treatments	Work actually performed on a patient.	18	31
47	`plan_templates`	`PlanTemplate`	plan-templates	Predefined treatment plans offered when recording treatment.	19	33
48	`doctor_signatures`	`DoctorSignature`	prescriptions	A saved signature image per login, placed on prescriptions.	19	32
49	`medicine_directory`	`MedicineDirectory`	prescriptions	The clinic's list of medicines used for prescribing.	19	32
50	`prescription_history`	`PrescriptionHistory`	prescriptions	Change history with before-snapshots for prescriptions.	19	32
51	`prescription_medicines`	`PrescriptionMedicine`	prescriptions	One medicine line on a prescription.	19	32
52	`prescription_templates`	`PrescriptionTemplate`	prescriptions	Print layouts for prescriptions.	19	32
53	`prescriptions`	`Prescription`	prescriptions	A prescription with status, version and signing details.	19	32
54	`treatment_plan_template_medicines`	`TreatmentPlanTemplateMedicine`	prescriptions	One medicine line inside a prescription preset.	19	32

#	Table	Entity class	Owning module	Business purpose	Data-model chapter	Behaviour chapter
55	`treatment_plan_templates`	`TreatmentPlanTemplate`	prescriptions	Prescription presets (ready-made medicine sets); not treatment plans.	19	32
56	`form_schema_versions`	`FormSchemaVersion`	engine	Versioned content of a lab order form.	26	43
57	`form_schemas`	`FormSchema`	engine	Stable identity of a lab order form.	26	43
58	`organization_specialties`	`OrganizationSpecialty`	engine	Whether an organization works in a given specialty.	26	43
59	`specialties`	`Specialty`	engine	Clinical specialties the engine is configured for.	26	43
60	`stage_template_steps`	`StageTemplateStep`	engine	One try-in round inside a stage template.	26	43
61	`stage_templates`	`StageTemplate`	engine	The rounds a kind of lab work goes through.	26	43
62	`transition_effects`	`TransitionEffect`	engine	Which effects a workflow transition runs, in order.	26	43
63	`workflow_effects`	`WorkflowEffect`	engine	Registry of code handlers a transition may run.	26	43
64	`workflow_states`	`WorkflowState`	engine	States of a published workflow version.	26	43
65	`workflow_template_versions`	`WorkflowTemplateVersion`	engine	Draft, published or retired versions of a workflow.	26	43
66	`workflow_templates`	`WorkflowTemplate`	engine	State machines for lab orders, items and stages.	26	43
67	`workflow_transitions`	`WorkflowTransition`	engine	Allowed moves between workflow states.	26	43
68	`billing_audit_logs`	`BillingAuditLog`	billing	Billing's own audit trail with before and after snapshots.	21	34
69	`cash_close_lines`	`CashCloseLine`	billing	Expected, counted and variance figures per payment account in a cash close.	21	34
70	`cash_closes`	`CashClose`	billing	Immutable end-of-period count of a location's counter.	21	34
71	`charges`	`Charge`	billing	One billable line for a patient; the atomic unit of billing.	20	34
72	`commission_entries`	`CommissionEntry`	billing	Append-only signed commission earned per charge or lab invoice line.	22	35
73	`commission_rules`	`CommissionRule`	billing	Per-service or per-category overrides of a compensation plan.	22	35
74	`commission_settlements`	`CommissionSettlement`	billing	A batch of commission entries paid through one payroll record.	22	35
75	`compensation_plans`	`CompensationPlan`	billing	Dated versions of how an assignment is paid: salary, commission or both.	22	35

#	Table	Entity class	Owning module	Business purpose	Data-model chapter	Behaviour chapter
76	`credit_packages`	`CreditPackage`	billing	Prepaid credit offers (backend only).	21	34
77	`discount_types`	`DiscountType`	billing	Owner-defined discount presets.	20	34
78	`document_sequences`	`DocumentSequence`	billing	Per-location, per-type, per-year counters behind invoice, receipt and refund numbers.	21	34
79	`expenses`	`Expense`	billing	A clinic running cost at a location.	21	34
80	`invoices`	`Invoice`	billing	An issued document grouping a patient's charges.	20	34
81	`ledger_entries`	`LedgerEntry`	billing	One-sided per-patient statement rows.	21	34
82	`ledger_journal_lines`	`LedgerJournalLine`	billing	Debit or credit sides of general ledger journals.	21	42
83	`ledger_journals`	`LedgerJournal`	billing	Balanced double-entry postings (lab money only).	21	42
84	`location_billing_settings`	`LocationBillingSettings`	billing	Per-location billing settings, including the document code.	20	34
85	`patient_credits`	`PatientCredit`	billing	A patient's purchase of a credit package.	21	34
86	`payment_accounts`	`PaymentAccount`	billing	Ways a location receives money: cash, bank, wallet.	20	34
87	`payment_allocations`	`PaymentAllocation`	billing	Signed records of how a payment was applied to charges.	20	34
88	`payments`	`Payment`	billing	Money received from a patient.	20	34
89	`refund_sources`	`RefundSource`	billing	Which payment funded which refund.	20	34
90	`refunds`	`Refund`	billing	Money returned to a patient.	20	34
91	`service_prices`	`ServicePrice`	billing	Location or doctor price overrides for a service.	20	34
92	`services`	`CatalogService`	billing	The clinic's billable service catalog (class CatalogService).	20	34
93	`consumption_template_items`	`ConsumptionTemplateItem`	inventory	One item and default quantity in a consumption kit.	23	39
94	`consumption_templates`	`ConsumptionTemplate`	inventory	Standard kits of materials for a procedure.	23	39
95	`inventory`	`Inventory`	inventory	Legacy single-table item list; not registered by any module.	23	39
96	`inventory_categories`	`InventoryCategory`	inventory	Groupings of stock items.	23	39

#	Table	Entity class	Owning module	Business purpose	Data-model chapter	Behaviour chapter
97	`inventory_items`	`InventoryItem`	inventory	A stocked item at a location with current stock.	23	39
98	`patient_consumption_items`	`PatientConsumptionItem`	inventory	One item used on a patient, with price snapshot.	23	39
99	`patient_consumptions`	`PatientConsumption`	inventory	Materials used on one patient.	23	39
100	`purchase_order_items`	`PurchaseOrderItem`	inventory	One item line on a purchase order.	23	39
101	`purchase_orders`	`PurchaseOrder`	inventory	An order to one supplier for one location.	23	39
102	`stock_movements`	`StockMovement`	inventory	Signed change to an item's stock with before and after.	23	39
103	`suppliers`	`Supplier`	inventory	Vendors of stock.	23	39
104	`chairs`	`Chair`	chairs	Treatment chairs of a location; entity only.	12	65
105	`notification_audience_settings`	`NotificationAudienceSetting`	notifications	A clinic's master switch and quiet hours per audience.	24	44
106	`notification_events`	`NotificationEvent`	notifications	Platform-defined kinds of notification event.	24	44
107	`notification_outbox`	`NotificationOutbox`	notifications	One message to one recipient on one channel.	24	59
108	`notification_rules`	`NotificationRule`	notifications	A clinic's on/off switch per event, audience and channel.	24	44
109	`notification_templates`	`NotificationTemplate`	notifications	Message wording per event, channel and language.	24	44
110	`patient_comm_prefs`	`PatientCommPref`	notifications	A patient's channel opt-outs and language.	24	44
111	`user_notifications`	`UserNotification`	notifications	A user's in-app inbox rows behind the bell.	24	44
112	`lab_clinic_links`	`LabClinicLink`	labs	A clinic's connection to a lab, with status and terms.	25	40

#	Table	Entity class	Owning module	Business purpose	Data-model chapter	Behaviour chapter
113	`lab_members`	`LabMember`	labs	Users who work at a lab as lab admin or technician.	25	41
114	`labs`	`Lab`	labs	A dental laboratory, external or in-house.	25	40
115	`magic_links`	`MagicLink`	lab-access	No-login links for a lab to one order; token hash only.	25	40
116	`lab_service_options`	`LabServiceOption`	lab-catalog	Materials or variants of a lab service with extra cost.	25	40
117	`lab_service_prices`	`LabServicePrice`	lab-catalog	List or negotiated price windows of a lab service.	25	40
118	`lab_services`	`LabService`	lab-catalog	Products a lab makes.	25	40
119	`order_templates`	`OrderTemplate`	lab-catalog	Saved starting points for lab orders.	25	40
120	`service_categories`	`ServiceCategory`	lab-catalog	Platform-level grouping of lab services.	25	40
121	`service_mappings`	`ServiceMapping`	lab-catalog	Which lab service fulfils which clinic billing service.	25	40
122	`attachments`	`Attachment`	lab-orders	Files on a lab order or item.	25	40
123	`dental_item_details`	`DentalItemDetails`	lab-orders	Typed dental columns promoted from form answers.	25	40
124	`item_form_responses`	`ItemFormResponse`	lab-orders	An item's form answers against its pinned form version.	25	40
125	`item_teeth`	`ItemTooth`	lab-orders	The teeth an order item applies to.	25	40
126	`lab_order_items`	`LabOrderItem`	lab-orders	One piece of lab work with frozen price snapshots.	25	40
127	`lab_orders`	`LabOrder`	lab-orders	A lab case for one patient, doctor and lab.	25	40
128	`order_charge_requests`	`OrderChargeRequest`	lab-orders	Extra charges a lab asks for after submit.	25	40

#	Table	Entity class	Owning module	Business purpose	Data-model chapter	Behaviour chapter
129	`order_messages`	`OrderMessage`	lab-orders	Conversation between clinic and lab on an order.	25	40
130	`order_revisions`	`OrderRevision`	lab-orders	Full snapshots of what the lab was asked to make, per revision.	25	40
131	`item_assignments`	`ItemAssignment`	lab-production	Which lab technician works on an item.	25	40
132	`item_warranties`	`ItemWarranty`	lab-production	Warranty that starts at delivery.	25	40
133	`order_item_stages`	`OrderItemStage`	lab-production	One try-in round of one item stage.	25	40
134	`remake_requests`	`RemakeRequest`	lab-production	Requests to redo an item, with the responsible party.	25	40
135	`shipment_items`	`ShipmentItem`	lab-production	One item inside a shipment.	25	40
136	`shipments`	`Shipment`	lab-production	A parcel between clinic and lab with a QR token.	25	40
137	`lab_billing_settings`	`LabBillingSettings`	lab-billing	Per-organization lab tax rate and due days.	25	42
138	`lab_credit_notes`	`LabCreditNote`	lab-billing	Downward corrections of issued patient lab invoices.	25	42
139	`lab_invoice_lines`	`LabInvoiceLine`	lab-billing	Lines of a patient lab invoice with frozen prices.	25	42
140	`lab_invoices`	`LabInvoice`	lab-billing	Patient invoices for lab work; drafts are estimates.	25	42
141	`lab_payment_allocations`	`LabPaymentAllocation`	lab-billing	How lab payments were applied to lab invoices.	25	42
142	`lab_payments`	`LabPayment`	lab-billing	Money from a patient for lab work.	25	42
143	`lab_refund_sources`	`LabRefundSource`	lab-billing	Which lab payment funded which lab refund.	25	42
144	`lab_refunds`	`LabRefund`	lab-billing	Money paid back to a patient for lab work.	25	42

#	Table	Entity class	Owning module	Business purpose	Data-model chapter	Behaviour chapter
145	`lab_statement_lines`	`LabStatementLine`	lab-payables	Lines of a lab's monthly statement.	25	42
146	`lab_statements`	`LabStatement`	lab-payables	A lab's monthly statement reconciled against bills.	25	42
147	`vendor_bill_lines`	`VendorBillLine`	lab-payables	Lines of a bill from a lab.	25	42
148	`vendor_bills`	`VendorBill`	lab-payables	What the clinic owes a lab for an order.	25	42
149	`vendor_payment_allocations`	`VendorPaymentAllocation`	lab-payables	How payments to a lab were applied to bills.	25	42
150	`vendor_payments`	`VendorPayment`	lab-payables	Money the clinic paid a lab.	25	42
151	`audit_logs`	`AuditLog`	common	Generic append-only change history.	26	60
152	`number_sequences`	`NumberSequence`	common	Per-organization counters such as LAB-0001.	26	62
153	`status_history`	`StatusHistory`	common	Append-only log of status changes.	26	60

The index holds 153 rows, one per table, matching the count in the entity catalog. Two tables are not registered by any Nest module (`chairs` and the legacy `inventory` table), so a development database built by the running application does not contain them, while a database built from the command-line data source does. Chapter 65 lists the tables that are never written or never read by any screen.

Appendix C — API Endpoint Summary

This appendix maps the Platform's backend API, one line per controller file. A controller is a group of related endpoints that share a URL prefix. The backend has **80 controller classes in 74 files, with 593 endpoints** in total. Where one file holds two small controllers, they share a line. Four of those endpoints belong to the Google and GitHub sign-in controller, which exists in the code but is not registered, so those routes do not exist at runtime. The full method-by-method detail of each endpoint is described in the module chapter named in the last column.

C.1 How every endpoint is protected

Every request passes five global guards in this order:

1. **JWT.** Checks the sign-in token, except on routes listed as public.
2. **Tenant context.** Checks that the user belongs to the organization and location named in the request headers.
3. **Feature flag.** Checks features required by `@RequireFeature`.
4. **Access.** Checks `@Can` permissions.
5. **Subscription.** Blocks writes when a clinic's subscription has expired (read-only mode).

Each endpoint also declares how access is decided:

- `@Can(…)`: needs specific permissions. This is the normal case.
- `@AnyMember`: any member of the organization may call it.
- `@Public`: no sign-in, such as the TV display or a lab magic link.
- `@ContextFree`: the handler checks access itself, because it runs before a location is chosen or outside a clinic.
- `@RequireSuperAdmin`: platform staff only.
- **Lab portal:** routes under `/lab-portal/…` use a separate lab-member guard instead of clinic permissions.

A test fails the build if any endpoint has none of these.

Technical references: `backend/src/app.module.ts` (guard order) · `backend/src/access/access-coverage.spec.ts` · `backend/src/labs/portal/lab-actor.ts` · `LabPortalController`

C.2 Controllers

Module	Base path	Endpoints	Access	What it is for	Chapter
app	`/`	1	none	Root greeting / health response	6
auth	`/auth`	17	Public + ContextFree	Register, verify email, login, location selection and switching, token refresh, logout, sessions, passwords	56
auth (OAuth)	`/auth/google`, `/auth/github`	4	Public	Google / GitHub sign-in; **not registered, routes do not exist** [Not implemented]	56
user	`/user`	6	ContextFree	User administration, staff login candidates	56

Module	Base path	Endpoints	Access	What it is for	Chapter
access	`/me` (`/me/access`) and `/access/catalog`	2	ContextFree / Can	The signed-in user's resolved access; the permission catalog for the roles screen	11, 57
access (navigation)	`/admin/navigation`	3	Can	The clinic's sidebar layout: read, save, reset	58
organizations	`/organizations`	7	SuperAdmin (settings read ContextFree)	Create and manage organizations and their settings	9
locations	`/locations`	7	ContextFree (checks `admin.locations.manage` itself)	Create and manage locations and location settings	9
feature-flags (platform)	`/feature-flags`	10	SuperAdmin (own list ContextFree)	Feature catalog; entitle and enable features per organization; seed	10
feature-flags (clinic)	`/admin/features`	4	Can	Owner switches features on and off, previews impact, sets location overrides	10, 58
settings-registry (theme)	`/public/theme`, `/theme`, `/admin/theme`, `/platform/theme`	8	Public / AnyMember / Can / SuperAdmin	Clinic theme and branding at platform, organization and location level	58
subscriptions (clinic)	`/account`	5	AnyMember	Clinic account and the setup wizard	45
subscriptions (platform)	`/admin/subscriptions`	6	SuperAdmin	Subscription status, manual payments, payment instructions	45
employees	`/employees`	8	Can	Employee records, status, staff login access, salary	37
employees (schedule)	`/employees/:employeeId/schedule`	3	Can	Weekly working hours per location	37
employees (availability)	`/employees/:employeeId/availability-overrides`	4	Can	One-off changes to a doctor's availability	37
employees (leave)	`/leaves`	8	Can	Leave requests, approval, balance	37
employees (roles list)	`/employment-roles`	5	Can	Roles used when assigning employees	11
employees (roles admin)	`/admin/roles`	6	Can	Owner's role builder: create, clone, edit, delete roles	11, 58
payroll	`/payroll`	10	Can	Generate monthly payroll, mark paid / on hold, export, payslip PDF	38
patients	`/patients`	43	Can	Patient records, import and export, and all clinical and CRM sub-records	28
appointments	`/appointments`	13	Can	Booking, availability, validation, confirm, cancel, no-show, check-in	29

Module	Base path	Endpoints	Access	What it is for	Chapter
queue (reception)	`/queue`	18	Can / AnyMember	Queue sessions, walk-ins, check-in, skip, rejoin, transfer, priority, policies, tickets	30
queue (doctor)	`/queue`	15	Can	The doctor's own queue: call next, select, recall, start consultation, send for test, complete	30
queue (display)	`/queue/displays`	1	Public	Data for the waiting-room TV display	30
encounters	`/encounters`	5	Can	The consultation record and workspace; treatments added during a visit	31
treatments	`/treatments`	3	Can	Treatments performed and the treatment catalog	31
plan-templates	`/plan-templates`	4	Can / AnyMember	Reusable treatment plan templates	33
prescriptions	`/prescriptions`	15	Can	Prescriptions: build, medicines, finalize, duplicate, PDF, history	32
prescriptions (medicines)	`/medicine-directory`	6	Can / AnyMember	The clinic's medicine directory	32
prescriptions (templates)	`/prescription-templates`	6	Can / AnyMember	Prescription print templates and the default template	32
prescriptions (protocols)	`/treatment-plan-templates`	5	Can / AnyMember	Reusable medicine protocols for prescriptions	32
prescriptions (signatures)	`/doctor-signatures`	4	Can / AnyMember	Doctors' signature images for printed prescriptions	32
billing (config)	`/billing`	16	Can	Service catalog, location prices, billing settings, payment accounts, discount types	34
billing (charges)	`/patients/:id/charges`, `/charges`, `/billing/ready-to-bill`	9	Can	Charges: list, add, from a treatment plan, edit, discount, void; the ready-to-bill list	34
billing (payments)	`/payments`, `/refunds`, `/patients/:id/advance`	10	Can	Collect payment (with preview), unapply, apply advance, refunds, daily collection	34
billing (invoices)	`/invoices`	9	Can	Invoices: list, stats, revisions, edit with preview, cancel, refund	34
billing (patient billing and closing)	`/patients/:id/billing`, `/billing/…`, `/treatment-plans/:id/billing-summary`	11	Can	Patient summary and statement, outstanding, cash close, doctor revenue, billing activity	34
billing (print)	`/print`	5	Can	Print data for invoices, receipts, refunds, statements, cash close	34
billing (expenses)	`/expenses`	5	Can	Clinic expenses and the expense summary	34

Module	Base path	Endpoints	Access	What it is for	Chapter
billing (credit packages)	`/credit-packages`	7	Can	Prepaid credit packages and patient credits (no screen)	34
billing (financial summary)	`/financial-summary`	3	Can	Daily, monthly and trend figures for the dashboard	36
billing (commissions)	`/commissions`, `/reports/doctor-earnings`, `/me/earnings`	3	Can	Commission ledger, doctor earnings, a doctor's own earnings	35
billing (compensation)	`/assignments/:id/compensation-plans`, `/compensation-plans`	3	Can	Doctors' compensation plans and commission rules	35
reports	`/reports`	5	Can / AnyMember	Report catalog, finance overview, clinic overview, run a report, export to Excel or PDF	36
inventory	`/inventory`	15	Can	Categories, items, stock levels, stock adjustments and movements	39
inventory (suppliers)	`/suppliers`	5	Can	Suppliers	39
inventory (purchase orders)	`/purchase-orders`	6	Can	Purchase orders: status, receive	39
inventory (consumption templates)	`/consumption-templates`	6	Can	Standard material lists per treatment type	39
inventory (patient consumption)	`/patient-consumptions`	4	Can	Materials used for a patient or appointment	39
notifications (inbox)	`/me/notifications`	4	ContextFree	The signed-in user's in-app notifications	44
notifications (settings)	`/notification-settings`	9	Can	Audiences, rules, templates, delivery log, patient preferences	44
uploads	`/uploads`	1	AnyMember	File upload to storage	28
labs (platform)	`/platform/labs`	10	SuperAdmin	The platform lab directory: labs, status, members, links	40
labs (clinic links)	`/lab-links`	7	Can / AnyMember	The clinic's connections to labs	40
lab catalog (platform)	`/platform/service-categories`	15	SuperAdmin	Lab service categories, services, options and prices	40
lab catalog (clinic)	`/lab-catalog`	13	Can / AnyMember	The clinic's lab price list, price quotes, service mappings, order templates	40
lab orders (clinic)	`/lab-orders`	14	Can	Lab orders: create, from template, submit, revise, repeat, messages, attachments, work order	40

Module	Base path	Endpoints	Access	What it is for	Chapter
lab access (clinic)	`/lab-orders/:id/magic-links`, `/lab-orders/:id/proxy`	8	Can	Send and revoke magic links; act for a lab that has no login	40
lab production (clinic)	`/lab-orders/…`, shipments	15	Can	Clinic side of production: try-ins, stage decisions, approve / reject / deliver items, remakes, receive shipments	40
lab access (magic link)	`/lab-link/:token`	5	Public	What a lab can do through a magic link without a login	41
lab access (order link)	`/order-links`	1	ContextFree	Resolve a short order link to the right place	41
lab portal	`/lab-portal`	8	Lab portal	The lab's own profile, team (members) and clinic connections	41
lab portal (orders)	`/lab-portal/orders`	12	Lab portal	Lab side of orders: accept, reject, clarification, expected date, messages, revisions, charge requests, attachments	41
lab portal (production)	`/lab-portal/production`	15	Lab portal	Production board: start, assign, finish, QC; shipments; remake disputes	41
lab portal (payables)	`/lab-portal/payables`	5	Lab portal	The lab's view of the clinic's bills, payments and statements	41
lab billing	`/lab-billing`	18	Can	Patient lab invoices: draft, issue, void, advance, credit notes, reminders, payments, refunds, settings	42
lab payables	`/lab-payables`	19	Can	Vendor bills, vendor payments and allocations, lab statements, aging / monthly-cost / case-profit reports	42
engine (workflows)	`/platform/workflow-templates`	15	SuperAdmin	Versioned workflow templates, states, transitions; stage templates; specialties	43
engine (forms)	`/platform/form-schemas`	15	SuperAdmin (reads AnyMember)	Versioned form schemas with validation, publish and retire	43

The table maps each API area to its module chapter. Most controllers use `@Can`, so a specific permission decides who may call them. A few kinds use other access rules:

- **Platform administration** (organizations, features, the lab directory, the lab catalog, Config Studio) is for the super admin only.
- **The lab portal controllers** serve lab staff and use their own guard.
- **Three kinds are public:** the TV display, magic links, and the public theme a clinic's login page needs before anyone signs in.

The endpoint counts add up to 593 including the four unregistered sign-in routes, so 589 routes are live. The access column is a summary: inside one controller, individual endpoints can need different permissions, and the module chapters list them per feature.

Appendix D — Permission Catalog

This appendix lists every permission the Platform knows. There are 70 in total, and the code defines them in one file. A permission is the smallest unit of access a role can be given, for example "void an invoice". An owner never invents new permissions. Instead, the owner combines these 70 into roles, using None / Basic / Full bundles per module or the Advanced mode for single permissions.

Five rules decide how a permission behaves:

- **Feature.** Each permission belongs to one feature. When that feature is not active for the organization, or is switched off at a location, the permission is dropped while access is worked out. It is not deleted from the role, so switching the feature back on restores access exactly as it was.
- **Scope.** Some permissions carry a scope: `own` (only records linked to the user, such as a doctor's own patients or queue), `location` (the active location) or `org` (every location of the organization). A wider scope includes the narrower ones.
- **Also grants.** Some permissions automatically include others. For example, editing patients includes viewing them.
- **Risk.** Permissions marked "high" ask for a confirmation and a reason when an owner grants them in the roles screen.
- **Who holds what.** Owners and the super admin hold every permission. Staff hold only what their role at the active location grants. A missing permission means no access: the system is fail-closed.

The tables below are grouped by the module the roles screen lists them under. Each row gives the permission key exactly as the code spells it, the label the roles screen shows, the feature it depends on, the scopes it accepts ("—" means it has no scope), the permissions it also grants, and whether it is high-risk.

Technical references: `backend/src/access/catalog/permissions.ts` · `PERMISSIONS` · `backend/src/access/catalog/bundles.ts` · `backend/src/access/catalog/templates.ts` · generated copy `web-app/src/features/access/generated/catalog.ts`

Patients

Permission	Label	Feature	Scopes	Also grants	Risk
`patients.view`	View patients	`patients.core`	own, location, org	—	—
`patients.create`	Register patients	`patients.core`	—	`patients.view`	—
`patients.edit`	Edit patient details	`patients.core`	—	`patients.view`	—
`patients.delete`	Delete patients	`patients.core`	—	`patients.view`	high
`patients.export`	Export patient data	`patients.core`	—	`patients.view`	high
`patients.price.view`	See treatment prices	`patients.core`	—	—	—
`patients.clinical.view`	View clinical records	`patients.core`	—	—	—
`patients.clinical.edit`	Edit clinical records	`patients.core`	—	`patients.clinical.view`	—
`patients.documents.manage`	Manage patient documents	`patients.documents`	—	—	—

Permission	Label	Feature	Scopes	Also grants	Risk
`patients.chart.edit`	Edit dental chart	`patients.teeth_chart`	—	`patients.clinical.view`	—
`plans.manage`	Manage treatment plans	`patients.treatment_plans`	—	—	—
`plans.templates.manage`	Manage treatment plan templates	`patients.treatment_plans`	—	—	—

Appointments

Permission	Label	Feature	Scopes	Also grants	Risk
`appointments.view`	View appointments	`appointments.core`	own, location, org	—	—
`appointments.book`	Book appointments	`appointments.core`	—	`appointments.view`	—
`appointments.reschedule`	Reschedule appointments	`appointments.core`	—	`appointments.view`	—
`appointments.cancel`	Cancel appointments	`appointments.core`	—	`appointments.view`	—
`appointments.override_rules`	Override booking rules	`appointments.core`	—	`appointments.book`	high

Queue

Permission	Label	Feature	Scopes	Also grants	Risk
`queue.board.view`	View reception queue board	`appointments.core`	—	—	—
`queue.checkin`	Check patients in	`appointments.core`	—	`queue.board.view`	—
`queue.entry.manage`	Skip, rejoin or remove queue entries	`appointments.core`	—	`queue.board.view`	—
`queue.entry.reprioritize`	Reprioritize the queue	`appointments.core`	—	`queue.board.view`	—
`queue.doctor.own`	Run own doctor queue	`appointments.core`	—	—	—
`queue.doctor.any`		`appointments.core`	—	`queue.doctor.own`	—
`queue.display.manage`	Manage TV display	`appointments.core`	—	—	—

Consultation

Permission	Label	Feature	Scopes	Also grants	Risk
`consultation.conduct`	Conduct consultations	`patients.core`	own, location, org	`patients.clinical.edit`	—

Permission	Label	Feature	Scopes	Also grants	Risk
`prescriptions.create`	Write prescriptions	`patients.prescriptions`	—	—	—
`prescriptions.library.manage`	Manage prescription library	`patients.prescriptions`	—	—	—

Billing

Permission	Label	Feature	Scopes	Also grants	Risk
`billing.desk.use`	Use the billing desk	`billing.invoices`	—	—	—
`billing.invoice.view`	View invoices	`billing.invoices`	own, location, org	—	—
`billing.invoice.edit`	Edit invoices	`billing.invoices`	—	`billing.invoice.view`	—
`billing.invoice.void`	Void invoices	`billing.invoices`	—	`billing.invoice.view`	high
`billing.payment.collect`	Collect payments	`billing.invoices`	—	`billing.invoice.view`	—
`billing.refund.issue`	Issue refunds	`billing.invoices`	—	`billing.invoice.view`	high
`billing.price.override`	Override prices	`billing.invoices`	—	—	high
`billing.cash_close`	Close the cash drawer	`billing.invoices`	—	—	—
`billing.activity.view`	View billing activity	`billing.invoices`	—	—	—

Discounts

Permission	Label	Feature	Scopes	Also grants	Risk
`billing.discount.apply`	Apply preset discounts	`billing.discounts`	—	—	—
`billing.discount.custom`	Give custom discounts	`billing.discounts`	—	`billing.discount.apply`	high

Expenses

Permission	Label	Feature	Scopes	Also grants	Risk
`expenses.view`	View expenses	`billing.expenses`	—	—	—
`expenses.manage`	Record and edit expenses	`billing.expenses`	—	`expenses.view`	—

Commissions

Permission	Label	Feature	Scopes	Also grants	Risk
`earnings.own.view`	See own earnings	`billing.commissions`	—	—	—
`earnings.all.view`		`billing.commissions`	—	`earnings.own.view`	—
`compensation.manage`	Manage salaries and commissions	`billing.commissions`	—	`earnings.all.view`	high

Lab

Permission	Label	Feature	Scopes	Also grants	Risk
`lab.orders.view`	View lab orders	`dental.lab_orders`	own, location, org	—	—
`lab.orders.create`	Create lab orders	`dental.lab_orders`	—	`lab.orders.view`	—
`lab.billing.manage`	Manage lab billing	`dental.lab_orders`	—	—	—
`lab.catalog.manage`	Manage lab catalog	`dental.lab_orders`	—	—	—
`lab.payables.manage`	Manage lab payables	`dental.lab_orders`	—	—	—
`lab.links.manage`	Manage lab links	`dental.lab_orders`	—	—	—
`lab.templates.share`	Share lab order templates	`dental.lab_orders`	—	—	—

Inventory

Permission	Label	Feature	Scopes	Also grants	Risk
`inventory.view`	View inventory	`inventory.core`	—	—	—
`inventory.stock.adjust`	Adjust stock	`inventory.core`	—	`inventory.view`	—
`inventory.po.manage`	Manage purchase orders	`inventory.core`	—	`inventory.view`	—

Team

Permission	Label	Feature	Scopes	Also grants	Risk
`employees.view`	View staff	``	—	—	—
`employees.manage`	Manage staff	``	—	`employees.view`	—
`payroll.manage`	Manage payroll	`hr.payroll`	—	—	high
`leave.approve`	Approve leave	`hr.leave`	—	—	—

Reports

Permission	Label	Feature	Scopes	Also grants	Risk
`reports.clinical.view`	View clinical reports	`reports.clinical`	own, location, org	—	—
`reports.financial.view`	View financial reports	`reports.financial`	own, location, org	—	—
`reports.export`	Export reports	`reports.exports`	—	—	high
`finance.overview.view`	View finance overview	`reports.financial`	—	—	—

Admin

Permission	Label	Feature	Scopes	Also grants	Risk
`admin.features.manage`	Turn clinic features on/off	``	—	—	high
`admin.roles.manage`	Manage roles and permissions	``	—	—	high
`admin.users.manage`	Manage users and their access	``	—	—	high
`admin.nav.manage`	Customize the sidebar	``	—	—	—
`admin.settings.manage`	Manage clinic settings	``	—	—	—
`admin.locations.manage`	Manage locations	``	—	—	—
`admin.audit.view`	View the audit log	``	—	—	—
`admin.theme.manage`	Manage clinic theme and branding	``	—	—	—
`notifications.settings.manage`	Manage notification settings	``	—	—	—

How to read these tables

These tables show the full set of building blocks for roles. A permission whose feature is `core` is always available, because no feature switch controls it. A permission with scopes listed must be granted with exactly one of those scopes. The roles screen offers the scope as a choice in Advanced mode, and the bundles pick a sensible default. For which permissions each of the five system roles (Admin, Doctor, Receptionist, Assistant, Accountant) receives, see Chapter 11, "Roles and permissions", which explains the templates in prose.

Appendix E — Scheduled Jobs and Background Loops

This appendix lists everything the Platform does on its own, without a user clicking anything. That covers timed jobs that run on a schedule, and background loops that deliver realtime events and notifications. It also explains which server process runs each one, because running a job twice can cause duplicate work.

E.1 Where background work runs

The backend is one program that can start in three process roles:

- `api` **role:** serves web requests and the realtime socket connection. It never runs scheduled jobs, so you can run several `api` processes side by side without jobs running twice.
- `worker` **role:** runs the scheduled jobs and the background relays. **Exactly one** worker process must run.
- `all` **role:** does both. It is the default in development and needs no Redis.

The code enables the job scheduler only in processes whose role is not `api`.

Technical references: `backend/src/common/runtime/process-role.ts` · `runsJobs` · `backend/src/app.module.ts` · `AppModule.forRole` · `backend/src/main.worker.ts`

E.2 Timed jobs

Job	Schedule	What it does	Status
Queue no-show marker	Every 60 seconds	Marks a booked patient as a no-show once they have not checked in by the appointment start plus the location's maximum lateness from the queue policy.	[Implemented]
Queue call timeout	Every 15 seconds	Auto-skips a called patient who never came to the doctor, when the queue policy has a call timeout set.	[Implemented]
Queue session day-end	Every 5 minutes	Closes queue sessions from a past clinic day when they are empty. Sessions that still have waiting or on-hold patients are left open for an admin and logged as an alert. They are never auto-cancelled.	[Implemented]
Queue cleanup	Every day at 03:15	Deletes idempotency keys older than 24 hours and published realtime outbox rows older than 7 days.	[Implemented]
Patient credit expiry	Every day at 01:00	Marks prepaid patient credits (from credit packages) whose expiry date has passed as expired.	[Implemented]
Billing invariant check	Every day at 02:30	Recomputes billing totals and checks the money rules (invariants), including the commission ledger rule, and logs any break it finds.	[Implemented]
Month-end payroll	05:00 on the 1st of every month	Generates payroll records for the month that just ended, for every active location of every organization. Locations already generated are skipped, so a re-run is safe.	[Implemented]
Inventory alerts	Every day at 08:00	Counts low-stock and soon-expiring items per location and writes a warning to the server log. No user is notified, and nothing appears in any inbox or screen.	[Partial]

The table lists eight timed jobs. The first four keep the live queue consistent. The other four do daily or monthly housekeeping for credits, billing checks, payroll and stock.

Times use the server clock. None of these jobs sets a timezone, so the daily and monthly schedules follow the timezone of the server running the worker. They do not follow each clinic's location timezone. For clinics in a different timezone from the server, "01:00" or "the 1st at 05:00" therefore happens at a different local clinic time. The queue day-end job avoids this problem. It decides what "a past clinic day" means for each location from that location's queue policy (including its day cut-off) and the Platform clock, not from the server date alone.

Code comment vs code. The payroll job's comment says it runs "at 00:05 on the 1st". The schedule expression `0 5 1 * *` actually means 05:00 on the 1st. This handbook follows the code.

The credit expiry job uses the server clock. It works out "today" from the server's local date rather than the location's clock. Every other clinic-day rule in the Platform uses the location's timezone, so a credit can expire up to a few hours early or late for a clinic whose timezone differs from the server's.

Technical references: `backend/src/queue/jobs/queue-jobs.service.ts` · `backend/src/billing/patient-credits.service.ts` · `checkExpiredCredits` · `backend/src/billing/invariants/billing-invariant.job.ts` · `backend/src/payroll/payroll.service.ts` · `month-end-payroll` · `backend/src/inventory/services/inventory-alerts.service.ts`

E.3 Background loops (always running in the worker)

Loop	How it wakes up	What it does	Status
Queue outbox relay	Database LISTEN on the queue outbox channel, plus a poll every 1 second as a fallback	Takes queue events written in the same transaction as each queue change and publishes them, through Redis, to the realtime socket rooms that reception screens, doctor screens and the TV display listen to.	[Implemented]
Notification relay	Database LISTEN on the notification outbox channel, plus a poll every 2 seconds	Delivers queued notifications to users' in-app inboxes.	[Implemented]
Realtime heartbeat and watchdog	Timer	Checks that the Redis publish and subscribe link between processes is alive.	[Implemented]

These three loops are what make the Platform feel live. The two relays follow the "transactional outbox" pattern. A change and the message announcing it are saved together in one database transaction, and a separate loop delivers the message afterwards. So a message is never lost, and is never sent for a change that was rolled back. If the database LISTEN connection drops, both relays log a warning and keep working by polling alone.

Technical references: `backend/src/queue/realtime/outbox-relay.service.ts` · `backend/src/notifications/delivery/notification-relay.service.ts` · `backend/src/queue/realtime/redis-realtime.publisher.ts` · `redis-realtime.subscriber.ts`